DSpace 7.x Documentation

Contents

1. Introduction . . . 5
1.1 Release Notes . . . 6
1.2 Functional Overview . . . 37
1.3 Technology Overview . . . 49
2. Installing DSpace . . . 50
2.1 7.0-7.1 Frontend Installation . . . 73
3. Upgrading DSpace . . . 77
3.1 Migrating DSpace to a new server . . . 88
4. Using DSpace . . . 90
4.1 Authentication and Authorization . . . 91
4.1.1 Authentication Plugins . . . 92
4.1.2 Bulk Access Management . . . 110
4.1.3 Embargo . . . 113
4.1.3.1 Pre-3.0 Embargo Lifter Commands . . . 118
4.1.4 Managing User Accounts . . . 119
4.1.4.1 Email Subscriptions . . . 122
4.1.5 Request a Copy . . . 124
4.2 CAPTCHA Verification . . . 133
4.3 Configurable Entities . . . 135
4.3.1 Entities Use Case: Compound Journal . . . 149
4.3.2 Entities Use Case: Research Management . . . 152
4.4 Curation System . . . 155
4.4.1 Bundled Tasks . . . 162
4.4.1.1 Bitstream Format Profiler Task . . . 163
4.4.1.2 Link Checker Tasks . . . 164
4.4.1.3 MetadataWebService Task . . . 165
4.4.1.4 MicrosoftTranslator Task . . . 167
4.4.1.5 NoOp Task . . . 168
4.4.1.6 Required Metadata Task . . . 169
4.4.1.7 Virus Scan Task . . . 170
4.5 Exporting Content and Metadata . . . 172
4.5.1 Linked (Open) Data . . . 173
4.5.2 SWORDv1 Client . . . 183
4.5.3 Exchanging Content Between Repositories . . . 184
4.5.4 OAI . . . 185
4.5.4.1 OAI-PMH Data Provider 2.0 (Internals) . . . 191
4.5.4.2 OAI 2.0 Server . . . 193
4.5.5 OpenAIRE4 Guidelines Compliancy . . . 200
4.5.6 Signposting . . . 202
4.6 Ingesting Content and Metadata . . . 203
4.6.1 Ingesting HTML Archives . . . 204
4.6.2 SWORDv2 Server . . . 205
4.6.3 SWORDv1 Server . . . 213
4.6.4 Exporting and Importing Community and Collection Hierarchy . . . 218
4.6.5 Importing Items via basic bibliographic formats (Endnote, BibTex, RIS, CSV, etc) and online services (arXiv, PubMed, CrossRef, CiNii, etc) . . . 220
4.6.6 Registering Bitstreams via Simple Archive Format . . . 222
4.6.7 Importing and Exporting Items via Simple Archive Format (SAF) . . . 224
4.6.8 Importing and Exporting Content via Packages . . . 232
4.6.9 Configurable Workflow . . . 236
4.6.10 Submission User Interface . . . 244
4.6.10.1 Live Import from external sources . . . 261
4.6.10.2 Simple HTML Fragment Markup . . . 268
4.6.10.3 Supervision Orders . . . 269
4.7 Items and Metadata . . . 272
4.7.1 Authority Control of Metadata Values . . . 273
4.7.1.1 ORCID Authority . . . 277
4.7.2 Batch Metadata Editing . . . 287
4.7.2.1 Batch Metadata Editing Configuration . . . 293
4.7.3 DOI Digital Object Identifier . . . 295
4.7.4 Item Level Versioning . . . 303
4.7.5 Mapping/Linking Items to multiple Collections . . . 310
4.7.6 Metadata Recommendations . . . 311
4.7.7 Moving Items . . . 312
4.7.8 PDF Citation Cover Page . . . 313
4.7.9 Updating Items via Simple Archive Format . . . 315
4.8 Managing Community Hierarchy . . . 317
4.9 ORCID Integration . . . 319
4.10 Researcher Profiles . . . 333
4.11 Statistics and Metrics . . . 336
4.11.1 SOLR Statistics . . . 337
4.11.1.1 SOLR Statistics Maintenance . . . 347
4.11.1.1.1 Testing Solr Shards . . . 352
4.11.2 DSpace Google Analytics Statistics . . . 355
4.11.3 Exchange usage statistics with IRUS . . . 358
4.12 User Interface . . . 360
4.12.1 User Interface Configuration . . . 361
4.12.2 User Interface Customization . . . 383

4.12.3 User Interface Debugging . . . 397
4.12.4 Accessibility . . . 398
4.12.5 Browse . . . 400
4.12.6 Discovery . . . 402
4.12.7 Contextual Help Tooltips . . . 418
4.12.8 IIIF Configuration . . . 420
4.12.9 Multilingual Support . . . 424
5. System Administration . . . 426
5.1 AIP Backup and Restore . . . 427
5.1.1 DSpace AIP Format . . . 442
5.2 Ant targets and options . . . 453
5.3 Command Line Operations . . . 455
5.3.1 Database Utilities . . . 457
5.3.2 Executing streams of commands . . . 459
5.4 Handle.Net Registry Support . . . 460
5.5 Logical Item Filtering and DOI Filtered Provider for DSpace . . . 463
5.6 Mediafilters for Transforming DSpace Content . . . 468
5.6.1 ImageMagick Media Filters . . . 472
5.7 Performance Tuning DSpace . . . 477
5.8 Ping or Healthcheck endpoints for confirming DSpace services are functional . . . 482
5.9 Scheduled Tasks via Cron . . . 483
5.10 Search Engine Optimization . . . 486
5.10.1 Google Scholar Metadata Mappings . . . 492
5.11 Troubleshooting Information . . . 493
5.12 Validating CheckSums of Bitstreams . . . 494
6. DSpace Development . . . 498
6.1 User Interface Design Principles & Accessibility . . . 499
6.2 REST API . . . 504
6.3 REST API v6 (deprecated) . . . 508
6.3.1 REST Based Quality Control Reports . . . 516
6.3.1.1 REST Reports - Collection Report Screenshots with Annotated API Calls . . . 521
6.3.1.2 REST Reports - Metadata Query Screenshots with Annotated API Calls . . . 527
6.3.1.3 REST Reports - Summary of API Calls . . . 531
6.4 Advanced Customisation . . . 534
6.4.1 DSpace Service Manager . . . 535
6.5 Curation Tasks . . . 537
6.5.1 Curation tasks in Jython . . . 541
6.6 Development Tools Provided by DSpace . . . 543
6.7 Services to support Alternative Identifiers . . . 544
6.8 Batch Processing . . . 548
6.9 Workflow . . . 549
7. DSpace Reference . . . 554
7.1 Configuration Reference . . . 555
7.2 DSpace Item State Definitions . . . 608
7.3 Directories and Files . . . 610
7.4 Metadata and Bitstream Format Registries . . . 613
7.5 Architecture . . . 618
7.5.1 Application Layer . . . 620
7.5.2 Business Logic Layer . . . 622
7.5.3 DSpace Services Framework . . . 639
7.5.4 Storage Layer . . . 643
7.6 History . . . 650
7.6.1 Changes in 7.x . . . 651
7.6.2 Changes in Older Releases . . . 654
8. Learning DSpace . . . 655
8.1 Community and Collection management . . . 657
8.1.1 'Assigning Roles' in Collections: How to Set Up Workflows and Grant Privileges . . . 658
8.1.2 Collection Management . . . 661
8.1.2.1 Create Collection . . . 662
8.1.2.2 Delete Collection . . . 667
8.1.2.3 Edit Collection . . . 670
8.1.2.4 Export Collection . . . 689
8.1.3 Community Management . . . 694
8.1.3.1 Create a Community . . . 695
8.1.3.2 Delete Community . . . 699
8.1.3.3 Edit Community . . . 702
8.2 Content (Item) management . . . 710
8.2.1 Add item . . . 711
8.2.2 Delete item . . . 722
8.2.3 Edit Item . . . 724
8.2.3.1 Authorizations (Manage access to an item) . . . 727
8.2.3.2 Collection Mapper . . . 734
8.2.3.3 Edit Bitstream . . . 740
8.2.3.4 Edit Metadata . . . 747
8.2.3.5 Edit Relationship . . . 756
8.2.3.6 Make an Item Private . . . 763
8.2.3.7 Move an Item . . . 767
8.2.3.8 Reinstate an item . . . 771
8.2.3.9 Versioned Item . . . 775

8.2.3.10 Withdraw an item . . . 782
8.2.4 Embargo an item . . . 786
8.2.5 Lease an item . . . 795
8.3 DSpace 7 Demo Quick Start . . . 805
8.4 Management sidebar . . . 806
8.5 Menus . . . 808
8.6 Registry management . . . 810
8.6.1 Metadata Registry Management . . . 811
8.7 Request-a-copy . . . 824
8.8 Search - Advanced . . . 825
8.9 User management . . . 826
8.9.1 Add or Manage an E-Person . . . 827
8.9.2 Create or manage a user group . . . 846

Introduction

DSpace is an open source software platform that enables organisations to:

- capture and describe digital material using a submission workflow module, or a variety of programmatic ingest options
- distribute an organisation's digital assets over the web through a search and retrieval system
- preserve digital assets over the long term

This system documentation includes a functional overview of the system, which is a good introduction to the capabilities of the system, and should be readable by non-technical folk. Everyone should read this section first because it introduces some terminology used throughout the rest of the documentation.

For people actually running a DSpace service, there is an installation guide, and sections on configuration and the directory structure. Support options are available in the DSpace Support Guide.

For those interested in the details of how DSpace works, and those potentially interested in modifying the code for their own purposes, there is a detailed ar chitecture section.

Other good sources of information are:

- The DSpace Support Guide lists various places to ask for help, report bugs or security issues, etc.
- The DSpace REST API contract which documents the REST API behavior, etc. If you want source code docs, we also provide JavaDocs for the Java API layer which can be built by running `mvn javadoc:javadoc`
- The DSpace Wiki contains stacks of useful information about the DSpace platform and the work people are doing with it. You are strongly encouraged to visit this site and add information about your own work.
- dspace.org has announcements and contains useful information about bringing up an instance of DSpace at your organization.
- The DSpace Community List. Join DSpace-Community to ask questions or join discussions about non-technical aspects of building and running a DSpace service. It is open to all DSpace users. Ask questions, share news, and spark discussion about DSpace with people managing other DSpace sites. Watch DSpace-Community for news of software releases, user conferences, and announcements about DSpace.
- The DSpace Technical List. DSpace developers & fellow community members help answer installation and technology questions, share information and help each other solve technical problems through the DSpace-Tech mailing list. Post questions or contribute your expertise to other developers working with the system.
- The DSpace Development List. Join Discussions among DSpace Developers. The DSpace-Dev listserv is for DSpace developers working on the DSpace platform to share ideas and discuss code changes to the open source platform. Join other developers to shape the evolution of the DSpace software. The DSpace community depends on its members to frame functional requirements and high-level architecture, and to facilitate programming, testing, documentation and to the project.

Release Notes

Upgrade from any past version of DSpace!

Installing DSpace provides an overview of the DSpace 7 installation process and all prerequisite software. You should review this before attempting an upgrade, in order to ensure you are running the required versions of Java, Node, etc.

Upgrading DSpace provides a guide for upgrading from *any old version* of DSpace to v7. As in the past, your data migrates automatically, no matter which older version you are running. *However, as the old XMLUI and JSPUI user interfaces are no longer supported, you must switch to using the new User Interface.*

Support for DSpace 7 ended on May 28, 2026. See DSpace Software Support Policy. DSpace 7.6.7 is the final release of the 7.x.x platform.

- 7.6.7 Release Notes
  - 7.6.7 Security Fixes
  - 7.6.7 Breaking Changes
  - 7.6.7 Major Bug Fixes and Improvements
  - 7.6.7 Acknowledgments
- 7.6.6 Release Notes
  - 7.6.6 Security Fixes
  - 7.6.6 Major Bug Fixes and Improvements
  - 7.6.6 New/Updated Language support
  - 7.6.6 Acknowledgments
- 7.6.5 Release Notes
  - 7.6.5 Major Bug Fixes and Improvements
  - 7.6.5 Acknowledgments
- 7.6.4 Release Notes
  - 7.6.4 Security Fixes
  - 7.6.4 Breaking Changes
  - 7.6.4 Major Bug Fixes and Improvements
  - 7.6.4 New/Updated Language support
  - 7.6.4 Acknowledgments
- 7.6.3 Release Notes
  - 7.6.3 Breaking Changes
  - 7.6.3 Major Bug Fixes and Improvements
  - 7.6.3 New/Updated Language support
  - 7.6.3 Acknowledgments
- 7.6.2 Release Notes
  - 7.6.2 Acknowledgments
- 7.6.1 Release Notes
  - 7.6.1 Acknowledgments
- 7.6 Release Notes
  - 7.6 Acknowledgments
- 7.5 Release Notes
  - 7.5 Acknowledgments
- 7.4 Release Notes
  - 7.4 Acknowledgments
- 7.3 Release Notes
  - 7.3 Acknowledgments
- 7.2.1 Release Notes (Backend Only)
- 7.2 Release Notes
  - 7.2 Acknowledgments
- 7.1.1 Release Notes (Backend Only)
- 7.1 Release Notes
  - 7.1 Acknowledgments
- 7.0 Release Notes
- 7.0 Configurations Removed
- 7.0 Acknowledgments
  - Major Contributing Institutions
  - Financial Contributors
  - Frontend / User Interface Acknowledgments
  - Backend / REST API Acknowledgments
  - Additional Thanks
- 7.0 Beta 1-5 Release Notes
  - 7.0 Beta 5 Release Notes
  - 7.0 Beta 4 Release Notes
  - 7.0 Beta 3 Release Notes
  - 7.0 Beta 2 Release Notes
  - 7.0 Beta 1 Release Notes

7.6.7 Release Notes

We highly recommend ALL DSpace 7.x users upgrade to 7.6.7, or patch their site.

DSpace 7.6.7 contains four security fixes related to the DSpace REST API, along with a patch for an Angular SSR vulnerability that could impact some installations of the DSpace User Interface. See details below.

DSpace 7.6.7 was released on May 27, 2026

To try out DSpace 7.6.7 immediately, see Try out DSpace 7.

To upgrade to DSpace 7.6.7 from 7.x *or any prior version*, see Upgrading DSpace

- To upgrade to 7.6.7, you MUST upgrade *both* the backend and frontend (user interface). Many bug fixes require updating *both.*

To install DSpace 7.6.7 for the first time, see Installing DSpace.

- Download DSpace 7.6.7 Backend: https://github.com/DSpace/DSpace/releases/tag/dspace-7.6.7
- Download DSpace 7.6.7 User Interface: https://github.com/DSpace/dspace-angular/releases/tag/dspace-7.6.7

DSpace 7.6.7 provides security patches, bug fixes & performance improvements to the 7.x platform. *No new features are provided.* As such this release should be an easier upgrade for sites already running 7.x.

7.6.7 Security Fixes

- **Fix for GHSA-9x82-rm84-c6x7** *(high severity*). Remote Code Execution (RCE) possible in Velocity templates used by LDN (Linked Data Notifications) when COAR Notify is enabled. *(NOTE: A CVE ID has been requested but not yet assigned)*
- **Fix for GHSA-9qm4-rh6w-pq5x** *(moderate severity).* Path traversal vulnerability possible in LDN (Linked Data Notifications) message generation when COAR Notify is enabled. *(NOTE: A CVE ID has been requested but not yet assigned)*
- **Fix for GHSA-v66x-68f2-pxf5** *(moderate severity).* Path Traversal Vulnerability is possible in Curation Task Reporter output path. *(NOTE: A CVE ID has been requested but not yet assigned)*
- **Fix for GHSA-c827-pw3m-67w7** *(moderate severity)*. ORE resource URI does not validate scheme for non-web resources when harvesting OAI content. *(NOTE: A CVE ID has been requested but not yet assigned)*
- **Patch for CVE-2026-27739 in Angular SSR** *(critical severity).* All versions of Angular SSR (Server Side Rendering) contain a critical SSRF (Server-Side Request Forgery) vulnerability. This SSRF vulnerability may be possible to exploit in DSpace sites that are **not** running DSpace behind a well-configured proxy (see mailing list announcement). This release of DSpace provides additional protection via the new "ui > baseUrl" setting (see "Breaking Changes" below)
- This release also contains many dependency updates in order to keep all DSpace sites secure. Some of these updates patch vulnerabilities that have been reported by those dependencies. But no exploits of these vulnerabilities have been confirmed in DSpace.

7.6.7 Breaking Changes

Some fixes in 7.6.7 required major code or configuration changes. The following changes *may* negatively impact or "break" your local customizations to prior versions of DSpace. Please be aware of them before upgrading.

- **Frontend's new "baseUrl" setting is required for robots.txt and redirects.** The frontend configuration has a newly added "ui > baseUrl" setting which should match the value of "dspace.ui.url" on the backend. This setting is REQUIRED for robots.txt and redirects to function properly. This new setting also helps to patch against the Angular SSR vulnerability CVE-2026-27739.
- **Replaced "webui.content_disposition_format" with "webui.content_disposition_inline"** - In prior versions the "webui.content_disposition_format" setting (in dspace.cfg) listed MIME Types which are **not allowed** be displayed inline (in a user's browser). This setting has been replaced by a "webui.content_disposition_inline" setting which lists MIME types which **are allowed** to be displayed inline. By default PDF, GIF, JPEG, PNG, MPG and MP4 are allowed to be displayed inline. This improves security of unknown or custom formats by ensuring they don't default to being displayed inline. See https://github.com/DSpace/DSpace/pull/11956

7.6.7 Major Bug Fixes and Improvements

- Fixed bug on Browse by Issue Date where the date was treated as a filter rather than a start date for decades. #10055 *(Donated by Atmire)*
- Fixed a bug where the IIIF viewer was not working due to a missing `mime.types` file. *#11804 (Donated by 4Science)*
- Fixed a bug where authentication methods would not appear if page refreshed before authentication token expired. #4662 *(Donated by Atmire)*
- Fixed a bug in hierarchical vocabulary browse where only the first 20 matches to a query were rendered. #4500 *(Donated by Atmire)*
- Fixed an issue where deleted bitstreams returned HTTP 401 unauthorized instead of 404. #11629 *(Donated by Jesiel Viana)*
- Fixed bug where LDAP authentication would fail with when a user's LDAP entry had no email field. #11292 *(Donated by dataquest)*
- Fixed bug where metadata export was no longer respecting `metadata.hide.*` properties #11197 *(Donated by Atmire)*
- Fixed an issue where the "New process" dropdown would not fully load when browser zoom was adjusted #4577 *(Donated by 4Science)*
- Fixed bug occurring during OAI update process when batch size exceeded and items had metadata-level embargoes. #12112 *(Donated by Toni Prieto)*
- Added missing Entity-related metadata fields to the Metadata Registry. #3353
- Fixed bug where OAI identifier prefix could revert back to un-interpolated value when not defined in config. #12505 *(Donated by Atmire)*
- Submission / Workflow enhancements and fixes
  - Fixed an issue where loading many mapped collection forms via `item-submission` could cause a Fetch error. #10750 *(Donated by Paulo Graça)*
  - Fixed bug where a Submitter could not deposit items via SWORD when the item had an embargo defined. #10404 *(Donated by dataquest)*
  - There is now a configurable limit on the number of items that can be added or edited in a single CSV metadata import. #9663 *(Donated by Neki-IT)*
  - Fixed an issue where controlled vocabulary lookup was not working for text with accented characters. #12097 *(Donated by Istvan Vig)*
- Administrative enhancements and fixes
  - Fixed an issue where the checksum checker could fail to complete due to memory constraints in repositories with many bitstreams. #7322 *(Donated by Miika Nurminen)*
  - Fixed an issue where the curation task CreateMissingIdentifiers would not work for an item without a handle. #11676 *(Donated by The Library Code)*
  - Fixed bug where it was not possible to set the DELETE resource policy. #10741 *(Donated by DSquare Technologies)*
- Performance improvements

    - Improved performance by optimizing the SQL query in the `findByEPerson` method. #11472 *(Donated by Toni Prieto)*
    - Improved performance on the signposting endpoint by adding a count query. #12305 *(Donated by Tina Schönborn)*
    - Improved loading times and cache behavior for the community list page. #9911 *(Donated by 4Science)*
    - Improved performance for relationship queries by caching the tables that store entity and relationship types. #12511 *(Donated by Atmire)*
- Fixed other small bugs. See Changes in 7.x for a list of all changes.

7.6.7 Acknowledgments

The DSpace application would not exist without the hard work and support of its community. Thank you to the many developers who have worked very hard to deliver all the bug fixes and improvements. This release was *entirely volunteer driven!*

Development Acknowledgments

A total of 39 unique individuals contributed to 7.6.7.

Frontend / User Interface Acknowledgments

The following 19 individuals have contributed directly to the new DSpace (Angular) User Interface in this release (ordered by number of GitHub commits): Tim Donohue (tdonohue), Zoltán Kanász-Nagy (kanasznagyzoltan), Oscar Chacon (oscar-escire), Toni Prieto (toniprieto), Gaurav Patel (GauravD2t), Alan Orth (alanorth), Art Lowel (artlowel), Guillermo Escalante (guillermo-escire), Andrea Barbasso (AndreaBarbasso), Bram Maegerman (bram-maegerman), Francesco Molinaro (FrancescoMolinaro), Jesiel Viana (jesielviana), Juraj Roka, Marcin Miosz (MMilosz), Shivani Rupnawar (ShivaniNR), Yury Bondarenko (ybnd), im-shubham-vish, Mukunda Rao Katta (MukundaKatta), Pawel Zlakowski (pzlakowski).

The above contributor list was determined based on contributions to the "dspace-angular" project in GitHub between 7.6.6 (after December 17, 2025) and 7.6.7 using "git shortlog" on the `dspace-7_x` branch and excluding all merge commits: `git shortlog -s -n -e --no-merges --since 2025-12-17`

Backend / REST API Acknowledgments

The following 26 individuals have contributed directly to the DSpace backend (REST API, Java API, OAI-PMH, etc.) in this release (ordered by number of GitHub commits): Tim Donohue (tdonohue), Kim Shepherd (kshepherd), Abhinav Sidharthan (AbhinavS96), Toni Prieto (toniprieto), Alan Orth (alanorth), Michaela Paurikova (Paurikova2), Istvan Vig (parphis), Pascal-Nicolas Becker (pnbecker), Paulo Graça (paulo-graca), Johnny Mendes do Carmo (JohnnyMendesC), Mark Patton (markpatton), Bram Luyten (bram-atmire), Milan Majchrak (milanmajchrak), April Herron (aprilherron), Drew Heles (dheles), Eike Löhden (EikLoe), Francesco Pio Scognamiglio (francescopioscognamiglio), Jesiel Viana (jesielviana), Marcin Miosz (MMilosz), Marie Verdonck (MarieVerdonck), Miika Nurminen (minurmin), Nathan Buckingham (ConfusionOrb221), Piaget Bouaka Donfack (PiagetBouaka), Stefano Maffei (steph-ieffam), Tina Schönborn (tinsch), Yury Bondarenko (ybnd).

The above contributor list was determined based on contributions to the "DSpace" project in GitHub between 7.6.6 (after December 17, 2025) and 7.6.7 using "git shortlog" on the `dspace-7_x` branch and excluding all merge commits: `git shortlog -s -n -e --no-merges --since 2025-12-17`

7.6.6 Release Notes

We highly recommend ALL DSpace 7.x users upgrade to 7.6.6, or patch their site.

DSpace 7.6.6 contains an update to Apache Tika v3.2.3 in the backend (REST API) which patches a severe vulnerability in that tool. See CVE-2025-54988, CVE-2025-66516 and #11678 for details.

DSpace 7.6.6 was released on December 17, 2025

To try out DSpace 7.6.6 immediately, see Try out DSpace 7.

To upgrade to DSpace 7.6.6 from 7.x *or any prior version*, see Upgrading DSpace

- To upgrade to 7.6.6, you MUST upgrade *both* the backend and frontend (user interface). Many bug fixes require updating *both.*

To install DSpace 7.6.6 for the first time, see Installing DSpace.

- Download DSpace 7.6.6 Backend: https://github.com/DSpace/DSpace/releases/tag/dspace-7.6.6
- Download DSpace 7.6.6 User Interface: https://github.com/DSpace/dspace-angular/releases/tag/dspace-7.6.6

DSpace 7.6.6 provides security patches, bug fixes & performance improvements to the 7.6.x platform. *No new features are provided.* As such this release should be an easier upgrade for sites already running 7.6.x.

7.6.6 Security Fixes

- **Patch for CVE-2025-66516 / CVE-2025-54988 in Apache Tika** *(critical severity)*. All versions of Apache Tika prior to version 3.2.2 contain a critical XML External Entity (XXE) vulnerability. This XXE vulnerability may be possible to exploit in DSpace if an attacker has submitter privileges. See #11678 for more details.
    - After upgrading, we recommend all sites recreate text extracted files. This is a safety measure to ensure that none of those text extracted files contain unexpected information because of a prior exploit of this XXE vulnerability. To recreate your text extracted files, run:

```
# This command will force all current text-extracted files to be deleted and recreated.
./dspace filter-media -f -p "Text Extractor"
```

- **Fix for potential XPath Injection in Controlled Vocabulary lookup.** This vulnerability only allows an attacker access to the controlled vocabulary XML file(s), which are already public in source code. So, this vulnerability poses no security risk but it has been patched for future code safety. See #11042 and #11075 *(Reported by odaysec. Fixed by odaysec and Mark Wood)*
- This release also contains many dependency updates in order to keep all DSpace sites secure. Some of these updates patch vulnerabilities that have been reported by those dependencies. But no exploits of these vulnerabilities have been confirmed in DSpace.

7.6.6 Major Bug Fixes and Improvements

- General user enhancements and fixes
  - Fixed bug where HTML tags in some metadata fields were wrongly evaluated in administrative workflow and search results. #1495 *(Donated by Atmire)*
  - Fixed bug where bitstreams with embargo lift date in metadata were not appearing on Item Page. #8640 *(Donated by Vir Softech)*
  - Fixed bug where Item Page could throw an error when signposting is enabled and the item has a large number of bitstreams. #8943 *(Donated by Nicholas Woodward)*
  - Fixed bug in "Browse by Title" where searching for a title beginning with certain articles (e.g. A, As, O, Os) did not return accurate results. #4166 *(Donated by 4Science)*
  - Fixed bug on "Browse by" pages where the search box no longer worked after clicking on a value on the page. #2739 *(Donated by Atmire)*
  - Fixed bug where exporting search results didn't work if the search contained quotation marks. #9589 *(Donated by Vir Softech)*
  - When an Entity's Item Page displays tabbed search, switching tabs no longer retains the query parameters from the prior tab. #2729 *(Donated by Atmire)*
  - Fixed issues with downloading bitstreams whose filenames contained non-ASCII characters using the Safari browser. #11191 *(Donated by Neki-IT)*
  - The language selected in a user's profile is now the default language displayed by DSpace unless a different language is selected via the header. #4241 *(Donated by Atmire)*
- Submission / Workflow enhancements and fixes
  - Fixed bug where dynamic tag input field was wrongly splitting values on unexpected characters (like "<"). Now it only splits on pressing Enter or adding a comma. #4664 *(Donated by The Library Code)*
  - Fixed bug where a newly created Researcher Profile could not be selected as an author of a new Publication Entity. #4648 *(Donated by Jesiel Viana)*
  - Fixed bug where file uploads would fail if the "assetstore.dir" was set to a location using a symbolic link. #11169 *(Donated by 4Science)*
  - Fixed memory leak in submission form which could occur if you closed and reopened the form several times in a row. #4645 *(Donated by Jukka Lipka)*
  - Fixed bug where output of "submission-forms-migrate" script was an invalid submission configuration. #8443 *(Donated by Neki-IT)*
- Statistics enhancements and fixes
  - Add several "usage-statistics.*" configurations to make it easier to quickly customize the statistical reports. #10142 *(Donated by Nicholas Woodward)*
  - Updated Solr-based internal statistics to only track downloads of bitstreams that are in a bundle listed in the existing "solr-statistics.query.filter.bundles" configuration. This defaults to only tracking downloads of bitstreams in the "ORIGINAL" bundle. #11033 *(Donated by Neki-IT)*
  - Fixed bug where IRUS statistics event processor used the wrong OAI-PMH url. #8852 *(Donated by Neki-IT)*
- Authentication fixes
  - Fixed database connection leak that could occur via many unsuccessful logins using password login or ORCID login. #11202 *(Donated by Atmire)*
  - Fixed issues with correctly populating special groups for several authentication methods. #11633 *(Donated by 4Science)*
  - Fixed bug in Shibboleth authentication where EPerson metadata was not being updated after login. #11568 *(Donated by Neki-IT)*
  - Fixed bug where the registration page and forgot password page were accessible to authenticated users. #4592 *(Donated by 4Science)*
  - Support for X.509 certificate authentication has been removed as it appears to be unused and non-functional. #11334
- Integration fixes
  - Fixed bug where import of a Person Entity from ORCID was not working. #11674 *(Donated by 4Science)*
  - Fixed bug where ORCID Integration would reject a work if it referenced an ISSN that already exists (e.g. an article referencing a journal via SSN). #10738 *(Donated by The Library Code)*
  - Fixed bug where ORCID queue list would include items that are still in workspace or workflow. #4654 *(Donated by Pierre Lasou)*
  - Improved mappings between ORCID document type and DSpace's dc.type field #11253 *(Donated by Pierre Lasou)*
  - Import from PubMed now supports adding your PubMed API Key via a new "pubmed.apiKey" configuration in "external-providers.cfg". #10356 *(Donated by Atmire)*
  - Fixed bug where OAI-PMH could crash if it encountered forbidden XML characters in metadata. #10721 *(Donated by Neki-IT)*
  - Fixed performance of SWORDv2 servicedocument when you have a large number of collections. #10740 (Donated by fribeiro-fccn)
  - Fixed bug where SWORDv2 read/update/delete actions were failing with a 500 error. #11547 *(Donated by Marsa Haoua)*
  - Fixed bug where SWORDv1 submission failed if the user was not an Administrator. #8647
- Search Engine Optimization (SEO) fixes
  - Fixed bug where private (non-discoverable) item pages were missing the "noindex" meta tag that tells crawlers to avoid indexing the item. #4835 *(Donated by Atmire)*
  - Added 'rel="nofollow"' to all search facets on the search page to tell well-behaved bots not to follow those links. #4785 *(Donated by Alan Orth)*
- Administrative enhancements and fixes
  - Fixed bug where Item Versioning could fail if special characters were added to the "Summary" field. #4327 *(Donated by Jukka Lipka)*
  - Fixed bug where client-side rendering (CSR) of the user interface could fail if the "ui.nameSpace" configuration was customized in "config.*.yml". #4637 *(Donated by Milan Majchrak)*
  - Fixed bug where backend would fail to startup when "citation-page.enabled_communities" configuration was enabled. #11167 *(Donated by Miika Nurminen)*
  - Fixed bug where "checker" and "bitstore-migrate" command-line scripts would not save updates / current status if the script was interrupted. #10507 and #10489 *(Donated by David Steelman)*
  - Fixed bug where "subscription-send" command-line script was not sending updates about restricted items when the user has permissions on those restricted items. #10506 *(Donated by dataquest)*
  - Fixed bug where "filter-media" command-line script did not understand orientation metadata in jpeg images. #10534 *(Donated by 4Science)*
  - Fixed bug where running "export" command-line script on an Item in workspace or workflow would return an error even though the export succeeds. #11074 *(Donated by Martin Walk)*

    - Fixed open connection leak in "import" command-line script. #10886 *(Donated by Atmire)*
    - Fixed bug where "cleanup" command-line script would fail if a request-for-copy existed in the database for the deleted bitstream. #9674 *(Donated by Atmire)*
    - Amazon S3 integration has been updated to use AWS SDK v2. #11532 *(Donated by Mark Patton)*
- Fixed other small bugs. See Changes in 7.x for a list of all changes.

7.6.6 New/Updated Language support

- Czech (eština) language updates donated by dataquest
- French (Français) language updates donated by Nima Behforouz (nimabehforouz)
- Ukrainian (Y) language updates donated by Olexandr Shaposhnyk (oshaposhnyk)

7.6.6 Acknowledgments

The DSpace application would not exist without the hard work and support of its community. Thank you to the many developers who have worked very hard to deliver all the bug fixes and improvements. This release was *entirely volunteer driven!*

Development Acknowledgments

A total of 41 unique individuals contributed to 7.6.6.

Frontend / User Interface Acknowledgments

The following 17 individuals have contributed directly to the new DSpace (Angular) User Interface in this release (ordered by number of GitHub commits): Alan Orth (alanorth), Francesco Molinaro (FrancescoMolinaro), Milan Majchrak (milanmajcrak), Alexandre Vryghem (alexandrevryghem), Jesiel Viana (jesielviana), Jukka Lipka (jlipka), Kim Shepherd (kshepherd), Tim Donohue (tdonohue), Abhinav Sidharthan (AbhinavS96), Matus Kasak, Kristof De Langhe (Atmire-Kristof), Nima Behforouz (nimabehforouz), Toni Prieto (toniprieto), Zahraa Chreim (ZahraaChreim-Atmire), Bram Maegerman (bram-maegerman), Nicholas Woodward (nwoodward), Olexandr Shaposhnyk (oshaposhnyk).

The above contributor list was determined based on contributions to the "dspace-angular" project in GitHub between 7.6.5 (after July 22, 2025) and 7.6.6 using "git shortlog" on the `dspace-7_x` branch and excluding all merge commits: `git shortlog -s -n -e --no-merges --since 2025-07-22`

Backend / REST API Acknowledgments

The following 33 individuals have contributed directly to the DSpace backend (REST API, Java API, OAI-PMH, etc.) in this release (ordered by number of GitHub commits): Tim Donohue (tdonohue), Johnny Mendes do Carmo (JohnnyMendesC), Nicholas Woodward (nwoodward), Alan Orth (alanorth), Miika Nurminen (minurmin), Kim Shepherd (kshepherd), Vincenzo Mecca (vins01-4science), Alexandre Vryghem (alexandrevryghem), David Steelman (dsteelma-umd), Michele Boychuk (Micheleboychuk), Sascha Szott (saschaszott), Elios Buzo (eliosbz), Joran De Braekeleer (gingyx), Kristof De Langhe (Atmire-Kristof), Pierre Lasou (pilasou), Zahraa Chreim (ZahraaChreim-Atmire), Abhinav Sidharthan (AbhinavS96), Andreas Czerniak (ACz-UniBi), Daniel Coelho (danielcoelhocgu), Jesiel Viana (jesielviana), Marie Verdonck (MarieVerdonck), Mark Wood (mwoodiupui), Mark Patton (markpatton), Marsa Haoua (marsaoua), Martin Walk (MW3000), Nathan Buckingham (ConfusionOrb221), Paulo Graça (paulo-graca), Stefano Maffei (steph-ieffam), fribeiro-fccn, Igor Baptista da Costa (IgorBaptist4), im-shivamb, im-shubham-vish, Jozef Misutka (vidiecan).

The above contributor list was determined based on contributions to the "DSpace" project in GitHub between 7.6.5 (after July 22, 2025) and 7.6.6 using "git shortlog" on the `dspace-7_x` branch and excluding all merge commits: `git shortlog -s -n -e --no-merges --since 2025-07-22`

7.6.5 Release Notes

DSpace 7.6.5 only includes dependency updates

DSpace 7.6.5 only includes a few dependency updates on top of the 7.6.4 release. The most significant of those updates is an update to log4j version 2.25.1, which fixes severe backend logging issues that were occurring in DSpace 7.6.4.

DSpace 7.6.5 was released on July 22, 2025

To try out DSpace 7.6.5 immediately, see Try out DSpace 7.

To upgrade to DSpace 7.6.5 from 7.x *or any prior version*, see Upgrading DSpace

- To upgrade to 7.6.5, you MUST upgrade *both* the backend and frontend (user interface). Many bug fixes require updating *both.*

To install DSpace 7.6.5 for the first time, see Installing DSpace.

- Download DSpace 7.6.5 Backend: https://github.com/DSpace/DSpace/releases/tag/dspace-7.6.5
- Download DSpace 7.6.5 User Interface: https://github.com/DSpace/dspace-angular/releases/tag/dspace-7.6.5

DSpace 7.6.5 only provides a few dependency updates to the 7.6.x platform. *No new features are provided.* As such this release should be an easier upgrade for sites already running 7.6.x.

7.6.5 Major Bug Fixes and Improvements

- Updates the backend to use log4j version 2.25.1. The prior version of log4j (v2.25.0) contains a severe bug that resulted in no `INFO` level messages being logged to the `dspace.log` file. See #11046
- Other minor dependency updates to the User Interface.
- See Changes in 7.x for a list of all changes.

7.6.5 Acknowledgments

The DSpace 7.6.5 release only included dependency updates from GitHub's dependabot and minor fixes to automated tests (from Tim Donohue). Nonetheless, we thank the community members who contributed to this release via testing of this release.

7.6.4 Release Notes

The 7.6.4 release uses log4j 2.25.0, which breaks backend logging.

Unfortunately, after 7.6.4 was released, we discovered that it included log4j version 2.25.0, which contains a severe bug that results in no `INFO` level messages being logged to the `dspace.log` file. In your logs, you will only see `WARN` and `ERROR` messages, regardless of your log4j configuration. See https://github.com/DSpace/DSpace/issues/11046

The fix for this bug is to upgrade to either 7.6.5 (see above) or patch your backend to use log4j version 2.25.1 by modifying the "log4j.version" tag in the DSpace backend's root `/pom.xml`

```
<log4j.version>2.25.1</log4j.version>
```

After making that modification, you will need to rebuild the DSpace backend ("mvn -U clean package") and reinstal ("ant update").

We highly recommend ALL DSpace 7.x users upgrade to 7.6.4 (or later), or patch their site.

DSpace 7.6.4 contains two security fixes to the backend (REST API) for all sites running 7.x (or below). See CVE-2025-53621 and CVE-2025-53622 for details.

DSpace 7.6.4 was released on July 14, 2025

To try out DSpace 7.6.4 immediately, see Try out DSpace 7.

To upgrade to DSpace 7.6.4 from 7.x *or any prior version*, see Upgrading DSpace

- To upgrade to 7.6.4, you MUST upgrade *both* the backend and frontend (user interface). Many bug fixes require updating *both.*

To install DSpace 7.6.4 for the first time, see Installing DSpace.

- Download DSpace 7.6.4 Backend: https://github.com/DSpace/DSpace/releases/tag/dspace-7.6.4
- Download DSpace 7.6.4 User Interface: https://github.com/DSpace/dspace-angular/releases/tag/dspace-7.6.4

DSpace 7.6.4 provides bug fixes, accessibility & performance improvements to the 7.6.x platform. *No new features are provided.* As such this release should be an easier upgrade for sites already running 7.6.x.

7.6.4 Security Fixes

- **Fix for CVE-2025-53621** *(moderate severity)*. XML External Entity (XXE) injection possible in import via Simple Archive Format (SAF) or import from external sources. See security advisory or mailing list “security notice” for details.
- **Fix for CVE-2025-53622** *(moderate severity).* Path traversal vulnerability in Simple Archive Format (SAF) package import via “contents” file. See security advisory or mailing list “security notice” for details.
- Fix possible "Zip Slip" in SWORDv2 ingest process. This potential issue was found via an automated code scan and no exploit has been confirmed. See #10725
- This release also includes many dependency updates in order to keep all DSpace sites secure. Some of these updates patch vulnerabilities that have been reported by those dependencies. But no exploits of these vulnerabilities have been confirmed in DSpace.

7.6.4 Breaking Changes

Some fixes in 7.6.4 required major code or configuration changes. The following changes *may* negatively impact or "break" your local customizations to prior versions of DSpace. Please be aware of them before upgrading.

- **Frontend configuration "ssr > paths" has been replaced by "ssr > excludePathPatterns".** In DSpace 7.6.3, it was possible to only enable SSR (server-side rendering) for specific User Interface paths (configured by the "ssr > paths" configuration in your "config.*.yml" file. This setting has been replaced by the inverse configuration, which allows you to **exclude** specific paths from SSR. See "excludePathPatterns" in the "ssr" section of the User Interface Configuration and #4227 *(Donated by 4Science)*
  - *This change was necessary to fix an SEO bug where error pages and Handle redirects would return "200 OK" responses when the prior configuration ("ssr > paths") was used.* See #4132 and #4042

7.6.4 Major Bug Fixes and Improvements

- General user enhancements and fixes
  - Fixed search bug where stale data could sometimes be returned. #3888 *(Donated by Atmire)*
  - Fixed bug where access-restricted thumbnails were not shown to users who have proper access rights. #4165 *(Donated by Atmire)*
  - Fixed statistics bug where "view" events could be sent twice when using scoped themes. #4099 *(Donated by Atmire)*
  - Fixed bug where sorting in descending order on Browse by Author may result in an error. #10617 *(Donated by Toni Prieto)*
- Submission / Workflow enhancements and fixes
  - Fixed issue where search filters could disappear from MyDSpace page. #4097 *(Donated by Arvo Consultores)*
  - Fixed infinite loading issue that could occur on submission forms. #4387 *(Donated by Atmire)*
  - Fixed bug where an admin could not upload a bitstream to an item in workflow when impersonating a user. #4307 *(Donated by 4Science)*

    - Fixed bug where fields of type "name" in submission forms could not be deleted properly. #4073 *(Donated by Atmire)*
    - Fixed bug where the "preloadLevel" for a controlled vocabulary was being ignored. This setting should let sites decide how many levels to automatically open in any hierarchical vocabulary. #3821 *(Donated by Atmire)*
    - ArXiv importer began failing because its configuration didn't use the HTTPS URL. *#10998 (Donated by Atmire)*
    - CrossRef importer wasn't mapping to default DSpace document types. #9900 *(Donated by Paulo Graça)*
    - Web of Science importer was enhanced to provide better error handling. #10692 *(Donated by 4Science)*
    - Improved proxy support for all external sources/APIs. All external connections should now use the DSpace Proxy Configuration. See also #10639 and #10723 *(Donated by 4Science)*
    - Alphabetized all external sources in the import dropdown by source name. #10806
- Administrative enhancements and fixes
    - Health page now includes an "SEO" validation check. A basic check of your DSpace site's Search Engine Optimization is now available on your Health page (/health) in the Admin sidebar. This SEO report checks that your sitemap is visible, your robots.txt is visible and that you have SSR (Server Side Rendering) enabled. See #10485 *(Donated by Atmire)*
        - This minor enhancement helps sites to find and resolve SEO issues which are frequently reported by Google Scholar (and similar search engines).
    - Fixed bug where list of bitstreams on "Edit Item > Bitstreams" tab would be duplicated when using pagination. #4015 *(Donated by Atmire)*
    - Fixed bug where editing a bitstream and changing its file format would still display the old (cached) format. #4137 *(Donated by Jesiel Viana)*
    - Fixed issued where Collection Admins could not add new policies to bitstreams. #10952 *(Donated by 4Science, backported by @max-nuding)*
    - Fixed issued where Collection Admins could not view metadata of withdrawn items. #10681 *(Donated by 4Science)*
    - Fixed issue where, when impersonating another users, an Administrator could not upload files to items in workflow approval. #4288 *(Donated by 4Science)*
    - Fixed pagination issue with Bulk Access Control script/page which could cause duplicate policies to be created. #10698 *(Donated by 4Science)*
- Performance/stability improvements:
    - Limited the number of Items that can be exported at once to 500. This limit is configurable on backend via "bulkedit.export.max.items" #10030 *(Donated by Atmire)*
    - Fixed issue where login endpoints to REST API didn't always close their database connection after login action was completed. #10598 *(Donated by Atmire)*
    - OpenSearch now limits its results per page to 100, though it is configurable via "websvc.opensearch.max_num_of_items_per_request" setting. #10645 *(Donated by Sascha Szott)*
    - Indexing improvements to the "index-discovery" script when your site has a larger number of relationships (between entities). #10353 *(Donated by Atmire)*
    - Fixed bug when DSpace could fail to startup when ORCID connections failed (if ORCID is enabled). #10551 *(Donated by The Library Code)*
    - Fixed bug where, when enabled, PDF CoverPages would throw an error if the PDF was opened in the browser. #9989 and #9665 *(Donated by The Library Code, backported by Miika Nurminen)*
- Accessibility improvements:
    - Accessibility Settings can be customized by users. Basic accessibility settings like Notification (popup) timeouts and ARIA Live Region timeouts can now be customized by each user of your site via the "Accessibility Settings" link in the footer. This allows users more control over how long confirmation and error messages are displayed. #3615 *(Built by Atmire, funded via the DSpace Development Fund)*
    - Truncated abstracts now use ellipsis for truncation instead of fading out. #4342 *(Built by 4Science, funded via the DSpace Development Fund)*
    - Improved screen reader accessibility for header and recent submissions. #4335 *(Donated by Atmire)*
    - Enhanced keyboard navigation on several pages. #4314 *(Donated by eScire)*
- OpenAIRE v4 compliance improvements to both the OpenAIRE submission forms and the OAI-PMH "oai_openaire" metadata format. #10174 *(Built by Atmire, funded from Ireland's National Open Research Forum (NORF) under the 2022 Open Research Fund)*
- Fixed a REST API bug where sending a PATCH request for an unknown metadata field could clear all object metadata. (This only impacts REST API and not the UI.) #10961 *(Donated by Atmire)*
- Fixed a large number of other small bugs. See Changes in 7.x for a list of all changes.

7.6.4 New/Updated Language support

- French (Français) language updates donated by Pierre Lasou (pilasou) and Carolyn Sullivan (Peredwel)
- German (Deutsch) language updates donated by Sascha Szott (saschaszott)

7.6.4 Acknowledgments

The DSpace application would not exist without the hard work and support of its community. Thank you to the many developers who have worked very hard to deliver all the bug fixes and improvements. This release was *entirely volunteer driven!*

Development Acknowledgments

A total of 42 unique individuals contributed to 7.6.4.

Frontend / User Interface Acknowledgments

The following 27 individuals have contributed directly to the new DSpace (Angular) User Interface in this release (ordered by number of GitHub commits): Andreas Awouters (AAwouters), Alexandre Vryghem (alexandrevryghem), Dan Gastardelli (DanGastardelli), Francesco Molinaro (FrancescoMolinaro), Jens Vannerum (jensvannerum), Adamo Fapohunda (AdamF42), Alan Orth (alanorth), Art Lowel (artlowel), Kristof De Langhe (Atmire-Kristof), Sascha Szott (saschaszott), Jesiel Viana (jesielviana), Oscar Chacón (oscar-escire), Sergio Fernández Celorio (sergius02), Tim Donohue (tdonohue), Lotte Hofstede (LotteHofstede), Carolyn Sullivan (Peredwel), Abhinav Sidharthan (AbhinavS96), Andrea Barbasso (AndreaBarbasso), Giuseppe Digilio (atarix83), Joran De Braekeleer (gingyx), Kim Shepherd (kshepherd), Nona Luypaert (nona-luypaert), Pierre Lasou (pilasou), Victor Hugo Duran Santiago (VictorHugoDuranS), anaparvo, Francesco Bacchelli (frabacche), Escalante Guillermo (guillermo2519).

The above contributor list was determined based on contributions to the "dspace-angular" project in GitHub between 7.6.3 (after February 7, 2025) and 7.6.4 using "git shortlog" on the `dspace-7_x` branch and excluding all merge commits: `git shortlog -s -n -e --no-merges --since 2025-02-07`

Backend / REST API Acknowledgments

The following 24 individuals have contributed directly to the DSpace backend (REST API, Java API, OAI-PMH, etc.) in this release (ordered by number of GitHub commits): Tim Donohue (tdonohue), Kim Shepherd (kshepherd), Stefano Maffei (steph-ieffam), Jens Vannerum (jensvannerum), Sascha Szott (saschaszott), Elios Buzo (eliosbz), Adamo Fapohunda (AdamF42), Michele Boychuk (Micheleboychuk), Max Nuding (max-nuding), Alan Orth (alanorth), Toni Prieto (toniprieto), Abhinav Sidharthan (AbhinavS96), Jesiel Viana (jesielviana), Alexandre Vryghem (alexandrevryghem), Agustina Martinez (amgciadev), Marcin Miosz (MMilosz), Mark Wood (mwoodiupui), Martin Walk (MW3000), Pascal-Nicolas Becker (pnbecker), Paulo Graça (paulo-graca), Piaget Bouaka Donfack (PiagetBouaka), Yury Bondarenko (ybnd), Zahraa Chreim (ZahraaChreim-Atmire), James Sullivan (jameswsullivan).

The above contributor list was determined based on contributions to the "DSpace" project in GitHub between 7.6.3 (after February 7, 2025) and 7.6.4 using "git shortlog" on the `dspace-7_x` branch and excluding all merge commits: `git shortlog -s -n -e --no-merges --since 2025-02-07`

7.6.3 Release Notes

DSpace 7.6.3 was released on February 7, 2025

To try out DSpace 7.6.3 immediately, see Try out DSpace 7.

To upgrade to DSpace 7.6.3 from 7.x *or any prior version*, see Upgrading DSpace

- To upgrade to 7.6.3, you MUST upgrade *both* the backend and frontend (user interface). Many bug fixes require updating *both.*

To install DSpace 7.6.3 for the first time, see Installing DSpace.

- Download DSpace 7.6.3 Backend: https://github.com/DSpace/DSpace/releases/tag/dspace-7.6.3
- Download DSpace 7.6.3 User Interface: https://github.com/DSpace/dspace-angular/releases/tag/dspace-7.6.3

DSpace 7.6.3 provides bug fixes, accessibility & performance improvements to the 7.6.x platform. *No new features are provided.* As such this release should be an easier upgrade for sites already running 7.6.x.

7.6.3 Breaking Changes

Some accessibility and bug fixes in 7.6.3 required larger code or configuration changes. The following changes *may* negatively impact or "break" your local customizations to prior versions of DSpace. Please be aware of them before upgrading.

- **Angular Server Side Rendering (SSR) is no longer performed for all pages in DSpace.** By default, new limitations are places on which pages (or parts of pages) are rendered via SSR. This was done to help reduce the CPU/memory that is used by bot traffic. Many bots will trigger SSR unnecessarily, resulting in potential performance issues. You may wish to review these new settings and adapt them, as necessary, to your production site. These new settings are all configurable, allowing you to decide if the new behavior or old behavior is desired.
  - By default, SSR is only performed for paths in the sitemap. This means search engine crawlers can continue to access these sitemap pages via SSR. However, it ensures bot traffic is limited to a subset of the site. See Server Side Rendering (SSR) Settings and #3682
  - By default, SSR is no longer performed for the search and browse components (on every page they are used). This ensures that embedded searches/browses on Community, Collection, or Item (Entity) pages are not triggered by search engine crawlers or bots. See Server Side Rendering (SSR) Settings and #3709
- **DSpace has a new, separate command-line log file (`[dspace]/log/dspace-cli.log-[date]`)** for logging the output/results of any scripts that are started from the command-line. The DSpace backend web application still logs to dspace.log-[date] in the same directory. You may need to update any local/custom log management scripts to include this new log file.
  - This change was necessary to fix a major bug where the backend webapp would sometimes stop logging if a "logrotate" was triggered. See #9832
- **The "Edit Item, Bitstreams tab" was refactored to use an HTML table.** This was necessary to fix major accessibility and keyboard navigation issues with the page. The entire page was rebuilt to us a `<table>` instead of using nested `<div>` tags. See #3464

7.6.3 Major Bug Fixes and Improvements

- Performance improvements in the User Interface
  - Angular Server Side Rendering (SSR) has been updated to attempt to reduce the CPU/memory necessary for managing bot traffic
    - By default, SSR is now only performed for paths in the sitemap (see new "paths" setting in Server Side Rendering (SSR) Settings). This means search engine crawlers can continue to access these sitemap pages via SSR. However, it ensures bot traffic is limited to a subset of the site: #3682 *(Donated by Vitor Silverio and Alan Orth)*
    - By default, SSR is no longer performed for the search and browse components (see Server Side Rendering (SSR) Settings). This ensures that embedded searches/browses on Community, Collection, or Item (Entity) pages are not triggered by search engine crawlers or bots: #3709 *(Donated by 4Science)*
    - Optionally, sites may configure SSR to occur via a separate, private/localhost REST API URL. This may provide performance benefits as it allows SSR processing to bypass DNS lookup and make private requests (including via localhost) directly to the machine running the REST API. See "ssrBaseUrl" in REST API Settings and #3358 *(Donated by 4Science)*
  - Ensure components are not sending requests for large sets of data (i.e. bypassing pagination): #2513 *(Donated by 4Science)*
  - Edit Item page would sometimes fail to load or encounter an infinite loop when visiting the version history page: #3252 *(Donated by Atmire)*
  - Workflow tasks loaded slowly when there were large numbers of groups: #9860 *(Donated by PCG Academia)*
  - Reduced number of requests when loading the simple item page: #3701 *(Donated by Atmire)*
  - Reduced the number of requests sent to backend when loading search results (by fixing a few caching bugs): #3415 *(Donated by Atmire)*

    - Browse pages were retrieving more data than necessary, resulting in poor performance for larger numbers of values: #10057 *(Donated by Toni Prieto)*
    - Several administrative pages could encounter infinite loading errors. These included EPerson and Group pages, Bitstream and Metadata registry, and pagination tools: #3585 *(Donated by Atmire)*
    - Improve performance of Group2Group cache to better support sites with many Groups. #9760 *(Donated by Alfeu U. Tavares)*
    - Reduced memory usage OAI-PMH full reindex: #10089 *(Donated by Toni Prieto)*
- Accessibility improvements in User Interface
    - Edit Item, Bitstreams tab was refactored to enhance accessibility and keyboard control: #3464 *(Donated by Atmire)*
    - Submission interface now supports keyboard reordering of multi-valued fields: #1271 *(Donated by 4Science)*
    - Ensured several pages have properly ordered HTML headings: #3071 *(Donated by 4Science)*
    - An invisible ARIA "live region" now exists which can communicate notifications and page changes to screen readers *(Donated by Atmire)*
    - All disabled buttons in DSpace are now accessible to users using a screen reader: #3249 *(Donated by Atmire)*
    - "All of DSpace" browse menu has improved keyboard navigation: #3085 *(Donated by Atmire)*
    - Controlled Vocabulary tree-view has improved support for keyboard navigation and screen readers: #3367 *(Donated by Neki-it)*
    - File download link has been updated to include the file name in the ARIA label (for screen readers): #3762 *(Donated by Neki-it)*
    - Date slider in search results was reloading the entire page of results, causing issues with accessibility: #3236 *(Donated by 4Science)*
    - Other minor accessibility improvements
- ORCID integration fixes. These fixes were all f*unded by ORCID Global Participation Fund and built by 4Science*
    - Fix several bugs with updating ORCID synchronization settings in the User Interface: #8925
    - ORCID integration was not sending updated information when an Item is versioned: #8662
    - Disconnecting your Person Entity from ORCID was not revoking your ORCID token: #9088
    - ORCID icons are now displayed next to any author who has an ORCID stored in DSpace. A tooltip will display the person's ORCID: #908

- Submission / Workflow fixes
    - Dropdown or lookup fields would sometimes reset when "Enter" key was pressed: #3724 *(Donated by Atmire)*
    - Reordering multi-valued fields was not always working properly: #3679 *(Donated by 4Science)*
    - Drag and drop of Subject Keywords in the submission form was not working properly when there were multiple rows of keywords: #3624 *(Donated by 4Science)*
    - Reviewers were not able to edit files, even when "workflow.reviewer.file-edit" was enabled: #9734 *(Donated by Agustina Martinez)*
    - Enforce unique Item IDs in the "workspace" database table: #9341 *(Donated by Atmire)*
- Administrative Fixes
    - Backend webapp would sometimes stop logging if a "logrotate" was triggered. This was fixed by creating a **separate** log file (dspace-cli.log-[date]) for command-line task logs: #9832 *(Donated by Atmire)*
    - Community Administrators where wrongly seeing the button to create top-level communities: #3288 *(Donated by 4Science)*
    - Edit Item, Metadata tab was truncating the list of available metadata fields: #2730 *(Donated by eScire)*
    - User Profile page was only showing the first 20 group memberships: #2357*(Donated by eScire)*
    - On the Administer Workflow page, the "Delete" button was not deleting the Item: #3086 *(Donated by eScire)*
    - If a file was larger than the full-text indexing limit, it was not being indexed properly*: #9893 (Donated by Brian Keese)*
    - If Tomcat shut down when a Process was running, the Process would stay in the "running" state forever: *#9727 and #10156 (Donated by Atmire)*
    - DOI Organizer would fail on the first error, ignoring the rest of the queue: #9828 *(Donated by Mark Wood)*
    - "check links" curation task would fail if it encountered a redirect: #9658 *(Donated by Atmire)*
- Other major fixes
    - Legacy Bitstream URLs (/bitstream/[handle-prefix]/[handle-suffix]/[index]/[filename]) would not redirect if the filename included a space: #3871 *(Donated by Scholars Portal)*
    - On Browse by Issue Date page, the Year dropdown would sometimes be empty when an invalid date was encountered in metadata. #3408 *(Donated by Atmire)*
    - Clicking on Browse by Author after Browse by Issue Date would generate a server error: #573 *(Donated by Atmire)*
    - After logging in via Shibboleth, the page content and admin sidebar didn't always load correctly: #3011 *(Donated by Michael Spalti)*
- Many updates to dependencies. Removal of some older, unused dependencies.
    - Backend dependency updates (includes upgrading to the latest version of Spring 5.3 and Spring Security 5.7)
    - Frontend dependency updates (includes upgrading to the latest version of Angular 15)
- *Many other minor bug fixes* as listed in Changes in 7.x.

7.6.3 New/Updated Language support

- Czech (eština) language updates donated by Milan Majchrak (milanmajcrak)
- French (Français) language updates donated by Pierre Lasou (pilasou) and Emmanuel Pastor (pastr)
- German (Deutsch) language updates donated by Sascha Szott (saschaszott)

7.6.3 Acknowledgments

The DSpace application would not exist without the hard work and support of its community. Thank you to the many developers who have worked very hard to deliver all the bug fixes and improvements. This release was *entirely volunteer driven!*

Development Acknowledgments

A total of 46 unique individuals contributed to 7.6.3.

Frontend / User Interface Acknowledgments

The following 30 individuals have contributed directly to the new DSpace (Angular) User Interface in this release (ordered by number of GitHub commits): Francesco Molinaro (FrancescoMolinaro), Andreas Awouters (AAwouters), Tim Donohue (tdonohue), Alexandre Vryghem (alexandrevryghem), Sascha Szott (saschaszott), Alisa Ismailati (alisaismailati), Andrea Guevara (Andrea-Guevara), Giuseppe Digilio (atarix83), Jens Vannerum (jensvannerum), Milan Majchrak (milanmajchrak), Alan Orth (alanorth), Andrea Barbasso (AndreaBarbasso), Pierre Lasou (pilasou), Julia Gilmore (alphapetjg), Mohana Sarmiento (msarmie), Simone Ramundi (Simone-Ramundi), Victor Hugo Duran Santiago (VictorHugoDuranS), Vincenzo Mecca (vins01-4science), Igor Baptista da Costa (IgorBaptist4), Arta Seyedian (aseyedia), Elvi Nemiz (eulereadgbe), Emmanuel Pastor (pastr), Kim Shepherd (kshepherd), Koen Pauwels (KoenP), Michael Spalti (mspalti), Miika Nurminen (minurmin), Vladzislav Novski (vNovski), Yana De Pauw (YanaDePauw), Yury Bondarenko (ybnd), Alfeu Tavares (autavares-dev).

The above contributor list was determined based on contributions to the "dspace-angular" project in GitHub between 7.6.2 (after July 9, 2024) and 7.6.3 using "git shortlog" on the `dspace-7_x` branch and excluding all merge commits: `git shortlog -s -n -e --no-merges --since 2024-07-09`

Backend / REST API Acknowledgments

The following 26 individuals have contributed directly to the DSpace backend (REST API, Java API, OAI-PMH, etc.) in this release (ordered by number of GitHub commits): Tim Donohue (tdonohue), Kim Shepherd (kshepherd), Sascha Szott (saschaszott), Giuseppe Digilio (atarix83), Mark Wood (mwoodiupui), Jens Vannerum (jensvannerum), Nona Luypaert (nona-luypaert), Kristof De Langhe (Atmire-Kristof), Vincenzo Mecca (vins01-4science), Alfeu Tavares (autavares-dev), Igor Baptista da Costa (IgorBaptist4), Florian Gantner (floriangantner), Jukka Lipka (jlipka), Marie Verdonck (MarieVerdonck), Mikhail Schastlivtsev (schastlivcev), Nathan Buckingham (ConfusionOrb221), Toni Prieto (toniprieto), Agustina Martinez (amgciadev), Alexandre Vryghem (alexandrevryghem), Andrew Alesik (AndrewAlesik), Brian Keese (bkeese), Chris Wilper (cwilper), Eike Löhden (Leano1998), Kevin Van de Velde (KevinVdV), Koen Pauwels (KoenP), Oscar Chacón (oscar-escire).

The above contributor list was determined based on contributions to the "DSpace" project in GitHub between 7.6.2 (after July 9, 2024) and 7.6.3 using "git shortlog" on the `dspace-7_x` branch and excluding all merge commits: `git shortlog -s -n -e --no-merges --since 2024-07-09`

7.6.2 Release Notes

DSpace 7.6.2 was released on July 9, 2024

To try out DSpace 7.6.2 immediately, see Try out DSpace 7.

To upgrade to DSpace 7.6.2 from 7.x *or any prior version*, see Upgrading DSpace

- To upgrade to 7.6.2, you MUST upgrade *both* the backend and frontend (user interface). Many bug fixes require updating *both.*

To install DSpace 7.6.2 for the first time, see Installing DSpace.

- Download DSpace 7.6.2 Backend: https://github.com/DSpace/DSpace/releases/tag/dspace-7.6.2
- Download DSpace 7.6.2 User Interface: https://github.com/DSpace/dspace-angular/releases/tag/dspace-7.6.2

DSpace 7.6.2 provides bug fixes, accessibility & performance improvements to the 7.6.x platform. *No new features are provided.* As such this release should be an easier upgrade for sites already running 7.6.x.

Security Fixes

- **Fix CVE-2024-38364** *(low severity)* by disabling the ability to open HTML/XML bitstreams in a user's browser. See https://github.com/DSpace/DSpace/security/advisories/GHSA-94cc-xjxr-pwvf (or mailing list announcements) for more details & configuration workaround. *(Discovered and reported by Muhammad Zeeshan (Xib3rR4dAr))*

Major bug fixes / improvements include:

- Performance Improvements
  - Disabled Angular "inlineCriticalCSS" in all Server Side Rendering (SSR). This provides a performance improvement to all SSR generated pages. See https://github.com/DSpace/dspace-angular/pull/2901 *(Donated by 4Science)*
  - Media filter performance improvements when filtering a large number of bitstreams (for thumbnail creation or full text indexing). *(Donated by 4Science)*
  - Submission form performance improvements. The submission form has been updated to ensure it no longer loads all related objects. *(Donated by Atmire)*
  - Submission configuration reloading performance improvements. This also improves performance of creating a new Collection. (Donated by Toni Prieto)
  - Updated robots.txt to stop crawlers from accessing search facets (*Donated by Atmire*)
- Accessibility improvements in User Interface
  - Hidden "Skip to main content" button now exists on all pages. (*Donated by Atmire)*
  - Header / Navbar / Admin Sidebar accessibility fixes *(Donated by 4Science)*
  - Community list accessibility fixes *(Donated by Hrafn Malmquist)*
  - Color contrast fixes to "dspace" theme *(Donated by Maciej Kleban)*
  - Search results / MyDSpace / Item Edit / Browse by / Login menu accessibility fixes *(Donated by Atmire)*
  - Community/Collection Homepage accessibility fixes *(Donated by Atmire)*
  - Additional keyboard controls in Submission form *(Donated by Atmire)*
  - Browse by Author accessibility fixes *(Donated by Neki-it)*
  - "Loading" message accessibility improvements *(Donated by Neki-it)*
  - Fixing issue with header menu being keyboard accessible on small screens *(Donated by Eike Löhden)*
  - Fix color contrast issues with cookie settings popup *(Donated by PCG Academia)*
- Submission form fixes
  - Fixed caching issues and instability of PATCH commands when editing date *(Donated by 4Science)*
  - Fixed issue where some changes could be lost after a save but reappear after reloading page *(Donated by Atmire)*

    - Fixed issue where metadata import (from external source) would only show first value for each metadata field instead of all values *(Donated by Atmire)*
    - Fixed issue where vocabulary displayed value was not always appearing when editing an existing submission *(Donated by Atmire)*
    - Provide a way to deselect a value from a metadata field dropdown *(Donated by Atmire)*
    - Fixed bugs related to creating/deleting Entity relationships in the submission form *(Donated by Atmire)*
    - Fixed bugs where type-bind wasn't working for radio buttons and checkboxes *(Donated by Max Nuding)*
    - Fixed issues with CrossRef and Scopus metadata import *(Donated by Sascha Szott)*
    - Fixed issue with display of no results from CrossRef search *(Donated by Philipp Rumpf)*
    - Fixed issues with DataCite metadata import *(Donated by Florian Gantner)*
- Statistics fixes
    - Solr Statistics: fixed issue where first visit to a repository was not always tracked because of a CSRF token mismatch.
    - Google Analytics 4 updated to only count file downloads from ORIGINAL bundle. See https://github.com/DSpace/DSpace/pull/8944 *(Donated by Atmire)*
    - Item Counts (webui.strengths) are now updating automatically again.
- SEO improvements
    - Legacy bitstream URLs now return a 301 redirect (instead of a 302) *(Donated by Atmire)*
    - Missing identifiers now return an HTTP code 404 *(Donated by Atmire)*
- Fixed bug where Community/Collection administrators could not add/edit a logo for a Community/Collection. *(Donated by 4Science)*
- Fixed bug where Amazon S3 data store would sometimes leave around temp files during download process. See https://github.com/DSpace/DSpace/pull/9477 *(Donated by 4Science)*
- Fixed issue where failing ZIP exports could sometimes leave around a work directory.
- Fixed issue where virtual metadata of Entities could cause failures during Item versioning & AIP import.
- Fixed issue where indexing may fail if Full Text extraction fails *(Donated by 4Science)*
- Fix several issues with editing Entity relationships *(Donated by Atmire)*
- Fix several pagination issues with Item Mapper *(Donated by Atmire)*
- Fixed issue where deleted admin users could cause the Processes page to no longer load properly. *(Donated by Atmire)*
- System alerts now support basic HTML. See https://github.com/DSpace/dspace-angular/pull/3044 *(Donated by Abel Gómez)*
- Fixed issue where batch imported bitstreams may be created without a resource policy type. *(Donated by Agustina Martinez)*
- Fixed issue where Items imported via SWORD may be created without a resource policy type. *(Donated by Paulo Graça)*
- Fixed display issue where Communities with diacritics were not sorted properly *(Donated by Paulo Graça)*
- Fixed issue where a client's "user-agent" was not being forwarded to backend (REST API) *(Donated by Alan Orth and Mark Cooper)*
- OAI-PMH DataCite crosswalk updated to support DataCite version 4.5 *(Sponsored by The Library Code)*
- Minor updates to various dependencies for security purposes (both for user interface and backend)
- *Many other minor bug fixes* as listed in Changes in 7.x.

Updated Language support:

- (NEW) Arabic () translation added by KnowledgeWare Technologies Est.
- Czech (eština) translation updates donated by NTK
- German (Deutsch) language updates donated by Sascha Szott (saschaszott)
- Portuguese (Português) language updates donated by Ricardo Saraiva (rsaraivac)

7.6.2 Acknowledgments

The DSpace application would not exist without the hard work and support of its community. Thank you to the many developers who have worked very hard to deliver all the bug fixes and improvements. This release was *entirely volunteer driven!*

Development Acknowledgments

A total of 58 unique individuals contributed to 7.6.2.

Frontend / User Interface Acknowledgments

The following 35 individuals have contributed directly to the new DSpace (Angular) User Interface in this release *(ordered by number of GitHub commits)*: Alexandre Vryghem (alexandrevryghem), Tim Donohue (tdonohue), Davide Negretti (davide-negretti), Art Lowel (artlowel), Alan Orth (alanorth), Ricardo Saraiva (rsaraivac), Sascha Szott (saschaszott), Lotte Hofstede (LotteHofstede), Jens Vannerum (jensvannerum), Oscar Chacón (oscar-escire), Pierre Lasou (pilasou), Yury Bondarenko (ybnd), Francesco Molinaro (FrancescoMolinaro), Giuseppe Digilio (atarix83), Kuno Vercammen, Michael Spalti (mspalti), Thomas Misilo (misilot), Abel Gómez (abelgomez), Andreas Awouters (AAwouters), Kim Shepherd (kshepherd), William Welling (wwelling), Yana De Pauw (YanaDePauw), Maciej Kleban (Dawnkai), Victor Hugo Duran Santiago (VictorHugoDuranS), Agustina Martinez (amgciadev), Andrea Barbasso (AndreaBarbasso), Bram Luyten (bram-atmire), Eike Löhden (Leano1998), Florian Gantner (floriangantner), Marie Verdonck (MarieVerdonck), Mohamed Ali, NTK, Nona Luypaert (nona-luypaert), Max Nuding (hutattedonmyarm), Reeta Kuukoski (reetagithub).

The above contributor list was determined based on contributions to the "dspace-angular" project in GitHub between 7.6.1 (after Nov 15, 2023) and 7.6.2 using "git shortlog" on the `dspace-7_x` branch and excluding all merge commits: `git shortlog -s -n -e --no-merges --since 2023-11-15`

Backend / REST API Acknowledgments

The following 38 individuals have contributed directly to the DSpace backend (REST API, Java API, OAI-PMH, etc.) in this release *(ordered by number of GitHub commits)*: Tim Donohue (tdonohue), Sascha Szott (saschaszott), Alan Orth (alanorth), Andrea Bollini (abollini), Toni Prieto (toniprieto), Agustina Martinez (amgciadev), Marie Verdonck (MarieVerdonck), Pascal-Nicolas Becker (pnbecker), Paulo Graça (paulo-graca), Yana De Pauw (YanaDePauw), Nicholas Woodward (nwoodward), Florian Gantner (floriangantner), Xiqinger, Damian Jozefowski (damian-joz), Francesco Bacchelli (frabacche), Thomas Misilo (misilot), Mohamed Eskander (eskander17), Alexandre Vryghem (alexandrevryghem), Eike Löhden (Leano1998), Jean-François Morin (jeffmorin), Kristof De Langhe (Atmire-Kristof), Philipp Rumpf (philipprumpf), Roy Brushini (Bezkup), William Welling (wwelling), Marsa Haoua (marsaoua), David Steelman, Jens Vannerum (jensvannerum), John Abrahams (jabrah), Kim Shepherd (kshepherd), Koen Pauwels (KoenP), Marian Berger (MajoBerger), Martin Walk (MW3000), Mirko Scherf (mirkoscherf), Nona Luypaert (nona-luypaert), Shankeerthan Kasilingam, Yannick Paulsen (YPaulsen-TLC), Max Nuding (hutattedonmyarm), Milan Majchrak (milanmajchrak).

The above contributor list was determined based on contributions to the "DSpace" project in GitHub between 7.6.1 (after Nov 15, 2023) and 7.6.2 using "git shortlog" on the `dspace-7_x` branch and excluding all merge commits: `git shortlog -s -n -e --no-merges --since 2023-11-15`

7.6.1 Release Notes

DSpace 7.6.1 was released on November 15, 2023

To try out DSpace 7.6.1 immediately, see Try out DSpace 7. This includes instructions for a quick-install via Docker, as well as information on our DSpace demo site.

To upgrade to DSpace 7.6.1 from 7.x *or any prior version*, see Upgrading DSpace

- To upgrade to 7.6.1, you MUST upgrade *both* the backend and frontend (user interface). Many bug fixes require updating *both.*

To install DSpace 7.6.1 for the first time, see Installing DSpace.

- Download DSpace 7.6.1 Backend: https://github.com/DSpace/DSpace/releases/tag/dspace-7.6.1
- Download DSpace 7.6.1 User Interface: https://github.com/DSpace/dspace-angular/releases/tag/dspace-7.6.1

DSpace 7.6.1 provides bug fixes & performance improvements to the 7.6.x platform. *No new features are provided.* As such this release should be an easier upgrade for sites already running 7.6.

Major bug fixes / improvements include:

- Performance improvements
  - User interface no longer repeatedly requests the "/api" endpoint of the backend. https://github.com/DSpace/dspace-angular/issues/2482 *(Donated by Atmire)*
  - "Edit Group" page performs much better for groups with a lot of members. https://github.com/DSpace/DSpace/issues/9052 and https://github.com/DSpace/dspace-angular/issues/2512
  - Workflow Tasks page loads more quickly for a logged-in EPerson who is a member of many groups. https://github.com/DSpace/DSpace/issues/9053 *(Donated by Atmire)*
  - OAI-PMH no longer has high memory usage during/after harvesting. https://github.com/DSpace/DSpace/issues/8846 *(Donated by Christian Bethge, ULB)*
  - Improved performance of "./dspace update-handle-prefix" script. https://github.com/DSpace/DSpace/issues/9066 *(Donated by 4Science)*
  - Improved performance of "./dspace checker" script. https://github.com/DSpace/DSpace/issues/9180 *(Donated by 4Science)*
  - Improved performance of "./dspace generate-sitemaps" script (and automatic sitemaps). https://github.com/DSpace/DSpace/issues/3182 *(Donated by 4Science)*
- General fixes
  - Specific browser plugins no longer cause the User Interface to appear blank. https://github.com/DSpace/dspace-angular/issues/2450 *(Donated by floriangantner, U of Bamberg)*
  - Emails sent by DSpace no longer have blank "Subject" fields. https://github.com/DSpace/DSpace/issues/8921 *(Donated by Mark Wood, Indiana University)*
  - Navbar menu on mobile devices now works in Firefox/Safari. https://github.com/DSpace/dspace-angular/issues/2372 *(Donated by eScire)*
  - Git is no longer required as a build dependency. https://github.com/DSpace/DSpace/pull/9032 *(Donated by Cottage Labs)*
- Item view fixes
  - Handle redirects now work properly when "ui.nameSpace" config (on frontend) is customized. https://github.com/DSpace/dspace-angular/issues/2517 *(Donated by Atmire)*
  - Bitstream URLs now work properly if "ui.nameSpace" config (on frontend) is customized. https://github.com/DSpace/dspace-angular/issues/2446 *(Donated by Atmire)*
  - Thumbnails now work properly for Bitstreams having special characters. https://github.com/DSpace/DSpace/issues/9112 *(Donated by floriangantner, U of Bamberg)*
  - Fixed display issue of media viewer (controls hidden behind header). https://github.com/DSpace/dspace-angular/issues/2341 *(Donated by 4Science)*
  - MathJax code is no longer displayed twice. https://github.com/DSpace/dspace-angular/issues/2170 *(Donated by Atmire)*
  - Fixed issue where some "Edit Item" pages were visible (but not usable) to anonymous users. https://github.com/DSpace/dspace-angular/issues/2609 *(Donated by 4Science)*
- Browse/Search fixes
  - Hierarchical browse indexes for controlled vocabularies can now be disabled. https://github.com/DSpace/DSpace/issues/8947 *(Donated by Toni Prieto)*
  - Pagination is now reset when changing searches. https://github.com/DSpace/dspace-angular/issues/2159 *(Donated by eScire)*
  - Unicode characters now work properly in search filters. https://github.com/DSpace/DSpace/issues/8914 *(Donated by Toni Prieto)*
- Submission/Workflow form fixes
  - Submission form no longer hangs / enters infinite loop in specific scenarios. https://github.com/DSpace/dspace-angular/issues/1924 and https://github.com/DSpace/dspace-angular/issues/2577 *(Donated by 4Science)*
  - Freetext is now supported in controlled vocabulary fields where "closed=false". https://github.com/DSpace/dspace-angular/issues/2435 *(Donated by Atmire, with support from the International Livestock Research Institute)*
  - Date field can now be modified easier when editing an existing submission. https://github.com/DSpace/dspace-angular/issues/2588 *(Donated by 4Science)*
  - "metadata.hide" fields are no longer hidden from submitters/reviewers. https://github.com/DSpace/dspace-angular/issues/1997 *(Donated by Toni Prieto)*
  - "Type" dropdown no longer changes to first entry in list when pressing Enter. https://github.com/DSpace/dspace-angular/issues/2145 *(Donated by eScire)*
  - Workflow curation tasks can now be "queued" for later. https://github.com/DSpace/DSpace/issues/9070 *(Donated by Agustina Martinez, Cambridge University)*
  - Removed "Add More" button when additional sections not available. https://github.com/DSpace/dspace-angular/issues/2535 *(Donated by Atmire)*

    - Added submitter information in "dc.description.provenance" as in older versions. https://github.com/DSpace/DSpace/issues/8585 *(Donated by Arvo Consultores)*
- Authentication fixes
    - Entering an incorrect password no longer results in a blank login menu in Firefox. https://github.com/DSpace/dspace-angular/issues/2515 *(Donated by Atmire)*
    - LDAP is no longer broken if groupmaps were enabled. https://github.com/DSpace/DSpace/issues/8920 *(Donated by wwuck)*
    - Special groups are only added for the authentication system the user used to login. https://github.com/DSpace/DSpace/issues/9127 *(Donated by 4Science)*
    - Login popup now lists login methods in order of configuration. https://github.com/DSpace/dspace-angular/issues/2365 *(Donated by Atmire)*
- Admin tools fixes
    - A new Metadata Schema in registry no longer requires a page reload to appear. https://github.com/DSpace/dspace-angular/issues/1081 *(Donated by eScire)*
    - Moving items between collections now inherits policies correctly. https://github.com/DSpace/DSpace/issues/8987 *(Donated by Atmire)*
- Statistics fixes
    - Google Analytics bitstream statistics are now limited to the "ORIGINAL" bundle. https://github.com/DSpace/DSpace/issues/8938 *(Donated by Atmire)*
- Theming fixes
    - Configuring themes using "handle" now works properly. https://github.com/DSpace/dspace-angular/issues/2348 *(Donated by Atmire)*
    - Extending a theme no longer causes it to potentially render multiple times. https://github.com/DSpace/dspace-angular/issues/2346 *(Donated by Atmire)*
- Search Engine Optimization fixes
    - Access restricted and "Non-Discoverable" Items are no longer listed in Sitemaps. https://github.com/DSpace/DSpace/issues/5394 and https://github.com/DSpace/DSpace/issues/5343 *(Donated by 4Science)*
- Accessibility fixes
    - Fixes to accessibility of Community List (/community-list) page. https://github.com/DSpace/dspace-angular/issues/2251 *(Donated by Cottage Labs)*
    - Added an invisible "Skip to main content" button on all pages. https://github.com/DSpace/dspace-angular/issues/2523 *(Donated by Atmire)*
- Translation bug fixes
    - Translation files (i18n files) are now hashed, so they are no longer reloaded unless they change. https://github.com/DSpace/dspace-angular/issues/2461 *(Donated by Atmire)*
    - Localized "default_[language-code].license" files are supported again. https://github.com/DSpace/DSpace/issues/8882 *(Donated by PCG Academia)*
    - Translations are now working properly for item status badges in MyDSpace. https://github.com/DSpace/dspace-angular/issues/2387 *(Donated by Mirko Scherf)*
- Replication Task Suite version 7.6 has been released to add compatibility with all DSpace 7.6.x releases. This Maven plugin can be used to provide extra curation tasks for AIP Backup and Restore.
- *Many other bug fixes and dependency updates* as listed in Changes in 7.x.

Updated Language support:

- Finnish (Suomi) translation updates donated by Reeta Kuukoski (reetagithub)
- Italian (Italiano) language updates donated by 4Science
- Polish (Polski) translation updates donated by Micha Dykas (michdyk)
- Brazilian Portuguese (Português do Brasil) updates donated by Marco Aurelio Cardoso (marcoaureliocardoso)
- Portugese (Português) language updates donated by José Carvalho (j-n-c)
- (New) Serbian Cyrillic () language translations donated by Milos Ivanovic (imilos)
- (New) Serbian Latin (Srpski (lat)) language translations donated by Milos Ivanovic (imilos)
- Spanish (Español) translation updates donated by Arvo Consultores

7.6.1 Acknowledgments

The DSpace application would not exist without the hard work and support of its community. Thank you to the many developers who have worked very hard to deliver all the bug fixes and improvements. This release was *entirely volunteer driven!*

Development Acknowledgments

A total of 38 unique individuals contributed to 7.6.1.

Frontend / User Interface Acknowledgments

The following 29 individuals have contributed directly to the new DSpace (Angular) User Interface in this release *(ordered by number of GitHub commits)*: Alexandre Vryghem (alexandrevryghem), Tim Donohue (tdonohue), Hugo Daniel Dominguez de la Cruz (hugo-escire), Davide Negretti (davide-negretti), Vladzislav Novski (vNovski), Eike Löhden (Leano1998), Hrafn Malmquist (J4bbi), Paulo Graça (paulo-graca), Art Lowel (artlowel), Micha Dykas (michdyk), Alan Orth (alanorth), Kristof De Langhe (Atmire-Kristof), Milos Ivanovic (imilos), Mirko Scherf (mirkoscherf), Sascha Szott (saschaszott), Mark Wood (mwoodiupui), William Welling (wwtamu), Lotte Hofstede (LotteHofstede), Flo (floriangantner), Hardy Pottinger (hardyoyo), Sergio Fernández Celorio (sergius02), Yury Bondarenko (ybnd), Agustina Martinez (amgciadev), Reeta Kuukoski (reetagithub), Nona Luypaert (nona-luypaert), Kim Shepherd (kshepherd), Yana De Pauw (YanaDePauw), Marie Verdonck (MarieVerdonck)

The above contributor lists were determined based on contributions to the "dspace-angular" project in GitHub between 7.6 (after June 23, 2023) and 7.6.1: https://github.com/DSpace/dspace-angular/graphs/contributors?from=2023-06-23&to=2023-11-15&type=c

Backend / REST API Acknowledgments

The following 20 individuals have contributed directly to the DSpace backend (REST API, Java API, OAI-PMH, etc.) in this release *(ordered by number of GitHub commits)*: Tim Donohue (tdonohue), Agustina Martinez (amgciadev), Mark Wood (mwoodiupui), Paulo Graça (paulo-graca), Toni Prieto (toniprieto), Adán Román Ruiz (aroman-arvo), Christian Bethge (ChrisBethgster), Alan Orth (alanorth), Nicholas Woodward (nwoodward), Nona Luypaert (nona-

luypaert), Alexandre Vryghem (alexandrevryghem), Hrafn Malmquist (J4bbi), Andrea Bollini (abollini), Luca Giamminonni (LucaGiamminonni), damian-joz, Michael Spalti (mspalti), Kim Shepherd (kshepherd), Martin Walk (MW3000)

The above contributor list was determined based on contributions to the "DSpace" project in GitHub between 7.6 (after June 23, 2023) and 7.6.1: https://github.com/DSpace/dspace/graphs/contributors?from=2023-06-23&to=2023-11-15&type=c

7.6 Release Notes

DSpace 7.6 was released on June 26, 2023

To try out DSpace 7.6 immediately, see Try out DSpace 7. This includes instructions for a quick-install via Docker, as well as information on our sandbox/demo site for DSpace 7.

To upgrade to DSpace 7.6 from 7.x *or any prior version*, see Upgrading DSpace.

To install DSpace 7.6 for the first time, see Installing DSpace.

- Download DSpace 7.6 Backend: https://github.com/DSpace/DSpace/releases/tag/dspace-7.6
- Download DSpace 7.6 User Interface: https://github.com/DSpace/dspace-angular/releases/tag/dspace-7.6

DSpace 7.6 provides new features & bug fixes to the 7.x platform.

New features include:

- **Bulk Access Management** allows someone with administrative privileges (site wide or over a single object) to perform bulk modifications to the permissions of objects they administer. This allows for the ability to add/remove embargoes or other access restrictions to many objects at once. *(Developed and co-funded by 4Science with the Support of University of Cambridge)*
- **Support for selecting Primary Bitstream for archived Items**, similar to version 6.x. The existing "primary bitstream" selector now functions properly when editing a Bitstream.
- **Item counts can now be displayed for all Communities/Collections** *similar to version 6.x. (Donated by PCG Academia)*
- **Browse Hierarchical Controlled Vocabularies** This new feature allows users to browse/search for items quickly using the same controlled vocabulary configured in your submission forms. *(Donated by Atmire)*
- **Signposting support for items and bitstreams.** This new feature embeds signposting links/metadata into pages and responses, to better support FAIR guiding principles. (*Developed and donated by 4Science)*
- **Import Simple Archive Format Zip files from a remote URL.** This feature enhances the existing batch import feature to allow you to specify the URL of the ZIP file to import, instead of using the drag & drop upload. *(Developed and donated by 4Science)*
- **ImageMagick Thumbnails for Video files (MP4).** A new "ImageMagick Video Thumbnail" plugin can be used to generate thumbnails from Video files using FFmpeg. *(Donated by Abel Gómez)*
- **Ability to map Item submission forms via Entity Type** to easily configure a submission form for all Collections accepting the same Entity Type. See the Configurable Entities documentation. (*Donated by Paulo Graça)*
- **New default Privacy Statement and End User Agreement** . The new default text of these policies can be found by visiting the links in the footer of our demo site. *(Donated by DSpaceDirect)*
- **Oracle support has been removed** as was previously announced in March 2022 on our mailing lists.

Major bug fixes / improvements include:

- User interface upgraded to Angular 15
- Performance improvements for Items with many bitstreams *(Donated by Michael Spalti)*
- Browse by fixes
  - Allow browse by to properly filter for numerical values. *(Donated by eScire)*
  - Added upper limit for year in Browse by Date *(Donated by Michael Spalti)*
  - Fixed issue with clicking on items when filtering by year in Browse by Date *(Donated by Michael Spalti)*
  - Fixed issue where non-existent browse could cause page loading issues *(Donated by Atmire)*
  - Fixed issue where special characters in UUID could cause page loading issues *(Donated by Atmire)*
- Search fixes
  - Fixed issue with scope-based searching in Discovery/Solr *(Donated by Atmire)*
  - Fixed issue where deselecting a filter could return an invalid query. *(Donated by eScire)*
  - Ensured Collections/Communities are now listed alphabetically in all search/selection popups (and sort order is configurable in config.yml) *(Donated by Arvo Consultores)*
  - Fixed issues with "-i" and "-r" params to index-discovery script *(Donated by Agustina Martinez)*
- Submission form fixes
  - Fixed issue where "list" input type would not work properly when set to required
  - Adding/removing new relationships (between Entities) would sometimes not update on the form *(Donated by Atmire)*
  - Date picker was not always respecting start/end dates for access options *(Donated by Mark Wood)*
  - Hint was missing from subject keywords field, and "list" and "tag" input types *(Donated by Atmire)*
  - Fixed several issues with imports from external sources to avoid cached errors and double submissions *(Donated by Atmire and 4Science and Alan Orth and others)*
  - Fixed issues where a required, hidden field could make it impossible to complete submission *(Donated by 4Science)*
  - Disabled collection box once submission is in workflow state. Collection cannot be changed during workflow. *(Donated by eScire)*
  - Fixed visibility of read-only fields and sections *(Donated by 4Science)*
  - Fixed issue where CC0 was displayed twice in Creative Commons section *(Donated by Atmire)*
- Statistics fixes
  - Fixed issue where Solr statistics were not loading after sharding *(Donated by Nicholas Woodward)*
  - Fixed issue where restricted objects appeared in Solr statistics with an empty name
  - Fixed issue where the referrer for a "view" event was almost always incorrect *(Donated by Atmire)*
  - Fixed issue where "search_result" events were not being captured in statistics. *(Donated by Atmire)*
- Permission inheritance fixes
  - When moving an Item to a new Collection *(Donated by Agustina Martinez)*

    - Where Bundles were accidentally inheriting access permissions for Bitstreams *(Donated by Kim Shepherd)*
- Administrative fixes
    - Enhanced functionality for deleting multiple bitstreams (of the same Item) at once. Fixed issue where sometimes only the first bitstream would be deleted. *(Donated by Atmire)*
    - Fixed issue where only one thumbnail was deleted when running media-filter in force mode. (*Developed and donated by 4Science)*
    - Fixed issue where an Admin couldn't reset another user's password if Captcha was enabled. *(Donated by @Ma-Tador)*
    - Fixed issues with checksum reporter/checker no longer working properly. (*Developed and donated by 4Science)*
    - Fixed issue where Collection Admin could not edit an Item Template *(Donated by D&L)*
    - Fixed issue where EPerson deletion could result in page loading issues (*Developed and donated by 4Science)*
    - Fixed bug where edit form for a Commuity/Collection/Item would not always update if you selected a different object immediately (*Developed and donated by 4Science)*
    - Improved validation of input fields when creating new Metadata schema or fields. *(Donated by Atmire)*
    - Fixed issue where workflow actions (from "Administer workflow" page) were not working. (*Developed and donated by 4Science)*
    - Fixed issue where curation tasks could not be performed by Community or Collection Admins. (*Developed and donated by 4Science)*
- Request a Copy fixes
    - Fixed issue with response page not loading
    - Fixed issue where "helpdesk" strategy required authentication in order to respond. *(Donated by Arvo Consultores)*
    - Moved text of approval/rejection emails to backend along with other email templates *(Donated by Mark Wood)*
- Added Multipart upload support to Amazon S3 Bitstore plugin (allows for larger uploads). (*Developed and donated by 4Science)*
- Added "webui.content_disposition_format" configuration to support always downloading (as an attachment) specific file formats. *(Donated by Atmire)*
- Fixed issue where page could fail to load if browse cookie language code was invalid *(Donated by Atmire)*
- Fixed issue where Shibboleth authentication would not reload the current page after authentication *(Donated by Michael Spalti)*
- Fixed several bugs with the ORCID Authority Control plugin and added more detailed documentation on enabling it. *(Donated by uofmsean with support from others)*
- Fixed issue where RSS feed was wrongly sorting alphabetically. Corrected to sort by date (like recent submissions). *(Donated by Nicholas Woodward)*
- Fixed issue where you could not enable the video viewer without also enabling the image viewer. *(Donated by Atmire)*
- Fixed issue where Batch import (Zip) would throw a confusing error if the file size was too large *(Donated by Nicholas Woodward)*
- Fixed issue where Batch import (Zip) was not cleaning up temporary files after an error occurred *(Donated by Nicholas Woodward)*
- GDPR compliance fix. An unnecessary external font was being loaded into the UI. *(Donated by Nicholas Woodward)*
- Minor updates/fixes to IIIF support *(Donated by Michael Spalti)*
- Made many more user interface components themeable
- Numerous user interface accessibility and usability enhancements *(Donated by Adam Doan and eScire and Atmire)*
- Docker script build enhancements *(Donated by DSpaceDirect)*
- Numerous other small bug fixes and dependency updates to frontend and backend.

New/Updated Language support:

- French (Français) language updates donated by Pierre Lasou (pilasou)
- German (Deutsch) language updates donated by fapsi and Sascha Szott (saschaszott)
- Hungarian (Magyar) language updates donated by Nagy Akos (akoscomp)
- Italian (Italiano) language added & donated by 4Science
- Portugese (Português) language updates donated by Ricardo Saraiva (rsaraivac)
- Spanish (Español) language updates donated by vlguerrero
- Vietnamese (Ting Vit) language added & donated by D&L (VuNguyen - Zerosoul1990)

7.6 Acknowledgments

The DSpace application would not exist without the hard work and support of its community. Thank you to the many developers who have worked very hard to deliver all the new features and improvements. We deeply appreciate the financial contributions by those institutions that have contributed to the DSpace Development Fund. Also, thanks to the users who provided input and feedback on the development, those who contributed documentation as well those who participated in the testathons.

Development Acknowledgments

A total of 45 unique individuals contributed to 7.6, with major institutional contributions coming from 4Science and Atmire.

Frontend / User Interface Acknowledgments

The following 33 individuals have contributed directly to the new DSpace (Angular) User Interface in this release (ordered by number of GitHub commits): Alexandre Vryghem (alexandrevryghem), enea4science, Kristof De Langhe (Atmire-Kristof), Nona Luypaert (nona-luypaert), Tim Donohue (tdonohue), Lotte Hofstede (LotteHofstede), Cristian Emanuelle Guzmán Suárez (CrisGuzmanS), Mark Wood (mwoodiupui), Art Lowel (artlowel), Michael Spalti (mspalti), Ricardo Saraiva (rsaraivac), Paulo Graça (paulo-graca), Aman Budgujar (amanbudgujar), Alan Orth (alanorth), Adam Doan (doana), VuNguyen (Zerosoul1990), Davide Negretti (davide-negretti), damian-joz, Yana De Pauw (YanaDePauw), Yury Bondarenko (ybnd), Nicholas Woodward (nwoodward), Pierre Lasou (pilasou), fapsi, Flo (floriangantner), Mattia Vianelli (Sondissimo), Reeta Kuuskoski (reetagithub), Agustina Martinez (amgciadev), Nagy Akos (akoscomp), Mark Cooper (mark-cooper), vlguerrero, Adán Román Ruiz (aroman-arvo), Sascha Szott (saschaszott), Kim Shepherd (kshepherd)

The above contributor lists were determined based on contributions to the "dspace-angular" project in GitHub between 7.5 (after February 17, 2023) and 7.6: https://github.com/DSpace/dspace-angular/graphs/contributors?from=2023-02-17&to=2023-06-23&type=c

Backend / REST API Acknowledgments

The following 27 individuals have contributed directly to the DSpace backend (REST API, Java API, OAI-PMH, etc.) in this release (ordered by number of GitHub commits): Tim Donohue (tdonohue), Nona Luypaert (nona-luypaert), Mark Wood (mwoodiupui), Alan Orth (alanorth), NikitaKr1vonosov, Paulo Graça (paulo-graca), Alexandre Vryghem (alexandrevryghem), Adán Román Ruiz (aroman-arvo), Andrea Bollini (abollini), Yana De Pauw (YanaDePauw), Francesco Pio Scognamiglio (francescopioscognamiglio), Agustina Martinez (amgciadev), Kim Shepherd (kshepherd), Michael Spalti (mspalti), Nicholas

Woodward (nwoodward), Abel Gómez (abelgomez), Kristof De Langhe (Atmire-Kristof), damian-joz, buithaihai, Chandaluri Vamsi Krishna (CVamsi27), eskander17, Vincenzo Mecca (vins01-4science), Kevin Van de Velde (KevinVdV), Luca Giamminonni (LucaGiamminonni), Hardy Pottinger (hardyoyo), Marie Verdonck (MarieVerdonck), Yury Bondarenko (ybnd)

The above contributor list was determined based on contributions to the "DSpace" project in GitHub between 7.5 (after February 17, 2023) and 7.6: https://github.com/DSpace/dspace/graphs/contributors?from=2023-02-17&to=2023-06-23&type=c

Financial Acknowledgments

We gratefully recognize those institutions who have generously pledged to financially support one or more 7.x releases via the DSpace Development Fund. A list of those funders can be found on the DSpace Development Fund page.

7.5 Release Notes

DSpace 7.5 was released on February 20, 2023

To try out DSpace 7.5 immediately, see Try out DSpace 7. This includes instructions for a quick-install via Docker, as well as information on our sandbox/demo site for DSpace 7.

To upgrade to DSpace 7.5 from 7.x *or any prior version*, see Upgrading DSpace.

To install DSpace 7.5 for the first time, see Installing DSpace.

- Download DSpace 7.5 Backend: https://github.com/DSpace/DSpace/releases/tag/dspace-7.5
- Download DSpace 7.5 User Interface: https://github.com/DSpace/dspace-angular/releases/tag/dspace-7.5

DSpace 7.5 provides new features & bug fixes to the 7.x platform.

New features include:

- **Subscribe to email updates from a Community or Collection** *(ported from the DSpace-CRIS project, partially donated and developed by 4Science)*
  - *NOTE: This feature requires new scheduled cron settings* to enable sending of emails. See Scheduled Tasks via Cron
- **Supervision orders** allow Administrators to assign Group(s) to supervise other user's (in-progress) submissions
- **System wide alerts** which allow you to display a site-wide banner to announce scheduled downtime or maintenance (Login as an Admin and visit the "System-wide Alert" menu option)
- **Ability to disable self-registration or restrict to specific email domains**. See Authentication by Password documentation for more details.
- **Search interface supports filtering by controlled vocabularies** See the new "vocabularies" option in config.*.yml. By default, only the SRSC vocabulary is available in DSpace, but additional ones may be added as described in Authority Control of Metadata Values
- **Support for custom Configurable Workflow steps, including "Select Single Reviewer Workflow" and "Score Review Workflow"** (which were in v6.x). See Additional workflow steps documentation for more details.
- **Ability to add Contextual Help Tooltips on any page.** Currently, only a basic example is provided, with more likely to be added in future DSpace releases. In the meantime, additional tooltips may be added manually on a page-by-page basis. See Contextual Help Tooltips for more details.
- **Basic MediaViewer now supports captioning for audio/video files.** *(Donated by yingjin)* The caption file must be provided alongside the video/audio file. See documentation at Media Viewer Settings
- **On Item pages, the "Edit Item > Metadata" tab has been redesigned** to allow for easier reordering of metadata values, etc.
- **On Item pages, metadata values can automatically link to "Browse By" results for that value.** *(Donated by The Library Code with the Support of German Institute for Urban Affairs)* The existing "webui.browse.links.<n>" settings in dspace.cfg are now supported in version 7.x. See Links to Other Browse Contexts section of the Configuration documentation.
- **"Show identifiers" submission allows you to optionally pre-register Handles or DOIs for all new submissions.** *(Donated by The Library Code with support of Technische Universität Hamburg and Technische Informationsbibliothek)* See Configuring the Identifiers step and Configuring pre-registration of Identifiers
- **New DataCite plugin for importing metadata from DataCite when starting a new submission.** *(Donated by @johannastaudinger, @florianga ntner and @philipprumpf)*
- **New "dspace database skip" command** can be used to skip problematic database migrations during upgrade to 7.x. See this common migration error that has impacted some older DSpace sites during their upgrade to 7.x.

Major bug fixes / improvements include:

- Performance improvements to User Interface
  - Optional page caching (of server-side rendered pages) for bots and/or anonymous users. Page caching for bots is enabled by default, while page caching for anonymous users is disabled. Enabling page caching for anonymous users can drastically speed up your site, but anonymous users may see slightly outdated content. See Cache Settings - Server Side Rendering(SSR) for details.
  - Decreased the initial download size of the UI application. *Between 7.4 and 7.5, we've decreased the initial download from 877KB to ~600KB* (gzipped sizes), even while adding new features.
- Performance improvements for the "metadata-import" and "cleanup" command-line scripts. *(Fixes to "cleanup" donated by Agustina Martinez)*
- Logging of slow SOLR queries is now enabled by default to simplify performance issues analysis (*Donated by 4Science*)
- Fixed issues with the "Request a Copy" helpdesk strategy, where all requests go to the Helpdesk email to respond. *(Donated by Mark Wood, IUPUI)*
- Number of bitstreams displayed on the Item page is now configurable (see "item > bitstream > pageSize" configuration). *(Donated by Arvo Consultores y Tecnología. S.L)*
- Updated Submission forms "<regex>" tag to use a common extended regular expression format.
- "Back to Results" button appears on Item pages when browsing/searching and clicking on an Item. This allows you to easily return to your prior browse/search. *(Donated by Michael Spalti)*
- Fixed breadcrumbs to display full object names on hover over *(Donated by eScire)*
- Facet search box on the main search interface can now search all words in a phrase facet value (previously it only searched the first word)
- Fixed issue where homepage hits could trigger 2 views in Google Analytics

- Fixed issues with Syndication Feed buttons on Community/Collection pages *(Donated by Nicholas Woodward)*
- Fixed issue where "Browse By Title" didn't allow you to filter results by multiple words *(Donated by Toni Prieto)*
- Fixed issue where opening a restricted bitstream in a different window or tab did not work properly.
- Fixed issue where items with thousands of bitstreams caused the Item page to load extremely slowly or crash entirely.
- Fixed robots.txt to include fully qualified URLs in Sitemap (necessary for search engine optimization)
- Fixed issue where "Browse By" menus were not working properly for Community/Collection Admins.
- Fixed issue where Collection Admins could not version Items.
- Fixed issue where Group descriptions could not be edited from the Admin UI.
- Fixed issue where sidebar options were displayed for authenticated users with read-only access. Sidebar has been enhanced to check user permissions more carefully.
- Fixed issue where thumbnails were sometimes generated with no permissions, making them not able to be displayed in the UI.
- Fixed issue where submission forms listed Collections in the Collection dropdown in a random order. They are now ordered by Title *(Donated by Arvo Consultores y Tecnología. S.L)*
- Fixed issue where scripts are listed in the start new process dropdown in a random order. They are now order alphabetically *(Donated by 4Science)*
- Fixed issues where configured Creative Commons jurisdiction was being ignored & locale wasn't returning translated questions *(Donated by Toni Prieto)*
- Fixed issues where OAI Harvesting "last harvested" date wasn't being updated or reported properly. *(Donated by Nicholas Woodword)*
- Fixed issue where title highlighting was not working in search results.
- Fixed issue where an Item could not be deleted if it had a "Request for Copy".
- Allow users to enhance ImageMagick generated PDF thumbnails via a new "density" setting & fixed layout issues with some PDF thumbnails. *(Donated by Alan Orth)*
- CSS variables can now be used in all HTML templates. See https://github.com/DSpace/dspace-angular/pull/1853
- Accessibility bug fixes for several pages *(Donated by eScire)*
- Support for Node v18 in the User Interface
- Numerous other small bug fixes and dependency updates

New/Updated Language support:

- Brazilian Portugese (Português do Brasil) language updates donated by Lucas Zinato Carraro (lucaszc)
- Catalan (Català) language added & donated by Toni Prieto
- French (Français) language updates donated by Pierre Lasou (pilasou)
- German (Deutsch) language updates donated by Sascha Szott (saschaszott)
- Greek () language updates donated by LisAtUP
- Kazakh () language updates donated by myrza1
- Polish (Polski) language added & donated by PCG Academia & Micha Dykas (michdyk)
- Spanish (Español) language updates donated by Toni Prieto & Cristian Emanuelle Guzmán Suárez (CrisGuzmanS)
- Ukrainian (Y) language added & donated by AndrukhivAndriy

7.5 Acknowledgments

The DSpace application would not exist without the hard work and support of its community. Thank you to the many developers who have worked very hard to deliver all the new features and improvements. We deeply appreciate the financial contributions by those institutions that have contributed to the DSpace Development Fund. Also, thanks to the users who provided input and feedback on the development, those who contributed documentation as well those who participated in the testathons.

Development Acknowledgments

A total of 44 unique individuals contributed to 7.5, with major institutional contributions coming from 4Science and Atmire.

Frontend / User Interface Acknowledgments

The following 30 individuals have contributed directly to the new DSpace (Angular) User Interface in this release (ordered by number of GitHub commits): Alexandre Vryghem (alexandrevryghem), Tim Donohue (tdonohue), Kim Shepherd (kshepherd), Kristof De Langhe (Atmire-Kristof), Michael Spalti (mspalti), Yana De Pauw (YanaDePauw), Yury Bondarenko (ybnd), Giuseppe Digilio (atarix83), Joost Fock (joost-atmire), Cristian Emanuelle Guzmán Suárez (CrisGuzmanS), nikunj59, yingjin, Marie Verdonck (MarieVerdonck), Toni Prieto (toniprieto), Mark Wood (mwoodiupui), Art Lowel (artlowel), Nicholas Woodward (nwoodward), Sergio Fernández Celorio (sergius02), Lucas Zinato Carraro (lucaszc), Sascha Szott (saschaszott), Micha Dykas (michdyk), Vincenzo Mecca (vins01-4science), Pierre Lasou (pilasou), Max Nuding (hutattedonmyarm), David Steelman (dsteelma-umd), Mykhaylo Boychuk (Micheleboychuk), myrza1, AndrukhivAndriy, LisAtUP, tony

The above contributor lists were determined based on contributions to the "dspace-angular" project in GitHub between 7.4 (after October 6, 2022) and 7.5: https://github.com/DSpace/dspace-angular/graphs/contributors?from=2022-10-06&to=2023-02-16&type=c

Backend / REST API Acknowledgments

The following 26 individuals have contributed directly to the DSpace backend (REST API, Java API, OAI-PMH, etc.) in this release (ordered by number of GitHub commits): Mykhaylo Boychuk (Micheleboychuk), Kim Shepherd (kshepherd), Tim Donohue (tdonohue), Marie Verdonck (MarieVerdonck), JensRoets-Atmire, Yana De Pauw (YanaDePauw), Vincenzo Mecca (vins01-4science), Nicholas Woodward (nwoodward), Kevin Van de Velde (KevinVdV), Toni Prieto (toniprieto), Adán Román Ruiz (aroman-arvo), Alex (sashashura), Philipp Rumpf (philipprumpf), Mark Wood (mwoodiupui), Alan Orth (alanorth), Giuseppe Digilio (atarix83), Joost Fock (joost-atmire), Pascal-Nicolas Becker (pnbecker), Johanna (johannastaudinger), Hrafn Malmquist (J4bbi), Raf Ponsaerts (Raf-atmire), Andrea Bollini (abollini), Francesco Pio Scognamiglio (francescopioscognamiglio), Christian Scheible (christian-scheible), Kristof De Langhe (Atmire-Kristof), Flo (floriangantner)

The above contributor list was determined based on contributions to the "DSpace" project in GitHub between 7.4 (after October 6, 2022) and 7.5: https://github.com/DSpace/DSpace/graphs/contributors?from=2022-10-06&to=2023-02-16&type=c

Financial Acknowledgments

We gratefully recognize those institutions who have generously pledged to financially support one or more 7.x releases via the DSpace Development Fund. A list of those funders can be found on the DSpace Development Fund page.

7.4 Release Notes

DSpace 7.4 was released on October 10, 2022

To upgrade to DSpace 7.4 from 7.x *or any prior version*, see Upgrading DSpace.

To install DSpace 7.4 for the first time, see Installing DSpace.

- Download DSpace 7.4 Backend: https://github.com/DSpace/DSpace/releases/tag/dspace-7.4
- Download DSpace 7.4 User Interface: https://github.com/DSpace/dspace-angular/releases/tag/dspace-7.4

DSpace 7.4 provides new features & bug fixes to the 7.x platform.

New and improved features include:

- **Recent Submissions are now listed on the homepage**. (See "recentSubmissions" options) *(donated by DSpaceDirect, developed by DSquare Technologies)*
- **Batch export (to Zip) of Collections or Items** available in the Admin UI
- **Batch import (from Zip)** available in the Admin UI.
- **Thumbnails are now displayed** in all search/browse screens. (See "showThumbnails" option) *(donated by Michael Spalti, Willamette University)*
- **Support for Google Captcha** on "New user registration" form. (See "registration.verification.enabled" in dspace.cfg) *(donated by 4Science)*
- **Support for Google Analytics 4** ("Universal Analytics" is still also supported)
- **Support for Markdown, HTML and MathJax** in Item abstracts. Support for line breaks in all metadata fields.
- **Support for Remote Handle Resolver** now included, allowing for Handle Server to be run remotely.
- **Enhanced support for Amazon S3 as a storage location**, including support for IAM Roles.
- **Allow for deletion of old Processes** from the Admin UI. Bulk deletion script (process-cleaner) also available.
- **Request a Copy can now be sent to multiple recipients** at once *(donated by Mark Wood, IUPUI)*
- Privacy statement and end user agreement can now be disabled. (See "info" options)
- Add configurations for page sizes to many browse/search pages. (See various "pageSize" options in User Interface Configuration) *(donated by Mark Wood, IUPUI, and Mark Cooper, LYRASIS)*

Major bug fixes / improvements include:

- Fixed issue where "start-handle-server" command line tool wasn't working properly *(donated by Jean-François Morin, Université Laval and Mark Wood, IUPUI)*
- MyDSpace: Entities are now displayed properly, workflow tasks now refresh after state change
- Submission: After drag & drop of a file, the user goes directly to the submission form. *(donated by Nicolas Boulay, Université Laval)*
- Statistics: fixed issue where file names were sometimes missing or replaced by UUID.
- Thumbnails: restricted thumbnails are now supported and are displayed if you have access.
- Search: admins were seeing private/withdrawn items in normal searches. Those now are only visible in the "Admin Search" page.
- Browse by pages: fixed issue where pagination was not working properly on all browse by pages and diacritics were not working properly in filter boxes
- Curation tasks: the output of the task is now visible from the Admin UI.
- Password authentication security enhancements: user must provide current password when changing their password. Password minimum requirements are now configurable, allowing for you to require users to create more secure passwords (See "authentication-password.regex-validation.pattern" in dspace.cfg)
- Shibboleth authentication: Admin menu now reloads automatically after login
- Numerous caching & performance fixes to ensure various pages will now automatically refresh whenever underlying objects change.
- See the 7.4 milestone for frontend and backend for a list of *all changes applied in 7.4.*

New/Updated Language support:

- Finnish (Suomi) language updates donated by Reeta Kuuskoski (reetagithub)
- French (Français) language updates donated by Pierre Lasou (pilasou)
- German (Deutsch) language updates donated by Sascha Szott (saschaszott)
- Greek () language added & donated by Kostis Alexandris (kostisalex)
- Hindi () language added & donated by DSquare Technologies
- Kazakh () language added & donated by myrza1
- Brazilian Portuguese (Português do Brasil) updates donated by João Fernandes (joao-uefrom), Lucas Zinato Carraro (lucaszc) and Danilo Felicio Jr (danilofjr)
- Spanish (Español) language updates donated by Arvo Consultores y Tecnología. S.L
- Swedish (Svenska) language added & donated by Urban Andersson (jokermanse). Additional updates donated by Reeta Kuuskoski (reetagithub)

7.4 Acknowledgments

The DSpace application would not exist without the hard work and support of its community. Thank you to the many developers who have worked very hard to deliver all the new features and improvements. We deeply appreciate the financial contributions by those institutions that have contributed to the DSpace Development Fund. Also, thanks to the users who provided input and feedback on the development, those who contributed documentation as well those who participated in the testathons.

Development Acknowledgments

A total of 49 unique individuals contributed to 7.4, with major institutional contributions coming from 4Science and Atmire.

Frontend / User Interface Acknowledgments

The following 39 individuals have contributed directly to the new DSpace (Angular) User Interface in this release (ordered by number of GitHub commits): Yury Bondarenko (ybnd), Michael Spalti (mspalti), Giuseppe Digilio (atarix83), Davide Negretti (davidenegretti-4science), Lotte Hofstede (LotteHofstede), nikunj59, Peter Wolfersberger (hpwBLN), Tim Donohue (tdonohue), Samuel Cambien (samuelcambien), Art Lowel (artlowel), Mark Wood (mwoodiupui), Alexandre Vryghem (alexandrevryghem), Pierre Lasou (pilasou), Vincenzo Mecca (vins01-4science), myrza1, Yana De Pauw (YanaDePauw), Lucas Zinato Carraro (lucaszc), Abhishek Raval (abhishekd2t), GauravD2t, Danilo Felicio Jr (danilofjr), Kostis Alexandris (kostisalex), Kristof De Langhe (Atmire-Kristof), Mark Cooper (mark-cooper), Toni Prieto (toniprieto), FelixNicol, João Fernandes (joao-uefrom), David Steelman (dsteelma-umd), Kostas Stamatis (kstamatis), Corrado Lombardi (corrad82-4s), Sergio Fernández Celorio (sergius02), Urban Andersson (jokermanse), yingjin, Ian van der Linde (Ianvdl), Andrea Bollini (abollini), Hardy Pottinger (hardyoyo), Reeta Kuuskoski (reetagithub), Sascha Szott (saschaszott), Kim Shepherd (kshepherd), Nicolas Boulay (nibou230)

The above contributor lists were determined based on contributions to the "dspace-angular" project in GitHub between 7.3 (after June 24, 2022) and 7.4: https://github.com/DSpace/dspace-angular/graphs/contributors?from=2022-06-24&to=2022-10-06&type=c

Backend / REST API Acknowledgments

The following 20 individuals have contributed directly to the DSpace backend (REST API, Java API, OAI-PMH, etc.) in this release (ordered by number of GitHub commits): Michele Boychuk (Micheleboychuk), Luca Giamminonni (LucaGiamminonni), Vincenzo Mecca (vins01-4science), Michael Spalti (mspalti), Tim Donohue (tdonohue), Francesco Pio Scognamiglio (francescopioscognamiglio), Mark Wood (mwoodiupui), Philipp Rumpf (philipprumpf), ammanabrolua, Andrea Bollini (abollini), Yury Bondarenko (ybnd), eskander17, Kim Shepherd (kshepherd), ballarinie, Davide Negretti (davidenegretti-4science), Toni Prieto (toniprieto), Ben Bosman (benbosman), Samuel Cambien (samuelcambien), Marie Verdonck (MarieVerdonck), Jean-François Morin (jeffmorin)

The above contributor list was determined based on contributions to the "DSpace" project in GitHub between 7.3 (after June 24, 2022) and 7.4: https://github.com/DSpace/DSpace/graphs/contributors?from=2022-06-24&to=2022-10-06&type=c

Financial Acknowledgments

We gratefully recognize those institutions who have generously pledged to financially support one or more 7.x releases via the DSpace Development Fund. A list of those funders can be found on the DSpace Development Fund page.

7.3 Release Notes

DSpace 7.3 was released on June 27, 2022

To upgrade to DSpace 7.3 from 7.x *or any prior version*, see Upgrading DSpace.

To install DSpace 7.3 for the first time, see Installing DSpace.

- Download DSpace 7.3 Backend: https://github.com/DSpace/DSpace/releases/tag/dspace-7.3
- Download DSpace 7.3 User Interface: https://github.com/DSpace/dspace-angular/releases/tag/dspace-7.3

DSpace 7.3 provides new features & bug fixes to the 7.x platform.

New and improved features include:

- **ORCID Authentication and synchronization to a DSpace Researcher Profile** *(ported from the DSpace-CRIS project, partially donated and developed by 4Science).* See ORCID Integration and Researcher Profiles
- **Import content directly from 9 new external services** including CrossRef, Scopus, Web of Science, PubMed Europe, CiNii, NASA Astrophysics Data System (ADS), VuFind.org, SciELO.org, and the European Patent Office (EPO) (*ported from the DSpace-CRIS project, partially donated and developed by 4Science). See Importing Items via basic bibliographic formats (Endnote, BibTex, RIS, CSV, etc) and online services (arXiv, PubMed, CrossRef, CiNii, etc)*
- **Configurable Entities now support Item Versioning.** It is now possible to create versions of Entities which automatically retain all relationships. Examples of how versioning works can be found in "Versioning Support" section of "Configurable Entities". Also refer to Item Level Versioning documentation for a general overview of how versioning works in DSpace.
- **Admin "Health" menu provides basic control panel functionality** *(based on 6.x Control Panel).* When logged in as an Administrator, select "Health" from the side menu. You'll see a "Status" tab which provides useful information about the status of the DSpace backend, and an "Info" tab which provides an overview of backend configurations and Java information.
- **Validate a Batch Metadata CSV before applying changes** *(similar to 6.x).* When uploading a CSV for batch updates (using "Import" menu), a new "Validate Only" option is selected by default. When selected, the uploaded CSV will only be validated & you'll receive a report of the detected changes in the CSV. This allows you to verify the changes are correct before applying them. (NOTE: applying the changes requires re-submitting the CSV with the "Validate Only" option deselected)
- **Export search results to a CSV** *(similar to 6.x).* When logged in as an Administrator, after performing a search a new "Export search results as CSV" button appears. Clicking it will export the metadata of all items in your search results to a CSV. This CSV can then be used to perform batch metadata updates (based on the items in your search results).
- **Submission forms support display of SHERPA/RoMEO publisher policies when an ISSN is entered** *(similar to 6.x).* A new "sherpaPolicies" optional submission step exists. When enabled, a new "Sherpa policies" section appears in the form. If a submitter enters an ISSN, then that form section will display publisher policies based on that ISSN. See "Configuring the Sherpa Romeo step" in the "Submission User Interface"
- **Submission forms support type-based fields** (*ported from the DSpace-CRIS project by Kim Shepherd of The Library Code, with support from Technische Universität Berlin) (similar to 6.x):* Based on the selected Type (dc.type), the form may dynamically add/change metadata fields available. See "Item Type Based Metadata Collection" in Submission User Interface
- **Preview items during workflow approval** *(similar to 6.x).* When viewing the list of all items under workflow (in "My DSpace" "Show: Workflow Tasks"), it's now possible to preview each item (before claiming or reviewing) by clicking the View button.
- **Withdrawn items now show a tombstone page** *(similar to 6.x).* When an item is withdrawn, accessing it's homepage as a non-Administrator now brings you to a "tombstone" page which notes that the Item was withdrawn. (Administrative users can see see the entire item when they are logged in)

- **RSS/Atom feeds for Site, Community & Collection pages** *(Donated by Atmire) (similar to 6.x).* On the homepage, and on every Community or Collection page, a Syndication Feed icon now appears. Clicking it brings you to an Atom feed of the most recent submissions (either sitewide, or specific to that Community/Collection).
- Optionally, **Item "access status" badges (e.g. "open access", "restricted", "metadata only", "embargoed") can be displayed in all Item lists.** *(Donated by Université Laval)* This feature can be enabled via the new "showAccessStatuses" setting in config.*.yml. See User Interface Configuration and https://github.com/DSpace/dspace-angular/issues/1566
- Optionally, **a welcome email can be sent to all newly registered users** *(Donated by Mark H. Wood of IUPUI)*. This can be enabled using the new "mail.welcome.enabled" backend configuration in your local.cfg.
- **Oracle Database support has been deprecated.** It will be removed in mid-2023. All sites should plan a migration to PostgreSQL. See https://github.com/DSpace/DSpace/issues/8214
- Migrated text extraction to use Apache Tika. There is now a single "Text Extractor" media filter plugin. See Mediafilters for Transforming DSpace Content for more details.

Major bug fixes / improvements include:

- Major frontend performance improvements: initial page load is much quicker. All assets are now zipped/minified. Caching improvements.
- Frontend can be easily run from a subpath (e.g. https://my.university.edu/dspace) *(Donated by Harvard University, developed by William Welling)*
- "custom" theme was throwing TypeErrors in 7.2. That has been fixed.
- Renamed "private/public" status to be "non-discoverable/discoverable", which is more accurate. This item status just meant the item was not findable/discoverable through DSpace search/browse. It has nothing to do with whether the item is accessible or not accessible anonymously.
- Added "Now Showing..." informational counts to all Browse By pages (similar to in 6.x)
- Changed Browse "jump to" boxes to act as a filter, rather than jumping to a specific page. Updated display to make this clearer.
- Fixed several bugs related to resource policies
    - Couldn't set an end date on a resource policy in some situations
    - Couldn't edit an existing resource policy (previously you had to delete & recreate the policy)
- Fixed bug where resubmitting an Item move could result in Item deletion.
- Fixed several bugs in submission form, including
    - Improved form validation and error display
    - Occasionally, null metadata values were being saved
    - In some scenarios, submissions could be completed without agreeing to license.
    - Fixed bug where Creative Commons step wasn't loading properly when configured
    - Fixed bug where radio buttons were not working properly when submission field was set as a "list" that was not repeatable.
    - Fixed several bugs in bitstream editing form
- Fixed bug with browse by issue date wouldn't work if it encountered an invalid date in the first item. *(Donated by 4Science)*
- Fixed bug where bitstream edit page wouldn't load if a bitstream's access policies were all deleted.
- Fixed bug where restricted bitstream downloads were not checking a user's "special groups" which may provide them permissions. *(Donated by 4Science)*
- Frontend has been upgraded to Angular 13.
- Backend has been updated to Spring Boot 2.6
- Numerous other dependency updates
- See the 7.3 milestone for frontend and backend for a list of *all changes applied in 7.3.*

New/Updated Language support:

- Bengali () language added & donated by Md. Topu Raihan (raihantopu)
- Finnish (Suomi) language updates donated by Reeta Kuuskoski (reetagithub)
- French (Français) language updates donated by Marie-Hélène Vézina (mhvenzina)
- German (Deutsch) language updates donated by Sascha Szott (saschaszott) and louiserumpf and Yannick Paulsen (YPaulsen-TLC) and WolfgangT71
- Turkish (Türkçe) language added & donated by stanbul Technical University IT Office

7.3 Acknowledgments

The DSpace application would not exist without the hard work and support of its community. Thank you to the many developers who have worked very hard to deliver all the new features and improvements. We deeply appreciate the financial contributions by those institutions that have contributed to the DSpace Development Fund. Also, thanks to the users who provided input and feedback on the development, those who contributed documentation as well those who participated in the testathons.

Development Acknowledgments

A total of 37 unique individuals contributed to 7.3, with major institutional contributions coming from 4Science and Atmire.

Frontend / User Interface Acknowledgments

The following 28 individuals have contributed directly to the new DSpace (Angular) User Interface in this release (*ordered by number of GitHub commits*): Giuseppe Digilio (atarix83), Yury Bondarenko (ybnd), Davide Negretti (davidenegretti-4science), Kim Shepherd (kshepherd), Tim Donohue (tdonohue), Sascha Szott (saschaszott), Luca Giamminonni (LucaGiamminonni), Alexandre Vryghem (alexandrevryghem), Art Lowel (artlowel), Yana De Pauw (YanaDePauw), Lotte Hofstede (LotteHofstede), Marie Verdonck (MarieVerdonck), Bruno Roemers (bruno-atmire), Michael Spalti (mspalti), stanbul Technical University IT Office (itudevops), Sufiyan Shaikh (SufiyanShaikhPeerbits), William Welling (wwelling), Michele Boychuk (Micheleboychuk), nikunj59, Kristof De Langhe (Atmire-Kristof), louiserumpf, Pierre Lasou (pilasou), Marie-Hélène Vézina (mhvenzina), Reeta Kuuskoski (reetagithub), pratik-peerbits, Md. Topu Raihan (raihantopu), Pascal-Nicolas Becker (pnbecker), Rezart Vata (rezartatis)

The above contributor lists were determined based on contributions to the "dspace-angular" project in GitHub between 7.2 (after Feb 3, 2022) and 7.3: https://github.com/DSpace/dspace-angular/graphs/contributors?from=2022-02-03&to=2022-06-24&type=c

Backend / REST API Acknowledgments

The following 20 individuals have contributed directly to the DSpace backend (REST API, Java API, OAI-PMH, etc.) in this release (*ordered by number of GitHub commits*): Luca Giamminonni (LucaGiamminonni), Bruno Roemers (bruno-atmire), Michele Boychuk (Micheleboychuk), Tim Donohue (tdonohue), ballarinie, Andrea Bollini (abollini), Mark Wood (mwoodiupui), Kristof De Langhe (Atmire-Kristof), Kim Shepherd (kshepherd), Antoine Snyers (antoine-atmire), Ben Bosman (benbosman), Marie Verdonck (MarieVerdonck), Corrado Lombardi (corrad82-4s), Yana De Pauw (YanaDePauw), eskander17, Michael Spalti (mspalti), Toni Prieto (toniprieto), Giuseppe Digilio (atarix83), Pascal-Nicolas Becker (pnbecker), Samuel Cambien (samuelcambien)

The above contributor list was determined based on contributions to the "DSpace" project in GitHub between 7.2 (after Feb 3, 2022) and 7.3: https://github.com/DSpace/DSpace/graphs/contributors?from=2022-02-03&to=2022-06-24&type=c

Financial Acknowledgments

We gratefully recognize the following institutions who have generously pledged financially to support the 7.3 release via the DSpace Development Fund.

- Charité - Universitätsmedizin Berlin - Medical Library
- Freie Universität Berlin, Universitätsbibliothek
- Georgia Tech Library
- Humboldt Universität zu Berlin
- Indiana University, Purdue University, Indianapolis (IUPUI)
- Technische Universität Berlin
- Universitäts- und Landesbibliothek Darmstadt
- Universitätsbibliothek Kassel
- Universität Konstanz
- University of Arizona Libraries
- University of Edinburgh
- University of Kansas
- Virginia Tech University Libraries

7.2.1 Release Notes (Backend Only)

All DSpace 7.0, 7.1 or 7.2 sites should ensure the backend is upgraded to 7.2.1 (or later) or manually patched

DSpace 7.0, 7.1 and 7.2 all used a bundled version of the Apache Spring Libraries which are vulnerable to RCE (remote command execution). The CVE-2022-22965 vulnerability is described in more detail at https://spring.io/blog/2022/03/31/spring-framework-rce-early-announcement

If you cannot upgrade immediately, other workarounds / alternative fixes are documented in the patch PR at https://github.com/DSpace/DSpace/pull/8231

DSpace 7.2.1 only contains an update to the Apache Spring Libraries to ensure DSpace is not vulnerable to CVE-2022-22965. As such, it was only a Backend / REST API release. The DSpace 7.2 Frontend (UI) can be used with the DSpace 7.2.1 Backend.

To ensure your 7.x site is completely secure, perform the following:

1. Upgrade your DSpace backend (REST API) to version 7.2.1 immediately. This backend is compatible with the DSpace Frontend version 7.2 (only)
    a. If you are unable to perform this upgrade, you may patch your 7.0 or 7.1 site by applying the changes in PR #8231. Instructions can be found in that PR.
2. Optionally, upgrade your Apache Tomcat to version 9.0.62 (which also has extra guards against this vulnerability).
3. Make sure to restart Tomcat after updates have been applied.

At this time, DSpace 6.x and below appear unaffected by CVE-2022-22965, as they all used Java/JDK 8 (or below) which is documented as not impacted. The vulnerability is only possible when using Java/JDK 9 or above.

7.2 Release Notes

DSpace 7.2 was released on February 7, 2022

To try out DSpace 7.2 immediately, see Try out DSpace 7. This includes instructions for a quick-install via Docker, as well as information on our sandbox/demo site for DSpace 7.

To upgrade to DSpace 7.2 from 7.x *or any prior version*, see Upgrading DSpace.

To install DSpace 7.2 for the first time, see Installing DSpace.

- Download DSpace 7.2 Backend: https://github.com/DSpace/DSpace/releases/tag/dspace-7.2
- Download DSpace 7.2 User Interface: https://github.com/DSpace/dspace-angular/releases/tag/dspace-7.2

DSpace 7.2 provides new features & bug fixes to the 7.x platform.

New and improved features include:

- **Runtime Configuration for the User Interface** *(Donated by Harvard University, developed by William Welling)*: In DSpace 7.0 and 7.1, changes to your User Interface Configuration required rebuilding the entire UI (which could take 10+ minutes). As of 7.2, *all User Interface configurations are loaded at runtime*. So, to change configurations just requires a quick restart of the User Interface (which usually takes only a few seconds). The configuration format also changed from Typescript to YAML to support this feature. A "yarn env:yaml" migration script is provided to migrate the old format to the new one. See User Interface Configuration for more details.
- **Add Item Embargoes / Restrictions in the Submission User Interface:** A new, optional "itemAccessConditions" step exists in the Submission configuration. Enabling it adds a section which allows you to select access restrictions, embargoes or leases. It also allows you to select whether the Item is discoverable via search/browse. See Submission User Interface and Embargo documentation for details.

- **Feedback Form:** A feedback form is now linked in the footer of every page, as long as a "feedback.recipient" is specified in your local.cfg. This feature allow users to contact the configured "feedback.recipient" from any page in the site.
- **OpenID Connect (OIDC) Authentication Plugin** *(Ported from the DSpace-CRIS project by Hardy Pottinger of California Digital Library, with support from 4Science)*: DSpace now supports single sign on using OpenID Connect (OIDC), which allows it to support authentication through providers such as Google, Microsoft, Amazon, etc. For more information on setting this up, see the Authentication Plugins page.
- **IIIF Enhancements** *(Donated by Michael Spalti of Willamette University)***:** Includes support for adding IIIF metadata using Importing and Exporting Items via Simple Archive Format (SAF), editing IIIF bitstream metadata from the User Interface (when editing an existing Bitstream), and a new "./dspace iiif-canvas-dimensions" CLI tool for auto-populating IIIF canvas dimensions in bulk.
- **Running "filter-media" (Mediafilters) from Processes User Interface.** Administrators can now run the "filter-media" script from the Admin UI ("Processes" menu), in order to immediately update thumbnails, full text indexing, etc. See Mediafilters for Transforming DSpace Content for more details about this script.
- **Improved support for custom "Browse By" configurations.** User Interface "Browse by" options are now retrieved dynamically from the REST API, based on the backend's configured browse by indexes (see "webui.browse.index.*" options documented in the Configuration Reference)
- **Backend has added support for JDK 17.** The DSpace backend now supports either JDK 11 or JDK 17.
- **Frontend has been upgraded to Angular 11.**
- **Solr now uses a connection pool by default** *(Donated by Mark H. Wood of IUPUI)*. See Configuration Reference for details of new "solr.client.*" configs in dspace.cfg.

Major bug fixes include:

- User interface would load indefinitely if the REST API was unavailable. Now, an error page is displayed to let you know the REST API is unresponsive.
- User Interface deployment required the "node_modules" folder to exist, making it more difficult to containerize (e.g. Docker). Now, the UI can be deployed via only the "dist" folder. See https://github.com/DSpace/dspace-angular/issues/1410 *(Donated by Harvard University, developed by William Welling)*
- Searches with invalid syntax or special characters would load indefinitely. Now an error is displayed if the syntax is invalid in some way.
- On Item page, very long metadata fields or file names would break the page layout
- When an Item had an invalid or empty "dspace.entity.type" metadata field, it was unable to be deleted.
- On Submission page, if you drag & drop a file to start a submission, the Collection selection window sometimes did not load properly.
- On Submission page, sometimes the "Deposit" button would not enable even when all required fields are filled out. "Deposit" button is now always enabled, but it will block submission if required fields are missing.
- On Submission page, fixed several bugs with editing / setting embargoes or access restrictions on uploaded files.
- Statistics were always accessible publicly, even if restricted to Administrators. Statistics now are only accessible to Admins when `usage-statistics.authorization.admin.usage` is set to true in local.cfg
- Administrators were not able to reset passwords of other users.
- On Processes page, scripts could not be run without parameters. Additionally, fixed display of dates so they always appear as UTC time
- When both Shibboleth and DSpace password authentication were enabled, users were able to change their password in DSpace in order to bypass Shibboleth.
- On backend, improved indexing performance. *(Donated by 4Science)*
- On backend, improved file download performance to avoid connection leaks when S3 is used as backend storage.
- Numerous other minor bug fixes or accessibility improvements. See the 7.2 milestone for frontend and backend for a list of *all changes applied in 7.2.*

New/Updated Language support:

- Scottish Gaelic (Gàidhlig) user interface support added *(Donated by Donald I Macdonald and Stòrlann Nàiseanta na Gàidhlig)*
- German (Deutsch) user interface support had a syntax error which caused it not to work properly

7.2 Acknowledgments

A total of 28 unique individuals contributed to 7.2, with major institutional contributions coming from 4Science and Atmire.

Frontend / User Interface Acknowledgments

The following 15 individuals have contributed directly to the new DSpace (Angular) User Interface in this release (*ordered by number of GitHub commits*): Giuseppe Digilio (atarix83), William Welling (wwelling), Tim Donohue (tdonohue), Davide Negretti (davidenegretti-4science), Michael Spalti (mspalti), Rezart Vata (rezartatis), Bruno Roemers (bruno-atmire), Lotte Hofstede (LotteHofstede), Art Lowel (artlowel), Yury Bondarenko (ybnd), Corrado Lombardi (corrad82-4s), Kristof De Langhe (Atmire-Kristof), Hardy Pottinger (hardyoyo), Donald I Macdonald (donaldi), Santiago Tettamanti (santit96)

The above contributor lists were determined based on contributions to the "dspace-angular" project in GitHub between 7.1 (after Oct 28, 2021) and 7.2: https://github.com/DSpace/dspace-angular/graphs/contributors?from=2021-10-28&to=2022-02-03&type=c

Backend / REST API Acknowledgments

The following 23 individuals have contributed directly to the DSpace backend (REST API, Java API, OAI-PMH, etc) in this release (*ordered by number of GitHub commits*): Tim Donohue (tdonohue), Michele Boychuk (Micheleboychuk), Michael Spalti (mspalti), Hardy Pottinger (hardyoyo), Yana De Pauw (YanaDePauw), Joost Fock (joost-atmire), Kevin Van de Velde (KevinVdV), Corrado Lombardi (corrad82-4s), Jose Vicente Ribelles Aguilar (jvribell), Davide Negretti (davidenegretti-4science), Luca Giamminonni (LucaGiamminonni), Yury Bondarenko (ybnd), Ben Bosman (benbosman), Marie Verdonck (MarieVerdonck), Bruno Roemers (bruno-atmire), Mark Wood (mwoodiupui), Hrafn Malmquist (J4bbi), Paulo Graça (paulo-graca), Andrea Bollini (abollini), Giuseppe Digilio (atarix83), William Welling (wwelling), Kristof De Langhe (Atmire-Kristof), Samuel Cambien (samuelcambien)

The above contributor list was determined based on contributions to the "DSpace" project in GitHub between 7.1 (after Oct 28, 2021) and 7.2: https://github.com/DSpace/DSpace/graphs/contributors?from=2021-10-28&to=2022-02-03&type=c

7.1.1 Release Notes (Backend Only)

All DSpace 7.0 or 7.1 sites should ensure the backend is upgraded to 7.1.1 (or above) or manually patched

DSpace 7.0 and 7.1 both used a bundled version of the Apache Log4j Library vulnerable to RCE (remote command execution). The CVE-2021-44228 vulnerability is described in more detail at https://cve.mitre.org/cgi-bin/cvename.cgi?name=CVE-2021-44228 and https://logging.apache.org/log4j/2.x/security.html#Fixed_in_Log4j_2.15.0

DSpace 7.1.1 only contains an update to the Apache Log4j Library to ensure DSpace is not vulnerable to CVE-2021-44228. As such, it was only a Backend / REST API release. The DSpace 7.1 Frontend (UI) can be used with the DSpace 7.1.1 Backend.

To ensure your 7.x site is completely secure, perform ALL the following:

1. Upgrade your DSpace backend (REST API) to version 7.1.1 immediately. This backend is compatible with the DSpace Frontend version 7.1
    a. If you are unable to perform this upgrade, you may patch your 7.0 or 7.1 site by applying the changes in PR #8065. Specifically, update your ./pom.xml to have <log4j.version>2.15.0</log4j.version>. Then rebuild & redeploy your backend. Make sure to restart Tomcat.
2. Upgrade to Apache Solr v8.11.1 (or above), to ensure your Solr is patched for CVE-2021-44228
    a. If you are unable to perform this upgrade, you may patch your current Solr by ensuring that `-Dlog4j2.formatMsgNoLookups=true` is specified in your `SOLR_OPTS` environment variable. For more information, see https://solr.apache.org/security.html#apache-solr-affected-by-apache-log4j-cve-2021-44228
3. If you use the Handle.Net Registry Support in DSpace 7.x, make sure to restart your Handle Server. This will ensure it is using the new version of log4j as well.

At this time, DSpace 6.x and below appear unaffected by CVE-2021-44228, as they all used log4j v1 exclusively with a default configuration that is not impacted.

Additional log4j patch available for 7.x

Immediately after version 7.1.1 was released, the log4j community announced a secondary, less severe vulnerability (CVE-2021-45046) which was patched in a log4j v 2.16.0 release.

This fix is NOT included in 7.1.1. But, you can immediately apply this secondary patch by applying the changes in https://github.com/DSpace/DSpace/pull/8070. This is again a one line change. Simply update your ./pom.xml to have <log4j.version>2.16.0</log4j.version>. Then rebuild & redeploy your backend.

7.1 Release Notes

We highly recommend ALL DSpace 7.0 users upgrade to 7.1

DSpace 7.1 contains a security fix to the backend (REST API) for all sites running 7.0. See CVE-2021-41189 for details.

DSpace 7.1 was released on November 1, 2021

To try out DSpace 7.1 immediately, see Try out DSpace 7. This includes instructions for a quick-install via Docker, as well as information on our sandbox/demo site for DSpace 7.

To upgrade to DSpace 7.1 from 7.0 *or any prior version*, see Upgrading DSpace.

To install DSpace 7.0 for the first time, see Installing DSpace.

- Download DSpace 7.1 Backend: https://github.com/DSpace/DSpace/releases/tag/dspace-7.1
- Download DSpace 7.1 User Interface: https://github.com/DSpace/dspace-angular/releases/tag/dspace-7.1

DSpace 7.1 provides new features, security & bug fixes to the 7.x platform.

New and improved features include:

- **Request a Copy** *(Backend donated by Mark H. Wood of IUPUI)*: Similar to v6.x, users can now ask the original author or submitter (or a help desk email) for an emailed copy of access restricted files. This provides users with a way to privately get copies of restricted files, should the request be approved. A request can be submitted by simply clicking on an access restricted bitstream in the UI. Approval or rejection of the request occurs by clicking the link sent in the request email.
- **Item Versioning:** Similar to v6.x, administrators or submitters can now create new versions of Items. A new Item version can be created by logging in & clicking the "Create new version" button (next to the "Edit this item" button) on an Item's page. The new version is then created via the normal Item submission form (prepopulated with all existing information). Once created, all versions of an Item are visible on the Item page in the "Version History" section.
    - *Item Versioning is enabled by default, but can be disabled via configuration.*
    - *Entities are not yet supported for Versioning.*
- **Configure Collections to harvest content via OAI-PMH (OAI Harvesting):** Similar to 6.x (XMLUI), on the "Edit Collection" page's "Content Source" tab, there's an option to specify "This collection harvests its content from an external source". When enabled for a Collection, you can configure an external OAI-PMH instance (including another DSpace site) to harvest from. Once configured in the UI, harvesting is completed based on the configured schedule in your backend's local.cfg or oai.cfg.
- **IIIF Support** *(Donated by Michael Spalti of Willamette University, with support & enhancements donated by 4Science)*: DSpace now supports the International Image Interoperability Framework (IIIF.io), including an embedded IIIF viewer (Mirador) in the UI. IIIF support is disabled by default, but can be easily enabled via configuration. Enabling IIIF also requires installing a IIIF image server (e.g. Cantaloupe). For more details, please see the linked documentation.
- **Ability to "extend" other User Interface themes:** In your environment.prod.ts, you can now specify that one theme "extends" another. This allows you to inherit all settings from the extended them by default. See the "Entending other Themes" section of the User Interface Customization documentation.
- **Configure one Entity Type per Collection** *(Ported by 4Science from their DSpace-CRIS project)*: When Configurable Entities are enabled, in the "Edit Collection" page you can select an Entity Type (e.g. Person, Project, Journal, etc) that Collection will accept. Once configured, this Collection will only accept new Submissions of that Entity Type, and will be one of the *recommended Collections* to Submitters whenever they start a new Submission of that Entity Type. See "Configure Collections for each Entity type" section of the Configurable Entities documentation.

- **Support for importing Entities & Relationships via the Simple Archive Format** (*Donated by tysonlt*)**:** This is achieved via a new, optional "relationships" file in the Simple Archive Format directory. See the documentation for more details.
- **Support for importing Project Entities with funding information via the OpenAIRE API** (*Donated by Paulo Graça*): When importing a new "Research Project" Entity, a new "Funding OpenAIRE API" option is available, allowing you to import a Project from the OpenAIRE API complete with all it's funding information (Funder, Funder Identifier, Funding Stream and Funding ID). This is implemented via a new external source via Live Import from external sources.
- **Command-line script to help test the connection between your UI and your REST API.** Several people who installed 7.0 early ran into issues configuring the UI and REST API properly. A new "yarn config:check:rest" script has been added to the frontend codebase to help validate the connection with your REST API. It should also provide more descriptive errors (should they occur) which will help us to debug future issues others may encounter. See the "Frontend Installation" instructions (step 4) in Installing DSpace for more details.
- **Logical Item Filtering and DOI Filtered Provider for DSpace.** This introduces a framework to define rules using boolean logic to filter items. Furthermore these filters can be used to decide whether a DOI should be minted for a certain item or not. Donated by The Library Code with the support of TUHH and TIB.

Security fixes include:

- **[HIGH] CVE-2021-41189**: In 7.0, a Community or Collection Admin could escalate their permissions to become a full Administrator. A quick fix is a lso provided for sites running 7.0. *(Reported by Andrea Bollini of 4Science)*

Major bug fixes include:

- Major performance improvements to UI and REST API
  - In 7.0, several UI pages (Homepage, MyDSpace, Search, etc) would respond slowly if a large number of Communities & Collections existed in the system. Performance has been drastically improved in 7.1.
- In Submission form, the "administrator" Bitstream access condition was fixed. Previously, depending on Collection's "DEFAULT_READ" settings, the "administrator" permission limitation may not have worked properly. See https://github.com/DSpace/DSpace/pull/8013
- In Login form, some special characters in passwords were not accepted. This was the result of an encoding error which is now fixed. See https://github.com/DSpace/dspace-angular/issues/1367
- In "Edit Item" on "Bitstreams" tab, the upload button was not working properly. This has been fixed. See https://github.com/DSpace/dspace-angular/issues/1286
- In "Edit Item" on "Relationships" tab, it was difficult to manage Entities with many relationships. This has been improved. See https://github.com/DSpace/dspace-angular/issues/1148
- Google Analytics reporting has been moved to an asynchronous/batch script to improve performance. See https://github.com/DSpace/DSpace/pull/2248
- Numerous other minor bug fixes or accessibility improvements. See the 7.1 milestone for frontend and backend for a list of *all changes applied in 7.1.*

New/Updated Language support:

- German (Deutsch) user interface support was updated *(Donated by The Library Code)*
- Spanish (Español) user interface support added *(Donated by Gustavo S. Ferreyro)*

7.1 Acknowledgments

A total of 27 unique individuals contributed to 7.1, with major institutional contributions coming from 4Science and Atmire.

Frontend / User Interface Acknowledgments

The following 18 individuals have contributed directly to the new DSpace (Angular) User Interface in this release (*ordered by number of GitHub commits*): Davide Negretti (davidenegretti-4science), Giuseppe Digilio (atarix83), Yury Bondarenko (ybnd), Tim Donohue (tdonohue), Michael Spalti (mspalti), Rezart Vata (rezartatis), Kristof De Langhe (Atmire-Kristof), Yana De Pauw (YanaDePauw), Lotte Hofstede (LotteHofstede), Alessandro Martelli (alemarte), Bill Branan (bbranan), Art Lowel (artlowel), Corrado Lombardi (corrad82-4s), Marie Verdonck (MarieVerdonck), Bruno Roemers (bruno-atmire), Yannick Paulsen (YPaulsen-TLC), Pascal-Nicolas Becker (pnbecker), Gustavo S Ferreyro (gsferreyro)

The above contributor lists were determined based on contributions to the "dspace-angular" project in GitHub between 7.0 (after July 29, 2021) and 7.1: https://github.com/DSpace/dspace-angular/graphs/contributors?from=2021-07-29&to=2021-10-27&type=c

Backend / REST API Acknowledgments

The following 16 individuals have contributed directly to the DSpace backend (REST API, Java API, OAI-PMH, etc) in this release (*ordered by number of GitHub commits*): Michele Boychuk (Micheleboychuk), Michael Spalti (mspalti), Mark Wood (mwoodiupui), Paulo Graça (paulo-graca), Tim Donohue (tdonohue), Andrea Bollini (abollini), tysonlt, Yury Bondarenko (ybnd), Corrado Lombardi (corrad82-4s), Yana De Pauw (YanaDePauw), Nicholas Woodward (nwoodward), Alan Orth (alanorth), Davide Negretti (davidenegretti-4science), Marie Verdonck (MarieVerdonck), Andrew Wood (AndrewZWood), Ben Bosman (benbosman).

The above contributor list was determined based on contributions to the "DSpace" project in GitHub between 7.0 (after July 29, 2021) and 7.1: https://github.com/DSpace/DSpace/graphs/contributors?from=2021-07-29&to=2021-10-27&type=c

7.0 Release Notes

DSpace 7.0 was released on July 29, 2021

Brief videos of some DSpace 7 features are available at https://duraspace.org/dspace/dspace-7/

DSpace 7.0 is the largest release in the history of DSpace software. While retaining the "out-of-the-box" aspects DSpace is known for, it represents a major evolution of the platform including:

- **A completely new User Interface (demo site).** This is the new Javascript-based frontend, built on Angular.io (with support for SEO provided by Angular Universal). This new interface is also customizable via HTML and CSS (Sass) and Bootstrap. For early theme building tips see User Interface Customization
- **A completely new, fully featured REST API (demo site)**, provided via a single "server" webapp backend. This new backend is not only a REST API, but also still supports OAI-PMH, SWORD (v1 or v2) and RDF. Anything you can do from the User Interface is now also possible in our REST API. See REST API documentation for more details.
- **A newly designed search box.** Search from the header of any page (click the magnifying glass). The search results page now features automatic search highlight, expandable & searchable filters, and optional thumbnail-based results (click on the “grid” view).
- **A new MyDSpace area to manage your submissions & reviews**, MyDSpace includes a new drag & drop area to start a new submission, and easily search your workflow tasks or in progress submissions to find what you were working on. (Login, click on your user profile icon, click “MyDSpace”). Find workflow tasks to claim by selecting “All tasks” in the “Show” dropdown.
- **A new configurable submission user interface,** featuring a one-page, drag & drop submission form. This form is completely configurable and can be prepopulated by dragging & dropping a metadata file (e.g. ArXiv, CSV/TSV, Endnote, PubMed, or RIS. etc) or by importing via external APIs (e.g ORCID, PubMed, Sherpa Journals or Sherpa Publishers, etc) (video). Local controlled vocabularies are also still supported (video). See Submission User Interface for more details.
- **Optional, new Configurable Entities feature**. DSpace now supports “entities”, which are DSpace Items of a specific ‘type’ which may have relationships to other entities. These entity types and relationships are configurable, with two examples coming out-of-the-box: a set of Journal hierarchy entities (Journal, Volume, Issue, Publication) and a set of Research entities (Publication, Project, Person, OrgUnit). For more information see Configurable Entities.
- **Dynamic user interface translations** (Click the globe, and select a language). Interested in adding more translations? See DSpace 7 Translation - Internationalization (i18n) - Localization (l10n).
- **A new Admin sidebar.** Login as an Administrator, and an administrative sidebar appears. Features available include:
    - Quickly create or edit objects from anywhere in the system. Either browse to the object first, or search for it using the Admin sidebar.
    - Processes UI (video) allows Administrators to run backend scripts/processes while monitoring their progress & completion. (Login as an Admin, select "Processes" in sidebar)
    - Administrative Search (video) combines retrieval of withdrawn items and private items, together with a series of quick action buttons.
    - Administer Active Workflows (video) allows Administrators to see every submission that is currently in the workflow approval process.
    - Bitstream Editing (video) has a drag-and-drop interface for re-ordering bitstreams and makes adding and editing bitstreams more intuitive.
    - Metadata Editing (video) introduces suggest-as-you-type for field name selection of new metadata.
    - Login As (Impersonate) another account allows Administrators to debug issues that a specific user is seeing, or do some work on behalf of that user. (Login as an Admin, Click "Access Control" in sidebar, Click "People". Search for the user account & edit it. Click the "Impersonate EPerson" button. You will be authenticated as that user until you click "Stop Impersonating EPerson" in the upper right.)
- **Improved GDPR alignment** (video)
    - User Agreement required for all authenticated users to read and agree to. (Login for first time, and sample user agreement will display. After agreeing to it, it will not appear again.)
    - Cookie Preferences are now available for all users (anonymous or authenticated). A cookie preference popup appears when first accessing the site. Users are given information on what cookies added by DSpace, including a Privacy Statement which can be used to describe how their data is used.
    - User Accounts can be deleted *even if they've submitted content* in the past.
- **Support for OpenAIREv4** Guidelines for Literature Repositories in OAI-PMH (See the new “openaire4” context in OAI-PMH).
- **Search Engine Optimization**: Tested and approved by the Google Scholar team, DSpace still includes all the SEO features you require: a robots.txt, Sitemaps and Google Scholar "citation" tags.
- **Video/Image Content Streaming** *(Kindly donated by Zoltán Kanász-Nagy and Dániel Péter Sipos of Qulto):* When enabled, DSpace can now stream videos & view images full screen, using an embedded viewer. (See the "mediaViewer" settings in the environment.common.ts to enable.)
- **Basic Usage Statistics** (video) are available for the entire site (See "Statistics" menu at top of homepage), or specific Communities, Collections or Items (Click on that same "Statistics" menu after browsing to a specific object
- Additional features are listed in the Beta release notes below. Also, give it a try on our demo site & see what you discover!

DSpace 7 does not yet include all the features of DSpace 6.x

DSpace 7.0 represents a major evolution of the platform into a new, modern web architecture. This means there are tons of new and redesigned features in 7.0. However, in order to get this release in your hands sooner, DSpace Steering decided to delay some 6.x features for later 7.x releases. So, if you don't see a 6.x feature yet in 7.0, it'll likely be coming soon in a later 7.x release. For a prioritized list of upcoming features see "What features are coming in a later 7.x release?" on our DSpace Release 7.0 Status page.

Additional major changes to be aware of in the 7.x platform (not an exhaustive list):

- **XMLUI and JSPUI are no longer supported or distributed with DSpace.** All users should immediately migrate to and utilize the new Angular User Interface. There is no migration path from either the XMLUI or JSPUI to the new User interface. However, the new user interface can be themed via HTML and CSS (SCSS).
- **The old REST API ("rest" webapp from DSpace v4.x-6.x) is deprecated and will be removed in v8.x.** The new REST API (provided in the "server" webapp) replaces all functionality available in the older REST API. If you have tools that rely on the old REST API, you can still (optionally) build & deploy it alongside the "server" webapp via the "-Pdspace-rest" Maven flag. See REST API v6 (deprecated)
- **The Submission Form configuration has changed.** The "item-submission.xml" file has changed its structure, and the "input-forms.xml" has been replaced by a "submission-forms.xml". See Submission User Interface
- **ElasticSearch Usage Statistics have been removed.** Please use SOLR Statistics or DSpace Google Analytics Statistics.
- **The traditional, 3-step Workflow system has been removed in favor of the Configurable Workflow System.** For most users, you should see no effect or difference. The default setup for this Configurable Workflow System is identical to the traditional, 3-step workflow ("Approve/Reject", "Approve/Reject/Edit Metadata", "Edit Metadata")
- **The old BTE import framework in favor of Live Import Framework** (features of BTE have been ported to Live Import)
- **Apache Solr is no longer embedded within the DSpace installer.** Solr now MUST be installed as a separate dependency alongside the DSpace backend. See Installing DSpace.
- **A large number of old/obsolete configurations were removed.** "7.0 Configurations Removed" section below.
- See Upgrading DSpace for more hints on the upgrade from any old version of DSpace to 7.x

Additional Resources

- Video presentations / Workshops from OR2021 (June 2021) showing off many of the new features & configurations of DSpace 7: DSpace 7 at OR2021

7.0 Configurations Removed

With the removal of the JSPUI and XMLUI, a large number of server-side (backend) configurations were made obsolete and were therefore removed between the 6.x and 7.0 release. Those Configurations removed included:

- Within the `[dspace]/config/` directory, these are the **configuration files which were deleted:**
  - `dc2mods.cfg`
  - `input-forms.xml / dtd` (*REPLACED BY submission-forms.xml, see Submission User Interface)*
  - `log4j.properties` (REPLACED BY `log4j2.xml`)
  - `log4j-console.properties` (REPLACED BY `log4j-console.xml`)
  - `log4j-solr.properties` (no replacement as Solr now must be installed separately)
  - `news-side.html`
  - `news-top.html`
  - `news-xmlui.xml`
  - `workflow.xml` (REPLACED BY `./spring/api/workflow.xml`)
  - `xmlui.xconf / dtd`
  - `emails/bte_*` (BTE import framework was removed in favor of Live Import from external sources)
  - `modules/controlpanel.cfg`
  - `modules/elastic-search-statistics.cfg` (Elastic Search support was removed in favor of Solr)
  - `modules/fetchccdata.cfg`
  - `modules/publication-lookup.cfg`
  - `spring/api/bte.xml` (BTE import framework was removed in favor of Live Import from external sources)
  - `spring/oai/*` (OAI is now part of the backend "server webapp" and needs no separate configurations)
  - `spring/xmlui/*`
- Within the `dspace.cfg` main configuration file, the following **settings were removed:**
  - `log.init.config` (replaced by `log4j2.xml`)
  - `webui.submit.blocktheses`
  - `webui.submit.upload.html5`
  - `webui.submission.restrictstep.enableAdvancedForm`
  - `webui.submission.restrictstep.groups`
  - `webui.submit.enable-cc`
  - `webui.browse.thumbnail.*`
  - `webui.item.thumbnail.*`
  - `webui.preview.enabled`
  - `webui.strengths.show`
  - `webui.browse.author-field`
  - `webui.browse.author-limit`
  - `webui.browse.render-scientific-formulas`
  - `recent.submissions.*`
  - `webui.collectionhome.*`
  - `plugin.sequence.org.dspace.plugin.SiteHomeProcessor`
  - `plugin.sequence.org.dspace.plugin.CommunityHomeProcessor`
  - `plugin.sequence.org.dspace.plugin.CollectionHomeProcessor`
  - `plugin.sequence.org.dspace.plugin.ItemHomeProcessor`
  - `plugin.single.org.dspace.app.webui.search.SearchRequestProcessor`
  - `plugin.single.org.dspace.app.xmlui.aspect.administrative.mapper.SearchRequestProcessor`
  - `plugin.named.org.dspace.app.webui.json.JSONRequest`
  - `plugin.single.org.dspace.app.webui.util.StyleSelection`
  - `webui.bitstream.order.*`
  - `webui.itemdisplay.*`
  - `webui.resolver.*`
  - `webui.preferred.identifier`
  - `webui.identifier.*`
  - `webui.mydspace.*`
  - `webui.suggest.*`
  - `webui.controlledvocabulary.enable`
  - `webui.session.invalidate`
  - `itemmap.*`
  - `jspui.*`
  - `xmlui.*`
  - `mirage2.*`

A full list of all changes / bug fixes in 7.x is available in the Changes in 7.x section.

7.0 Acknowledgments

DSpace 7.0 was the largest release in the history of DSpace, with 1,026,797 lines of code changed and 79 unique individuals contributing to either the frontend or backend.

Major Contributing Institutions

The following institutions have been major code contributors to the DSpace 7 release (in general)

- Atmire - also hosted/maintained DSpace 7 UI demo site from 2018-2023
- 4Science - also hosted/maintained DSpace 7 REST demo site from 2018-2023
- FCT / RCAAP

Financial Contributors

We gratefully recognize the following institutions who together have generously contributed financially to support the DSpace 7 staged release program (see DSpace 7 Release Goals), and individuals who devoted time to fundraising:

- Auburn University
- Cornell University
- Pascal Becker
- Dalhousie University
- Duke University
- ETH Zurich, ETH Library
- Fraunhofer Gesellschaft
- Imperial College London
- Indiana University–Purdue University, Indianapolis
- LYRASIS
- National Library of Finland
- Beate Rajski
- Staats- und Universitätsbibliothek Hamburg – Carl von Ossietzky
- Technische Universität Berlin
- Technische Universität Hamburg (TUHH)
- The DSpace-Konsortium Deutschland
- The Helmut-Schmidt-Universität/Universität der Bundeswehr Hamburg
- The Library Code GmbH
- The Ohio State University
- Texas Digital Library
- University of Arizona
- University of Edinburgh
- University of Kansas
- University of Minnesota
- University of Missouri
- University of Toronto
- World Bank
- ZHAW

Frontend / User Interface Acknowledgments

The following 55 individuals have contributed directly to the new DSpace (Angular) User Interface in this release (*ordered by number of GitHub commits*): Giuseppe Digilio (atarix83), Kristof De Langhe (Atmire-Kristof), Lotte Hofstede (LotteHofstede), Art Lowel (artlowel), Marie Verdonck (MarieVerdonck), Julius Gruber (Flusspferd123), Yury Bondarenko (ybnd), William Welling (wwelling and wellingWilliam), Yana De Pauw (YanaDePauw), Tim Donohue (tdonohue), Alessandro Martelli (alemarte), Michael Spalti (mspalti), Jonas Van Goolen (jonas-atmire), Laura Henze (lhenze), Dániel Péter Sipos (dsipos-dev), Samuel Cambien (samuelcambien), Bruno Roemers (bruno-atmire), Matteo Perelli (sourcedump), Bram Luyten (bram-atmire), Ben Bosman (benbosman), Terry Brady (terrywbrady), Raf Ponsaerts (Raf-atmire), Danilo Di Nuzzo (ddinuzzo), Andrea Chiapparelli (andreachiapparelli), Antoine Snyers (antoine-atmire), Corrado Lombardi (corrad82-4s), Courtney Pattison (courtneypattison), Àlex Magaz Graça (rivaldi8), Chris Wilper (cwilper), Christi an Scheible (christian-scheible), Andrew Wood (AndrewZWood), Reeta Kuuskoski (reetagithub), Vítor Silvério Rodrigues (vitorsilverio), Alexander Sulfrian (AlexanderS), muiltje, José Carvalho (josekarvalho), Claudia Jürgen (cjuergen), fernandaruizm, Ivan Masar (helix84), Paulo Graça (paulo-graca), Philip Vissenaekens (PhilipVis), Nagy Akos (akoscomp), Kevin Van de Velde (KevinVdV), Sascha Szott (saschaszott), Mohamed Mohideen Abdul Rasheed (mohideen), David Cavrenne (davidatmire), Hardy Pottinger (hardyoyo), Luca Giamminonni (LucaGiamminonni), Mateus Mercer (MatMercer), Denijs Balodis (Denijsb), Pascal-Nicolas Becker (pnbecker), Mikus Zarins (MixonZ), marciofoz, Andrea Bollini (abollini), Martin Walk (MW3000).

Out of the above list, the following individuals contributed a translation of the new interface (ordered alphabetically by language): Ivan Masar (Czech), Marina Muilwijk (Dutch), Reeta Kuuskoski (Finnish), David Cavrenne (French), Claudia Jürgen and Sascha Szott (German), Nagy Akos and Transylvanian Museum Society (Hungarian), Mikus Zarins (Latvian), Vítor Silvério Rodrigues and marciofoz (Brazilian Portuguese), José Carvalho (Portuguese) and Maria Fernanda Ruiz (Spanish).

The above contributor lists were determined based on historical contributions to the "dspace-angular" project in GitHub until 7.0: https://github.com/DSpace/dspace-angular/graphs/contributors?from=2016-11-27&to=2021-07-29&type=c

Backend / REST API Acknowledgments

The following 55 individuals have contributed directly to the DSpace backend (REST API, Java API, OAI-PMH, etc) in this release (*ordered by number of GitHub commits*): Raf Ponsaerts (Raf-atmire), Tim Donohue (tdonohue), Andrea Bollini (abollini), Michele Boychuk (Micheleboychuk), Mark Wood (mwoodiupui), Marie Verdonck (MarieVerdonck), Ben Bosman (benbosman), Luigi Andrea Pascarelli (lap82), Terry Brady (terrywbrady), Tom Desair (tomdesair), Yana De Pauw (YanaDePauw), Chris Wilper (cwilper), Peter Nijs (peter-atmire), Kevin Van de Velde (KevinVdV), Bruno Roemers (bruno-atmire), Giuseppe Digilio (atarix83), Pasquale Cavallo (pasqualecvl), Jelle Pelgrims (jpelgrims-atmire), Andrew Wood (AndrewZWood), Samuel Cambien (samuelcambien), Antoine Snyers (antoine-atmire), Kim Shepherd (kshepherd), Yury Bondarenko (ybnd), Michael Spalti (mspalti), Alessandro Martelli (alemarte), Oliver Goldschmidt (olli-gold), Jonas Van Goolen (jonas-atmire), Kristof De Langhe (Atmire-Kristof), Alexander Sulfrian (AlexanderS), Patrick Trottier (PTrottier), Pablo Prieto (ppmdo), Hardy Pottinger (hardyoyo), Pascal-Nicolas Becker (pnbecker), William Tantzen (tantz001), Paulo Graça (paulo-graca), Luca Giamminonni (LucaGiamminonni), Ivan Masar (helix84), Hrafn Malmquist (J4bbi), Ian Little (ilittle-cnri), Anis Moubarik (anis-moubarik), Claudia Jürgen (cjuergen), Alan Orth (alanorth), xuejiangtao, Danilo Di Nuzzo (ddinuzzo), James Creel (jcreel), Marsa Haoua (marsaoua), Philip Vissenaekens (PhilipVis), Miika Nurminen (minurmin), Bram Luyten (bram-atmire), Christian Scheible (christian-scheible), Nicholas Woodward (nwoodward), József Marton (jmarton), Mohamed Mohideen Abdul Rasheed (mohideen), Saiful Amin (saiful-semantic), Àlex Magaz Graça (rivaldi8)

The above contributor list was determined based on contributions to the "DSpace" project in GitHub between 6.0 (after Oct 24, 2016) and 7.0: https://github.com/DSpace/DSpace/graphs/contributors?from=2016-10-24&to=2021-07-29&type=c Therefore this list may include individuals who contributed to later 6.x releases, but *only if their bug fix was also applied to 7.0*.

Additional Thanks

Additional thanks to our DSpace Leadership Group and DSpace Steering Group for their ongoing DSpace support and advice. Thanks also to LYRASIS for your leadership, collaboration & support in helping to speed up the development process of DSpace 7.

Thanks also to the various developer & community Working Groups who have worked diligently to help make DSpace 7 a reality. These include:

- DSpace 7 Working Group (2016-2023) - This is the team behind the code
- DSpace 7 Entities Working Group (2018-19) - This team designed & implemented Configurable Entities
- DSpace 7 Marketing Working Group (2016-2020) - This team did all our DSpace 7marketing, press releases & announcements.
- DSpace Community Advisory Team (DCAT) - This team helped organize/lead the DSpace 7.0 Testathon (to bang on the system to find any last bugs), and they also provided us with advice on features, etc.

We apologize to any contributor accidentally left off this list. DSpace has such a large, active development community that we sometimes lose track of all our contributors. Acknowledgments to those left off will be made in future releases.

7.0 Beta 1-5 Release Notes

DSpace 7.0 was developed via a series of Beta releases from 2020-21. The release notes for each Beta are retained here for reference.

7.0 Beta 5 Release Notes

Released April 2021

Included in Beta 5

- **Support for custom theme(s) in UI & accessibility cleanup of base theme**. See early information at DSpace UI Design principles and guidelines and the "themes" section of the environment.common.ts
  - Updated the "base" theme (default Bootstrap look & feel) for consistency and better accessibility. (Additional accessibility analysis will be performed during Testathon)
  - Added a simple "dspace" theme (this is the new default theme, and primarily shows an example of customizing color scheme & homepage)
  - Added a "custom" theme folder with all necessary files. These files can be directly modified to create a completely custom theme.
- **Major performance improvements to UI** by making better use of caching & smart reloading
- **Video/Image Content Streaming** *(Kindly donated by Zoltán Kanász-Nagy and Dániel Péter Sipos of Qulto):* When enabled, DSpace can now stream videos & view images full screen, using an embedded viewer.
  - See the new "mediaViewer" settings in the environment.common.ts to enable. Sample screenshots of the feature can also be found at https://github.com/DSpace/dspace-angular/issues/885
- New Administrative Features
  - Add ability to modify Community/Collection resource policies (i.e. permissions). Edit a Community or Collection and look at the "Authorizations" tab.
  - Add ability to edit/delete user Groups.
  - Add private/withdrawn item badges for Administrators to quickly see which Items are private or withdrawn. These are viewable throughout the browse/search when logged in as an Administrative user.
- Configurable Entities Improvements
  - Entities now report their Entity type in the URL path (e.g. Person entities use URL path /entities/person/[uuid] and Publication entities use the URL path /entities/publication/[uuid])
  - Each Entity type now has a custom Submission form.
    - These can be most easily seen in the Demo site. Submitting to the "People" collection uses the "Person" Entity Form. Submitting to the "Articles" collection uses the "Publication" Entity Form. The full list of Entity-specific Collection submission mappings can be found in the example in item-submission.xml (this example is enabled on our Demo Site)
  - General performance improvements for Entities. Introduction of "tilted" relationships for Configurable Entities that may have hundreds or thousands of relationships.
- Improvements to Upgrade process
  - Added a new Submission form migration script to help DSpace 5/6 institutions migrate their old Submission configuration files to the new /updated format for v7.
- Security fixes
  - Added CSRF (Cross-Site Request Forgery) protection to REST API. UI (and any other clients) now must be *trusted* to login to the REST API.
  - Improved permissions checks/validation in UI for Administrator, Community/Collection Administrator and Submitter roles.
  - Fixed several other security issues auto-reported by LGTM
- Many bug fixes
  - Fixed issue where mapped items were not appearing
  - Fixed issue where Handles were not redirecting
  - Fixed issues with Sherpa and ORCID integrations
  - Fixed several small issues with OpenAIRE v4 support in OAI-PMH
  - Fixed many bugs in MyDSpace and Submission UI
  - Fixed several bugs in CSV import/export process.
  - Fixes to search/browse pagination & breadcrumb trail
  - Improved performance of Browse by Community/Collection hierarchy
  - LDAP Authentication support is working again
- Many dependency upgrades

    - Upgrade UI to Angular v10
    - Upgrade UI to Node v12 or v14 support
    - Upgrade Backend to Solr v8 support
    - Upgrade to ORCID v3 support
    - Upgrade to SHERPA v2 support
- Removal of obsolete features
    - Removal of the old BTE framework in favor of Live Import Framework (features of BTE have been ported to Live Import)
    - Removal of Traditional/Basic workflow in favor of Configurable Workflow (default workflow is still the same as in DSpace 6)

Changelog

- All User Interface changes: https://github.com/DSpace/dspace-angular/issues?q=is%3Aclosed+milestone%3A7.0beta5
- All Backend changes: https://github.com/DSpace/DSpace/issues?q=is%3Aclosed+milestone%3A7.0beta5

7.0 Beta 4 Release Notes

Released October 2020

Included in Beta 4

- **Live Import framework** (video) support has been added to the Submission Form (and REST API `/api/integration/externalsources` endpoint)
    - Search an external site for works to import (From your MyDSpace page, click the "Import metadata from external source" button in upper right). Currently supports Library of Congress Names, ORCID, PubMed, Sherpa Journals or Sherpa Publishers.
    - Drag and drop a bibliographic file into Submission form or MyDSpace page to prepopulate metadata. Supported formats include ArXiv, CSV (or TSV), Endnote, PubMed, or RIS.
- **Controlled Vocabulary support** (video) in Submission Form. Depending on the field configuration, this can include autocomplete of known terms (see default "Subject Keywords" field), dropdown support (see default "Type" field) and hierarchical tree views
    - Includes support for Controlled Vocabs, Authority Control and "Value-Pairs" (from submission configs)
- **Curation Tasks** are now supported via the Admin UI and the Processes UI. (Login as an Admin, select "Curation Tasks")
- **Import / Export metadata from/to CSV** (i.e. Batch Metadata Editing) is now available from the Admin UI. (Login as an Admin, select "Export" > "Metadata", select "Import" > "Metadata")
- **Basic Usage Statistics** (video) are available for the entire site (See "Statistics" menu at top of homepage), or specific Communities, Collections or Items (Click on that same "Statistics" menu after browsing to a specific object).
    - Support for exchanging usage data to IRUS was added. See new "irus-statistics.cfg" and DS-626
- Improved GDPR Alignment (video)
    - **User Agreement** required for all authenticated users to read and agree to. (Login for first time, and sample user agreement will display. After agreeing to it, it will not appear again.)
    - **Cookie Preferences** are now available for all users (anonymous or authenticated). A cookie preference popup appears when first accessing the site. Users are given information on what cookies added by DSpace, including a **Privacy Statement** which can be used to describe how their data is used.
    - User Accounts can be deleted *even if they've submitted content* in the past.
        - When a user is deleted, their past submissions are kept but the submitter field is set to empty (null).
        - Users cannot be deleted if they are the only member of a workflow approval group. Admins must either delete that group first, or assign another member to the group. This ensures Workflows are kept even if a user account needs to be deleted.
- **Language preferences** are now kept for all users (anonymous or logged in). By default, DSpace will try to use your browser's preferred language (if found in Accept-Language header and a translation in that language exists). Users can override it by either saving a preferred language in their user profile, or by manually selecting a different language from the globe icon (upper right).
- **IP-based authorization** lets you restrict (or provide access to) objects based on the user's IP address. This uses the same "authentication-ip.cfg" configuration as DSpace 6, allowing you to map IP ranges to specific DSpace Groups. Users within that IP range are added to the mapped DSpace Group for the remainder of their session.
- **Search Engine Optimization**: Addition of robots.txt, Sitemaps and Google Scholar "citation" tags. These optimizations are being tested by the Google Scholar team and may be improved further in the upcoming beta 5 release.
    - For improved SEO, Sitemaps are now enabled by default and automatically update once per day.
- Security Fixes and Dependency upgrades
    - Enhancements to new `/api/authz/features` endpoint in REST API to provide additional feature-specific permission checks
    - Flyway database engine was upgraded to version 6.5.5
    - Indexing enhancements (some objects were being indexed twice, see PR#2960)
    - Fixes to Shibboleth login
    - Additional bug fixes to both UI and REST API

Changelog

- All User Interface changes: https://github.com/DSpace/dspace-angular/issues?q=is%3Aclosed+milestone%3A7.0beta4
- All Backend changes: https://github.com/DSpace/DSpace/issues?q=is%3Aclosed+milestone%3A7.0beta4

7.0 Beta 3 Release Notes

Released July 2020

Included in Beta 3

- **Processes Admin UI** (video) allows Administrators to run backend scripts/processes while monitoring their progress & completion. (Login as an Admin, select "Processes" in sidebar)
    - Currently supported processes include "index-discovery" (reindex site), "metadata-export" (batch metadata editing CSV export), and "metadata-import" (batch metadata editing CSV import).
- **Manage Account Profile** allows logged in users to update their name, language or password. (Login, click on the account icon, and select "Profile")

- **New User Registration** (video) and password reset on the Login Screen
- **Login As (Impersonate)** another account allows Administrators to debug issues that a specific user is seeing, or do some work on behalf of that user. (Login as an Admin, Click "Access Control" in sidebar, Click "People". Search for the user account & edit it. Click the "Impersonate EPerson" button. You will be authenticated as that user until you click "Stop Impersonating EPerson" in the upper right.)
    - Requires "webui.user.assumelogin=true" to be set in your local.cfg on backend. Also be aware that you can only "impersonate" a user who is *not* a member of the Administrator group.
- **Manage Authorization Policies** of an Item allows Administrators to directly change/update the access policies of an Item, its Bundles or Bitstreams. (Login as an Admin, Click "Edit" "Item" in sidebar, and search for the Item. Click the "Authorization.." button on its "Status" tab.
- **Manage Item Templates** of a Collection allows Administrators to create/manage template metadata that all new Items will start with when submitted to that Collection. (Login as an Admin, Click "Edit" "Collection" in sidebar and search for the Collection. Click the "Add" button under "Template Item" to get started.)
    - NOTE: unfortunately there's a known bug that while you can create these templates, the submission process is not yet using them. See https://github.com/DSpace/dspace-angular/issues/748
- **Administer Active Workflows** (video) allows Administrators to see every submission that is currently in the workflow approval process. From there, they have the option to delete Items (if they are no longer needed), or send them back to the workflow pool (to allow another user to review them). (Login as an Admin, Click "Administer Workflow" in sidebar)
- **CC License** step allows your users to select a Creative Commons License as part of their submission. Once enabled in the "item-submission.xml" (on the backend) it appears as part of the submission form.
- **Angular CLI** compatibility was added to the User Interface. This allows developers to easily update the User Interface using standard Angular commandline tools. More information (including tutorials) is available at https://cli.angular.io/
- English, Latvian, Dutch, German, French, Portuguese, Spanish and Finnish language catalogs
- Numerous bugs were fixed based on early user testing. (*Thanks to all who've tested Beta 1 or Beta 2 and reported your feedback!*) Some bugs fixed include:
    - Login/Logout session fixes (including compatibility with Firefox and Safari browsers)
    - Improved Community/Collection tree browsing performance
    - Fixes to editing Communities, Collections and Items. This includes improved drag & drop reordering of bitstreams in an Item.
    - Improved performance of Collection dropdown in submission
    - Ability to download restricted bitstreams (previously these would error out)
    - Authorization & security improvements in both REST API and UI
- Upgraded all REST API dependencies (Spring, Spring Boot, HAL Browser) and enhanced our automated testing via additional Integration Tests.
- *All features previous mentioned in 7.0 Beta 2 Release Notes and 7.0 Beta 1 Release Notes below*

Learn More: New videos are available highlighting features of the MyDSpace area:

- Manage Submissions in MyDSpace (video)
- Manage Tasks in MyDSpace (video)

Changelog

- All User Interface changes: https://github.com/DSpace/dspace-angular/issues?q=is%3Aclosed+milestone%3A7.0beta3
- All Backend changes: https://github.com/DSpace/DSpace/issues?q=is%3Aclosed+milestone%3A7.0beta3

7.0 Beta 2 Release Notes

Released April 2020

Included in Beta 2

- **Administrative Search** (video) combines retrieval of withdrawn items and private items, together with a series of quick action buttons.
- **EPeople, Groups and Roles** can now be viewed, created and updated.
    - Manage Groups (Login as an Admin Access Control Groups)
    - Manage EPeople (Login as an Admin Access Control EPeople)
    - Manage Community/Collection Roles (Login as an Admin Edit Community/Collection Assign Roles). Note: this feature is Admin-only in beta 2, but will be extended to Community/Collection Admins in beta 3.
- **Bitstream Editing** (video) has a drag-and-drop interface for re-ordering bitstreams and makes adding and editing bitstreams more intuitive.
- **Metadata Editing** (video) introduces suggest-as-you-type for field name selection of new metadata.
- **Update Profile / Change Password** (Login Select user menu in upper right Profile)
- **Shibboleth Authentication**
- **Viewing Item Version History** (requires upgrading from a 6.x site that includes Item Versioning)
- Collection and Community (video) creation and edit pages.
- English, Latvian, Dutch, German, French, Portuguese and Spanish language catalogs
- Security and authorization improvements, including REST API support hiding specific metadata fields (metadata.hide property) and upgrades of different software packages on which DSpace 7 depends.
- *All features previous mentioned in 7.0 Beta 1 Release Notes below*

A full list of all changes / bug fixes in 7.x is available in the Changes in 7.x section.

7.0 Beta 1 Release Notes

Released March 2020

New features to look for

- **A completely new User Interface (demo site).** This is the new Javascript-based frontend, built on Angular.io (with support for SEO provided by Angular Universal). This new interface is also via HTML and CSS (SCSS). For early theme building training, see the “Getting Started with DSpace 7 Workshop” from the North American User Group meeting: slides or video recording.

- **A completely new, fully featured REST API (demo site)**, provided via a single "server" webapp backend. This new backend is not only a REST API, but also still supports OAI-PMH, SWORD (v1 or v2) and RDF. See the REST API's documentation / contract at https://github.com/DSpace/Rest7Contract/blob/master/README.md
- **A newly designed search box.** Search from the header of any page (click the magnifying glass). The search results page now features automatic search highlight, expandable & searchable filters, and optional thumbnail-based results (click on the “grid” view).
- **A new MyDSpace area**, including a new, one-page, drag & drop submission form, a new workflow approval process, and searchable past submissions. (Login, click on your user profile icon, click “MyDSpace”). Find workflow tasks to claim by selecting “All tasks” in the “Show” dropdown.
- **Dynamic user interface translations** (Click the globe, and select a language). Anyone interested in adding more translations? See DSpace 7 Translation - Internationalization (i18n) - Localization (l10n).
- **A new Admin sidebar.** Login as an Administrator, and an administrative sidebar appears. Use this to create a new Community/Collection/Item, edit existing ones, and manage registries. (*NOTE: A number of Administrative tools are still missing or greyed out. They will be coming in future Beta releases.*)
- Optional, new **Configurable Entities feature**. DSpace now supports “entities”, which are DSpace Items of a specific ‘type’ which may have relationships to other entities. These entity types and relationships are configurable, with two examples coming out-of-the-box: a set of Journal hierarchy entities (Journal, Volume, Issue, Publication) and a set of Research entities (Publication, Project, Person, OrgUnit). For more information see “The Power of Configurable Entities” from OR2019: slides or video recording. Additionally, a test data set featuring both out-of-the-box examples can be used when trying out DSpace 7 via Docker. Early documentation is available at Configurable Entities.
- **Support for OpenAIREv4** Guidelines for Literature Repositories in OAI-PMH (See the new “openaire4” context in OAI-PMH).

Additional major changes to be aware of in the 7.x platform (not an exhaustive list):

- **XMLUI and JSPUI are no longer supported or distributed with DSpace.** All users should immediately migrate to and utilize the new Angular User Interface. There is no migration path from either the XMLUI or JSPUI to the new User interface. However, the new user interface can be themed via HTML and CSS (SCSS).
- **The old REST API ("rest" webapp from DSpace v4.x-6.x) is deprecated and will be removed in v8.x.** The new REST API (provided in the "server" webapp) replaces all functionality available in the older REST API. If you have tools that rely on the old REST API, you can still (optionally) build & deploy it alongside the "server" webapp via the "-Pdspace-rest" Maven flag.
- **The Submission Form configuration has changed.** The "item-submission.xml" file has changed its structure, and the "input-forms.xml" has been replaced by a "submission-forms.xml". For early documentation see Configuration changes in the submission process
- **ElasticSearch Usage Statistics have been removed.** Please use SOLR Statistics or DSpace Google Analytics Statistics.
- **The traditional, 3-step Workflow system has been removed in favor of the Configurable Workflow System.** For most users, you should see no effect or difference. The default setup for this Configurable Workflow System is identical to the traditional, 3-step workflow ("Approve/Reject", "Approve/Reject/Edit Metadata", "Edit Metadata")
- **Apache Solr is no longer embedded within the DSpace installer (and has been upgraded to Solr v7).** Solr now MUST be installed as a separate dependency alongside the DSpace backend. See Installing DSpace.
- **Some command-line tools/scripts are enabled in the new REST API** (e.g. index-discovery): See new Scripts endpoint: https://github.com/DSpace/Rest7Contract/blob/master/scripts-endpoint.md
- **DSpace now has a single, backend "server" webapp to deploy in Tomcat (or similar).** In DSpace 6.x and below, different machine interfaces (OAI-PMH, SWORD v1 or v2, RDF, REST API) were provided via separate deployable webapps. Now, all those interfaces along with the new REST API are in a single, "server" webapp built on Spring Boot. You can now control which interfaces are enabled, and what path they appear on via configuration (e.g. "oai.enabled=true" and "oai.path=oai"). See https://jira.lyrasis.org/browse/DS-4257
- **Configuration has been upgraded to Apache Commons Configuration version 2.** For most users, you should see no effect or difference. *No DSpace configuration files were modified during this upgrade and no configurations or settings were renamed or changed.* However, if you locally modified or customized the `[dspace]/config/config-definition.xml` (DSpace's Apache Commons Configuration settings), you will need to ensure those modifications are compatible with Apache Commons Configuration version 2. See the Apache Commons Configuration's configuration definition file reference for more details.
- **Handle Server has been upgraded to version 9.x :** https://jira.lyrasis.org/browse/DS-4205
- **DSpace now has sample Docker images (configurations)** which can be used to try out DSpace quickly. See Try out DSpace 7 ("Install via Docker" section).

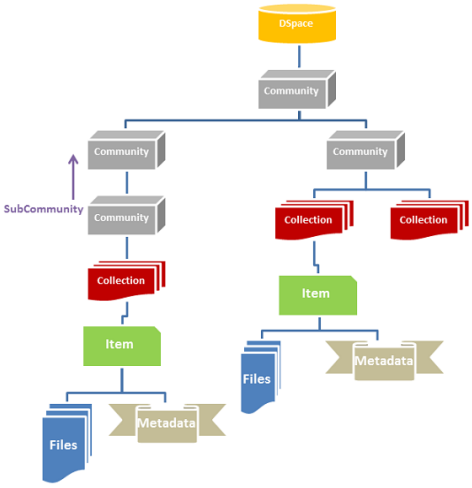
Functional Overview

The following sections describe the various functional aspects of the DSpace system.

1 Online access to your digital assets
    1.1 Full-text search
    1.2 Navigation
    1.3 Supported file types
    1.4 Optimized for Google Indexing
    1.5 OpenURL Support
    1.6 Support for modern browsers
2 Metadata Management
    2.1 Metadata
    2.2 Choice Management and Authority Control
3 Licensing
    3.1 Collection and Community Licenses
    3.2 License granted by the submitter to the repository
    3.3 Creative Commons Support for DSpace Items
4 Persistent URLs and Identifiers
    4.1 Handles
    4.2 Bitstream 'Persistent' Identifiers
5 Getting content into DSpace
    5.1 The Manual DSpace Submission and Workflow System
        5.1.1 Workflow Steps
        5.1.2 Submission Workflow in DSpace
    5.2 Command line import facilities
    5.3 Registration for externally hosted files
    5.4 SWORD Support
6 Getting content out of DSpace
    6.1 OAI Support
    6.2 Signposting
    6.3 Command Line Export Facilities
    6.4 Packager Plugins
    6.5 Crosswalk Plugins
    6.6 Supervision and Collaboration
7 User Management
    7.1 User Accounts (E-Person)
    7.2 Subscriptions
    7.3 Groups
8 Access Control
    8.1 Authentication
    8.2 Authorization
9 Usage Metrics
    9.1 Item, Collection and Community Usage Statistics
    9.2 System Statistics
10 Digital Preservation
    10.1 Checksum Checker
11 System Design
    11.1 Data Model
    11.2 Amazon S3 Support

Online access to your digital assets

The online presentation of your content in an organized tree of Communities and Collections is a main feature of DSpace. Users can access pages for individual items, these are metadata descriptions together with files available for download. The structure is summarised in this diagram (click to see the image at full size).



Full-text search

DSpace can process uploaded text based contents for full-text searching. This means that not only the metadata you provide for a given file will be searchable, but all of its contents will be indexed as well. This allows users to search for specific keywords that only appear in the actual content and not in the provided description.

Navigation

DSpace allows users to find their way to relevant content in a number of ways, including:

- **Searching** for one or more keywords in metadata or extracted full-text
- **Faceted browsing** through any field provided in the item description.
- Through **external reference**, such as a Handle
- By clicking on Community and Collection titles to explore their contents

Another important mechanism for discovery in DSpace is the browse. This is the process whereby the user views a particular index, such as the title index, and navigates around it in search of interesting items. The browse subsystem provides a simple API for achieving this by allowing a caller to specify an index, and a subsection of that index. The browse subsystem then discloses the portion of the index of interest. Indices that may be browsed are item title, item issue date, item author, and subject terms. Additionally, the browse can be limited to items within a particular collection or community.

For more information on Search/Browse functionality in DSpace, see Discovery.

Supported file types

DSpace can accommodate any type of uploaded file. While DSpace is most known for hosting text based materials including scholarly communication and electronic theses and dissertations (ETDs), there are many stakeholders in the community who use DSpace for multimedia, data and learning objects. While some restrictions apply, DSpace can even serve as a store for HTML Archives.

Files that have been uploaded to DSpace are often referred to as "Bitstreams". The reason for this is mainly historic and tracks back to the technical implementation. After ingestion, files in DSpace are stored on the file system as a stream of bits without the file extension.

By default, DSpace only recognizes specific file types, as defined in its Bitstream Format Registry. The default Bitstream Format Registry recognizes many common file formats, but it can be enhanced at your local institution via the Admin User Interface.

Optimized for Google Indexing

The Duraspace community fosters a close relation with Google to ensure optimal indexing of DSpace content, primarily in the Google Search and Google Scholar products. For the purpose of Google Scholar indexing, DSpace added specific metadata in the page head tags facilitating indexing in Scholar. More information can be retrieved on the Google Scholar Metadata Mappings page. Popular DSpace repositories often generate over 60% of their visits from Google pages.

OpenURL Support

DSpace supports the OpenURL protocol in a rather simple fashion. If your institution has an SFX server, DSpace will display an OpenURL link on every item page, automatically using the Dublin Core metadata. Additionally, DSpace can respond to incoming OpenURLs. Presently it simply passes the information in the OpenURL to the search subsystem. A list of results is then displayed, which usually gives the relevant item (if it is in DSpace) at the top of the list.

Support for modern browsers

The DSpace developer community aims to rely on modern web standards and well tested libraries where possible. As a rule of thumb, users can expect that the DSpace web interfaces work on modern web browsers. DSpace developers routinely test new interface developments on recent versions of Firefox, Safari, Chrome and Microsoft Edge. Because of fast moving, automatic, incremental updates to these browsers, support is no longer targeted at specific versions of these browsers. (Please note that we do not recommend or support using Internet Explorer as it is considered "end of life" by Microsoft.)

Metadata Management

Metadata

Broadly speaking, DSpace holds three sorts of metadata about archived content:

- **Descriptive Metadata**: DSpace can support multiple flat metadata schemas for describing an item. A qualified Dublin Core metadata schema loosely based on the Library Application Profile set of elements and qualifiers is provided by default. This default schema is described in more detail in Metadata and Bitstream Format Registries. However, you can configure multiple schemas and select metadata fields from a mix of configured schemas to describe your items. Other descriptive metadata about items (e.g. metadata described in a hierarchical schema) may be held in serialized bitstreams.
- **Administrative Metadata**: This includes preservation metadata, provenance and authorization policy data. Most of this is held within DSpace's relational DBMS schema. Provenance metadata (prose) is stored in Dublin Core records. Additionally, some other administrative metadata (for example, bitstream byte sizes and MIME types) is replicated in Dublin Core records so that it is easily accessible outside of DSpace.
- **Structural Metadata**: This includes information about how to present an item, or bitstreams within an item, to an end-user, and the relationships between constituent parts of the item. As an example, consider a thesis consisting of a number of TIFF images, each depicting a single page of the thesis. Structural metadata would include the fact that each image is a single page, and the ordering of the TIFF images/pages. Structural metadata in DSpace is currently fairly basic; within an item, bitstreams can be arranged into separate bundles as described above. A bundle may also optionally have a *primary bitstream*. This is currently used by the HTML support to indicate which bitstream in the bundle is the first HTML file to send to a browser. In addition to some basic technical metadata, a bitstream also has a 'sequence ID' that uniquely identifies it within an item. This is used to produce a 'persistent' bitstream identifier for each bitstream. Additional structural metadata can be stored in serialized bitstreams, but DSpace does not currently understand this natively.

Choice Management and Authority Control

This is a configurable framework that lets you define plug-in classes to control the choice of values for specified DSpace metadata fields. It also lets you configure fields to include "authority" values along with the textual metadata value. The choice-control system includes a user interface in both the Configurable Submission UI and the Admin UI (edit Item pages) that assists the user in choosing metadata values.

Introduction and Motivation

Definitions

Choice Management

This is a mechanism that generates a list of choices for a value to be entered in a given metadata field. Depending on your implementation, the exact choice list might be determined by a proposed value or query, or it could be a fixed list that is the same for every query. It may also be closed (limited to choices produced internally) or open, allowing the user-supplied query to be included as a choice.

Authority Control

This works in addition to choice management to supply an authority key along with the chosen value, which is also assigned to the Item's metadata field entry. Any authority-controlled field is also inherently choice-controlled.

About Authority Control

The advantages we seek from an authority controlled metadata field are:

1. **There is a simple and positive way to test whether two values are identical**, by comparing authority keys.
    - Comparing plain text values can give false positive results e.g. when two different people have a name that is written the same.
    - It can also give false negative results when the same name is written different ways, e.g. "J. Smith" vs. "John Smith".
2. **Help in entering correct metadata values.** The submission and admin UIs may call on the authority to check a proposed value and list possible matches to help the user select one.
3. **Improved interoperability.** By sharing a name authority with another application, your DSpace can interoperate more cleanly with other applications.
    - For example, a DSpace institutional repository sharing a naming authority with the campus social network would let the social network construct a list of all DSpace Items matching the shared author identifier, rather than by error-prone name matching.
    - When the name authority is shared with a campus directory, DSpace can look up the email address of an author to send automatic email about works of theirs submitted by a third party. That author does not have to be an EPerson.
4. Authority keys are normally invisible in the public web UIs. They are only seen by administrators editing metadata. The value of an authority key is not expected to be meaningful to an end-user or site visitor.

Authority control is different from the controlled vocabulary of keywords already implemented in the submission UI:

1. **Authorities are external to DSpace.** The source of authority control is typically an external database or network resource.
    - Plug-in architecture makes it easy to integrate new authorities without modifying any core code.
2. This authority proposal impacts all phases of metadata management.
    - The keyword vocabularies are only for the submission UI.

- Authority control is asserted everywhere metadata values are changed, including unattended/batch submission, SWORD package submission, and the administrative UI.

Some Terminology

Authority	An authority is a source of fixed values for a given domain, each unique value identified by a key.
.	For example, the OCLC LC Name Authority Service.
Authority Record	The information associated with one of the values in an authority; may include alternate spellings and equivalent forms of the value, etc.
Authority Key	An opaque, hopefully persistent, identifier corresponding to exactly one record in the authority.

Licensing

DSpace offers support for licenses on different levels

Collection and Community Licenses

Each community and collection in the hierarchy of a DSpace repository can contain its own license terms. This allows an institution to use the repository both for collections where certain rights are reserved and others from which the content may be accessed and distributed more freely.

License granted by the submitter to the repository

At the end of the manual submission process, the submitter is asked to grant the repository service an appropriate distribution license. This license can be easily customized on a per collection basis. In its most common form, the submitter grants to the repository service a non-exclusive distribution license, meaning that he officially gives the repository service the right to share his or her work with the world.

Creative Commons Support for DSpace Items

DSpace provides support for Creative Commons licenses to be attached to items in the repository. They represent an alternative to traditional copyright. To learn more about Creative Commons, visit their website. Support for license selection is controlled by a site-wide configuration option, and since license selection involves interaction with the Creative Commons website, additional parameters may be configured to work with a proxy server. If the option is enabled, users may select a Creative Commons license during the submission process, or select to don't assign a Creative Commons license at all. If a selection is made, metadata and a copy of the license in the RDF format is stored along with the item in the repository. There is also an indication - text and a Creative Commons icon - in the item display page of the web user interface when an item is licensed under Creative Commons. The RDF license is embedded in the html page of the item to allow machine understanding of the licensing terms. For specifics of how to configure and use Creative Commons licenses, see the configuration section.

Persistent URLs and Identifiers

Handles

Researchers require a stable point of reference for their works. The simple evolution from sharing of citations to emailing of URLs broke when Web users learned that sites can disappear or be reconfigured without notice, and that their bookmark files containing critical links to research results couldn't be trusted in the long term. To help solve this problem, a core DSpace feature is the creation of a persistent identifier for every item, collection and community stored in DSpace. To persist identifiers, DSpace requires a storage- and location- independent mechanism for creating and maintaining identifiers. DSpace uses the CNRI Handle System for creating these identifiers. The rest of this section assumes a basic familiarity with the Handle system.

DSpace uses Handles primarily as a means of assigning globally unique identifiers to objects. Each site running DSpace needs to obtain a unique Handle 'prefix' from CNRI, so we know that if we create identifiers with that prefix, they won't clash with identifiers created elsewhere.

Presently, Handles are assigned to communities, collections, and items. Bundles and bitstreams are not assigned Handles, since over time, the way in which an item is encoded as bits may change, in order to allow access with future technologies and devices. Older versions may be moved to off-line storage as a new standard becomes de facto. Since it's usually the *item* that is being preserved, rather than the particular bit encoding, it only makes sense to persistently identify and allow access to the item, and allow users to access the appropriate bit encoding from there.

Of course, it may be that a particular bit encoding of a file is explicitly being preserved; in this case, the bitstream could be the only one in the item, and the item's Handle would then essentially refer just to that bitstream. The same bitstream can also be included in other items, and thus would be citable as part of a greater item, or individually.

The Handle system also features a global resolution infrastructure; that is, an end-user can enter a Handle into any service (e.g. Web page) that can resolve Handles, and the end-user will be directed to the object (in the case of DSpace, community, collection or item) identified by that Handle. In order to take advantage of this feature of the Handle system, a DSpace site must also run a 'Handle server' that can accept and resolve incoming resolution requests. All the code for this is included in the DSpace source code bundle.

Handles can be written in two forms:

```
hdl:1721.123/4567
http://hdl.handle.net/1721.123/4567
```

The above represent the same Handle. The first is possibly more convenient to use only as an identifier; however, by using the second form, any Web browser becomes capable of resolving Handles. An end-user need only access this form of the Handle as they would any other URL. It is possible to enable some browsers to resolve the first form of Handle as if they were standard URLs using CNRI's Handle Resolver plug-in, but since the first form can always be simply derived from the second, DSpace displays Handles in the second form, so that it is more useful for end-users.

It is important to note that DSpace uses the CNRI Handle infrastructure only at the 'site' level. For example, in the above example, the DSpace site has been assigned the prefix '1721.123'. It is still the responsibility of the DSpace site to maintain the association between a full Handle (including the '4567' local part) and the community, collection or item in question.

Bitstream 'Persistent' Identifiers

Similar to handles for DSpace items, bitstreams also have 'Persistent' identifiers. They are more volatile than Handles, since if the content is moved to a different server or organization, they will no longer work (hence the quotes around 'persistent'). However, they are more easily persisted than the simple URLs based on database primary key previously used. This means that external systems can more reliably refer to specific bitstreams stored in a DSpace instance.

Each bitstream has a sequence ID, unique within an item. This sequence ID is used to create a persistent ID, of the form:

dspace url/bitstream/handle/sequence ID/filename

For example:

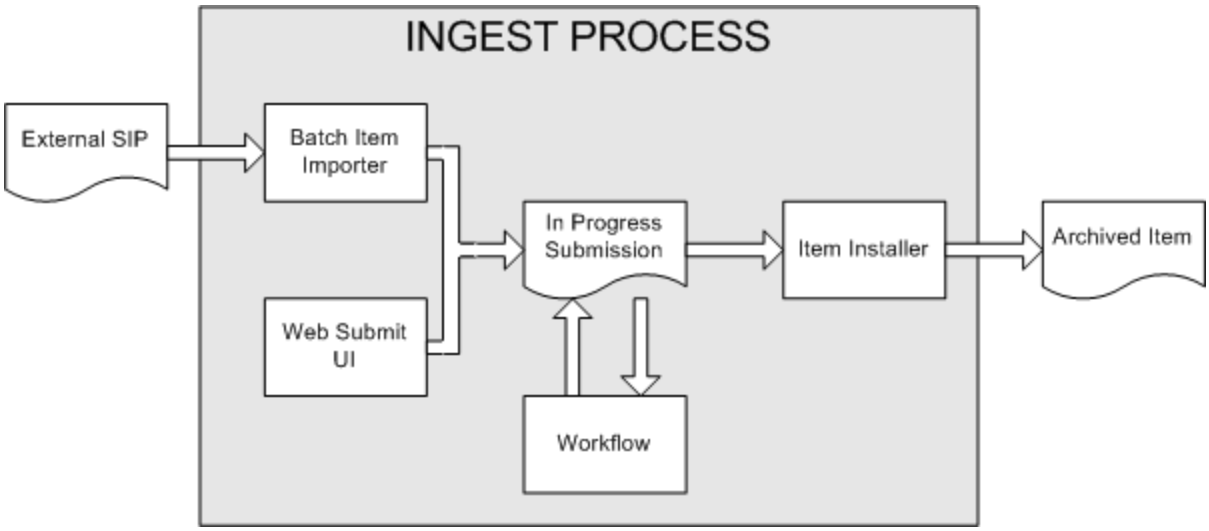
```
https://dspace.myu.edu/bitstream/123.456/789/24/foo.html
```

The above refers to the bitstream with sequence ID 24 in the item with the Handle *hdl:123.456/789*. The *foo.html* is really just there as a hint to browsers: Although DSpace will provide the appropriate MIME type, some browsers only function correctly if the file has an expected extension.

Getting content into DSpace

The Manual DSpace Submission and Workflow System

Rather than being a single subsystem, ingesting is a process that spans several. Below is a simple illustration of the current ingesting process in DSpace.



DSpace Ingest Process

The batch item importer is an application, which turns an external SIP (an XML metadata document with some content files) into an "in progress submission" object. The Web submission UI is similarly used by an end-user to assemble an "in progress submission" object.

Depending on the policy of the collection to which the submission in targeted, a workflow process may be started. This typically allows one or more human reviewers or 'gatekeepers' to check over the submission and ensure it is suitable for inclusion in the collection.

When the Batch Ingester or Submission UI completes the InProgressSubmission object, and invokes the next stage of ingest (be that workflow or item installation), a provenance message is added to the Dublin Core which includes the filenames and checksums of the content of the submission. Likewise, each time a workflow changes state (e.g. a reviewer accepts the submission), a similar provenance statement is added. This allows us to track how the item has changed since a user submitted it.

Once any workflow process is successfully and positively completed, the InProgressSubmission object is consumed by an "item installer", that converts the InProgressSubmission into a fully blown archived item in DSpace. The item installer:

- Assigns an accession date
- Adds a "date.available" value to the Dublin Core metadata record of the item

- Adds an issue date if none already present
- Adds a provenance message (including bitstream checksums)
- Assigns a Handle persistent identifier
- Adds the item to the target collection, and adds appropriate authorization policies
- Adds the new item to the search and browse index

Workflow Steps

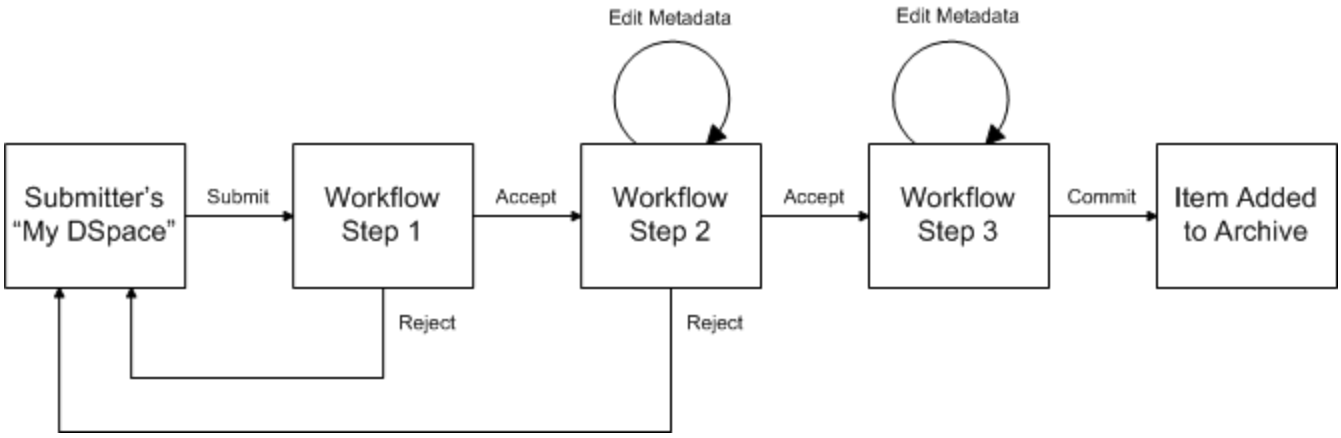
By default, a collection's workflow may have up to three steps. Each collection may have an associated e-person group for performing each step; if no group is associated with a certain step, that step is skipped. If a collection has no e-person groups associated with any step, submissions to that collection are installed straight into the main archive. Keep in mind, however, that this is only the **default** behavior, and the workflow process can be configured /customized easily, see Configurable Workflow.

In other words, the default sequence is this: The collection receives a submission. If the collection has a group assigned for workflow step 1, that step is invoked, and the group is notified. Otherwise, workflow step 1 is skipped. Likewise, workflow steps 2 and 3 are performed if and only if the collection has a group assigned to those steps.

When a step is invoked, the submission is put into the 'task pool' of the step's associated group. One member of that group takes the task from the pool, and it is then removed from the task pool, to avoid the situation where several people in the group may be performing the same task without realizing it.

The member of the group who has taken the task from the pool may then perform one of three actions:

Workflow Step	Possible actions
review (step 1)	Can accept submission for inclusion, or reject submission.
edit (step 2)	Can edit metadata provided by the user with the submission, but cannot change the submitted files. Can accept submission for inclusion, or reject submission.
finaledit (step 3)	Can edit metadata provided by the user with the submission, but cannot change the submitted files. Must then commit to archive; may not reject submission.



Submission Workflow in DSpace

If a submission is rejected, the reason (entered by the workflow participant) is e-mailed to the submitter, and it is returned to the submitter's 'My DSpace' page. The submitter can then make any necessary modifications and re-submit, whereupon the process starts again.

If a submission is 'accepted', it is passed to the next step in the workflow. If there are no more workflow steps with associated groups, the submission is installed in the main archive.

One last possibility is that a workflow can be 'aborted' by a DSpace site administrator. This is accomplished using the Administration UI.

Command line import facilities

DSpace includes batch tools to import items in a simple directory structure, where the Dublin Core metadata is stored in an XML file. This may be used as the basis for moving content between DSpace and other systems. For more information see Item Importer and Exporter.

DSpace also includes various package importer tools, which support many common content packaging formats like METS. For more information see Package Importer and Exporter. Additionally, DSpace can import/export Archival Information Packages (AIPs), see AIP Backup and Restore.

Registration for externally hosted files

Registration is an alternate means of incorporating items, their metadata, and their bitstreams into DSpace by taking advantage of the bitstreams already being in accessible computer storage. An example might be that there is a repository for existing digital assets. Rather than using the normal interactive ingest process or the batch import to furnish DSpace the metadata and to upload bitstreams, registration provides DSpace the metadata and the location of the bitstreams. DSpace uses a variation of the import tool to accomplish registration.

SWORD Support

SWORD (Simple Web-service Offering Repository Deposit) is a protocol that allows the remote deposit of items into repositories. SWORD was further developed in SWORD version 2 to add the ability to retrieve, update, or delete deposits. DSpace supports the SWORD protocol via the 'sword' web application and SWord v2 via the swordv2 web application. The specification and further information can be found at http://swordapp.org. See also SWORDv1 Server and SWORDv2 Server.

Getting content out of DSpace

OAI Support

The Open Archives Initiative has developed a protocol for metadata harvesting. This allows sites to programmatically retrieve or 'harvest' the metadata from several sources, and offer services using that metadata, such as indexing or linking services. Such a service could allow users to access information from a large number of sites from one place.

DSpace exposes the Dublin Core metadata for items that are publicly (anonymously) accessible. Additionally, the collection structure is also exposed via the OAI protocol's 'sets' mechanism. OCLC's open source OAICat framework is used to provide this functionality.

You can also configure the OAI service to make use of any crosswalk plugin to offer additional metadata formats, such as MODS.

DSpace's OAI service does support the exposing of deletion information for withdrawn items, but not for items that are 'expunged' (see above). DSpace also supports OAI-PMH resumption tokens. See OAI for more information.

Signposting

DSpace supports FAIR Signposting Profile at Level 2: By supporting the FAIR Signposting Profile at Level 2, your platform demonstrates a commitment to improving the machine accessibility, interoperability, and reusability of scholarly resources. It ensures that the information you provide is standardized, consistent, and easily navigable by both human users and machine agents, contributing to a more efficient and FAIR scholarly web ecosystem. For more information see Signposting.

Command Line Export Facilities

DSpace includes batch tools to export items in a simple directory structure, where the Dublin Core metadata is stored in an XML file. This may be used as the basis for moving content between DSpace and other systems. For more information see Item Importer and Exporter.

DSpace also includes various package exporter tools, which support many common content packaging formats like METS. For more information see Package Importer and Exporter. Additionally, DSpace can import/export Archival Information Packages (AIPs), see AIP Backup and Restore.

Packager Plugins

Packagers are software modules that translate between DSpace Item objects and a self-contained external representation, or "package". A *Package Ingester* interprets, or *ingests*, the package and creates an Item. A *Package Disseminator* writes out the contents of an Item in the package format.

A package is typically an archive file such as a Zip or "tar" file, including a *manifest* document which contains metadata and a description of the package contents. The IMS Content Package is a typical packaging standard. A package might also be a single document or media file that contains its own metadata, such as a PDF document with embedded descriptive metadata.

Package ingesters and package disseminators are each a type of named plugin (see Plugin Manager), so it is easy to add new packagers specific to the needs of your site. You do not have to supply both an ingester and disseminator for each format; it is perfectly acceptable to just implement one of them.

Most packager plugins call upon Crosswalk Plugins to translate the metadata between DSpace's object model and the package format.

More information about calling Packagers to ingest or disseminate content can be found in the Package Importer and Exporter section of the System Administration documentation.

Crosswalk Plugins

Crosswalks are software modules that translate between DSpace object metadata and a specific external representation. An *Ingestion Crosswalk* interprets the external format and crosswalks it to DSpace's internal data structure, while a *Dissemination Crosswalk* does the opposite.

For example, a MODS ingestion crosswalk translates descriptive metadata from the MODS format to the metadata fields on a DSpace Item. A MODS dissemination crosswalk generates a MODS document from the metadata on a DSpace Item.

Crosswalk plugins are named plugins (see Plugin Manager), so it is easy to add new crosswalks. You do not have to supply both an ingester and disseminator for each format; it is perfectly acceptable to just implement one of them.

There is also a special pair of crosswalk plugins which use XSL stylesheets to translate the external metadata to or from an internal DSpace format. You can add and modify XSLT crosswalks simply by editing the DSpace configuration and the stylesheets, which are stored in files in the DSpace installation directory.

The Packager plugins and OAH-PMH server make use of crosswalk plugins.

Supervision and Collaboration

In order to facilitate, as a primary objective, the opportunity for thesis authors to be supervised in the preparation of their e-theses, a supervision order system exists to bind groups of other users (thesis supervisors) to an item in someone's pre-submission workspace. The bound group can have system policies associated with it that allow different levels of interaction with the student's item; a small set of default policy groups are provided:

- Full editorial control
- View item contents

Once the default set has been applied, a system administrator may modify them as they would any other policy set in DSpace

This functionality could also be used in situations where researchers wish to collaborate on a particular submission, although there is no particular collaborative workspace functionality.

See Supervision Orders for more details.

User Management

Although many of DSpace's functions such as document discovery and retrieval can be used anonymously, some features (and perhaps some documents) are only available to certain "privileged" users. E-People and Groups are the way DSpace identifies application users for the purpose of granting privileges. This identity is bound to a session of a DSpace application such as the Web UI or one of the command-line batch programs. Both E-People and Groups are granted privileges by the authorization system described below.

User Accounts (E-Person)

DSpace holds the following information about each e-person:

- E-mail address
- First and last names
- Whether the user is able to log in to the system via the Web UI, and whether they must use an X509 certificate to do so;
- A password (encrypted), if appropriate
- A list of collections for which the e-person wishes to be notified of new items
- Whether the e-person 'self-registered' with the system; that is, whether the system created the e-person record automatically as a result of the end-user independently registering with the system, as opposed to the e-person record being generated from the institution's personnel database, for example.
- The network ID for the corresponding LDAP record, if LDAP authentication is used for this E-Person.

Subscriptions

Not yet been implemented. This listed in Tier 3 (see #6 in Tier 3): DSpace Release 7.0 Status#Tier3:MediumPriority

As noted above, end-users (e-people) may 'subscribe' to collections in order to be alerted when new items appear in those collections. Each day, end-users who are subscribed to one or more collections will receive an e-mail giving brief details of all new items that appeared in any of those collections the previous day. If no new items appeared in any of the subscribed collections, no e-mail is sent. Users can unsubscribe themselves at any time. RSS feeds of new items are also available for collections and communities.

Groups

Groups are another kind of entity that can be granted permissions in the authorization system. A group is usually an explicit list of E-People; anyone identified as one of those E-People also gains the privileges granted to the group.

However, an application session can be assigned membership in a group *without* being identified as an E-Person. For example, some sites use this feature to identify users of a local network so they can read restricted materials not open to the whole world. Sessions originating from the local network are given membership in the "LocalUsers" group and gain the corresponding privileges.

Administrators can also use groups as "roles" to manage the granting of privileges more efficiently.

Access Control

Authentication

Authentication is when an application session positively identifies itself as belonging to an E-Person and/or Group. In DSpace, it is implemented by a mechanism called *Stackable Authentication*: the DSpace configuration declares a "stack" of authentication methods. An application (like the Web UI) calls on the Authentication Manager, which tries each of these methods in turn to identify the E-Person to which the session belongs, as well as any extra

Groups. The E-Person authentication methods are tried in turn until one succeeds. Every authenticator in the stack is given a chance to assign extra Groups. This mechanism offers the following advantages:

- Separates authentication from the Web user interface so the same authentication methods are used for other applications such as non-interactive Web Services
- Improved modularity: The authentication methods are all independent of each other. Custom authentication methods can be "stacked" on top of the default DSpace username/password method.
- Cleaner support for "implicit" authentication where username is found in the environment of a Web request, e.g. in an X.509 client certificate.

For more information see Authentication Plugins

Authorization

DSpace's authorization system is based on associating actions with objects and the lists of EPeople who can perform them. The associations are called Resource Policies, and the lists of EPeople are called Groups. There are two built-in groups: 'Administrators', who can do anything in a site, and 'Anonymous', which is a list that contains all users. Assigning a policy for an action on an object to anonymous means giving everyone permission to do that action. (For example, most objects in DSpace sites have a policy of 'anonymous' READ.) Permissions must be explicit - lack of an explicit permission results in the default policy of 'deny'. Permissions also do not 'commute'; for example, if an e-person has READ permission on an item, they might not necessarily have READ permission on the bundles and bitstreams in that item. Currently Collections, Communities and Items are discoverable in the browse and search systems regardless of READ authorization.

The following actions are possible:

Collection

ADD/REMOVE	add or remove items (ADD = permission to submit items)
DEFAULT_ITEM_READ	inherited as READ by all submitted items
DEFAULT_BITSTREAM_READ	inherited as READ by Bitstreams of all submitted items. Note: only affects Bitstreams of an item at the time it is initially submitted. If a Bitstream is added later, it does *not* get the same default read policy.
COLLECTION_ADMIN	collection admins can edit items in a collection, withdraw items, map other items into this collection.

Item

ADD/REMOVE	add or remove bundles
READ	can view item (item metadata is always viewable)
WRITE	can modify item

Bundle

ADD/REMOVE	add or remove bitstreams to a bundle

Bitstream

READ	view bitstream
WRITE	modify bitstream

Note that there is no 'DELETE' action. In order to 'delete' an object (e.g. an item) from the archive, one must have REMOVE permission on all objects (in this case, collection) that contain it. The 'orphaned' item is automatically deleted.

Policies can apply to individual e-people or groups of e-people.

Usage Metrics

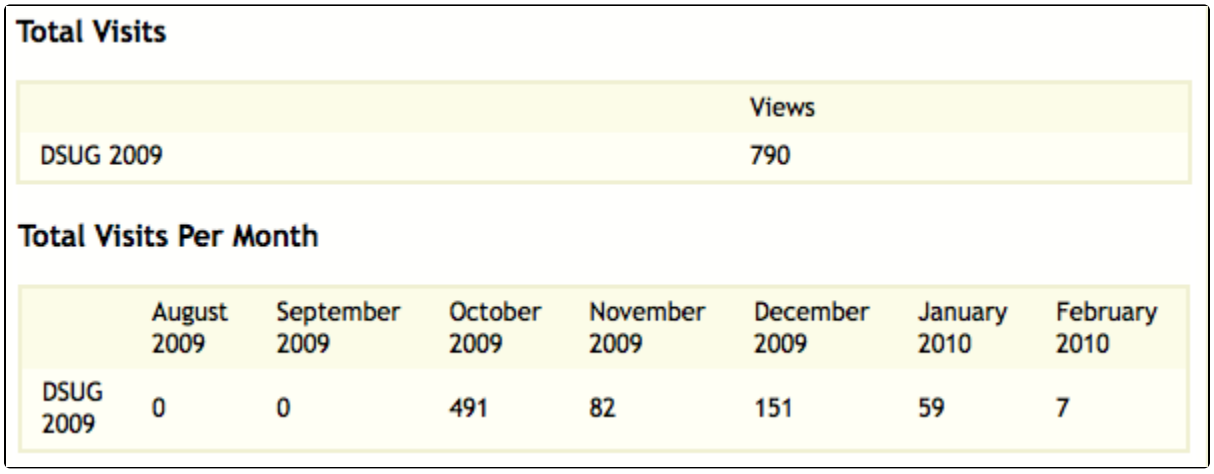
DSpace is equipped with SOLR based infrastructure to log and display pageviews and file downloads.

Item, Collection and Community Usage Statistics

Usage statistics can be retrieved from individual item, collection and community pages. These Usage Statistics pages show:

- Total page visits (all time)
- Total Visits per Month
- File Downloads (all time)*
- Top Country Views (all time)
- Top City Views (all time)

*File Downloads information is only displayed for item-level statistics. Note that downloads from separate bitstreams are also recorded and represented separately. DSpace is able to capture and store File Download information, even when the bitstream was downloaded from a direct link on an external website.



Total Visits

	Views
DSUG 2009	790

Total Visits Per Month

	August 2009	September 2009	October 2009	November 2009	December 2009	January 2010	February 2010
DSUG 2009	0	0	491	82	151	59	7

System Statistics

Various statistical reports about the contents and use of your system can be automatically generated by the system. These are generated by analyzing DSpace's log files. Statistics can be broken down monthly.

The report includes following sections

- A customizable general overview of activities in the archive, by default including:
  - Number of items archived
  - Number of bitstream views
  - Number of item page views
  - Number of collection page views
  - Number of community page views
  - Number of user logins
  - Number of searches performed
  - Number of license rejections
  - Number of OAI Requests
- Customizable summary of archive contents
- Broken-down list of item viewings
- A full break-down of all performed actions
- User logins
- Most popular searches
- Log Level Information
- Processing information!stats_genrl_overview.png!
  The results of statistical analysis can be presented on a by-month and an in-total report, and are available via the user interface. The reports can also either be made public or restricted to administrator access only.

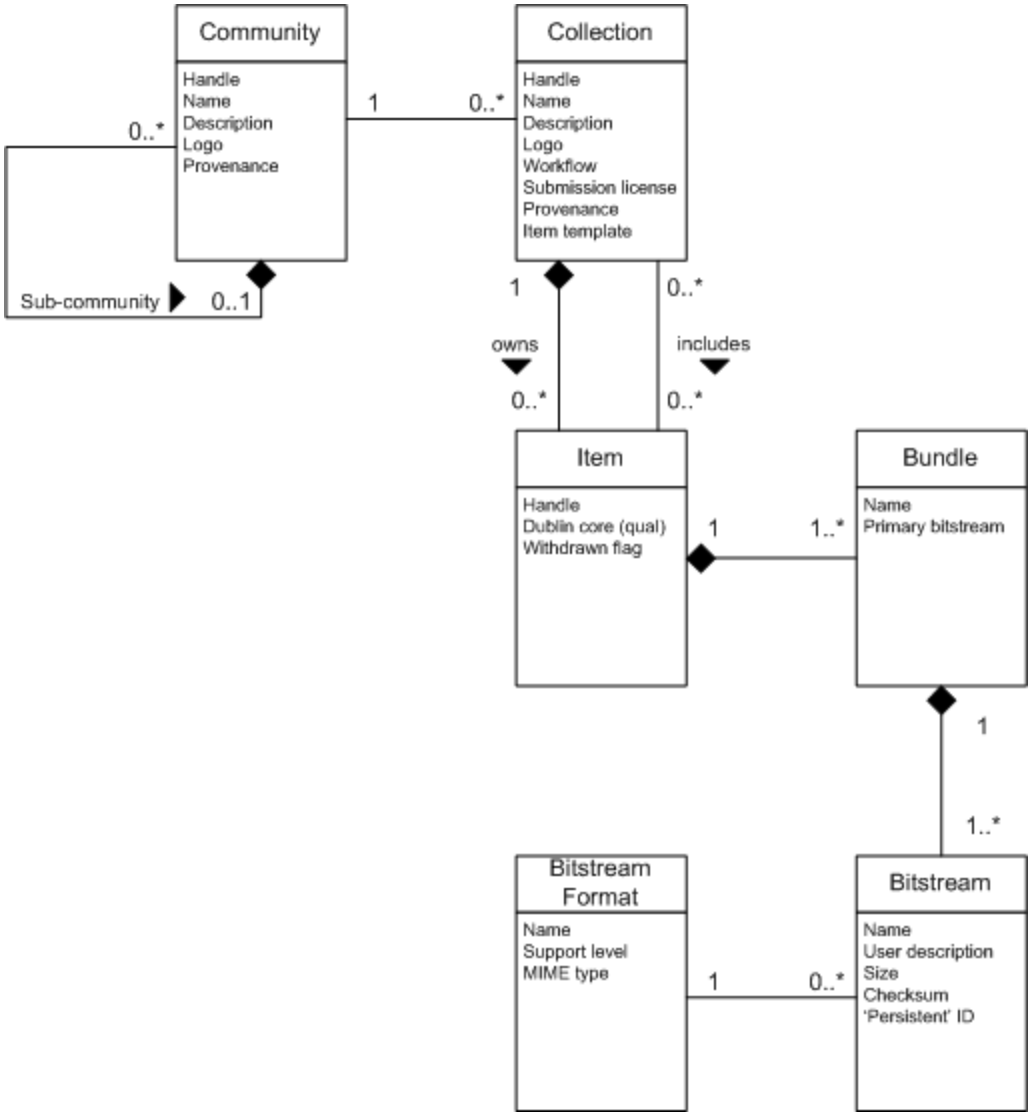
Digital Preservation

Checksum Checker

The purpose of the checker is to verify that the content in a DSpace repository has not become corrupted or been tampered with. The functionality can be invoked on an ad-hoc basis from the command line, or configured via cron or similar. Options exist to support large repositories that cannot be entirely checked in one run of the tool. The tool is extensible to new reporting and checking priority approaches.

System Design

Data Model



Data Model Diagram

The way data is organized in DSpace is intended to reflect the structure of the organization using the DSpace system. Each DSpace site is divided into *communities*, which can be further divided into *sub-communities* reflecting the typical university structure of college, department, research center, or laboratory.

Communities contain *collections*, which are groupings of related content. A collection may only appear in one community at this time.

Each collection is composed of *items*, which are the basic archival elements of the archive. Each item is owned by one collection. Additionally, an item may appear in additional collections; however every item has one and only one owning collection.

Items are further subdivided into named *bundles* of *bitstreams*. Bitstreams are, as the name suggests, streams of bits, usually ordinary computer files. Bitstreams that are somehow closely related, for example HTML files and images that compose a single HTML document, are organized into bundles.

In practice, most items tend to have these named bundles:

- *ORIGINAL* – the bundle with the original, deposited bitstreams
- *THUMBNAILS* – thumbnails of any image bitstreams
- *TEXT* – extracted full-text from bitstreams in ORIGINAL, for indexing
- *LICENSE* – contains the deposit license that the submitter granted the host organization; in other words, specifies the rights that the hosting organization have
- *CC_LICENSE* – contains the distribution license, if any (a icenommons license) associated with the item. This license specifies what end users downloading the content can do with the content

Each bitstream is associated with one *Bitstream Format*. Because preservation services may be an important aspect of the DSpace service, it is important to capture the specific formats of files that users submit. In DSpace, a bitstream format is a unique and consistent way to refer to a particular file format. An integral part of a bitstream format is an either implicit or explicit notion of how material in that format can be interpreted. For example, the interpretation for bitstreams encoded in the JPEG standard for still image compression is defined explicitly in the Standard ISO/IEC 10918-1. The interpretation of bitstreams in Microsoft Word 2000 format is defined implicitly, through reference to the Microsoft Word 2000 application. Bitstream formats can be more specific than MIME types or file suffixes. For example, *application/ms-word* and *.doc* span multiple versions of the Microsoft Word application, each of which produces bitstreams with presumably different characteristics.

Each bitstream format additionally has a *support level*, indicating how well the hosting institution is likely to be able to preserve content in the format in the future. There are three possible support levels that bitstream formats may be assigned by the hosting institution. The host institution should determine the exact meaning of each support level, after careful consideration of costs and requirements. MIT Libraries' interpretation is shown below:

Supported	The format is recognized, and the hosting institution is confident it can make bitstreams of this format usable in the future, using whatever combination of techniques (such as migration, emulation, etc.) is appropriate given the context of need.
Known	The format is recognized, and the hosting institution will promise to preserve the bitstream as-is, and allow it to be retrieved. The hosting institution will attempt to obtain enough information to enable the format to be upgraded to the 'supported' level.
Unsupported	The format is unrecognized, but the hosting institution will undertake to preserve the bitstream as-is and allow it to be retrieved.

Each item has one qualified Dublin Core metadata record. Other metadata might be stored in an item as a serialized bitstream, but we store Dublin Core for every item for interoperability and ease of discovery. The Dublin Core may be entered by end-users as they submit content, or it might be derived from other metadata as part of an ingest process.

Items can be removed from DSpace in one of two ways: They may be 'withdrawn', which means they remain in the archive but are completely hidden from view. In this case, if an end-user attempts to access the withdrawn item, they are presented with a 'tombstone,' that indicates the item has been removed. For whatever reason, an item may also be 'expunged' if necessary, in which case all traces of it are removed from the archive.

Object	Example
Community	Laboratory of Computer Science; Oceanographic Research Center
Collection	LCS Technical Reports; ORC Statistical Data Sets
Item	A technical report; a data set with accompanying description; a video recording of a lecture
Bundle	A group of HTML and image bitstreams making up an HTML document
Bitstream	A single HTML file; a single image file; a source code file
Bitstream Format	Microsoft Word version 6.0; JPEG encoded image format

Amazon S3 Support

DSpace offers two means for storing bitstreams. The first is in the file system on the server. The second is using Amazon S3. For more information, see Storage Layer

Technology Overview

DSpace open source software is free to use, and community supported.

DSpace consists of both a frontend (User Interface) and a backend (REST API & other machine interfaces). A brief overview of the technologies used for each is provided below.

- DSpace Frontend (UI) Technologies
- DSpace Backend (REST API) Technologies
- How the Frontend and Backend interact

DSpace Frontend (UI) Technologies

The DSpace Frontend provides the User Interface which allows people to interact with DSpace. It requires a DSpace backend, and cannot be run standalone.

The DSpace Frontend is built on the Angular platform, written in the Typescript language. It uses Bootstrap & HTML5 for theming/styling & strives for WCAG 2.1 AA alignment. The frontend also uses Angular Universal for "server-side rendering", which allows it to function even when Javascript is unavailable in the browser. For more information on Angular Universal, see the the Angular University guide.

More information on installing the DSpace Frontend can be found in the Installing DSpace guide.

DSpace Backend (REST API) Technologies

The DSpace Backend provides the REST API, which is required by the DSpace Frontend. It also provides additional machine interfaces for interacting with data in DSpace, such as OAI-PMH, SWORDv2 Server, SWORDv1 Server and various command-line (CLI) tools. The DSpace backend can be run standalone, but it doesn't provide a user friendly web interface (which is why the DSpace frontend is recommended).

The DSpace Backend is built on Spring Boot, written in Java. The REST API portion of the backend is built on Spring Technologies, including Spring REST, Spring HATEOAS, and aligns with Spring Data REST. The REST API uses the Spring Data REST Hal Browser as a basic web interface for exploring the REST API. All REST API responses are returned in JSON format.

The DSpace Backend requires a relational database (usually PostgreSQL), used to store all the metadata and relationships between objects. All files uploaded into DSpace are stored on the filesystem (any operating system is supported). Apache Solr is also required, and is used to index all objects for searching/browsing.

More information on installing the DSpace Backend can be found in the Installing DSpace guide. More information on the REST API specifically can be found in our REST Contract.

How the Frontend and Backend interact

Here is a high level overview of what happens when a user interacts with DSpace when the user interface is running in *production* mode:

1. *Initial static page via server-side rendering (SSR):* When a user initially visits any page in the DSpace user interface (UI), this triggers server-side rendering (SSR) via Angular Universal. This means that the UI (Javascript) application is run *on the server* by Node.js. The result is that a *static* HTML page is generated, which will be sent back to the user.
    a. This process of rendering the static HTML page will result in Node.js making requests to REST API to gather all the data necessary to build the *static* HTML page.
2. *Static page is dynamically replaced by UI application:* The user briefly sees the generated static HTML page *while the UI (Javascript) application is downloading to their browser* . This allows the user to immediately see the DSpace User Interface even before it becomes interactive. As soon as the UI application finishes downloading, it dynamically replaces that static HTML page, making the User Interface interactive to the user. (The time between the UI page appearing and becoming interactive is usually unnoticeable to a user.) This entire process is handled by Angular Universal.
3. *Interactions with the UI application send requests to the REST API (client-side rendering):* As soon as the UI becomes interactive, it runs entirely in the user's browser (as any other Javascript application). This means that when the user interacts with the application (by clicking links/buttons or typing in fields, etc), this will send requests from the user's browser to the REST API (backend). This is called client-side rendering (CSR) as all HTML is generated within the user's browser.
    a. At this point, every action in the User Interface will generate one or more requests to the REST API to gather necessary data. These requests are all visible in the user's browser (in the "Network" tab of the browser's "Developer tools").

Keep in mind, SSR can be potentially taxing for very large pages with a lot of objects or data display. This is because Node.js has to make requests to the REST API to gather all the data for the page before rendering the static HTML. Because of this, we do also document some Performance Tuning suggestions for the User Interface (e.g. there is an option to cache these SSR generated static pages in order to generate them less frequently).

Some bots and clients may use server-side rendering at all times

For bots or clients *without the ability to run Javascript*, every page request will trigger SSR (server-side rendering). This is because the static HTML page can never be dynamically replaced by the User Interface application (in step 2 above). However, this behavior is necessary to support Search Engine Optimization. Some search engine bots cannot run Javascript & therefore cannot index sites which do not generate static HTML pages.

Running the user interface in development mode disables SSR and may impact SEO

Running the user interface (frontend) in *development* mode will only utilize client-side rendering (CSR) (as described in step 3 above). This means server-side rendering (SSR) will never occur, and all HTML will be generated in the user's browser. The result is that bots or clients *without the ability to run Javascript* will be unable to interact with the site (which can negatively impact Search Engine Optimization)

Installing DSpace

- Installation Overview
- Installing the Backend (Server API)
  - Backend Requirements
  - Backend Installation
- Installing the Frontend (User Interface)
  - Frontend Requirements
  - Frontend Installation
- What Next?
- Common Installation Issues
  - Troubleshoot an error or find detailed error messages
  - User Interface never appears (no content appears) or "Proxy server received an invalid response"
  - User Interface partially load but then spins (never fully loads or some content doesn't load)
  - "500 Service Unavailable" from the User Interface
  - "No _links section found at..." error from User Interface
  - "RangeError: Maximum call stack size exceeded"
  - "XMLHttpRequest.. has been blocked by CORS policy" or "CORS error" or "Invalid CORS request"
  - Cannot login from the User Interface with a password that I know is valid
  - "403 Forbidden" error with a message that says "Access is denied. Invalid CSRF Token"
  - Using a Self-Signed SSL Certificate causes the Frontend to not be able to access the Backend
  - My REST API is running under HTTPS, but some of its "link" URLs are switching to HTTP
  - My User Interface's robots.txt has incorrect sitemap URLs
  - Cannot upload file from User Interface
  - Javascript heap out of memory
  - Solr responds with "Expected mime type application/octet-stream but got text/html" (404 Not Found)
  - Database errors occur when you run ant fresh_install

Installation Overview

Try out DSpace 7 before you install

If you'd like to quickly try out DSpace 7 before a full installation, see Try out DSpace 7 for instructions on a quick install via Docker.

As of version 7 (and above), the DSpace application is split into a "frontend" (User Interface) and a "backend" (Server API). Most institutions will want to install BOTH. However, you can decide whether to run them on the same machine or separate machines.

- The DSpace Frontend consists of a User Interface built on Angular.io. It is a Node.js web application, i.e. once it is built/compiled, it only require Node.js to run. It cannot be run "standalone", as it *requires* a valid DSpace Backend to function. The frontend provides all user-facing functionality.
- The DSpace Backend consists of a Server API ("server" webapp), built on Spring Boot. It is a Java web application. It can be run standalone, however it has no user interface. The backend provides all machine-based interfaces, including the REST API, OAI-PMH, SWORD (v1 and v2) and RDF.

We recommend installing the Backend **first**, as the Frontend requires a valid Backend to run properly.

Installing the Backend (Server API)

Backend Requirements

- UNIX-like OS or Microsoft Windows
- Java JDK 11 or 17 (OpenJDK or Oracle JDK)
- Apache Maven 3.5.4 or above (Java build tool)
  - Configuring a Maven Proxy
- Apache Ant 1.10.x or later (Java build tool)
- Relational Database (PostgreSQL)
  - PostgreSQL 12.x, 13.x, 14.x, 15.x, 16.x or 17.x (with pgcrypto installed)
  - Oracle (UNSUPPORTED AS OF 7.6)
- Apache Solr 8.x (full-text index/search service)
- Servlet Engine (Apache Tomcat 9, Jetty, Caucho Resin or equivalent)
- (Optional) IP to City Database for Location-based Statistics

UNIX-like OS or Microsoft Windows

- UNIX-like operating system (Linux, HP/UX, Mac OSX, etc.) : Many distributions of Linux/Unix come with some of the dependencies below pre-installed or easily installed via updates. You should consult your particular distribution's documentation or local system administrators to determine what is already available.
- Microsoft Windows: While DSpace can be run on Windows servers, most institutions tend to run it on a UNIX-like operating system.

Java JDK 11 or 17 (OpenJDK or Oracle JDK)

JDK17 support was first available in DSpace 7.2. DSpace 7.1 and 7.0 only supported JDK 11.

- OpenJDK download and installation instructions can be found here http://openjdk.java.net/install/. Most operating systems provide an easy path to install OpenJDK. Just be sure to install the full JDK (development kit), and not the JRE (which is often the default example).
- Oracle's Java can be downloaded from the following location: http://www.oracle.com/technetwork/java/javase/downloads/index.html. Make sure to download the appropriate version of the Java SE JDK.

Make sure to install the JDK and not just the JRE

DSpace requires the full JDK (Java Development Kit) be installed, rather than just the JRE (Java Runtime Environment). So, please be sure that you are installing the full JDK and not just the JRE.

Only JDK11 and JDK 17 are fully supported

Older versions of Java are unsupported. This includes JDK v7-10.

Newer versions of Java may work (e.g. JDK v12-16), but we do not recommend running them in Production. We highly recommend running only Java LTS (Long Term Support) releases in Production, as non-LTS releases may not receive ongoing security fixes. As of this DSpace release, JDK11 and JDK 17 are the two most recent Java LTS releases. As soon as the next Java LTS release is available, we will analyze it for compatibility with this release of DSpace. For more information on Java releases, see the Java roadmaps for Oracle and/or OpenJDK.

Apache Maven 3.5.4 or above (Java build tool)

We recommend using the most recent version of Maven that you can, as newer releases may include performance improvements and security updates. We recommend avoiding any that are "end of life" per https://maven.apache.org/docs/history.html

Maven is necessary in the first stage of the build process to assemble the installation package for your DSpace instance. It gives you the flexibility to customize DSpace using the existing Maven projects found in the *[dspace-source]/dspace/modules* directory or by adding in your own Maven project to build the installation package for DSpace, and apply any custom interface "overlay" changes.

Maven can be downloaded from http://maven.apache.org/download.html It is also provided via many operating system package managers.

Configuring a Maven Proxy

You can configure a proxy to use for some or all of your HTTP requests in Maven. The username and password are only required if your proxy requires basic authentication (note that later releases may support storing your passwords in a secured keystore, in the meantime, please ensure your *settings.xml* file (usually *${user.home}/.m2/settings.xml*) is secured with permissions appropriate for your operating system).

Example:

```
<settings>
  .
  .
  <proxies>
   <proxy>
      <active>true</active>
      <protocol>http</protocol>
      <host>proxy.somewhere.com</host>
      <port>8080</port>
      <username>proxyuser</username>
      <password>somepassword</password>
      <nonProxyHosts>www.google.com|*.somewhere.com</nonProxyHosts>
    </proxy>
  </proxies>
  .
  .
</settings>
```

Apache Ant 1.10.x or later (Java build tool)

While Apache Ant recommends using v1.10.x for Java 11, we've also had some success with recent versions of 1.9.x (specifically v1.9.15 seems to work fine with Java 11). That said, earlier versions of v1.9.x are not compatible with Java 11.

Apache Ant is required for the second stage of the build process (deploying/installing the application). First, Maven is used to construct the installer (`[dspace-source]/dspace/target/dspace-installer`), after which Ant is used to install/deploy DSpace to the installation directory.

Ant can be downloaded from the following location: http://ant.apache.org It is also provided via many operating system package managers.

Relational Database (PostgreSQL)

PostgreSQL 12.x, 13.x, 14.x, 15.x, 16.x or 17.x (with pgcrypto installed)

- PostgreSQL can be downloaded from http://www.postgresql.org/. It is also provided via many operating system package managers.
  - Make sure to select a version of PostgreSQL that is still under support from the PostgreSQL team.
  - If the version of Postgres provided by your package manager is outdated, you may wish to use one of the official PostgreSQL provided repositories:
    - Linux users can select their OS of choice for detailed instructions on using the official PostgreSQL apt or yum repository: http://www.postgresql.org/download/linux/
    - Windows users will need to use the windows installer: http://www.postgresql.org/download/windows/
    - Mac OSX users can choose their preferred installation method: http://www.postgresql.org/download/macosx/
- Install the pgcrypto extension. It will also need to be enabled on your DSpace Database (see Installation instructions below for more info). The pgcrypto extension allows DSpace to create UUIDs (universally unique identifiers) for all objects in DSpace, which means that (internal) object identifiers are now globally unique and no longer tied to database sequences.
  - On most Linux operating systems (Ubuntu, Debian, RedHat), this extension is provided in the "postgresql-contrib" package in your package manager. So, ensure you've installed "postgresql-contrib".
  - On Windows, this extension should be provided automatically by the installer (check your "[PostgreSQL]/share/extension" folder for files starting with "pgcrypto")
- Unicode (specifically UTF-8) support must be enabled (but this is enabled by default).
- Once installed, you need to enable TCP/IP connections (DSpace uses JDBC):
  - In `postgresql.conf`: uncomment the line starting: `listen_addresses = 'localhost'`. This is the default, in recent PostgreSQL releases, but you should at least check it.
  - Then tighten up security a bit by editing `pg_hba.conf` and adding this line:

    ```
    host dspace dspace 127.0.0.1 255.255.255.255 md5
    ```

    This should appear *before* any lines matching `all` databases, because the first matching rule governs.
  - Then restart PostgreSQL.

Oracle (UNSUPPORTED AS OF 7.6)

As of the 7.6 release, Oracle databases are no longer supported

Oracle support has been removed as was previously announced in March 2022 on our mailing lists. See https://github.com/DSpace/DSpace/issues/8214

All DSpace sites should now use PostrgreSQL (see above)

- Details on acquiring Oracle can be downloaded from the following location: http://www.oracle.com/database/. You will need to create a database for DSpace. Make sure that the character set is one of the Unicode character sets. DSpace uses UTF-8 natively, and it is suggested that the Oracle database use the same character set. You will also need to create a user account for DSpace (e.g. *dspace*) and ensure that it has permissions to add and remove tables in the database. Refer to the Quick Installation for more details.
  - **NOTE:** If the database server is not on the same machine as DSpace, you must install the Oracle client to the DSpace server and point `tnsnames.ora` and `listener.ora` files to the database the Oracle server.

Apache Solr 8.x (full-text index/search service)

Solr 8.11.1 or above is recommended as all prior 8.x releases are vulnerable to CVE-2021-44228 (log4j critical vulnerability). If you must use a prior version of 8.x, make sure to add "-Dlog4j2.formatMsgNoLookups=true" to your SOLR_OPTS environment variable, see https://solr.apache.org/security.html#apache-solr-affected-by-apache-log4j-cve-2021-44228

Solr 9.x is not supported by DSpace 8.x out-of-the-box. However, it is possible to run DSpace 7.x with Solr 9.x if you make minor modifications to the "solrconfig.xml" files that come with DSpace. See this single commit which provides those changes, or this below comment for details.

NOTE: with Solr 9.8+ or later, you **must** specify "`-Dsolr.config.lib.enabled=true`" in your `SOLR_OPTS` environment variable, or on the command-line when you start Solr. This "solr.config.lib.enabled" setting allows for "solrconfig.xml" configuration files to still use "<lib>" tags to import plugins. This is **required** for DSpace, because several of our Solr cores (search, oai, etc) import plugins via these "<lib>" tags.

Make sure to install Solr with Authentication disabled (which is the default). DSpace does not yet support authentication to Solr (see https://github.com/DSpace/DSpace/issues/3169). Instead, we recommend placing Solr behind a firewall and/or ensuring port 8983 (which Solr runs on) is not available for public/anonymous access on the web. Solr only needs to be accessible to requests from the DSpace backend.

Solr can be obtained at the Apache Software Foundation site for Solr. You may wish to read portions of the quick-start tutorial to make yourself familiar with Solr's layout and operation. Unpack a Solr .tgz or .zip archive in a place where you keep software that is not handled by your operating system's package management tools, and arrange to have it running whenever DSpace is running. You should ensure that Solr's index directories will have plenty of room to grow. You should also ensure that port 8983 is not in use by something else, or configure Solr to use a different port.

If you are looking for a good place to put Solr, consider `/opt` or `/usr/local`. You can simply unpack Solr in one place and use it. Or you can configure Solr to keep its indexes elsewhere, if you need to – see the Solr documentation for how to do this.

It is not necessary to dedicate a Solr instance to DSpace, if you already have one and want to use it. Simply copy DSpace's cores to a place where they will be discovered by Solr. See below.

Servlet Engine (Apache Tomcat 9, Jetty, Caucho Resin or equivalent)

Only Tomcat 9 is supported at this time. Tomcat 10 is incompatible with Tomcat 9 and will not be supported until DSpace 8.0 at the earliest. See https://github.com/DSpace/DSpace/issues/8713 for more details

- **Apache Tomcat 9.** Tomcat can be downloaded from the following location: http://tomcat.apache.org. It is also provided via many operating system package managers.
  - *The Tomcat owner* (i.e. the user that Tomcat runs as) ***must have read/write access*** *to the DSpace installation directory* (i.e. `[dspace]`). There are a few common ways this may be achieved:
    - One option is to specifically give the Tomcat user (often named "tomcat") ownership of the [dspace] directories, for example:

```
# Change [dspace] and all subfolders to be owned by "tomcat"
chown -R tomcat:tomcat [dspace]
```

        - Another option is to have Tomcat itself *run as* a new user named "dspace" (see installation instructions below). Some operating systems make modifying the Tomcat "run as" user easily modifiable via an environment variable named TOMCAT_USER. This option may be more desirable if you have multiple Tomcat instances running, and you do not want all of them to run under the same Tomcat owner.
    - *On Debian systems*, you may also need to modify or override the "tomcat.service" file to specify the DSpace installation directory in the list of ReadWritePaths. For example:

```
# Replace [dspace] with the full path of your DSpace install
ReadWritePaths=[dspace]
```

    - You need to ensure that Tomcat a) has enough memory to run DSpace, and b) uses UTF-8 as its default file encoding for international character support. So ensure in your startup scripts (etc) that the following environment variable is set: *JAVA_OPTS="-Xmx512M -Xms64M -Dfile.encoding=UTF-8"*
    - **Modifications in *[tomcat]/conf/server.xml*** : You also need to alter Tomcat's default configuration to support searching and browsing of multi-byte UTF-8 correctly. You need to add a configuration option to the *<Connector>* element in *[tomcat]/config/server.xml*: *URIEncoding="UTF-8"* e.g. if you're using the default Tomcat config, it should read:

```
<!-- Define a non-SSL HTTP/1.1 Connector on port 8080 -->
<Connector port="8080"
               minSpareThreads="25"
               enableLookups="false"
               redirectPort="8443"
               connectionTimeout="20000"
               disableUploadTimeout="true"
               URIEncoding="UTF-8"/>
```

      You may change the port from 8080 by editing it in the file above, and by setting the variable *CONNECTOR_PORT* in *server.xml*. You should set the URIEncoding even if you are running Tomcat behind a reverse proxy (Apache HTTPD, Nginx, etc.) via AJP.
- **Jetty or Caucho Resin**
    - DSpace 7 has not been tested with Jetty or Caucho Resin, after the switch to Java 11
    - Older versions of DSpace were able to run on a Tomcat-equivalent servlet Engine, such as Jetty (https://www.eclipse.org/jetty/) or Caucho Resin (http://www.caucho.com/). If you choose to use a different servlet container, please ensure that it supports Servlet Spec 3.1 (or above).
    - Jetty and Resin are configured for correct handling of UTF-8 by default.

(Optional) IP to City Database for Location-based Statistics

Optionally, if you wish to record the geographic locations of clients in DSpace usage statistics records, you will need to install (and regularly update) one of the following:

- Either, a copy of MaxMind's GeoLite City database (in MMDB format)
    - NOTE: Installing MaxMind GeoLite2 is *free*. However, you **must** sign up for a (free) MaxMind account in order to obtain a license key to use the GeoLite2 database.
    - You may download GeoLite2 directly from MaxMind, or many Linux distributions provide the `geoipupdate` tool directly via their package manager. You will still need to configure your license key prior to usage.
    - Once the "GeoLite2-City.mmdb" database file is installed on your system, you will need to configure its location as the value of `usage-statistics.dbfile` in your `local.cfg` configuration file.
    - See the "Managing the City Database File" section of SOLR Statistics for more information about using a City Database with DSpace.
- Or, you can alternatively use/install DB-IP's City Lite database (in MMDB format)
    - This database is also free to use, but does **not** require an account to download.
    - Once the "dbip-city-lite.mmdb" database file is installed on your system, you will need to configure its location as the value of `usage-statistics.dbfile` in your `local.cfg` configuration file.
    - See the "Managing the City Database File" section of SOLR Statistics for more information about using a City Database with DSpace.

Backend Installation

1. Install all the Backend Requirements listed above.
2. **Create a DSpace operating system user *(optional)*** . As noted in the prerequisites above, Tomcat (or Jetty, etc) **must run as** an operating system user account that has full read/write access to the DSpace installation directory (i.e. `[dspace]`). Either you must ensure the Tomcat owner also owns `[dspace]`, OR you can create a new "dspace" user account, and ensure that Tomcat also runs as that account:

```
useradd -m dspace
```

The choice that makes the most sense for you will probably depend on how you installed your servlet container (Tomcat/Jetty/etc). If you installed it from source, you will need to create a user account to run it, and that account can be named anything, e.g. 'dspace'. If you used your operating system's package manager to install the container, then a user account should have been created as part of that process and it will be much easier to use that account than to try to change it.

3. **Download** the latest DSpace release from the DSpace GitHub Repository. You can choose to either download the zip or tar.gz file provided by GitHub, or you can use "git" to checkout the appropriate tag (e.g. `dspace-7.2`) or branch.
4. **Unpack the DSpace software**. After downloading the software, based on the compression file format, choose one of the following methods to unpack your software:
    a. **Zip file**. If you downloaded *dspace-7.2.zip* do the following:

    ```
    unzip dspace-7.2.zip
    ```

    b. **.gz file**. If you downloaded *dspace-7.2.tar.gz* do the following:

    ```
    gunzip -c dspace-7.2.tar.gz | tar -xf -
    ```

    For ease of reference, we will refer to the location of this unzipped version of the DSpace release as *[dspace-source]* in the remainder of these instructions. After unpacking the file, the user may wish to change the ownership of the dspace-7.x folder to the "dspace" user. (And you may need to change the group).
5. **Database Setup**
    - *PostgreSQL:*
        - Create a `dspace` database user (this user can have any name, but we'll assume you name it "dspace"). This is entirely separate from the `dspace` operating-system user created above:

        ```
        createuser --username=postgres --no-superuser --pwprompt dspace
        ```

        You will be prompted (twice) for a password for the new `dspace` user. Then you'll be prompted for the password of the PostgreSQL superuser (`postgres`).
        - Create a `dspace` database, owned by the `dspace` PostgreSQL user. Similar to the previous step, this can only be done by a "superuser" account in PostgreSQL (e.g. `postgres`):

        ```
        createdb --username=postgres --owner=dspace --encoding=UNICODE dspace
        ```

        You will be prompted for the password of the PostgreSQL superuser (`postgres`).
        - Finally, you MUST enable the pgcrypto extension on your new dspace database. Again, this can only be enabled by a "superuser" account (e.g. `postgres`)

        ```
        # Login to the database as a superuser, and enable the pgcrypto extension on this database
        psql --username=postgres dspace -c "CREATE EXTENSION pgcrypto;"
        ```

        The "CREATE EXTENSION" command should return with no result if it succeeds. If it fails or throws an error, it is likely you are missing the required pgcrypto extension (see Database Prerequisites above).

            - ***Alternative method: How to enable pgcrypto via a separate database schema.*** While the above method of enabling pgcrypto is perfectly fine for the majority of users, there may be some scenarios where a database administrator would prefer to install extensions into a database schema that is *separate from* the DSpace tables. Developers also may wish to install pgcrypto into a separate schema if they plan to "clean" (recreate) their development database frequently. Keeping extensions in a separate schema from the DSpace tables will ensure developers would NOT have to continually re-enable the extension each time you run a "`./dspace database clean`". If you wish to install pgcrypto in a separate schema here's how to do that:

```
# Login to the database as a superuser
psql --username=postgres dspace
# Create a new schema in this database named "extensions" (or whatever you want to
name it)
CREATE SCHEMA extensions;
# Enable this extension in this new schema
CREATE EXTENSION pgcrypto SCHEMA extensions;
# Grant rights to call functions in the extensions schema to your dspace user
GRANT USAGE ON SCHEMA extensions TO dspace;


# Append "extensions" on the current session's "search_path" (if it doesn't already
exist in search_path)
# The "search_path" config is the list of schemas that Postgres will use
SELECT set_config('search_path',current_setting('search_path') || ',extensions',
false) WHERE current_setting('search_path') !~ '(^|,)extensions(,|$)';
# Verify the current session's "search_path" and make sure it's correct
SHOW search_path;
# Now, update the "dspace" Database to use the same "search_path" (for all future
sessions) as we've set for this current session (i.e. via set_config() above)
ALTER DATABASE dspace SET search_path FROM CURRENT;
```

- *Oracle (UNSUPPORTED AS OF 7.6):*
  - Setting up DSpace to use Oracle is a bit different now. You will need still need to get a copy of the Oracle JDBC driver, but instead of copying it into a lib directory you will need to install it into your local Maven repository. (You'll need to download it first from this location: http://www.oracle.com/technetwork/database/enterprise-edition/jdbc-112010-090769.html.) Run the following command (all on one line):

    ```
    mvn install:install-file
        -Dfile=ojdbc6.jar
        -DgroupId=com.oracle
        -DartifactId=ojdbc6
        -Dversion=11.2.0.4.0
        -Dpackaging=jar
        -DgeneratePom=true
    ```

  - You need to compile DSpace with an Oracle driver (ojdbc6.jar) corresponding to your Oracle version - update the version in *[dspace-source]/pom.xml* E.g.:

    ```
    <dependency>
      <groupId>com.oracle</groupId>
      <artifactId>ojdbc6</artifactId>
      <version>11.2.0.4.0</version>
    </dependency>
    ```

  - Create a database for DSpace. Make sure that the character set is one of the Unicode character sets. DSpace uses UTF-8 natively, and it is required that the Oracle database use the same character set. Create a user account for DSpace (e.g. *dspace*) and ensure that it has permissions to add and remove tables in the database.
  - NOTE: You will need to ensure the proper `db.*` settings are specified in your `local.cfg` file (see next step), as the defaults for all of these settings assuming a PostgreSQL database backend.

```
db.url = jdbc:oracle:thin:@host:port/SID
# e.g. db.url = jdbc:oracle:thin:@//localhost:1521/xe
# NOTE: in db.url, SID is the SID of your database defined in tnsnames.ora
# the default Oracle port is 1521
# You may also use a full SID definition, e.g.
# db.url = jdbc:oracle:thin:@(description=(address_list=(address=(protocol=TCP)
(host=localhost)(port=1521)))(connect_data=(service_name=DSPACE)))

# Oracle driver and dialect
db.driver = oracle.jdbc.OracleDriver
db.dialect = org.hibernate.dialect.Oracle10gDialect

# Specify DB username, password and schema to use
db.username =
db.password =
db.schema = ${db.username}
# For Oracle, schema is equivalent to the username of your database account,
# so this may be set to ${db.username} in most scenarios
```

    - Later, during the Maven build step, don't forget to specify `mvn -Ddb.name=oracle package`
6. **Initial Configuration (local.cfg):** Create your own `[dspace-source]/dspace/config/local.cfg` configuration file. You may wish to simply copy the provided `[dspace-source]/dspace/config/local.cfg.EXAMPLE`. This local.cfg file can be used to store *any* configuration changes that you wish to make which are local to your installation (see local.cfg configuration file documentation). ANY setting may be copied into this local.cfg file from the dspace.cfg or any other *.cfg file in order to override the default setting (see note below). For the initial installation of DSpace, there are some key settings you'll likely want to override. Those are provided in the `[dspace-source]/dspace/config/local.cfg.EXAMPLE`. (NOTE: Settings followed with an asterisk (*) are highly recommended, while all others are optional during initial installation and may be customized at a later time.)
    - `dspace.dir*` - must be set to the *[dspace]* (installation) directory (*NOTE: On Windows be sure to use forward slashes for the directory path!* For example: "`C:/dspace`" is a valid path for Windows.)
    - `dspace.server.url*` - complete URL of this DSpace backend (including port and any subpath). **Do not end with '/'**. For example: http://localhost:8080/server
    - `dspace.ui.url*` - complete URL of the DSpace frontend (including port and any subpath). REQUIRED for the REST API to fully trust requests from the DSpace frontend. **Do not end with '/'**. For example: http://localhost:4000
    - `dspace.name` - Human-readable, "proper" name of your server, e.g. "My Digital Library".
    - `solr.server*` - complete URL of the Solr server. DSpace makes use of Solr for indexing purposes. http://localhost:8983/solr unless you changed the port or installed Solr on some other host.
    - `default.language` - Default language for all metadata values (defaults to "en_US")
    - `db.url*` - The full JDBC URL to your database (examples are provided in the `local.cfg.EXAMPLE`)
    - `db.driver*` - Which database driver to use for PostgreSQL (default should be fine)
    - `db.dialect*` - Which database dialect to use for PostgreSQL (default should be fine)
    - `db.username*` - the database username used in the previous step.
    - `db.password*` - the database password used in the previous step.
    - `db.schema*` - the database schema to use (examples are provided in the local.cfg.EXAMPLE)
    - `mail.server` - fully-qualified domain name of your outgoing mail server.
    - `mail.from.address` - the "From:" address to put on email sent by DSpace.
    - `feedback.recipient` - mailbox for feedback mail.
    - `mail.admin` - mailbox for DSpace site administrator.
    - `alert.recipient` - mailbox for server errors/alerts (not essential but very useful!)
    - `registration.notify`- mailbox for emails when new users register (optional)

    > Your local.cfg file can override ANY settings from other *.cfg files in DSpace
    >
    > The provided `local.cfg.EXAMPLE` only includes a small subset of the configuration settings available with DSpace. It provides a good starting point for your own `local.cfg` file.
    >
    > However, you should be aware that ANY configuration can now be copied into your `local.cfg` to override the default settings. This includes ANY of the settings/configurations in:
    >
    > - The primary dspace.cfg file (`[dspace]/config/dspace.cfg`)
    > - Any of the module configuration files (`[dspace]/config/modules/*.cfg` files)
    > - Any of the Spring Boot settings (`[dspace-src]/dspace-server-webapp/src/main/resources/application.properties`)
    >
    > Individual settings may also be commented out or removed in your `local.cfg`, in order to re-enable default settings.
    >
    > See the Configuration Reference section for more details.
7. **DSpace Directory:** Create the directory for the DSpace backend installation (i.e. `[dspace]`). As *root* (or a user with appropriate permissions), run:

```
mkdir [dspace]
chown dspace [dspace]
```

(Assuming the *dspace* UNIX username.)

8. **Build the Installation Package:** As the *dspace* UNIX user, generate the DSpace installation package.

```
cd [dspace-source]
mvn package
```

Building with Oracle Database Support (UNSUPPORTED AS OF 7.6)

Without any extra arguments, the DSpace installation package is initialized for PostgreSQL. If you want to use Oracle instead, you should build the DSpace installation package as follows:
`mvn -Ddb.name=oracle package`

9. **Install DSpace Backend:** As the *dspace* UNIX user, install DSpace to `[dspace]`:

```
cd [dspace-source]/dspace/target/dspace-installer
ant fresh_install
```

To see a complete list of build targets, run: `ant help` *The most likely thing to go wrong here is the test of your database connection. See the Common Installation Issues Section below for more details.*

10. **Initialize your Database:** While this step is optional (as the DSpace database should auto-initialize itself on first startup), it's always good to verify one last time that your database connection is working properly. To initialize the database run:

```
[dspace]/bin/dspace database migrate
```

    a. After running this script, it's a good idea to run "./dspace database info" to check that your database has been fully initialized. A fully initialized database should list the state of all migrations as either "Success" or "Out of Order". If any migrations have failed or are still listed as "Pending", then you need to check your "dspace.log" for possible "ERROR" messages. If any errors appeared, you will need to resolve them before continuing.

11. **Deploy Server web application:** The DSpace backend consists of a single "server" webapp (in `[dspace]/webapps/server`). You need to deploy this webapp into your Servlet Container (e.g. Tomcat). Generally, there are two options (or techniques) which you could use...either configure Tomcat to find the DSpace "server" webapp, or copy the "server" webapp into Tomcat's own webapps folder.
    - *Technique A.* Tell your Tomcat/Jetty/Resin installation where to find your DSpace web application(s). As an example, in the file `[tomcat]/conf/server.xml` you can add a Context member under the `<Host>` (but replace `[dspace]`with your installation location):

DEFINE A CONTEXT PATH FOR DSpace Server webapp: server.xml

```
<Host name="localhost"  appBase="webapps" unpackWARs="true" autoDeploy="true">
       <Context path="/server" docBase="[dspace]/webapps/server" reloadable="true" />
```

      The name of the file (not including the suffix ".xml") will be the name of the context, so for example `server.xml` defines the context at http://host:8080/server. To define the *root context* (http://host:8080/), name that context's file `ROOT.xml`. Optionally, you can also choose to install the old, deprecated "rest" webapp if you
    - *Technique B.* Simple and complete. You copy only (or all) of the DSpace Web application(s) you wish to use from the [dspace]/webapps directory to the appropriate directory in your Tomcat/Jetty/Resin installation. For example:
      `cp -R [dspace]/webapps/* [tomcat]/webapps` (This will copy all the web applications to Tomcat).
      `cp -R [dspace]/webapps/server [tomcat]/webapps` (This will copy only the Server web application to Tomcat.)

      To define the *root context* (http://host:8080/), name that context's directory `ROOT`.

12. **Optionally, also install the deprecated DSpace 6.x REST API web application.** If you previously used the DSpace 6.x REST API, for backwards compatibility the old, deprecated "rest" webapp is still available to install (in `[dspace]/webapps/rest`). It is NOT used by the DSpace frontend. So, most users should skip this step.
13. **Copy Solr cores:** DSpace installation creates a set of four empty Solr cores already configured.

    a. Copy them from `[dspace]`/solr to the place where your Solr instance will discover them. For example:

```
# [solr] is the location where Solr is installed.
# NOTE: On Debian systems the configsets may be under /var/solr/data/configsets
cp -R [dspace]/solr/* [solr]/server/solr/configsets

# Make sure everything is owned by the system user who owns Solr
# Usually this is a 'solr' user account
# See https://solr.apache.org/guide/8_1/taking-solr-to-production.html#create-the-solr-user
chown -R solr:solr [solr]/server/solr/configsets
```

    b. Start (or re-start) Solr. For example:

```
[solr]/bin/solr restart
```

c. You can check the status of Solr and your new DSpace cores by using its administrative web interface. Browse to `${solr.server}` (e.g. `http://localhost:8983/solr/`) to see if Solr is running well, then look at the cores by selecting (on the left) Core Admin or using the Core Selector drop list.

i. For example, to test that your "search" core is setup properly, try accessing the URL `${solr.server}/search/select`. It should run an empty query against the "search" core, returning an empty JSON result. If it returns an error, then that means your "search" core is missing or not installed properly.

14. **Create an Administrator Account:** Create an initial administrator account from the command line:

```
[dspace]/bin/dspace create-administrator
```

15. **Initial Startup!** Now the moment of truth! Start up (or restart) Tomcat/Jetty/Resin.
    a. *REST API Interface* - (e.g.) http://dspace.example.edu:8080/server/
    b. *OAI-PMH Interface* - (e.g.) http://dspace.example.edu:8080/server/oai/request?verb=Identify
    c. For an example of what the default backend looks like, visit the Demo Backend: https://demo.dspace.org/server/
16. **Setup scheduled tasks for behind-the-scenes processes:** For all features of DSpace to work properly, there are some scheduled tasks you MUST setup to run on a regular basis. Some examples are tasks that help create thumbnails (for images), do full-text indexing (of textual content) and send out subscription emails. See the Scheduled Tasks via Cron for more details.
17. **Production Installation (adding HTTPS support)**: *Running the DSpace Backend on HTTP & port 8080 is only usable for local development environments (where you are running the UI and REST API from the same machine, and only accessing them via localhost URLs).* **If you want to run DSpace in Production, you MUST run the backend with HTTPS support** (otherwise logins will not work outside of your local domain).
    a. For HTTPS support, we recommend installing either Apache HTTPD or Nginx, configuring SSL at that level, and proxying all requests to your Tomcat installation. Keep in mind, if you want to host both the DSpace Backend and Frontend on the same server, you can use one installation of Apache HTTPD or NGinx to manage HTTPS/SSL and proxy to both.
    b. *Apache HTTPD:* These instructions are specific to Apache HTTPD, but a similar setup can be achieved with NGinx (see below)
        i. Install Apache HTTPD, e.g. `sudo apt install apache2`
        ii. Install mod_headers, mod_proxy and mod_proxy_ajp (or mod_proxy_http) modules, e.g. `sudo a2enmod headers; sudo a2enmod proxy; sudo a2enmod proxy_ajp`
            1. Alternatively, you can choose to use mod_proxy_http to create an http proxy. A separate example is commented out below
        iii. For mod_proxy_ajp to communicate with Tomcat, you'll need to enable Tomcat's AJP connector in your Tomcat's server.xml:

```
<Connector protocol="AJP/1.3" port="8009" redirectPort="8443" URIEncoding="UTF-8" />
```

        iv. Restart Apache to enable these modules
        v. Obtain an SSL certificate for HTTPS support. If you don't have one yet, you can use Let's Encrypt (for free) using the "certbot" tool: https://certbot.eff.org/
        vi. Now, setup a new VirtualHost for your site (using HTTPS / port 443) which proxies all requests to Tomcat's AJP connector (running on port 8009)

```
<VirtualHost _default_:443>
    # Add your domain here. We've added "dspace.example.edu" as an example
    ServerName dspace.example.edu
    .. setup your host how you want, including log settings...     .. setup your host how
you want, including log settings...

    # Most installs will need these options enabled to ensure DSpace knows its hostname and
scheme (http or https)
    # Also required to ensure correct sitemap URLs appear in /robots.txt for User Interface.
    ProxyPreserveHost On
    RequestHeader set X-Forwarded-Proto https

    SSLEngine on
    SSLCertificateFile [full-path-to-PEM-cert]
    SSLCertificateKeyFile [full-path-to-cert-KEY]
    # LetsEncrypt certificates (and possibly others) may require a chain file be specified
    # in order for the UI / Node.js to validate the HTTPS connection.
    #SSLCertificateChainFile [full-path-to-chain-file]

    # Proxy all HTTPS requests to "/server" from Apache to Tomcat via AJP connector
    ProxyPass /server ajp://localhost:8009/server
    ProxyPassReverse /server ajp://localhost:8009/server

    # If you would rather use mod_proxy_http as an http proxy to port 8080
    # then use these settings instead
    #ProxyPass /server http://localhost:8080/server
    #ProxyPassReverse /server http://localhost:8080/server
</VirtualHost>
```

c. *NGinx:* These instructions are specific to NGinx.
   i. Install/Setup NGinx
   ii. Sample NGinx "server block" configuration. Keep in mind we are only providing basic example settings.

```
# Setup HTTP to redirect to HTTPS
server {
  listen 80;
  # Add your domain here. We've added "dspace.example.edu" as an example
  server_name dspace.example.edu;
  rewrite ^ https://dspace.example.edu permanent;
}

# Setup HTTPS access
server {
  listen 443 ssl;
  # Add your domain here. We've added "dspace.example.edu" as an example
  server_name dspace.example.edu;

  # Add your SSL certificate/key path here
  # NOTE: For LetsEncrypt, the certificate should be the full certificate chain file
  ssl_certificate dspace.example.edu.crt (or PEM);
  ssl_certificate_key dspace.example.edu.key;

  # Proxy all HTTPS requests to "/server" from NGinx to Tomcat on port 8080
  location /server {
    proxy_set_header X-Forwarded-Proto https;
    proxy_set_header X-Forwarded-Host $host;
    proxy_pass http://localhost:8080/server;
  }
}
```

d. After switching to HTTPS, make sure to go back and update the URLs (primarily `dspace.server.url`) in your local.cfg to match the new URL of your backend (REST API). This will require briefly rebooting Tomcat.

Installing the Frontend (User Interface)

Below instructions are specific to 7.2 (or later)

The Frontend Instructions below are specific to 7.2 or later. For Frontend Installation instructions for 7.0 or 7.1, see 7.0-7.1 Frontend Installation

Frontend Requirements

- UNIX-like OS or Microsoft Windows
- Node.js (v16.x or v18.x or v20.x or v22.x)
- Yarn (v1.x)
- PM2 (or another Process Manager for Node.js apps) (optional, but recommended for Production)
- DSpace 7.x Backend (see above)

UNIX-like OS or Microsoft Windows

- UNIX-like operating system (Linux, HP/UX, Mac OSX, etc.) : Many distributions of Linux/Unix come with some of the dependencies below pre-installed or easily installed via updates. You should consult your particular distribution's documentation or local system administrators to determine what is already available.
- Microsoft Windows: While DSpace can be run on Windows servers, most institutions tend to run it on a UNIX-like operating system.

Node.js (v16.x or v18.x or v20.x or v22.x)

Node v16.x may no longer work as of DSpace 7.6.3. Because Node.js 16 is end-of-life, some dependencies of DSpace no longer support Node 16, which may result in build failures.

Node v20.x and v22.x may only be supported in DSpace 7.6.2 or later. See https://github.com/DSpace/dspace-angular/pull/2849

- Node.js can be found at https://nodejs.org/. It may be available through your Linux distribution's package manager. We recommend running a Long Term Support (LTS) version (even numbered releases). Non-LTS versions (odd numbered releases) are not recommended.
- Node.js is a Javascript runtime that also provides npm (Node Package Manager). It is used to both build and run the frontend.

Yarn (v1.x)

- Yarn v1.x is available at https://classic.yarnpkg.com/. It can usually be install via NPM (or through your Linux distribution's package manager). *We do NOT currently support Yarn v2.*

```
# You may need to run this command using "sudo" if you don't have proper privileges
npm install --global yarn
```

- Yarn is used to build the frontend.

PM2 (or another Process Manager for Node.js apps) (*optional, but recommended for Production*)

- In Production scenarios, we *highly recommend* starting/stopping the User Interface using a Node.js process manager. There are several available, but our current favorite is PM2. The rest of this installation guide assumes you are using PM2.
- PM2 is very easily installed via NPM

```
# You may need to run this command using "sudo" if you don't have proper privileges
npm install --global pm2
```

DSpace 7.x Backend (see above)

- The DSpace User Interface (Frontend) cannot function without an installed DSpace Backend. Follow the instructions above.
- The Frontend and Backend *do not need to be installed on the same machine/server*. They may be installed on separate machines as long as the two machines can connect to one another via HTTP or HTTPS.

Frontend Installation

Below instructions are specific to 7.2 (or later)

The Frontend Instructions below are specific to 7.2 or later. For Frontend Installation instructions for 7.0 or 7.1, see 7.0-7.1 Frontend Installation

1. **Download Code (to [dspace-angular]):** Download the latest dspace-angular release from the DSpace GitHub repository. You can choose to either download the zip or tar.gz file provided by GitHub, or you can use "git" to checkout the appropriate tag (e.g. `dspace-7.2`) or branch.
    a. NOTE: For the rest of these instructions, we'll refer to the source code location as `[dspace-angular]`.
2. **Install Dependencies:** Install all required local dependencies by running the following from within the unzipped `[dspace-angular]` directory

```
# change directory to our repo
cd [dspace-angular]

# install the local dependencies
yarn install

# NOTE: Some dependencies occasionally get overly strict over exact versions of Node & Yarn.
# If you are running a supported version of Node & Yarn, but see a message like
# `The engine "node" is incompatible with this module.`, you can disregard it using this flag:
# yarn install --ignore-engines
```

3. **Build/Compile:** Build the User Interface for Production. This builds source code (under `[dspace-angular]/src/`) to create a compiled version of the User Interface in the `[dspace-angular]/dist` folder. This `/dist` folder is what we will deploy & run to start the UI.

```
yarn build:prod
```

   a. You only need to rebuild the UI application if you change source code (under `[dspace-angular]/src/`). Simply changing the configurations (e.g. config.prod.yml, see below) do not require a rebuild, but only require restarting the UI.
4. **Deployment (to [dspace-ui-deploy]):** *(Only recommended for Production setups)* Choose/Create a directory on your server where you wish to run the compiled User Interface. We'll call this `[dspace-ui-deploy]`.

[dspace-ui-deploy] vs [dspace-angular]

`[dspace-angular]` is the directory where you've downloaded and built the UI source code (per the instructions above). For deployment /running the UI, we recommend creating an entirely separate `[dspace-ui-deploy]` directory. This keeps your running, production User Interface separate from your source code directory and also minimizes downtime when rebuilding your UI. You may even choose to deploy to a `[dspace-ui-deploy]` directory on a different server (and copy the `/dist` directory over via FTP or similar).

If you are installing the UI for the first time, or just want a simple setup, you can choose to have [dspace-ui-deploy] and [dspace-angular] be the *same directory.* This would mean you don't have to copy your /dist folder to another location. However, the downside is that your running site will become unresponsive whenever you do a re-build/re-compile (i.e. rerun "yarn build:prod") as this build process will first delete the `[dspace-angular]/dist` directory before rebuilding it.

   a. Copy the entire `[dspace-angular]/dist/` folder to this location. For example:

```
cp -r [dspace-angular]/dist [dspace-ui-deploy]
```

   b. WARNING: At this time, you MUST copy the entire "dist" folder and make sure NOT to rename it. Therefore, the directory structure should look like this:

Contents of [dspace-ui-deploy] folder

```
[dspace-ui-deploy]
    /dist
       /browser (compiled client-side code)
       /server  (compiled server-side code, including "main.js")
    /config     (Optionally created in the "Configuration" step below)
       /config.prod.yml (Optionally created in the "Configuration" step below)
```

   c. NOTE: the OS account which runs the UI via Node.js (see below) MUST have write privileges to the `[dspace-ui-deploy]` directory (because on startup, the runtime configuration is written to `[dspace-ui-deploy]/dist/browser/assets/config.json`)
5. **Configuration:** You have two options for User Interface Configuration, Environment Variables or YAML-based configuration (`config.prod.yml`). Choose one!

   a. *YAML configuration:* Create a "config.prod.yml" at `[dspace-ui-deploy]/config/config.prod.yml`. You may wish to use the `[dspace-angular]/config/config.example.yml` as a starting point. This `config.prod.yml` file can be used to override any of the default configurations listed in the `config.example.yml` (in that same directory). *At a minimum* this file MUST include a "rest" section (and may also include a "ui" section), similar to the following (keep in mind, you only need to include settings that you need to modify).

Example config.prod.yml

```
# The "ui" section defines where you want Node.js to run/respond. It often is a *localhost* (non-public) URL, especially if you are using a Proxy.
# However, the "baseUrl" setting will define the public URL of your DSpace instance. It should be set to the same value as "dspace.ui.url" on the backend.
# In this example, we are setting up our UI to just use localhost, port 4000. But, the public URL is "https://dspace.example.edu"
# This is a common setup for when you want to use Apache or Nginx to handle HTTPS and proxy requests to Node on port 4000
ui:
  ssl: false
  host: localhost
  port: 4000
  nameSpace: /
  # This is the public URL where the site will respond (often defined in your Apache/Nginx proxy)
  # It MUST be kept in sync with the value of "dspace.ui.url" in the backend's local.cfg
  baseUrl: https://dspace.example.edu

# This example is valid if your Backend is publicly available at https://dspace.example.edu/server/
# The REST settings MUST correspond to the primary/public URL of the backend. Usually, this means they must be kept in sync
# with the value of "dspace.server.url" in the backend's local.cfg
rest:
  ssl: true
  host: dspace.example.edu
  port: 443
  nameSpace: /server
```

b. *Environment variables:* Every configuration in the UI may be specified via an Environment Variable. See Configuration Override in the User Interface Configuration documentation for more details. For example, the below environment variables provide the same setup as the config.prod.yml example above.

Example Environment Variables

```
# All environment variables MUST
# (1) be prefixed with "DSPACE_"
# (2) use underscores as separators (no dots allowed), and
# (3) use all uppercase

# "ui" section
DSPACE_UI_SSL = false
DSPACE_UI_HOST = localhost
DSPACE_UI_PORT = 4000
DSPACE_UI_NAMESPACE = /
DSPACE_UI_BASEURL = dspace.example.edu

# "rest" section
DSPACE_REST_SSL = true
DSPACE_REST_HOST = dspace.example.edu
DSPACE_REST_PORT = 443
DSPACE_REST_NAMESPACE = /server
```

   i. NOTE: When using PM2, some may find it easier to use Environment variables, as it allows you to specify DSpace UI configs within your PM2 configuration. See PM2 instructions below.

c. Configuration Hints:
   i. See the User Interface Configuration documentation for a list of all available configurations.
   ii. In the "ui" section above, you may wish to start with "ssl: false" and "port: 4000" just to be certain that everything else is working properly **before** adding HTTPS support. KEEP IN MIND, we highly recommend always using HTTPS for Production. (See section on HTTPS below)
   iii. (Optionally) *Test the connection to your REST API from the UI from the command-line.* This is not required, but it can sometimes help you discover immediate configuration issues if the test fails.
      1. If you are using YAML configs, copy your config.prod.yml back into your source code folder at `[dspace-angular]/config/config.prod.yml`
      2. From `[dspace-angular]`, run `yarn test:rest` This script will attempt a basic Node.js connection to the REST API that is configured in your "config.prod.yml" file and validate the response.
      3. A successful connection should return a 200 Response and all JSON validation checks should return "true"

4. If you receive a connection error or different response code, you MUST fix your REST API before the UI will be able to work. See also the "Common Installation Issues" below. If you receive an SSL error, see "Using a Self-Signed SSL Certificate causes the Frontend to not be able to access the Backend"

iv. When using a subpath (nameSpace) in your UI server base URL (e.g. "http://localhost:4000/mysite/" instead of "http://localhost:4000/"), you must make sure that the URL **without** the subpath is added to the `rest.cors.allowed-origins` list in `[dspace]/config/modules/rest.cfg` or the `local.cfg` override. The default value used for this configuration assumes that Origin and DSpace URL are identical, but CORS origins do not contain a subpath.

6. **Start up the User Interface:** The compiled User Interface only requires Node.js to run. However, most users may want to use PM2 (or a similar Node.js process manager) in Production to provide easier logging and restart tools.
   a. *Quick Start:* To quickly startup / test the User Interface, you can just use Node.js. This is only recommended for quickly testing the UI is working, as no logs are available.

```
# You MUST start the UI from within the deployment directory
cd [dspace-ui-deploy]

# Run the "server/main.js" file to startup the User Interface
node ./dist/server/main.js

# Stop the UI by killing it via Ctrl+C
```

   b. *Run via PM2:* Using PM2 (or a different Node.js process manager) is highly recommended for Production scenarios. Here's an example of a Production setup of PM2.
      i. First you need to create a PM2 JSON configuration file which will run the User Interface. This file can be named anything & placed where ever you like, but you may want to save it to your deployment directory (e.g. `[dspace-ui-deploy]/dspace-ui.json`).

dspace-ui.json

```
{
    "apps": [
        {
           "name": "dspace-ui",
           "cwd": "/full/path/to/dspace-ui-deploy",
           "script": "dist/server/main.js",
           "instances": "max",
           "exec_mode": "cluster",
           "env": {
              "NODE_ENV": "production"
           }
        }
    ]
}
```

1. NOTE: The "cwd" setting MUST correspond to your `[dspace-ui-deploy]` folder path.
2. NOTE #2: The "exec_mode" and "instances" settings are optional but highly recommended. Setting "exec_mode" to "cluster" enable's PM2's cluster mode. This will provide better performance in production as it allows PM2 to scale your site across multiple CPUs. The "instances" setting tells PM2 how many CPUs to scale across ("max" means all CPUs, but you can also specify a number.)
3. NOTE #3: If you wanted to configure your UI using Environment Variables, specify those Environment Variables under the "env" section. For example:

Configuration via Environment Variables

```
"env": {
   "NODE_ENV": "production",
   "DSPACE_REST_SSL": "true",
   "DSPACE_REST_HOST": "demo.dspace.org",
   "DSPACE_REST_PORT": "443",
   "DSPACE_REST_NAMESPACE": "/server"
}
```

4. NOTE #4: If you are using Windows, there are two other rules to keep in mind in this JSON configuration. First, *all paths must include double backslashes* (e.g. "C:\\dspace-ui-deploy"). Second, "cluster" mode is *required*. Here's an example configuration for Windows:

dspace-ui.json (for Windows)

```
{
    "apps": [
        {
           "name": "dspace-ui",
           "cwd": "C:\\full\\path\\to\\dspace-ui-deploy",
           "script": "dist\\server\\main.js",
           "instances": "max",
           "exec_mode": "cluster",
           "env": {
              "NODE_ENV": "production"
           }
        }
    ]
}
```

ii. Now, start the application using PM2 using the configuration file you created in the previous step

```
# In this example, we are assuming the config is named "dspace-ui.json"
pm2 start dspace-ui.json

# To see the logs, you'd run
# pm2 logs

# To stop it, you'd run
# pm2 stop dspace-ui.json

# If you need to change your PM2 configs, delete the old config and restart
# pm2 delete dspace-ui.json
```

iii. For more PM2 commands see https://pm2.keymetrics.io/docs/usage/quick-start/
iv. HINT: You may also want to install/configure pm2-logrotate to ensure that PM2's log folder doesn't fill up over time.
v. *Did* PM2 *not work or throw an immediate error?* It's likely that something in your UI installation or configuration is incorrect. Check the PM2 logs ("pm2 logs") first for errors. If the problem is not obvious, try to see if you can run the UI using the "Quick Start" method (using just Node.js) instead. Once "Quick Start" is working, try PM2 again.
vi. *If neither PM2 nor the "Quick Start" method works for you:* then see the "User Interface never appears (no content appears)" section in the Commons Installation Issues below

7. **Test it out:** At this point, the User Interface should be available at the URL you configured!
    a. For an example of what the default frontend looks like, visit the Demo Frontend: https://demo.dspace.org/
    b. If the UI fails to start or throws errors, it's likely a configuration issue. See Commons Installation Issues below for common error messages you may see and how to resolve them.
    c. If you have an especially difficult issue to debug, you may wish to *stop* PM2. Instead, try running the UI via the "Quick Start" method (using just Node.js). This command might provide a more specific error message to you, if PM2 is not giving enough information back.
8. **Add HTTPS support:** For HTTPS (port 443) support, you have two options
    a. *(Recommended)* Install either Apache HTTPD or Nginx to act as a "reverse proxy" for the frontend (and backend). This allows you to manage HTTPS (SSL certificates) in either Apache HTTPD or Nginx, and proxy all requests to the frontend (running on port 4000) and backend (running on port 8080). This is our current recommended approach. These instructions are specific to Apache, but a similar setup can be achieved with Nginx.
        i. If you already have Apache / Nginx installed for the backend, you can use the same Apache / Nginx. You can also choose to install a separate one (either approach is fine).
            1. Install Apache HTTPD, e.g. `sudo apt install apache2`
            2. Install the mod_proxy and mod_proxy_http modules, e.g. `sudo a2enmod proxy; sudo a2enmod proxy_http`
            3. Restart Apache to enable
            4. Obtain an SSL certificate for HTTPS support. If you don't have one yet, you can use Let's Encrypt (for free) using the "certbot" tool: https://certbot.eff.org/
        ii. *Apache HTTPD sample configuration:*

            1. Now, setup (or update) the new VirtualHost for your UI site (preferably using HTTPS / port 443) which proxies all requests to PM2 running on port 4000.

```
<VirtualHost _default_:443>
    # Add your domain here. We've added "dspace.example.edu" as an example
    ServerName dspace.example.edu
    .. setup your host how you want, including log settings...

    # Most installs will need these options enabled to ensure DSpace knows its
hostname and scheme (http or https)
    # Also required to ensure correct sitemap URLs appear in /robots.txt for User
Interface.
    ProxyPreserveHost On
    RequestHeader set X-Forwarded-Proto https

    # These SSL settings are identical to those for the backend installation (see
above)
    # If you already have the backend running HTTPS, just add the new Proxy settings
below.
    SSLEngine on
    SSLCertificateFile [full-path-to-PEM-cert]
    SSLCertificateKeyFile [full-path-to-cert-KEY]
    # LetsEncrypt certificates (and possibly others) may require a chain file be
specified
    # in order for the UI / Node.js to validate the HTTPS connection.
    #SSLCertificateChainFile [full-path-to-chain-file]

    # These Proxy settings are for the backend. They are described in the backend
installation (see above)
    # If you already have the backend running HTTPS, just append the new Proxy
settings below.
    # Proxy all HTTPS requests to "/server" from Apache to Tomcat via AJP connector
    # (In this example: https://dspace.example.edu/server/ will display the REST API)
    ProxyPass /server ajp://localhost:8009/server
    ProxyPassReverse /server ajp://localhost:8009/server

    # [NEW FOR UI:] Proxy all HTTPS requests from Apache to PM2 on localhost, port
4000
    # NOTE that this proxy URL must match the "ui" settings in your config.prod.yml
    # (In this example: https://dspace.example.edu/ will display the User Interface)
    ProxyPass / http://localhost:4000/
    ProxyPassReverse / http://localhost:4000/
</VirtualHost>
```

iii. *NGinx sample configuration*

1. Sample NGinx "server block" configuration. Keep in mind we are only providing basic example settings.

```
# Setup HTTPS access
server {
  listen 443 ssl;
  # Add your domain here. We've added "dspace.example.edu" as an example
  server_name dspace.example.edu;

  # Add your SSL certificate/key path here
  # NOTE: For LetsEncrypt, the certificate should be the full certificate chain file
  # These SSL settings are identical to those for the backend installation (see
above)
  # If you already have the backend running HTTPS, just add the new Proxy settings
below.
  ssl_certificate dspace.example.edu.crt (or PEM);
  ssl_certificate_key dspace.example.edu.key;

  # Proxy all HTTPS requests to "/server" from NGinx to Tomcat on port 8080
  # These Proxy settings are for the backend. They are described in the backend
installation (see above)
  location /server {
    proxy_set_header X-Forwarded-Proto https;
    proxy_set_header X-Forwarded-Host $host;
    proxy_pass http://localhost:8080/server;
  }

  # [NEW FOR UI:] Proxy all HTTPS requests from NGinx to PM2 on localhost, port 4000
  # NOTE that this proxy URL must match the "ui" settings in your config.prod.yml
  # (In this example: https://dspace.example.edu/ will display the User Interface)
  location / {
    proxy_set_header X-Forwarded-Proto https;
    proxy_set_header X-Forwarded-Host $host;
    proxy_pass http://localhost:4000/;
  }
}
```

iv. HINT#1: Because you are using a proxy for HTTPS support, in your User Interface Configuration, your "ui" settings will still have "ssl: false" and "port: 4000". This is perfectly OK!
v. HINT#2: to force the UI to connect to the backend using HTTPS, you should verify your "rest" settings in your User Interface Configuration **match** the "dspace.server.url" in your backend's "local.cfg" and both use the HTTPS URL. So, if your backend (REST API) is proxied to https://dspace.example.edu/server/, both those settings should specify that HTTPS URL.
vi. HINT#3: to force the backend to recognize the HTTPS UI, make sure to update your "dspace.ui.url" in your backend's "local.cfg" is updated to use the new HTTPS UI URL (e.g. https://dspace.example.edu).

b. *(Alternatively)* You can use the basic HTTPS support built into our UI and Node server. (This may currently be better for non-Production environments as it has not been well tested)
   i. Create a `[dspace-ui-deploy]/config/ssl/` folder and add a `key.pem` and `cert.pem` to that folder (they must have those exact names)
   ii. In your User Interface Configuration, go back and update the following:
      1. Set "ui > ssl" to true
      2. Update "ui > port" to be 443
         a. In order to run Node/PM2 on port 443, you also will likely need to provide node with special permissions, like in this example.
   iii. Restart the UI
   iv. Keep in mind, while this setup is simple, you may not have the same level of detailed, Production logs as you would with Apache HTTPD or Nginx

What Next?

After a successful installation, you may want to take a closer look at

- Performance Tuning DSpace: If you are noticing any slowness in your Production site, we have a guide for how you might speed things up.
- User Interface Customization: Documentation on customizing the User Interface with your own branding / theme(s)
- User Interface Configuration: Additional configurations available in the User Interface.
- Submission User Interface: Options to configure/customize the default Submission (deposit) process
- Configurable Workflow: Options to configure/customize the default Workflow approval process
- Scheduled Tasks via Cron : Several DSpace features **require** that a command-line script is run regularly via cron.
- Configuration Reference : Details on the configuration options available to the Backend
- Handle Server installation: Optionally, you may wish to enable persistent URLs for your DSpace site using CRNI's Handle.Net Registry
- Statistics and Metrics: Optionally, you may wish to configuration one (or more) Statistics options within DSpace, including Google Analytics and (internal) Solr Statistics
- Multilingual Support: Optionally, you may wish to enable multilingual support in your DSpace site.
- Using DSpace : Various other pages which describe usage and additional configurations related to other DSpace features.
- System Administration: Various other pages which describe additional backend installation options/configurations.

If you've run into installation problems, you may want to...

- Visit the Troubleshoot an error guide for tips on locating the cause of the error
- Review Commons Installation Issues (see below)
- Ask for Support via one of the support options documented on that page

Common Installation Issues

Troubleshoot an error or find detailed error messages

See the Troubleshoot an error guide, look for the section on "DSpace 7.x". This will provide you hints on locating error messages both in the User Interface (frontend) and in the REST API (backend)

User Interface never appears (no content appears) or "Proxy server received an invalid response"

Chances are your User Interface (UI) is throwing a severe error or not starting properly. The best way to debug this issue would be to start the User Interface in development mode to see if it can give you a more descriptive error.

1. First, create a `[dspace-ui-deploy]/config/config.dev.yml` configuration file for development. This file supports the same configs as your existing config.prod.yml. So, you can copy over any settings you want to test out.
2. Start the UI in development mode (this doesn't require a proxy like Apache or Nginx)

   ```
   yarn start:dev
   ```

3. This will boot up the User Interface on whatever port you specified in "config.dev.yml"
4. At this point, attempt to access the UI from your web browser. Even if it isn't fully working, you should be able to still get more information from your browser's DevTools regarding the underlying error. See the Troubleshoot an error page, look for the section on "DSpace 7.x". It has a guide for locating UI error messages in your browser's Developer Tools.

Once you've found the underlying error, it may be one of the "common installation issues" listed below.

User Interface partially load but then spins (never fully loads or some content doesn't load)

Chances are your User Interface (UI) is throwing an error or receiving an unexpected response from the REST API backend. Since the UI is Javascript based, it runs entirely in your browser. That means the error it's hitting is most easily viewed in your browser (and in fact the error may never appear in log files).

See the Troubleshoot an error page, look for the section on "DSpace 7.x". It has a guide for locating UI error messages in your browser's Developer Tools.

"500 Service Unavailable" from the User Interface

This error is saying that the frontend is working, but it is unable to communicate with your backend. It's the same as the "No _links section found at..." error described in the next section. Please follow the troubleshooting details in that section.

"No _links section found at..." error from User Interface

When starting up the User Interface for the first time, you may see an error that looks similar to this...

```
No _links section found at [rest-api-url]
ERROR Error: undefined doesn't contain the link sites
     at MapSubscriber.project
```

This error means that the UI is unable to contact the REST API listed at `[rest-api-url]` and/or the response from that `[rest-api-url]` is unexpected (as it doesn't contain the "_links" to the endpoints available at that REST API). A valid DSpace `[rest-api-url]` will respond with JSON similar to our demo API at https://demo.dspace.org/server/api

First, *test the connection to your REST API from the UI from the command-line.*

```
# This script will attempt a basic Node.js connection to the REST API
# configured in your "[dspace-angular]/config/config.prod.yml" and
# validate the response.(NOTE: config.prod.yml MUST be copied to
# to [dspace-angular]/config/ for this script to find it!)
yarn test:rest

# In DSpace 7.1 a different command was needed
# yarn config:check:rest
```

- A successful connection should return a 200 Response and all JSON validation checks should return "true".
- If you receive a connection error or different response code, you MUST fix your REST API before the UI will be able to work (see additional hints below for likely causes).

Usually, the core problem is caused by one of the following scenarios:

- *A possible configuration issue* in the frontend or backend.
  - Check the "rest" section of your `config.*.yml` (or `environment.*.ts` for 7.1 or 7.0) configuration file for the User Interface. That configuration section defines which REST API the UI will attempt to use. If the settings do NOT map to a valid DSpace REST API, then you will see this "No _links section found.." error. Keep in mind, **the REST API must use HTTPS** (the only exception is if both the frontend and backend are running on "localhost"-based URLs)
  - Check the "dspace.ui.url" configuration of your backend & verify it corresponds to the public URL of the User Interface (i.e. the exact same URL you use in your browser)
  - Verify the backend "trusts" the frontend via the "rest.cors.allowed-origins" configuration (in rest.cfg or local.cfg). This setting must list all web-based clients which are trusted by the backend (REST API). By default, "dspace.ui.url" should be listed... but you should verify it has not been modified.
- *A possible SSL certificate issue*. This issue may also appear if the REST API's SSL Certificate is either *untrusted (by the frontend) or expired*.
  - If you are using a Let's Encrypt style certificate, you may need to modify your backend's Apache settings to also provide a Chain File as follows:

    ```
    # For example: /etc/letsencrypt/live/[domain]/cert.pem
    SSLCertificateFile [full-path-to-PEM-cert]
    # For example: /etc/letsencrypt/live/[domain]/privkey.pem
    SSLCertificateKeyFile [full-path-to-cert-KEY]
    # For example: /etc/letsencrypt/live/[domain]/chain.pem
    SSLCertificateChainFile [full-path-to-chain-file]
    ```

  - Per the Apache docs, you can also use the SSLCertificateFile setting to specify intermediate CA certificates along with the main cert.
  - For self-signed certs, see also "Using a Self-Signed SSL Certificate causes the Frontend to not be able to access the Backend" common issue listed below.
- *Something blocking access to the REST API.* This may be a proxy issue, a firewall issue, or something else generally blocking the port (e.g. port 443 for SSL).
  - Verify that you can access the REST API from the machine where Node.js is running (i.e. your UI is running). For example try a simple "wget" or "curl" to verify the REST API is returning expected JSON similar to our demo API at https://demo.dspace.org/server/api

    ```
    # Attempt to access the REST API via HTTPS from command-line on the machine where Node.js is
    running.
    # If this fails or throws a SSL cert error, you must fix it.
    wget https://[rest.host]/server/api
    ```

  - **In most production scenarios, your REST API should be publicly accessible on the web**, unless you are guaranteed that all your DSpace users will access the site behind a VPN or similar. So, this "No _links section found" error may also occur if you are accessing the UI from a client computer/web browser which is *unable to access the REST API.*

If none of the above suggestions helped, you may want to look closer at the request logs in your browser (using browser's Dev Tools) and server-side logs, to be sure that the requests from your UI are going where you expect, and see if they appear also on the backend. Tips for finding these logs can be found in the "DSpace 7.x" section of our Troubleshoot an error guide.

"RangeError: Maximum call stack size exceeded"

When starting up the User Interface for the first time, you may see an error that looks similar to this...

```
ERROR RangeError: Maximum call stack size exceeded
```

This error means that the UI is trying to contact your REST API, but is having issues doing so (possibly because either a proxy or an HTTPHTTPS redirect is causing issues or a redirect loop).

Double check your "`dspace.server.url`" setting in your local.cfg on the backend. Is it the same URL you use in your browser to access the backend? Keep in mind the mode (http vs https), domain, port, and subpath(s) all must match, and it *must not end in a trailing slash.*

Also double check the "rest" section of your `config.*.yml` (or `environment.*.ts` for 7.1 or 7.0) configuration file for the User Interface. Make sure it's also pointing to the exact same URL as that "`dspace.server.url`" setting. Again, check the mode, domain, port and paths all match exactly.

"XMLHttpRequest.. has been blocked by CORS policy" or "CORS error" or "Invalid CORS request"

If you are seeing a CORS error in your browser, this means that you are accessing the REST API via an "untrusted" client application. To fix this error, you must change your REST API / Backend configuration to trust the application.

- By default, the DSpace REST API / Backend will only trust the application at `dspace.ui.url`. Therefore, you should first verify that your `dspace.ui.url` setting (in your local.cfg) exactly matches the *primary URL* of your User Interface (i.e. the URL you see in the browser). This must be an exact match: mode (http vs https), domain, port, and subpath(s) all must match.

- If you need to trust *additional* client applications / URLs, those MUST be added to the `rest.cors.allowed-origins` configuration. See REST API for details on this configuration.
- Also, check your Tomcat (or servlet container) log files. If Tomcat throws a syntax or other major error, it may return an error response that triggers a CORS error. In this scenario, the CORS error is only a side effect of a larger error.

If you modify either of the above settings, you will need to restart Tomcat for the changes to take effect.

Cannot login from the User Interface with a password that I know is valid

If you cannot login via the user interface with a valid password, you should check to see what underlying error is being returned by the REST API. The easiest way to do this is by using your web browser's Dev Tools as described in our Troubleshoot an error guide (see the "Try this first" section for DSpace 7).

If the password is valid, more than likely you'll see the underlying error is "403 Forbidden" error with a message that says "Access is denied. Invalid CSRF Token" (see hints on solving this in the very next section)

"403 Forbidden" error with a message that says "Access is denied. Invalid CSRF Token"

First, double check that you are seeing that exact error message. A `403 Forbidden` error may be thrown in a variety of scenarios. For example, a 403 may be thrown if a page requires a login, if you have entered an invalid username or password, or even sometimes when there is a CORS error (see previous installation issue for how to solve that).

If you are seeing the message "Invalid CSRF Token" message (especially on every login), this is usually the result of a configuration / setup issue.

Here's some things you should double check:

1. If you site had been working, and this error seems random, it is possibly that `DSPACE-XSRF-COOKIE` cookie in your browser just got "out of sync" (this can occur if you are logging into the REST API and UI separately in the same browser).
    a. Logout and login & try the same action again. If it works this time, then that cookie was just "out of sync". If it fails a second time, then there is a likely configuration issue...see suggestions below.
2. **Make sure your backend is running HTTPS**! This is the most common cause of this error. The only scenario where you can run the backend in HTTP is when both the frontend & backend URLs are "localhost"-based URLs.
    a. The reason for this HTTPS requirement is that most modern browsers will automatically block cross-domain cookies when using HTTP. Cross-domain cookies are *required* for successful authentication. The only *exception* is when both the frontend and backend are using localhost URLs (as in that scenario the cookies no longer need to be sent cross-domain). A more technical description of this behavior is in the sub-bullets below.
        i. If the REST API Backend is running HTTP, then it will always send the required `DSPACE-XSRF-COOKIE` cookie with a value of `SameSite=Lax`. This setting means that the cookie will *not* be sent (by your browser) to any other domains. Effectively, this will block all logins from any domain that is not the same as the REST API (as this cookie will not be sent back to the REST API as required for CSRF validation). In other words, running the REST API on HTTP is only possible if the User Interface is running on the exact same domain. For example, running both on 'localhost' with HTTP is a common development setup, and this will work fine.
        ii. In order to allow for cross-domain logins, you MUST enable HTTPS on the REST API. This will result in the `DSPACE-XSRF-COOKIE` cookie being set to `SameSite=None; Secure`. This setting means the cookie will be sent cross domain, but only for HTTPS requests. It also allows the user interface (or other client applications) to be on any domain, provided that the domain is trusted by CORS (see `rest.cors.allowed-origins` setting in REST API)
3. Verify that your User Interface's "rest" section matches the value of "`dspace.server.url`" configuration on the Backend. This simply ensures your UI is sending requests to the correct REST API. Also pay close attention that both specify HTTPS when necessary (see previous bullet).
4. Verify that your "`dspace.server.url`" configuration on the Backend matches the primary URL of the REST API (i.e. the URL you see in the browser). This must be an exact match: mode (http vs https), domain, port, and subpath(s) all must match, and it *must not end in a trailing slash* (e.g. "https://demo.dspace.org/server" is valid, but "https://demo.dspace.org/server/" may cause problems).
5. Verify that your "`dspace.ui.url`" configuration on the Backend matches the primary URL of your User Interface (i.e. the URL you see in the browser). This must be an exact match: mode (http vs https), domain, port, and subpath(s) all must match, and it *must not end in a trailing slash* (e.g. "https://demo.dspace.org" is valid, but "https://demo.dspace.org/" may cause problems).
6. Verify that nothing (e.g. a proxy) is blocking Cookies and HTTP Headers from being passed between the UI and REST API. DSpace's CSRF protection relies on the client (User Interface) being able to return both a valid `DSPACE-XSRF-COOKIE` cookie and a matching `X-XSRF-TOKEN` header back to the REST API for validation. See our REST Contract for more details https://github.com/DSpace/RestContract/blob/main/csrf-tokens.md
7. If you are running a custom application, or accessing the REST API from the command-line (or other third party tool like Postman), you MUST ensure you are sending the CSRF token on every modifying request. See our REST Contract for more details https://github.com/DSpace/RestContract/blob/main/csrf-tokens.md

For additional information on how DSpace's CSRF Protection works, see our REST Contract at https://github.com/DSpace/RestContract/blob/main/csrf-tokens.md

Using a Self-Signed SSL Certificate causes the Frontend to not be able to access the Backend

If you setup the backend to use HTTPS with a self-signed SSL certificate, then Node.js (which the frontend runs on) may not "trust" that certificate by default. This will result in the Frontend not being able to make requests to the Backend.

One possible workaround (untested as of yet) is to try setting the NODE_EXTRA_CA_CERTS environment variable (which tells Node.js to trust additional CA certificates).

```
# May be necessary for self-signed certificates.
export NODE_EXTRA_CA_CERTS="/etc/ssl/my.dspace.pem"
```

Another option is to avoid using a self-signed SSL certificate. Instead, create a real, issued SSL certificate using something like Let's Encrypt (or similar free services)

My REST API is running under HTTPS, but some of its "link" URLs are switching to HTTP

This scenario may occur when you are running the REST API behind an HTTP proxy (e.g. Apache HTTPD's `mod_proxy_http`, Ngnix's `proxy_pass` or any other proxy that is forwarding from HTTPS to HTTP).

The fix is to ensure the DSpace REST API is sent the `X-Forwarded-Proto` header (by your proxying service), telling it that the forwarded protocol is HTTPS

```
X-Forwarded-Proto: https
```

In general, when running behind a proxy, the DSpace REST API depends on accurate X-Forwarded-* headers to be sent by that proxy.

My User Interface's robots.txt has incorrect sitemap URLs

This scenario may occur when you are running the User Interface behind an HTTP proxy (e.g. Apache HTTPD's `mod_proxy_http`, Ngnix's `proxy_pass` or any other proxy that is forwarding from HTTPS to HTTP).

The fix is to ensure the DSpace User Interface (frontend) is sent the correct `X-Forwarded-Proto` and `Host` (or X-Forwarded-Host) headers to tell it the correct hostname and scheme (HTTP or HTTPS)

Apache HTTPD example

```
ProxyPreserveHost on
RequestHeader set X-Forwarded-Proto https
```

Cannot upload file from User Interface

If everything seems to be working, but you cannot upload files, it's important to first check your logs for any possible backend errors. See the Troubleshoot an error page.

If you are running DSpace on a Debian-based system (e.g. Ubuntu), some users have reported that it's **required** grant "ReadWrite" access to Apache Tomcat (where the backend is running) via the service file (e.g. /lib/systemd/system/tomcat9.service). In the `[Service]` section you need to add something like this:

```
# Give Tomcat read/write on the DSpace installation
# Make sure to update the "/PATH/TO" to be the full path of your DSpace install
ReadWritePaths=/PATH/TO/dspace

# NOTE: If you don't want to give Tomcat read/write to all of DSpace,
# you could limit this further to just these folders
# dspace/assetstore
# dspace/solr
# dspace/log
```

Alternatively, instead of modifying the /lib/systemd/system/tomcat9.service file, consider creating a systemd override configuration file so that future upgrades of the tomcat service via apt-get do not overwrite the above changes.

To create the /etc/systemd/system/tomcat9.service.d/override.conf file, on the commandline, type:

sudo systemctl edit tomcat9.service

Content of the file:

[Service]

ReadWritePaths=/PATH/TO/dspace/

Javascript heap out of memory

On some versions of Node.js or some operating systems, sites have reported seeing a "Javascript heap out of memory" error when trying to run the User Interface *in development mode* (`yarn start:dev`). This does not seem to occur on every system, but the fix is always the same. You should ensure that Node.js is given *at least* 4GB of memory via the "NODE_OPTIONS" environment variable

```
# Set the "NODE_OPTIONS" environment variable on your system. This example will work for Linux/macOS
# Ensure the "max-old-space-size" is set to 4GB (4096MB) or greater.
export NODE_OPTIONS=--max-old-space-size=4096
```

NOTE: More discussion on this issue can be found in https://github.com/DSpace/dspace-angular/issues/2259 It appears to only occur on systems where the default memory allocated for Node isn't sufficient to build DSpace in development mode.

This same setting may also be used in production scenarios to give Node.js more memory to work with. See Performance Tuning DSpace for more details.

Solr responds with "Expected mime type application/octet-stream but got text/html" (404 Not Found)

This error occurs when Solr is either not initialized properly, or your DSpace backend is unable to find/communicate with Solr. Here's a few things you should double check:

1. Verify that Solr is running and/or check for errors in its logs. Try to restart it (usually via a command like `[solr]/bin/solr restart`), and verify it's accessible via wget or a web browser (usually at a URL like `http://localhost:8983/solr`)
2. Verify that your `solr.server` setting (in local.cfg) is correct for your Solr installation. This should correspond to the main URL of your Solr site (usually something like `http://localhost:8983/solr`). If you use `wget` or a browser from the machine running your DSpace backend, you should get a response from that URL (it should return the Solr Admin UI).
3. Verify that the required DSpace Solr cores have been properly installed/configured (per installation instructions above). When properly installed, you should be able to get a response from them. For example, the URL `${solr.server}/search/select` should run an empty query against the "search" core, returning an empty JSON result.
4. If Solr is running & you are sure `solr.server` is set properly, double check that nothing else could be blocking the DSpace backend from accessing Solr. For instance, if Solr is on a separate machine, verify that there is no firewall or proxy that could be blocking access between the DSpace backend machine and the Solr machine.

Database errors occur when you run `ant fresh_install`

There are two common errors that occur.

- If your error looks like this:

```
[java] 2004-03-25 15:17:07,730 INFO
            org.dspace.storage.rdbms.InitializeDatabase @ Initializing Database
[java] 2004-03-25 15:17:08,816 FATAL
            org.dspace.storage.rdbms.InitializeDatabase @ Caught exception:
[java] org.postgresql.util.PSQLException: Connection refused. Check
            that the hostname and port are correct and that the postmaster is
            accepting TCP/IP connections.
[java]     at
            org.postgresql.jdbc1.AbstractJdbc1Connection.openConnection(AbstractJd
bc1Connection.java:204)
[java]     at org.postgresql.Driver.connect(Driver.java:139)
```

  it usually means you haven't yet added the relevant configuration parameter to your PostgreSQL configuration (see above), or perhaps you haven't restarted PostgreSQL after making the change. Also, make sure that the *db.username* and *db.password* properties are correctly set in *[dspace]/config/dspace.cfg*. An easy way to check that your DB is working OK over TCP/IP is to try this on the command line:

```
psql -U dspace -W -h localhost
```

  Enter the *dspace* database password, and you should be dropped into the psql tool with a *dspace*=> prompt.
- Another common error looks like this:

```
[java] 2004-03-25 16:37:16,757 INFO
            org.dspace.storage.rdbms.InitializeDatabase @ Initializing Database
[java] 2004-03-25 16:37:17,139 WARN
            org.dspace.storage.rdbms.DatabaseManager @ Exception initializing DB
            pool
[java] java.lang.ClassNotFoundException: org.postgresql.Driver
[java]     at java.net.URLClassLoader$1.run(URLClassLoader.java:198)
[java]     at java.security.AccessController.doPrivileged(Native
           Method)
[java]     at
           java.net.URLClassLoader.findClass(URLClassLoader.java:186)
```

  This means that the PostgreSQL JDBC driver is not present in *[dspace]/lib*. See above.

7.0-7.1 Frontend Installation

Frontend (UI) Installation Guide for 7.0 or 7.1 ONLY

In DSpace 7.0 and 7.1, the Frontend installation required configuring your UI before you could build the UI. This required a slightly different installation process, and this installation process required a full rebuild whenever configurations changed. The configuration file and several scripts also had different names. This page provides a guide for those who are still on 7.0 or 7.1.

This guide will not work for 7.2!

If you are running DSpace 7.2 (and later), you MUST instead following the new Frontend Installation process documented at Installing DSpace. The new Frontend Installation instructions are easier and also allow you to rebuild/redeploy with minimal downtime.

- 7.0/7.1 Frontend Requirements
- 7.0/7.1 Frontend Installation

7.0/7.1 Frontend Requirements

- UNIX-like OS or Microsoft Windows
- Node.js (v12.x or v14.x)
- Yarn (v1.x)
- PM2 (or another Process Manager for Node.js apps) (optional, but recommended for Production)
- DSpace 7.x Backend (see above)

UNIX-like OS or Microsoft Windows

- UNIX-like operating system (Linux, HP/UX, Mac OSX, etc.) : Many distributions of Linux/Unix come with some of the dependencies below pre-installed or easily installed via updates. You should consult your particular distribution's documentation or local system administrators to determine what is already available.
- Microsoft Windows: While DSpace can be run on Windows servers, most institutions tend to run it on a UNIX-like operating system.

Node.js (v12.x or v14.x)

- Node.js can be found at https://nodejs.org/. It may be available through your Linux distribution's package manager. We recommend running a Long Term Support (LTS) version (even numbered releases). Non-LTS versions (odd numbered releases) are not recommended.
- Node.js is a Javascript runtime that also provides npm (Node Package Manager). It is used to both build and run the frontend.

Yarn (v1.x)

- Yarn v1.x is available at https://classic.yarnpkg.com/. It can usually be install via NPM (or through your Linux distribution's package manager). *We do NOT currently support Yarn v2.*

```
# You may need to run this command using "sudo" if you don't have proper privileges
npm install --global yarn
```

- Yarn is used to build/install the frontend.

PM2 (or another Process Manager for Node.js apps) (*optional, but recommended for Production*)

- In Production scenarios, we *highly recommend* starting/stopping the User Interface using a Node.js process manager. There are several available, but our current favorite is PM2. The rest of this installation guide assumes you are using PM2.
- PM2 is very easily installed via NPM

```
# You may need to run this command using "sudo" if you don't have proper privileges
npm install --global pm2
```

DSpace 7.x Backend (see above)

- The DSpace User Interface (Frontend) cannot function without an installed DSpace Backend. Follow the instructions above.
- The Frontend and Backend *do not need to be installed on the same machine/server*. They may be installed on separate machines as long as the two machines can connect to one another via HTTP or HTTPS.

7.0/7.1 Frontend Installation

1. First, install all the Frontend Requirements above & verify the backend/REST API is publicly accessible.
2. Download the latest dspace-angular release from the DSpace GitHub repository. You can choose to either download the zip or tar.gz file provided by GitHub, or you can use "git" to checkout the appropriate tag (e.g. `dspace-7.0 or dspace-7.1`) or branch.
3. Install all necessary local dependencies by running the following from within the unzipped "dspace-angular" directory

```
# change directory to our repo
cd dspace-angular

# install the local dependencies
yarn install
```

4. Create a Production Configuration file at *`[dspace-angular]/src/environments/environment.prod.ts`*. You may wish to use the environment.common.ts as a starting point. This `environment.prod.ts` file can be used to override any of the default configurations listed in the environment.common.ts (in that same directory). *At a minimum* this file MUST include a "rest" section (and may also include a "ui" section), similar to the following (keep in mind, you only need to include settings that you need to modify).

Syntax for 7.1 or 7.0 (environment.prod.ts)

```
export const environment = {
  // The "ui" section defines where you want Node.js to run/respond. It often is a *localhost* (non-
public) URL, especially if you are using a Proxy.
  // In this example, we are setting up our UI to just use localhost, port 4000.
  // This is a common setup for when you want to use Apache or Nginx to handle HTTPS and proxy requests
to Node on port 4000
  ui: {
      ssl: false,
      host: 'localhost',
      port: 4000,
      // NOTE: Space is capitalized because 'namespace' is a reserved string in TypeScript
      nameSpace: '/'
  },
  // This example is valid if your Backend is publicly available at https://api.mydspace.edu/server/
  // The REST settings MUST correspond to the primary URL of the backend. Usually, this means they must
be kept in sync
  // with the value of "dspace.server.url" in the backend's local.cfg
  rest: {
      ssl: true,
      host: 'api.mydspace.edu',
      port: 443,
      // NOTE: Space is capitalized because 'namespace' is a reserved string in TypeScript
      nameSpace: '/server'
  }
};
```

   a. (Optionally) *Test the connection to your REST API from the UI from the command-line.* This is not required, but it can sometimes help you discover immediate configuration issues if the test fails.
      i. In DSpace 7.1, this could be tested by running `yarn config:check:rest` This script will attempt a basic Node.js connection to the REST API that is configured in your "environment.prod.ts" file and validate the response.
      ii. A successful connection should return a 200 Response and all JSON validation checks should return "true"
      iii. If you receive a connection error or different response code, you MUST fix your REST API before the UI will be able to work. See also the Commons Installation Issues. If you receive an SSL error, see "Using a Self-Signed SSL Certificate causes the Frontend to not be able to access the Backend"
   b. HINT #1: In the "ui" section above, you may wish to start with "ssl: false" and "port: 4000" just to be certain that everything else is working properly. With those settings, you can quickly test your UI by running "`yarn start`" and trying to access it via `http://[mydspace.edu]:4000/` from your web browser. KEEP IN MIND, we highly recommend always using HTTPS for Production.
   c. HINT #2: If Node throws an error saying "listen EADDRNOTAVAIL: address not available", try setting the "host" to "0.0.0.0" or "localhost". Usually that error is a sign that the "host" is not recognized.
   d. If there are other settings you know you need to modify in the sample `environment.common.ts` configuration file you can also copy them into this same file.
5. Build the User Interface for Production. This uses your `environment.common.ts` and the source code to create a compiled version of the UI in the `[dspace-angular]/dist` folder

```
yarn run build:prod
```

   a. In 7.1 or 7.0: anytime you change/update your `environment.prod.ts`, *then you will need to rebuild* the UI application (i.e. rerun this" yarn run build:prod" command).
6. Assuming you are using PM2, create a JSON configuration file describing how to run our UI application. This need NOT be in the same directory as the dspace-angular codebase itself (in fact you may want to put the parent directory or another location). Keep in mind the "cwd" setting (on line 5) must be the full path to your `[dspace-angular]` folder.

dspace-angular.json

```
{
    "apps": [
        {
           "name": "dspace-angular",
           "cwd": "/home/dspace/dspace-angular",
           "script": "yarn",
           "args": "run serve:ssr",
           "interpreter": "none"
        }
    ]
}
```

a. *Not using PM2?* That's OK. The key command that your process manager should run is `yarn run serve:ssr`. This is the command that starts the app (after it was built using `yarn run build:prod`)
b. *Using Windows?* This "dspace-angular.json" file needs to have a slightly different structure on Windows. First, *all paths must include double backslashes* (e.g. C:\\dspace-angular). Second, "cluster" mode is required. Finally, because of a known issue in PM2, you must point the "script" at the "npm/node_modulesyarn/bin/yarn.js" file directly. So, here's how this configuration looks on Windows platforms:

dspace-angular.json (For Windows only)

```
{
    "apps": [
        {
           "name": "dspace-angular",
           "cwd": "C:\\path\\to\\dspace-angular",
           "script": "C:\\path\\to\\npm\\node_modules\\yarn\\bin\\yarn.js",
           "args": "run serve:ssr",
           "interpreter": "none",
           "exec_mode": "cluster"
        }
    ]
}
```

7. Now, start the application using PM2 using the configuration file you created in the previous step

```
# In this example, we are assuming the config is named "dspace-angular.json"
pm2 start dspace-angular.json

# To see the logs, you'd run
# pm2 logs

# To stop it, you'd run
# pm2 stop dspace-angular.json
```

a. For more PM2 commands see https://pm2.keymetrics.io/docs/usage/quick-start/
b. HINT: You may also want to install/configure pm2-logrotate to ensure that PM2's log folder doesn't fill up over time.
c. *Did* PM2 *not work or throw an immediate error?* It's likely that something in your UI installation or configuration is incorrect. Check the PM2 logs ("pm2 logs") first for errors. If the problem is not obvious, try to see if you can run the UI using "yarn start" (this builds & starts the app in one step). Once you successfully get the UI running via "yarn start", you should be able to go back to using PM2 successfully.

8. At this point, the User Interface should be available at the URL you configured in your `environment.prod.ts`
    a. For an example of what the default frontend looks like, visit the Demo Frontend: https://demo7.dspace.org/
    b. If the UI fails to start or throws errors, it's likely a configuration issue. See Commons Installation Issues for common error messages you may see and how to resolve them.
    c. If you have an especially difficult issue to debug, you may wish to *stop* PM2. Instead, try running the UI via `yarn start` (which is a simple build & deploy process for the UI). This command might provide a more specific error message to you, if PM2 is not giving enough information back.
9. For HTTPS (port 443) support, you have two options
    a. (Recommended) You can install either Apache HTTPD or Nginx , configuring SSL at that level, and proxy requests to PM2 (on port 4000). This is our current recommended approach. Plus, as a bonus, if you want to host the UI and Backend on the same server, you can use just one Apache HTTPD (or Nginx) to proxy to both. These instructions are specific to Apache.
        i. Install Apache HTTPD, e.g. `sudo apt install apache2`
        ii. Install the mod_proxy and mod_proxy_http modules, e.g. `sudo en2mod proxy; sudo a2enmod proxy_http`
        iii. Restart Apache to enable
        iv. Now, setup a new VirtualHost for your site (preferably using HTTPS / port 443) which proxies all requests to PM2 running on port 4000.

```
<VirtualHost _default_:443>
    .. setup your host how you want, including log settings...

    SSLEngine on
    SSLCertificateFile [full-path-to-PEM-cert]
    SSLCertificateKeyFile [full-path-to-cert-KEY]

    # Proxy all HTTPS requests from Apache to PM2 on port 4000
    # NOTE that this proxy URL must match the "ui" settings in your config.prod.yml
    ProxyPass / http://localhost:4000/
    ProxyPassReverse / http://localhost:4000/
</VirtualHost>
```

b. (Alternatively) You can use the basic HTTPS support built into dspace-angular node server. (This may currently be better for non-Production environments as it has not been well tested)
    i. Create a `[dspace-angular]/config/ssl/` folder and add a `key.pem` and `cert.pem` to that folder (they must have those exact names)
    ii. Enable "ui.ssl" (set to true)
    iii. Update your "ui.port" to be 443
        1. In order to run Node/PM2 on port 443, you also will likely need to provide node with special permissions, like in this example.
    iv. Rebuild and then restart the app in PM2
    v. Keep in mind, while this setup is simple, you may not have the same level of detailed, Production logs as you would with Apache HTTPD or Nginx

10. Additional UI configurations are described in User Interface Configuration. A guide to customizing the look and feel or branding via a Theme is also available in User Interface Customization

Upgrading DSpace

What versions does this guide cover?

These instructions are valid for any of the following upgrade paths:

- *Upgrading ANY prior version (1.x.x, 3.x, 4.x, 5.x, 6.x or 7.x) of DSpace to DSpace 7.x (latest version)*

For more information about new features or major changes in previous releases of DSpace, please refer to following:

- Releases - Provides links to release notes for all prior releases of DSpace
- Version History - Provides detailed listing of all changes in all prior releases of DSpace

Should I upgrade or migrate my current installation?

DSpace offers two approaches to getting your installation up-to-date.

- Upgrading to the latest release (covered by this guide). This provides a walkthrough of how to install the latest version of the code *over top* of your existing DSpace installation, in order to upgrade to the latest version.
- Migrating to the latest version. This separate guide provides a walkthrough of installing a new copy of DSpace and migrating your existing production data into it. This approach may be more useful if you wish to move your DSpace to a different server, or want to start "fresh" with the same data.

The approach you choose is up to you. Upgrading is often easiest for minor upgrades (e.g. 7.x latest 7.x). Migrating may be useful for major upgrades (e.g. 7.x 8.x), or if you need to move your DSpace installation.

Please refrain from customizing the DSpace database tables. It will complicate your next upgrade!

As DSpace automatically upgrades your database structure (using FlywayDB migrations), *we highly recommend AGAINST customizing the DSpace database tables/structure or backporting any features that change the DSpace tables/structure*. Doing so will often cause the automated database upgrade process to fail (and therefore will complicate your next upgrade).

If you must add features requiring new database tables/structure, we recommend creating new tables (instead of modifying existing ones), as that is usually much less disruptive to our automated database upgrade.

Test Your Upgrade Process

In order to minimize downtime, it is always recommended to first perform a DSpace upgrade using a Development or Test server. You should note any problems you may have encountered (and also how to resolve them) before attempting to upgrade your Production server. It also gives you a chance to "practice" at the upgrade. Practice makes perfect, and minimizes problems and downtime. Additionally, if you are using a version control system, such as git, to manage your locally developed features or modifications, then you can do all of your upgrades in your local version control system on your Development server and commit the changes. That way your Production server can checkout your well tested and upgraded code.

In the notes below `[dspace]` refers to the install directory for your existing DSpace installation, and `[dspace-source]` to the source directory for DSpace. Whenever you see these path references, be sure to replace them with the actual path names on your local system.

1 Release Notes / Significant Changes
2 Upgrading the Backend (Server API)
  2.1 Backup your DSpace Backend
  2.2 Update Backend Prerequisite Software
  2.3 Upgrading the Backend Steps
3 Upgrading the Frontend (User Interface)
4 Troubleshooting Upgrade Issues
  4.1 See all "Common Installation Issues"
  4.2 Database migrate errors: "Migration V5.7_2017.04.11__DS-3563_Index_metadatavalue_resource_type_id_column.sql failed" or "Migration V5.7_2017.05.05__DS-3431.sql failed"
  4.3 Database migrate errors because of custom database tables/columns
  4.4 Running "Ignored" Flyway Migrations
  4.5 Manually updating the Metadata Registries

Release Notes / Significant Changes

DSpace 7.6.1 only provided improvements and bug fixes. See the Release Notes for all the details. A few fixes to be aware of when you upgrade:

- Git is no longer required to build DSpace. You can remove Git from your system unless you require Git for your own local version control.
- Hierarchical browse indexes for controlled vocabularies can now be disabled via a backend configuration. See Configuration Reference#HierarchicalBrowseIndexes for details.

DSpace 7.6 added some new features and many improvements and bug fixes. See the Release Notes for all the details. These major features are most likely to impact your upgrade:

- **Oracle support has been removed** as was previously announced in March 2022 on our mailing lists.
- **Item counts can now be displayed for all Communities/Collections** *similar to version 6.x.*
- **New default Privacy Statement and End User Agreement** . The new default text of these policies can be found by visiting the links in the footer of our demo site.

DSpace 7.5 added some new features and many improvements and bug fixes. See the Release Notes for all the details. These major features are most likely to impact your upgrade:

- **Subscribe to email updates from a Community or Collection** - This REQUIRES enabling a new "subscription-send" Scheduled Tasks via Cron.
- **New "dspace database skip" command** - Useful tool for some sites upgrading from 6.x or below *if* they encounter this common migration error
- **User interface now provides page caching for better performance** - Sites which have encountered slower UI performance may wish to use the new settings in Cache Settings - Server Side Rendering(SSR)
- **User interface now requires Node.js 16 or 18.** While it may work for other versions, it's best to no longer use Node.js 14, as it goes EOL in April 2023

DSpace 7.4 added some new features and many improvements and bug fixes. See the Release Notes for all the details. These major features are most likely to impact your upgrade:

- **Recent Submissions are now listed on the homepage.** (See "recentSubmissions" options)
- **Thumbnails are now displayed** in all search/browse screens. (See "showThumbnails" option)

DSpace 7.3 added many new features and improvements. See the Release Notes for all the details. These major features are most likely to impact your upgrade:

- **Oracle Database support has been deprecated.** It will be removed in mid-2023 (likely in the 7.6 release, which is tentative for June 2023). All sites should plan to migrate to PostgreSQL. See https://github.com/DSpace/DSpace/issues/8214
- **ORCID Authentication and synchronization to a DSpace Researcher Profile** now exists (requires first enabling Configurable Entities). See ORCID Integration and Researcher Profiles
- **Admin "Health" menu provides basic control panel functionality** *(based on 6.x Control Panel).* When logged in as an Administrator, select "Health" from the side menu. You'll see a "Status" tab which provides useful information about the status of the DSpace backend, and an "Info" tab which provides an overview of backend configurations and Java information.
- **Media Filter updates to use Apache Tika.** There is now a single "Text Extractor" media filter which you should use (see updated settings in dspace.cfg and Mediafilters for Transforming DSpace Content for more details).

DSpace 7.2 made some major changes to the User Interface build process, including a new configuration file format. See the Release Notes for all the details. These major features are most likely to impact your upgrade:

- **New Configuration for User Interface to support Runtime Configuration:** *In the User Interface, the "environment.*.ts" configuration files have been replaced with a new "config.*.yml" file.* A migration script is provided which can migrate your UI configurations from the old format to the new one. More information on that migration is available in the User Interface Configuration documentation.
- **Submission Process now supports Item Embargoes / access restrictions.** It is disabled by default, but can be easily enabled by uncommenting (or adding) the "itemAccessConditions" step in your item-submission.xml on the backend. See Submission User Interface and Embargo for more details.
- **Feedback Form** now exists. It is enabled by default in the UI's footer as long as you set a "feedback.recipient" in your local.cfg on the backend.
- **OpenID Connect (OIDC) Authentication Plugin** now exists. See the Authentication Plugins page for how to enable it.
- **Improved support for custom "Browse By" configurations.** If you had previously configured custom "Browse by" types in your UI configuration file, those settings can be removed. The "Browse by" types are now read dynamically from the REST API based on configured indexes. See User Interface Configuration for more details.

DSpace 7.1 primarily added new features and bug fixes on top of 7.0. See the Release Notes for all the details. A few key changes to be aware of:

- **New Collection "Entity Type" Configuration**: If you were using Configurable Entities in 7.0, the Entity Type to Collection "mapping" has been moved to the Collection's "Edit Metadata" screen as a new "Entity Type" dropdown. This formalizes the recommended mapping between a Collection and an Entity Type (so we highly recommend each Entity Type be stored in its own Collection). See the "Configure Collections for each Entity type" section of the Configurable Entities documentation for more details.

DSpace 7.0 features some significant changes which you may wish to be aware of before beginning your upgrade:

- **XMLUI and JSPUI are no longer supported or distributed with DSpace.** All users should install and utilize the new Angular User Interface. See the "Installing the Frontend (User Interface)" instructions in Installing DSpace
- **The old REST API ("rest" webapp from DSpace v4.x-6.x) is deprecated and will be removed in v8.x.** The new REST API (provided in the "server" webapp) replaces all functionality available in the older REST API. If you have tools that rely on the old REST API, you can still (optionally) build & deploy it alongside the "server" webapp via the "-Pdspace-rest" Maven flag. See REST API v6 (deprecated)
- **Solr must be installed separately** due to changes in the packaging of recent Solr releases. The indexes have been reconfigured and must be rebuilt. See below.
- **GeoIP location database must be installed separately** due to changes in Maxmind's terms and conditions. MaxMind has changed the terms and procedure for obtaining and using its GeoLite2 location database. Consequently, DSpace no longer automatically downloads the database during installation or update, and the DSpace-specific database update tool has been removed. If you wish to (continue to) record client location data in SOLR Statistics, you will need to make new arrangements. See below.
- **The Submission Form configuration has changed.** The "item-submission.xml" file has changed its structure, and the "input-forms.xml" has been replaced by a "submission-forms.xml". See Submission User Interface
- **The traditional, 3-step Workflow system has been removed in favor of the Configurable Workflow System .** For most users, you should see no effect or difference. The default setup for this Configurable Workflow System is identical to the traditional, 3-step workflow ("Approve/Reject", "Approve/Reject/Edit Metadata", "Edit Metadata")
- **The old BTE import framework in favor of Live Import Framework** (features of BTE have been ported to Live Import)
- **ElasticSearch Usage Statistics have been removed.** Please use SOLR Statistics or DSpace Google Analytics Statistics.
- **Configuration has been upgraded to Apache Commons Configuration version 2.** For most users, you should see no effect or difference. *No DSpace configuration files were modified during this upgrade and no configurations or settings were renamed or changed.* However, if you locally modified or customized the `[dspace]/config/config-definition.xml` (DSpace's Apache Commons Configuration settings), you will need to ensure those modifications are compatible with Apache Commons Configuration version 2. See the Apache Commons Configuration's configuration definition file reference for more details.

- **Handle server has been updated to v9.3.** Most users will see no effect or difference, however a minor change to the Handle Server configuration is necessary, see below.
- **Many backend prerequisites have been upgraded to avoid "end of life" versions.** Therefore, pay very close attention to the required prerequisites listed below.
- **A large number of old/obsolete configurations were removed.** "7.0 Configurations Removed" section in the Release Notes.

Upgrading the Backend (Server API)

Backup your DSpace Backend

Before you start your upgrade, it is strongly recommended that you create a backup of your DSpace content. Backups are easy to recover from; a botched install/upgrade is very difficult if not impossible to recover from. The DSpace specific things to backup are: configs, source code modifications, database, and assetstore. On your server that runs DSpace, you might additionally consider checking on your cron/scheduled tasks, servlet container, and database.

Make a complete backup of your system, including:

- Database: Make a snapshot/dump of the database. For the PostgreSQL database use Postgres' pg_dump command. For example:

```
pg_dump -U [database-user] -f [backup-file-location] [database-name]
```

- Assetstore: Backup the directory (`[dspace]/assetstore` by default, and any other assetstores configured in `[dspace]/config/spring/api/bitstore.xml`)
- Configuration: Backup the entire directory content of `[dspace]/config`.
- Customizations: If you have custom code, such as themes, modifications, or custom scripts, you will want to back them up to a safe location.
- Statistics data: what to back up depends on what you were using before: the options are the default SOLR Statistics, or the legacy statistics. Legacy stats utilizes the dspace.log files, while SOLR Statistics stores data in `[dspace]/solr/statistics`. A simple copy of the logs or the Solr core directory tree should give you a point of recovery, should something go wrong in the update process. We can't stress this enough: your users depend on these statistics more than you realize. You need a backup.
- Authority data: stored in `[dspace]/solr/authority`. As with the statistics data, making a copy of the directory tree should enable recovery from errors.

Update Backend Prerequisite Software

DSpace 7.x requires the following versions of prerequisite software:

- **Updated:** Java 11 (Oracle or OpenJDK)
- **Updated:** Apache Maven 3.3.x or above
- **Updated:** Apache Ant 1.10.x or above
- Database
  - **Updated:** PostgreSQL 11.x, 12.x or 13.x (with pgcrypto installed), OR
  - **Deprecated:** Oracle 10g or above (Oracle support is deprecated, see https://github.com/DSpace/DSpace/issues/8214)
- **Updated:** Tomcat 9.x
- **New:** Solr 8.x. *See Installing new Solr cores when upgrading from DSpace 6.x or below.*
- **New:** (optional) IP to City Database for location-based Statistics (either MaxMind's GeoLite City Database or DB-IP's City Lite Database). *See Set up IP to City Database.*

Refer to the Backend Requirements section of "Installing DSpace" for more details around configuring and installing these prerequisites.

Upgrading the Backend Steps

Migration guide also available

If during the upgrade you are migrating your DSpace backend to a new server/machine, see Migrating DSpace to a new server guide for hints/tips.

1. **Download** the latest DSpace release from the DSpace GitHub Repository. You can choose to either download the zip or tar.gz file provided by GitHub, or you can use "git" to checkout the appropriate tag (e.g. `dspace-7.2`) or branch.
    a. Unpack it using "unzip" or "gunzip". If you have an older version of DSpace installed on this same server, you may wish to unpack it to a different location than that release. This will ensure no files are accidentally overwritten during the unpacking process, and allow you to compare configs side by side.
    b. For ease of reference, we will refer to the location of this unzipped version of the DSpace release as *[dspace-source]* in the remainder of these instructions.
2. **If upgrading from 6.x or below, a few extra steps are required *before* you install DSpace 7.x.** *If you are upgrading from a previous version of 7.x, skip this and move along.*
    a. **Ensure that your database is compatible:** Starting with DSpace 6.x, there are new database requirements for DSpace (refer to the Backend Requirements section of "Installing DSpace" for full details).
        i. *PostgreSQL databases*: PostgreSQL 9.4 or above is required and the "pgcrypto" extension must be installed.
            1. Notes on installing pgcrypto
                a. On most Linux operating systems (Ubuntu, Debian, RedHat), this extension is provided in the "postgresql-contrib" package in your package manager. So, ensure you've installed "postgresql-contrib".
                b. On Windows, this extension should be provided automatically by the installer (check your "[PostgreSQL]/share/extension" folder for files starting with "pgcrypto")
            2. Enabling pgcrypto on your DSpace database. (Additional options/notes in the Installation Documentation)

```
# Login to your "dspace" database as a superuser
psql --username=postgres dspace
# Enable the pgcrypto extension on this database
CREATE EXTENSION pgcrypto;
```

ii. *Oracle databases: Oracle support has been deprecated in DSpace.* It will no longer be supported as of June/July 2023. See https://github.com/DSpace/DSpace/issues/8214 for more details.

b. **From your old version of DSpace, dump your `authority` and `statistics` Solr cores.** (Only necessary if you want to keep both your authority records and/or SOLR Statistics)

```
[dspace]/bin/dspace solr-export-statistics -i authority
[dspace]/bin/dspace solr-export-statistics -i statistics
```

The dumps will be written to the directory `[dspace]/solr-export`. *This may take a long time and require quite a lot of storage.* In particular, the statistics core is likely to be huge, perhaps double the size of the content of `solr/statistics/data`. You should ensure that you have sufficient free storage.

This is not the same as the disaster-recovery backup that was done above. These dumps will be reloaded into new, reconfigured cores later.

If you were **sharding** your statistics data, you will need to dump each shard separately. The index names for prior years will be `statistics-YYYY` (for example: `statistics-2017 statistics-2018` etc.) The current year's statistics shard is named `statistics` and you should dump that one too.

Unfortunately, the "solr-export-statistics" script was not created until DSpace 5.x. Therefore, you will not be able to upgrade statistics from 4.x or below *unless you first upgrade to either 5.x or 6.x.* This upgrade could be done in a test environment, to allow you to export your statistics (so they can be reimported into 7.x below). But, there's unfortunately no direct way to migrate 4.x (or 3.x or 1.x.x) Solr Statistics into 7.x.

c. **Move your old Solr cores to a safe location** in case of trouble with the upgrade procedure. If you leave them in place, you will get a mixture of old and new files that the new Solr will refuse to load.

d. ***(If upgrading from 5.x or below)* Replace your old build.properties file with a local.cfg** : As of DSpace 6.0, the `build.properties` configuration file has been replaced by an enhanced `local.cfg` configuration file. Therefore, any old `build.properties` file (or similar `[dspace-source]/*.properties` files) WILL BE IGNORED. Instead, you should create a new `local.cfg` file, based on the provided `[dspace-source]/dspace/config/local.cfg.EXAMPLE` and use it to specify all of your locally customized DSpace configurations. This new `local.cfg` can be used to override ANY setting in any other configuration file (`dspace.cfg` or `modules/*.cfg`). To override a default setting, simply copy the configuration into your `local.cfg` and change its value(s). For much more information on the features of local.cfg, see the Configuration Reference documentation and the local.cfg Configuration File section on that page.

```
cd [dspace-source]
cp dspace/config/local.cfg.EXAMPLE local.cfg

# Then edit the local.cfg, specifying (at a minimum) your basic DSpace configuration settings.
# Optionally, you may copy any settings from other *.cfg configuration files into your local.cfg
to override them.
# After building DSpace, this local.cfg will be copied to [dspace]/config/local.cfg, where it will
also be used at runtime.
```

3. **Build DSpace Backend.** Run the following commands to compile DSpace :

```
cd [dspace-source]
mvn -U clean package
```

The above command will re-compile the DSpace source code and build its "installer". You will find the result in `[dspace-source]/dspace/target/dspace-installer`

Defaults to PostgreSQL settings

Without any extra arguments, the DSpace installation package is initialized for PostgreSQL. If you use Oracle instead, you should build the DSpace installation package as follows:

```
mvn -Ddb.name=oracle -U clean package
```

4. **Stop Tomcat (or servlet container).** Take down your servlet container.

   a. For Tomcat, use the `$CATALINA_HOME/shutdown.sh` script. (Many Unix-based installations will have a startup/shutdown script in the `/etc/init.d` or `/etc/rc.d` directories.)

5. **Update your DSpace Configurations.** Depending on the version of DSpace you are upgrading from, not all steps are required.

   a. **If you are upgrading from a prior version of DSpace 7.x**, *you will need to perform the following steps.*

      i. As of DSpace 7.3, the "db.dialect" configuration has changed from "org.dspace.storage.rdbms.hibernate.postgres.DSpacePostgreSQL82Dialect" to "org.hibernate.dialect.PostgreSQL94Dialect". Therefore, MAKE SURE that your dspace.cfg or local.cfg has this setting:

```
db.dialect = org.hibernate.dialect.PostgreSQL94Dialect
```

ii. You may wish to review the Release Notes for details about new features. There may be new configurations you may wish to tweak to enable/disable those features.
iii. *Make sure your existing 7.x local.cfg is in the source directory* (e.g. `[dspace-source]/dspace/config/local.cfg`). That way your existing 7.x configuration gets reinstalled alongside the new version of DSpace.

b. **If you are upgrading from DSpace 6.x or below,** *you will need to perform these steps.*
   i. **Review your customized configurations (recommended to be in local.cfg):** As mentioned above, we recommend any local configuration changes be placed in a local.cfg Configuration File. With any major upgrade some configurations may have changed. Therefore, it is recommended to review all configuration changes that exist in the `config` directory, and its subdirectories, concentrating on configurations your previously customized in your local.cfg. See also the Configuration Reference.
   ii. **Remove obsolete configurations.** With the removal of the JSPUI and XMLUI, a large number of server-side (backend) configurations were made obsolete and were therefore removed between the 6.x and 7.0 release. A full list can be found in the Release Notes.
   iii. **Remove BTE Spring configuration:** If it exists, remove the `[dspace]/config/spring/api/bte.xml` Spring Configuration. This file is no longer needed as the BTE framework was removed in favor of Live Import from external sources.
   iv. **M igrate or recreate your Submission configuration.** As of DSpace 7, the submission configuration has changed. The format of the "item-submission.xml" file has been updated, and the older "input-forms.xml" has been replaced by a new "submission-forms.xml". You can choose to either start fresh with the new v7 configuration files, or you can use the steps below to migrate your old configurations into the new format. See the Submission User Interface for more information
      1. First, create a temporary folder to copy your old v6 configurations into

```
# Example of creating a [dspace]/config/temp folder for this migration
# You must replace [dspace] with the full path of your DSpace 7 installation.
cd [dspace]/config
mkdir temp
```

      2. Copy your old (v5 or v6) "item-submission.xml" and "input-forms.xml" into that temporary folder
      3. Run the command-line migration script to migrate them to v7 configuration files

```
# This example uses [dspace] as a placeholder for all paths.
# Replace it with either the absolute or relative path of these files
[dspace]/bin/dspace submission-forms-migrate -s [dspace]/config/temp/item-submission.
xml -f [dspace]/config/temp/input-forms.xml
```

      4. The result will be two files. These are valid v7 configurations based on your original submission configuration files.
         a. `[dspace]/config/item-submission.xml.migrated`
         b. `[dspace]/config/submission-forms.xml.migrated`
      5. These "*.migrated" files have *no inline comments*, so you may want to edit them further before installing them (by removing the ".migrated" suffix). Alternatively, you may choose to copy sections of the *.migrated files into the default configurations in the `[dspace]/config/` folder, therefore retaining the inline comments in those default files.
   v. **City IP Database file for Solr Statistics has been renamed.** The old `[dspace]/config/GeoLiteCity.dat` file is no longer maintained by its provider. You can delete it. The new file is named `GeoLite2-City.mmdb` by default. If you have configured a different name and/or location for this file, you should check the setting of `usage-statistics.dbfile` in `[dspace]/config/modules/usage-statistics.cfg` (and perhaps move your custom setting to `local.cfg`).
   vi. **tm-extractors media filtering (WordFilter) no longer exists:** the PoiWordFilter plugin now fulfills this function. If you still have WordFilter configured, remove from `dspace.cfg` and/or `local.cfg` all lines referencing `org.dspace.app.mediafilter.WordFilter` and uncomment all lines referencing `org.dspace.app.mediafilter.PoiWordFilter`.
   vii. **Re-configure Solr URLs:** change the value of `solr.server` to point at your new Solr external service. It will probably become something like `solr.server = https://localhost:8983/solr`. Solr only needs to be accessible to the DSpace backend, and should not be publicly available on the web. It can either be run on localhost or via a hostname (if run on a separate server from the backend). Also review the values of
      1. `discovery.search.server`
      2. `oai.solr.url`
      3. `solr.authority.server`
      4. `solr-statistics.server`
   viii. **Sitemaps are now automatically generated/updated:** A new `sitemap.cron` setting exists in the dspace.cfg which controls when Sitemaps are generated. By default they are enabled to update once per day, for optimal SEO. See Search Engine Optimization docs for more detail
      1. Because of this change, if you had a system cron job which ran "`./dspace generate-sitemaps`", this system cron job can be removed in favor of the new `sitemap.cron` setting.

c. **If you are upgrading from DSpace 5.x or below,** there are a few additional configuration changes to be aware of.
   i. **Search/Browse requires Discovery:** As of DSpace 6, only Discovery (Apache Solr) is supported for search/browse. Support for Legacy Search (using Apache Lucene) and Legacy Browse (using database tables) has been removed, along with all their configurations.
   ii. **XPDF media filtering no longer exists:** XPDF media filtering, deprecated in DSpace 5, has been removed. If you used this, you will need to reconfigure using the remaining alternatives (e.g. PDF Text Extractor and/or ImageMagick PDF Thumbnail Generator).

6. **Update DSpace Installation.** Update the DSpace installation directory with the new code and libraries. Issue the following commands:

```
cd [dspace-source]/dspace/target/dspace-installer
ant update
```

7. **Upgrade your database** (*required for all upgrades*). The DSpace code will automatically upgrade your database (*from any prior version of DSpace*). By default, this database upgrade occurs automatically when you restart Tomcat (or your servlet container). However, if you have a large repository or are upgrading across multiple versions of DSpace at once, you may wish to manually perform the upgrade (as it could take some time, anywhere from 5-15 minutes for large sites).
    a. (Optional) If desired, you can optionally verify which migrations have not yet been run on your database. You can use this to double check that DSpace is recognizing your database version appropriately

    ```
    [dspace]/bin/dspace database info

    # If you are upgrading from 5.x or later, then this will list all migrations
    #    which were previously run, along with any which are "PENDING" or "IGNORED"
    #    that need to be run to upgrade your database.
    # If you are upgrading from 4.x or earlier, this will attempt to detect which
    #    version of DSpace you are upgrading from. Look for a line at the bottom
    #    that says something like:
    #    "Your database looks to be compatible with DSpace version ___"
    ```

    b. (Optional) In some rare scenarios, if your database's "sequences" are outdated, inconsistent or incorrect, a database migration error may occur (in your DSpace logs). While this is seemingly a rare occurrence, you may choose to run the "update-sequences" command PRIOR to upgrading your database. If your database sequences are inconsistent or incorrect, this "update-sequences" command will auto-correct them (otherwise, it will do nothing).

    ```
    # This command only works if upgrading from DSpace 7.0 or later
    [dspace]/bin/dspace database update-sequences

    # If upgrading from DSpace 6 or below, this script had to be run via psql from [dspace]/etc
    /postgres/update-sequences.sql
    # For example:
    # psql -U [database-user] -f [dspace]/etc/postgres/update-sequences.sql [database-name]

    # NOTE: It is important to run the "update-sequences" script which came with the OLDER version of
    DSpace (the version you are upgrading from)!  If you've misplaced # this older version of the
    script, you can download it from our codebase & run it via the "psql" command above.
    # DSpace 6.x version of "update-sequences.sql": https://github.com/DSpace/DSpace/blob/dspace-6_x
    /dspace/etc/postgres/update-sequences.sql
    # DSpace 5.x version of "update-sequences.sql": https://github.com/DSpace/DSpace/blob/dspace-5_x
    /dspace/etc/postgres/update-sequences.sql
    ```

    c. (REQUIRED) Then, you can upgrade your DSpace database to the latest version of DSpace. (NOTE: check the DSpace log, `[dspace]/log/dspace.log.[date]`, for any output from this command)

    ```
    # If upgrading from DSpace 6.x or below
    [dspace]/bin/dspace database migrate ignored

    # If upgrading from an earlier version of DSpace 7.x
    [dspace]/bin/dspace database migrate
    ```

    If you are upgrading from DSpace 6.x or below be sure you include the "ignored" parameter! There are database changes which were previously optional but now are mandatory (specifically Configurable Workflow database changes).
    d. *If the database upgrade process fails or throws errors,* then look at the "Troubleshooting Upgrade Issues" section below for possible tips /hints.
    e. More information on the "database" command can be found in Database Utilities documentation.

By default, your site will be automatically reindexed after a database upgrade

If any database migrations are run (even during minor release upgrades), then by default DSpace will automatically reindex all content in your site. This process is run automatically in order to ensure that any database-level changes are also immediately updated within the search/browse interfaces. See the notes below under "**Restart Tomcat (servlet container)**" for more information.

However, you may choose to **skip automatic reindexing.** Some sites choose to run the reindex process manually in order to better control when /how it runs.

To disable automatic reindexing, set `discovery.autoReindex = false` in `config/local.cfg` or `config/modules/discovery.cfg`.

As you have disabled automatic reindexing, make sure to manually reindex your site by running `[dspace]/bin/dspace index-discovery -b` *(This must be run after restarting Tomcat)*

*WARNING: It is not recommended to skip automatic reindexing, unless you will **manually reindex** at a later time, or have verified that a reindex is not necessary. Forgetting to reindex your site after an upgrade may result in unexpected errors or instabilties.*

8. **Deploy Server web application:** The DSpace backend consists of a single "server" webapp (in `[dspace]/webapps/server`). You need to deploy this webapp into your Servlet Container (e.g. Tomcat). Generally, there are two options (or techniques) which you could use...either configure Tomcat to find the DSpace "server" webapp, or copy the "server" webapp into Tomcat's own webapps folder. For more information & example commands, see the Installation Guide

    a. Optionally, you may also install the deprecated DSpace 6.x REST API web application ("rest" webapp). If you previously used the DSpace 6.x REST API, for backwards compatibility the old, deprecated "rest" webapp is still available to install (in `[dspace]/webapps/rest`). It is NOT used by the DSpace UI/frontend. *So, most users should skip this step.*
9. **If upgrading from a previous version of 7.x,** a few extra steps *may be required before starting Tomcat*.
    a. **Update your Solr schema definition(s)**
        i. If you are upgrading from one 7.x release to another, you will need to update your 'search' Solr schema definition with the new version (For example, in 7.1 a new "search.entitytype" field was added to this schema. In 7.6, a new "lastModified_dt" field was added to this schema.).

            1. Copy the updated schema.xml to your Solr 'search' core, e.g.

```
# The destination directory may differ per OS.
# Just replace the existing 'search' core "schema.xml" with the new one.
cp [dspace]/solr/search/conf/schema.xml /var/solr/data/search/conf/
```

            2. Restart Solr

```
[solr]/bin/solr restart
```

            3. Reindex your site

```
[dspace]/bin/dspace index-discovery -b
```

10. **If upgrading from 6.x or below,** *a few extra steps are required to before starting Tomcat. If you are upgrading from a previous version of 7.x, skip this and move along.*
    a. **Install new Solr cores and rebuild your indexes.** *(Required when upgrading from 6.x or below. This may be done after starting Tomcat, but is required for DSpace 7.x to function properly.)*
        i. Copy the new, empty Solr cores to your new Solr instance.

```
cp -R [dspace]/solr/* [solr]/server/solr/configsets
chown -R solr:solr [solr]/server/solr/configsets
```

        ii. Start Solr, or restart it if it is running, so that these new cores are loaded.

```
[solr]/bin/solr restart
```

        iii. You can check the status of Solr and your new DSpace cores by using its administrative web interface. Browse to `${solr.server}` (e.g. `http://localhost:8983/solr/`) to see if Solr is running well, then look at the cores by selecting (on the left) Core Admin or using the Core Selector drop list.

            1. For example, to test that your "search" core is setup properly, try accessing the URL `${solr.server}/search/select`. It should run an empty query against the "search" core, returning an empty JSON result. If it returns an error, then that means your "search" core is missing or not installed properly.

        iv. Load `authority` and `statistics` from the CSV dumps that you made earlier in Step 2 above.

```
[dspace]/bin/dspace solr-import-statistics -i authority
[dspace]/bin/dspace solr-import-statistics -i statistics
```

            This could take quite some time.

            If you had sharded your statistics, you will need to load the dump of each shard separately into the "statistics" core. DSpace 7 does not support Solr shards at this time. Unfortunately, this will involve renaming all CSV export files to remove the year (e.g. rename "statistics-2012_export_2013-12_5.csv" to "statistics_export_2013-12_5.csv") and rerunning "[dspace]/bin/dspace solr-import-statistics -i statistics". More advice on this process can be found in this dspace-tech mailing list thread.

        v. For the Statistics core(s) only, upgrade legacy DSpace Object Identifiers (pre-6.4 statistics) to UUID Identifiers.

```
[dspace]/bin/dspace solr-upgrade-statistics-6x -i statistics
```

Again If you had sharded your statistics, you will need to run this for each shard separately. See also SOLR Statistics Maintenance#UpgradeLegacyDSpaceObjectIdentifiers(pre-6xstatistics)toDSpace6xUUIDIdentifiers

vi. Rebuild the `oai` and `search` cores.

```
[dspace]/bin/dspace oai import
[dspace]/bin/dspace index-discovery -b
```

If you have a great deal of content, this could take a long time.

b. **Update Handle Server Configuration.** *(Required when upgrading from 6.x or below)* Because we've updated to Handle Server v9, if you are using the built-in Handle server (most installations do), you'll need to add the follow to the end of the *server_config* section of your `[dspace]/handle-server/config.dct` file (the only new line is the "enable_txn_queue" line)

```
"case_sensitive" = "no"
"storage_type" = "CUSTOM"
"storage_class" = "org.dspace.handle.HandlePlugin"
"enable_txn_queue" = "no"
```

i. Alternatively, you could re-run the `./dspace make-handle-config` script, which is in charge of updating this `config.dct` file.

c. **(Optional) Set up IP to City database for location-based statistics.** If you wish to (continue to) record the geographic origin of client activity, you will need to install (*and regularly update*) one of the following:

i. Either, a copy of MaxMind's GeoLite City database (in MMDB format)
- NOTE: Installing MaxMind GeoLite2 is *free*. However, you **must** sign up for a (free) MaxMind account in order to obtain a license key to use the GeoLite2 database.
- You may download GeoLite2 directly from MaxMind, or many Linux distributions provide the `geoipupdate` tool directly via their package manager. You will still need to configure your license key prior to usage.
- Once the "GeoLite2-City.mmdb" database file is installed on your system, you will need to configure its location as the value of `usage-statistics.dbfile` in your `local.cfg` configuration file .
- You can discard any old GeoLiteCity.dat database(s) found in the `config/` directory (if they exist).
- See the "Managing the City Database File" section of SOLR Statistics for more information about using a City Database with DSpace.

ii. Or, you can alternatively use/install DB-IP's City Lite database (in MMDB format)
- This database is also free to use, but does **not** require an account to download.
- Once the "dbip-city-lite.mmdb" database file is installed on your system, you will need to configure its location as the value of `usage-statistics.dbfile` in your `local.cfg` configuration file .
- See the "Managing the City Database File" section of SOLR Statistics for more information about using a City Database with DSpace.

d. **Check your cron / Task Scheduler jobs.** In recent versions of DSpace, some of the scripts names have changed.

i. Check the Scheduled Tasks via Cron documentation for details. If you have been using the `dspace stats-util --optimize` tool, it is no longer recommended and you should stop.

ii. WINDOWS NOTE: If you are running the Handle Server on a Windows machine, a new `[dspace]/bin/start-handle-server.bat` script is available to more easily startup your Handle Server.

11. **Restart Tomcat (servlet container).** Now restart your servlet container (Tomcat/Jetty/Resin) and test out the upgrade.

a. **Upgrade of database:** If you didn't manually upgrade your database in the previous step, then your database will be automatically upgraded to the latest version. This may take some time (seconds to minutes), depending on the size of your repository, etc. Check the DSpace log (`[dspace]/log/dspace.log.[date]`) for information on its status.

12. **Reindexing of all content for search/browse:** If your database was just upgraded (either manually or automatically), all the content in your DSpace will be automatically re-indexed for searching/browsing. As the process can take some time (minutes to hours, depending on the size of your repository), it is performed in the background; meanwhile, DSpace can be used as the index is gradually filled. *But, keep in mind that not all content will be visible until the indexing process is completed.* Again, check the DSpace log (`[dspace]/log/dspace.log.[date]`) for information on its status.

a. *If you wish to skip automatic reindexing, please see the Note above under the "Upgrade your Database" step.*
b. *When upgrading from 7.0/7.1/7.2 to 7.3, it is REQUIRED to reindex your content.* If reindexing does not occur automatically, or you disabled it, then run "./dspace index-discovery -b" to reindex your site.

```
# Manually reindex all content
./dspace index-discovery -b
```

13. **Review / Update your scheduled tasks (e.g. cron jobs).** For all features of DSpace to work properly, there are some scheduled tasks you MUST setup to run on a regular basis. Some examples are tasks that help create thumbnails (for images), do full-text indexing (of textual content) and send out subscription emails. See the Scheduled Tasks via Cron for more details.
    a. *When upgrading to 7.5 (or later)* , you will want to make sure the new "subscription-send" task is added to your existing scheduled tasks (in cron or similar). This new task is in charge of sending Email Subscriptions for any users who have subscribed to updates. (NOTE: "subscription-send" *replaces* the older "sub-daily" task from 6.x or below). See the Scheduled Tasks via Cron for more details.
14. **Install or Upgrade the new User Interface (see below)**

Upgrading the Frontend (User Interface)

1. **If upgrading from 6.x or below, install the new User Interface per the Installing DSpace guide.** The JSPUI and XMLUI are no longer supported and cannot work with the DSpace 7 backend. You will need to install the new (Angular.io) User Interface.
    a. JSPUI or XMLUI based themes cannot be migrated. That said, since the new Angular UI also uses Bootstrap, you may be able to borrow some basic CSS from your old themes. But any HTML-level changes will need to be reimplemented in the new UI.
2. **If upgrading from a prior version of 7.x**, upgrading requires installing the latest version of the User Interface code
    a. **Upgrade Node.js if necessary**
        i. *If upgrading to 7.5*, we highly recommend using either Node.js 16 or 18 as Node.js 14 will enter EOL (end of life) as of 30 April 2023.
    b. **Download the latest dspace-angular release from the DSpace GitHub repository**. You can choose to either download the zip or tar.gz file provided by GitHub, or you can use "git" to checkout the appropriate tag (e.g. `dspace-7.2`) or branch.
        i. If you've cloned or copied this code into your own GitHub or GitLab repository, you may wish to simply pull the latest tagged code into your codebase using Git. That will allow you to more easily address any "code conflicts" between your local changes and the new version of DSpace (if any are found).
        ii. NOTE: For the rest of these instructions, we'll refer to the source code location as `[dspace-angular]`.
    c. **Install any updated local dependencies** using Yarn in the `[dspace-angular]` source code directory:

```
# change directory to our repo
cd [dspace-angular]


# Remove any cached dependencies from older DSpace releases
yarn run clean

# install/update the local dependencies
yarn install
```

    d. **Build the latest User Interface code for Production**: This rebuilds the latest code into the `[dspace-angular]/dist` directory

```
yarn build:prod
```

    e. **If upgrading from 7.0 or 7.1, read the updated Installation documentation.** As of 7.2, we now recommend deploying the compiled User Interface (in `[dspace-angular]/dist`) to a different directory (which we refer to as `[dspace-ui-deploy]`) in order to keep your running UI separate from the source code. While it's still possible to run the UI using "yarn start" or "yarn run serve:ssr" (both of which use `[dspace-angular]/dist`), that older approach will mean that your site goes down / becomes unavailable anytime you rebuild (yarn build:prod). To solve this issue:
        i. Create a separate `[dspace-ui-deploy]` location as described in the Installation documentation.
        ii. Copy the `[dspace-angular]/dist` folder to that location, as described in the Installation documentation.
        iii. Update your PM2 configuration or local startup scripts to use Node.js instead of Yarn. Again, see the Installation documentation.
    f. **If upgrading from 7.0 or 7.1, migrate your UI Configurations to YAML.** In 7.2, the format of the UI configuration file changed from Typescript to YAML to support runtime configuration. This means that the older `./src/environment/environment.*.ts` configuration files have all been replaced by corresponding `./config/config.*.yml` configuration files (e.g. environment.prod.ts config.prod.yml).
        i. Either, manually move your "environment.prod.ts" configurations into a new "./config/config.prod.yml" file, using the "./config/config.example.yml" as a guide, along with the User Interface Configuration documentation.
        ii. OR, you can migrate your configurations using the provided "yarn env:yaml" migration script. For detailed instructions, see the Migrate environment file to YAML second of the User Interface Configuration documentation.
    g. **Review Configuration changes** to see if you wish to update any new configurations
        i. In 7.1, we added the ability to "extend" themes in the "themes" section. See the User Interface Configuration documentation for details.
        ii. In 7.2, themes now support optional "headTags" which can be used to customize favicons per theme (see User Interface Customization). Additionally, "browseBy > types" configurations were removed, as they are now dynamically retrieved from the REST API (see User Interface Configuration).
        iii. In 7.3, you can optionally enable Item Access labels in the UI to display the status of an Item as a badge (e.g. "open access", "restricted", "metadata only" or "embargoed"). See User Interface Configuration
        iv. In 7.4, many new user interface configurations were added. See Release Notes and User Interface Configuration for details.
        v. In 7.5, a new "Server Side Rendering" page caching option was added which can drastically speed up the initial response of your site. Other new settings also were added. See Release Notes and User Interface Configuration for details.
        vi. In general, review the Release Notes, especially the "Breaking Changes" section.
    h. **Update your theme (if necessary)**, if you've created a custom theme in "src/themes" (or modified the existing "custom" or "dspace" themes in that location). Pay close attention to the following...
        i. In 7.3, a new "eager-theme.module.ts" and "lazy-theme.module.ts" has been added to both the "custom" and "dspace" themes to improve performance. Make sure to copy those to your custom theme. Additionally, this new "eager-theme.module.ts" for your theme MUST be imported/enabled in "src/themes/eager-themes.module.ts". For example, for a local theme under "src/theme/my-theme":

src/themes/eager-themes.module.ts

```
import { EagerThemeModule as MyThemeEagerThemeModule } from './my-theme/eager-theme.module';
...
@NgModule({
```

```
  imports: [
    MyThemeEagerThemeModule,
  ],
})
```

ii. Additional minor changes may have been made. It's usually best to look for changes to whichever theme you started from. If you started your theme from the "custom" theme, look for any new changes made under "/src/themes/custom". If you started your theme from the "dspace" theme, look for any new changes made under "/src/themes/dspace".
    1. Using a tool like "git diff" from the commandline is often an easy way to see changes that occurred only in that directory.

```
# Example which will show all the changes to "src/themes/dspace" (and all subfolders)
# between dspace-7.4 (tag) and dspace-7.5 (tag)
git diff dspace-7.4 dspace-7.5 -- src/themes/dspace/
```

iii. For the "custom" theme, the largest changes are often:
    1. New themeable components (subdirectories) may be added under "src/themes/custom/app", allowing you the ability to now change the look & feel of those components.
    2. The "src/themes/custom/theme.module.ts" file will likely have minor updates. This file registers any new themeable components (in the "`const DECLARATIONS`" section), and also registers new Modules, i.e. new UI features, (in the "`@NgModule`" "imports" section). Make sure those sections are updated in your copy of this file!
    3. Sometimes, new styles may be added in the "styles" folder, or new imports to "styles/theme.scss"
    4. If you have locally customized the styles or look & feel of any component, you should also verify that the component itself (in src/app) hasn't had updates.

iv. For the "dspace" theme, the largest changes are often:
    1. Existing customized components (subdirectories) under "src/themes/dspace/app/" may have minor updates, if improvements were made to that component.
    2. The "src/themes/custom/theme.module.ts" file will likely have minor updates. This file registers any new themeable components (in the "`const DECLARATIONS`" section), and also registers new Modules, i.e. new UI features, (in the "`@NgModule`" "imports" section). Make sure those sections are updated in your copy of this file!
    3. Sometimes, new styles may be added in the "styles" folder, or new imports to "styles/theme.scss"
    4. If you have locally customized the styles or look & feel of any additional component, you should also verify that the component itself (in src/app) hasn't had updates.

i. **Restart the User Interface.**
    i. If you are using PM2 as described in the Installing DSpace instructions, you'd stop it and then start it back up as follows

```
# Stop the running UI
pm2 stop dspace-ui.json

# If you had to update your PM2 configs, you may need to delete your old configuration from
PM2
# pm2 delete dspace-ui.json

# Start it back up
pm2 start dspace-ui.json
```

    ii. If you are using a different approach, you simply need to *stop the running UI*, and re-run:

```
# First stop the running UI process by killing it (or similar)

# You MUST restart the UI from within the deployment directory
# See Installation Instructions for more info on this directory
cd [dspace-ui-deploy]

# Then restart the UI via Node.js
node ./dist/server/main.js
```

j. **Verify the UI and REST API are both working properly.**
    i. If you hit errors, see the "Troubleshooting Upgrade Issues" section below. Additionally, check the "Common Installation Issues" section of the Installing DSpace documentation for other common misconfiguration or setup issues.

Troubleshooting Upgrade Issues

See all "Common Installation Issues"

At times the upgrade process may involve configuration changes which could result in one of our "Common Installation Issue" error messages. So, make sure to check that section of the Installing DSpace documentation, especially if you find your UI is no longer connecting to your REST API.

Database migrate errors: "Migration V5.7_2017.04.11__DS-3563_Index_metadatavalue_resource_type_id_column.sql failed" or " Migration V5.7_2017.05.05__DS-3431.sql failed"

If you are upgrading to DSpace 7.x and receive *either* of the two following errors after running "./dspace database migrate ignored":

```
Migration V5.7_2017.04.11__DS-3563_Index_metadatavalue_resource_type_id_column.sql failed

[or]

Migration V5.7_2017.05.05__DS-3431.sqln failed
```

This means your database never ran those older migrations during a past upgrade from 5.x6.x (or similar).

Luckily, though, these migrations are both obsolete in DSpace 7.x (and later). This means *you can skip these migration safely*.

As of DSpace 7.5, a new "./dspace database skip" command is provided to easily skip one (or both) of these failing migrations as follows:

```
# If you need to skip "V5.7_2017.04.11__DS-3563_Index_metadatavalue_resource_type_id_column.sql", run this:
./dspace database skip "5.7.2017.04.11"

# If you need to skip "V5.7_2017.05.05__DS-3431.sql", run this:
./dspace database skip "5.7.2017.05.05"
```

For more information on the "./dspace database skip" command see Database Utilities.

Database migrate errors because of custom database tables/columns

If you have modified your database structure directly (or installed custom plugins which do so), then it is possible a "./dspace database migrate" will fail if it encounters an unexpected database structure. In this scenario, you may need to do some manual migrations before the automatic migrations will succeed. The general process would be something like this:

1. Revert back to your current DSpace database
2. Manually upgrade just your database **past** the failing migration. For example, if you are current using DSpace 1.5 and the "V1.6" migration is failing, you may need to first manually upgrade your database to 1.6 compatibility. This may involve either referencing the upgrade documentation for that older version of DSpace, or running the appropriate SQL script from under `[dspace-src]/dspace-api/src/main/resources/org/dspace/storage/rdbms/sqlmigration/`)
3. Then, re-run the migration process from that point forward (i.e. re-run `./dspace database migrate`)

Running "Ignored" Flyway Migrations

During some upgrades, a Flyway database migration will be "ignored." One known instance of this is documented in https://github.com/DSpace/DSpace/issues/6762. If you are upgrading from DSpace 5.x to a later version of DSpace, the migration put in place to address https://github.com/DSpace/DSpace/pull/1128 will be "ignored" because it is not necessary. There is a special command you can run which will un-flag this migration as "ignored."

```
dspace database migrate ignored
```

Manually updating the Metadata Registries

The database migration (./dspace database migrate) should *automatically trigger* your metadata/file registries to be updated (based on the config files in `[dspace]/config/registries/`). However, if this update was NOT triggered, you can also manually run these registry updates (they will not harm existing registry contents) as follows:

```
cd [dspace]/bin/
./dspace registry-loader -metadata ../config/registries/dcterms-types.xml
./dspace registry-loader -metadata ../config/registries/dublin-core-types.xml
./dspace registry-loader -metadata ../config/registries/eperson-types.xml
./dspace registry-loader -metadata ../config/registries/local-types.xml
./dspace registry-loader -metadata ../config/registries/sword-metadata.xml
./dspace registry-loader -metadata ../config/registries/workflow-types.xml
```

Migrating DSpace to a new server

These instructions are meant as a general guideline to how you can migrate your DSpace site/data to a new server while also Upgrading DSpace to the latest release. Keep in mind that you MUST also review the Installing DSpace and Upgrading DSpace guides when performing a migration (e.g. you must ensure you have correct dependencies installed and you must ensure you perform all upgrade steps).

Overview of migration approaches

There are two main approaches to migrating your DSpace to a new server.

1. Install a fresh copy of DSpace & migrate the database/files into it - This is the approach documented on this page. It is the recommended approach as it ensures zero data loss. However, it does involve more steps to complete the migration.
2. Install a fresh copy of DSpace & use the AIP Backup and Restore - This is an alternative approach where you can use the AIP export tools to export AIPs from your old site, and then import them into the new site. While this also works, keep in mind that not all DSpace data can be exported to AIPs, so you will lose some data during this migration (namely any submissions not yet completed or still in workflow approval will be lost, see the AIP Backup and Restore documentation for more details on what data is currently missing from AIPs).

Step 1: Install a fresh copy of DSpace

On your new server, follow the Installing DSpace instructions and install a fresh (empty) copy of the latest version of DSpace. **BEFORE PROCEEDING**, ensure that this fresh copy of DSpace is correctly installed and shows no errors when you startup the site. (The site will obviously appear empty though, and that's OK)

You can also use this time to get your basic configurations setup properly for both the backend (local.cfg) and the frontend (config.prod.yml).

Step 2: Prepare your data to copy from the old DSpace to the new one

There are three main areas of data you need to migrate in order to ensure no data loss.

Perform these steps on the *old server*

1. First, you should STOP tomcat on the old server. These steps require the site to be down.
2. Update sequences *(optional)* - When migrating content, sometimes sites will find that database sequences will be outdated or incorrect. This can result in "duplicate key" errors during the database migration to the latest version. To avoid this, **before** you export your data, run this older copy of the "update-sequences" command on your database. This should ensure your database sequences are updated before you dump your data.

```
# If upgrading from DSpace 6 or below, run this on your old database
psql -U [database-user] -f [dspace]/etc/postgres/update-sequences.sql [database-name]
# e.g. psql -U dspace -f [dspace]/etc/postgres/update-sequences.sql dspace
```

    a. NOTE: It is important to run the "update-sequences" script which came with the OLDER version of DSpace (the version you are migrating from)! If you've misplaced this older version of the script, you can download it from our codebase & run it via the "psql" command above.
        i. DSpace 6.x version of "update-sequences.sql": https://github.com/DSpace/DSpace/blob/dspace-6_x/dspace/etc/postgres/update-sequences.sql
        ii. DSpace 5.x version of "update-sequences.sql": https://github.com/DSpace/DSpace/blob/dspace-5_x/dspace/etc/postgres/update-sequences.sql
3. The database data - Make sure to export the database data from your old DSpace site using a tool like "pg_dump" (for PostgresSQL). If you use "pg_dump", you'll end up with a large SQL file which contains all the data from your old database.

```
# Example of using pg_dump to export a database to an output file
pg_dump -U [db_username] [db_name] > [output_file.sql]
```

4. The "assetstore" folder - This folder is in your DSpace installation directory and it contains all the files stored in your DSpace. You will need **all the contents** of this folder (including all subdirectories), so you could choose to zip it up or you could copy it over directly.
5. The Solr data *(optional)* - Both DSpace authority and statistics are stored in Solr. If you want to keep these, you will want to export them from the old Solr and move them over. Use the "solr-export-statistics" tool provided with DSpace: see "Export SOLR Statistics" in the Solr Statistics Maintenance guide. *(Requires Solr to be running. Keep in mind, this may require you to start Tomcat back up if Solr is running in Tomcat.)*

Step 3: Copy over the prepared data and import it into the new DSpace

Copy the data you've prepared in Step 2 over to the new server.

Now, you'll import this data into your new installation of DSpace (created in Step 1).

Perform these steps on the *new server.*

1. First, you must STOP Tomcat on the new server.
2. The database data - Before you can import the data, you must delete the new, empty database.
    a. Delete/Clean the new, empty database (created in step 1) as you will have empty tables created during the installation. The easiest way to achieve this is to run the "./dspace database clean" command. Keep in mind it requires temporarily enabling it via "db.cleanDisabled=false" in your local.cfg. (After the "clean" command succeeds, make sure to remove this configuration.)

```
# Delete everything in your database
# Requires temporarily setting "db.cleanDisabled=false" in your local.cfg
./dspace database clean
```

        i. Alternatively, PostgreSQL users could delete the entire database (using dropdb command, e.g. "dropdb -U [db_username] [db_name]") and recreate it based on the "Database Setup" instructions in Installing DSpace.
    b. Import the database dump you created in Step 2 (above), which will recreate this database with all your old data in it. For Postgres, you can use the "psql" command.

```
# Example of using psql to import data from a SQL file into a database
psql -U [db_username] [db_name] < [output_file.sql]
```

    (NOTICE the direction of the angle character... in this command you are telling Postgres to execute all the commands contained in your "output_file.sql", which will cause it to recreate all the database data in your new database.)
3. The "assetstore" folder - Delete the **empty** assetstore folder on the new server. Copy the entire assestore folder (and all subdirectories) from the old server to the new one. In the end, you should have a several subdirectory hierarchies (containing your files) under the `[dspace]/assetstore/` folder on the new server.
4. The Solr data (optional) - If you exported the statistics or authority data in Step 2, then you can import this data from the exported files using the "solr-import-statistics" tool provided with DSpace, see "Import SOLR Statistics" in the Solr Statistics Maintenance guide. *(Requires Solr to be running)*

Step 4: Update the database, Start DSpace & Reindex

Now that all the data is copied over, you need to ensure it's updated and reindexed properly (for the new version of DSpace).

Perform these steps on the *new server.*

1. Migrate/Upgrade the database to the latest version - Now that your old data is migrated, you MUST ensure it's using the latest database updates based on the new DSpace you've installed. Review the database steps in Upgrading DSpace and follow the instructions there.

```
# Migrate your old data to the latest DSpace version
# WARNING: You must review the Upgrading DSpace docs to see if there are any additional database steps
listed there!
./dspace database migrate ignored
```

    NOTE: You should check the logs (dspace.log) for errors. Additional steps may be documented in the Upgrading DSpace guide.
2. Start Tomcat. This will bring your new DSpace back up, with the migrated data in place. Check the backend logs (dspace.log and Tomcat log) to ensure no errors occur on startup.
3. Reindex all content - This will ensure all search/browse functionality works in the DSpace site. Optionally, if you use OAI-PMH, you will want to reindex content into there as well.

```
# Reindex all your content in DSpace
./dspace index-discovery -b

# (Optionally) also reindex everything into OAI-PMH endpoint
./dspace oai import
```

    NOTE: Until this command completes (it may take a while for large sites), you will not be able to fully browse/search the content from the User Interface. To check the progress of the reindex, check your dspace.log file.

Step 5: Review the Upgrade Instructions and final cleanup

If you've changed the version of DSpace you are running, you should review the Upgrading DSpace guide for instructions related to necessary configuration changes or other necessary updates. You should perform any upgrade steps that you have NOT yet performed above (keep in mind you already should have upgraded your database, and reindexed your content).

At this time, you also may wish to review your configurations on your *old* DSpace site, and see if there are any configurations that you wish to copy over into your new DSpace site. This step is optional, as you can also choose to start "fresh" with a new local.cfg file.

FINALLY, test the new site and verify that all the content, user accounts, etc. have moved over successfully. If you encounter any issues, see our Troubleshoot an error guide for hints/tips on finding the underlying error message & reporting it to Support lists/channels. Also make sure to check our list of Common Installation Issues in the Installing DSpace guide.

Using DSpace

This page offers access to all aspects of the documentation relevant to using DSpace after it has been properly installed or upgraded. These pages assume that DSpace is functioning properly. Please refer to the section on System Administration if you are looking for information on diagnosing DSpace issues and measures you can take to restore your DSpace to a state in which it functions properly.

Authentication and Authorization

- Authentication Plugins
- Bulk Access Management
- Embargo
- Managing User Accounts
- Request a Copy

Authentication Plugins

1 Stackable Authentication Method(s)
- 1.1 Authentication by Password
  - 1.1.1 Enabling Authentication by Password
  - 1.1.2 Configuring Authentication by Password
- 1.2 Open ID Connect (OIDC) Authentication
  - 1.2.1 Enabling OIDC Authentication
  - 1.2.2 Configuring OIDC Authentication
    - 1.2.2.1 Sample/Test OIDC Configuration
- 1.3 Shibboleth Authentication
  - 1.3.1 Enabling Shibboleth Authentication
  - 1.3.2 Configuring Shibboleth Authentication
    - 1.3.2.1 Apache "mod_shib" Configuration (required)
    - 1.3.2.2 Sample shibboleth2.xml Configuration
    - 1.3.2.3 Sample attribute-map.xml Configuration (for samltest.id)
    - 1.3.2.4 DSpace Shibboleth Configuration Options
- 1.4 LDAP Authentication
  - 1.4.1 Introduction to LDAP specific terminology
  - 1.4.2 Enabling LDAP Authentication
  - 1.4.3 Configuring LDAP Authentication
  - 1.4.4 Debugging LDAP connection and configuration
  - 1.4.5 Enabling Hierarchical LDAP Authentication
  - 1.4.6 Configuring Hierarchical LDAP Authentication
- 1.5 ORCID Authentication
  - 1.5.1 Enabling ORCID Authentication
- 1.6 IP Authentication
  - 1.6.1 Enabling IP Authentication
  - 1.6.2 Configuring IP Authentication
- 1.7 X.509 Certificate Authentication
  - 1.7.1 Enabling X.509 Certificate Authentication
  - 1.7.2 Configuring X.509 Certificate Authentication
- 1.8 Example of a Custom Authentication Method

Stackable Authentication Method(s)

Since many institutions and organizations have existing authentication systems, DSpace has been designed to allow these to be easily integrated into an existing authentication infrastructure. It keeps a series, or "stack", of *authentication methods*, so each one can be tried in turn. This makes it easy to add new authentication methods or rearrange the order without changing any existing code. You can also share authentication code with other sites.

Configuration File:	**`[dspace]/config/modules/authentication.cfg`**
Property:	`plugin.sequence.org.dspace.authenticate.AuthenticationMethod`
Example Value:	`plugin.sequence.org.dspace.authenticate.AuthenticationMethod = org.dspace.authenticate.PasswordAuthentication`

The configuration property `plugin.sequence.org.dspace.authenticate.AuthenticationMethod` defines the authentication stack. It is a comma-separated list of class names. Each of these classes implements a different `authentication method`, or way of determining the identity of the user. They are invoked in the order specified until one succeeds.

Existing Authentication Methods include

- Authentication by Password (class: `org.dspace.authenticate.PasswordAuthentication`) (DEFAULT)
- Open ID Connect (OIDC) Authentication (class: `org.dspace.authenticate.OidcAuthentication`)
- Shibboleth Authentication (class: `org.dspace.authenticate.ShibAuthentication`)
- LDAP Authentication (class: `org.dspace.authenticate.LDAPAuthentication`)
- ORCID Authentication (class: `org.dspace.authenticate.OrcidAuthentication`)
- IP Address based Authentication (class: `org.dspace.authenticate.IPAuthentication`)
- X.509 Certificate Authentication (class: `org.dspace.authenticate.X509Authentication`)

An authentication method is a class that implements the interface `org.dspace.authenticate.AuthenticationMethod`. It `authenticates` a user by evaluating the *credentials* (e.g. username and password) he or she presents and checking that they are valid.

Authentication by Password

Enabling Authentication by Password

By default, this authentication method is enabled in DSpace.

However, to enable Authentication by Password, you must ensure the `org.dspace.authenticate.PasswordAuthentication` class is listed as one of the AuthenticationMethods in the following configuration:

Configuration File:	**`[dspace]/config/modules/authentication.cfg`**
Property:	`plugin.sequence.org.dspace.authenticate.AuthenticationMethod`
Example Value:	`plugin.sequence.org.dspace.authenticate.AuthenticationMethod = org.dspace.authenticate.PasswordAuthentication`

Configuring Authentication by Password

The default method `org.dspace.authenticate.PasswordAuthentication` has the following properties:

- Use of inbuilt e-mail address/password-based log-in. This is achieved by sending login information to the "/api/authn/login" endpoint of the REST API, in order to obtain a JSON Web Token. This JSON Web token must be sent on every later request which requires authentication.
- Users can register themselves (i.e. add themselves as e-people without needing approval from the administrators), and can set their own passwords when they do this
- Users are not members of any special (dynamic) e-person groups
- You can restrict the domains from which new users are able to register. To enable this feature, uncomment the following line from dspace.cfg: `authentication.password.domain.valid = example.com` Example options might be '`@example.com`' to restrict registration to users with addresses ending in @example.com, or '`@example.com, .ac.uk`' to restrict registration to users with addresses ending in @example.com or with addresses in the .ac.uk domain.

A full list of all available Password Authentication Configurations:

Configuration File:	**`[dspace]/config/modules/authentication-password.cfg`**
Property:	`user.registration`
Example Value:	`user.registration = false`
Informational Note:	This option allows you to disable all self-registration. When set to "false", no one will be able to register new accounts with your system. Default is "true".
Property:	`authentication-password.domain.valid`
Example Value:	`authentication-password.domain.valid = @mit.edu, .ac.uk`
Informational Note:	This option allows you to limit self-registration to email addresses ending in a particular domain value. The above example would limit self-registration to individuals with "@mit.edu" email addresses and all ".ac.uk" email addresses. (This setting only works when user.registration=true)
Property:	`authentication-password.login.specialgroup`
Example Value:	`authentication-password.login.specialgroup = My DSpace Group`
Informational Note:	This option allows you to automatically add all password authenticated user sessions to a specific DSpace Group (the group must exist in DSpace) for the remainder of their logged in session.
Property:	`authentication-password.digestAlgorithm`
Example Value:	`authentication-password.digestAlgorithm = SHA-512`
Informational Note:	This option specifies the hashing algorithm to be used in converting plain-text passwords to more secure password digests. The example value is the default. You may select any digest algorithm available through java.security.MessageDigest on your system. At least MD2, MD5, SHA-1, SHA-256, SHA-384, and SHA-512 should be available, but you may have installed others. Most sites will not need to adjust this.
Property:	`authentication-password.regex-validation.pattern`
Example Value:	`authentication-password.regex-validation.pattern = ^.{8\,}$`

Informational Note:	This option specifies a regular expression which all new passwords MUST validate against. By default, DSpace just requires a new password to be 8 or more characters (see above example value). However, sites can modify this regex in order to require more robust passwords of all users. One example of a complex rule is: `authentication-password.regex-validation.pattern = ^(?=.*?[a-z])(?=.*?[A-Z])(?=\\S*?[0-9])(?=\\S*?[!?$@#$%^&+=]).{8\,15}$` This example requires all users to adopt a more complex password: • `(?=.*?[a-z])` - the password must contain at least one lowercase character • `(?=.*?[A-Z])` - the password must contain at least one uppercase character • `(?=\\S*?[0-9])` - the password must contain at least one numeric character • `(?=\\S*?[!?$@#$%^&+=])` - the password must contain at least one of the following special character: !?$@#$%^&+= • `.{8\,15}` - the password must be at least 8 and at most 15 characters long (NOTE: the "\," is required to escape the comma, which is a special character)

Open ID Connect (OIDC) Authentication

Open ID Connect (OIDC) Authentication is only available in DSpace 7.2 or above.

Enabling OIDC Authentication

To enable OIDC Authentication, you must ensure the `org.dspace.authenticate.OidcAuthentication` class is listed as one of the AuthenticationMethods in the following configuration:

Configuration File:	**`[dspace]/config/modules/authentication.cfg`**
Property:	`plugin.sequence.org.dspace.authenticate.AuthenticationMethod`
Example Value:	`plugin.sequence.org.dspace.authenticate.AuthenticationMethod = org.dspace.authenticate.OidcAuthentication` (*NOTE: This setting may be repeated to support multiple AuthenticationMethods*) (WARNING: it's easy to miss, the "camel case" for `OidcAuthentication` might catch you off guard. It's important to *not* use `OIDCAuthentication` in this line, because that class does not exist. *Case matters*.

Configuring OIDC Authentication

OpenID Connect is an identity layer on top of the OAuth 2.0 protocol. It allows Clients to verify the identity of the End-User based on the authentication performed by an Authorization Server, as well as to obtain basic profile information about the End-User in an interoperable and REST-like manner. There are many server implementations of OpenID Connect, including Keycloak and AWS Cognito.

Configuration File:	**`[dspace]/config/modules/authentication-oidc.cfg`**
Property:	`authentication-oidc.auth-server-url`
Example Value:	`authentication-oidc.auth-server-url = https://auth.example.com`
Informational Note:	(Optional) The root URL of the OpenID Connect server. This is optional, as it's only used to fill out each of the "-endpoint" configs below (see below). So, for some setups, it may be easier to configure the "-endpoint" configs directly INSTEAD OF the "auth-server-url" and "auth-server-realm"
Property:	`authentication-oidc.auth-server-realm`
Example Value:	`authentication-oidc.auth-server-realm = dspace-realm`
Informational Note:	(Optional) The realm to authenticate against on the OpenID Connect server. This is optional, as it's only used to fill out each of the "-endpoint" configs below (see below). So, for some setups, it may be easier to configure the "-endpoint" configs directly INSTEAD OF the "auth-server-url" and "auth-server-realm"
Property:	`authentication-oidc.token-endpoint`
Example Value:	`authentication-oidc.token-endpoint = ${authentication-oidc.auth-server-url}/auth/realms/${authentication-oidc.auth-server-realm}/protocol/openid-connect/token`
Informational Note:	(Required) The URL of the OIDC Token endpoint. This defaults to using the configured "auth-server-url" and "auth-server-realm" to determine the likely OIDC path for this endpoint (see example above for the default value). However, if that default path is incorrect, you may choose to hardcode the correct URL in this field.

Property:	`authentication-oidc.authorize-endpoint`
Example Value:	`authentication-oidc.authorize-endpoint = ${authentication-oidc.auth-server-url}/auth/realms /${authentication-oidc.auth-server-realm}/protocol/openid-connect/auth`
Informational Note:	(Required) The URL of the OIDC Authorize endpoint. This defaults to using the configured "auth-server-url" and "auth-server-realm" to determine the likely OIDC path for this endpoint (see example above for the default value). However, if that default path is incorrect, you may choose to hardcode the correct URL in this field.
Property:	`authentication-oidc.user-info-endpoint`
Example Value:	`authentication-oidc.user-info-endpoint = ${authentication-oidc.auth-server-url}/auth/realms /${authentication-oidc.auth-server-realm}/protocol/openid-connect/userinfo`
Informational Note:	(Required) The URL of the OIDC Userinfo endpoint. This defaults to using the configured "auth-server-url" and "auth-server-realm" to determine the likely OIDC path for this endpoint (see example above for the default value). However, if that default path is incorrect, you may choose to hardcode the correct URL in this field.
Property:	`authentication-oidc.client-id`
Example Value:	`authentication-oidc.client-id = our-dspace`
Informational Note:	(Required) The registered OIDC client id for our DSpace server's use. No default value.
Property:	`authentication-oidc.client-secret`
Example Value:	`authentication-oidc.client-secret = some-sort-of-hash`
Informational Note:	(Required) The registered OIDC client secret for our DSpace server's use. No default value.
Property:	`authentication-oidc.redirect-url`
Example Value:	`authentication-oidc.redirect-url = ${dspace.server.url}/api/authn/oidc`
Informational Note:	The URL users will be redirected to after a successful login. The example above is the default value, and it usually does not need to be updated.
Property:	`authentication-oidc.scopes`
Example Value:	`authentication-oidc.scopes = openid,email,profile`
Informational Note:	The scopes to request from the OIDC server. The example above is the default value
Property:	`authentication-oidc.can-self-register`
Example Value:	`authentication-oidc.can-self-register = true`
Informational Note:	Specify if the user can self register using OIDC (true|false). If not specified, true is assumed. If this is set to false, then only users with an existing EPerson in DSpace will be able to authenticate through OIDC. When set to true, an EPerson will be automatically created for each person who successfully authenticates through OIDC.
Property:	`authentication-oidc.user-info.email`
Example Value:	`authentication-oidc.user-info.email = email`
Informational Note:	Specify the attribute present in the user info json related to the user's email. The default value is "email"
Property:	`authentication-oidc.user-info.first-name`
Example Value:	`authentication-oidc.user-info.first-name = given_name`
Informational Note:	Specify the attribute present in the user info json related to the user's first/given name. The default value is "given_name"
Property:	`authentication-oidc.user-info.last-name`
Example Value:	`authentication-oidc.user-info.last-name = family_name`
Informational Note:	Specify the attribute present in the user info json related to the user's last/family name. The default value is "family_name"

Sample/Test OIDC Configuration

One way to easily test OIDC Authentication is to use the Auth0 test site at https://auth0.com/. This site allows you to create a random OIDC client & a random OIDC user to authenticate as. So, it can be used to verify that DSpace's OIDC authentication is working in your system, but obviously is only meant for development/testing purposes.

In order to use Auth0 you first need to create a new tenant on that website, and enable some authentication methods. Be sure to also configure at least the following fields under Applications > Settings:

- Allowed Callback URLs: http://localhost:4000/login, http://localhost:8080/server/api/authn/oidc
- Allowed Logout URLs: http://localhost:4000/
- Allowed Web Origins: http://localhost:4000

On the same page you will find some information about your application (under Basic Informattion), to finish the setup you will need to copy some of that config in your local.cfg:

local.cfg updates for PhantAuth

```
# Enable OIDC
plugin.sequence.org.dspace.authenticate.AuthenticationMethod = org.dspace.authenticate.OidcAuthentication

# Settings for OIDC authentication
authentication-oidc.authorize-endpoint = https://YOUR-TENANT.eu.auth0.com/authorize
authentication-oidc.token-endpoint = https://YOUR-TENANT.eu.auth0.com/oauth/token
authentication-oidc.user-info-endpoint = https://YOUR-TENANT.eu.auth0.com/userinfo

# Obtain your Client ID & Client Secret under Applications > Settings
authentication-oidc.client-id =
authentication-oidc.client-secret =
```

Shibboleth Authentication

Enabling Shibboleth Authentication

To enable Shibboleth Authentication, you must ensure the `org.dspace.authenticate.ShibAuthentication` class is listed as one of the AuthenticationMethods in the following configuration:

Configuration File:	**`[dspace]/config/modules/authentication.cfg`**
Property:	`plugin.sequence.org.dspace.authenticate.AuthenticationMethod`
Example Value:	`plugin.sequence.org.dspace.authenticate.AuthenticationMethod = org.dspace.authenticate.ShibAuthentication` (*NOTE: This setting may be repeated to support multiple AuthenticationMethods*)

Configuring Shibboleth Authentication

Shibboleth is a distributed authentication system for securely authenticating users and passing attributes about the user from one or more identity providers. In the Shibboleth terminology DSpace is a Service Provider which receives authentication information and then based upon that provides a service to the user. To use Shibboleth, DSpace *requires* that you use Apache installed with the mod_shib module acting as a proxy for all HTTP requests for your servlet container (typically Tomcat). DSpace will receive authentication information from the mod_shib module through HTTP headers.

Before DSpace will work with Shibboleth, you **must** have the following:

1. An Apache web server with the "mod_shib" module installed. As mentioned, this mod_shib module acts as a proxy for all HTTP requests for your servlet container (typically Tomcat). Any requests to DSpace that require authentication via Shibboleth should be redirected to 'shibd' (the shibboleth daemon) by this "mod_shib" module. Details on installing/configuring mod_shib in Apache are available at: https://wiki.shibboleth.net/confluence/display/SHIB2/NativeSPApacheConfig We also have a sample Apache + mod_shib configuration provided below.
2. An external Shibboleth IdP (Identity Provider). Using mod_shib, DSpace will only act as a Shibboleth SP (Service Provider). The actual Shibboleth Authentication & Identity information must be provided by an external IdP. If you are using Shibboleth at your institution already, then there already should be a Shibboleth IdP available. More information about Shibboleth IdPs versus SPs is available at: https://wiki.shibboleth.net/confluence/display/SHIB2/UnderstandingShibboleth

For more information on installing and configuring a Shibboleth Service Provider see: https://wiki.shibboleth.net/confluence/display/SHIB2/Installation

Note about Shibboleth Active vs Lazy Sessions:

When configuring your Shibboleth Service Provider there are two Shibboleth paradigms you may use: Active or Lazy Sessions. Active sessions is where the mod_shib module is configured to product an entire URL space. No one will be able to access that URL without first authenticating with Shibboleth. Using this method you will need to configure shibboleth to protect the URL: "/shibboleth-login". The alternative, Lazy Session does not protect any specific URL. Instead Apache will allow access to any URL, and when the application wants to it may initiate an authenticated session.

The Lazy Session method is preferable for most DSpace installations, as you usually want to provide public access to (most) DSpace content, while restricting access to only particular areas (e.g. administration UI/tools, private Items, etc.). When Active Sessions are enabled your *entire* DSpace site will be access restricted. In other words, when using Active Sessions, Shibboleth will require everyone to first authenticate before they can access any part of your repository (which essentially results in a "dark archive", as anonymous access will not be allowed).

Apache "mod_shib" Configuration (required)

As mentioned above, you must have Apache with the "mod_shib" module installed in order for DSpace to be able to act as a Shibboleth Service Provider (SP). The mod_shib module acts as a proxy for all HTTP requests for your servlet container (typically Tomcat). Any requests to DSpace that require authentication via Shibboleth should be redirected to 'shibd' (the shibboleth daemon) by this "mod_shib" module. Details on installing/configuring mod_shib in Apache are available at: https://wiki.shibboleth.net/confluence/display/SHIB2/NativeSPApacheConfig General information about installing /configuring Shibboleth Service Providers (SPs) can be found at: https://wiki.shibboleth.net/confluence/display/SHIB2/Installation

A few extra notes/hints when configuring mod_shib & Apache:

- In Debian based environments, "mod_shib" tends to be in a package named something like "libapache2-mod-shib2"
- The Shibboleth setting "ShibUseHeaders" is no longer required to be set to "On", as DSpace will correctly utilize attributes instead of headers.
    - When "ShibUseHeaders" is set to "Off" (which is recommended in the mod_shib documentation), proper configuration of Apache to pass attributes to Tomcat (via either mod_jk or mod_proxy) can be a bit tricky, SWITCH has some great documentation on exactly what you need to do. We will eventually paraphrase/summarize this documentation here, but for now, the SWITCH page will have to do.
- When initially setting up Apache & mod_shib, https://samltest.id/ provides a great testing ground for your configurations. This site provides a sample/demo Shibboleth IdP (as well as a sample Shibboleth SP) which you can test against. It acts as a "sandbox" to get your configurations working properly, before you point DSpace at your production Shibboleth IdP.
- You also may wish to review the Shibboleth setup in our "dspace-shibboleth" Docker setup which the development team uses for testing (and it uses https://samltest.id as the IdP). It may provide you with good examples/hints on getting everything setup. However, keep in mind this code has not been tested in Production scenarios.

Below, we have provided a sample Apache configuration. However, as every institution has their own specific Apache setup/configuration, it is highly likely that you will need to tweak this configuration in order to get it working properly. Again, see the official mod_shib documentation for much more detail about each of these settings: https://wiki.shibboleth.net/confluence/display/SHIB2/NativeSPApacheConfig These configurations are meant to be added to an Apache <VirtualHost> which acts as a proxy to your Tomcat (or other servlet container) running DSpace. More information on Apache VirtualHost settings can be found at: https://httpd.apache.org/docs/2.2/vhosts/

```
#### SAMPLE MOD_SHIB CONFIGURATION FOR APACHE2 (it may require local modifications based on your Apache setup)
####
# While this sample VirtualHost is for HTTPS requests (recommended for Shibboleth, obviously),
# you may also need/want to create one for HTTP (*:80)
<VirtualHost *:443>
   ...
   # PLEASE NOTE: We have omitted many Apache settings (ServerName, LogLevel, SSLCertificateFile, etc)
   # which you may need/want to add to your VirtualHost

   # As long as Shibboleth module is installed, enable all Shibboleth/mod_shib related settings
   <IfModule mod_shib>
       # Shibboleth recommends turning on UseCanonicalName
       # See "Prepping Apache" in https://wiki.shibboleth.net/confluence/display/SHIB2/NativeSPApacheConfig
       UseCanonicalName On

       # Most DSpace instances will want to use Shibboleth "Lazy Session", which ensures that users
       # can access DSpace without first authenticating via Shibboleth.
       # This section turns on Shibboleth "Lazy Session". Also ensures that once they have authenticated
       # (by accessing /Shibboleth.sso/Login path), then their Shib session is kept alive
       <Location />
         AuthType shibboleth
         ShibRequireSession Off
         require shibboleth
         # If your "shibboleth2.xml" file specifies an <ApplicationOverride> setting for your
         # DSpace Service Provider, then you may need to tell Apache which "id" to redirect Shib requests to.
         # Just uncomment this and change the value "my-dspace-id" to the associated @id attribute value.
         #ShibRequestSetting applicationId my-dspace-id
       </Location>

       # If a user attempts to access the DSpace shibboleth endpoint, force them to authenticate via Shib.
       <Location "/server/api/authn/shibboleth">
         Order deny,allow
         Allow from all
         AuthType shibboleth
         ShibRequireSession On
```

```
         # Please note that setting ShibUseHeaders to "On" is a potential security risk.
         # You may wish to set it to "Off". See the mod_shib docs for details about this setting:
         # https://wiki.shibboleth.net/confluence/display/SHIB2/NativeSPApacheConfig#NativeSPApacheConfig-
AuthConfigOptions
         # Here's a good guide to configuring Apache + Tomcat when this setting is "Off":
         # https://www.switch.ch/de/aai/support/serviceproviders/sp-access-rules.html#javaapplications
         ShibUseHeaders On
         Require shibboleth
       </Location>

       # If a user attempts to access the DSpace login endpoint, ensure Shibboleth is supported but other auth
methods can be too.
       <Location "/server/api/authn/login">
          Order deny,allow
          Allow from all
          AuthType shibboleth
          # For DSpace, this is required to be off otherwise the available auth methods will be not visible
          ShibRequireSession Off
          # Please note that setting ShibUseHeaders to "On" is a potential security risk.
          # You may wish to set it to "Off". See the mod_shib docs for details about this setting:
          # https://wiki.shibboleth.net/confluence/display/SHIB2/NativeSPApacheConfig#NativeSPApacheConfig-
AuthConfigOptions
          # Here's a good guide to configuring Apache + Tomcat when this setting is "Off":
          # https://www.switch.ch/de/aai/support/serviceproviders/sp-access-rules.html#javaapplications
          ShibUseHeaders On
       </Location>

       # Ensure /Shibboleth.sso path (in Apache) can be accessed
       # By default it may be inaccessible if your Apache security is tight.
       <Location "/Shibboleth.sso">
         Order deny,allow
         Allow from all
         # Also ensure Shibboleth/mod_shib responds to this path
         SetHandler shib
       </Location>

       # Finally, you may need to ensure requests to /Shibboleth.sso are NOT redirected
       # to Tomcat (as they need to be handled by mod_shib instead).
       # NOTE: THIS SETTING IS LIKELY ONLY NEEDED IF YOU ARE USING mod_proxy TO REDIRECT
       # ALL REQUESTS TO TOMCAT (e.g. ProxyPass /server ajp://localhost:8009/server)
       ProxyPass /Shibboleth.sso !
   </IfModule>

   ...


   # You will likely need Proxy settings to ensure Apache is proxying requests to Tomcat for the DSpace REST API
   # The below is just an example of proxying for REST API only. It requires installing & enabling "mod_proxy"
and "mod_proxy_ajp"
   ## Proxy / Forwarding Settings ##
   <Proxy *>
      AddDefaultCharset Off
      Order allow,deny
      Allow from all
   </Proxy>

   # Proxy all requests to /server to Tomcat via AJP
   ProxyPass /server ajp://localhost:8009/server
   ProxyPassReverse /server ajp://localhost:8009/server

   # Optionally, also proxy Angular UI (if on same server). This requires "mod_proxy_http"
   #ProxyPass / http://localhost:4000/
   #ProxyPassReverse / http://localhost:4000/
</VirtualHost>
```

Sample shibboleth2.xml Configuration

In addition, here's a sample "ApplicationOverride" configuration for "shibboleth2.xml". This particular "ApplicationOverride" is configured to use the Test IdP provided by https://samltest.id/ and is just meant as an example. In order to enable it for testing purposes, you **must** specify `ShibRequestSetting applicationId samltest` in your Apach `mod_shib` configuration (see above). An additional, more detailed example is provided in our "dspace-shibboleth" Docker configurations at https://github.com/DSpace/DSpace/blob/main/dspace/src/main/docker/dspace-shibboleth/shibboleth2.xml

```
        <!-- *** Sample Shibboleth Settings for  https://samltest.id/  ***     -->
        <!-- This provides a simple sample of how you could configure            -->
        <!-- shibboleth2.xml for DSpace sites.                                   -->
        <!-- TO ENABLE: You'd need to specify "applicationId" as "samltest" in   -->
        <!-- your mod_shib settings, e.g.                                        -->
        <!-- <Location />                                                        -->
        <!--     ...                                                             -->
        <!--     ShibRequestSetting applicationId samltest                       -->
        <!-- </Location>                                                         -->
        <ApplicationOverride id="samltest" entityID="http://[mydspace.edu]/shibboleth" REMOTE_USER="eppn
persistent-id targeted-id">

            <!-- We'll use a TEST IdP, hosted by the awesome https://samltest.id/ testing service. -->
            <!-- See also: https://wiki.shibboleth.net/confluence/display/SHIB2/NativeSPServiceSSO -->
            <!-- DSPACE 7 requires Shibboleth to use "SameSite=None" property for its Cookies -->
            <Sessions lifetime="28800" timeout="3600" checkAddress="false" relayState="ss:mem" handlerSSL="
true" cookieProps="; path=/; SameSite=None; secure; HttpOnly">
               <SSO entityID="https://samltest.id/saml/idp">
                 SAML2 SAML1
               </SSO>
            </Sessions>

            <!-- Loads and trusts a metadata file that describes the IdP and how to communicate with it. -->
            <!-- By default, metadata is retrieved from the TEST IdP at https://samltest.id/ -->
            <!-- and is cached in a local file named "samltest-metadata.xml". -->
            <!-- See also: https://wiki.shibboleth.net/confluence/display/SHIB2/NativeSPMetadataProvider -->
            <MetadataProvider type="XML" uri="https://samltest.id/saml/idp"
                          backingFilePath="samltest-metadata.xml" reloadInterval="180000"/>
        </ApplicationOverride>
```

Sample attribute-map.xml Configuration (for samltest.id)

In order to use the above example for https://samltest.id/, you may also need to modify your attribute-map.xml to support their attributes. Again, a more complete example is in our "dspace-shibboleth" Docker configurations at https://github.com/DSpace/DSpace/blob/main/dspace/src/main/docker/dspace-shibboleth/attribute-map.xml

```
<Attributes xmlns="urn:mace:shibboleth:2.0:attribute-map" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance">
    <!-- Custom Attributes specific to samltest.id -->
    <Attribute name="urn:oid:0.9.2342.19200300.100.1.1" id="uid"/>
    <Attribute name="urn:oid:0.9.2342.19200300.100.1.3" id="mail"/>
    <Attribute name="urn:oid:2.5.4.4" id="sn"/>
    <Attribute name="urn:oid:2.16.840.1.113730.3.1.241" id="displayName"/>
    <Attribute name="urn:oid:2.5.4.20" id="telephoneNumber"/>
    <Attribute name="urn:oid:2.5.4.42" id="givenName"/>
    <Attribute name="https://samltest.id/attributes/role" id="role"/>

    ...


    <!-- In addition to the attribute mapping, DSpace expects the following Shibboleth Headers to be set:
           - SHIB-NETID
           - SHIB-MAIL
           - SHIB-GIVENNAME
           - SHIB-SURNAME
         These are set by mapping the respective IdP attribute (left hand side) to the header attribute (right
hand side).
    -->
    <Attribute name="urn:oid:0.9.2342.19200300.100.1.1" id="SHIB-NETID"/>
    <Attribute name="urn:mace:dir:attribute-def:uid" id="SHIB-NETID"/>

    <Attribute name="urn:oid:0.9.2342.19200300.100.1.3" id="SHIB-MAIL"/>
    <Attribute name="urn:mace:dir:attribute-def:mail" id="SHIB-MAIL"/>

    <Attribute name="urn:oid:2.5.4.42" id="SHIB-GIVENNAME"/>
    <Attribute name="urn:mace:dir:attribute-def:givenName" id="SHIB-GIVENNAME"/>

    <Attribute name="urn:oid:2.5.4.4" id="SHIB-SURNAME"/>
    <Attribute name="urn:mace:dir:attribute-def:sn" id="SHIB-SURNAME"/>


</Attributes>
```

DSpace Shibboleth Configuration Options

Authentication Methods:

DSpace supports authentication using NetID, or email address. A user's NetID is a unique identifier from the IdP that identifies a particular user. The NetID can be of almost any form such as a unique integer, string, or with Shibboleth 2.0 you can use "targeted ids". You will need to coordinate with your shibboleth federation or identity provider. There are three ways to supply identity information to DSpace:

*1) NetID from Shibboleth Header (**best**)*

The NetID-based method is superior because users may change their email address with the identity provider. When this happens DSpace will not be able to associate their new address with their old account.

*2) Email address from Shibboleth Header (**okay**)*

In the case where a NetID header is not available or not found DSpace will fall back to identifying a user based-upon their email address.

*3) Tomcat's Remote User (**worst**)*

In the event that neither Shibboleth headers are found then as a last resort DSpace will look at Tomcat's remote user field. This is the least attractive option because Tomcat has no way to supply additional attributes about a user. Because of this the autoregister option is not supported if this method is used.

Identity Scheme Migration Strategies:

If you are currently using Email based authentication (either 1 or 2) and want to upgrade to NetID based authentication then there is an easy path. Simply enable shibboleth to pass the NetID attribute and set the netid-header below to the correct value. When a user attempts to log in to DSpace first DSpace will look for an EPerson with the passed NetID, however when this fails DSpace will fall back to email based authentication. Then DSpace will update the user's EPerson account record to set their NetID so all future authentications for this user will be based upon NetID. One thing to note is that DSpace will prevent an account from switching NetIDs. If an account already has a NetID set and then they try and authenticate with a different NetID the authentication will fail.

EPerson Metadata:

One of the primary benefits of using Shibboleth based authentication is receiving additional attributes about users such as their names, telephone numbers, and possibly their academic department or graduation semester if desired. DSpace treats the first and last name attributes differently because they (along with email address) are the three pieces of minimal information required to create a new user account. For both first and last name supply direct mappings to the Shibboleth headers. In additional to the first and last name DSpace supports other metadata fields such as phone, or really anything you want to store on an eperson object. Beyond the phone field, which is accessible in the user's profile screen, none of these additional metadata fields will be used by DSpace out-of-the box. However if you develop any local modification you may access these attributes from the EPerson object. The Vireo ETD workflow system utilizes this to aid students when submitting an ETD.

Role-based Groups:

DSpace is able to place users into pre-defined groups based upon values received from Shibboleth. Using this option you can place all faculty members into a DSpace group when the correct affiliation's attribute is provided. When DSpace does this they are considered 'special groups', these are really groups but the user's membership within these groups is not recorded in the database. Each time a user authenticates they are automatically placed within the pre-defined DSpace group, so if the user loses their affiliation then the next time they login they will no longer be in the group.

Depending upon the shibboleth attributed use in the role-header it may be scoped. Scoped is shibboleth terminology for identifying where an attribute originated from. For example a students affiliation may be encoded as "student@tamu.edu". The part after the @ sign is the scope, and the preceding value is the value. You may use the whole value or only the value or scope. Using this you could generate a role for students and one institution different than students at another institution. Or if you turn on ignore-scope you could ignore the institution and place all students into one group.

The values extracted (a user may have multiple roles) will be used to look up which groups to place the user into. The groups are defined as "authentication -shibboleth.role.<role-name>" which is a comma separated list of DSpace groups.

Having issues getting Safari working?

In addition to the below settings, you may need to ensure your Shibboleth IdP is *trusted* by the DSpace backend by adding it to your `rest.cors.allowed-origins` configuration. *This is required for Safari web browsers to work with DSpace's Shibboleth plugin.*

For example, if your IdP is https://samltest.id/, then you need to append that URL to the comma-separated list of "allowed-origins" like:

```
rest.cors.allowed-origins = ${dspace.ui.url}, https://samltest.id
```

More information on this configuration can be found in the REST API documentation.

Configuration File:	**`[dspace]/config/modules/authentication-shibboleth.cfg`**
Property:	`authentication-shibboleth.lazysession`
Example Value:	`authentication-shibboleth.lazysession = true`
Informational Note:	Whether to use lazy sessions or active sessions. For more DSpace instances, you will likely want to use lazy sessions. Active sessions will force **every** user to authenticate via Shibboleth before they can access your DSpace (essentially resulting in a "dark archive").
Property:	`authentication-shibboleth.lazysession.loginurl`
Example Value:	authentication-shibboleth.`lazysession.loginurl = /Shibboleth.sso/Login`
Informational Note:	The url to start a shibboleth session (only for lazy sessions). Generally this setting will be "/Shibboleth.sso/Login"
Property:	`authentication-shibboleth.lazysession.secure`
Example Value:	`authentication-shibboleth.lazysession.secure = true`
Informational Note:	Force HTTPS when authenticating (only for lazy sessions). Generally this is recommended to be "true".
Property:	`authentication-shibboleth.netid-header`
Example Value:	`authentication-shibboleth.netid-header = SHIB-NETID`
Informational Note:	The HTTP header where shibboleth will supply a user's NetID. This HTTP header should be specified as an Attribute within your Shibboleth "attribute-map.xml" configuration file.
Property:	`authentication-shibboleth.email-header`
Example Value:	`authentication-shibboleth.email-header = SHIB-MAIL`
Informational Note:	The HTTP header where the shibboleth will supply a user's email address. This HTTP header should be specified as an Attribute within your Shibboleth "attribute-map.xml" configuration file.
Property:	`authentication-shibboleth.email-use-tomcat-remote-user`

Example Value:	`authentication-shibboleth.email-use-tomcat-remote-user = false`
Informational Note:	Used when a netid or email headers are not available should Shibboleth authentication fall back to using Tomcat's remote user feature? Generally this is not recommended. See the "Authentication Methods" section above.
Property:	`authentication-shibboleth.reconvert.attributes`
Example Value	`authentication-shibboleth.reconvert.attributes = false`
Informational Note:	Shibboleth attributes are by default UTF-8 encoded. Some servlet container automatically converts the attributes from ISO-8859-1 (latin-1) to UTF-8. As the attributes already were UTF-8 encoded it may be necessary to reconvert them. If you set this property true, DSpace converts all shibboleth attributes retrieved from the servlet container from UTF-8 to ISO-8859-1 and uses the result as if it were UTF-8. This procedure restores the shibboleth attributes if the servlet container wrongly converted them from ISO-8859-1 to UTF-8. Set this true, if you notice character encoding problems within shibboleth attributes.
Property:	`authentication-shibboleth.autoregister`
Example Value:	`authentication-shibboleth.autoregister = true`
Informational Note:	Should we allow new users to be registered automatically?
Property:	`authentication-shibboleth.sword.compatibility`
Example Value:	`authentication-shibboleth.sword.compatibility = false`
Informational Note:	SWORD compatibility will allow this authentication method to work when using SWORD. SWORD relies on username and password based authentication and is entirely incapable of supporting shibboleth. This option allows you to authenticate username and passwords for SWORD sessions with out adding another authentication method onto the stack. You will need to ensure that a user has a password. One way to do that is to create the user via the create-administrator command line command and then edit their permissions. WARNING: If you enable this option while ALSO having "PasswordAuthentication" enabled, then you should ensure that "PasswordAuthentication" is listed prior to "ShibAuthentication" in your authentication.cfg file. Otherwise, ShibAuthentication will be used to authenticate all of your users INSTEAD OF PasswordAuthentication.
Property:	`authentication-shibboleth.firstname-header`
Example Value:	`authentication-shibboleth.firstname-header = SHIB_GIVENNAME`
Informational Note:	The HTTP header where the shibboleth will supply a user's given name. This HTTP header should be specified as an Attribute within your Shibboleth "attribute-map.xml" configuration file.
Property:	`authentication-shibboleth.lastname-header`
Example Value:	`authentication-shibboleth.lastname-header = SHIB_SN`
Informational Note:	The HTTP header where the shibboleth will supply a user's surname. This HTTP header should be specified as an Attribute within your Shibboleth "attribute-map.xml" configuration file.
Property:	`authentication-shibboleth.eperson.metadata`

Example Value:	`authentication-shibboleth.eperson.metadata = \`
	` SHIB-telephone => phone, \`
	` SHIB-cn => cn`

Informational Note:	Additional user attributes mapping, multiple attributes may be stored for each user. The left side is the Shibboleth-based metadata Header and the right side is the eperson metadata field to map the attribute to.
Property:	`authentication-shibboleth.eperson.metadata.autocreate`
Example Value:	`authentication-shibboleth.eperson.metadata.autocreate = true`
Informational Note:	If the eperson metadata field is not found, should it be automatically created?
Property:	`authentication-shibboleth.role-header`
Example Value:	`authentication-shibboleth.role-header = SHIB_SCOPED_AFFILIATION`
Informational Note:	The Shibboleth header holding the user's Shibboleth roles. See the "Role-based Groups" section above for more info.
Property:	`authentication-shibboleth.role-header.ignore-scope`

Example Value:	`authentication-shibboleth.role-header.ignore-scope = true`
Informational Note:	Whether to ignore roles' scopes (everything after the @ sign for scoped attributes)
Property:	`authentication-shibboleth.role-header.ignore-value`
Example Value:	`authentication-shibboleth.role-header.ignore-value = false`
Informational Note:	Whether to ignore roles' values (everything before the @ sign for scoped attributes)
Property:	`authentication-shibboleth.role.[affiliation-attribute]`

Example Value:	`authentication-shibboleth.role.faculty = Faculty, Member`
	`authentication-shibboleth.role.staff = Staff, Member`
	`authentication-shibboleth.role.student = Students, Member`

Informational Note:	Mapping of affiliation values to DSpace groups. See the "Role-based Groups" section above for more info.
Property:	`authentication-shibboleth.default-roles`
Example Value:	`authentication-shibboleth.default-roles = GenericUser`
Informational Note:	These roles are assumed if no roles were sent by Shibboleth or there was no header with name matching the value of `authentication-shibboleth.role_header`. May be repeated to provide multiple default roles.

LDAP Authentication

Introduction to LDAP specific terminology

If you are unfamiliar with LDAP, the following introduction to some of its terminology might come in handy:

https://stackoverflow.com/questions/18756688/what-are-cn-ou-dc-in-an-ldap-search

Enabling LDAP Authentication

To enable LDAP Authentication, you must ensure the `org.dspace.authenticate.LDAPAuthentication` class is listed as one of the AuthenticationMethods in the following configuration:

Configuration File:	**`[dspace]/config/modules/authentication.cfg`**
Property:	`plugin.sequence.org.dspace.authenticate.AuthenticationMethod`
Example Value:	`plugin.sequence.org.dspace.authenticate.AuthenticationMethod = org.dspace.authenticate.LDAPAuthentication`

Configuring LDAP Authentication

If LDAP is enabled, then new users will be able to register by entering their username and password without being sent the registration token. If users do not have a username and password, then they can still register and login with just their email address the same way they do now.

If you want to give any special privileges to LDAP users, create a stackable authentication method to automatically put people who have a netid into a special group. You might also want to give certain email addresses special privileges. Refer to the Custom Authentication Code section below for more information about how to do this.

Ensure required commas are escaped in LDAP configuration

NOTE: As of DSpace 6, commas (,) are now a special character in the Configuration system. As some LDAP configuration may contain commas, you must be careful to escape any required commas by adding a backslash (\) before each comma, e.g. "\,". The configuration reference for authentication-ldap.cfg has been updated below with additional examples.

Here is an explanation of each of the different LDAP configuration parameters:

Configuration File:	**`[dspace]/config/modules/authentication-ldap.cfg`**
Property:	`authentication-ldap.enable`
Example Value:	`authentication-ldap.enable = false`
Informational Note:	This setting will enable or disable LDAP authentication in DSpace. With the setting off, users will be required to register and login with their email address. With this setting on, users will be able to login and register with their LDAP user ids and passwords.
Property:	`authentication-ldap.autoregister`
Example Value:	`authentication-ldap.autoregister = true`
Informational Note:	This will turn LDAP autoregistration on or off. With this on, a new EPerson object will be created for any user who successfully authenticates against the LDAP server when they first login. With this setting off, the user must first register to get an EPerson object by entering their ldap username and password and filling out the forms.
Property:	`authentication-ldap.provider_url`
Example Value:	`authentication-ldap.provider_url = ldap://ldap.myu.edu/o=myu.edu\,ou=mydept`
Informational Note:	This is the url to your institution's LDAP server. You may or may not need the /o=myu.edu part at the end. Your server may also require the ldaps:// protocol. (This field has no default value) **NOTE:** As of DSpace 6, commas (,) are now a special character in the Configuration system. Therefore, be careful to escape any required commas in this configuration by adding a backslash (\) before each comma, e.g. "\,"
Property:	`authentication-ldap.starttls`
Example Value:	`authentication-ldap.starttls = false`
Informational Note:	Should we issue StartTLS after establishing TCP connection in order to initiate an encrypted connection? Note: This (TLS) is different from LDAPS: • TLS is a tunnel for plain LDAP and is typically recognized on the same port (standard LDAP port: 389) • LDAPS is a separate protocol, deprecated in favor of the standard TLS method. (standard LDAPS port: 636)
Property:	`authentication-ldap.id_field`
Example Value:	`authentication-ldap.id_field = uid`
Explanation:	This is the unique identifier field in the LDAP directory where the username is stored. (This field has no default value)
Property:	`authentication-ldap.object_context`
Example Value:	`authentication-ldap.object_context = ou=people\,o=myu.edu`
Informational Note:	This is the LDAP object context to use when authenticating the user. By default, DSpace will use this value to create the user's DN in order to attempt to authenticate them. It is appended to the *id_field* and username. For example `uid=username\,ou=people\,o=myu.edu`. You will need to modify this to match your LDAP configuration. (This field has no default value) If your users do NOT all exist under a single "object_context" in LDAP, then you should ignore this setting and INSTEAD use the Hierarchical LDAP Authentication settings below (especially see "`search.user`" or "`search.anonymous`") **NOTE:** As of DSpace 6, commas (,) are now a special character in the Configuration system. Therefore, be careful to escape any required commas in this configuration by adding a backslash (\) before each comma, e.g. "\,"
Property:	`authentication-ldap.search_context`
Example Value:	`authentication-ldap.search_context = ou=people`
Informational Note:	This is the search context used when looking up a user's LDAP object to retrieve their data for autoregistering. With `autoregister=true`, when a user authenticates without an EPerson object we search the LDAP directory to get their name (`id_field`) and email address (`email_field`) so that we can create one for them. So after we have authenticated against *uid=username, ou=people,o=byu.edu* we now search in ou=people for filtering on [uid=username]. Often the `search_context` is the same as the `object_context` parameter. But again this depends on your LDAP server configuration. (This field has no default value, and it MUST be specified when either `search.anonymous=true` or `search.user` is specified) **NOTE:** As of DSpace 6, commas (,) are now a special character in the Configuration system. Therefore, be careful to escape any required commas in this configuration by adding a backslash (\) before each comma, e.g. "\,"
Property:	`authentication-ldap.email_field`

Example Value:	`authentication-ldap.email_field = mail`
Informational Note:	This is the LDAP object field where the user's email address is stored. "mail" is the most common for LDAP servers. (This field has no default value) If the "email_field" is unspecified, or the user has no email address in LDAP, his/her username (id_field value) will be saved as the email in DSpace (or appended to `netid_email_domain`, when specified)
Property:	`authentication-ldap.netid_email_domain`
Example Value:	`authentication-ldap.netid_email_domain = @example.com`
Informational Note:	If your LDAP server does not hold an email address for a user (i.e. no `email_field`), you can use the following field to specify your email domain. This value is appended to the netid (`id_field`) in order to make an email address (which is then stored in the DSpace EPerson). For example, a netid of 'user' and `netid_email_domain` as `@example.com` would set the email of the user to be user@example.com *Please note:* this field will only be used if "`email_field`" is unspecified OR the user in question has no email address stored in LDAP. If both "`email_field`" and "`netid_email_domain`" are unspecified, then the "`id_field`" will be used as the email address.
Property:	`authentication-ldap.surname_field`
Example Value:	`authentication-ldap.surname_field = sn`
Informational Note:	This is the LDAP object field where the user's last name is stored. "`sn`" is the most common for LDAP servers. If the field is not found the field will be left blank in the new eperson object. (This field has no default value)
Property:	`authentication-ldap.givenname_field`
Example Value:	`authentication-ldap.givenname_field = givenName`
Informational Note:	This is the LDAP object field where the user's given names are stored. I'm not sure how common the givenName field is in different LDAP instances. If the field is not found the field will be left blank in the new eperson object. (This field has no default value)
Property:	`authentication-ldap.phone_field`
Example Value:	`authentication-ldap.phone_field = telephoneNumber`
Informational Note:	This is the field where the user's phone number is stored in the LDAP directory. If the field is not found the field will be left blank in the new eperson object. (This field has no default value)
Property:	`authentication-ldap.login.specialgroup`
Example Value:	`authentication-ldap.login.specialgroup = group-name`
Informational Note:	If specified, all user sessions successfully logged in via LDAP will automatically become members of this DSpace Group (for the remainder of their current, logged in session). This DSpace Group **must** already exist (it will not be automatically created). This is useful if you want a DSpace Group made up of all internal authenticated users. This DSpace Group can then be used to bestow special permissions on any users who have authenticated via LDAP (e.g. you could allow anyone authenticated via LDAP to view special, on campus only collections or similar)
Property:	`login.groupmap.*`

Example Value:	`authentication-ldap.login.groupmap.1 = ou=Students:ALL_STUDENTS`
	`authentication-ldap.login.groupmap.2 = ou=Employees:ALL_EMPLOYEES`
	`authentication-ldap.login.groupmap.3 = ou=Faculty:ALL_FACULTY`

Informational Note:	The left part of the value (before the ":") must correspond to a portion of a user's DN (unless "`login.group.attribute`" is specified..please see below). The right part of the value corresponds to the name of an existing DSpace group.
	
	For example, if the authenticated user's DN in LDAP is in the following form:
	
	`cn=jdoe,OU=Students,OU=Users,dc=example,dc=edu`
	
	that user would get assigned to the ALL_STUDENTS DSpace group for the remainder of their current session.
	
	However, if that same user later graduates and is employed by the university, their DN in LDAP may change to:
	
	`cn=jdoe,OU=Employees,OU=Users,dc=example,dc=edu`
	
	Upon logging into DSpace after that DN change, the authenticated user would now be assigned to the ALL_EMPLOYEES DSpace group for the remainder of their current session.
	
	Note: This option can be used independently from the login.specialgroup option, which will put all LDAP users into a single DSpace group. Both options may be used together.

Property:	`authentication-ldap.login.groupmap.attribute`
Example Value:	`authentication-ldap.login.groupmap.attribute = group`
Informational Note:	The value of the "`authentication-ldap.login.groupmap.attribute`" should specify the name of a single LDAP attribute. If this property is uncommented, it changes the meaning of the left part of "`authentication-ldap.login.groupmap.*`" (see above) as follows: • If the authenticated user has this LDAP attribute, look up the value of this LDAP attribute in the left part (before the ":") of the `authentication-ldap.login.groupmap.*` value • If that LDAP value is found in any "`authentication-ldap.login.groupmap.*`" field, assign this authenticated user to the DSpace Group specified by the right part (after the ":") of the `authentication-ldap.login.groupmap.*` value. For example: • `authentication-ldap.login.groupmap.attribute = group` • `authentication-ldap.login.groupmap.1 = mathematics:Mathematics_Group` The above would ensure that any authenticated users where their LDAP "group" attribute equals "mathematics" would be added to the DSpace Group named "Mathematics_Group" for the remainder of their current session. However, if that same user logged in later with a new LDAP "group" value of "computer science", he/she would no longer be a member of the "Mathematics_Group" in DSpace.

Debugging LDAP connection and configuration

As every LDAP is different, configuring your DSpace to communicate with your LDAP can sometimes be a challenge. We recommend using third-party LDAP tools to test your LDAP connection / username / password, and perform sample searches to better understand what information is being returned from your local LDAP. This will help ensure that LDAP configuration goes more smoothly.

One example of such an LDAP tool is the `ldapsearch` commandline tool available in most Linux operating systems (e.g. in Debian / Ubuntu it's available in the "ldap-utils" package). Below are some example ldapsearch commands that can be used to determine (and/or debug) specific configurations in your `authentication-ldap.cfg`. In the below examples, we've used the names of specific DSpace configurations as placeholders (in square brackets).

```
# Basic anonymous connection (for VERBOSE, add -v)
ldapsearch -x -H [provider_url]

# Debug a connection error (add -d-1)
# If you are connecting to an LDAPS URL and see connection errors (e.g. "peer cert untrusted or revoked")
# then see below note about "SSL Connection Errors"
ldapsearch -x -H [provider_url] -d-1


# Attempt to connect to [provider_url] as [search.user] (will prompt for search.user's password)
# This doesn't actually perform a query, just ensures that authentication is working
# NOTE: "search.user" is USUALLY either the full user DN (e.g. "cn=dspaceadmin,ou=people,o=myu.edu")
# or "DOMAIN\USERNAME" (e.g. "MYU\DSpaceUser"). The latter is more likely with Windows Active Directory
ldapsearch -x -H [provider_url] -D [search.user] -W


# Attempt to list the first 100 users in a given [search_context], returning the "cn", "mail" and "sn" fields
for each
ldapsearch -x -H [provider_url] -D [search.user] -W -b [search_context] -z 100 cn mail sn


# Attempt to find the first 100 users whose [id_field] starts with the letter "t", returning the [id_field],
"cn", "mail" and "sn" fields for each
ldapsearch -x -H [provider_url] -D [search.user] -W -b [search_context] -z 100 -s sub "([id_field]=t*)"
[id_field] cn mail sn
```

SSL Connection Errors: If you are using ldapsearch with an LDAPS connection (secure connection), you may receive "peer cert untrusted or revoked" errors if the LDAP SSL certificate is self-signed. You can temporarily tell LDAP to accept any security certificate by setting `TLS_REQCERT allow` in your ldapsearch's `ldap.conf` file. *Be sure to remove this setting however after you are done testing!*

```
# FOR TESTING ONLY! This setting disables the check for a valid LDAP Server security certificate,
# which is considered a security issue for production LDAP setups. Setting this to "allow" tells
# the LDAP client to accept any security certificates that it cannot verify or validate.
TLS_REQCERT allow
```

More information on this SSL workaround can be found at:

- http://www.bind9.net/manual/openldap/2.3/tls.html
- http://muzso.hu/2012/03/29/how-to-configure-ssl-aka.-ldaps-for-libnss-ldap-auth-client-config-in-ubuntu

Enabling Hierarchical LDAP Authentication

Please note, that DSpace doesn't contain the `LDAPHierarchicalAuthentication` class anymore. This functionality is now supported by `LDAPAuthentication`, which uses the same configuration options.

If your users are spread out across a hierarchical tree on your LDAP server, you may wish to have DSpace search for the user name in your tree. Here's how it works:

1. DSpace gets the user name from the login form
2. DSpace binds to LDAP as an administrative user with right to search in DNs (LDAP may be configured to allow anonymous users to search)
3. DSpace searches for the user name as within DNs (username is a part of full DN)
4. DSpace binds with the found full DN and password from login form
5. DSpace logs user in if LDAP reports successful authentication; refuses login otherwise

Configuring Hierarchical LDAP Authentication

Hierarchical LDAP Authentication shares all the above standard LDAP configurations, but has some additional settings.

You can optionally specify the search scope. If anonymous access is not enabled on your LDAP server, you will need to specify the full DN and password of a user that is allowed to bind in order to search for the users.

Configuration File:	**`[dspace]/config/modules/authentication-ldap.cfg`**
Property:	`authentication-ldap.search_scope`
Example Value:	`authentication-ldap.search_scope = 2`

Informational Note:	This is the search scope value for the LDAP search during autoregistering (`autoregister=true`). This will depend on your LDAP server setup, and is only really necessary if your users are spread out across a hierarchical tree on your LDAP server. This value must be one of the following integers corresponding to the following values:
	`object scope : 0`
	`one level scope : 1`
	`subtree scope : 2`
	
	Please note that "`search_context`" in the LDAP configurations must also be specified.

Property:	`authentication-ldap.search.anonymous`
Example Value:	`authentication-ldap.search.anonymous = true`
Informational Note:	If true, DSpace will anonymously search LDAP (in the "`search_context`") for the DN of the user trying to login to DSpace. This setting is "false" by default. By default, DSpace will either use "`search.user`" to authenticate for the LDAP search (if search.user is specified), or will use the "`object_context`" value to create the user's DN.
Property:	`authentication-ldap.search.user` `authentication-ldap.search.password`
Example Value:	`authentication-ldap.search.user = cn=admin\,ou=people\,o=myu.edu` `authentication-ldap.search.password = password`
Informational Note:	The full DN and password of a user allowed to connect to the LDAP server and search (in the "`search_context`") for the DN of the user trying to login. By default, if unspecified, DSpace will either search LDAP anonymously for the user's DN (when `search.anonymous=true`), or will use the "`object_context`" value to create the user's DN. **NOTE:** As of DSpace 6, commas (,) are now a special character in the Configuration system. Therefore, be careful to escape any required commas in this configuration by adding a backslash (\) before each comma, e.g. "\,"

ORCID Authentication

Enabling ORCID Authentication

Enabling ORCID Authentication **requires** also enabling Configurable Entities and Researcher Profiles

To enable ORCID Authentication, see the documentation for enabling the ORCID Integration. You do not need to enable ORCID synchronization, but you currently must enable Researcher Profiles and Configurable Entities.

IP Authentication

Enabling IP Authentication

To enable IP Authentication, you must ensure the `org.dspace.authenticate.IPAuthentication` class is listed as one of the AuthenticationMethods in the following configuration:

Configuration File:	**[dspace]/config/modules/authentication.cfg**
Property:	`plugin.sequence.org.dspace.authenticate.AuthenticationMethod`
Example Value:	`plugin.sequence.org.dspace.authenticate.AuthenticationMethod = org.dspace.authenticate.IPAuthentication`

Configuring IP Authentication

Configuration File:	**[dspace]/config/modules/authentication-ip.cfg**

Once enabled, you are then able to map DSpace groups to IP addresses in `authentication-ip.cfg` by setting `ip.GROUPNAME = iprange[, iprange ...]`, e.g:

```
authentication-ip.MY_UNIVERSITY = 10.1.2.3, \                # Full IP
13.5, \                      # Partial IP
11.3.4.5/24, \               # with CIDR
12.7.8.9/255.255.128.0, \    # with netmask
2001:18e8::32                # IPv6 too
```

Negative matches can be set by prepending the entry with a '-'. For example if you want to include all of a class B network except for users of a contained class c network, you could use: 111.222,-111.222.333.

Notes:

- If the Groupname contains blanks you must escape the spaces, e.g. "Department\ of\ Statistics"
- If your DSpace installation is hidden behind a web proxy, remember to set the `useProxies` configuration option within the 'Logging' section of `dspace.cfg` to use the IP address of the user rather than the IP address of the proxy server.

X.509 Certificate Authentication

Obsolete. X.509 Certificate Authentication was removed in DSpace 7.6.6. See https://github.com/DSpace/DSpace/pull/11567

Enabling X.509 Certificate Authentication

The X.509 authentication method uses an X.509 certificate sent by the client to establish his/her identity. It requires the client to have a personal Web certificate installed on their browser (or other client software) which is issued by a Certifying Authority (CA) recognized by the web server.

1. See the HTTPS installation instructions to configure your Web server. If you are using HTTPS with Tomcat, note that the `<Connector>` tag *must* include the attribute `clientAuth="true"` so the server requests a personal Web certificate from the client.
2. Add the `org.dspace.authenticate.X509Authentication` plugin `first` to the list of stackable authentication methods in the value of the configuration key `plugin.sequence.org.dspace.authenticate.AuthenticationMethod`

Configuration File:	**[dspace]/config/modules/authentication.cfg**
Property:	`plugin.sequence.org.dspace.authenticate.AuthenticationMethod`
Example Value:	`plugin.sequence.org.dspace.authenticate.AuthenticationMethod = org.dspace.authenticate.X509Authentication` `plugin.sequence.org.dspace.authenticate.AuthenticationMethod = org.dspace.authenticate.PasswordAuthentication`

Configuring X.509 Certificate Authentication

Configuration File:	**[dspace]/config/modules/authentication-x509.cfg**

1. You must also configure DSpace with the same CA certificates as the web server, so it can accept and interpret the clients' certificates. It can share the same keystore file as the web server, or a separate one, or a CA certificate in a file by itself. Configure it by *one*of these methods, either the Java keystore

```
authentication-x509.keystore.path =  path to Java keystore file
authentication-x509.keystore.password =  password to access the keystore
```

   ...or the separate CA certificate file (in PEM or DER format):

```
authentication-x509.ca.cert =  path to certificate file for CA whose client certs to accept.
```

2. Choose whether to enable auto-registration: If you want users who authenticate successfully to be automatically registered as new E-Persons if they are not already, set the `autoregister` configuration property to `true`. This lets you automatically accept all users with valid personal certificates. The default is `false`.

TODO: document the remaining authentication-x509.* properties

Example of a Custom Authentication Method

Also included in the source is an implementation of an authentication method used at MIT, *edu.mit.dspace.MITSpecialGroup*. This does not actually authenticate a user, it *only* adds the current user session to a special (dynamic) group called 'MIT Users' (which must be present in the system!). This allows us to create authorization policies for MIT users without having to manually maintain membership of the MIT users group.

By keeping this code in a separate method, we can customize the authentication process for MIT by simply adding it to the stack in the DSpace configuration. None of the code has to be touched.

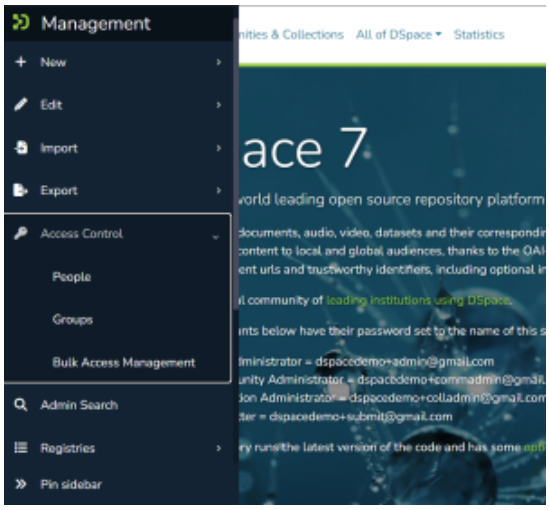
You can create your own custom authentication method and add it to the stack. Use the most similar existing method as a model, e.g. `org.dspace.authenticate.PasswordAuthentication` for an "explicit" method (with credentials entered interactively) or `org.dspace.authenticate.X509Authentication` for an implicit method.

Bulk Access Management

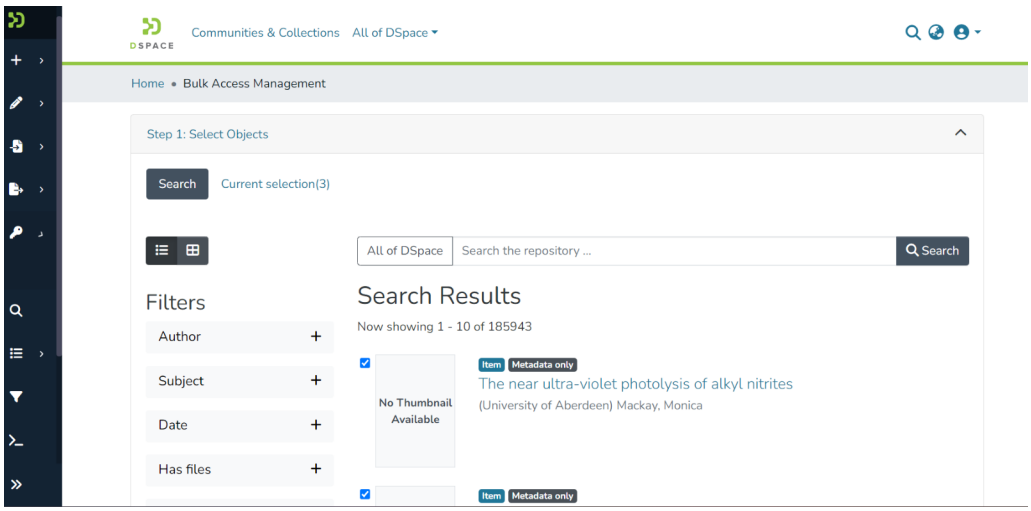
The Bulk Access Management will let administrators edit massively the access conditions of Metadata and Bitstreams on selected objects.

Usage:

- When logged as Administrator, it is possible to change the access condition to Metadata and Bitstreams on items by following the path: "Management > Access Control > Bulk Access Management".
- Community or Collection Administrators may also use this tool from the "Access Control" tab of the "Edit Community", "Edit Collection" or "Edit Item" page. In this scope, the tool will only perform changes within the selected Community/Collection/Item.



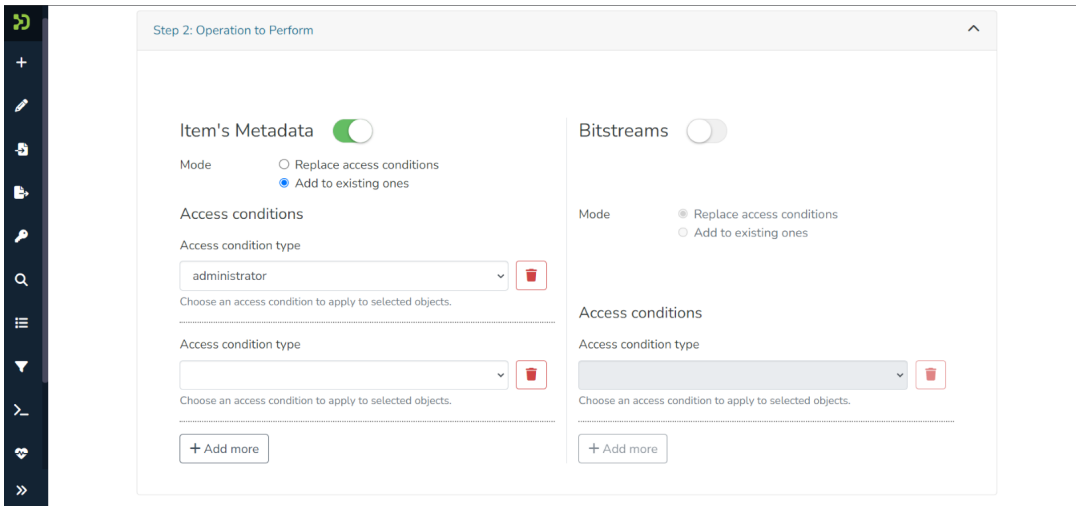
In the first box (Step 1) should be selected the objects on which access conditions will be changed.



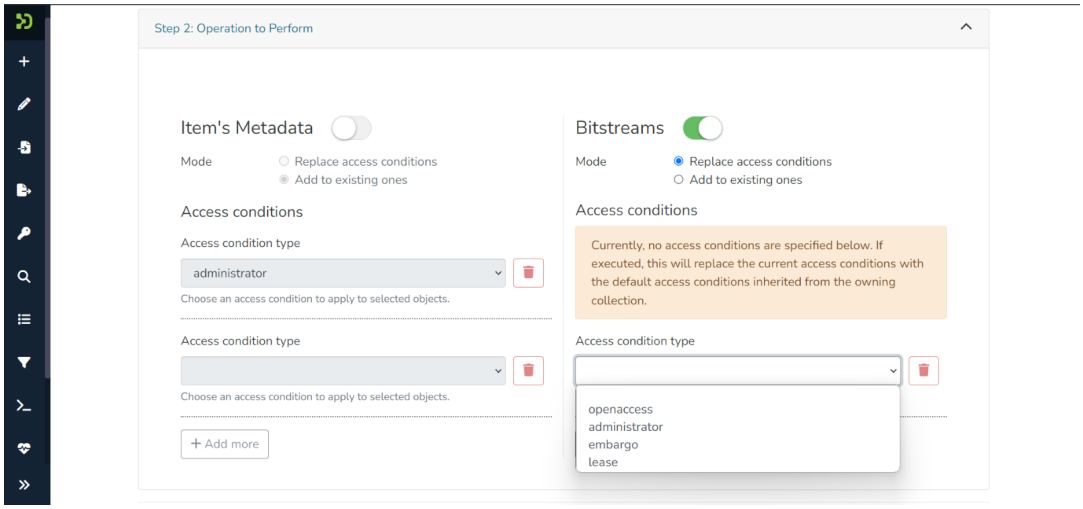
In the second box (Step 2) the administrator will choose if change the access conditions on Metadata, or on Bitstreams or both.

The actions that can be performed are:

- Replacing the existent access conditions
- Add new conditions to the existing ones



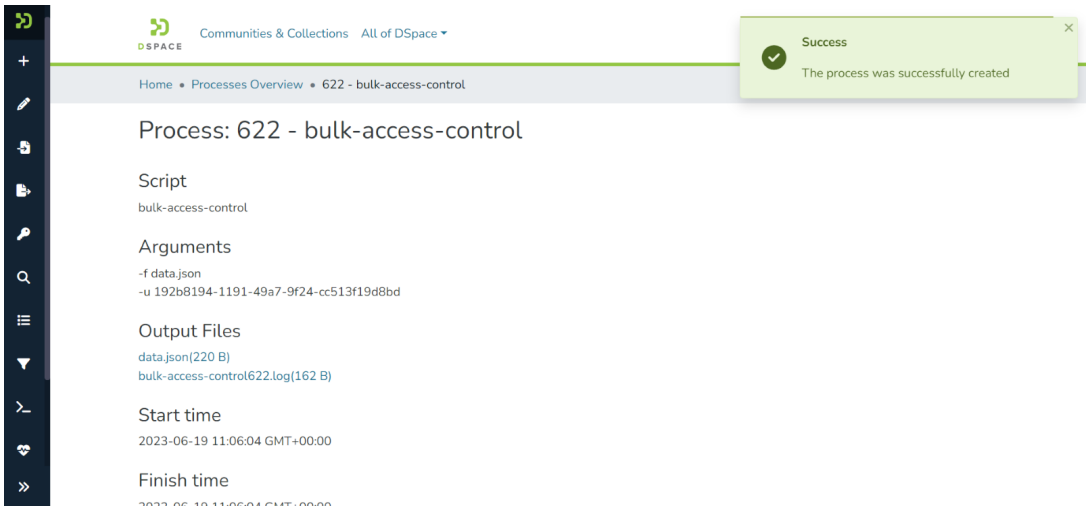
If there is no previous access condition defined, a warning box will appear.



The access conditions that can be chosen by default are:

- Openaccess
- Administrator
- Embargo
- Lease

When done, click Execute. The process will start. If it succeeded, the process page and a success message will show.



At the moment, DSpace supports a single feature configuration defined by the defaultBulkAccessConditionConfiguration, in which are specified the access conditions available for the Item's and the Bitstream's Metadata.

The access conditions listed in the dropdown menu are set by default as Openaccess, Administrator, Embargo, and Lease.

They can be edited by changing and adding options in the code:

```
<bean id="defaultBulkAccessConditionConfiguration"
        class="org.dspace.app.bulkaccesscontrol.model.BulkAccessConditionConfiguration">
      <property name="name" value="default"/>
      <property name="itemAccessConditionOptions">
          <list>
              <ref bean="openAccess"/>
              <ref bean="administrator"/>
              <ref bean="embargoed" />
              <ref bean="lease"/>
          </list>
      </property>
      <property name="bitstreamAccessConditionOptions">
          <list>
              <ref bean="openAccess"/>
              <ref bean="administrator"/>
              <ref bean="embargoed" />
              <ref bean="lease"/>
          </list>
      </property>
  </bean>

  <bean id="bulkAccessConditionConfigurationService"
        class="org.dspace.app.bulkaccesscontrol.service.BulkAccessConditionConfigurationService">
      <property name="bulkAccessConditionConfigurations">
          <list>
              <ref bean="defaultBulkAccessConditionConfiguration"/>
          </list>
      </property>
  </bean>
```

Please refer to this page for further information: https://github.com/DSpace/DSpace/blob/main/dspace/config/spring/api/access-conditions.xml#L78-L106

Embargo

1 What is an Embargo?
2 DSpace Embargo Functionality
    2.1 Managing Embargoes on existing Items
3 Configuring and using Embargo in DSpace Submission User Interface
    3.1 Enabling Item-level Embargo
    3.2 Configuring Embargo / Access Restriction options
    3.3 Private/Public (or Non-Discoverable/Discoverable) Item
    3.4 Pre-3.0 Embargo Migration Routine
4 Technical Specifications
    4.1 Introduction
    4.2 ResourcePolicy
    4.3 Item
    4.4 Item.inheritCollectionDefaultPolicies(Collection c)
    4.5 AuthorizeService
    4.6 Withdraw Item
    4.7 Reinstate Item
    4.8 Pre-DSpace 3.0 Embargo Compatibility
5 Creating Embargoes via Metadata
    5.1 Introduction
    5.2 Setting Embargo terms via metadata
        5.2.1 Terms assignment
        5.2.2 Terms interpretation/imposition
        5.2.3 Embargo period
    5.3 Configuration of metadata fields
    5.4 Operation
    5.5 Extending embargo functionality
        5.5.1 Setter
        5.5.2 Lifter

What is an Embargo?

An embargo is a temporary access restriction placed on metadata or bitstreams (i.e. files). Its scope or duration may vary, but the fact that it eventually expires is what distinguishes it from other content restrictions. For example, it is not unusual for content destined for DSpace to come with permanent restrictions on use or access based on license-driven or other IP-based requirements that limit access to institutionally affiliated users. Restrictions such as these are imposed and managed using standard administrative tools in DSpace, typically by attaching specific access policies (aka "resource policies") to Items, Collections, Bitstreams, etc.

Embargo functionality was originally introduced as part of DSpace 1.6, enabling embargoes on the level of items that applied to all bitstreams included in the item. Since DSpace 3.0, this functionality has been extended to the Submission User Interface, enabling embargoes on the level of individual bitstreams.

DSpace Embargo Functionality

Embargoes can be applied per *item (including metadata)* and per *bitstream (i.e. file)*. The *item* level embargo will be the default for every *bitstream*, although it could be customized at *bitstream* level.

When an embargo is set on either an item level or a bitstream level, a new ResourcePolicy (i.e. access policy) is added to the corresponding Item or Bitstream. **This ResourcePolicy will automatically control the lifting of the embargo (when the embargo date passes).** An embargo lift date is generally stored as the "start date" of such a policy. Essentially, this means that the access rights defined in the policy do not get applied until *after that date passes* (and prior to that date, the access rights will default to Admin only).

The scheduled, manual "embargo-lifter" commands (used prior to DSpace 3) are no longer necessary and not recommended to run.

Managing Embargoes on existing Items

Administrators are able to change the lift date of any embargo by editing the authorization policy (ResourcePolicy) on the object. These authorization policies can be managed from the Edit Item screen by clicking on "Authorizations".

- To add an embargo, edit the appropriate policy and set a "start date". To add an full Item embargo (including metadata), edit the Item policy. To embargo individual bitstreams, edit the appropriate Bitstream policy.
- To remove an embargo, edit the appropriate policy, and clear out the "start date".
- To change an embargo, edit the appropriate policy, and change the "start date" to a new date.

Changes to the embargo should take effect immediately. However, as Administrators have full access to embargoed items, you may need to log out first. After logging out, you will be subject to the embargo.

Configuring and using Embargo in DSpace Submission User Interface

Available in DSpace 7.2 and above

In DSpace 7.2 and above, both Item-level embargoes and bitstream (file) level embargoes are supported in the Submission user interface.
In DSpace 7.1 and 7.0, the Submission user interface only supported embargoes on specified bitstreams (files). However, item-level embargoes could be added after submissions were accepted using the "Manage Embargoes on existing Items" approach described above.

Enabling Item-level Embargo

While Bitstream-level embargoes are enabled by default, Item-level embargoes currently are not. However, enabling them is easy. Simply update your `item-submission.xml` to include this tag in your <submission-process>:

```
<submission-process name="traditional">
   ...

   <!-- This step enables embargoes and other access restrictions at the Item level -->
   <step id="itemAccessConditions"/>
</submission-process>
```

After making this update, you will need to restart your backend (REST API) for the changes to take effect.

Configuring Embargo / Access Restriction options

Starting in DSpace 7, embargo (and lease) settings are configurable via a Spring Bean configuration file `[dspace]/config/spring/api/access-conditions.xml`

For detailed information on configuring your Embargo options (and other related options like lease or restrict to a particular group of users), see the Submission User Interface documentation. Specifically these two sections:

- For Bitstream embargo / access options, see the section on "*Configuring the File Upload step*" of the Submission User Interface
- For Item embargo / access options, see the section on "*Configuring the Item Access Conditions step*" of the Submission User Interface

Private/Public (or Non-Discoverable/Discoverable) Item

It is also possible to adjust the Private/Public (or Non-Discoverable/Discoverable) state of an item after it has been archived in the repository. This can be achieved from either the "Admin Search" (/admin/search), or from the "Status" tab under "Edit Item".

Private (or non-Discoverable) items are not retrievable through the DSpace search, browse or Discovery indexes. However, they are accessible via a direct link. It is possible to create a publicly accessible, non-discoverable item...in which case it can only be shared via a direct link. But, once anyone has that link, it is available anonymously.

Therefore, an "Admin Search" option is provided, which allows you to search across all items, including private or withdrawn items. You can also filter your results to display only private items.

Pre-3.0 Embargo Migration Routine

If you have just upgraded from a DSpace 1.x.x version, any embargoes that are currently "in effect" will need to be migrated into ResourcePolicies. Prior to 3.0, embargoes in DSpace were managed entirely in metadata fields (and required running a scheduled "embargo-lifter" command). However, DSpace now stores all embargo information directly on ResourcePolicies (i.e. "access policies"). These ResourcePolicies automatically "lift" an embargo after the embargo date passes.

In order to migrate old embargoes into ResourcePolicies, a migration routine has been developed. **Please note that this migration routine should only need to be run ONCE** (immediately after an upgrade from 1.x.x to a more recent version of DSpace). After that point, any newly defined embargoes will automatically be stored on ResourcePolicies.

To execute it, run the following command:

```
[dspace]/bin/dspace migrate-embargo -a
```

Technical Specifications

Introduction

The following sections illustrate the technical changes that have been made to the *back-end* to add the new *Advanced Embargo* functionality.

ResourcePolicy

When an embargo is set at *item* level or *bitstream* level, a new *ResourcePolicy* will be added.

Three new attributes have been introduced in the *ResourcePolicy* class:

- *rpname*: resource policy name
- *rptype*: resource policy type
- *rpdescription*: resource policy description

While *rpname* and *rpdescription* are fields manageable by users, the *rptype* is managed by DSpace itself. It represents a type that a resource policy can assume, among the following:

- TYPE_SUBMISSION: all the policies added automatically during the submission process
- TYPE_WORKFLOW: all the policies added automatically during the workflow stage
- TYPE_CUSTOM: all the custom policies added by users
- TYPE_INHERITED: all the policies inherited from the enclosing object (for Item, a Collection; for Bitstream, an Item).

Here is an example of all information contained in a single policy record:

```
policy_id: 4847
resource_type_id: 2
resource_id: 89
action_id: 0
eperson_id:
epersongroup_id: 0
start_date: 2013-01-01
end_date:
rpname: Embargo Policy
rpdescription:  Embargoed through 2012
rptype: TYPE_CUSTOM
```

Item

To manage **Private/Public** state a new *boolean* attribute has been added to the Item:

- isDiscoverable

When an Item is private, the attribute will assume the value ***false***.

Item.inheritCollectionDefaultPolicies(Collection c)

This method has been adjusted to leave custom policies, added by the users, in place and add the default collection policies only if there are no custom policies.

AuthorizeService

Some methods have been changed on *AuthorizeService* to manage the new fields and some convenience methods have been introduced:

```
public static List<ResourcePolicy> findPoliciesByDSOAndType(Context c, DSpaceObject o, String type);
public static void removeAllPoliciesByDSOAndTypeNotEqualsTo(Context c, DSpaceObject o, String type);
public static boolean isAnIdenticalPolicyAlreadyInPlace(Context c, DSpaceObject o, ResourcePolicy rp);
public static ResourcePolicy createOrModifyPolicy(ResourcePolicy policy, Context context, String name, int
idGroup, EPerson ePerson, Date embargoDate, int action, String reason, DSpaceObject dso);
```

Withdraw Item

The feature to withdraw an item from the repository has been modified to keep all the custom policies in place.

Reinstate Item

The feature to reinstate an item in the repository has been modified to preserve existing custom policies.

Pre-DSpace 3.0 Embargo Compatibility

The Pre-DSpace 3.0 embargo functionality (see below) has been modified to adjust the policies setter and lifter. These classes now also set the dates within the policy objects themselves in addition to setting the date in the item metadata.

Creating Embargoes via Metadata

Introduction

Prior to DSpace 3.0, all DSpace embargoes were stored as metadata. While embargoes are no longer stored permanently in metadata fields (they are now stored on ResourcePolicies, i.e. access policies), embargoes *can still be initialized via metadata fields.*

This ability to create/initialize embargoes via metadata is extremely powerful if you wish to submit embargoed content via electronic means (such as Importing Items via Simple Archive Format, SWORDv1, SWORDv2, etc).

Setting Embargo terms via metadata

Functionally, the embargo system allows you to attach "terms" to an item before it is placed into the repository, which express how the embargo should be applied. What do we mean by "terms" here? They are really any expression that the system is capable of turning into (1) the time the embargo expires, and (2) a concrete set of access restrictions. Some examples:

"2020-09-12" - an absolute date (i.e. the date embargo will be lifted)
"6 months" - a time relative to when the item is accessioned
"forever" - an indefinite, or open-ended embargo
"local only until 2015" - both a time and an exception (public has no access until 2015, local users OK immediately)
"Nature Publishing Group standard" - look-up to a policy somewhere (typically 6 months)

These terms are interpreted by the embargo system to yield a specific date on which the embargo can be removed (or "lifted"), and a specific set of access policies. Obviously, some terms are easier to interpret than others (the absolute date really requires none at all), and the default embargo logic understands only the most basic terms (the first and third examples above). But as we will see below, the embargo system provides you with the ability to add your own interpreters to cope with any terms expressions you wish to have. This date that is the result of the interpretation is stored with the item. The embargo system detects when that date has passed, and removes the embargo ("lifts it"), so the item bitstreams become available. Here is a more detailed life-cycle for an embargoed item:

Terms assignment

The first step in placing an embargo on an item is to attach (assign) "terms" to it. If these terms are missing, no embargo will be imposed. As we will see below, terms are carried in a configurable DSpace metadata field, so assigning terms just means assigning a value to a metadata field. This can be done in a web submission user interface form, in a SWORD deposit package, a batch import, etc. - anywhere metadata is passed to DSpace. The terms are not immediately acted upon, and may be revised, corrected, removed, etc, up until the next stage of the life-cycle. Thus a submitter could enter one value, and a collection editor replace it, and only the last value will be used. Since metadata fields are multivalued, theoretically there can be multiple terms values, but in the default implementation only one is recognized.

Terms interpretation/imposition

In DSpace terminology, when an Item has exited the last of any workflow steps (or if none have been defined for it), it is said to be "installed" into the repository. At this precise time, the interpretation of the terms occurs, and a computed "lift date" is assigned, and recorded as part of the ResourcePolicy (aka policy) of the Item. Once the lift date has been assigned to the ResourcePolicy, the metadata field which defined the embargo is **cleared**. From that point forward, all embargo information is controlled/defined by the ResourcePolicy.

It is important to understand that this interpretation happens only once, (just like the installation). Therefore, **updating/changing an embargo cannot be done via metadata fields.** Instead, all embargo updates must be made to the ResourcePolicies themselves (e.g. ResourcePolicies can be managed from the Admin UI in the Edit Item screens).

Also note that since these policy changes occur before installation, there is no time during which embargoed content is "exposed" (accessible by non-administrators). The terms interpretation and imposition together are called "setting" the embargo, and the component that performs them both is called the embargo "setter".

Embargo period

After an embargoed item has been installed, the policy restrictions remain in effect until the embargo date passes. Once the embargo date passes, the policy restrictions are automatically lifted. An embargo lift date is generally stored as the "start date" of a policy. Essentially, this means that the policy does not get applied until after that date passes (and prior to that date, the object defaults to Admin only access).

Administrators are able to change the lift date of the embargo by editing the policy (ResourcePolicy). These policies can be managed from the Edit Item screens.

Configuration of metadata fields

DSpace embargoes utilize standard metadata fields to hold both the "terms" and the "lift date". Which fields you use are configurable, and no specific metadata element is dedicated or pre-defined for use in embargo. Rather, you must specify exactly what field you want the embargo system to examine when it needs to find the terms or assign the lift date.

The properties that specify these assignments live in dspace.cfg:

```
# DC metadata field to hold the user-supplied embargo terms
embargo.field.terms = SCHEMA.ELEMENT.QUALIFIER

# DC metadata field to hold computed "lift date" of embargo
embargo.field.lift = SCHEMA.ELEMENT.QUALIFIER
```

You replace the placeholder values with real metadata field names. If you only need the "default" embargo behavior - which essentially accepts only absolute dates as "terms" - this is the only configuration required, except as noted below.

There is also a property for the special date of "forever":

```
# string in terms field to indicate indefinite embargo
embargo.terms.open = forever
```

which you may change to suit linguistic or other preference.

You are free to use existing metadata fields, or create new fields. If you choose the latter, you must understand that the embargo system does **not** create or configure these fields: i.e. you must follow all the standard documented procedures for actually creating them (i.e. adding them to the metadata registry, or to display templates, etc) - this does not happen automatically. Likewise, if you want the field for "terms" to appear in submission screens and workflows, you must follow the documented procedure for configurable submission (basically, this means adding the field to submission-forms.xml). The flexibility of metadata configuration makes if easy for you to restrict embargoes to specific collections, since configurable submission can be defined per collection.

Key recommendations:

1. Use a local metadata schema. Breaking compliance with the standard Dublin Core in the default metadata registry can create a problem for the portability of data to/from of your repository.
2. If using existing metadata fields, avoid any that are automatically managed by DSpace. For example, fields like "date.issued" or "date. accessioned" are normally automatically assigned, and thus must not be recruited for embargo use.
3. Do not place the field for "lift date" in submission screens. This can potentially confuse submitters because they may feel that they can directly assign values to it. As noted in the life-cycle above, this is erroneous: the lift date gets assigned by the embargo system based on the terms. Any pre-existing value will be over-written. But see next recommendation for an exception.
4. As the life-cycle discussion above makes clear, after the terms are applied, that field is no longer actionable in the embargo system. Conversely, the "lift date" field is not actionable **until** the application. Thus you may want to consider configuring both the "terms" and "lift date" to use the same metadata field. In this way, during workflow you would see only the terms, and after item installation, only the lift date. If you wish the metadata to retain the terms for any reason, use 2 distinct fields instead.

Operation

After the fields defined for terms and lift date have been assigned in dspace.cfg, and created and configured wherever they will be used, you can begin to embargo items simply by entering data (dates, if using the default setter) in the terms field. They will automatically be embargoed as they exit workflow, and that the computed lift date will be stored on the ResourcePolicy

Extending embargo functionality

The embargo system supplies a default "interpreter/imposition" class (the "Setter") .

Setter

The default setter recognizes only two expressions of terms: either a literal, non-relative date in the fixed format "yyyy-mm-dd" (known as ISO 8601), or a special string used for open-ended embargo (the default configured value for this is "forever", but this can be changed in dspace.cfg to "toujours", "unendlich", etc). It will perform a minimal sanity check that the date is not in the past. Similarly, the default setter will only remove all read policies as noted above, rather than applying more nuanced rules (e.g allow access to certain IP groups, deny the rest). Fortunately, the setter class itself is configurable and you can "plug in" any behavior you like, provided it is written in java and conforms to the setter interface. The dspace.cfg property:

```
# implementation of embargo setter plugin - replace with local implementation if applicable
plugin.single.org.dspace.embargo.EmbargoSetter = org.dspace.embargo.DefaultEmbargoSetter
```

controls which setter to use.

Lifter

DEPRECATED: The Lifter is no longer used in the DSpace API, and is not recommended to utilize. Embargo lift dates are now stored on ResourcePolicies and, as such, are "lifted" automatically when the embargo date passes. Manually running a "lifter" may bypass this automatic functionality and result in unexpected results.

The default lifter behavior as described above - essentially applying the collection policy rules to the item - might also not be sufficient for all purposes. It also can be replaced with another class:

```
implementation of embargo lifter plugin - - replace with local implementation if applicable
plugin.single.org.dspace.embargo.EmbargoLifter = org.dspace.embargo.DefaultEmbargoLifter
```

Pre-3.0 Embargo Lifter Commands

DEPRECATED - Not recommended to use

The old "embargo-lifter" command is no longer necessary to run. All Embargoes in DSpace are now stored on ResourcePolicies and are lifted automatically after the lift date passed. See Embargo documentation for more information.

Continuing to run the "embargo-lifter" is not recommended and this feature will be removed entirely in a future DSpace release.

If you have implemented the pre DSpace 3.0 Embargo feature, you will need to run it periodically to check for Items with expired embargoes and lift them.

Command used:	`[dspace]/bin/dspace embargo-lifter`
Java class:	org.dspace.embargo.EmbargoManager
Arguments short and (long) forms):	Description
`-c` or `--check`	ONLY check the state of embargoed Items, do NOT lift any embargoes
`-i` or `--identifier`	Process ONLY this handle identifier(s), which must be an Item. Can be repeated.
`-l` or `--lift`	Only lift embargoes, do NOT check the state of any embargoed items.
`-n` or `--dryrun`	Do no change anything in the data model, print message instead.
`-v` or `--verbose`	Print a line describing the action taken for each embargoed item found.
`-q` or `--quiet`	No output except upon error.
`-h` or `--help`	Display brief help screen.

You must run the Embargo Lifter task periodically to check for items with expired embargoes and lift them from being embargoed. For example, to check the status, at the CLI:

```
[dspace]/bin/dspace embargo-lifter -c
```

To lift the actual embargoes on those items that meet the time criteria, at the CLI:

```
[dspace]/bin/dspace embargo-lifter -l
```

Managing User Accounts

- From the browser
- From the command line
  - The user command
    - To create a new user account:
    - To list accounts:
    - To modify an account:
    - To delete an account:
  - The Groomer
    - Find accounts with unsalted passwords
    - Find (and perhaps delete) disused accounts
- Cryptographic properties

When a user registers an account for the purpose of subscribing to change notices, submitting content, or the like, DSpace creates an EPerson record in the database. Administrators can manipulate these records in several ways.

From the browser

- Login as an Administrator
- Sidemenu "Access Control" "People"
- Browse or search for the account you wish to modify or delete.

To modify user permissions / group memberships:

- Login as an Administrator
- Sidemenu "Access Control" "Groups"
- Edit the Group
- Search for the EPerson & add/remove them from that group.

To debug issues for a specific user, it's possible to login as (or "impersonate") that user account

- On the backend, first you MUST enable the "assumelogin" feature. This feature is disabled by default. Update this setting in your local.cfg or dspace.cfg

```
# Required to use "Impersonate EPerson" feature
# When enabled, a full Administrator can impersonate any other non-Administrative user
webui.user.assumelogin = true
```

- Then, from the user interface, login as an Administrator
- Sidemenu "Access Control" "People"
- Browse or search for the account you wish to login as
- Edit that User, and click the "Impersonate EPerson" button.
- You are now logged in as that user. You'll see an Impersonate icon/button in the header.
- You are able to temporarily manage any activities as that user.
- Once your are done, click the "Stop impersonating EPerson".
- Optionally, you may wish to disable this feature again in your local.cfg by setting the above configuration to "false" or commenting it out.

From the command line

The `user` command

The `dspace user` command adds, lists, modifies, and deletes EPerson records.

To create a new user account:

```
[dspace]/bin/dspace user --add --email jquser@example.com -g John -s User --password hiddensecret
[dspace]/bin/dspace user --add --netid jquser --telephone 555-555-1234 --password hiddensecret
```

One of the options `--email or --netid` is required to name the record. The complete options are:

-a	--add	required
-m	--email	email address
-n	--netid	"netid" (a username in an external system such as a directory – see Authentication Methods for details)
-p	--password	a password for the account. Required.

-g	--givenname	First or given name
-s	--surname	Last or surname
-t	--telephone	Telephone number
-l	--language	Preferred language
-c	--requireCertificate	Certificate required? See X.509 Authentication for details.

To list accounts:

```
[dspace]/bin/dspace user --list
```

This simply lists some characteristics of each EPerson.

short	long	meaning
-L	--list	required

To modify an account:

```
[dspace]/bin/dspace user --modify -m george@example.com
```

short	long	meaning
-M	--modify	required
-m	--email	identify the account by email address
-n	--netid	identify the account by netid
-g	--givenname	First or given name
-s	--surname	Last or surname
-t	--telephone	telephone number
-l	--language	preferred language
-c	--requireCertificate	certificate required?
-C	--canLogIn	is the account enabled or disabled?
-i	--newEmail	set or change email address
-l	--newNetid	set or change netid
-w	--newPassword	set or change password

To delete an account:

```
[dspace]/bin/dspace user --delete -n martha
```

short	long	meaning
-d	--delete	required
-m	--email	identify the account by email address
-n	--netid	identify the account by netid

The Groomer

This tool inspects all user accounts for several conditions.

short	long	meaning
-a	--aging	find accounts not logged in since a given date
-u	--unsalted	find accounts not using salted password hashes
-b	--before	date cutoff for --aging
-d	--delete	delete disused accounts (used with --aging)

Find accounts with unsalted passwords

Earlier versions of DSpace used an "unsalted hash" method to protect user passwords. Recent versions use a salted hash. You can find accounts which have never been converted to salted hashing:

Discovering accounts with unsalted password hashes

```
[DSpace]/bin/dspace dsrun org.dspace.eperson.Groomer -u
```

The output is a list of email addresses for matching accounts.

Find (and perhaps delete) disused accounts

You can list accounts which have not logged on since a given date:

Discovering disused accounts

```
[DSpace]/bin/dspace dsrun org.dspace.eperson.Groomer -a -b 07/20/1969
```

The output is a tab-separated-value table of the EPerson ID, last login date, email address, netid, and full name for each matching account.

You can also have the tool delete matching accounts:

Deleting disused accounts

```
[DSpace]/bin/dspace dsrun org.dspace.eperson.Groomer -a -b 07/20/1969 -d
```

Cryptographic properties

The cryptographic properties used for generating the salted hashes, to ensure encryption at rest for user passwords, can be found and adjusted in:

https://github.com/DSpace/DSpace/blob/main/dspace-api/src/main/java/org/dspace/eperson/PasswordHash.java

Email Subscriptions

Introduction
Adding new subscriptions
Managing your subscriptions
Enable sending out emails

Introduction

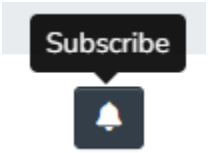
This feature is available in 7.5 or later.

Registered users can subscribe to communities or collections in DSpace. After subscribing, users will receive a regular email containing the new and modified items in the communities/collections they are subscribed to.

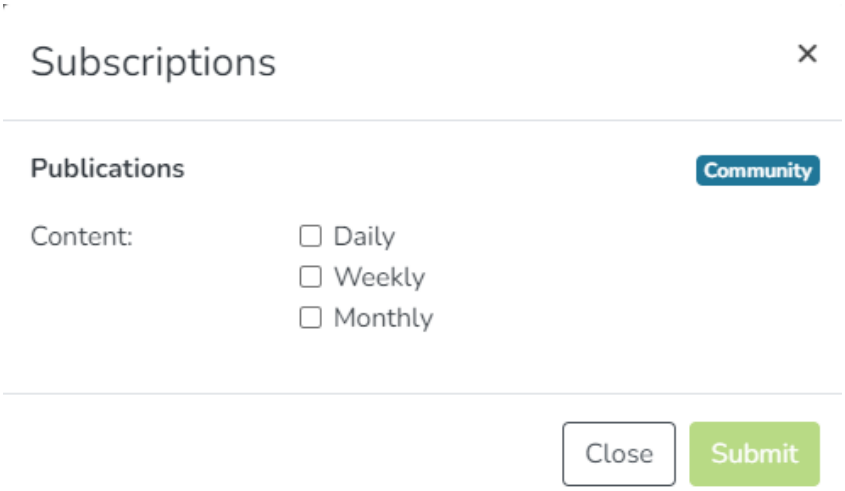
Adding new subscriptions

Adding new email subscriptions is only available to users who are logged in.

In the User interface, browse to the Community or Collection you wish to subscribe to, and click on the Subscribe button.



After clicking that button, you'll see a popup window which allows you to select the frequency of subscription you'd like.

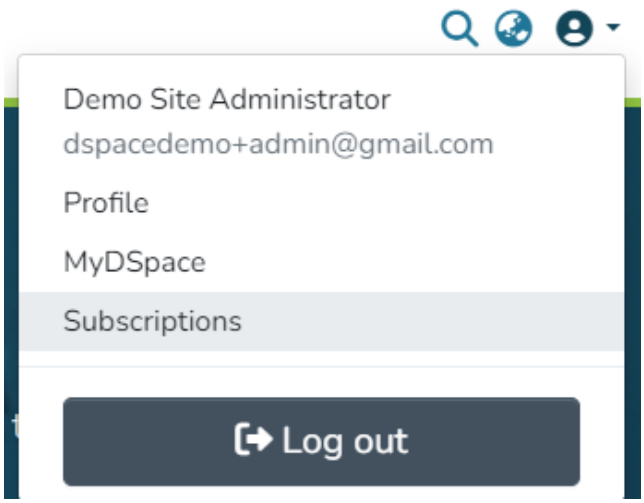


Currently, three frequency options are available:

- Daily: Receive a daily email of Items under the Community/Collection which have been updated in the last day.
- Weekly: Receive a weekly email of Items under the Community/Collection which have been updated in the last week.
- Monthly: Receive a monthly email of Items under the Community/Collection which have been updated in the last month.

Managing your subscriptions

To manage your subscriptions, visit your "Subscriptions" page under your user profile:



From this page, you are able to see all your current Community/Collection subscriptions. You can choose to edit or delete any in the list.

Enable sending out emails

Run "subscription-send" to enable

⚠ NOTE: Until you enable the "subscription-send" script, users will not receive the email updates for their subscriptions. It is HIGHLY RECOMMENDED to enable this script via Scheduled Tasks via Cron. See sample settings on that page.

To send out the subscription emails you MUST invoke the **subscription-send** script from the DSpace command-line or Processes UI. It is advised to setup this script as a Scheduled Tasks via Cron. See sample settings on that page.

```
# Example of running subscription-send to send out all "Daily" subscription emails
./dspace subscription-send -f D
```

This script requires the "-f" (--frequency) parameter with a value of "D" (Daily), "W" (Weekly), or "M" (Monthly). Keep in mind, you will want to schedule it to run on a Daily, Weekly and Monthly basis to send the appropriate emails.

Request a Copy

Introduction
Requesting a copy using the User Interface
(Optional) Requesting a copy with Help Desk workflow
Email templates
Configuration parameters
Selecting Request a Copy strategy
    Configure who gets request via a metadata field
    Configure all requests to go to a helpdesk email
    Configure all requests to go to the administrators of a Collection
    Combine multiple strategies

Introduction

Supported in 7.1 or above

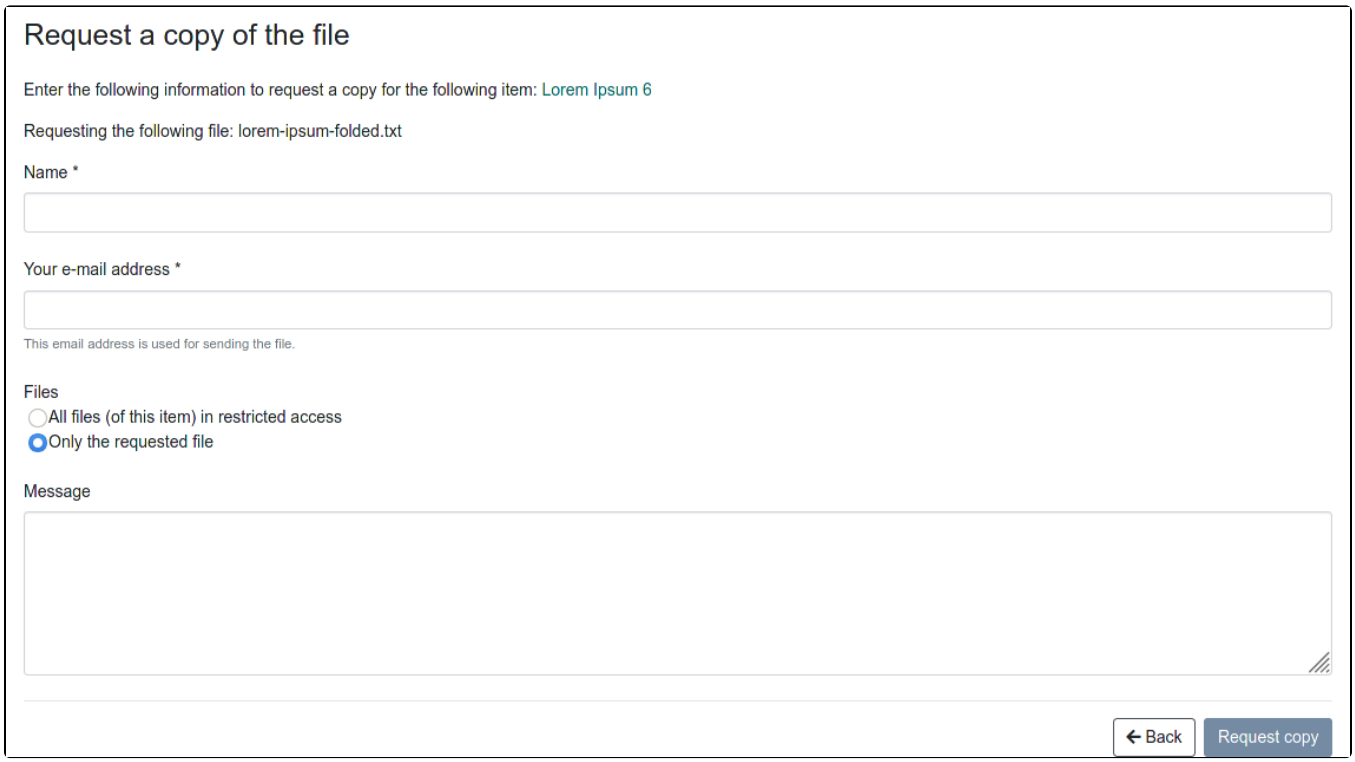
Request a Copy was not available in DSpace 7.0. It was restored in DSpace 7.1. See DSpace Release 7.0 Status

The request a copy functionality was added to DSpace as a measure to facilitate access in those cases when uploaded content can not be openly shared with the entire world immediately after submission into DSpace. It gives users an efficient way to request access to the original submitter of the item, who can approve this access with the click of a button. This practice complies with most applicable policies as the submitter interacts directly with the requester on a case by case basis.

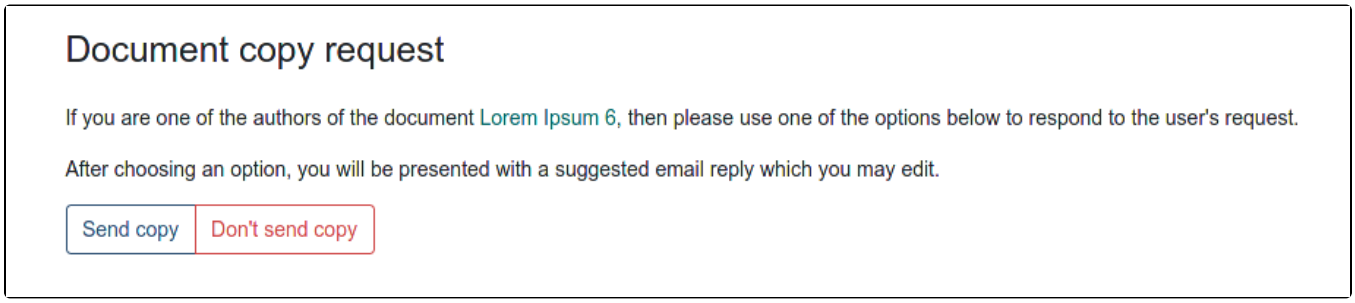
Requesting a copy using the User Interface

Users can request a copy by clicking the file thumbnail or the name of a file that the user is restricted from viewing.

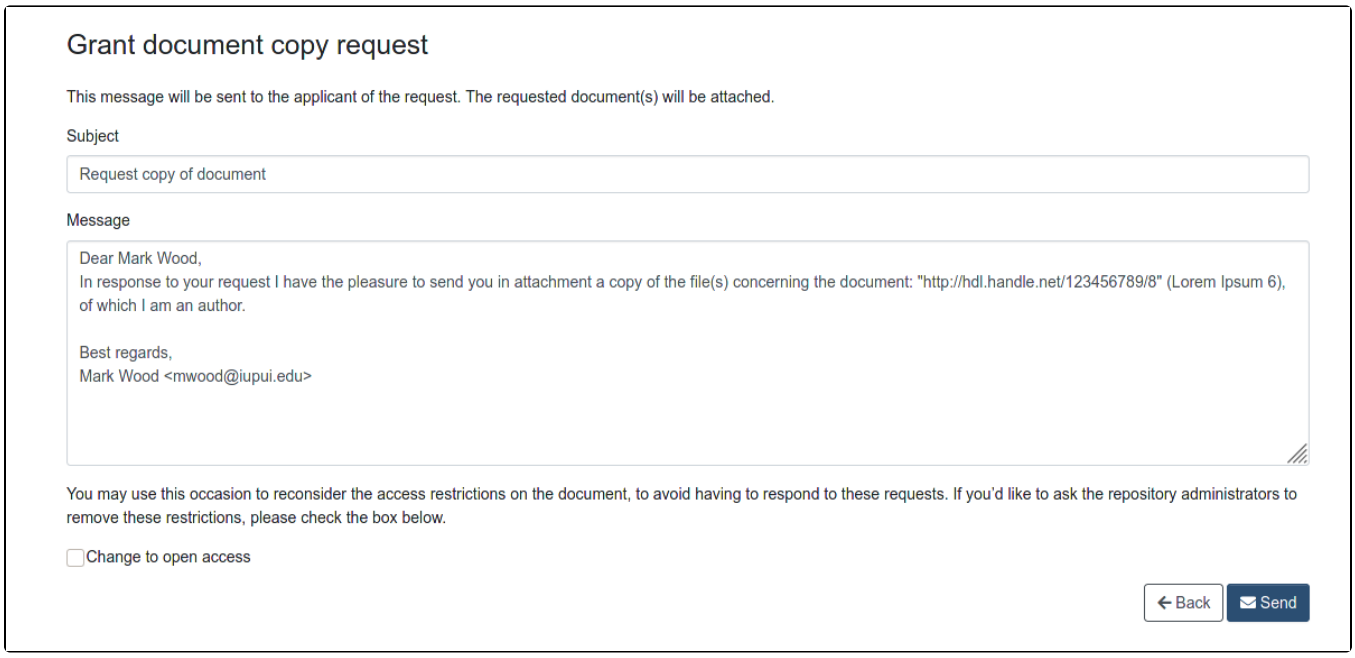
The request form asks the user for his or her name, email address and message where the reason for requesting access can be entered.



After clicking "Request copy" at the bottom of this form, the original submitter of the item will receive an email containing the details of the request. The email also contains a link with a token that brings the original submitter to a page where he or she can either grant or reject access. If the original submitter can not evaluate the request, he or she can forward this email to the right person, who can use the link containing the token without having to log into DSpace.



Each of these buttons registers the choice of the submitter, displaying the following form in which an additional reason for granting or rejecting the access can be added.



After hitting send, the contents of this form will be sent together with the associated files to the email address of the requester. In case the access is rejected, only the reason will be sent to the requester.

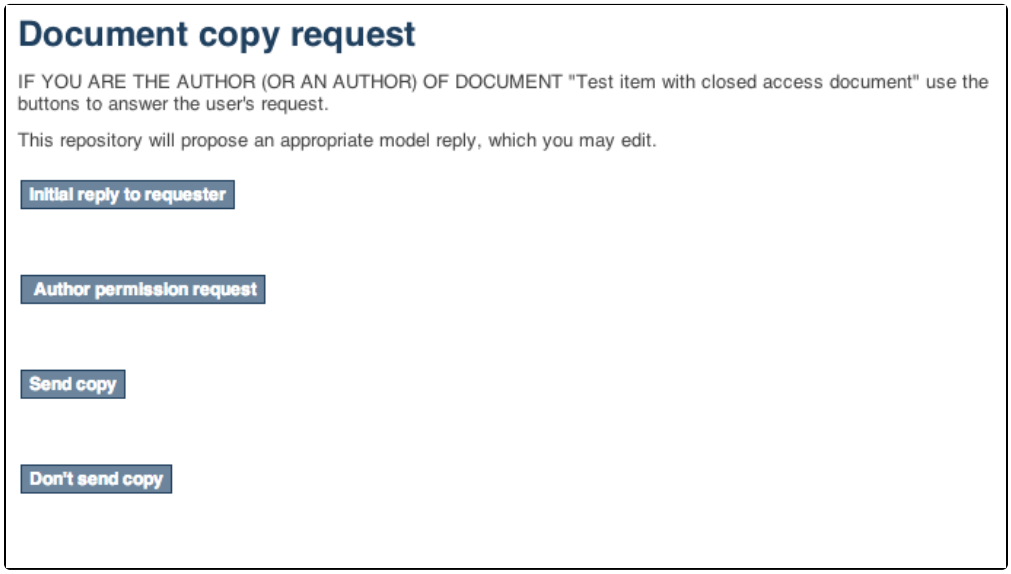
While responding positively to a request for copy, the person who approved may also ask the repository administrator to alter the access rights of the item, allowing unrestricted open access to everyone, by checking "Change to open access".

(Optional) Requesting a copy with Help Desk workflow

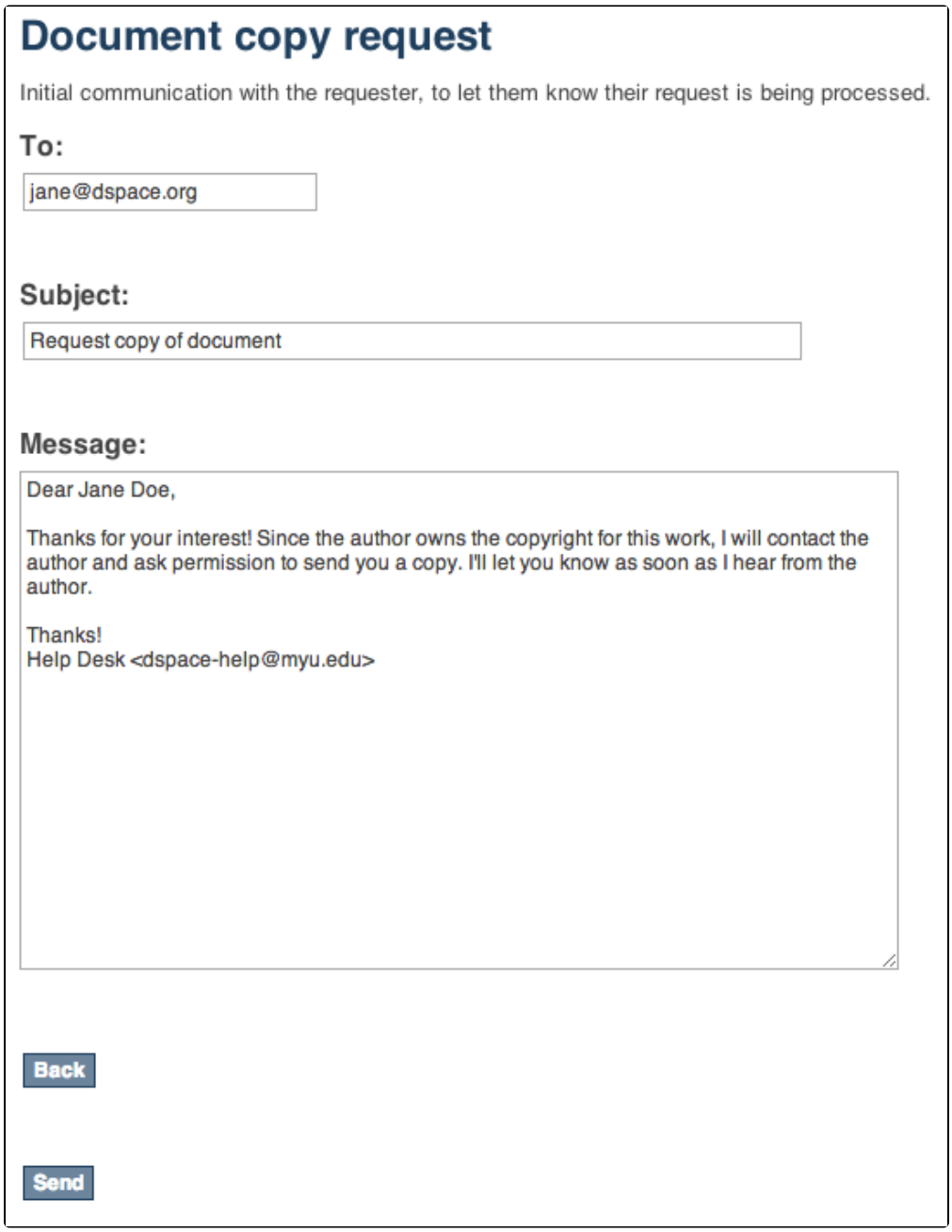
Available in 7.5 or later. However, in 7.5, users approving/rejecting these requests via the HelpDesk workflow **must** first authenticate. This is a known bug as described in https://github.com/DSpace/DSpace/issues/8636

As of 7.6, the HelpDesk workflow can be performed without requiring authentication (issue #8636 has been fixed)

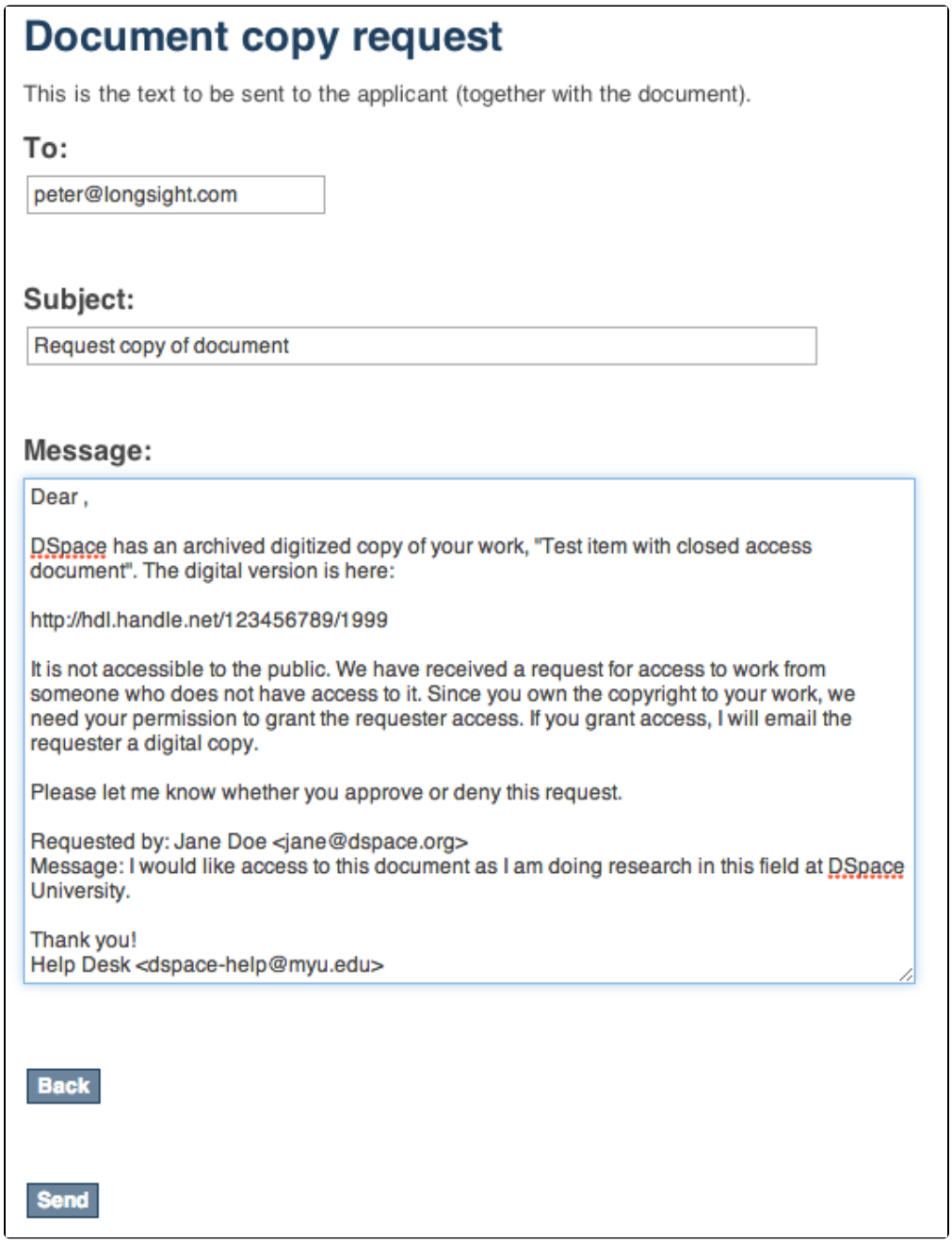
(Optional) Request Item with HelpDesk intermediary, is steered towards having your Repository Support staff act as a helpdesk that receives all incoming RequestItem requests, and then processes them. This adds the options of "Initial Reply to Requestor" to let the requestor know that their request is being worked on, and an option "Author Permission Request" which allows the helpdesk to email the author of the document, as not all documents are deposited by the author, or the author will need to be tracked down by a support staff, as DSpace might not have their current email address.



Initial Reply to Requester



Author permission request, includes information about the original request (requester name, requester email, requester's reason for requesting). The author /submitter's name and email address will be pre-populated in the form from the submitter, but the email address and author name are editable, as the submitter's of content to DSpace aren't always the author.



Email templates

Most of the email templates used by Request a Copy are treated just like other email templates in DSpace. The templates can be found in the /config /emails directory and can be altered just by changing the contents and restarting tomcat.

request_item. admin	template for the message that will be sent to the administrator of the repository, after the original submitter requests to have the permissions changed for this item.
request_item. author	template for the message that will be sent to the original submitter of an item with the request for copy.

The templates for emails that the requester receives, that could have been customized by the approver in the aforementioned dialog are not managed as separate email template files. These defaults are stored in the Messages.properties file under the keys

itemRequest.response.body.approve	Default message for informing the requester of the approval

itemRequest.response.body.reject	Default message for informing the requester of the rejection
itemRequest.response.body.contactAuthor	Default message for the helpdesk to contact the author
itemRequest.response.body.contactRequester	Default message for the helpdesk to contact the requester

Configuration parameters

Request a copy is enabled by default. These configuration parameters in dspace.cfg relate to Request a Copy:

Property:	`request.item.type`
Example Value	`request.item.type = all`
Informational Note	This parameter manages who can file a request for an item. The parameter is optional. When it is empty or commented out, request a copy is disabled across the entire repository. When set to **all**, any user can file a request for a copy. When set to **logged**, only registered users can file a request for copy.
Property:	`mail.helpdesk`
Example Value	`mail.helpdesk = foo@bar.com`
Informational Note	The email address assigned to this parameter will receive the emails both for granting or rejecting request a copy requests, as well as requests to change item policies. This parameter is optional. If it is empty or commented out, it will default to `mail.admin`. *WARNING:* This setting is only utilized if the `RequestItemHelpdeskStrategy` bean is enabled in `[dspace]/config/spring/api/requestitem.xml` (see below)
Property:	`request.item.helpdesk.override`
Example Value	`request.item.helpdesk.override = true`
Informational Note	Should all Request Copy emails go to the `mail.helpdesk` instead of the item submitter? Default is `false`, which sends Item Requests to the item submitter. *WARNING:* This setting is only utilized if the `RequestItemHelpdeskStrategy` bean is enabled in `[dspace]/config/spring/api/requestitem.xml` (see below)

Selecting Request a Copy strategy

The process that DSpace uses to determine who is the recipient of the Item Request is configurable in this Spring file: `[dspace]/config/spring/api/requestitem.xml`

New in DSpace 7

The strategy is selected using a Spring `<alias alias='org.dspace.app.requestitem.RequestItemAuthorExtractor' name='theStrategyClass'/>`. Previously this was done by moving the `RequestItemAuthorExtractor id` to the selected `<bean>` definition.

New in DSpace 7.6

The strategy is selected by configuring it into the `<bean/>` for `RequestItemMetadataStrategy` as a constructor argument.

Configure who gets request via a metadata field

By default the `RequestItemMetadataStrategy` is enabled, but falls back to the Item Submitter eperson's name and email. You can configure the `RequestItemMetadataStrategy` to load the author's name and email address if you set that information into an item metadata field. For example:

Syntax for 7.6 or later

```
<!-- This alias defines that you want to use the RequestItemMetadataStrategy (this is enabled by default) -->
<!-- This bean specifies that you want to use the RequestItemMetadataStrategy (this is enabled by default) -->
<bean class="org.dspace.app.requestitem.RequestItemEmailNotifier" lazy-init='false'>
        <description>This sends various emails between the requestor and the grantor.</description>

        <!-- Modify the "ref" here to point at the "RequestItemHelpdeskStrategy" -->
        <constructor-arg index='0'
                         ref='org.dspace.app.requestitem.RequestItemMetadataStrategy'/>
</bean>

<!-- This bean allows you to specify which metadata field is used (if any) -->
<bean class="org.dspace.app.requestitem.RequestItemMetadataStrategy"
      id="org.dspace.app.requestitem.RequestItemMetadataStrategy">
  <!--
  Uncomment these properties if you want lookup in metadata the email and the name of the author to contact for
request copy.
  If you don't configure that or if the requested item doesn't have these metadata the submitter data are used
as fail over

  <property name="emailMetadata" value="schema.element.qualifier" />
  <property name="fullNameMatadata" value="schema.element.qualifier" />

  -->
</bean>
```

Syntax for 7.5 or earlier

```
<!-- This alias defines that you want to use the RequestItemMetadataStrategy (this is enabled by default) -->
<alias alias='org.dspace.app.requestitem.RequestItemAuthorExtractor'
       name='org.dspace.app.requestitem.RequestItemMetadataStrategy'/>

<!-- This bean allows you to specify which metadata field is used (if any) -->
<bean class="org.dspace.app.requestitem.RequestItemMetadataStrategy"
      id="org.dspace.app.requestitem.RequestItemMetadataStrategy"
      autowireCandidate="true">
  <!--
  Uncomment these properties if you want lookup in metadata the email and the name of the author to contact for
request copy.
  If you don't configure that or if the requested item doesn't have these metadata the submitter data are used
as fail over

  <property name="emailMetadata" value="schema.element.qualifier" />
  <property name="fullNameMatadata" value="schema.element.qualifier" />

  -->
</bean>
```

Configure this as follows:

1. Create a metadata field which you'd like to use to store this email address (and optionally a second metadata field for the full name).
    a. Hint: You may wish to add this metadata field to your "metadata.hide.*" settings in local.cfg in order to ensure this metadata field is hidden from normal users & is only visible to Administrative users. That way this email address will NOT appear in Item display pages (except to Administrators)
2. Uncomment the "emailMetadata" setting above, and configure it's "value" to use the new metadata field.
3. Edit the Item(s) which you wish to use this field. Add the new metadata field to those items, given it a value of the email address which will *receive* the request for copy. By default, if an Item does NOT have this metadata field, the request for copy will still go to the Item's submitter.

Configure all requests to go to a helpdesk email

Prior to 7.6, all users who wish to respond to a request to the helpdesk email *must first login* to DSpace. See https://github.com/DSpace/DSpace/issues/8636 This was fixed in 7.6.

Another common request strategy is the use a single Helpdesk email address to receive all of these requests (see corresponding helpdesk configs in dspace.cfg above). If you wish to use the Helpdesk Strategy, you must replace the references to the default `RequestItemMetadataStrategy`, bean with the `RequestItemHelpdeskStrategy` bean:

Syntax for 7.6 or later

```
<!-- To change the settings, you need to modify the constructor-arg (see below) to use the
"RequestItemEmailNotifier" bean.-->
<bean class="org.dspace.app.requestitem.RequestItemEmailNotifier" lazy-init='false'>
        <description>This sends various emails between the requestor and the grantor.</description>

        <!-- Modify the "ref" here to point at the "RequestItemHelpdeskStrategy" -->
        <constructor-arg index='0'
                         ref='org.dspace.app.requestitem.RequestItemHelpdeskStrategy'/>
</bean>
```

Syntax for 7.5 or earlier

```
<!-- Change this alias to use "RequestItemHelpdeskStrategy" bean-->
<alias alias='org.dspace.app.requestitem.RequestItemAuthorExtractor'
       name='org.dspace.app.requestitem.RequestItemHelpdeskStrategy'/>

<!-- Ensure the bean is uncommented (it should be by default) -->
<bean class="org.dspace.app.requestitem.RequestItemHelpdeskStrategy"
        id="org.dspace.app.requestitem.RequestItemHelpdeskStrategy"
        autowireCandidate="true"/>
```

Configure all requests to go to the administrators of a Collection

This strategy sends mail to all of the members of the administrators group for the collection which owns the item.

Syntax for 7.6 or later

```
<!-- To change the settings, you need to modify the constructor-arg (see below) to use the
"CollectionAdministratorsRequestItemStrategy" bean.-->
<bean class="org.dspace.app.requestitem.RequestItemEmailNotifier" lazy-init='false'>
        <description>This sends various emails between the requestor and the grantor.</description>

        <!-- Modify the "ref" here to point at the "CollectionAdministratorsRequestItemStrategy" -->
        <constructor-arg index='0'
                         ref='org.dspace.app.requestitem.CollectionAdministratorsRequestItemStrategy'/>
</bean>
```

Syntax for 7.5 or earlier

```
<!-- Change this alias to use "CollectionAdministratorsRequestItemStrategy" bean-->
<alias alias='org.dspace.app.requestitem.RequestItemAuthorExtractor'
       name='org.dspace.app.requestitem.CollectionAdministratorsRequestItemStrategy'/>

<!-- Ensure the bean is uncommented (it should be by default) -->
<bean class='org.dspace.app.requestitem.CollectionAdministratorsRequestItemStrategy'
      id='org.dspace.app.requestitem.CollectionAdministratorsRequestItemStrategy'
      autowireCandidate='true'/>
```

Combine multiple strategies

This strategy combines the results of other strategies into a single list of email recipients. Pass the strategy a single constructor argument, being a list of the strategy beans whose output should be combined.

In the following example, email will be sent to the address(es) found in the configured metadata fields (or to the submitter if none), *and* to the owning collection's administrators.

Syntax for 7.6 or later

```
<!-- To change the settings, you need to modify the constructor-arg (see below) to use the
"CombiningRequestItemStrategy" bean.-->
<bean class="org.dspace.app.requestitem.RequestItemEmailNotifier" lazy-init='false'>
        <description>This sends various emails between the requestor and the grantor.</description>

        <!-- Modify the "ref" here to point at the "CombiningRequestItemStrategy" -->
        <constructor-arg index='0'
                         ref='org.dspace.app.requestitem.CombiningRequestItemStrategy'/>
</bean>

<!-- This bean is where you can combine multiple strategies by referencing them in the <list> below -->
<bean class='org.dspace.app.requestitem.CombiningRequestItemStrategy'
      id='org.dspace.app.requestitem.CombiningRequestItemStrategy'>
    <constructor-arg>
        <description>A list of references to RequestItemAuthorExtractor beans</description>
        <list>
            <ref bean='org.dspace.app.requestitem.RequestItemMetadataStrategy'/>
            <ref bean='org.dspace.app.requestitem.CollectionAdministratorsRequestItemStrategy'/>
        </list>
    </constructor-arg>
</bean>
```

Syntax for 7.5 or earlier

```
<!-- Change this alias to use "CombiningRequestItemStrategy" bean-->
<alias alias='org.dspace.app.requestitem.RequestItemAuthorExtractor'
       name='org.dspace.app.requestitem.CombiningRequestItemStrategy'/>

<!-- This bean is where you can combine multiple strategies by referencing them in the <list> below -->
<bean class='org.dspace.app.requestitem.CombiningRequestItemStrategy'
      id='org.dspace.app.requestitem.CombiningRequestItemStrategy'
      autowireCandidate='true'>
    <constructor-arg>
        <description>A list of references to RequestItemAuthorExtractor beans</description>
        <list>
            <ref bean='org.dspace.app.requestitem.RequestItemMetadataStrategy'/>
            <ref bean='org.dspace.app.requestitem.CollectionAdministratorsRequestItemStrategy'/>
        </list>
    </constructor-arg>
</bean>
```

CAPTCHA Verification

This feature is available starting from DSpace 7.4

This feature, when enabled, offers a powerful additional layer of protection against possible unwanted behaviors like massive registrations performed by bots using random or stolen email addresses. Feature can be enabled or disabled by decision of DSpace instance administrator, and is based on **Google reCAPTCHA**.

ReCAPTCHA supported versions are v2 with both invisible (https://developers.google.com/recaptcha/docs/invisible) and checkbox (https://developers.google.com/recaptcha/docs/display) verification modes, and v3 (https://developers.google.com/recaptcha/docs/v3)

Prerequisites

Before enabling the feature, a valid site and secret pair should be obtained from Google reCAPTCHA system, by registering the DSpace application on which verification will be set on reCAPTCHA admin panel (https://www.google.com/recaptcha/admin)

How to enable the feature

Once site and secret are available, following property, set in configuration files (dspace.cfg or local.cfg) enables the CAPTCHA verification

```
registration.verification.enabled = true
```

Whereas, in case **v2** of Google reCAPTCHA is to be enabled, these properties, in configuration files, must be set

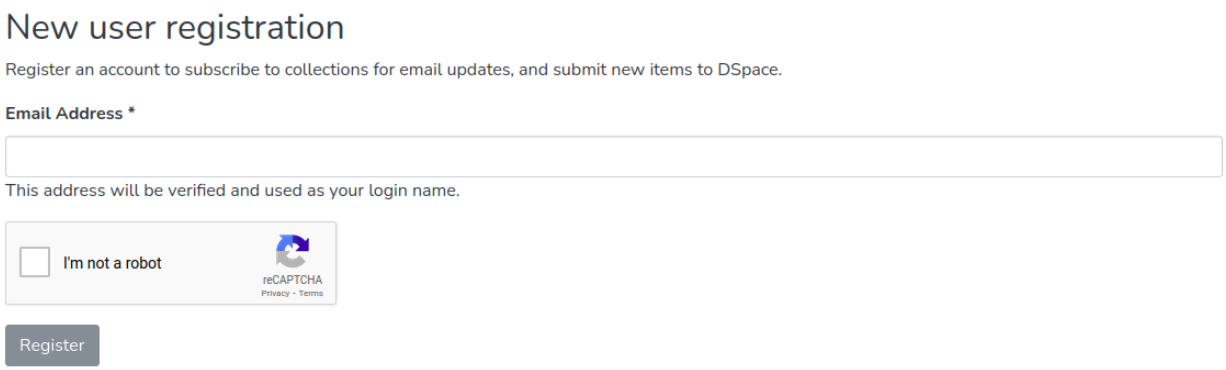
```
google.recaptcha.version = v2
google.recaptcha.mode = <invisible or checkbox depending on which mode is wanted>
google.recaptcha.key.site = <your site here>
google.recaptcha.key.secret = <your secret here>
```

In case **v3** of Google reCAPTCHA is to be enabled, properties to be set are:

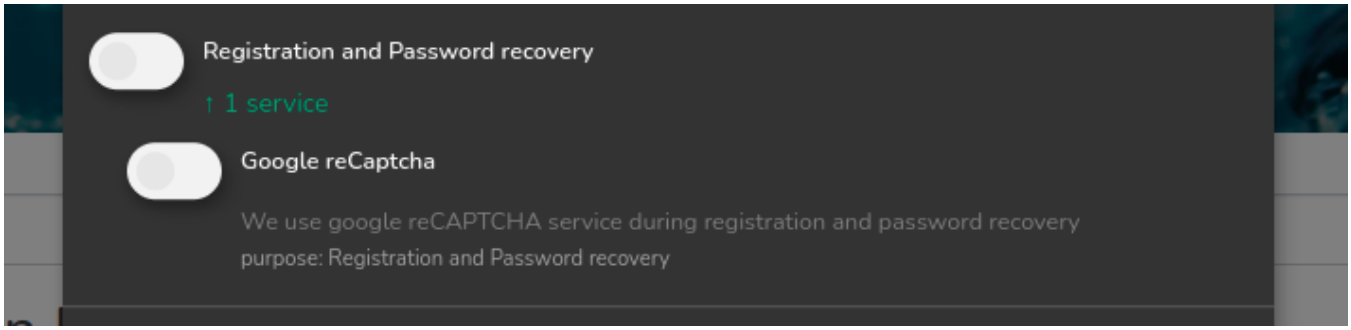
```
google.recaptcha.version = v3
google.recaptcha.key.site = <your site here>
google.recaptcha.key.secret = <your secret here>
google.recaptcha.site-verify = https://www.google.com/recaptcha/api/siteverify
google.recaptcha.key.threshold = <score threshold>
google.recaptcha.mode = invisible
```

`google.recaptcha.key.threshold` property is related to reCAPTCHA verification logic. v3 assigns to each request made against verification APIs, in this case by DSpace system during registration process. reCAPTCHA v3 returns a score (1.0 is very likely a good interaction, 0.0 is very likely a bot). By default a good threshold could be 0.5. For further information, see https://developers.google.com/recaptcha/docs/v3#interpreting_the_score

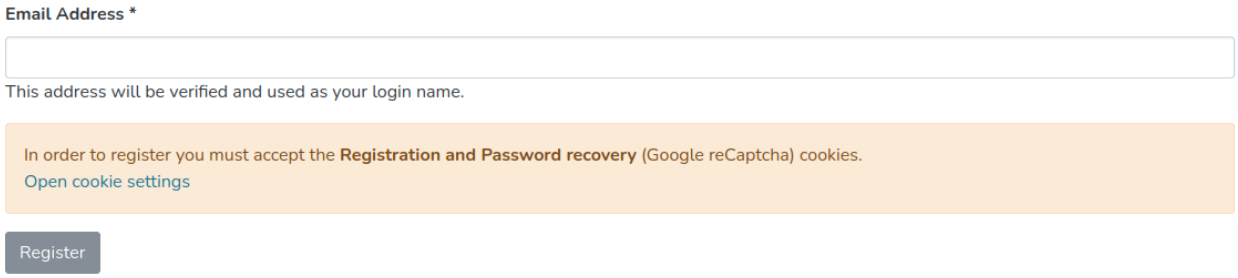
Once feature is enabled, the user registration will actually be performed if and only if the CAPTCHA token, passed in registration payload, is verified during registration process itself and is considered valid. Each registration request, even if made using DSpace REST APIs must have a captcha token in its header.



A new type of cookie has been added to DSpace cookie set, "Registration and Password Recovery". This cookie is proposed only when CAPTCHA verification is enabled.



This cookie option must be enabled by users before registering, otherwise they won't be able to perform a registration



Configurable Entities

- Introduction
- Default Entity Models
  - Research Entities
  - Journals
- Enabling Entities
  - 1. Configure your entity model (optionally)
  - 2. Import entity model into the database
  - 3. Configure Collections for each Entity type
  - 4. Configure Submission Forms for each Entity type
    - 4.1 Use of collection-entity-type attribute for default Submission forms per Entity Type
  - 5. Configure Workflow for each Entity type (optionally)
  - 6. Configure Virtual Metadata to display for related Entities (optionally)
- Designing your own Entity model
  - Thinking about the object model
  - Configuring the object model
  - Configuring the metadata fields
  - Configure the submission forms
  - Configuring the item display pages
  - Configuring virtual metadata
  - Configuring discovery
- Additional Technical Details
  - Database Storage
  - Ordering of Relations (place column)
    - Unidirectional example
    - Bidirectional example
  - Tilted relationships
    - Use case: OrgUnit vs Publication relationship
    - Impact of the tilted relationships
- Versioning Support
  - Example of the latest status of a relationship (technical details)
  - Metadata fields that represent relations
  - Configure versioning for an entity type

Introduction

DSpace users have expressed the need for DSpace to be able to provide more support for different types of digital objects related to open access publications, such as authors/author profiles, data sets etc. Configurable Entities are designed to meet that need.

In DSpace, **an Entity is a special type of Item which often has Relationships to other Entities**. Breaking it down with more details...

- **Entity:** Every Entity is an Item.
  - This means they must belong to a Collection, just like a normal Item. (Community & Collection objects are unchanged and unaffected by Entities.)
  - Normal Items are still the "default" Item, and they are unchanged. So, not every Item is an Entity.
  - Because Entities are all Items, they are immediately usable in submission/workflow process, batch import/export, OAI-PMH, etc.
- **Entity (or Item) Type:** Entities all have a "dspace.entity.type" metadata field which defines their Entity/Item "type". For example, this type may be "Person", "Project", "Publication", "Journal", etc. It's highly visible within the User Interface as a label.
- **Relationships:** Based on that "type", an Entity may be related to other Entities via a Relationship. One Entity type may support several relationship types at once. Examples of relationship types include "isPersonOfProject" or "isPublicationOfAuthor". These relationship types are named based on the Entity "type" (as you can likely tell). Relationships also appear on Entities as metadata using the "relation" schema.
- **Virtual Metadata:** Entities of different types may also have customized visualizations in the User Interface. These visualizations may also dynamically pull in metadata from related Entities. For example, a Publication entity may be displayed in the User Interface with an author name dynamically pulled in from a related Person entity. The metadata "appears" as though it is part of the Entity you are viewing, but it is dynamically pulled via the Relationship.

Entities and their Relationships are also completely configurable. DSpace provides some sample models out of the box, which you can use directly or adapt as needed.

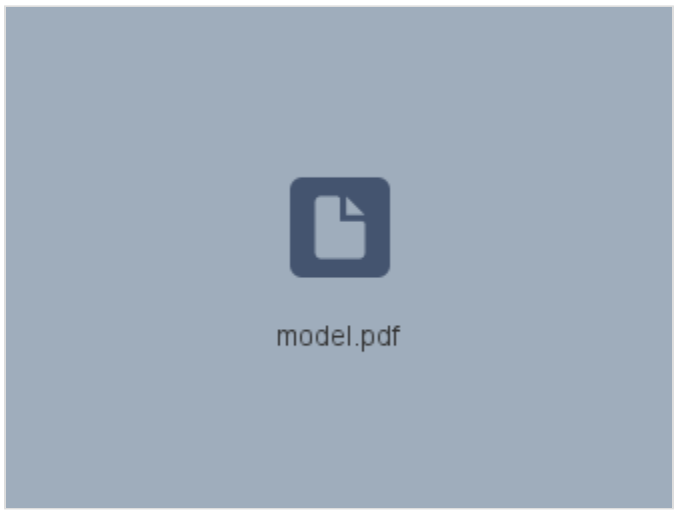
The Entity model also has similarities with the Portland Common Data Model (PCDM), with an Entity roughly mapping to a "pcdm:Object" and existing Communities and Collections roughly mapping to a "pcdm:Collection". However, at this time DSpace Entities concentrate more on building a graph structure of relationships, instead of a tree structure.

Default Entity Models

DSpace currently comes with the following Entity models, both of which are defined in `[dspace]/config/entities/relationship-types.xml`. These Entity models are not used by default, but may be enabled as described below.

Research Entities

Research Entities include Person, OrgUnit, Project and Publication. They allow you to create author profiles (Person) in DSpace, and relate those people to their department(s) (OrgUnit), grant project(s) (Project) and works (Publication).



- Each publication can link to projects, people and org units
- Each person can link to projects, publications and org units
- Each project can link to publications, people and org units
- Each org units can link to projects, people and publications

Journals

Journal Entities include Journal, Journal Volume, Journal Issue and Publication (article). They allow you to represent a Journal hierarchy more easily within DSpace, starting at the overall Journal, consisting of multiple Volumes, and each Volume containing multiple Issues. Issues then link to all articles (Publication) which were a part of that journal issue.

NOTE: that this model includes the same "Publication" entity as the Research Entities model described above. This Entity overlap allows you to link an article (Publication) both to its author (Person) as well as the Journal Issue it appeared in.

Enabling Entities

By default, Entities are not used in DSpace. But, as described above several models are available out-of-the-box that may be optionally enabled.

Keep in mind, there are a few DSpace import/export features that do not yet support Entities in DSpace 7.0. These will be coming in future 7.x releases. See DSpace Release 7.0 Status for prioritization information, etc.

- AIP Backup and Restore does not fully support entity types or relationships. In other words, Entities are only represented as normal Items in AIPs
- Importing and Exporting Items via Simple Archive Format does not fully support entity types or relationships. In other words, Entities are only represented as normal Items in SAF. (Note: early work to bring this support is already begun in https://github.com/DSpace/DSpace/pull/3322)
- SWORDv1 Server and SWORDv2 Server does not yet support Entity or relationship creation.

1. Configure your entity model (optionally)

As described above, DSpace provides two default entity models defined in `[dspace]/config/entities/relationship-types.xml`. These models may be used as-is, or modified.

You can also design your own model from scratch (see "Designing your own model" section below). So, feel free to start by modifying `relationship-types.xml`, or creating your own model based on the `relationship-types.dtd`.

A simple XML configuration file is used to determine the relations. This configuration is read into the database (making modeling the data from the UI possible in future versions as well). A sample of how to define a relationship between Publication and Person is shown below:

```
<!-- This relationship defines both the Publication and Person Entities, along with how they are related
together. -->
<relationships>
    <type>
        <leftType>Publication</leftType>
        <rightType>Person</rightType>
        <leftLabel>isAuthorOfPublication</leftLabel>
        <rightLabel>isPublicationOfAuthor</rightLabel>
        <leftCardinality>
            <min>0</min>
            <!--<max></max> not specified, unlimited-->
        </leftCardinality>
        <rightCardinality>
            <min>0</min>
            <!--<max></max> not specified, unlimited-->
        </rightCardinality>
    </type>
</relationships>
```

2. Import entity model into the database

In order to enable a defined entity model, it MUST be imported into the DSpace database This is achieved by using the "initialize-entities" script. The example below will import the "out-of-the-box" entity models into your DSpace installation

```
# The -f command requires a full path to an Entities model configuration file.
[dspace]/bin/dspace initialize-entities -f [dspace]/config/entities/relationship-types.xml
```

If an Entity (of same type name) already exists, it will be updated with any new relationships defined in relationship-types.xml

If an Entity (of same type name) doesn't exist, the new Entity type will be created along with its relationships defined in relationship-types.xml

Once imported into the Database, the overall structure is as follows:

- All valid Entity Types are stored in the "entity_type" database table.
- All Relationship type definitions are stored in the "relationship_type" database table
- All Relationships between entities get stored in the "relationship" table.
- Entities themselves are stored alongside Items in the 'item' table. Every Entity must have a "dspace.entity.type" metadata field whose value is a valid Entity Type (from the "entity_type" table).

Keep in mind, your currently enabled Entity model is defined in your database, and NOT in the "relationship-types.xml". Anytime you want to update your data model, you'd update/create a configuration (like relationship-types.xml) and re-run the "initialize-entities" command.

3. Configure Collections for each Entity type

Because all Entities are Items, they MUST belong to a Collection. Therefore, the *recommended way* to create a different submission forms per Entity type (e.g. Person, Project, Journal, Publication, etc) is to *ensure you create a Collection for each Entity Type* (as each Collection can have a custom Submission Form).

1. Create at least one Collection for each Entity Type needing a custom Submission form. For example, a Collection for "Person" entities, and a separate one for "Publication" entities.
2. Edit the Collection. On the "Edit Metadata" page, use the "Entity Type" dropdown to select the Entity Type for this Collection.
    a. This "Entity Type" selection will ensure that every Item submitted to this collection is automatically assigned that Entity type. So, it ties this Collection to that type of Entity (i.e. *no other type of Entity can be submitted to this Collection).*
        i. *NOTE: Entity Type is currently not modifiable after being set.* This is because changing the Entity type may result in odd behavior (or errors) with in-progress submissions (as they will continue to use the old Entity Type). If you really need to modify the Entity Type, you can do so by changing the "dspace.entity.type" metadata value on the Collection object. At this time, changing that metadata field would need to be done at the database level.
    b. *NOTE:* In 7.0, this "Entity Type" dropdown did not exist. In that release, you have to create a "Template Item" from that page. In the In the Template Item, add a single metadata field "dspace.entity.type". Give it a value matching the Entity type (e.g. Publication, Person, Project, OrgUnit, Journal, JournalVolume, JournalIssue). This value IS CASE SENSITIVE and it MUST match the Entity type name defined in relationship-types.xml
        i. As of 7.1 (or above) , if you previously created a Template Item in 7.0, the "dspace.entity.type" field value will be migrated to the "Entity Type" dropdown automatically (via a database migration).
3. In the Edit Collection page, switch to the "Assign Roles" tab and create a "Submitters" group. Add any people who should be allowed to submit /create this new Entity type.
    a. If you only want Administrators to create this Entity type, you can skip this step. Administrators can submit to any Collection.
4. If you want to hide this Collection, you can choose to only make it visible to that same Submitters group (or Administrators). This does NOT hide the Entities from search or browse, but it will hide the Collection itself.
    a. In the Edit Collection page, switch to the "Authorizations" tab.

b. Add a new Authorization of TYPE_CUSTOM, restricting "READ" to the Submitters group created above (or Administrators if there is no Submitters group). You can also add multiple READ policies as needed. WARNING: The Submitters group MUST have READ privileges to be able to submit/create new Entities.
c. Remove the default READ policy giving Anonymous permissions.
d. Assuming you want the Entities to still be publicly available, make sure the DEFAULT_ITEM_READ policy is set to "Anonymous"!

Each of these Collections must be contained in a Community. The simplest thing to do would be to create a new top-level Community, place your entity Collections there, and (optionally) hide the Community in a fashion similar to "hide this Collection" above.

Obviously, how you organize your Entity Types into Collections is up to you. You can create a single Collection for all Entities of that type (e.g. an "Author Profiles" collection could be where all "Person" Entities are submitted/stored). Or, you could create many Collections for each Entity Type (e.g. each Department in your University may have it's own Community, and underneath have a "Staff Profiles" Collection where all "Person" Entities for that department are submitted/stored). A few example structures are shown below.

Example Structure based on the departments:

- Department of Architecture
  - Building Technology Program
  - Theses - Department of Architecture
- Department of Biology
  - Theses - Biology
- People
- Projects

OR

- Department of Architecture
  - Building Technology Program
  - Theses - Department of Architecture
  - People in Department of Architecture
  - Projects in Department of Architecture
- Department of Biology
  - Theses - Biology
  - People in Department of Biology
  - Projects in Department of Biology

Example Structure based on the publication type:

- Books
  - Book Chapter
  - Edited Volume
  - Monograph
- Theses
  - Bachelor Thesis
  - Doctoral Thesis
  - Habilitation Thesis
  - Master Thesis
- People
- Projects

4. Configure Submission Forms for each Entity type

You should have already created Entity-specific Collections in the previous step. Now, we just need to map those Collections to Submission processes specific to each Entity.

On the backend, you will now need to modify the `[dspace]/config/item-submission.xml` to "map" this Collection (or Collections) to the submission process for this Entity type.

- DSpace comes with sample submission forms for each Entity type.
  - The sample <submission-process> is defined in `item-submission.xml` and named based on the Entity type (e.g. Publication, Person, Project, etc).
  - The metadata fields captured for each Entity are defined in a custom step in `submission-forms.xml`, and named in the format "[entityType]Step" (where the entity type is camelcased). For example: "publicationStep", "personStep", "projectStep".
- Optionally, modify those sample submission forms. See Submission User Interface for hints/tips on customizing the `item-submission.xml` or `submission-forms.xml` files
- As of 7.6, you can simply map each Entity Type to a specific submission form as follows in your `item-submission.xml` (This section already exists, just uncomment it)

```
<name-map collection-entity-type="Publication" submission-name="Publication"/>
<name-map collection-entity-type="Person" submission-name="Person"/>
<name-map collection-entity-type="Project" submission-name="Project"/>
<name-map collection-entity-type="OrgUnit" submission-name="OrgUnit"/>
<name-map collection-entity-type="Journal" submission-name="Journal"/>
<name-map collection-entity-type="JournalVolume" submission-name="JournalVolume"/>
<name-map collection-entity-type="JournalIssue" submission-name="JournalIssue"/>
```

- WARNING: If you create a new Collection using a specific Entity Type, you must currently restart your servlet container (e.g. Tomcat) for the submission form configuration to take effect for the new Collection. This is the result of a known bug where the Submission forms are cached until the servlet container is restarted. See this issue ticket: https://github.com/DSpace/DSpace/issues/7985
- In 7.5 and earlier, you needed to map each Collection's handle one by one to a Submission form in `item-submission.xml`. Map your Collection's handle (findable on the Collection homepage) to the submission form you want it to use. In the below example, we've mapped a single Collection to each of the out-of-the-box Entity types.

```
<name-map collection-handle="123456789/5" submission-name="Publication"/>
<name-map collection-handle="123456789/6" submission-name="Person"/>
<name-map collection-handle="123456789/7" submission-name="Project"/>
<name-map collection-handle="123456789/8" submission-name="OrgUnit"/>
<name-map collection-handle="123456789/28" submission-name="Journal"/>
<name-map collection-handle="123456789/29" submission-name="JournalVolume"/>
<name-map collection-handle="123456789/30" submission-name="JournalIssue"/>
```

Once your modifications to the submission process are complete, you will need to quickly reboot Tomcat (or your servlet container) to reload the current settings.

4.1 Use of collection-entity-type attribute for default Submission forms per Entity Type

Alternatively to a collection's Handle, Entities Types can be used as an attribute. So, instead of specifying the collection handle, you will need to use the `collection-entity-type` attribute and what Entity Type to use (like: Person, Project). Please mind that your Collections with Entity Type need to be previously created.

```
<name-map collection-entity-type="Publication" submission-name="Publication"/>
<name-map collection-entity-type="Person" submission-name="Person"/>
<name-map collection-entity-type="Project" submission-name="Project"/>
<name-map collection-entity-type="OrgUnit" submission-name="OrgUnit"/>
<name-map collection-entity-type="Journal" submission-name="Journal"/>
<name-map collection-entity-type="JournalVolume" submission-name="JournalVolume"/>
<name-map collection-entity-type="JournalIssue" submission-name="JournalIssue"/>
```

Once your modifications to the submission process are complete, you will need to quickly reboot Tomcat (or your servlet container) to reload the current settings.

For DSpace 7.6 release it requires Tomcat Restart for every new collection

Due to the way SubmissionConfigReader is loaded into memory (on a initialize process) currently there is no implemented way to reload submission forms. So, every time you assign an entity type to a collection, or create a new collection with an associated entity type, **you will need to do a Tomcat restart** for that collection to be available at the item submission config. There is an on going fix for that.

DSpace 7.6.1 introduced a fix and you don't need to do a Tomcat Restart anymore

DSpace 7.6.1 adds a way to reload Submission Configs, so you no longer need to do a Tomcat Restart after creating a new collection with an entity type, or assigning to a existing one.

5. Configure Workflow for each Entity type (optionally)

The DSpace workflow can be used for reviewing all objects in the Object Model since these objects are all Items, and separate collections can be used. The workflow used for e.g. a Person Object can be configured to be identical to a publication, different from a publication, or use no workflow at all.

See Configurable Workflow for more information on configuring workflows per Collection.

6. Configure Virtual Metadata to display for related Entities (optionally)

"Virtual Metadata" is metadata that is dynamically determined (at the time of access) based on an Entity's relationship to other Entities. A basic example is displaying a Person Entity's name in the "dc.contributor.author" field of a related Publication Entity. That "dc.contributor.author" field doesn't actually exist on the Publication, but is dynamically added as "virtual metadata" simply because the Publication is linked to the Person (via a relationship).

Virtual Metadata is configurable for all Entities and all relationships. DSpace comes with default settings for its default Entity model, and those can be found in `[dspace]/config/spring/api/virtual-metadata.xml`. In that Spring Bean configuration file, you'll find a map of each relationship type to a metadata field & its value. Here's a summary of how it works:

- The "org.dspace.content.virtual.VirtualMetadataPpopulator" bean maps every Relationship type (from `relationship-types.xml`) to a <util: map> definition (of a given ID) also in the `virtual-metadata.xml`

```
<!-- For example, the isAuthorOfPublication relationship is linked to a map of ID
"isAuthorOfPublicationMap" -->
<entry key="isAuthorOfPublication" value-ref="isAuthorOfPublicationMap"/>
```

- That <util:map> defintion defines which DSpace metadata field will store the virtual metadata. It also links to the bean which will dynamically define the value of this metadata field.

```
<!-- In this example, isAuthorOfPublication will be displayed in the "dc.contributor.author" field -->
<!-- The *value* of that field will be defined by the "publicationAuthor_author" bean -->
<util:map id="isAuthorOfPublicationMap">
    <entry key="dc.contributor.author" value-ref="publicationAuthor_author"/>
</util:map>
```

- A bean of that ID then defines the value of the field, based on the related Entity. In this example, these fields are pulled from the related Person entity and concatenated. If the Person has "person.familyName=Jones" and "person.givenName=Jane", then the value of "dc.contributor.author" on the related Publication will be dynamically set to "Jones, Jane.

```
 <bean class="org.dspace.content.virtual.Concatenate" id="publicationAuthor_author">
    <property name="fields">
        <util:list>
            <value>person.familyName</value>
            <value>person.givenName</value>
            <value>organization.legalName</value>
        </util:list>
    </property>
    <property name="separator">
        <value>, </value>
    </property>
    <property name="useForPlace" value="true"/>
    <property name="populateWithNameVariant" value="true"/>
</bean>
```

If the default Virtual Metadata looks good to you, no changes are needed. If you make any changes, be sure to restart Tomcat to update the bean definitions.

Designing your own Entity model

When using a different entities model, the new model has to be configured an loaded into your repository

Thinking about the object model

First step: identify the entity types

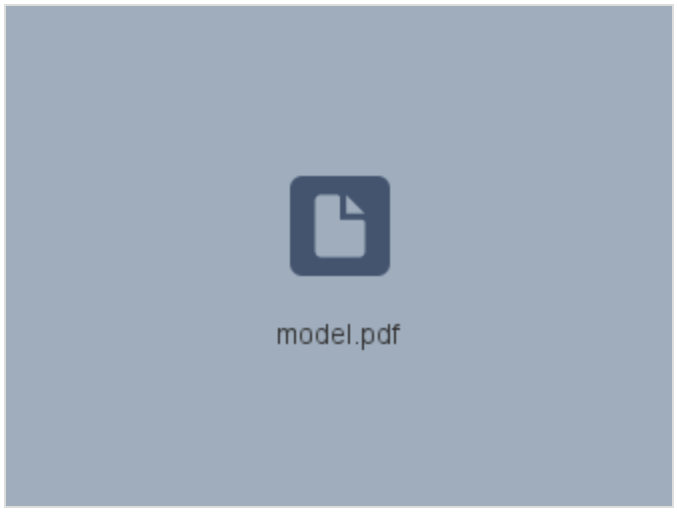
- Which types of objects would you want to create items for: e.g. Person, Publication, JournalVolume
- Be careful not to confuse a type with a relationship. A Person is a type, an author is a relationship between the publication and the person

Second step: identify the relationship types

- Which relationship types would you want to create between the entity items from the previous step: e.g. isAuthorOfPublication, isEditorOfPublication, isProjectOfPublication, isOrgUnitOfPerson, isJournalIssueOfPublication
- Multiple relationships between the same 2 types can be created: isAuthorOfPublication, isEditorOfPublication
- Relationships are automatically bidirectional, so no need to worry about whether you want to display the authors in a publication or the publications of an author

Third step: visualize your model

- By creating a drawing of your model, you'll be able to quickly verify whether anything is missing



Configuring the object model

Configure the model in relationship-types.xml

- Similar to the default relationship-types.xml, configure a relationship type per connection between two Entity Types
- Include the two Entity Type names which are being connected.
- Determine a clear an unambiguous name for the relation in both directions
- Optionally: determine the cardinality (min/max occurrences) for the relationships
- Optionally: determine default behavior for copying metadata if the relationship is deleted

Configuring the metadata fields

Determining the metadata fields to use

- Dublin Core works for publications, but not for a Person, JournalVolume, …
- There are many standards which can be easily configured: schema.org, eurocris, datacite, …
- Pick a schema which suits your needs

Configure the submission forms

- Add a form in submission-forms.xml for each entity type, containing the relevant metadata fields
- See also Submission User Interface documentation.
- Configure which relationships to create

Configuring the item display pages

- The metadata configuration is not specific to configurable entities.
- Similar to other customizations to the item display pages, configure in Angular which metadata fields to display and their label. A template per entity type can be created
- The relationship display is similar to the metadata configuration
- Similar to the metadata configuration: configure in Angular which relationship to display and their label

Configuring virtual metadata

- The isAuthorOfPublication relationship can be displayed for the Publication item as dc.contributor.author
- The isOrgUnitOfPerson relationship can be displayed for the Person item as organization.legalName
- This can be configured in virtual-metadata.xml

Configuring discovery

- Configure the discovery facets, filters, sort options, …
  - The facets for a Person can be job title, organization, project, …
  - The filters for a Person can be person.familyName, person.givenName, …

Additional Technical Details

The original Entities design document is available in Google Docs at: https://docs.google.com/document/d/1wEmHirFzrY3qgGtRr2YBQwGOvH1IuTVGmxDIdnqvwxM/edit While this content has been copied into the DSpace wiki, the file remains available as needed.

A talk on Configurable Entities was also presented at DSpace 7 at OR2021

Database Storage

The actual type of entity is stored in the "dspace.entity.type" metadata field.

Flyway will create tables to store the entity type and relationship configuration. The entity type table contains a unique ID and name for the entity type. This table typically only contains a few rows. It uses database indexes to easily and quickly find the entity types.

Table "entity_type"

```
 Column |         Type          | Modifiers
--------+-----------------------+-----------
 id     | integer               | not null
 label  | character varying(32) | not null
```

The relationship type table contains a unique ID, the 2 entity type IDs (foreign keys), the labels for the relation, and the cardinality details (min and max occurrences in each direction). This table typically only contains a few rows. It uses database indexes to easily and quickly find the relations types.

Table "relationship_type"

```
        Column         |         Type          | Modifiers
-----------------------+-----------------------+-----------
 id                    | integer               | not null
 left_type             | integer               | not null
 right_type            | integer               | not null
 left_label            | character varying(32) | not null
 right_label           | character varying(32) | not null
 left_min_cardinality  | integer               |
 left_max_cardinality  | integer               |
 right_min_cardinality | integer               |
 right_max_cardinality | integer               |
```

The actual relations are stored in another new database table, containing a row per relation between two entities.

Table "relationship"

```
   Column    |  Type   | Modifiers
-------------+---------+-----------
 id          | integer | not null
 left_id     | uuid    | not null
 type_id     | integer | not null
 right_id    | uuid    | not null
 left_place  | integer |
 right_place | integer |
```

It contains a unique ID, the ID of the relationship type (a foreign key to the table above), and the left and right item (foreign keys to the actual items), and place values to keep track of the position of the entity for the given relationship type. It uses database indexes to easily and quickly find all relations from the current item.

Ordering of Relations (place column)

The “place” columns for relations are similar to the "place" column in the metadatavalue table, which also determines the order of the values. It gets updated automatically with the next number if a relation is created, similar to the current solution to populate the "place" column in the metadatavalue table.

The reason why there's a left and right place separately, is because we don't want the model to restrict which part of the relation has a "place".

Unidirectional example

For a journal, there are relations to the journal volumes as visible in REST. The relevant section is:

https://demo.dspace.org/entities/journalvolume/f9b89a11-b44e-4a64-a3b4-ab24a33553c7

"leftId": "a23eae5a-7857-4ef9-8e52-989436ad2955",
"rightId": "f9b89a11-b44e-4a64-a3b4-ab24a33553c7",
"leftPlace": 1,
"rightPlace": 1,

https://demo.dspace.org/entities/journalvolume/343d3263-2733-4367-9dc4-216a01b4a461

"leftId": "a23eae5a-7857-4ef9-8e52-989436ad2955",
"rightId": "343d3263-2733-4367-9dc4-216a01b4a461",
"leftPlace": 2,
"rightPlace": 1,

For this particular relation, the leftPlace is relevant because the leftId contains the current item ID. First the item with leftPlace 1 is displayed. Hereafter the item with leftPlace 2 is displayed.

Bidirectional example

For an article, there are relations to the OrgUnits visible in REST. The relevant section is:

https://demo.dspace.org/entities/orgunit/d30de96b-1e76-40ae-8ef9-ab426b6f9763

"leftId": "96715576-3748-4761-ad45-001646632963",
"rightId": "d30de96b-1e76-40ae-8ef9-ab426b6f9763",
"leftPlace": 1,
"rightPlace": 2,

https://demo.dspace.org/entities/orgunit/c216201f-ed10-4361-b0e0-5a065405bd3e

"leftId": "96715576-3748-4761-ad45-001646632963",
"rightId": "c216201f-ed10-4361-b0e0-5a065405bd3e",

"leftPlace": 2,
"rightPlace": 2,

For this particular relation, the leftPlace is relevant because the leftId contains the current item ID. First the item with leftPlace 1 is displayed. Hereafter the item with leftPlace 2 is displayed.

In the other direction, on the OrgUnit item page there are 6 publications visible in REST. The relevant section is:

https://demo.dspace.org/entities/publication/e98b0f27-5c19-49a0-960d-eb6ad5287067

"leftId": "e98b0f27-5c19-49a0-960d-eb6ad5287067",
"rightId": "d30de96b-1e76-40ae-8ef9-ab426b6f9763",
"leftPlace": 1,
"rightPlace": 1,

https://demo.dspace.org/entities/publication/96715576-3748-4761-ad45-001646632963

"leftId": "96715576-3748-4761-ad45-001646632963",
"rightId": "d30de96b-1e76-40ae-8ef9-ab426b6f9763",
"leftPlace": 1,
"rightPlace": 2,

https://demo.dspace.org/entities/publication/2f4ec582-109e-4952-a94a-b7d7615a8c69

"leftId": "2f4ec582-109e-4952-a94a-b7d7615a8c69",
"rightId": "d30de96b-1e76-40ae-8ef9-ab426b6f9763",
"leftPlace": 2,
"rightPlace": 3,

For this particular relation, the rightPlace is relevant because the rightId contains the current item ID. First the item with rightPlace 1 is displayed. Hereafter the item with rightPlace 2 is displayed, …

The relation with rightPlace 2 is the same relation as mentioned for the article above

The default DSpace database tables will not need to be modified as the entity type is part of the regular metadata.

Tilted relationships

The tilted relationships are a default DSpace 7 feature, developed in https://github.com/DSpace/DSpace/pull/3134

It's designed to improve performance when an entity has 1000s of relationships. It will avoid loading the relationships in the configured direction unless explicitly requested to retrieve them. It's mainly used for setups where there are so many related items that it doesn't make sense to list them all, and they are rather made available via search. Using tilted relationships, those setups can get a big performance boost.

The behavior can most easily be explained using a real-life example

Use case: OrgUnit vs Publication relationship

DSpace 7 has an `isOrgUnitOfPublication` relationship type defined by default. When there are OrgUnits with 1000s or more Publications linked to them, the default `isOrgUnitOfPublication` relationship can run into performance issues.

When setting the tilted to `left`, where the left entity type is `Publication` and the right entity type is `OrgUnit`, you are specifying that the `Publication` should still load the related `OrgUnit`s (expected to be a small amount anyway), but the `OrgUnit` is not supposed to load all the related `Publication`s.

The implications of this setup are:

- When checking the `OrgUnit` metadata, it won't populate the `relation.isPublicationOfOrgUnit` virtual metadata
- When checking the `OrgUnit` relationships, it won't return the relationships based on this type
- But using the relationship search angular component, the related `Publication`s can still be displayed on the `OrgUnit` item page
- It is also still possible to use the relationship list angular component on the `OrgUnit` item page without a performance impact, but for the `OrgUnit` entities, the pageable list doesn't have much added value
- When checking the `Publication` metadata, it will still populate the `relation.isOrgUnitOfPublication` virtual metadata, and can also populate any other virtual metadata such as the OrgUnit name on the Publication
- When checking the `Publication` relationships, it will include the relationships based on this type
- The relationship list angular component on the `Publication` item page can be used, and won't have any issues

Impact of the tilted relationships

Performance tests were performed in https://github.com/DSpace/DSpace/pull/3134 (some tests were specific to improvements which are independent from tilted relationships, and which are already applied in all cases)

In general:

- Retrieving the item REST representation which includes virtual metadata when using the tilted relationships for items with 600 relationships reduced the duration to 5% (so a 95% reduction in time spent)
- Creating new relationships when using the tilted relationships for items with 1000 relationships or more reduced the duration as well, but more recent tests revealed this reduced the duration to 25% (so a 75% reduction in time spent)

Using tilted relationships, there are production environments with single entities having over 40k relationships using a single relationship type. Using the tilted relationships, this setup doesn't impact the performance.

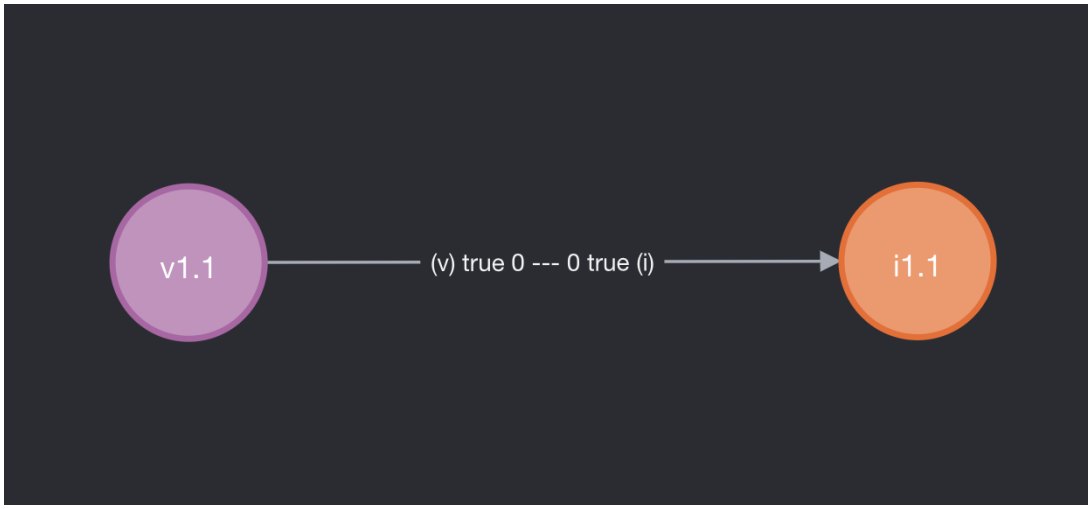
Using the OrgUnit to Person relationship, it's possible to have an OrgUnit with 40k Person entities, and the virtual metadata of the Person entity contains the Org Unit's name, ID, ….

Versioning Support

DSpace entities **fully support versioning**. For the most part, this works like any other item. For example, when creating a new version of an item, a new item is created and all metadata values of the preceding item are copied over to the new item. Special care was taken to version relationships between entities.

Example of the latest status of a relationship (technical details)

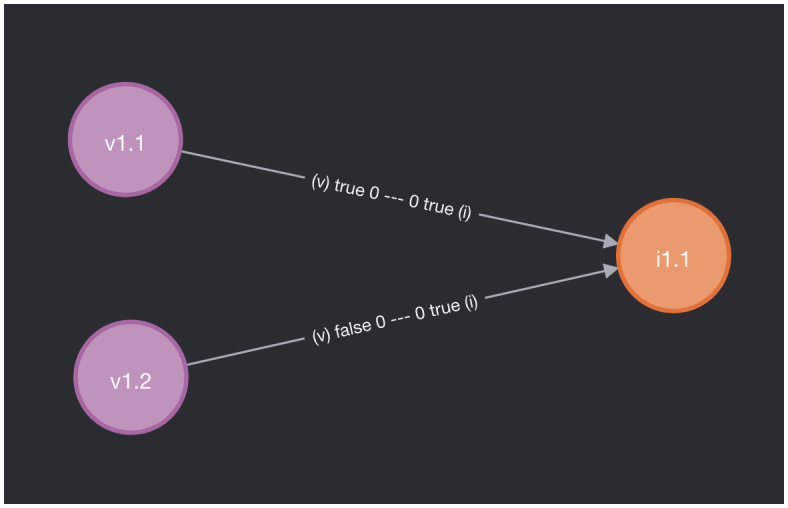
To understand how versioning between entities with relationships works, let's walk through the following example:



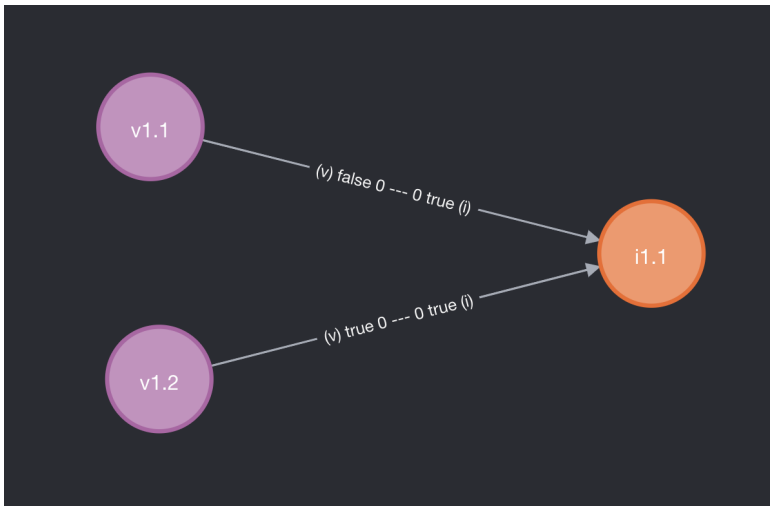
Consider Volume 1.1 (left side) and Issue 1.1 (right side). Both are archived and both are the first version. Note that on the arrow, representing the relation between the volume and the issue, two booleans and two numbers are indicated.

- The boolean on side `(v)` is true if and only if volume 1.1 is the latest version that is relevant to issue 1 (even though it may be possible that volume 1.2, the second version of volume 1, exists). This means that on the item page of issue 1.1, a link to the item page of volume 1.1 should be displayed. It also means that searching for (the uuid of) issue 1.1 should yield volume 1.1.
- The boolean on side `(i)` is true if and only if issue 1.1 is the latest version that is relevant to volume 1.1 (even though it may be possible that issue 1.1, the second version of issue 1, exists). This means that on the item page of volume 1.1, a link to the item page of issue 1.1 should be displayed. It also means that searching for (the uuid of) volume 1.1 should yield issue 1.1.
- The number on side `(v)` indicates the place at which the virtual metadata representing this relationship (if any) will appear on volume 1.1. E.g. using the out-of-the-box configuration in `virtual-metadata.xml`, metadata field `publicationissue.issueNumber` of issue 1.1 would appear as metadata field `publicationissue.issueNumber` on volume 1.1 on place 0 (i.e. as the first metadata value).
- The number on side `(i)` indicates the place at which the virtual metadata representing this relationship (if any) will appear on issue 1.1. E.g. using the out-of-the-box configuration in `virtual-metadata.xml`, metadata field `publicationvolume.volumeNumber` of volume 1.1 would appear as metadata field `publicationvolume.volumeNumber` on issue 1.1 on place 0 (i.e. as the first metadata value).

With the groundwork out of the way, let's see what happens when we create a new version of volume 1.1. The new version is not yet archived, because it still has to be edited in the submission UI.

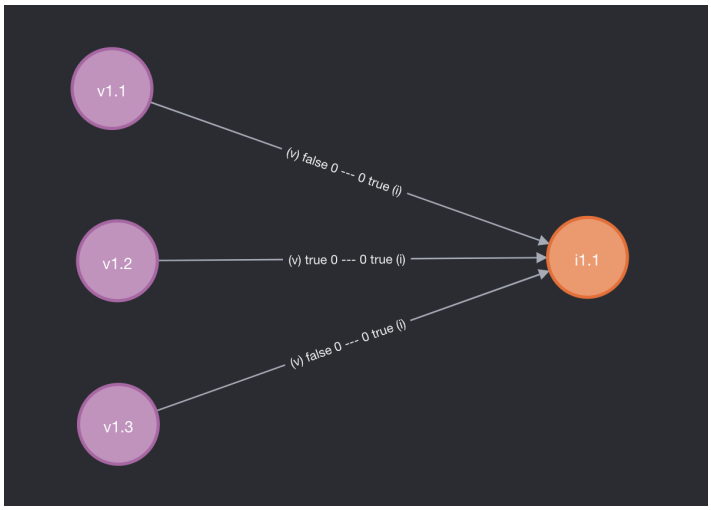


At this moment, when viewing the item page of issue 1.1, the user should only see volume 1.1 (as volume 1.2 is not yet archived). When viewing the item page of volume 1.1, nothing has changed: only a link to issue 1.1 will appear. When viewing the item page of volume 1.2 (e.g. as an admin), a link to issue 1.1 will appear as well.

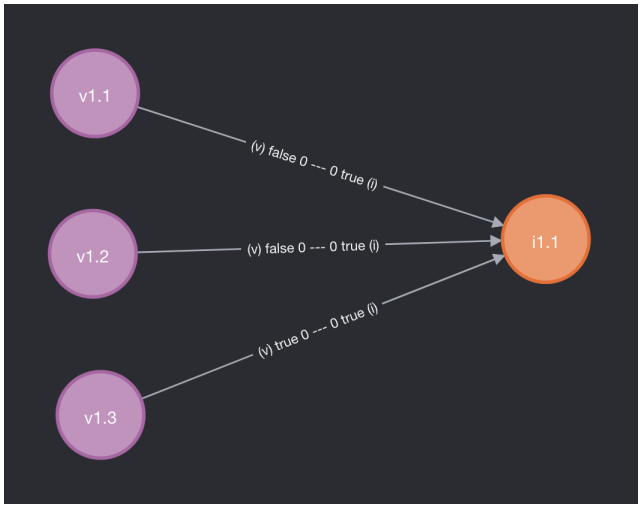


As soon as volume 1.2 is deposited (archived), the "latest status" of both volume 1.1 and volume 1.2 are updated. When viewing the item page of issue 1.1, volume 1.2 should be visible. When viewing the item pages of the volumes, nothing has changed.

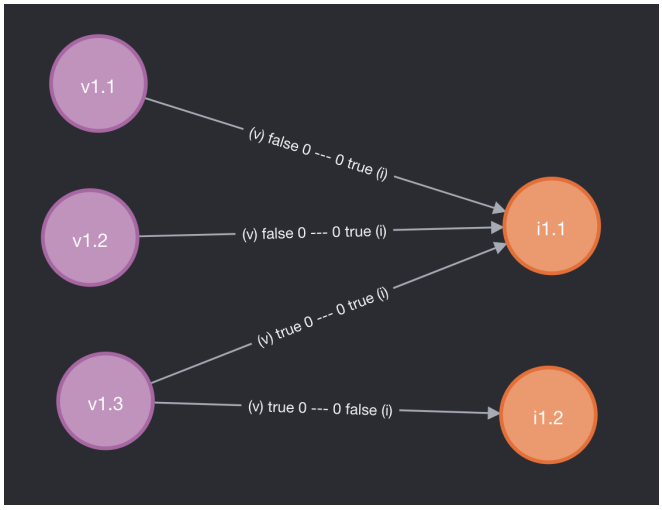
Let's create another version of the volume (not archived):



And after archiving volume 1.3:



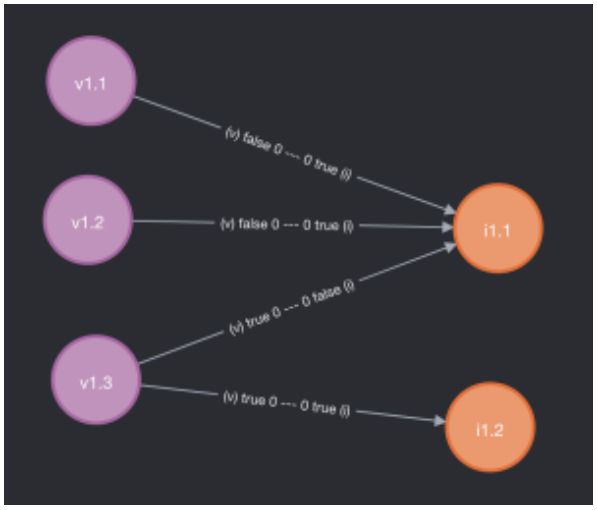
What happens if we create a new version of issue 1.1?



Only the relationship with volume 1.3 is copied. For issue 1.1, no relationship was displayed with volume 1.1 and 1.2. (The relationships still exist in the database, but are not visible in the UI.). For volume 1.1, a relationship to issue 1.1 remains present, but it should not be updated to issue 1.2. For issue 1.2, these relationships are longer relevant, so they are not copied.

On the item pages of volume 1.1, volume 1.2 and volume 1.3, you should see issue 1.1 (as 1.2 is not archived yet)

Because issue 1.2 is not yet archived, all volumes are still pointing to issue 1.1. Let's archive it:



Now on the item pages of volume 1.1 and volume 1.2, you should see issue 1.1; it's the latest issue at the time that those volumes were superseded by volume 1.3. On the item page of volume 1.3, you'll see issue 1.3. On the item page of issue 1.1 you'll still see volume 1.3 as well.

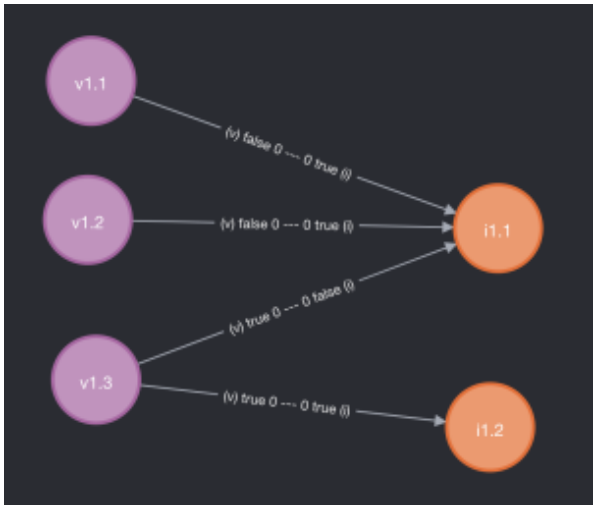
Metadata fields that represent relations

If you have a closer look at items with relationships, you'll notice two categories of metadata fields that are controlled by DSpace:

- `relation.*` fields, for example `relation.isIssueOfJournalVolume` on volume items
- `relation.*.latestForDiscovery` fields, for example `relation.isIssueOfJournalVolume.latestForDiscovery` on volume items

Metadata fields of the first category (`relation.*`) contain all uuids of related items that the current item can see. I.e. a relationship has to exist between the current item and the other item, and the other item needs to have "latest status" for that specific relationship.

As an example take the following state of the previous section:



Item issue 1.1 will contain metadata field `relation.isJournalVolumeOfIssue` with as value the uuid of volume 1.3. Volume 1.1 and 1.2 are not included because they don't have "latest status" on the relevant relationships.

Metadata fields of the second category (`relation.*.latestForDiscovery`) contain all uuids of the items for which the current item is visible. I.e. a relationship has to exist between the current item and the other item, and the current item needs to have "latest status" for that specific relationship. These fields are particularly important for indexing and search, because they allow to us to surface all the items that a particular item is referring to.

Continuing on the example above, issue 1.1 will have metadata field `relation.isJournalVolumeOfIssue.latestForDiscovery` containing the uuids of volume 1.1 and 1.2.

With issue 1.1 containing volume 1.1 and 1.2 in `relation.isJournalVolumeOfIssue.latestForDiscovery`, a search on the volume 1.1 page for all issues containing volume 1.1 will display issue 1.1 thanks to this setup.

Configure versioning for an entity type

DSpace contains a bunch of example entity types that support versioning out of the box. What follows is an overview of the requirements to make entity versioning work.

1. when introducing a relationship type, make sure to add four new metadata fields to `config/registries/relationship-formats.xml`. E.g. `relation.isAuthorOfPublication`, `relation.isAuthorOfPublication.latestForDiscovery`, `relation.isPublicationOfAuthor` and `relation.isPublicationOfAuthor.latestForDiscovery`
2. when introducing an entity type, filter items on `latestVersion:true` in `discovery.xml`. This will be the default search, which ensures older versions are not shown
    - If you want to show all related items, including older versions, you can create another discovery config without `latestVersion:true`. This should be used for item pages displaying the related items to the current item using the discovery search.
    - The entity types configured out-of-the-box have discovery config `<entity-type>` and discovery config `<entity-type>Relationships` for that purpose.

Note that versioning support is enabled by default, but can be turned off by setting `versioning.enabled = false` in `versioning.cfg` or `local.cfg` . For more details on item versioning, see: https://wiki.lyrasis.org/display/DSDOC7x/Item+Level+Versioning.

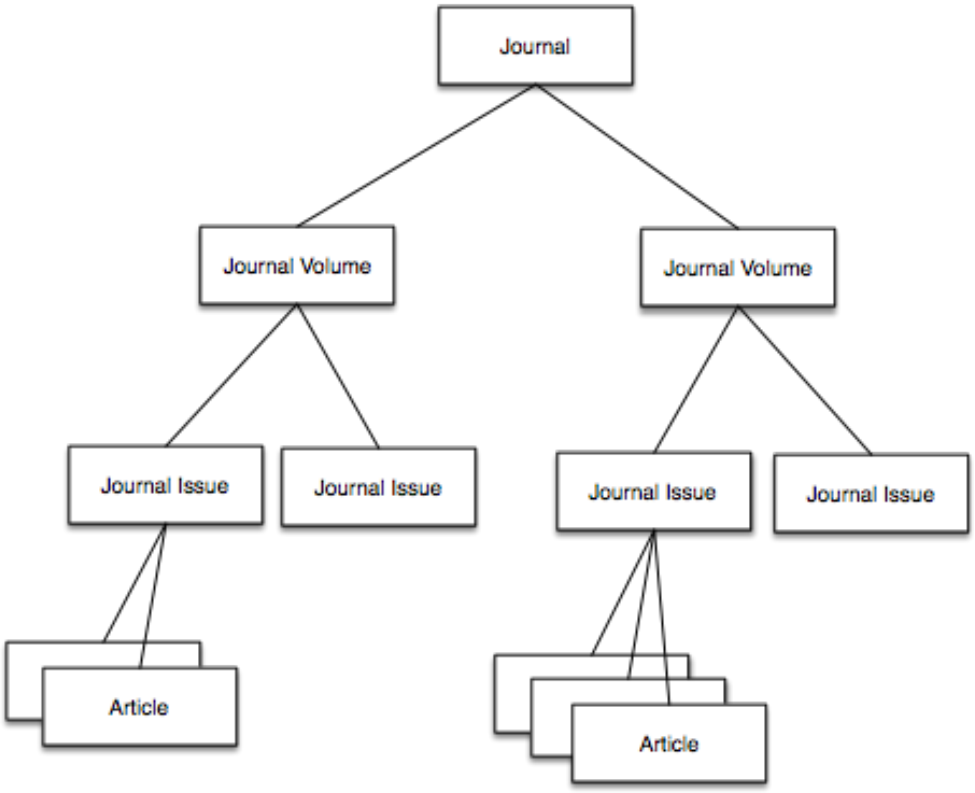
Entities Use Case: Compound Journal

This page describes an out-of-the-box Configurable Entities use case which is to represent a "Compound Journal" hierarchy.

Structure

The structure displays the types of entities, and the relations between them.

Each Journal consists of multiple volumes, and each volume contains multiple issues.



Data Model Storage

Storage in the database

The entity types represent the 3 blocks from the structure.

The labels displayed here will also be stored in the item metadata to easily identify the type of item.

```
select * from entity_type;

 id |     label
----+---------------
  1 | Publication
  5 | Journal
  6 | JournalVolume
  7 | JournalIssue
```

The relationship types are the lines connecting the blocks from the structure. The cardinality columns have been omitted in this document to reduce the size of the table. The labels displayed here will also be used for e.g. the DSpace CSV import to build relations and for displaying the relations in REST.

```
select id, left_type, right_type, left_label, right_label from relationship_type;

 id | left_type | right_type |       left_label            |      right_label
----+-----------+------------+-----------------------------+------------------------
  7 |         5 |          6 | isVolumeOfJournal           | isJournalOfVolume
  8 |         6 |          7 | isIssueOfJournalVolume      | isJournalVolumeOfIssue
  9 |         7 |          1 | isPublicationOfJournalIssue | isJournalIssueOfPublication
```

Data Model Design in XML

The XML file designed to create the rows in the database mentioned above.

```
<?xml version="1.0" encoding="UTF-8"?>

<relationships>
    <type>
        <leftType>Journal</leftType>
        <rightType>JournalVolume</rightType>
        <leftLabel>isVolumeOfJournal</leftLabel>
        <rightLabel>isJournalOfVolume</rightLabel>
        <leftCardinality>
            <min>0</min>
            <!--<max></max> not specified, unlimited-->
        </leftCardinality>
        <rightCardinality>
            <min>1</min> <!--each volume should have one journal-->
            <max>1</max> <!--each volume should have one journal-->
        </rightCardinality>
    </type>
    <type>
        <leftType>JournalVolume</leftType>
        <rightType>JournalIssue</rightType>
        <leftLabel>isIssueOfJournalVolume</leftLabel>
        <rightLabel>isJournalVolumeOfIssue</rightLabel>
        <leftCardinality>
            <min>0</min>
        </leftCardinality>
        <rightCardinality>
            <min>1</min> <!--each issue should have one volume—>
            <max>1</max> <!--each volume should have one journal-->
        </rightCardinality>
    </type>
    <type>
        <leftType>JournalIssue</leftType>
        <rightType>Publication</rightType>
        <leftLabel>isPublicationOfJournalIssue</leftLabel>
        <rightLabel>isJournalIssueOfPublication</rightLabel>
        <leftCardinality>
            <min>0</min>
        </leftCardinality>
        <rightCardinality>
            <min>0</min>
            <max>1</max>
        </rightCardinality>
    </type>
</relationships>
```

Importing this XML using the command:

[install_dir]/bin/dspace dsrun org.dspace.app.util.InitializeEntities -f [install_dir]/config/relationship-types.xml

creates the database tables.

Relation Storage

Storage in the database

The relations are the actual items which are connected, and contain a foreign key to both items, and to the relationship type. The place column has been omitted to reduce the size of the table. The item IDs have been reduced in size to reduce the size of the table.

The table below indicates in the first row that:

the item (a journal)

https://demo.dspace.org/entities/journal/d4af6c3e-53d0-4757-81eb-566f3b45d63a

is related to the item (a journal volume)

https://demo.dspace.org/entities/journalvolume/07c6249f-4bf7-494d-9ce3-6ffdb2aed538

via the relation `isVolumeOfJournal`.

The row ID 60 states the item (a journal volume)

https://demo.dspace.org/entities/journalvolume/07c6249f-4bf7-494d-9ce3-6ffdb2aed538

is related to the item (a journal issue)

https://demo.dspace.org/entities/journalissue/44c29473-5de2-48fa-b005-e5029aa1a50b

via the relation `isIssueOfJournalVolume`.

```
select id, left_id, type_id, right_id from relationship;

 id |               left_id           | type_id |               right_id
----+----------------------------------+---------+--------------------------------
 56 | d4af6c3e-53d0-81eb-566f3b45d63a |       7 | 07c6249f-4bf7-9ce3-6ffdb2aed538
 57 | d4af6c3e-53d0-81eb-566f3b45d63a |       7 | 66bb4e5d-b419-a648-f270a527f17c
 58 | a23eae5a-7857-8e52-989436ad2955 |       7 | f9b89a11-b44e-a3b4-ab24a33553c7
 59 | a23eae5a-7857-8e52-989436ad2955 |       7 | 343d3263-2733-9dc4-216a01b4a461
 60 | 07c6249f-4bf7-9ce3-6ffdb2aed538 |       8 | 44c29473-5de2-b005-e5029aa1a50b
 61 | 07c6249f-4bf7-9ce3-6ffdb2aed538 |       8 | c3076837-e5df-80bc-2661cd390a7b
 62 | 66bb4e5d-b419-a648-f270a527f17c |       8 | a4a63ab5-8c0b-b5f7-5b5d9828cb69
 63 | f9b89a11-b44e-a3b4-ab24a33553c7 |       8 | 77877343-3f75-9492-6ed7c98ed84e
 64 | f9b89a11-b44e-a3b4-ab24a33553c7 |       8 | f4dcd8a6-4cc4-8bb9-a7e8202e05b0
 65 | 343d3263-2733-9dc4-216a01b4a461 |       8 | b7003f66-80e9-99a2-3695e8150b80
 66 | 343d3263-2733-9dc4-216a01b4a461 |       8 | db55298c-a21f-8793-a21f1194a226
141 | 44c29473-5de2-b005-e5029aa1a50b |       9 | e98b0f27-5c19-960d-eb6ad5287067
142 | c3076837-e5df-80bc-2661cd390a7b |       9 | 96715576-3748-ad45-001646632963
143 | a4a63ab5-8c0b-b5f7-5b5d9828cb69 |       9 | 047556d1-3d01-bc68-0cee7ad7ed4e
```

Virtual Metadata

For Journal Volumes, it's relevant to include the ISSN and journal name of the parent journal in the metadata. A sample of this functionality can be seen at e.g. https://demo.dspace.org/server/#https://demo.dspace.org/server/api/core/items/66bb4e5d-b419-42b7-a648-f270a527f17c where `journal.identifier.issn` and `journal.title` is populated with the metadata of the parent journal

For Journal Issues, it's relevant to include the volume number of the parent journal volume in the metadata. A sample of this functionality can be seen at e.g. https://demo.dspace.org/server/#https://demo.dspace.org/server/api/core/items/a4a63ab5-8c0b-4456-b5f7-5b5d9828cb69 where `journalvolume.identifier.volume` is populated with the metadata of the parent journal

Since this is virtual metadata, the metadata is not duplicated, but only displayed.

The configuration of this virtual metadata can be found at https://github.com/DSpace/DSpace/blob/main/dspace/config/spring/api/virtual-metadata.xml

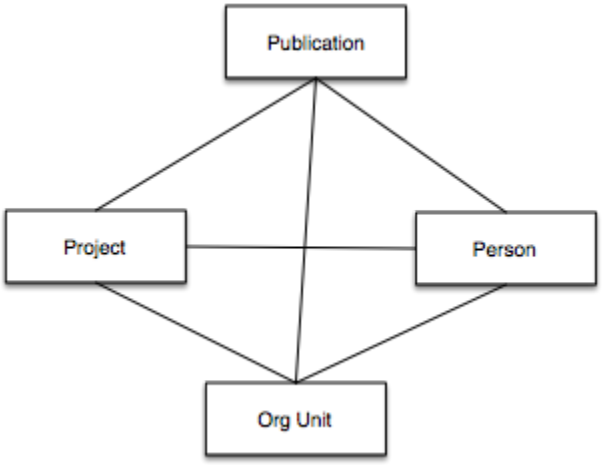
Entities Use Case: Research Management

This page describes an out-of-the-box Configurable Entities use case which is to represent research data similar to a "Current Research Information System" (CRIS) or "Research Information Management System" (RIMS).

Structure

The structure displays the types of entities, and the relations between them.

- Each publication can link to projects, people and org units
- Each person can link to projects, publications and org units
- Each project can link to publications, people and org units
- Each org units can link to projects, people and publications



Model storage

Storage in the database

The entity types represent the 4 blocks from the structure.

The labels displayed here will also be stored in the item metadata to easily identify the type of item.

```
select * from entity_type;

 id |     label
----+---------------
  1 | Publication
  2 | Person
  3 | Project
  4 | OrgUnit
```

The relationship types are the lines connecting the blocks from the structure. The cardinality columns have been omitted in this document to reduce the size of the table. The labels displayed here will also be used for e.g. the DSpace CSV import to build relations and for displaying the relations in REST.

```
select id, left_type, right_type, left_label, right_label from relationship_type;

 id | left_type | right_type |       left_label       |      right_label
----+-----------+------------+------------------------+------------------------
  1 |         1 |          2 | isAuthorOfPublication  | isPublicationOfAuthor
  2 |         1 |          3 | isProjectOfPublication | isPublicationOfProject
  3 |         1 |          4 | isOrgUnitOfPublication | isPublicationOfOrgUnit
  4 |         2 |          3 | isProjectOfPerson      | isPersonOfProject
  5 |         2 |          4 | isOrgUnitOfPerson      | isPersonOfOrgUnit
  6 |         3 |          4 | isOrgUnitOfProject     | isProjectOfOrgUnit
```

Data Model Design in XML

The XML file designed to create the rows in the database mentioned above

```
<relationships>
    <type>
        <leftType>Publication</leftType>
        <rightType>Person</rightType>
        <leftLabel>isAuthorOfPublication</leftLabel>
        <rightLabel>isPublicationOfAuthor</rightLabel>
        <leftCardinality>
            <min>0</min>
            <!--<max></max> not specified, unlimited-->
        </leftCardinality>
        <rightCardinality>
            <min>0</min>
            <!--<max></max> not specified, unlimited-->
        </rightCardinality>
    </type>
    <type>
        <leftType>Publication</leftType>
        <rightType>Project</rightType>
        <leftLabel>isProjectOfPublication</leftLabel>
        <rightLabel>isPublicationOfProject</rightLabel>
        <leftCardinality>
            <min>0</min>
        </leftCardinality>
        <rightCardinality>
            <min>0</min>
        </rightCardinality>
    </type>
    <type>
        <leftType>Publication</leftType>
        <rightType>OrgUnit</rightType>
        <leftLabel>isOrgUnitOfPublication</leftLabel>
        <rightLabel>isPublicationOfOrgUnit</rightLabel>
        <leftCardinality>
            <min>0</min>
        </leftCardinality>
        <rightCardinality>
            <min>0</min>
        </rightCardinality>
    </type>
    <type>
        <leftType>Person</leftType>
        <rightType>Project</rightType>
        <leftLabel>isProjectOfPerson</leftLabel>
        <rightLabel>isPersonOfProject</rightLabel>
        <leftCardinality>
            <min>0</min>
        </leftCardinality>
        <rightCardinality>
            <min>0</min>
        </rightCardinality>
    </type>
    <type>
        <leftType>Person</leftType>
        <rightType>OrgUnit</rightType>
        <leftLabel>isOrgUnitOfPerson</leftLabel>
        <rightLabel>isPersonOfOrgUnit</rightLabel>
        <leftCardinality>
            <min>0</min>
        </leftCardinality>
        <rightCardinality>
            <min>0</min>
        </rightCardinality>
    </type>
    <type>
        <leftType>Project</leftType>
        <rightType>OrgUnit</rightType>
        <leftLabel>isOrgUnitOfProject</leftLabel>
        <rightLabel>isProjectOfOrgUnit</rightLabel>
```

```
        <leftCardinality>
            <min>0</min>
        </leftCardinality>
        <rightCardinality>
            <min>0</min>
        </rightCardinality>
    </type>
</relationships>
```

Importing this file created the database tables

Relation Storage

Storage in the database

The relations are the actual items which are connected, and contain a foreign key to both items, and to the relationship type. The place column has been omitted to reduce the size of the table. The item IDs have been reduced in size to reduce the size of the table.

The table below indicates

in the first row the item (a publication)

https://demo.dspace.org/entities/publication/e98b0f27-5c19-49a0-960d-eb6ad5287067

is related to the item (a person)

https://demo.dspace.org/entities/person/0ffbee3f-e7ea-42bc-92fe-2fbef1a52c0f

via the relation `isAuthorOfPublication`.

```
select id, left_id, type_id, right_id from relationship;

 id |               left_id           | type_id |               right_id
----+----------------------------------+---------+--------------------------------
 52 | e98b0f27-5c19-960d-eb6ad5287067 |       1 | 0ffbee3f-e7ea-92fe-2fbef1a52c0f
 53 | e98b0f27-5c19-960d-eb6ad5287067 |       1 | 5a3f7c7a-d3df-b8a2-f00ede62c60a
 66 | 72635f7f-37b5-b4f2-5ff45d97a09b |       1 | 0ffbee3f-e7ea-92fe-2fbef1a52c0f
 67 | 674f695e-8001-8f9c-095c536a6bcb |       1 | 5a3f7c7a-d3df-b8a2-f00ede62c60a
 68 | a64719f8-ba7b-8eb6-f8feb0c000b7 |       1 | f2235aa6-6fe7-a690-598b72dd8e44
 69 | a64719f8-ba7b-8eb6-f8feb0c000b7 |       1 | 0ffbee3f-e7ea-92fe-2fbef1a52c0f
 70 | e98b0f27-5c19-960d-eb6ad5287067 |       2 | 0de99067-c898-a82c-9555f3311288
 71 | e7bd0d24-e83a-bc0c-8aaaeb19dc7d |       2 | 0de99067-c898-a82c-9555f3311288
 76 | 2f4ec582-109e-a94a-b7d7615a8c69 |       2 | 18e7924c-f15b-9fe3-3de370bccc97
 77 | 99c2e55c-6326-9f36-fcac333b0e8c |       2 | 18e7924c-f15b-9fe3-3de370bccc97
 78 | 96715576-3748-ad45-001646632963 |       2 | 18e7924c-f15b-9fe3-3de370bccc97
 79 | e98b0f27-5c19-960d-eb6ad5287067 |       3 | d30de96b-1e76-8ef9-ab426b6f9763
 80 | e98b0f27-5c19-960d-eb6ad5287067 |       3 | 506a7e54-8d7c-8636-d5f6411483de
 81 | e98b0f27-5c19-960d-eb6ad5287067 |       3 | c216201f-ed10-b0e0-5a065405bd3e
 82 | 96715576-3748-ad45-001646632963 |       3 | d30de96b-1e76-8ef9-ab426b6f9763
 95 | a64719f8-ba7b-8eb6-f8feb0c000b7 |       3 | c216201f-ed10-b0e0-5a065405bd3e
 96 | a64719f8-ba7b-8eb6-f8feb0c000b7 |       3 | d30de96b-1e76-8ef9-ab426b6f9763
 97 | 0de99067-c898-a82c-9555f3311288 |       6 | d30de96b-1e76-8ef9-ab426b6f9763
```

Virtual Metadata

For publications, it's relevant to include the names of the related authors in the metadata. A sample of this functionality can be seen at e.g. https://demo.dspace.org/server/#https://demo.dspace.org/server/api/core/items/e98b0f27-5c19-49a0-960d-eb6ad5287067 where `dc.contributor.author` is populated with the name of the related author. `dc.contributor.other` is also populated with the name of the Org Unit here

For a person, it's relevant to include the names of the org units in the metadata. A sample of this functionality can be seen at e.g. https://demo.dspace.org/server/#https://demo.dspace.org/server/api/core/items/f2235aa6-6fe7-4174-a690-598b72dd8e44 where `person.contributor.other` is populated with the name of the related Org Unit

Since this is virtual metadata, the metadata is not duplicated

The configuration of this virtual metadata can be found at https://github.com/DSpace/DSpace/blob/main/dspace/config/spring/api/virtual-metadata.xml

Curation System

DSpace supports running curation tasks, which are described in this section. DSpace includes several useful tasks out-of-the-box, but the system also is designed to allow new tasks to be added between releases, both general purpose tasks that come from the community, and locally written and deployed tasks.

1 Tasks
2 Activation
3 Task Invocation
    3.1 On the command line
    3.2 In the admin UI
    3.3 Protecting reporter and task file directories
    3.4 In workflow
    3.5 In arbitrary user code
4 Asynchronous (Deferred) Operation
5 Task Output and Reporting
    5.1 Status Code
    5.2 Result String
    5.3 Reporting Stream
6 Relative Path Calcuation
7 Task Properties
8 Task Parameters
9 Scripted Tasks

Tasks

The goal of the curation system ("CS") is to provide a simple, extensible way to manage routine content operations on a repository. These operations are known to CS as "tasks", and they can operate on any DSpaceObject (i.e. subclasses of DSpaceObject) - which means the entire Site, Communities, Collections, and Items - viz. core data model objects. Tasks may elect to work on only one type of DSpace object - typically an Item - and in this case they may simply ignore other data types (tasks have the ability to "skip" objects for any reason). The DSpace core distribution will provide a number of useful tasks, but the system is designed to encourage local extension - tasks can be written for any purpose, and placed in any java package. This gives DSpace sites the ability to customize the behavior of their repository without having to alter - and therefore manage synchronization with - the DSpace source code. What sorts of activities are appropriate for tasks?

Some examples:

- apply a virus scan to item bitstreams (this will be our example below)
- profile a collection based on format types - good for identifying format migrations
- ensure a given set of metadata fields are present in every item, or even that they have particular values
- call a network service to enhance/replace/normalize an item's metadata or content
- ensure all item bitstreams are readable and their checksums agree with the ingest values

Since tasks have access to, and can modify, DSpace content, performing tasks is considered an administrative function to be available only to knowledgeable collection editors, repository administrators, sysadmins, etc. No tasks are exposed in the public interfaces.

Activation

For CS to run a task, the code for the task must of course be included with other deployed code (to `[dspace]/lib`, WAR, etc) but it must also be declared and given a name. This is done via a configuration property in `[dspace]/config/modules/curate.cfg` as follows:

```
### Task Class implementations
plugin.named.org.dspace.curate.CurationTask = org.dspace.ctask.general.NoOpCurationTask = noop
plugin.named.org.dspace.curate.CurationTask = org.dspace.ctask.general.ProfileFormats = profileformats
plugin.named.org.dspace.curate.CurationTask = org.dspace.ctask.general.RequiredMetadata = requiredmetadata
plugin.named.org.dspace.curate.CurationTask = org.dspace.ctask.general.ClamScan = vscan
plugin.named.org.dspace.curate.CurationTask = org.dspace.ctask.general.MicrosoftTranslator = translate
plugin.named.org.dspace.curate.CurationTask = org.dspace.ctask.general.MetadataValueLinkChecker = checklinks
```

For each activated task, a key-value pair is added. The key is the fully qualified class name and the value is the *taskname* used elsewhere to configure the use of the task, as will be seen below. Note that the curate.cfg configuration file, while in the config directory, is located under "modules". The intent is that tasks, as well as any configuration they require, will be optional "add-ons" to the basic system configuration. Adding or removing tasks has no impact on dspace.cfg.

For many tasks, this activation configuration is all that will be required to use it. But for others, the task needs specific configuration itself. A concrete example is described below, but note that these task-specific configuration property files also reside in `[dspace]/config/modules`

Task Invocation

Tasks are invoked using CS framework classes that manage a few details (to be described below), and this invocation can occur wherever needed, but CS offers great versatility "out of the box":

On the command line

A simple tool "CurationCli" provides access to CS via the command line. This tool bears the name "curate" in the DSpace launcher. For example, to perform a virus check on collection "4":

```
[dspace]/bin/dspace curate -t vscan -i 123456789/4
```

The complete list of options:

option	meaning
-t taskname	name of task to perform.
-T filename	name of file containing a list of tasknames to be performed. This option is available only via the command line.
-e epersonID	(required) email address or netid of the E-Person performing the task. This option is available only via the command line.
-i identifier	ID of object to curate. May be (1) a Handle, (2) a workflow ID, or (3) 'all' to operate on the whole repository.
-q queue	name of queue to process. -i and -q are mutually exclusive.
-l limit	maximum number of objects in Context cache. If absent, unlimited objects may be added.
-s scope	declare a scope for database transactions. Scope must be: (1) 'open' (default value), (2) 'curation' or (3) 'object'.
-v	emit verbose output
-r filename	emit reporting to the named file. '-r -' writes reporting to standard out. If not specified, report is discarded silently. This option is available only via the command line.
-p name=value	set a runtime task parameter `name` to the value `value`. May be repeated as needed. See "Task parameters" below.

As with other command-line tools, these invocations could be placed in a cron table and run on a fixed schedule, or run on demand by an administrator.

In the admin UI

In the UI, there are several ways to execute configured Curation Tasks:

1. **From the "Curate" tab/button that appears on each "Edit Community/Collection/Item" page:** this tab allows an Administrator, Community Administrator or Collection Administrator to run a Curation Task on that particular Community, Collection or Item. When running a task on a Community or Collection, that task will also execute on all its child objects, unless the Task itself states otherwise (e.g. running a task on a Collection will also run it across all Items within that Collection).
    - NOTE: Community Administrators and Collection Administrators can only run Curation Tasks on the Community or Collection which they administer, along with any child objects of that Community or Collection. For example, a Collection Administrator can run a task on that specific Collection, or on any of the Items within that Collection.
2. **From the Administrator's "Curation Tasks" page:** This option is only available to DSpace Administrators, and appears in the Administrative side-menu. This page allows an Administrator to run a Curation Task across a single object, or all objects within the entire DSpace site.
    - In order to run a task from this interface, you must enter in the handle for the DSpace object. To run a task site-wide, you can use the handle: `[your-handle-prefix]/0`

Each of the above pages exposes a drop-down list of configured tasks, with a button to 'perform' the task, or queue it for later operation (see section below). Not all activated tasks need appear in the Curate tab - you filter them by means of a configuration property. This property also permits you to assign to the task a more user-friendly name than the PluginManager *taskname*. The property resides in `[dspace]/config/modules/curate.cfg`:

```
curate.ui.tasknames = profileformats = Profile Bitstream Formats
curate.ui.tasknames = requiredmetadata = Check for Required Metadata
```

When a task is selected from the drop-down list and performed, the tab displays both a phrase interpreting the "status code" of the task execution, and the "result" message if any has been defined. When the task has been queued, an acknowledgement appears instead. You may configure the words used for status codes in curate.cfg (for clarity, language localization, etc):

```
curate.ui.statusmessages = -3 = Unknown Task
curate.ui.statusmessages = -2 = No Status Set
curate.ui.statusmessages = -1 = Error
curate.ui.statusmessages = 0 = Success
curate.ui.statusmessages = 1 = Fail
curate.ui.statusmessages = 2 = Skip
curate.ui.statusmessages = other = Invalid Status
```

Report output from tasks run in this way is collected by configuring a Reporter plugin. You must have exactly one Reporter configured. The default is to use the FileReporter, which writes a single report of the output of all tasks in the run over all of the selected objects, to a file in the reports directory (configured as report.dir). See `[DSpace]/config/modules/submission-configuration.cfg` for the value of `plugin.single.org.dspace.curate.Reporter`. Other Reporter implementations are provided, or you may supply your own.

As the number of tasks configured for a system grows, a simple drop-down list of **all** tasks may become too cluttered or large. DSpace 1.8+ provides a way to address this issue, known as *task groups*. A task group is a simple collection of tasks that the Admin UI will display in a separate drop-down list. You may define as many or as few groups as you please. If no groups are defined, then all tasks that are listed in the *ui.tasknames* property will appear in a single drop-down list. If at least *one* group is defined, then the admin UI will display **two** drop-down lists. The first is the list of task groups, and the second is the list of task names associated with the selected group. A few key points to keep in mind when setting up task groups:

- a task can appear in more than one group if desired
- tasks that belong to no group are *invisible* to the admin UI (but of course available in other contexts of use)

The configuration of groups follows the same simple pattern as tasks, using properties in `[dspace]/config/modules/curate.cfg`. The group is assigned a simple logical name, but also a localizable name that appears in the UI. For example:

```
# ui.taskgroups contains the list of defined groups, together with a pretty name for UI display
curate.ui.taskgroups = replication = Backup and Restoration Tasks
curate.ui.taskgroups = integrity = Metadata Integrity Tasks
.....
# each group membership list is a separate property, whose value is comma-separated list of logical task names
curate.ui.taskgroup.integrity = profileformats, requiredmetadata
....
```

Protecting reporter and task file directories

As at DSpace 7.6.7, directories for reading task files and writing reporter output are enforced. If you wish to write curation reports to locations outside the defaults listed, you may modify or add to this configuration

```
# allowed base path(s) of curation task files
# it is recommended to restrict this path as much as possible
# so that the DSpace Processes framework may only load files as "tasks"
# from a trusted location. For multiple paths, repeat this configuration
# property for each trusted path
# Default: ${dspace.dir}
#curate.taskfile.base = ${dspace.dir}

# allowed base path of reporter output.
# Default: ${dspace.dir}/log
#curate.reporter.base = ${dspace.dir}/log
```

In workflow

CS provides the ability to attach any number of tasks to standard DSpace workflows. Using a configuration file `[dspace]/config/workflow-curation.xml`, you can declaratively (without coding) wire tasks to any step in a workflow. An example:

```
<taskset-map>
    <mapping collection-handle="default" taskset="cautious" />
</taskset-map>
<tasksets>
    <taskset name="cautious">
        <flowstep name="editstep">
            <task name="vscan">
                <workflow>reject</workflow>
                <notify on="fail">$flowgroup</notify>
                <notify on="fail">$colladmin</notify>
                <notify on="error">$siteadmin</notify>
            </task>
        </flowstep>
    </taskset>
</tasksets>
```

This markup would cause a virus scan to occur during the "editstep" of workflow for any collection, and automatically reject any submissions with infected files. It would further notify (via email) both the reviewers ("editstep" role/group), and the collection administrators, if either of these are defined. If it could not perform the scan, the site administrator would be notified.

The notifications use the same procedures that other workflow notifications do - namely email. There is a new email template defined for curation task use: `[dspace]/config/emails/flowtask_notify`. This may be language-localized or otherwise modified like any other email template.

Tasks wired in this way are normally performed *as soon as the workflow step is entered*, and the outcome action (defined by the 'workflow' element) immediately follows. It is also possible to delay the performance of the task - which will ensure a responsive system - by queuing the task instead of directly performing it:

```
...
    <taskset name="cautious">
        <flowstep name="editstep" queue="workflow">
...
```

This attribute (which must always follow the "name" attribute in the flowstep element), will cause all tasks associated with the step to be placed on the queue named "workflow" (or any queue you wish to use, of course), and further has the effect of **suspending** the workflow. When the queue is emptied (meaning all tasks in it performed), then the workflow is restarted. Each workflow step may be separately configured,

Like configurable submission, you can assign these task rules per collection, as well as having a default for any collection.

As with task invocation from the administrative UI, workflow tasks need to have a Reporter configured in `submission-configuration.cfg`.

In arbitrary user code

If these pre-defined ways are not sufficient, you can of course manage curation directly in your code. You would use the CS helper classes. For example:

```
Collection coll = (Collection)HandleManager.resolveToObject(context, "123456789/4");
Curator curator = new Curator();
curator.setReporter(System.out);
curator.addTask("vscan").curate(coll);
System.out.println("Result: " + curator.getResult("vscan"));
```

would do approximately what the command line invocation did. the method "curate" just performs all the tasks configured (you can add multiple tasks to a curator).

The above directs report output to standard out. Any class which implements Appendable may be set as the reporter class.

Asynchronous (Deferred) Operation

Because some tasks may consume a fair amount of time, it may not be desirable to run them in an interactive context. CS provides a simple API and means to defer task execution, by a queuing system. Thus, using the previous example:

```
Curator curator = new Curator();
curator.addTask("vscan").queue(context, "123456789/4", "monthly");
```

would place a request on a named queue "monthly" to virus scan the collection. To read (and process) the queue, we could for example:

```
[dspace]/bin/dspace curate -q monthly
```

use the command-line tool, but we could also read the queue programmatically. Any number of queues can be defined and used as needed.
In the administrative UI curation "widget", there is the ability to both perform a task, but also place it on a queue for later processing.

Task Output and Reporting

Few assumptions are made by CS about what the 'outcome' of a task may be (if any) - it. could e.g. produce a report to a temporary file, it could modify DSpace content silently, etc. But the CS runtime does provide a few pieces of information whenever a task is performed:

Status Code

This was mentioned above. This is returned to CS whenever a task is called. The complete list of values:

```
 -3 NOTASK  - CS could not find the requested task
 -2 UNSET   - task did not return a status code because it has not yet run
 -1 ERROR   - task could not be performed
  0 SUCCESS - task performed successfully
  1 FAIL    - task performed, but failed
  2 SKIP    - task not performed due to object not being eligible
```

In the administrative UI, this code is translated into the word or phrase configured by the *ui.statusmessages* property (discussed above) for display.

Result String

The task may define a string indicating details of the outcome. This result is displayed, in the "curation widget" described above:

```
"Virus 12312 detected on Bitstream 4 of 1234567789/3"
```

CS does not interpret or assign result strings, the task does it. A task may not assign a result, but the "best practice" for tasks is to assign one whenever possible.

Reporting Stream

For very fine-grained information, a task may write to a *reporting* stream. This stream may be sent to a file or to standard out, when running a task from the command line. Tasks run from the administrative UI or a workflow use a configured Reporter class to collect report output. Your own code may collect the report using any implementation of the Appendable interface. Unlike the result string, there is no limit to the amount of data that may be pushed to this stream.

Relative Path Calcuation

The utility that validates input and output for the Curation framework expects an absolute path. If a relative path is given to `-r` or `-T` , the Curation system will calculate an absolute path using the CLI user's current working directory as a root directory. This is then validated against the allowed base directories in `curate.taskfile.base` or `curate.reporter.base` . For the most predictable results, it is recommended to use absolute paths for these CLI paramters.

Task Properties

DSpace 1.8 introduces a new "idiom" for tasks that require configuration data. It is available to any task whose implementation extends `AbstractCuratio nTask`, but is completely optional. There are a number of problems that task properties are designed to solve, but to make the discussion concrete we will start with a particular one: the problem of hard-coded configuration file names. A task that relies on configuration data will typically encode a fixed reference to a configuration file name. For example, the virus scan task reads a file called "`clamav.cfg`", which lives in `[dspace]/config/modules`. It could look up its configuration properties in the ordinary way. But tasks are supposed to be written by anyone in the community and shared around (without prior coordination), so if another task uses the same configuration file name, there is a name **collision** here that can't be easily fixed, since the reference is hard-coded in each task. In this case, if we wanted to use both at a given site, we would have to alter the source of one of them - which introduces needless code localization and maintenance.

Task properties gives us a simple solution. Here is how it works: suppose that both colliding tasks instead use the task properties facility instead of ordinary configuration lookup. For example, each asks for the property `clamav.service.host`. At runtime, the curation system **resolves** this request to a set of configuration properties, and it uses the *name the task has been configured as* as the prefix of the properties. So, for example, if both were installed (in, say, `curate.cfg`) as:

```
org.dspace.ctask.general.ClamAv = vscan,
org.community.ctask.ConflictTask = virusscan,
....
```

then the task property `foo` will resolve to the property named `vscan.foo` when called from ClamAv task, but `virusscan.foo` when called from ConflictTask's code. Note that the "`vscan`" etc are locally assigned names, so we can always prevent the "collisions" mentioned, and we make the tasks much more portable, since we remove the "hard-coding" of config names.

Another use of task properties is to support multiple task profiles. Suppose we have a task that we want to operate in one of two modes. A good example would be a mediafilter task that produces a thumbnail. We can either create one if it doesn't exist, or run with "-force" which will create one regardless. Suppose this behavior was controlled by a property in a config file. If we configured the task as "thumbnail", then we would have in (perhaps) `[dspace] /config/modules/thumbnail.cfg`:

```
...other properties...
thumbnail.thumbnail.maxheight = 80
thumbnail.thumbnail.maxwidth = 80
thumbnail.forceupdate=false
```

The thumbnail generating task code would then resolve "forcedupdate" to see whether filtering should be forced.

But an obvious use-case would be to want to run force mode **and** non-force mode from the admin UI on different occasions. To do this, one would have to stop Tomcat, change the property value in the config file, and restart, etc However, we can use task properties to elegantly rescue us here. All we need to do is go into the config/modules directory, and create a new file perhaps called: `thumbnail.force.cfg`. In this file, we put the properties:

```
thumbnail.force.thumbnail.maxheight = 80
thumbnail.force.thumbnail.maxwidth = 80
thumbnail.force.forceupdate=true
```

Then we add a new task (really just a new name, no new code) in curate.cfg:

```
org.dspace.ctask.general.ThumbnailTask = thumbnail
org.dspace.ctask.general.ThumbnailTask = thumbnail.force
```

Consider what happens: when we perform the task "`thumbnail`" (using taskProperties), it uses the `thumbnail.*` properties and operates in "non-force" profile (since the value is false), but when we run the task "`thumbnail.force`" the curation system uses the `thumbnail.force.*` properties. Notice that we did all this via local configuration - we have not had to touch the source code at all to obtain as many "profiles" as we would like.

See Task Properties in Curation Tasks for details of how properties are resolved in task code.

Task Parameters

New in DSpace 7, you can pass parameters to a task at invocation time. These runtime parameters will be presented to the task as if they were task properties (see above) and, if present, will override the value of identically-named properties. Example:

Task parameters

```
bin/dspace curate -t reticulate -i 123456789/36 -p foreground=red -p background=green
```

Scripted Tasks

The procedure to set up curation tasks in Jython is described on a separate page: Curation tasks in Jython

DSpace 1.8 includes limited (and somewhat experimental) support for deploying and running tasks written in languages other than Java. Since version 6, Java has provided a standard way (API) to invoke so-called scripting or dynamic language code that runs on the java virtual machine (JVM). Scripted tasks are those written in a language accessible from this API. The exact number of supported languages will vary over time, and the degree of maturity of each language, or suitability of the language for curation tasks will also vary significantly. However, preliminary work indicates that Ruby (using the JRuby runtime) and Groovy may prove viable task languages.

Support for scripted tasks does **not** include any DSpace pre-installation of the scripting language itself - this must be done according to the instructions provided by the language maintainers, and typically only requires a few additional jars on the DSpace classpath. Once one or more languages have been installed into the DSpace deployment, task support is fairly straightforward. One new property must be defined in `[dspace]/config/modules/curate.cfg`:

```
curate.script.dir = ${dspace.dir}/scripts
```

This merely defines the directory location (usually relative to the deployment base) where task script files should be kept. This directory will contain a "catalog" of scripted tasks named `task.catalog` that contains information needed to run scripted tasks. Each task has a 'descriptor' property with value syntax:

`<engine>|<relFilePath>|<implClassCtor>`

An example property for a link checking task written in Ruby might be:

```
linkchecker = ruby|rubytask.rb|LinkChecker.new
```

This descriptor means that a "`ruby`" script engine will be created, a script file named "`rubytask.rb`" in the directory `<script.dir>` will be loaded and the resolver will expect an evaluation of "`LinkChecker.new`" will provide a correct implementation object. Note that the task must be configured in all other ways just like java tasks (in `ui.tasknames, ui.taskgroups`, etc).

Script files may embed their descriptors to facilitate deployment. To accomplish this, a script must include the descriptor string with syntax: `$td=<descriptor>` somewhere on a comment line. For example:

```
# My descriptor $td=ruby|rubytask.rb|LinkChecker.new
```

For reasons of portability, the <relFilePath> component may be omitted in this context. Thus, "`$td=ruby||LinkChecker.new`" will be expanded to a descriptor with the name of the embedding file.

Bundled Tasks

DSpace bundles a small number of tasks of general applicability. Those that do not require configuration (or have usable default values) are activated by default to demonstrate the use of the curation system. They may be deactivated by means of configuration, if desired, without affecting system integrity. Those that require configuration may be enabled (activated) by means editing DSpace configuration files. Each task is briefly described in this section.

All bundled tasks are in the package `org.dspace.ctask.general`. So, for example, to activate the no-operation task, which is implemented in the class `NoOpCurationTask`, one would configure:

```
plugin.named.org.dspace.curate.CurationTask = org.dspace.ctask.general.NoOpCurationTask = noop
```

Bitstream Format Profiler Task

The task with the taskname 'formatprofiler' (in the admin UI it is labeled "Profile Bitstream Formats") examines all the bitstreams in an item and produces a table ("profile") which is assigned to the result string. It is activated by default, and is configured to display in the administrative UI. The result string has the layout:

```
10 (K) Portable Network Graphics
5  (S) Plain Text
```

where the left column is the count of bitstreams of the named format and the letter in parentheses is an abbreviation of the repository-assigned support level for that format:

```
U  Unsupported
K  Known
S  Supported
```

The profiler will operate on any DSpace object. If the object is an item, then only that item's bitstreams are profiled; if a collection, all the bitstreams of all the items; if a community, all the items of all the collections of the community.

Link Checker Tasks

Two link checker tasks, BasicLinkChecker and MetadataValueLinkChecker, can be used to check for broken or unresolvable links appearing in item metadata.

This task is intended as a prototype / example for developers and administrators who are new to the curation system.

These tasks are not configurable.

Basic Link Checker

BasicLinkChecker iterates over all metadata fields ending in "uri" (eg. dc.relation.uri, dc.identifier.uri, dc.source.uri ...), attempts a GET to the value of the field, and checks for a 200 OK response.
Results are reported in a simple "one row per link" format.

Metadata Value Link Checker

MetadataValueLinkChecker parses all metadata fields for valid HTTP URLs, attempts a GET to those URLs, and checks for a 200 OK response.
Results are reported in a simple "one row per link" format.

MetadataWebService Task

DSpace item metadata can contain any number of identifiers or other field values that participate in networked information systems. For example, an item may include a DOI which is a controlled identifier in the DOI registry. Many web services exist to leverage these values, by using them as 'keys' to retrieve other useful data. In the DOI case for example, CrossRef provides many services that given a DOI will return author lists, citations, etc. The MetadataWebService task enables the use of such services, and allows you to obtain and (optionally) add to DSpace metadata the results of any web service call to any service provider. You simply need to describe what service you want to call, and what to do with the results. Using the task code (`[task code]`), you can create as many distinct tasks as you have services you want to call.

Each task description lives in a configuration file in 'config/modules' (or in your local.cfg), and is a simple properties file, like all other DSpace configuration files (see Configuration Reference). All of the settings associated with a given task should be prepended with the task name (as assigned in `config/modules/curate.cfg`). For example, if the task name is `issn2pubname` in curate.cfg, then all settings should start with "`issn2pubname.`" Your settings can either be set in your `local.cfg` , or in a new configuration file which is included (`include = path/to/new/file.cfg`) into either your local.cfg or the dspace.cfg. See the Configuration Reference for examples of including configuration files, or modifying your `local.cfg`

There are a few required properties you must configure for any service, and for certain services, a few additional ones. An example will illustrate best.

ISSN to Publisher Name

Suppose items (holding journal articles) include 'dc.identifier.issn' when available. We might also want to catalog the publisher name (in 'dc.publisher'). The cataloger could look up the name given the ISSN in various sources, but this 'research' is tedious, costly and error-prone. There are many good quality, free web services that can furnish this information. So we will configure a MetadataWebService task to call a service, and then automatically assign the publisher name to the item metadata. As noted above, all that is needed is a description of the service, and what to do with the results. Create a new file in 'config/modules' called 'issn2pubname.cfg' (or whatever is mnemonically useful to you). The first property in this file describes the service in a 'template'. The template is just the URL to call the web service, with parameters to substitute values in. Here we will use the 'Sherpa/Romeo' service:

```
[taskcode].template=http://www.sherpa.ac.uk/romeo/api29.php?issn={dc.identifier.issn}
```

When the task runs, it will replace '{dc.identifier.issn}' with the value of that field in the item, If the field has multiple values, the first one will be used. As a web service, the call to the above URL will return an XML document containing information (including the publisher name) about that ISSN. We need to describe what to do with this response document, i.e. what elements we want to extract, and what to do with the extracted content. This description is encoded in a property called the 'datamap'. Using the example service above we might have:

```
[taskcode].datamap=//publisher/name=>dc.publisher,//romeocolor
```

Each separate instruction is separated by a comma, so there are 2 instructions in this map. The first instruction essentially says: find the XML element 'publisher name' and assign the value or values of this element to the 'dc.publisher' field of the item. The second instruction says: find the XML element 'romeocolor', but do not add it to the DSpace item metadata - simply add it to the task result string (so that it can be seen by the person running the task). You can have as many instructions as you like in a datamap, which means that you can retrieve multiple values from a single web service call. A little more formally, each instruction consists of one to three parts. The first (mandatory) part identifies the desired data in the response document. The syntax (here '//publisher/name') is an XPath 1.0 expression, which is the standard language for navigating XML trees. If the value is to be assigned to the DSpace item metadata, then 2 other parts are needed. The first is the 'mapping symbol' (here '=>'), which is used to determine how the assignment should be made. There are 3 possible mapping symbols, shown here with their meanings:

```
'->' mapping will add to any existing value(s) in the item field
'=>' mapping will replace any existing value(s) in the item field
'~>' mapping will add *only if* item field has no existing value(s)
```

The third part (here 'dc.publisher') is simply the name of the metadata field to be updated. These two mandatory properties (template and datamap) are sufficient to describe a large number of web services. All that is required to enable this task is to edit '`config/modules/curate.cfg`' (or your `local.cfg`), and add '`issn2pubname`' to the list of tasks:

```
plugin.named.org.dspace.curate.CurationTask = org.dspace.ctask.general.MetadataWebService = issn2pubname
plugin.named.org.dspace.curate.CurationTask = org.dspace.ctask.general.MetadataWebService = doi2crossref
```

If you wish the task to be available in the Admin UI, see the Invocation from the Admin UI documentation (above) about how to configure it. The remaining sections describe some more specialized needs using the MetadataWebService task.

HTTP Headers

For some web services, protocol and other information is expressed not in the service URL, but in HTTP headers. Examples might be HTTP basic auth tokens, or requests for a particular media type response. In these cases, simply add a property to the configuration file (our example was 'issn2pubname.cfg') containing all headers you wish to transmit to the service:

```
[taskcode].headers=Accept: application/xml||Cache-Control: no-cache
```

You can specify any number of headers, just separate them with a 'double-pipe' ('||'). Ensure that any commas in values are escaped (with backslash comma, i.e. '\,').

Transformations

One potential problem with the simple parameter substitutions performed by the task is that the service might expect a different format or expression of a value than the way it is stored in the item metadata. For example, a DOI service might expect a bare prefix/suffix notation ('10.000/12345'), whereas the DSpace metadata field might have a URI representation ('http://dx.doi.org/10.000/12345'). In these cases one can declare a 'transformation' of a value in the template. For example:

```
[taskcode].template=http://www.crossref.org/openurl/?id={doi:dc.relation.isversionof}&format=unixref
```

The 'doi:' prepended to the metadata field name declares that the value of the 'dc.relation.isversionof' field should be *transformed* before the substitution into the template using a transformation named 'doi'. The transformation is itself defined in the same configuration file as follows:

```
[taskcode].transform.doi=match 10. trunc 60
```

This would be read as: exclude the value string up to the occurrence of '10.', then truncate any characters after length 60. You may define as many transformations as you want in any task, although generally 1 or 2 will suffice. They keywords 'match', 'trunc', etc are names of 'functions' to be applied (in the order entered). The currently available functions are:

```
'cut' <number> = remove number leading characters
'trunc' <number> = remove trailing characters after number length
'match' <pattern> = start match at pattern
'text' <characters> = append literal characters (enclose in ' ' when whitespace needed)
```

When the task is run, if the transformation results in an invalid state (e.g. cutting more characters than there are in the value), the un-transformed value will be used and the condition will be logged. Transformations may also be applied to values returned from the web service. That is, one can apply the transformation to a value before assigning it to a metadata field. In this case, the declaration occurs in the datamap property, not the template:

```
[taskcode].datamap=//publisher/name=>shorten:dc.publisher,//romeocolor
```

Here the task will apply the 'shorten' transformation (which must be defined in the same config file) before assigning the value to 'dc.publisher'.

Result String Programatic Use

Normally a task result string appears in a window in the admin UI after it has been invoked. The MedataWebService task will concatenate all the values declared in the 'datamap' property and place them in the result string using the format: 'name:value name:value' for as many values as declared. In the example above we would get a string like 'publisher: Nature romeocolor: green'. This format is fine for simple display purposes, but can be tricky if the values contain spaces. You can override the space separator using an optional property 'separator' (put in the config file, with all other properties). If you use:

```
[taskcode].separator=||
```

for example, it becomes easy to parse the result string and preserve spaces in the values. This use of the result string can be very powerful, since you are essentially creating a map of returned values, which can then be used to populate a user interface, or any other way you wish to exploit the data (drive a workflow, etc).

Limits and Use

A few limitations should be noted. First, since the response parsing utilizes XPath, the service can only operate on XML, (not JSON) response documents. Most web services can provide either, so this should not be a major obstacle. The MetadataWebService can be used in many ways: showing an admin a value in the result string in a UI, run in a batch to update a set of items, etc. One excellent configuration is to wire these tasks into submission workflow, so that 'automatic cataloging' of many fields can be performed on ingest.

MicrosoftTranslator Task

Microsoft Translator uses the Microsoft Translate API to translate metadata values from one source language into one or more target languages.

This task cab be configured to process particular fields, and use a default language if no authoritative language for an item can be found. Bing API v2 key is needed.

MicrosoftTranslator extends the more generic AbstractTranslator. This now seems wasteful, but a GoogleTranslator had also been written to extend AbstractTranslator. Unfortunately, Google has announced they are decommissioning free Translate API service, so this task hasn't been included in DSpace's general set of curation tasks.

Translated fields are added in addition to any existing fields, with the target language code in the 'language' column. This means that running a task multiple times over one item with the same configuration could result in duplicate metadata.

This task is intended as a prototype / example for developers and administrators who are new to the curation system.

Configure Microsoft Translator

An example configuration file can be found in `[dspace]/config/modules/translator.cfg`.

```
#---------------------------------------------------------------#
#----------TRANSLATOR CURATION TASK CONFIGURATIONS--------------#
#---------------------------------------------------------------#
# Configuration properties used solely by MicrosoftTranslator   #
# Curation Task (uses Microsoft Translation API v2)             #
#---------------------------------------------------------------#
## Translation field settings
##
## Authoritative language field
## This will be read to determine the original language an item was submitted in
## Default: dc.language
translator.field.language = dc.language

## Metadata fields you wish to have translated
translator.field.targets = dc.description.abstract, dc.title, dc.type

## Translation language settings
##
## If the language field configured in translate.field.language is not present
## in the record, set translate.language.default to a default source language
## or leave blank to use autodetection
translator.language.default = en

## Target languages for translation
translator.language.targets = de, fr

## Translation API settings
##
## Your Bing API v2 key and/or Google "Simple API Access" Key
## (note to Google users: your v1 API key will not work with Translate v2,
## you will need to visit https://code.google.com/apis/console and activate
## a Simple API Access key)
##
## You do not need to enter a key for both services.
translator.api.key.microsoft = YOUR_MICROSOFT_API_KEY_GOES_HERE
translator.api.key.google = YOUR_GOOGLE_API_KEY_GOES_HERE
```

NoOp Task

This task does absolutely nothing. It is intended as a starting point for developers and administrators wishing to learn more about the curation system.

Required Metadata Task

The "`requiredmetadata`" task examines item metadata and determines whether fields that the web submission (`input-forms.xml`) marks as required are present. It sets the result string to indicate either that all required fields are present, or constructs a list of metadata elements that are required but missing. When the task is performed on an item, it will display the result for that item. When performed on a collection or community, the task be performed on each item, and will display the *last* item result. If all items in the community or collection have all required fields, that will be the last in the collection. If the task fails for any item (i.e. the item lacks all required fields), the process is halted. This way the results for the 'failed' items are not lost.

Virus Scan Task

The "vscan" task performs a virus scan on the bitstreams of items using the ClamAV software product.

Clam AntiVirus is an open source (GPL) anti-virus toolkit for UNIX. A port for Windows is also available. The virus scanning curation task interacts with the ClamAV virus scanning service to scan the bitstreams contained in items, reporting on infection(s). Like other curation tasks, it can be run against a container or item, in the GUI or from the command line. It should be installed according to the documentation at http://www.clamav.net. It should not be installed in the dspace installation directory. You may install it on the same machine as your dspace installation, or on another machine which has been configured properly.

Setup the service from the ClamAV documentation.

This plugin requires a ClamAV daemon installed and configured for TCP sockets. Instructions for installing ClamAV (http://www.clamav.net/doc/latest/**clamdoc**.pdf)

NOTICE: The following directions assume there is a properly installed and configured clamav daemon. Refer to links above for more information about ClamAV.
The Clam anti-virus database must be updated regularly to maintain the most current level of anti-virus protection. Please refer to the ClamAV documentation for instructions about maintaining the anti-virus database.

DSpace Configuration

In `[dspace]/config/modules/curate.cfg`, activate the task:

- Add the plugin to the list of curation tasks.

```
### Task Class implementations
plugin.named.org.dspace.curate.CurationTask = org.dspace.ctask.general.NoOpCurationTask = noop
plugin.named.org.dspace.curate.CurationTask = org.dspace.ctask.general.ProfileFormats = profileformats
plugin.named.org.dspace.curate.CurationTask = org.dspace.ctask.general.RequiredMetadata = requiredmetadata
# This is the ClamAV scanner plugin
plugin.named.org.dspace.curate.CurationTask = org.dspace.ctask.general.ClamScan = vscan
plugin.named.org.dspace.curate.CurationTask = org.dspace.ctask.general.MicrosoftTranslator = translate
plugin.named.org.dspace.curate.CurationTask = org.dspace.ctask.general.MetadataValueLinkChecker = checklinks
```

- Optionally, add the vscan friendly name to the configuration to enable it in the administrative it in the administrative user interface.

```
curate.ui.tasknames = profileformats = Profile Bitstream Formats
curate.ui.tasknames = requiredmetadata = Check for Required Metadata
curate.ui.tasknames = checklinks = Check Links in Metadata
# Enable ClamAV from UI
curate.ui.tasknames = vscan = Virus Scan
```

- In `[dspace]/config/modules`, edit configuration file clamav.cfg:

```
clamav.service.host = 127.0.0.1
# Change if not running on the same host as your DSpace installation.
clamav.service.port = 3310
# Change if not using standard ClamAV port
clamav.socket.timeout = 120
# Change if longer timeout needed
clamav.scan.failfast = false
# Change only if items have large numbers of bitstreams
```

- Finally, if desired virus scanning can be enabled as part of the submission process upload file step. In `[dspace]/config/modules`, edit configuration file `submission-curation.cfg`:

NOT YET SUPPORTED IN 7.x or later

```
submission-curation.virus-scan = true
```

Task Operation from the Administrative user interface

Curation tasks can be run against container and item dspace objects by e-persons with administrative privileges. A curation tab will appear in the administrative ui after logging into DSpace:

1. Click on the curation tab.
2. Select the option configured in ui.tasknames above.
3. Select Perform.

Task Operation from the Item Submission user interface

If desired virus scanning can be enabled as part of the submission process upload file step. In `[dspace]/config/modules`, edit configuration file `submission-curation.cfg`:

NOT YET SUPPORTED IN 7.x or later

```
submission-curation.virus-scan = true
```

Task Operation from the curation command line client

To output the results to the console:

```
[dspace]/bin/dspace curate -t vscan -i <handle of container or item dso> -r -
```

Or capture the results in a file:

```
[dspace]/bin/dspace curate -t vscan -i <handle of container or item dso> -r - > /<path...>/<name>
```

Table 1 – Virus Scan Results Table

GUI (Interactive Mode)	FailFast	Expectation
Container	T	Stop on 1st Infected Bitstream
Container	F	Stop on 1st Infected Item
Item	T	Stop on 1st Infected Bitstream
Item	F	Scan all bitstreams
Command Line		
Container	T	Report on 1st infected bitstream within an item/Scan all contained Items
Container	F	Report on all infected bitstreams/Scan all contained Items
Item	T	Report on 1st infected bitstream
Item	F	Report on all infected bitstreams

Exporting Content and Metadata

General top level page to group all DSpace facilities for exporting content and metadata.

- Linked (Open) Data
- SWORDv1 Client
- Exchanging Content Between Repositories
- OAI
- OpenAIRE4 Guidelines Compliancy
- Signposting

Linked (Open) Data

- Introduction
  - Exchanging repository contents
  - Terminology
- Linked (Open) Data Support within DSpace
  - Architecture / Concept
  - Install a Triple Store
  - Default configuration and what you should change
  - Configuration Reference
    - [dspace-source]/dspace/config/modules/rdf.cfg
    - [dspace-source]/dspace/config/modules/rdf/constant-data-*.ttl
    - [dspace-source]/dspace/config/modules/rdf/metadata-rdf-mapping.ttl
    - [dspace-source]/dspace/config/modules/rdf/fuseki-assembler.ttl
    - [dspace-source]/dspace/config/spring/api/rdf.xml
  - Maintenance

Introduction

Exchanging repository contents

Most sites on the Internet are oriented towards human consumption. While HTML may be a good format for presenting information to humans, it is not a good format to export data in a way easy for a computer to work with. Like most software for building repositories, DSpace supports OAI-PMH as an interface to expose the stored metadata. While OAI-PMH is well known in the field of repositories, it is rarely known elsewhere (e.g. Google retired its support for OAI-PMH in 2008). The Semantic Web is a generic approach to publish data on the Internet together with information about its semantics. Its application is not limited to repositories or libraries and it has a growing user base. RDF and SPARQL are W3C-released standards for publishing structured data on the web in a machine-readable way. The data stored in repositories is particularly suited for use in the Semantic Web, as the metadata are already available. It doesn't have to be generated or entered manually for publication as Linked Data. For most repositories, at least for Open Access repositories, it is quite important to share their stored content. Linked Data is a rather big chance for repositories to present their content in a way that can easily be accessed, interlinked and (re)used.

Terminology

We don't want to give a full introduction into the Semantic Web and its technologies here as this can be easily found in many places on the web. Nevertheless, we want to give a short glossary of the terms used most often in this context to make the following documentation more readable.

Semantic Web	The term "Semantic Web" refers to the part of the Internet containing Linked Data. Just like the World Wide Web, the Semantic Web is also woven together by links among the data.
Linked Data **Linked Open Data**	Data in RDF, following the Linked Data Principles are called Linked Data. The Linked Data Principles describe the expected behavior of data publishers who shall ensure that the published data are easy to find, easy to retrieve, can be linked easily and link to other data as well. Linked Open Data is Linked Data published under an open license. There is no technical difference between Linked Data and Linked Open Data (often abbreviated as LOD). It is only a question of the license used to publish it.
RDF **RDF/XML** **Turtle** **N-Triples** **N3-Notation**	RDF is an acronym for Resource Description Framework, a metadata model. Don't think of RDF as a format, as it is a model. Nevertheless, there are different formats to serialize data following RDF. RDF/XML, Turtle, N-Triples and N3-Notation are probably the most well-known formats to serialize data in RDF. While RDF/XML uses XML, Turtle, N-Triples and N3-Notation don't and they are easier for humans to read and write. When we use RDF in DSpace configuration files, we currently prefer Turtle (but the code should be able to deal with any serialization).
Triple Store	A triple store is a database to natively store data following the RDF model. Just as you have to provide a relational database for DSpace, you have to provide a Triple Store for DSpace if you want to use the LOD support.
SPARQL	The SPARQL Protocol and RDF Query Language is a family of protocols to query triple stores. Since version 1.1, SPARQL can be used to manipulate triple stores as well, to store, delete or update data in triple stores. DSpace uses SPARQL 1.1 Graph Store HTTP Protocol and SPARQL 1.1 Query Language to communicate with the Triple Store. The SPARQL 1.1 Query Language is often referred to simply as SPARQL, so expect the SPARQL 1.1 Query Language if no other specific protocol out of the SPARQL family is explicitly specified.
SPARQL endpoint	A SPARQL endpoint is a SPARQL interface of a triple store. Since SPARQL 1.1, a SPARQL endpoint can be either read-only, allowing o nly to query the stored data; or readable and writable, allowing to modify the stored data as well. When talking about a SPARQL endpoint without specifying which SPARQL protocol is used, an endpoint supporting SPARQL 1.1 Query Language is meant.

Linked (Open) Data Support within DSpace

Starting with DSpace 5.0, DSpace provides support for publishing stored contents in form of Linked (Open) Data.

Architecture / Concept

To publish content stored in DSpace as Linked (Open) Data, the data have to be converted into RDF. The conversion into RDF has to be configurable as different DSpace instances may use different metadata schemata, different persistent identifiers (DOI, Handle, ...) and so on. Depending on the content to convert, configuration and other parameters, conversion may be time-intensive and impact performance. Content of repositories is much more often read then created, deleted or changed because the main goal of repositories is to safely store their contents. For this reason, the content stored within DSpace is converted and stored in a triple store immediately after it is created or updated. The triple store serves as a cache and provides a SPARQL endpoint to make the converted data accessible using SPARQL. The conversion is triggered automatically by the DSpace event system and can be started manually using the command line interface – both cases are documented below. There is no need to backup the triple store, as all data stored in the triple store can be recreated from the contents stored elsewhere in DSpace (in the assetstore(s) and the database). Beside the SPARQL endpoint, the data should be published as RDF serialization as well. With dspace-rdf DSpace offers a module that loads converted data from the triple store and provides it as an RDF serialization. It currently supports RDF/XML, Turtle and N-Triples.

Repositories use Persistent Identifiers to make content citable and to address content. Following the Linked Data Principles, DSpace uses a Persistent Identifier in the form of HTTP(S) URIs, converting a Handle to http://hdl.handle.net/<handle> and a DOI to http://dx.doi.org/<doi>. Altogether, DSpace Linke d Data support spans all three Layers: the storage layer with a triple store, the business logic with classes to convert stored contents into RDF, and the application layer with a module to publish RDF serializations. Just like DSpace allows you to choose Oracle or Postgresql as the relational database, you may choose between different triple stores. The only requirements are that the triple store must support SPARQL 1.1 Query Language and SPARQL 1.1 Graph Store HTTP Protocol which DSpace uses to store, update, delete and load converted data in/out of the triple store and uses the triple store to provide the data over a SPARQL endpoint.

Store public data only in the triple store!

The triple store should contain only data that are public, because the DSpace access restrictions won't affect the SPARQL endpoint. For this reason, DSpace converts only archived, discoverable (non-private) Items, Collections and Communities which are readable for anonymous users. Please consider this while configuring and/or extending DSpace Linked Data support.

The org.dspace.rdf.conversion package contains the classes used to convert the repository content to RDF. The conversion itself is done by plugins. The or g.dspace.rdf.conversion.ConverterPlugin interface is really simple, so take a look at it you if can program in Java and want to extend the conversion. The only thing important is that plugins must only create RDF that can be made publicly available, as the triple store provides it using a sparql endpoint for which the DSpace access restrictions do not apply. Plugins converting metadata should check whether a specific metadata field needs to be protected or not (see org.dspace.app.util.MetadataExposure on how to check that). The MetadataConverterPlugin is heavily configurable (see below) and is used to convert the metadata of Items. The StaticDSOConverterPlugin can be used to add static RDF Triples (see below). The SimpleDSORelationsConverterPlugi creates links between items and collections, collections and communities, subcommunitites and their parents, and between top-level communities and the information representing the repository itself.

As different repositories uses different persistent identifiers to address their content, different algorithms to create URIs used within the converted data can be implemented. Currently HTTP(S) URIs of the repository (called local URIs), Handles and DOIs can be used. See the configuration part of this document for further information. If you want to add another algorithm, take a look at the org.dspace.rdf.storage.URIGenerator interface.

Install a Triple Store

In addition to a normal DSpace installation you have to install a triple store. You can use any triple store that supports SPARQL 1.1 Query Language and SPARQL 1.1 Graph Store HTTP Protocol. If you do not have one yet, you can use Apache Fuseki. Download Fuseki from its official download page and unpack the downloaded archive. The archive contains several scripts to start Fuseki. Use the start script appropriate to the OS of your choice with the options '--localhost --config=<dspace-install>/config/modules/rdf/fuseki-assembler.ttl'. Instead of changing to the directory into which you unpacked Fuseki, you may set the variable FUSEKI_HOME. If you're using Linux and bash, you unpacked Fuseki to /usr/local/jena-fuseki-1.0.1 and you installed DSpace to [dspace-install], this would look like this:

```
export FUSEKI_HOME=/usr/local/jena-fuseki-1.0.1 ; $FUSKI_HOME/fuseki-server --localhost --config [dspace-
install]/config/modules/rdf/fuseki-assembler.ttl
```

Fuseki's archive contains a script to start Fuseki automatically at startup as well.

Make Fuseki connect to localhost only, by using the argument --localhost when launching if you use the configuration provided with DSpace! The configuration contains a writeable SPARQL endpoint that allows any connection to change/delete the content of your triple store.

Use Apache mod proxy, mod rewrite or any other appropriate web server/proxy to make localhost:3030/dspace/sparql readable from the internet. Use the address under which it is accessible as the address of your public sparql endpoint (see the property public.sparql.endpoint in the configuration reference below.).

The configuration provided within DSpace makes it store the files for the triple store under [dspace-install]/triplestore. Using this configuration, Fuseki provides three SPARQL endpoints: two read-only endpoints and one that can be used to change the data of the triple store. **You should not use this configuration if you let Fuseki connect to the internet directly** as it would make it possible for anyone to delete, change or add information to the triple store. The option --localhost tells Fuseki to listen only on the loopback device. You can use Apache mod_proxy or any other web or proxy server to make the read-only SPARQL endpoint accessible from the internet. With the configuration described, Fueski listens to the port 3030 using HTTP. Using the address *http://localhost:3030/* you can connect to the Fuseki Web UI. *http://localhost:3030/dspace/data* addresses a writeable SPARQL 1.1 HTTP Graph Store Protocol endpoint, and *http://localhost:3030/dspace/get* a read-only one. Under *http://localhost:3030/dspace/sparql* a read-only SPARQL 1.1 Query Language endpoint can be found. **The first one of these endpoints must be not accessible by the internet**, while the last one should be accessible publicly.

Default configuration and what you should change

First, you'll want to ensure the Linked Data endpoint is enabled/configured. In your `local.cfg`, add `rdf.enabled = true`. You can optionally change it's path by setting `rdf.path` (it defaults to "rdf" which means the Linked Data endpoint is available at `[dspace.server.url]/rdf/` (where `dspace.server.url` is also specified in your `local.cfg`)

In the file `[dspace]/config/dspace.cfg` you should look for the property `event.dispatcher.default.consumers` and add `rdf` there. Adding rdf there makes DSpace update the triple store automatically as the publicly available content of the repository changes.

As the Linked Data support of DSpace is highly configurable this section gives a short list of things you probably want to configure before using it. Below you can find more information on what is possible to configure.

In the file `[dspace]/config/modules/rdf.cfg` you want to configure the address of the public sparql endpoint and the address of the writable endpoint DSpace use to connect to the triple store (the properties `rdf.public.sparql.endpoint`, `rdf.storage.graphstore.endpoint`). In the same file you want to configure the URL that addresses the dspace-rdf module which is depending on where you deployed it (property `rdf.contextPath`) and switch content negotiation on (set property `rdf.contentNegotiation.enable = true`).

In the file `[dspace]/config/modules/rdf/constant-data-general.ttl` you should change the links to the Web UI of the repository and the public readable SPARQL endpoint. The URL of the public SPARQL endpoint should point to a URL that is proxied by a webserver to the Triple Store. See the section Install a Triple Store above for further information.

In the file `[dspace]/config/modules/rdf/constant-data-site.ttl` you may add any triples that should be added to the description of the repository itself.

If you want to change the way the metadata fields are converted, take a look into the file `[dspace]/config/modules/rdf/metadata-rdf-mapping.ttl`. This is also the place to add information on how to map metadata fields that you added to DSpace. There is already a quite acceptable default configuration for the metadata fields which DSpace supports out of the box. If you want to use some specific prefixes in RDF serializations that support prefixes, you have to edit `[dspace]onfig/modules/rdf/metadata-prefixes.ttl`.

Configuration Reference

There are several configuration files to configure DSpace's LOD support. The main configuration file can be found under `[dspace-source]/dspace/config/modules/rdf.cfg`. Within DSpace we use Spring to define which classes to load. For DSpace's LOD support this is done within `[dspace-source]/dspace/config/spring/api/rdf.xml`. All other configuration files are positioned in the directory `[dspace-source]/dspace/config/modules/rdf/`. Configurations in `rdf.cfg` can be modified directly, or overridden via your `local.cfg` config file (see Configuration Reference). You'll have to configure where to find and how to connect to the triple store. You may configure how to generate URIs to be used within the generated Linked Data and how to convert the contents stored in DSpace into RDF. We will guide you through the configuration file by file.

[dspace-source]/dspace/config/modules/rdf.cfg

Property:	rdf.enabled
Example Value:	rdf.enabled = true
Informational Note:	Defines whether the RDF endpoint is enabled or disabled (disabled by default). If enabled, the RDF endpoint is available at ${dspace.server.url}/${rdf.path}. Changing this value requires rebooting your servlet container (e.g. Tomcat)
Property:	rdf.path
Example Value:	rdf.path = rdf
Informational Note:	Defines the path of the RDF endpoint, if enabled. For example, a value of "rdf" (the default) means the RDF interface/endpoint is available at ${dspace.server.url}/rdf (e.g. if "dspace.server.url = http://localhost:8080/server", then it'd be available at "http://localhost:8080/server/rdf". Changing this value requires rebooting your servlet container (e.g. Tomcat)

Property:	rdf.contentNegotiation.enable
ExampleValue:	rdf.contentNegotiation.enable = true
InformationalNote:	Defines whether content negotiation should be activated. Set this true, if you use Linked Data support.
Property:	rdf.contextPath
ExampleValue:	rdf.contextPath = ${dspace.baseUrl}/rdf
InformationalNote:	The content negotiation needs to know where to refer if anyone asks for RDF serializations of content stored within DSpace. This property sets the URL where the dspace-rdf module can be reached on the Internet (depending on how you deployed it).
Property:	rdf.public.sparql.endpoint
ExampleValue:	rdf.public.sparql.endpoint = http://${dspace.baseUrl}/sparql
InformationalNote:	Address of the read-only public SPARQL endpoint supporting SPARQL 1.1 Query Language.
Property:	rdf.storage.graphstore.endpoint
ExampleValue:	rdf.storage.graphstore.endpoint = http://localhost:3030/dspace/data
InformationalNote:	Address of a writable SPARQL 1.1 Graph Store HTTP Protocol endpoint. This address is used to create, update and delete converted data in the triple store. If you use Fuseki with the configuration provided as part of DSpace 5, you can leave this as it is. If you use another Triple Store or configure Fuseki on your own, change this property to point to a writeable SPARQL endpoint supporting the SPARQL 1.1 Graph Store HTTP Protocol.

Property:	rdf.storage.graphstore.authentication
Example Value:	rdf.storage.graphstore.authentication = no
Informational Note:	Defines whether to use HTTP Basic authentication to connect to the writable SPARQL 1.1 Graph Store HTTP Protocol endpoint.
Properties:	rdf.storage.graphstore.login rdf.storage.graphstore.password
Example Values:	rdf.storage.graphstore.login = dspace rdf.storage.graphstore.password =ecapsd
Informational Note:	Credentials for the HTTP Basic authentication if it is necessary to connect to the writable SPARQL 1.1 Graph Store HTTP Protocol endpoint.
Property:	rdf.storage.sparql.endpoint
Example Value:	rdf.storage.sparql.endpoint = http://localhost:3030/dspace/sparql
Informational Note:	Besides a writable SPARQL 1.1 Graph Store HTTP Protocol endpoint, DSpace needs a SPARQL 1.1 Query Language endpoint, which can be read-only. This property allows you to set an address to be used to connect to such a SPARQL endpoint. If you leave this property empty the property ${rdf.public.sparql.endpoint} will be used instead.
Properties:	rdf.storage.sparql.authentication rdf.storage.sparql.login rdf.storage.sparql.password
Example Values:	rdf.storage.sparql.authentication = yes rdf.storage.sparql.login = dspace rdf.storage.sparql.password = ecapsd
Informational Note:	As for the SPARQL 1.1 Graph Store HTTP Protocol you can configure DSpace to use HTTP Basic authentication to authenticate against the (read-only) SPARQL 1.1 Query Language endpoint.

Property:	rdf.converter.DSOtypes
Example Value:	rdf.converter.DSOtypes = SITE, COMMUNITY, COLLECTION, ITEM
Informational Note:	Define which kind of DSpaceObjects should be converted. Bundles and Bitstreams will be converted as part of the Item they belong to. Don't add EPersons here unless you really know what you are doing. All converted data is stored in the triple store that provides a publicly readable SPARQL endpoint. So all data converted into RDF is exposed publicly. Every DSO type you add here must have an HTTP URI to be referenced in the generated RDF, which is another reason not to add EPersons here currently.
The following properties configure the StaticDSOConverterPlugin.	
Properties:	rdf.constant.data.GENERAL rdf.constant.data.COLLECTION rdf.constant.data.COMMUNITY rdf.constant.data.ITEM rdf.constant.data.SITE
Example Values:	rdf.constant.data.GENERAL = ${dspace.dir}/config/modules/rdf/constant-data-general.ttl rdf.constant.data.COLLECTION = ${dspace.dir}/config/modules/rdf/constant-data-collection.ttl rdf.constant.data.COMMUNITY = ${dspace.dir}/config/modules/rdf/constant-data-community.ttl rdf.constant.data.ITEM = ${dspace.dir}/config/modules/rdf/constant-data-item.ttl rdf.constant.data.SITE = ${dspace.dir}/config/modules/rdf/constant-data-site.ttl
Informational Note:	These properties define files to read static data from. These data should be in RDF, and by default Turtle is used as serialization. The data in the file referenced by the property ${rdf.constant.data.GENERAL} will be included in every Entity that is converted to RDF. E.g. it can be used to point to the address of the public readable SPARQL endpoint or may contain the name of the institution running DSpace. The other properties define files that will be included if a DSpace Object of the specified type (collection, community, item or site) is converted. This makes it possible to add static content to every Item, every Collection, ...
The following properties configure the MetadataConverterPlugin.	
Property:	rdf.metadata.mappings
Example Value:	rdf.metadata.mappings = ${dspace.dir}/config/modules/rdf/metadata-rdf-mapping.ttl
Informational Note:	Defines the file that contains the mappings for the MetadataConverterPlugin. See below the description of the configuration file [dspace-source] /dspace/config/modules/rdf/metadata-rdf-mapping.ttl.
Property:	rdf.metadata.schema
Example Value:	rdf.metadata.schema = file://${dspace.dir}/config/modules/rdf/metadata-rdf-schema.ttl

Informational Note:	Configures the URL used to load the RDF Schema of the DSpace Metadata RDF mapping Vocabulary. Using a file:// URI makes it possible to convert DSpace content without having an internet connection. The version of the schema has to be the right one for the used code. In DSpace 5.0 we use the version 0.2.0. This Schema can be found here as well: http://digital-repositories.org/ontologies/dspace-metadata-mapping/0.2.0. The newest version of the Schema can be found here: http://digital-repositories.org/ontologies/dspace-metadata-mapping/.
Property:	rdf.metadata.prefixes
Example Value:	rdf.metadata.prefixes = ${dspace.dir}/config/modules/rdf/metadata-prefixes.ttl
Informational Note:	If you want to use prefixes in RDF serializations that support prefixes, you can define these prefixes in the file referenced by this property.
The following properties configure the SimpleDSORelationsConverterPlugin	
Property:	rdf.simplerelations.prefixes
Example Value:	rdf.simplerelations.prefixes = ${dspace.dir}/config/modules/rdf/simple-relations-prefixes.ttl
Informational Note:	If you want to use prefixes in RDF serializations that support prefixes, you can define these prefixes in the file referenced by this property.
Property:	rdf.simplerelations.site2community
Example Value:	rdf.simplerelations.site2community = http://purl.org/dc/terms/hasPart, http://digital-repositories.org/ontologies/dspace/0.1.0#hasCommunity
Informational Note:	Defines the predicates used to link from the data representing the whole repository to the top level communities. Defining multiple predicates separated by commas will result in multiple triples.
Property:	rdf.simplerelations.community2site
Example Value:	rdf.simplerelations.community2site = http://purl.org/dc/terms/isPartOf, http://digital-repositories.org/ontologies/dspace/0.1.0#isPartOfRepository

Informational Note:	Defines the predicates used to link from the top level communities to the data representing the whole repository. Defining multiple predicates separated by commas will result in multiple triples.
Property:	rdf.simplerelations.community2subcommunity
Example Value:	rdf.simplerelations.community2subcommunity = http://purl.org/dc/terms/hasPart, http://digital-repositories.org/ontologies/dspace/0.1.0#hasSubcommunity
Informational Note:	Defines the predicates used to link from communities to their subcommunities. Defining multiple predicates separated by commas will result in multiple triples.
Property:	rdf.simplerelations.subcommunity2community
Example Value:	rdf.simplerelations.subcommunity2community = http://purl.org/dc/terms/isPartOf, http://digital-repositories.org/ontologies/dspace/0.1.0#isSubcommunityOf
Informational Note:	Defines the predicates used to link from subcommunities to the communities they belong to. Defining multiple predicates separated by commas will result in multiple triples.
Property:	rdf.simplerelations.community2collection
Example Value:	rdf.simplerelations.community2collection = http://purl.org/dc/terms/hasPart, http://digital-repositories.org/ontologies/dspace/0.1.0#hasCollection
Informational Note:	Defines the predicates used to link from communities to their collections. Defining multiple predicates separated by commas will result in multiple triples.
Property:	rdf.simplerelations.collection2community
Example Value:	rdf.simplerelations.collection2community = http://purl.org/dc/terms/isPartOf, http://digital-repositories.org/ontologies/dspace/0.1.0#isPartOfCommunity

Informational Note:	Defines the predicates used to link from collections to the communities they belong to. Defining multiple predicates separated by commas will result in multiple triples.
Property:	rdf.simplerelations.collection2item
Example Value:	rdf.simplerelations.collection2item = http://purl.org/dc/terms/hasPart, http://digital-repositories.org/ontologies/dspace/0.1.0#hasItem
Informational Note:	Defines the predicates used to link from collections to their items. Defining multiple predicates separated by commas will result in multiple triples.
Property:	rdf.simplerelations.item2collection
Example Value:	rdf.simplerelations.item2collection = http://purl.org/dc/terms/isPartOf, http://digital-repositories.org/ontologies/dspace/0.1.0#isPartOfCollection
Informational Note:	Defines the predicates used to link from items to the collections they belong to. Defining multiple predicates separated by commas will result in multiple triples.
Property:	rdf.simplerelations.item2bitstream
Example Value:	rdf.simplerelations.item2bitstream = http://purl.org/dc/terms/hasPart, http://digital-repositories.org/ontologies/dspace/0.1.0#hasBitstream
Informational Note:	Defines the predicates used to link from item to their bitstreams. Defining multiple predicates separated by commas will result in multiple triples.

[dspace-source]/dspace/config/modules/rdf/constant-data-*.ttl

As described in the documentation of the configuration file [dspace-source]/dspace/config/modules/rdf.cfg, the constant-data-*.ttl files can be used to add static RDF to the converted data. The data are written in Turtle, but if you change the file suffix (and the path to find the files in rdf.cfg) you can use any other RDF serialization you like to. You can use this, for example, to add a link to the public readable SPARQL endpoint, add a link to the repository homepage, or add a triple to every community or collection defining it as an entity of a specific type like a bibo:collection. The content of the file [dspace-source]/dspace/config/modules/rdf/constant-data-general.ttl will be added to every DSpaceObject that is converted. The content of the file [dspace-source] /dspace/config/modules/rdf/constant-data-community.ttl to every community, the content of the file [dspace-source]/dspace/config/modules/rdf/constant-data-collection.ttl to every collection and the content of the file [dspace-source]/dspace/config/modules/rdf/constant-data-item.ttl to every Item. You can use the file [dspace-source]/dspace/config/modules/rdf/constant-data-site.ttl to specify data representing the whole repository.

[dspace-source]/dspace/config/modules/rdf/metadata-rdf-mapping.ttl

This file should contain several metadata mappings. A metadata mapping defines how to map a specific metadata field within DSpace to a triple that will be added to the converted data. The MetadataConverterPlugin uses these metadata mappings to convert the metadata of a item into RDF. For every metadata field and value it looks if any of the specified mappings matches. If one does, the plugin creates the specified triple and adds it to the converted data. In the file you'll find a lot of examples on how to define such a mapping.

For every mapping a metadata field name has to be specified, e.g. dc.title, dc.identifier.uri. In addition you can specify a condition that is matched against the field's value. The condition is specified as a regular expression (using the syntax of the java class java.util.regex.Pattern). If a condition is defined, the mapping will be used only on fields those values which are matched by the regex defined as condition.

The triple to create by a mapping is specified using reified RDF statements. The DSpace Metadata RDF Mapping Vocabulary defines some placeholders that can be used. The most important placeholder is dm:DSpaceObjectIRI which is replaced by the URI used to identify the entity being converted to RDF. That means if a specific Item is converted the URI used to address this Item in RDF will be used instead of dm:DSpaceObjectIRI. There are three placeholders that allow reuse of the value of a meta data field. dm:DSpaceValue will be replace by the value as it is. dm:LiteralGenerator allows one to specify a regex and replacement string for it (see the syntax of the java classes java.util.regex.Pattern and java.util.regex.Matcher) and creates a Literal out of the field value using the regex and the replacement string. dm:ResourceGenerator does the same as dm:LiteralGenerator but it generates a HTTP (S) URI that is used in place. So you can use the resource generator to generate URIs containing modified field values (e.g. to link to classifications). If you know regular expressions and turtle, the syntax should be quite self explanatory.

[dspace-source]/dspace/config/modules/rdf/fuseki-assembler.ttl

This is a configuration for the triple store Fuseki of the Apache Jena project. You can find more information on the configuration it provides in the section Install a Triple Store above.

[dspace-source]/dspace/config/spring/api/rdf.xml

This file defines which classes are loaded by DSpace to provide the RDF functionality. There are two things you might want to change: the class that is responsible to generate the URIs to be used within the converted data, and the list of Plugins used during conversion. To change the class responsible for the URIs, change the following line:

```
<property name="generator" ref="org.dspace.rdf.storage.LocalURIGenerator"/>
```

This line defines how URIs should be generated, to be used within the converted data. The LocalURIGenerator generates URIs using the ${dspace.url} property. The HandleURIGenerator uses handles in form of HTTP URLs. It uses the property ${handle.canonical.prefix} to convert handles into HTTPS URLs. The class org.dspace.rdf.storage.DOIURIGenerator uses DOIs in the form of HTTP URLs if possible, or local URIs if there are no DOIs. It uses the DOI resolver "http://dx.doi.org" to convert DOIs into HTTP URLs. The class org.dspace.rdf.storage.DOIHandleGenerator does the same but uses Handles as fallback if no DOI exists. The fallbacks are necessary as DOIs are currently used for Items only and not for Communities or Collections.

All plugins that are instantiated within the configuration file will automatically be used during the conversion. Per default the list looks like the following:

```
<!-- configure all plugins the converter should use. If you don't want to
         use a plugin, remove it here. -->
    <bean id="org.dspace.rdf.conversion.SimpleDSORelationsConverterPlugin" class="org.dspace.rdf.conversion.
SimpleDSORelationsConverterPlugin"/>
    <bean id="org.dspace.rdf.conversion.MetadataConverterPlugin" class="org.dspace.rdf.conversion.
MetadataConverterPlugin"/>
    <bean id="org.dspace.rdf.conversion.StaticDSOConverterPlugin" class="org.dspace.rdf.conversion.
StaticDSOConverterPlugin"/>
```

You can remove plugins if you don't want them. If you develop a new conversion plugin, you want to add its class to this list.

Maintenance

As described above you should add `rdf` to the property `event.dispatcher.default.consumers` and in dspace.cfg. This configures DSpace to automatically update the triple store every time the publicly available content of the repository is changed. Nevertheless there is a command line tool that gives you the possibility to update the content of the triple store. As the triple store is used as a cache only, you can delete its content and reindex it every time you think it is necessary of helpful. The command line tool can be started by the following command which will show its online help:

```
[dspace-install]/bin/dspace rdfizer --help
```

The online help should give you all necessary information. There are commands to delete one specific entity; to delete all information stored in the triple store; to convert one item, one collection or community (including all subcommunities, collections and items) or to convert the complete content of your repository. If you start using the Linked Open Data support on a repository that already contains content, you should run `[dspace-install]/bin/dspace rdfizer --convert-all` once.

Every time content of DSpace is converted or Linked Data is requested, DSpace will try to connect to the triple store. So ensure that it is running (as you do with e.g. your sevlet container or relational database).

SWORDv1 Client

The embedded SWORD Client allows a user (currently restricted to an administrator) to copy an item to a SWORD server. This allows your DSpace installation to deposit items into another SWORD-compliant repository (including another DSpace install).

DSpace 7.0 does not yet support

The SWORDv1 Client is not available in DSpace 7.0. It may be restored in a later 7.x release, see DSpace Release 7.0 Status

1 Enabling the SWORD Client
2 Configuring the SWORD Client

Enabling the SWORD Client

The SWORDv1 Client is not available in DSpace 7.0. It may be restored in a later 7.x release, see DSpace Release 7.0 Status

Configuring the SWORD Client

All the relevant configuration can be found in `sword-client.cfg`. These may be overridden in your `local.cfg` config (see Configuration Reference).

Configuration File:	**`[dspace]/config/modules/sword-client.cfg`**
Property:	`sword-client.targets`

Example value:	`sword-client.targets = http://localhost:8080/sword/servicedocument, \`
	`  http://client.swordapp.org/client/servicedocument, \`
	`  http://dspace.swordapp.org/sword/servicedocument, \`
	`  http://sword.eprints.org/sword-app/servicedocument, \`
	`  http://sword.intralibrary.com/IntraLibrary-Deposit/service, \`
	`  http://fedora.swordapp.org/sword-fedora/servicedocument`

Informational note:	List of remote Sword servers. Used to build the drop-down list of selectable SWORD targets.
Property:	`sword-client.file-types`
Example value:	`sword-client.file-types = application/zip`
Informational note:	List of file types from which the user can select. If a type is not supported by the remote server it will not appear in the drop-down list.
Property:	`sword-client.package-formats`
Example value:	`sword-client.package-formats = http://purl.org/net/sword-types/METSDSpaceSIP`
Informational note:	List of package formats from which the user can select. If a format is not supported by the remote server it will not appear in the drop-down list.

Exchanging Content Between Repositories

1 Transferring Content via Export and Import
    1.1 Transferring Communities, Collections, or Items using Packages
2 Transferring Items using Simple Archive Format
3 Transferring Items using OAI-ORE/OAI-PMH Harvester

Transferring Content via Export and Import

To migrate content from one DSpace to another, you can export content from the Source DSpace and import it into the Destination DSpace.

Transferring Communities, Collections, or Items using Packages

You may transfer any DSpace content (Communities, Collections or Items) from one DSpace to another by utilizing the AIP Backup and Restore tool. This tool allows you to export content into a series of Archival Information Packages (AIPs). These AIPs can be used to restore content (from a backup) or move /migrate content to another DSpace installation.

For more information see AIP Backup and Restore.

Transferring Items using Simple Archive Format

Where items are to be moved between DSpace instances (for example from a test DSpace into a production DSpace) the Item Exporter and Item Importer can be used.

First, you should export the DSpace Item(s) into the Simple Archive Format, as detailed at: Importing and Exporting Items via Simple Archive Format (SAF). Be sure to use the --migrate option, which removes fields that would be duplicated on import. Then import the resulting files into the other instance.

Transferring Items using OAI-ORE/OAI-PMH Harvester

You may also choose to enable the OAI-ORE Harvester. This OAI-ORE Harvester allows one DSpace installation to harvest Items (via OAI-ORE) from another DSpace Installation (or any other system supporting OAI-ORE). Items are harvested from a remote DSpace Collection into a local DSpace Collection. Harvesting can also be scheduled to run automatically (or by demand).

See OAI - Harvesting from another DSpace

OAI

OAI Interfaces

1 OAI-PMH Server
    1.1 OAI-PMH Server Activation
    1.2 OAI-PMH Server Maintenance
2 OAI-PMH / OAI-ORE Harvester (Client)
    2.1 Harvesting from another DSpace
    2.2 OAI-PMH / OAI-ORE Harvester Configuration
    2.3 Setting up a harvest to import content into a collection
        2.3.1 Using the "harvest" script
            2.3.1.1 Examples of harvesting a collection through CLI commands
        2.3.2 Setting up a harvest content source from the UI
3 DSpace 7 Demo - OAI-PMH

OAI-PMH Server

In the following sections and subpages, you will learn how to configure OAI-PMH server and activate additional OAI-PMH crosswalks. The user is also referred to OAI-PMH Data Provider for greater depth details of the program.

The OAI-PMH Interface may be used by other systems to harvest metadata records from your DSpace.

OAI-PMH Server Activation

DSpace's OAI-PMH server is enabled by default. However, you can choose to enable/disable it in your local.cfg using these configurations:

```
# Enable (true) or disable (false) OAI-PMH server
oai.enabled = true

# When enabled, OAI-PMH server is available at this path
oai.path = oai
```

If you modify either of these configuration, you must restart your Servlet Container (usually Tomcat).

- You can test that it is working by sending a request to: `[dspace.server.url]/[oai.path]/request?verb=Identify` (e.g. http://localhost:8080/server/oai/request?verb=Identify)
- The response should look similar to the response from the DSpace 7 Demo Server: https://demo.dspace.org/server/oai/request?verb=Identify

If you're using a recent browser, you should see a HTML page describing your repository. What you're getting from the server is in fact an XML file with a link to an XSLT stylesheet that renders this HTML in your browser (client-side). Any browser that cannot interpret XSLT will display pure XML. The default stylesheet is located in `[dspace-source]/dspace-oai/src/main/resources/static/style.xsl` and can be changed by configuring the `stylesheet` attribute of the `Configuration` element in `[dspace]/config/crosswalks/oai/xoai.xml`.

Relevant Links

- OAI 2.0 Server - basic information needed to configure and use the OAI Server in DSpace
- OAI-PMH Data Provider 2.0 (Internals) - information on how it's implemented
- http://www.openarchives.org/pmh/ - information on the OAI-PMH protocol and its usage (not DSpace-specific)

OAI-PMH Server Maintenance

After activating the OAI-PMH server, you need to also ensure its index is updated on a regular basis. Currently, this doesn't happen automatically within DSpace. Instead, you must schedule the `[dspace.dir]/bin/dspace oai import` commandline tool to run on a regular basis (usually at least nightly, but you could schedule it more frequently).

Here's an example cron that can be used to schedule an OAI-PMH reindex on a nightly basis (for a full list of recommended DSpace cron tasks see Scheduled Tasks via Cron):

```
# Update the OAI-PMH index with the newest content at midnight every day
# NOTE: ONLY NECESSARY IF YOU ARE RUNNING OAI-PMH
# (This ensures new content is available via OAI-PMH)
0 0 * * * [dspace.dir]/bin/dspace oai import > /dev/null
```

More information about the `dspace oai` commandline tool can be found in the OAI Manager documentation.

OAI-PMH / OAI-ORE Harvester (Client)

This section describes the parameters used in configuring the OAI-ORE / OAI-ORE harvester. This harvester can be used to harvest content (bitstreams and metadata) into DSpace from an external OAI-PMH or OAI-ORE server.

Supported in 7.1 or above

OAI Harvesting was not available in DSpace 7.0. It was restored in DSpace 7.1. See DSpace Release 7.0 Status

Harvesting from another DSpace

If you are harvesting content (bitstreams and metadata) **from** an external DSpace installation via OAI-PMH & OAI-ORE, you first should verify that the external DSpace installation allows for OAI-ORE harvesting.

If the external DSpace is running v6.x or below, it must be running both the OAI-PMH interface and the XMLUI interface to support harvesting content from it via OAI-ORE.

If the external DSpace is running v7.x or above, it just needs to be running the OAI-PMH interface.

You can verify that OAI-ORE harvesting option is enabled by following these steps:

1. First, check to see if the external DSpace reports that it will support harvesting ORE via the OAI-PMH interface. Send the following request to the DSpace's OAI-PMH interface: `http://[full-URL-to-OAI-PMH]/request?verb=ListRecords&metadataPrefix=ore`
    - The response should be an XML document containing ORE, similar to the response from the DSpace Demo Server: http://demo.dspace.org/server/oai/request?verb=ListRecords&metadataPrefix=ore
2. For 6.x or below, you can verify that the XMLUI interface supports OAI-ORE (it should, as long as it's a current version of DSpace). First, find a valid Item Handle. Then, send the following request to the DSpace's XMLUI interface: `http://[full-URL-to-XMLUI]/metadata/handle/[item-handle]/ore.xml`
    - The response should be an OAI-ORE (XML) document which describes that specific Item. It should look similar to the response from the DSpace Demo Server: http://demo.dspace.org/xmlui/metadata/handle/10673/3/ore.xml

For examples of how to set up harvesting via the User Interface, see the "Content Source" settings of the "Edit Collection" documentation

OAI-PMH / OAI-ORE Harvester Configuration

There are many possible configuration options for the OAI harvester. Most of these are contained in the `[dspace]/config/modules/oai.cfg` file (unless otherwise noted below). They may be updated there or overridden in your `local.cfg` config file (see Configuration Reference).

Configuration File:	**`[dspace]/config/modules/oai.cfg`**
Property:	`oai.harvester.eperson`
Example Value:	`oai.harvester.eperson = admin@myu.edu`
Informational Note:	The EPerson under whose authorization automatic harvesting will be performed. This field does not have a default value and must be specified in order to use the harvest scheduling system. This will most likely be the DSpace admin account created during installation.
Property:	`oai.url`
Example Value:	`oai.url = ${dspace.server.url}/${oai.path}`
Informational Note:	The base url of the OAI-PMH disseminator webapp (i.e. do not include the /request on the end). This is necessary in order to mint URIs for ORE Resource Maps. The default value of `${dspace.baseUrl}/oai` will work for a typical installation, but should be changed if appropriate. Please note that `dspace.baseUrl` is defined in your `dspace.cfg` configuration file.
Property:	`oai.ore.authoritative.source`
Example Value:	`oai.ore.authoritative.source = oai`
Informational Note:	The webapp responsible for minting the URIs for ORE Resource Maps. If using oai, the `oai.url` config value must be set. • When set to 'oai', all URIs in ORE Resource Maps will be relative to the OAI-PMH URL (configured by `oai.url` above) • The URIs generated for ORE ReMs follow the following convention for either setting: http://\[base-URL\]/metadata/handle/\[item-handle\]/ore.xml
Property:	`oai.harvester.autoStart`
Example Value:	`oai.harvester.autoStart = false`
Informational Note:	Determines whether the harvest scheduler process starts up automatically when DSpace webapp is redeployed.

Property:	`oai.harvester.metadataformats.PluginName`
Example Value:	`oai.harvester.metadataformats.PluginName = \` `http://www.openarchives.org/OAI/2.0/oai_dc/, Simple Dublin Core`
Informational Note:	This field can be repeated and serves as a link between the metadata formats supported by the local repository and those supported by the remote OAI-PMH provider. It follows the form `oai.harvester.metadataformats.PluginName = NamespaceURI,Optional Display Name` . The pluginName designates the metadata schemas that the harvester "knows" the local DSpace repository can support. Consequently, the PluginName must correspond to a previously declared ingestion crosswalk. The namespace value is used during negotiation with the remote OAI-PMH provider, matching it against a list returned by the ListMetadataFormats request, and resolving it to whatever metadataPrefix the remote provider has assigned to that namespace. Finally, the optional display name is the string that will be displayed to the user when setting up a collection for harvesting. If omitted, the PluginName:NamespaceURI combo will be displayed instead.
Property:	`oai.harvester.oreSerializationFormat.OREPrefix`
Example Value:	`oai.harvester.oreSerializationFormat.OREPrefix = \` `http://www.w3.org/2005/Atom`
Informational Note:	This field works in much the same way as `oai.harvester.metadataformats.PluginName` . The `OREPrefix` must correspond to a declared ingestion crosswalk, while the Namespace must be supported by the target OAI-PMH provider when harvesting content.
Property:	`oai.harvester.timePadding`
Example Value:	`oai.harvester.timePadding = 120`
Informational Note:	Amount of time subtracted from the from argument of the PMH request to account for the time taken to negotiate a connection. Measured in seconds. Default value is 120.
Property:	`oai.harvester.harvestFrequency`
Example Value:	`oai.harvester.harvestFrequency = 720`
Informational Note:	How frequently the harvest scheduler checks the remote provider for updates. Should always be longer than *timePadding* . Measured in minutes. Default value is 720.
Property:	`oai.harvester.minHeartbeat`
Example Value:	`oai.harvester.minHeartbeat = 30`
Informational Note:	The heartbeat is the frequency at which the harvest scheduler queries the local database to determine if any collections are due for a harvest cycle (based on the *harvestFrequency*) value. The scheduler is optimized to then sleep until the next collection is actually ready to be harvested. The *minHeartbeat* and *maxHeartbeat* are the lower and upper bounds on this timeframe. Measured in seconds. Default value is 30.
Property:	`oai.harvester.maxHeartbeat`
Example Value:	`oai.harvester.maxHeartbeat = 3600`
Informational Note:	The heartbeat is the frequency at which the harvest scheduler queries the local database to determine if any collections are due for a harvest cycle (based on the *harvestFrequency*) value. The scheduler is optimized to then sleep until the next collection is actually ready to be harvested. The *minHeartbeat* and *maxHeartbeat* are the lower and upper bounds on this timeframe. Measured in seconds. Default value is 3600 (1 hour).
Property:	`oai.harvester.maxThreads`
Example Value:	`oai.harvester.maxThreads = 3`
Informational Note:	How many harvest process threads the scheduler can spool up at once. Default value is 3.
Property:	`oai.harvester.threadTimeout`
Example Value:	`oai.harvester.threadTimeout = 24`
Informational Note:	How much time passes before a harvest thread is terminated. The termination process waits for the current item to complete ingest and saves progress made up to that point. Measured in hours. Default value is 24.
Property:	`oai.harvester.unknownField`

Example Value:	`oai.harvester.unkownField = fail | add | ignore`
Informational Note:	You have three (3) choices. When a harvest process completes for a single item and it has been passed through ingestion crosswalks for ORE and its chosen descriptive metadata format, it might end up with DIM values that have not been defined in the local repository. This setting determines what should be done in the case where those DIM values belong to an already declared schema. *Fail* will terminate the harvesting task and generate an error. **Ignore** will quietly omit the unknown fields. **Add** will add the missing field to the local repository's metadata registry. Default value: **fail**.
Property:	`oai.harvester.unknownSchema`
Example Value:	`oai.harvester.unknownSchema = fail | add | ignore`
Informational Note:	When a harvest process completes for a single item and it has been passed through ingestion crosswalks for ORE and its chosen descriptive metadata format, it might end up with DIM values that have not been defined in the local repository. This setting determines what should be done in the case where those DIM values belong to an unknown schema. **Fail** will terminate the harvesting task and generate an error. **Ignore** will quietly omit the unknown fields. **Add** will add the missing schema to the local repository's metadata registry, using the schema name as the prefix and "unknown" as the namespace. Default value: **fail**.
Property:	`oai.harvester.acceptedHandleServer`
Example Value:	`oai.harvester.acceptedHandleServer = \` `hdl.handle.net, handle.test.edu`
Informational Note:	A harvest process will attempt to scan the metadata of the incoming items (identifier.uri field, to be exact) to see if it looks like a handle. If so, it matches the pattern against the values of this parameter. If there is a match the new item is assigned the handle from the metadata value instead of minting a new one. Default value: *hdl.handle.net* .
Property:	`oai.harvester.rejectedHandlePrefix`
Example Value:	`oai.harvester.rejectedHandlePrefix = 123456789, myeduHandle`
Informational Note:	Pattern to reject as an invalid handle prefix (known test string, for example) when attempting to find the handle of harvested items. If there is a match with this config parameter, a new handle will be minted instead. Default value: *123456789* .

Setting up a harvest to import content into a collection

There are two options to set up a collection for harvesting. One is by using the DSpace scripts "harvest", the other is by setting up the content source of a collection through the UI.

Using the "harvest" script

The harvest script can be called from both the CLI and REST API by calling "harvest". It uses the paramaters as defined in the following table.

Short option	Long option	Argument	Explanation
-p	--purge	*[none]*	Delete all the items in the collection provided with the `-c` parameter.
-r	--run	*[none]*	Run the standard harvesting procedure for the collection provided with the `-c` parameter.
-g	--ping	*[none]*	Verify that the server provided through the `-a` parameter and the set provided through the `-i` parameter can be resolved and work.
-s	--setup	*[none]*	Set the collection provided with the `-c` parameter up for harvesting. The server will need to be provided through the `-a` parameter, and the oai set id needs to be provided by the `-i` parameter.
-S	--start	*[none]*	Start the harvest loop for all collections.
-R	--reset	*[none]*	Reset the harvest status on all collections.
-P	--purgeCollections	*[none]*	Purge all harvestable collections.
-o	--reimport	*[none]*	Reimport all items the items in the collection provided by the `-c` parameter. This is the equivalent of running both the `-p` and the `-r` command for the provided collection.
-c	--collection	[id-or-handle]	The harvesting collection (handle or id)
-t	--type	[type-code]	The type of harvesting: 0 for no harvesting, 1 for metadata only, 2 for metadata and bitstream references (requires ORE support), 3 for metadata and bitstreams (requires ORE support)
-a	--address	[url]	The address of the OAI-PMH server to be harvested
-i	--oai_set_id	[set-id]	The id of the PMH set representing the harvested collection. In case all sets need to harvested the value "all" should be provided.

-m	--metadata_format	[format]	The name of the desired metadata format for harvesting, resolved to namespace and crosswalk in the dspace.cfg
-h	--help	*[none]*	Print the help message
-e	--eperson	[email]	**(CLI ONLY)** The eperson that performs the harvest. When the command is used from the REST API, the currently logged in user will be used.

Examples of harvesting a collection through CLI commands

1. Verify whether the harvester source can be reached

```
dspace/bin/dspace harvest -g -a https://harvest.source.org -i harvest-set
```

Replace `https://harvest.source.org` with the source you want to use, the `harvest-set` with the set/sets you want to harvest or `all` in case you want to harvest all sets.

2. Set up a collection for harvesting

```
dspace/bin/dspace harvest -s -c 123456789/123 -e harvest-user@dspace.org -a https://harvest.source.org -i harvest-set -m dc -t 1
```

Replace the `123456789/123` with your collection, the `harvest-user@dspace.org` with an existing user in DSpace that has sufficient rights to perform the ingestion, `https://harvest.source.org` with the source you want to use, the `harvest-set` with the set/sets you want to harves or `all` in case you want to harvest all sets. The `-m` parameter indicated the metadata format to be used and the `-t` parameter indicates the harvest type to be used. When the value `0` is used for `-t` , harvesting will be disabled.

3. Run the harvest for the set up collection

```
dspace/bin/dspace harvest -r -c 123456789/123 -e harvest-user@dspace.org
```

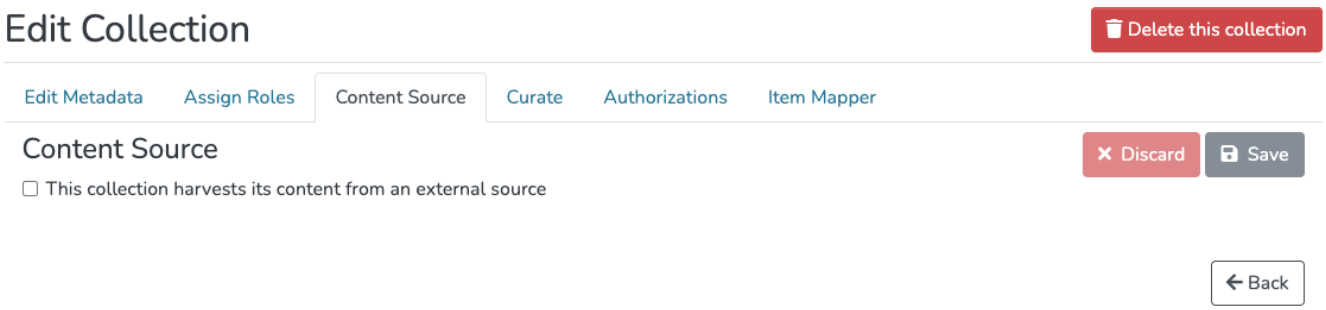
Replace the `123456789/123` with your collection, the `harvest-user@dspace.org` with an existing user in DSpace that has sufficient rights to perform the ingestion.

Setting up a harvest content source from the UI

A collection can be configured to retrieve its content from an external source. This can be done from the "Edit Collection" UI by using the following steps.

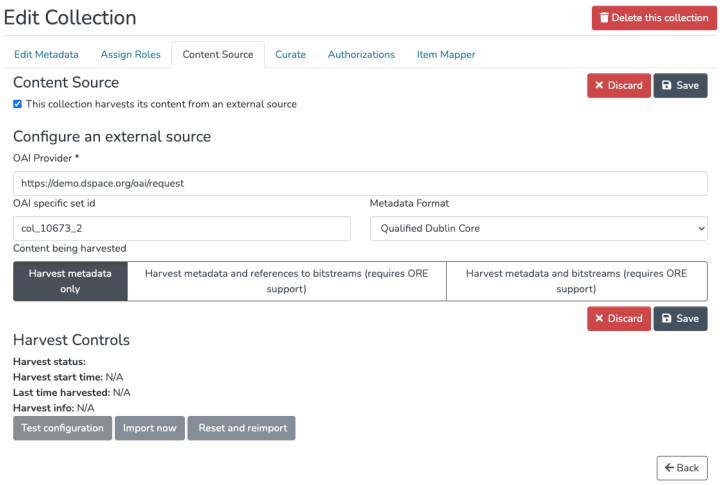
1. Configure the collection to harvest its content from an external source

Navigate to the "Edit collection" > "Content Source" tab. Tick the checkbox "This collection harvests its content from an external source".



2. Configure the harvest source

Once the checkbox has been ticket, the OAI provider, set id and metadata format can be configured. An example of the configuration can be found in the image below.

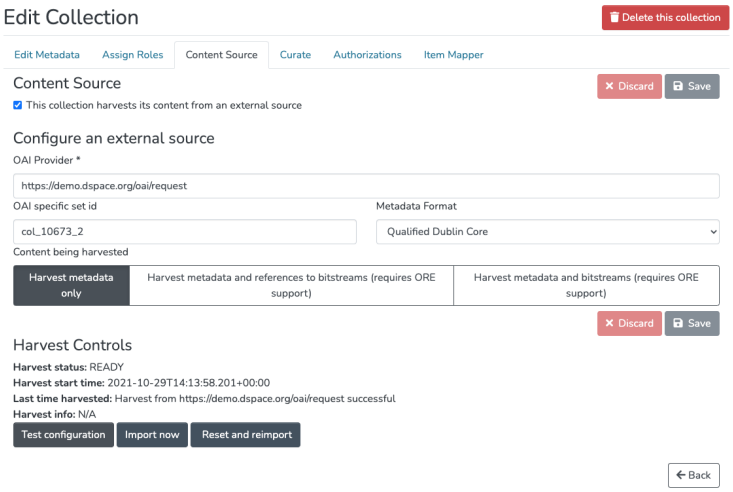


When all sets need to be harvested, the field can be left empty.

The server configuration will be tested upon clicking the "Save" button.

3. Start the harvest

Click the "Import Now" button to start the import. When the import has started, the button will indicate that the import is in progress, however, there is no need to remain on this page as the harvest will continue to run after leaving this page.



If the current server configuration needs to be retested at a later point, the "Test configuration" button can be used. To fully reset the collection by purging all items and starting a reimport, click the "Reset and reimport" button.

DSpace 7 Demo - OAI-PMH

- https://demo.dspace.org/server/oai/request?verb=Identify

OAI-PMH Data Provider 2.0 (Internals)

1 OAI-PMH Data Provider 2.0 (Internals)
- 1.1 Sets
- 1.2 Unique Identifier
- 1.3 Access control
- 1.4 Modification Date (OAI Date Stamp)
- 1.5 "About" Information
- 1.6 Deletions
- 1.7 Flow Control (Resumption Tokens)

OAI-PMH Data Provider 2.0 (Internals)

The DSpace platform supports the Open Archives Initiative Protocol for Metadata Harvesting (OAI-PMH) version 2.0 as a data provider. This is accomplished using the XOAI OAI-PMH Java Toolkit.

The DSpace build process builds a single backend webapp, which optionally includes an OAI-PMH endpoint (when oai.enabled=true) In a typical configuration, this endpoint is deployed at `${dspace.server.url}/oai` (configured by "oai.path"), containing request, driver and openaire contexts, for example:

```
http://dspace.myu.edu/server/oai/request?verb=Identify
```

The "base URL" of this DSpace deployment would be:

```
http://dspace.myu.edu/server/oai/request
```

But one could also provide the Driver or OpenAIRE contexts:

```
http://dspace.myu.edu/server/oai/driver
http://dspace.myu.edu/server/oai/openaire
```

It is this URL that should be registered with www.openarchives.org.

DSpace provides implementations of the XOAI data sources interfaces.

Sets

OAI-PMH allows repositories to expose an hierarchy of sets in which records may be placed. A record can be in zero or more sets.

DSpace exposes collections and communities as sets.

Each community and collection has a corresponding OAI set, discoverable by harvesters via the ListSets verb. The setSpec is based on the community /collection handle, with the "/" converted to underscore to form a legal setSpec. The setSpec is prefixed by "com_" or "col_" for communities and collections, respectively (this is a change in set names in DSpace 3.0 / OAI 2.0). For example:

```
col_1721.1_1234
```

Naturally enough, the community/collection name is also the name of the corresponding set.

Unique Identifier

Every item in OAI-PMH data repository must have an unique identifier, which must conform to the URI syntax. As of DSpace 1.2, Handles are not used; this is because in OAI-PMH, the OAI identifier identifies the *metadata record* associated with the *resource*. The *resource* is the DSpace item, whose *resource identifier* is the Handle. In practical terms, using the Handle for the OAI identifier may cause problems in the future if DSpace instances share items with the same Handles; the OAI metadata record identifiers should be different as the different DSpace instances would need to be harvested separately and may have different metadata for the item.

The OAI identifiers that DSpace uses are of the form:

```
oai:PREFIX:handle
```

For example:

```
oai:dspace.myu.edu:123456789/345
```

If you wish to use a different scheme, this can easily be changed by editing the value of identifier.prefix at [dspace]/config/modules/oai.cfg file.

Access control

OAI provides no authentication/authorisation details, although these could be implemented using standard HTTP methods. It is assumed that all access will be anonymous for the time being.

A question is, "is all metadata public?" Presently the answer to this is yes; all metadata is exposed via OAI-PMH, even if the item has restricted access policies. The reasoning behind this is that people who do actually have permission to read a restricted item should still be able to use OAI-based services to discover the content. But, exposed data could be changed by changing the XSLT defined at [dspace]/config/crosswalks/oai/metadataFormats.

Modification Date (OAI Date Stamp)

OAI-PMH harvesters need to know when a record has been created, changed or deleted. DSpace keeps track of a "last modified" date for each item in the system, and this date is used for the OAI-PMH date stamp. This means that any changes to the metadata (e.g. admins correcting a field, or a withdrawal) will be exposed to harvesters.

"About" Information

As part of each record given out to a harvester, there is an optional, repeatable "about" section which can be filled out in any (XML-schema conformant) way. Common uses are for provenance and rights information, and there are schemas in use by OAI communities for this. Presently DSpace does not provide any of this information, but XOAI core library allows its definition. This requires to dive into code and perform some changes.

Deletions

As DSpace supports two forms of deletions (withdrawals or permanent expunging), this has an impact on how OAI-PMH exposes delitions. During a permanent deletion (expunge), DSpace no longer retains any information about the deleted object. Therefore, permanent deletions "disappear" from OAI-PMH, as DSpace no longer has any information about the object. This is considered a "transient" approach to deletion based on OAI-PMH definitions.

When an item is withdrawn in DSpace, the item still exists but it hidden from public view. Withdrawn items will report a "<header status="deleted">" in OAI-PMH when a GetRecord request is made for a withdrawn item (however, they are NOT shown in an OAI-PMH "ListRecords" request by default). Keep in mind that the OAI-PMH index does NOT update automatically, so withdrawn items will not show this "deleted" status until "./dspace oai import" is next run.

Once an item has been withdrawn, OAI-PMH harvests of the date range in which the withdrawal occurred will find the "deleted" record header. Harvests of a date range prior to the withdrawal will *not* find the record, despite the fact that the record did exist at that time. As an example of this, consider an item that was created on 2002-05-02 and withdrawn on 2002-10-06. A request to harvest the month 2002-10 will yield the "record deleted" header. However, a harvest of the month 2002-05 will not yield the original record.

Flow Control (Resumption Tokens)

An OAI data provider can prevent any performance impact caused by harvesting by forcing a harvester to receive data in time-separated chunks. If the data provider receives a request for a lot of data, it can send part of the data with a resumption token. The harvester can then return later with the resumption token and continue.

DSpace supports resumption tokens for "ListRecords", "ListIdentifiers" and "ListSets" OAI-PMH requests.

Each OAI-PMH ListRecords request will return at most 100 records (by default) but it could be configured in the `[dspace]/config/crosswalks/oai/xoai.xml` file.

When a resumption token is issued, the optional *completeListSize* and *cursor* attributes are included. OAI 2.0 resumption tokens are persistent, so *expirationDate* of the resumption token is undefined, they do not expire.

Resumption tokens contain all the state information required to continue a request.

OAI 2.0 Server

1 Introduction
- 1.1 What is OAI 2.0?
- 1.2 Why OAI 2.0?
- 1.3 Concepts (XOAI Core Library)

2 OAI 2.0
- 2.1 Indexing OAI content
  - 2.1.1 OAI Manager
  - 2.1.2 Scheduled Tasks
- 2.2 Client-side stylesheet
- 2.3 Metadata Formats
- 2.4 Encoding problems

3 Configuration
- 3.1 Basic Configuration
- 3.2 Advanced Configuration
  - 3.2.1 General options
  - 3.2.2 Add/Remove Metadata Formats
  - 3.2.3 Add/Remove Metadata Fields

4 Driver/OpenAIRE compliance
- 4.1 Driver Compliance
- 4.2 OpenAIRE compliance

5 Sanity check your OAI interface with the OAI Validator

Introduction

Open Archives Initiative Protocol for Metadata Harvesting is a low-barrier mechanism for repository interoperability. Data Providers are repositories that expose structured metadata via OAI-PMH. Service Providers then make OAI-PMH service requests to harvest that metadata. OAI-PMH is a set of six verbs or services that are invoked within HTTP.

What is OAI 2.0?

OAI 2.0 is a Java implementation of an OAI-PMH data provider interface (originally developed by Lyncode) that uses XOAI, an OAI-PMH Java Library.

Why OAI 2.0?

Projects like OpenAIRE have specific metadata requirements (to the published content through the OAI-PMH interface). As the OAI-PMH protocol doesn't establish any frame to these specifics, OAI 2.0 can, in a simple way, have more than one instance of an OAI interface (feature provided by the XOAI core library) so one could define an interface for each project. That is the main purpose, although, OAI 2.0 allows much more than that.

Concepts (XOAI Core Library)

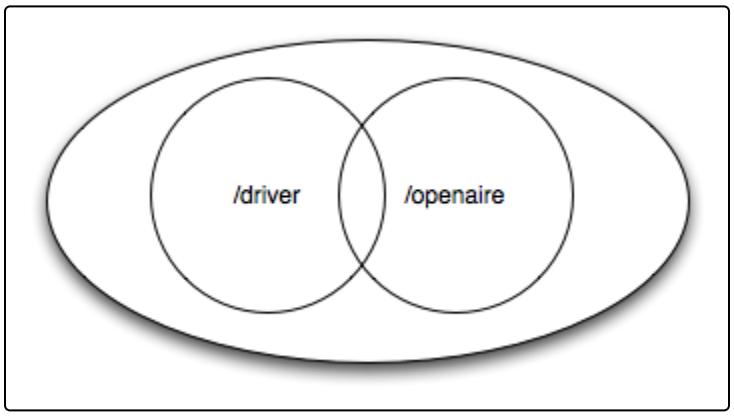
To understand how XOAI works, one must understand the concept of Filter, Transformer and Context. With a Filter it is possible to select information from the data source. A Transformer allows one to make some changes in the metadata before showing it in the OAI interface. XOAI also adds a new concept to the OAI-PMH basic specification, the concept of context. A context is identified in the URL:

```
http://www.example.com/oai/<context>
```

Contexts could be seen as virtual distinct OAI interfaces, so with this one could have things like:

- http://www.example.com/oai/request
- http://www.example.com/oai/driver
- http://www.example.com/oai/openaire

With this ingredients it is possible to build a robust solution that fulfills all requirements of *Driver*, *OpenAIRE* and also other project-specific requirements. As shown in Figure 1, with contexts one could select a subset of all available items in the data source. So when entering the *OpenAIRE* context, all OAI-PMH request will be restricted to that subset of items.



At this stage, contexts could be seen as sets (also defined in the basic OAI-PMH protocol). The magic of XOAI happens when one need specific metadata format to be shown in each context. Metadata requirements by *Driver* slightly differs from the *OpenAIRE* ones. So for each context one must define its specific transformer. So, contexts could be seen as an extension to the concept of sets.

To implement an OAI interface from the XOAI core library, one just need to implement the datasource interface.

OAI 2.0

OAI 2.0 is deployed as a part of the DSpace server (backend) webapp. OAI 2.0 has a configurable data source, by default it will not query the DSpace SQL database at the time of the OAI-PMH request. Instead, it keeps the required metadata in its Solr index (currently in a separate "oai" Solr core) and serves it from there. It's also possible to set OAI 2.0 to only use the database for querying purposes if necessary, but this decreases performance significantly. Furthermore, it caches the requests, so doing the same query repeatedly is very fast. In addition to that it also compiles DSpace items to make uncached responses much faster.

Details about OAI 2.0 internals can be found here.

The OAI 2.0 Server only uses Solr for its indexing. The previous capability to use Database indexing has been removed.

Indexing OAI content

OAI 2.0 uses Solr for all indexing of content.

The Solr index can be updated at your convenience, depending on how fresh you need the information to be. Typically, the administrator sets up a nightly cron job to update the Solr index from the SQL database.

OAI Manager

OAI manager is a utility that allows one to do certain administrative operations with OAI. You can call it from the command line using the dspace launcher:

Syntax

[dspace]/bin/dspace oai <action> [parameters]

Actions

- import *Imports DSpace items into OAI Solr index (also cleans OAI cache)*
- clean-cache *Cleans the OAI cache*

Parameters

- -c *Clears the Solr index before indexing (it will import all items again)*
- -v *Verbose output*
- -h *Shows an help text*

Scheduled Tasks

In order to refresh the OAI Solr index, it is required to run the `[dspace]/bin/dspace oai import` command periodically. You can add the following task to your crontab:

```
0 3 * * * [dspace]/bin/dspace oai import
```

Note that `[dspace]` should be replaced by the correct value, that is, the value defined in `dspace.cfg` parameter `dspace.dir`.

Client-side stylesheet

The OAI-PMH response is an XML file. While OAI-PMH is primarily used by harvesting tools and usually not directly by humans, sometimes it can be useful to look at the OAI-PMH requests directly - usually when setting it up for the first time or to verify any changes you make. For these cases, XOAI provides an XSLT stylesheet to transform the response XML to a nice looking, human-readable and interactive HTML. The stylesheet is linked from the XML response and the transformation takes place in the user's browser (this requires a recent browser, older browsers will only display the XML directly). Most automated tools are interested only in the XML file itself and will not perform the transformation. If you want, you can change which stylesheet will be used by placing it into the `[dspace]/webapps/oai/static` directory (or into the `[dspace-src]/dspace-xoai/dspace-xoai-webapp/src/main/webapp/static` after which you have to rebuild DSpace), modifying the "stylesheet" attribute of the "Configuration" element in `[dspace]/config/crosswalks/oai/xoai.xml` and restarting your servlet container.

Metadata Formats

By default OAI 2.0 provides 12 metadata formats within the /request context:

1. OAI_DC
2. DIDL
3. DIM
4. ETDMS
5. METS
6. MODS
7. OAI-ORE
8. QDC
9. RDF
10. MARC
11. UKETD_DC
12. XOAI

At /driver context it provides:

1. OAI_DC
2. DIDL
3. METS

At /openaire context it provides:

1. OAI_DC
2. METS

And at /openaire4 context it provides:

1. OAI_OPENAIRE

Encoding problems

There are two main potential sources of encoding problems:

a) The servlet connector port has to use the correct encoding. E.g. for Tomcat, this would be `<Connector port="8080" ... URIEncoding="UTF-8" />`, where the port attribute specifies port of the connector that DSpace is configured to access Solr on (this is usually 8080, 80 or in case of AJP 8009).

b) System locale of the dspace command line script that is used to do the oai import. Make sure the user account launching the script (usually from cron) has the correct locale set (e.g. en_US.UTF-8). Also make sure the locale is actually present on your system.

Configuration

Basic Configuration

Configuration File:	**`[dspace]/config/modules/oai.cfg`**
Property:	oai.enabled
Example Value:	`oai.enabled = true (default)`
Information Note:	Allows you to enable or disable the OAI module/endpoint
Property:	oai.path
Example Value:	`oai.path = oai (default)`
Information Note:	Allows you to specify the path where the OAI module will be deployed. This path is relative to the dspace.server.url. So, for example, if "dspace.server.url=http://localhost:8080/server", then by default the OAI module is available at http://localhost:8080/server/oai/
Property:	`oai.storage`

Example Value:	`oai.storage = solr`
Information Note:	This allows to choose the OAI data source between solr and database. ONLY "solr" is supported at this time.
Property:	`oai.solr.url`
Example Value:	`oai.solr.url = ${solr.server}/oai`
Informational Note:	Solr Server location
Property:	`oai.identifier.prefix`
Example Value:	`oai.identifier.prefix = ${dspace.hostname}`
Informational Note:	OAI persistent identifier prefix. Format - oai:PREFIX:HANDLE
Property:	`oai.config.dir`
Example Value:	`oai.config.dir = ${dspace.dir}/config/crosswalks/oai`
Informational Note:	Configuration directory, used by XOAI (core library). Contains xoai.xml, metadata format XSLTs and transformer XSLTs.
Property:	`oai.cache.enabled`
Example Value:	`oai.cache.enabled = true`
Informational Note:	Whether to enable the OAI cache. Default is true (for better performance).
Property:	`oai.cache.dir`
Example Value:	`oai.cache.dir = ${dspace.dir}/var/oai`
Informational Note:	Directory to store runtime generated files (for caching purposes).

Advanced Configuration

OAI 2.0 allows you to configure following advanced options:

- Contexts
- Transformers
- Metadata Formats
- Filters
- Sets

It's an XML file commonly located at: **[dspace]/config/crosswalks/oai/xoai.xml**

General options

These options influence the OAI interface globally. "per page" means per request, next page (if there is one) can be requested using resumptionToken provided in current page.

- identation [boolean] - whether the output XML should be indented to make it human-readable
- maxListIdentifiersSize [integer] - how many identifiers to show per page (verb=ListIdentifiers)
- maxListRecordsSize [integer] - how many records to show per page (verb=ListRecords)
- maxListSetsSize [integer] - how many sets to show per page (verb=ListSets)
- stylesheet [relative file path] - an xsl stylesheet used by client's web browser to transform the output XML into human-readable HTML

Their location and default values are shown in the following fragment:

```
<Configuration xmlns="http://www.lyncode.com/XOAIConfiguration"
        identation="false"
        maxListIdentifiersSize="100"
        maxListRecordsSize="100"
        maxListSetsSize="100"
        stylesheet="static/style.xsl">
```

Add/Remove Metadata Formats

Each context could have its own metadata formats. So to add/remove metadata formats to/from it, just need add/remove its reference within xoai.xml, for example, imagine one need to remove the XOAI schema from:

```
<Context baseurl="request">
  <Format refid="oaidc" />
  <Format refid="mets" />
  <Format refid="xoai" />
  <Format refid="didl" />
  <Format refid="dim" />
  <Format refid="ore" />
  <Format refid="rdf" />
  <Format refid="etdms" />
  <Format refid="mods" />
  <Format refid="qdc" />
  <Format refid="marc" />
  <Format refid="uketd_dc" />
</Context>
```

Then one would have:

```
<Context baseurl="request">
  <Format refid="oaidc" />
  <Format refid="mets" />
  <Format refid="didl" />
  <Format refid="dim" />
  <Format refid="ore" />
  <Format refid="rdf" />
  <Format refid="etdms" />
  <Format refid="mods" />
  <Format refid="qdc" />
  <Format refid="marc" />
  <Format refid="uketd_dc" />
</Context>
```

It is also possible to create new metadata format by creating a specific XSLT for it. All already defined XSLT for DSpace can be found in the **[dspace]/config/crosswalks/oai/metadataFormats** directory. So after producing a new one, add the following information (location marked using brackets) inside the <*Formats*> element in [dspace]/config/crosswalks/oai/xoai.xml:

```
<Format id="[IDENTIFIER]">
  <Prefix>[PREFIX]</Prefix>
  <XSLT>metadataFormats/[XSLT]</XSLT>
  <Namespace>[NAMESPACE]</Namespace>
  <SchemaLocation>[SCHEMA_LOCATION]</SchemaLocation>
</Format>
```

where:

Parameter	Description
IDENTIFIER	The identifier used within context configurations to reference this specific format, must be unique within all Metadata Formats available.
PREFIX	The prefix used in OAI interface (metadataPrefix=PREFIX).
XSLT	The name of the XSLT file within **[dspace]/config/crosswalks/oai/metadataFormats** directory
NAMESPACE	XML Default Namespace of the created Schema
SCHEMA_LOCATION	URI Location of the XSD of the created Schema

NOTE: Changes in [dspace]/config/crosswalks/oai/xoai.xml requires reloading/restarting the servlet container.

Add/Remove Metadata Fields

The internal DSpace fields (Dublin Core) are exposed in the internal XOAI format (xml). All other metadata formats exposed via OAI are mapped from this XOAI format using XSLT (xoai.xsl itself is just an identity transformation). These XSLT stylesheets are found in the **[dspace]/config/crosswalks/oai/metadataFormats** directory. So e.g. oai_dc.xsl is a transformation from the XOAI format to the oai_dc format (unqualified Dublin Core).

Therefore exposing any DSpace metadata field in any OAI format is just a matter of modifying the corresponding output format stylesheet (This assumes the general knowledge of how XSLT works. For a tutorial, see e.g. http://www.w3schools.com/xsl/).

For example, if you have a DC field "local.note.librarian" that you want to expose in oai_dc as <dc:note> (please note that this is not a valid DC field and thus breaks compatibility), then edit oai_dc.xsl and add the following lines just above the closing tag </oai_dc:dc>:

```
<xsl:for-each select="doc:metadata/doc:element[@name='local']/doc:element[@name='note']/doc:element/doc:element
/doc:field[@name='librarian']">
    <dc:note><xsl:value-of select="." /></dc:note>
</xsl:for-each>
```

If you need to add/remove metadata fields, you're changing the output format. Therefore it is recommended to create a new metadata format as a copy of the one you want to modify. This way the old format will remain available along with the new one and any upgrades to the original format during DSpace upgrades will not overwrite your customizations. If you need the format to have the same name as the original format (e.g. the default oai_dc format), you can create a new context in xoai.xsl containing your modified format with the original name, which will be available as /oai/context-name.

NOTE: Please, keep in mind that the OAI provider caches the transformed output, so you have to run `[dspace]/bin/dspace oai clean-cache` after any .xsl modification and reload the OAI page for the changes to take effect. When adding/removing metadata formats, making changes in [dspace]/config /crosswalks/oai/xoai.xml requires reloading/restarting the servlet container.

Driver/OpenAIRE compliance

The default OAI 2.0 installation provides two new contexts. They are:

- Driver context, which only exposes Driver compliant items;
- OpenAIRE context, which only exposes OpenAIRE compliant items;

However, in order to be exposed DSpace items must be compliant with Driver/OpenAIRE guide-lines.

Driver Compliance

DRIVER Guidelines for Repository Managers and Administrators on how to expose digital scientific resources using OAI-PMH and Dublin Core Metadata, creating interoperability by homogenizing the repository output. The OAI-PMH `driver` set is based on DRIVER Guidelines 2.0.

This set is used to expose items of the repository that are available for open access. It's not necessary for all the items of the repository to be available for open access.

What specific metadata values are expected?

To have items in this set, you must configure your `input-forms.xml` file in order to comply with the DRIVER Guidelines:

- Must have a publication date - dc.date.issued (already configured in DSpace items)
- dc.language must use ISO639-3
- the value of dc.type must be one of the 16 types named in the guidelines

How do you easily add those metadata values?

As DRIVER guidelines use Dublin Core, all the needed items are already registered in DSpace. You just need to configure the deposit process.

OpenAIRE compliance

For OpenAIRE v4 compliance, see OpenAIRE4 Guidelines Compliancy

The OpenAIRE Guidelines 2.0 provide the OpenAIRE compatibility to repositories and aggregators. By implementing these Guidelines, repository managers are facilitating the authors who deposit their publications in the repository in complying with the EC Open Access requirements. For developers of repository platforms, the Guidelines provide guidance to add supportive functionalities for authors of EC-funded research in future versions.

The name of the set in OAI-PMH is "ec_fundedresources" and will expose the items of the repository that comply with these guidelines. These guidelines are based on top of DRIVER guidelines. See version 2.0 of the Guidelines.

See the Application Profile of OpenAIRE.

What specific metadata values are expected?

These are the OpenAIRE metadata values only, to check these and driver metadata values check page 11 of the OpenAIRE guidelines 2.0.

- dc:relation with the project ID (see p.8)
- dc:rights with the access rights information from vocabulary (possible values here)

Optionally:

- dc:date with the embargo end date (recommended for embargoed items)

```
<dc:date>info:eu-repo/date/embargoEnd/2011-05-12<dc:date>
```

How do you easily add those metadata values?

- Have a dc:relation field in `input-forms.xml` with a list of the projects. You can also use the OpenAIRE Authority Control Addon to facilitate the process of finding the project.
- Just use a combo-box for dc:rights to input the 4 options:
  - info:eu-repo/semantics/closedAccess
  - info:eu-repo/semantics/embargoedAccess
  - info:eu-repo/semantics/restrictedAccess
  - info:eu-repo/semantics/openAccess
- Use an input-box for dc:date to insert the embargo end date

Relevant Links

- OAI 2.0 is a standard part of DSpace 3.0
- Download & Install OAI 2.0 for DSpace 1.8.x: http://www.lyncode.com/dspace/addons/xoai/

Sanity check your OAI interface with the OAI Validator

There is a very useful validator for OAI interfaces available at http://validator.oaipmh.com, we urge you to use this validator to confirm your OAI interface is in fact usable.

OpenAIRE4 Guidelines Compliancy

Loading of Entities and Fields

OpenAIRE4 features depends on Configurable Entities feature and its default configurations. In order to have your repository compliant with OpenAIRE4 guidelines you need to follow some steps:

The default submission-forms.xml file configures the form fields that allow the creation of the specific OpenAIRE entities and their relationships. In order to use those forms you need to configure your item-submission.xml and add these to the `<submission-map>`:

item-submission.xml

```
<name-map collection-handle="123456789/2" submission-name="openAIREPublicationSubmission" />
<name-map collection-handle="123456789/3" submission-name="openAIREPersonSubmission" />
<name-map collection-handle="123456789/5" submission-name="openAIREProjectSubmission" />
<name-map collection-handle="123456789/4" submission-name="openAIREOrganizationSubmission" />
```

Please note that my `collection-handle="123456789/4"` will be different in your system and it refers to the collection that will gather a specific Entity type like Publications, Persons, Projects or Organizations.

To load OpenAIRE Entities model you must firstly run the following:

loading openaire entity-relationships model

```
[/dspace]/bin/dspace initialize-entities -f [/dspace]/config/entities/openaire4-relationships.xml
```

and load the required metadata fields

loading openaire registries

```
[/dspace]/bin/dspace registry-loader -metadata [/dspace]/config/registries/openaire4-types.xml
[/dspace]/bin/dspace initialize-entities -f [/dspace]/config/entities/datacite-types.xml
```

After those steps your repository will have the required fields and entities for the compliancy.

OAI interface

As decided in our Entities meeting (2019-11-19 DSpace 7 Entities WG Meeting), the XOAI Default Context should only display Publications or non Entity Items. For OpenAIRE4 it will also be considered only Publications as the main Entity to be processed and all the related ones will be loaded in the process.

OpenAIRE4 is accessible in a specific OAI context through the URL:
`http://[dspace-server-url]/oai/openaire4?verb=ListRecords&metadataPrefix=oai_openaire`

in order to use it, you must first ensure you have the oai.cfg setting uncommented:
`oai.enabled = true`
(NOTE: you may need to restart your tomcat service)

If you need to display additional metadata at the oai_openaire metadata format, you could rename the file:

`[/dspace/]config/spring/api/virtual-metadata.xml.openaire`

and replace it with the existing one:

```
[/dspace/]config/spring/api/virtual-metadata.xml
```

Please note if you do this you should restart your tomcat service container.

This additional virtual metadata will enable to represent something like this in this XML in the oai_openaire metadata format, where you have, for instance, author identifiers:

oai datacite:creators example

```
<datacite:creators>
 <datacite:creator>
   <datacite:creatorName>Evans, R.J.</datacite:creatorName>
   <datacite:affiliation>Institute of Science and Technology</datacite:affiliation>
   <datacite:nameIdentifier nameIdentifierScheme="ORCID" schemeURI="http://orcid.org"> 1234-1234-1234-1234 <
/datacite:nameIdentifier>
 </datacite:creator>
</datacite:creators>
```

Then you may need to run the OAI import from the command line with the cleaning cache parameter to reload all data to OAI:

```
[/dspace/]/bin/dspace oai import -c
```

Signposting

Overview

The concept of Signposting is aimed at facilitating machine agents in navigating scholarly information systems easily. Signposting uses typed links to clarify patterns found in scholar portals, offering a standard approach to address the issue of making the descriptive metadata and links in landing pages, usually optimized for human use, readable for machine agents.

To provide machine-friendly authorship information, the publisher can include author links in the Link header of the HTTP response. Additionally, the publisher can use a "cite-as" link to fetch the persistent identifier of the resource. These links enable bots to follow them and discover relevant additional information related to the resource.

By adopting Signposting techniques, the users contribute to improving the machine accessibility and navigation of scholarly web resources, enhancing the overall efficiency and interoperability of scholarly information systems.

More information can be found on the Signposting website: https://signposting.org/

DSpace supports FAIR Signposting Profile at Level 2: By supporting the FAIR Signposting Profile at Level 2, your platform demonstrates a commitment to improving the machine accessibility, interoperability, and reusability of scholarly resources. It ensures that the information you provide is standardized, consistent, and easily navigable by both human users and machine agents, contributing to a more efficient and FAIR scholarly web ecosystem. More information on: https://github.com/DSpace/RestContract/blob/main/signposting.md

The FAIR Signposting profile (more information on: https://signposting.org/FAIR/) is based on the FAIR principles (Findable, Accessible, Interoperable, and Reusable - https://www.go-fair.org/fair-principles/).

- Findability: Your system ensures that scholarly resources are easily discoverable by both humans and machines. It includes the use of persistent identifiers, such as DOIs (Digital Object Identifiers), to uniquely identify and locate resources. These identifiers are included in the signposting links provided in the HTTP responses.
- Accessibility: Your system supports accessibility by providing machine-readable metadata and links that facilitate automated processing. The Signposting Patterns specified in the profile guide the inclusion of links in the HTTP Link headers, HTML link elements, or Link Sets. These links convey essential information about the resource, such as authorship, identifiers, and relationships to other resources.
- Interoperability: Your system promotes interoperability by adopting standardized formats and protocols. It ensures that the signposting links and metadata adhere to established conventions and vocabularies, making it easier for machines to interpret and process the information consistently. By implementing the FAIR Signposting Profile, your system aligns with a community-accepted standard for interoperability.
- Reusability: Your system supports reusability by providing clear and structured metadata about scholarly resources. This includes information about licenses, permissions, and terms of use. By including this information in the signposting links or associated metadata, your system enables users and machines to understand the conditions under which the resources can be reused.

Enabling / Disabling

Signposting is enabled by default in DSpace 7 (starting with version 7.6). When enabled on the backend, the `${dspace.server.url}/signposting/` REST Endpoint will be available and can be used based on the documentation at https://github.com/DSpace/RestContract/blob/main/signposting.md. When disabled, this endpoint will return a 404.

However, if you wish to disable it, you can change this configuration in your local.cfg

```
signposting.enabled = false
```

Modifications to this setting require rebooting your servlet container (e.g. Tomcat)

Configuration

Additional signposting configuration options are available in `[dspace]/config/modules/signposting.cfg`. For most sites, the default settings should be all you need.

Ingesting Content and Metadata

This is a new top level page grouping all documentation concerning all different ways to ingest content and metadata into DSpace

- Ingesting HTML Archives
- SWORDv2 Server
- SWORDv1 Server
- Exporting and Importing Community and Collection Hierarchy
- Importing Items via basic bibliographic formats (Endnote, BibTex, RIS, CSV, etc) and online services (arXiv, PubMed, CrossRef, CiNii, etc)
- Registering Bitstreams via Simple Archive Format
- Importing and Exporting Items via Simple Archive Format (SAF)
- Importing and Exporting Content via Packages
- Configurable Workflow
- Submission User Interface

The section on **Batch Metadata Editing** also contains information on how to add items through spreadsheet ingest.

Ingesting HTML Archives

Not yet supported in DSpace 7. See https://github.com/DSpace/DSpace/issues/8635

For the most part, at present DSpace simply supports uploading and downloading of bitstreams as-is. This is fine for the majority of commonly-used file formats – for example PDFs, Microsoft Word documents, spreadsheets and so forth. HTML documents (Web sites and Web pages) are far more complicated, and this has important ramifications when it comes to digital preservation:

- Web pages tend to consist of several files – one or more HTML files that contain references to each other, and stylesheets and image files that are referenced by the HTML files.
- Web pages also link to or include content from other sites, often imperceptibly to the end-user. Thus, in a few year's time, when someone views the preserved Web site, they will probably find that many links are now broken or refer to other sites than are now out of context.In fact, it may be unclear to an end-user when they are viewing content stored in DSpace and when they are seeing content included from another site, or have navigated to a page that is not stored in DSpace. This problem can manifest when a submitter uploads some HTML content. For example, the HTML document may include an image from an external Web site, or even their local hard drive. When the submitter views the HTML in DSpace, their browser is able to use the reference in the HTML to retrieve the appropriate image, and so to the submitter, the whole HTML document appears to have been deposited correctly. However, later on, when another user tries to view that HTML, their browser might not be able to retrieve the included image since it may have been removed from the external server. Hence the HTML will seem broken.
- Often Web pages are produced dynamically by software running on the Web server, and represent the state of a changing database underneath it.

Dealing with these issues is the topic of much active research. Currently, DSpace bites off a small, tractable chunk of this problem. DSpace can store and provide on-line browsing capability for *self-contained, non-dynamic* HTML documents. DSpace allows relative links between HTML documents stored together in a single item to work. In practical terms, this means:

- No dynamic content (CGI scripts and so forth)
- All links to preserved content must be *relative links*, that do not refer to 'parents' above the 'root' of the HTML document/site:
  - *diagram.gif* is OK
  - *image/foo.gif* is OK
  - *../index.html* is only OK in a file that is at least a directory deep in the HTML document/site hierarchy
  - */stylesheet.css* is not OK (the link will break)
  - *http://somedomain.com/content.html* is not OK (the link will continue to link to the external site which may change or disappear)
- Any 'absolute links' (e.g. *http://somedomain.com/content.html*) are stored 'as is', and will continue to link to the external content (as opposed to relative links, which will link to the copy of the content stored in DSpace.) Thus, over time, the content referred to by the absolute link may change or disappear.

SWORDv2 Server

SWORD (Simple Web-service Offering Repository Deposit) is a protocol that allows the remote deposit of items into repositories. DSpace implements the SWORD protocol via the 'sword' web application. The specification and further information can be found at http://swordapp.org/.

SWORD is based on the Atom Publish Protocol and allows service documents to be requested which describe the structure of the repository, and packages to be deposited.

1 Enabling SWORD v2 Server
2 Configuring SWORD v2 Server
3 Deposit to SWORDv2 Server
    3.1 Other example SWORDv2 commands
4 Troubleshooting
    4.1 Missing expression of encoding in XML header
5 DSpace Demo Server: Service Documents

Enabling SWORD v2 Server

To enable DSpace's SWORD v2 server, set the following in your local.cfg:

```
swordv2-server.enabled = true
# Optionally, if you wish to change its path
swordv2-server.path = swordv2
```

Keep in mind, modifying these settings will require restarting your Servlet Container (usually Tomcat).

Once enabled, the SWORDv2 module will be available at ${dspace.server.url}/${swordv2-server.path}. For example, if "dspace.server.url=http://localhost:8080/server", then (by default) it will be available at http://localhost:8080/server/swordv2/

Configuring SWORD v2 Server

These are the SWORD (v2) configurations. They may be edited directly or overridden in your local.cfg config (see Configuration Reference).

Configuration File:	**`[dspace]/config/modules/swordv2-server.cfg`**
Property:	`swordv2-server.enabled`
Example Value:	`swordv2-server.enabled = true  (default is false)`
Informational Note:	Whether SWORDv2 module is enabled or disabled (disabled by default). Modifying this setting will require restarting your Servlet Container (usually Tomcat).
Property:	`swordv2-server.path`
Example Value:	`swordv2-server.path = swordv2`
Informational Note:	When enabled, this is the subpath where the SWORDv2 module is deployed. This path is relative to `${dspace.server.url}`. Modifying this setting will require restarting your Servlet Container (usually Tomcat).
Property:	`swordv2-server.url`
Example Value:	`swordv2-server.url = ${dspace.server.url}/${swordv2-server.path}`
Informational Note:	The base url of the SWORD 2.0 system. This defaults to `${dspace.server.url}/${swordv2-server.path}`
Property:	`swordv2-server.collection.url`
Example Value:	`swordv2-server.collection.url = http://www.myu.ac.uk/swordv2/collection`
Informational Note:	The base URL of the SWORD collection. This is the URL from which DSpace will construct the deposit location URLs for collections. This defaults to `${dspace.server.url}/${swordv2-server.path}/collection`
Property:	`swordv2-server.servicedocument.url`
Example Value:	`swordv2-server.servicedocument.url = http://www.myu.ac.uk/swordv2/servicedocument`
Informational Note:	The service document URL of the SWORD collection. The base URL of the SWORD service document. This is the URL from which DSpace will construct the service document location urls for the site, and for individual collections. This defaults to `${dspace.server.url}/${swordv2-server.path}/servicedocument`

Property:	`swordv2-server.accept-packaging.collection`

Example Value:	`swordv2-server.accept-packaging.collection.METSDSpaceSIP = http://purl.org/net/sword/package/METSDSpaceSIP`
	`swordv2-server.accept-packaging.collection.SimpleZip = http://purl.org/net/sword/package/SimpleZip`
	`swordv2-server.accept-packaging.collection.Binary = http://purl.org/net/sword/package/Binary`

Informational Note:	The accept packaging properties, along with their associated quality values where appropriate. Package format information • METSDSpaceSIP: zipfile containing mets.xml file describing the resources packed together with it in the root of the zipfile. • Binary: Binary resource that should be taken in as-is, not unpacked • SimpleZip: Zip file that should be unpacked and each file in the zip should be ingested separately. No metadata provided /ingested.
Property:	`swordv2-server.accept-packaging.item`

Example Value:	`swordv2-server.accept-packaging.item.METSDSpaceSIP = http://purl.org/net/sword/package/METSDSpaceSIP`
	`swordv2-server.accept-packaging.item.SimpleZip = http://purl.org/net/sword/package/SimpleZip`
	`swordv2-server.accept-packaging.item.Binary = http://purl.org/net/sword/package/Binary`

Informational Note:	The accept packaging properties for items. It is possible to configure this for specific collections by adding the handle of the collection to the setting, for example `swordv2-server.accept-packaging.collection.[handle].METSDSpaceSIP = http://purl.org/net/sword-types/METSDSpaceSIP` Package format information • METSDSpaceSIP: zipfile containing mets.xml file describing the resources packed together with it in the root of the zipfile. • Binary: Binary resource that should be taken in as-is, not unpacked • SimpleZip: Zip file that should be unpacked and each file in the zip should be ingested separately. No metadata provided /ingested.
Property:	`swordv2-server.accepts`
Example Value:	`swordv2-server.accepts = application/zip, image/jpeg`
Informational Note:	A comma-separated list of MIME types that SWORD will accept. To accept all mimetypes, the value can be set to "*/*"
Property:	`swordv2-server.expose-communities`
Example Value:	`swordv2-server.expose-communities = false`
Informational Note:	Whether or not the server should expose a list of all the communities to a service document request. As deposits can only be made into a collection, it is recommended to leave this set to false.
Property:	`swordv2-server.max-upload-size`
Example Value:	`swordv2-server.max-upload-size = 0`
Informational Note:	The maximum upload size of a package through the SWORD interface (measured in bytes). This will be the combined size of all the files, metadata, and manifest file in a package - this is different to the maximum size of a single bitstream. If this is set to 0, no maximum file size will be enforced.
Property:	`swordv2-server.keep-original-package`

Example Value:	`swordv2-server.keep-original-package = true`
Informational Note:	Should DSpace store a copy of the orignal SWORD deposit package? This will cause the deposit process to be slightly slower and for more disk to be used, however original files will be preserved. It is recommended to leave this option enabled.
Property:	`swordv2-server.bundle.name`
Example Value:	`swordv2-server.bundle.name = SWORD`
Informational Note:	The bundle name that SWORD should store incoming packages within if `swordv2-server.keep-original-package` is set to true.
Property:	`swordv2-server.bundle.deleted`
Example Value:	`swordv2-server.bundle.deleted = DELETED`
Informational Note:	The bundle name that SWORD should use to store deleted bitstreams if `swordv2-server.versions.keep` is set to true. This will be used in the case that individual files are updated or removed via SWORD. If the entire Media Resource (files in the ORIGINAL bundle) is removed this will be backed up in its entirety in a bundle of its own
Property:	`swordv2-server.keep-package-on-fail`
Example Value:	`swordv2-server.keep-package-on-fail = false`
Informational Note:	In the event of package ingest failure, provide an option to store the package on the file system. The default is false. The location can be set using the `swordv2-server.failed-package-dir` setting.
Property:	`swordv2-server.failed-package-dir`
Example Value:	`swordv2-server.failed-package-dir = /dspace/upload`
Informational Note:	If `swordv2-server.keep-package-on-fail` is set to true, this is the location where the package would be stored.
Property:	`swordv2-server.on-behalf-of.enable`
Example Value:	`swordv2-server.on-behalf-of.enable = true`
Informational Note:	Should DSpace accept mediated deposits? See the SWORD specification for a detailed explanation of deposit On-Behalf-Of another user.
Property:	`swordv2-server.on-behalf-of.update.mediators`
Example Value:	`swordv2-server.on-behalf-of.update.mediators = admin@mydspace.edu, mediator@mydspace.edu`
Informational Note:	Which user accounts are allowed to do updates on items which already exist in DSpace, on-behalf-of other users? If this is left blank, or omitted, then all accounts can mediate updates to items, which could be a security risk, as there is no implicit checking that the authenticated user is a "legitimate" mediator
Property:	`swordv2-server.verbose-description.receipt.enable`

Example Value:	`swordv2-server.verbose-description.receipt.enable = false`
Informational Note:	Should the deposit receipt include a verbose description of the deposit? For use by developers - recommend to set to "false" for production systems
Property:	`swordv2-server.verbose-description.error.enable`
Example Value:	`swordv2-server.verbose-description.error.enable = true`
Informational Note:	should the error document include a verbose description of the error? For use by developers, although you may also wish to leave this set to "true" for production systems
Property:	`swordv2-server.error.alternate.url`
Example Value:	`swordv2-server.error.alternate.url = http://mydspace.edu/contact`
Informational Note:	The error document can contain an alternate url, which the client can use to follow up any issues. For example, this could point to the Contact-Us page
Property:	`swordv2-server.error.alternate.content-type`
Example Value:	`swordv2-server.error.alternate.content-type = text/html`
Informational Note:	The `swordv2-server.error.alternate.url` may have an associated content type, such as `text/html` if it points to a web page. This is used to indicate to the client what content type it can expect if it follows that url.
Property:	`swordv2-server.generator.url`
Example Value:	`swordv2-server.generator.url = http://www.dspace.org/ns/sword/2.0/`
Informational Note:	The URL which identifies DSpace as the software that is providing the SWORD interface.
Property:	`swordv2-server.generator.version`
Example Value:	`swordv2-server.generator.version = 2.0`
Informational Note:	The version of the SWORD interface.
Property:	`swordv2-server.auth-type`
Example Value:	`swordv2-server.auth-type = Basic`
Informational Note:	Which form of authentication to use. Normally this is set to `Basic` in order to use HTTP Basic.
Property:	`swordv2-server.upload.tempdir`
Example Value:	`swordv2-server.upload.tempd = /dspace/upload`
Informational Note:	The location where uploaded files and packages are stored while being processed.

Property:	`swordv2-server.updated.field`
Example Value:	`swordv2-server.updated.field = dc.date.updated`
Informational Note:	The metadata field in which to store the updated date for items deposited via SWORD.
Property:	`swordv2-server.slug.field`
Example Value:	`swordv2-server.slug.field = dc.identifier.slug`
Informational Note:	The metadata field in which to store the value of the slug header if it is supplied.
Property:	`swordv2-server.author.field`
Example Value:	`swordv2-server.author.field = dc.contributor.author`
Informational Note:	The metadata field in which to store the value of the atom entry author if it supplied.
Property:	`swordv2-server.title.field`
Example Value:	`swordv2-server.title.field = dc.title`
Informational Note:	The metadata field in which to store the value of the atom entry title if it supplied.
Property:	`swordv2-server.disseminate-packaging`
Example Value:	`swordv2-server.disseminate-packaging.METSDSpaceSIP = http://purl.org/net/sword/package/METSDSpaceSIP` `swordv2-server.disseminate-packaging.SimpleZip = http://purl.org/net/sword/package/SimpleZip`
Informational Note:	Supported packaging formats for the dissemination of packages.
Property:	`swordv2-server.statement.bundles`
Example Value:	`swordv2-server.statement.bundles = ORIGINAL, SWORD, LICENSE`
Informational Note:	Which bundles should the Statement include in its list of aggregated resources? The Statement will automatically mark any bitstreams which are in the bundle identified by the `${bundle.name}` property, provided that bundle is also listed here (i.e. if you want Original Deposits to be listed in the Statement then you should add the SWORD bundle to this list)
Property:	`plugin.single.org.dspace.sword2.WorkflowManager`
Example Value:	`plugin.single.org.dspace.sword2.WorkflowManager = org.dspace.sword2.WorkflowManagerDefault`
Informational Note:	Which workflow manager to use.
Property:	`swordv2-server.workflowmanagerdefault.always-update-metadata`
Example Value	`swordv2-server.workflowmanagerdefault.always-update-metadata = true`

Informational Note	Should the WorkflowManagerDefault plugin allow updates to the item's metadata to take place on items which are in states other than the workspace (e.g. in the workflow, archive, or withdrawn) ?
Property:	`swordv2-server.workflowmanagerdefault.file-replace.enable`
Example Value	`swordv2-server.workflowmanagerdefault.file-replace.enable = false`
Informational Note	Should the server allow PUT to individual files? If this is enabled, then DSpace may be used with the DepositMO SWORD extensions, BUT the caveat is that DSpace does not formally support Bitstream replace, so this is equivalent to a DELETE and then a POST, which violates the RESTfulness of the server. The resulting file DOES NOT have the same identifier as the file it was replacing. As such it is STRONGLY RECOMMENDED to leave this option turned off unless working explicitly with DepositMO enabled client environments
Property:	`swordv2-server.mets-ingester.package-ingester`
Example Value:	`swordv2-server.mets-ingester.package-ingester = METS`
Informational Note:	Which package ingester to use for METS packages.
Property:	`swordv2-server.restore-mode.enable`
Example Value:	`swordv2-server.restore-mode.enable = false`
Informational Note:	Should the SWORD server enable restore-mode when ingesting new packages. If this is enabled the item will be treated as a previously deleted item from the repository. If the item has previously been assigned a handle then that same handle will be restored to activity.
Property:	`swordv2-server.simpledc.*`
Example Value:	`swordv2-server.simpledc.abstract = dc.description.abstractsimpledc.date = dc.datesimpledc.rights = dc.rights`
Informational Note:	Configuration of metadata field mapping used by the SimpleDCEntryIngester, SimpleDCEntryDisseminator and FeedContentDisseminator
Property:	`swordv2-server.atom.*`
Example Value	`swordv2-server.atom.author = dc.contributor.author`
Informational Note:	Configuration of metadata field mapping used by the SimpleDCEntryIngester, SimpleDCEntryDisseminator and FeedContentDisseminator
Property:	`swordv2-server.metadata.replaceable`
Example Value	`swordv2-server.metadata.replaceable = dc.description.abstract, dc.rights, dc.title.alternative, dc.identifier.citation`
Informational Note	Used by SimpleDCEntryIngester: Which metadata fields can be replaced during a PUT to the Item of an Atom Entry document? Fields listed here are the ones which will be removed when a new PUT comes through (irrespective of whether there is a new incoming value to replace them)
Property:	`swordv2-server.multipart.entry-first`
Example Value:	`swordv2-server.multipart.entry-first = false`
Informational Note:	The order of precedence for importing multipart content. If this is set to `true` then metadata in the package will override metadata in the atom entry, otherwise the metadata in the atom entry will override that from the package.
Property:	`swordv2-server.workflow.notify`

Example Value:	`swordv2-server.workflow.notify = true`
Informational Note:	If the workflow gets started (the collection being deposited into has a workflow configured), should a notification get sent?
Property:	`swordv2-server.versions.keep`
Example Value:	`swordv2-server.versions.keep = true`
Informational Note:	When content is replaced, should the old version be kept? This creates a copy of the ORIGINAL bundle with the name V_YYYY-MM-DD.X where YYYY-MM-DD is the date the copy was created, and X is an integer from 0 upwards.
Property:	`swordv2-server.state.*`

Example Value:	`swordv2-server.state.workspace.uri = http://dspace.org/state/inprogress`
	`swordv2-server.state.workspace.description = The item is in the user workspace`
	`swordv2-server.state.workflow.uri = http://dspace.org/state/inreview`
	`swordv2-server.state.workflow.description = The item is undergoing review prior to acceptance in the archive`

Informational Note:	Pairs of states (URI and description) than items can be in. Typical states are `workspace`, `workflow`, `archive`, and `withdrawn`.
Property:	`swordv2-server.workspace.url-template`
Example Value	`swordv2-server.workspace.url-template = http://mydspace.edu/workspaceitems/#wsid#/edit`
Informational Note	URL template for links to items in the workspace (items in the archive will use the handle). The `#wsid#` url parameter will be replaced with the workspace id of the item. The example above shows how to construct this URL for the UI.

Other configuration options exist that define the mapping between mime types, ingesters, and disseminators. A typical configuration looks like this:

```
plugin.named.org.dspace.sword2.SwordContentIngester = \
  org.dspace.sword2.SimpleZipContentIngester = http://purl.org/net/sword/package/SimpleZip, \
  org.dspace.sword2.SwordMETSIngester = http://purl.org/net/sword/package/METSDSpaceSIP, \
  org.dspace.sword2.BinaryContentIngester = http://purl.org/net/sword/package/Binary

plugin.single.org.dspace.sword2.SwordEntryIngester = \
  org.dspace.sword2.SimpleDCEntryIngester

plugin.single.org.dspace.sword2.SwordEntryDisseminator = \
  org.dspace.sword2.SimpleDCEntryDisseminator

# note that we replace ";" with "_" as ";" is not permitted in the PluginManager names
plugin.named.org.dspace.sword2.SwordContentDisseminator = \
  org.dspace.sword2.SimpleZipContentDisseminator = http://purl.org/net/sword/package/SimpleZip, \
  org.dspace.sword2.FeedContentDisseminator = application/atom+xml, \
  org.dspace.sword2.FeedContentDisseminator = application/atom+xml_type_feed

# note that we replace ";" with "_" as ";" is not permitted in the PluginManager names
plugin.named.org.dspace.sword2.SwordStatementDisseminator = \
  org.dspace.sword2.AtomStatementDisseminator = atom, \
  org.dspace.sword2.OreStatementDisseminator = rdf, \
  org.dspace.sword2.AtomStatementDisseminator = application/atom+xml_type_feed, \
  org.dspace.sword2.OreStatementDisseminator = application/rdf+xml
```

Deposit to SWORDv2 Server

If you'd like to deposit content to your repository via the installed SWORD Server, you'll need to select a SWORD Client to do so.

- Some SWORDv2 Clients are available at http://swordapp.org/sword-v2/
- It's possible to simply deposit a valid SWORDv2 Zip package via common Linux commandline tools (e.g. curl). For example:

```
# Deposit a SWORD Zip package named "sword-package.zip" into a DSpace Collection (Handle 123456789/2) as
user "test@dspace.org"
# (Please note that you WILL need to obviously modify the Collection location, user/password and name of
the SWORD package)

curl -i --data-binary "@sword-package.zip" -H "Content-Disposition:attachment; filename=sword-package.
zip" -H "Content-Type:application/zip" -H "Packaging:http://purl.org/net/sword/package/METSDSpaceSIP" -u
test@dspace.org:[password] -X POST http://[dspace.url]/swordv2/collection/123456789/2

# Template 'curl' command:
#curl -i --data-binary "@[zip-package-name]" -H "Content-Disposition:attachment; filename=[zip-package-
name]" -H "Content-Type:application/zip" -H "Packaging:http://purl.org/net/sword/package/METSDSpaceSIP" -
u [user]:[password] -X POST http://[dspace.url]/swordv2/collection/[collection-handle]
```

  NOTE: `-H "Packaging:http://purl.org/net/sword/package/METSDSpaceSIP"` is required for SWORDv2 and `-H "X-Packaging:http://purl.org/net/sword-types/METSDSpaceSIP"` is required for SWORD. X-Packaging/Packaging and the URLs are different.
- A sample SWORD Zip package (which works for both SWORDv1 or SWORDv2) is available in the codebase at https://github.com/DSpace/DSpace/tree/main/dspace-sword/example. Look for the file named example.zip

Other example SWORDv2 commands

```
# Example of retrieving Item information via "edit-media" path in ATOM format (can be run on any item within
DSpace, but requires authentication)
# NOTE: Accept header is required, and must be a format supported by a SwordContentDisseminator plugin (see
configuration above)
curl -i -H "Accept:application/atom+xml" -u test@dspace.org:[password] -X GET http://[dspace.url]/swordv2/edit-
media/[internal-item-identifier]
```

Troubleshooting

Missing expression of encoding in XML header

If your SWORD Deposit requests are unsuccessful, please check that the XML in your initial metadata deposit correctly specifies the encoding.

If you use:

```
<?xml version="1.0"?>
```

DSpace will default to UTF-32.

So to successfully deposit an XML in UTF-8, make sure you use:

```
<?xml version="1.0" encoding="utf-8" ?>
```

DSpace Demo Server: Service Documents

In DSpace, the SWORD interfaces are on the backend, so the correct URLs are

- https://demo.dspace.org/server/sword/servicedocument
- https://demo.dspace.org/server/swordv2/servicedocument

SWORDv1 Server

SWORD (Simple Web-service Offering Repository Deposit) is a protocol that allows the remote deposit of items into repositories. DSpace implements the SWORD protocol via the 'sword' web application. The version of SWORD v1 currently supported by DSpace is 1.3. The specification and further information can be found at http://swordapp.org.

SWORD is based on the Atom Publish Protocol and allows service documents to be requested which describe the structure of the repository, and packages to be deposited.

1 Enabling SWORD Server
2 Configuring SWORD Server
3 Deposit to SWORD Server
4 DSpace Demo Server: Service Documents

Enabling SWORD Server

To enable DSpace's SWORD v1 server, set the following in your local.cfg:

```
sword-server.enabled = true
# Optionally, if you wish to change its path
sword-server.path = sword
```

Keep in mind, modifying these settings will require restarting your Servlet Container (usually Tomcat).

Once enabled, the SWORD v1 module will be available at ${dspace.server.url}/${sword-server.path}. For example, if "dspace.server.url=http://localhost:8080/server", then (by default) it will be available at http://localhost:8080/server/sword/

Configuring SWORD Server

These are the SWORD (v1) configurations. They may be edited directly or overridden in your local.cfg config (see Configuration Reference).

Configuration File:	**`[dspace]/config/modules/sword-server.cfg`**
Property:	`sword-server.enabled`
Example Value:	`sword-server.enabled = true  (default is false)`
Informational Note:	Whether SWORDv1 module is enabled or disabled (disabled by default). Modifying this setting will require restarting your Servlet Container (usually Tomcat).
Property:	`sword-server.path`
Example Value:	`sword-server.path = sword`
Informational Note:	When enabled, this is the subpath where the SWORDv1 module is deployed. This path is relative to `${dspace.server.url}`. Modifying this setting will require restarting your Servlet Container (usually Tomcat).
Property:	`sword-server.mets-ingester.package-ingester`
Example Value:	`sword-server.mets-ingester.package-ingester = METS`
Informational Note:	The property key tell the SWORD METS implementation which package ingester to use to install deposited content. This should refer to one of the classes configured for: `plugin.named.org.dspace.content.packager.PackageIngester` The value of sword.mets-ingester.package-ingester tells the system which named plugin for this interface should be used to ingest SWORD METS packages.

Properties:	`mets.default.ingest.crosswalk.EPDCX`
	`mets.default.ingest.crosswalk.*`
	(NOTE: These configs are in the `dspace.cfg` file as they are used by many interfaces)

Example Value:	`mets.submission.crosswalk.EPDCX = EPDCX`

Informational Note:	Define the metadata types which can be accepted/handled by SWORD during ingest of a package. Currently, EPDCX (EPrints DC XML) is the recommended default metadata format, but others are supported. An example of an EPDCX SWORD package can be found at [dspace-src]/dspace-sword/example/example.zip. Additional metadata types can be added to this list by just defining new configurations. For example, you can map a new "mdtype" MYFORMAT to a custom crosswalk named MYFORMAT: `mets.submission.crosswalk.MYFORMAT = MYFORMAT` You'd also want to map your new custom crosswalk to a stylesheet using the next configuration (crosswalk.submission.*.stylesheet).
Property:	`crosswalk.submission.EPDCX.stylesheet` (NOTE: This configuration is in the `dspace.cfg` file)
Example Value:	`crosswalk.submission.EPDCX.stylesheet = crosswalks/sword-swap-ingest.xsl`
Informational Note:	Define the stylesheet which will be used by the self-named XSLTIngestionCrosswalk class when asked to load the SWORD configuration (as specified above). This will use the specified stylesheet to crosswalk the incoming SWAP metadata to the DIM format for ingestion. Additional crosswalk types can be added to this list by just defining new configurations. For example, you can map a custom crosswalk named MYFORMAT to use a specific "my-crosswalk.xsl" stylesheet: `crosswalk.submission.MYFORMAT.stylesheet = crosswalks/my-crosswalk.xsl` Keep in mind, you'll need to also ensure MYFORMAT crosswalk is defined by the previous configuration (mets.submission.crosswalk.*).
Property:	`sword-server.deposit.url`
Example Value:	`sword-server.deposit.url = http://www.myu.ac.uk/sword/deposit`
Informational Note:	The base URL of the SWORD deposit. This is the URL from which DSpace will construct the deposit location URLs for collections. The default is `${dspace.server.url}/${sword-server.path}/deposit` . In the event that you are not deploying DSpace as the ROOT application in the servlet container, this will generate incorrect URLs, and you should override the functionality by specifying in full as shown in the example value.
Property:	`sword-server.servicedocument.url`
Example Value:	`sword-server.servicedocument.url = http://www.myu.ac.uk/sword/servicedocument`
Informational Note:	The base URL of the SWORD service document. This is the URL from which DSpace will construct the service document location URLs for the site, and for individual collections. The default is `${dspace.server.url}/${sword-server.path}/servicedocument`. In the event that you are not deploying DSpace as the ROOT application in the servlet container, this will generate incorrect URLs, and you should override the functionality by specifying in full as shown in the example value.
Property:	`sword-server.media-link.url`
Example Value:	`sword-server.media-link.url = http://www.myu.ac.uk/sword/media-link`
Informational Note:	The base URL of the SWORD media links. This is the URL which DSpace will use to construct the media link URLs for items which are deposited via sword. The default is `${dspace.server.url}/${sword-server.path}/media-link`. In the event that you are not deploying DSpace as the ROOT application in the servlet container, this will generate incorrect URLs, and you should override the functionality by specifying in full as shown in the example value.
Property:	`sword-server.generator.url`
Example Value:	`sword-server.generator.url = http://www.dspace.org/ns/sword/1.3.1`
Informational Note:	The URL which identifies the SWORD software which provides the sword interface. This is the URL which DSpace will use to fill out the atom:generator element of its atom documents. The default is: http://www.dspace.org/ns/sword/1.3.1 If you have modified your SWORD software, you should change this URI to identify your own version. If you are using the standard 'dspace-sword' module you will not, in general, need to change this setting.
Property:	`sword-server.updated.field`

Example Value:	`sword-server.updated.field = dc.date.updated`
Informational Note:	The metadata field in which to store the updated date for items deposited via SWORD.
Property:	`sword-server.slug.field`
Example Value:	`sword-server.slug.field = dc.identifier.slug`
Informational Note:	The metadata field in which to store the value of the slug header if it is supplied.
Properties:	`sword-server.accept-packaging.METSDSpaceSIP.identifier` `sword-server.accept-packaging.METSDSpaceSIP.q`
Example Value:	`sword-server.accept-packaging.METSDSpaceSIP.identifier = http://purl.org/net/sword-types/METSDSpaceSIP` `sword-server.accept-packaging.METSDSpaceSIP.q = 1.0`
Informational Note:	The accept packaging properties, along with their associated quality values where appropriate. This is a Global Setting; these will be used on all DSpace collections
Property:	`sword-server.accepts`
Example Value:	`sword-server.accepts = application/zip, foo/bar`
Informational Note:	A comma separated list of MIME types that SWORD will accept.
Properties:	`sword-server.accept-packaging.[handle].METSDSpaceSIP.identifier` `sword-server.accept-packaging.[handle].METSDSpaceSIP.q`
Example Value:	`sword-server.accept-packaging.[handle].METSDSpaceSIP.identifier = http://purl.org/net/sword-types/METSDSpaceSIP` `sword-server.accept-packaging.[handle].METSDSpaceSIP.q = 1.0`
Informational Note:	Collection Specific settings: these will be used on the collections with the given handles.
Property:	`sword-server.expose-items`
Example Value:	`sword-server.expose-items = false`
Informational Note:	Should the server offer up items in collections as sword deposit targets. This will be effected by placing a URI in the collection description which will list all the allowed items for the depositing user in that collection on request. **NOTE:** this will require an implementation of deposit onto items, which will not be forthcoming for a short while.
Property:	`sword-server.expose-communities`
Example Value:	`sword-server.expose-communities = false`
Informational Note:	Should the server offer as the default the list of all Communities to a Service Document request. If false, the server will offer the list of all collections, which is the default and recommended behavior at this stage. **NOTE:** a service document for Communities will not offer any viable deposit targets, and the client will need to request the list of Collections in the target before deposit can continue.
Property:	`sword-server.max-upload-size`
Example Value:	`sword-server.max-upload-size = 0`
Informational Note:	The maximum upload size of a package through the sword interface, in bytes. This will be the combined size of all the files, the metadata and any manifest data. It is NOT the same as the maximum size set for an individual file upload through the user interface. If not set, or set to 0, the sword service will default to no limit.
Property:	`sword-server.keep-original-package`
Example Value:	`sword-server.keep-original-package = true`

Informational Note:	Whether or not DSpace should store a copy of the original sword deposit package. **NOTE:** this will cause the deposit process to run slightly slower, and will accelerate the rate at which the repository consumes disk space. BUT, it will also mean that the deposited packages are recoverable in their original form. It is strongly recommended, therefore, to leave this option turned on. When set to "true", this requires that the configuration option `upload.temp.dir` (in `dspace.cfg`) is set to a valid location.
Property:	`sword-server.bundle.name`
Example Value:	`sword-server.bundle.name = SWORD`
Informational Note:	The bundle name that SWORD should store incoming packages under if sword.keep-original-package is set to true. The default is "SWORD" if not value is set
Properties:	`sword-server.keep-package-on-fail` `sword-server.failed-package.dir`
Example Value:	`sword-server.keep-package-on-fail=true` `sword-server.failed-package.dir=${dspace.dir}/upload`
Informational Note:	In the event of package ingest failure, provide an option to store the package on the file system. The default is false.
Property:	`sword-server.identify-version`
Example Value:	`sword-server.identify-version = true`
Informational Note:	Should the server identify the sword version in a deposit response. It is recommended to leave this unchanged.
Property:	`sword-server.on-behalf-of.enable`
Example Value:	`sword-server.on-behalf-of.enable = true`
Informational Note:	Should mediated deposit via sword be supported. If enabled, this will allow users to deposit content packages on behalf of other users.
Property:	`sword-server.restore-mode.enable`
Example Value:	`sword-server.restore-mode.enable = true`
Informational Note:	Should the sword server enable restore-mode when ingesting new packages. If this is enabled the item will be treated as a previously deleted item from the repository. If the item had previously been assigned a handle then that same handle will be restored to activity. If that item had not been previously assign a handle, then a new handle will be assigned.
Property:	`plugin.named.org.dspace.sword.SWORDingester`

Example Value:	`plugin.named.org.dspace.sword.SWORDIngester = \`
	`  org.dspace.sword.SWORDMETSIngester = http://purl.org/net/sword-types/METSDSpaceSIP \`
	`  org.dspace.sword.SimpleFileIngester = SimpleFileIngester`

Informational Note:	Configure the plugins to process incoming packages. The form of this configuration is as per the Plugin Manager's Named Plugin documentation: `plugin.named.[interface] = [implementation] = [package format identifier]` (see `dspace.cfg`). Package ingesters should implement the SWORDIngester interface, and will be loaded when a package of the format specified above in: `sword-server.accept-packaging.[package format].identifier = [package format identifier]` is received. In the event that this is a simple file deposit, with no package format, then the class named by "SimpleFileIngester" will be loaded and executed where appropriate. This case will only occur when a single file is being deposited into an existing DSpace Item.

Deposit to SWORD Server

If you'd like to deposit content to your repository via the installed SWORD Server, you'll need to select a SWORD Client to do so.

- A variety of SWORDv1 Clients (in various languages/tools) are available off of http://swordapp.org/sword-v1/
- DSpacealso comes with an optional SWORDv1 Client which can be enabled to deposit content from one DSpace to another.
- An example SWORDv1 ZIP package is available in the DSpace Codebase at: https://github.com/DSpace/DSpace/tree/main/dspace-sword/example
- Finally, it's also possible to simply deposit a valid SWORD Zip package via common Linux commandline tools (e.g. curl). For example:

```
# Deposit a SWORD Zip package named "sword-package.zip" into a DSpace Collection (Handle 123456789/2) as
user "test@dspace.org"
# (Please note that you WILL need to obviously modify the Collection location, user/password and name of
the SWORD package)

curl -i --data-binary "@sword-package.zip" -H "Content-Disposition:filename=sword-package.zip" -H
"Content-Type:application/zip" -H "X-Packaging:http://purl.org/net/sword-types/METSDSpaceSIP" -u
test@dspace.org:[password] http://[dspace.url]/sword/deposit/123456789/2

# Template 'curl' command:
#curl -i --data-binary "@[zip-package-name]" -H "Content-Disposition:filename=[zip-package-name]" -H
"Content-Type:application/zip" -H "X-Packaging:http://purl.org/net/sword-types/METSDSpaceSIP" -u [user]:
[password] http://[dspace.url]/sword/deposit/[collection-handle
```

DSpace Demo Server: Service Documents

In DSpace, the SWORD interfaces are on the backend, so the correct URLs are

- https://demo.dspace.org/server/sword/servicedocument
- https://demo.dspace.org/server/swordv2/servicedocument

Exporting and Importing Community and Collection Hierarchy

1 Community and Collection Structure Importer
    1.1 Usage
    1.2 XML Import Format
2 Community and Collection Structure Exporter
    2.1 Usage

Community and Collection Structure Importer

This Command-Line tool gives you the ability to import a community and collection structure directory from a source XML file.

Usage

Command used:	`[dspace]/bin/dspace structure-builder`
Java class:	`org.dspace.administer.StructBuilder`
Argument: short and long (if available) forms:	Description of the argument
-f	Source xml file. The presence of this argument engages import mode.
-o	Output xml file. **Required.** A copy of the input augmented with the Handles assigned to each new Community or Collection.
-e	Email of DSpace Administrator. **Required.**

XML Import Format

The administrator need to build the source xml document in the following format:

```
<import_structure>
        <community>
                <name>Community Name</name>
                <description>Descriptive text</description>
                <intro>Introductory text</intro>
                <copyright>Special copyright notice</copyright>
                <sidebar>Sidebar text</sidebar>
                <community>
                   <name>Sub Community Name</name>
                   <community> ...[ad infinitum]...
                   </community>
                </community>
                <collection>
                   <name>Collection Name</name>
                   <description>Descriptive text</description>
                   <intro>Introductory text</intro>
                   <copyright>Special copyright notice</copyright>
                   <sidebar>Sidebar text</sidebar>
                   <license>Special licence</license>
                   <provenance>Provenance information</provenance>
                </collection>
         </community>
</import_structure>
```

The resulting output document will be as follows:

```
<import_structure>
     <community identifier="123456789/1">
                <name>Community Name</name>
                <description>Descriptive text</description>
                <intro>Introductory text</intro>
                <copyright>Special copyright notice</copyright>
                <sidebar>Sidebar text</sidebar>
                <community identifier="123456789/2">
                        <name>Sub Community Name</name>
                        <community identifier="123456789/3"> ...[ad infinitum]...
                        </community>
                </community>
                <collection identifier="123456789/4">
                        <name>Collection Name</name>
                        <description>Descriptive text</description>
                        <intro>Introductory text</intro>
                        <copyright>Special copyright notice</copyright>
                        <sidebar>Sidebar text</sidebar>
                        <license>Special licence</license>
                        <provenance>Provenance information</provenance>
                </collection>
        </community>
</import_structure>
```

This command-line tool gives you the ability to import a community and collection structure directly from a source XML file. It is executed as follows:

```
[dspace]/bin/dspace structure-builder -f /path/to/source.xml -o path/to/output.xml -e admin@user.com
```

This will examine the contents of *source.xml*, import the structure into DSpace while logged in as the supplied administrator, and then output the same structure to the output file, but including the handle for each imported community and collection as an attribute.

Community and Collection Structure Exporter

This command-line tool writes the current Community and Collection structure into an XML document in the format which is read by the importer. See above for format details.

Usage

```
[dspace]/bin/dspace structure-builder [-h] [-?] -x -e <eperson> -o <output>
```

Argument: short and long (if available) forms:	**Description of the argument**
-x, --export	Export the current structure as XML. The presence of this argument engages export mode.
-e, --eperson *email or netid*	User who is manipulating the repository's structure. **Required.** This user's rights determine access to communities and collections.
-o, --output *file path*	The exported structure is written here. **Required.**
-h or -?	Help

Importing Items via basic bibliographic formats (Endnote, BibTex, RIS, CSV, etc) and online services (arXiv, PubMed, CrossRef, CiNii, etc)

1 Introduction
2 Supported External Sources
3 Disabling an External source
4 Submitting starting from external sources
5 Submitting starting from bibliographic file
6 More Information

In DSpace 7.0, the Biblio-Transformation-Engine (BTE) was removed in favor of Live Import from external sources. All online services and bibliographic formats previously supported by BTE have been moved or are being moved to the External Sources framework.

Introduction

This documentation explains the features and the usage of the importer framework. The importer framework is built into both the REST API and User Interface. Currently supported formats include:

- Drag & drop of Endnote, BibTex, RIS, TSV, CSV, arXiv, PubMed. From the MyDSpace page, dragging & dropping one of these files will start a new submission, extracting the metadata from the file.
- Import via ORCID, PubMed, Sherpa Journals, Sherpa Publishers. From the MyDSpace page, you can select to start a new submission by searching an external source.

Supported External Sources

DSpace supports importing metadata from a variety of online services. Some of these services can ONLY support out-of-the-box Configurable Entities.

Supported online services are all configured on the backend in the `[dspace]/config/spring/api/external-services.xml` file. To disable a service, simply comment it out in that file.

Online providers available out of the box include:

- NASA Astrophysics Data System (ADS) lookup (Supported for creating new Items, or "Publication" Entities). Can be configured via "ads.*" settings in external-providers.cfg. *REQUIRES an API key to function, signup at https://ui.adsabs.harvard.edu/help/api/*
- arXiv lookup (Supported for creating new Items, or "Publication" Entities).
- CiNii lookup (Supported for creating new Items, or "Publication" Entities). Can be configured via "cinii.*" settings in external-providers.cfg. *REQUIRES an API key to function, signup at https://support.nii.ac.jp/en/cinii/api/developer*
- CrossRef lookup (Supported for creating new Items, or "Publication" Entities). Can be configured via "crossref.*" settings in external-providers.cfg
- European Patent Office (EPO) lookup (Supported for creating new Items, or "Publication" Entities). Can be configured via "epo.*" settings in external-providers.cfg. *REQUIRES an API key to function, signup at https://developers.epo.org/*
- ORCID
  - ORCID author lookup (Only supported for creating new "Person" Entities). Can be configured via "orcid.*" settings in orcid.cfg.
  - ORCID publication lookup (Supported for creating new Items, or "Publication" Entities). Allows you to lookup a publication based on an author's ORCID. Can be configured via "orcid.*" settings in orcid.cfg.
- PubMed
  - Search PubMed (Supported for creating new Items, or "Publication" Entities). Can be configured via "pubmed.*" settings in external-providers.cfg
  - Search PubMed Europe (Supported for creating new Items, or "Publication" Entities). Can be configured via "pubmedeurope.*" settings in external-providers.cfg
- SciELO lookup (Supported for creating new Items, or "Publication" Entities). Can be configured via "scielo.*" settings in external-providers.cfg.
- Scopus lookup (Supported for creating new Items, or "Publication" Entities). Can be configured via "scopus.*" settings in external-providers.cfg. *REQUIRES an API key to function, signup at https://dev.elsevier.com*
- Sherpa Romeo
  - Sherpa Journals by ISSN (Only supported for creating new "Journal" Entities)
  - Sherpa Journals (Only supported for creating new "Journal" Entities) - supports looking up a Journal by title
  - Sherpa Publishers (Only supported for creating new "OrgUnit" Entities)
- VuFind lookup (Supported for creating new Items, or "Publication" Entities). Can be configured via "vufind.*" settings in external-providers.cfg
- Web of Science lookup (Supported for creating new Items, or "Publication" Entities). Can be configured via "wos.*" settings in external-providers.cfg. *REQUIRES a paid API key to function, signup at https://developer.clarivate.com/apis/wos*
  - Currently this WOS integration requires a paid license and does NOT yet support the WOS Starter API. See this issue ticket for more information: https://github.com/DSpace/DSpace/issues/8695

Disabling an External source

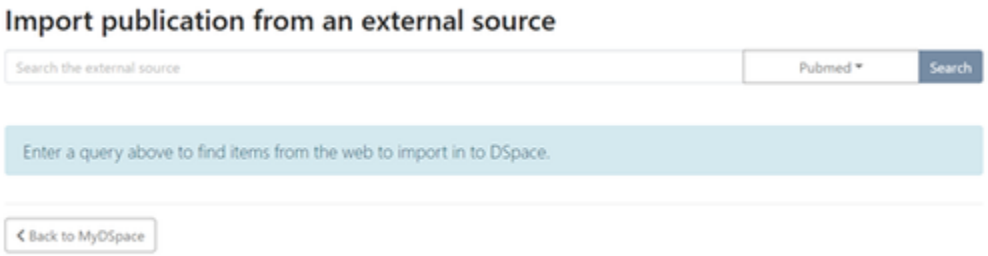
By default, DSpace has all external sources enabled in the `[dspace]/config/spring/api/external-services.xml` file. However, because some external services may require a **paid subscription** , sites may wish to disable any external sources that they do not want to use.

To disable an external source, simply comment out its "`<bean>`" tag in the `external-services.xml` file. Comment it out using XML comment tags (<!-- and -->).

For example, this will disable the Scopus external service (which is one that requires a paid subscription):

```
<!--
<bean id="scopusLiveImportDataProvider" class="org.dspace.external.provider.impl.LiveImportDataProvider">
    <property name="metadataSource" ref="ScopusImportService"/>
    <property name="sourceIdentifier" value="scopus"/>
    <property name="recordIdMetadata" value="dc.identifier.scopus"/>
    <property name="supportedEntityTypes">
        <list>
            <value>Publication</value>
        </list>
    </property>
</bean>
-->
```

Submitting starting from external sources



1. From the MyDSpace page a new submission can be started not only using the submission form but also automatically populating metadata, importing them from several online services.
2. After choosing the external source to import from and inserting a term in search bar, the system will show the list of matching results.
3. When selecting an item, the system will display the metadata to be imported, according to the configured mapping.
4. Clicking on “Start submission” the system will display the submission forms filled with the imported metadata.

Submitting starting from bibliographic file

1. From the MyDSpace page, drag & drop the bibliographic file (e.g. bibtex, endnote, etc) onto the file dropbox
2. Select the collection to submit to
3. Submission will be created, with the metadata parsed out of that bibliographic file. The bibliographic file will also be attached as a Bitstream (in case some fields could not be successfully parsed). You can choose to keep it or delete it.

More Information

More information on configuring metadata mappings for various import formats / services can be found in the Live Import from external sources documentation. See the "Editing Metadata Mapping" section.

Registering Bitstreams via Simple Archive Format

1 Overview
- 1.1 Accessible Storage
- 1.2 Registering Items Using the Item Importer
- 1.3 Internal Identification and Retrieval of Registered Items
- 1.4 Exporting Registered Items
- 1.5 Deleting Registered Items

Registering is not Importing

The procedures below will **not import** the actual bitstreams into DSpace. They will merely inform DSpace of an existing location where these Bitstreams can be found. Please refer to Importing and Exporting Items via Simple Archive Format (SAF) for information on importing metadata and bitstreams.

Overview

Registration is an alternate means of incorporating items, their metadata, and their bitstreams into DSpace by taking advantage of the bitstreams already being in storage accessible to DSpace. An example might be that there is a repository for existing digital assets. Rather than using the normal interactive ingest process or the batch import to furnish DSpace the metadata and to upload bitstreams, registration provides DSpace the metadata and the location of the bitstreams. DSpace uses a variation of the import tool to accomplish registration.

Accessible Storage

To register an item its bitstreams must reside on storage accessible to DSpace and therefore referenced by an asset store number in *dspace.cfg*. The configuration file *dspace.cfg* establishes one or more asset stores through the use of an integer asset store number. This number relates to a directory in the DSpace host's file system or a set of SRB account parameters. This asset store number is described in The *dspace.cfg* Configuration Properties File section and in the *dspace.cfg* file itself. The asset store number(s) used for registered items should generally not be the value of the *assetstore.incoming* property since it is unlikely that you will want to mix the bitstreams of normally ingested and imported items and registered items.

Registering Items Using the Item Importer

DSpace uses the same import tool that is used for batch import except that several variations are employed to support registration. The discussion that follows assumes familiarity with the import tool.

The DSpace Simple Archive Format for registration does not include the actual content files (bitstreams) being registered. The format is however a directory full of items to be registered, with a subdirectory per item. Each item directory contains a file for the item's descriptive metadata (*dublin_core.xml*) and a file listing the item's content files (*contents*), but not the actual content files themselves.

The *dublin_core.xml* file for item registration is exactly the same as for regular item import.

The *contents* file, like that for regular item import, lists the item's content files, one content file per line, but each line has the one of the following formats:

```
-r -s n -f filepath
-r -s n -f filepath\tbundle:bundlename
-r -s n -f filepath\tbundle:bundlename\tpermissions: -[r|w] 'group name'
-r -s n -f filepath\tbundle:bundlename\tpermissions: -[r|w] 'group        name'\tdescription: some text
```

where

- `-r` indicates this is a file to be registered
- `-s n` indicates the asset store number (*n*)
- `-f filepath` indicates the path and name of the content file to be registered (filepath)
- `\t` is a tab character
- `bundle:bundlename` is an optional bundle name
- `permissions: -[r|w] 'group name'` is an optional read or write permission that can be attached to the bitstream
- `description: some text` is an optional description field to add to the file
  The bundle, that is everything after the filepath, is optional and is normally not used.

The command line for registration is just like the one for regular import:

```
[dspace]/bin/dspace import -a -e joe@user.com -c collectionID -s items_dir -m mapfile
```

(or by using the long form)

```
[dspace]/bin/dspace import --add --eperson=joe@user.com --collection=collectionID --source=items_dir --
map=mapfile
```

The `--workflow` and `--test` flags will function as described in Importing Items.

The `--delete` flag will function as described in Importing Items but the registered content files will not be removed from storage. See Deleting Registered Items.

The `--replace` flag will function as described in Importing Items but care should be taken to consider different cases and implications. With old items and new items being registered or ingested normally, there are four combinations or cases to consider. Foremost, an old registered item deleted from DSpace using `--replace` will not be removed from the storage. See Deleting Registered Items. where is resides. A new item added to DSpace using `--replace` will be ingested normally or will be registered depending on whether or not it is marked in the *contents* files with the `-r`.

Internal Identification and Retrieval of Registered Items

Once an item has been registered, superficially it is indistinguishable from items ingested interactively or by batch import. But internally there are some differences:

First, the randomly generated internal ID is not used because DSpace does not control the file path and name of the bitstream. Instead, the file path and name are that specified in the *contents* file.

Second, the *store_number* column of the bitstream database row contains the asset store number specified in the *contents* file.

Third, the *internal_id* column of the bitstream database row contains a leading flag (`-R`) followed by the registered file path and name. For example, `-Rfilepath` where *filepath* is the file path and name relative to the asset store corresponding to the asset store number. The asset store could be traditional storage in the DSpace server's file system or an SRB account.

Fourth, an MD5 checksum is calculated by reading the registered file if it is in local storage.

Registered items and their bitstreams can be retrieved transparently just like normally ingested items.

Exporting Registered Items

Registered items may be exported as described in Exporting Items. If so, the export directory will contain actual copies of the files being exported but the lines in the contents file will flag the files as registered. This means that if DSpace items are "round tripped" (see Transferring Items Between DSpace Instances) using the exporter and importer, the registered files in the export directory will again registered in DSpace instead of being uploaded and ingested normally.

Deleting Registered Items

If a registered item is deleted from DSpace, (either interactively or by using the `--delete` or `--replace` flags described in Importing and Exporting Items via Simple Archive Format (SAF)) the item will disappear from DSpace but its registered content files will remain in place just as they were prior to registration. Bitstreams not registered but added by DSpace as part of registration, such as `license.txt` files, will be deleted.

Importing and Exporting Items via Simple Archive Format (SAF)

1 Item Importer and Exporter
- 1.1 DSpace Simple Archive Format
  - 1.1.1 dublin_core.xml or metadata_[prefix].xml
  - 1.1.2 contents file
  - 1.1.3 relationships file
- 1.2 Configuring metadata_[prefix].xml for a Different Schema
- 1.3 Importing Items
  - 1.3.1 Adding Items to a Collection from a directory
  - 1.3.2 Adding Items to a Collection from a zipfile
  - 1.3.3 Replacing Items in a Collection
  - 1.3.4 Deleting or Unimporting Items in a Collection
  - 1.3.5 Other Options
  - 1.3.6 UI Batch Import
- 1.4 Exporting Items
  - 1.4.1 UI Batch Export

Item Importer and Exporter

DSpace has a set of command line tools for importing and exporting items in batches, using the DSpace Simple Archive Format (SAF). Apart from the offered functionality, these tools serve as an example for users who aim to implement their own item importer.

DSpace Simple Archive Format

The basic concept behind the DSpace's Simple Archive Format (SAF) is to create an archive, which is a directory containing one subdirectory per item. Each item directory contains a file for the item's descriptive metadata, and the files that make up the item.

```
archive_directory/
    item_000/
        dublin_core.xml         -- qualified Dublin Core metadata for metadata fields belonging to the 'dc'
schema.
        metadata_[prefix].xml   -- metadata in another schema.  The prefix is the name of the schema as
registered with the metadata registry.
        contents                -- text file containing one line per filename.
                collections                                 -- (Optional) text file that contains the handles of
the collections the item will belong to. Each handle in a row.
                                                                -- Collection in first line will be the owning
collection.
                handle                                          -- contains the handle assigned/to be assigned to
this resource
        relationships           -- (Optional) If importing Entities, you can specify one or more relationships
to create on import
        file_1.doc              -- files to be added as bitstreams to the item.
        file_2.pdf
    item_001/
        dublin_core.xml
        contents
        file_1.png
        ...
```

dublin_core.xml or metadata_[prefix].xml

The `dublin_core.xml` or `metadata_[prefix].xml` file has the following format, where each metadata element has its own entry within a `<dcvalue>` tagset. There are currently three tag attributes available in the `<dcvalue>` tagset:

- `element` - the Dublin Core element
- `qualifier` - the element's qualifier
- `language` - (optional) ISO language code for element

```
<dublin_core>
    <dcvalue element="title" qualifier="none">A Tale of Two Cities</dcvalue>
    <dcvalue element="date" qualifier="issued">1990</dcvalue>
    <dcvalue element="title" qualifier="alternative" language="fr">J'aime les Printemps</dcvalue>
</dublin_core>
```

(Note the optional language tag attribute which notifies the system that the optional title is in French.)

When providing urls as values for fields that contain the ampersand (&) symbol, the ampersands in these urls have to be encoded as **&**

Every metadata field used, must be registered via the metadata registry of the DSpace instance first. See Metadata and Bitstream Format Registries.

Recommended Metadata

It is recommended to minimally provide "dc.title" and, where applicable, "dc.date.issued". Obviously you can (and should) provide much more detailed metadata about the Item. For more information see: Metadata Recommendations.

contents file

The `contents` file is a plain text document that simply enumerates, one file per line, the bitstream file names. See the following example:

```
file_1.doc
file_2.pdf
license
```

Please notice that the `license` is optional, and if you wish to have one included, you can place the file in the .../item_001/ directory, for example.

The bitstream name may *optionally* be followed by any of the following:

- `\tbundle:BUNDLENAME`
- `\tpermissions:PERMISSIONS`
- `\tdescription:DESCRIPTION`
- `\tprimary:true`

Where '\t' is the Tab character (not a literal "\t" string)

'BUNDLENAME' is the name of the bundle to which the bitstream should be added. Without specifying the bundle, items will go into the default bundle, ORIGINAL.

'PERMISSIONS' is text with the following format: `-[r|w] 'group name'`

- -r = read permissions for this group
- -w = write permissions for this group

'DESCRIPTION' is text of the files description.

Primary is used to specify the primary bitstream.

Supported in 7.2 or above for 'import' only

The IIIF metadata feature was added in 7.2 and is only supported on *import* ('add' mode) of an SAF package.

For IIIF enabled items, the bitstream name may optionally be followed by any of the following:

- `\tiiif-label:IIIFLABEL`
- `\tiiif-toc:IIIFTOC`
- `\tiiif-width:IIIFWIDTH`
- `\tiiif-height:IIIFHEIGHT`

Where:

'IIIFLABEL' is the label that will be used for the image in the viewer.

'IIIFTOC' is the label that will be used for a table of contents entry in the viewer.

'IIIFWIDTH' is the image width that will be used for the IIIF canvas.

'IIIFHEIGHT' is the image height that will be used for the IIIF canvas.

relationships file

Supported in 7.1 or above for 'import' only.

This feature was added in 7.1. Currently the 'relationships' file is only supported on *import* ('add' mode) of an SAF package. See note at bottom of this section about using the "metadata_relation.xml" if you wish to export & update relationships.

The optional `relationships` file enumerates the relationships of this Entity to other Entities (either already in the system, or also specified in your SAF import batch). This allows entities to be linked to new or existing entities during import. Entities can be linked to other entities in this import set by referring to their import subfolder name. Because relationships can only be created for Entities, it can only be used when importing Configurable Entities.

Each line in the file contains a relationship type key and an item identifier in the following format:

```
relation.<relation_key> <handle|uuid|folderName:import_item_folder|schema.element[.qualifier]:value>
```

The input_item_folder should refer the folder name of another item in this import batch. Example:

```
relation.isAuthorOfPublication 5dace143-1238-4b4f-affb-ed559f9254bb
relation.isAuthorOfPublication 123456789/1123
relation.isOrgUnitOfPublication folderName:item_001
relation.isProjectOfPublication project.identifier.id:123
relation.isProjectOfPublication project.identifier.name:A Name with Spaces
```

During initial import, new items are stored in a map keyed by the item folder name. Once the initial import is complete, a second pass checks for a 'relationships' manifest file in each folder and creates a relationship of the specified type to the specified item.

Don't forget Entities require a "dspace.entity.type" metadata field

Remember, if you are creating *new* Entities via an SAF package, those Entities MUST specify a "dspace.entity.type" metadata field. Because this metadata field is in the "dspace" schema, it MUST be specified in a "metadata_dspace.xml", similar to:

metadata_dspace.xml

```
<dublin_core schema="dspace">
  <dcvalue element="entity" qualifier="type">Publication</dcvalue>
</dublin_core>
```

Relationships to existing Entities can also be created via metadata_relation.xml

If you already know the UUID of an existing Entity that you want to relate to, you can also create/update the "metadata_relation.xml" file to add/update the relationship, similar to:

metadata_relation.xml

```
<dublin_core schema="relation">
  <dcvalue element="isAuthorOfPublication">5dace143-1238-4b4f-affb-ed559f9254bb</dcvalue>
</dublin_core>
```

The "relationships" file is primarily for creating relationships between Entities in the *same import batch*. Of course, you can also choose to use the "relationships" file to create new relationships to existing Entities instead of creating/updating the "metadata_relation.xml" file. The main advantage of the "metadata_relation.xml" file is that it is used both on export and import, while the "relationships" file is only used on import at this time.

Configuring `metadata_[prefix].xml` for a Different Schema

It is possible to use other Schema such as EAD, VRA Core, etc. Make sure you have defined the new schema in the DSpace Metadata Schema Registry.

1. Create a separate file for the other schema named `metadata_[prefix].xml`, where the `[prefix]` is replaced with the schema's prefix.
2. Inside the xml file use the same Dublin Core *syntax*, but on the `<dublin_core>` element include the attribute `schema=[prefix]`.
3. Here is an example for ETD metadata, which would be in the file `metadata_etd.xml`:

```
<dublin_core schema="etd">
     <dcvalue element="degree" qualifier="department">Computer Science</dcvalue>
     <dcvalue element="degree" qualifier="level">Masters</dcvalue>
     <dcvalue element="degree" qualifier="grantor">Michigan Institute of Technology</dcvalue>
</dublin_core>
```

Importing Items

Before running the item importer over items previously exported from a DSpace instance, please first refer to Transferring Items Between DSpace Instances.

Command used:	`[dspace]/bin/dspace import`

Java class:	`org.dspace.app.itemimport.ItemImport`
Arguments short and (long) forms:	Description
`-a or --add`	Add items to DSpace ‡
`-r or --replace`	Replace items listed in mapfile ‡
`-d or --delete`	Delete items listed in mapfile ‡
`-s or --source`	Source of the items (directory)
`-c or --collection`	Destination Collection by its Handle or database ID
`-m or --mapfile`	Where the mapfile for items can be found (name and directory)
`-e or --eperson`	Email of eperson doing the importing
`-w or --workflow`	Send submission through collection's workflow
`-n or --notify`	Kicks off the email alerting of the item(s) has(have) been imported
`-v or --validate`	Test run, do not actually import items
`-p or --template`	Apply the collection template
`-R or --resume`	Resume a failed import (Used on Add only)
`-h or --help`	Command help
`-z or --zip`	Name of zipfile

‡ These are mutually exclusive.

The item importer is able to batch import unlimited numbers of items for a particular collection using a very simple CLI command and 'arguments'.

Adding Items to a Collection from a directory

To add items to a collection, you gather the following information:

- eperson
- Collection ID (either Handle (e.g. 123456789/14) or UUID
- Source directory where the items reside
- Mapfile. Since you don't have one, you need to determine where it will be (e.g. /Import/Col_14/mapfile)
  At the command line:

```
[dspace]/bin/dspace import --add --eperson=joe@user.com --collection=CollectionID --source=items_dir --
mapfile=mapfile
```

or by using the short form:

```
[dspace]/bin/dspace import -a -e joe@user.com -c CollectionID -s items_dir -m mapfile
```

The above command would cycle through the archive directory's items, import them, and then generate a map file which stores the mapping of item directories to item handles. **SAVE THIS MAP FILE.** You can use it for replacing or deleting (unimporting) the mapped items.

Testing. You can add `--validate` (or `-v`) to the command to simulate the entire import process without actually doing the import. This is extremely useful for verifying your import files before doing the actual import.

Adding Items to a Collection from a zipfile

To add items to a collection, you gather the following information:

- eperson
- Collection ID (either Handle (e.g. 123456789/14) or Database ID (e.g. 2)
- Source directory where your zipfile containing the items resides
- Zipfile
- Mapfile. Since you don't have one, you need to determine where it will be (e.g. /Import/Col_14/mapfile)
  At the command line:

```
[dspace]/bin/dspace import --add --eperson=joe@user.com --collection=CollectionID --source=zipfile_dir --
zip=filename.zip --mapfile=mapfile
```

or by using the short form:

```
[dspace]/bin/dspace import -a -e joe@user.com -c CollectionID -s zipfile_dir -z filename.zip -m mapfile
```

The above command would unpack the zipfile, cycle through the archive directory's items, import them, and then generate a map file which stores the mapping of item directories to item handles. **SAVE THIS MAP FILE.** You can use it for replacing or deleting (unimporting) the mapped items.

Testing. You can add `--validate` (or `-v`) to the command to simulate the entire import process without actually doing the import. This is extremely useful for verifying your import files before doing the actual import.

Replacing Items in a Collection

Replacing existing items is relatively easy. Remember that mapfile you saved above? Now you will use it. The command (in short form):

```
[dspace]/bin/dspace import -r -e joe@user.com -c collectionID -s items_dir -m mapfile
```

Long form:

```
[dspace]/bin/dspace import --replace --eperson=joe@user.com --collection=collectionID --source=items_dire --
mapfile=mapfile
```

If you wish to replace content using a Zipfile, that's also possible. The command is similar. But, in this situation "-s" refers to the directory of the zip file, and "-z" gives the name of the zipfile:

```
[dspace]/bin/dspace import -r -e joe@user.com -c collectionID -s zipfile_dir -z filename.zip -m mapfile
```

Deleting or Unimporting Items in a Collection

You are able to unimport or delete items provided you have the mapfile. Remember that mapfile you saved above? The command is (in short form):

```
[dspace]/bin/dspace import -e joe@user.com -d -m mapfile
```

In long form:

```
[dspace]/bin/dspace import --eperson=joe@user.com --delete --mapfile mapfile
```

Other Options

- **Workflow**. The importer usually bypasses any workflow assigned to a collection. But add the `--workflow` (`-w`) argument will route the imported items through the workflow system.
- **Templates**. If you have templates that have constant data and you wish to apply that data during batch importing, add the `--template` (`-p`) argument.
- **Resume**. If, during importing, you have an error and the import is aborted, you can use the `--resume` (`-R`) flag to resume the import where you left off after you fix the error.
- **Specifying the owning collection on a per-item basis from the command line administration tool**

  If you omit the -c flag, which is otherwise mandatory, the ItemImporter searches for a file named "collections" in each item directory. This file should contain a list of collections, one per line, specified either by their handle, or by their internal db id. The ItemImporter then will put the item in each of the specified collections. The owning collection is the collection specified in the first line of the collections file.

  If both the -c flag is specified and the collections file exists in the item directory, the ItemImporter will ignore the collections file and will put the item in the collection specified on the command line.

  Since the collections file can differ between item directories, this gives you more fine-grained control of the process of batch adding items to collections.

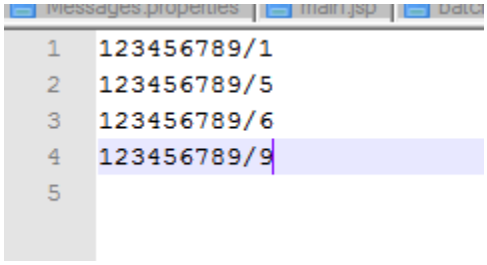
UI Batch Import

Available in DSpace 7.4 and above.

Batch import can also take place via the Administrator's UI. The steps to follow are:

A. Prepare the data

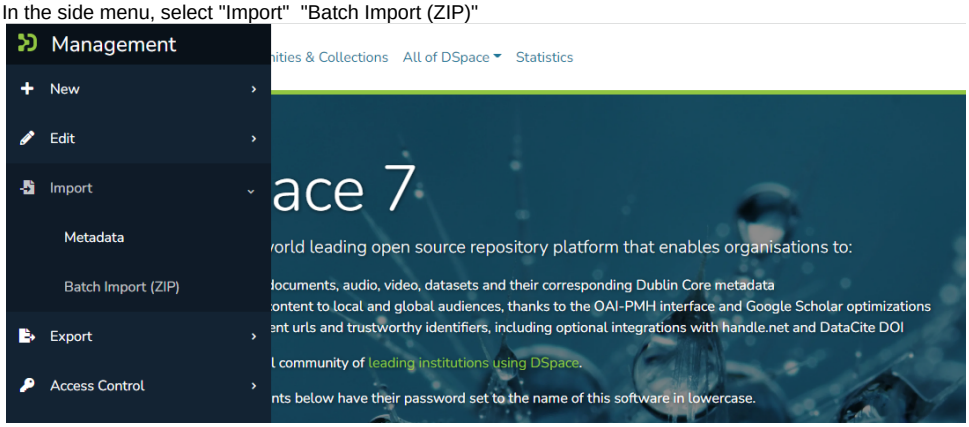
1. Items, i.e. the metadata and their bitstreams, must be in the Simple Archive Format described earlier in this chapter. Thus, for each item there must be a separate directory that contains the corresponding files of the specific item.
2. Moreover, in each item directory, there can be another file that describes the collection or the collections that this item will be added to. The name of this file must be "collections" and it is optional. It has the following format:



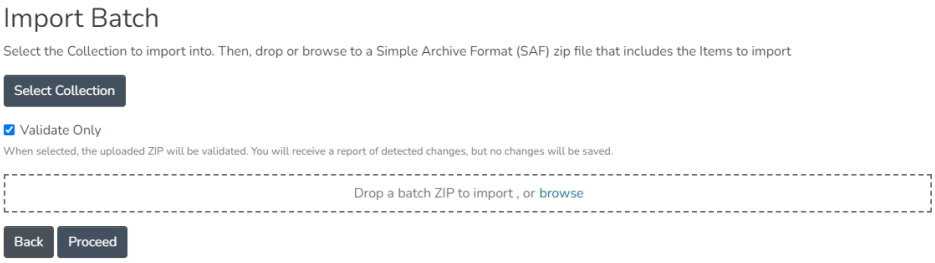
   Each line contains the handle of the collection. The collection in the first line is the owning collection while the rest are the other collections that the item should belong to.
3. Compress the item directories into a ZIP file. Please note that you need to zip the actual item directories and not just the directory that contains the item directories. Thus, the final zip file must directly contain the item directories.

B. Import the items via the UI

1. Login as an Administrator.
2. In the side menu, select "Import" "Batch Import (ZIP)"



3. From the "Import Batch" page:
   a. Select the Collection you are importing into.
   b. Drag & drop the ZIP file into the drop box (or browse to it on your filesystem).
   c. Choose whether you want to "Validate Only" or not.
      i. When selected, DSpace will **test** the batch import process, but no content will be batch imported. This allows you to validate the results of the import process before doing the import.
      ii. When deselected, DSpace will do the batch import.



4. Clicking "Proceed" will start the Batch Import. This creates a new "Process" which begins the upload of the batch. *Depending on the size of the batch, this process may take some time to complete.* You can refresh to page to see the current status, or go back to the list of processes ("Processes" menu in sidebar) to check on its status. Once the process is COMPLETED, you will see a log of the results and a mapfile (which can be used to make later updates).
5. All prior imports will be listed in the "Processes" menu, until their corresponding process entry is deleted. Once you are satisfied with the import and have no need to see the logs or mapfile, you may wish to delete that process entry in order to free up storage space (as your uploaded ZIP will be retained in DSpace until the process is deleted). A "process-cleaner" script can also be started from the "Processes" page which can be used to bulk delete old processes.

It is also possible to start an "import" directly from the "Processes" menu. This allows you to specify additional options/flags which are normally only available to the command-line "import" tool (see documentation above).

Exporting Items

The item exporter can export a single item or a collection of items, and creates a DSpace simple archive in the aforementioned format for each exported item. The items are exported in a sequential order in which they are retrieved from the database. As a consequence, the sequence numbers of the item subdirectories (item_000, item_001) are not related to DSpace handle or item IDs.

Command used:	`[dspace]/bin/dspace export`
Java class:	*org.dspace.app.itemexport.ItemExport*
Arguments short and (long) forms:	Description
`-t` or `--type`	Type of export. *COLLECTION* will inform the program you want the whole collection. *ITEM* will be only the specific item. (You will actually key in the keywords in all caps. See examples below.)
`-i` or `--id`	The ID or Handle of the Collection or Item to export.
`-d` or `--dest`	The destination path where you want the file of items to be placed.
`-n` or `--number`	Sequence number to begin with. Whatever number you give, this will be the name of the first directory created for your export. The layout of the export directory is the same as the layout used for import.
`-m` or `--migrate`	Export the item/collection for migration. This will remove the handle and any other metadata that will be re-created in the new instance of DSpace.
-x or --exclude-bitstreams	Do not export bitstreams. See the usage scenario below.
`-h` or `--help`	Brief Help.

Exporting a Collection

The CLI command to export the items of a collection:

```
[dspace]/bin/dspace export --type=COLLECTION --id=collectionID_or_handle --dest=/path/to/destination --
number=seq_num
```

Short form:

```
[dspace]/bin/dspace export -t COLLECTION -i collectionID_or_handle -d /path/to/destination -n seq_num
```

The keyword *COLLECTION* means that you intend to export an entire collection. The ID can either be the database ID or the handle. The exporter will begin numbering the simple archives with the sequence number that you supply.

Exporting a Single Item

To export a single item use the keyword *ITEM* and give the item ID as an argument:

```
[dspace]/bin/dspace export --type=ITEM --id=itemID_or_handle --dest=/path/to/destination --number=seq_num
```

Short form:

```
[dspace]/bin/dspace export -t ITEM -i itemID_or_handle -d /path/to/destination -n seq_num
```

Each exported item will have an additional file in its directory, named "handle". This will contain the handle that was assigned to the item, and this file will be read by the importer so that items exported and then imported to another machine will retain the item's original handle.

The `-m` Argument

Using the `-m` argument will export the item/collection and also perform the migration step. It will perform the same process that the next section Exchanging Content Between Repositories performs. We recommend that section to be read in conjunction with this flag being used.

The -x Argument

Using the -x argument will do the standard export except for the bitstreams which will not be exported. If you have full SAF without bitstreams and you have the bitstreams archive (which might have been imported into DSpace earlier) somewhere near, you could symlink original archive files into SAF directories and have an exported collection which almost doesn't occupy any space but otherwise is identical to the exported collection (i.e. could be imported into DSpace). In case of huge collections -x mode might be substantially faster than full export.

UI Batch Export

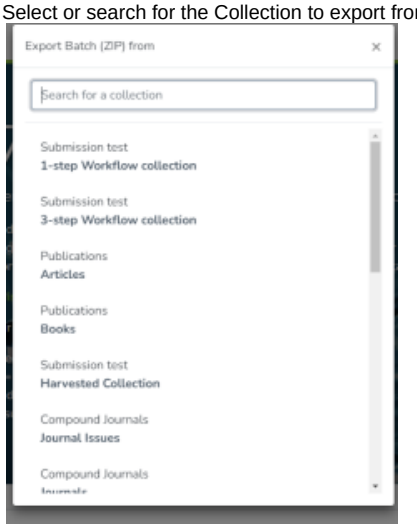
Available in DSpace 7.4 and above.

Batch export can also take place via the Administrator's UI. The default file size upload limit is 512MB, and is being configured in the spring boot application.properties file.

The steps to follow are:

1. Login as an Administrator.
2. In the side menu, select "Import" "Batch Export (ZIP)"

3. Select or search for the Collection to export from:



4. Clicking "Export" will start the Batch Export. This creates a new "Process" which begins export process. *Depending on the size of the export, this process may take some time to complete.* You can refresh to page to see the current status, or go back to the list of processes ("Processes" menu in sidebar) to check on its status. Once the process is COMPLETED, you will see a log of the results and an exported ZIP file which you can download for the results.
5. All prior exports will be listed in the "Processes" menu, until their corresponding process entry is deleted. Once you are satisfied with the export and have downloaded the ZIP, you may wish to delete that process entry in order to free up storage space (as your exported ZIP will be retained in DSpace until the process is deleted). A "process-cleaner" script can also be started from the "Processes" page which can be used to bulk delete old processes.

It is also possible to start an "export" directly from the "Processes" menu. This allows you to specify additional options/flags which are normally only available from the command-line "export" tool (see documentation above). It also allows you to export a single Item.

Importing and Exporting Content via Packages

1 Package Importer and Exporter
- 1.1 Supported Package Formats
- 1.2 Ingesting
  - 1.2.1 Ingestion Modes & Options
    - 1.2.1.1 Ingesting a Single Package
    - 1.2.1.2 Ingesting Multiple Packages at Once
  - 1.2.2 Restoring/Replacing using Packages
    - 1.2.2.1 Default Restore Mode
    - 1.2.2.2 Restore, Keep Existing Mode
    - 1.2.2.3 Force Replace Mode
- 1.3 Disseminating
  - 1.3.1 Disseminating a Single Object
  - 1.3.2 Disseminating Multiple Objects at Once
- 1.4 Archival Information Packages (AIPs)
- 1.5 METS packages

Package Importer and Exporter

This command-line tool gives you access to the Packager plugins. It can *ingest* a package to create a new DSpace Object (Community, Collection or Item), or *disseminate* a DSpace Object as a package.

To see all the options, invoke it as:

```
[dspace]/bin/dspace packager --help
```

This mode also displays a list of the names of package ingestion and dissemination plugins that are currently installed in your DSpace. Each Packager plugin also may allow for custom options, which may provide you more control over how a package is imported or exported. You can see a listing of all specific packager options by invoking `--help` (or `-h`) with the `--type` (or `-t`) option:

```
[dspace]/bin/dspace packager --help --type METS
```

The above example will display the normal help message, while also listing any additional options available to the "METS" packager plugin.

Supported Package Formats

DSpace comes with several pre-configured package ingestion and dissemination plugins, which allow you to import/export content in a variety of formats.

Pre-Configured Submission Package (SIP) Types

- AIP - Ingests content which is in the DSpace Archival Information Package (AIP) format. This is used as part of the DSpace AIP Backup and Restore process
- DSPACE-ROLES - Ingests DSpace users/groups in the DSPACE-ROLES XML Schema. This is primarily used by the DSpace AIP Backup and Restore process to ingest/replace DSpace Users & Groups.
- METS - Ingests content which is in the DSpace METS SIP format
- PDF - Ingests a single PDF file (where basic metadata is extracted from the file properties in the PDF Document).

Pre-Configured Dissemination Package (DIP) Types

- AIP - Exports content which is in the DSpace Archival Information Package (AIP) format. This is used as part of the DSpace AIP Backup and Restore process
- DSPACE-ROLES - Exports DSpace users/groups in the DSPACE-ROLES XML Schema. This is primarily used by the DSpace AIP Backup and Restore process to export DSpace Users & Groups.
- METS - Exports content in the DSpace METS SIP format

For a list of all package ingestion and dissemination plugins that are currently installed in your DSpace, you can execute:

```
[dspace]/bin/dspace packager --help
```

Some packages ingestion and dissemination plugins also have custom options/parameters. For example, to see a listing of the custom options for the "METS" plugin, you can execute:

```
[dspace]/bin/dspace packager --help --type METS
```

Ingesting

Ingestion Modes & Options

When ingesting packages DSpace supports several different "modes". (Please note that not all packager plugins may support all modes of ingestion)

1. Submit/Ingest Mode (`-s` option, default) – submit package to DSpace in order to create a new object(s)
2. Restore Mode (`-r` option) – restore pre-existing object(s) in DSpace based on package(s). This also attempts to restore all handles and relationships (parent/child objects). This is a specialized type of "submit", where the object is created with a known Handle and known relationships.
3. Replace Mode (`-r -f` option) – replace existing object(s) in DSpace based on package(s). This also attempts to restore all handles and relationships (parent/child objects). This is a specialized type of "restore" where the contents of existing object(s) is replaced by the contents in the AIP(s). By default, if a normal "restore" finds the object already exists, it will back out (i.e. rollback all changes) and report which object already exists.

Ingesting a Single Package

To ingest a single package from a file, give the command:

```
[dspace]/bin/dspace packager -e [user-email] -p [parent-handle] -t [packager-name] /full/path/to/package
```

Where *[user-email]* is the e-mail address of the E-Person under whose authority this runs; *[parent-handle]* is the Handle of the Parent Object into which the package is ingested, *[packager-name]* is the plugin name of the package ingester to use, and */full/path/to/package* is the path to the file to ingest (or *"-"* to read from the standard input).

Here is an example that loads a PDF file with internal metadata as a package:

```
[dspace]/bin/dspace packager -e admin@myu.edu -p 4321/10 -t PDF thesis.pdf
```

This example takes the result of retrieving a URL and ingests it:

```
wget -O - http://alum.mit.edu/jarandom/my-thesis.pdf | [dspace]/bin/dspace packager -e admin@myu.edu -p 4321
/10  -t PDF -
```

Ingesting Multiple Packages at Once

Some Packager plugins support bulk ingest functionality using the `--all` (or `-a`) flag. When `--all` is used, the packager will attempt to ingest all child packages referenced by the initial package (and continue on recursively). Some examples follow:

- For a Site-based package - this would ingest **all** Communities, Collections & Items based on the located package files
- For a Community-based package - this would ingest that Community and all SubCommunities, Collections and Items based on the located package files
- For a Collection - this would ingest that Collection and all contained Items based on the located package files
- For an Item – this just ingest the Item (including all Bitstreams & Bundles) based on the package file.

Here is a basic example of a bulk ingest 'packager' command template:

```
[dspace]/bin/dspace packager -s -a -t AIP -e <eperson> -p <parent-handle> <file-path>
```

for example:

```
[dspace]/bin/dspace packager -s -a -t AIP -e admin@myu.edu -p 4321/12 collection-aip.zip
```

The above command will ingest the package named "collection-aip.zip" as a child of the specified Parent Object (handle="4321/12"). The resulting object is assigned a new Handle (since `-s` is specified). In addition, any child packages directly referenced by "collection-aip.zip" are also recursively ingested (a new Handle is also assigned for each child AIP).

Not All Packagers Support Bulk Ingest

Because the packager plugin must know how to locate all child packages from an initial package file, not all plugins can support bulk ingest. Currently, in DSpace the following Packager Plugins support bulk ingest capabilities:

- METS Packager Plugin
- AIP Packager Plugin

Restoring/Replacing using Packages

Restoring is slightly different than just **ingesting**. When restoring, the packager makes every attempt to restore the object as it **used to be** (including its handle, parent object, etc.).

There are currently three restore modes:

1. Default Restore Mode (`-r`) = Attempt to restore object (and optionally children). Rollback all changes if any object is found to already exist.
2. Restore, Keep Existing Mode (`-r -k`) = Attempt to restore object (and optionally children). If an object is found to already exist, skip over it (and all children objects), and continue to restore all other non-existing objects.
3. Force Replace Mode (`-r -f`) = Restore an object (and optionally children) and **overwrite** any existing objects in DSpace. Therefore, if an object is found to already exist in DSpace, its contents are replaced by the contents of the package. *WARNING: This mode is potentially dangerous as it will permanently destroy any object contents that do not currently exist in the package. You may want to first perform a backup, unless you are sure you know what you are doing!*

Default Restore Mode

By default, the restore mode (`-r` option) will rollback all changes if any object is found to already exist. The user will be informed if which object already exists within their DSpace installation.

Use this 'packager' command template:

```
[dspace]/bin/dspace packager -r -t AIP -e <eperson> <file-path>
```

For example:

```
[dspace]/bin/dspace packager -r -t AIP -e admin@myu.edu aip4567.zip
```

Notice that unlike `-s` option (for submission/ingesting), the `-r` option does not require the Parent Object (`-p` option) to be specified if it can be determined from the package itself.

In the above example, the package "aip4567.zip" is restored to the DSpace installation with the Handle provided within the package itself (and added as a child of the parent object specified within the package itself). If the object is found to already exist, all changes are rolled back (i.e. nothing is restored to DSpace)

Restore, Keep Existing Mode

When the "Keep Existing" flag (`-k` option) is specified, the restore will attempt to skip over any objects found to already exist. It will report to the user that the object was found to exist (and was not modified or changed). It will then continue to restore all objects which do not already exist. This flag is most useful when attempting a bulk restore (using the `--all` (or `-a`) option.

One special case to note: If a Collection or Community is found to already exist, its child objects are also skipped over. So, this mode will not auto-restore items to an existing Collection.

Here's an example of how to use this 'packager' command:

```
[dspace]/bin/dspace packager -r -a -k -t AIP -e <eperson> <file-path>
```

For example:

```
[dspace]/bin/dspace packager -r -a -k -t AIP -e admin@myu.edu aip4567.zip
```

In the above example, the package "aip4567.zip" is restored to the DSpace installation with the Handle provided within the package itself (and added as a child of the parent object specified within the package itself). In addition, any child packages referenced by "aip4567.zip" are also recursively restored (the `-a` option specifies to also restore all child pacakges). They are also restored with the Handles & Parent Objects provided with their package. If any object is found to already exist, it is skipped over (child objects are also skipped). All non-existing objects are restored.

Force Replace Mode

When the "Force Replace" flag (`-f` option) is specified, the restore will **overwrite** any objects found to already exist in DSpace. In other words, existing content is deleted and then replaced by the contents of the package(s).

Potential for Data Loss

Because this mode actually **destroys** existing content in DSpace, it is potentially dangerous and may result in data loss! It is recommended to always perform a full backup (assetstore files & database) before attempting to replace any existing object(s) in DSpace.

Here's an example of how to use this 'packager' command:

```
[dspace]/bin/dspace packager -r -f -t AIP -e <eperson> <file-path>
```

For example:

```
[dspace]/bin/dspace packager -r -f -t AIP -e admin@myu.edu aip4567.zip
```

In the above example, the package "aip4567.zip" is restored to the DSpace installation with the Handle provided within the package itself (and added as a child of the parent object specified within the package itself). In addition, any child packages referenced by "aip4567.zip" are also recursively ingested. They are also restored with the Handles & Parent Objects provided with their package. *If any object is found to already exist, its contents are replaced by the contents of the appropriate package.*

If any error occurs, the script attempts to rollback the entire replacement process.

Disseminating

Disseminating a Single Object

To disseminate a single object as a package, give the command:

```
[dspace]/bin/dspace packager -d -e [user-email] -i [handle] -t [packager-name] [file-path]
```

Where *[user-email]* is the e-mail address of the E-Person under whose authority this runs; *[handle]* is the Handle of the Object to disseminate; *[packager-name]* is the plugin name of the package disseminator to use; and *[file-path]* is the path to the file to create (or *"-"* to write to the standard output). For example:

```
[dspace]/bin/dspace packager -d -e admin@myu.edu -i 4321/4567 -t METS 4567.zip
```

The above code will export the object of the given handle (4321/4567) into a METS file named "4567.zip".

Disseminating Multiple Objects at Once

To export an object hierarchy, use the `-a` (or `--all`) package parameter.

For example, use this 'packager' command template:

```
[dspace]/bin/dspace packager -d -a -e [user-email] -i [handle] -t [packager-name][file-path]
```

for example:

```
[dspace]/bin/dspace packager -d -a -t METS -e admin@myu.edu -i 4321/4567 4567.zip
```

The above code will export the object of the given handle (4321/4567) into a METS file named "4567.zip". In addition it would export all children objects to the same directory as the "4567.zip" file.

Archival Information Packages (AIPs)

Since DSpace 1.7, DSpace can backup and restore all of its contents as a set of AIP Files. This includes all Communities, Collections, Items, Groups and People in the system.

This feature came out of a requirement for DSpace to better integrate with DuraCloud, and other backup storage systems. One of these requirements is to be able to essentially "backup" local DSpace contents into the cloud (as a type of offsite backup), and "restore" those contents at a later time.

Essentially, this means DSpace can export the entire hierarchy (i.e. bitstreams, metadata and relationships between Communities/Collections/Items) into a relatively standard format (a METS-based, AIP format). This entire hierarchy can also be re-imported into DSpace in the same format (essentially a restore of that content in the same or different DSpace installation).

For more information, see the section on AIP backup & Restore for DSpace.

METS packages

Since DSpace 1.4 release, the software includes a package disseminator and matching ingester for the DSpace METS SIP (Submission Information Package) format. They were created to help end users prepare sets of digital resources and metadata for submission to the archive using well-defined standards such as METS, MODS, and PREMIS. The plugin name is *METS* by default, and it uses MODS for descriptive metadata.

The DSpace METS SIP profile is available at: DSpaceMETSSIPProfile

Configurable Workflow

1 Introduction
2 How to Configure your Workflows
    2.1 WORKFLOWS
    2.2 STEPS
    2.3 ROLES
    2.4 ACTIONS
    2.5 CURATION
    2.6 HOW IT WORKS
3 Data Migration
    3.1 Workflowitem conversion/migration scripts
        3.1.1 Automatic migration
        3.1.2 Java based migration
4 Configuration
    4.1 Main workflow configuration
        4.1.1 workflowFactory bean (org.dspace.xmlworkflow.XmlWorkflowFactoryImpl)
        4.1.2 workflow beans (org.dspace.xmlworkflow.state.Workflow)
        4.1.3 role beans (org.dspace.xmlworkflow.Role)
        4.1.4 step beans (org.dspace.xmlworkflow.state.Step)
    4.2 Workflow actions configuration
        4.2.1 API configuration
            4.2.1.1 User Selection Action
            4.2.1.2 Processing Action
5 Authorizations
6 Database
    6.1 cwf_workflowitem
    6.2 cwf_collectionrole
    6.3 cwf_workflowitemrole
    6.4 cwf_pooltask
    6.5 cwf_claimtask
    6.6 cwf_in_progress_user
7 Additional workflow steps/actions and features
    7.1 Optional workflow steps: Select single reviewer workflow
    7.2 Optional workflow steps: Score review workflow
    7.3 Workflow overview features

Introduction

Workflows can be used to define how documents should be reviewed or edited after being submitted and/or imported into DSpace. The primary focus of the workflow framework is to create a more flexible solution for the administrator to configure, and even to allow an application developer to implement custom steps, which may be configured in the workflow for the collection through a simple configuration file. Each workflow can be compared to an action that is performed on an item between its submission to the repository and the moment it is archived and published in the repository. The concept behind this approach was modeled on the configurable submission system already present in DSpace.

How to Configure your Workflows

Every submission to DSpace goes through a workflow before it is published in the repository. A workflow consists of a series of steps, each of which is an opportunity for a reviewer, editor or collection administrator to modify and/or approve/reject the submission. A step may also begin by running a Curation Task over the submission.

Each collection is associated with a workflow. If no explicit association is made, the collection is assigned the default workflow. These associations are configured in `config/spring/api/workflow.xml` using the workflowMapping property of the XmlWorkflowFactory bean. To make an explicit association, add an entry to the list with the collection's Handle as the 'key' and the 'name' of a Workflow bean as the 'value-ref'.

Each step in a workflow is associated with a "role" which defines who can perform that step. Role members will be notified when a new submission needs their attention. Roles are defined by DSpace user groups. If you wish to have reviewers interact with incoming submissions, you must create and fill the necessary groups. See below for details.

WORKFLOWS

To create a new workflow, add another bean with the 'class' 'org.dspace.xmlworkflow.state.Workflow' and a unique 'name'. Give it a 'steps' property containing a list of the steps that should be entered in sequence, and a 'firstStep' property which names the step to be entered first. See the default workflow for an example. An existing step may be re-used if appropriate, or you can create one to suit.

STEPS

Aside from its name, a step has a "user selection method", a "role", "actions" and "outcomes".

A step's 'userSelectionMethod' is the name of an "action" of the user-selection type. A step may, for example, let itself be claimed (for a given submission) by a single user, or it may combine the actions of multiple users. A step has exactly one 'userSelectionMethod'. See more on actions below.

A step's role defines the set of users who may perform actions on a submission that has entered that step. See more on roles below.

A step's actions are the types of work that are done in the step. See more on actions below. More than one action may be listed.

A step's outcomes connect the role members' decisions with the next step to be performed. For example, this allows a role member to accept a submission and skip subsequent steps by going directly to the final step in the workflow.

To create a new step, add a bean with 'class' org.dspace.xmlworkflow.state.Step and the necessary properties, as discussed above. See the existing steps in workflow.xml for examples.

ROLES

You may re-use existing roles, or add your own. A role has a 'name', a 'scope', and optionally a 'description'. There are three kinds of roles:

- A COLLECTION role refers to a user group associated with a specific collection. It will be named {collectionID}_{roleName}. For example, a role 'editor' with COLLECTION scope, applied to collection 123, will refer to the user group named 'editor_123', while the same role applied to collection 456 will refer to the user group 'editor_456'.
- A REPOSITORY role refers to a fixed user group, whose name is the role's name. A REPOSITORY role named 'fred' will always refer to the user group 'fred'.
- An ITEM role is assigned by a previous action in the workflow. [NEEDS MORE EXPLANATION]

To create a new role, add a bean with 'class' org.dspace.xmlworkflow.Role, the appropriate 'scope', and a unique 'name'. Be sure that the related groups exist.

ACTIONS

Actions are defined separately in 'config/spring/api/workflow-actions.xml'.

A number of actions are already defined, and these should serve most needs. Actions are implemented in Java code, so if you need a new one then you will need to write some Java in addition to configuring it here.

There are two kinds of actions: user assignment and processing. A user assignment action selects one or more role members to execute a step. A processing action modifies the state of the submission.

To configure a new Action, create a bean with a unique 'id', 'class' equal to the fully qualified name of the Java class which implements the action, and 'scope' "prototype". Add properties, constructor arguments, etc. as required by the code.

CURATION

To attach a Curation Task to a workflow step, see Curation System. Tasks are executed at the beginning of a step, before role members are notified.

HOW IT WORKS

For details of how these concepts are implemented (for example, to create new actions) see the Workflow page under DSpace Development.

Data Migration

As of DSpace 7, Configurable Workflow is the only workflow system available in DSpace. It has fully replaced the older "traditional/basic workflow" system. One major difference is that Configurable Workflow is dynamic – if a user is added to a workflow approval task *after* a workflow has already begun, they will immediately get access to any existing items in workflow. Previously, this was not possible in the "traditional" workflow system.

Workflowitem conversion/migration scripts

Depending on the workflow that is used by a DSpace installation, different scripts can be used when migrating to the new workflow.

Automatic migration

As part of the upgrade to DSpace 7 or above, all your old policies, roles, tasks and workflowitems will be automatically updated from the original workflow to the Configurable Workflow framework. This is done via this command:

```
[dspace]/bin/dspace database migrate ignored
```

The "ignored" parameter will tell DSpace to run any previously-ignored migrations on your database. As the Configurable Workflow migrations have existed in the DSpace codebase for some time, this is the only way to force them to be run.

For more information on the "database migrate" command, please see Database Utilities.

Java based migration

In case your DSpace installation uses a customized version of the workflow, the migration script might not work properly and a different approach is recommended. Therefore, an additional Java based script has been created that restarts the workflow for all the workflowitems that exist in the original workflow framework. The script will take all the existing workflowitems and place them in the first step of the configurable workflow framework thereby taking into account the XML configuration that exists at that time for the collection to which the item has been submitted. This script can also be used to restart the workflow for workflowitems in the original workflow but not to restart the workflow for items in the configurable workflow.

To execute the script, run the following CLI command:

```
[dspace]/bin/dspace dsrun org.dspace.xmlworkflow.migration.RestartWorkflow -e admin@myrespository.org
```

The following arguments can be specified when running the script:

- -e: specifies the username of an administrator user
- -n: if sending submissions through the workflow, send notification emails
- -p: the provenance description to be added to the item
- -h: help

Configuration

Main workflow configuration

As of DSpace 7, the `workflow.xml` configuration file has been migrated to use Spring Bean syntax (instead of a custom XML format). The structure of this XML has changed. If you need help migrating your old `workflow.xml` file (which started with a `<wf-config>` tag) to the new format (using `<bean>` tags), an XSLT script is available: workflow-migration.xsl

The workflow main configuration can be found in the workflow.xml file, located in **`[dspace]/config/spring/api/workflow.xml`** . An example of this workflow configuration file can be found below.

```
<beans>
    <bean class="org.dspace.xmlworkflow.XmlWorkflowFactoryImpl">
        <property name="workflowMapping">
            <util:map>
                <entry key="defaultWorkflow" value-ref="defaultWorkflow"/>
<!--            <entry key="123456789/4" value-ref="selectSingleReviewer"/>-->
<!--            <entry key="123456789/5" value-ref="scoreReview"/>-->
            </util:map>
        </property>
    </bean>

    <!--Standard DSpace workflow-->
    <bean name="defaultWorkflow" class="org.dspace.xmlworkflow.state.Workflow">
        <property name="firstStep" ref="reviewstep"/>
        <property name="steps">
            <util:list>
                <ref bean="reviewstep"/>
                <ref bean="editstep"/>
                <ref bean="finaleditstep"/>
            </util:list>
        </property>
    </bean>

    <bean id="{workflow.id}"
                class="org.dspace.xmlworkflow.state.Workflow">
            <!-- Another workflow configuration-->
    </bean>

        <!-- Role beans.  See below. -->

        <!-- Step beans.  See below. -->

</beans>
```

workflowFactory bean (org.dspace.xmlworkflow.XmlWorkflowFactoryImpl)

The workflow map contains a mapping between collections in DSpace and a workflow configuration, and is defined by the `workflowMapping` property of the workflow factory. Similar to the configuration of the submission process, the mapping can be done based on the handle of the collection. The mapping with "defaultWorkflow" as the value for the collection mapping, will be used for the collections not occurring in other mapping tags. Each mapping is defined by a "`entry`" element with two attributes:

- key: can either be a collection handle or "defaultWorkflow"
- value-ref: the value of this attribute points to one of the workflow configurations defined by the "Workflow" beans

workflow beans (org.dspace.xmlworkflow.state.Workflow)

The workflow bean is a repeatable XML element and represents one workflow process. It requires the following:

- "`name`" attribute: a unique name used for the identification of the workflow and used in the workflow to collection mapping
- "`firstStep`" property: the identifier of the first step of the workflow. This step will be the entry point of this workflow-process. When a new item has been committed to a collection that uses this workflow, the step configured in the "`firstStep`" property will he the first step the item will go through.
- "`steps`" property: a list of all steps within this workflow (in the order they will be processed).

role beans (org.dspace.xmlworkflow.Role)

Each workflow step has defined "role" property. A role represents one or more existing DSpace EPersons or Groups and can be used to assign them to one or more steps in the workflow process. One role is represented by one "role" bean and has the following:

- "`id`" attribute: a unique identifier (in one workflow process) for the role
- "`description`" property: optional attribute to describe the role
- "`scope`" property: optional attribute that is used to find our group and must have one of the following values, which are defined as constant fields of `org.dspace.xmlworkflow.Role.Scope`:
  - COLLECTION: The collection value specifies that the group will be configured at the level of the collection. This type of groups is the same as the type that existed in the original workflow system. In case no value is specified for the scope attribute, the workflow framework assumes the role is a collection role.
  - REPOSITORY: The repository scope uses groups that are defined at repository level in DSpace. The name attribute should exactly match the name of a group in DSpace.
  - ITEM: The item scope assumes that a different action in the workflow will assign a number of EPersons or Groups to a specific workflow-item in order to perform a step. These assignees can be different for each workflow item.
- "`name`" property: The name specified in the name attribute of a role will be used to lookup an eperson group in DSpace. The lookup will depend on the scope specified in the "`scope`" attribute:
  - COLLECTION: The workflow framework will look for a group containing the name specified in the name attribute and the ID of the collection for which this role is used.
  - REPOSITORY: The workflow framework will look for a group with the same name as the name specified in the `name` attribute.
  - ITEM: in case the item scope is selected, the name of the role attribute is not required.

```
<bean id="reviewer" class="org.dspace.xmlworkflow.Role">
    <property name="scope" value="#{ T(org.dspace.xmlworkflow.Role.Scope).COLLECTION}"/>
    <property name="name" value="Reviewer"/>
    <property name="description" value="The people responsible for this step are able to edit the metadata of
incoming submissions, and then accept or reject them."/>
</bean>
```

step beans (org.dspace.xmlworkflow.state.Step)

The step element represents one step in the workflow process. A step represents a number of actions that must be executed by one specified role. In case no `role` attribute is specified, the workflow framework assumes that the DSpace system is responsible for the execution of the step and that no user interface will be available for each of the actions in this step. The step element has the following in order to further configure it:

- "`name`" attribute: The name attribute specifies a unique identifier for the step. This identifier will be used when configuring other steps in order to point to this step. This identifier can also be used when configuring the start step of the workflow item.
- "`userSelectionMethod`" property: This attribute defines the `UserSelectionAction` that will be used to determine how to attach users to this step for a workflow-item. The value of this attribute must refer to the identifier of an action bean in the workflow-actions.xml. Examples of the user attachment to a step are the currently used system of a task pool or as an alternative directly assigning a user to a task.
- "`role`" property: optional attribute that must point to the `id` attribute of a role element specified for the workflow. This role will be used to define the epersons and groups used by the `userSelectionMethod`.
- RequiredUsers

```
<bean name="reviewstep" class="org.dspace.xmlworkflow.state.Step">
    <property name="userSelectionMethod" ref="claimaction"/>
    <property name="role" ref="reviewer"/>
    <property name="outcomes">
        <util:map>
            <entry key="#{ T(org.dspace.xmlworkflow.state.actions.ActionResult).OUTCOME_COMPLETE}"
                   value-ref="editstep"/>
        </util:map>
    </property>
    <property name="actions">
        <util:list>
            <ref bean="reviewaction"/>
        </util:list>
    </property>
</bean>
```

Each step contains a number of actions that the workflow item will go through. In case the action has a user interface, the users responsible for the exectution of this step will have to execute these actions before the workflow item can proceed to the next action or the end of the step.

There is also an optional subsection that can be defined for a step part called "`outcomes`". This can be used to define outcomes for the step that differ from the one specified in the nextStep attribute. Each action returns an integer depending on the result of the action. The default value is "0" and will make the workflow item proceed to the next action or to the end of the step.
In case an action returns a different outcome than the default "0", the alternative outcomes will be used to lookup the next step. The "`outcomes`" element contains a number of steps, each having a status attribute. This status attribute defines the return value of an action. The value of the element will be used to lookup the next step the workflow item will go through in case an action returns that specified status.

Workflow actions configuration

API configuration

The workflow actions configuration is located in the `[dspace]/config/spring/api/` directory and is named "`workflow-actions.xml`". This configuration file describes the different Action Java classes that are used by the workflow framework. Because the workflow framework uses Spring framework for loading these action classes, this configuration file contains Spring configuration.

This file contains the beans for the actions and user selection methods referred to in the `workflow.xml`. In order for the workflow framework to work properly, each of the required actions must be part of this configuration.

```
<beans
    xmlns="http://www.springframework.org/schema/beans"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xmlns:util="http://www.springframework.org/schema/util"
    xsi:schemaLocation="http://www.springframework.org/schema/beans http://www.springframework.org/schema/beans
/spring-beans-2.0.xsd
                        http://www.springframework.org/schema/util http://www.springframework.org/schema/util
/spring-util-2.0.xsd">

    <!-- At the top are our bean class identifiers --->
    <bean id="{action.api.id}" class="{class.path}" scope="prototype"/>
    <bean id="{action.api.id.2}" class="{class.path}" scope="prototype"/>

        <!-- Below the class identifiers come the declarations for out actions/userSelectionMethods -->

        <!-- Use class workflowActionConfig for an action -->
        <bean id="{action.id}" class="oorg.dspace.xmlworkflow.state.actions.WorkflowActionConfig" scope="
prototype">
            <constructor-arg type="java.lang.String" value="{action.id}"/>

            <property name="processingAction" ref="{action.api.id}"/>
            <property name="requiresUI" value="{true/false}"/>
        </bean>

        <!-- Use class UserSelectionActionConfig for a user selection method -->
        <!--User selection actions-->
        <bean id="{action.api.id.2}" class="org.dspace.xmlworkflow.state.actions.UserSelectionActionConfig"
scope="prototype">
            <constructor-arg type="java.lang.String" value="{action.api.id.2}"/>

            <property name="processingAction" ref="{user.selection.bean.id}"/>
            <property name="requiresUI" value="{true/false}"/>
        </bean>
</beans>
```

Two types of actions are configured in this Spring configuration file:

- User selection action: This type of action is always the first action of a step and is responsible for the user selection process of that step. In case a step has no role attached, no user will be selected and the `NoUserSelectionAction` is used.
- Processing action: This type of action is used for the actual processing of a step. Processing actions contain the logic required to execute the required operations in each step. Multiple processing actions can be defined in one step. These user and the workflow item will go through these actions in the order they are specified in the workflow configuration unless an alternative outcome is returned by one of them.

User Selection Action

Each user selection action that is used in the workflow configuration refers to a bean definition in the `workflow-actions.xml` file. In order to define a new user selection action, the following XML code is used:

```
<bean id="{action.api.id.2}" class="org.dspace.xmlworkflow.state.actions.UserSelectionActionConfig" scope="
prototype">
    <constructor-arg type="java.lang.String" value="{action.api.id.2}"/>

    <property name="processingAction" ref="{user.selection.bean.id}"/>
    <property name="requiresUI" value="{true/false}"/>
</bean>
```

This bean defines a new UserSelectionActionConfig and the following child tags:

- constructor-arg: This is a constructor argument containing the ID of the task. This is the same as the `id` attribute of the bean and is used by the workflow configuration to refer to this action.
- property `processingAction`: This tag refers the the ID of the API bean, responsible for the implementation of the API side of this action. This bean should also be configured in this XML.
- property `requiresUI`: In case this property is true, the workflow framework will expect a user interface for the action. Otherwise the framework will automatically execute the action and proceed to the next one.

Processing Action

Processing actions are configured similarly to the user selection actions. The only difference is that these processing action beans are implementations of the WorkflowActionConfig class instead of the UserSelectionActionConfig class.

Authorizations

Currently, the authorizations are always granted and revoked based on the tasks that are available for certain users and groups. The types of authorization policies that is granted for each of these is always the same:

- READ
- WRITE
- ADD
- DELETE

Database

The workflow uses a separate metadata schema named `workflow`. The fields this schema contains can be found in the `[dspace]/config/registries` directory and in the file `workflow-types.xml`. This schema is only used when using the score reviewing system at the moment, but one could always use this schema if metadata is required for custom workflow steps.

The following tables have been added to the DSpace database. All tables are prefixed with 'cwf_' to avoid any confusion with the existing workflow related database tables:

cwf_workflowitem

The cwf_workflowitem table contains the different workflowitems in the workflow. This table has the following columns:

- workflowitem_id: The identifier of the workflowitem and primary key of this table
- item_id: The identifier of the DSpace item to which this workflowitem refers.
- collection_id: The collection to which this workflowitem is submitted.
- multiple_titles: Specifies whether the submission has multiple titles (important for submission steps)
- published_before: Specifies whether the submission has been published before (important for submission steps)
- multiple_files: Specifies whether the submission has multiple files attached (important for submission steps)

cwf_collectionrole

The cwf_collectionrole table represents a workflow role for one collection. This type of role is the same as the roles that existed in the original workflow meaning that for each collection a separate group is defined to described the role. The cwf_collectionrole table has the following columns:

- collectionrol_id: The identifier of the collectionrole and the primaty key of this table
- role_id: The identifier/name used by the workflow configuration to refer to the collectionrole
- collection_id: The collection identifier for which this collectionrole has been defined
- group_id: The group identifier of the group that defines the collection role

cwf_workflowitemrole

The cwf_workflowitemrole table represents roles that are defined at the level of an item. These roles are temporary roles and only exist during the execution of the workflow for that specific item. Once the item is archived, the workflowitemrole is deleted. Multiple rows can exist for one workflowitem with e.g. one row containing a group and a few containing epersons. All these rows together make up the workflowitemrole The cwf_workflowitemrole table has the following columns:

- workflowitemrole_id: The identifier of the workflowitemrole and the primaty key of this table
- role_id: The identifier/name used by the workflow configuration to refer to the workflowitemrole
- workflowitem_id: The cwf_workflowitem identifier for which this workflowitemrole has been defined
- group_id: The group identifier of the group that defines the workflowitemrole role
- eperson_id: The eperson identifier of the eperson that defines the workflowitemrole role

cwf_pooltask

The cwf_pooltask table represents the different task pools that exist for a workflowitem. These task pools can be available at the beginning of a step and contain all the users that are allowed to claim a task in this step. Multiple rows can exist for one task pool containing multiple groups and epersons. The cwf_pooltask table has the following columns:

- pooltask_id: The identifier of the pooltask and the primaty key of this table
- workflowitem_id: The identifier of the workflowitem for which this task pool exists
- workflow_id: The identifier of the workflow configuration used for this workflowitem
- step_id: The identifier of the step for which this task pool was created
- action_id: The identifier of the action that needs to be displayed/executed when the user selects the task from the task pool
- eperson_id: The identifier of an eperson that is part of the task pool
- group_id: The identifier of a group that is part of the task pool

cwf_claimtask

The cwf_claimtask table represents a task that has been claimed by a user. Claimed tasks can be assigned to users or can be the result of a claim from the task pool. Because a step can contain multiple actions, the claimed task defines the action at which the user has arrived in a particular step. This makes it possible to stop working halfway the step and continue later. The cwf_claimtask table contains the following columns:

- claimtask_id: The identifier of the claimtask and the primary key of this table
- workflowitem_id: The identifier of the workflowitem for which this task exists
- workflow_id: The id of the workflow configuration that was used for this workflowitem
- step_id: The step that is currenlty processing the workflowitem
- action_id: The action that should be executed by the owner of this claimtask
- owner_id: References the eperson that is responsible for the execution of this task

cwf_in_progress_user

The cwf_in_progess_user table keeps track of the different users that are performing a certain step. This table is used because some steps might require multiple users to perform the step before the workflowitem can proceed. The cwf_in_progress_user table contains the following columns:

- in_progress_user_id: The identifier of the in progress user and the primary key of this table
- workflowitem_id: The identifier of the workflowitem for which the user is performing or has performed the step.
- user_id: The identifier of the eperson that is performing or has performe the task
- finished: Keeps track of the fact that the user has finished the step or is still in progress of the execution

Additional workflow steps/actions and features

These optional steps are only supported in 7.5 and later.

These optional workflow steps are pre-defined in the "workflow.xml" but are not used by default.

Optional workflow steps: Select single reviewer workflow

This workflow makes it possible to assign a single user to review an item. This workflow configuration skips the task pool option meaning that the assigned reviewer no longer needs to claim the task. The configuration consists of the following 2 steps.

- selectReviewerStep: During this step, a user has the ability to select a responsible user to review the workflowitem. This means that for each workflowitem, a different user can be selected. Because a user is assigned, the task pool is no longer required.
    - The available users to select from are defined in the "action.selectrevieweraction.group" setting in workflow.cfg. This setting must list the name of a group of reviewers to select from (default value = "Reviewers" group).
- singleUserReviewStep: The start of the reviewstep is different than the typical task pool. Instead of having a task pool, the user will be automatically assigned to the task. However, the user still has the option to reject the task (in case he or she is not responsible for the assigned task) or review the item. In case the user rejects the task, the workflowitem will be sent to the another step in the workflow as an alternative to the default outcome.
    - In workflow.cfg, there's an option to allow these reviewers to be able to edit files & metadata. When "workflow.reviewer.file-edit=true", the selected user will be allowed to edit the workflow item. By default they cannot.

Optional workflow steps: Score review workflow

The score review system allows reviewers to give the reviewed item a rating. Depending on the results of the rating, the item will be approved to go to the next workflow step or will be sent to an alternative step. The scrore review workflow consists of the following 2 steps.

- scoreReviewStep: The group of responsible users for the score reviewing will be able to claim the task from the taskpool. Depending on the configuration, a different number of users can be required to execute the task (default is requiredusers=2). This means that the task will be available in the task pool until the required number of users has at least claimed the task. Once everyone of them has finished the task, the next (automatic) processing step is activated.
    - In workflow.cfg, there's an option to allow these reviewers to be able to edit files & metadata. When "workflow.reviewer.file-edit=true", the selected user will be allowed to edit the workflow item. By default they cannot.
- evaluationStep: During the evaluationstep, no user interface is required. The workflow system will automatically execute the step that evaluates the different scores (which corresponds to a rating from 1-5). In case the average score is greater than the average "minimumAcceptanceScore", the item is approved, otherwise it is rejected. (The minimum average score is set by adjusting the `minimumAcceptanceScore` property passed to `evaluationactionAPI` in `config/spring/api/workflow-actions.xml`.)

Workflow overview features

The DSpace UI also provides a feature to allow Administrators to see & administer all active workflows (workitems). This feature is provided in the "Administer Workflow" menu option. Currently, the Administrator has the ability to permanently delete the workflowitem, or to send it back to the original submitter.

Submission User Interface

This page explains various customization and configuration options that are available within DSpace for the Item Submission user interface.

1 Default Submission Process
    1.1 Optional Steps
2 Understanding the Submission Configuration Files
    2.1 The Structure of item-submission.xml
    2.2 Defining Steps (<step>) within the item-submission.xml
        2.2.1 Where to place your <step-definition>
        2.2.2 The ordering of <step> tags matter!
        2.2.3 Structure of the <step-definition> tag
3 Reordering/Removing/Adding Submission Steps
4 Assigning a custom Submission Process to a Collection
    4.1 Getting A Collection's Handle
    4.2 Assigning a default Submission Process per Entity Type
5 Custom Metadata-entry Steps for Submission
    5.1 Introduction
    5.2 Describing Custom Metadata Forms
    5.3 The Structure of submission-forms.xml
        5.3.1 Using a form in a submission process for a Collection
        5.3.2 Adding a Form
            5.3.2.1 Forms and Pages
            5.3.2.2 Composition of a Field
                5.3.2.2.1 Visibility configuration examples
            5.3.2.3 Item type Based Metadata Collection
        5.3.3 Configuring Controlled Vocabularies
        5.3.4 Adding Value-Pairs
            5.3.4.1 Example
    5.4 Deploying Your Custom Forms
6 Configuring the File Upload step
    6.1 Basic Settings
    6.2 Modifying metadata form presented for Bitstreams
    6.3 Modifying access conditions (embargo, etc.) presented for Bitstreams
    6.4 Define upload of files mandatory or optional per collection
7 Configuring the Item Access Conditions step
    7.1 Enabling the step
    7.2 Modifying access conditions (embargo, etc.) presented for Items
    7.3 Examples of selecting Item and Bitstream access conditions
8 Configuring the Sherpa Romeo step
    8.1 Enabling the Sherpa Romeo step
9 Configuring the "Identifiers" step
10 Creating new Submission Steps Programmatically.

DSpace Submission Configuration changed in v7.x

The name and structure of the Submission configuration files changed in 7.x. The DSpace 6.x (and below) "item-submission.xml" and "input-forms.xml" configuration files are no longer supported. In 7.x and above, the format of the "item-submission.xml" file has been updated, and the older "input-forms.xml" has been replaced by a new "submission-forms.xml".

You can choose to either start fresh with the new v7 configuration files (see documentation below) and/or use the "./dspace submission-forms-migrate" script to migrate your old configurations into new ones. See the Upgrading DSpace guide (step on "Update your DSpace Configurations") for more information on using the migration script.

Default Submission Process

The DSpace Submission process consists of a series of "steps", where each "step" corresponds to one or "sections" in the Submission UI. By default, the DSpace Submission process includes the following steps/sections, in this order:

1. **"Select Collection"** (id="collection"), appears as dropdown: If not already selected, the user must select a collection to deposit the Item into. As of DSpace 7, you also can change the Collection you are submitting into at any time. However, be aware that there may be some metadata lost if the Collection you switch two uses a *different* submission form & you already began entering metadata in the current submission.
2. **"Describe" sections** (id="traditionalpageone" and "traditionalpagetwo"): This is where the user may enter descriptive metadata about the Item. This step may consist of one or more sections of metadata entry. By default, there are two sections of metadata-entry . For information on modifying the metadata entry pages, please see Custom Metadata-entry Pages for Submission section below.
3. **"Upload" section** (id="upload"): This is where the user may upload one or more files to associate with the Item. As of DSpace 7, you can also drag and drop files anywhere on the page to trigger an upload. For more information on file upload, also see Configuring the File Upload step below.
4. **"License" section** (id="license"): This is where the user **must** agree to the repository distribution license in order to complete the deposit. This repository distribution license is defined in the `[dspace]/config/default.license` file. It can also be customized per-collection from the Collection Edit UI.
5. **"Deposit" button**: Once all required fields/sections are completed, the "Deposit" button becomes enabled. After clicking it, the new Item will either become immediately available or undergo a workflow approval process (depending on the Collection policies). For more information on the workflow approval process see Configurable Workflow

To modify or reorganize these submission steps, just modify the `[dspace]/config/item-submission.xml` file. Please see the section below on Reordering/Removing/Adding Submission Steps.

You can also choose to have different submission processes for different DSpace Collections. For more details, please see the section below on Assigning a custom Submission Process to a Collection.

Optional Steps

DSpace also ships with several optional steps which you may choose to enable if you wish. In no particular order:

- **"Item Access" (or Embargo) section** (id="itemAccessConditions"): *Only available in 7.2 or above.* This step allows the user to (optionally) modify access rights or set an embargo during the deposit of an Item. For more information on this step, and Embargo options in general, please see the Embargo documentation.
- **"CC License" section** (id="cclicense"): This step allows the user to (optionally) assign a Creative Commons license to a particular Item. Please see the Configuring Creative Commons License section of the Configuration documentation for more details.
- **"Extraction" section** (id="extractionstep"): This step will automatically attempt to extract metadata from uploaded files. Currently it only supports bibliographic formats documented in Importing Items via basic bibliographic formats (Endnote, BibTex, RIS, TSV, CSV) and online services (OAI, arXiv, PubMed, CrossRef, CiNii). Any extracted metadata is immediately populated in the submission form (without notifying the user).
  - By default it is disabled, as it populates metadata automatically (without notifying the user). This means it can sometimes result in duplicative metadata in the submission form.
  - The behavior of this step can be more fully configured via the 'config/spring/api/step-processing-listener.xml' configuration
  - NOTE: this action is also only triggered when a request is performed (e.g. when a file is uploaded or the submission form is saved). You can configure the Angular UI to autosave based on a timer in order to force this action to be done more regularly.
- Various Configurable Entities related steps: These steps are "Describe" steps that are specific to different Entity types. They provide a list of metadata fields of specific interest to those Entities.

To enable any of these optional submission steps, just uncomment the step definition within the `[dspace]/config/item-submission.xml` file. Please see the section below on Reordering/Removing/Adding Submission Steps.

You can also choose to enable certain steps only for specific DSpace Collections. For more details, please see the section below on Assigning a custom Submission Process to a Collection.

Understanding the Submission Configuration Files

The `[dspace]/config/item-submission.xml` contains the submission configurations for the DSpace UI. This configuration file contains detailed documentation within the file itself, which should help you better understand how to best utilize it.

The Structure of *item-submission.xml*

The structure of this file changed slightly in DSpace 7

As of DSpace 7, the following structural changes were made to item-submission.xml:

- Step definitions under <step-definitions> now use the <step-definition> tag (previously, in 6.x, the tag was named <step>)
- Every step definition now needs to be defined under <step-definitions> (previously, in 6.x, you could also define them in <submission-process>), and have a unique ID
- Each <step-definition> now only represents a single "section" of the Submission UI. (previously, in 6.x, some steps like Describe represented multiple pages)
- An attribute "mandatory=[true|false]" was added to the <step> element. When true, that section is always displayed to the user. When false, it's not displayed by default, but instead must be activated explicitly by the user by choosing to add the section in the Submission UI.
- The old <workflow-editable> element has been replaced with a <scope> element which defines when/how this <step> should be displayed.

Because this file is in XML format, you should be familiar with XML before editing this file. By default, this file contains the "traditional" Item Submission Process for DSpace, which consists of the following Steps (in this order):

Select Collection -> Describe (two steps) -> Upload -> License -> Complete

If you would like to customize the steps used or the ordering of the steps, you can do so within the <*submission-definition*> section of the *item-submission.xml* .

In addition, you may also specify different Submission Processes for different DSpace Collections. This can be done in the <*submission-map*> section. The *item-submission.xml* file itself documents the syntax required to perform these configuration changes.

Defining Steps (<*step*>) within the *item-submission.xml*

This section describes how Steps of the Submission Process are defined within the *item-submission.xml*.

Where to place your <*step-definition*>

The <*step-definition*> always appear within the <*step-definitions*> section of the *item-submission.xml* configuration file.

- This section allows all <*step*> definitions to be defined globally (i.e. so they may be used in multiple <*submission-process*> definitions). Steps defined in this section **must** define a unique *id* which can be used to reference this step.
- For example:

```
<step-definitions>
        <step-definition id="custom-step">
           ...
        </step>
        ...
</step-definitions>
```

- The above step definition could then be referenced from within a <*submission-process*> as simply <*step id="custom-step"/*>

The ordering of <*step*> tags matter!

The ordering of the <*step*> tags within a <*submission-process*> definition directly corresponds to the order in which those steps will appear!

For example, the following defines a Submission Process where the *License* step directly precedes the *Describe* step (more information about the structure of the information under each <step> tag can be found in the section on Structure of the <step> Definition below):

```
<submission-process>
    <!--Step 1 will be to Sign off on the License-->
    <step id="license"/>

    <!--Step 2 & 3 will be to ask for metadata-->
    <step id="traditionalpageone"/>
    <step id="traditionalpagetwo"/>

        ...[other steps]...
</submission-process>
```

Structure of the <step-definition> tag

The structure of the <step-definition> tag is as follows:

```
<step-definition id="traditionalpageone" mandatory="true">
    <heading>submit.progressbar.describe.stepone</heading>
    <processing-class>org.dspace.app.rest.submit.step.DescribeStep</processing-class>
    <type>submission-form</type>
    <!-- <scope visibility="hidden" visibilityOutside="hidden">submission</scope> -->
</step-definition>
```

Each *step* contains the following elements/attributes. The required elements are so marked:

- **mandatory** *(attribute)*: [true|false] When true, the step's section is displayed by default to all users in the UI. When false, the step is not displayed and must be activated explicitly by the user by selecting it in the UI or supplying data of interest to the section.
- **heading**: Partial I18N key (defined in the UI's language packs) which corresponds to the text that should be displayed in section header for this step. This partial I18N key is prefixed with "submission.sections.". Therefore, the full i18n key is "submission.sections.[heading]" in the User Interface's language packs (e.g. en.json5 for English)
- **processing-class (Required):** Full Java path to the Processing Class for this Step. This Processing Class **must** perform the primary processing of any information gathered in this step. All valid step processing classes must extend the abstract `org.dspace.submit.AbstractProcessingStep` class (or alternatively, extend one of the pre-existing step processing classes in `org.dspace.submit.step.*`)
- **type (Required)**: The type of step defined. Most steps are of type "submission-form", which means they directly map to a `<form>` defined in the s`ubmission-forms.xml` configuration file. In this situation, the `<step-definition>` "id" attribute MUST map to a `<form>` "name" attribute defined in submission-forms.xml. Any value is allowed, and only "submission-form" has a special meaning at this time.
- **scope:** Optionally, allows you to limit the "scope" of this particular step, and define whether the step is visible outside that scope. Valid scope values include "submission" (limited to the submission form) and "workflow" (limited to workflow approval process).
    - "visibility" attribute defines the visibility of the step while **within** the given scope. Can be set to "read-only" (in this scope you can see this step but not edit it), or "hidden" (in this scope you cannot see this step).
    - "visibilityOutside" attribute defines the visibility of the step while **outside** the given scope. Can be set to "read-only" (in other scopes you can see this step but not edit it), or "hidden" (in other scopes you cannot see this step).

Reordering/Removing/Adding Submission Steps

The removal of existing steps and reordering of existing steps is a relatively easy process!

Reordering steps

1. Locate the `<submission-process>` tag which defines the Submission Process that you are using. If you are unsure which Submission Process you are using, it's likely the one with `name="traditional"`, since this is the traditional DSpace submission process.
2. Reorder the `<step>` tags within that `<submission-process>` tag. Be sure to move the *entire* `<step>` tag.

Removing one or more steps

1. Locate the `<submission-process>` tag which defines the Submission Process that you are using. If you are unsure which Submission Process you are using, it's likely the one with `name="traditional"`, since this is the traditional DSpace submission process.
2. Comment out (i.e. surround with `<! --` and `-->`) the `<step>` tags which you want to remove from that *<submission-process>* tag. Be sure to comment out the *entire* `<step >` *tag.*
    - *Hint:* You cannot remove the *"collection"* step, as an DSpace Item cannot exist without belonging to a Collection.

Adding one or more optional steps

1. Locate the `<submission-process>` tag which defines the Submission Process that you are using. If you are unsure which Submission Process you are using, it's likely the one with `name="traditional"`, since this is the traditional DSpace submission process.
2. Uncomment (i.e. remove the `<! --` and `-->`) the `<step>` tag(s) which you want to add to that `<submission-process>` tag. Be sure to uncomment the *entire* `<step>` *tag.*

Assigning a custom Submission Process to a Collection

Assigning a custom submission process to a Collection in DSpace involves working with the *submission-map* section of the *item-submission.xml*. For a review of the structure of the *item-submission.xml* see the section above on Understanding the Submission Configuration File.

Each *name-map* element within *submission-map* associates a collection with the name of a submission definition.

There are two ways to configure this mapping:

1. The traditional way is to use the "collection-handle" attribute to map a submission form to it's Collection. Its *collection-handle* attribute is the Handle of the collection. Its *submission-name* attribute is the submission definition name, which must match the *name* attribute of a *submission-process* element (in the *submission-definitions* section of *item-submission.xml*.
    a. For example, the following fragment shows how the collection with handle "12345.6789/42" is assigned the "custom" submission process:

```
<submission-map>
    <name-map collection-handle="12345.6789/42" submission-name="custom" />
    ...
</submission-map>

<submission-definitions>
    <submission-process name="custom">
    ...
</submission-definitions>
```

2. As of 7.6, another option is to use the "collection-entity-type" attribute to map *all Collections* which use that Entity Type (requires Configurable Entities) to a specific submission definition name (via the *submission-name* attribute, similar to above).
    a. For example, the following fragment shows how to map all Collections which use the out-of-the-box Entity Types to a submission definition of the same name:

```
<submission-map>
    ...
    <name-map collection-entity-type="Publication" submission-name="Publication"/>
    <name-map collection-entity-type="Person" submission-name="Person"/>
    <name-map collection-entity-type="Project" submission-name="Project"/>
    <name-map collection-entity-type="OrgUnit" submission-name="OrgUnit"/>
    <name-map collection-entity-type="Journal" submission-name="Journal"/>
    <name-map collection-entity-type="JournalVolume" submission-name="JournalVolume"/>
    <name-map collection-entity-type="JournalIssue" submission-name="JournalIssue"/>
    ...
<submission-map>
```

It's a good idea to keep the definition of the *default* name-map, so there is always a default for collections which do not have a custom form set.

Getting A Collection's Handle

You will need the *handle* of a collection in order to assign it a custom form set. To discover the handle, go to the Community or Collection in the DSpace UI. Look for the "Permanent URI" listed near the top of the page. It should look something like:

```
http://myhost.my.edu/handle/12345.6789/42
```

The handle is everything after "handle/" (in the above example it is "12345.6789/42"). It should look familiar to any DSpace administrator. That is what goes in the *collection-handle* attribute of your *name-map* element.

Assigning a default Submission Process per Entity Type

Alternatively to a collection's Handle, Entities Types can be used as an attribute. With these configurations you will enable default submission forms per Entity type. You don't have to specify every collection's handle to use for a particular submission form if you intend to use entities.
In order to do it so, instead of `collection-handle` attribute you need to use `collection-entity-type`. The possible values for this attribute are the ones that you use or that you specified in `relationship-types.xml` file (please check the documentation for more information). In order the submission process to be assigned to an entity type, you need to previously have associated an Entity Type to a Collection (please check: Configurable Entities#3.ConfigureCollectionsforeachEntitytype).

As an example, for every time you need to insert a new person in a Person's collection. You just need to specify the submission form to be used, like: `submission-name="customPerson"` in the example and also the entity type that is associated, like `collection-entity-type="Person"`.

```
<submission-map>
    <name-map collection-entity-type="Person" submission-name="customPerson" />
    ...
</submission-map>

<submission-definitions>
    <submission-process name="customPerson">
    ...
</submission-definitions>
```

If a collection `collection-handle="12345.6789/42"` configuration will prevail over this configuration. Meaning that if a `collection-entity-type` is defined and a collection-handle is also defined and if a collection handle overlaps in both configurations, then, the submission to be considered it will be the one that is defined by `collection-handle` (it will prevail the one with more granularity).

Custom Metadata-entry Steps for Submission

Introduction

This section explains how to customize the Web forms used by submitters and editors to enter and modify the metadata for a new item. These metadata web forms are controlled by the *Describe* step within the Submission Process. However, they are also configurable via their own XML configuration file `[dspace]/config/submission-forms.xml`.

In this configuration you can create alternate metadata forms, which can then be mapped to a "submission-form" step in the "item-submission.xml" (see above).

In creating custom metadata forms, you can choose:

- Which fields appear on each form, and their sequence. (Keep in mind, each "form" represents to a "step" or section)
- Labels, prompts, and other text associated with each field.
- Ability to display smaller fields side-by-side in a single "row"
- List of available choices for each menu-driven field.

All of the custom metadata-entry forms for a DSpace instance are controlled by a single XML file, `submission-forms.xml`, in the *config* subdirectory under the DSpace home, `[dspace]/config/submission-forms.xml`. DSpace comes with a number of sample forms which implement the traditional metadata-entry forms, and also serves as a well-documented example. Some default forms include:

- "bitstream-metadata" - This is a special form which defines the metadata fields available for every uploaded bitstream (file)
- "traditionalpageone" - A sample form which is used by the first "Describe" step defined in item-submission.xml
- "traditionalpagetwo" - A sample form which is used by the second "Describe" step defined in item-submission.xml
- A number of sample forms for various out-of-the-box Configurable Entities. These forms all have a corresponding <step> defined in item-submission.xml. In conjunction to those <step> definitions, these forms may be used to submit new Entities of specific types. Usually this is done by mapping that Entity-specific submission-process (in item-submission.xml) to a Collection which is used for new submissions of that Entity.

The rest of this section explains how to create your own sets of custom forms.

Describing Custom Metadata Forms

The description of a set of fields through which submitters enter their metadata is called a *form* (in the UI, each "form" is displayed in a separate collapsible section). A form is identified by a unique symbolic *name*. In the XML structure, the *form* is broken down into *rows of fields*. This allows you to place smaller fields side-by-side in a single, horizontal row, or alternatively decide to display one field per row.

The Structure of *submission-forms.xml*

The name & structure of this file changed slightly in DSpace 7

As of DSpace 7, the following structural changes were made to this configuration:

- input-forms.xml (v6) was renamed to submission-forms.xml
- <form-map> top-level element was removed. All Collection mappings are now in item-submission.xml
- <page> element under <form> was removed. As described below, <form> element now represent a single section of the submission process.
- <row> element under <form> was added. As described below, multiple fields can now be displayed in one horizontal row.
- A new form named "bitstream-metadata" was introduced to allow you to configure which metadata is requested for a bitstream during submission.

The XML configuration file has a single top-level element, *input-forms*, which contains two elements in a specific order. The outline is as follows:

```
<input-forms>

    <--  Form Set Definitions -->
    <form-definitions>
        <form name="traditionalpageone">
            ...
        </form>
        ...
    </form-definitions>

    <--  Name/Value Pairs used within Multiple Choice Widgets -->
    <form-value-pairs>
        <value-pairs value-pairs-name="common_iso_languages" dc-term="language_iso">
            ...
        </value-pairs>
        ...
    </form-value-pairs>
</input-forms>
```

Using a form in a submission process for a Collection

Keep in mind, the "submission-forms.xml" only defines *forms* and *value-pairs* (used for specific fields like selectboxes). To enable a form requires also updating the "item-submission.xml" configuration to use that form (see also above):

1. In "item-submission.xml", a <step-definition> of type "submission-form" must be created, with an "id" matching the *name* of the *form* (see above for more details on step-definition)
2. In "item-submission.xml", a <submission-process> must be created/updated to use that newly defined "step".
3. Finally, also in "item-submission.xml", a Collection must be setup to use that submission process in the <submission-map> section.

So, if you modify submission-forms.xml, you may need to double check your changes will be used in your item-submission.xml.

Adding a Form

You can add a new form by creating a new *form* element within the *form-definitions* element. It has one attribute, *name*, which as described above must match the "id" of a <step-definition> in "item-submission.xml".

Forms and Pages

The content of the *form* is a sequence of *row* elements. Each of these corresponds to a single, horizontal row, containing metadata input fields. The rows are presented in sequence, with the first row displayed at the top of the form. A form is displayed as a section (or step) within the submission process.

A *form* may contain any number of rows. A row generally only contains one or two input fields (including more than one input field may require the "style" setting, see below). Each field defines an interactive dialog where the submitter enters one of the Dublin Core metadata items.

Composition of a Field

Each *field* contains the following elements, in the order indicated. The required sub-elements are so marked:

- **dc-schema** (Required) : Name of metadata schema employed, e.g. *dc* for Dublin Core. This value must match the value of the *schema* element defined in *dublin-core-types.xml*
- **dc-element** (Required) : Name of the Dublin Core element entered in this field, e.g. *contributor*.
- **dc-qualifier**: Qualifier of the Dublin Core element entered in this field, e.g. when the field is *contributor.advisor* the value of this element would be *advisor*. Leaving this out means the input is for an unqualified DC element.
- **language**: If set to *true* a drop down menu will be shown, containing languages. The selected language will be used as language tag of the metadata field. A compulsory argument *value-pairs-name* must be given containing the name of the value pair that contains all the languages: e. g. `<language value-pairs-name="common_iso_languages">true</language>`.
- **repeatable**: Value is *true* when multiple values of this field are allowed, *false* otherwise. When you mark a field repeatable, the UI will add an "Add more" control to the field, allowing the user to ask for more fields to enter additional values. Intended to be used for arbitrarily-repeating fields such as subject keywords, when it is impossible to know in advance how many input boxes to provide. Repeatable fields also support reordering of values.
- **label** (Required): Text to display as the label of this field, describing what to enter, e.g. "*Your Advisor's Name*".

- **input-type** (Required): Defines the kind of interactive widget to put in the form to collect the Dublin Core value. Content must be one of the following keywords:
    - **onebox** – A single text-entry box (i.e. a normal input textbox)
    - **textarea** – Large block of text that can be entered on multiple lines, e.g. for an abstract.
    - **name** – Personal name, with separate fields for family name and first name. When saved they are appended in the format 'LastName, FirstName'. *(By default, this input type is unused. Author fields now use the "onebox" type to support different types of names.)*
    - **date** – Calendar date. When required, demands that at least the year be entered.
    - **series** – Series/Report name and number. Separate fields are provided for series name and series number, but they are appended (with a semicolon between) when saved.
    - **dropdown** – Choose value(s) from a "drop-down" menu list.
        - Requires that you include a value for the *value-pairs-name* attribute to specify a list of menu entries from which to choose. Use this to make a choice from a restricted set of options, such as for the *language* item.
    - **qualdrop_value** – Enter a "qualified value", which includes *both* a qualifier from a drop-down menu and a free-text value. Used to enter items like alternate identifiers and codes for a submitted item, e.g. the DC *identifier* field.
        - Similar to the *dropdown* type, *requires* that you include the *value-pairs-name* attribute to specify a menu choice list.
        - Because the "qualdrop_value" dynamically sets the *qualifier* (based on the drop-down menu), the <dc-qualifier> field *MUST* be empty. The <dc-qualifier> element cannot be used with this field type.
    - **list** – Choose value(s) from a checkbox or radio button list. If the *repeatable* attribute is set to *true*, a list of checkboxes is displayed. If the *repeatable* attribute is set to *false*, a list of radio buttons is displayed. *(By default, this input type is unused.)*
        - Requires that you include a value for the *value-pairs-name* attribute to specify a list of values from which to choose.
    - **tag** - A free-text field which allows you to add multiple labels/tags as values. An example is the "Subject Keywords" field.
        - Note: A tag field MUST be marked as <repeatable>true</repeatable>.
- **hint** (Required): Content is the text that will appear as a "hint", or instructions, below the input field. Can be left empty, but the tag must be present.
- **required**: When this element is included with any content, it marks the field as a required input. If the user saves the form without entering a value for this field, that text is displayed as a warning message. For example, *<required>You must enter a title.</required>* Note that leaving the required element empty will *not* mark a field as required, e.g.:*<required></required>*
- **vocabulary:** When specified, this field uses a controlled vocabulary defined in `[dspace]/config/controlled-vocabularies/[name].xml`. This setting may be used to provide auto-complete functionality, for example in the "Subject Keywords" field (which uses the "tag" input type). See also the "Configuring Controlled Vocabularies" section below.
- **regex:** When specified, this field will be validated against the Regular Expression, and only successfully validating values will be saved. An example is commented out in the default "Author" field. If the validation fails, the following error message will be shown by default: *"This input is restricted by the current pattern: {{ pattern }}."*. This can be customized, by adding an entry to the internalization files with the key `error.validation.pattern.schema_element_qualifier` and the schema, element and qualifier of the field. For example: *"error.validation.pattern.dc_identifier": "The identifier can only consist of numbers"*. For instructions on how to add custom entries see: Customize UI labels using Internationalization (i18n) files
- **style:** When specified, this provides a CSS style recommendation to the UI for how to style that field. This is primarily used when displaying multiple fields per row, so that you can tell the UI how many columns each field should use in that row. Keep in mind, these styles should follow the Bootstrap Grid System, where the number of columns adds up to 12. An example can be see in the default "Date of Issue" and "Publisher" fields, which are configured to use 4 (col-sm-4) and 8 (col-sm-8) columns respectively.
- **visibility:** the submission scope for which the field should be visible. Values allowed are *submission* or *workflow.* When one of the two options is given the filed will be visible only for the scope provided and it will be hidden otherwise.
- **readonly:** this option can be used only together with the *visibility* element, and it means the field should be a read-only input instead of being hidden out of the scope provided by the *visibility* element. The value allowed is *readonly*, e.g.: *<readonly>readonly</readonly>*

Visibility configuration examples

A field configured to be visible only with *submission* scope, while is hidden with *workflow* scope

```
<field>
    <dc-schema>dc</dc-schema>
    <dc-element>title</dc-element>
    <dc-qualifier>alternative</dc-qualifier>
    <repeatable>true</repeatable>
    <label>Other Titles</label>
    <input-type>onebox</input-type>
    <hint>If the item has any alternative titles, please enter them here.</hint>
    <required></required>
    <visibility>submission</visibility>
</field>
```

A field configured to be visible only with *workflow* scope, while is read-only with *submission* scope

```
<field>
    <dc-schema>dc</dc-schema>
    <dc-element>title</dc-element>
    <dc-qualifier>alternative</dc-qualifier>
    <repeatable>true</repeatable>
    <label>Other Titles</label>
    <input-type>onebox</input-type>
    <hint>If the item has any alternative titles, please enter them here.</hint>
    <required></required>
    <readonly>readonly</readonly>
    <visibility>workflow</visibility>
</field>
```

Item type Based Metadata Collection

Available in 7.3 and later

A field can be made visible depending on the value of *dc.type*. A new field element, <type-bind>, has been introduced to facilitate this. The <type-bind> takes a comma separated list of publication types. If the field is missing or empty, it will always be visible. In this example the field will only be visible if a value of "thesis" or "ebook" has been entered into *dc.type* on an earlier page:

```
<field>
    <dc-schema>dc</dc-schema>
    <dc-element>identifier</dc-element>
    <dc-qualifier>isbn</dc-qualifier>
    <label>ISBN</label>
    <type-bind>thesis,ebook</type-bind>
</field>
```

A field may be configured multiple times in the submission configuration with different values in type-bind. This is useful if a field is required for one type but not another, or should display a different label and hint message depending on the publication type:

```
<field>
    <dc-schema>dc</dc-schema>
    <dc-element>identifier</dc-element>
    <dc-qualifier>isbn</dc-qualifier>
    <label>ISBN</label>
    <type-bind>book,ebook</type-bind>
        <required>You must enter an ISBN for this book</required>
</field>

<field>
    <dc-schema>dc</dc-schema>
    <dc-element>identifier</dc-element>
    <dc-qualifier>isbn</dc-qualifier>
    <label>ISBN of Parent Publication</label>
    <type-bind>thesis,book chapter,letter</type-bind>
        <hint>Enter the ISBN of the book in which this was published</hint>
</field>
```

If a field is *required* but is bound to a type that does not match the submitted publication, the *required* value will be ignored.

Note: When the submitter changes the Type field, other fields (usually just below it) dynamically appear. There's a brief demo of this feature in the 2022-07-13 - DSpace 7 Q&A webinar at time 19:05. The submission process is one page, but it has collapsible sections, each of which corresponds to one of the old "pages".

Configuring Controlled Vocabularies

DSpace supports controlled vocabularies to confine the set of keywords that users can use while describing items. The need for a limited set of keywords is important since it eliminates the ambiguity of a free description system, consequently simplifying the task of finding specific items of information. The controlled vocabulary allows the user to choose from a defined set of keywords organized in an tree (taxonomy) and then use these keywords to describe items while they are being submitted.

The taxonomies are described in XML following this (very simple) structure:

```
<node id="acmccs98" label="ACMCCS98">
    <isComposedBy>
        <node id="A." label="General Literature">
            <isComposedBy>
                <node id="A.0" label="GENERAL"/>
                <node id="A.1" label="INTRODUCTORY AND SURVEY"/>
                ...
            </isComposedBy>
        </node>
        ...
    </isComposedBy>
</node>
```

You are free to use any application you want to create your controlled vocabularies. A simple text editor should be enough for small projects. Bigger projects will require more complex tools. You may use Protegé to create your taxonomies, save them as OWL and then use a XML Stylesheet (XSLT) to transform your documents to the appropriate format. Future enhancements to this add-on should make it compatible with standard schemas such as OWL or RDF.

New vocabularies should be placed in `[dspace]/config/controlled-vocabularies/` and must be according to the structure described.

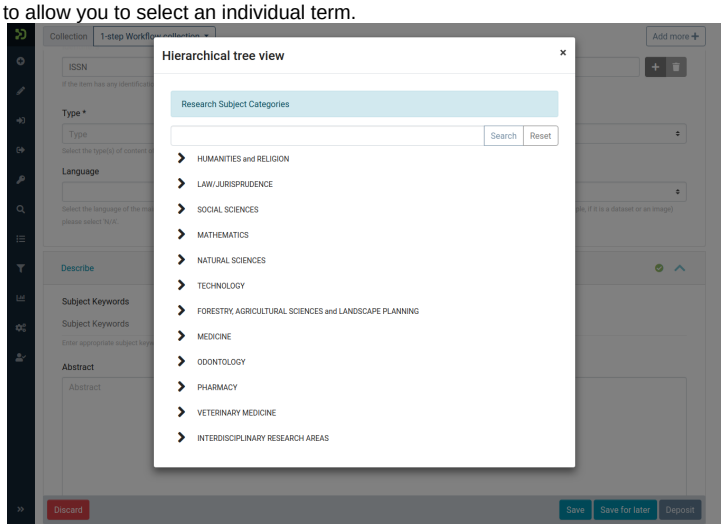
Vocabularies need to be associated with the corresponding metadata fields. Edit the file `[dspace]/config/submission-forms.xml` and place a `"vocabulary"` tag under the `"field"` element that you want to control. Set value of the `"vocabulary"` element to the name of the file that contains the vocabulary, leaving out the extension (the add-on will only load files with extension "*.xml"). For example:

```
<field>
    <dc-schema>dc</dc-schema>
    <dc-element>subject</dc-element>
    <dc-qualifier></dc-qualifier>
    <repeatable>true</repeatable>
    <label>Subject Keywords</label>
    <input-type>onebox</input-type>
    <hint>Enter appropriate subject keywords or phrases below.</hint>
    <required></required>
    <vocabulary>srsc</vocabulary>
</field>
```

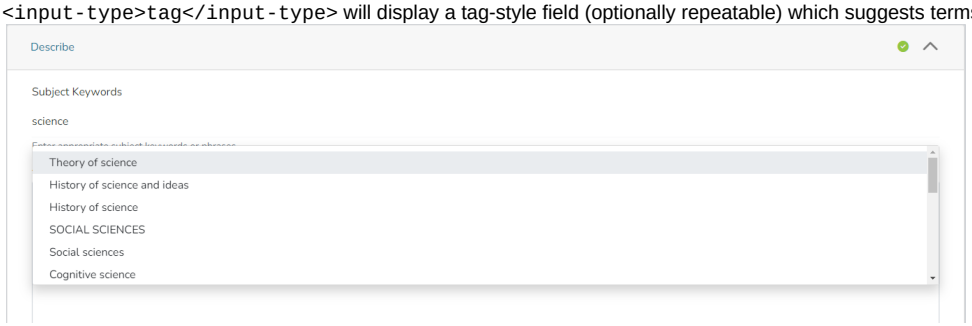
The vocabulary element has an optional boolean attribute closed that can be used to force input only with the Javascript of controlled-vocabulary add-on. The default behaviour (i.e. without this attribute) is as set closed="false". This allow the user also to enter the value in free way.

Controlled vocabularies have two main display types in the submission form:

1. `<input-type>onebox</input-type>` will display a onebox style field (optionally repeatable) which pops up the entire hierarchical vocabulary to allow you to select an individual term.



2. `<input-type>tag</input-type>` will display a tag-style field (optionally repeatable) which suggests terms within the vocabulary as you type.



The following vocabularies are currently available by default:

- **nsi** - *nsi.xml* - The Norwegian Science Index
- **srsc** - *srsc.xml* - Swedish Research Subject Categories

Adding Value-Pairs

Finally, your custom form description needs to define the "value pairs" for any fields with input types that refer to them. Do this by adding a *value-pairs* element to the contents of *form-value-pairs*. It has the following required attributes:

- **value-pairs-name** – Name by which an *input-type* refers to this list.
- **dc-term** – Dublin Core field for which this choice list is selecting a value.

Each *value-pairs* element contains a sequence of *pair* sub-elements, each of which in turn contains two elements:

- **displayed-value** – Name shown (on the web page) for the menu entry.
- **stored-value** – Value stored in the DC element when this entry is chosen. Unlike the HTML *select* tag, there is no way to indicate one of the entries should be the default, so the first entry is always the default choice.

Example

Here is a menu of types of common identifiers:

```
<value-pairs value-pairs-name="common_identifiers" dc-term="identifier">
    <pair>
        <displayed-value>Gov't Doc #</displayed-value>
        <stored-value>govdoc</stored-value>
    </pair>
    <pair>
        <displayed-value>URI</displayed-value>
        <stored-value>uri</stored-value>
    </pair>
    <pair>
        <displayed-value>ISBN</displayed-value>
        <stored-value>isbn</stored-value>
    </pair>
</value-pairs>
```

It generates the following HTML, which results in the menu widget below. (Note that there is no way to indicate a default choice in the custom input XML, so it cannot generate the HTML *SELECTED* attribute to mark one of the options as a pre-selected default.)

```
<select name="identifier_qualifier_0">
    <option VALUE="govdoc">Gov't Doc #</option>
    <option VALUE="uri">URI</option>
    <option VALUE="isbn">ISBN</option>
</select>
```

Deploying Your Custom Forms

The DSpace web application only reads your custom form definitions when it starts up, so it is important to remember:

- *You must always restart Tomcat* (or whatever servlet container you are using) for changes made to the *submission-forms.xml* and/or *item-submission.xml* to take effect.

Any mistake in the syntax or semantics of the form definitions, such as poorly formed XML or a reference to a nonexistent field name, may result in errors in the DSpace REST API & UI. The exception message (at the top of the stack trace in the *dspace.log* file) usually has a concise and helpful explanation of what went wrong. Don't forget to stop and restart the servlet container before testing your fix to a bug.

Configuring the File Upload step

Basic Settings

The *Upload* step in the DSpace submission process has a few configuration options which can be set with your *[dspace]/config/local.cfg* configuration file. They are as follows:

- *spring.servlet.multipart.max-file-size (default=512MB)* - Spring Boot's maximum allowable file upload size. For DSpace, we default it to 512MB (in application.properties). But, you may wish to override the default value in your local.cfg. Example values include "512MB", "1GB", or even "-1" (to allow unlimited). See Spring's documentation on this setting: https://spring.io/guides/gs/uploading-files/#_tuning_file_upload_limits
    - NOTE: Increasing this value significantly does NOT guarantee that DSpace will be able to successfully upload files of a very large size via the web. Large uploads depend on many other factors including bandwidth, web server settings, internet connection speed, etc. Therefore, for very large files, you may need to consider importing via command-line tools or similar.
- *spring.servlet.multipart.max-request-size (default=512MB)* - Spring Boot's maximum allowable upload *request* size (i.e. the maximum total upload size for all files in a multi-file upload). For DSpace, we default it to 512MB (in application.properties). But, you may wish to override the default value in your local.cfg. Example values include "512MB", "1GB", or even "-1" (to allow unlimited). See Spring's documentation on this setting: https://spring.io/guides/gs/uploading-files/#_tuning_file_upload_limits
    - NOTE: Increasing this value significantly does NOT guarantee that DSpace will be able to successfully upload files of a very large size via the web. Large uploads depend on many other factors including bandwidth, web server settings, internet connection speed, etc. Therefore, for very large files, you may need to consider importing via command-line tools or similar.
- *webui.submit.upload.required* - Whether or not all users are *required* to upload a file when they submit an item to DSpace. It defaults to 'true'. When set to 'false' users will see an option to skip the upload step when they submit a new item.

Modifying metadata form presented for Bitstreams

After uploading a file (bitstream) in the Submission UI, you can optionally edit that bitstream's metadata. The form displayed on that edit screen is built by the "bitstream-metadata" form defined in submission-forms.xml. You can modify that form to change the fields captured for a Bitstream. *However, the "dc.title" field is REQUIRED in order to store the name of the file.*

```
<form-definitions>
    <!-- Form used for entering in Bitstream/File metadata after uploading a file -->
    <form name="bitstream-metadata">
        ...
    </form>
</form-definitions>
```

Modifying access conditions (embargo, etc.) presented for Bitstreams

After uploading a file (bitstream) in the Submission UI, you can optionally edit that bitstream's access conditions. This allows you to embargo a bitstream, lease it, or limit it to Administrators only.

These access conditions are defined in a new Spring Bean configuration file `[dspace]/config/spring/api/access-conditions.xml`

- The "uploadConfigurationService" bean maps an existing "UploadConfiguration" bean (default is "uploadConfigurationDefault") to a specific step /section name used in item-submission.xml.

    ```
    <!-- This default configuration says the <step-definition id="upload"> defined in item-submission.xml
    uses "uploadConfigurationDefault" -->
    <bean id="uploadConfigurationService" class="org.dspace.submit.model.UploadConfigurationService">
        <property name="map">
            <map>
                <entry key="upload" value-ref="uploadConfigurationDefault" />
            </map>
        </property>
    </bean>
    ```

- One or more UploadConfiguration beans may exist, providing different options for different upload sections. An "UploadConfiguration" consists of several properties:
    - **name** (Required): The unique name of this upload configuration
    - **configurationService** (Required through 7.3): reference to the DSpace ConfigurationService (should always be "org.dspace.services.ConfigurationService"). Starting in 7.4 this is no longer required and should not be set.
    - **metadata** (Required): The metadata "form" to use for this upload configuration. The value specified here MUST correspond to a <form> defined in your submission-forms.xml. In the below example, the "bitstream-metadata" form is used by the "uploadConfigurationDefault" bean...meaning that form will be used to capture metadata about the uploaded bitstream.

- **options** (Required, but can be empty): list of all "AccessConditionOption" beans to enable. This list will be shown to the user to let them select which access restrictions to place on each bitstream. NOTE: *To disable the ability to select bitstream access restrictions, comment out all <ref> tags to create an empty list of options.*
- **maxSize:** Optionally, you can specify a maximum size of file accepted by this UploadConfiguration. If unspecified, default is to use the maximum file upload limits specified in Spring Boot (see "Basic Settings" above)
- **required**: Optionally, you can specify if a file upload is required for this UploadConfiguration. If true, upload is required and users cannot complete a submission without uploading at least one file. If false, no upload is required to complete the submission. If unspecified, default is to use "webui.submit.upload.required" configuration in dspace.cfg/local.cfg, which defaults to "true" (file upload required).

```
<bean id="uploadConfigurationDefault" class="org.dspace.submit.model.UploadConfiguration">
    <property name="name" value="upload"></property>
        <property name="configurationService" ref="org.dspace.services.ConfigurationService"/>
        <property name="metadata" value="bitstream-metadata" />
    <property name="options">
        <!-- This is the list of access options which will be displayed on the "bitstream-
metadata" form -->
        <!-- If no <ref> tags appear in this list, then access restrictions will not be allowed on
bitstreams -->
        <list>
            <ref bean="openAccess"/>
            <ref bean="lease"/>
            <ref bean="embargoed" />
            <ref bean="administrator"/>
        </list>
    </property>
</bean>
```

- Any number of "AccessConditionOption" beans may be added for applying different types of access permissions to uploaded files (based on which one the user selects). These beans are easy to add/update, and just require the following
  - **id** (Required): Each defined bean MUST have a unique "id" and have "class=org.dspace.submit.model.AccessConditionOption".
  - **groupName**: Optionally, define a specific DSpace Group which this Access Condition relates to. This group will be saved to the ResourcePolicy when this access condition is applied.
  - **name**: Give a unique name for this Access Condition. This name is stored in the ResourcePolicy "name" when this access condition is applied.
  - **hasStartDate**: If the access condition is time-based, you can decide whether a start date is required. (true = required start date, false = disabled/not required). This start date will be saved to the ResourcePolicy when this access condition is applied.
  - **startDateLimit**: If the access condition is time-based, you can optionally set an start date limit (e.g. +36MONTHS). This field is used to set an upper limit to the start date based on the current date. In other words, a value of "+36MONTHS" means that users cannot set a start date which is more than 3 years from today. This setting's value uses Solr's Date Math Syntax, and is always based on today (NOW).
  - **hasEndDate**: If the access condition is time-based, you can enable/disable whether an end date is required. (true = required end date, false = disabled/not required). This end date will be saved to the ResourcePolicy when this access condition is applied.
  - **endDateLimit**: If the access condition is time-based, you can optionally set an end date limit (e.g. +6MONTHS). This field is used to set an upper limit to the start date based on the current date. In other words, a value of "+6MONTHS" means that users cannot set an end date which is more than 6 months from today. This setting's value use Solr's Date Math Syntax, and is always based on today (NOW).

```
<!-- Example access option named "embargo", which lets users specify a future date
    (not more than 3 years from now) when this file will be available to Anonymous users -->
<bean id="embargoed" class="org.dspace.submit.model.AccessConditionOption">
        <property name="groupName" value="Anonymous"/>
        <property name="name" value="embargo"/>
        <property name="hasStartDate" value="true"/>
        <property name="startDateLimit" value="+36MONTHS"/>
        <property name="hasEndDate" value="false"/>
</bean>
```

- NOTE: It's possible to **test** the Date math syntax via command-line to see what the value would look like starting from today:

```
./dspace dsrun org.dspace.util.DateMathParser +999YEARS
Applied +999YEARS to implicit current time:  Wed Jun 25 19:42:48 UTC 3023
```

- By default, DSpace comes with these out-of-the-box Access Conditions (which you can customize/change based on local requirements)
  - "administrator" - access restricts the bitstream to the Administrator group immediately (after submission completes)
  - "openAccess" - makes the bitstream immediately accessible to Anonymous group (after submission completes)
  - "embargoed" - embargoes the bitstream for a period of time (maximum of 3 years, as defined in startDateLimit default setting), after which it becomes anonymously accessible. See also Embargo for discussion of how embargoes work in DSpace.
  - "lease" - makes the bitstream anonymously accessible immediately (after submission completes), but that access *expires* after a period of time (maximum of 6 months, as defined in endDateLimit default setting). After that date it is no longer accessible (except to Administrators)

Define upload of files mandatory or optional per collection

You will need to make the following changes to the *access-conditions.xml* file

1. Create a new <bean> below “uploadConfigurationDefault” with the following settings, adding only the line `<property name="required" value="false">`

```
<!-- Define a new bean that sets "required=false" for the UploadConfiguration -->
<bean id="uploadConfigurationWithoutMandatory"
      class="org.dspace.submit.model.UploadConfiguration">
    <property name="name" value="upload" />
    <property name="metadata" value="bitstream-metadata" />
    <property name="required" value="false" />
    <property name="options">
        <list>
            <ref bean="openAccess" />
            <ref bean="lease" />
            <ref bean="embargoed" />
            <ref bean="administrator" />
        </list>
     </property>
</bean>
```

2. Create a new <bean> below "uploadConfigurationService" with the following settings:

```
<!-- Define an UploadConfigurationService which makes Bitstream uploading optional -->
<bean id="uploadConfigurationServiceWithoutMandatory"
    class="org.dspace.submit.model.UploadConfigurationService">
    <property name="map">
        <map>
           <entry key="uploadOptional" value-ref="uploadConfigurationWithoutMandatory" />
        </map>
    </property>
</bean>
```

Now, in the *item-submission.xml* file, you need to make the following changes:

1. Create a new <step-definition> below “upload” with the following settings:

```
<!-- Create an uploadOptional step which will use the
     new "uploadConfigurationServiceWithoutMandatory"
     that you defined in your access-conditions.xml -->
<step-definition id="uploadOptional">
    <heading>submit.progressbar.upload</heading>
    <processing-class>org.dspace.app.rest.submit.step.UploadStep</processing-class>
    <type>upload</type>
</step-definition>
```

2. Finally, what you need to do is use “uploadOptional” as a <step>, replacing default upload, for example:

```
<!-- This first example is a submission form that *REQUIRES* file upload
     by using the existing "upload" step. -->
<submission-process name="traditional">
    <step id="collection" />
    <step id="traditionalpageone" />
    <step id="traditionalpagetwo" />
    <step id="upload" />
    <step id="license" />
</submission-process>

<!-- This second example is a submission form that makes file upload *OPTIONAL*
     by using the newly defined "uploadOptional" step. -->
<submission-process name="traditionalUploadOptional">
    <step id="collection" />
    <step id="traditionalpageone" />
    <step id="traditionalpagetwo" />
    <step id="uploadOptional" />
```

```
    <step id="license" />
</submission-process>
```

At this point, any Collection which uses the "traditionalUploadOptional" submission-process will no longer require files to be uploaded. But, any Collection which still uses the "traditional" submission-process will still require files to be uploaded.

Configuring the Item Access Conditions step

Enabling the step

By default, the "Item Access Conditions" step is disabled. To enable it, simply update your `item-submission.xml` to include this tag in your <submission-process>:

```
<submission-process name="traditional">
   ...

   <!-- This step enables embargoes and other access restrictions at the Item level -->
   <step id="itemAccessConditions"/>
</submission-process>
```

After making this update, you will need to restart your backend (REST API) for the changes to take effect.

Modifying access conditions (embargo, etc.) presented for Items

The "Item Access Conditions" step uses a similar access condition configuration as the "Upload" step as described in the Modifying access conditions (embargo, etc.) presented for Bitstreams documentation above.

All available Item access conditions are defined in a new Spring Bean configuration file `[dspace]/config/spring/api/access-conditions.xml`

- One or more "AcccessConditionConfiguration" beans may exist, providing different options for different submission forms (only one should be in use in a form at a time). By default an "accessConditionConfigurationDefault" bean is defined. An "AccessConditionConfiguration" consists of several properties:
    - **name** (Required): The unique name of this configuration. It must match the "id" of the step defined in your `item-submission.xml`
    - **canChangeDiscoverable:** Whether this configuration allows users to change the discoverability of an Item. A "discoverable" item is one that is findable through all search/browse interfaces, provided that you have access to see that Item. A "non-discoverable" item is one that will never be findable through search/browse (except by Administrators)... instead a direct link is necessary to view the Item. See also DSpace Item State Definitions. When "canChangeDiscoverable" is "true", the user can modify discoverability in this submission section. When set to "false", the user cannot modify this setting and all submitted Items will be "discoverable".
    - **options** (Required): list of all "AccessConditionOption" beans to enable for this Item access conditions step. This list will be shown to the user to let them select which access restrictions to place on this Item.
- This step uses the same "AccessConditionOption" beans as the "Upload" step, as described in the Modifying access conditions (embargo, etc.) presented for Bitstreams documentation above. You can choose to enable the same options for both Items and Bitstreams, or provide different options for each.
- By default, DSpace comes with these out-of-the-box Access Conditions (which you can customize/change based on local requirements)
    - "administrator" - access restricts the bitstream to the Administrator group immediately (after submission completes)
    - "openAccess" - makes the bitstream immediately accessible to Anonymous group (after submission completes)
    - "embargoed" - embargoes the bitstream for a period of time (maximum of 3 years, as defined in startDateLimit default setting), after which it becomes anonymously accessible. See also Embargo for discussion of how embargoes work in DSpace.
    - "lease" - makes the bitstream anonymously accessible immediately (after submission completes), but that access *expires* after a period of time (maximum of 6 months, as defined in endDateLimit default setting). After that date it is no longer accessible (except to Administrators)

Examples of selecting Item and Bitstream access conditions

What happens when a User selects different access conditions for an Item (via the "Item Access Conditions" step) and its files (via the "Upload" step)? Generally speaking, both

Generally speaking, both access restrictions will be applied. Here's some examples:

- If a user selects "openAccess" in the "Item Access Conditions" step AND "embargo" in the "Upload" step for one Bitstream
    - Then, the Item's metadata will be publicly visible, but that single Bitstream will be embargoed.
- If a user selects "openAccess" in the "Item Access Conditions" step AND "administrator" in the "Upload" step for one Bitstream
    - Then, the Item's metadata will be publicly visible, but that single Bitstream will only be visible to Administrators
- If a user selects "administrator" in the "Item Access Conditions" step AND nothing in the "Upload" step.
    - Then, the Item's metadata and all Bitstreams will only be accessible to administrators.
- If a user selects "embargo" in the "Item Access Conditions" step AND nothing in the "Upload" step.
    - Then, the Item's metadata and all Bitstreams will be embargoed. Nothing will be visible in the system until the embargo data passes.
- If a user selects "embargo" in the "Item Access Conditions" step AND "openAccess" in the "Upload" step for one Bitstream.
    - Then, the Item's metadata will be embargoed (making it impossible to find the Item unless you are an Administrator). HOWEVER, the bitstream will be publicly accessible immediately (but only via a direct link, as it won't be searchable in the system until the embargo date passes).

- (To test other scenarios, submit a test Item with those permissions applied. Then, edit that Item, visit the "Status" tab, and click "Authorizations" to see what access restrictions were applied to the Item and its bitstreams.)

Configuring the Sherpa Romeo step

Enabling the Sherpa Romeo step

By default, the "Sherpa RoMEO Policy" step is disabled. To enable it, simply update your `item-submission.xml` to include this tag in your <submission-process>:

```
<submission-process name="traditional">
   ...
   <!-- This step shows when appropriate publisher policies retrieved from SHERPA/RoMEO -->
   <step id="sherpaPolicies"/>
</submission-process>
```

you must also obtain your `sherpa.romeo.apikey` registering you client application here https://v2.sherpa.ac.uk/api/ and put them in the `local.cfg`

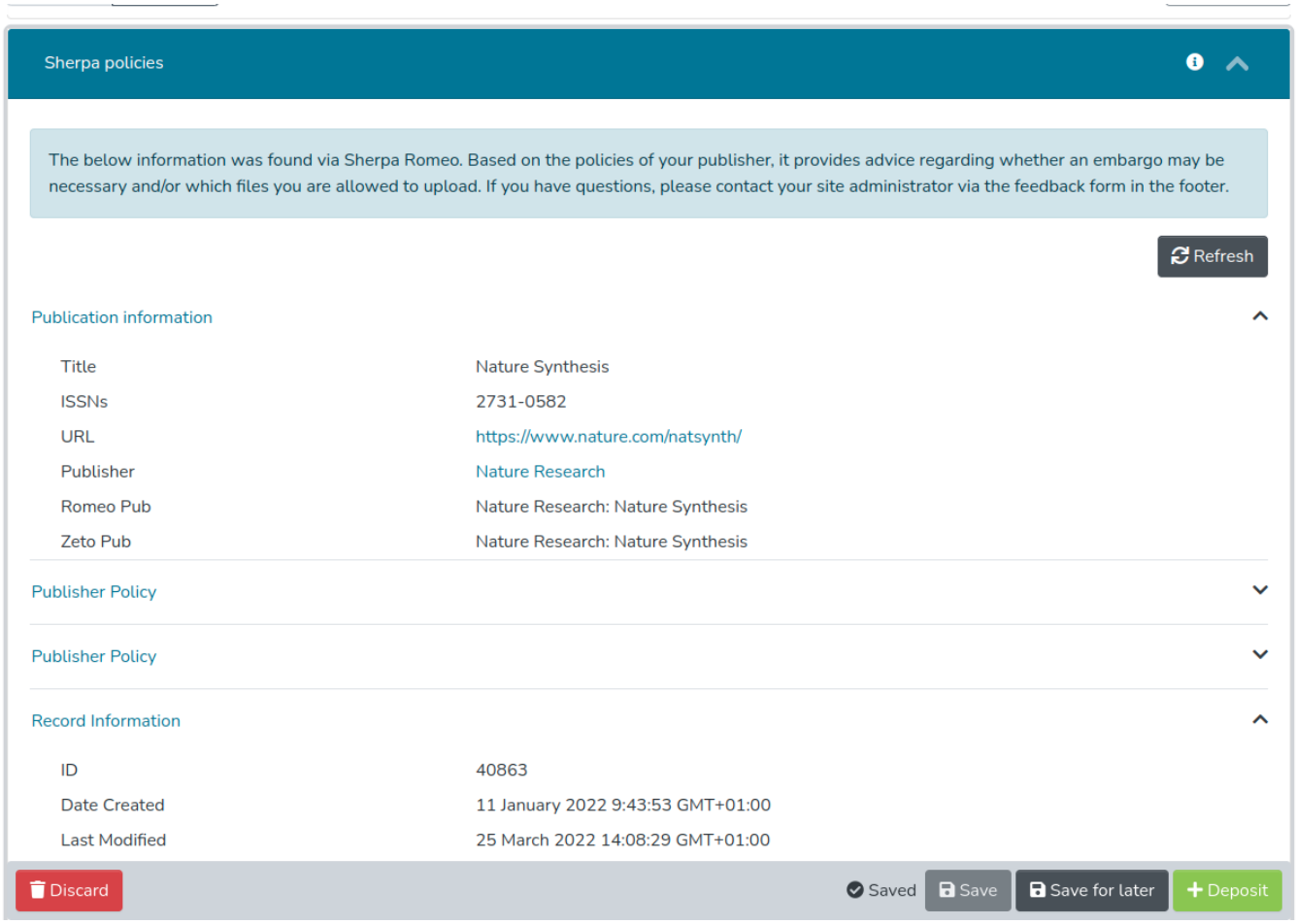
```
sherpa.romeo.apikey = <YOUR-API-KEY>
```

The step needs to extract the ISSN of the Journal where the publication has been submitted/published to query the Sherpa/RoMEO database in order to visualize the publisher policies, this is done by an implementation of the `org.dspace.app.sherpa.submit.ISSNItemExtractor` interface configured in the `org.dspace.app.sherpa.submit.SHERPASubmitConfigurationService`

The configuration is provided by Spring `config/spring/api/sherpa.xml`

```
        <bean class="org.dspace.app.sherpa.submit.SHERPASubmitConfigurationService"
                id="org.dspace.app.sherpa.submit.SHERPASubmitConfigurationService">
                <property name="issnItemExtractors">
                        <list>
                                <bean class="org.dspace.app.sherpa.submit.MetadataValueISSNExtractor">
                                        <property name="metadataList">
                                                <list>
                                                        <value>dc.identifier.issn</value>
                                                </list>
                                        </property>
                                </bean>
                                <!-- Uncomment this bean if you have SHERPARoMEOJournalTitle enabled
                                <bean class="org.dspace.app.sherpa.submit.MetadataAuthorityISSNExtractor">
                                        <property name="metadataList">
                                                <list>
                                                        <value>dc.title.alternative</value>
                                                </list>
                                        </property>
                                </bean>  -->
                        </list>
                </property>
        </bean>
```

out-of-box implementations able to extract the ISSN from the metadata value or authority are provided.



Configuring the "Identifiers" step

By default, the "Identifiers" step is disabled. To enable it, update your `item-submission.xml` to include this tag in your <submission-process>:

```
<submission-process name="traditional">

   ...

   <!-- This step shows identifiers already registered for this in-progress item

   <step id="identifiers"/>

   ...

</submission-process>
```

It is recommended to display this step above most others so that the submitter can clearly see any identifiers that will be created while completing their submission.

You must also enable registration of identifiers for workspace and workflow items in `dspace/config/modules/identifiers.cfg` or `local.cfg` (this is disabled by default):

```
identifiers.submission.register = true
```

While editing this configuration, pay attention to the filter configuration - logical item filters can be referenced here to apply some conditions as to whether an item qualifies for a DOI or not (eg. based on metadata entered, the type of work, or so on).

Any identifiers registered for the current submission or workflow item will be displayed in a read-only section. If no identifiers are registered, a placeholder “no identifiers” message will be displayed.

If DOI registration is configured for logical item filtering, the DOI will be minted (in a 'pending' state) or deleted as appropriate whenever the in-progress item is saved, depending on whether it passes the filter test.

See `dspace/config/modules/identifiers.cfg`

Creating new Submission Steps Programmatically.

First, a brief warning: *Creating a new Submission Step requires some Java knowledge, and is therefore recommended to be undertaken by a Java programmer whenever possible.*

In most scenarios, this is NOT necessary, as it's much easier to configure a custom Submission Step using DescribeStep or similar.

That being said, at a higher level, creating a new Submission Step requires the following (in this relative order):

1. Create a new Step Processing class
    - This class **must** extend the abstract `org.dspace.submit.AbstractProcessingStep` class and implement all methods defined by that abstract class.
    - This class should be built in such a way that it can process the input gathered from the UI
2. Add a valid Step Definition to the `item-submission.xml` configuration file.
    - This may also require that you add an I18N (Internationalization) key for this step's *heading to the UI*
    - For more information on *<step-definition> tags* within the `item-submission.xml`, see the section above on Defining Steps (*<step>*) within the `item-submission.xml`.
3. For the UI, you will need to..
    - Add a new section type to SectionsType enum matching to the type of step you are creating
    - Create a new Component for this new SectionsType, annotated with "@renderSectionFor()".... see existing section components under src/app/submission/sections for examples.
    - *(Other steps may be necessary... this process has not been fully documented at this time.)*

Live Import from external sources

1 General Framework
- 1.1 Introduction
- 1.2 Features
- 1.3 Abstraction of input format
- 1.4 Editing Metadata Mapping
- 1.5 Transformation to DSpace Item
  - 1.5.1 Implementation of an import source for External Sources
  - 1.5.2 Implementation of an import source for files
  - 1.5.3 Mapping raw data to Metadata
  - 1.5.4 Inherited methods
  - 1.5.5 Spring configuration for External Sources
  - 1.5.6 Metadata mapping
  - 1.5.7 Available Metadata Contributor classes
- 1.6 Framework Sources Implementations
  - 1.6.1 PubMed Integration
    - 1.6.1.1 Introduction
    - 1.6.1.2 Publication Lookup URL
    - 1.6.1.3 PubMed Metadata Mapping
    - 1.6.1.4 PubMed specific classes Config
      - 1.6.1.4.1 Metadata mapping classes
      - 1.6.1.4.2 Service classes
  - 1.6.2 ArXiv Integration
    - 1.6.2.1 ArXiv Metadata Mapping

General Framework

Introduction

This framework is used by both the REST API and User Interface to help enhance or enrich submissions. One examples usage is in Importing Items via basic bibliographic formats (Endnote, BibTex, RIS, CSV, etc) and online services (arXiv, PubMed, CrossRef, CiNii, etc)

Features

- lookup publications from remote sources
- Support for multiple implementations

Abstraction of input format

The importer framework does not enforce a specific input format. Each importer implementation defines which input format it expects from a remote source. The import framework uses generics to achieve this. Each importer implementation will have a type set of the record type it receives from the remote source's response. This type set will also be used by the framework to use the correct MetadataFieldMapping for a certain implementation. Read "I mplementation of an import source" below for more information and how to enable the framework.

Editing Metadata Mapping

At a simple level, metadata mapping configurations are all in Spring configs in `[dspace.dir]/config/spring/api/`

In that directory, you'll find a mapping file per import source, e.g. "arxiv-integration.xml", "bibtex-integration.xml", "endnote-integration.xml", "pubmed-integration.xml", etc.

There are two different mapping types.

1. First, mapping from a file-based import (e.g. bibtex, endnote, ris, etc) to a DSpace metadata field.
   a. The list of all of the enabled mappings can be found in a "MetadataFieldConfig" <util:map>, usually at the top of the config file.

```
<util:map id="bibtexMetadataFieldMap" key-type="org.dspace.importer.external.metadatamapping.
MetadataFieldConfig"
             value-type="org.dspace.importer.external.metadatamapping.contributor.
MetadataContributor">
    <description>Defines which metadatum is mapped on which metadatum. Note that while the key
must be unique it
        only matters here for postprocessing of the value. The mapped MetadatumContributor has
full control over
           what metadatafield is generated.
    </description>
    <!-- These entry tags are the enabled mappings. The "value-ref" must map to a <bean> ID. -->
    <entry key-ref="dcTitle" value-ref="bibtexTitleContrib" />
    <entry key-ref="dcAuthors" value-ref="bibtexAuthorsContrib" />
    <entry key-ref="dcJournal" value-ref="bibtexJournalContrib" />
    <entry key-ref="dcIssued" value-ref="bibtexIssuedContrib" />
    <entry key-ref="dcJissn" value-ref="bibtexJissnContrib" />
</util:map>
```

b. Each field in the file is mapped to a DSpace metadata field in a "SimpleMetadataContributor" bean definition. *NOTE: a large number of DSpace defined metadata fields are already configured as MetadataFieldConfig beans in the "dublincore-metadata-mapper.xml" Spring Config in the same directory. These may be reused in other configurations.*

```
<!-- This example bean for BibTex says the "title" key in the BibTex" file should be mapped to the
DSpace metadata field
     defined in the "dcTitle" bean.  This "dcTitle" bean is found in "dublincore-metadata-mapper.
xml" and obviously maps to "dc.title" -->
<bean id="bibtexTitleContrib" class="org.dspace.importer.external.metadatamapping.contributor.
SimpleMetadataContributor">
    <property name="field" ref="dcTitle"/>
    <property name="key" value="title" />
</bean>
```

2. Second, mapping from an external API query import (e.g. arxiv, pubmed, etc) to a DSpace metadata field.
   a. Similar to above, The list of all of the enabled mappings can be found in a "MetadataFieldConfig" <util:map>, usually at the top of the config file.

```
<util:map id="arxivMetadataFieldMap" key-type="org.dspace.importer.external.metadatamapping.
MetadataFieldConfig"
             value-type="org.dspace.importer.external.metadatamapping.contributor.
MetadataContributor">
    <description>Defines which metadatum is mapped on which metadatum. Note that while the key
must be unique it
        only matters here for postprocessing of the value. The mapped MetadatumContributor has
full control over
        what metadatafield is generated.
    </description>
    <!-- These entry tags are the enabled mappings. The "value-ref" must map to a <bean> ID. -->
    <entry key-ref="arxiv.title" value-ref="arxivTitleContrib"/>
    <entry key-ref="arxiv.summary" value-ref="arxivSummaryContrib"/>
    <entry key-ref="arxiv.published" value-ref="arxivPublishedContrib"/>
    <entry key-ref="arxiv.arxiv.doi" value-ref="arxivDoiContrib"/>
    <entry key-ref="arxiv.arxiv.journal_ref" value-ref="arxivJournalContrib"/>
    <entry key-ref="arxiv.category.term" value-ref="arxivCategoryTermContrib"/>
    <entry key-ref="arxiv.author.name" value-ref="arxivAuthorContrib"/>
    <entry key-ref="arxiv.identifier.other" value-ref="arxivOtherContrib"/>
</util:map>
```

   b. Each field in the file is mapped to a DSpace metadata field, usually in a "SimpleXPathMetadatumContributor" bean definition which also uses a "MetadataFieldConfig" bean. *NOTE: a large number of DSpace defined metadata fields are already configured as MetadataFieldConfig beans in the "dublincore-metadata-mapper.xml" Spring Config in the same directory. These may be reused in other configurations.*

```
<!-- This first bean define an XPath query ("ns:title") to map to a field (ID="arxiv.title") in
DSpace -->
<bean id="arxivTitleContrib" class="org.dspace.importer.external.metadatamapping.contributor.
SimpleXpathMetadatumContributor">
    <property name="field" ref="arxiv.title"/>
    <property name="query" value="ns:title"/>
    <property name="prefixToNamespaceMapping" ref="arxivBasePrefixToNamespaceMapping"/>
</bean>
<!-- This second bean then defines which DSpace field to use when "arxiv.title" is references. In
other words, between these two beans,
     the "ns:title" XPath query value is saved to "dc.title". -->
<bean id="arxiv.title" class="org.dspace.importer.external.metadatamapping.MetadataFieldConfig">
    <constructor-arg value="dc.title"/>
</bean>
```

Transformation to DSpace Item

The framework produces an 'ImportRecord' that is completely decoupled from DSpace. It contains a set of metadata DTO's that contain the notion of schema,element and qualifier. The specific implementation is responsible for populating this set. It is then very simple to create a DSpace item from this list.

Implementation of an import source for External Sources

Each external source/API importer implementation must at least implements `org.dspace.importer.external.service.components.QuerySource`, which provides the query method used by the framework to retrieve data from the remote source (e.g. Pubmed, ArXiv, etc). Each external source importer must implements, according to the provider APIs, the declared methods.
An useful abstract for remote sources is `org.dspace.importer.external.service.components.AbstractRemoteMetadataSource`.This class contains functionality to handle request timeout and to retry requests. Using this abstract, the query method must implements java.util.concurrent.Callable.

Implementation of an import source for files

Each file importer implementation must at least implements `org.dspace.importer.external.service.components.FileSource`, which provides the basic methods used by the framework to parse and load data from the file (e.g. CSV, Endnote, etc).

Each importer must implements the method:

```
public List<ImportRecord> getRecords(InputStream inputStream) throws FileSourceException;
```

This method is responsible to transform the input data into an ImportRecord list, which will then managed by the top layer of the framework.

The conversion from raw data to an ImportRecord could be done using the framework too, using the metadata mapping structure (see below).

File sources needs to know which file extensions they have to supports. This is done by the default method `isValidSourceForFile` in `FileSource`, and is controlled by the entries in the list returned by declared method `public List<String> getSupportedExtensions();`

An useful abstract for file source is `org.dspace.importer.external.service.components.AbstractPlainMetadataSource`. It should be used whenever it is possible to model the data in the file as a list of key-value lists (e.g. for CSV files, any row is a key value list).

Mapping raw data to Metadata

The framework core is a mid-layer component which allow the conversion of raw data into metadata (ImportRecord) using xml configurable spring beans.

The core of this approach is `org.dspace.importer.externa.service.AbstractImportMetadataSourceService`. Any service that wants to generate metadata from raw data should go through this abstract.

Our service then should extends AbstractImportMetadataSourceService, and use `transformSourceRecords` to transform raw data into ImportRecords.

The most relevant concept in the framework is `private MetadataFieldMapping<RecordType, MetadataContributor<RecordType>> metadataFieldMapping`. This is the place where the framework take the mapping between row data and the associated metadatum. This map must be injected in the service, and will be used by `transformSourceRecords` to convert the data.

`RecordType` is a generic type, which rapresent a single entry of the list of data, and will be mapped to a single ImportRecord. Any metadatum will be mapped to a specific field in the RecordType using a Contributor as described in Metadata mapping.

Inherited methods

Method getImportSource() should return a unique identifier. Importer implementations should not be called directly, but class *org.dspace.importer.external.service.ImportService* should be called instead. This class contains the same methods as the importer implementations, but with an extra parameter 'url'. This url parameter should contain the same identifier that is returned by the getImportSource() method of the importer implementation you want to use.

The other inherited methods are used to query the remote source.

Spring configuration for External Sources

In order to make the live import providers available, them must be mapped as spring beans into `dspace-api/src/main/resources/spring/spring-dspace-addon-import-services.xml`.

This is an example of a provider which allow to import both files and remote source.

```
<bean id="PubmedImportService"
      class="org.dspace.importer.external.pubmed.service.PubmedImportMetadataSourceServiceImpl" scope="
singleton">
    <property name="metadataFieldMapping" ref="PubmedMetadataFieldMapping"/>
    <property name="supportedExtensions">
        <list>
            <value>xml</value>
        </list>
    </property>
    ...
</bean>
```

Here is defined the service responsible to fetch and transform the data `PubmedImportMetadataSourceServiceImpl`, which is an extension of `AbstractImportMetadataSourceService` as described above.

The field `metadataFieldMapping` is an instance of `Map<MetadataFieldConfig,MetadataContributor>` and contains the effective mapping.

`supportedExtensions` is the file extension this provider supports.

To expose this provider as Live Import provider, we need to construct a bean of type `org.dspace.external.provider.impl.LiveImportDataProvider` in the following way

```
<bean id="pubmedLiveImportDataProvider" class="org.dspace.external.provider.impl.LiveImportDataProvider">
    <property name="metadataSource" ref="PubmedImportService"/>
    <property name="sourceIdentifier" value="pubmed"/>
    <property name="recordIdMetadata" value="dc.identifier.other"/>
</bean>
```

where `metadataSource` is the bean referencing to live import service as described in “Metadata mapping”, `sourceIdentifier` the name of the provider in the live import framework and `recordIdMetadata` the metadatum used as id of the ImportRecord.

Metadata mapping

When using an implementation of `AbstractImportSourceService`, a mapping of remote record fields to DSpace metadata fields can be created.

first create an implementation of class `AbstractMetadataFieldMapping` with the same type set used for the importer implementation.

Then create a spring configuration file in `[dspace.dir]/config/spring/api`.

Each DSpace metadata field that will be used for the mapping must first be configured as a spring bean of class*org.dspace.importer.external.metadatamapping.MetadataFieldConfig*.

```
   <bean id="dc.title" class="org.dspace.importer.external.metadatamapping.MetadataFieldConfig">
       <constructor-arg value="dc.title"/>
   </bean>
```

NOTE: A large number of these MetadataFieldConfig definitions are already provided out-of-the-box in `[dspace.dir]/config/spring/api/dublincore-metadata-mapper.xml` This allows most service-specific Spring configurations to just reuse those existing MetadataFieldConfig definitions

Now this metadata field can be used to create a mapping. To add a mapping for the "dc.title" field declared above, a new spring bean configuration of a class class *org.dspace.importer.external.metadatamapping.contributor.MetadataContributor* needs to be added. This interface contains a type argument. The type needs to match the type used in the implementation of AbstractImportSourceService. The responsibility of each MetadataContributor implementation is to generate a set of metadata from the retrieved document. How it does that is completely opaque to the AbstractImportSourceService but it is assumed that only one entity (i.e. item) is fed to the metadatum contributor.

For example `java SimpleXpathMetadatumContributor implements MetadataContributor<OMElement>` can parse a fragment of xml and generate one or more metadata values.

This bean expects 2 property values:

- field: A reference to the configured spring bean of the DSpace metadata field. e.g. the "dc.title" bean declared above.
- query: The xpath expression used to select the record value returned by the remote source.

```
    <bean id="titleContrib" class="org.dspace.importer.external.metadatamapping.contributor.
SimpleXpathMetadatumContributor">
        <property name="field" ref="dc.title"/>
        <property name="query" value="dc:title"/>
    </bean>
```

Multiple record fields can also be combined into one value. To implement a combined mapping first create a *SimpleXpathMetadatumContributor* as explained above for each part of the field.

```
    <bean id="lastNameContrib" class="org.dspace.importer.external.metadatamapping.contributor.
SimpleXpathMetadatumContributor">
        <property name="field" ref="dc.contributor.author"/>
        <property name="query" value="x:authors/x:author/x:surname"/>
    </bean>
    <bean id="firstNameContrib" class="org.dspace.importer.external.metadatamapping.contributor.
SimpleXpathMetadatumContributor">
        <property name="field" ref="dc.contributor.author"/>
        <property name="query" value="x:authors/x:author/x:given-name"/>
    </bean>
```

Note that namespace prefixes used in the xpath queries are configured in bean "FullprefixMapping" in the same spring file.

```
    <util:map id="FullprefixMapping" key-type="java.lang.String" value-type="java.lang.String">
        <description>Defines the namespace mappin for the SimpleXpathMetadatum contributors</description>
        <entry key="http://purl.org/dc/elements/1.1/" value="dc"/>
        <entry key="http://www.w3.org/2005/Atom" value="x"/>
    </util:map>
```

Then create a new list in the spring configuration containing references to all *SimpleXpathMetadatumContributor* beans that need to be combined.

```
   <util:list id="combinedauthorList" value-type="org.dspace.importer.external.metadatamapping.contributor.
MetadataContributor" list-class="java.util.LinkedList">

        <ref bean="lastNameContrib"/>
        <ref bean="firstNameContrib"/>
    </util:list>
```

Finally create a spring bean configuration of class*org.dspace.importer.external.metadatamapping.contributor.CombinedMetadatumContributor*. This bean expects 3 values:

- field: A reference to the configured spring bean of the DSpace metadata field. e.g. the "dc.title" bean declared above.
- metadatumContributors: A reference to the list containing all the single record field mappings that need to be combined.
- separator: These characters will be added between each record field value when they are combined into one field.

```
    <bean id="authorContrib" class="org.dspace.importer.external.metadatamapping.contributor.
CombinedMetadatumContributor">
        <property name="separator" value=", "/>
        <property name="metadatumContributors" ref="combinedauthorList"/>
        <property name="field" ref="dc.contributor.author"/>
    </bean>
```

Each contributor must also be added to the "MetadataFieldMap" used by the *MetadataFieldMapping* implementation. Each entry of this map maps a metadata field bean to a contributor. For the contributors created above this results in the following configuration:

```
    <util:map id="org.dspace.importer.external.metadatamapping.MetadataFieldConfig"
              value-type="org.dspace.importer.external.metadatamapping.contributor.MetadataContributor">
        <entry key-ref="dc.title" value-ref="titleContrib"/>
        <entry key-ref="dc.contributor.author" value-ref="authorContrib"/>
    </util:map>
```

Note that the single field mappings used for the combined author mapping are not added to this list.

Available Metadata Contributor classes

Class	Description
SimpleXpathMetadatumContributor	Use an XPath expression to map the XPath result to a metadatum
SimpleMetadataContributor	This contributor is used in plain metadata as exposed above. Mapping is easy because it is based on the key used in the DTO.
CombinedMetadatumContributor	Use a LinkedList of MetadataContributor to combine into the value the resulting value for each contributor.

Framework Sources Implementations

PubMed Integration

Introduction

First read the base documentation on external importing (see above). This documentation explains the implementation of the importer framework using PubMed (http://www.ncbi.nlm.nih.gov/pubmed) as an example.

Publication Lookup URL

To be able to do the lookup for our configured import-service, we need to be able to know what URL to use to check for publications. This URL the `publication-lookup.url` setting defined within the `[dspace.dir]/config/modules/publication-lookup.cfg`. You may choose to modify this setting or override it within your local.cfg.

This setting can be modified in one of two ways:

- You can choose to specific a single, specific URL. This will tell the lookup service to only use one location to lookup publication information. Valid URLs are any that are defined as a `baseAddress` for beans within the `[src]/dspace-api/src/main/resources/spring/spring-dspace-addon-import-services.xml` Spring config file.
    - For example, this setting will ONLY use PubMed for lookups: `publication-lookup.url=http://eutils.ncbi.nlm.nih.gov/entrez/eutils/`
- By default, `publication-lookup.url` is set to an asterisk ('*'). This default value will attempt to lookup the publication using ALL configured importServices in the `[src]/dspace-api/src/main/resources/spring/spring-dspace-addon-import-services.xml` Spring config file

PubMed Metadata Mapping

The PubMed metadata mappings are defined in the `[dspace.dir]/config/spring/api/pubmed-integration.xml` Spring configuration file. These metadata mappings can be tweaked as desired. The format of this file is described in the "Metadata mapping" section above

PubMed specific classes Config

These classes are simply implementations based of the base classes defined in importer/external. They add characteristic behavior for services/mapping for the PubMed specific data.

Metadata mapping classes

- "PubmedFieldMapping". An implementation of AbstractMetadataFieldMapping, linking to the bean that serves as the entry point of other metadata mapping
- "PubmedDateMetadatumContributor"/"PubmedLanguageMetadatumContributor". Pubmed specific implementations of the "MetadataContributor" interface

Service classes

- "GeneratePubmedQueryService". Generates the pubmed query which is used to retrieve the records. This is based on a given item.
- "PubmedImportMetadataSourceServiceImpl". Child class of "AbstractImportMetadataSourceService", retrieving the records from pubmed.

ArXiv Integration

ArXiv Metadata Mapping

The ArXiv metadata mappings are defined in the `[dspace.dir]/config/spring/api/arxiv-integration.xml` Spring configuration file. These metadata mappings can be tweaked as desired. The format of this file is described in the "Metadata mapping" section above

Simple HTML Fragment Markup

A few features of the user interface, such as the deposit license text & some metadata fields, can be marked up using a subset of HTML. This HTML subset is defined by Angular, as we use Angular's "[innerHtml]" property to display these HTML-based fields.

Angular automatically sanitizes any HTML passed to "[innerHtml]" in order to avoid XSS attacks. See Angular docs at https://angular.io/guide/security#preventing-cross-site-scripting-xss

At this time, Angular does NOT have a formal reference of elements/attributes which are allowed, but we've compiled a list below of currently known acceptable elements. This list may change in later releases of Angular, but is currently maintained in Angular's "html_sanitizer.ts": https://github.com/angular/angular/blob/main/packages/core/src/sanitization/html_sanitizer.ts

As of the writing of this page, these HTML5 elements may be used:

- h1, h2, h3, h4, h5, h6
- p, div
- a
- img
- audio
- video
- map
- table-based elements (table, td, th, tr, etc)
- list-based elements (ol, ul, li, etc)
- other formatting elements (b, i, u, br, hr, small, font, etc)

Not all DSpace fields support HTML, but the User Interface should make it clear which fields do. When adding HTML to a field, you should not create a complete HTML document (surrounded with "<html>" tags). Just add an HTML fragment.

Supervision Orders

Available in 7.5 or later

In order to facilitate, as a primary objective, the opportunity for thesis authors to be supervised in the preparation of their e-theses, a supervision order system exists to bind groups of other users (thesis supervisors) to an item in someone's pre-submission workspace. The bound group can have system policies associated with it that allow different levels of interaction with the student's item; a small set of default policy groups are provided:

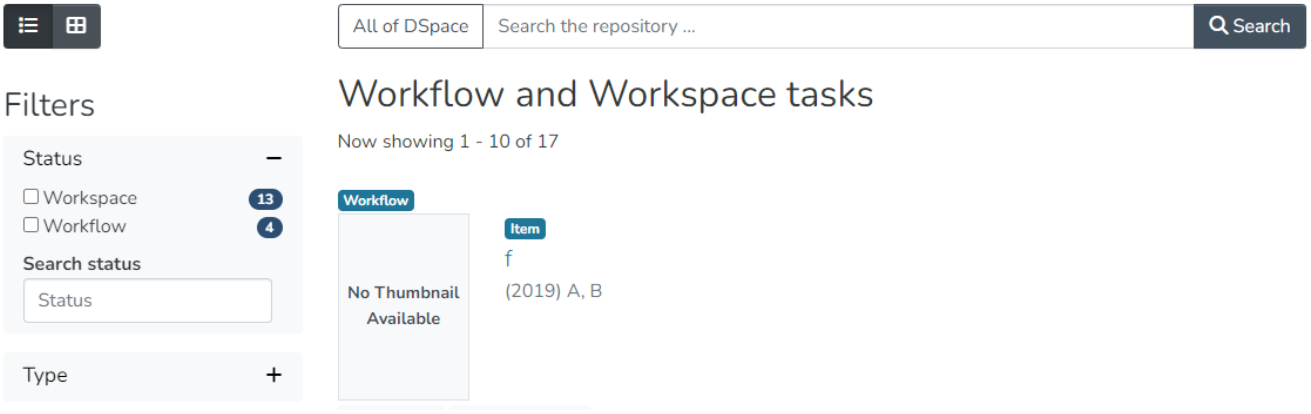
- Full editorial control (EDITOR)
- View item contents (OBSERVER)

Once the default set has been applied, a system administrator may modify them as they would any other policy set in DSpace

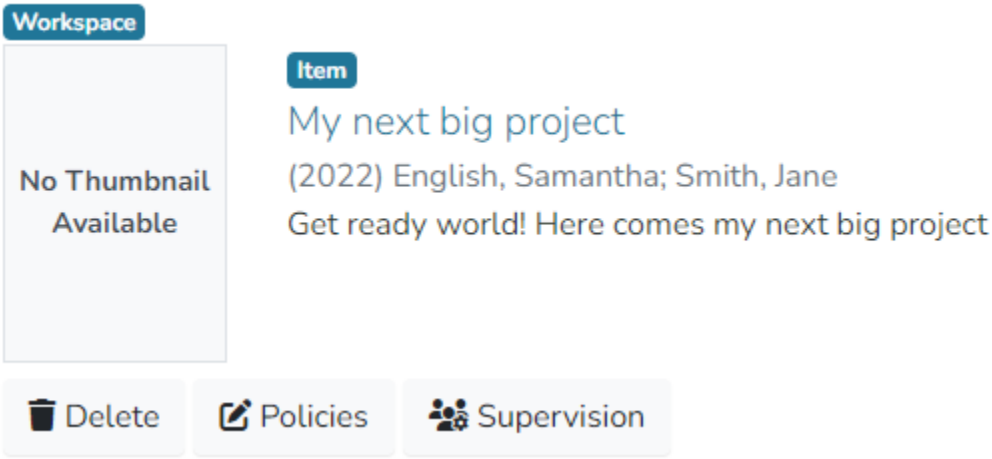
This functionality could also be used in situations where researchers wish to collaborate on a particular submission, although there is no particular collaborative workspace functionality.

Creating a Supervision Order

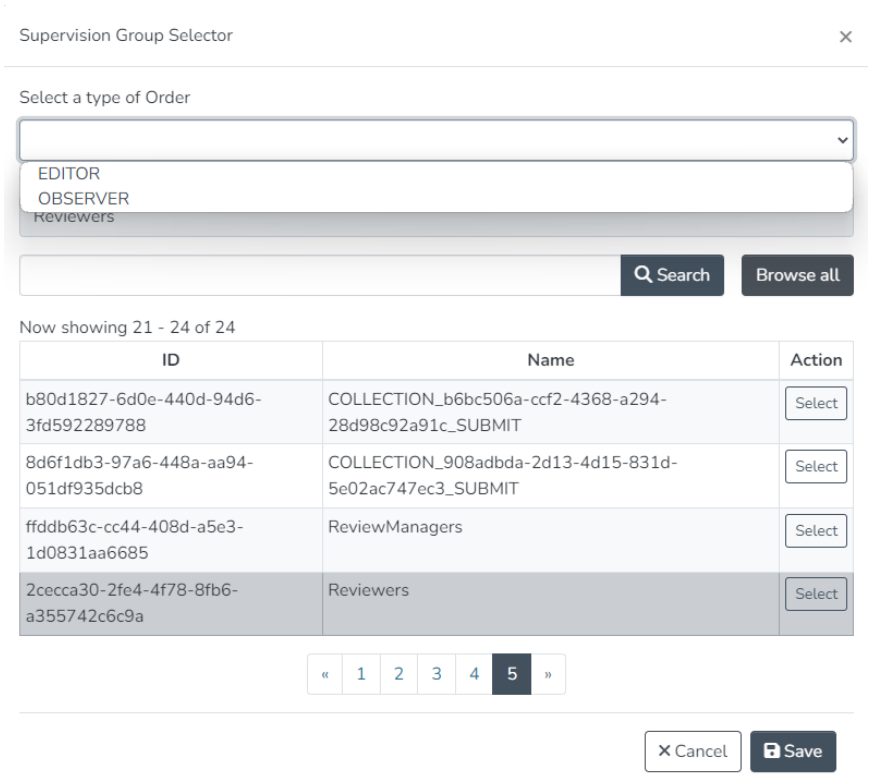
Login as an Administrator, and visit the "Administer Workflow" sidebar menu. From this screen you can see all Items that are either in the "Workspace" (pre-submission) or "Workflow" (workflow approval process) status.



For Items that are in the "Workspace", it is possible to create a supervision order by clicking on the "Supervision" button.



After clicking "Supervision", you'll be able to create a Supervision order by selecting the "Type of Order" (EDITOR or OBSERVER) and assigning those permissions to an existing DSpace Group.



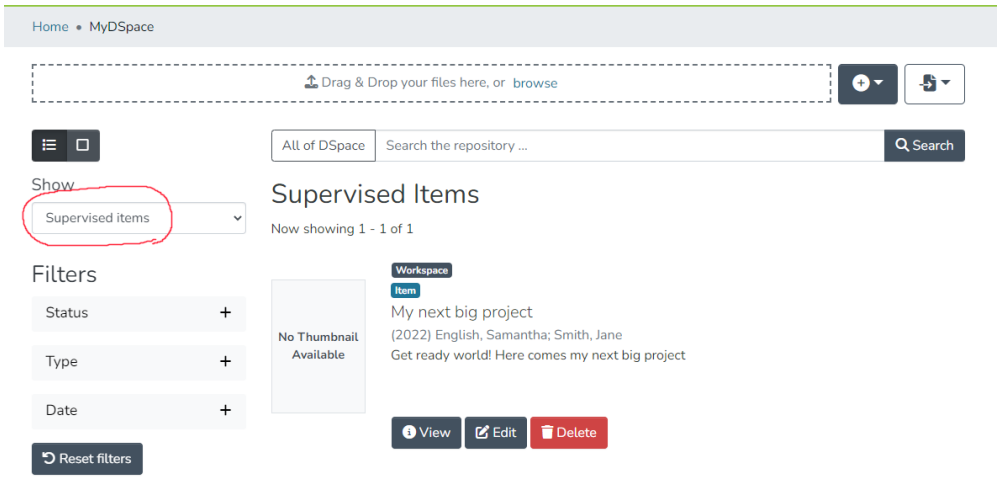
In DSpace, there are currently two Types of Orders:

- EDITOR - The supervising group is given ADD, WRITE, and READ access to the item (but not any bundles or bitstreams that already exist). Any new bundles or bitstreams inherit the supervising group's policy to permit ADD, WRITE and READ operations.
    - *NOTE:* Keep in mind, this does NOT give the supervising group REMOVE policies on the Bundle or DELETE on the Item. This means the supervising group is only able to edit the metadata & add additional bitstreams. They are NOT able to remove existing bitstreams or delete the Item unless additional policies are manually added.
- OBSERVER - The supervising group is given READ access to the item (but not to any bundles or bitstreams that already exist). Any new bundles or bitstreams inherit the supervision group's policy to permit READ operations.
    - NOTE: At this time, there is a known issue where the OBSERVER group will still see the "Edit" and "Delete" buttons, but are unable to perform those actions. https://github.com/DSpace/dspace-angular/issues/2094

Keep in mind, *you can adjust the permissions defined to any order after creating the order!* Simply click on the "Policies" button on the "Administer Workflow" page to adjust the default policies for that supervising group!

Supervising a Submission

Once a Supervision Order is created (see above step), all group members for the supervising group will see that Item in their "Supervised Items" list on their MyDSpace page:

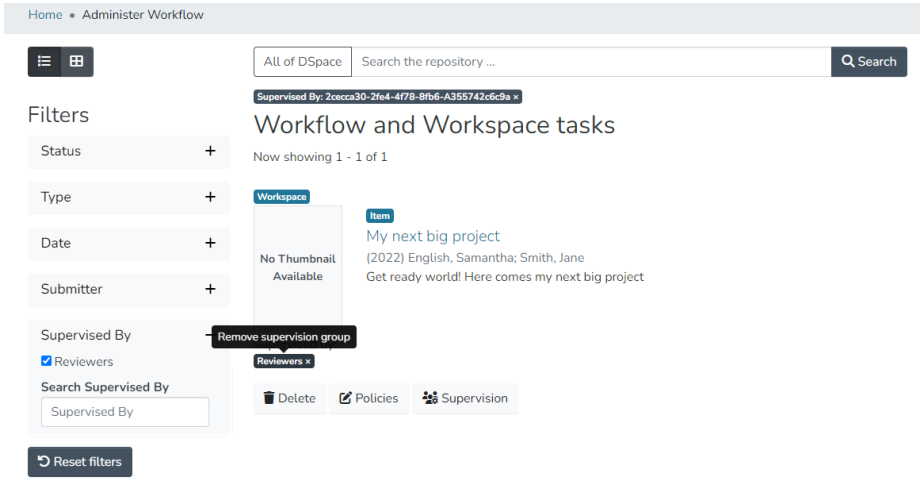


Based on the type of Submission Order (or additional permissions provided), all members of the supervising group will be able to view and/or edit that in-progress submission.

Managing Supervision Orders

At any time, an Administrator can remove or recreate Supervision Orders. This is also done from the "Administer Workflow" page.

On that page, a "Supervised By" filter exists, allowing you to locate all currently supervised items by the assigned group:



You can click on the "Supervised by" label under the supervised item to **remove** the existing supervision order. New orders can be added by clicking the "Supervision" button. You can also adjust any supervising group permissions by editing the policies directly by clicking on the "Policies" button.

Items and Metadata

- Authority Control of Metadata Values
- Batch Metadata Editing
- DOI Digital Object Identifier
- Item Level Versioning
- Mapping/Linking Items to multiple Collections
- Metadata Recommendations
- Moving Items
- PDF Citation Cover Page
- Updating Items via Simple Archive Format

Authority Control of Metadata Values

1 Introduction
2 Simple choice management for DSpace submission forms
    2.1 Example
    2.2 Use simple choice management to add language tags to metadata fields
3 Hierarchical Taxonomies and Controlled Vocabularies
    3.1 Default Hierarchical Controlled Vocabularies
    3.2 Enabling / Disabling a Hierarchical Controlled Vocabulary
    3.3 Additional configurations for Hierarchical Controlled Vocabularies
    3.4 How to invoke a controlled vocabulary from submission-forms.xml
4 Authority Control: Enhancing DSpace metadata fields with Authority Keys
    4.1 How it works
    4.2 Original source:

Introduction

With DSpace you can describe digital objects such as text files, audio, video or data to facilitate easy retrieval and high quality search results. These descriptions are organized into metadata fields that each have a specific designation. For example: dc.title stores the title of an object, while dc.subject is reserved for subject keywords.

For many of these fields, including title and abstract, free text entry is the proper choice, as the values are likely to be unique. Other fields are likely to have values drawn from controlled sets. Such fields include unique names, subject keywords, document types and other classifications. For those kinds of fields the overall quality of the repository metadata increases if values with the same meaning are normalized across all items. Additional benefits can be gained if unique identifiers are associated as well in addition to canonical text values associated with a particular metadata field.

This page covers features included in the DSpace submission forms that allow repository managers to enforce the usage of normalized terms for those fields where this is required in their institutional use cases. DSpace offers simple and straightforward features, such as definitions of simple text values for dropdowns, as well as more elaborate integrations with external vocabularies such as the Library of Congress Naming Authority.

Simple choice management for DSpace submission forms

The DSpace Submission forms, defined in the submission-forms.xml file, allows the inclusion of value pairs that can be organized in lists in order to populate dropdowns or other multiple choice elements. If you explore the default submission-forms.xml file, you can see that a number of such value pair lists are already pre defined.

Example

```
<value-pairs value-pairs-name="common_identifiers" dc-term="identifier">
    <pair>
        <displayed-value>Gov't Doc #</displayed-value>
        <stored-value>govdoc</stored-value>
    </pair>
    <pair>
        <displayed-value>URI</displayed-value>
        <stored-value>uri</stored-value>
    </pair>
    <pair>
        <displayed-value>ISBN</displayed-value>
        <stored-value>isbn</stored-value>
    </pair>
</value-pairs>
```

It generates the following HTML, which results in the menu widget below.

```
<select name="identifier_qualifier_0">
    <option VALUE="govdoc">Gov't Doc #</option>
    <option VALUE="uri">URI</option>
    <option VALUE="isbn">ISBN</option>
</select>
```

A list of value pairs has following required attributes:

- **value-pairs-name** – Name by which an *input-type* refers to this list.
- **dc-term** – Dublin Core field for which this choice list is selecting a value.

Each *value-pairs* element contains a sequence of *pair* sub-elements, each of which in turn contains two elements:

- **displayed-value** – Name shown (on the web page) for the menu entry.
- **stored-value** – Value stored in the DC element when this entry is chosen. Unlike the HTML *select* tag, there is no way to indicate one of the entries should be the default, so the first entry is always the default choice.

Use simple choice management to add language tags to metadata fields

DSpace uses the simple choice management to provide a controlled list of language tags. Out of the box DSpace comes with a list of ISO language tags. You can add further language lists or use the provided one to let submitters tag languages of metadata fields. Take a look at the part of this documentation about the configuration of the Submission User Interface.

Hierarchical Taxonomies and Controlled Vocabularies

The value pairs system works well for short and flat lists of choices. DSpace offers a second way of structuring and managing more complex, hierarchical controlled vocabularies. In contrast to the value pairs system, these controlled vocabularies are managed in separate XML files in the `[dspace]/config/controlled-vocabularies/` directory instead of being entered straight into submission-forms.xml

The taxonomies are described in XML according to this structure:

```
<node id="acmccs98" label="ACMCCS98">
    <isComposedBy>
        <node id="A." label="General Literature">
            <isComposedBy>
                <node id="A.0" label="GENERAL"/>
                <node id="A.1" label="INTRODUCTORY AND SURVEY"/>
                ...
            </isComposedBy>
        </node>
        ...
    </isComposedBy>
</node>
```

As you can see, each node element has an id and label attribute. It can contain the isComposedBy element, which in its turn, consists of a list of other nodes.

You are free to use any application you want to create your controlled vocabularies. A simple text editor should be enough for small projects. Bigger projects will require more complex tools. You may use Protegé to create your taxonomies, save them as OWL and then use a XML Stylesheet (XSLT) to transform your documents to the appropriate format. Future enhancements to this add-on should make it compatible with standard schemas such as OWL or RDF.

Default Hierarchical Controlled Vocabularies

By default, DSpace includes two out-of-the-box hierarchical controlled vocabularies in the `[dspace]/config/controlled-vocabularies/` directory.

- **nsi** - *nsi.xml* - The Norwegian Science Index (in the Norweigen language)
- **srsc** - *srsc.xml* - Swedish Research Subject Categories (in the English language, with notes in Swedish)

You may create your own hierarchical controlled vocabulary by using either of those as a model. All valid hierarchical vocabularies should align with the "controlledvocabulary.xsd" schedule available in that same directory.

Enabling / Disabling a Hierarchical Controlled Vocabulary

To enable a hierarchical controlled vocabulary, simply configure it's usage in one (or more) of your fields in your "submission-forms.xml" (as documented below).

To disable a hierarchical controlled vocabulary, simply remove it from all your fields in your "submission-forms.xml". You can also disable **all** controlled vocabularies by commenting out the "DSpaceControlledVocabulary" plugin in "authority.cfg":

authority.cfg

```
plugin.selfnamed.org.dspace.content.authority.ChoiceAuthority = \
 org.dspace.content.authority.DCInputAuthority, \
 org.dspace.content.authority.DSpaceControlledVocabulary
```

Additional configurations for Hierarchical Controlled Vocabularies

Each hierarchical controlled vocabulary supports additional configurations in the "authority.cfg" file (or local.cfg). These settings override the default behavior of a Controlled Vocabulary in the "org.dspace.content.authority.DSpaceControlledVocabulary" Java class.

Property:	`vocabulary.plugin.[vocab-name].hierarchy.store`
Example Value:	`vocabulary.plugin.srsc.hierarchy.store = true`
Informational Note:	Store entire hierarchy along with selected value. Default is true
Property:	`vocabulary.plugin.[vocab-name].hierarchy.suggest`
Example Value:	`vocabulary.plugin.srsc.hierarchy.suggest = false`
Informational Note:	Display the entire hierarchy in the suggestion list. Default is false.
Property:	`vocabulary.plugin.[vocab-name].hierarchy.delimiter`
Example Value:	`vocabulary.plugin.srsc.hierarchy.delimiter = "::"`
Informational Note:	Delimiter to use when building hierarchy string. Default is "::" (double colon)
Property:	`vocabulary.plugin.[vocab-name].hierarchy.preloadLevel`
Example Value:	`vocabulary.plugin.srsc.hierarchy.preloadLevel = 1`
Informational Note:	The number of levels of the hierarchy to load at once (e.g. in the "Browse by [vocab-name]" in the user interface). The value "1" means to just load the top level members of the hierarchy. The value "2" means to load the top level members and their children. The value "3" means to load the top level members, their children and their grandchildren. Keep in mind, the "Browse by" options in the User Interface may be slower when you load more levels at once. Default is "1".

How to invoke a controlled vocabulary from submission-forms.xml

Vocabularies need to be associated with the correspondent DC metadata fields. Edit the file `[dspace]/config/submission-forms.xml` and place a "`vocabulary`" tag under the "`field`" element that you want to control. Set value of the "`vocabulary`" element to the name of the file that contains the vocabulary, leaving out the extension (the add-on will only load files with extension "*.xml"). For example:

```
<field>
    <dc-schema>dc</dc-schema>
    <dc-element>subject</dc-element>
    <dc-qualifier></dc-qualifier>
    <repeatable>true</repeatable>
    <label>Subject Keywords</label>
    <input-type>onebox</input-type>
    <hint>Enter appropriate subject keywords or phrases below.</hint>
    <required></required>
    <vocabulary>srsc</vocabulary>
</field>
```

The vocabulary element has an optional boolean attribute `closed` that can be used to force input only with the Javascript of controlled-vocabulary add-on. The default behaviour (i.e. without this attribute) is `closed="false"`. This allows the user to enter values as free text in addition to selecting them from the controlled vocabulary.

Authority Control: Enhancing DSpace metadata fields with Authority Keys

The aforementioned features only deal with text representations of controlled values. DSpace also offers support for adding authority keys and confidence values to a specific text value entered in a metadata field. The following terminology applies in the description of this area of DSpace functionality:

- **Authority** An *authority* is an external source of fixed values for a given domain, each unique value identified by a *key*. For example, the OCLC LC Name Authority Service, ORCID or VIAF.
- **Authority Record** The information associated with one of the values in an authority; may include alternate spellings and equivalent forms of the value, etc.
- **Authority Key** An opaque, hopefully persistent, identifier corresponding to exactly one record in the authority.

The fact that this functionality deals with **external** sources of authority makes it inherently different from the functionality for controlled vocabularies. Another difference is that the authority control is asserted *everywhere* metadata values are changed, including unattended/batch submission, SWORD package submission, and the administrative UI.

How it works

TODO

Original source:

Authority Control of Metadata Values original development proposal for DSpace 1.6

ORCID Authority

1 Introduction
2 Use case and high level benefits
3 Enabling the ORCID authority control
    3.1 Settings to enable in local.cfg
    3.2 Enabling the ORCID beans
4 Importing existing authors & keeping the index up to date
    4.1 Different possible use cases for Index-authority script
        4.1.1 Metadata value WITHOUT authority key in metadata
        4.1.2 Metadata that already has an authority key from an external source (NOT auto-generated by DSpace)
        4.1.3 Metadata that has already a new dspace generated uid authority key
        4.1.4 Processing on records in the authority cache
    4.2 Submission of new DSpace items - Author lookup
    4.3 Admin Edit Item
    4.4 Editing existing items using Batch CSV Editing
    4.5 Storage of related metadata
5 Configuration
6 Adding additional fields under ORCID
7 Integration with other systems beside ORCID
8 FAQ
    8.1 Which information from ORCID is currently indexed in the authority cache?
    8.2 How can I index additional fields in the authority cache?
    8.3 How can I use the information stored in the authority cache?
    8.4 How to add additional metadata fields in the authority cache that are not related to ORCID?
    8.5 What happens to data if another authority control was already present?
    8.6 Where can I find the URL that is used to lookup ORCIDs?

ORCID Authority can only pull data from ORCID and link it to a DSpace metadata field

ORCID Authority allows you to link up DSpace metadata fields (added during the submission process) to a person's ORCID identifier. The main use case for this feature is to allow you to link author metadata fields to their ORCID identifier. This is a very basic ORCID integration that has existed since DSpace 5.x.

If you re looking for ORCID Authentication & the ability to synchronize data from DSpace to an ORCID profile, then you should be using the ORCID Integration feature instead.

Introduction

The ORCID integration adds ORCID compatibility to the existing solutions for Authority control in DSpace. String names of authors are still being stored in DSpace metadata. The authority key field is leveraged to store a uniquely generated internal ID that links the author to more extended metadata, including the ORCID ID and alternative author names.

This extended metadata is stored and managed in a dedicated SOLR index, the DSpace authority cache.

Use case and high level benefits

The vision behind this project consists of the following two aspects:

Lowering the threshold to adopt ORCID for the members of the DSpace community

ORCID's API has enabled developers across the globe to build points of integration between ORCID and third party applications. Up until today, this meant that members of the DSpace community were still required to implement front-end and back-end modifications to the DSpace source code in order to leverage these APIs. As DSpace aims to provide turnkey Institutional Repository functionality, the platform is expected to provide more functionality out of the box. Only an elite selection of members in the DSpace community has software development resources readily available to implement this kind of functionality. By contributing a solution directly to the core DSpace codebase, this threshold to adopt ORCID functionality in DSpace repositories is effectively lowered. The ultimate goal is to allow easy adoption of ORCID without customization of the DSpace software, by allowing repository administrators to enable or disable functionality by means of user friendly configuration.

Address generic use cases with appealing end user functionality

This proposal aims to provide user friendly features for both repository administrators as well as non- technical end users of the system. The addition of ORCID functionality to DSpace should not come at the cost of making the system more difficult for administrators and end users to use. Scope With this vision in mind, the project partners wanted to tackle the first phases for repository managers of existing DSpace repositories: ensuring that ORCIDs are properly associated with new works entering the system, as well as providing functionality to efficiently batch-update content already existing in the system, with unambiguous author identity information.

Enabling the ORCID authority control

To enable ORCID authority control requires settings in both local.cfg and updating the registered beans in "orcid-authority-services.xml" as described below.

Settings to enable in local.cfg

If you wish to enable this feature, some changes are required to the `local.cfg` file. The first step is to activate the authority as a valid option for authority control, this is done by adding/setting an additional plugin in the `plugin.named.org.dspace.content.authority.ChoiceAuthority` property. An example of this can be found below.

authority.cfg

```
plugin.named.org.dspace.content.authority.ChoiceAuthority = \
    org.dspace.content.authority.SolrAuthority = SolrAuthorAuthority
```

The feature relies on the following configuration parameters in `authority.cfg, solrauthority.cfg and orcid.cfg`. To activate the default settings it suffices to remove the comment hashes ("#") for the following lines or copy them into your local.cfg. See the section at the bottom of this page what these parameters mean exactly and how you can tweak the configuration.

```
# This setting should already be specified in your solrauthority.cfg
solr.authority.server=${solr.server}/${solr.multicorePrefix}authority

# These settings can be found in your authority.cfg (or could be added to local.cfg)
choices.plugin.dc.contributor.author = SolrAuthorAuthority
choices.presentation.dc.contributor.author = authorLookup
authority.controlled.dc.contributor.author = true
authority.author.indexer.field.1=dc.contributor.author
```

Beginning with DSpace 7, you must specify which ORCID API you wish to use. A Client ID/Secret is also required, but can be obtained for free for the Public API: https://info.orcid.org/documentation/features/public-api/

If you are an ORCID Member Institution, you can use the Member API instead. The Member API is required for additional ORCID Integration features, but is NOT required for this basic ORCID Authority feature.

```
# These ORCID settings are now required for ORCID Authority
# They can be found in your orcid.cfg (or can be added to local.cfg)
orcid.domain-url = https://orcid.org
# You can use either the Public API or Member API in this next setting
orcid.api-url = https://pub.orcid.org/v3.0

# You do NOT need to pay for a Member API ID to use ORCID Authority.
# Instead, you just need a Public API ID from a free ORCID account.
orcid.application-client-id = MYID
orcid.application-client-secret = MYSECRET
```

The final part of configuration is to add the authority consumer in front of the list of event consumers (in dspace.cfg or local.cfg). Add "authority" in front of the list as displayed below.

```
event.dispatcher.default.consumers = authority, versioning, discovery, eperson
```

Enabling the ORCID beans

You must also uncomment the ORCID beans in `config/spring/api/orcid-authority-services.xml`. These are commented out by default as they require setting the "orcid.*" settings described above.

Simply uncomment these settings as-is & restart Tomcat. They will pull their configs from orcid.cfg or your local.cfg.

orcid-authority-services.xml

```
<!-- This bean & alias are commented out by default. Simply uncomment them -->
<alias name="OrcidSource" alias="AuthoritySource"/>
<bean name="OrcidSource" class="org.dspace.authority.orcid.Orcidv3SolrAuthorityImpl">
    <property name="clientId" value="${orcid.application-client-id}" />
    <property name="clientSecret" value="${orcid.application-client-secret}" />
    <property name="OAUTHUrl" value="${orcid.token-url}" />
    <property name="orcidRestConnector" ref="orcidRestConnector"/>
</bean>

<!-- Also uncomment Orcidv3AuthorityValue in the list of supported types below.
     The other settings in this AuthorityTypes bean can be left as-is. -->
<bean name="AuthorityTypes" class="org.dspace.authority.AuthorityTypes">
        <property name="types">
            <list>
                <bean class="org.dspace.authority.orcid.Orcidv3AuthorityValue"/>
                <bean class="org.dspace.authority.PersonAuthorityValue"/>
            </list>
        </property>
...
</bean>
```

Importing existing authors & keeping the index up to date

When first enabled the authority index will be empty, to populate the authority index run the following script:

```
[dspace]/bin/dspace index-authority
```

This will iterate over every metadata under authority control and create records of them in the authority index. The metadata without an authority key will each be updated with an auto generated authority key. These will not be matched in any way with other existing records. The metadata with an authority key that does not already exist in the index will be indexed with those authority keys. The metadata with an authority key that already exist in the index will be re-indexed the same way. These records remain unchanged.

Different possible use cases for Index-authority script

Metadata value WITHOUT authority key in metadata

“Luyten, Bram” is present in the metadata without any authority key.
GOAL: “Luyten, Bram” gets added in the cache ONCE

All occurences of “Luyten, Bram” in the DSpace item metadata will become linked with the same generated uid.

Metadata that already has an authority key from an external source (NOT auto-generated by DSpace)

“Snyers, Antoine” is present with authority key “u12345”

The old authority key needs to be preserved in the item metadata and duplicated in the cache.
“u12345” will be copied to the authority cache and used as the authority key there.

Metadata that has already a new dspace generated uid authority key

Item metadata already contains an author with name “Haak, Danielle” and a uid in the authority field 3dda2571-6be8-4102-a47b-5748531ae286

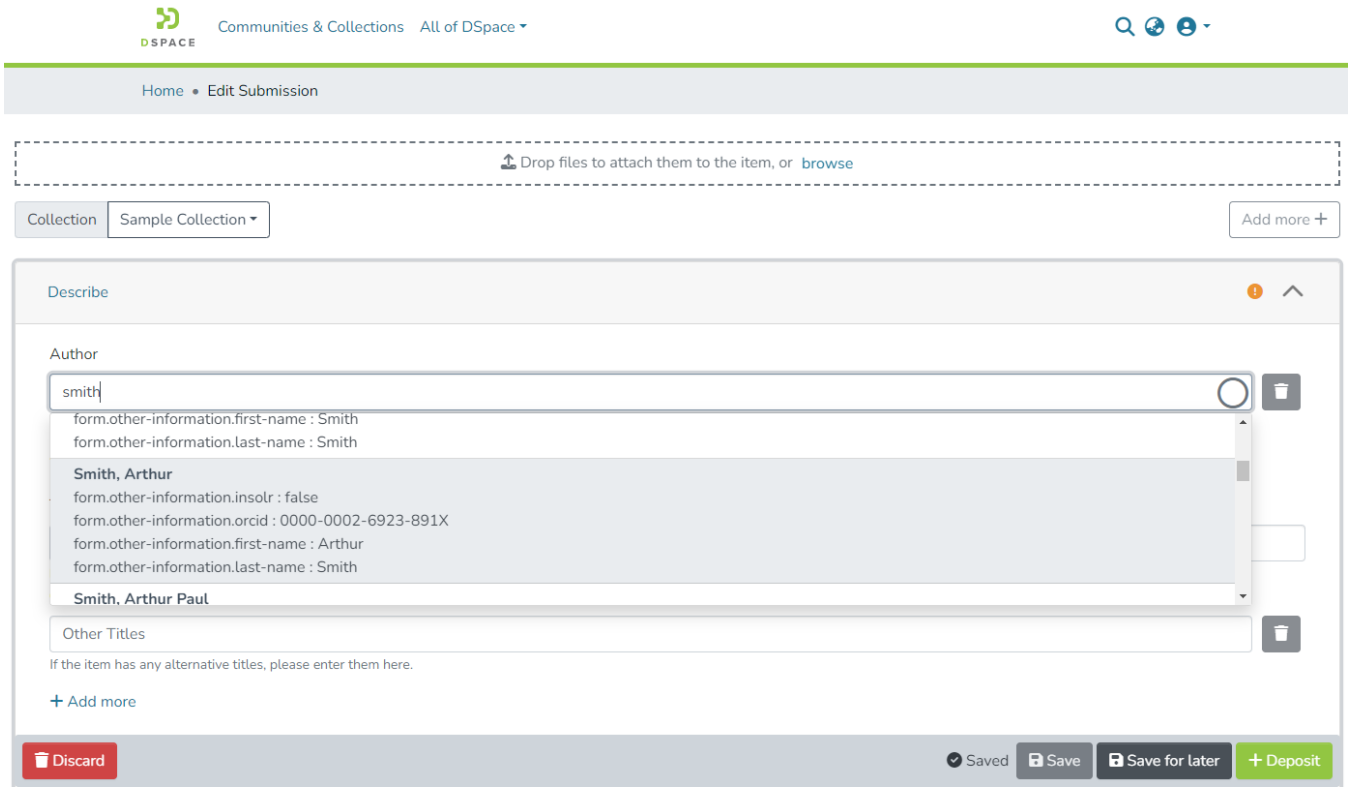
This uid is preserved and no new record is being created in the authority index.

Processing on records in the authority cache

Running this script again will update the index and keep the index clean. For example if an author occurs in a single item and that item is deleted the script will need to be run again to remove it from the index. When run again it will remove all records that no longer have a link to existing authors in the database.

Submission of new DSpace items - Author lookup

When ORCID Authority is enabled, the Author field can be used to search entries in ORCID. Simply type in an Author name to search your locally indexed authors and authors in ORCID.



Select an author entry from the list to add that Author. The List of authors is updated as you type.

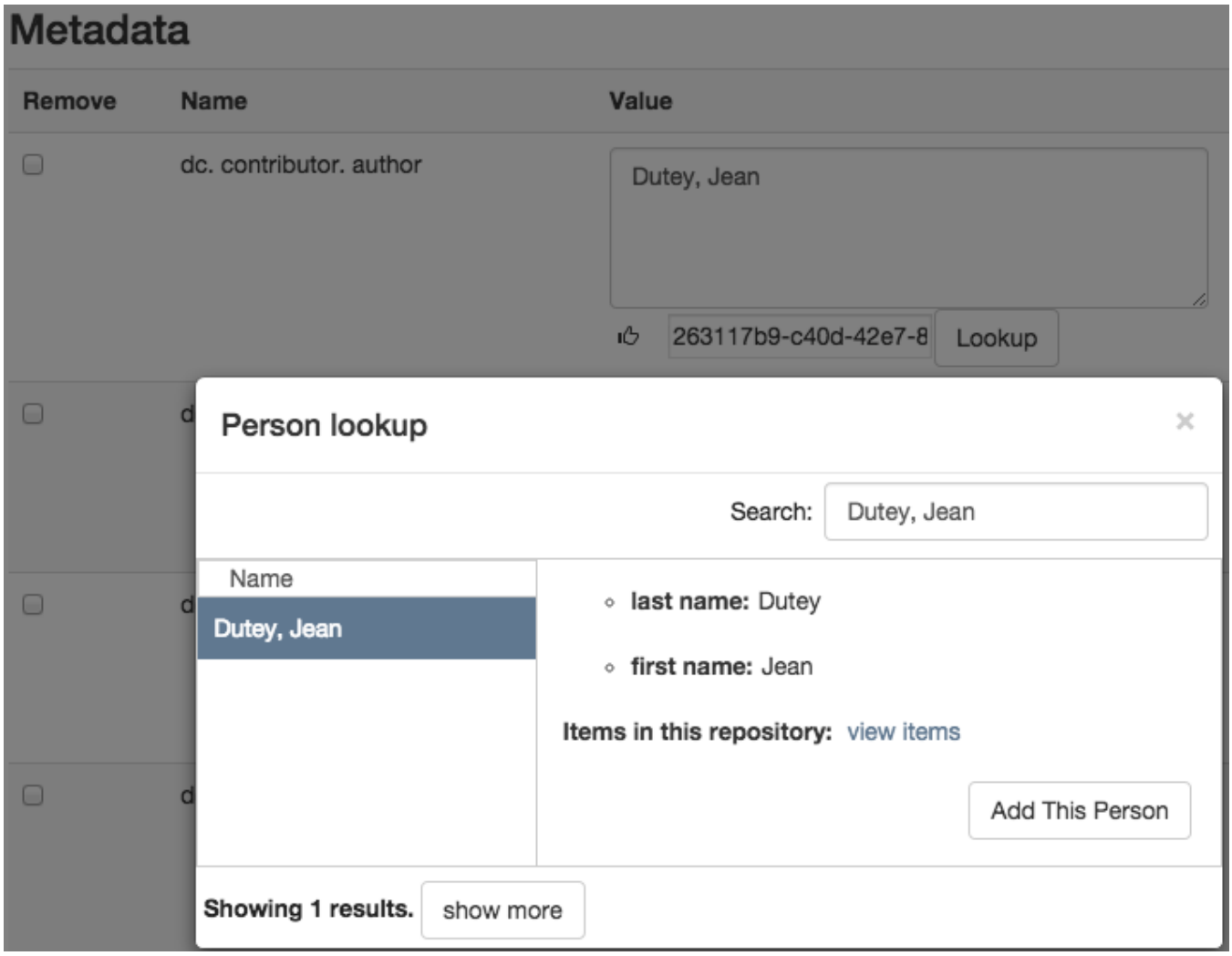
Authors that already appear somewhere in the repository are differentiated from the authors that have been retrieved from ORCID.

Admin Edit Item

Not yet available in DSpace 7.x. The below screenshots are from 6.x

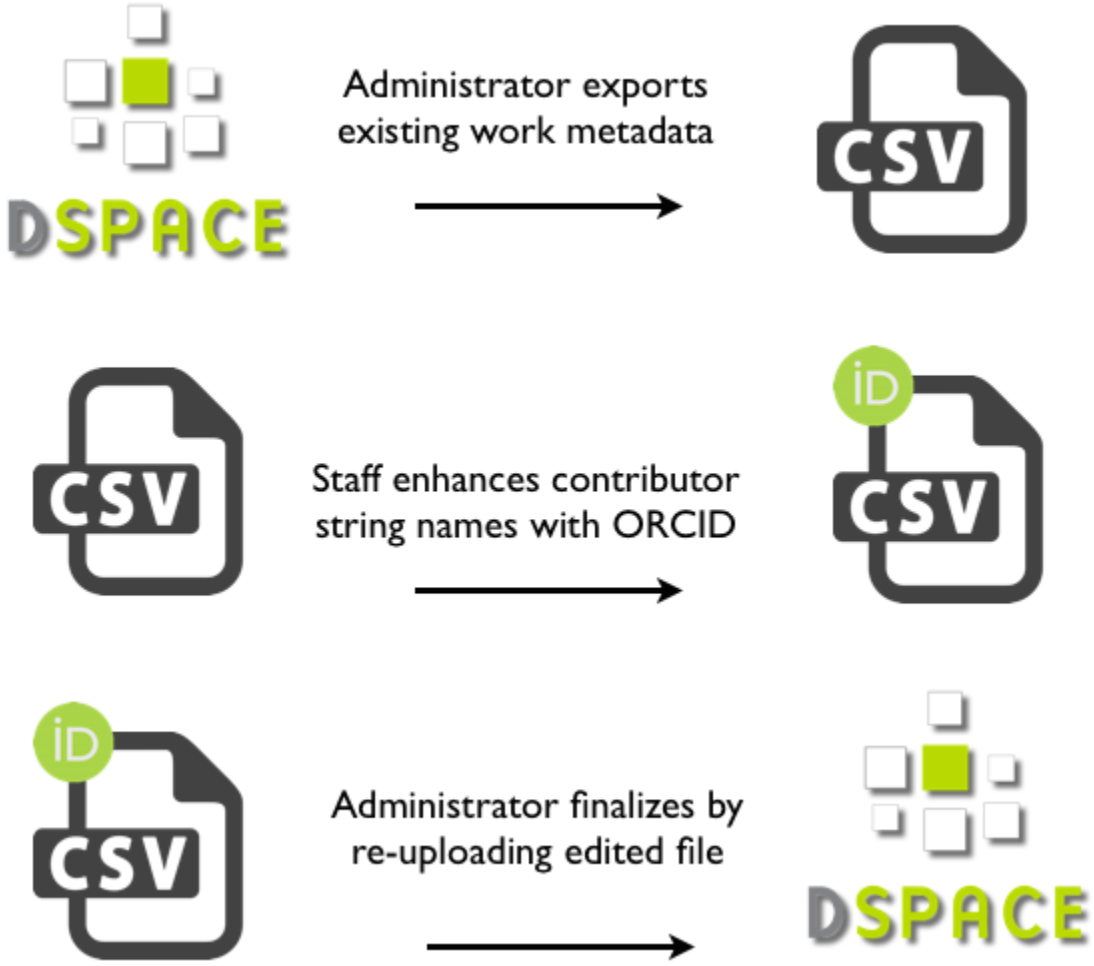
In the edit metadata page, under the values for the dc.contributor.author fields, an extra line shows the author ID together with a lock icon and a Lookup button. The author ID cannot be changed manually. However the Lookup button will help you change the author name and ID at the same time.

Clicking the Lookup button brings back the Lookup User Interface. This works just the same way as in the submission forms.



Editing existing items using Batch CSV Editing

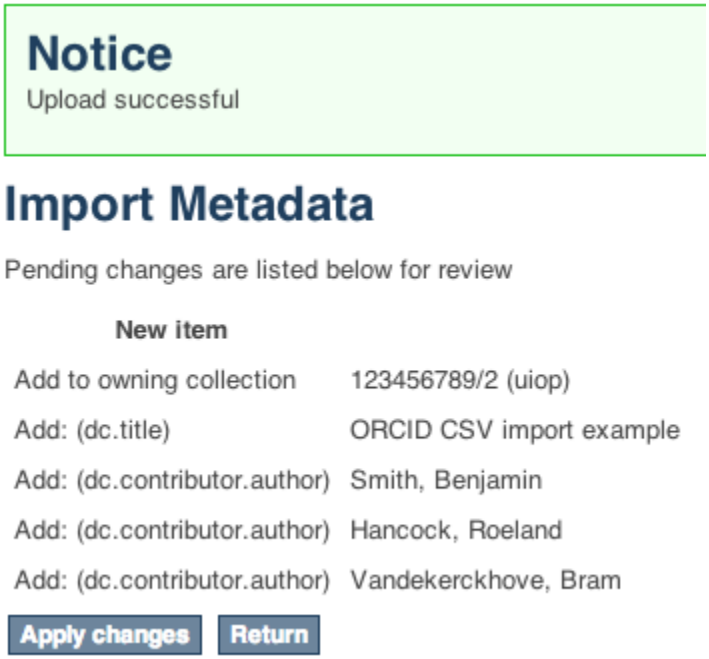
Instructions on how to use the Batch CSV Editing are found on the Batch Metadata Editing documentation page.



ORCID Integration is provided through the Batch CSV Editing feature with an extra available headers "ORCID:dc.contributor.author". The usual CSV headers only contain the metadata fields: e.g. "dc.contributor.author". In addition to the traditional header, another dc.contributor.author header can be added with the "ORCID:" prefix. The values in this column are supposed to be ORCIDs.

id	collection	dc.title	ORCID:dc.contributor.author	dc.contributor.author
+	123456789/2	ORCID CSV import example	0000-0001-8932-6872||0000-0001-9152-849X	Smith, Benjamin

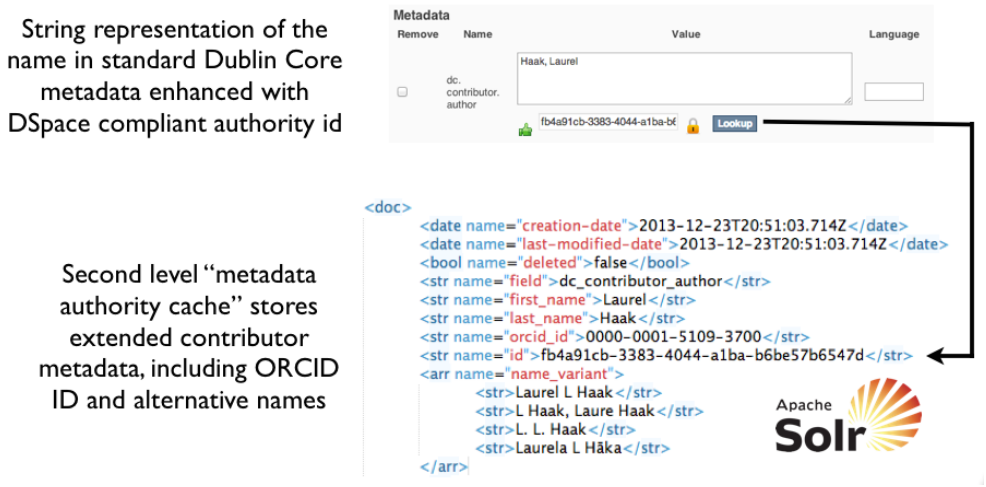
For each of the ORCID authors a lookup will be done and their names will be added to the metadata. All the non-ORCID authors will be added as well. The authority keys and solr records are added when the reported changes are applied.



Storage of related metadata

ORCID authorities not only link a digital identifier to a name. It regroups a load of metadata going from alternative names and email addresses to keywords about their works and much more. The metadata is obtained by querying the ORCID web services. In order to avoid querying the ORCID web services every time, all these related metadata is gathered in a "metadata authority cache" that DSpace can access directly.

In practice the cache is provided by an apache solr server. When a look-up is made and an author is chosen that is not yet in the cache, a record is created from an ORCID profile and added to the cache with the list of related metadata. The value of the Dublin Core metadata is based on the first and last name as they are set in the ORCID profile. The authority key for this value links directly to the solr document's id. DSpace does not provide a way to edit these records manually.



The information in the authority cache can be updated by running the following command line operation:

Command used:	`[dspace]/bin/dspace dsrun org.dspace.authority.UpdateAuthorities`
Arguments	description
-i	update specific solr records with the given internal ids (comma-separated)

-h	prints this help message

This will iterate over every solr record currently in use (unless the -i argument is provided), query the ORCID web service for the latest data and update the information in the cache. If configured, the script will also update the metadata of the items in the repository where applicable.

The configuration property can be set in `config/modules/solrauthority.cfg`, or overridden in your `local.cfg` (see Configuration Reference).

```
solrauthority.auto-update-items = false | true
```

When set to true and this is script is run, if an authority record's information is updated the whole repository will be scanned for this authority. Every metadata field with this authority key will be updated with the value of the updated authority record.

Configuration

In the Enabling the ORCID authority control section, you have been told to add this block of configuration.

NOTE: you can use local.cfg for these

For all of the configuration options described below, you can use either dspace.cfg or local.cfg. Either will work. It is possible that, when you compile your code with Maven, and you have tests enabled, your build will fail. DSpace unit tests utilize parts of dspace.cfg, and the configuration options you will utilize below are known to cause unit test errors. The easiest way to avoid this situation is to use the local.cfg file.

```
solr.authority.server=${solr.server}/authority
choices.plugin.dc.contributor.author = SolrAuthorAuthority
choices.presentation.dc.contributor.author = authorLookup
authority.controlled.dc.contributor.author = true
authority.author.indexer.field.1=dc.contributor.author

# These ORCID settings are now required for ORCID Authority
orcid.domain-url = https://orcid.org
# You can use either the Public API or Member API
orcid.api-url = https://pub.orcid.org/v3.0

# You do NOT need to pay for a Member API ID to use ORCID Authority.
# Instead, you just need a Public API ID from a free ORCID account.
# https://info.orcid.org/documentation/features/public-api/
orcid.application-client-id = MYID
orcid.application-client-secret = MYSECRET
```

The ORCID Integration feature is an extension on the authority control in DSpace. Most of these properties are extensively explained on the Authority Control of Metadata Values documentation page. These will be revisited but first we cover the properties that have been newly added.

- The **`solr.authority.server`** is the url to the solr core. Usually this would be on the **`solr.server`** next to the oai, search and statistics cores.
- **`authority.author.indexer.field.1`** and the subsequent increments configure which fields will be indexed in the authority cache. However before adding extra fields into the solr cache, please read the section about Adding additional fields under ORCID.

That's it for the novelties. Moving on to the generic authority control properties:

- With the **`authority.controlled`** property every metadata field that needs to be authority controlled is configured. This involves every type of authority control, not only the fields for ORCID integration.
- The **`choices.plugin`** should be configured for each metadata field under authority control. Setting the value on SolrAuthorAuthority tells DSpace to use the solr authority cache for this metadatafield, cfr. Storage of related metadata.
- The **`choices.presention`** should be configured for each metadata field as well. The traditional values for this property are `select|suggest|lookup`. A new value, `authorLookup`, has been added to be used in combination with the SolrAuthorAuthority choices plugin. While the other values can still be used, the authorLookup provides a richer user interface in the form of a popup on the submission page.
- The browse indexes need to point to the new authority-controlled index: `webui.browse.index.2 = author:metadata:dc.contributor.*,dc.creator:text` should become **webui.browse.index.2 = author:metadataAuthority:dc.contributor.author:authority**
- More existing configuration properties are available but their values are independent of this feature and their default values are usually fine: **`choices.closed`** , **`authority.required`**, **`authority.minconfidence`** .

For the cache update script, one property can be set in `config/modules/solrauthority.cfg`:

```
auto-update-items = false | true
```

The default value for when the property is missing is false.

The final part of configuration is to add the authority consumer in front of the list of event consumers. Add "authority" in front of the list as displayed below.

```
event.dispatcher.default.consumers = authority, versioning, discovery, eperson, harvester
```

Without the consumer there is no automatic indexing of the authority cache and the metadata will not even have authority keys.

Changes to the configuration always require a server restart before they're in effect.

Adding additional fields under ORCID

Other metadata fields besides "dc.contributor.author" can benefit from the ORCID authority control at the same time. Here is an example of how to get the same ORCID functionality for the "dc.contributor.editor" metadata field assuming that "dc.contributor.author" is already configured correctly. It can be achieved by modifying configuration files only.

First add the same configuration fields that have been added for the "dc.contributor.author"

```
choices.plugin.dc.contributor.editor = SolrAuthorAuthority
choices.presentation.dc.contributor.editor = authorLookup
authority.controlled.dc.contributor.editor = true

authority.author.indexer.field.1=dc.contributor.author
authority.author.indexer.field.2=dc.contributor.editor
```

This is enough to get the look-up interface on the submission page and on the edit metadata page. The authority keys will be added and indexed with the information from orcid just as it happens with the Authors.

But you're not completely done yet, There is one more configuration step. Because now when adding new editors in the metadata that are not retrieved through the external look-up, their first and last name will not be displayed in the look-up interface next time you look for them.

To fix this, open the file at `config/spring/api/orcid-authority-services.xml` and find this spring bean:

```
<bean name="AuthorityTypes" class="org.dspace.authority.AuthorityTypes">
    <property name="types">
        <list>
            <bean class="org.dspace.authority.orcid.Orcidv3AuthorityValue"/>
            <bean class="org.dspace.authority.PersonAuthorityValue"/>
        </list>
    </property>
    <property name="fieldDefaults">
        <map>
            <entry key="dc_contributor_author">
                <bean class="org.dspace.authority.PersonAuthorityValue"/>
            </entry>
        </map>
    </property>
</bean>
```

The map inside the "fieldDefaults" property needs an additional entry for the editor field:

```
<entry key="dc_contributor_editor">
    <bean class="org.dspace.authority.PersonAuthorityValue"/>
</entry>
```

With this last change everything is set up to work correctly. The rest of this configuration file is meant for JAVA developers that wish to provide integration with other systems beside ORCID. Developers that wish to display other fields than first and last name can also have a look in that section.

Note: Each metadata field has a separate set of authority records. Authority keys are not shared between different metadata fields. E. g. multiple dc.contributor.author can have the same authority key and point to the same authority record in the cache. But when an ORCID is chosen for a dc.contributor.editor field, a separate record is made in the cache. Both records are updated from the same source and will contain the same information. The difference is that when performing a look-up of a person that has been introduced as an authority for an author field but not yet as an editor, it will show as record that is not yet present in the repository cache.

Integration with other systems beside ORCID

The authority cache and look-up functionality can be extended to use other sources than ORCID or to show more information in the look-up interface. However some JAVA development is necessary for this. Specific instructions can be found in the readme file of the org.dspace.authority package.

FAQ

Which information from ORCID is currently indexed in the authority cache?

Here is a breakdown of the fields stored in the solr cache.

The system/dspace related fields are: *id, field, value, deleted, creation_date, last_modified_date, authority_type*.

The fields with data coming directly from ORCID are: *first_name, last_name, name_variant, orcid_id, label_researcher_url, label_keyword, label_external_identifier, label_biography, label_country.* The field *all_labels* contains all the values from the other fields starting with "label_".

How can I index additional fields in the authority cache?

There is currently no configuration to control which fields are indexed. The only way to achieve this is to modify the source code.

List of the files at work for this job:
`config/spring/api/orcid-authority-services.xml`: OrcidSource contains the URL for orcid's REST API.
org.dspace.authority.orcid.Orcid makes the REST call
+ org.dspace.authority.orcid.xml.XMLtoBio converts the received XML to a java object (Bio).
+ org.dspace.authority.orcid.model.Bio
+ org.dspace.authority.orcid.OrcidAuthorityValue#create(org.dspace.authority.orcid.model.Bio) inserts all the values from Bio into the AuthorityValue subclass.
+ org.dspace.authority.orcid.OrcidAuthorityValue#getSolrInputDocument defines what's included in solr.

The files preceded with a '+' would be necessary to modify to add more info into the cache.

How can I use the information stored in the authority cache?

The look-up UI is currently the only place this information is sent to. However just a limited number of fields are sent. The place in the source code to modify to get more fields there is org.dspace.authority.orcid.OrcidAuthorityValue#choiceSelectMap. This is also documented in the readme of the org.dspace.authority package.

How to add additional metadata fields in the authority cache that are not related to ORCID?

Make the same configuration step as for adding additional fields under ORCID. Currently the ORCID suggestions cannot be turned off for specific fields, that would require custom code.

What happens to data if another authority control was already present?

As long as the metadata does not get indexed, there will be no changes. However every time any metadata of an item is modified, the metadata under authority control for that item will be re-indexed. When that happens a record will be inserted in the solr cache. That record's ID will be the authority key of the metadata. This can be done for all metadata at once with the index-authority script.

In short: authority keys that exist prior to enabling the solrauthority are kept. They just won't show in the look-up until they are indexed.

Where can I find the URL that is used to lookup ORCIDs?

It is found in the `config/spring/api/orcid-authority-services.xml` configuration file. Look for the `<bean name="OrcidSource">`, which is initialized with a URL of http://pub.orcid.org

Batch Metadata Editing

1 Batch Metadata Editing Tool
- 1.1 Export Function
  - 1.1.1 Web Interface Export
  - 1.1.2 Command Line Export
- 1.2 Import Function
  - 1.2.1 Web Interface Import
  - 1.2.2 Command Line Import
- 1.3 CSV Format
  - 1.3.1 File Structure
- 1.4 Editing the CSV
  - 1.4.1 Editing Collection Membership
  - 1.4.2 Adding Metadata-Only Items
  - 1.4.3 Deleting Metadata
  - 1.4.4 Performing 'actions' on items
  - 1.4.5 Migrating Data or Exchanging data
  - 1.4.6 Common Issues
    - 1.4.6.1 Metadata values in CSV export seem to have duplicate columns
    - 1.4.6.2 DSpace responds with "No changes were detected" when CSV is uploaded
- 1.5 Batch Editing, Entities and Relationships
  - 1.5.1 Background about entities and virtual metadata
  - 1.5.2 Admin CSV export
  - 1.5.3 Admin CSV import

Batch Metadata Editing Tool

DSpace provides a batch metadata editing tool. The batch editing tool is able to produce a comma delimited file in the CSV format. The batch editing tool facilitates the user to perform the following:

- Batch editing of metadata (e.g. perform an external spell check)
- Batch additions of metadata (e.g. add an abstract to a set of items, add controlled vocabulary such as LCSH)
- Batch find and replace of metadata values (e.g. correct misspelled surname across several records)
- Mass move items between collections
- Mass deletion, withdrawal, or re-instatement of items
- Enable the batch addition of new items (without bitstreams) via a CSV file
- Re-order the values in a list (e.g. authors)

For information about configuration options for the Batch Metadata Editing tool, see Batch Metadata Editing Configuration

Export Function

Web Interface Export

Batch metadata exports (to CSV) can be performed from the Administrative menu:

- Login as an Administrative user
- In Sidebase, select "Export" "Metadata". Type in the Community/Collection name.
  - Alternatively, browse to the Community or Collection you wish to export, and then go to "Export" "Metadata". That Community /Collection will be preselected.
- Click "Export". A new Process will be created (in "Processes" menu). Once completed, download the resulting CSV.

Exporting search results to CSV was not added until DSpace 7.3

As of DSpace 7.3, it is possible to Export search results to a CSV *(similar to 6.x).* When logged in as an Administrator, after performing a search a new "Export search results as CSV" button appears. Clicking it will export the metadata of all items in your search results to a CSV. This CSV can then be used to perform batch metadata updates (based on the items in your search results). **- Release Notes#7.3ReleaseNotes**

Please see below documentation for more information on the CSV format and actions that can be performed by editing the CSV.

Command Line Export

The following table summarizes the basics.

Command used:	`[dspace]/bin/dspace metadata-export`
Java class:	org.dspace.app.bulkedit.MetadataExport
Arguments short and (long) forms):	Description
`-f` or `--file`	Required. The filename of the resulting CSV.
`-i` or `--id`	The Item, Collection, or Community handle or Database ID to export. If not specified, **all** items will be exported.

`-a` or `--all`	Include all the metadata fields that are not normally changed (e.g. provenance) or those fields you configured in the `[dspace]/config/modules/bulkedit.cfg` to be ignored on export.
`-h` or `--help`	Display the help page.

To run the batch editing exporter, at the command line:

```
[dspace]/bin/dspace metadata-export -f name_of_file.csv -i 1023/24
```

Example:

```
[dspace]/bin/dspace metadata-export -f /batch_export/col_14.csv -i /1989.1/24
```

In the above example we have requested that a collection, assigned handle '*1989.1/24*' export the entire collection to the file '*col_14.csv*' found in the '*/batch_export*' directory.

Please see below documentation for more information on the CSV format and actions that can be performed by editing the CSV .

Import Function

Importing large CSV files

It is not recommended to import CSV files of more than 1,000 lines (i.e. 1,000 items). When importing files larger than this, it may be difficult for an Administrator to accurately verify the changes that the import tool states it will make. In addition, depending on the memory available to the DSpace site, large files may cause 'Out Of Memory' errors part way through the import process.

Web Interface Import

Batch metadata imports (from CSV) can be performed from the Administrative menu:

- First, complete all editing of the CSV and save your changes
- Login as an Administrative User
- In sidebar, select "Import" "Metadata" and drag & drop the CSV file

Validate a Batch Metadata CSV was not added until DSpace 7.3

As of DSpace 7.3, it is now possible to *validate* a Batch Metadata CSV before applying changes *(similar to 6.x)*. When uploading a CSV for batch updates (using "Import" menu), a new "Validate Only" option is selected by default. When selected, the uploaded CSV will only be validated & you'll receive a report of the detected changes in the CSV. This allows you to verify the changes are correct before applying them. (NOTE: applying the changes requires re-submitting the CSV with the "Validate Only" option deselected) **- Release Notes#7.3ReleaseNotes**

Command Line Import

The following table summarizes the basics.

Command used:	`[dspace]/bin/dspace metadata-import`
Java class:	org.dspace.app.bulkedit.MetadataImport
Arguments short and (long) forms:	Description
`-f` or `--file`	Required. The filename of the CSV file to load.
`-s` or `--silent`	Silent mode. The import function does not prompt you to make sure you wish to make the changes.
`-e` or `--email`	The email address of the user. This is only required when adding new items.
`-w` or `--workflow`	When adding new items, the program will queue the items up to use the Collection Workflow processes.
`-n` or `--notify`	when adding new items using a workflow, send notification emails.
`-t` or `--template`	When adding new items, use the Collection template, if it exists.
`-h` or `--help`	Display the brief help page.

Silent Mode should be used carefully. It is possible (and probable) that you can overlay the wrong data and cause irreparable damage to the database.

To run the batch importer, at the command line:

```
[dspace]/bin/dspace metadata-import -f name_of_file.csv
```

Example

```
[dspace]/bin/dspace metadata-import -f /dImport/col_14.csv
```

If you are wishing to upload new metadata **without** bitstreams, at the command line:

```
[dspace]/bin/dspace metadata-import -f /dImport/new_file.csv -e joe@user.com -w -n -t
```

In the above example we threw in all the arguments. This would add the metadata and engage the workflow, notification, and templates to all be applied to the items that are being added.

CSV Format

The CSV (comma separated values) files that this tool can import and export abide by the RFC4180 CSV format. This means that new lines, and embedded commas can be included by wrapping elements in double quotes. Double quotes can be included by using two double quotes. The code does all this for you, and any good csv editor such as Excel or OpenOffice will comply with this convention.

All CSV files are also in UTF-8 encoding in order to support all languages.

File Structure

The first row of the CSV must define the metadata values that the rest of the CSV represents. ***The first column must always be "id" which refers to the item's internal database ID***. *All other columns are optional.* The other columns contain the dublin core metadata fields that the data is to reside.

A typical heading row looks like:

```
id,collection,dc.title,dc.contributor,dc.date.issued,etc,etc,etc.
```

Subsequent rows in the csv file relate to items. A typical row might look like:

```
350,2292,Item title,"Smith, John",2008
```

If you want to store multiple values for a given metadata element, they can be separated with the double-pipe '||' (or another character that you defined in your `modules/bulkedit.cfg` file). For example:

```
Horses||Dogs||Cats
```

Elements are stored in the database in the order that they appear in the CSV file. You can use this to order elements where order may matter, such as authors, or controlled vocabulary such as Library of Congress Subject Headings.

Editing the CSV

If you are editing with Microsoft Excel, be sure to open the CSV in Unicode/UTF-8 encoding

By default, Microsoft Excel may not correctly open the CSV in Unicode/UTF-8 encoding. This means that special characters may be improperly displayed and also can be "corrupted" during re-import of the CSV.

You need to tell Excel this CSV is Unicode, by importing it as follows. (*Please note these instructions are valid for MS Office 2007 and 2013. Other Office versions may vary*)

- First, open Excel (with an empty sheet/workbook open)
- Select "Data" tab
- Click "From Text" button (in the "External Data" section)
- Select your CSV file
- Wizard Step 1
  - Choose "Delimited" option
  - Start import at row: 1
  - In the "File origin" selectbox, select "65001 : Unicode (UTF-8)"
    - NOTE: these encoding options are sorted alphabetically, so "Unicode (UTF-8)" appears near the bottom of the list.
  - Click Next
- Wizard Step 2
  - Select "Comma" as the only delimiter
  - Click Next
- Wizard Step 3
  - Select "Text" as the "Column data format" (*Unfortunately, this must be done for each column individually in Excel*)
    - At a minimum, you MUST ensure all date columns (e.g. dc.date.issued) are treated as "Text" so that Excel doesn't autoconvert DSpace's YYYY-MM-DD format into MM/DD/YYYY
    - To avoid such autoconversion, it is safest to ensure each column is treated as "Text". Unfortunately, this means selecting each column one-by-one and choosing "Text" as the "Column data format".
  - Click Finish
- Choose whether to open CSV in the existing sheet or a new one

Tips to Simplify the Editing Process

When editing a CSV, here's a couple of basic tips to keep in mind:

1. The "id" column MUST remain intact. This column also must always have a value in it.
2. To simplify the CSV, you can simply remove any columns you do NOT wish to edit (except for "id" column, see #1). Don't worry, removing the entire column won't delete metadata (see #3)
3. When importing a CSV file, the importer will *overlay* the metadata onto what is already in the repository to determine the differences. It *only* acts on the contents of the CSV file, rather than on the complete item metadata. This means that the CSV file that is exported can be manipulated quite substantially before being re-imported. Rows (items) or Columns (metadata elements) can be removed and will be ignored.
   a. For example, if you only want to edit "dc.subject", you can remove ALL columns EXCEPT for "id" and "dc.subject" so that you can just manipulate the "dc.subject" field. On import, DSpace will see that you've only included the "dc.subject" field in your CSV and therefore will only update the "dc.subject" metadata field for any items listed in that CSV.
4. Because removing an entire column does NOT delete metadata value(s), if you actually wish to delete a metadata value you should leave the column intact, and simply clear out the appropriate row's value (in that column).

Editing Collection Membership

Items can be moved between collections by editing the collection handles in the 'collection' column. Multiple collections can be included. The first collection is the 'owning collection'. The owning collection is the primary collection that the item appears in. Subsequent collections (separated by the field separator) are treated as mapped collections. These are the same as using the map item functionality in the DSpace user interface. To move items between collections, or to edit which other collections they are mapped to, change the data in the collection column.

Adding Metadata-Only Items

New metadata-only items can be added to DSpace using the batch metadata importer. To do this, enter a plus sign '+' in the first 'id' column. The importer will then treat this as a new item. If you are using the command line importer, you will need to use the -e flag to specify the user email address or id of the user that is registered as submitting the items.

Deleting Metadata

It is possible to perform metadata deletes across the board of certain metadata fields from an exported file. For example, let's say you have used keywords (dc.subject) that need to be removed *en masse*. You would leave the column (dc.subject) intact, but remove the data in the corresponding rows.

Performing 'actions' on items

It is possible to perform certain 'actions' on items. This is achieved by adding an 'action' column to the CSV file (after the id, and collection columns). There are three possible actions:

1. ***'expunge'*** This permanently deletes an item. Use with care! This action must be enabled by setting 'allowexpunge = true' in `modules/bulkedit.cfg`
2. ***'withdraw'*** This withdraws an item from the archive, but does not delete it.
3. ***'reinstate'*** This reinstates an item that has previously been withdrawn.

If an action makes no change (for example, asking to withdraw an item that is already withdrawn) then, just like metadata that has not changed, this will be ignored.

Migrating Data or Exchanging data

It is possible that you have data in one Dublin Core (DC) element and you wish to really have it in another. An example would be that your staff have input Library of Congress Subject Headings in the Subject field (dc.subject) instead of the LCSH field (dc.subject.lcsh). Follow these steps and your data is migrated upon import:

1. Insert a new column. The first row should be the new metadata element. (We will refer to it as the TARGET)
2. Select the column/rows of the data you wish to change. (We will refer to it as the SOURCE)
3. Cut and paste this data into the new column (TARGET) you created in Step 1.
4. Leave the column (SOURCE) you just cut and pasted from empty. Do not delete it.

Common Issues

Metadata values in CSV export seem to have duplicate columns

DSpace responds with "No changes were detected" when CSV is uploaded

Unfortunately, this response may be caused in many ways

- It's possible the CSV was not saved properly after editing. Check that the edits are in the CSV, and that there were no backend errors in the DSpace logs (which would be an indication of an invalid or corrupted CSV file)
- Depending on the version of DSpace, you may be encountering this known bug with processing linebreaks in CSV files: https://github.com/DSpace/DSpace/issues/6600
- If you are setting a new embargo date in the CSV, ensure that the embargo lift date is a future date. It's been reported that past dates may cause DSpace to ignore item changes.

Batch Editing, Entities and Relationships

Consider the following page for this topic: Configurable Entities

Background about entities and virtual metadata

- In DSpace 7, we have entities. Entities are items with an entity type (there can still be items without an entity type).
- Two entities can be linked to each other. For this purpose relations are defined, which indicate the relationship between the entities. Relationships between two entities are defined by the metadata schema *relation*. The relation reflects how two entities are related to each other, for example *isPersonOfProject* or *isPublicationOfAuthor*.
- Until the introduction of entities, we could only link items to each other by inserting DOIs or URLs of other items into metadata fields *dc.relation.**. What is new about the linking between entities in DSpace 7 is that UUIDs are entered into the fields, i.e. internal identifiers of other entities that DSpace can easily resolve. DSpace "knows" which entities are linked to each other and how.
- On the item view of an entity (remember: an entity is an item with an entity type) metadata of other entities can be displayed. DSpace refers to this as virtual metadata. Virtual metadata does not belong to the item in whose item view it is displayed, but to a linked entity. They can be changed only in the linked entity. As an example: we have the entities journal and journal issue. All journal issues display the title of the journal in their item view. This title is stored only in the journal and is only (dynamically) displayed in the issues.

Admin CSV export

- Virtual metadata is exported with the entities in which it is included. For example, when you export projects, you see a column for the *project.investigator* field. Here, the names of two people have been included as virtual metadata. However, the names are not stored in the project, but exported from the respective person entities at the time of export. The specification *::virtual::* marks this. The specifications *::8585::* and *::27946::* are examples for this documentation and represent IDs of the relations. The specification *::600* comes from the DSpace Authority, which is also specified due to technical circumstances.
- The relation itself is also included in the CSV export, in the *relation.isPersonOfProject* field. Additionally, a *relation.isPersonOfProject.latestForDiscovery* field is created. This field has internal reasons in DSpace and should help to make things faster discoverable. In the fields you again see the *::virtual::8585::600* specification, which are already explained above. Instead of the values of individual metadata fields, you now have the UUIDs of the items that are linked. You can always get these UUIDs from the URL of the item view of an item.

An example heading row of the CSV export file (project entity):

```
id,collection,dc.title,project.investigator,relation.isPersonOfProject,etc,etc,etc.
```

Subsequent example row in the CSV export file (project entity):

```
350,2292,Project title,"Smith, John::virtual::8585::600||Doe, Jane::virtual::27946::600","d89c1eb1-2e7c-4912-
a1eb-f27b17fd6848::virtual::8585::600||e3595b14-6937-47b9-b718-1972cb683943::virtual::27946::600"
```

Admin CSV import

- As always, only the columns and rows that will be changed should be specified. You do not want to import the columns that contain virtual metadata, because they are not stored in the imported items, but in the linked items. So in the above example you don't want to import the *project.investigator* column, but delete it from the CSV.
- To link one item to another you need to create a corresponding column of the *relation* metadata schema, so in our example above *relation.isPersonOfProject*. All columns of the form *relation.*.latestForDiscovery* are created and updated automatically, so you don't want to import them. If you want to create a new relation, of course you don't know the ID of the relation, you can replace it with a +, then DSpace will assign it on its own. Of course, people can also be removed from the column or completely new relations can be created for new items, even if there are no old ones to be taken over.

An example heading row for the CSV import file (project entity):

```
id,collection,dc.title,relation.isPersonOfProject,etc,etc,etc.
```

Subsequent example row for the CSV import file (project entity):

```
350,2292,Project title,"d89c1eb1-2e7c-4912-a1eb-f27b17fd6848::virtual::8585::600||e3595b14-6937-47b9-b718-
1972cb683943::virtual::+::600"
```

Batch Metadata Editing Configuration

The Batch Metadata Editing Tool allows the administrator to extract from the DSpace database a set of records for editing via a CSV file. It provides an easier way of editing large collections.

A full list of all available Batch Metadata Editing Configurations:

Configuration File:	**`[dspace]/config/modules/bulkedit.cfg`**
Property:	`bulkedit.valueseparator`
Example Value:	`bulkedit.valueseparator = ||`
Informational note	The delimiter used to separate values within a single field. For example, this will place the double pipe between multiple authors appearing in one record (Smith, William || Johannsen, Susan). This applies to any metadata field that appears more than once in a record. The user can change this to another character.
Property:	`bulkedit.fieldseparator`
Example Value:	`bulkedit.fieldseparator = ,`
Informational note	The delimiter used to separate fields (defaults to a comma for CSV). Again, the user could change it something like '$'. If you wish to use a tab, semicolon, or hash (#) sign as the delimiter, set the value to be `tab`, `semicolon` or `hash`. `bulkedit.fieldseparator = tab`
Property:	`bulkedit.authorityseparator`
Example Value:	`bulkedit.authorityseparator = ::`
Informational note	The delimiter used to separate authority data (defaults to a double colon ::)
Property:	`bulkedit.export.max.items`
Example Value:	`bulkedit.export.max.items = 500`
Informational note	(As of 7.6.4) Maximum number of items which can be exported at one time. This corresponds to the maximum number of data rows which will appear in the exported CSV. Default is 500 items.
Property:	`bulkedit.gui-item-limit`
Example Value:	`bulkedit.gui-item-limit = 20`
Informational note	When using the WEBUI, this sets the limit of the number of items allowed to be edited in one processing. There is no limit when using the CLI.
Property:	`bulkedit.ignore-on-export`

Example Value:	`bulkedit.ignore-on-export = dc.date.accessioned, \`
	`dc.date.available, \`
	`dc.date.updated, dc.description.provenance`

Informational note	Metadata elements to exclude when exporting via the user interfaces, or when using the command line version and not using the -a (all) option.
Property:	`bulkedit.allowexpunge`
Example Value:	`bulkedit.allowexpunge = false`
Informational note	Should the 'action' column allow the 'expunge' method. By default this is set to false
Property	`bulkedit.allow-bulk-deletion`
Example Value:	`bulkedit.allow-bulk-deletion = dspace.agreements.end-user`

Informational note	Comma separated list of metadata fields that can be deleted in bulk using the 'metadata-deletion' script. By default only the 'dspace.agreements.end-user' field can be deleted in bulk, as doing so allows an Administrator to force all users to re-review the End User Agreement on their next login. However, you may choose to enable additional fields. Keep in mind, any fields listed here may be batch deleted from the Processes UI & such metadata deletions cannot be undone.

DOI Digital Object Identifier

- Persistent Identifier
- DOI Registration Agencies
  - Configure DSpace to use the DataCite API
    - dspace.cfg
    - Metadata conversion
    - Identifier Service
    - Sending metadata updates to DataCite
    - DOIs using DataCite and Item Level Versioning
    - Command Line Interface
    - 'cron' job for asynchronous reservation/registration
    - Limitations of DataCite DOI support
  - Configure DSpace to use EZID service for registration of DOIs
    - Limitations of EZID DOI support
- Adding support for other Registration Agencies
- Configuring pre-registration of Identifiers
  - Why mint in submission?
  - Enable the Identifiers step
  - Configure filters and behaviour
  - Administrator registration

Persistent Identifier

It is good practice to use Persistent Identifiers to address items in a digital repository. There are many different systems for Persistent Identifiers: Handle , DOI , urn:nbn, purl and many more. It is far out of the scope of this document to discuss the differences of all these systems. For several reasons the Handle System is deeply integrated in DSpace, and DSpace makes intensive use of it. With DSpace 3.0 the Identifier Service was introduced that makes it possible to also use external identifier services within DSpace.

DOIs are Persistent Identifiers like Handles are, but as many big publishing companies use DOIs they are quite well-known to scientists. Some journals ask for DOIs to link supplemental material whenever an article is submitted. Beginning with DSpace 4.0 it is possible to use DOIs in parallel to the Handle System within DSpace. By "using DOIs" we mean automatic generation, reservation and registration of DOIs for every item that enters the repository. These newly registered DOIs will not be used as a means to build URLs to DSpace items. Items will still rely on handle assignment for the item urls.

DOI Registration Agencies

To register a DOI one has to enter into a contract with a DOI registration agency which is a member of the International DOI Foundation. Several such agencies exist. Different DOI registration agencies have different policies. Some of them offer DOI registration especially or only for academic institutions, others only for publishing companies. Most of the registration agencies charge fees for registering DOIs, and all of them have different rules describing for what kind of item a DOI can be registered. To make it quite clear: to register DOIs with DSpace you have to enter into a contract with a DOI registration agency.

DataCite is an international initiative to promote science and research, and a member of the International DOI Foundation. The members of DataCite act as registration agencies for DOIs. Some DataCite members provide their own APIs to reserve and register DOIs; others let their clients use the DataCite API directly. Starting with version 4.0 DSpace supports the administration of DOIs by using the DataCite API directly or by using the API from EZID (which is a service of the University of California Digital Library). This means you can administer DOIs with DSpace if your registration agency allows you to use the DataCite API directly or if your registration agency is EZID.

Configure DSpace to use the DataCite API

If you use a DOI registration agency that lets you use the DataCite API directly, you can follow the instructions below to configure DSpace. In case EZID is your registration agency the configuration of DSpace is documented here: Configure DSpace to use EZID service for registration of DOIs.

To use DOIs within DSpace you have to configure several parts of DSpace:

- enter your DOI prefix and the credentials to use the API from DataCite in dspace.cfg,
- configure the script which generates some metadata,
- activate the DOI mechanism within DSpace,
- configure a cron job which transmits the information about new and changed DOIs to the registration agency.

dspace.cfg

After you enter into a contract with a DOI registration agency, they'll provide you with user credentials and a DOI prefix. You have to enter these in the dspace cfg. Here is a list of DOI configuration options in dspace.cfg:

Configuration File:	**[dspace]/config/dspace.cfg**
Property:	`identifier.doi.user`

Example Value:	`identifier.doi.user = user123`
Informational Note:	Username to login into the API of the DOI registration agency. You'll get it from your DOI registration agency.
Property:	`identifier.doi.password`
Example Value:	`identifier.doi.password = top-secret`
Informational Note:	Password to login into the API of the DOI registration agency. You'll get it from your DOI registration agency.
Property:	`identifier.doi.prefix`
Example Value:	`identifier.doi.prefix = 10.5072`
Informational Note:	The prefix you got from the DOI registration agency. All your DOIs start with this prefix, followed by a slash and a suffix generated from DSpace. The prefix can be compared with a namespace within the DOI system.
Property:	`identifier.doi.namespaceseparator`
Example Value:	`identifier.doi.namespaceseparator = dspace-`
Informational Note:	This property is optional. If you want to use the same DOI prefix in several DSpace installations or with other tools that generate and register DOIs it is necessary to use a namespace separator. All the DOIs that DSpace generates will start with the DOI prefix, followed by a slash, the namespace separator and some number generated by DSpace. For example, if your prefix is 10.5072 and you want all DOIs generated by DSpace to look like 10.5072/dspace-1023 you have to set this as in the example value above.
Property:	`identifier.doi.resolver`
Example Value:	`identifier.doi.resolver = https://doi.org`
Informational Note:	URL for the DOI resolver. This will be the stem for generated DOIs. This property is optional, and defaults to the example value above.
Property:	crosswalk.dissemination.DataCite.publisher
Example Value:	crosswalk.dissemination.DataCite.publisher = My University Press
Informational Note:	The name of the publishing institution or publisher.
Property:	crosswalk.dissemination.DataCite.hostingInstitution
Example Value:	crosswalk.dissemination.DataCite.hostingInstitution = My University
Informational Note:	The name of the organization/institution which hosts this instance of the object. If not configured, it will default to the value of crosswalk.dissemination.DataCite.publisher.
Property:	crosswalk.dissemination.DataCite.dataManager
Example Value:	crosswalk.dissemination.DataCite.dataManager = My University Department of Geology
Informational Note:	If not configured, it will default to the value of crosswalk.dissemination.DataCite.publisher.

Please don't use the test prefix 10.5072 with DSpace. The test prefix 10.5072 differs from other prefixes: It answers GET requests for all DOIs even for DOIs that are unregistered. DSpace checks that it mint only unused DOIs and will create an Error: "Register DOI ... failed: DOI_ALREADY_EXISTS". Your registration agency can provide you an individual test prefix, that you can use for tests.

Metadata conversion

To reserve or register a DOI, DataCite requires that metadata be supplied which describe the object that the DOI addresses. The file [dspace]/config /crosswalks/DIM2DataCite.xsl controls the conversion of metadata from the DSpace internal format into the DataCite format. If you are running a version of DSpace earlier than 6.0, you have to add your DOI prefix, namespace separator and the name of your institution to this file:

[dspace]/config/crosswalks/DIM2DataCite.xsl

```
<!--
    Document   : DIM2DataCite.xsl
    Created on : January 23, 2013, 1:26 PM
    Author     : pbecker, ffuerste
    Description: Converts metadata from DSpace Intermediat Format (DIM) into
                 metadata following the DataCite Schema for the Publication and
                 Citation of Research Data, Version 2.2
-->
<xsl:stylesheet xmlns:xsl="http://www.w3.org/1999/XSL/Transform"
                xmlns:dspace="http://www.dspace.org/xmlns/dspace/dim"
                xmlns="http://datacite.org/schema/kernel-2.2"
                version="1.0">

    <!-- CONFIGURATION -->
    <!-- Please add your DOI-Prefix and your namespace separator here (e.g. 10.5072-dspace-). -->
    <xsl:variable name="prefix">10.5072-dspace-</xsl:variable>
    <!-- The content of the following variable will be used as element publisher. -->
    <xsl:variable name="publisher">My University</xsl:variable>
    <!-- The content of the following variable will be used as element contributor with contributorType
datamanager. -->
    <xsl:variable name="datamanager"><xsl:value-of select="$publisher" /></xsl:variable>
    <!-- The content of the following variable will be used as element contributor with contributorType
hostingInstitution. -->
    <xsl:variable name="hostinginstitution"><xsl:value-of select="$publisher" /></xsl:variable>
    <!-- Please take a look into the DataCite schema documentation if you want to know how to use these
elements.
         http://schema.datacite.org -->


    <!-- DO NOT CHANGE ANYTHING BELOW THIS LINE EXCEPT YOU REALLY KNOW WHAT YOU ARE DOING! -->
...
```

If you are running DSpace 6.0 or later, then you do not have to change the XSL to change the publisher, datamanager or hostinginstitution. Just change the configured using the `crosswalk.dissemination.DataCite.*` properties in `local.cfg` (see `dspace.cfg` for examples).

If you want to know more about the DataCite Schema, have a look at the documentation. If you change this file in a way that is not compatible with the DataCite schema, you won't be able to reserve and register DOIs anymore. Do not change anything if you're not sure what you're doing. To get the XML on which the XSLT processor will start, use the following command:

```
[dspace]/bin/dspace dsrun org.dspace.content.crosswalk.XSLTDisseminationCrosswalk dim 123456789/3
```

To get the XML that will be send to DataCite replace 'dim' with 'DataCite'. If the DOI is not stored in the metadata, DSpace will add it automatically as identifier. So don't worry if the XML produced by this command does not contain the DOI. Once the DOI is stored in the metadata, it should also be contained in the XML.

Identifier Service

The Identifier Service manages the generation, reservation and registration of identifiers within DSpace. You can configure it using the config file located in [dspace]/config/spring/api/identifier-service.xml. In the file you should already find the code to configure DSpace to register DOIs. Just read the comments and remove the comment signs around the two appropriate beans.

After removing the comment signs the file should look something like this (I removed the comments to make the listing shorter):

[dspace]/config/spring/api/identifier-service.xml

```
<!--
    Copyright (c) 2002-2010, DuraSpace.  All rights reserved
    Licensed under the DuraSpace License.

    A copy of the DuraSpace License has been included in this
    distribution and is available at: http://www.dspace.org/license
-->

<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xsi:schemaLocation="http://www.springframework.org/schema/beans
           http://www.springframework.org/schema/beans/spring-beans-2.5.xsd">

    <bean id="org.dspace.identifier.service.IdentifierService"
          class="org.dspace.identifier.IdentifierServiceImpl"
          autowire="byType"
          scope="singleton"/>

    <bean id="org.dspace.identifier.DOIIdentifierProvider"
        class="org.dspace.identifier.DOIIdentifierProvider"
        scope="singleton">
        <property name="configurationService"
            ref="org.dspace.services.ConfigurationService" />
        <property name="DOIConnector"
            ref="org.dspace.identifier.doi.DOIConnector" />
    </bean>

    <bean id="org.dspace.identifier.doi.DOIConnector"
        class="org.dspace.identifier.doi.DataCiteConnector"
        scope="singleton">
        <property name='DATACITE_SCHEME' value='https'/>
        <property name='DATACITE_HOST' value='mds.test.datacite.org'/>
        <property name='DATACITE_DOI_PATH' value='/doi/' />
        <property name='DATACITE_METADATA_PATH' value='/metadata/' />
        <property name='disseminationCrosswalkName' value="DataCite" />
    </bean>
</beans>
```

If you use other IdentifierProviders beside the DOIIdentifierProvider there will be more beans in this file.

Please pay attention to configure the property DATACITE_HOST. Per default it is set to the DataCite test server. To reserve real DOIs you will have to change it to mds.datacite.org. Ask your registration agency if you're not sure about the correct address.

Sending metadata updates to DataCite

DSpace should send updates to DataCite whenever the metadata of an item changes. To do so, you must enable the DOIConsumer in your dspace.cfg (or local.cfg. You should remove the comments in front of the two following properties or add them to the local.cfg:

[dspace]/config/dspace.cfg

```
event.consumer.doi.class = org.dspace.identifier.doi.DOIConsumer
event.consumer.doi.filters = Item+Modify_Metadata
```

Then you should add 'doi' to the property `event.dispatcher.default.consumers`. After adding it, this property may look like this:

[dspace]/config/dspace.cfg

```
event.dispatcher.default.consumers = versioning, discovery, eperson, harvester, doi
```

DOIs using DataCite and Item Level Versioning

If you enabled Item Level Versioning you should enable the `VersionedDOIIdentifierProvider` instead of the `DOIIdentifierProvider`. The `VersionedDOIIdentifierProvider` ensures that newer versions of the same Item gets a DOI looking as the DOI of the first version of an item, extended by a dot and the version number. With DSpace 6 this also became the default for handles if Item Level Versioning is enabled. In the configuration file `[dspace]/config/spring/api/identifier-service.xml` you'll find the possiblity to enable the `VersionedDOIIdentifierProvider`. If you want to use versioned DOIS, please comment out the `DOIIdentifierProvider` as only one of both DOIProviders should be enabled at the same time.

Command Line Interface

To make DSpace resistant to outages of DataCite we decided to separate the DOI support into two parts. When a DOI should be generated, reserved or minted, DSpace does this in its own database. To perform registration and/or reservation against the DOI registration agency a job has to be started using the command line. Obviously this should be done by a cron job periodically. In this section we describe the command line interface, in case you ever want to use it manually. In the next section you'll see the cron job that transfers all DOIs designated for reservation and/or registration.

The command line interface in general is documented here: Command Line Operations.

The command used for DOIs is '`doi-organiser`'. You can use the following options:

Option (short)	Option (long)	Parameter	Description
-d	--delete-all		Transmit information to the DOI registration agency about all DOIs that were deleted.
	--delete-doi	DOI	Transmit information to the DOI registration agency that the specified DOI was deleted. The DOI must already be marked for deletion; you cannot use this command to delete a DOI for an exisiting item.
-h	--help		Print online help.
-l	--list		List all DOIs whose changes were not committed to the registration agency yet.
-q	--quiet		The doi-organiser sends error reports to the mail address configured in the property alert.recipient in dspace.cfg. If you use this option no output should be given to stdout. If you do not use this option the doi-organiser writes information about successful and unsuccessful operations to stdout and stderr. You can find information in dspace.log of course.
-r	--register-all		Transmit information about all DOIs that should be registered.
	--register-doi	DOI | ItemID | handle	If a DOI is marked for registration, you can trigger the registration at the DOI registration agency by this command. Specify either the DOI, the ID of the item, or its handle.
-s	--reserve-all		Transmit to the DOI registration agency information about all DOIs that should be reserved.
	--reserve-doi	DOI | ItemID | handle	If a DOI is marked for registration, you can trigger the registration at the DOI registration agency by this command. Specify either the DOI, the ID of the item, or its handle.
-u	--update-all		If a DOI is reserved for an item, the metadata of the item will be sent to DataCite. This command transmits new metadata for items whose metadata were changed since the DOI was reserved.
	--update-doi	DOI | ItemID | handle	If a DOI needs an update of the metadata of the item it belongs to, you can trigger this update with this command. Specify either the DOI, the ID of the item, or its handle.

Currently you cannot generate new DOIs with this tool. You can only send information about changes in your local DSpace database to the registration agency.

'cron' job for asynchronous reservation/registration

When a DOI should be reserved, registered, deleted or its metadata updated, DSpace just writes this information into its local database. A command line interface is supplied to send the necessary information to the registration agency. This behavior makes it easier to react to outages or errors while using the API. This information should be sent regularly, so it is a good idea to set up a cron job instead of doing it manually.

There are four commands that should be run regularly:

- Update the metadata of all items that have changed since their DOI was reserved.
- Reserve all DOIs marked for reservation
- Register all DOIs marked for registration
- Delete all DOIs marked for deletion

In DSpace, a DOI can have the state "registered", "reserved", "to be reserved", "to be registered", "needs update", "to be deleted", or "deleted". After updating an item's metadata the state of its assigned DOI is set back to the last state it had before. So, e.g., if a DOI has the state "to be registered" and the metadata of its item changes, it will be set to the state "needs update". After the update is performed its state is set to "to be registered" again. Because of this behavior **the order of the commands above matters**: the update command must be executed before all of the other commands above.

The cron job should perform the following commands with the rights of the user your DSpace installation runs as:

```
[dspace]/bin/dspace doi-organiser -u -q
[dspace]/bin/dspace doi-organiser -s -q
[dspace]/bin/dspace doi-organiser -r -q
[dspace]/bin/dspace doi-organiser -d -q
```

The doi-organiser sends error messages as email and logs some additional information. The option -q tells DSpace to be quiet. If you don't use this option the doi-organiser will print messages to stdout about every DOI it successfully reserved, registered, updated or deleted. Using a cron job these messages would be sent as email.

In case of an error, consult the log messages. If there is an outage of the API of your registration agency, DSpace will not change the state of the DOIs so that it will do everything necessary when the cron job starts the next time and the API is reachable again.

The frequency the cron job runs depends on your needs and your hardware. The more often you run the cron job the faster your new DOIs will be available online. If you have a lot of submissions and want the DOIs to be available really quickly, you probably should run the cron job every fifteen minutes. If there are just one or two submissions per day, it should be enough to run the cron job twice a day.

To set up the cron job, you just need to run the following command as the *dspace* UNIX user:

```
crontab -e
```

The following line tells cron to run the necessary commands twice a day, at 1am and 1pm. Please notice that the line starting with the numbers is one line, even it it should be shown as multiple lines in your browser.

```
# Send information about new and changed DOIs to the DOI registration agency:
0 1,13 * * *  [dspace]/bin/dspace doi-organiser -u -q ; [dspace]/bin/dspace doi-organiser -s -q ; [dspace]/bin
/dspace doi-organiser -r -q ; [dspace]/bin/dspace doi-organiser -d -q
```

Limitations of DataCite DOI support

Every DSpace installation expects to be the only application that generates DOIs which start with the prefix and the namespace separator you configured. DSpace does not check whether a DOI it generates is reserved or registered already.

That means if you want to use other applications or even more than one DSpace installation to register DOIs with the same prefix, you'll have to use a unique namespace separator for each of them. Also you should not generate DOIs manually with the same prefix and namespace separator you configured within DSpace. For example, if your prefix is 10.5072 you can configure one DSpace installation to generate DOIs starting with 10.5072/papers-, a second installation to generate DOIs starting with 10.5072/data- and another application to generate DOIs starting with 10.5072/results-.

DOIs will be used in addition to Handles. This implementation does not replace Handles with DOIs in DSpace. That means that DSpace will still generate Handles for every item, every collection and every community, and will use those Handles as part of the URL of items, collections and communities.

DSpace currently generates DOIs for items only. There is no support to generate DOIs for Communities and collections yet.

When using DSpace's support for the DataCite API probably not all information would be restored when using the AIP Backup and Restore (see https://github.com/DSpace/DSpace/issues/5203). The DOIs included in metadata of Items will be restored, but DSpace won't update the metadata of those items at DataCite anymore. You can even get problems when minting new DOIs after you restored older ones using AIP.

Configure DSpace to use EZID service for registration of DOIs

The EZID IdentifierProvider operates synchronously, so there is much less to configure. You will need to un-comment the `org.dspace.identifier.EZIDIdentifierProvider` bean in `config/spring/api/identifier-service.xml` to enable DOI registration through EZID.

In `config/dspace.cfg` you will find a small block of settings whose names begin with `identifier.doi.ezid`. You should uncomment these properties and give them appropriate values. Sample values for a test account are supplied.

name	meaning
identifier.doi.ezid.shoulder	The "shoulder" is the DOI prefix issued to you by the EZID service. DOIs minted by this instance of DSpace will be the concatenation of the "shoulder" and a locally unique token.
identifier.doi.ezid.user identifier.doi.ezid.password	The username and password by which you authenticate to EZID.
identifier.doi.ezid.publisher	You may specify a default value for the required `datacite.publisher` metadatum, for use when the Item has no publisher.
crosswalk.dissemination.DataCite.publisher	Should match identifier.doi.ezid.publisher.
crosswalk.dissemination.DataCite.hostingInstitution	Name of the hosting institution. If not configured, it will be set to the value of crosswalk.dissemination.DataCite.publisher.
crosswalk.dissemination.DataCite.dataManager	Name of the data manager. If not configured, it will be set to the value of crosswalk.dissemination.DataCite.publisher.

Back in `config/spring/api/identifier-service.xml` you will see some other configuration of the `EZIDIdentiferProvider` bean. In most situations, the default settings should work well. But, here's an explanation of options available:

- *EZID Provider / Registrar settings:* By default, the EZIDIdentifierProvider is configured to use the CDLib provider (ezid.cdlib.org) in the `EZID_SCHEME`, `EZID_HOST` and `EZID_PATH` settings. In most situations, the default values should work for you. However, you may need to modify these values (especially the `EZID_HOST`) if you are registered with a different EZID provider. In that situation, please check with your provider for valid "host" and "path" settings. If your provider provides EZID service at a particular path on its host, you may set that in `EZID_PATH`.
    - NOTE: As of the writing of this documentation, the default CDLib provider settings should also work for institutions that use Purdue (ezid.lib.purdue.edu) as a provider. Currently, Purdue and CDLib currently share the same infrastructure, and both `ezid.cdlib.org` and `ezid.lib.purdue.edu` point to the same location.
- *Metadata mappings:* You can alter the mapping between DSpace and EZID metadata, should you choose. The `crosswalk` property is a map from DSpace metadata fields to EZID fields, and can be extended or changed. The `key` of each `entry` is the name of an EZID metadata field; the `value` is the name of the corresponding DSpace field, from which the EZID metadata will be populated.
- *Crosswalking / Transforms:* You can also supply transformations to be applied to field values using the `crosswalkTransform` property. Each `key` is the name of an EZID metadata field, and its `value` is the name of a Java class which will convert the value of the corresponding DSpace field to its EZID form. The only transformation currently provided is one which converts a date to the year of that date, named `org.dspace.identifier.ezid.DateToYear`. In the configuration as delivered, it is used to convert the date of issue to the year of publication. You may create new Java classes with which to supply other transformations, and map them to metadata fields here. If an EZID metadatum is not named in this map, the default mapping is applied: the string value of the DSpace field is copied verbatim.

Limitations of EZID DOI support

DOIs will be used in addition to Handles. This implementation does not replace Handles with DOIs in DSpace. That means that DSpace will continue to generate Handles for every item, every collection and every community, and will use those Handles as part of the URL of items, collections and communities.

Currently, the EZIDIdentifierProvider has a known issue where it stores its DOIs in the `dc.identifier` field, instead of using the `dc.identifier.uri` field (which is the one used by DataCite DOIs and Handles). See https://github.com/DSpace/DSpace/pull/1006 for more details. This will be corrected in a future version of DSpace.

DSpace currently generates DOIs for items only. There is no support to generate DOIs for Communities and Collections yet.

Adding support for other Registration Agencies

If you want DSpace to support other registration agencies, you just have to write a Java class that implements the interface DOIConnector ([dspace-source]/dspace-api/src/main/java/org/dspace/identifier/doi/DOIConnector.java). You might use the DataCiteConnector ([dspace-source]/dspace-api/src/main/java/org/dspace/identifier/doi/DataCiteConnector.java) as an example. After developing your own DOIConnector, you configure DSpace as if you were using the DataCite API directly. Just use your DOIConnector when configuring the IdentifierService instead of the DataCiteConnector.

Configuring pre-registration of Identifiers

Why mint in submission?

Users often want to see what DOI they **will** get so they can alter their PDF, coverpage, other metadata, and so on.

This feature should ensure that users can see their future DOI, and if necessary, a warning that if certain conditions are not met, the DOI will not be registered after approval.

Keeping a DOI in pending status does use up an integer from the total DOI namespace, but it also ensures that the submitter, reviewers, administrators etc know what the DOI will be if it is ever registered in the future.

If this is really not desired, eg. there are many item types which should never get a DOI, then there is a way to configure a filter that avoids minting a new PENDING DOI at all unless conditions are met in submission.

Enable the Identifiers step

See Submission User Interface#Configuringthe%22Identifiers%22step

Configure filters and behaviour

To enable this feature and configure the exact way it works, edit the `${dspace.dir}/dspace/modules/identifiers.cfg` configuration file

Property:	`identifiers.submission.register`
Example Value:	`true`

Informational Note:	Enable this feature. Default: false. Handles will be registered at time of submission. DOIs (if item filters evaluate to true) will be minted in a "pending" state for items, to be registered or queued for registration at archival.
Property:	`identifiers.submission.filter.install`
Example Value:	`doi-filter`
Informational Note:	Bean ID of a logical item filter (see `config/modules/spring/api/item-filters.xml`) that will be used to evaluate whether a DOI should be queued for registration when this item is installed (archived) in DSpace. This filter will be applied whether or not a "pending" DOI is already minted for the item. (If a filter is absent or null, an item will always be evaluated as 'true')
Property:	`identifiers.submission.filter.workspace`
Example Value:	`always_true_filter`
Informational note	Bean ID of a logical item filter (see `config/modules/spring/api/item-filters.xml`) that will be used to evaluate whether a DOI should be minted as "pending" for registration when this item is first submitted as a workspace item in DSpace. Depending on the value of `identifiers.submission.strip_pending_during_submission` this filter will be checked whenever the workspace item changes, to see if it now qualifies for a DOI. Default: always_true_filter (If a filter is absent or null, an item will always be evaluated as 'true')
Property:	`identifiers.submission.strip_pending_during_submission`
Example Value:	`true`
Informational Note:	If, during workspace item changes, the workspace filter **no longer** evaluates to true, should any DOIs be stripped? (moved to MINTED or DELETED status) This is useful in situations where the submitter needs real-time feedback as to whether their item qualifies for a DOI.
Property:	`identifiers.item-status.register-doi`
Example Value:	`false`
Informational Note:	Allow administrators to queue DOIs for registration in the Item Status page. Default: false. **Important:** This configuration property **must** be set, even if it matches the default, as it is exposed as a REST configuration property to the frontend.

Administrator registration

If an item does not have a DOI at all, or if an item has a MINTED or PENDING DOI, a user with ADMIN rights over the item may queue the DOI registration from the Item Status page. No filters will be applied to this action. This requires `identifiers.item-status.register-doi` to be `true` in identifiers configuration (see above)

Item Level Versioning

1 What is Item Level Versioning?
2 Important warnings
3 Disabling Item Level Versioning
4 Initial Requirements
5 User Interface
    5.1 General behaviour: Linear Versioning
    5.2 Creating a new version of an item
    5.3 View the history and older versions of an item
6 Architecture
    6.1 Versioning model
7 Configuration
    7.1 Versioning Service Override
    7.2 Identifier Service Override
    7.3 Version History Visibility
        7.3.1 Hide Editor/Submitter details in version table
    7.4 Allowing submitters to version their items
8 Identified Challenges & Known Issues
    8.1 Conceptual compatibility with Embargo
    8.2 Conceptual compatibility with Item Level Statistics

What is Item Level Versioning?

Versioning is a new functionality to build the history of an item. Users will have the opportunity to create new version of an existing item any time the will make a change.

Supported in 7.1 or above

Item level versioning was not fully supported in DSpace 7.0 (you were only able to view existing versions). It was restored in DSpace 7.1. See DSpace Release 7.0 Status

Important warnings

Item Level Versioning on Entities configuration

Configurable Entities are supported in Item Level Versioning support starting from version 7.3. More details about the configuration specific to Configurable Entities can be found on that page.

AIP Backup & Restore functionality only works with the Latest Version of Items

If you are using the AIP Backup and Restore functionality to backup / restore / migrate DSpace Content, you must be aware that the "Item Level Versioning" feature is **not yet compatible** with AIP Backup & Restore. **Using them together may result in accidental data loss.** Currently the AIPs that DSpace generates only store the *latest version* of an Item. Therefore, past versions of Items will always be lost when you perform a restore / replace using AIP tools. See https://github.com/DSpace/DSpace/issues/4751.

DSpace 6+ changed the way Handles are created for versioned Items

Starting with 6.0, the way DSpace crates Handles for versioned Items was changed. If you want to keep the old behavior of DSpace 4 and 5 you have to enable the `VersionedHandleIdentifierProviderWithCanonicalHandles` in the XML configuration files `[dspace]/config/spring/api/identifier-service.xml`. See IdentifierServiceOverride below for details and the comments in the configuration file.

Disabling Item Level Versioning

By default, Item Level Versioning is enabled in DSpace 7. You may choose to disable it by updating this configuration in your local.cfg:

```
versioning.enabled = false
```

Additionally, you will need to make the following changes to disable all versioning-related features:

- Switch to using the basic "HandleIdentifierProvider" in your `[dspace]/config/spring/api/identifier-services.xml`. Make sure to comment out the "VersionedHandleIdentifierProvider" and replace it with this:

```
<!-- This HandleIdentiferProvider should be used when versioning is disabled -->
<bean id="org.dspace.identifier.HandleIdentifierProvider" class="org.dspace.identifier.
HandleIdentifierProvider" scope="singleton">
    <property name="configurationService" ref="org.dspace.services.ConfigurationService"/>
</bean>
```

- Remove the "versioning" consumer from the list of default Event Consumers in either your dspace.cfg or local.cfg. Look for this configuration:

```
# Remove the "versioning" entry in this list
# (NOTE: Your list of consumers may be different based on the features you've enabled)
#event.dispatcher.default.consumers = versioning, discovery, eperson

# For example:
event.dispatcher.default.consumers = discovery, eperson
```

Once these changes are made, you will need to restart your servlet container (e.g. Tomcat) for the new settings to take effect.

Initial Requirements

The Item Level Versioning implementation builds on following requirements identified by the stakeholders who supported this contribution: Initial Requirements Analysis

1. What should be *Versionable*
    a. Versioning happens at the level of an Individual Item
    b. Versioning should preserve the current state of *metadata*, *bitstreams* and *resource policies* attached to the item.
2. Access, Search and Discovery
    a. Only the most recent version of an item is available via the search interface
    b. Previous versions of Items should continue to be visible, citable and accessible
    c. The Bitstreams for previous versions are retained. If something was once retrievable, it should always be retrievable.
3. Identifiers
    a. Each version of an Item is represented by a separate "*versioned*" identifier
    b. A base "*versionhistory*" Identifier points to the most recent version of the Item.
    c. A revision identifier also exists that is unique to the specific version.
    d. When a new version of an Item is deposited, a new revision identifier will be created.
4. Presentation
    a. On the item page, there is a link to view previous/subsequent versions.
    b. By examining the metadata or identifiers, it is possible to determine whether an item is the most recent version, the original version, or an intermediate version.
5. Access Control and Rights
    a. Certain roles should be able to generate a new version of the item via submission.
    b. To submitters, collection manager, administrators will be given to option to create new version of an item.
    c. Rights to access a specific Item should transmute as well to previous versions
    d. Rights to access a specific Bitstream should also transmute to previous versions.
6. Data Integrity
    a. The relationships between versions should not be brittle and breakable by manipulating Item metadata records.
    b. The relationships between versions should be preserved and predictable in various Metadata Exports (OAI, Packagers, ItemExport)
    c. The relationships between versions should be maintained in SWORD and AIP packaging and be maintained in updates and restorations.

User Interface

General behaviour: Linear Versioning

From the user interface, DSpace offers **linear** versioning. As opposed to hierarchical versioning, linear version has following properties:

- A new version can be created started from any available version but will be always be put at the end of the version history (it will be the latest)
- Only one in-progress version can exist at any time. When new version has been created and still needs to pass certain steps of the workflow, it is temporarily impossible to create another new version until the workflow steps are finished and the new version has replaced the previous one.

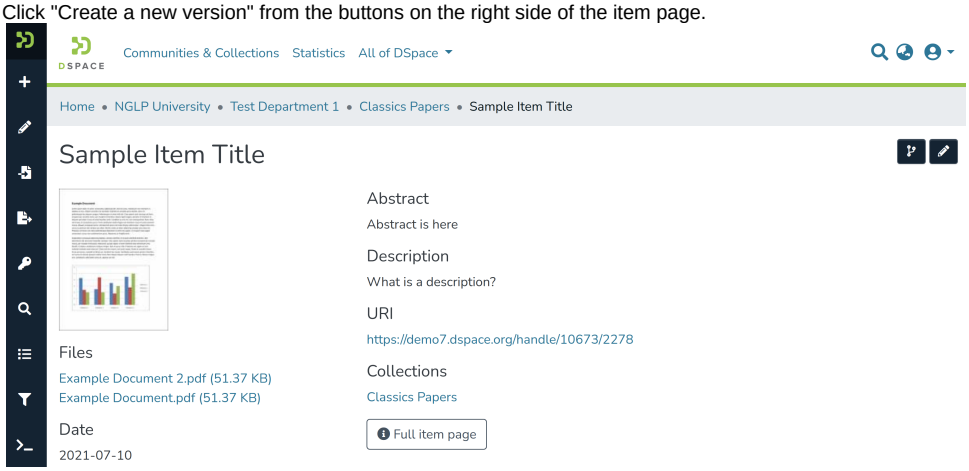
Creating a new version of an item

Administrators, collection/community administrators and eventually the original submitter can create new versions of an item from the Item View page. By default the original submitter is not allowed to create new version but the permission can be granted with the following property

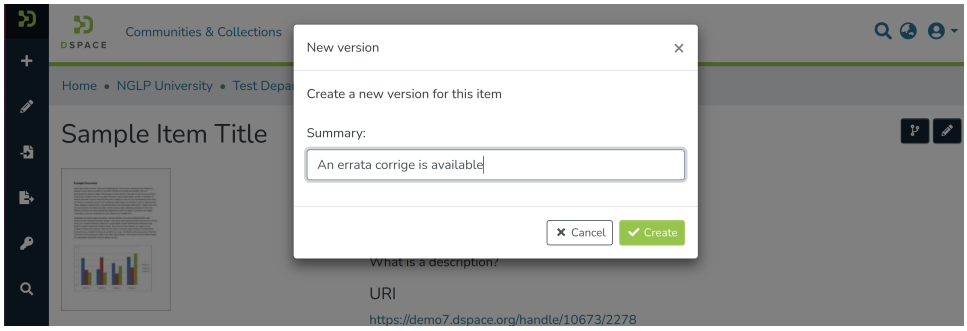
modules/versioning.cfg

```
versioning.submitterCanCreateNewVersion=false
```

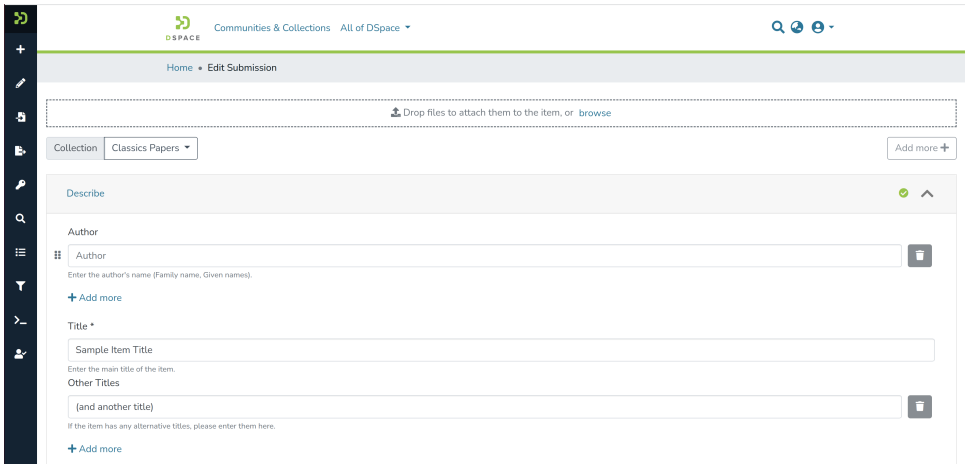
1. Click "Create a new version" from the buttons on the right side of the item page.



2. Provide the reason for creating a new version that will later on be stored and displayed in the version summary.

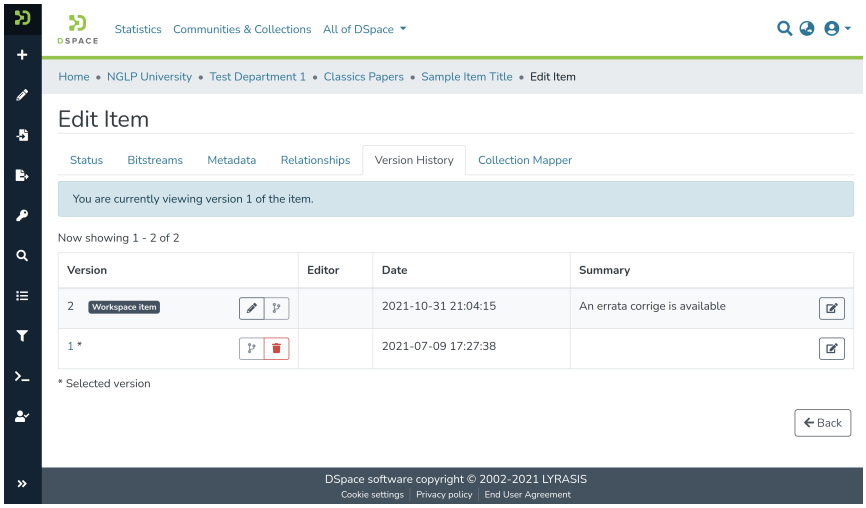


3. Your new version is now creates as a new Item in your Workspace. It requires you to go through the submission and workflow steps like you would do for a normal, new submission to the collection. The rationale behind this is that if you are adding new files or metadata, you will also need to accept the license for them. In addition to this, the versioning functionality does not bypass any quality control embedded in the workflow steps.



After the submission steps and the execution of subsequent workflow steps, the new version becomes available in the repository.

Versions can be also managed via the edit item page, in the dedicated versions tab



View the history and older versions of an item

An overview of the version history, including links to older versions of an item, is available at the bottom of an Item View page. The repository administrator can decide whether the version history should be available to all users or restricted to administrators. By default, this information is available to all users. Information displayed includes the version number, Submitter/Editor name (only if enabled), date, and summary/description. As necessary, you may change the visibility of this table or the "Editor" column using the "Version History Visibility" configurations below.

Version History

You are currently viewing version 2 of the item.

Now showing 1 - 2 of 2

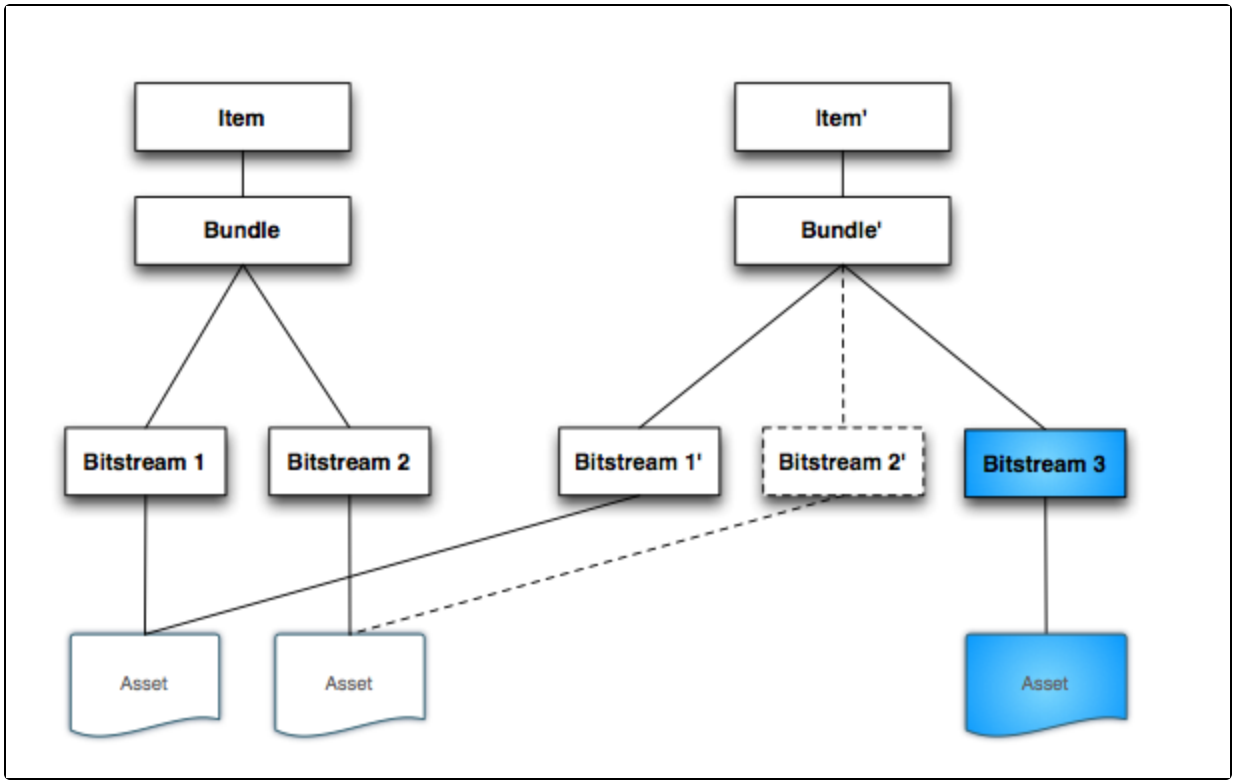
Version	Editor	Date	Summary
2 *		2021-10-31 21:04:15	An errata corrige is available
1		2021-07-09 17:27:38	

* Selected version

Architecture

Versioning model

For every new Version a separate DSpace Item will be created that replicates the metadata, bundle and bitstream records. The bitstream records will point to the same file on the disk.



The *Cleanup* method has been modified to retain the file if another Bitstream record point to it (the dotted lines in the diagram represent a bitstream deleted in the new version), in other words the file will be deleted only if the Bitstream record processed is the only one to point to the file *(count(INTERNAL_ID)=1).*

Configuration

Versioning Service Override

You can override the default behaviour of the Versioning Service using following XML configuration file, deployed under your dspace installation directory:

[dspace_installation_dir]/config/spring/api/versioning-service.xml

In this file, you can specify which metadata fields are automatically "reset" (i.e. cleared out) during the creation of a new item version. By default, all metadata values (and bitstreams) are copied over to the newly created version, with the exception of **dc.date.accessioned** and **dc.description.provenance.** You may specify additional metadata fields to reset by adding them to the "ignoredMetadataFields" property in the "versioning-service.xml" file:

```
<!-- Default Item Versioning Provider, defines behavior for replicating
     Item, Metadata, Budles and Bitstreams. Autowired at this time. -->
<bean class="org.dspace.versioning.DefaultItemVersionProvider">
    <property name="ignoredMetadataFields">
         <set>
            <value>dc.date.accessioned</value>
            <value>dc.description.provenance</value>
         </set>
    </property>
</bean>
```

Identifier Service Override

Persistent Identifiers are used to address Items within DSpace. The handle system is deeply integrated within DSpace, but since version 4 DSpace can also mint DOIs. DSpace 4 and 5 supported one type of versioned handle: The initial version of an Item got a handle, e.g. 10673/100. This handle was called the canonical one. When a newer version was created, the canonical handle was moved to identify the newest version. The previously newest version got a new handle build out of the canonical handle extended by a dot and the version number. In the image below you see a version history of an item using this handle strategy.

The canonical handle will always point to the newest version of an Item. This makes sense if you hide the version history. Normal users won't be able to find older versions and will always see just the newest one. Please keep in mind, that older versions can be accessed by "guessing" the versioned Handle if you do not remove the read policies manually. The downside of this identifier strategy is that there is no permanent handle to cite the currently newest version, as it will get a new Handle when a newer version is created.

With DSpace 6 versioned DOIs (using DataCite as DOI registration agency) were added and the default versioned Handle strategy was changed. Starting with DSpace 6 the `VersionedHandleIdentifierProvider` creates a handle for the first version of an item. Every newer version gets the same handle extended by a dot and the version number. To stay by the example from above, the first version of an Item gets the Handle 10673/100, the second version 10673/100.2, the third version 10673.3 and so on. This strategy has the downside that there is no handle pointing always to the newest version. But each version gets an identifier that can be use to cite exactly this version. If page numbers changes in newer editions the old citations stay valid.

In DSpace 4 and 5 only the strategy using canonical handles (one handle that always points to the newest version) were implemented. In DSpace 6 the strategy of creating a new handle for each version was implemented. With DSpace 6 this new strategy become the default. The strategy using canonical handle still exists in DSpace but you have to enable the `VersionedHandleIdentifierWithCanonicalHandles` in the file `[dspace]/config/spring/api/identifier-service.xml`. With DSpace 6 versioned DOIs were introduced using the strategy that every new version gets a new DOI (extended by a dot and the version numbers for versions >= 2). To use versioned Handle you have to enable DOIs, you have to use DataCite as registration agency and you have to enable the `VersionedDOIIdentifierProvider` in the named configuration file.

You can configure which persistent identifiers should be used by editing following XML configuration file, deployed under your dspace installation directory:

[dspace_installation_dir]/config/spring/api/identifier-service.xml

No changes to this file are required to enable Versioning. This file is currently only relevant if you want to keep the identifier strategy from DSpace 4 and 5 or if you want to enable DOIs or even versioned DOIs.

Version History Visibility

By default, **all** users will be able to see the version history. To ensure that only administrators can see the Version History, enable `versioning.item.history.view.admin` in the `[dspace]/config/modules/versioning.cfg` OR in your `local.cfg` file.

```
# Setting this to "true" will hide the entire "Version History" table from
# all users *except* Administrators
versioning.item.history.view.admin=false
```

Hide Editor/Submitter details in version table

In either the `[dspace]/config/modules/versioning.cfg` configuration file or your `local.cfg`, you can customize the configuration option `versioning.item.history.include.submitter`. By default this is set to false, which means that information about the submitter is only viewable by administrators. If you want to expose the submitters information to everyone (which be useful if all submitters uses generic institutional email addresses, but may conflict with local privacy laws if personal details are included) you can set this configuration property to true.

```
# This property controls whether the name of the submitter/editor
# of a version should be included in the version history of an item.
# Set to "true" to show the submitter to all users who have access to the table.
# Set to "false" to hide the submitter information from everyone except Administrators
versioning.item.history.include.submitter=false
```

Allowing submitters to version their items

With DSpace 6.0 it became possible to allow submitters to create new versions of their own items. The new versions are going through the normal workflow process if the collection is configured this way. To allow submitters to create new versions of Item they originally submitted, you have to change the configuration property `versioning.submitterCanCreateNewVersion` and set it to `true`. It is part of the configuration file `[dspace]/config/modules/versioning.cfg` but can be overridden in your `local.cfg` too.

Identified Challenges & Known Issues

Item Level Versioning has a substantial conceptual impact on many DSpace features. Therefore it has been accepted into DSpace as an optional feature. Following challenges have been identified in the current implementation. As an early adopter of the Item Level Versioning feature, your input is greatly appreciated on any of these.

Conceptual compatibility with Embargo

Lifting an embargo currently does not interact with Item Level Versioning. Additional implementation would be required to ensure that lifting an embargo actually creates a new version of the item.

Conceptual compatibility with Item Level Statistics

Both on the level of pageviews and downloads, different versions of an item are treated as different items. As a result, an end user will have the impression that the stats are being "reset" once a new version is installed, because the previous downloads and pageviews are allocated to the previous version.

One possible solution would be to present an end user with aggregated statistics across all viewers, and give administrators the possibility to view statistics per version.

Mapping/Linking Items to multiple Collections

- Introduction
- Using the Item Mapper
- Implications
    - Mapping collection vs Owning collection
    - Mapping an item does not modify access rights

Introduction

The Item Mapper is a tool in the DSpace web user interface allowing repository managers to display the same item in multiple collections at once. Thanks to this feature, a repository manager is not forced to duplicate items to display them in different collections

Using the Item Mapper

In the User Interface, the item mapper can be accessed when editing an Item.

- Login as someone with Edit permissions
- Search/browse to the Item
- Click the "Edit this Item" button
- Click "Mapped" collections" button on the "Status" tab
    - You'll be shown a list of currently mapped collections (if any)
    - You can also map the item to a new collection by clicking on "Map new collections" tab, and searching for the Collection(s)

Implications

Mapping collection vs Owning collection

The relation between an item and the collection in which it is mapped is different from the relation that this item has with the collection to which it was originally submitted. This second collection is referred to as the "owning" collection. When an item is deleted from the owning collection, it automatically disappears from the mapping collection. From within the mapping collection, the only thing that can be deleted is the mapping relation. Removing this mapping relation does not affect the presence of the item in the owning collection.

Mapping an item does not modify access rights

When an item gets mapped into a collection, it does not receive new access rights. It retains the authorizations that it inherited from the collection that "owns" it. Collection admins who do not have read access to an item will not be able to map them to other collections.

Metadata Recommendations

1 Recommended Metadata Fields
2 Local Fields

Recommended Metadata Fields

DSpace provides a broad list of metadata fields out of the box (see: Metadata and Bitstream Format Registries), and a variety of options for adding content to DSpace (both from the UI and from other services). No matter which Ingest option you use, DSpace recommends ensuring that the following metadata fields are specified:

- **Title** (`dc.title`)
  - When submitting an Item via the DSpace web user interface, this field is **required**.
  - If you add an Item to DSpace through another means (SWORD, etc), it is recommend to specify a title for an Item. Without a title, the Item will show up in DSpace a "Untitled".
- **Publication Date** (`dc.date.issued`)
  - When submitting an Item via the DSpace web user interface, this field is **required** (by default).
    - However, your System Administrator can choose to enable the "Initial Questions" step within the Submission User Interface. Enabling this step will cause the following to occur: If the item is said to be "published", then the Publication Date will be required. If the item is said to be "unpublished" then the Publication Date will be auto-set to today's date (date of submission). *WARNING:* Google Scholar has recommended against automatically assigning this "dc.date.issued" field to the date of submission as it often results in incorrect dates in Google Scholar results. See https://github.com/DSpace/DSpace/issues/4850 and https://github.com/DSpace/DSpace/issues/5112 for more details.
  - If you add and Item to DSpace through another means (SWORD, etc), it is recommended to specify the date in which an Item was published, in ISO-8601 (e.g. 2007, 2008-01, or 2011-03-04). This ensures DSpace can accurately report the publication date to services like Google Scholar. If an item is unpublished, you can either chose to leave this blank, or pass in the literal string "today" (which will tell DSpace to automatically set it to the date of ingest)

DSpace will not auto-assign a "dc.date.issued"

As of DSpace 4.0, the system will not assign a "dc.date.issued" when unspecified. Previous versions of DSpace (3.0 or below) would set "dc.date.issued" to the date of accession (dc.date.accessioned), if it was unspecified during ingest.

If you are adding content to DSpace without using the DSpace web user interface, there are two recommended options for assigning "dc.date.issued"

- If the item is previously published before, please set "dc.date.issued" to the date of publication in ISO-8601(e.g. 2007, 2008-01, or 2011-03-04)
- If the item has never been previously published, you may set "dc.date.issued='today'" (the literal string "today"). This will cause DSpace to automatically assign "dc.date.issued" to the date of accession (dc.date.accessioned), as it did previously
  - You can also chose to leave "dc.date.issued" as unspecified, but then the new Item will have an empty date within DSpace.

Obviously, we recommend specifying as much metadata as you can about a new Item. For a full list of supported metadata fields, please see: Metadata and Bitstream Format Registries

Local Fields

You may encounter situations in which you will require an appropriate place to store information that does not immediately fit with the description of a field in the default registry. The recommended practice in this situation is to create new fields in a separate schema. You can choose your own name and prefix for this schema such as *local.* or *myuni.*

It is generally discouraged to use any of the fields from the default schema as a place to store information that doesn't correspond with the fields description. This is especially true if you are ever considering the option to open up your repository metadata for external harvesting.

Moving Items

1 Moving Items via Web UI
2 Moving Items via the Batch Metadata Editor

Moving Items via Web UI

It is possible for Administrators to move items one at a time from the User Interface.

- Login as an Administrator
- Browse/search for the item.
- Click "Edit this Item" on the item page
- When editing an item, on the 'Edit item' screen, click the "Move..." button.
- Search for the new Collection for the item to appear in. By default, when the item is moved, it will take its authorizations (who can READ / WRITE it) with it.
    - If you wish for the item to take on the default authorizations of the destination collection, tick the 'Inherit policies' checkbox. This is useful if you are moving an item from a private collection to a public collection, or from a public collection to a private collection.
    - Note: When selecting the 'Inherit policies' option, ensure that this will not override system-managed authorizations such as those imposed by the embargo system.

Moving Items via the Batch Metadata Editor

Items may also be moved in bulk by using the CSV batch metadata editor (see Editing Collection Membership section under Batch Metadata Editing).

PDF Citation Cover Page

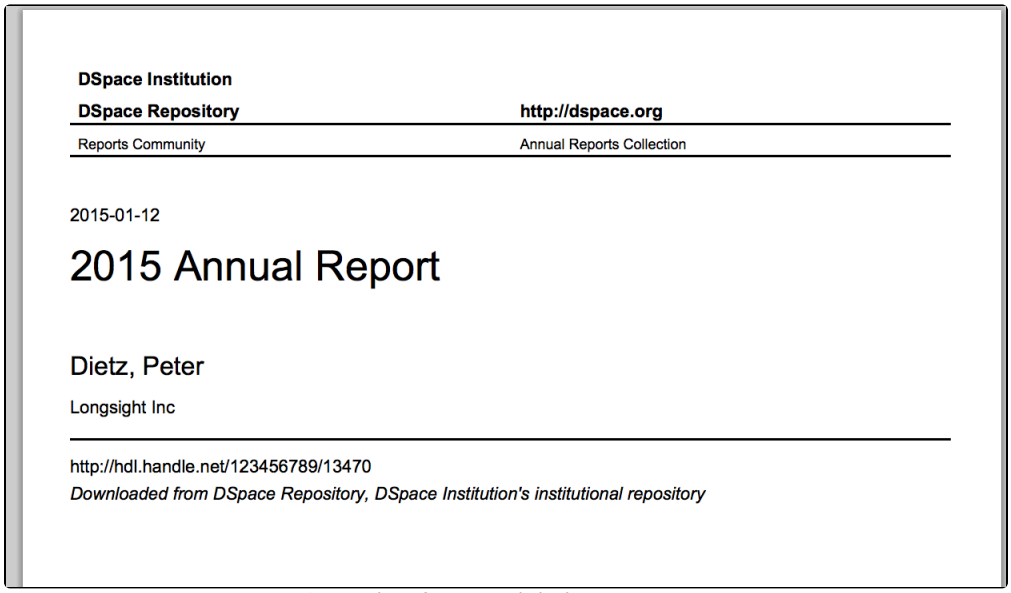
Enabling PDF Cover Pages may affect your site's visibility in Google Scholar (and similar search engines)

Google Scholar specifically warns against automatically generating PDF Cover Pages, as they can break the metadata extraction techniques used by their search engine. Be aware that enabling PDF Cover Pages may also cause those items to no longer be indexed by Google Scholar. For more information, please see the "Indexing Repositories: Pitfalls and Best Practices" talk from Anurag Acharya (co-creator of Google Scholar) presented at the Open Repositories 2015 conference.

A known issue with the current implementation of the PDF Citation Cover Page is that primarily only English/Roman characters are supported. This is due to a limitation in the tool used to generate PDFs. See https://github.com/DSpace/DSpace/issues/5590 for more details on this issue

Adding a cover page to retrieved documents from DSpace that include additional citation information has been sought, as documents uploaded to the repository might have had their context stripped from them, when they are just a PDF. Context that might have surrounded the document would be the journal, publisher, edition, and more. Without that information, the document might just be a few pages of text, with no way to piece it together. Since repository policy might be to include this information as metadata to the Item, this metadata can be added to the citation cover page, so that the derivative PDF includes all of this information.

The citation cover page works by only storing the original PDF in DSpace, and then generating the citation-cover-page PDF on-the-fly. An alternative set up would be to run the PDF Citation Coverpage Curation Task on the DSpace repository contents, and then disseminate the pre-generated citation-version instead of generating it on the fly.



Screenshot of generated citation cover page

Configuration settings for Citation Cover Page

Configuration file renamed to citation-page.cfg and configurations names have changed

As of DSpace 6.0, the configuration file for this feature was **renamed** from `disseminate-citation.cfg` to `citation-page.cfg`. The renaming was to clarify the purpose of this configuration file, as its previous name was misleading / confusing to some users.

In addition, all configurations below have now been prefixed with "citation-page" (e.g. the `enable_globally` configuration has been renamed to `citation-page.enable_globally`)

In the `{dspace.dir}/config/modules/citation-page.cfg` file review the following fields to make sure they are uncommented:

Property:	**citation-page.enable_globally**
Example Values:	citation-page.enable_globally = true
Informational Note:	Boolean to determine is citation-functionality is enabled globally for entire site. This will enable the citation cover page generator for all PDFs. Default: disabled
Property:	**citation-page.enabled_collections**
Example Values:	citation-page.enabled_collections = 1811/123, 1811/234

Informational Note:	List of collection handles to enable the cover page generator for bitstreams within. Default: blank
Property:	**citation-page.enabled_communities**
Example Values:	citation-page.enabled_communities = 1811/222, 1811/333
Informational Note:	List of community handles to enable the cover page generator for bitstreams within. Default: blank
Property:	**citation-page.citation_as_first_page**
Example Values:	citation-page.citation_as_first_page = true
Informational Note:	Should the citation page be the first page cover (true), or the last page (false). Default: true, (first page)
Property:	**citation-page.header1**
Example Values:	citation-page.header1 = University of Higher Education
Informational Note:	First row of header, perhaps for institution / university name. Commas separate multiple sections of the header (see screenshot above) Default Value: DSpace Institution
Property:	**citation-page.header2**
Example Values:	citation-page.header2 = Scholar Archive\, http://archive.example.com
Informational Note:	Second row of header, perhaps put your DSpace instance branded name, and url to your DSpace. A comma is used to separate instance name, and the URL Default Value: DSpace Repository, http://dspace.org
Property:	**citation-page.fields**
Example Values:	citation-page.fields = dc.date.issued, dc.title, dc.creator, dc.contributor.author, dc.publisher, _line_, dc.identifier.citation, dc.identifier.uri
Informational Note:	Metadata fields to display on the citation PDF. Specify in schema.element.qualifier form, and separate fields by a comma. If you want to have a horizontal line break, use _line_ Default Value: dc.date.issued,dc.title,dc.creator,dc.contributor.author,dc.publisher,_line_,dc.identifier.citation,dc.identifier.uri
Property:	**citation-page.footer**
Example Values:	citation-page.footer = Downloaded from Scholar Archive at University of Higher Education\, an open access institutional repository. All Rights Reserved.
Informational Note:	Footer text at the bottom of the citation page. It might be some type of license or copyright information, or just letting the recipient know where they downloaded the file from. Default Value: Downloaded from DSpace Repository\, DSpace Institution's institutional repository NOTE: any commas appearing in this text should be escaped as "\,". See example above.

Updating Items via Simple Archive Format

1 Item Update Tool
1.1 DSpace Simple Archive Format
1.2 ItemUpdate Commands
1.2.1 CLI Examples

Item Update Tool

ItemUpdate is a batch-mode command-line tool for altering the metadata and bitstream content of existing items in a DSpace instance. It is a companion tool to ItemImport and uses the DSpace simple archive format to specify changes in metadata and bitstream contents. Those familiar with generating the source trees for ItemImport will find a similar environment in the use of this batch processing tool.

For metadata, ItemUpdate can perform 'add' and 'delete' actions on specified metadata elements. For bitstreams, 'add' and 'delete' are similarly available. All these actions can be combined in a single batch run.

ItemUpdate supports an undo feature for all actions except bitstream deletion. There is also a test mode, as with ItemImport. However, unlike ItemImport, there is no resume feature for incomplete processing. There is more extensive logging with a summary statement at the end with counts of successful and unsuccessful items processed.

One probable scenario for using this tool is where there is an external primary data source for which the DSpace instance is a secondary or down-stream system. Metadata and/or bitstream content changes in the primary system can be exported to the simple archive format to be used by ItemUpdate to synchronize the changes.

A note on terminology: **item** refers to a DSpace item. **metadata element** refers generally to a qualified or unqualified element in a schema in the form `[schema].[element].[qualifier]` or `[schema].[element]` and occasionally in a more specific way to the second part of that form. **metadata field** refers to a specific instance pairing a metadata element to a value.

DSpace Simple Archive Format

As with ItemImporter, the idea behind the DSpace's simple archive format is to create an archive directory with a subdirectory per item. There are a few additional features added to this format specifically for ItemUpdate. Note that in the simple archive format, the item directories are merely local references and only used by ItemUpdate in the log output.

The user is referred to the previous section DSpace Simple Archive Format.

Additionally, the use of a **delete_contents** is now available. This file lists the bitstreams to be deleted, one bitstream ID per line. Currently, no other identifiers for bitstreams are usable for this function. This file is an addition to the Archive format specifically for ItemUpdate.

The optional suppress_undo file is a flag to indicate that the 'undo archive' should not be written to disk. This file is usually written by the application in an undo archive to prevent a recursive undo. This file is an addition to the Archive format specifically for ItemUpdate.

ItemUpdate Commands

Command used:	`[dspace]/bin/dspace itemupdate`
Java class:	*org.dspace.app.itemupdate.ItemUpdate*
Arguments short and (long) forms:	Description
`-a` or `--addmetadata [metadata element]`	Repeatable for multiple elements. The metadata element should be in the form dc.x or dc.x.y. The mandatory argument indicates the metadata fields in the dublin_core.xml file to be added unless already present (multiple fields should be separated by a semicolon ';'). However, duplicate fields will not be added to the item metadata without warning or error.
`-d` or `--deletemetadata [metadata element]`	Repeatable for multiple elements. All metadata fields matching the element will be deleted.
`-A` or `--addbitstreams`	Adds bitstreams listed in the contents file with the bitstream metadata cited there.

`-D` or `--deletebitstreams [filter plug classname or alias]`	Not repeatable. With no argument, this operation deletes bitstreams listed in the `deletes_contents` file. Only bitstream IDs are recognized identifiers for this operation. The optional filter argument is the classname of an implementation of `org.dspace.app.itemdupate.BitstreamFilter` class to identify files for deletion or one of the aliases (e.g. ORIGINAL, ORIGINAL_AND_DERIVATIVES, TEXT, THUMBNAIL) which reference existing filters based on membership in a bundle of that name. In this case, the `delete_contents` file is not required for any item. The filter properties file will contains properties pertinent to the particular filer used. Multiple filters are not allowed.
`-h` or `--help`	Displays brief command line help.
`-e` or `--eperson`	Email address of the person or the user's database ID **(Required)**
`-s` or `--source`	Directory archive to process **(Required)**
`-i` or `--itemfield`	Specifies the metadata field that contains the item's identifier; Default value is "dc.identifier.uri" (Optional)
`-t` or `--test`	Runs the process in test mode with logging. But no changes applied to the DSpace instance. (Optional)
`-P` or `--provenance`	Prevents any changes to the provenance field to represent changes in the bitstream content resulting from an Add or Delete. In other words, when this flag is specified, no new provenance information is added to the DSpace Item when adding/deleting a bitstream. No provenance statements are written for thumbnails or text derivative bitstreams, in keeping with the practice of MediaFilterManager. (Optional)
`-F` or `--filter-properties`	The filter properties files to be used by the delete bitstreams action (Optional)
`-v` or `--verbose`	Turn on verbose logging.

CLI Examples

Adding Metadata:

```
[dspace]/bin/dspace itemupdate -e joe@user.com -s [path/to/archive] -a dc.description
```

This will update all DSpace Items listed in your archive directory, adding a new `dc.description` metadata field. Items will be located in DSpace based on the handle found in 'dc.identifier.uri' (since the `-i` argument wasn't used, the default metadata field, dc.identifier.uri, from the dublin_core.xml file in the archive folder, is used).

Managing Community Hierarchy

1 Sub-Community Management

Sub-Community Management

Reindex content for new permissions to take effect

After moving or changing an existing Community hierarchy, it is important to reindex your content. Moving a Community under a new parent may result in the inheritance of new/different permissions from that new parent Community. These new permissions will not take effect until you reindex your content. Keep in mind, you may not need to reindex all content, but may be able to simply reindex the content under the new parent Community.

```
./dspace index-discovery -i [new-parent-uuid]
```

DSpace provides an administrative tool, 'CommunityFiliator', for managing community sub-structure. It has two operations, either establishing a community to sub-community relationship, or dis-establishing an existing relationship.

The familiar parent/child metaphor can be used to explain how it works. Every community in DSpace can be either a 'parent' community, meaning it has at least one sub-community, or a 'child' community, meaning it is a sub-community of another community, or both or neither. In these terms, an 'orphan' is a community that lacks a parent (although it can be a parent); 'orphans' are referred to as 'top-level' communities in the DSpace user-interface, since there is no parent community 'above' them. The first operation, establishing a parent/child relationship - can take place between any community and an orphan. The second operation - removing a parent/child relationship, will make the child an orphan.

Command used:	`[dspace]/bin/dspace community-filiator`
Java class:	*org.dspace.administer.CommunityFiliator*
Arguments short and (long) forms:	Description
`-s` or `--set`	Set a parent/child relationship
`-r` or `--remove`	Remove a parent/child relationship
`-c` or `--child`	Child community (Handle or database ID)
`-p` or `--parent`	Parent community (Handle or database ID
`-h` or `--help`	Online help.

Set a parent/child relationship, issue the following at the CLI:

```
[dspace]/bin/dspace community-filiator --set --parent=parentID --child=childID
```

(or using the short form)

```
[dspace]/bin/dspace community-filiator -s -p parentID -c childID
```

where '-s' or '-set' means establish a relationship whereby the community identified by the '-p' parameter becomes the parent of the community identified by the '-c' parameter. Both the 'parentID' and 'childID' values may be handles or database IDs.

The reverse operation looks like this:

```
[dspace]/bin/dspace community-filiator --remove --parent=parentID --child=childID
```

(or using the short form)

```
[dspace]/bin/dspace community-filiator -r -p parentID -c childID
```

where '-r' or '-remove' means dis-establish the current relationship in which the community identified by 'parentID' is the parent of the community identified by 'childID'. The outcome will be that the 'childID' community will become an orphan, i.e. a top-level community.

If the required constraints of operation are violated, an error message will appear explaining the problem, and no change will be made. An example in a removal operation, where the stated child community does not have the stated parent community as its parent: "Error, child community not a child of parent community".

It is possible to effect arbitrary changes to the community hierarchy by chaining the basic operations together. For example, to move a child community from one parent to another, simply perform a 'remove' from its current parent (which will leave it an orphan), followed by a 'set' to its new parent.

It is important to understand that when any operation is performed, all the sub-structure of the child community follows it. Thus, if a child has itself children (sub-communities), or collections, they will all move with it to its new 'location' in the community tree.

ORCID Integration

Since DSpace 7.3 a bidirectional ORCID integration is available for DSpace. This feature allows for authentication via ORCID, as well as synchronizing data between ORCID and DSpace, via the usage of Researcher Profiles.

Acknowledgments

The ORCID integration was originally developed by 4Science in DSpace-CRIS. It is the result of years of collaboration with several institutions and the ORCID team that has helped to correct, improve and broaden the scope of the integration. We want to thank the University of Hong Kong that was the first institution to fund development activities in this regard back in 2015 and the TUHH Hamburg University of Technology that have funded the initial porting of the ORCID integration to the new Angular/REST architecture introduced in DSpace 7. Last but not least, funds have been received by the DSpace community to port this feature from DSpace-CRIS to DSpace.

- Overview
- User features
  - Login via ORCID
  - Connect/Disconnect the local profile to ORCID
  - Pushing data to ORCID
  - Import publications from ORCID
  - Import a person from ORCID
  - Displaying ORCID iDs on work landing pages
- Configuration
  - Enable the integration
  - Configure the push of information from DSpace to ORCID
    - Mapping of the DSpace Person Items to ORCID Works
    - Mapping of DSpace Publication items to ORCID Works
    - Mapping of DSpace Project items to ORCID Funding
  - Configure the import features
  - Configure the author lookup in submission
- Troubleshooting & common issues
  - I'm having trouble testing the ORCID integration. What should I check?
  - I cannot find the ORCID features described by this page in my installation
  - I'm unable to authenticate via ORCID
  - After logging in via ORCID, a new DSpace account was created instead of using my existing DSpace account
  - I'm having trouble creating test accounts on ORCID to experiment with the features
  - I have configured my Public ORCID API credentials in DSpace but I get an error attempting to login via ORCID
  - I don't find my publications looking up for my ORCID iD
  - I cannot push all my publications, only few or none of them are listed in the queue
  - Push of publications to ORCID fails
  - Push of projects to ORCID fails

Overview

DSpace provides a bidirectional integration with ORCID based on the ORCID API v3.0. Both the Public ORCID API and the Membership API are supported.

The table below summarizes the supported features according to the type of ORCID API configured.

Feature	No credentials*	Public API	Member API
Authentication		✓	✓
Connect local profile to ORCID (authenticated ID)		✓	✓
ORCID Registry Lookup - import Person records	✓	✓	✓
ORCID Registry Lookup - as authority	✓	✓	✓
Import publication from ORCID	✓	✓	✓
Push biographic data to ORCID			✓
Push publications to ORCID (works)			✓
Push projects to ORCID (fundings)			✓

*** No credentials:** please note that ORCID strongly recommends to apply at least for a free public API Key as this will help to trouble-shoot integration problems and get support from ORCID. There is also a chance to get better performance/priority over "unknown" client.

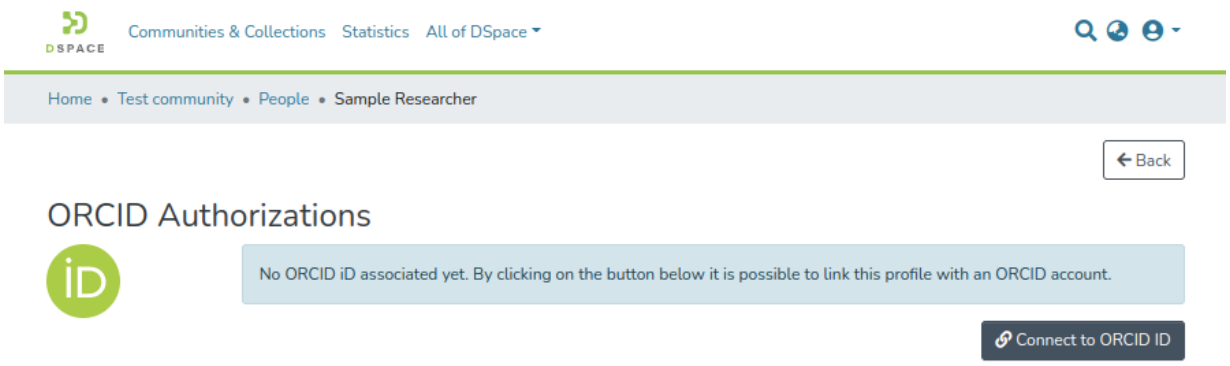
User features

Login via ORCID

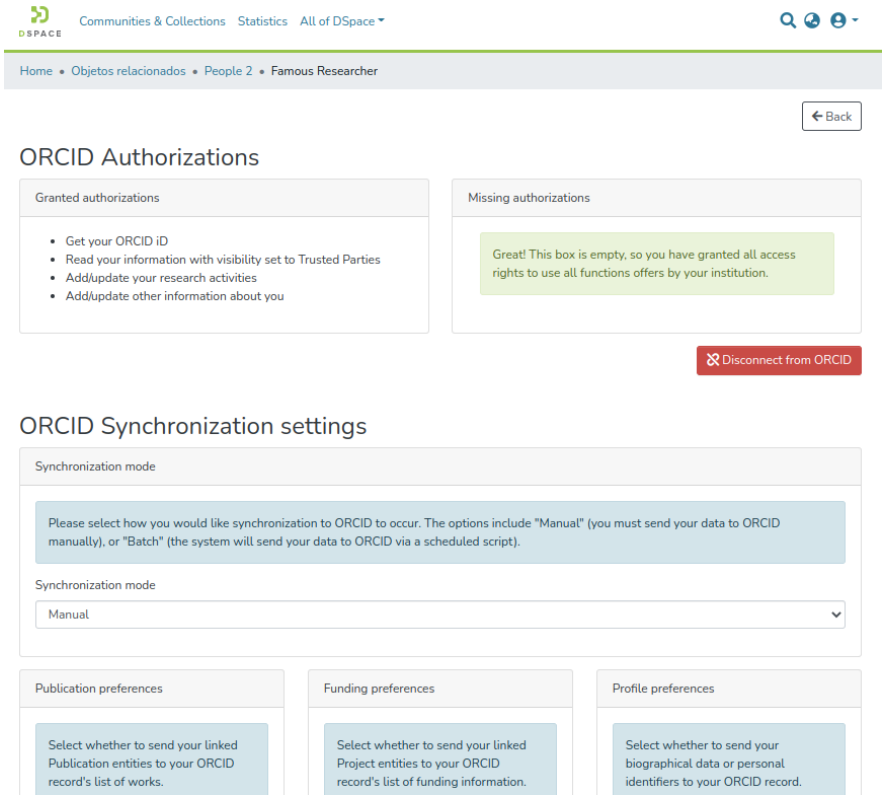
Once enabled, an option to login via ORCID is provided to the user among the other authentication methods configured in the system. When a user chooses to login to DSpace using ORCID, if the user's email address is set to "private" visibility in their ORCID record, they will get an error unless the ORCID Login workflow has been configured correctly with code that is not part of the default DSpace 7.x codebase but can be used as needed in local DSpace 7.x implementations: see ORCID Login flow for private emails and the ORCID Login: Guide to Repository access and Account linking for more information. The ORCID authentication doesn't allow the user to reset their password from DSpace.

Connect/Disconnect the local profile to ORCID

The researcher can connect (or disconnect) their DSpace local Researcher Profile with ORCID from the Person item detail page.



If a user clicks the "Disconnect from ORCID" button, their ORCID access token will be revoked and the DSpace repository will no longer be a "trusted party" on the user's ORCID account, according to ORCID's best practices.

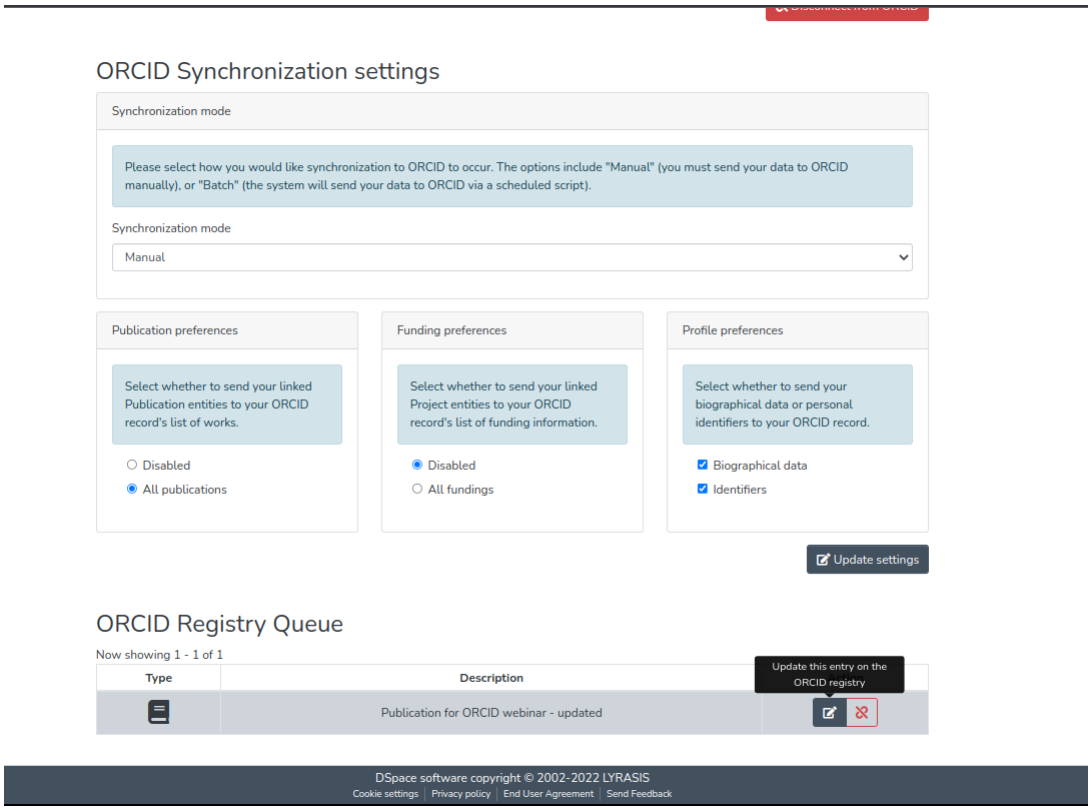


Pushing data to ORCID

Once a profile has been connected they can manage their synchronization preferences deciding what should be pushed to ORCID, including:

- biographic data
- Publication (entities) linked with their Researcher Profile. (Publication entities are synced to Works in ORCID.)
- Project (entities) linked with their Researcher Profile. (Project entities are synced to Fundings in ORCID.)
- Code for pushing Product and Patent information to ORCID is available for organizations who wish to use it; see: https://github.com/DSpace/DSpace/pull/9850

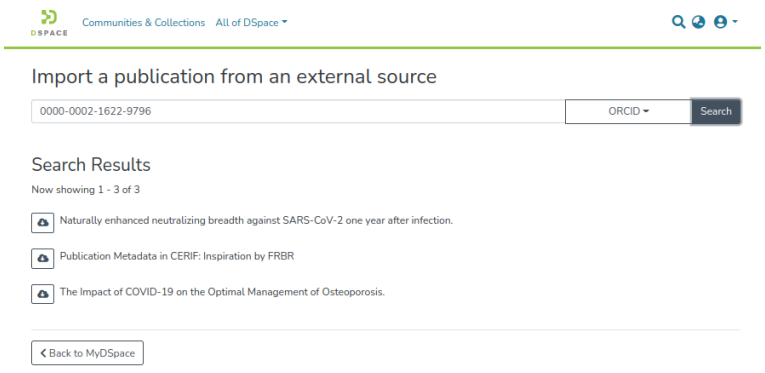
NOTE: The ORCID synchronization feature is disabled by default, even when ORCID Authentication is enabled. See Configuration section below for how to enable it.



The synchronization can happen automatically over the night or manually. The list of information that should be pushed or updated from DSpace to ORCID is presented in a queue and can be manually discarded or immediately pushed by the researcher.

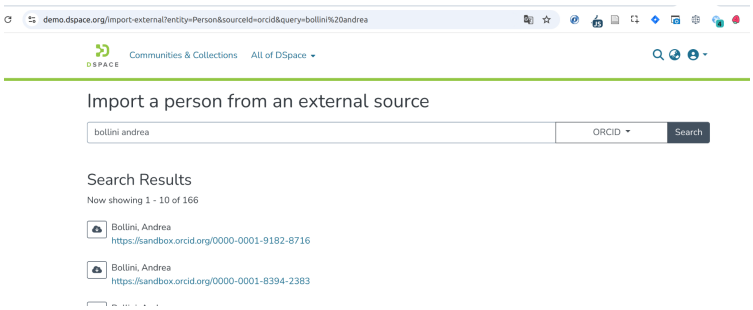
Import publications from ORCID

It is possible to import a publication from ORCID using the "Import from external sources" button in the home page. Once you select the Publication entity type you will be able to find ORCID as a Source and you can get the list of publications (ORCID works) that appear in an ORCID profile by searching for its ORCID iD.



Import a person from ORCID

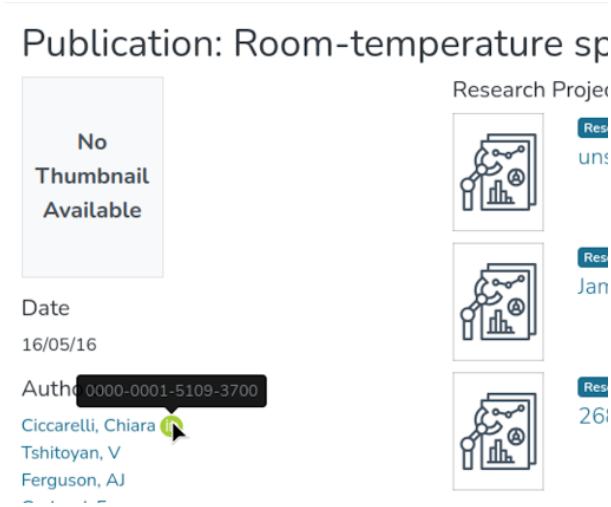
It is possible to import a person from ORCID using the "Import from external sources" button in the MyDSpace page. Once you select the Person entity type you will be able to find ORCID as a Source and you can get the list of person matching the provided query terms



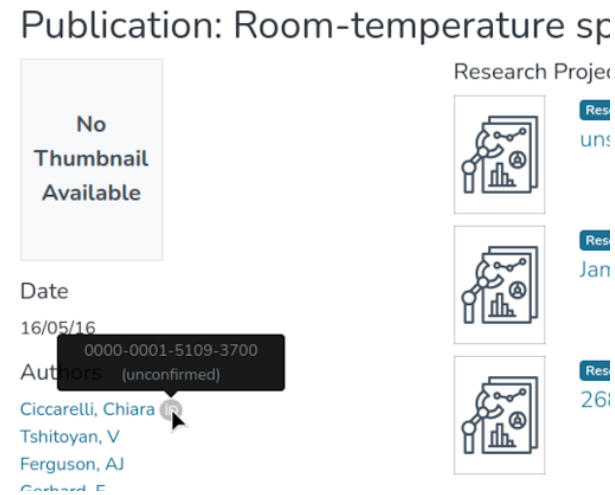
the person import from ORCID is more useful and common to be used in the context of the management of relationship among research outputs and activities (publications, projects, etc) and person, see the Configure the author lookup in submission

Displaying ORCID iDs on work landing pages

For ORCID iDs that have been associated with a user profile (Person Entity), either by a user connecting their authenticated ORCID iD with their profile, or an administrator adding an unauthenticated ORCID iD to a Person Entity profile: the ORCID iD icon will appear next to the user's name on any works landing pages where the user is an author, with the user's ORCID iD displayed upon hovering over the ORCID iD icon. Authenticated ORCID iDs will appear as a green ORCID iD icon:



Unauthenticated ORCID iDs will appear as a grey icon with the word "unconfirmed" in parentheses:



Configuration

Enable the integration

All the ORCID features requires a minimal common set of properties to configure in the `local.cfg`

```
# These URLs are for testing against ORCID's Sandbox API
# These are only useful for testing, and you must first request a Sandbox API Key from ORCID
orcid.domain-url= https://sandbox.orcid.org
orcid.api-url = https://api.sandbox.orcid.org/v3.0
orcid.public-url = https://pub.sandbox.orcid.org/v3.0
# Keep in mind, these API keys MUST be for the Sandbox API if you use "sandbox.orcid.org" URLs above!
orcid.application-client-id = <YOUR-SANDBOX-ORCID-CLIENT-ID>
orcid.application-client-secret = <YOUR-SANDBOX-ORCID-CLIENT-SECRET>

# Once you are ready to switch to Production, you need to update these settings to use ORCID's production API
# See https://github.com/ORCID/ORCID-Source/tree/master/orcid-api-web#endpoints
# orcid.domain-url= https://orcid.org
# orcid.api-url = https://api.orcid.org/v3.0
# orcid.public-url = https://pub.orcid.org/v3.0
# DON'T FORGET TO UPDATE YOUR API KEY! It must be a valid Public or Member API Key
# orcid.application-client-id = <YOUR-PRODUCTION-ORCID-CLIENT-ID>
# orcid.application-client-secret = <YOUR-PRODUCTION-ORCID-CLIENT-SECRET>
```

- **Enable in Production:** To enable the main integration (i.e. connect a local profile with ORCID and push data to the ORCID registry) you MUST to be an ORCID Member, get a Member API Key and properly enable and configure the feature in DSpace. See also "How do I register for Member API credentials?" from ORCID.
- **Enable in Testing:** To test ORCID integration, it's possible to use the ORCID Sandbox (without being an ORCID member). However, to do so, you must request a Sandbox Member API Key. See also "How do I register a public api client?".
- **Setting the "redirect URL" in ORCID:** In the ORCID API Credentials request form you will be asked to enter one or more redirect URLs for your application (DSpace). You will need to enter here the root URLs of your REST and user interfaces, which could be different. If the root URLs of both are the same, then just enter the URL of your user interface.
    - For example, for the DSpace 7 official demo, we use these redirect URLs:
        - User Interface: https://demo7.dspace.org (please note the absence of a /home or any subpaths)
        - REST API: https://api7.dspace.org (please note the absence of a /server or any subpaths)
    - For more information on valid ORCID redirect URLs, see "How do redirect URIs work?" from ORCID.
- **Configure the Client ID and Client Secret in DSpace:** Once ORCID has reviewed and approved your request, you will get from them the Client ID and Client Secret that need to be set in the `local.cfg` among other properties See the configuration examples above.

Please note that by default DSpace will request permissions to READ and WRITE all the information from the ORCID profile, as this will enable support for all of the features. You can fine-tune that by overriding the following properties. Please note that if you are going to configure Public API Credentials you MUST update this configuration keeping only the `/authenticate` scope as all the other scopes require Member API.

```
# The scopes to be granted by the user during the login on ORCID (see https://info.orcid.org/faq/what-is-an-
oauth-scope-and-which-scopes-does-orcid-support/)
orcid.scope = /authenticate
# The below scopes are ONLY valid if you have a Member API Key. They should be commented out if you only have a
Public API Key
orcid.scope = /read-limited
orcid.scope = /activities/update
orcid.scope = /person/update
```

To enable **ORCID Authentication** you need to uncomment the following line in the `modules/authentication.cfg` file or add it to your `local.cfg`

```
plugin.sequence.org.dspace.authenticate.AuthenticationMethod = org.dspace.authenticate.OrcidAuthentication
```

Please note that you are NOT required to enable the ORCID Authentication to use the other ORCID features, including the synchronisation ones. It is also possible to use just the ORCID Authentication without enabling all the other features.

When a user loggs in via ORCID the system will attempt to reuse an existing account looking up by email. If none is found then a new account is created in DSpace. It is possible to disable the creation of new accounts by setting the following property:

```
authentication-orcid.can-self-register = false
```

To enable **ORCID Synchronization**, you need to uncomment the following or add it to your `local.cfg`

```
# the properties below are required only for the sync / linking part (not for authentication or import)
orcid.synchronization-enabled = true
# you need to enable the orcidqueue consumer to keep track of what need to be sync between DSpace and ORCID
event.dispatcher.default.consumers = versioning, discovery, eperson, orcidqueue
```

The push of DSpace data (Person, Publication, Project) to ORCID is based on mappings defined in the `config/modules/orcid.cfg` file. You will find details below in the dedicated paragraphs.

The ORCID Synchronization features depend on other features that must be enabled: DSpace User Profile, Configurable Entities at least Person, Publication, Project & OrgUnit.

The synchronization features are classified as *experimental* at the time of 7.3 and it **MUST** be enabled manually. Due to the strict validation rules applied on the ORCID side and the absence of friendly edit UI for the archived items in DSpace (see issues#2876), it is hard at this time to achieve an optimal UX.

Configure the push of information from DSpace to ORCID

Please note that many fields on the ORCID side are subject to validation, i.e. only values from controlled-list can be used and some fields are mandatory. The table below summarizes the validations that are defined at the time of DSpace 7.3. ORCID updates such rules periodically, usually modifying (enlarging) the controlled-list, and changes to the mandatory fields can also happen.

ORCID Entity	Corresponding DSpace Entity	Mandatory fields	Controlled fields
Person	Person		Country (iso-3166 2 code letter)
Work	Publication	Title Type Publication Date (>= 1900) External Identifier (at least one)	Type (https://info.orcid.org/documentation/integration-and-api-faq/#easy-faq-2682) Identifier.Type (https://pub.orcid.org/v3.0/identifiers)
Funding	Project	Title External Identifier (at least one) Funding Agency (Organisation) Currency if an Amount is provided	Amount.Currency (https://www.iso.org/iso-4217-currency-codes.html)
Organisation	OrgUnit	External Identifier (at least one) Address City Country	See https://support.orcid.org/hc/en-us/articles/360006894674-Metadata-in-the-Funding-section Country (iso-3166 2 code letter) Identifier.Type (ROR, LEI, CrossRef Funder ID, RINGOLD, GRID)

To provide more meaningful messages to the user DSpace implements a local validation before trying to push the record to ORCID. This validation verifies the data using the rules above so that a specific message is displayed to the user. If for any reason another error is returned by ORCID a generic message is shown to the user and the exact technical message received by ORCID is logged in the `dspace.log` file and stored in the `orcidhistory` table.

The local validation can be turned off. This validation is mainly intended as a development/debug option and should be not enabled in production.

```
orcid.validation.work.enabled = true
orcid.validation.funding.enabled = true
```

Mapping of the DSpace Person Items to ORCID Works

Metadata in a DSpace "Person" item, once that the item has been linked to ORCID by the researcher owning the ORCID profile, can be pushed to ORCID to fill the Profile Section of the ORCID Person using the `org.dspace.orcid.service.OrcidProfileSectionFactoryService` configured via this mapping bean:

```
    <!-- Configuration of ORCID profile sections factory.
    Each entry of sectionFactories must be an implementation of OrcidProfileSectionFactory.-->
    <bean class="org.dspace.orcid.service.impl.OrcidProfileSectionFactoryServiceImpl">
            <constructor-arg name="sectionFactories">
                    <list>

                            <bean class="org.dspace.orcid.model.factory.impl.OrcidSimpleValueObjectFactory">
                                    <constructor-arg name="sectionType" value="OTHER_NAMES" />
                                    <constructor-arg name="preference" value="BIOGRAPHICAL" />
                                    <property name="metadataFields" value="${orcid.mapping.other-names}" />
                                </bean>

                            <bean class="org.dspace.orcid.model.factory.impl.OrcidSimpleValueObjectFactory">
                                    <constructor-arg name="sectionType" value="KEYWORDS" />
                                    <constructor-arg name="preference" value="BIOGRAPHICAL" />
                                    <property name="metadataFields" value="${orcid.mapping.keywords}" />
                                </bean>

                            <bean class="org.dspace.orcid.model.factory.impl.OrcidSimpleValueObjectFactory">
                                    <constructor-arg name="sectionType" value="COUNTRY" />
                                    <constructor-arg name="preference" value="BIOGRAPHICAL" />
                                    <property name="metadataFields" value="${orcid.mapping.country}" />
                                </bean>

                            <bean class="org.dspace.orcid.model.factory.impl.
OrcidPersonExternalIdentifierFactory">
                                    <constructor-arg name="sectionType" value="EXTERNAL_IDS" />
                                    <constructor-arg name="preference" value="IDENTIFIERS" />
                                    <property name="externalIds" value="${orcid.mapping.person-external-
ids}" />
                                </bean>

                            <bean class="org.dspace.orcid.model.factory.impl.OrcidSimpleValueObjectFactory">
                                    <constructor-arg name="sectionType" value="RESEARCHER_URLS" />
                                    <constructor-arg name="preference" value="IDENTIFIERS" />
                                    <property name="metadataFields" value="${orcid.mapping.researcher-
urls}" />
                                </bean>

                    </list>
            </constructor-arg>
    </bean>
```

The above configuration links each piece of information that can be synchronized from DSpace to ORCID with a preference that the user can manage on the DSpace side (i.e. sync of the keywords is linked to the BIOGRAPHICAL preference) and defines which DSpace metadata will be used to fill the ORCID field. The bean reads the metadata mapping from the `config/modules/orcid.cfg`

```
### Other names mapping ###
orcid.mapping.other-names = person.name.variant
orcid.mapping.other-names = person.name.translated

### Keywords mapping ###
orcid.mapping.keywords = dc.subject

### Country mapping ###
orcid.mapping.country = person.country
orcid.mapping.country.converter =

### Person External ids mapping ###
##orcid.mapping.person-external-ids syntax is <metadatafield>::<type>
orcid.mapping.person-external-ids = person.identifier.scopus-author-id::SCOPUS
orcid.mapping.person-external-ids = person.identifier.rid::RID

### Researcher urls mapping ###
orcid.mapping.researcher-urls = dc.identifier.uri
```

Mapping of DSpace Publication items to ORCID Works

A DSpace "Publication" item is pushed to ORCID as a "Work" using the `org.dspace.orcid.model.factory.impl.OrcidWorkFactory` configured via this mapping bean:

```
        <bean id="orcidWorkFactoryFieldMapping" class="org.dspace.app.orcid.model.OrcidWorkFieldMapping" >
                <property name="contributorFields" value="${orcid.mapping.work.contributors}" />
                <property name="externalIdentifierFields" value="${orcid.mapping.work.external-ids}" />
                <property name="publicationDateField" value="${orcid.mapping.work.publication-date}" />
                <property name="titleField" value="${orcid.mapping.work.title}" />
                <property name="journalTitleField" value="${orcid.mapping.work.journal-title}" />
                <property name="shortDescriptionField" value="${orcid.mapping.work.short-description}" />
                <property name="subTitleField" value="${orcid.mapping.work.sub-title}" />
                <property name="languageField" value="${orcid.mapping.work.language}" />
                <property name="languageConverter" ref="${orcid.mapping.work.language.converter}" />
                <property name="typeField" value="${orcid.mapping.work.type}" />
                <property name="typeConverter" ref="${orcid.mapping.work.type.converter}" />
                <property name="citationType" value="${orcid.mapping.work.citation.type}" />
        </bean>
```

that reads the mapping from the `config/modules/orcid.cfg` file:

```
### Work (Publication) mapping ###
orcid.mapping.work.title = dc.title
orcid.mapping.work.sub-title =
orcid.mapping.work.short-description = dc.description.abstract
orcid.mapping.work.publication-date = dc.date.issued
orcid.mapping.work.language = dc.language.iso
orcid.mapping.work.language.converter = mapConverterDSpaceToOrcidLanguageCode
orcid.mapping.work.journal-title = dc.relation.ispartof
orcid.mapping.work.type = dc.type
orcid.mapping.work.type.converter = mapConverterDSpaceToOrcidPublicationType

##orcid.mapping.work.contributors syntax is <metadatafield>::<role>
orcid.mapping.work.contributors = dc.contributor.author::author
orcid.mapping.work.contributors = dc.contributor.editor::editor

##orcid.mapping.work.external-ids syntax is <metadatafield>::<type> or $simple-handle::<type>
##The full list of available external identifiers is available here https://pub.orcid.org/v3.0/identifiers
orcid.mapping.work.external-ids = dc.identifier.doi::doi
orcid.mapping.work.external-ids = dc.identifier.scopus::eid
orcid.mapping.work.external-ids = dc.identifier.pmid::pmid
orcid.mapping.work.external-ids = $simple-handle::handle
orcid.mapping.work.external-ids = dc.identifier.isi::wosuid
orcid.mapping.work.external-ids = dc.identifier.issn::issn
```

In the above configuration the "simple" properties are mapped matching the ORCID field name on the left (i.e. short-description) with the DSpace metadata that holds such information (i.e dc.description.abstract). For the ORCID Type field a special "converter" is configured so that the value of the DSpace metadata (i.e. dc.type) is mapped to the controlled-list of types accepted by ORCID (https://info.orcid.org/faq/what-work-types-does-orcid-support/). The value of the `orcid.mapping.work.type.converter` matches the name of a bean defined in the `config/spring/api/orcid-services.xml`

```
    <bean name="mapConverterDSpaceToOrcidPublicationType" class="org.dspace.util.SimpleMapConverter" init-
method="init">
                <property name="converterNameFile" value="orcid/mapConverter-dspace-to-orcid-publication-type.
properties" />
                <property name="configurationService" ref="org.dspace.services.ConfigurationService" />
                <property name="defaultValue" value="other"/>
        </bean>
```

Finally a special treatment is needed for the external ids as this is a complex field on the ORCID side composed of two values: the identifier type (from a controlled-list) and the identifier value. In this case the configuration maps a DSpace metadata field (i.e. dc.identifier.doi) to a specific identifier type (i.e. the part after :: , doi).

Mapping of DSpace Project items to ORCID Funding

A DSpace "Project" item is pushed to ORCID as a "Funding" using the `org.dspace.orcid.model.factory.impl.OrcidFundingFactory` configured via this mapping bean:

```
        <bean id="orcidFundingFactoryFieldMapping" class="org.dspace.orcid.model.OrcidFundingFieldMapping" >
                <property name="contributorFields" value="${orcid.mapping.funding.contributors}" />
                <property name="externalIdentifierFields" value="${orcid.mapping.funding.external-ids}" />
                <property name="titleField" value="${orcid.mapping.funding.title}" />
                <property name="typeField" value="${orcid.mapping.funding.type}" />
                <property name="typeConverter" ref="${orcid.mapping.funding.type.converter}" />
                <property name="amountField" value="${orcid.mapping.funding.amount}" />
                <property name="amountCurrencyField" value="${orcid.mapping.funding.amount.currency}" />
                <property name="amountCurrencyConverter" ref="${orcid.mapping.funding.amount.currency.
converter}" />
                <property name="descriptionField" value="${orcid.mapping.funding.description}" />
                <property name="startDateField" value="${orcid.mapping.funding.start-date}" />
                <property name="endDateField" value="${orcid.mapping.funding.end-date}" />
                <property name="organizationRelationshipType" value="${orcid.mapping.funding.organization-
relationship-type}" />
        </bean>
```

that reads the mapping from the `config/modules/orcid.cfg` file

```
### Funding mapping ###
orcid.mapping.funding.title = dc.title
orcid.mapping.funding.type =
orcid.mapping.funding.type.converter = mapConverterDSpaceToOrcidFundingType
##orcid.mapping.funding.external-ids syntax is <metadatafield>::<type>
##The full list of available external identifiers is available here https://pub.orcid.org/v3.0/identifiers
orcid.mapping.funding.external-ids = dc.identifier::grant_number
orcid.mapping.funding.external-ids = dc.identifier.other::other-id
orcid.mapping.funding.description = dc.description
orcid.mapping.funding.start-date = project.startDate
orcid.mapping.funding.end-date = project.endDate
##orcid.mapping.funding.contributors syntax is <metadatafield>::<type>
orcid.mapping.funding.contributors = project.investigator::lead
orcid.mapping.funding.organization-relationship-type = isOrgUnitOfProject
orcid.mapping.funding.amount = project.amount
orcid.mapping.funding.amount.currency = project.amount.currency
orcid.mapping.funding.amount.currency.converter = mapConverterDSpaceToOrcidAmountCurrency
```

in the above configuration the "simple" properties are mapped matching the ORCID field name on the left (i.e. description) with the DSpace metadatafield that holds such information (i.e dc.description). For the ORCID Type field a special "converter" is configured so that the value of the DSpace metadata (i.e. dc.type) is mapped to the controlled-list of types accepted by ORCID (https://support.orcid.org/hc/en-us/articles/360006894674-Metadata-in-the-Funding-section). The value of the `orcid.mapping.funding.type.converter` matches the name of a bean defined in the `config/spring/api/orcid-services.xml` The same apply for the currency `orcid.mapping.funding.amount.currency.converter = mapConverterDSpaceToOrcidAmountCurrency`

```
<bean name="mapConverterDSpaceToOrcidFundingType" class="org.dspace.util.SimpleMapConverter" init-method="init">
        <property name="converterNameFile" value="orcid/mapConverter-dspace-to-orcid-funding-type.properties" />
        <property name="configurationService" ref="org.dspace.services.ConfigurationService" />
        <property name="defaultValue" value=""/>
</bean>

<bean name="mapConverterDSpaceToOrcidAmountCurrency" class="org.dspace.util.SimpleMapConverter" init-method="
init">
        <property name="converterNameFile" value="orcid/mapConverter-dspace-to-orcid-amount-currency.
properties" />
        <property name="configurationService" ref="org.dspace.services.ConfigurationService" />
        <property name="defaultValue" value=""/>
</bean>
```

Finally a special treatment is needed for Funder that is a mandatory field on the ORCID side. In this case the mapping defines which relation is used to link the Project with the Funder (OrgUnit)

```
orcid.mapping.funding.organization-relationship-type = isOrgUnitOfProject
```

Configure the import features

The Import features from ORCID have been implemented using the Live Import Framework

The following bean is used to configure the **import of person records from ORCID**. It is activated as an external source in `config/spring/api/external-services.xml`

```
    <bean class="org.dspace.external.provider.impl.OrcidV3AuthorDataProvider" init-method="init">
        <property name="sourceIdentifier" value="orcid"/>
        <property name="orcidUrl" value="${orcid.domain-url}" />
        <property name="clientId" value="${orcid.application-client-id}" />
        <property name="clientSecret" value="${orcid.application-client-secret}" />
        <property name="OAUTHUrl" value="${orcid.token-url}" />
        <property name="orcidRestConnector" ref="orcidRestConnector"/>
        <property name="supportedEntityTypes">
            <list>
                <value>Person</value>
            </list>
        </property>
    </bean>
```

the mapping between ORCID Person and the DSpace Person Item is the following, currently hard-coded:

ORCID	DSpace
Name/FamilyName	person.firstName
Name/GivenName	person.givenName
Name/Path	person.identifier.orcid
ORCID Profile URL	dc.identifier.uri

The following bean is instead used to configure the **import of publication records** from ORCID (Work)

```
    <bean id="orcidPublicationDataProvider" class="org.dspace.external.provider.impl.
OrcidPublicationDataProvider">
        <property name="sourceIdentifier" value="orcidWorks"/>
        <property name="fieldMapping" ref="orcidPublicationDataProviderFieldMapping"/>
        <property name="supportedEntityTypes">
            <list>
                <value>Publication</value>
            </list>
        </property>
    </bean>
```

The mapping of the ORCID Work metadata to the DSpace metadata is performed by the following bean in `config/spring/api/orcid-services.xml`

```
        <bean id="orcidPublicationDataProviderFieldMapping" class="org.dspace.orcid.model.
OrcidWorkFieldMapping" >
                <property name="contributorFields" value="${orcid.external-data.mapping.publication.
contributors}" />
                <property name="externalIdentifierFields" value="${orcid.external-data.mapping.publication.
external-ids}" />
                <property name="publicationDateField" value="${orcid.external-data.mapping.publication.issued-
date}" />
                <property name="titleField" value="${orcid.external-data.mapping.publication.title}" />
                <property name="journalTitleField" value="${orcid.external-data.mapping.publication.is-part-
of}" />
                <property name="shortDescriptionField" value="${orcid.external-data.mapping.publication.
description}" />
                <property name="languageField" value="${orcid.external-data.mapping.publication.language}" />
                <property name="languageConverter" ref="${orcid.external-data.mapping.publication.language.
converter}" />
                <property name="typeField" value="${orcid.external-data.mapping.publication.type}" />
                <property name="typeConverter" ref="${orcid.external-data.mapping.publication.type.converter}"
/>
        </bean>
```

that reads the mapping from the `config/modules/orcid.cfg` file.

Please note that such mapping is separated from the mapping used to push information from DSpace to ORCID but usually, as provided in the default configuration, the mapping should be the same.

```
### Work (Publication) external-data.mapping ###
orcid.external-data.mapping.publication.title = dc.title

orcid.external-data.mapping.publication.description = dc.description.abstract
orcid.external-data.mapping.publication.issued-date = dc.date.issued
orcid.external-data.mapping.publication.language = dc.language.iso
orcid.external-data.mapping.publication.language.converter = mapConverterOrcidToDSpaceLanguageCode
orcid.external-data.mapping.publication.is-part-of = dc.relation.ispartof
orcid.external-data.mapping.publication.type = dc.type
orcid.external-data.mapping.publication.type.converter = mapConverterOrcidToDSpacePublicationType

##orcid.external-data.mapping.publication.contributors syntax is <metadatafield>::<role>
orcid.external-data.mapping.publication.contributors = dc.contributor.author::author
orcid.external-data.mapping.publication.contributors = dc.contributor.editor::editor

##orcid.external-data.mapping.publication.external-ids syntax is <metadatafield>::<type> or $simple-handle::
<type>
##The full list of available external identifiers is available here https://pub.orcid.org/v3.0/identifiers
orcid.external-data.mapping.publication.external-ids = dc.identifier.doi::doi
orcid.external-data.mapping.publication.external-ids = dc.identifier.scopus::eid
orcid.external-data.mapping.publication.external-ids = dc.identifier.pmid::pmid
orcid.external-data.mapping.publication.external-ids = dc.identifier.isi::wosuid
orcid.external-data.mapping.publication.external-ids = dc.identifier.issn::issn
```

Configure the author lookup in submission

Please note that there are two different possibilities:

- via an ORCID lookup authority available for a DSpace repository that is not using Configurable Entities
- via a relation among the research output item (Publication, etc.) and a Person Item bound to the ORCID Person External Source defined in the previous paragraph

Troubleshooting & common issues

The troubleshooting guide from ORCID can help as well https://info.orcid.org/documentation/integration-guide/troubleshooting/

I'm having trouble testing the ORCID integration. What should I check?

Please double check the documentation and the other FAQs to be sure that you have followed all of the instructions to enable the integration correctly. If you still have trouble, contact the DSpace tech community via email or slack providing as much detail as possible. If the issue is related to the synchronization of DSpace local data with ORCID it would be useful to share information about the content of your `orcidhistory` table and any relevant message that you could have in the `dspace.log` file.

I cannot find the ORCID features described by this page in my installation

The ORCID features must be enabled by changing some configuration files. Please refer to the ***Enable the integration section*** above.

I'm unable to authenticate via ORCID

If you have correctly enabled the ORCID authentication feature and you are able to start the OAuth flow with ORCID but get a failure when you are redirected back to DSpace, it could be due to privacy settings on your ORCID record. The DSpace ORCID authentication **requires that you release an email address to match your ORCID account with a DSpace account** or to create a new one at your first login. Make your ORCID account email address public or visible to trusted parties. This is often not the case for a freshly created account on ORCID.

If you are encountering this issue, you'll see a message like this in your "dspace.log" file on the backend:

```
2022-08-04 11:43:42,124 ERROR unknown unknown org.dspace.authenticate.OrcidAuthenticationBean @ An error occurs
registering a new EPerson from ORCID
java.lang.IllegalStateException: The email is configured private on orcid
```

After logging in via ORCID, a new DSpace account was created instead of using my existing DSpace account

Currently, the ORCID integration with DSpace relies on a matching email address to find your existing account. If your ORCID account and DSpace account have **different** email addresses associated with them, then it is possible that a new (duplicative) user account will be created.

I'm having trouble creating test accounts on ORCID to experiment with the features

Please refer to the ORCID trouble-shooting guide https://info.orcid.org/documentation/integration-guide/troubleshooting/ A frequent mistake working with the ORCID sandbox environment is to forget that only email addresses @mailinator.com are allowed for account created on the sandbox. Remember to validate your email address once the account as been created visiting the online inbox at mailinator.com

I have configured my Public ORCID API credentials in DSpace but I get an error attempting to login via ORCID

When you use public ORCID API credentials you can only use a subset of the integration features (check). Moreover you need to limit the scopes (permissions) requested to the user via the ORCID authentication to the `/authenticate` scope. Please check the ***Enable the integration section*** above.

I don't find my publications looking up for my ORCID iD

Please check "`config/modules/orcid.cfg`" (or your `local.cfg`) to see if the system has been properly configured to use the production ORCID API. There is a chance that your installation is still configured to use the ORCID Sandbox that is appropriate for the testing and development phase of the integration. The ORCID sandbox doesn't contain the same data as the Public environment.

I cannot push all my publications, only few or none of them are listed in the queue

Please double check that the `orcidqueue` consumer has been enabled (in dspace.cfg or local.cfg) and that the orcid settings of your profile have the "All publications" checkbox flagged. ORCID features require the use of the new Configurable Entities. Only Publication item are synchronized with ORCID; simple "untyped" Items will not be synchronized. Please consider to convert your legacy collection to "Publication" collection and set a `dspace.entity.type = Publication` metadata on your legacy items.

Push of publications to ORCID fails

This is usually due to validation errors. ORCID could complain about missing mandatory fields or invalid values for fields that are linked to a controlled-list. Please check the table in the ***Configure the push of information from DSpace to ORCID*** and the ***Mapping of DSpace Publication items to ORCID Works paragraph*** to solve this. Your dspace.log file may also provide useful error messages.

Push of projects to ORCID fails

This is usually due to validation errors. Make sure that all required metadata fields exist on the Project Entity and any linked OrgUnit Entities. ORCID could complain about missing mandatory fields or invalid values for fields that are linked to a controlled-list. Please check the table in the ***Configure the push of information from DSpace to ORCID*** and the ***Mapping of DSpace Project items to ORCID Funding paragraph*** to solve this. Your dspace.log file may also provide useful error messages.

Researcher Profiles

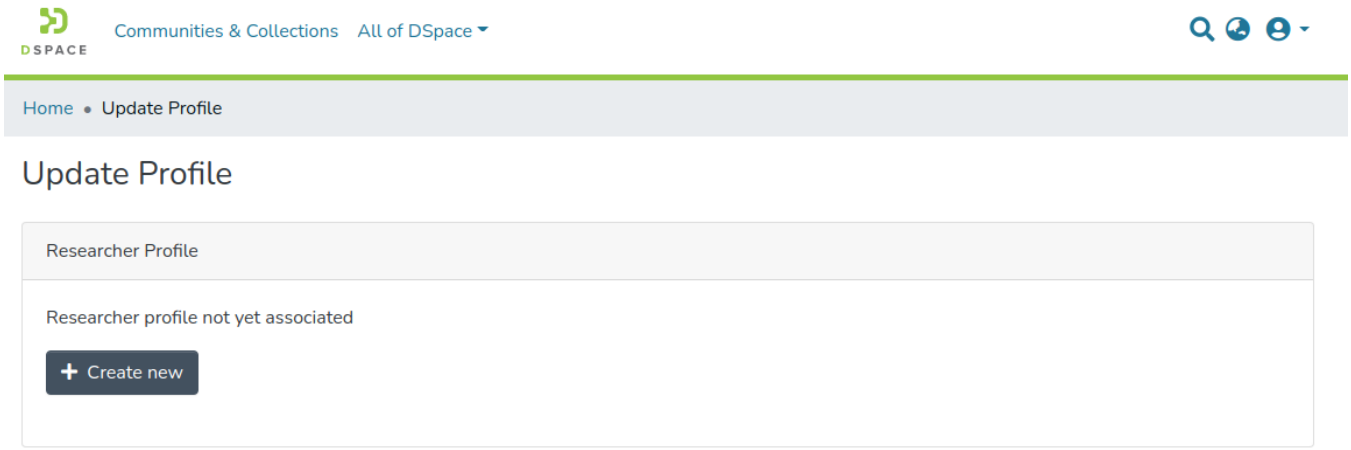
The DSpace Researcher Profile feature has been introduced in DSpace 7.3 to support the ORCID Integration work but can be used alone. It is turned off by default and must be enabled manually

A DSpace Researcher Profile is a special Person Entity (item) that is linked with exactly one EPerson (DSpace account). This linked EPerson owns the profile (Person Item), including having WRITE permission on it. The link between the Person Item and the EPerson is managed in the Person's `dspace.object.owner` metadata field. This field is configured to hold authority values and will contain the `UUID` of the EPerson that owns the profile.

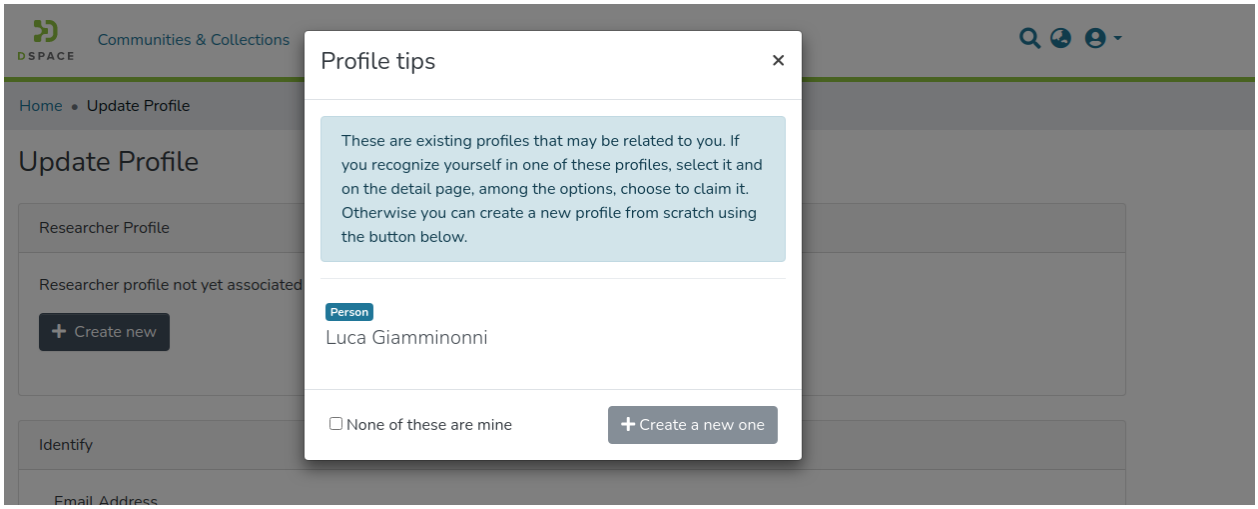
Here a summary of the key concepts & requirements of the feature:

- Profiles require Configurable Entities to be enabled, as every Researcher Profile is represented by a Person Entity.
- A profile can be linked to only one EPerson (user account). That EPerson has full rights to manage the profile, including whether the profile is publicly visible or private.
- Optionally, Profiles can be synchronized (or initially created) via ORCID Integration.

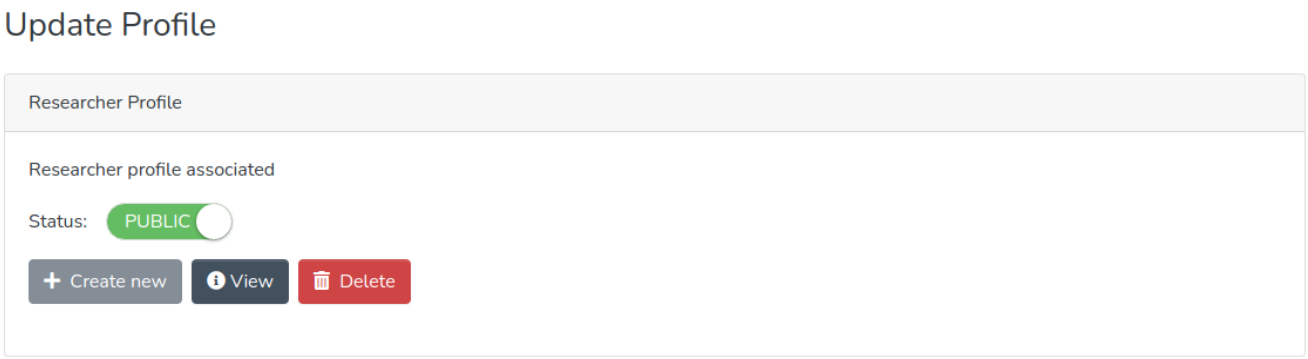
When the feature is enabled, the user can create a researcher profile from his Profile (account page)



If a Person Item already exists in the system, matching the account email address, this Person Item is offered to the user:



Once a profile has been created or claimed, the user can make it public (Anonymous READ) or private:



By default, deletion of the researcher profile does NOT delete the corresponding Person Item. Instead, it just unlinks the Person Item from the EPerson account. This behavior can be changed as specified in the ***Advanced configuration*** section below.

Enable the feature

To enable the feature you need to set the following property (uncommenting it in the `config/modules/researcher-profile.cfg` or adding it to the `config/local.cfg`)

```
researcher-profile.entity-type = Person
```

You can specify a different Entity Type for the item that can be used as profile. This is an advanced setting -- change it only if you know what are you doing and have implemented specific customisation.

You need to enable also the EPerson authority for the `dspace.object.owner`. Uncomment the following lines in the `config/modules/authority.cfg`:

```
# Configuration settings required for Researcher Profiles
# These settings ensure "dspace.object.owner" field are indexed by Authority Control
choices.plugin.dspace.object.owner = EPersonAuthority
choices.presentation.dspace.object.owner = suggest
authority.controlled.dspace.object.owner = true
```

Last, you need to ensure that at least one Collection is configured to accept Person entities. Only EPersons having the submission right in such a Collection will be able to create profiles. There are many possibilities for using these settings to control who may or may not create a profile.

Advanced configuration

You can configure some aspects of the Profile feature in the `config/modules/researcher-profile.cfg`

Property:	`researcher-profile.entity-type`
Example Value:	`Person`
Informational Note:	The type of Entity to use for Researcher Profile items. By default, the Person Entity is used, as this is provided out-of-the-box in DSpace. This would only need to be modified if you have created a heavily customized Configurable Entities data model which does NOT include Person.
Property:	`researcher-profile.collection.uuid`

Example Value:	[collection-uuid]
Informational Note:	UUID of the Collection where all Researcher Profiles should be created by default. This Collection MUST be configured to accept Person Entities (or the entity type specified in "researcher-profile.entity-type"). By default this is UNSPECIFIED. The default behavior is that the person's Researcher Profile will be created in the Collection in DSpace which is configured to accept Person Entities and where the user has permissions to submit. If multiple Collections of this type are available, then the first one found will be used.
Property:	`researcher-profile.hard-delete.enabled`
Example Value:	`false`
Informational note	Whether to enable "hard delete" when a Researcher Profile is deleted by an EPerson. When "hard delete" is enabled (set to true), then anytime an EPerson deletes their Researcher Profile, the underlying Person Entity will be deleted (i.e. this acts as a permanent deletion). When "hard delete" is disabled (set to false, the default value), then anytime an EPerson deletes their Researcher Profile, it will simply be "unlinked". In other words, the underlying Person Entity will be kept in the system.
Property:	`researcher-profile.set-new-profile-visible`
Example Value:	false
Informational note	Whether to make a new Researcher Profile "visible" (i.e. allow anonymous access) on creation. When set to "false" (default value), a newly created Researcher Profile will only be accessible to the EPerson who created it. That EPerson may chose to make to visible (i.e. allow anonymous access) at a later time. When set to "true", a newly created Researcher Profile will be immediately accessible to anonymous users. But, the EPerson who created it may chose to hide it (i.e. disallow anonymous access) at a later time.

Troubleshooting

I cannot find this feature

The feature needs to be enabled explicitly. Please follow the instruction in the ***Enable the feature*** section above.

The users sees an error when they try to create their profile

The feature requires that the Person entity be configured in the data model (see Configurable Entities) and the user must have permission to submit in at least one collection configured to accept Person entities. Please double check that the EPersonAuthority is bound to the `dspace.object.owner` metada ta -- see the ***Enable the feature*** section above.

Statistics and Metrics

- SOLR Statistics
- DSpace Google Analytics Statistics
- Exchange usage statistics with IRUS

SOLR Statistics

DSpace uses the Apache SOLR application underlying the statistics. SOLR enables performant searching and adding to vast amounts of (usage) data. Unlike previous versions, enabling statistics in DSpace does not require additional installation or customization. All the necessary software is included.

1 What is exactly being logged ?
- 1.1 Common stored fields for all usage events
- 1.2 Unique stored fields for bitstream downloads
- 1.3 Unique stored fields for search queries
- 1.4 Unique stored fields for workflow events

2 Disabling Tracking of Statistics
3 Web User Interface Elements
- 3.1 Pageview and Download statistics
  - 3.1.1 Home page
  - 3.1.2 Community home page
  - 3.1.3 Collection home page
  - 3.1.4 Item home page
- 3.2 Search Query Statistics
- 3.3 Workflow Event Statistics

4 Architecture
5 Configuration settings for Statistics
- 5.1 Pre-1.6 Statistics settings

6 Statistics Administration
- 6.1 Converting older DSpace logs into SOLR usage data
- 6.2 Statistics Client Utility
- 6.3 Anonymizing Statistics

7 Custom Reporting - Querying SOLR Directly
- 7.1 Resources
- 7.2 Examples
  - 7.2.1 Top downloaded items by a specific user

8 Managing the City Database File

What is exactly being logged ?

After the introduction of the SOLR Statistics logging, every pageview and file download is logged in a dedicated SOLR statistics core.

In addition to the already existing logging of pageviews and downloads, DSpace also logs search queries users enter in the DSpace search dialog and workflow events.

DSpace 7.x and 8.x does not yet support all features

In DSpace 7.x & 8.x, only usage statistics (pageview, downloads) are logged. Search statistics and workflow reports (which were available in 6.x and below) are not yet supported. See their related tickets: https://github.com/DSpace/DSpace/issues/2880 and https://github.com/DSpace/DSpace/issues/2851

Workflow Events logging

Only workflow events, initiated and executed by a physical user are being logged. Automated workflow steps or ingest procedures are currently **not** being logged by the workflow events logger.

The logging happens at the server side, and doesn't require a javascript like Google Analytics does, to provide usage data. Definition of which fields are to be stored happens in the file **dspace/solr/statistics/conf/schema.xml**.

Although they are stored in the same index, the stored fields for views, search queries and workflow events are different. A new field, statistics_type determines which kind of a usage event you are dealing with. The three possible values for this field are **view, search** and **workflow.**

```
<field name="statistics_type" type="string" indexed="true" stored="true" required="true" />
```

Common stored fields for all usage events

```
<field name="type" type="integer" indexed="true" stored="true" required="true" />
<field name="id" type="integer" indexed="true" stored="true" required="true" />
<field name="ip" type="string" indexed="true" stored="true" required="false" />
<field name="time" type="date" indexed="true" stored="true" required="true" />
<field name="epersonid" type="integer" indexed="true" stored="true" required="false" />
<field name="continent" type="string" indexed="true" stored="true" required="false"/>
<field name="country" type="string" indexed="true" stored="true" required="false"/>
<field name="countryCode" type="string" indexed="true" stored="true" required="false"/>
<field name="city" type="string" indexed="true" stored="true" required="false"/>
<field name="longitude" type="float" indexed="true" stored="true" required="false"/>
<field name="latitude" type="float" indexed="true" stored="true" required="false"/>
<field name="owningComm" type="integer" indexed="true" stored="true" required="false" multiValued="true"/>
<field name="owningColl" type="integer" indexed="true" stored="true" required="false" multiValued="true"/>
<field name="owningItem" type="integer" indexed="true" stored="true" required="false" multiValued="true"/>
<field name="dns" type="string" indexed="true" stored="true" required="false"/>
<field name="userAgent" type="string" indexed="true" stored="true" required="false"/>
<field name="isBot" type="boolean" indexed="true" stored="true" required="false"/>
<field name="referrer" type="string" indexed="true" stored="true" required="false"/>
<field name="uid" type="uuid" indexed="true" stored="true" default="NEW" />
<field name="statistics_type" type="string" indexed="true" stored="true" required="true" default="view" />
```

The combination of type and id determines which resource (either community, collection, item page or file download) has been requested.

Unique stored fields for bitstream downloads

```
<field name="bundleName" type="string" indexed="true" stored="true" required="false" multiValued="true" />
```

Unique stored fields for search queries

```
<field name="query" type="string" indexed="true" stored="true" required="false" multiValued="true"/>
<field name="scopeType" type="integer" indexed="true" stored="true" required="false" />
<field name="scopeId" type="integer" indexed="true" stored="true" required="false" />
<field name="rpp" type="integer" indexed="true" stored="true" required="false" />
<field name="sortBy" type="string" indexed="true" stored="true" required="false" />
<field name="sortOrder" type="string" indexed="true" stored="true" required="false" />
<field name="page" type="integer" indexed="true" stored="true" required="false" />
```

Unique stored fields for workflow events

```
<field name="workflowStep" type="string" indexed="true" stored="true" required="false" multiValued="true"/>
<field name="previousWorkflowStep" type="string" indexed="true" stored="true" required="false" multiValued="
true"/>
<field name="owner" type="string" indexed="true" stored="true" required="false" multiValued="true"/>
<field name="submitter" type="integer" indexed="true" stored="true" required="false" />
<field name="actor" type="integer" indexed="true" stored="true" required="false" />
<field name="workflowItemId" type="integer" indexed="true" stored="true" required="false" />
```

Disabling Tracking of Statistics

By default, Statistics are captured by the REST API for all visits (page hits) and downloads that occur via both the User Interface and the REST API.

Disabling statistical tracking currently must be done by modifying the backend's Spring configuration in [dspace]/config/spring/rest/event-service-listeners.xml. In that file, you must **comment out** the "SolrLoggerUsageEventListener" in order to disable all tracking

event-service-listeners.xml

```
<beans>
    ...
    <!-- Comment out this bean, as shown below, to disable all tracking of usage statistics in Solr -->
    <!-- Inject the SolrLoggerUsageEventListener into the EventService  -->
    <!--
    <bean class="org.dspace.statistics.SolrLoggerUsageEventListener">
        <property name="eventService" ref="org.dspace.services.EventService"/>
    </bean>
    -->
</beans>
```

After commenting out that bean, you will need to restart Tomcat.

NOTE: This only disables **tracking** statistics in Solr. The "Statistics" link will still appear in the header menu of the User Interface. However, you can limit its visibility by setting it to only be visible to administrative users. Update this configuration in your local.cfg or user-statistics.cfg:

```
# Limit access to Admins, Community/Collection Admins
usage-statistics.authorization.admin.usage = true
```

Statistics Menu link can only be removed by customizing DSpace

At this time, there is no flag to remove the "Statistics" menu link completely. See https://github.com/DSpace/DSpace/issues/9698

Web User Interface Elements

Pageview and Download statistics

In the UI, pageview and download statistics can be accessed from the "Statistics" navigation menu near the header. That statistics page is "context aware", so it will show the usage statistics for whatever page (site, Community, Collection) you are currently on.

If you are not seeing the menu, it's likely that they are only enabled for administrators in your installation. Change the configuration parameter "authorization.admin.usage" in usage-statistics.cfg to false in order to make statistics visible for all repository visitors.

Home page

Starting from the repository homepage, the statistics page displays the top 10 most popular items of the entire repository.

Community home page

The following statistics are available for the community home pages:

- Total visits of the current community home page
- Visits of the community home page over a timespan of the last 7 months
- Top 10 country from where the visits originate
- Top 10 cities from where the visits originate

Collection home page

The following statistics are available for the collection home pages:

- Total visits of the current collection home page
- Visits of the collection home over a timespan of the last 7 months
- Top 10 country from where the visits originate
- Top 10 cities from where the visits originate

Item home page

The following statistics are available for the item home pages:

- Total visits of the item
- Total visits for the bitstreams attached to the item
- Visits of the item over a timespan of the last 7 months
- Top 10 country views from where the visits originate
- Top 10 cities from where the visits originate

Search Query Statistics

Only supported in DSpace 6 and below

Search query statistics are only supported in DSpace 6.x and below at this time. The below screenshots and instructions are for 6.x and will need updating if this feature is ported to later versions of DSpace. See https://github.com/DSpace/DSpace/issues/2880

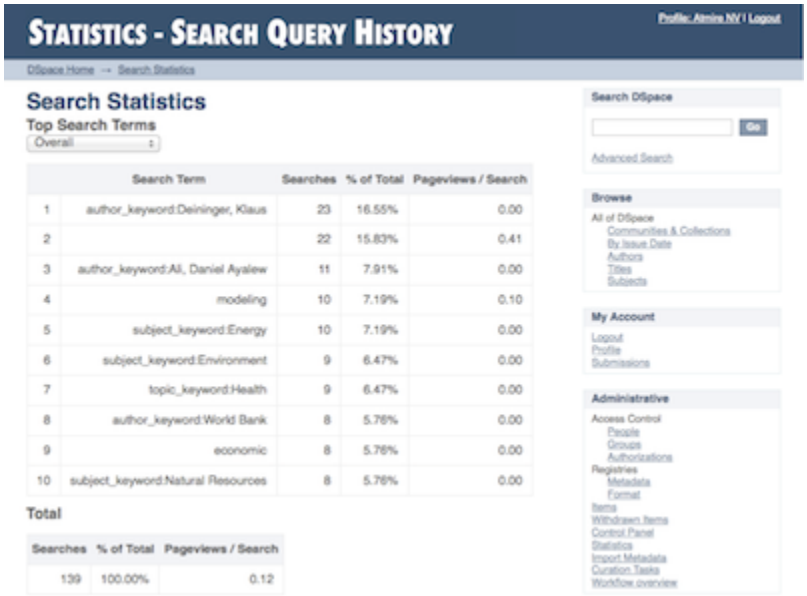
In the UI, search query statistics can be accessed from the lower end of the navigation menu.

If you are not seeing the link labelled "search statistics", it is likely that they are only enabled for administrators in your installation. Change the configuration parameter "authorization.admin.search" in usage-statistics.cfg to false in order to make statistics visible for all repository visitors.

The dropdown on top of the page allows you to modify the time frame for the displayed statistics.

The Pageviews/Search column tracks the amount of pages visited after a particular search term. Therefore a zero in this column means that after executing a search for a specific keyword, not a single user has clicked a single result in the list.

If you are using Discovery, note that clicking the facets also counts as a search, because clicking a facet sends a search query to the Discovery index.



Workflow Event Statistics

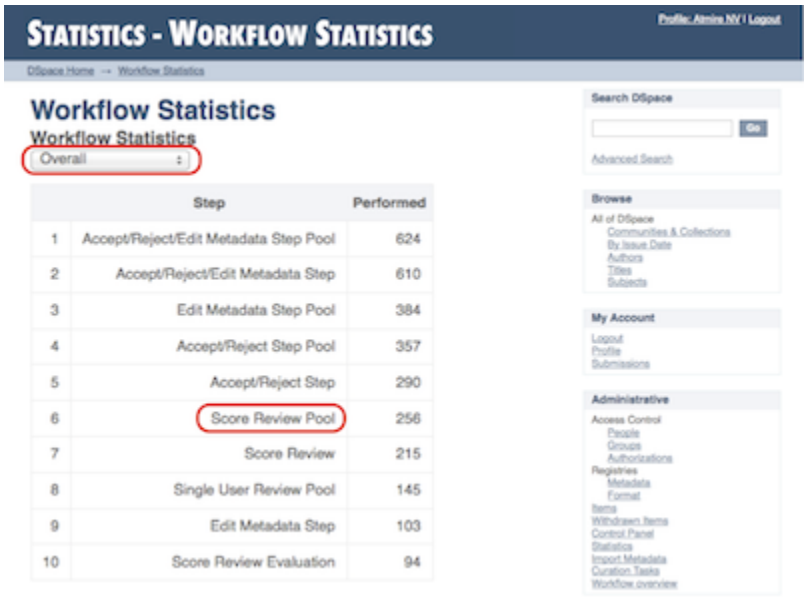
Only supported in DSpace 6 and below

Workflow Event statistics are only supported in DSpace 6.x and below at this time. The below screenshots and instructions are for 6.x and will need updating if this feature is ported to later versions of DSpace. See https://github.com/DSpace/DSpace/issues/2851

In the UI, search query statistics can be accessed from the lower end of the navigation menu.

If you are not seeing the link labelled "Workflow statistics", it is likely that they are only enabled for administrators in your installation. Change the configuration parameter "authorization.admin.workflow" in usage-statistics.cfg to false in order to make statistics visible for all repository visitors.

The dropdown on top of the page allows you to modify the time frame for the displayed statistics.



Architecture

The DSpace Statistics Implementation is a Client/Server architecture based on Solr for collecting usage events in the User Interface or REST API applications of DSpace. Solr must be installed separately from DSpace.

Configuration settings for Statistics

In the `{dspace.dir}/config/modules/solr-statistics.cfg` file review the following fields. These fields can be edited in place, or overridden in your own local.cfg config file (see Configuration Reference).

Property:	solr-statistics.server
Example Values:	solr-statistics.server = http://127.0.0.1/solr/statistics solr-statistics.server = ${solr.server}/statistics
Informational Note:	Is used by the SolrLogger Client class to connect to the Solr server over http and perform updates and queries. In most cases, this can (and should) be set to localhost (or 127.0.0.1). To determine the correct path, you can use a tool like `wget` to see where Solr is responding on your server. For example, you'd want to send a query to Solr like the following: `wget http://127.0.0.1/solr/statistics/select?q=*:*` Assuming you get an HTTP 200 OK response, then you should set `solr.log.server` to the '/statistics' URL of 'http://127.0.0.1/solr/statistics' (essentially removing the "/select?q=:" query off the end of the responding URL.)
Property:	solr-statistics.query.filter.bundles
Example Value:	solr-statistics.query.filter.bundles=ORIGINAL
Informational Note:	A comma seperated list that contains the bundles for which the file statistics will be displayed.
Property:	solr-statistics.query.filter.spiderIp
Example Value:	solr-statistics.query.filter.spiderIp = false

Informational Note:	If true, statistics queries will filter out spider IPs -- use with caution, as this often results in extremely long query strings.
Property:	solr-statistics.query.filter.isBot
Example Value:	solr-statistics.query.filter.isBot = true
Informational Note:	If true, statistics queries will filter out events flagged with the "isBot" field. This is the recommended method of filtering spiders from statistics.
Property:	solr-statistics.autoCommit
Example Value:	solr-statistics.autoCommit = true
Informational Note:	If true (default), then all view statistics will be committed to Solr whenever the next autoCommit is triggered. This is recommended behavior. If false, then view statistics will be committed to Solr *immediately* (i.e. via an explicit commit call). This setting is untested in Production scenarios, and is primarily used by automated integration tests (to verify that the statistics engine is working properly).
Property:	solr-statistics.spiderips.urls

Example Value:	solr-statistics.spiderips.urls =
	`http://iplists.com/google.txt, \`
	`http://iplists.com/inktomi.txt, \`
	`http://iplists.com/lycos.txt, \`
	`http://iplists.com/infoseek.txt, \`
	`http://iplists.com/altavista.txt, \`
	`http://iplists.com/excite.txt, \`
	`http://iplists.com/misc.txt`

Informational Note:	List of URLs to download spiders files into [dspace]/config/spiders. These files contain lists of known spider IPs and are utilized by the SolrLogger to flag usage events with an "isBot" field, or ignore them entirely. The "stats-util" command can be used to force an update of spider files, regenerate "isBot" fields on indexed events, and delete spiders from the index. For usage, run: `dspace stats-util -h` from your [dspace]/bin directory

In the `{dspace.dir}/config/modules/`**`usage-statistics`**`.cfg` file review the following fields. These fields can be edited in place, or overridden in your own local.cfg config file (see Configuration Reference).

Property:	usage-statistics.dbfile
Example Value:	usage-statistics.dbfile = ${dspace.dir}/config/GeoLite2-City.mmdb
Informational Note:	References the location of the installed GeoLite or DB-IP City "mmdb" database file. This file is utilized by the LocationUtils to calculate the location of client requests based on IP address. NOTE: This database file MUST be downloaded, installed and updated using third-party tools. See the "Managing the City Database File" section below.
Property:	usage-statistics.resolver.timeout

Example Value:	usage-statistics.resolver.timeout = 200
Informational Note:	Timeout in milliseconds for DNS resolution of origin hosts/IPs. Setting this value too high may result in solr exhausting your connection pool.
Property:	useProxies (Set in dspace.cfg)
Example Value:	useProxies = true
Informational Note:	Will cause Statistics logging to look for X-Forward URI to detect clients IP that have accessed it through a Proxy service (e.g. the Apache mod_proxy). Allows detection of client IP when accessing DSpace. [Note: This setting is found in the DSpace Logging section of dspace.cfg]
Property:	usage-statistics.authorization.admin.usage
Example Value:	usage-statistics.authorization.admin.usage = true
Informational Note:	When set to true, only general administrators, collection and community administrators are able to access the pageview and download statistics from the web user interface. As a result, the links to access statistics are hidden for non logged-in admin users. Setting this property to "false" will display the links to access statistics to anyone, making them publicly available.
Property:	usage-statistics.authorization.admin.search
Example Value:	usage-statistics.authorization.admin.search = true
Informational Note:	When set to true, only system, collection or community administrators are able to access statistics on search queries.
Property:	usage-statistics.authorization.admin.workflow
Example Value:	usage-statistics.authorization.admin.workflow = true
Informational Note:	When set to true, only system, collection or community administrators are able to access statistics on workflow events.
Property:	usage-statistics.logBots
Example Value:	usage-statistics.logBots = true
Informational Note:	When this property is set to false, and IP is detected as a spider, the event is not logged. When this property is set to true, the event will be logged with the "isBot" field set to true. (see solr-statistics.query.filter.* for query filter options)

Property:	usage-statistics.shardedByYear
Example Value:	usage-statistics.shardedByYear = false
Informational Note:	When set to "true", the DSpace statistics engine will look for additional Solr Shards (per year) when compiling all usage statistics. Therefore, if you are regularly running "stats-utils -s" (as documented in the "Solr Sharding By Year" section of the "SOLR Statistics Maintenance" page), then you should set this to "true". By default, it is "false", which tells the statistics engine to only compile usage statistics based on what is found in the current Solr core.

Pre-1.6 Statistics settings

Only supported in DSpace 6 and below

Search query statistics are only supported in DSpace 6.x and below at this time. See https://github.com/DSpace/DSpace/issues/2852

Older versions of DSpace featured static reports generated from the log files. They still persist in DSpace today but are completely independent from the SOLR based statistics.
The following configuration parameters applicable to these reports can be found in dspace.cfg.

```
###### Statistical Report Configuration Settings ######

 # should the stats be publicly available?  should be set to false if you only
 # want administrators to access the stats, or you do not intend to generate
 # any
 report.public = false

 # directory where live reports are stored
 report.dir = ${dspace.dir}/reports/
```

These fields are not used by the new 1.6 Statistics, but are only related to the Statistics from previous DSpace releases

Statistics Administration

Converting older DSpace logs into SOLR usage data

If you have upgraded from a previous version of DSpace, converting older log files ensures that you carry over older usage stats from before the upgrade.

Statistics Client Utility

The command line interface (CLI) scripts can be used to clean the usage database from additional spider traffic and other maintenance tasks. As of DSpace 3.0, a script has been added to split up the monolithic SOLR core into individual cores each containing a year of statistics.

Anonymizing Statistics

DSpace provides a commandline script (./dspace anonymize-statistics) which allows you to anonymize your statistics to better comply with GDPR and similar privacy regulations.

The script will anonymise the IP values by rewriting (‘masking’) the last part. This mask is configurable, both for ipv4 and ipv6 addresses.

- For IPv4 addresses, the last number will be replaced by the mask, defined by the configuration key ‘anonymise_statistics.ip_v4_mask’ which defaults to ‘254’.
  For example, 109.74.16.171 is rewritten as 109.74.16.254
- For IPv6 address, the last two numbers will be replaced by the mask, defined by the configuration key ‘anonymise_statistics.ip_v6_mask’ which defaults to ‘FFFF:FFFF’. For example, 2001:0db8:85a3:0000:0000:8a2e:0370:7334 is rewritten as 2001:0db8:85a3:0000:0000:8a2e:FFFF:FFFF

For each anonymised record, the DNS field is also replaced by “anonymised”.

Script options available:

- The program only processes records older than 90 days. This period can be altered with the config ‘anonymise_statistics.time_limit’ (expressed in days) in usage-statistics.cfg.
- "-s [sleep]" : The script takes an optional parameter ‘-s [sleep]’ (expressed in ms), which will make the Java thread sleep between the calls to Solr to reduce the load impact.
- "-t [threads]" : The Solr service commit mechanism is also optimised by adding multi-threading support. The script takes an optional parameter ‘-t [threads]’ to indicate how many threads the Solr service can use for this, if not given the thread count defaults to 2.

Statistical records can also be anonymised the moment they are created. Enabling this feature can be done by setting the configuration parameter "anonymise_statistics.anonymise_on_log" to true in "usage-statistics.cfg" When this configuration property is not set, the feature is disabled by default.

Custom Reporting - Querying SOLR Directly

When the web user interface does not offer you the statistics you need, you can greatly expand the reports by querying the SOLR index directly.

Resources

- https://www.safaribooksonline.com/library/view/apache-solr-enterprise/9781782161363/
- https://lucidworks.com/blog/faceted-search-with-solr/

Examples

Top downloaded items by a specific user

Query:

```
http://localhost:8983/solr/statistics/select?indent=on&version=2.2&start=0&rows=10&fl=*%
2Cscore&qt=standard&wt=standard&explainOther=&hl.fl=&facet=true&facet.field=epersonid&q=type:0
```

Explained:

facet.field=epersonid — You want to group by epersonid, which is the user id.
type:0 — Interested in bitstreams only

```
<lst name="facet_counts">
    <lst name="facet_fields">
        <lst name="epersonid">
            <int name="66">1167</int>

<int name="117">251</int>

<int name="52">42</int>

<int name="19">36</int>

<int name="88">20</int>

<int name="112">18</int>

<int name="110">9</int>

<int name="96">0</int>

</lst>
    </lst>
</lst>
```

Managing the City Database File

If you wish to record the geographic locations of clients in your DSpace statistics records (e.g. the City or Country where they are accessing your DSpace), you **must** install (and regularly update) one of the following IP to City Database Files (in MMDB format). We recommend installing a City-level database, as it provides more granular location information than a Country-level database (which can only provide the country where the access originated).

- Either install a copy of MaxMind's GeoLite City database (in MMDB format)
    - Installing MaxMind GeoLite2 is *free*. However, you **must** sign up for a (free) MaxMind account in order to obtain a license key to use the GeoLite2 database.
    - You will need to arrange regular downloads of the GeoLite2 database. MaxMind offers an updater tool (geoipupdate) to do the downloading/updating, and a number of Linux distributions package it (as `geoipupdate`). You will still need to configure your license key prior to usage. Use it before restarting DSpace, to get an up-to-date database.
    - Once the "GeoLite2-City.mmdb" database file is installed on your system, you will need to configure its location as the value of `usage-statistics.dbfile` in your `local.cfg` configuration file.
    - NOTE: This file is frequently updated by MaxMind.com, so you will need to refresh it regularly (ideally by scheduling the updater tool via a cron job or similar). As this is written, the database is updated monthly, and to be allowed to obtain it you need to agree to keep your copy updated.
- Or, you can alternatively use/install DB-IP's City Lite database (in MMDB format)
    - This database is also free to use, but does **not** require an account to download.
    - You will need to arrange regular downloads of the City Lite database. DB-IP offers an updater tool (dbip-update) to do the downloading /updating, but it requires PHP to run.
    - Once the "dbip-city-lite.mmdb" database file is installed on your system, you will need to configure its location as the value of `usage-statistics.dbfile` in your `local.cfg` configuration file.

- NOTE: This file is frequently updated by DB-IP.com, so you will need to refresh it regularly (ideally by scheduling the updater tool via a cron job or similar). As this is written, the database is updated monthly with the latest available at https://db-ip.com/db/download/ip-to-city-lite

SOLR Statistics Maintenance

1 DSpace Log Converter
2 Filtering and Pruning Spiders
3 Export SOLR records to intermediate format for import into another tool/instance
4 Export SOLR statistics, for backup and moving to another server
5 Import SOLR statistics, for restoring lost data or moving to another server
6 Reindex SOLR statistics, for upgrades or whenever the Solr schema for statistics is changed
7 Upgrade Legacy DSpace Object Identifiers (pre-6x statistics) to DSpace 6x UUID Identifiers
8 Solr Sharding By Year
    8.1 Technical implementation details
    8.2 Testing Solr Shards

DSpace Log Converter

The use of Solr for statistics in DSpace makes it possible to have a database of statistics. With this in mind, there is the issue of the older log files and how a site can use them. The following command process is able to convert the existing log files and then import them for Solr use. The user will need to perform this conversion only once.

The Log Converter program converts log files from dspace.log into an intermediate format that can be inserted into Solr.

Command used:	`[dspace]/bin/dspace stats-log-converter`
Java class:	org.dspace.statistics.util.ClassicDSpaceLogConverter
Arguments short and long forms):	Description
`-i` or `--in`	Input file
`-o` or `--out`	Output file
`-m` or `--multiple`	Adds a wildcard at the end of input and output, so it would mean if `-i dspace.log -m` was specified, dspace.log* would be converted. (i.e. all of the following: *dspace.log, dspace.log.1, dspace.log.2, dspace.log.3,* etc.)
`-n` or `--newformat`	If the log files have been created with DSpace 1.6 or newer
`-v` or `--verbose`	Display verbose output (helpful for debugging)
`-h` or `--help`	Help

The command loads the intermediate log files that have been created by the aforementioned script into Solr.

Command used:	`[dspace]/bin/dspace stats-log-importer`
Java class:	org.dspace.statistics.util.StatisticsImporter
Arguments (short and long forms):	Description
`-i` or `--in`	input file
`-m` or `--multiple`	Adds a wildcard at the end of the input, so it would mean dspace.log* would be imported
`-s` or `--skipdns`	To skip the reverse DNS lookups that work out where a user is from. (The DNS lookup finds the information about the host from its IP address, such as geographical location, etc. This can be slow, and wouldn't work on a server not connected to the internet.)
`-v` or `--verbose`	Display verbose ouput (helpful for debugging)
`-l` or `--local`	For developers: allows you to import a log file from another system, so because the handles won't exist, it looks up random items in your local system to add hits to instead.
`-h` or `--help`	Help

Although the DSpace Log Convertor applies basic spider filtering (googlebot, yahoo slurp, msnbot), it is far from complete. Please refer to Filtering and Pruning Spiders for spider removal operations, after converting your old logs.

Filtering and Pruning Spiders

Command used:	`[dspace]/bin/dspace stats-util`
Java class:	org.dspace.statistics.util.StatisticsClient
Arguments (short and long forms):	Description
`-b` or `--reindex-bitstreams`	Reindex the bitstreams to ensure we have the bundle name
`-r` or `--remove-deleted-bitstreams`	While indexing the bundle names remove the statistics about deleted bitstreams
`-u` or `--update-spider-files`	Update Spider IP Files from internet into `[dspace]/config/spiders`. Downloads Spider files identified in `dspace.cfg` under property `solr.spiderips.urls`. See Configuration settings for Statistics
`-f` or `--delete-spiders-by-flag`	Delete Spiders in Solr By isBot Flag. Will prune out all records that have `isBot:true`
`-i` or `--delete-spiders-by-ip`	Delete Spiders in Solr By IP Address, DNS name, or Agent name. Will prune out all records that match spider identification patterns.
`-m` or `--mark-spiders`	Update isBot Flag in Solr. Marks any records currently stored in statistics that have IP addresses matched in spiders files
`-h` or `--help`	Calls up this brief help table at command line.

Notes:

The usage of these options is open for the user to choose. If you want to keep spider entries in your repository, you can just mark them using "`-m`" and they will be excluded from statistics queries when "`solr.statistics.query.filter.isBot = true`" in the `dspace.cfg`. If you want to keep the spiders out of the solr repository, just use the "`-i`" option and they will be removed immediately.

Spider IPs are specified in files containing one pattern per line. A line may be a comment (starting with "#" in column 1), empty, or a single IP address or DNS name. If a name is given, it will be resolved to an address. Unresolvable names are discarded and will be noted in the log.

There are guards in place to control what can be defined as an IP range for a bot. In `[dspace]/config/spiders`, spider IP address ranges have to be at least 3 subnet sections in length 123.123.123 and IP Ranges can only be on the smallest subnet [123.123.123.0 - 123.123.123.255]. If not, loading that row will cause exceptions in the dspace logs and exclude that IP entry.

Spiders may also be excluded by DNS name or Agent header value. Place one or more files of patterns in the directories `[dspace]/config/spiders/domains` and/or `[dspace]/config/spiders/agents`. Each line in a pattern file should be either empty, a comment starting with "#" in column 1, or a regular expression which matches some names to be recognized as spiders.

Export SOLR records to intermediate format for import into another tool/instance

Command used:	`[dspace]/bin/dspace stats-util`
Java class:	org.dspace.statistics.util.StatisticsClient
Arguments (short and long forms):	Description
`-e` or `--export`	Export SOLR view statistics data to usage statistics intermediate format

This exports the records to `[dspace]/temp/usagestats_0.csv`. This will chunk the files at 10,000 records to new files. This can be imported with `stats-log-importer` to SOLR Statistics

Export SOLR statistics, for backup and moving to another server

Command used:	`[dspace]/bin/dspace solr-export-statistics`
Java class:	org.dspace.util.SolrImportExport
Arguments (short and long forms):	Description
`-i` or - -index-name	optional, the name of the index to process. "statistics" is the default. "authority" can also be exported.
`-l or --last` *`integer`*	optionally export only *integer* many days worth of statistics
`-d or --directory`	optional, directory to use for storing the exported files. By default, `[dspace]/solr-export` is used. If that is not appropriate (due to storage concerns), we recommend you use this option to specify a more appropriate location.

-f or --force-overwrite	optional, overwrite export file if it exists (DSpace 6.1 and later)

Import SOLR statistics, for restoring lost data or moving to another server

Command used:	`[dspace]/bin/dspace solr-import-statistics`
Java class:	org.dspace.util.SolrImportExport
Arguments (short and long forms):	Description
-i or --index-name	optional, the name of the index to process. "statistics" is the default. "authority" can also be imported.
`-c or --clear`	optional, clears the contents of the existing stats core before importing
`-d or --directory`	optional, directory which contains the files for importing. By default, `[dspace]/solr-export` is used. If that is not appropriate (due to storage concerns), we recommend you use this option to specify a more appropriate location.

Reindex SOLR statistics, for upgrades or whenever the Solr schema for statistics is changed

Command used:	`[dspace]/bin/dspace solr-reindex-statistics`
Java class:	org.dspace.util.SolrImportExport
Arguments (short and long forms):	Description
-i or --index-name	optional, the name of the index to process. "statistics" is the default
`-k or --keep`	optional, tells the script to keep the intermediate export files for possible later use (by default all exported files are removed at the end of the reindex process).
`-d or --directory`	optional, directory to use for storing the exported files (temporarily, unless you also specify `--keep`, see above). By default, `[dspace]/solr-export` is used. If that is not appropriate (due to storage concerns), we recommend you use this option to specify a more appropriate location. Not sure about your space requirements? You can estimate the space required by looking at the current size of `[dspace]/solr/statistics`
-f or --force-overwrite	optional, overwrite export file if it exists (DSpace 6.1 and later)

NOTE: `solr-reindex-statistics` is safe to run on a live site. The script stores incoming usage data in a temporary SOLR core, and then merges that new data into the reindexed data when the reindex process completes.

Upgrade Legacy DSpace Object Identifiers (pre-6x statistics) to DSpace 6x UUID Identifiers

This command was introduced in **DSpace 7.0** and will be included in the **DSpace 6.4** release as well.

It is recommended that all DSpace instances with legacy identifiers perform this one-time upgrade of legacy statistics records.

This action is safe to run on a live site. As a precaution, it is recommended that you backup you statistics shards before performing this action.

Note: a link to this section of the documentation should be added to the DSpace 6.4 Release Notes. (It is already noted in the DSpace 7.0 Upgrading DSpace page, step 11d)

The DSpace 6x code base changed the primary key for all DSpace objects from an integer id to UUID identifiers. Statistics records that were created before upgrading to DSpace 6x contain the legacy identifiers.

While the DSpace user interfaces make some attempt to correlate legacy identifiers with uuid identifiers, it is recommended that users perform this one time upgrade of legacy statistics records.

If you have sharded your statistics repository, this action must be performed on each shard.

Command used:	`[dspace]/bin/dspace solr-upgrade-statistics-6x`

Java class:	org.dspace.util.SolrUpgradePre6xStatistics
Arguments (short and long forms):	Description
- i or - -index-name	Optional, the name of the index to process. "statistics" is the default
`-n or --num_rec`	Optional. Total number of records to update (defaut=100,000). To process all records, set -n to 10000000 or to 100000000 (10M or 100M) If possible, please allocate 2GB of memory to this process (e.g. -Xmx2000m)
`-b or --batch_size`	Number of records to batch update to SOLR at one time (default=10,000).

NOTE: This process will rewrite most solr statistics records and may temporarily double the size of your statistics repositories.

If a UUID value cannot be found for a legacy id, the legacy id will be converted to the form "xxxx-unmigrated" where xxxx is the legacy id.

Solr Sharding By Year

The DSpace tool described below for managing Solr data through yearly sharding no longer functions in DSpace 7.x (see also https://github.com/DSpace/DSpace/issues/8478). Using these tools to manage Solr shards is no longer recommended. Alternative approaches are being explored and this page will be updated to reflect those findings.

Command used:	`[dspace]/bin/dspace stats-util`
Java class:	org.dspace.statistics.util.StatisticsClient
Arguments (short and long forms):	Description
`-s` or `--shard-solr-index`	Splits the data in the main Solr core up into a separate core for each year. This will upgrade the performance of Solr.

Notes:

Yearly Solr sharding is a routine that can drastically improve the performance of your DSpace SOLR statistics. It was introduced in DSpace 3.0 and is not backwards compatible. The routine decreases the load created by the logging of new usage events by reducing the size of the SOLR Core in which new usage data are being logged. By running the script, you effectively split your current SOLR core, containing all of your usage events, into different SOLR cores that each contain the data for one year. In case your DSpace has been logging usage events for less than one year, you will see no notable performance improvements until you run the script after the start of a new year. Both writing new usage events as well as read operations should be more performant over several smaller SOLR Shards instead of one monolithic one.

It is highly recommended that you execute this script once at the start of every year. To ensure this is not forgotten, you can include it in your crontab or other system scheduling software. Here's an example cron entry (just replace [dspace] with the full path of your DSpace installation):

```
# At 12:00AM on January 1, "shard" the DSpace Statistics Solr index.  Ensures each year has its own Solr index
- this improves performance.
0 0 1 1 * [dspace]/bin/dspace stats-util -s
```

You MUST restart Tomcat after sharding

After running the statistics shard process, the "View Usage Statistics" page(s) in DSpace will **not** automatically recognize the new shard.

Restart tomcat to ensure that the new shard is recognized & included in usage statistics queries.

Repair of Shards Created Before DSpace 5.7 or DSpace 6.1

If you ran the shard process before upgrading to DSpace 5.7 or DSpace 6.1, the multi-value fields such as owningComm and onwningColl are likely be corrupted. Previous versions of the shard process lost the multi-valued nature of these fields. Without the multi-valued nature of these fields, it is difficult to query for statistics records by community / collection / bundle.

You can verify this problem in the solr admin console by looking at the owningComm field on existing records and looking for the presence of "\\," within that field.

The following process may be used to repair these records.

1. Backup your solr statistics-xxxx directories while tomcat is down.
2. Backup and delete the contents of the dspace-install/solr-export directory
3. For each "statistics-xxxx" shard that exists, export the repository

```
dspace solr-export-statistics -i statistics-xxxx -f
```

4. Run the following to repair records in the dspace-install/solr-export directory

```
for file in *
do
sed -E -e "s/[\\]+,/,/g" -i $file
done
```

5. For each shard that was exported, run the following import

```
dspace solr-import-statistics -i statistics-xxxx -f
```

If you repeat the query that was run previously, the fields containing "\\," should now contain an array of owning community ids.

Shard Naming

Prior to the release of DSpace 6.1, the shard names created were off by one year in timezones with a positive offset from GMT.

Shar

See Unable to locate Jira server for this macro. It may be due to Application Link configuration.

Technical implementation details

After sharding, the Solr data cores are located in the [dspace.dir]/solr directory. There is no need to define the location of each individual core in solr.xml because they are automatically retrieved at runtime. This retrieval happens in the *static* method located in the *org.dspace.statistics.SolrLogger* class. These cores are stored in the *statisticYearCores* list. Each time a query is made to Solr, these cores are added as shards by the *addAdditionalSolrYearCores* method. The cores share a common configuration copied from your original *statistics* core. Therefore, no issues should be resulting from subsequent an`t updates`.

The actual sharding of the of the original Solr core into individual cores by year is done in the *shardSolrIndex* method in the *org.dspace.statistics.SolrLogger* class. The sharding is done by first running a facet on the time to get the facets split by year. Once we have our years from our logs we query the main Solr data server for all information on each year & download these as CSVs. When we have all data for one year, we upload it to the newly created core of that year by using the update csv handler. Once all data of one year have been uploaded, those data are removed from the main Solr (by doing it this way if our Solr crashes we do not need to start from scratch).

Multiple Shard Fix (DSpace 6.1)

A bug exists in the DSpace 6.0 release that prevents tomcat from starting when multiple shards are present.

To address this issue, the initialization of SOLR shards is deferred until the first SOLR related requests are processed.

See Unable to locate Jira server for this macro. It may be due to Application Link configuration.

Testing Solr Shards

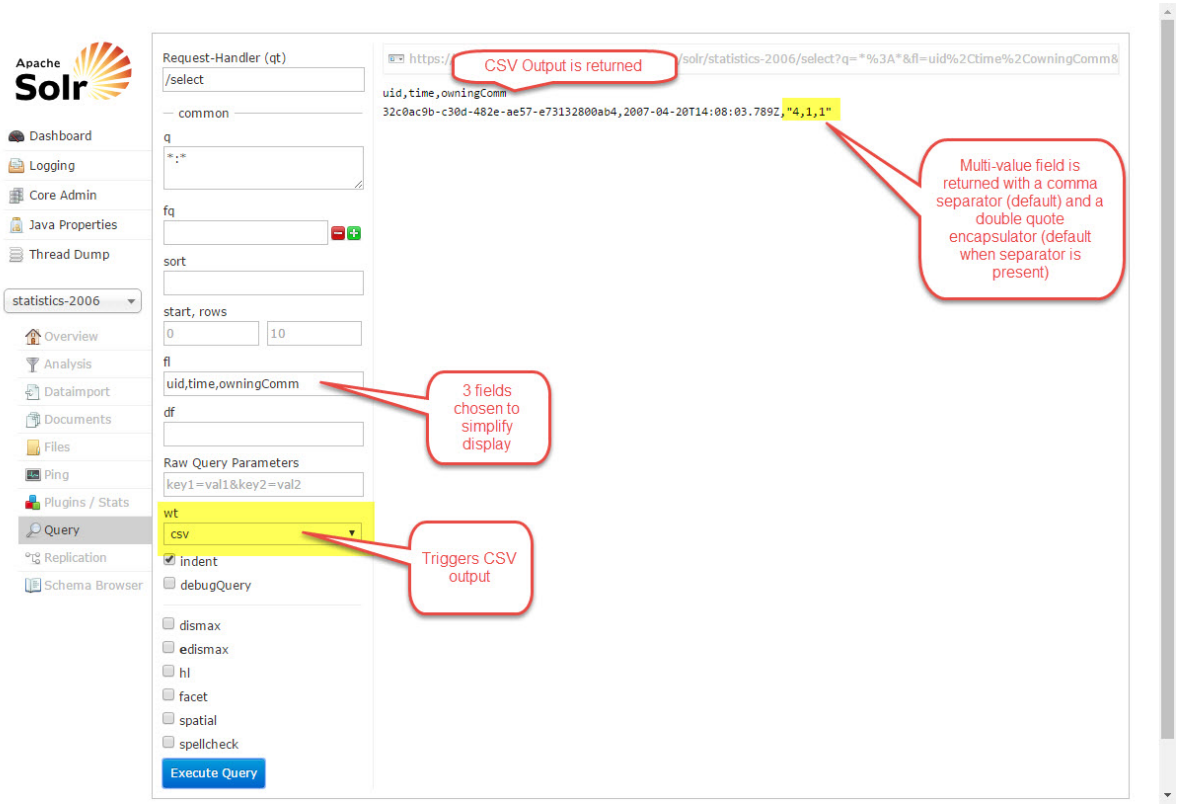
Testing Solr Shards

Testing Solr Shards

These notes detail how to test and manipulate SOLR statistics shards.

Testing CSV Export

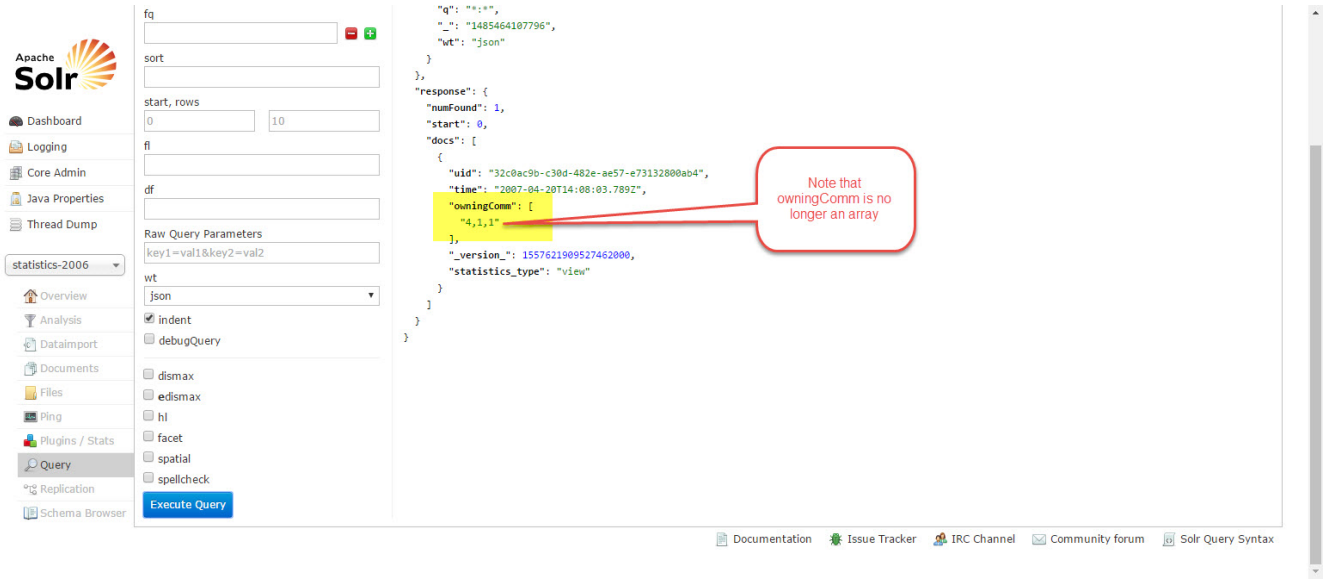
The SOLR Admin Console provides a mechanism to test the CSV Export Process and Parameters



Testing CSV Import

The SOLR Admin Console provides a mechanism to access the CSV Upload process. Unfortunately, it does not all parameters to be provided.

Note that the multi-value field is corrupted if you import by this manner.



It is possible to csv import parameters using curl.

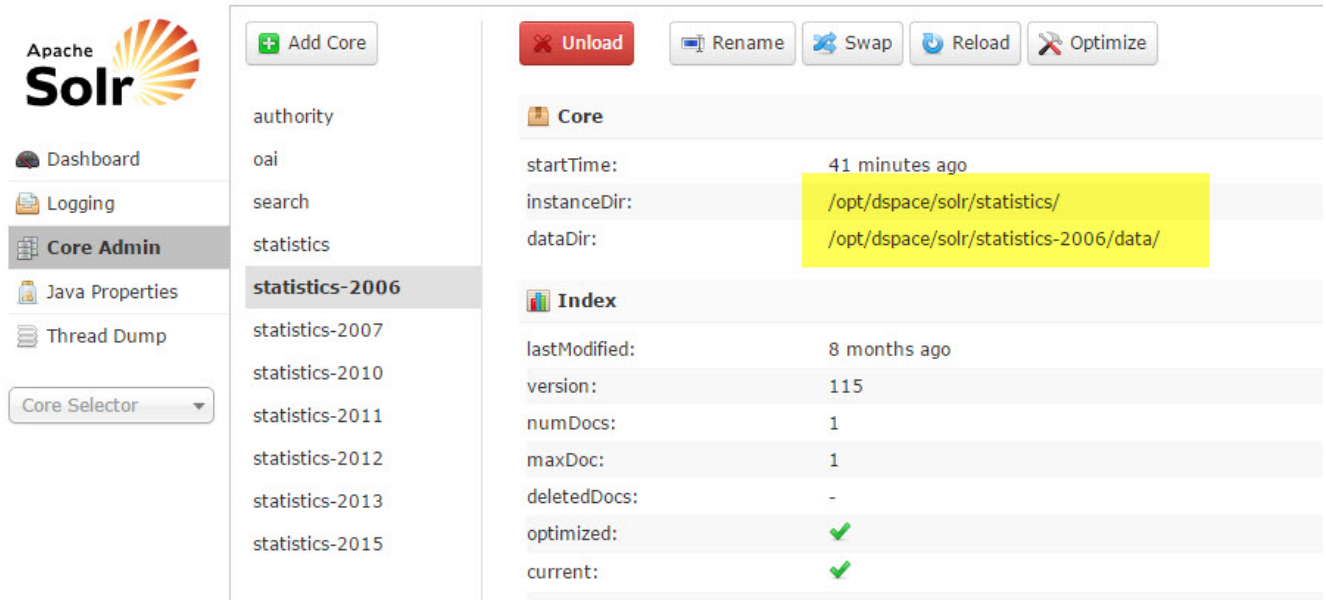
Running CSV Upload with curl

```
curl -F "data=@statistics-2006_export_2007-04.csv" "http://localhost/solr/statistics-2006/update/csv?skip=_version_&csv.mv.escape=%5C&f.owningColl.split=true&f.owningColl.separator=%7C&f.owningComm.split=true&f.owningComm.separator=,&f.owningItem.split=true&f.owningItem.separator=%7C&f.bundleName.split=true&f.bundleName.separator=%7C&stream.contentType=text%2Fcsv%3Bcharset%3Dutf-8&commit=true&softCommit=false&waitSearcher=true&wt=javabin&version=2"
```

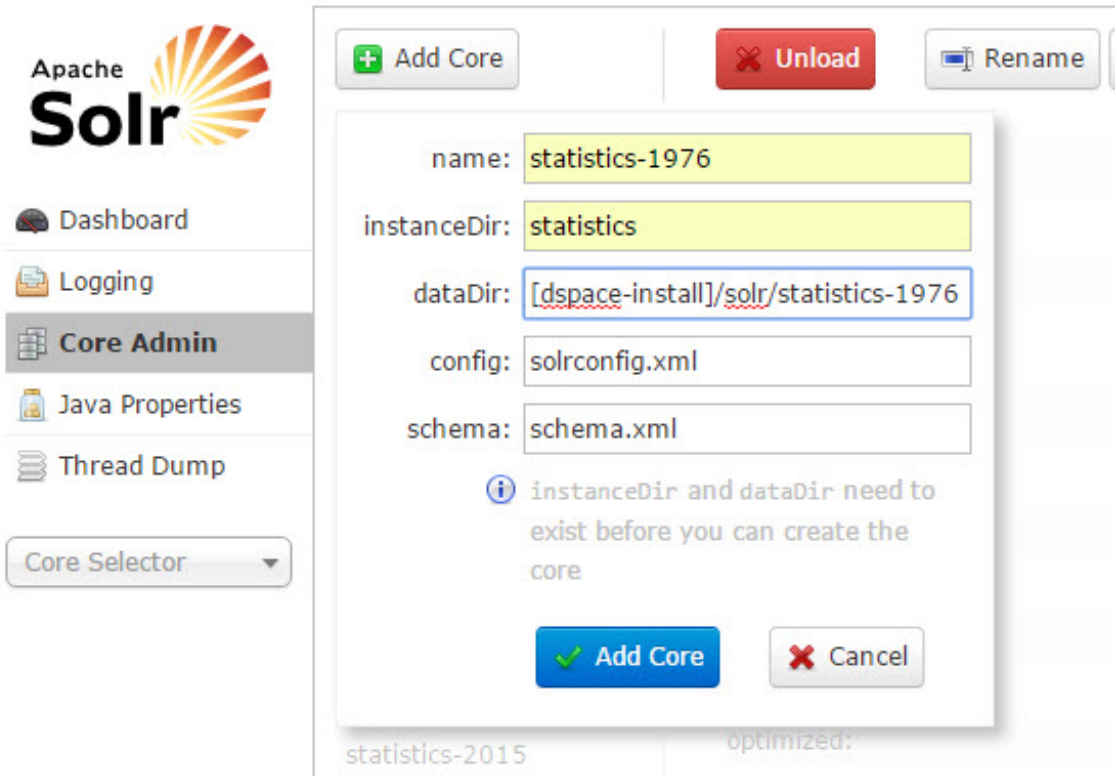
Creating a Shard in the Admin Console

While this is probably not necessary, it is possible to create an empty shard in the Solr Admin console.

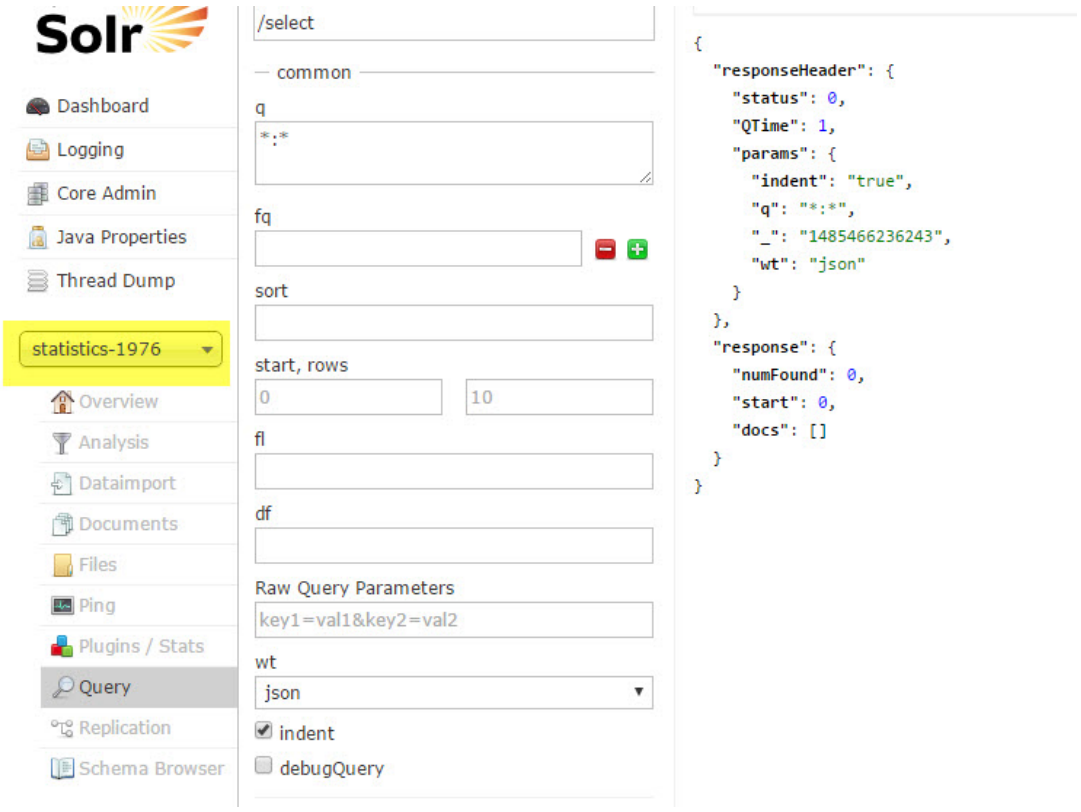
Note that existing shards use the statistics directory as an "instance" directory.



Manually create a new shard



The new shard can be queried like the other ones



DSpace Google Analytics Statistics

- Google Analytics Support
  - Enabling Google Analytics
  - Configuring Google Analytics
- Google Analytics Reports in DSpace UI
  - Configuration settings for Google Analytics Statistics

Google Analytics development and debugging
- Identifying outgoing events in Chrome Dev Tools
- The GA4 Measurement protocol and event parameters
- Analytics Debug View and Debugger Chrome extension

Google Analytics Support

It is possible to record User Interface traffic by enabling the recording of Google Analytics data within DSpace. DSpace supports Google Analytics 4 and as of July 1, 2024, Google Analytics 4 has replaced Universal Analytics. Also, under GA4 it currently supports the Google Tag (gtag.js), but not Google Tag Manager (GTM). See Where is My Google Tag ID for more information.

Enabling Google Analytics

By default, Google Analytics is disabled in DSpace. To enable it, simply set the value of `google.analytics.key` in either your local.cfg or dspace.cfg:

```
# For Google Analytics 4
google.analytics.key = G-XXXXX
```

When Google Analytics is disabled, you will see 404 responses returned from the REST API whenever the User Interface attempts to access `${dspace.server.url}/api/config/properties/google.analytics.key`. This is expected behavior, as that 404 response is the REST API telling the User Interface that Google Analytics is not configured. When the UI sees that 404 from the REST API, it disables Google Analytics tracking the UI.

Configuring Google Analytics

Additional configuration are provided to allow for enhanced Google Analytics support.

Property:	`google.analytics.buffer.limit`
Example Value:	`google.analytics.buffer.limit = 256`
Informational Note:	Maximum number of events held in the buffer to send to Google Analytics. Used in conjunction with "cron" settings below.
Property:	`google.analytics.cron`
Example Value:	`google.analytics.cron = 0 0/5 * * * ?`
Informational Note:	REQUIRED if you want to send file download events to Google Analytics (where they will be tracked as Google "events"). This defines the schedule for how frequently events tracked on the backend (like file downloads) will be sent to Google Analytics. Syntax is defined at https://www.quartz-scheduler.org/api/2.3.0/org/quartz/CronTrigger.html The above example will run this task every 5 minutes (0 0/5 * * * ?) For Google Analytics 4, you MUST also add the "api-secret" below to support sending download events.
Property:	`google.analytics.api-secret`
Example Value:	`google.analytics.api-secret = mysecret`

Informational Note:	(Only used for Google Analytics 4) Defines a Measurement Protocol API Secret to be used to track interactions which occur outside of the user's browser. This is REQUIRED to track downloads of bitstreams. For more details see https://developers.google.com/analytics/devguides/collection/protocol/ga4 Steps to create your API secret are also available from https://www.monsterinsights.com/docs/how-to-create-your-measurement-protocol-api-secret-in-ga4/
Property:	`google-analytics.bundles`
Example Value:	`google-analytics.bundles = ORIGINAL`
Informational Note:	Which Bundles to include in Bitstream statistics. By default, set to ORIGINAL bundle only.

Google Analytics Reports in DSpace UI

DSpace 7 does not yet support

Google Analytics Reporting is not available in DSpace 7.0. While DSpace 7 can capture statistics via Google Analytics (see above), it is not able to display Google Analytics reports in the DSpace User Interface (like was supported in the XMLUI). It is under discussion as it's unclear how many sites used this feature. See DSpace Release 7.0 Status

As of DSpace version 5.0 it has also become possible to expose that recorded Google Analytics data within DSpace. The data is retrieved from Google using the Google Analytics Reporting API v3. This feature is disabled by default, to enable it please follow the instructions below.

Please read the documentation found at https://developers.google.com/analytics/devguides/reporting/core/v3/ and https://developers.google.com/accounts/docs/OAuth2ServiceAccount. It is the definitive documentation, however, it is over detailed for our purposes so the critical steps are summarized below. The theory is that as a developer you would create a Google project, write your application and store the code in the Google code repository, then create a Google Service Account which your application could use to retrieve data from the Google Analytics API. In our case we already have our application, DSpace, but we still have to go through the motions of creating a project in order to be able to be able to generate the Service Account which we need to allow DSpace to talk to the Google Analytics API.

1. Logon to the Google Developers Console https://console.developers.google.com/project with whatever email address you use to access/manage your existing Google Analytics account(s).
2. Create a new Google Project. The assumption is that you are developing some new software and will make use of the Google code repository. This is not the case but you need to create the skeleton project before you can proceed to the next step.
3. Enable the Analytics API for the project. In the sidebar on the left, expand **APIs & auth**. Next, click **APIs**. In the list of APIs, make sure the status is **ON** for the Analytics API.
4. In the sidebar on the left, select **Credentials**.
5. Select **OAuth / Create new Client ID**, then in the subsequent popup screen select **Service account**. This will automatically generate the required Service Account email address and certificate.
6. Go to your Google Analytics dashboard http://www.google.com/analytics/. Create an account for the newly generated Service Account email address and give it permission to 'Read and Analyze' at account level. See *Note below.
7. The generated certificate needs to be placed somewhere that your DSpace application can access and be referenced as described below in the configuration section..

*Note:- The Google documentation specifies that the Service Account email address should only require 'Read and Analyze' permission. However, it would appear this may not be the case and it may be necessary to grant greater permissions, at least initially.

Configuration settings for Google Analytics Statistics

In the `[dspace.dir]/config/modules/google-analytics.cfg` file review the following fields. These should be either edited directly or overridden in your local.cfg config file (see Configuration Reference).

Property:	google-analytics.application.name
Value:	Dummy Project
Informational Note:	Not sure if this property is required but it was in the example code provided by Google. Please do not delete.
Property:	google-analytics.table.id
Example Value:	ga:12345678

Informational Note:	Logon to the Google Analytics Dashboard and select the Property (or website in plain English) that you wish to target. Then select the Admin section for the property. You should then be able to select the 'view settings' for the view you are interested in. The View ID should replace 12345678 below (note - confusingly the Reporting API documentation refers to the View ID as Table ID).
Property:	google-analytics.account.email
Example Value:	xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx@developer.gserviceaccount.com
Informational Note:	The email address automatically generated when you created the Service Account.
Property:	google-analytics.certificate.location
Example Value:	/home/example/dslweb--privatekey.p12
Informational Note:	The certificate file automatically generated when you created the Service Account.
Property:	google-analytics.authorization.admin.usage
Example Value:	true
Informational Note:	Control if the statistics pages should be only shown to authorized users. If enabled, only the administrators for the DSpaceObject will be able to view the statistics. If disabled, anyone with READ permissions on the DSpaceObject will be able to view the statistics.

Google Analytics development and debugging

Identifying outgoing events in Chrome Dev Tools

Through "Inspect Element" in Chrome DevTools, you can identify outgoing requests to Google Analytics filtering on requests starting with "collect?"

The GA4 Measurement protocol and event parameters

Google Analytics 4 Measurement Protocol provides a handy overview of the parameters that are sent along.

Analytics Debug View and Debugger Chrome extension

DebugView documentation: GA4 Monitor events in DebugView

Exchange usage statistics with IRUS

1 Introduction
2 Prerequisite
3 Configuration
4 Re-trying failed attempts

IRUS is being retired at the end of April 2026. See the announcement at https://research.jiscinvolve.org/wp/2025/09/02/preparing-for-changes-to-irus/

Introduction

IRUS (Institutional Repository Usage Statistics) enables Institutional Repositories to share and expose statistics based on the COUNTER standard.
It offers opportunities for benchmarking and acts as an intermediary between repositories and other agencies.

IRUS is currently available in the following areas/countries:

- United Kingdom: https://irus.jisc.ac.uk/
- Australia and New-Zealand: https://irus.jisc.ac.uk/irus-anz/
- United States: https://www.lyrasis.org/programs/Pages/IRUS-US.aspx

Prerequisite

The DSpace server should be able to access the tracker's base production and test URL's.
The tracker's base production URL will depend on the area/country where your repository is located:

- United Kingdom: http://irus.jisc.ac.uk/counter/
- Australia and New-Zealand: https://irus.jisc.ac.uk/counter/anz/
- United States: https://irus.jisc.ac.uk/counter/us/

The tracker's base test URL is common to all areas/countries:

https://irus.jisc.ac.uk/counter/test/

Access to the tracker's base URLs can easily be verified using a *wget* command with the applicable URL, e.g.:

```
wget https://irus.jisc.ac.uk/counter/test/
```

The above command should return a HTTP 200.

Configuration

The IRUS statistics tracker can be configured in the irus-statistics.cfg file which can be found [dspace-src]/dspace/config/modules.

Property	Description	Default
`irus.statistics.tracker.enabled`	Configuration used to enable the IRUS statistics tracker. Set to true to enable.	`false`
`irus.statistics.tracker.type-field`	Metadata field to check if certain items should be excluded from tracking. If empty or commented out, all items are tracked.	
`irus.statistics.tracker.type-value`	The values in the above metadata field that will be considered to be tracked.	
`irus.statistics.tracker.entity-types`	The entity types to be included in the tracking. If left empty, only publication hits will be tracked. If entities are disabled in DSpace (the default in DSpace 7.1), then all Items will be included in tracking.	`Publication`
`irus.statistics.tracker.environment`	The tracker environment determines to which url the statistics are exported (test or prod).	`test`
`irus.statistics.tracker.testurl`	The url to which the trackings are exported when testing. (In theory, this should be https://irus.jisc.ac.uk/counter/test/)	
`irus.statistics.tracker.produrl`	The url to which the trackings are exported in production. (this will depend on your area/country, refer to the Prerequisite section)	
`irus.statistics.tracker.urlversion`	Tracker version	

`irus.statistics.spider.agentregex.url`	External URL pointing to the COUNTER user agents file. The user agents file is downloaded from the provided URL as part of the Apache ant build process. Item views determined by DSpace to have been generated by bots/spiders are not sent to IRUS. Including this additional (and optional) agents file can reduce unnecessary network traffic by reducing the need to transfer view data that will be ignored by IRUS. Example value: https://raw.githubusercontent.com/atmire/COUNTER-Robots/master/generated/COUNTER_Robots_list.txt	
`irus.statistics.spider.agentregex.regexfile`	Location where the user agents file should be downloaded to. The Apache ant build process that retrieves the user agents file from the URL specified above places it in the location specified here. Example value: ${dspace.dir}/config/spiders/agents/COUNTER_Robots_list.txt	

Re-trying failed attempts

If the IRUS tracker is down or some other kind of error should occur preventing DSpace from committing to the tracker, the record is stored in the database in a separate table (`OpenUrlTracker`) that is being created automatically during deployment. Committing these entries can be tried again using the following command.

```
[deployed-dspace]/bin/dspace retry-tracker
```

This will iterate over all the logged entries and retry committing them. If they fail again, they remain in the table, if they succeed, they are removed.

It is strongly advised to schedule this script to be executed daily or weekly (preferable at low load-times during the night or weekend). If there are no failed entries, the script will not perform any actions and exit immediately.

User Interface

- User Interface Configuration
- User Interface Customization
- User Interface Debugging
- Accessibility
- Browse
- Discovery
- Contextual Help Tooltips
- IIIF Configuration
- Multilingual Support

User Interface Configuration

- Overview
- Configuration File Format
  - Migrate environment file to YAML
- Configuration Override
  - In 7.2 or above
  - In 7.1 or 7.0
- Configuration Reference
  - Production Mode
  - UI Core Settings
  - Server Side Rendering (SSR) Settings
  - REST API Settings
  - Cache Settings - General
  - Cache Settings - Server Side Rendering (SSR)
  - Authentication Settings
  - Form Settings
  - Notification Settings
  - Submission Settings
  - Language Settings
  - Browse By Settings
  - Community-List Settings
  - Homepage Settings
  - Undo Settings
  - Item Access Labels
  - Item Page Settings
  - Theme Settings
  - Media Viewer Settings
    - Uploading video captioning files
  - Toggle end-user agreement and privacy policy
  - Settings for rendering Markdown, HTML and MathJax in metadata
  - Controlled Vocabularies in Search Filters
  - Universal (Server-side Rendering) Settings
  - Debug Settings

Overview

As the DSpace 7 User Interface is built on Angular.io, it aligns with many of the best practices of that platform & the surrounding community. One example is that our UI uses the TypeScript language. That said, you do NOT need to be deeply familiar with TypeScript to edit themes or other configuration.

In DSpace 7.2 and later, the UI Configuration format changed to support runtime configuration loading

As of DSpace 7.2, the UI configuration format has changed to YAML in order to support runtime configuration. This means that reloading configurations now simply requires restarting the UI (which generally takes a few seconds).

In DSpace 7.1 and below, you had to rebuild the UI anytime you modified a configuration setting. The UI configuration format was Typescript which required recompiling each time a setting changed.

Configuration File Format

In DSpace 7.2 and above, the Configuration format is now YAML and is located at `./config/config.*.yaml`.

In DSpace 7.1 and 7.0, the Configuration format was a Typescript file and was located at `./src/environments/environment.*.ts`. The structure of this file was essentially a JSON like format.

If you are upgrading from 7.0 or 7.1 to 7.2 (or later), you will either need to migrate your old configuration file (from Typescript to YAML) or start fresh. You can migrate your old (7.0 or 7.1) "environment.*.ts" configuration file to the new "config.*.yml" format (see below).

Migrate environment file to YAML

These instructions will allow you to migrate your old (7.0 or 7.1) "environment.*.ts" configuration file to the new "config.*.yml" format.

1. First, you will need to make minor modifications to the old "environment.*.ts" configuration file(s) to ensure it no longer imports any other files:

Replace all imports in environment.*.ts

```
// (1) FIRST, you must comment out or remove all 4 imports at the top of the file. For example:

//import { GlobalConfig } from '../config/global-config.interface';
//import { NotificationAnimationsType } from '../app/shared/notifications/models/notification-animations-type';
//import { BrowseByType } from '../app/browse-by/browse-by-switcher/browse-by-decorator';
//import { RestRequestMethod } from '../app/core/data/rest-request-method';

// (2) SECOND, replace those 4 imported objects with these 4 objects.
// Paste this code at the top of the file where those imports were.

interface GlobalConfig { }

enum NotificationAnimationsType {
    Fade = 'fade',
    FromTop = 'fromTop',
    FromRight = 'fromRight',
    FromBottom = 'fromBottom',
    FromLeft = 'fromLeft',
    Rotate = 'rotate',
    Scale = 'scale'
}

enum BrowseByType {
    Title = 'title',
    Metadata = 'metadata',
    Date = 'date'
}

enum RestRequestMethod {
    GET = 'GET',
    POST = 'POST',
    PUT = 'PUT',
    DELETE = 'DELETE',
    OPTIONS = 'OPTIONS',
    HEAD = 'HEAD',
    PATCH = 'PATCH'
}
```

2. Now, you are ready to run the "yarn env:yaml" command to transform this old configuration into the new format.

Migrate from environment.*.ts to config.*.yml

```
yarn env:yaml [relative-path-to-environment.ts] [optional-relative-path-to-YAML]

# For example, from the 7.2 (or above) root directory, run this:
# yarn env:yaml relative/path/to/old/environment.prod.ts config/config.prod.yml
```

3. Finally, you should replace the old environment.*.ts config file(s) with the stock versions. They continue to provide default configuration values, but customization should be done in the YAML files. If you had created *additional* environment files, those can be deleted.

Configuration Override

In 7.2 or above

Starting in 7.2, if you make a configuration update, you only need to restart the frontend. There is no need to rebuild unless you have made code changes in "/src" directory or similar.

The UI configuration files reside in the `./config/` folder in the Angular UI source code. The default configuration is provided in `config.yml`.

To change the default configuration values, you simply create (one or more) local files that override the parameters you need to modify. You can use `config.example.yml` as a starting point.

- For example, create a new `config.dev.yml` file in `config/` for a `development` environment;

- For example, create a new `config.prod.yml` file in `config/` for a `production` environment;

Configurations can also be overridden via one of the following

- Using Environment variables. All environment variables MUST (1) be prefixed with "DSPACE_", (2) use underscores as separators (no dots allowed), and (3) use all uppercase. Some examples are below:

```
# "ui" settings environment variables
ui.ssl => DSPACE_UI_SSL # Whether the angular application uses SSL [true/false]
ui.host => DSPACE_UI_HOST # The host name
ui.port => DSPACE_UI_PORT # The port number
ui.nameSpace => DSPACE_UI_NAMESPACE # The namespace
ui.baseUrl => DSPACE_UI_BASEURL  # The Base URL for SSR

# "rest" settings environment variables
rest.ssl => DSPACE_REST_SSL # Whether the angular REST uses SSL [true/false]
rest.host => DSPACE_REST_HOST # The host name of the REST application
rest.port => DSPACE_REST_PORT # The port number of the REST application
rest.nameSpace => DSPACE_REST_NAMESPACE # The namespace of the REST application

# Other examples
defaultLanguage => DSPACE_DEFAULTLANGUAGE
mediaViewer.video => DSPACE_MEDIAVIEWER_VIDEO

# Multi-valued setting examples
# If a setting can have multiple values (e.g. theme names), then use an index number (starting with zero)
# to specify the multiple values.
# The below example is equivalent to:
# themes:
#   - name: 'dspace'
#   - name: 'mytheme'
#     handle: '10673/123'
DSPACE_THEMES_0_NAME = 'dspace'
DSPACE_THEMES_1_NAME = 'mytheme'
DSPACE_THEMES_1_HANDLE = '10673/123'
```

- Or, by creating a `.env` (environment) file in the project root directory and setting the environment variables in that location.

The override priority ordering is as follows (with items listed at the top overriding all other settings)

1. Environment variables
2. The `.env` file
3. The `./config/config.prod.yml`, `./config/config.dev.yml` or `./config/config.test.yml` files (depending on current mode)
4. The `./config/config.yml` file
5. The hardcoded defaults in `./src/config/default-app-config.ts`

In 7.1 or 7.0

The UI configuration files reside in the ./`src/environments/` folder in the Angular UI source code. The default configuration are in `environment.common.ts` in that directory.

To change the default configuration values, you simply create (one or more) local files that override the parameters you need to modify. You can use `environment.template.ts` as a starting point.

- For example, create a new `environment.dev.ts` file in `src/environments/` for a `development` environment;
- For example, create a new `environment.prod.ts` file in `src/environments/` for a `production` environment;

The "ui" and "rest" sections of the configuration may also be overridden separately via one of the following

- By setting any of the following environment variables in your system:

```
# "ui" settings environment variables
DSPACE_HOST # The host name of the angular application
DSPACE_PORT # The port number of the angular application
DSPACE_NAMESPACE # The namespace of the angular application
DSPACE_SSL # Whether the angular application uses SSL [true/false]

# "rest" settings environment variables
DSPACE_REST_HOST # The host name of the REST application
DSPACE_REST_PORT # The port number of the REST application
DSPACE_REST_NAMESPACE # The namespace of the REST application
DSPACE_REST_SSL # Whether the angular REST uses SSL [true/false]
```

- Or, by creating a `.env` (environment) file in the project root directory and setting the environment variables in that location.

The override priority ordering is as follows (with items listed at the top overriding all other settings)

1. Environment variables
2. The ".env" file
3. The "environment.prod.ts", "environment.dev.ts" or "environment.test.ts"
4. The "environment.common.ts"

Configuration Reference

The following configurations are available in `./src/environments/environment.common.ts` These settings may be overridden as described above

Production Mode

Only valid for 7.1 or 7.0

As of 7.2 and above, Angular production mode is automatically enabled whenever you are running the app in Production mode (NODE_ENV=production, or 'yarn start:prod' or 'yarn serve:ssr'). Angular production mode is automatically disabled when you are running the app in Development mode (NODE_ENV=development, or 'yarn start:dev" or 'yarn serve')

When Production mode is enabled, this enables Angular's runtime production mode and compresses the built application. This should always be enabled in Production scenarios.

```
production: true
```

UI Core Settings

The "ui" (user interface) section defines where you want Node.js to run/respond. It may correspond to your primary/public URL, but it also may not (if you are running behind a proxy). In this example, we are setting up our UI to just use localhost, port 4000. This is a common setup for when you want to use Apache or Nginx to handle HTTPS and proxy requests to Node.js running on port 4000.

Format for 7.2 or later (config.*.yml)

```
ui:
  ssl: false
  host: localhost
  port: 4000
  # NOTE: Space is capitalized because 'namespace' is a reserved string in TypeScript
  nameSpace: /
  # [New in 7.6.7+] Specify the public URL that this user interface responds to. This corresponds to the
"dspace.ui.url" property in your backend's local.cfg.
  # SSR is only enabled when the client's "Host" HTTP header matches this baseUrl. The baseUrl is also used for
redirects and SEO links (in robots.txt).
  baseUrl: http://localhost:4000
  # The rateLimiter settings limit each IP to a 'max' of 500 requests per 'windowMs' (1 minute).
  rateLimiter:
    windowMs: 60000 # 1 minute
    max: 500 # limit each IP to 500 requests per windowMs
```

Format for 7.1 or 7.0 (environment.*.ts)

```
ui: {
  ssl: false,
  host: 'localhost',
  port: 4000,
  // NOTE: Space is capitalized because 'namespace' is a reserved string in TypeScript
  nameSpace: '/',
  // The rateLimiter settings limit each IP to a "max" of 500 requests per "windowMs" (1 minute).
  rateLimiter: {
    windowMs: 1 * 60 * 1000,   // 1 minute
    max: 500 // limit each IP to 500 requests per windowMs
  }
},
```

The "rateLimiter" sub-section can be used to protect against a DOS (denial of service) attack when the UI is processed on the server side (i.e. server-side rendering). Default settings are usually OK. In Angular, server-side rendering occurs to support better Search Engine Optimization (SEO), as well as to support clients which cannot use Javascript. See the next section for more details.

Sub-path in frontend URL: When using a subpath (nameSpace) in your UI server base URL (e.g. "http://localhost:4000/mysite/" instead of "http://localhost:4000/"), you must make sure that the URL **without** the subpath is added to the `rest.cors.allowed-origins` list in `[dspace]/config/modules/rest.cfg` or the `local.cfg` override. The default value used for this configuration assumes that Origin and DSpace URL are identical, but CORS origins do not contain a subpath. Without this change you will see CORS policy errors preventing communication between the frontend and backend servers.

Server Side Rendering (SSR) Settings

The "universal" section defines configurations related to Angular's Server Side Rendering (SSR) (previously called Angular Universal). DSpace enables Angular SSR by default as required for Search Engine Optimization (SEO), as well as to support clients which cannot use Javascript. The default settings attempt to provide a balance between SEO and performance. However, you may choose to customize this behavior based on your local needs.

config.*.yml

```
universal:
  # (7.6.2 and later) Whether to tell Angular to inline "critical" styles into the server-side rendered HTML.
  # Determining which styles are critical is a relatively expensive operation; this option is
  # disabled (false) by default to boost server performance at the expense of loading smoothness.
  inlineCriticalCss: false
  # (7.6.3 only) Path prefixes to enable SSR for. By default these are limited to paths of primary DSpace
objects listed in the DSpace sitemap.
  # Paths are matched based on whether they "start with" a string in this configuration.  Wildcards are not
supported.
  # To disable this feature, specify [ '/' ], as that will result in all paths being enabled for SSR.
  paths: [ '/home', '/items/', '/entities/', '/collections/', '/communities/', '/bitstream/', '/bitstreams/', '
/handle/' ]
  # (7.6.4 and later) Path prefixes to *disable* SSR for. By default, excludes community and collection browse,
global browse,
  # global search, community list, statistics and various administrative tools.
  # The defined patterns will be run as regexes against the path of the page to check if SSR is allowed.
  # If the path matches any of the regexes it will be served directly in CSR (client side rendering).
  # NOTE: This configuration *replaces* the "paths" setting that existed in versions 7.6.3 and 8.1.
  excludePathPatterns:
    - pattern: "^/communities/[a-f0-9-]{36}/browse(/.*)?$"
      flag: "i"
    - pattern: "^/collections/[a-f0-9-]{36}/browse(/.*)?$"
      flag: "i"
    - pattern: "^/browse/"
    - pattern: "^/search$"
    - pattern: "^/community-list$"
    - pattern: "^/admin/"
    - pattern: "^/processes/?"
    - pattern: "^/notifications/"
    - pattern: "^/statistics/?"
    - pattern: "^/access-control/"
    - pattern: "^/health$"
  # (7.6.3 and later) Whether to enable rendering of Search component in SSR.
  # If set to true the component will be included in the HTML returned from the server side rendering.
  # If set to false the component will not be included in the HTML returned from the server side rendering.
  enableSearchComponent: false
  # (7.6.3 and later) Whether to enable rendering of Browse component on SSR.
  # If set to true the component will be included in the HTML returned from the server side rendering.
  # If set to false the component will not be included in the HTML returned from the server side rendering.
  enableBrowseComponent: false
  # (7.6.3 and later) Enable state transfer from the server-side application to the client-side application.
(Defaults to true)
  # Note: When using an external application cache layer, it's recommended not to transfer the state to avoid
caching it.
  # Disabling it ensures that dynamic state information is not inadvertently cached, which can improve security
and
  # ensure that users always use the most up-to-date state.
  transferState: true
  # (7.6.3 and later) When a different REST base URL is used for the server-side application, the generated
state contains references to
  # REST resources with the internal URL configured. By default, these internal URLs are replaced with public
URLs.
  # Disable this setting to avoid URL replacement during SSR. In this the state is not transferred to avoid
security issues.
  replaceRestUrl: true
```

The "paths" setting defines which DSpace pages will be processed in server-side rendering. For proper Search Engine Optimization, you should ensure this "paths" setting includes all URL paths that you want to be indexed by search engine bots. Any paths **not** listed in this setting will be inaccessible to most bots/crawlers (at least any that cannot process Javascript). The default "paths" settings are listed above, and they correspond to every page which is listed in the Sitemaps or may be of interest to search engines. DSpace purposefully does not add "/search" or "/browse" pages to these paths because bot requests to those pages may result in performance issues or require a larger amount of CPU to process the SSR. Keep in mind that values of "paths" are matched against the start of a path, so "/items/" will match every Item page in the system.

The "enableSearchComponent" and "enableBrowseComponent" settings control whether the Angular Components related to search/browse are enabled for SSR or not. These components are used in many different pages within the DSpace site, and these configurations are obeyed site wide. For example, while the "/communities/" and "/collections/" pages do undergo SSR by default, both of these pages also include embedded browse options as separate tabs. Setting "enableBrowseComponent: false" will ensure that the browse tabs for all Communities and Collections are **not** processed by SSR. Similarly, some "/entities/" pages (e.g. Person Entities) may include an embedded search section on the entity page. Setting "enableSearchComponent: false" will ensure that embedded search section for all Entities is **not** processed by SSR.

The "transferState" and "replaceRestUrl" settings are ONLY used if you've set the "ssrBaseUrl" setting the REST API Settings below (see next section). When the "ssrBaseUrl" is specified, then a different REST API URL is being used for all SSR requests. Setting "transferState" to true (default vaule) will ensure that the state of the server-side application will be transferred to the client-side after SSR completes. This is recommended behavior unless you are using an external cache layer. Setting "replaceRestUrl" to true (default value) will ensure that all URLs returned in the JSON response of an SSR request to the "ssrBaseUrl" will be updated to use the public REST API URL (set by the "ssl", "host", "port" and "nameSpace" in the REST API Settings). This is recommended behavior to ensure the client-side application has access to the public URLs you can use. Setting "replaceRestUrl" to false will disable this URL replacement, but will also disable "transferState" to ensure the state is not transferred while including potentially "private" URLs.

REST API Settings

The "rest" (REST API) section defines which REST API the UI will use. The REST settings MUST correspond to the primary URL of the backend. Usually, this means they must be kept in sync
with the value of `dspace.server.url` in the backend's `local.cfg`. See Configuration Reference.

This example is valid if your Backend is publicly available at https://mydspace.edu/server/ . Keep in mind that the "port" must always be specified even if it's a standard port (i.e. port 80 for HTTP and port 443 for HTTPS).

Format for 7.2 or later (config.*.yml)

```
rest:
  ssl: true
  host: mydspace.edu
  port: 443
  # NOTE: Space is capitalized because 'namespace' is a reserved string in TypeScript
  nameSpace: /server
  # (7.6.3 and later) OPTIONAL: Provide a different REST API URL to be used during SSR execution.
  # It must contain the whole URL including protocol, server port and server namespace
  ssrBaseUrl: http://localhost:8080/server
```

Format for 7.1 or 7.0 (environment.*.ts)

```
rest: {
  ssl: true,
  host: 'api.mydspace.edu',
  port: 443,
  // NOTE: Space is capitalized because 'namespace' is a reserved string in TypeScript
  nameSpace: '/server'
},
```

The "ssl", "host", "port" and "nameSpace" settings are all **required** , and are used to construct the URL used to contact the REST API. As noted above, these four settings should be kept in sync with the value of `dspace.server.url` in the backend's `local.cfg`

The "ssrBaseUrl" provides the option to use a **different** URL for the REST API when performing Server Side Rendering (SSR) (see prior section). This is not enabled by default, but it can provide potential SSR performance benefits, as it allows all SSR requests to bypass DNS lookup. For example, the primary REST API settings (ssl, host, port, nameSpace) should always reference a public URL like "https://mydspace.edu/server". But, you could set the "ssrBaseUrl" to a localhost URL (e.g. "http://localhost:8080/server") if your REST API is running on the same machine as the User Interface. This would result in all client-side code (running in the user's browser) accessing the REST API via the public URL, while the server-side code (triggered by SSR) would access the REST API via the localhost URL. A few important tips:

- The "ssrBaseUrl" need not always be a localhost URL, but it **must** be accessible to the machine where the User Interface's server-side code is running.
- When the "ssrBaseUrl" is specified, you **must also set** the corresponding `dspace.server.ssr.url` in the backend's local.cfg. See Configuration Reference.
- When the "ssrBaseUrl" is specified, there are additional "ssr" settings available including "transferState" and "replaceRestUrl". See Server Side Rendering (SSR) above.

Cache Settings - General

The "cache" section controls how long objects/responses will remain in the UI cache. The defaults should be OK for most sites.

Format for 7.2 or later (config.*.yml)

```
cache:
  # NOTE: how long should objects be cached for by default
  msToLive:
    default: 900000 # 15 minutes
  # Default 'Cache-Control' HTTP Header to set for all static content (including compiled *.js files)
  # Defaults to one week. This lets a user's browser know that it can cache these files for one week,
  # after which they will be "stale" and need to be redownloaded.
  control: max-age=604800 # one week
  # (New in 7.6.7+) These static files should not be cached (paths relative to dist/browser, including the
leading slash)
  noCacheFiles:
    - '/index.html'
  autoSync:
    defaultTime: 0
    maxBufferSize: 100
    timePerMethod:
      PATCH: 3 # time in seconds
```

Format for 7.1 or 7.0 (environment.*.ts)

```
cache: {
  // NOTE: how long should objects be cached for by default
  msToLive: {
    default: 15 * 60 * 1000, // 15 minutes
  },
  control: 'max-age=60', // revalidate browser
  autoSync: {
    defaultTime: 0,
    maxBufferSize: 100,
    timePerMethod: {[RestRequestMethod.PATCH]: 3} as any // time in seconds
  }
},
```

Cache Settings - Server Side Rendering (SSR)

Available in 7.5 or later

Caching options are also available for the User Interface's "server-side rendering" (which uses Angular Universal). Server-side rendering is used to pre-generate full HTML pages and pass those back to users. This is necessary for Search Engine Optimization (SEO) as some web crawlers cannot use Javascript. It also can be used to immediately show the first HTML page to users while the Javascript app loads in the user's browser.

While server-side-rendering is highly recommended on all sites, it can result in Node.js having to pre-generate many HTML pages at once when a site has a large number of simultaneous users/bots. This may cause Node.js to spend a lot of time processing server-side-rendered content, slowing down the entire site.

Therefore, DSpace provides some basic caching of server-side rendered pages, which allows the same pre-generated HTML to be sent to many users /bots at once & decreases the frequency of server-side rendering.

Two cache options are provide: `botCache` and `anonymousCache` . As the names suggest, the botCache is used for known web crawlers / bots, while the anonymousCache may be used for all anonymous (non-authenticated) users. By default, only the botCache is enabled. But highly active sites may wish to enable the anonymousCache as well, since it can provide users with a more immediate response when they encounter cached pages.

Keep in mind, when the "anonymousCache" is enabled, this means that all non-authenticated users will utilize this cache. This cache can result in massive speed improvements (for initial page load), as the majority of users may be interacting with cached content. However, these users may occasionally encounter cached pages which are outdated or "stale" (based on values of "timeToLive" and "allowStale"). This means that these users will not immediately see new updates or newly added content (Communities, Collections, Items) until the cache has refreshed itself. That said, when "timeToLive" is set to a low value (like 10 seconds), this risk is minimal for highly active pages/content.

config.*.yml

```
cache:
  ...
  serverSide:
    # Set to true to see all cache hits/misses/refreshes in your console logs. Useful for debugging SSR caching
issues.
    debug: false
    # When enabled (i.e. max > 0), known bots will be sent pages from a server side cache specific for bots.
    # (Keep in mind, bot detection cannot be guarranteed. It is possible some bots will bypass this cache.)
    botCache:
      # Maximum number of pages to cache for known bots. Set to zero (0) to disable server side caching for
bots.
      # Default is 1000, which means the 1000 most recently accessed public pages will be cached.
      # As all pages are cached in server memory, increasing this value will increase memory needs.
      # Individual cached pages are usually small (<100KB), so max=1000 should only require ~100MB of memory.
      max: 1000
      # Amount of time after which cached pages are considered stale (in ms). After becoming stale, the cached
      # copy is automatically refreshed on the next request.
      # NOTE: For the bot cache, this setting may impact how quickly search engine bots will index new content
on your site.
      # For example, setting this to one week may mean that search engine bots may not find all new content for
one week.
      timeToLive: 86400000 # 1 day
      # When set to true, after timeToLive expires, the next request will receive the *cached* page & then re-
render the page
      # behind the scenes to update the cache. This ensures users primarily interact with the cache, but may
receive stale pages (older than timeToLive).
      # When set to false, after timeToLive expires, the next request will wait on SSR to complete & receive a
fresh page (which is then saved to cache).
      # This ensures stale pages (older than timeToLive) are never returned from the cache, but some users will
wait on SSR.
      allowStale: true
    # When enabled (i.e. max > 0), all anonymous users will be sent pages from a server side cache.
    # This allows anonymous users to interact more quickly with the site, but also means they may see slightly
    # outdated content (based on timeToLive)
    anonymousCache:
      # Maximum number of pages to cache. Default is zero (0) which means anonymous user cache is disabled.
      # As all pages are cached in server memory, increasing this value will increase memory needs.
      # Individual cached pages are usually small (<100KB), so a value of max=1000 would only require ~100MB of
memory.
      max: 0
      # Amount of time after which cached pages are considered stale (in ms). After becoming stale, the cached
      # copy is automatically refreshed on the next request.
      # NOTE: For the anonymous cache, it is recommended to keep this value low to avoid anonymous users seeing
outdated content.
      timeToLive: 10000 # 10 seconds
      # When set to true, after timeToLive expires, the next request will receive the *cached* page & then re-
render the page
      # behind the scenes to update the cache. This ensures users primarily interact with the cache, but may
receive stale pages (older than timeToLive).
      # When set to false, after timeToLive expires, the next request will wait on SSR to complete & receive a
fresh page (which is then saved to cache).
      # This ensures stale pages (older than timeToLive) are never returned from the cache, but some users will
wait on SSR.
      allowStale: true
```

Authentication Settings

The "auth" section provides some basic authentication-related settings. Currently, it's primarily settings related to when a session timeout warning will be showed to your users, etc.

Format for 7.2 or later (config.*.yml)

```
auth:
  # Authentication UI settings
  ui:
    # the amount of time before the idle warning is shown
    timeUntilIdle: 900000 # 15 minutes
    # the amount of time the user has to react after the idle warning is shown before they are logged out.
    idleGracePeriod: 300000 # 5 minutes
  # Authentication REST settings
  rest:
    # If the rest token expires in less than this amount of time, it will be refreshed automatically.
    # This is independent from the idle warning. Defaults to automatic refresh when the token will
    # expire within 2 minutes. Because token expires after 30 minutes by default, this means automatic
    # refresh would occur every ~28 minutes.
    timeLeftBeforeTokenRefresh: 120000 # 2 minutes
```

Format for 7.1 or 7.0 (environment.*.ts)

```
auth: {
  // Authentication UI settings
  ui: {
    // the amount of time before the idle warning is shown
    timeUntilIdle: 15 * 60 * 1000, // 15 minutes
    // the amount of time the user has to react after the idle warning is shown before they are logged out.
    idleGracePeriod: 5 * 60 * 1000, // 5 minutes
  },
  // Authentication REST settings
  rest: {
    // If the rest token expires in less than this amount of time, it will be refreshed automatically.
    // This is independent from the idle warning.
    timeLeftBeforeTokenRefresh: 2 * 60 * 1000, // 2 minutes
  },
},
```

Form Settings

The "form" section provides basic settings for any forms displayed in the UI. At this time, these settings only include a validatorMap, which is not necessary to modify for most sites

Format for 7.2 or later (config.*.yml)

```
form:
  # (7.5 and above) Whether to enable "spellcheck" attribute of textareas in forms.
  spellCheck: true
  # NOTE: Map server-side validators to comparative Angular form validators
  validatorMap:
    required: required
    regex: pattern
```

Format for 7.1 or 7.0 (environment.*.ts)

```
form: {
  // NOTE: Map server-side validators to comparative Angular form validators
  validatorMap: {
    required: 'required',
    regex: 'pattern'
  }
},
```

Notification Settings

The "notifications" section provides options related to where user notifications will appear in your UI. By default, they appear in the top right corner, and timeout after 5 seconds.

Format for 7.2 or later (config.*.yml)

```
notifications:
  rtl: false
  position:
    - top
    - right
  maxStack: 8
  # NOTE: after how many seconds notification is closed automatically. If set to zero notifications are not
closed automatically
  timeOut: 5000 # 5 second
  clickToClose: true
  # NOTE: 'fade' | 'fromTop' | 'fromRight' | 'fromBottom' | 'fromLeft' | 'rotate' | 'scale'
  animate: scale
```

Format for 7.1 or 7.0 (environment.*.ts)

```
notifications: {
  rtl: false,
  position: ['top', 'right'],
  maxStack: 8,
  // NOTE: after how many seconds notification is closed automatically. If set to zero notifications are not
closed automatically
  timeOut: 5000, // 5 second
  clickToClose: true,
  // NOTE: 'fade' | 'fromTop' | 'fromRight' | 'fromBottom' | 'fromLeft' | 'rotate' | 'scale'
  animate: NotificationAnimationsType.Scale
},
```

The set of valid animations can be found in the NotificationAnimationsType, and are implemented in `./src/shared/animations/`

Submission Settings

The "submission" section provides some basic Submission/Deposit UI options. These allow you to optionally enable an autosave (disabled by default), and custom styles/icons for metadata fields or authority confidence values.

Format for 7.2 or later (config.*.yml)

```
submission:
  autosave:
    # NOTE: which metadata trigger an autosave
    metadata: []
    # NOTE: after how many time (milliseconds) submission is saved automatically
    # eg. timer: 300000 # 5 minutes
    timer: 0
  icons:
    metadata:
      # NOTE: example of configuration
      #   # NOTE: metadata name
      # - name: dc.author
      #   # NOTE: fontawesome (v5.x) icon classes and bootstrap utility classes can be used
      #   style: fas fa-user
      - name: dc.author
        style: fas fa-user
      # default configuration
      - name: default
        style: ''
    authority:
      confidence:
        # NOTE: example of configuration
        #   # NOTE: confidence value
        # - name: dc.author
        #   # NOTE: fontawesome (v5.x) icon classes and bootstrap utility classes can be used
        #   style: fa-user
        - value: 600
          style: text-success
        - value: 500
          style: text-info
        - value: 400
          style: text-warning
        # default configuration
        - value: default
          style: text-muted
```

Format for 7.1 or 7.0 (environment.*.ts)

```
submission: {
    autosave: {
      // NOTE: which metadata trigger an autosave
      metadata: [],
      /**
       * NOTE: after how many time (milliseconds) submission is saved automatically
       * eg. timer: 5 * (1000 * 60); // 5 minutes
       */
      timer: 0
    },
    icons: {
      metadata: [
        /**
         * NOTE: example of configuration
         * {
         *    // NOTE: metadata name
         *    name: 'dc.author',
         *    // NOTE: fontawesome (v5.x) icon classes and bootstrap utility classes can be used
         *    style: 'fa-user'
         * }
         */
        {
          name: 'dc.author',
          style: 'fas fa-user'
        },
        // default configuration
        {
          name: 'default',
          style: ''
        }
      ],
      authority: {
        confidence: [
          /**
           * NOTE: example of configuration
           * {
           *    // NOTE: confidence value
           *    value: 'dc.author',
           *    // NOTE: fontawesome (v4.x) icon classes and bootstrap utility classes can be used
           *    style: 'fa-user'
           * }
           */
          {
            value: 600,
            style: 'text-success'
          },
          {
            value: 500,
            style: 'text-info'
          },
          {
            value: 400,
            style: 'text-warning'
          },
          // default configuration
          {
            value: 'default',
            style: 'text-muted'
          },

        ]
      }
    }
  },
```

Language Settings

The "defaultLanguage" and "languages" sections allow you to customize which languages to support in your User Interface. See also Multilingual Support.

Format for 7.2 or later (config.*.yml)

```
#  Default Language in which the UI will be rendered if the user's browser language is not an active language
defaultLanguage: en

# Languages. DSpace Angular holds a message catalog for each of the following languages.
# When set to active, users will be able to switch to the use of this language in the user interface.
# All out of the box language packs may be found in the ./src/assets/i18n/ directory
languages:
  - code: en
    label: English
    active: true
  - code: cs
    label: eština
    active: true
  - code: de
    label: Deutsch
    active: true
  - ...
```

Format for 7.1 or 7.0 (environment.*.ts)

```
// Default Language in which the UI will be rendered if the user's browser language is not an active language
defaultLanguage: 'en',
// Languages. DSpace Angular holds a message catalog for each of the following languages.
// When set to active, users will be able to switch to the use of this language in the user interface.
languages: [{
  code: 'en',
  label: 'English',
  active: true,
}, {
  code: 'de',
  label: 'Deutsch',
  active: true,
},
...
],
```

The DSpace UI requires that a corresponding language pack file (named with the language code and ending in ".json5") be placed in `./src/assets/i18n/`. See also DSpace 7 Translation - Internationalization (i18n) - Localization (l10n) for information about how to create and contribute these files.

Browse By Settings

In 7.2 or above, the "browseBy" section only provides basic UI configurations for "Browse by" pages (/browse path). The "Browse by" options that appear in the "All of DSpace" header menu *are determined dynamically from the REST API*. This allows the UI to change dynamically based on the configured browse indexes in Discovery.

Format for 7.2 or later (config.*.yml)

```
browseBy:
  # Amount of years to display using jumps of one year (current year - oneYearLimit)
  oneYearLimit: 10
  # Limit for years to display using jumps of five years (current year - fiveYearLimit)
  fiveYearLimit: 30
  # The absolute lowest year to display in the dropdown (only used when no lowest date can be found for all
items)
  defaultLowerLimit: 1900
  # If true, thumbnail images for items will be added to BOTH search and browse result lists. (default: true)
  showThumbnails: true
  # The number of entries in a paginated browse results list.
  # Rounded to the nearest size in the list of selectable sizes on the settings menu.
  pageSize: 20

# NOTE: The "types" section no longer exists, as it is determined dynamically via the REST API
```

NOTE: The "pageSize" configuration will always round to the closest "pageSizeOptions" value listed in "page-component-options.model.ts"

In 7.1 or 7.0, the "browseBy" section allowed you to customize which "Browse by" options appear in the "All of DSpace" header menu at the top of your DSpace site. The "id" MUST correspond to the *name* of a valid Browse index available from your REST API (see documentation on the REST API /api /discover/browses endpoint). It is possible to configure additional indexes on the Backend using Discovery, and any configured index appears in your REST API.

Format for 7.1 or 7.0 (environment.*.ts)

```
browseBy: {
  // Amount of years to display using jumps of one year (current year - oneYearLimit)
  oneYearLimit: 10,
  // Limit for years to display using jumps of five years (current year - fiveYearLimit)
  fiveYearLimit: 30,
  // The absolute lowest year to display in the dropdown (only used when no lowest date can be found for all
items)
  defaultLowerLimit: 1900,
  // List of all the active Browse-By types
  // Adding a type will activate their Browse-By page and add them to the global navigation menu,
  // as well as community and collection pages
  // Allowed fields and their purpose:
  //    id:             The browse id to use for fetching info from the rest api
  //    type:           The type of Browse-By page to display
  //    metadataField:  The metadata-field used to create starts-with options (only necessary when the type is
set to 'date')
  types: [
    {
      id: 'title',
      type: BrowseByType.Title,
    },
    {
      id: 'dateissued',
      type: BrowseByType.Date,
      metadataField: 'dc.date.issued'
    },
    {
      id: 'author',
      type: BrowseByType.Metadata
    },
    {
      id: 'subject',
      type: BrowseByType.Metadata
    }
  ]
},
```

Community-List Settings

Available in 7.4 or later

The "communityList" section allows you to configure the behavior of the "Communities & Collections" page (/community-list path), which is linked in the header.

config.*.yml

```
communityList:
  # Number of communities to list per expansion (i.e. each time you click "show more")
  pageSize: 20
```

NOTE: The "pageSize" configuration will always round to the closest "pageSizeOptions" value listed in "page-component-options.model.ts"

Homepage Settings

Available in 7.4 or later

The "homePage" section allows you to configure the behavior of the DSpace homepage (/ path).

config.*.yml

```
homePage:
  recentSubmissions:
    # The number of item showing in recent submissions list. Set to "0" to hide all recent submissions
    pageSize: 5
    # Date field to use to sort recent submissions
    sortField: 'dc.date.accessioned'
  topLevelCommunityList:
    # Number of communities to list (per page) on the home page
    # This will always round to the nearest number from the list of page sizes. e.g. if you set it to 7 it'll
use 10
    pageSize: 5
```

NOTE: The "pageSize" configuration will always round to the closest "pageSizeOptions" value listed in "page-component-options.model.ts"

Undo Settings

Both the "item" edit and "collection" edit screens allow you to undo changes within a specific time. This is controlled by these settings:

Format for 7.2 or later (config.*.yml)

```
item:
  edit:
    undoTimeout: 10000 # 10 seconds

collection:
  edit:
    undoTimeout: 10000 # 10 seconds
```

Format for 7.1 or 7.0 (environment.*.ts)

```
item: {
  edit: {
    undoTimeout: 10000 // 10 seconds
  }
},
collection: {
  edit: {
    undoTimeout: 10000 // 10 seconds
  }
},
```

Item Access Labels

Available in 7.3 or later

Item access labels allow to display for each item in search results if it is Open Access, under embargo, restricted or metadata only (does not contain any file/bitstream). This feature is disabled by default, but can be enabled in your config.*.yml.

config.*.yml

```
# Item Config
item:
  # Show the item access status label in items lists (default=false)
  showAccessStatuses: true
```

Item Page Settings

Available in 7.5 or later

The "item" section allows you to configure the behavior of the Item pages.

config.*.yml

```
item:
  ...
  bitstream:
    # Number of entries in the bitstream list in the item view page.
    pageSize: 5
```

NOTE: The "pageSize" configuration will always round to the closest "pageSizeOptions" value listed in "page-component-options.model.ts"

Theme Settings

The "themes" section allows you to configure which theme(s) are enabled for your DSpace site (with the default theme being the "dspace" one). You can enable a single theme across all pages, and/or enable specific alternative themes based on a specific Community, Collection or Item (by UUID or Handle), or based on a Regex match of a URL pattern. This allows you fine grained control over how your site looks, including the ability to customize it per Community or Collection or even per specific pages in the site. *See User Interface Customization for details of how to create a new, custom theme.*

Format for 7.2 or later (config.*.yml)

```
themes:
  # Add additional themes here. In the case where multiple themes match a route, the first one
  # in this list will get priority. It is advisable to always have a theme that matches
  # every route as the last one
  #
  # # A theme with a handle property will match the community, collection or item with the given
  # # handle, and all collections and/or items within it
  # - name: 'custom',
  #   handle: '10673/1233'
  #
  # # A theme with a regex property will match the route using a regular expression. If it
  # # matches the route for a community or collection it will also apply to all collections
  # # and/or items within it
  # - name: 'custom',
  #   regex: 'collections\/e8043bc2.*'
  #
  # # A theme with a uuid property will match the community, collection or item with the given
  # # ID, and all collections and/or items within it
  # - name: 'custom',
  #   uuid: '0958c910-2037-42a9-81c7-dca80e3892b4'
  #
  # # The extends property specifies an ancestor theme (by name). Whenever a themed component is not found
  # # in the current theme, its ancestor theme(s) will be checked recursively before falling back to default.
  # - name: 'custom-A',
  #   extends: 'custom-B',
  #   # Any of the matching properties above can be used
  #   handle: '10673/34'
  #
  # - name: 'custom-B',
  #   extends: 'custom',
  #   handle: '10673/12'
  #
  # # A theme with only a name will match every route
  # name: 'custom'
  #
  # # This theme will use the default bootstrap styling for DSpace components
  # - name: BASE_THEME_NAME
  #
  - name: dspace
    # (New in 7.6.7+) This theme's CSS will be prefetched via https://developer.mozilla.org/en-US/docs/Web/HTML
/Reference/Attributes/rel/prefetch
    # to improve peformance & style stability on initial load.
    # Can be omitted for themes that only apply to a specific handle, regex or uuid to save bandwidth.
    prefetch: true
    # Whenever this theme is active, the following tags will be injected into the <head> of the page.
    # Example use case: set the favicon based on the active theme.
    headTags:
    - tagName: link
      attributes:
        rel: icon
        href: assets/dspace/images/favicons/favicon.ico
        sizes: any
    - tagName: link
      attributes:
        rel: icon
        href: assets/dspace/images/favicons/favicon.svg
        type: image/svg+xml
    - tagName: link
      attributes:
        rel: apple-touch-icon
        href: assets/dspace/images/favicons/apple-touch-icon.png
    - tagName: link
      attributes:
        rel: manifest
        href: assets/dspace/images/favicons/manifest.webmanifest
```

Format for 7.1 or 7.0 (environment.*.ts)

```
themes: [
  // Add additional themes here. In the case where multiple themes match a route, the first one
  // in this list will get priority. It is advisable to always have a theme that matches
  // every route as the last one

  // {
  //   // A theme with a handle property will match the community, collection or item with the given
  //   // handle, and all collections and/or items within it
  //   name: 'custom',
  //   handle: '10673/1233'
  // },
  // {
  //   // A theme with a regex property will match the route using a regular expression. If it
  //   // matches the route for a community or collection it will also apply to all collections
  //   // and/or items within it
  //   name: 'custom',
  //   regex: 'collections\/e8043bc2.*'
  // },
  // {
  //   // A theme with a uuid property will match the community, collection or item with the given
  //   // ID, and all collections and/or items within it
  //   name: 'custom',
  //   uuid: '0958c910-2037-42a9-81c7-dca80e3892b4'
  // },
  // {
  //    // Using the "extends" property allows a theme to extend/borrow from an ancestor theme (by name).
  //    // Wherever a theme component is now found in this themes, its ancestor theme(s) will be checked
  //    // recursively before falling back to default.
  //    name: 'custom-A',
  //    extends: 'custom-B',
  //    // Any of the matching properties above can be used
  //    handle: 10673/34,
  // },
  // {
  //    name: 'custom-B',
  //    extends: 'custom',
  //    handle: 10673/12,
  // },
  // {
  //   // A theme with only a name will match every route
  //   name: 'custom'
  // },
  // {
  //   // This theme will use the default bootstrap styling for DSpace components
  //   name: BASE_THEME_NAME
  // },
  {
    // The default dspace theme
    name: 'dspace'
    // Whenever this theme is active, the following tags will be injected into the <head> of the page.
    // Example use case: set the favicon based on the active theme.
    headTags: [
      {
        // Insert <link rel="icon" href="assets/dspace/images/favicons/favicon.ico" sizes="any"/> into the
<head> of the page.
        tagName: 'link',
        attributes: {
          'rel': 'icon',
          'href': 'assets/dspace/images/favicons/favicon.ico',
          'sizes': 'any',
        }
      },
      ...
    ]
  },
],
```

Media Viewer Settings

The DSpace UI comes with a basic, out-of-the-box Media Viewer (disabled by default). This media viewer can support any files which have a MIME Type that *begins with* either "image/*", "video/*", or "audio/*".

Format for 7.2 or later (config.*.yml)

```
# Whether to enable media viewer for image and/or video Bitstreams (i.e. Bitstreams whose MIME type starts with
'image' or 'video').
# When "image: true", this enables a gallery viewer where you can zoom or page through images.
# When "video: true", this enables embedded video streaming.  This embedded video streamer also supports audio
files.
mediaViewer:
  image: false
  video: false
```

Uploading video captioning files

As of 7.5 (or later), the Video viewer also supports WebVTT (or VTT) Captioning. Video captioning requires that a WebVTT Caption file (.vtt) be uploaded into the DSpace Item (DSpace is not able to create or generate these .vtt files). Here's an example of how to setup captioning:

- The Item must already have a Bitstream which is a video file (in a "video/*" format) in the ORIGINAL bundle. In this example, we'll assume it is named "myVideo.mp4"
- Upload a corresponding WebVTT Caption file named "`[video-filename]-[languageCode].vtt`" to the ORIGINAL bundle.
  - For a video named "myVideo.mp4", an English caption file would be named "myVideo.mp4-en.vtt".
  - If an additional Spanish language caption file was uploaded, it should be named "myVideo.mp4-es.vtt".
  - All WebVTT Caption files MUST use two-letter ISO 639-1 Language Codes. A list of all supported Language Codes can be found in "src/app/item-page/media-viewer/media-viewer-video/language-helper.ts"
- Once the Caption file is uploaded, reload the video viewer (on the Item page). You should now see the "Captions" (or CC) option is now available. (Depending on the browser you use, this option may appear in the lower menu of the video, or require you to open an options menu.) Selecting it will enable captioning in your language of choice.

Format for 7.1 or 7.0 (environment.*.ts)

```
// Whether to enable media viewer for image and/or video Bitstreams (i.e. Bitstreams whose MIME type starts
with "image" or "video").
// For images, this enables a gallery viewer where you can zoom or page through images.
// For videos, this enables embedded video streaming
mediaViewer: {
  image: false,
  video: false,
},
```

Toggle end-user agreement and privacy policy

Available in 7.4 or later

The DSpace UI comes with basic end-user agreement and privacy policy functionality. Since release 7.4 these features can be disabled in a configuration file. More information on what disabling on of these features results in is documented in the default app configuration (see code snippet below).

config.*.yml

```
info:
  # Whether the end user agreement is required before users may use the repository.
  # If enabled, the user will be required to accept the agreement before they can use the repository.
  # If disabled, the page will not exist and no agreement is required to use the repository
  enableEndUserAgreement: false
  # Whether the privacy statement should exist or not.
  enablePrivacyStatement: false
```

By default the features are enabled.

Settings for rendering Markdown, HTML and MathJax in metadata

Available in 7.4 or later

The DSpace UI can support Markdown (using https://commonmark.org/) and MathJax (https://www.mathjax.org) in metadata field values. Both Markdown and MathJax are disabled by default.

HTML is a part of markdown, so enabling the markdown option will ensure HTML tags in metadata field values get rendered as well

```
# Whether to enable Markdown (https://commonmark.org/) and MathJax (https://www.mathjax.org/)
# display in supported metadata fields. By default, only dc.description.abstract is supported.
markdown:
  enabled: false
  mathjax: false
```

Mathjax will only be rendered if markdown is enabled, so configuring 'markdown.mathjax = true' with 'markdown.enabled = false' will have no effect.

By default, only the "dc.description.abstract" metadata supports these formats when enabled. To enable markdown for other metadata fields, a custom sub-component of the ItemPageFieldComponent has to be created for that metadata field, with the enableMarkdown field set to true. Refer to the ItemPageAbstractFieldComponent component for an example.

Controlled Vocabularies in Search Filters

Available in 7.5 or later

When using hierarchical controlled vocabularies (e.g. SRSC as described in Authority Control of Metadata Values), it's possible to search using the controlled vocabulary hierarchy via the search filters. To enable this feature, you must specify the filter and vocabulary to enable as follows:

```
# Which vocabularies should be used for which search filters
# and whether to show the filter in the search sidebar
# Take a look at the filter-vocabulary-config.ts file for documentation on how the options are obtained
vocabularies:
  - filter: 'subject'
    vocabulary: 'srsc'
    enabled: true
```

Keep in mind, the "filter" MUST be a valid search filter (e.g. subject, author) as seen on the "/api/discover/facets" REST API endpoint. The "vocabulary" MUST be a valid controlled vocabulary installed in your DSpace backend (under "[dspace]/config/controlled-vocab/" folder based on the documentation at Authority Control of Metadata Values.

When this feature is enabled, you should see a "Browse [filter] tree" link in the search filter on the search results page (and anywhere search filters are shown). This "Browse [filter] tree" link will allow you to select a search filter from within the configured hierarchical vocabulary.

Universal (Server-side Rendering) Settings

Only valid for 7.1 or 7.0

As of DSpace 7.2, these settings are no longer editable. Universal is automatically enabled at all times to support Search Engine Optimization.

The "universal" section pertains to enabling/disabling Angular Universal for Server-side rendering. DSpace requires server-side rendering to support Search Engine Optimization. When it's turned off, your site may not be able to be indexed in Google, Google Scholar and other search engines.

environment.*.ts

```
// Angular Universal settings
universal: {
  preboot: true,
  async: true,
  time: false
},
```

Debug Settings

The "debug" property allows you to turn on debugging in the Angular UI. When enabled, your environment and all Redux actions/transfers are logged to the console. This is only ever needed if you are debugging a tricky issue.

Format for 7.2 or later (config.*.yml)

```
# NOTE: will log all redux actions and transfers in console
debug: false
```

Format for 7.1 or 7.0 (environment.*.ts)

```
// NOTE: will log all redux actions and transfers in console
debug: false
```

User Interface Customization

- Angular Overview
- Theme Technologies
- Running the UI in Developer Mode
- Creating a Custom Theme
    - Theme Directories & Design Principles
    - Getting Started
    - Global style/font/color customizations
    - Customize Logo in Header
    - Customize Navigation Links in Header
    - Customize Footer
    - Customize Favicon for site or theme
    - Customize Home Page News
    - Customize the simple Item page
    - Customize other Components in your Theme
    - Customize UI labels using Internationalization (i18n) files
    - Extending other Themes
    - Adding Component Directories to your Theme
    - Removing Component Directories from your Theme
    - Debugging which theme is being used
    - Finding which component is generating the content on a page
- Additional Theming Resources

Angular Overview

The DSpace User Interface (UI) is built on the Angular.io framework. All data comes from the REST API (DSpace Backend), and the final HTML pages are generated via TypeScript.

Before getting started in customizing or branding the UI, there are some basic Angular concepts to be aware of. Y*ou do not need to know Angular or TypeScript to theme or brand the UI.* But, understanding a few basic concepts will allow you to better understand the folder structure / layout of the codebase.

Angular Components: In Angular, every webpage consists of a number of "components" which define the structure of a page. They are the main "building block" of any Angular application. Components are reusable across many pages. So, for example, there's only one "header" and "footer" component, even though they appear across all pages.

Each Component has:

- A `*.component.ts` (TypeScript) file which contains the logic & name ("selector") of the component
- A `*.component.html` (HTML) file which contains the HTML markup for the component (and possibly references to other embedded components). This is also called the "template".
    - In HTML files, components are named/referenced as HTML-like tags (e.g. `<ds-header>`, `<ds-footer>`). In DSpace's UI, every component starts with "ds-" in order to distinguish it as an out-of-the-box DSpace component.
- A `*.component.scss` (Sass / CSS) file which contains the style for the component.

If you want a deeper dive into Angular concepts of Components and Templates, see https://angular.io/guide/architecture-components

Theme Technologies

The DSpace UI uses:

- Bootstrap (v4.x) website framework for general layout & webpage components (buttons, alerts, etc)
- Sass, a CSS preprocessor, for stylesheets. Sass is very similar to CSS (an in fact, any CSS is valid Sass). But, Sass allows you to nest CSS rules & have variables and functions. For a brief overview on Sass, see https://sass-lang.com/guide
- HTML5, the latest specification of the HTML language

Familiarity with these technologies (or even just CSS + HTML) is all you need to do basic theming of the DSpace UI.

Running the UI in Developer Mode

Whenever you are testing changes in the User Interface, may wish to see you changes "live" instead of rebuilding after each change. The easiest way to achieve this is to run the User Interface locally (i.e. on localhost) in developer mode by running:

```
yarn start:dev
```

This mode has several development-specific advantages:

- UI starts more rapidly
- UI will use a separate "config.dev.yml" configuration file (in 7.1 or 7.0 this file was named "environment.dev.ts"). This lets you have development specific configs, separate from your Production settings in "config.prod.yml" (in 7.1 or 7.0 this file was named "environment.prod.ts")

- UI will automatically reload anytime you modify a file. Essentially the UI will constantly "watch" for changes (as you make them) & will reload anytime you modify a file. This lets you find issues/bugs more rapidly and also test more rapidly.

Keep in mind, you should NEVER run the UI in developer mode in production scenarios. Production mode provides much better performance and ensures your site fully supports SEO, etc.

Creating a Custom Theme

Theme Directories & Design Principles

A theme's directory should include the following files and directories

- `app/` contains the theme's Angular components and should mirror the structure of `src/app/`
- `assets/` contains the theme's custom assets, such as fonts or images
- `styles/` contains the theme's global styles
- `eager-theme.module.ts` declares the components that should be included in the app's main bundle, such as
    - *Eager components* are those that should be available immediately when first loading, such as the main parts of the homepage and components that are present on every page.
    - *Entry components* that are registered via a decorator such as `@listableObjectComponent`. These must also be included in the module's `providers`.
- `lazy-theme.module.ts` declares all the other components of the theme.

Out of the box, there are three theming layers/directories to be aware of:

- **Base Theme** (`src/app/` directories): The primary look & feel of DSpace (e.g. HTML layout, header/footer, etc) is defined by the HTML5 templates under this directory. Each HTML5 template is stored in a subdirectory named for the Angular component where that template is used. The base theme includes very limited styling (CSS, etc), based heavily on *default Bootstrap (4.x) styling*, and only allowing for minor tweaks to improve WCAG 2.1 AA accessibility.
- **Custom Theme** (`src/themes/custom` directories): This directory acts as the scaffolding or template for creating a new custom theme. It provides (empty) Angular components/templates which allow you to change the theme of individual components. Since all files are empty by default, if you enable this theme (without modifying it), it will look *identical* to the Base Theme.
- **DSpace Theme** (`src/themes/dspace` directories): This is the default theme for DSpace 7. It's a very simple example theme providing a custom color scheme, header & homepage on top of the Base Theme. It's important to note that this theme ONLY provides custom CSS/images to override our Base Theme. All HTML5 templates are included at the Base Theme level, as this ensures those HTML5 templates are also available to the Custom Theme.

The DSpace UI design principles & technologies are described in more detail at DSpace UI Design principles and guidelines

Getting Started

1. *Choose a theme to start from:* As documented above, there are two "src/theme/" directories provided out of the box: "custom" or "dspace". You should select one to use as the basis for your theme. Which you choose is up to you, but here are a few things to consider:
    a. *DSpace Theme* (`src/themes/dspace`): This is a simple, example theme for novice users. Primarily, in this theme, you can immediately customize the CSS, header & homepage components. You can add other components as needed (see "Adding Component Directories to your Theme" below).
        i. Advantages: This theme is small and simple. It provides an easy starting point / example for basic themes. Future User Interface upgrades (e.g. from 7.1 7.2) are likely to be easier because the theme is smaller in size.
        ii. Disadvantages: It has very few component directories by default. But you can always add more. See "Adding Component Directories to your Theme" below.
    b. *Custom Theme* (`src/themes/custom`): This theme provides all available theme-able components for more advanced or complex theming options. This provides you full control over everything that is theme-able in the User Interface
        i. Advantages: All theme-able components are provided in subdirectories. This makes it easier to modify the look and feel of any area of the User Interface.
        ii. Disadvantages: After creating your theme, you may wish to remove any component directories that you didn't modify (see "Removing Component Directories from your Theme" below). Generally speaking, upgrades (e.g. from 7.1 7.2) are often easier if your theme includes fewer components (as your theme may require updates if any component it references change significantly).
2. *Create your own theme folder OR edit the existing theme folder:* Either edit the theme directory in place, or copy it (and all its contents) into a new folder under `src/themes/` (choose whatever folder name you want)
3. *Register your theme folder (only necessary if you create a new folder in previous step):* Now, we need to make the UI aware of this new theme folder, before it can be used in configuration.
    a. Modify `angular.json` (in the root folder), adding your theme folder's main "theme.scss" file to the "styles" list. The below example is for a new theme folder named `src/themes/mydspacesite/`

```
"styles": [
  "src/styles/startup.scss",
  {
     "input": "src/styles/base-theme.scss",
     "inject": false,
     "bundleName": "base-theme"
  },
  ...
  {
     "input": "src/themes/mydspacesite/styles/theme.scss",
     "inject": false,
     "bundleName": "mydspacesite-theme"
  },
]
```

NOTE: the "bundleName" for your custom them MUST use the format "${folder-name}-theme". E.g. if the folder is named "src/themes/amazingtheme", then the "bundleName" MUST be "amazingtheme-theme"

4. (As of 7.3 or above) Import the new theme's `eager-theme.module.ts` in `themes/eager-themes.module.ts`. If you're switching from one theme to another, remove the old theme from the imports. Below is an example for a theme named "my-theme":

themes/eager-themes.module.ts

```
// COMMENT out the imports for any themes you are NOT using
//import { EagerThemeModule as DSpaceEagerThemeModule } from './dspace/eager-theme.module';
//import { EagerThemeModule as CustomEagerThemeModule } from './custom/eager-theme.module';

// Add a new import for your custom theme. Give its EagerThemeModule a unique name (e.g. "as [choose-a-
unique-name]").
// Make sure the path points at its "eager-theme.module.ts" (see 'from' portion of the import statement).
// NOTE: You can import multiple themes if you plan to use multiple themes
import { EagerThemeModule as MyThemeEagerThemeModule } from './my-theme/eager-theme.module';

...
@NgModule({
  imports: [
    // Again, comment out any themes you are NOT using
    //DSpaceEagerThemeModule,
    //CustomEagerThemeModule,

    // Add your custom theme's EagerThemeModule to this list
    // NOTE: you can add multiple theme's to this list if you plan to use multiple themes.
    MyThemeEagerThemeModule,
  ],
})
```

5. *Enable your theme:* Modify your `config/config.*.yml` configuration file (in 7.1 or 7.0 this file was named `src/environments/environment.*.ts`), adding your new theme to the "themes" array in that file. Pay close attention to modify the correct configuration file (e.g. modify config.dev.yml if running in dev mode, or config.prod.yml if running in prod mode). We recommend starting in "dev mode" (config.dev.yml) as this mode lets you see your changes immediately in a browser without a full rebuild of the UI – see next step.

Format for 7.2 or above (config.*.yml)

```
# In this example, we only show one theme enabled. It's possible to enable multiple (see below note)
themes:
  - name: 'mydspacesite'
```

Format for 7.1 or 7.0 (environment.*.ts)

```
// In this example, we only show one theme enabled. It's possible to enable multiple (see below note)
themes: [
 {
    name: 'mydspacesite'
 },
]
```

NOTE: The "name" used is the name of the theme's folder, so the example is for enabling a theme at `src/themes/mydspacesite/` globally. You should also comment out the default "dspace" theme, if you intend to replace it entirely.
NOTE #2: You may also choose to enable multiple themes for your site, and even specify a different theme for different Communities, Collections, Items or URL paths. See User Interface Configuration for more details on "Theme Settings"

6. *Verify your settings by starting the UI* (ideally in Dev mode): At this point, you should verify the basic settings you've made all "work". We recommend doing your theme work while running the UI in "dev mode", as the UI will auto-restart anytime you save a new change. This will allow you to quickly see the impact of each change in your browser.

```
# Start in dev mode (which uses config.dev.yml)
yarn start:dev
```

7. At this point, you can start making changes to your theme. See the following sections for examples of how to make common changes.

Global style/font/color customizations

Changes to the global Bootstrap variables or styles will apply to all pages / Angular components across the entire site.

1. *Global style changes:* All global style changes can be made in your theme's `styles` folder (e.g. `src/themes/mydspacesite/styles`). There are four main files in that folder:
    a. `_theme_sass_variable_overrides.scss` - May be used to override Bootstrap's default Sass variables. This is the file you may wish to use for **most** style changes. There are a large number of Bootstrap variables available which control everything from fonts, colors and the base style for all Bootstrap web components. For a full list of Bootstrap variables you can override in this file, see the `node_modules/bootstrap/scss/_variables.scss` file (which is installed in your source directory when you run "yarn install"). More information may also be found in the Bootstrap Sass documentation at https://getbootstrap.com/docs/4.0/getting-started/theming/#sass
    b. `_theme_css_variable_overrides.scss` - May be used to override DSpace's default CSS variables. DSpace's UI uses CSS variables for all its components. These variables all start with "--ds-*", and are listed in `src/styles/_custom_variables.scss`. You can also use this file to add your own, custom CSS variables which you want to use for your theme. If you create custom variables, avoid naming them with a "--ds-*" or a "--bs-*" prefix, as those are reserved for DSpace and Bootstrap variables.
    c. `_global-styles.scss` - May be used to modify the global CSS/SCSS for the site. This file may be used to override the default global style contained in `src/styles/_global-styles.scss`. Keep in mind, many styles can be more quickly changed by simply updating a variable in one of the above "*_variable_overrides.scss" files. So, it's often easier to use those first, where possible.
    d. `theme.scss` - This just imports all the necessary Sass files to create the theme. It's unnecessary to modify directly, unless you with to add new Sass files to your theme.
2. *Modifying the default font:* By default, DSpace uses Bootstrap's "native font stack", which just uses system UI fonts. However, the font used in your site can be quickly updated via Bootstrap variables in your theme's `_theme_sass_variable_overrides.scss` file.
    a. One option is to add a new import statement and modify the "$font-family-sans-serif" variable:

```
// Import the font (from a URL)
@import url('https://fonts.googleapis.com/css?family=Source+Sans+Pro');

// Configure Bootstrap to use this font (and list a number of backup fonts to use on various
systems)
$font-family-sans-serif: 'Source Sans Pro', -apple-system, BlinkMacSystemFont, "Segoe UI",
"Roboto", "Helvetica Neue", Arial, sans-serif, "Apple Color Emoji", "Segoe UI Emoji", "Segoe UI
Symbol" !default;
```

    b. If your font requires installing local files, you can do the following
        i. Copy your font file(s) in your theme's `assets/fonts/` folder
        ii. Create a new ".scss" file specific to your font in that folder, e.g. `assets/fonts/open-sans.scss`, and use the "@font-face" CSS rule to load that font:

open-sans.scss

```
@font-face {
  font-family: "Open Sans";
  src: url("/assets/fonts/OpenSans-Regular-webfont.woff2") format("woff2"),
       url("/assets/fonts/OpenSans-Regular-webfont.woff") format("woff");
}
```

iii. Then, import that new "open-sans.scss" file and use it in the "$font-family-sans-serif" variable

```
// Import the font via the custom SCSS file
@import '../assets/fonts/open-sans';

// Configure Bootstrap to use this font (and list a number of backup fonts to use on
various systems)
$font-family-sans-serif: 'Open Sans', -apple-system, BlinkMacSystemFont, "Segoe UI",
"Roboto", "Helvetica Neue", Arial, sans-serif, "Apple Color Emoji", "Segoe UI Emoji",
"Segoe UI Symbol" !default;
```

c. Keep in mind, as changing the font just requires adjusting Bootstrap Sass variables, there are a lot of Bootstrap guides out there that can help you make more advanced changes

3. *Modifying default color scheme:* The colors used in your site can be quickly updated via Bootstrap variables in your theme's `_theme_sass_variable_overrides.scss` file.
   a. Again, you can use entirely Bootstrap variables to adjust color schemes. See the Bootstrap Theming Colors documentation
   b. A list of all Bootstrap color variables can be found in the `node_modules/bootstrap/scss/_variables.scss` file
   c. Additional examples can be found in the out-of-the-box "dspace" theme, which adjusts the default Bootstrap colors slightly for both accessibility & to match the DSpace logo.
4. Any changes require rebuilding your UI. If you are running in "dev mode" (yarn start:dev), then the UI will restart automatically whenever changes are detected.

CSS custom properties vs. SASS variables

⚠

Support for theme switching at runtime requires that components use CSS custom properties (which vary at runtime) rather than SASS variables (which are fixed at build time). Thus, SASS variables will be undefined in individual components' stylesheets. The Bootstrap SASS variables are mapped to CSS properties for use in these places. For example, `$red` is mapped to `--bs-red` and may be referenced as `var("--bs-red")`.

Customize Logo in Header

1. Copy your logo to your theme's `assets/images/` folder. Anything in this theme folder will be deployed to `/assets/[theme-name]/images/` URL path.
2. Edit your theme's `app/header/header.component.ts` file. Swap the "templateUrl" property that your theme is using the local copy of "header.component.html"

header.component.ts

```
@Component({
  selector: 'ds-header',
  // styleUrls: ['header.component.scss'],
  styleUrls: ['../../../../app/header/header.component.scss'],
  // Uncomment the templateUrl which references the "header.component.html" file in your theme directory
  templateUrl: 'header.component.html',
  // Comment out the templateUrl which references the default "src/app/header/header.component.html"
file.
  //templateUrl: '../../../../app/header/header.component.html',
})
```

3. Your theme's version of the `header.component.html` file will be empty by default. Copy over the default HTML code from `src/app/header/header.component.html` into your version of this file.
4. Then, modify your copy of header.component.html to use your logo. In this example, we're assuming your theme name is "mytheme" and the logo file is named "my-logo.svg"

```
<header>
  <div class="container">
    <div class="d-flex flex-row justify-content-between">
      <a class="navbar-brand my-2" routerLink="/home">
        <!-- Modify the logo on the next line -->
        <img src="/assets/mytheme/images/my-logo.svg" [attr.alt]="'menu.header.image.logo' | translate"/>
      </a>
      ...
</header>
```

5. Obviously, you can also make additional modifications to the HTML of the header in this file! You'll also see that the header references several other DSpace UI components (e.g. `<ds-search-navbar>` is the search icon in the header). You can easily comment out these tags to disable them, or move them around to change where that component appears in the header.
6. Any changes require rebuilding your UI. If you are running in "dev mode" (yarn start:dev), then the UI will restart automatically whenever changes are detected.
7. NOTE: If you have a theme based on the "dspace" theme, be aware that theme places the header logo in two locations. This allows the "dspace" theme to support a single-line header (whereas the "custom" theme's header is multi-line):
    a. The Header component (*as described above*) is only used on user profile pages
    b. The Navbar component (`src/app/navbar/navbar.component.html`) is used everywhere else. The Navbar component can be customized in the same way as the Header Component. Just edit the logo path in the "navbar.component.html".

Customize Navigation Links in Header

This provides a basic example of adding in a hardcoded link to the header menu (displayed on every page in DSpace).

1. Edit your theme's existing `app/navbar/navbar.component.html` file. This file defines the entire <nav> which displays the navigation header across the entire DSpace site. While much of the content in this <nav> is loaded dynamically via other Angular components, it is possible to easily add a hardcoded link to the existing header. Find the section of this <nav> which is the , as that's where you'll want to add your new link. See inline comments in the example below.

navbar.component.html

```
<nav>
...
   <!-- This DIV is where the header links are added dynamically.
        You should see it surrounding all links in the header if you view HTML source -->
   <div id="collapsingNav" ... >
    <!-- The links themselves are in an unordered list (UL) -->
    <ul class="navbar-nav" ... >
       ...
       <!-- Add your new link at the end (or beginning) of this UL in a new LI tag -->
       <!-- NOTE: All classes used below are the same Bootstrap CSS classes used by our 'dspace' and
'custom' themes.
            You can modify them if the link doesn't look correct in your theme. -->
       <li class="nav-item d-flex align-items-center">
          <div class="text-md-center">
            <a href="http://dspace.org" class="nav-link">DSpace.org</a>
          </div>
       </li>
    </ul>
   </div>
</nav>
```

2. Obviously, you can also make additional modifications to the HTML of the header in this file, as necessary for your navigation header. Keep in mind though that anything you remove may impact the dynamic content that is pulled into this navigation header.
    a. An example is that the header logo for the "dspace" theme also exists in this same file.
3. Any changes require rebuilding your UI. If you are running in "dev mode" (yarn start:dev), then the UI will restart automatically whenever changes are detected.

Customize Footer

1. First, you'll want to decide if you want to modify just the footer's HTML, or the footer's styles (CSS/Sass), or both.
    a. If you want to modify the HTML, you'll need to create a copy of "footer.component.html" in your theme, where you place your changes.
    b. If you want to modify the styles, you'll need to create a copy of "footer.component.scss" in your theme, where you place your changes.
2. Edit your theme's `app/footer/footer.component.ts` file. Swap the "templateUrl" and "styleUrls" properties, based on which you want to modify in your theme.

footer.component.ts

```
@Component({
  selector: 'ds-footer',
  // If you want to modify styles, then...
  // Uncomment the styleUrls which references the "footer.component.scss" file in your theme's directory
  // and comment out the one that references the default "src/app/footer/footer.component.scss"
  styleUrls: ['footer.component.scss'],
  //styleUrls: ['../../../../app/footer/footer.component.scss'],
  // If you want to modify HTML, then...
  // Uncomment the templateUrl which references the "footer.component.html" file in your theme's
directory
  // and comment out the one that references the default "src/app/footer/footer.component.html"
  templateUrl: 'footer.component.html'
  //templateUrl: '../../../../app/footer/footer.component.html'
})
```

3. Now, based on what you want to modify, you will need to either update your theme's copy of `footer.component.html` or `footer.component.scss` or both.
    a. To change footer HTML: Your theme's version of the `footer.component.html` file will be empty by default. Copy over the default HTML code from `src/app/footer/footer.component.html` into your version of this file.
    b. To change footer Styles: Your theme's version of the `footer.component.scss` file will be empty by default. Copy over the default Sass code from `src/app/footer/footer.component.scss` into your version of this file.
4. Modify the HTML or Sass as you see fit.
    a. If you want to add images, add them to your theme's `assets/images` folder. Then reference them at the `/assets/[theme-name]/images/` URL path.
    b. Keep in mind, all Bootstrap variables, utility classes & styles can be used in these files. Take advantage of Bootstrap when you can do so.
5. DSpace also has a option to display a two-level footer, which is becoming more common these days. By default. DSpace just displays a small, bottom footer. But, you can enable a top footer (above that default footer) by add this line into your theme's `footer.component.ts`

footer.component.ts

```
export class FooterComponent extends BaseComponent {
   // This line will enable the top footer in your theme
   showTopFooter = true;
}
```

This top footer appears in the `footer.component.html` within a div. Notice the "`*ngIf='showTopFooter'`", which only shows that div when that variable is set to "true"

footer.component.html

```
<footer class="text-lg-start">
  <!-- This div and everything within it only displays if showTopFooter=true -->
  <div *ngIf="showTopFooter" class="top-footer">
    ...
  </div>
  <!-- The bottom footer always displays -->
  <div class="bottom-footer ...">
    ...
  </div>
</footer>
```

6. Any changes require rebuilding your UI. If you are running in "dev mode" (yarn start:dev), then the UI will restart automatically whenever changes are detected.

Customize Favicon for site or theme

Available in 7.2 or above. In 7.1 or 7.0, the only way to change the favicon is to modify the `src/assets/images/favicon.ico` file.

Each theme has the ability to add a set of (attribute-only) HTML tags in the <head> section of the page. This is useful for example to change the **favicon** based on the active theme. Whenever the theme changes, the head tags are reset. A theme can inherit head tags from the parent theme **only** if it doesn't have any head tags itself. (E.g. theme B extends theme A; if theme B does not have head tags, the head tags of theme A will be used (if any). However, if theme B does have head tags, only the tags from theme B will be used.) If none of the themes in the inheritance hierarchy have head tags configured, the head tags of the default theme (if any) will be used.

Note that a simple hardcoded favicon is set in case no head tags are currently active. The hardcoded favicon is stored at `src/assets/images/favicon.ico`. This implies that if head tags are added to a theme, the favicon should also be configured explicitly for that theme, else the behavior is undefined.

1. In the "themes" section of your `config/config.*.yml` configuration file, add (one or more) "headTags", pointing at the favicon file you want to use. For example:

```
themes:
  # The default dspace theme
  - name: dspace
    # Whenever this theme is active, the following tags will be injected into the <head> of the page.
    # Example use case: set the favicon based on the active theme.
    headTags:
      # Insert <link rel="icon" href="assets/dspace/images/favicons/favicon.ico" sizes="any"/> into the
<head> of the page.
      - tagName: link
        attributes:
          rel: icon
          href: assets/dspace/images/favicons/favicon.ico
          sizes: any
      # Insert <link rel="icon" href="assets/dspace/images/favicons/favicon.svg" type="image/svg+xml"/>
into the <head> of the page.
      - tagName: link
        attributes:
          rel: icon
          href: assets/dspace/images/favicons/favicon.svg
          type: image/svg+xml
      # Insert <link rel="apple-touch-icon" href="assets/dspace/images/favicons/apple-touch-icon.png"/>
into the <head> of the page.
      - tagName: link
        attributes:
          rel: apple-touch-icon
          href: assets/dspace/images/favicons/apple-touch-icon.png
      # Insert <link rel="manifest" href="assets/dspace/images/favicons/manifest.webmanifest"/> into the
<head> of the page.
      - tagName: link
        attributes:
          rel: manifest
          href: assets/dspace/images/favicons/manifest.webmanifest
```

2. In 7.2 or above, any changes to this configuration just require restarting your site (no rebuild necessary). In 7.1 or 7.0, you must rebuild your site after modifying the favicon.ico.
3. NOTE: If you specify multiple formats for your favicon (e.g. favicon.svg and favicon.ico), then your browser will select which one it prefers (e.g. Chrome seems to favor SVG over ICO). However, if you want to force all browser to use a single favicon, then you may wish to only specify one "icon" format in your `headTags` section.

Customize Home Page News

The primary Home page content is all included in the source code under "src/app/home-page". The News section is specifically under "src/app/home-page/home-news".

1. First, you'll want to decide if you want to modify just the Home Page News HTML, or styles (CSS/Sass), or both.
    a. If you want to modify the HTML, you'll need to create a copy of the HTML in "app/home-page/home-news/home-news.component.html" in your theme. This is where you place your changes.
    b. If you want to modify the styles, you'll need to create a copy of the CSS in "app/home-page/home-news/home-news.component.scss" in your theme. This is where you place your changes.
2. Edit your theme's `app/home-page/home-news/home-news.component.ts` file. Swap the "templateUrl" and "styleUrls" properties, based on which you want to modify in your theme.

home-news.component.ts

```
@Component({
  selector: 'ds-home-news',
  // If you want to modify styles, then...
  // Uncomment the styleUrls which references the "home-news.component.scss" file in your theme's
directory
  // and comment out the one that references the default "src/app/home-page/home-news/home-news.
component.scss"
  styleUrls: ['./home-news.component.scss'],
  //styleUrls: ['../../../../../app/home-page/home-news/home-news.component.scss'],
  // If you want to modify HTML, then...
  // Uncomment the templateUrl which references the "home-news.component.html" file in your theme's
directory
  // and comment out the one that references the default "src/app/home-page/home-news/home-news.
component.html"
  templateUrl: './home-news.component.html'
  //templateUrl: '../../../../../app/home-page/home-news/home-news.component.html'
})
```

3. Now, based on what you want to modify, you will need to either update your theme's copy of `home-news.component.html` or `home-news.component.scss` or both.
    a. To change HTML: Your theme's version of the `home-news.component.html` file will be empty by default. Copy over the default HTML code from `src/app/home-page/home-news/home-news.component.html` into your version of this file.
    b. To change Styles: Your theme's version of the `home-news.component.scss` file will be empty by default. Copy over the default Sass code from `src/app/home-page/home-news/home-news.component.scss` into your version of this file.
4. Modify the HTML or Sass as you see fit.
    a. If you want to add images, add them to your theme's `assets/images` folder. Then reference them at the `/assets/[theme-name]/images/` URL path.
    b. Keep in mind, all Bootstrap variables, utility classes & styles can be used in these files. Take advantage of Bootstrap when you can do so.
5. Any changes require rebuilding your UI. If you are running in "dev mode" (yarn start:dev), then the UI will restart automatically whenever changes are detected.

Customize the simple Item page

The "simple" Item page is the default display for an Item (when you visit any item via a URL like [dspace.ui.url]/items/[uuid]). If you want to modify the metadata fields displayed on that page by default, that can be done quite easily.

- Normal Item: The code for the simple Item page for a normal Item (i.e. not an Entity) can be found in the source code at "`src/app/item-page/simple/item-types/untyped-item/`"
- Publication Entity: If you are wanting to modify the display of Publication Entities, it has separate source code under "`src/app/item-page/simple/item-types/publication/`"

Here's the basics of modifying this page. The below examples assume you are working with a normal Item. But the same logic would work for modifying the Publication pages (you'd just need to modify it's HTML/CSS instead)

1. First, you'll want to decide if you want to modify just the Item Page HTML, or styles (CSS/Sass), or both.
    a. If you want to modify the HTML, you'll need to create a copy of the HTML in "`src/app/item-page/simple/item-types/untyped-item/untyped-item.component.html`" in your theme. This is where you place your changes.
    b. If you want to modify the styles, you'll need to create a copy of the CSS in "`src/app/item-page/simple/item-types/untyped-item/untyped-item.component.scss`" in your theme. This is where you place your changes.
2. Edit your theme's `app/item-page/simple/item-types/untyped-item/untyped-item.component.ts` file. Swap the "templateUrl" and "styleUrls" properties, based on which you want to modify in your theme. Also, MAKE SURE the "@listableObjectComponent" is using your theme... the last parameter should be the name of your theme!

untyped-item.component.ts

```
// MAKE SURE that the final parameter here is the name of your theme. This one assumes your theme is
named "custom".
@listableObjectComponent(Item, ViewMode.StandalonePage, Context.Any, 'custom')
@Component({
  selector: 'ds-untyped-item',
  // If you want to modify styles, then...
  // Uncomment the styleUrls which references the "untyped-item.component.scss" file in your theme's
directory
  // and comment out the one that references the default in "src/app/"
  styleUrls: ['./untyped-item.component.scss'],
  //styleUrls: ['../../../../../../../app/item-page/simple/item-types/untyped-item/untyped-item.
component.scss'],
  // If you want to modify HTML, then...
  // Uncomment the templateUrl which references the "untyped-item.component.html" file in your theme's
directory
  // and comment out the one that references the default "src/app/"
  templateUrl: './untyped-item.component.html',
  //templateUrl: '../../../../../../../app/item-page/simple/item-types/untyped-item/untyped-item.
component.html',
})
```

3. Now, based on what you want to modify, you will need to either update your theme's copy of `untyped-item.component.html` or `untyped-item.component.scss` or both.
    a. To change HTML: Your theme's version of the `untyped-item.component.html` file may be empty by default. Copy over the default HTML code from `src/item-page/simple/item-types/untyped-item/untyped-item.component.html` into your version of this file.
    b. To change Styles: Your theme's version of the `untyped-item.component.scss` file may be empty by default. Copy over the default Sass code from `src/item-page/simple/item-types/untyped-item/untyped-item.component.scss` into your version of this file
4. In the HTML of the Simple Item page, you'll see a number of custom "ds-" HTML-like tags which make displaying individual metadata fields easier. These tags make it easier to add/remove metadata fields on this page.
    a. `<ds-generic-item-page-field>` - This tag can be used to display the value of any metadata field (as a string).
        i. Put the name of the metadata field in the "[fields]" attribute... see existing fields as an example.
        ii. Put the i18n label you want to use for this field in the "[label]" attribute. Again, see existing fields as an example. This i18n tag MUST then be added to your "`src/assets/i18n/en.json5`" file (or corresponding file depending on your language)
        iii. For example, here's a new ds-generic-item-page-field which displays the value of the "dc.title.alternative" field with a label defined in the "
    b. `<ds-item-page-uri-field>` - This tag can be used to display the value of any metadata field as an HTML link. (The value must already be a URL)
        i. This field has the same attributes at `<ds-generic-item-page-field>` above.
    c. Some specific tags exist for other fields (e.g. `<ds-item-page-date-field>` and `<ds-item-page-abstract-field>`). These are currently separate components under "`src/app/item-page/simple/field-components/specific-field/`" directories. They are hardcoded to use a specific metadata field and label, but you could customize the components in that location.
5. To add new fields, just add new "<ds-generic-item-page-field>" tags (or similar). To remove fields, just comment them out or remove the HTML. You can also restructure the columns on that page using simple HTML and Bootstrap CSS.
6. Any changes require rebuilding your UI. If you are running in "dev mode" (yarn start:dev), then the UI will restart automatically whenever changes are detected.

NOTE: If your changes to the simple item page don't appear to be working, make sure that you have updated the `eager-theme.module.ts` to load your custom theme as described in the "Getting Started" section above! This small change is REQUIRED for the untyped-item component to work in a custom theme. You also may wish to restart your dev server to ensure nothing is being cached.

Customize other Components in your Theme

By now, if you've followed this entire guide, you'll notice a pattern! Customizing specific DSpace UI components requires just three steps:

1. *Configure your theme to use its copies of files:* Modify the corresponding `*.component.ts` in your theme.
    a. If you want to modify component style, replace the "styleUrls" in that file to point at the copy of `*.component.scss` in your theme.
    b. If you want to modify component HTML, replace the "template" in that file to point at the copy of `*.component.html` in your theme.
2. *Copy the default UI code into your theme file(s)*
    a. If you want to modify component style, copy the default `*.component.scss` code (from `src/app/`) into your theme's `component.scss` file.
    b. If you want to modify component HTML, copy the default `*.component.html` code (from `src/app/`) into your theme's `component.html` file.
3. *Modify those theme-specific files*
    a. If you want to add images, add them to your theme's `assets/images` folder. Then reference them at the `/assets/[theme-name]/images/` URL path.
    b. Keep in mind, all Bootstrap variables, utility classes & styles can be used in these files. Take advantage of Bootstrap when you can do so.

4. Remember to either rebuild the UI after each change, or run in dev mode (yarn start:dev) while you are doing theme work.

Customize UI labels using Internationalization (i18n) files

Much of the text (like headers and labels) displayed by the DSpace UI is captured in translation (language pack) files to support the use of DSpace in multiple languages. This model of multi-language support is called internationalization (abbreviated i18n). The default set of these i18n files are stored in `src/assets/i18n` and named using the language code, so `en.json5` is the English translation, `fr.json5` is the French translation, etc. In each of these files, a set of "keys" is mapped to text in the given language.

If you would like to change the text displayed in the UI, you will need to edit the i18n translation files. There are two approaches you can take:

1. You can edit the `src/assets/i18n/*.json5` file(s) directly
    a. For example, to change the label for the browse menu when viewing the UI in English (which defaults to "All of DSpace"), you would edit `src/assets/i18n/en.json5` and change the value for `menu.section.browse_global`.
2. And/or, you can create a separate *.json5 file in your theme which only lists the keys you have changed. This can keep your language changes in your theme, and will override the default keys in the `src/assets/i18n/` files. However, a specific setup is necessary, see the "Theme override approach" instructions below.

Theme override approach

The following "theme override" approach to capture i18n changes within a theme is only supported in DSpace 7.1 or above.

While editing the default i18n files directly is effective, the recommended approach is to capture i18n changes in your theme. This ensures that your changes to the default values are easy to find and review and also removes the risk of losing your changes when upgrading to newer versions of DSpace.

To capture i18n changes in your theme, you will need to:

1. Create an i18n directory under `src/themes/[theme-name]/assets`
2. For each language you would like to update, add a file to the new i18n directory following the naming scheme in the default i18n directory (en.json5 for English, fr.json5 for French, etc)
3. In each translation file add only the settings that you wish to add or override

There is an example of this configuration in the custom theme, which you can find in `src/themes/custom/assets/i18n`.

Once you have changes in place within your theme, they need to be applied by executing a script:

```
yarn merge-i18n -s src/themes/[theme-name]/assets/i18n
```

The `merge-i18n` script will merge the changes captured in your theme with the default settings, resulting in updated versions of the default i18n files. Any setting you included in your theme will override the default setting. Any new properties will be added. Files will be merged based on file name, so en.json5 in your theme will be merged with the en.json5 file in the default i18n directory.

Extending other Themes

This is only supported in 7.1 and above

Themes can extend other themes using the "extends" configuration. See User Interface Configuration for more examples.

Extending another theme means that you inherit all the settings of the extended theme. So, if the current theme does NOT specify a component style, its ancestor theme(s) will be checked recursively for their styles before falling back to the default. In other words, this "extends" setting allows for a theme to **inherit** all styles/components from the extended theme, and only override those styles/components you wish to override.

Here's a basic example:

Format for 7.2 or above (config.*.yml)

```
themes:
  # grandchild theme
  - name: custom-A
    extends: custom-B
    handle: '10673/34'
  # child theme
  - name: custom-B
    extends: custom
    handle: 10673/2
  # default theme
  - name: custom
```

Format for 7.1 or 7.0 (environment.*.ts)

```
themes: [
  // grandchild theme
  {
    name: 'custom-A',
    extends: 'custom-B',
    handle: '10673/34',
  },
  // child theme
  {
    name: 'custom-B',
    extends: 'custom',
    handle: '10673/2',
  },
  // default theme
  {
    name: 'custom',
  },
],
```

In the above examples:

- When the object at Handle '10673/2' (and any child objects) is viewed, the 'custom-B' theme will be used. By default, you'll have the same styles as the extended 'custom' theme. However, you can override individual styles in your 'custom-B' theme.
- When the object at Handle '10673/34' (and any child objects) is viewed, the 'custom-A' theme will be used. By default, your overall theme will be based on the 'custom' theme (in this case a "grandparent" theme). But, you can override those styles in your 'custom-B' theme or 'custom-A' theme.
    - The order of priority is 'custom-A', then 'custom-B', then 'custom'. If a style/component is in 'custom-A' it will be used. If not, 'custom-B' will be checked and if it's there, that version will be used. If not in either 'custom-A' or 'custom-B', then the style/component from 'custom' will be used. If the style/component is not in ANY of those themes, then the default (base theme) style will be used.

Adding Component Directories to your Theme

If you come across an Angular Component which is NOT in your theme but want to customize it, you can add it into your theme directory. This involves copying the component from the "Custom" theme over into your theme.

You can add/copy over a Component Directory as follows:

1. First, copy the Angular Component directory in question from the "Custom" theme folder (src/themes/custom) into your theme's folder. *NOTE: at this time, not all components are theme-able. So, if it doesn't exist in the "Custom" theme folder, then it may not be possible to theme.*
    a. For example, if you wanted to add the Footer Component to your theme, it can be found in the "Custom" theme at "src/themes/custom/app/footer".
    b. Copy that entire folder into your theme folder, retaining the same relative path. For example, to add the Footer Component, copy "src/themes/custom/app/footer" (and all contents) into "src/themes/[your-theme]/app/footer".
2. Now, you need to "register" that component in one of your theme's module files: lazy-theme.module.ts or eager-theme.module.ts. For performance it's best to put as many components into lazy-theme.module.ts as that means they'll only be downloaded if they're needed. Components in eager-theme.module.ts are included in the initial JS download for the app, so you should only add components there that are necessary on every page, such as the header and footer, these should be added to the `DECLARATIONS` array. You should also include components using one of our custom decorators (such as @listableObjectComponent), because those decorators need to be registered when the app starts to be able to be picked up. These should be added to the `ENTRY_COMPONENTS` array, which will both declare them as well as ensure they're loaded when the app starts.
3. Add an import of the new component file, or **copy** the corresponding import from "src/themes/custom/lazy-theme.module.ts" or "src/themes/custom/eager-theme.module.ts". For example, the Footer Component import can be found in "src/themes/custom/eager-theme.module.ts" and looks like this:

    ```
    import { FooterComponent } from './app/footer/footer.component';
    ```

4. In that same module file, also add this imported component to the "DECLARATIONS" section. (Again, you can optionally look in the custom theme's module files to see how its done). For example, the Footer Component would then be added to the list of DECLARATIONS (the order of the declarations list doesn't matter):

    ```
    const DECLARATIONS = [
      ....
      FooterComponent,
      ....
    ];
    ```

5. At this point, you should rebuild/restart your UI to ensure nothing has broken. If you did everything correctly, no build errors will occur. Generally speaking, it's best to add Components one by one, rebuilding in between.
6. Now, you can customize your newly added Component by following the "Customizing Other Components in your Theme" instructions above.

Removing Component Directories from your Theme

While there is no harm in keeping extra, unmodified component directories in your theme, it can be beneficial to *remove component directories which are unchanged.*

The main advantage to keeping your theme simple/small is that it can make future upgrades easier. Generally speaking, the fewer components you have in your theme, the less likely your theme will need modification in a future upgrade (as generally your theme may require updates if one of the components it references underwent structural/major changes).

You can remove a Component directory as follows:

1. First you MUST remove all references to that directory/component from your theme's `lazy-theme.module.ts` and `eager-theme.module.ts` files.
    a. For example, to delete the "./app/login-page" directory, you'd want to find which component(s) use that directory in your `lazy-theme.module.ts` file.
    b. If you search that file, you'd fine this reference:

    ```
    import { LoginPageComponent } from './app/login-page/login-page.component';
    ```

    c. That means you not only need to remove that "import" statement. You'd also need to remove all other references to "LoginPageComponent" in that same `lazy-theme.module.ts` file. So, you'd also need to remove it from the DECLARATIONS section:

    ```
    const DECLARATIONS = [
      ....
      LoginPageComponent,
      ....
    ];
    ```

2. Finally, delete the directory in question from your theme.
3. At this point, you should rebuild/restart your UI to verify nothing has broken. If you did everything correctly, no build errors will occur.
    a. If you failed to edit your `lazy-theme.module.ts` correctly, you may see "Cannot find module [path-to-module]" errors which reference the directories that Angular/Node can no longer find in your theme. Either restore those directories, or remove the reference (s) from the `lazy-theme.module.ts` similar to step 1 above.

Debugging which theme is being used

While you are working on themes, sometimes you may discover that it's difficult to tell which theme is being used to generate specific HTML elements. Luckily, there's an easy way to determine which theme is used on every HTML element.

Simply view the HTML source of the page, and look for the "data-used-theme" attribute. This attribute will tell you which named theme matched that HTML element. By default, a name of "base" references the core or "base" code (under ./src/app) was used.

For example:

HTML source

```
<!-- This example shows the theme named "dspace" was used for the "themed-header-navbar-wrapper.component.ts" --
>
<ds-themed-header-navbar-wrapper ... data-used-theme="dspace"></ds-themed-header-navabar-wrapper>

<main>
        <!-- But, on the same page, the theme named "base" (core code) was used for the "themed-breadcrumbs.
component.ts" -->
    <ds-themed-breadcrumbs ... data-used-theme="base"></ds-themed-breadcrumbs>
</main>
```

Finding which component is generating the content on a page

See "Finding which component is generating the content on a page" in our User Interface Debugging guide.

Additional Theming Resources

- User Interface Debugging - Our guide to debugging basic behaviors in the User Interface

- "Getting Started with DSpace 7.0" Basic Workshop at OR2021 Conference
- Bootstrap Documentation - DSpace's UI strives to be compliant with "out-of-the-box" Bootstrap as much as possible. Therefore, Bootstrap knowledge is very beneficial in customizing DSpace.
- Sass Documentation - Bootstrap and DSpace both use Sass to enhance your ability to customize styles quickly via variables, etc. Some familiarity with Sass is recommended, though you need not be an expert.

User Interface Debugging

This page provides some basic guidelines for debugging issues that may occur in the User Interface when following the User Interface Customization guide or similar.

- How to find what the User Interface is doing when you click something
- Finding which component is generating the content on a page

How to find what the User Interface is doing when you click something

HINT: This is very similar to our Troubleshoot an error guide.

If you want to determine what action the User Interface is taking when you click a button/link, you can find that information in your browser's Developer Tools.

- Open a new browser tab and visit your homepage.
- With that browser tab open, open "Developer Tools" in your browser. This "Developer Tools" tab/window will now follow all the actions you perform in your other browser tab.
  - In Chrome, go to "More Tools Developer Tools".
  - In Firefox, go to "Web Developer Web Developer Tools".
  - In Microsoft Edge, go to "More Tools Developer Tools".
- Now, go back to your open browser tab. Browse to the page you are interested in and click that button/link.
- Back in Developer Tools, you'll see what happened when you clicked that link
  - The "Network" tab will provide information on what requests the User Interface sent to the REST API. Clicking any will give you the details of the request and the response. If any appear in "red", that means the REST API returned an error status. Clicking the red request will show you the exact Headers sent/returned and the error Response.
  - If any errors occurred, you'll also see details about them in your "Console" tab.

Finding which component is generating the content on a page

Every DSpace Angular component has a corresponding `<ds-*>` HTML-like tag. The HTML tag is called the "selector" (or CSS selector), and it is defined in the "*.component.ts" tag using the "@Component" decorator (see Angular's Component Overview for the basics). The key point to remember is that *if you can find the "<ds-* >" tag, then it is easy to determine which component generated that tag*!

So, supposing you are trying to determine which component is generating part of a DSpace page.

1. View the HTML source of the page in your browser. Search for that section of the page. (Or, right click on that part of the page and select "Inspect")
   a. For example, on the homepage view the source of the "Communities in DSpace" heading
2. Look for a parent HTML tag that begins with "ds-". This is the component selector!
   a. Continuing the example, if you view the source of the "Communities in DSpace" heading, you'll see something like this (all HTML attributes have been removed to make the example more simple):

```
<ds-top-level-community-list>
  <div>
    <h2> Communities in DSpace </h2>
    <p>Select a community to browse its collections.</p>
  </div>
</ds-top-level-community-list>
```

   b. Based on the above HTML source, you can see that the "Communities in DSpace" header/content is coming from a component who's selector is "ds-top-level-community-list"
3. Now, search the source code (./src/app/) directories for a ".component.ts" file which includes that "ds-" tag name. This can most easily be done in an IDE, but also could be done using command line tools (e.g. grep like this).
   a. Continuing the example, if you search the `./src/app/` directories for "ds-top-level-community-list" you'll find a match in the "`src/app/home-page/top-level-community-list/top-level-community-list.component.ts`" file:

```
@Component({
  selector: 'ds-top-level-community-list',
  ...
})
```

   b. This lets you know that to modify the display of that section of the page, you may need to edit either the "`top-level-community-list.component.ts`" file **or** it's corresponding HTML file at "`top-level-community-list.component.html`"
4. Once you've located the component, you can edit that component's HTML file (ending in "component.html") to change that section of the page.
   a. Keep in mind, the component's HTML file may reference other "ds-" tags! Those are other components in DSpace which you can find again by searching the "./src/app" directories for that tag.

Accessibility

- Accessibility Statement
- Conformance status
- How we test for accessibility
- Accessibility Compliance Report (VPAT)
- Known limitations
- Report accessibility issues

Accessibility Statement

DSpace is an international, open-source digital repository application that aspires to be as inclusive as possible for all users, including people with disabilities. As a community of users and developers who build and maintain this application, we are dedicated to creating an accessible and interoperable user interface. We are guided by the recommendations of the Web Content Accessibility Guidelines (WCAG) and we continually strive to meet and exceed these standards.

Conformance status

The Web Content Accessibility Guidelines (WCAG) defines requirements for designers and developers to improve accessibility for people with disabilities. It defines three levels of conformance: Level A, Level AA, and Level AAA.

DSpace strives to conform with the current version of WCAG level AA. However, we acknowledge that achieving full accessibility is a work-in-progress at this time.

How we test for accessibility

Development on DSpace is active and ongoing and we use several methods to ensure accessibility for both existing and new development.

- We use design principles and coding standards informed by accessibility concerns as documented in User Interface Design Principles & Accessibility.
- We run automated accessibility scanning tools (Axe by Deque) across the user interface in our end-to-end tests (run via Cypress). These automated tests run for every GitHub pull request submitted to our user interface codebase.
- We ask institutions who use DSpace to share any of their own accessibility testing results with DSpace developers. Accessibility issues discovered are turned into bug tickets for developers to address in upcoming DSpace releases.
    - If your institution has accessibility testing results to share, please contact Tim Donohue or anyone on our DSpace Steering Group.
- In 2021, we conducted an accessibility audit of the DSpace application with Deque to get specific feedback on our accessibility conformance. Their feedback has guided our design and coding standards mentioned above.

Accessibility Compliance Report (VPAT)

Some organizations require a Voluntary Product Accessibility Template (VPAT) for software applications in use or being considered. VPATs are documents that describe how well a product meets accessibility standards. To access the latest VPAT for DSpace (based on version 7.6.2 and 8.0), please visit the DSpace website.

Known limitations

Despite our best efforts to ensure accessibility of DSpace, there may be some limitations. Below is a description of known limitations:

1. We track all known DSpace accessibility issues in our GitHub issue tracker with the "accessibility" label.
2. DSpace development is primarily volunteer-based, and therefore some accessibility tickets may be waiting on a volunteer to claim them. While we do our best to ensure critical issues are addressed quickly, non-critical issues may not receive attention until a volunteer gets to them. We accept code contributions from anyone (in the form of GitHub Pull Requests).
    a. If an issue is important to you and you have developers on staff (or can hire a service provider), please consider contributing a fix back to DSpace. Please claim open tickets by commenting on the issue ticket - this ensures that no other institutions will duplicate efforts.
3. Since the DSpace User Interface allows users to upload content, we cannot ensure the accessibility of user contributions. DSpace has some features that allow administrators to make uploaded content more accessible, but some limitations exist
    a. The MediaViewer (used to view video/audio content) supports subtitles/captioning. However, at this time, the WebVTT captioning files must be uploaded separately alongside the original video.
    b. At this time, DSpace does not support custom alternative text (alt text) for either thumbnail images (generated from uploaded files) or Community/Collection logos.

Report accessibility issues

We consider all accessibility issues to be bugs. Please report any accessibility issues as a GitHub issue ticket. We will prioritize accessibility issues all of our future releases.

In the ticket, please include the following details:

- What is the accessibility issue you've found? If you know of a way to fix the issue, please include it as well.

- Which page(s) of the DSpace web application can this issue be found on? For example, provide the URL of the page or a description of how to get to that page.
- How could someone reproduce this issue? For example, what tool or browser plugin did you use when you found this issue? If the issue is browser-specific, also note which browser(s) are affected.
- If possible, provide links/screenshots to document the issue or potential fixes. This might include a screenshot showing the issue, a link to WCAG describing the issue or a description from an internal accessibility audit.

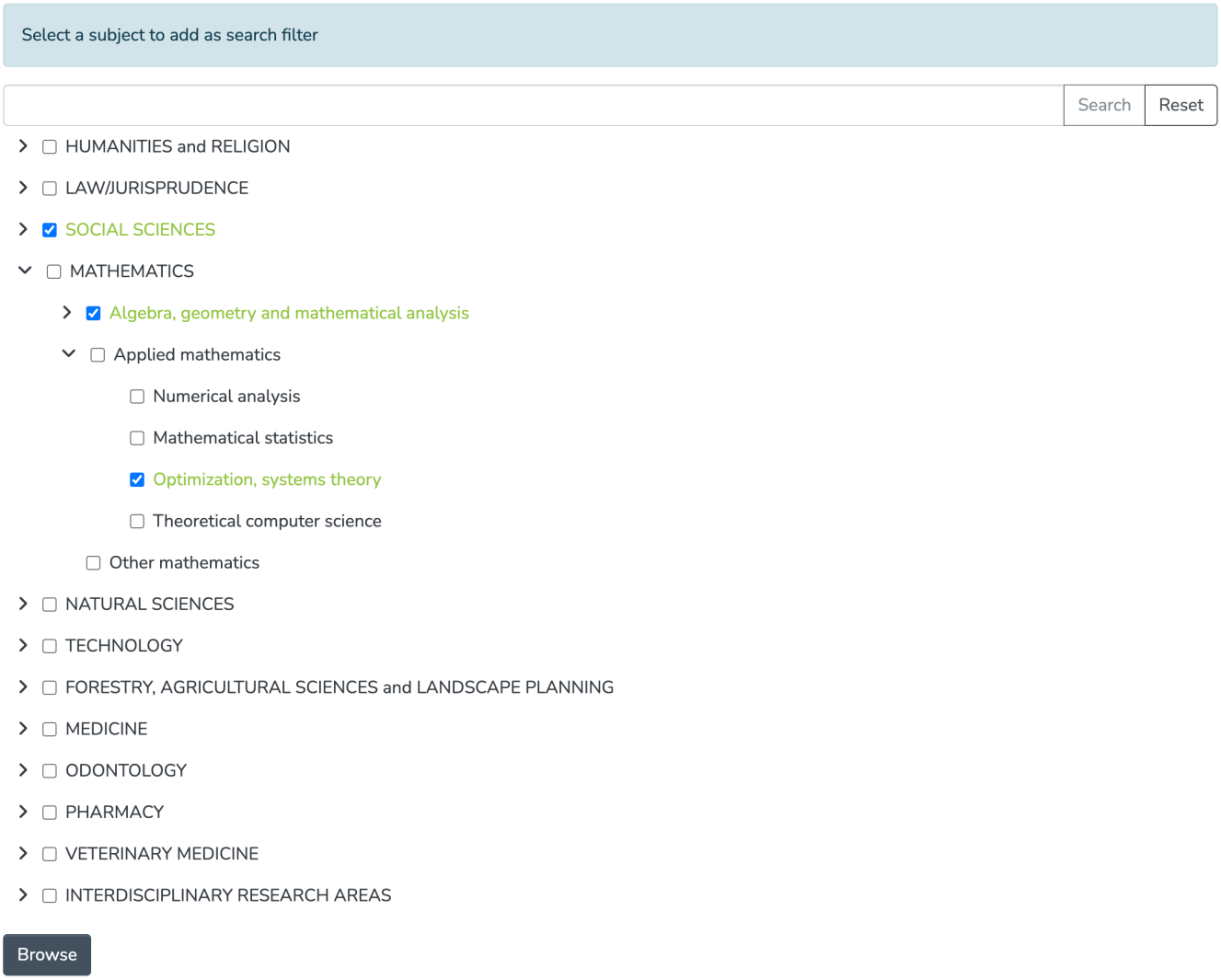
We also welcome contributions / accessibility fixes from anyone. If you've found a way to fix the issue, please submit a GitHub pull request to our codebase . Service providers are also available for hire to fix issues and donate them back to the DSpace codebase.

Browse

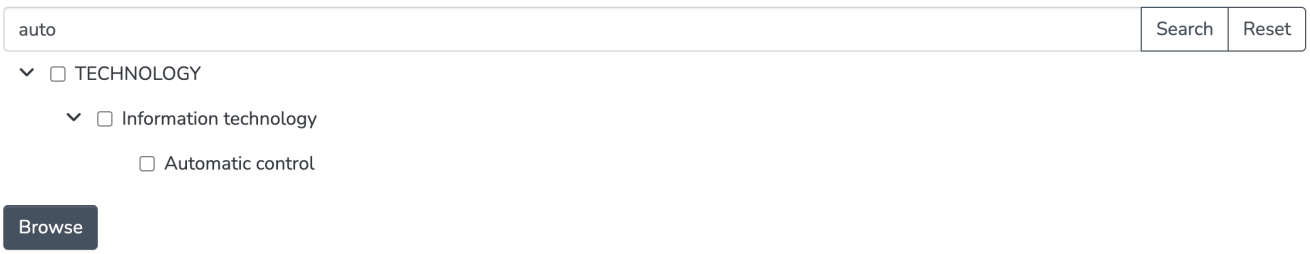
- Browse By Subject Category

Browse By Subject Category

You can find the link to this page by hovering over the "All of DSpace" menu option. On the page, you are presented with the values of the "srsc" vocabulary in a hierarchical tree structure. You can open/close different "branches" of the tree and you can select/deselect values you wish to search on.



You can search for specific values by using the search bar on top of the tree and clicking "Search". Clicking "Reset" will not only reset the tree itself, but also the values you previously selected.



After you're done (de)selecting values, click "Browse". This will redirect you to the search page, where your selected values are used as search filters:

- If one value was selected, the search results will consist of every item which has that value in their `dc.subject` metadata field.
- If multiple values were selected, the search results will consist of the items which have all of the values in their `dc.subject` metadata field. (E.g. if you selected TECHNOLOGY and MEDICINE, only items with *both subjects* will show up.)

To configure Browse by Subject Category options, see "Hierarchical Browse Indexes" in the Configuration Reference.

Discovery

1 What is DSpace Discovery
    1.1 What is a Sidebar Facet
    1.2 What is a Search Filter
    1.3 What is a tag cloud facet
2 Configuration files
3 General Discovery settings (config/modules/discovery.cfg)
4 Browse settings (config/dspace.cfg)
5 Modifying the Discovery User Interface (config/spring/api/discovery.xml)
    5.1 Structure Summary
    5.2 Default settings
    5.3 Non indexed metadata fields
    5.4 Search filters & sidebar facets Customization
        5.4.1 Hierarchical (taxonomies based) sidebar facets
    5.5 Sort option customization for search results
    5.6 DiscoveryConfiguration
        5.6.1 Configuring lists of sidebarFacets and searchFilters
        5.6.2 Configuring and customizing search sort fields
        5.6.3 Adding default filter queries (OPTIONAL)
        5.6.4 Access Rights Awareness
            5.6.4.1 Access Rights Awareness - technical details
        5.6.5 Customizing the Recent Submissions display
        5.6.6 Customizing hit highlighting & search snippets
            5.6.6.1 Hit highlighting technical details
        5.6.7 "More like this" configuration
            5.6.7.1 "More like this" technical details
        5.6.8 "Did you mean" spellcheck aid for search configuration
            5.6.8.1 "Did you mean" spellcheck aid for search technical details
        5.6.9 Customizing the "Tag Cloud" facet
        5.6.10 Disabling the "Has file(s)" facet
6 Discovery Solr Index Maintenance
7 Advanced Solr Configuration

What is DSpace Discovery

The Discovery Module enables faceted searching & browsing for your repository.

Although these techniques are new in DSpace, they might feel familiar from other platforms like Aquabrowser or Amazon, where facets help you to select the right product according to facets like price and brand. DSpace Discovery offers very powerful browse and search configurations that were only possible with code customization in the past.

Watch the DSpace Discovery introduction video

Since 6.0, Discovery is the only out-of-the-box Search and Browse infrastructure provided in DSpace.

What is a Sidebar Facet

From the user perspective, faceted search (also called faceted navigation, guided navigation, or parametric search) breaks up search results into multiple categories, typically showing counts for each, and allows the user to "drill down" or further restrict their search results based on those facets.

When you have successfully enabled Discovery in your DSpace, you will notice that the different enabled facets are visualized in a "Discover" section in your sidebar, by default, right below the Browse options.

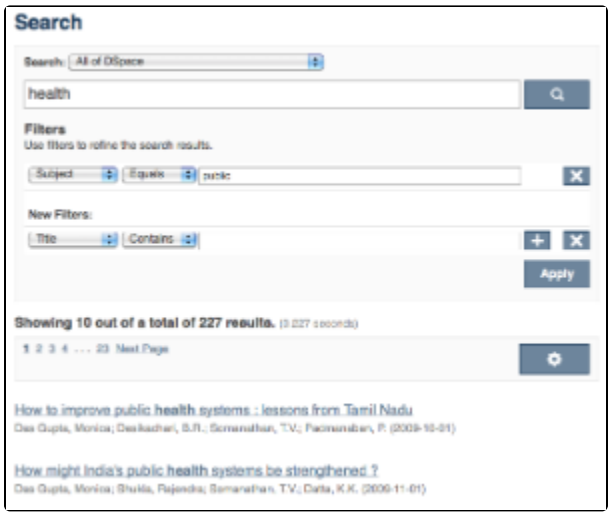
In this example, there are 3 Sidebar Facets: Author, Subject and Date Issued. It's important to know that multiple metadata fields can be included in one facet. For example, the Author facet above includes values from both dc.contributor.author as well as dc.creator.

Another important property of Sidebar Facets is that their contents are automatically updated to the context of the page. On collection homepages or community homepages it will include information about the items included in that particular collection or community.

What is a Search Filter

In a standard search operation, a user specifies his complete query prior to launching the operation. If the results are not satisfactory, the user starts over again with a (slightly) altered query.

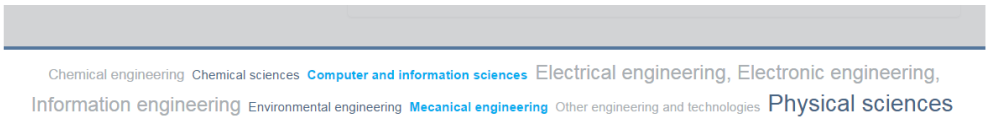
In a faceted search, a user can modify the list of displayed search results by specifying additional "filters" that will be applied on the list of search results. In DSpace, a filter is a contain condition applied to specific facets. In the example below, a user started with the search term "health", which yielded 500 results. After applying the filter "public" on the facet "Subject", only 227 results remain. Each time a user selects a sidebar facet it will be added as a filter. Active filters can be altered or removed in the 'filters' section of the search interface.



Another example: Using the standard search, a user would search for something like [**wetland + "dc.author=Mitsch, William J" + dc.subject="water quality"**]. With filtered search, they can start by searching for [**wetland**], and then filter the results by the other attributes, author and subject.

What is a tag cloud facet

Tag cloud facet is another way to display facets of your repository in a "tag cloud" form in which the importance of each tag is show with font size or color. This format is useful for quickly perceiving the most prominent terms.



This is a classic "tag cloud" facet in a DSpace repository.

Configuration files

The configuration for discovery is located in 2 separate files.

- General settings: The `discovery.cfg` file located in the `[dspace-install-dir]/config/modules directory`.
- Browse settings: The dspace.cfg file located in `[dspace-install-dir]/config/` includes "webui.browse.index.*" settings
- User Interface Configuration: The `discovery.xml` file is located in `[dspace-install-dir]/config/spring/api/` directory.

General Discovery settings (`config/modules/discovery.cfg`)

The `discovery.cfg` file is located in the `[dspace]/config/modules` directory and contains following properties. Any of these properties may be overridden in your `local.cfg` (see Configuration Reference):

Property:	**discovery.search.server**
Example Value:	`discovery.search.server=[http://localhost:8080/solr/search]`
Informational Note:	Discovery relies on a Solr index for storage and retrieval of its information. This parameter determines the location of the Solr index. If you are uncertain whether this property is set correctly, you can use a commandline tool like "wget" to perform a query against the Solr index (and ensure Solr responds). For example, the below query searches the Solr index for "test" and returns the response on standard out: `wget -O - http://localhost:8080/solr/search/select?q=test`
Property:	**discovery.index.authority.ignore[.field]**

Example Value:	`discovery.index.authority.ignore=true`
	
	`discovery.index.authority.ignore.dc.contributor.author=false`

Informational Note:	By default, Discovery will use the authority information in the metadata to disambiguate homonyms. Setting this property to false will make the indexing process the same as the metadata doesn't include authority information. The configuration can be different on a field (<schema>.<element>.<qualifier>) basis. Setting the property without a field will change the default value.
Property:	**discovery.browse.authority.ignore[.browse-index]**

Example Value:	`discovery.browse.authority.ignore=true`
	
	`discovery.browse.authority.ignore.author=false`

Informational Note:	*Similar property to "discovery.index.authority.ignore", except specific to the "Browse By" indexes.* By default, Discovery will use the authority information in the metadata to disambiguate homonyms. Setting this property to false will make the indexing process the same as the metadata doesn't include authority information. The configuration can be different on a browse index basis. Setting the property without a browse index will change the default value.
Property:	**discovery.index.authority.ignore-prefered[.field]**

Example Value:	`discovery.index.authority.ignore-prefered=true`
	
	`discovery.index.authority.ignore-prefered.dc.contributor.author=false`

Informational Note:	By default, Discovery will use the authority information in the metadata to query the authority for the preferred label. Setting this property to false will make the indexing process the same as the metadata doesn't include authority information (i.e. the preferred form is the one recorded in the metadata value). The configuration can be different on a field (<schema>.<element>.<qualifier>) basis. Setting the property without a field will change the default value. **If the authority is a remote service, disabling this feature can greatly improve performance.**
Property:	**discovery.browse.authority.ignore-prefered[.browse-index]**

Example Value:	`discovery.browse.authority.ignore-prefered=true`
	
	`discovery.browse.authority.ignore-prefered.author=false`

Informational Note:	*Similar property to "discovery.index.authority.ignore-prefered", except specific to the "Browse By" indexes.* By default, Discovery will use the authority information in the metadata to query the authority for the preferred label. Setting this property to false will make the indexing process the same as the metadata doesn't include authority information (i.e. the preferred form is the one recorded in the metadata value). The configuration can be different on a browse index basis. Setting the property without a browse index will change the default value. **If the authority is a remote service, disabling this feature can greatly improve performance.**
Property:	**discovery.index.authority.ignore-variants[.field]**

Example Value:	`discovery.index.authority.ignore-variants=true`
	
	`discovery.index.authority.ignore-variants.dc.contributor.author=false`

Informational Note:	By default, Discovery will use the authority information in the metadata to query the authority for variants. Setting this property to false will make the indexing process the same, as the metadata doesn't include authority information. The configuration can be different on a per-field (<schema>.<element>.<qualifier>) basis. Setting the property without a field will change the default value. **If authority is a remote service, disabling this feature can greatly improve performance.**
Property:	**discovery.browse.authority.ignore-variants[.browse-index]**

Example Value:	`discovery.browse.authority.ignore-variants=true`
	
	`discovery.browse.authority.ignore-variants.author=false`

Informational Note:	*Similar property to "discovery.index.authority.ignore-variants", except specific to the "Browse By" indexes.* By default, Discovery will use the authority information in the metadata to query the authority for variants. Setting this property to false will make the indexing process the same, as the metadata doesn't include authority information. The configuration can be different on a browse index basis. Setting the property without a browse index will change the default value. **If authority is a remote service, disabling this feature can greatly improve performance.**

Browse settings (`config/dspace.cfg`)

See the "Browse Index Configuration" section of the Configuration Reference for all "Browse By" configurations. These Configurations control both which fields are indexed for browsing, as well as which Browse by options appear in the user interface. Changing these configurations requires reindexing.

If you add new browse fields then you should coordinate changes here with the message catalog(s) in the UI. You will need to add several entries:

key	value
browse.comcol.by.*	label the browse field in the Browse menu of a community or collection page
menu.section.browse_global_by_*	label the browse field in the "browse" dropdown at the top of the page
browse.metadata.*	label the browse field in the body of the browsing page
browse.metadata.*.breadcrumbs	label the browse field in the page's "breadcrumb trail"

Modifying the Discovery User Interface (`config/spring/api/discovery.xml`)

The `discovery.xml` file is located in the `[dspace]/config/spring/api` directory. Modifying these settings can change the behavior of the Search pages in the user interface, allowing you to customize the facets, filters, sort options, etc.

If you add new search fields, sorts, etc. then you should coordinate changes here with the message catalog(s) in the UI. You will need to add e.g. `search.filters.filter.NAME.head` (where NAME is the `indexFieldName` of the field definition) to label a new search field.

Structure Summary

This file is in XML format. You should be familiar with XML before editing this file. The configurations are organized together in beans, depending on the purpose these properties are used for.
This purpose can be derived from the class of the beans. Here's a short summary of classes you will encounter throughout the file and what the corresponding properties in the bean are used for.

Download the configuration file and review it together with the following parameters

Class:	**DiscoveryConfigurationService**
Purpose:	Defines the mapping between separate Discovery configurations and individual collections/communities
Default:	All communities, collections and the homepage (key=default) are mapped to defaultConfiguration. Also controls the metadata fields that should not be indexed in the search core (item provenance for example).
Class:	**DiscoveryConfiguration**
Purpose:	Groups configurations for sidebar facets, search filters, search sort options and recent submissions
Default:	There is one configuration by default called defaultConfiguration
Class:	**DiscoverySearchFilter**
Purpose:	Defines that specific metadata fields should be enabled as a search filter
Default:	dc.title, dc.contributor.author, dc.creator, dc.subject.* and dc.date.issued are defined as search filters
Class:	**DiscoverySearchFilterFacet**
Purpose:	Defines which metadata fields should be offered as a contextual sidebar browse options, each of these facets has also got to be a search filter
Default:	dc.contributor.author, dc.creator, dc.subject.* and dc.date.issued
Class:	**HierarchicalSidebarFacetConfiguration**
Purpose:	Defines which metadata fields contain hierarchical data and should be offered as a contextual sidebar option
Class:	**DiscoverySortConfiguration**
Purpose:	Further specifies the sort options to which a DiscoveryConfiguration refers
Default:	dc.title and dc.date.issued are defined as alternatives for sorting, other than Relevance (hard-coded)
Class:	**DiscoveryHitHighlightingConfiguration**
Purpose:	Defines which metadata fields can contain hit highlighting & search snippets
Default:	dc.title, dc.contributor.author, dc.subject, dc.description.abstract & full text from text files.
Class:	**TagCloudFacetConfiguration**
Purpose:	Defines the tag cloud appearance configuration bean and the search filter facets to appear in the tag cloud form. You can have different "**TagCloudFacetConfiguration**" per community or collection or the home page

Default settings

In addition to the summarized descriptions of the default values, following details help you to better understand these defaults. If you haven't already done so, download the configuration file and review it together with the following parameters.
The file contains one default configuration that defines following sidebar facets, search filters, sort fields and recent submissions display:

- Sidebar facets
  - **searchFilterAuthor:** groups the metadata fields dc.contributor.author & dc.creator with a facet limit of 10, sorted by occurrence count
  - **searchFilterSubject:** groups all subject metadata fields (dc.subject.*) with a facet limit of 10, sorted by occurrence count
  - **searchFilterIssued:** contains the dc.date.issued metadata field, which is identified with the type "date" and sorted by specific date values

- Search filters
  - **searchFilterTitle:** contains the dc.title metadata field
  - **searchFilterAuthor:** contains the dc.contributor.author & dc.creator metadata fields
  - **searchFilterSubject:** contains the dc.subject.* metadata fields
  - **searchFilterIssued:** contains the dc.date.issued metadata field with the type "date"
- Sort fields
  - **sortTitle**: contains the dc.title metadata field
  - **sortDateIssued:** contains the dc.date.issued metadata field, this sort has the type date configured.
- defaultFilterQueries
  - The default configuration contains no defaultFilterQueries
  - The default filter queries are disabled by default but there is an example in the default configuration in comments which allows discovery to only return items (as opposed to also communities/collections).
- Recent Submissions
  - The recent submissions are sorted by dc.date. accessioned which is a date and a maximum number of 5 recent submissions are displayed.
- Hit highlighting
  - The fields dc.title, dc.contributor.author & dc.subject can contain hit highlighting.
  - The dc.description.abstract & full text field are used to render search snippets.
- Non indexed metadata fields
  - **Community/Collections:** dc.rights (copyright text)
  - **Items:** dc.description.provenance

Many of the properties contain lists that use references to point to the configuration elements. This way a certain configuration type can be used in multiple discovery configurations so there is no need to duplicate them.

Non indexed metadata fields

The discovery.xml file has configuration to not index certain metadata fields for communities/collections/items. The configuration is handled in the "toIgnoreMetadataFields" property located in the "org.dspace.discovery.configuration.DiscoveryConfigurationService" bean. Below is an example configuration that excludes dc.description.provenance for items & dc.rights for communities/collections:

```
<property name="toIgnoreMetadataFields">
    <map>
        <entry>
            <key><util:constant static-field="org.dspace.core.Constants.COMMUNITY"/></key>
            <list>
                <!--Introduction text-->
                <!--<value>dc.description</value>-->
                <!--Short description-->
                <!--<value>dc.description.abstract</value>-->
                <!--News-->
                <!--<value>dc.description.tableofcontents</value>-->
                <!--Copyright text-->
                <value>dc.rights</value>
                <!--Community name-->
                <!--<value>dc.title</value>-->
            </list>
        </entry>
        <entry>
            <key><util:constant static-field="org.dspace.core.Constants.COLLECTION"/></key>
            <list>
                <!--Introduction text-->
                <!--<value>dc.description</value>-->
                <!--Short description-->
                <!--<value>dc.description.abstract</value>-->
                <!--News-->
                <!--<value>dc.description.tableofcontents</value>-->
                <!--Copyright text-->
                <value>dc.rights</value>
                <!--Collection name-->
                <!--<value>dc.title</value>-->
            </list>
        </entry>
        <entry>
            <key><util:constant static-field="org.dspace.core.Constants.ITEM"/></key>
            <list>
                <value>dc.description.provenance</value>
            </list>
        </entry>
    </map>
</property>
```

By adding additional values to the appropriate lists additional metadata can be excluded from the search core, a reindex is required after altering this file to ensure that the values are removed from the index.

Search filters & sidebar facets Customization

This section explains the properties for search filters & sidebar facets. Each sidebar facet must occur in the reference list of the search filters. Below is an example configuration of a search filter that is not used as a sidebar facet.

```
<bean id="searchFilterTitle" class="org.dspace.discovery.configuration.DiscoverySearchFilter">
    <property name="indexFieldName" value="title"/>
    <property name="metadataFields">
        <list>
            <value>dc.title</value>
        </list>
    </property>
    <property name="pageSize" value="10"/>
</bean>
```

The id & class attributes are mandatory for this type of bean. The properties that it contains are discussed below.

- **indexFieldName** (Required)**:** A unique search filter name, the metadata will be indexed in Solr under this field name.
- **metadataFields** (Required): A list of the metadata fields that need to be included in the facet.

Sidebar facets extend the search filter and add some extra properties to it. Below is an example of a search filter that is also used as a sidebar facet.

```
<bean id="searchFilterAuthor" class="org.dspace.discovery.configuration.DiscoverySearchFilterFacet">
        <property name="indexFieldName" value="author"/>
        <property name="metadataFields">
            <list>
                <value>dc.contributor.author</value>
                <value>dc.creator</value>
            </list>
        </property>
        <property name="facetLimit" value="5"/>
        <property name="sortOrderSidebar" value="COUNT"/>
        <property name="sortOrderFilterPage" value="COUNT"/>
        <property name="isOpenByDefault" value="true"/>
        <property name="type" value="text"/>
    </bean>
```

Note that the class has changed from **DiscoverySearchFilter** to **DiscoverySearchFilterFacet**. This is needed to support the extra properties.

- **facetLimit** (optional)**:** The maximum number of values to be shown by default. This property is optional, if none is specified the default value "10" will be used. If the filter has the type **date**, this property will not be used since dates are automatically grouped together.
- **sortOrder** (optional)**:**The sort order for the sidebar facets, it can either be COUNT or VALUE. The default value is COUNT.
    - **COUNT** Facets will be sorted by the number of times they appear in the repository
    - **VALUE** Facets will be sorted alphabetically
- **type** (optional): the type of the sidebar facet it can either be "date" or "text". "text" is the default value.
    - **text:** The facets will be treated as is (DEFAULT)
    - **date**: Only the year will be stored in the Solr index. These years are automatically displayed in ranges that get smaller when you select one.

Hierarchical (taxonomies based) sidebar facets

Discovery supports specialized drill down in hierarchically structured metadata fields. For this drill down to work, the metadata in the field for which you enable this must be composed out of terms, divided by a splitter. For example, you could have a dc.subject.taxonomy field in which you keep metadata like "CARTOGRAPHY::PHOTOGRAMMETRY", in which Cartography and Photogrammetry are both terms, divided by the splitter "::". Initially the sidebar will only display the top level facets. When clicking on "view more" all the facet options will be displayed.

```
<bean id="searchFilterSubject" class="org.dspace.discovery.configuration.HierarchicalSidebarFacetConfiguration">
    <property name="indexFieldName" value="subject"/>
    <property name="metadataFields">
        <list>
            <value>dc.subject</value>
        </list>
    </property>
    <property name="sortOrder" value="COUNT"/>
    <property name="splitter" value="::"/>
    <property name="skipFirstNodeLevel" value="false"/>
</bean>
```

Note that the class has changed from **SidebarFacetConfiguration** to **HierarchicalSidebarFacetConfiguration**. This is needed to support the extra properties.

- **splitter** (required)**:** The splitter used to split up the separate nodes
- **skipFirstNodeLevel** (optional)**:** Whether or not to show the root node level. For some hierarchical data there is a single root node. In most cases it doesn't need to be shown since it isn't relevant. **This property is true by default.**

Sort option customization for search results

This section explains the properties of an individual SortConfiguration, like sortTitle and sortDateIssued from the default configuration. In order to create custom sort options, you can either modify specific properties of those that already exist or create a totally new one from scratch.

Here's what the sortTitle SortConfiguration looks like:

```
<bean id="sortTitle" class="org.dspace.discovery.configuration.DiscoverySortFieldConfiguration">
       <property name="metadataField" value="dc.title"/>
       <property name="type" value="text"/>
 </bean>
```

The `id` and `class` attributes are mandatory for this type of bean. The properties that it contains are discussed below.

- **metadataField** (Required): The metadata field indicating the sort values
- **type** (optional): the type of the sort option can either be date or text, if none is defined text will be used.

DiscoveryConfiguration

The DiscoveryConfiguration groups configurations for sidebar facets, search filters, search sort options and recent submissions. If you want to show the same sidebar facets, use the same search filters, search options and recent submissions everywhere in your repository, you will only need one DiscoveryConfiguration and you might as well just edit the defaultConfiguration.

The DiscoveryConfiguration makes it very easy to use custom sidebar facets, search filters, ... on specific communities or collection homepage. This is particularly useful if your collections are heterogeneous. For example, in a collection with conference papers, you might want to offer a sidebar facet for conference date, which might be more relevant than the actual issued date of the proceedings. In a collection with papers, you might want to offer a facet for funding bodies or publisher, while these fields are irrelevant for items like learning objects.

A DiscoveryConfiguration consists of six parts:

- The list of applicable sidebarFacets
- The list of applicable searchFilters
- The list of applicable searchSortFields
- Any default filter queries (optional)
- The configuration for the Recent submissions display
- The configuration of the tag cloud facet

Configuring lists of sidebarFacets and searchFilters

After modifying sidebarFacets and searchFilters, don't forget to reindex existing items by running `[dspace]/bin/dspace index-discovery -b`, otherwise the changes will not appear.

Below is an example of how one of these lists can be configured. It's important that each of the bean references corresponds to the exact name of the earlier defined facets, filters or sort options.

Each sidebar facet must also occur in the list of the search filters.

```
<property name="sidebarFacets">
    <list>
        <ref bean="sidebarFacetAuthor" />
        <ref bean="sidebarFacetSubject" />
        <ref bean="sidebarFacetDateIssued" />
    </list>
</property>
```

Configuring and customizing search sort fields

The search sort field configuration block contains the available sort fields and the possibility to configure a default sort field and sort order.
Below is an example of the sort configuration.

```
<property name="searchSortConfiguration">
    <bean class="org.dspace.discovery.configuration.DiscoverySortConfiguration">
        <!--<property name="defaultSort" ref="sortDateIssued"/>-->
        <!--DefaultSortOrder can either be desc or asc (desc is default)-->
        <property name="defaultSortOrder" value="desc"/>
        <property name="sortFields">
            <list>
                <ref bean="sortTitle" />
                <ref bean="sortDateIssued" />
            </list>
        </property>
    </bean>
</property>
```

The property name & the bean class are mandatory. The property field names are discusses below.

- **defaultSort** (optional): The default field on which the search results will be sorted. This must be a reference to an existing search sort field bean. If none is given relevance will be the default. Sorting according to the internal relevance algorithm is always available, even though it's not explicitly mentioned in the sortFields section.
- **defaultSortOrder** (optional): The default sort order can either be asc or desc.
- **sortFields** (mandatory): The list of available sort options, each element in this list must link to an existing sort field configuration bean.

Adding default filter queries (OPTIONAL)

Default filter queries are applied on all search operations and sidebar facet clicks. One useful application of default filter queries is ensuring that all returned results are items. As a result, subcommunities and collections that are returned as results of the search operation, are filtered out.
Similar to the lists above, the default filter queries are defined as a list. They are optional.

```
<property name="defaultFilterQueries">
    <list>
        <value>query1</value>
        <value>query2</value>
    </list>
</property>
```

This property contains a simple list which in turn contains the queries. Some examples of possible queries:

- search.resourcetype:2
- dc.subject:test
- dc.contributor.author: "Van de Velde, Kevin"
- ...

Access Rights Awareness

By default, when searching and browsing using Discovery, you will only see items that you have access to. So, your search/browse results may differ if you are logged into DSpace. This Access Rights Awareness feature ensures that anonymous users (and search engines) are not able to access information (both files and metadata) about embargoed or private items. It also provides you with more direct control over who can see individual items within your DSpace.

How does Access Rights Awareness work?

Access Rights Awareness checks the "READ" access on the Item.

If the "Anonymous" group has "READ" access on the Item, then anonymous/public users will be able to view that Item's metadata and locate that Item via DSpace's search/browse system. In addition, search engines will also be able to index that Item's metadata. However, even with Anonymous READ set at the Item-level, you may still choose to access-restrict the downloading/viewing of *files* within the Item. To do so, you would restrict "READ" access on individual Bitstream(s) attached to the Item.

If the "Anonymous" group does NOT have "READ" access on the Item, then anonymous users will never see that Item appear within their search/browse results (essentially the Item is "invisible" to them). In addition, that Item will be invisible to search engines, so it will never be indexed by them. However, any users who have been given READ access will be able to find/locate the item after logging into DSpace. For example, if a "Staff" group was provided "READ" access on the Item, then members of that "Staff" group would be able to locate the item via search/browse after logging into DSpace.

How can I disable Access Rights Awareness?

If you prefer to allow all access-restricted or embargoed Items to be findable within your DSpace, you can choose to turn off Access Rights Awareness. However, please be aware that this means that restricting "READ" access on an Item will not really do anything – the Item metadata will be available to the public no matter what group(s) were given READ access on that Item.

This feature can be switched off by going to the `[dspace.dir]/config/spring/api/discovery.xml` file & commenting out the bean & the alias shown below.

```
<bean class="org.dspace.discovery.SolrServiceResourceRestrictionPlugin" id="solrServiceResourceIndexPlugin"/>

<alias name="solrServiceResourceIndexPlugin" alias="org.dspace.discovery.SolrServiceResourceRestrictionPlugin"/>
```

The Browse Engine only supports the "Access Rights Awareness" if the Solr/Discovery backend is enabled (see Defining the Storage of the Browse Data). However, it is enabled by default for DSpace 3.x and above.

Access Rights Awareness - technical details

The *DSpaceObject* class has an *updateLastModified()* method which will be triggered each time an authorization policy changes. This method is only implemented in the item class where the last_modified timestamp will be updated and a modify event will be fired. By doing this we ensure that the discovery consumer is called and the item is reindexed. Since this feature can be switched off a separate plugin has been created: the *SolrServiceResourceRestrictionPlugin.* Whenever we reindex a DSpace object all the read rights will be stored in the read field. We make a distinction between groups and users by adding a *'g'* prefix for groups and the *'e'* prefix for epersons.

When searching in discovery all the groups the user belongs to will be added as a filter query as well as the users identifier. If the user is an admin all items will be returned since an admin has read rights on everything.

Customizing the Recent Submissions display

The recent submissions configuration element contains all the configuration settings to display the list of recently submitted items on the home page or community/collection page. Because the recent submission configuration is in the discovery configuration block, it is possible to show 10 recently submitted items on the home page but 5 on the community/collection pages.

Below is an example configuration of the recent submissions.

```
<property name="recentSubmissionConfiguration">
    <bean class="org.dspace.discovery.configuration.DiscoveryRecentSubmissionsConfiguration">
        <property name="metadataSortField" value="dc.date.accessioned"/>
        <property name="type" value="date"/>
        <property name="max" value="5"/>
    </bean>
</property>
```

The property name and the bean class are mandatory. The property field names are discusses below.

- **metadataSortField** (mandatory): The metadata field to sort on to retrieve the recent submissions
- **max** (mandatory): The maximum number of results to be displayed as recent submissions
- **type** (optional): the type of the search filter. It can either be date or text, if none is defined text will be used.

Customizing hit highlighting & search snippets

The hit highlighting configuration element contains all settings necessary to display search snippets & enable hit highlighting.

Disabling hit highlighting / search snippets

You can disable hit highlighting / search snippets by commenting out the entire `<property name="hitHighlightingConfiguration">` Configuration in the `[dspace]/config/spring/api/discovery.xml` configuration file.

PLEASE BE AWARE there are two sections where this <property> definition exists. You should comment out both. One is under the `<bean id="defaultConfiguration">` and one is under the `<bean id="homepageConfiguration">`

Alternatively, you may also choose to tweak which fields are shown in hit highlighting, or modify the number of matching words shown (snippets) and/or number of characters shown around the matching word (maxSize).

For this change to take effect in the User Interface, you will need to restart Tomcat.

Changes made to the configuration will not automatically be displayed in the user interface. By default, only the following fields are displayed: dc.title, dc.contributor.author, dc.creator, dc.contributor, dc.date.issued, dc.publisher, dc.description.abstract and fulltext.

If additional fields are required, look for the "itemSummaryList" template.

Below is an example configuration of hit highlighting.

```
<property name="hitHighlightingConfiguration">
    <bean class="org.dspace.discovery.configuration.DiscoveryHitHighlightingConfiguration">
        <property name="metadataFields">
            <list>
                <bean class="org.dspace.discovery.configuration.DiscoveryHitHighlightFieldConfiguration">
                    <property name="field" value="dc.title"/>
                    <property name="snippets" value="5"/>
                </bean>
                <bean class="org.dspace.discovery.configuration.DiscoveryHitHighlightFieldConfiguration">
                    <property name="field" value="dc.contributor.author"/>
                    <property name="snippets" value="5"/>
                </bean>
                <bean class="org.dspace.discovery.configuration.DiscoveryHitHighlightFieldConfiguration">
                    <property name="field" value="dc.subject"/>
                    <property name="snippets" value="5"/>
                </bean>
                <bean class="org.dspace.discovery.configuration.DiscoveryHitHighlightFieldConfiguration">
                    <property name="field" value="dc.description.abstract"/>
                    <!-- Max number of characters to display around the matching word (Warning setting to 0
returns entire field) -->
                    <property name="maxSize" value="250"/>
                    <!-- Max number of snippets (matching words) to show -->
                    <property name="snippets" value="2"/>
                </bean>
                <bean class="org.dspace.discovery.configuration.DiscoveryHitHighlightFieldConfiguration">
                                    <!-- Displays snippets from indexed full text of document (for
supported formats) -->
                    <property name="field" value="fulltext"/>
                    <!-- Max number of characters to display around the matching word (Warning setting to 0
returns entire field) -->
                    <property name="maxSize" value="250"/>
                                    <!-- Max number of snippets (matching words) to show -->
                    <property name="snippets" value="2"/>
                </bean>
            </list>
        </property>
    </bean>
</property>
```

The property name and the bean class are mandatory. The property field names are:

- **field** (mandatory)**:** The metadata field to be highlighted (can also be * if all the metadata fields should be highlighted).
- **maxSize** (optional): Limit the number of characters displayed to only the relevant part (use metadata field as search snippet).
- **snippets** (optional)**:** The maximum number of snippets that can be found in one metadata field.

Hit highlighting technical details

The *org.dspace.discovery.DiscoveryQuery* object has a setter & getter for the hit highlighting configuration set in Discovery configuration. If this configuration is given the *resolveToSolrQuery* method located in the *org.dspace.discovery.SolrServiceImpl* class will use the standard Solr highlighting feature (http://wiki.apache.org/solr/HighlightingParameters). The *org.dspace.discovery.DiscoverResult* class has a method to set the highlighted fields for each object & field.

The rendering of search results is no longer handled by the METS format but uses a special type of list named "TYPE_DSO_LIST". Each metadata field (& fulltext if configured) is added in the DRI and IF the field contains hit higlighting the Java code will split up the string & add *DRI highlights* to the list. The XSL for the themes also contains special rendering XSL for the DRI; for Mirage, the changes are located in the *discovery.xsl* file. For themes using the old themes based on structural.xsl, look for the template matching "*dri:list[@type='dsolist']*".

"More like this" configuration

The "more like this"-configuration element contains all the settings for displaying related items on an item display page.
Below is an example of the "more like this" configuration.

```
<property name="moreLikeThisConfiguration">
    <bean class="org.dspace.discovery.configuration.DiscoveryMoreLikeThisConfiguration">
        <property name="similarityMetadataFields">
            <list>
                <value>dc.title</value>
                <value>dc.contributor.author</value>
                <value>dc.creator</value>
                <value>dc.subject</value>
            </list>
        </property>
        <!--The minimum number of matching terms across the metadata fields above before an item is found as
related -->
        <property name="minTermFrequency" value="5"/>
        <!--The maximum number of related items displayed-->
        <property name="max" value="3"/>
        <!--The minimum word length below which words will be ignored-->
        <property name="minWordLength" value="5"/>
    </bean>
</property>
```

The property name and the bean class are mandatory. The property field names are discussed below.

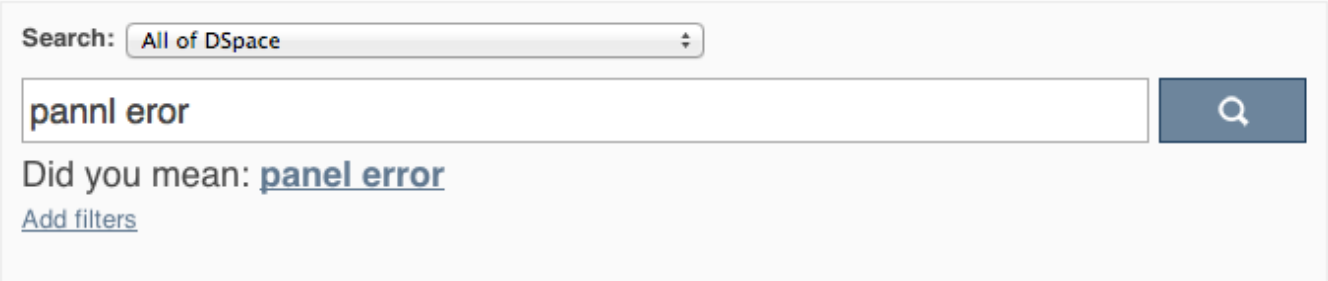
- similarityMetadataFields: the metadata fields checked for similarity
- minTermFrequency: The minimum number of matching terms accross the metadata fields above before an item is found as related
- max: The maximum number of related items displayed
- minWordLength: The minimum word length below which words will be ignored

"More like this" technical details

The *org.dspace.discovery.SearchService* object has received a *getRelatedItems()* method. This method requires an item & the more-like-this configuration bean from above. This method is implemented in the *org.dspace.discovery.SolrServiceImpl* which uses the item as a query & uses the default Solr parameters for more-like-this to pass the bean configuration to solr (https://cwiki.apache.org/confluence/display/solr/MoreLikeThis). The result will be a list of items or if none found an empty list.

"Did you mean" spellcheck aid for search configuration

DSpace 4 introduces the use of SOLR's SpellCheckComponent as an aid for search. When a user's search does not return any hits, the user is presented with a suggestion for an alternative search query.



The feature currently only one line of configuration to discovery.xml. Changing the value from true to false will disable the feature.

```
<property name="spellCheckEnabled" value="true" />
```

"Did you mean" spellcheck aid for search technical details

Similar to the More like this configuration, SOLR's spell check component is used with default configuration values. Any of these values can be overridden in the solrconfig.xml file located in dspace/solr/search/conf/. Following links provide more information about the SOLR SpellCheckComponent:

http://wiki.apache.org/solr/SpellCheckComponent

https://cwiki.apache.org/confluence/display/solr/Spell+Checking

Customizing the "Tag Cloud" facet

Not supported in DSpace 7.x

```
<!-- Set TagCloud configuration per discovery configuration -->
<property name="tagCloudFacetConfiguration" ref="defaultTagCloudFacetConfiguration"/>
```

*Declare the bean (of class: **TagCloudFacetConfiguration**) that holds the configuration for the tag cloud facet.*

```
<!--TagCloud configuration bean for homepage discovery configuration-->
<bean id="homepageTagCloudFacetConfiguration" class="org.dspace.discovery.configuration.
TagCloudFacetConfiguration">
        <!-- Actual configuration of the tagcloud (colors, sorting, etc.) -->
        <property name="tagCloudConfiguration" ref="tagCloudConfiguration"/>
        <!-- List of tagclouds to appear, one for every search filter, one after the other -->
        <property name="tagCloudFacets">
            <list>
                <ref bean="searchFilterSubject" />
            </list>
        </property>
</bean>
```

This bean has two properties:

- **tagCloudConfiguration**: is the bean which describes the actual appearance parameters
- **tagCloudFacets**: the search filter facets which will be used for the tag cloud. If you leave the list empty, no tag cloud will appear. If you declare more than one, such number of tag clouds will appear for each search filter, one after the other.

The appearance configuration can have the following properties, as shown in the following bean:

```
<bean id="tagCloudConfiguration" class="org.dspace.discovery.configuration.TagCloudConfiguration">
             <!-- Should display the score of each tag next to it? Default: false -->
             <property name="displayScore" value="true"/>
             <!-- Should display the tag as center aligned in the page or left aligned? Possible values: true
| false. Default: true  -->
             <property name="shouldCenter" value="true"/>
             <!-- How many tags will be shown. Value -1 means all of them. Default: -1 -->
             <property name="totalTags" value="-1"/>
             <!-- The letter case of the tags.
                     Possible values: Case.LOWER | Case.UPPER | Case.CAPITALIZATION | Case.PRESERVE_CASE |
Case.CASE_SENSITIVE
                     Default: Case.PRESERVE_CASE -->
             <property name="cloudCase" value="Case.PRESERVE_CASE"/>
             <!-- If the 3 CSS classes of the tag cloud should be independent of score (random=yes) or based
on the score. Possible values: true | false . Default: true-->
             <property name="randomColors" value="true"/>
             <!-- The font size (in em) for the tag with the lowest score. Possible values: any decimal.
Default: 1.1 -->
             <property name="fontFrom" value="1.1"/>
             <!-- The font size (in em) for the tag with the lowest score. Possible values: any decimal.
Default: 3.2 -->
             <property name="fontTo" value="3.2"/>
             <!-- The score that tags with lower than that will not appear in the rag cloud. Possible values:
any integer from 1 to infinity. Default: 0 -->
             <property name="cuttingLevel" value="0"/>
             <!-- The distance (in px) between the tags. Default: 5 -->
             <property name="marginRight" value="5"/>
             <!-- The ordering of the tags (based either on the name or the score of the tag)
                     Possible values: Tag.NameComparatorAsc | Tag.NameComparatorDesc | Tag.ScoreComparatorAsc
| Tag.ScoreComparatorDesc
                     Default: Tag.NameComparatorAsc  -->
             <property name="ordering" value="Tag.NameComparatorAsc"/>
    </bean>
```

When tagCloud is rendered there are some CSS classes that you can change in order to change the appearance of the tag cloud.

Class	Note
tagcloud	General class for the whole tagcloud
tagcloud_1	Specific tag class for tag of type 1 (based on score)
tagcloud_2	Specific tag class for tag of type 2 (based on score)
tagcloud_3	Specific tag class for tag of type 3 (based on score)

Disabling the "Has file(s)" facet

Since DSpace 6, a new "Has file(s)" facet has been enabled by default. This facet shows whether items have or do not have any bitstreams in the "ORIGINAL" bundle.
Should you want to turn this off, you can edit `[dspace]/config/spring/api/discovery.xml` to remove the following line from the `defaultConfiguration` and `homepageConfiguration` beans (in the `sidebarFacets` property):

```
<ref bean="searchFilterContentInOriginalBundle"/>
```

Then restart your servlet container.

Discovery Solr Index Maintenance

All released DSpace 7 versions have an issue with `-i <object handle>` command (https://github.com/DSpace/DSpace/issues/9764). Handles aren't supported as parameters in those versions. For the expected result, the user should use the `-i <object UUID>` instead.
This issue will be fixed in 9.0 / 8.1 and 7.6.3 versions.

Command used:	`[dspace]/bin/dspace index-discovery [-cbhf[r <item handle>]]`

Java class:	org.dspace.discovery.IndexClient
Arguments (short and long forms):	Description
	called without any options, will update/clean an existing index
`-b`	(re)build index, wiping out current one if it exists
`-c`	clean existing index removing any documents that no longer exist in the db
`-f`	if updating existing index, force each handle to be reindexed even if uptodate
`-h`	print this help message
`-i <object handle>`	Reindex an individual object (and any child objects). When run on an Item, it just reindexes that single Item. When run on a Collection, it reindexes the Collection itself and all Items in that Collection. When run on a Community, it reindexes the Community itself and all sub-Communities, contained Collections and contained Items.
`-r <object handle>`	remove an Item, Collection or Community from index based on its handle
-s	Rebuild the spellchecker, can be combined with -b and -f.

It is recommended to run maintenance on the Discovery Solr index occasionally (from crontab or your system's scheduler), to prevent your servlet container from running out of memory:

```
[dspace]/bin/dspace index-discovery
```

Advanced Solr Configuration

Discovery is built as an application layer on top of the Solr open source enterprise search server. Therefore, Solr configuration can be applied to the Solr cores that are shipped with DSpace.
The DSpace Solr instance currently runs several cores (which means indexes in Solr parlance). The "statistics" core is for collection of DSpace usage events for statistical purposes (if you have been collecting statistics for multiple years, you may have chosen to use sharding and you will see one core per each year collected). The "search" core is used by Discovery for for search and faceting, for displaying the collection/community hierarchy and item counts. The "authority" core is used by SolrAuthority to store information about authors, including their data imported from the ORCID registry.

```
solr
 search
    conf
        protwords.txt
        schema.xml
        solrconfig.xml
        stopwords.txt
        synonyms.txt
|
 ...
 statistics
     conf
         protwords.txt
         schema.xml
         solrconfig.xml
         stopwords.txt
         synonyms.txt
```

Contextual Help Tooltips

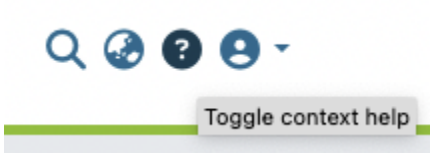
Available in 7.5 or later.

Contextual help tooltips are a feature to provide additional information about how to use DSpace to less experienced users without cluttering the interface for more advanced users who do not need additional instruction.

- User perspective
- Adding new tooltips

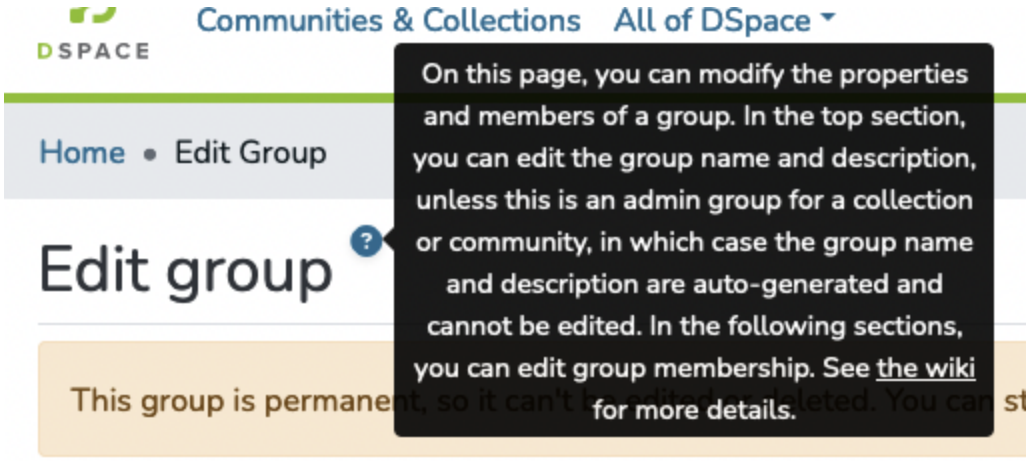
User perspective

If the user visits a page where contextual help tooltips are available, a "toggle context help" button appears in the header, in between the language switch menu and user profile menu. Clicking this button toggles the visibility of the tooltips on the page (by default, they are invisible).



When tooltip visibility is turned on, similar looking buttons appear on the page where ever a tooltip is available.

Clicking any of these buttons makes a text bubble appear containing the contextual help; clicking anywhere outside of the bubble makes it disappear again.



Adding new tooltips

Any HTML element can be given a tooltip by setting the `*dsContextHelp` attribute.

The value assigned to the attribute should be an object of type `ContextHelpDirectiveInput`:

```
export interface ContextHelpDirectiveInput {
  content: string;
  id: string;
  tooltipPlacement?: PlacementArray;
  iconPlacement?: PlacementDir;
}
```

- The mandatory `content` field represents a key in i18n files (src/assets/i18n/*.json5). You will need to add a new key to this file to store the help text.
- `id` should be a unique identifier for this tooltip, to distinguish it from other tooltips on the page.
- `tooltipPlacement` (optional) determines where the text bubble appears relative to the help button. Its type is an array of `Placements`; see the ng-bootstrap documentation for more information.
- `iconPlacement` (optional) should be assigned either 'left' or 'right', and determines whether the tooltip will be placed on the left or on the right of the element.

This is what the template looks like for the "Edit group" example in the "User Perspective" picture above:

```
<h2 class="border-bottom pb-2">
          <span
            *dsContextHelp="{
              content: 'admin.access-control.groups.form.tooltip.editGroupPage',
              id: 'edit-group-page',
              iconPlacement: 'right',
              tooltipPlacement: ['right', 'bottom']
            }"
          >
            {{messagePrefix + '.head.edit' | translate}}
          </span>
</h2>
```

A few important notes:

- Note the use of the `span` tags: setting `*dsContextHelp` directly on the `h2` element makes the help button appear all the way on the right of the page, instead of directly to the right of the "Edit group" text.
- The 'content' field maps to the i18n key which is used to display the help text. This i18n key's value may include markdown-style **links** (only)**.** At this time, other formatting is not supported. To display a link, use the following markdown syntax in your i18n value:

  ```
  "admin.access-control.groups.form.tooltip.editGroupPage": "This is the text that would be displayed in the tooltip. And here is a link to the [DSpace 7 documentation](https://wiki.lyrasis.org/display/DSDOC7x/)."
  ```

IIIF Configuration

- Overview
  - Format Support
- Enable IIIF Support on Backend
  - Install a IIIF Image Server
    - Installing and Configuring Cantaloupe
  - Required IIIF Configuration
  - Additional Configuration Options
  - CORS Configuration
  - IIIF Search API
- Enable/Install the Mirador Viewer on Frontend
  - Configuring Mirador
- Configure IIIF viewer via Metadata Fields

Overview

Supported in 7.1 or above

IIIF support was first added to DSpace in version 7.1. It was not available in 7.0 or below.

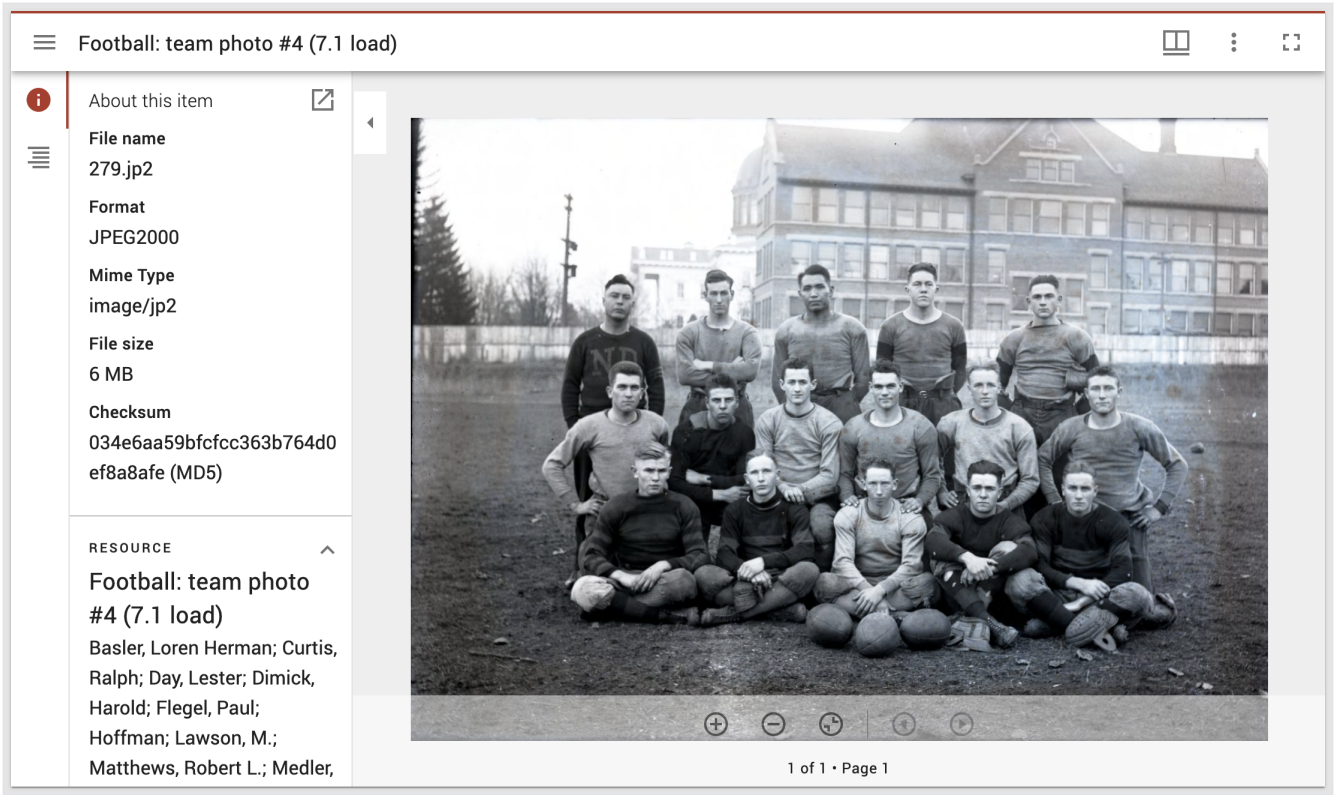
DSpace supports the International Image Interoperability Framework (IIIF). The DSpace REST API implements the IIIF Presentation API version 2.1.1, IIIF Image API version 2.1.1, and the IIIF Search API version 1.0 (experimental). The DSpace Angular frontend uses the Mirador 3.0 viewer.

Administrators can configure IIIF behavior at the Collection, Item, Bundle and Bitstream levels using metadata. To support additional sharing, viewing, comparing, and annotating, DSpace can be configured to share IIIF metadata with external IIIF clients (see CORS Configuration). IIIF REST endpoints implement the same security protocol as the primary REST API so that DSpace authorization policies are enforced for IIIF access as well.

IIIF Image Server

Running IIIF in production requires an IIIF-compatible image server. You are free to use any compatible image server you choose. However, instructions for configuring the Cantaloupe Image Server are included below. A preconfigured Cantaloupe image server can be started via docker-compose to simplify evaluation and testing.

Use full screen for better viewing.



Format Support

Currently, DSpace only supports IIIF viewing of Image formats (any format whose MIME type starts with "image/*"). For example, PDF viewing is not currently supported.

Enable IIIF Support on Backend

DSpace IIIF support is not enabled by default. To enable IIIF, you first need to install a IIIF Image Server, and then update your DSpace configuration as described below.

Install a IIIF Image Server

The Cantaloupe Image Server is currently recommended for use with DSpace, but you are free to use the image server of your choice. A list of IIIF-compliant image servers is maintained by the IIIF community.

Here is a brief overview of how the IIIF image server works with DSpace.

First, the base path to the image server is defined in config/modules/iiif.cfg.

```
iiif.image.server = https://imageserver.mycampus.edu/image-server/cantaloupe/iiif/2/
```

Given this configuration, the IIIF `manifest` returned by the DSpace backend will include an image resource annotation like the following:

IIIF Image Resource Annotation

```
resource: {
        @id: "https://imageserver.mycampus.edu/image-server/cantaloupe/iiif/2/4b415036-57a8-42f4-a971-
c5e982f55f92/full/full/0/default.jpg",
        @type: "dctypes:Image",
        service: {
                @context: "http://iiif.io/api/image/2/context.json",
                @id: "https://imageserver.mycampus.edu/image-server/cantaloupe/iiif/2/4b415036-57a8-42f4-a971-
c5e982f55f92",
                profile: "http://iiif.io/api/image/2/level1.json",
                protocol: "http://iiif.io/api/image"
        },
        format: "image/jp2"
}
```

The Mirador viewer (see below) uses this annotation to communicate with the image server using the IIIF Image API.

Finally, notice that the image server needs to retrieve the requested bitstream from DSpace. There are a number of ways to do this and the details vary with the image server chosen. The easiest approach is for the image server to request the bitstream via HTTP and the DSpace API, e.g.:

```
 http:/dspace.mycampus.edu:8080/server/api/core/bitstreams/4b415036-57a8-42f4-a971-c5e982f55f92/content
```

Installing and Configuring Cantaloupe

The Cantaloupe getting started page provides installation instructions. The basic installation process is simple.

The simplest way to configure Cantaloupe to retrieve images from DSpace is to use HTTPSource with the following configuration.

```
HttpSource.BasicLookupStrategy.url_prefix = <dspace-url>/server/api/core/bitstreams/
HttpSource.BasicLookupStrategy.url_suffix = /content
```

Required IIIF Configuration

To enable IIIF, edit `config/modules/iiif.cfg` or your `local.cfg` file and set **`iiif.enabled`** to be "true".

```
iiif.enabled = true
```

In addition, you need to provide the URL for your newly installed IIIF image server. e.g.:

```
iiif.image.server = http://localhost:8182/iiif/2/
```

Finally, update `dspace.cfg` or your `local.cfg` file by adding "iiif" to the default event dispatcher, as shown below:

```
event.dispatcher.default.consumers = versioning, discovery, eperson, iiif
```

With these changes in place, DSpace will be ready to respond to IIIF requests. Restart your DSpace backend (i.e. Tomcat) for these changes to all take effect.

Additional Configuration Options

The full set if IIIF configuration options can be found in config/modules/iiif.cfg.

Property	Description
iiif.enabled	Enables the DSpace IIIF service.
iiif.image.server	Base URL path for the IIIF image server. e.g. http://localhost:8182/iiif/2/
iiif.document.viewing.hint	Default viewing hint. Can be overridden with the metadata setting described below.
iiif.logo.image	Optional URL for a small image. This will be included in all IIIF manifests.
iiif.cors.allowed-origins	Comma separated list of allowed CORS origins. The list must include the default value: ${dspace.ui.url}.
iiif.metadata.item	Sets the Dublin Core metadata that will be added to the IIIF resource manifest. This property can be repeated.
iiif.metadata.bitstream	Sets the Bitstream metadata that will be added to the IIIF canvas metadata for individual images. This property can be repeated.
iiif.license.uri	Sets the metadata used for information about the resource usage rights.
iiif.attribution	The text to use as attribution in the iiif manifests. Defaults to: ${dspace.name}
iiif.document.viewing.hint	Either "individuals", "paged" or "continuous". Can be overridden with the metadata setting described below.
iiif.canvas.default-width	Default value for the canvas size. Can be overridden at the item, bundle or bitstream level.
iiif.canvas.default-height	Default value for the canvas size. Can be overridden at the item, bundle or bitstream level.

Canvas Dimensions

As of 7.2, the canvas dimension options (*iiif.canvas.default-width* and *iiif.canvas.default-height*) are updated with additional behaviors.

- If you do not provide your own default dimensions in `iiif.cfg`, DSpace will attempt to optimize canvas dimensions when dimension metadata is missing from the first bitstream in the item. This will often produce more accurate viewer layouts, but note that it is not sufficient to assure accurate layouts in all cases.
- If you decide to add your own default dimensions in `iiif.cfg` file your dimensions are used for every bitstream that lacks dimension metadata.
- You may also set both default dimensions in `iiif.cfg` to the value `-1`. In this case, DSpace creates accurate default dimensions for every bitstream that lacks dimension metadata. Note that this impacts performance.

It is recommended that *iiif.image.width* and *iiif.image.height* metadata be added to Item, Bundle, or Bitstream metadata to assure accurate layout and top performance. Default dimension configurations are intended to improve the user experience when dimension metadata has not yet been added.

CORS Configuration

The wildcard "*" configuration is the default CORS setting for IIIF. With this setting, all remote viewers and applications can retrieve manifests, assuring maximum interoperability. You can restrict CORS origins using the `iiif.cors.allowed-origins` property defined in `iiif.cfg`. Remove the wildcard and add a comma-separated list of origins instead.

IIIF Search API

DSpace includes a plugin to support the IIIF Search API. This plugin is designed to work specifically with the Solr OCR Highlighting Plugin and METS /ALTO data. You are welcome to experiment with the plugin. To do so,uncomment the following settings in `config/modules/iiif.cfg`:

```
iiif.search.url = ${solr.server}/word_highlighting
iiif.search.plugin = org.dspace.app.rest.iiif.service.WordHighlightSolrSearch
```

Once you have successfully indexed ALTO files using the Solr plugin, you can enable search within a DSpace Item by adding the `iiif.search.enabled metadata` field.

Indexing Support

Support for indexing OCR files using the the Solr OCR Highlighting Plugin or other services is not currently provided by DSpace. Institutions will need to develop their own approach to indexing their data.

Enable/Install the Mirador Viewer on Frontend

The Mirador 3.0 viewer is included in the dspace-angular (UI) source code. Before enabling Mirador, be sure to review the instructions for installing the Angular frontend if you haven't already.

To add the Mirador viewer to your DSpace frontend installation, run the following command:

```
# This builds and runs the DSpace UI with the Mirador Viewer in a single step
yarn run start:mirador:prod
```

This will build and copy Mirador to your `dist/` directory and start the frontend server.

The actual steps for deploying the Angular UI with Mirador into Production will likely vary with your setup. However, one possible command-line scenario is the following:

```
# Build Mirador viewer
yarn run build:mirador
# Build DSpace UI for production
yarn run build:prod
# Run the DSpace UI with Mirador viewer
yarn run serve:ssr
```

Running in Development

In the Dspace 7.1 release, the Mirador viewer cannot be used when running in development mode. For now, you need to use a production build.

Configuring Mirador

The Mirador viewer is highly configurable. The Mirador configuration file for DSpace includes a number of settings that you can override manually, including CSS values for styling. Note that some of the Mirador behavior (like the inclusion of thumbnail navigation on the right) is set by the Angular component at runtime. You can choose to override these runtime settings if you like.

Configure IIIF viewer via Metadata Fields

IIIF configuration at the Item-level is quite flexible and is managed using metadata. Canvas sizes, image labels, ranges and and other settings are controlled by using the following fields.

Required Field

Note that the `dspace.iiif.enabled` metadata field **MUST** be added to the Item and set to "true". Otherwise, the Item display will use the default DSpace view.

Schema	Element	Qualifier	Scope	Description
dspace	iiif	enabled	Item	Stores a boolean text value (true or false) to indicate if the iiif feature is enabled or not for the dspace object. If absent the value is derived from the parent dspace object.
iiif	label		Bitstream	Metadata field used to set the IIIF label associated with the canvas resource otherwise the system will derive one according to the configuration setting or the canvas.naming metadata field.
iiif	description		Item	Metadata field used to set the IIIF description associated with the resource.
iiif	canvas	naming	Item	Metadata field used to set the base label used to name all the canvas in the Item. The canvas label will be generated using the value of this metadata as prefix and the canvas position. e.g. Page 1, Page 2, etc.
iiif	viewing	hint	Item	Metadata field used to set the viewing hint overriding the configuration value if any. Possible values are "individuals" and "paged". Default value: individuals.
iiif	image	width	Item, Bundle, or Bitstream	Metadata field used to store the width of an image in pixels. Determines the canvas size.
iiif	image	height	Item, Bundle, or Bitstream	Metadata field used to store the height of an image in pixels. Determines the canvas size.
iiif	toc		Bitstream	Metadata field used to set the position of the iiif resource in the "table of contents" structure.
iiif	search	enabled	Item	Metadata field used to enable the IIIF Search service at the item level. This feature is experimental and requires additional setup.

Multilingual Support

DSpace supports a number of languages & you can even add your own translation. This may also be referred to as Localization (l10n) or Internationalization (i18n).

- Multilingual Support on the Backend (REST API)
  - Where to find the message catalog
  - Where to edit the message catalog
  - Localization of email messages
  - Metadata localization
  - Localization of submission-forms.xml
  - Localization of license.default
- Multilingual Support on the Frontend (UI)

Multilingual Support on the Backend (REST API)

In order to deploy a multilingual version of DSpace you have to configure two parameters in *`[dspace-source]/dspace/config/local.cfg`:*

- *default.locale*, e.g. `default.locale = en`
- *webui.supported locales*, e.g. `webui.supported.locales = en, de`

The Locales might have the form country, country_language, country_language_variant.

According to the languages you wish to support, you have to make sure that all the i18n related files are available.

Where to find the message catalog

The latest **English** message catalog is part of the main DSpace distribution and can be found at: `[dspace-source]/dspace-api/src/main/resources/Messages.properties`

The **different translations** for this message catalog are being managed separately from the DSpace core project, in order to release updates for these files more frequently than the DSpace software itself. Visit the dspace-api-lang project on Github.

Where to edit the message catalog

In some cases you may want to add additional keys to the message catalog or changing the particular wording of DSpace concepts. For example, you may want to change "Communities" into "Departments". These kind of changes may get automatically overwritten again when you upgrade to the newest version of DSpace. It is therefore advised to keep such changes isolated in the following location: `[dspace-source]/dspace/modules/server/src/main/resources/Messages.properties`

After rebuilding DSpace, any messages files placed in this directory will be automatically included in the Server web application. Files of the same name will override any default files. By default, this full directory path may not exist or may be empty. If it does not exist, you can simply create it. You can place any number of translation catalogues in this directory. To add additional translations, just add another copy of the *Messages.properties* file translated into the specific language and country variant you need.

For more information about the `[dspace-source]/dspace/modules/` directory, and how it may be used to "overlay" (or customize) the default Server Webapp, classes and files, please see: Advanced Customisation

Localization of email messages

All email templates used by DSpace can be found in `[dspace]/config/emails/`

The contents of the emails can be edited and translated.

Metadata localization

DSpace associates each metadata field value with a language code (though it may be left empty, e.g. for numeric values).

Localization of submission-forms.xml

The display labels for submission-forms.xml are currently not managed in the messages catalogs. To localize this file, you can create versions of this file in the same folders, appending _COUNTRY at the end of the filename, before the extension. For example, `submission-forms_de.xml` can be used to translate the submission form labels in German.

There is a known bug that any translated submission forms (e.g. submission-forms_de.xml) must include all the form-definitions available in the system. When they are not all included, DSpace will fall back to the default submission forms / locale. See https://github.com/DSpace/DSpace/issues/2827

Localization of license.default

The text in the default submission license (license.default) is currently not managed in the messages catalogs. It is translatable by appending _COUNTRY at the end of the filename, before the extension like for the localization of the input-forms.xml.

Multilingual Support on the Frontend (UI)

By default, DSpace will look at the user's browser language. If it has a language file in the user's language, it will render the interface in that language. If not, it will default to English or another default that you have configured.

The User Interface translations can be found in the `/src/assets/i18n/` folder of your UI's codebase. You can add additional translations & contribute them back to the project. For details see DSpace 7 Translation - Internationalization (i18n) - Localization (l10n)

All translations of the UI are provided in JSON5 format, which includes support for inline comments.

You can choose which languages you wish to enable/support in your UI by modifying the language section of your config.prod.yml file, which in turn, will generate a section like this in your `environment.prod.ts` configuration file:

```
// Default Language in which the UI will be rendered if the user's browser language is not an active language
defaultLanguage: 'en',
// Languages. DSpace Angular holds a message catalog for each of the following languages.
// When set to active, users will be able to switch to the use of this language in the user interface.
languages: [{
  code: 'en',
  label: 'English',
  active: true,
}, {
  code: 'de',
  label: 'Deutsch',
  active: true,
}, {
  code: 'cs',
  label: 'eština',
  active: true,
}, {
  code: 'nl',
  label: 'Nederlands',
  active: true,
}],
```

As shown above, the "defaultLanguage" is the language that your UI will use *by default* , if the user's browser has not specified a preferred language

The array of "languages" are all of the additional languages you wish to support.

- The "code" must match the prefix of a `*.json5` language file located in your `/src/assets/i18n/` folder
- The "label" is the text you want to display in the UI language selector (the globe in the header)
- The "active" setting allows you to decide whether that language appears in the UI language selector or not.

Any changes to the language settings require rebuilding & redeploying your UI.

System Administration

This top level node intends to hold all system administration aspects of DSpace including but not limited to:

- Installation
- Upgrading
- Troubleshooting system errors
- Managing Dependencies

In this context System administration is defined as all technical tasks required to get DSpace in a state in which it operates properly so its behaviour is predictable and can be used according to all the guidelines under "Using DSpace".

Introduction to DSpace System Administration

DSpace operates on several levels: as a Java servlet (in a servlet container like Tomcat), cron jobs, and on-demand operations. This section explains many of the on-demand operations. Some of the command operations may be also set up as cron jobs. Many of these operations are performed at the Command Line Interface (CLI) also known as the Unix prompt ($). Future references will use the term CLI when a command needs to be run at the command line.

Below is the "Command Help Table". This table explains what data is contained in the individual command/help tables in the sections that follow.

Command used:	*The directory and where the command is to be found.*
Java class:	*The actual java program doing the work.*
Arguments:	*The required/mandatory or optional arguments available to the user.*

DSpace Command Launcher

Many/most commands and scripts have a simple `[dspace]/bin/dspace <command>` command. See the Application Layer chapter for the details of the DSpace Command Launcher, and the Command Line Operations guide for common commands.

- AIP Backup and Restore
  - DSpace AIP Format
- Ant targets and options
- Command Line Operations
  - Database Utilities
  - Executing streams of commands
- Handle.Net Registry Support
- Logical Item Filtering and DOI Filtered Provider for DSpace
- Mediafilters for Transforming DSpace Content
  - ImageMagick Media Filters
- Performance Tuning DSpace
- Ping or Healthcheck endpoints for confirming DSpace services are functional
- Scheduled Tasks via Cron
- Search Engine Optimization
  - Google Scholar Metadata Mappings
- Troubleshooting Information
- Validating CheckSums of Bitstreams

AIP Backup and Restore

1 Background & Overview
- 1.1 How does this differ from traditional DSpace Backups? Which Backup route is better?
- 1.2 How does this help backup your DSpace to remote storage or cloud services (like DuraCloud)?
- 1.3 AIPs are Archival Information Packages
- 1.4 AIP Structure / Format

2 Running the Code
- 2.1 Exporting AIPs
  - 2.1.1 Export Modes & Options
  - 2.1.2 Exporting just a single AIP
  - 2.1.3 Exporting AIP Hierarchy
    - 2.1.3.1 Exporting Entire Site
- 2.2 Ingesting / Restoring AIPs
  - 2.2.1 Ingestion Modes & Options
    - 2.2.1.1 The difference between "Submit" and "Restore/Replace" modes
  - 2.2.2 Submitting AIP(s) to create a new object
    - 2.2.2.1 Submitting a Single AIP
    - 2.2.2.2 Submitting an AIP Hierarchy
    - 2.2.2.3 Submitting AIP(s) while skipping any Collection Approval Workflows
  - 2.2.3 Restoring/Replacing using AIP(s)
    - 2.2.3.1 Default Restore Mode
    - 2.2.3.2 Restore, Keep Existing Mode
    - 2.2.3.3 Force Replace Mode
    - 2.2.3.4 Restoring Entire Site
- 2.3 Cleaning up from a failed import
- 2.4 Performance considerations
- 2.5 Disable User Interaction for Cron

3 Command Line Reference
- 3.1 Additional Packager Options
  - 3.1.1 How to use additional options

4 Configuration in 'dspace.cfg'
- 4.1 AIP Metadata Dissemination Configurations
- 4.2 AIP Ingestion Metadata Crosswalk Configurations
- 4.3 AIP Ingestion EPerson Configurations
- 4.4 AIP Configurations To Improve Ingestion Speed while Validating

5 Common Issues or Error Messages

Background & Overview

AIP Backup & Restore doesn't yet work for Configurable Entities

Configurable Entities are not fully supported by AIP Backup & Restore. Since Entities are Items, their metadata and files can be exported/imported via AIPs. However, their relationships to other Entities cannot yet be exported (or imported) via AIPs. Therefore, **restoring Entities via AIP Backup & Restore may result in accidental data loss** (namely loss of relationships). See https://github.com/DSpace/DSpace/issues/2882 for more information.

AIP Backup & Restore functionality only works with the Latest Version of Items

If you are using the Item Level Versioning functionality (disabled by default), you must be aware that this "Item Level Versioning" feature is **not yet compatible** with AIP Backup & Restore. **Using them together may result in accidental data loss.** Currently the AIPs that DSpace generates only store the *latest version* of an Item. Therefore, past versions of Items will always be lost when you perform a restore / replace using AIP tools.

Additional background information available in the Open Repositories 2010 Presentation entitled Improving DSpace Backups, Restores & Migrations

DSpace can backup and restore all of its contents as a set of AIP Files. This includes all Communities, Collections, Items, Groups and People in the system.

This feature came out of a requirement for DSpace to better integrate with DuraCloud, and other backup storage systems. One of these requirements is to be able to essentially "backup" local DSpace contents into the cloud (as a type of offsite backup), and "restore" those contents at a later time.

Essentially, this means DSpace can export the entire hierarchy (i.e. bitstreams, metadata and relationships between Communities/Collections/Items) into a relatively standard format (a METS-based, AIP format). This entire hierarchy can also be re-imported into DSpace in the same format (essentially a restore of that content in the same or different DSpace installation).

Benefits for the DSpace community:

- Allows one to more easily move entire Communities or Collections between DSpace instances.
- Allows for a potentially more consistent backup of this hierarchy (e.g. to DuraCloud, or just to your own local backup system), rather than relying on synchronizing a backup of your Database (stores metadata/relationships) and assetstore (stores files/bitstreams).
- Provides a way for people to more easily get their data out of DSpace (whatever the purpose may be).
- Provides a relatively standard format for people to migrate entire hierarchies (Communities/Collections) from one DSpace to another (or from another system into DSpace).

How does this differ from traditional DSpace Backups? Which Backup route is better?

Traditionally, it has always been recommended to backup and restore DSpace's database and files (also known as the "assetstore") separately. This is described in more detail in the Storage Layer section of the DSpace System Documentation. The traditional backup and restore route is still a recommended and supported option.

However, the new AIP Backup & Restore option seeks to try and resolve many of the complexities of a traditional backup and restore. The below table details some of the differences between these two valid Backup and Restore options.

	Traditional Backup & Restore (Database and Files)	**AIP Backup & Restore**
Supported Backup /Restore Types		
Can Backup & Restore all DSpace Content easily	Yes (Requires two backups/restores – one for Database and one for Files)	Yes (Though, will not backup/restore items which are not officially "in archive")
Can Backup & Restore a Single Community /Collection/Item easily	No (It is possible, but requires a strong understanding of DSpace database structure & folder organization in order to only backup & restore metadata/files belonging to that single object)	Yes
Backups can be used to move one or more Community/Collection /Items to another DSpace system easily.	No (Again, it is possible, but requires a strong understanding of DSpace database structure & folder organization in order to only move metadata/files belonging to that object)	Yes
Can Backup & Restore Item Versions	Yes (Requires two backups/restores – one for Database and one for Files)	**No** (Currently, AIP Backup & Restore is not fully compatible with Item Level Versioning. AIP Backup & Restore can only backup/restore the *latest version* of an Item)
Can Backup & Restore Configurable Entities	Yes (Requires two backups/restores – one for Database and one for Files)	**No** (Currently, AIP Backup & Restore is not fully compatible with Configurable Entities. AIP Backup & Restore can only backup/restore the metadata & files of the Entity, but cannot backup/restore relationships to other Entities)
Supported Object Types During Backup & Restore		
Supports backup/restore of all Communities/Collections /Items (including metadata, files, logos, etc.)	Yes	Yes
Supports backup/restore of all People/Groups /Permissions	Yes	Yes
Supports backup/restore of all Collection-specific Item Templates	Yes	Yes
Supports backup/restore of all Collection Harvesting settings (only for Collections which pull in all Items via OAI-PMH or OAI-ORE)	Yes	No (This is a known issue. All previously harvested Items will be restored, but the OAI-PMH/OAI-ORE harvesting settings will be lost during the restore process.)
Supports backup/restore of all Withdrawn (but not deleted) Items	Yes	Yes
Supports backup/restore of Item Mappings between Collections	Yes	Yes (During restore, the AIP Ingester may throw a false "Could not find a parent DSpaceObject" error (see Common Issues or Error Messages), if it tries to restore an Item Mapping to a Collection that it hasn't yet restored. But this error can be safely bypassed using the 'skipIfParentMissing' flag (see Additional Packager Options for more details).
Supports backup/restore of all in-process, uncompleted Submissions (or those currently in an approval workflow)	Yes	No (AIPs are only generated for objects which are completed and considered "in archive")
Supports backup/restore of Items using custom Metadata Schemas & Fields	Yes	Yes (Custom Metadata Fields will be automatically recreated. Custom Metadata Schemas must be manually created first, in order for DSpace to be able to recreate custom fields belonging to that schema. See Common Issues or Error Messages for more details.)

Supports backup/restore of all local DSpace Configurations and Customizations	Yes (if you backup your **entire** DSpace directory as part of backing up your files)	Not by default (unless you also backup parts of your DSpace directory – note, you wouldn't need to backup the '[dspace]/assetstore' folder again, as those files are already included in AIPs)

Based on your local institutions needs, you will want to choose the backup & restore process which is most appropriate to you. You may also find it beneficial to use both types of backups on different time schedules, in order to keep to a minimum the likelihood of losing your DSpace installation settings or its contents. For example, you may choose to perform a Traditional Backup once per week (to backup your local system configurations and customizations) and an AIP Backup on a daily basis. Alternatively, you may choose to perform daily Traditional Backups and only use the AIP Backup as a "permanent archives" option (perhaps performed on a weekly or monthly basis).

Don't Forget to Backup your Configurations and Customizations

If you choose to use the AIP Backup and Restore option, do not forget to also backup your local DSpace configurations and customizations. Depending on how you manage your own local DSpace, these configurations and customizations are likely in one or more of the following locations:

- `[dspace]` - The DSpace installation directory (Please note, if you also use the AIP Backup & Restore option, you do **not** need to backup your `[dspace]/assetstore` directory, as those files already exist in your AIPs).
- `[dspace-source]` - The DSpace source directory

How does this help backup your DSpace to remote storage or cloud services (like DuraCloud)?

While AIP Backup and Restore is primarily a way to export your DSpace content objects to a local filesystem (or mounted drive), it can also be used as the basis for ensuring your content is safely backed up in a remote location (e.g. DuraCloud or other cloud backup services).

Simply put, these AIPs can be generated and then replicated off to remote storage or a cloud backup service for safe keeping. You can then pull them down either as an entire set, or individually, in order to restore one or more objects into your DSpace instance. While you could simply backup your entire DSpace database and "assetstore" to a cloud service, you'd have to download the **entire** database backup again in order to restore any content. With AIPs, you can instead just download the individual AIP files you need (which can decrease your I/O costs, if any exist) for that restoration.

This upload/download of your AIPs to a backup location can be managed in a manual fashion (e.g. via your own custom code or shell scripts), or you can use a DSpace Replication Task Suite add-on to help ease this process

The Replication Task Suite add-on for DSpace allows you the ability to backup and restore DSpace contents to /from AIPs via the DSpace Administrative Web Interface. It also includes "connectors" to the DuraCloud API, so you can configure it to automatically backup/retrieve your AIPs to/from DuraCloud. Installing this add-on means you can now easily backup and restore DSpace to DuraCloud (or other systems) simply via the DSpace Administrative Web Interface. More information on installing and configuring this add-on can be found on the Replication Task Suite page.

Makeup and Definition of AIPs

AIPs are Archival Information Packages

- AIP is a package describing **one archival object** in DSpace.
  - The **archival object** may be a single **Item**, **Collection**, **Community**, or **Site** (Site AIPs contain site-wide information). Bitstreams are included in an Item's AIP.
  - Each AIP is logically self-contained, can be restored without rest of the archive. (So you could restore a single Item, Collection or Community)
  - Collection or Community AIPs do **not** include all child objects (e.g. Items in those Collections or Communities), as each AIP only describes **one** object. However, these container AIPs do contain references (links) to all child objects. These references can be used by DSpace to automatically restore all referenced AIPs when restoring a Collection or Community.
  - AIPs are only generated for objects which are currently in the "in archive" state in DSpace. This means that in-progress, uncompleted submissions are not described in AIPs and cannot be restored after a disaster. Permanently removed objects will also no longer be exported as AIPs after their removal. However, withdrawn objects will continue to be exported as AIPs, since they are still considered under the "in archive" status.
  - AIPs with identical contents will always have identical checksums. This provides a basic means of validating whether the contents within an AIP have changed. For example, if a Collection's AIP has the same checksum at two different points in time, it means that Collection has not changed during that time period.
  - AIP profile favors completeness and accuracy rather than presenting the semantics of an object in a standard format. It conforms to the quirks of DSpace's internal object model rather than attempting to produce a universally understandable representation of the object. When possible, an AIP tries to use common standards to express objects.
  - An AIP *can* serve as a DIP (Dissemination Information Package) or SIP (Submission Information Package), especially when transferring custody of objects to another DSpace implementation.
  - In contrast to SIP or DIP, the AIP should include all available DSpace structural and administrative metadata, and basic provenance information. AIPs also describe some basic system level information (e.g. Groups and People).

AIP Structure / Format

Generally speaking, an AIP is an Zip file containing a METS manifest and all related content bitstreams.

For more specific details of AIP format / structure, along with examples, please see DSpace AIP Format.

Running the Code

Exporting AIPs

Export Modes & Options

All AIP Exports are done by using the Dissemination Mode (`-d` option) of the `packager` command.

There are two types of AIP Dissemination you can perform:

- **Single AIP** (default, using `-d` option) - Exports just an AIP describing a single DSpace object. So, if you ran it in this default mode for a Collection, you'd just end up with a single Collection AIP (which would not include AIPs for all its child Items)
- **Hierarchy of AIPs** (using the `-d --all` or `-d -a`option) - Exports the requested AIP describing an object, plus the AIP for all child objects. Some examples follow:
    - For a Site - this would export **all** Communities, Collections & Items within the site into AIP files (in a provided directory)
    - For a Community - this would export that Community and all SubCommunities, Collections and Items into AIP files (in a provided directory)
    - For a Collection - this would export that Collection and all contained Items into AIP files (in a provided directory)
    - For an Item – this just exports the Item into an AIP as normal (as it already contains its Bitstreams/Bundles by default)

Exporting just a single AIP

To export in single AIP mode (default), use this "packager" command template:

```
[dspace]/bin/dspace packager -d -t AIP -e <eperson> -i <handle> <file-path>
```

for example:

```
[dspace]/bin/dspace packager -d -t AIP -e admin@myu.edu -i 4321/4567 aip4567.zip
```

The above code will export the object of the given handle (4321/4567) into an AIP file named "aip4567.zip". This will **not** include any child objects for Communities or Collections.

Exporting AIP Hierarchy

To export an AIP hierarchy, use the `-a` (or `--all`) package parameter.

For example, use this 'packager' command template:

```
[dspace]/bin/dspace packager -d -a -t AIP -e <eperson> -i <handle> <file-path>
```

for example:

```
[dspace]/bin/dspace packager -d -a -t AIP -e admin@myu.edu -i 4321/4567 aip4567.zip
```

The above code will export the object of the given handle (4321/4567) into an AIP file named "aip4567.zip". In addition it would export all children objects to the same directory as the "aip4567.zip" file. The child AIP files are all named using the following format:

- File Name Format: `<Obj-Type>@<Handle-with-dashes>.zip`
    - e.g. COMMUNITY@123456789-1.zip, COLLECTION@123456789-2.zip, ITEM@123456789-200.zip
    - This general file naming convention ensures that you can easily locate an object to restore by its name (assuming you know its Object Type and Handle).
- Alternatively, if object doesn't have a Handle, it uses this File Name Format: `<Obj-Type>@internal-id-<DSpace-ID>.zip` (e.g. ITEM@internal-id-234.zip)

AIPs are only generated for objects which are currently in the "in archive" state in DSpace. This means that in-progress, uncompleted submissions are not described in AIPs and cannot be restored after a disaster.

Exporting Entire Site

To export an entire DSpace Site, pass the packager the Handle `<site-handle-prefix>/0`. For example, if your site prefix is "4321", you'd run a command similar to the following:

```
[dspace]/bin/dspace packager -d -a -t AIP -e admin@myu.edu -i 4321/0 sitewide-aip.zip
```

Again, this would export the DSpace Site AIP into the file "sitewide-aip.zip", and export AIPs for **all** Communities, Collections and Items into the same directory as the Site AIP.

Ingesting / Restoring AIPs

Ingestion Modes & Options

Ingestion of AIPs is a bit more complex than Dissemination, as there are several different "modes" available:

1. Submit/Ingest Mode (`-s` option, default) – submit AIP(s) to DSpace in order to create a new object(s) (i.e. AIP is treated like a SIP – Submission Information Package)
2. Restore Mode (`-r` option) – restore pre-existing object(s) in DSpace based on AIP(s). This also attempts to restore all handles and relationships (parent/child objects). This is a specialized type of "submit", where the object is created with a known Handle, known UUID and known relationships.
3. Replace Mode (`-r -f` option) – replace existing object(s) in DSpace based on AIP(s). This also attempts to restore all handles and relationships (parent/child objects). This is a specialized type of "restore" where the contents of existing object(s) is replaced by the contents in the AIP(s). By default, if a normal "restore" finds the object already exists, it will back out (i.e. rollback all changes) and report which object already exists.

Again, like export, there are two types of AIP Ingestion you can perform (using any of the above modes):

- **Single AIP** (default) - Ingests just an AIP describing a single DSpace object. So, if you ran it in this default mode for a Collection AIP, you'd just create a DSpace Collection from the AIP (but not ingest any of its child objects)
- **Hierarchy of AIPs** (by including the `--all` or `-a`option after the mode) - Ingests the requested AIP describing an object, plus the AIP for all child objects. Some examples follow:
  - For a Site - this would ingest **all** Communities, Collections & Items based on the located AIP files
  - For a Community - this would ingest that Community and all SubCommunities, Collections and Items based on the located AIP files
  - For a Collection - this would ingest that Collection and all contained Items based on the located AIP files
  - For an Item – this just ingest the Item (including all Bitstreams & Bundles) based on the AIP file.

The difference between "Submit" and "Restore/Replace" modes

It's worth understanding the primary differences between a Submission (specified by `-s` parameter) and a Restore (specified by `-r` parameter).

- **Submission Mode** (`-s` mode) - creates a new object (AIP is treated like a SIP)
  - By default, a new Handle is always assigned
    - However, you can force it to use the handle specified in the AIP by specifying `-o ignoreHandle=false` as one of your parameters
  - By default, a new Parent object **must** be specified (using the `-p` parameter). This is the location where the new object will be created.
    - However, you can force it to use the parent object specified in the AIP by specifying `-o ignoreParent=false` as one of your parameters
  - By default, will respect a Collection's Workflow process when you submit an Item to a Collection
    - However, you can specifically *skip* any workflow approval processes by specifying `-w` parameter.
  - **Always** adds a new Deposit License to Items
  - **Always** adds new DSpace System metadata to Items (includes new "dc.date.accessioned", "dc.date.available", "dc.date.issued" and "dc.description.provenance" entries)
  - **WARNING:** Submission mode may not be able to maintain Item Mappings between Collections. Because these mappings are recorded via the Collection Handles, mappings may be restored improperly if the Collection handle has changed when moving content from one DSpace instance to another.

- **Restore / Replace Mode** (`-r` mode) - restores a previously existing object (as if from a backup)
  - By default, the Handle specified in the AIP is restored
    - However, for restores, you can force a new handle to be generated by specifying `-o ignoreHandle=true` as one of your parameters. (NOTE: Doesn't work for *replace* mode as the new object always retains the handle of the replaced object)
    - Restore/Replace restores Handles as well as UUIDs. (*NOTE: UUID restoration only possible in 7.1 or above*)
  - By default, the object is restored under the Parent specified in the AIP
    - However, for restores, you can force it to restore under a different parent object by using the `-p` parameter. (NOTE: Doesn't work for *replace* mode, as the new object always retains the parent of the replaced object)
  - **Always** skips any Collection workflow approval processes when restoring/replacing an Item in a Collection
  - **Never** adds a new Deposit License to Items (rather it restores the previous deposit license, as long as it is stored in the AIP)
  - **Never** adds new DSpace System metadata to Items (rather it just restores the metadata as specified in the AIP)

Changing Submission/Restore Behavior

It is possible to change some of the default behaviors of both the Submission and Restore/Replace Modes. Please see the Additional Packager Options section below for a listing of command-line options that allow you to override some of the default settings described above.

Submitting AIP(s) to create a new object

The Submission mode (`-s`) always creates a new object with a newly assigned handle. In addition by default it respects all existing Collection approval workflows (so items may require approval unless the workflow is skipped by using the `-w` option). For information about how the "Submission Mode" differs from the "Replace / Restore mode", see The difference between "Submit" and "Restore/Replace" modes above.

Submitting a Single AIP

AIPs treated as SIPs

This option allows you to essentially use an AIP as a SIP (Submission Information Package). The default settings will create a new DSpace object (with a new handle and a new parent object, if specified) from your AIP.

To ingest a single AIP and create a new DSpace object under a parent of your choice, specify the `-p` (or `--parent`) package parameter to the command. Also, note that you are running the `packager` in `-s` (submit) mode.

NOTE: This only ingests the single AIP specified. It does **not** ingest all children objects.

```
[dspace]/bin/dspace packager -s -t AIP -e <eperson> -p <parent-handle> <file-path>
```

If you leave out the `-p` parameter, the AIP package ingester will attempt to install the AIP under the same parent it had before. As you are also specifying the `-s` (submit) parameter, the `packager` will assume you want a new Handle to be assigned (as you are effectively specifying that you are submitting a **new** object). If you want the object to retain the Handle specified in the AIP, you can specify the `-o ignoreHandle=false` option to force the packager to *not* ignore the Handle specified in the AIP.

Submitting an AIP Hierarchy

AIPs treated as SIPs

This option allows you to essentially use a set of AIPs as SIPs (Submission Information Packages). The default settings will create a new DSpace object (with a new handle and a new parent object, if specified) from each AIP

To ingest an AIP hierarchy from a directory of AIPs, use the `-a` (or `--all`) package parameter.

For example, use this 'packager' command template:

```
[dspace]/bin/dspace packager -s -a -t AIP -e <eperson> -p <parent-handle> <file-path>
```

for example:

```
[dspace]/bin/dspace packager -s -a -t AIP -e admin@myu.edu -p 4321/12 aip4567.zip
```

The above command will ingest the package named "aip4567.zip" as a child of the specified Parent Object (handle="4321/12"). The resulting object is assigned a new Handle (since `-s` is specified). In addition, any child AIPs referenced by "aip4567.zip" are also recursively ingested (a new Handle is also assigned for each child AIP).

Another example – **Ingesting a Top-Level Community** (by using the Site Handle, `<site-handle-prefix>/0`):

```
[dspace]/bin/dspace packager -s -a -t AIP -e admin@myu.edu -p 4321/0 community-aip.zip
```

The above command will ingest the package named "community-aip.zip" as a **top-level community** (i.e. the specified parent is "4321/0" which is a Site Handle). Again, the resulting object is assigned a new Handle. In addition, any child AIPs referenced by "community-aip.zip" are also recursively ingested (a new Handle is also assigned for each child AIP).

May want to skip Collection Approvals Workflows

Please note: If you are submitting a larger amount of content (e.g. multiple Communities/Collections) to your DSpace, you may want to tell the 'packager' command to skip over any existing Collection approval workflows by using the `-w` flag. By default, all Collection approval workflows will be respected. This means if the content you are submitting includes a Collection with an enabled workflow, you may see the following occur:

1. First, the Collection will be created & its workflow enabled
2. Second, each Item belonging to that Collection will be created & placed into the workflow approval process

   Therefore, if this content has already received some level of approval, you may want to submit it using the `-w` flag, which will skip any workflow approval processes. For more information, see Submitting AIP(s) while skipping any Collection Approval Workflows.

Item Mappings may not be maintained when submitting an AIP hierachy

When an Item is mapped to one or more Collections, this mapping is recorded in the AIP using the mapped Collection's handle. Unfortunately, since the submission mode (-s) assigns **new handles** to all objects in the hierarchy, this may mean that the mapped Collection's handle will have changed (or even that a different Collection will be available at the original mapped Collection's handle). DSpace does not have a way to uniquely identify Collections other than by handle, which means that item mappings are only able to be retained when the Collection handle is *also retained.*

If you encounter this issue, there are a few possible workarounds:

1. Use the restore/replace mode (-r) instead, as it will retain existing Collection Handles. Unfortunately though, this may not work if the content is being moved from a Test DSpace to a Production DSpace, as these existing handles may not be valid.
2. OR, use the submission mode with the "--o ignoreHandle=false". This will also retain existing Collection Handles. Unfortunately though, this may not work if the content is being moved from a Test DSpace to a Production DSpace, as these existing handles may not be valid.
3. OR, remove all existing Item Mappings and re-export AIPs (without Item Mappings). Then, import the hierarchy into the new DSpace instance (again without Item Mappings). Finally, recreate the necessary Item Mappings using a different tool, e.g. the Batch Metadata Editing tool supports bulk editing of Collection memberships/mappings.

Missing Groups or EPeople cannot be created when submitting an individual Community or Collection AIP

Please note, if you are using AIPs to move an entire Community or Collection from one DSpace to another, there is a known issue (see https://github.com/DSpace/DSpace/issues/4477) that the new DSpace instance will be unable to (re-)create any DSpace Groups or EPeople which are referenced by a Community or Collection AIP. The reason is that the Community or Collection AIP itself doesn't contain enough information to create those Groups or EPeople (rather that info is stored in the SITE AIP, for usage during Full Site Restores).

However, there are two possible ways to get around this known issue:

- **EITHER**, you can manually recreate all referenced Groups/EPeople in the new DSpace that you are submitting the Community or Collection AIP into.
- **OR**, you can temporarily disable the import of Group/EPeople information when submitting the Community or Collection AIP to the new DSpace. This would mean that after you submit the AIP to the new DSpace, you'd have to manually go in and add in any special permissions (as needed). To disable the import of Group/EPeople information, add these settings to your `dspace.cfg`file, and re-run the submission of the AIP with these settings in place:

```
mets.dspaceAIP.ingest.crosswalk.METSRIGHTS = NIL
mets.dspaceAIP.ingest.crosswalk.DSPACE-ROLES = NIL
```

  - Don't forget to remove these settings after you import your Community or Collection AIP. *Leaving them in place will mean that every time you import an AIP, all of its Group/EPeople/Permissions would be ignored.*

Submitting AIP(s) while skipping any Collection Approval Workflows

By default, the Submission mode (`-s`) always respects existing Colleciton approval workflows. So, if a Collection has a workflow, then a newly submitted Item will be placed into that workflow process (rather than immediately appearing in DSpace).

However, if you'd like to skip all workflow approval processes you can use the `-w` flag to do so. For example, the following command will skip any Collection approval workflows and immediately add the Item to a Collection.

```
[dspace]/bin/dspace packager -s -w -t AIP -e <eperson> -p <parent-handle> <file-path>
```

This `-w` flag may also be used when Submitting an AIP Hierarchy. For example, if you are migrating one or more Collections/Communities from one DSpace to another, you may choose to submit those AIPs with the `-w` option enabled. This will ensure that, if a Collection has a workflow approval process enabled, all its Items are available immediately rather than being all placed into the workflow approval process.

Restoring/Replacing using AIP(s)

Restoring is slightly different than just **submitting**. When restoring, we make every attempt to restore the object as it **used to be** (including its handle, parent object, etc.). For more information about how the "Replace/Restore Mode" differs from the "Submit mode", see The difference between "Submit" and "Restore/Replace" modes above.

There are currently three restore modes:

1. Default Restore Mode (`-r`) = Attempt to restore object (and optionally children). Rollback all changes if any object is found to already exist.
2. Restore, Keep Existing Mode (`-r -k`) = Attempt to restore object (and optionally children). If an object is found to already exist, skip over it (and all children objects), and continue to restore all other non-existing objects.
3. Force Replace Mode (`-r -f`) = Restore an object (and optionally children) and **overwrite** any existing objects in DSpace. Therefore, if an object is found to already exist in DSpace, its contents are replaced by the contents of the AIP. *WARNING: This mode is potentially dangerous as it will permanently destroy any object contents that do not currently exist in the AIP. You may want to perform a secondary backup, unless you are sure you know what you are doing!*

Default Restore Mode

By default, the restore mode (`-r` option) will throw an error and rollback all changes if any object is found to already exist. The user will be informed if which object already exists within their DSpace installation.

Restore a Single AIP: Use this 'packager' command template to restore a single object from an AIP (not including any child objects):

```
[dspace]/bin/dspace packager -r -t AIP -e <eperson> <AIP-file-path>
```

Restore a Hierarchy of AIPs: Use this 'packager' command template to restore an object from an AIP along with all child objects (from their AIPs):

```
[dspace]/bin/dspace packager -r -a -t AIP -e <eperson> <AIP-file-path>
```

For example:

```
[dspace]/bin/dspace packager -r -a -t AIP -e admin@myu.edu aip4567.zip
```

Notice that unlike `-s` option (for submission/ingesting), the `-r` option does not require the Parent Object (`-p` option) to be specified if it can be determined from the package itself.

In the above example, the package "aip4567.zip" is restored to the DSpace installation with the Handle provided within the package itself (and added as a child of the parent object specified within the package itself). In addition, any child AIPs referenced by "aip4567.zip" are also recursively ingested (the `-a` option specifies to also restore all child AIPs). They are also restored with the Handles & Parent Objects provided with their package. If any object is found to already exist, all changes are rolled back (i.e. nothing is restored to DSpace)

Highly Recommended to Update Database Sequences after a Large Restore

In some cases, when you restore a large amount of content to your DSpace, the internal database counts (called "sequences") may get out of sync with the Handles of the content you just restored. As a best practice, it is **highly recommended to always** re-run the "update-sequences" script on your DSpace database after a larger scale restore. This database script should be run while DSpace is stopped (you may either stop Tomcat or just the DSpace webapps). PostgreSQL/Oracle must be running. Simply run:
`[dspace]/bin/dspace database update-sequences`

More Information on using Default Restore Mode with Community/Collection AIPs

- Using the Default Restore Mode without the `-a` option, will only restore the **metadata** for that specific Community or Collection. No child objects will be restored.
- Using the Default Restore Mode with the `-a` option, will only successfully restore a Community or Collection if that object along with any child objects (Sub-Communities, Collections or Items) do not already exist. In other words, if any objects belonging to that Community or Collection already exist in DSpace, the Default Restore Mode will report an error that those object(s) could not be recreated. If you encounter this situation, you will need to perform the restore using either the Restore, Keep Existing Mode *or* the Force Replace Mode (depending on whether you want to keep or replace those existing child objects).

Restore, Keep Existing Mode

When the "Keep Existing" flag (`-k` option) is specified, the restore will attempt to skip over any objects found to already exist. It will report to the user that the object was found to exist (and was not modified or changed). It will then continue to restore all objects which do not already exist.

One special case to note: If a Collection or Community is found to already exist, its child objects are also skipped over. So, this mode will not auto-restore items to an existing Collection.

Restore a Hierarchy of AIPs: Use this 'packager' command template to restore an object from an AIP along with all child objects (from their AIPs):

```
[dspace]/bin/dspace packager -r -a -k -t AIP -e <eperson> <AIP-file-path>
```

For example:

```
[dspace]/bin/dspace packager -r -a -k -t AIP -e admin@myu.edu aip4567.zip
```

In the above example, the package "aip4567.zip" is restored to the DSpace installation with the Handle provided within the package itself (and added as a child of the parent object specified within the package itself). In addition, any child AIPs referenced by "aip4567.zip" are also recursively restored (the `-a` option specifies to also restore all child AIPs). They are also restored with the Handles & Parent Objects provided with their package. If any object is found to already exist, it is skipped over (child objects are also skipped). All non-existing objects are restored.

Force Replace Mode

When the "Force Replace" flag (`-f` option) is specified, the restore will **overwrite** any objects found to already exist in DSpace. In other words, existing content is deleted and then replaced by the contents of the AIP(s).

May also be useful in some specific restoration scenarios

This mode may also be used to restore missing objects which refer to existing objects. For example, if you are restoring a missing Collection which had existing Items linked to it, you can use this mode to auto-restore the Collection and update those existing Items so that they again link back to the newly restored Collection.

Potential for Data Loss

Because this mode actually **destroys** existing content in DSpace, it is potentially dangerous and may result in data loss! You may wish to perform a secondary full backup (assetstore files & database) before attempting to replace any existing object(s) in DSpace.

Replace using a Single AIP: Use this 'packager' command template to replace a single object from an AIP (not including any child objects):

```
[dspace]/bin/dspace packager -r -f -t AIP -e <eperson> <AIP-file-path>
```

Replace using a Hierarchy of AIPs: Use this 'packager' command template to replace an object from an AIP along with all child objects (from their AIPs):

```
[dspace]/bin/dspace packager -r -a -f -t AIP -e <eperson> <AIP-file-path>
```

For example:

```
[dspace]/bin/dspace packager -r -a -f -t AIP -e admin@myu.edu aip4567.zip
```

In the above example, the package "aip4567.zip" is restored to the DSpace installation with the Handle provided within the package itself (and added as a child of the parent object specified within the package itself). In addition, any child AIPs referenced by "aip4567.zip" are also recursively ingested. They are also restored with the Handles & Parent Objects provided with their package. *If any object is found to already exist, its contents are replaced by the contents of the appropriate AIP.*

If any error occurs, the script attempts to rollback the entire replacement process.

Restoring Entire Site

In order to restore an entire Site from a set of AIPs, you must do the following:

1. Install a completely "fresh" version of DSpace by following the Installation instructions in the DSpace Manual
    - At this point, you should have a completely empty, but fully-functional DSpace installation. You will need to create an initial Administrator user in order to perform this restore (as a full-restore can only be performed by a DSpace Administrator).
2. Once DSpace is installed, run the following command to restore all its contents from AIPs

    ```
    [dspace]/bin/dspace packager -r -a -f -t AIP -e <eperson> -i <site-handle-prefix>/0 -o
    skipIfParentMissing=true /full/path/to/your/site-aip.zip
    ```

    a. While the "`-o skipIfParentMissing=true`" flag is optional, it is often necessary whenever you are performing a large hierarchical site restoration. Please see the Additional Packager Options section below.

Please note the following about the above restore command:

- Notice that you are running this command in "Force Replace" mode (`-r -f`). This is necessary as your empty DSpace install will already include a few default groups (Administrators and Anonymous) and your initial administrative user. You need to replace these groups in order to restore your prior DSpace contents completely.
- `<eperson>` should be replaced with the Email Address of the initial Administrator (who you created when you reinstalled DSpace).
- `<site-handle-prefix>` should be replaced with your DSpace site's assigned Handle Prefix. This is equivalent to the `handle.prefix` setting in your `dspace.cfg`
- `/full/path/to/your/site-aip.zip` is the full path to the AIP file which represents your DSpace SITE. This file will be named whatever you named it when you actually exported your entire site. All other AIPs are assumed to be referenced from this SITE AIP (in most cases, they should be in the same directory as that SITE AIP).

Highly Recommended to Update Database Sequences after a Large Restore

In some cases, when you restore a large amount of content to your DSpace, the internal database counts (called "sequences") may get out of sync with the Handles of the content you just restored. As a best practice, it is **highly recommended to always** re-run the "update-sequences" script on your DSpace database after a larger scale restore. This database script should be run while DSpace is stopped (you may either stop Tomcat or just the DSpace webapps). PostgreSQL/Oracle must be running. Simply run:

```
[dspace]/bin/dspace database update-sequences
```

Cleaning up from a failed import

Sometimes your packager import of AIP packages can fail, due to lack of memory (see below for advice on better performance, please use JAVA_OPTS to set your memory higher than the default). If that happens, DSpace by design will leave the bitstreams it **did** import sucessfully, but they will be oprphaned, and will just occupy space in your assetstore. The standard DSpace cleanup cron job will clean up these orphaned bitstreams, however, you can also clean them up manually by running the following command:

Clean up after a failed import

```
[dspace]/bin/dspace cleanup -v
```

Performance considerations

When importing large structures like the whole site or a large collection/community, keep in mind that this can require a lot of memory, more than the default amount of heap allocated to the command-line launcher (256 Mb: `JAVA_OPTS="-Xmx256m -Dfile.encoding=UTF-8"`). This memory must be allocated in addition to the normal amount of memory allocated to Tomcat. For example, a site of 2500 fulltext items (2 Gb altogether) requires 5 Gb of maximum heap space and takes around 1 hour, including import and indexing.

You can raise the limit for a single run of the packager command by specifying memory options in the JAVA_OPTS environment variable, e.g.:

```
JAVA_OPTS="-Xmx4096m -Dfile.encoding=UTF-8" /dspace/bin/dspace packager -u -r -a -f -t AIP -e dspace@example.
com -i 123456789/0 sitewide-aip.zip
```

If the importer runs out of heap memory, it will crash either with "java.lang.OutOfMemoryError: GC overhead limit exceeded", which can be suppressed by adding "-XX:-UseGCOverheadLimit" to JAVA_OPTS, or with "java.lang.OutOfMemoryError: Java heap space". You can increase the allocated heap memory and try again, but keep in mind that although no changes were made in the database, the unsuccessfully imported files are still left in the assetstore (see https://github.com/DSpace/DSpace/issues/5593).

Disable User Interaction for Cron

If you wish to run any of the following commands from a cron job (or similar), then you may wish to **disable all user interaction** using the `-u` (`--no-user-interaction`) flag. For example, supposing you wanted to perform a full Site Backup (see Exporting Entire Site above) via a cronjob, you could simply run that command passing it the "-u" flage like this:

```
# Perform a full site backup to AIPs(with user interaction disabled) every Sunday at 1:00AM
# NOTE: Make sure to replace "123456789" with your actual Handle Prefix, and "admin@myu.edu" with your
Administrator account email.
0 1 * * * [dspace]/bin/dspace packager -u -d -a -t AIP -e admin@myu.edu -i 123456789/0 [full-path-to-backup-
folder]/sitewide-aip.zip
```

Command Line Reference

The following flags are valid to pass to the `[dspace]/bin/dspace packager` command:

Flag	Ingest or Export	Description / Usage
`-a (--all)`	both ingest and export	*For Ingest*: recursively ingest all child AIPs (referenced from this AIP). *For Export*: recursively export all child objects (referenced from this parent object)
`-d (--disseminate)`	export-only	This flag simply triggers the export of AIPs from the system. See Exporting AIPs
`-e (–eperson) [email-address]`	ingest-only	The email address of the EPerson who is ingesting the AIPs. Oftentimes this should be an Administrative account.
`-f (--force-replace)`	ingest-only	Ingest the AIPs in "Force Replace Mode" (must be specified in conjunction with `-r` flag), where existing objects will be replaced by the contents of the AIP.

`-h (--help)`	both ingest and export	Return help information. You should specify with `-t` for additional package specific help information
`-i (--identifier) [handle]`	both ingest and export	*For Ingest*: Only valid in "Force Replace Mode". In that mode this is the identifier of the object to replace. *For Export:* The identifier of the object to export to an AIP
`-k (--keep-existing)`	ingest-only	Specifies to use "Restore, Keep Existing Mode" during ingest (must be specified in conjunction with `-r` flag). In this mode, existing objects in DSpace will NOT be replaced by their AIPs, but missing objects will be restored from AIPs.
`-o (--option) [setting]=[value]`	both ingest and export	This flag is used to pass Additional Packager Options to the Packager command. Each type of packager may define its own custom Additional Options. For AIPs, the valid options are documented in the Additional Packager Options section below. This is repeatable (e.g. `-o [setting1]=[value] -o [setting2]=value`)
`-p (--parent) [handle]`	ingest only	Handle(s) of the parent Community or Collection to into which an AIP should be ingested. This may be repeatable.
`-r (--restore)`	ingest only	Specifies that this ingest is either "Restore Mode" (when standalone), "Restore, Keep Existing Mode" (when used with `-k` flag) or "Force Replace Mode" (when used with `-f` flag)
`-s (--submit)`	ingest only	Specifies that this ingest is in "Submit Mode" where an AIP is treated as a **new** object and assigned a new Handle /Identifier, etc.
`-t (--type) [package-type]`	both ingest and export	Specifies the type of package which is being ingested or exported. This controls which Ingester or Disseminator class is called. For AIPs, this is **always** set to "`-t AIP`"
`-u (--no-user-interaction)`	both ingest and export	Skips over all user interaction (e.g. question prompts). This flag can be used when running the packager from a script or cron job to bypass all user interaction. See also Disable User Interaction for Cron

Additional Packager Options

In additional to the various "modes" settings described under "Running the Code" above, the AIP Packager supports the following packager options. These options allow you to better tweak how your AIPs are processed (especially during ingests/restores/replaces).

Option	Ingest or Export	Default Value	Description
`filterBundles=[value]`	export-only	defaults to exporting all Bundles	This option can be used to limit the Bundles which are exported to AIPs for each DSpace Item. By default, all file Bundles will be exported into Item AIPs. You could use this option to limit the size of AIPs by only exporting certain Bundles. *WARNING: any bundles* **not** *included in AIPs will obviously be unable to be restored.* This option can be run in two ways: • **Exclude Bundles:** By default, you can provide a comma-separated list of bundles to be excluded from AIPs (e.g. "TEXT, THUMBNAIL") • **Include Bundles:** If you prepend the list with the "+" symbol, then the list specifies the bundles to be **included** in AIPs (e.g. "+ORIGINAL,LICENSE" would only include those two bundles). This second option is identical to using "includeBundles" option described below. (NOTE: If you choose to no longer export LICENSE or CC_LICENSE bundles, you will also need to disable the License Dissemination Crosswalks in the `aip.disseminate.rightsMD` configuration for the changes to take affect)
`ignoreHandle=[value]`	ingest-only	Restore /Replace Mode defaults to 'false', Submit Mode defaults to 'true'	If 'true', the AIP ingester will ignore any Handle specified in the AIP itself, and instead create a new Handle during the ingest process (this is the default when running in Submit mode, using the `-s` flag). If 'false', the AIP ingester attempts to restore the Handles specified in the AIP (this is the default when running in Restore/replace mode, using the `-r` flag).

`ignoreParent=[value]`	ingest-only	Restore/Replace Mode defaults to 'false', Submit Mode defaults to 'true'	If 'true', the AIP ingester will ignore any Parent object specified in the AIP itself, and instead ingest under a new Parent object (this is the default when running in Submit mode, using the `-s` flag). The new Parent object must be specified via the `-p` flag (run `dspace packager -h` for more help). If 'false', the AIP ingester attempts to restore the object directly under its old Parent (this is the default when running in Restore/replace mode, using the `-r` flag).
`includeBundles=[value]`	export-only	defaults to "all"	This option can be used to limit the Bundles which are exported to AIPs for each DSpace Item. By default, all file Bundles will be exported into Item AIPs. You could use this option to limit the size of AIPs by only exporting certain Bundles. *WARNING: any bundles **not** included in AIPs will obviously be unable to be restored.* This option expects a comma separated list of bundle names (e.g. "ORIGINAL,LICENSE,CC_LICENSE,METADATA"), or "all" if all bundles should be included. (See "filterBundles" option above if you wish to exclude particular Bundles. However, this "includeBundles" option **cannot** be used at the same time as "filterBundles".) (NOTE: If you choose to no longer export LICENSE or CC_LICENSE bundles, you will also need to disable the License Dissemination Crosswalks in the `aip.disseminate.rightsMD` configuration for the changes to take affect)
`manifestOnly=[value]`	both ingest and export	false	If 'true', the AIP Disseminator will only import/export a METS Manifest XML file (i.e. result will be an unzipped 'mets.xml' file), instead of a full AIP. This METS Manifest contains URI references to all content files, but does *not* contain any content files. **This option is experimental and is meant for debugging purposes only. It should never be set to 'true' if you want to be able to restore content files.** Again, please note that when you use this option, the final result will be an XML file, NOT the normal ZIP-based AIP format.
`passwords=[value]`	export-only	false	If 'true' (and the 'DSPACE-ROLES' crosswalk is enabled, see #AIP Metadata Dissemination Configurations), then the AIP Disseminator will export user password hashes (i.e. encrypted passwords) into Site AIP's METS Manifest. This would allow you to restore user's passwords from Site AIP. If 'false', then user password hashes are not stored in Site AIP, and passwords cannot be restored at a later time.
`skipIfParentMissing=[value]`	ingest-only	false	If 'true', ingestion will skip over any "Could not find a parent DSpaceObject" errors that are encountered during the ingestion process (Note: those errors will still be logged as "warning" messages in your DSpace log file). If you are performing a full site restore (or a restore of a larger Community/Collection hierarchy), you may encounter these errors if you have a larger number of Item mappings between Collections (i.e. Items which are mapped into several collections at once). When you are performing a recursive ingest, skipping these errors should not cause any problems. Once the missing parent object is ingested it will automatically restore the Item mapping that caused the error. For more information on this "Could not find a parent DSpaceObject" error see Common Issues or Error Messages.
`unauthorized=[value]`	export-only	*unspecified*	If 'skip', the AIP Disseminator will skip over any unauthorized Bundle or Bitstream encountered (i.e. it will not be added to the AIP). If 'zero', the AIP Disseminator will add a Zero-length "placeholder" file to the AIP when it encounters an unauthorized Bitstream. If unspecified (the default value), the AIP Disseminator will throw an error if an unauthorized Bundle or Bitstream is encountered.
`updatedAfter=[value]`	export-only	*unspecified*	This option works as a basic form of "incremental backup". This option requires that an ISO-8601 date is specified. When specified, the AIP Disseminator will only export Item AIPs which have a last-modified date **after** the specified ISO-8601 date. This option has no affect on the export of Site, Community or Collection AIPs as DSpace does not record a last-modified date for Sites, Communities or Collections. For example, when this option is specified during a full-site export, the AIP Disseminator will export the Site AIP, all Community AIPs, all Collection AIPs, and only Item AIPs modified after that date and time.
`validate=[value]`	both ingest and export	Export defaults to 'true', Ingest defaults to 'false'	If 'true', every METS file in AIP will be validated before ingesting or exporting. By default, DSpace will validate everything on export, but will skip validation during import. Validation on export will ensure that all exported AIPs properly conform to the METS profile (and will throw errors if any do not). Validation on import will ensure every METS file in every AIP is first validated before importing into DSpace (this will cause the ingestion processing to take longer, but tips on speeding it up can be found in the "AIP Configurations To Improve Ingestion Speed while Validating" section below). *DSpace recommends minimally validating AIPs on export. Ideally, you should validate both on export and import, but import validation is disabled by default in order to increase the speed of AIP restores.*

How to use additional options

These options can be passed in two main ways:

From the Command Line

From the command-line, you can add the option to your command by using the `-o` or `--option` parameter.

```
 [dspace]/bin/dspace packager -r -a -t AIP -o [option1]=[value] -o [option2]=[value] -e admin@myu.edu aip4567.zip
```

For example:

```
 [dspace]/bin/dspace packager -r -a -t AIP -o ignoreParent=false -o createMetadataFields=false -e admin@myu.edu aip4567.zip
```

Via the Java API call

If you are programmatically calling the `org.dspace.content.packager.DSpaceAIPIngester` from your own custom script, you can specify these options via the `org.dspace.content.packager.PackageParameters` class.

As a basic example:

```
PackageParameters params = new PackageParameters;
params.addProperty("createMetadataFields", "false");
params.addProperty("ignoreParent", "true");
```

Configuration in 'dspace.cfg'

The following new configurations relate to AIPs:

AIP Metadata Dissemination Configurations

The following configurations allow you to specify what metadata is stored within each METS-based AIP. In 'dspace.cfg', the general format for each of these settings is:

- `aip.disseminate.<setting> = <mdType>:<DSpace-crosswalk-name> [, ...]`
  - <setting> is the setting name (see below for the full list of valid settings)
  - <mdType> is optional. It allows you to specify the value of the @MDTYPE or @OTHERMDTYPE attribute in the corresponding METS element.
  - <DSpace-crosswalk-name> is required. It specifies the name of the DSpace Crosswalk which should be used to generate this metadata.
  - Zero or more `<label-for-METS>:<DSpace-crosswalk-name>` may be specified for each setting

AIP Metadata Recommendations

It is recommended to **minimally** use the default settings when generating AIPs. DSpace can only restore information that is included within an AIP. Therefore, if you choose to no longer include some information in an AIP, DSpace will no longer be able to restore that information from an AIP backup

The default settings in 'dspace.cfg' are:

- `aip.disseminate.techMD` - Lists the DSpace Crosswalks (by name) which should be called to populate the `<techMD>` section of the METS file within the AIP (Default: `PREMIS, DSPACE-ROLES`)
  - The `PREMIS` crosswalk generates PREMIS metadata for the object specified by the AIP
  - The `DSPACE-ROLES` crosswalk exports DSpace Group / EPerson information into AIPs in a DSpace-specific XML format. Using this crosswalk means that AIPs can be used to recreated Groups & People within the system. (NOTE: The `DSPACE-ROLES` crosswalk should be used alongside the `METSRights` crosswalk if you also wish to restore the *permissions* that Groups/People have within the System. See below for more info on the `METSRights` crosswalk.)
- `aip.disseminate.sourceMD` - Lists the DSpace Crosswalks (by name) which should be called to populate the `<sourceMD>` section of the METS file within the AIP (Default: `AIP-TECHMD`)
  - The AIP-TECHMD Crosswalk generates technical metadata (in DIM format) for the object specified by the AIP
- `aip.disseminate.digiprovMD` - Lists the DSpace Crosswalks (by name) which should be called to populate the `<digiprovMD>` section of the METS file within the AIP (Default: *None*)
- `aip.disseminate.rightsMD` - Lists the DSpace Crosswalks (by name) which should be called to populate the `<rightsMD>` section of the METS file within the AIP (Default: `DSpaceDepositLicense:DSPACE_DEPLICENSE, CreativeCommonsRDF:DSPACE_CCRDF, CreativeCommonsText:DSPACE_CCTEXT, METSRights`)
  - The `DSPACE_DEPLICENSE` crosswalk ensures the DSpace Deposit License is referenced/stored in AIP
  - The `DSPACE_CCRDF` crosswalk ensures any Creative Commons RDF Licenses are reference/stored in AIP
  - The `DSPACE_CCTEXT` crosswalk ensures any Creative Commons Textual Licenses are referenced/stored in AIP
  - The `METSRights` crosswalk ensures that Permissions/Rights on DSpace Objects (Communities, Collections, Items or Bitstreams) are referenced/stored in AIP. Using this crosswalk means that AIPs can be used to restore permissions that a particular Group or Person had on a DSpace Object. (NOTE: The `METSRights` crosswalk should always be used in conjunction with the `DSPACE-ROLES` crosswalk (see above) or a similar crosswalk. The `METSRights` crosswalk can only restore permissions, and cannot re-create Groups or EPeople in the system. The `DSPACE-ROLES` can actually re-create the Groups or EPeople as needed.)
- `aip.disseminate.dmd` - Lists the DSpace Crosswalks (by name) which should be called to populate the `<dmdSec>`section of the METS file within the AIP (Default: MODS, DIM)
  - The MODS crosswalk translates the DSpace descriptive metadata (for this object) into MODS. As MODS is a relatively "standard" metadata schema, it may be useful to include a copy of MODS metadata in your AIPs if you should ever want to import them into another (non-DSpace) system.
  - The DIM crosswalk just translates the DSpace internal descriptive metadata into an XML format. This XML format is proprietary to DSpace, but stores the metadata in a format similar to Qualified Dublin Core.

AIP Ingestion Metadata Crosswalk Configurations

The following configurations allow you to specify what DSpace Crosswalks are used during the ingestion/restoration of AIPs. These configurations also allow you to ignore areas of the METS file (in the AIP) if you do not want that area to be restored.

In `dspace.cfg`, the general format for each of these settings is:

- `mets.dspaceAIP.ingest.crosswalk.<mdType> = <DSpace-crosswalk-name>`
  - <mdType> is the type of metadata as specified in the METS file. This corresponds to the value of the @MDTYPE attribute (of that metadata section in the METS). When the @MDTYPE attribute is "OTHER", then the <mdType> corresponds to the @OTHERMDTYPE attribute value.
  - <DSpace-crosswalk-name> specifies the name of the DSpace Crosswalk which should be used to ingest this metadata into DSpace. You can specify the "NULLSTREAM" crosswalk if you specifically want this metadata to be ignored (and skipped over during ingestion).

By default, the settings in `dspace.cfg` are:

```
mets.dspaceAIP.ingest.crosswalk.DSpaceDepositLicense = NULLSTREAM
mets.dspaceAIP.ingest.crosswalk.CreativeCommonsRDF = NULLSTREAM
mets.dspaceAIP.ingest.crosswalk.CreativeCommonsText = NULLSTREAM
```

The above settings tell the ingester to **ignore** any metadata sections which reference DSpace Deposit Licenses or Creative Commons Licenses. These metadata sections can be safely ignored as long as the "LICENSE" and "CC_LICENSE" bundles are included in AIPs (which is the default setting). As the Licenses are included in those Bundles, they will already be restored when restoring the bundle contents.

More Info on Default Crosswalks used

If unspecified in the above settings, the AIP ingester will automatically use the Crosswalk which is named the same as the @MDTYPE or @OTHERMDTYPE attribute for the metadata section. For example, a metadata section with an @MDTYPE="PREMIS" will be processed by the DSpace Crosswalk named "PREMIS".

AIP Ingestion EPerson Configurations

The following setting determines whether the AIP Ingester should create an EPerson (if necessary) when attempting to restore or ingest an Item whose Submitter cannot be located in the system. By default it is set to "false", as for AIPs the creation of EPeople (and Groups) is generally handled by the `DSPACE-ROLES` crosswalk (see #AIP Metadata Dissemination Configurations for more info on `DSPACE-ROLES` crosswalk.)

- `mets.dspaceAIP.ingest.createSubmitter = false`

AIP Configurations To Improve Ingestion Speed while Validating

It is recommended to validate all AIPs on ingestion (when possible). But validation can be extremely slow, as each validation request first must download all referenced Schema documents from various locations on the web (sometimes as many as 10 schemas may be necessary to download in order to validate a single METS file). To make matters worse, the same schema will be re-downloaded each time it is used (i.e. it is not cached locally). So, if you are validating just 20 METS files which each reference 10 schemas, that results in 200 download requests.

In order to perform validations in a speedy fashion, you can pull down a local copy of **all** schemas. Validation will then use this local cache, which can sometimes increase the speed up to 10 x.

To use a local cache of XML schemas when validating, use the following settings in 'dspace.cfg'. The general format is:

- `mets.xsd.<abbreviation> = <namespace> <local-file-name>`
  - `<abbreviation>` is a unique abbreviation (of your choice) for this schema
  - `<namespace>` is the Schema namespace
  - `<local-file-name>` the full name of the cached schema file (which should reside in your `[dspace]/config/schemas/` directory, by default this directory does not exist – you will need to create it)

The default settings are all commented out. But, they provide a full listing of all schemas currently used during validation of AIPs. In order to utilize them, uncomment the settings, download the appropriate schema file, and save it to your `[dspace]/config/schemas/` directory (by default this directory does not exist – you will need to create it) using the specified file name:

```
#mets.xsd.mets = http://www.loc.gov/METS/ mets.xsd
#mets.xsd.xlink = http://www.w3.org/1999/xlink xlink.xsd
#mets.xsd.mods = http://www.loc.gov/mods/v3 mods.xsd
#mets.xsd.xml = http://www.w3.org/XML/1998/namespace xml.xsd
#mets.xsd.dc = http://purl.org/dc/elements/1.1/ dc.xsd
#mets.xsd.dcterms = http://purl.org/dc/terms/ dcterms.xsd
#mets.xsd.premis = http://www.loc.gov/standards/premis PREMIS.xsd
#mets.xsd.premisObject = http://www.loc.gov/standards/premis PREMIS-Object.xsd
#mets.xsd.premisEvent = http://www.loc.gov/standards/premis PREMIS-Event.xsd
#mets.xsd.premisAgent = http://www.loc.gov/standards/premis PREMIS-Agent.xsd
#mets.xsd.premisRights = http://www.loc.gov/standards/premis PREMIS-Rights.xsd
```

Common Issues or Error Messages

The below table lists common fixes to issues you may encounter when backing up or restoring objects using AIP Backup and Restore.

Issue / Error Message	How to Fix this Problem

Ingest /Restore Error: "Group Administrator already exists"	If you receive this problem, you are likely attempting to Restore an Entire Site, but are not running the command in Force Replace Mode (`-r -f`). Please see the section on Restoring an Entire Site for more details on the flags you should be using.
Ingest /Restore Error: "Unknown Metadata Schema encountered (mycustomschema)"	If you receive this problem, one or more of your Items is using a custom metadata schema which DSpace is currently not aware of (in the example, the schema is named "mycustomschema"). Because DSpace AIPs do not contain enough details to recreate the missing Metadata Schema, you must create it manually via the DSpace Admin UI. **Please note that you only need to create the Schema. You do not need to manually create all the fields belonging to that schema, as DSpace will do that for you as it restores each AIP.** Once the schema is created in DSpace, re-run your restore command. DSpace will automatically re-create all fields belonging to that custom metadata schema as it restores each Item that uses that schema.
Ingest Error: "Could not find a parent DSpaceObject referenced as 'xxx/xxx'"	When you encounter this error message it means that an object could not be ingested/restored as it belongs to a **parent** object which doesn't currently exist in your DSpace instance. During a full restore process, this error can be skipped over and treated as a warning by specifying the '`-o skipIfParentMissing=true`' option (see Additional Packager Options). If you have a larger number of Items which are mapped to multiple Collections, the AIP Ingester will sometimes attempt to restore an item mapping before the **Collection itself** has been restored (thus throwing this error). Luckily, this is not anything to be concerned about. As soon as the Collection is restored, the Item Mapping which caused the error will also be automatically restored. So, if you encounter this error during a full restore, it is safe to bypass this error message using the '`-o skipIfParentMissing=true`' option. All your Item Mappings should still be restored correctly.
Submit Error: PSQLException: ERROR: duplicate key value violates unique constraint "handle_handle_key"	This error means that while submitting one or more AIPs, DSpace encountered a Handle conflict. This is a general error the may occur in DSpace if your Handle sequence has somehow become out-of-date. However, it's easy to fix. Just run the `[dspace]/bin/dspace database update-sequences`

DSpace AIP Format

1 Makeup and Definition of AIPs
- 1.1 AIPs are Archival Information Packages.
- 1.2 General AIP Structure / Examples
  - 1.2.1 Customizing What Is Stored in Your AIPs

2 AIP Details: METS Structure

3 Metadata in METS
- 3.1 DIM (DSpace Intermediate Metadata) Schema
  - 3.1.1 DIM Descriptive Elements for Item objects
  - 3.1.2 DIM Descriptive Elements for Collection objects
  - 3.1.3 DIM Descriptive Elements for Community objects
  - 3.1.4 DIM Descriptive Elements for Site objects
- 3.2 MODS Schema
- 3.3 AIP Technical Metadata Schema (AIP-TECHMD)
  - 3.3.1 AIP Technical Metadata for Item
  - 3.3.2 AIP Technical Metadata for Bitstream
  - 3.3.3 AIP Technical Metadata for Collection
  - 3.3.4 AIP Technical Metadata for Community
  - 3.3.5 AIP Technical Metadata for Site
- 3.4 PREMIS Schema
  - 3.4.1 PREMIS Metadata for Bitstream
- 3.5 DSPACE-ROLES Schema
  - 3.5.1 Example of DSPACE-ROLES Schema for a SITE AIP
  - 3.5.2 Example of DSPACE-ROLES Schema for a Community or Collection
- 3.6 METSRights Schema
  - 3.6.1 Example of METSRights Schema for an Item
  - 3.6.2 Example of METSRights Schema for a Collection
  - 3.6.3 Example of METSRights Schema for a Community

Makeup and Definition of AIPs

AIPs only store the Latest Version of Items

If you are using the Item Level Versioning functionality (disabled by default), you must be aware that this "Item Level Versioning" feature is **not yet compatible** with AIP Backup & Restore. **Using them together may result in accidental data loss.** Currently the AIPs that DSpace generates only store the *latest version* of an Item. Therefore, past versions of Items will always be lost when you perform a restore / replace using AIP tools.

AIPs are Archival Information Packages.

- AIP is a package describing **one archival object** in DSpace.
  - The **archival object** may be a single **Item**, **Collection**, **Community**, or **Site** (Site AIPs contain site-wide information). Bitstreams are included in an Item's AIP.
  - Each AIP is logically self-contained, can be restored without rest of the archive. (So you could restore a single Item, Collection or Community)
  - Collection or Community AIPs do **not** include all child objects (e.g. Items in those Collections or Communities), as each AIP only describes **one** object. However, these container AIPs do contain references (links) to all child objects. These references can be used by DSpace to automatically restore all referenced AIPs when restoring a Collection or Community.
  - AIPs are only generated for objects which are currently in the "in archive" state in DSpace. This means that in-progress, uncompleted submissions are not described in AIPs and cannot be restored after a disaster. Permanently removed objects will also no longer be exported as AIPs after their removal. However, withdrawn objects will continue to be exported as AIPs, since they are still considered under the "in archive" status.
  - AIPs with identical contents will always have identical checksums. This provides a basic means of validating whether the contents within an AIP have changed. For example, if a Collection's AIP has the same checksum at two different points in time, it means that Collection has not changed during that time period.
  - AIP profile favors completeness and accuracy rather than presenting the semantics of an object in a standard format. It conforms to the quirks of DSpace's internal object model rather than attempting to produce a universally understandable representation of the object. When possible, an AIP tries to use common standards to express objects.
  - An AIP *can* serve as a DIP (Dissemination Information Package) or SIP (Submission Information Package), especially when transferring custody of objects to another DSpace implementation.
  - In contrast to SIP or DIP, the AIP should include all available DSpace structural and administrative metadata, and basic provenance information. AIPs also describe some basic system level information (e.g. Groups and People).

General AIP Structure / Examples

Generally speaking, an AIP is an Zip file containing a METS manifest and all related content bitstreams, license files and any other associated files.

Some examples include:

- Site AIP (Sample: SITE-example.zip)
  - METS contains basic metadata about DSpace Site and persistent IDs referencing all Top Level Communities
  - METS also contains a list of all Groups and EPeople information defined in the DSpace system. (NOTE: By default, user passwords are not stored in AIPs, unless you specify the 'passwords' flag. See Additional Packager Options.)
- Community AIP (Sample: COMMUNITY@123456789-1.zip)

    - METS contains all metadata for Community and persistent IDs referencing all members (SubCommunities or Collections). Package may also include a Logo file, if one exists.
    - METS contains any Group information for Community-specific groups (e.g. `COMMUNITY_<ID>_ADMIN` group).
    - METS contains all Community permissions/policies (translated into METSRights schema)
- Collection AIP (Sample: COLLECTION@123456789-2.zip)
    - METS contains all metadata for Collection and persistent IDs referencing all members (Items). Package may also include a Logo file, if one exists.
    - METS contains any Group information for Collection-specific groups (e.g. `COLLECTION_<ID>_ADMIN`, `COLLECTION_<ID>_SUBMIT`, etc.).
    - METS contains all Collection permissions/policies (translated into METSRights schema)
    - If the Collection has an Item Template, the METS will also contain all the metadata for that Item Template.
- Item AIP (Sample: ITEM@123456789-8.zip)
    - METS contains all metadata for Item and references to all Bundles and Bitstreams. Package also includes all Bitstream files.
    - METS contains all Item/Bundle/Bitstream permissions/policies (translated into METSRights schema)

Notes:

- Bitstreams and Bundles are second-class archival objects; they are recorded in the context of an Item.
- BitstreamFormats are not even second-class; they are described implicitly within Item technical metadata, and reconstructed from that during restoration
- EPeople are only defined in Site AIP, but may be referenced from Community or Collection AIPs
- Groups may be defined in Site AIP, Community AIP or Collection AIP. Where they are defined depends on whether the Group relates specifically to a single Community or Collection, or is just a general site-wide group.

What is NOT in AIPs

- DSpace Site configurations ([dspace]/config/ directory) or customizations (themes, stylesheets, etc) are not described in AIPs
- DSpace Database model (or customizations therein) is not described in AIPs
- Any objects which are not currently in the "In Archive" state are not described in AIPs. This means that in-progress, unfinished submissions are never included in AIPs.

Customizing What Is Stored in Your AIPs

If you choose, you can customize exactly what information is stored in your AIPs. However, you should be aware that you can only restore information which is stored within your AIPs. If you choose to remove information from your AIPs, you will be unable to restore it later on (unless you are also backing up your entire DSpace database and assetstore folder).

AIP Recommendations

It is recommended to minimally use the default settings when generating AIPs. DSpace can only restore information that is included within an AIP. Therefore, if you choose to no longer include some information in an AIP, DSpace will no longer be able to restore that information from an AIP backup

There are two ways to go about customizing your AIP format:

1. You can customize your `dspace.cfg` settings pertaining to AIP generation. These configurations will allow you to specify exactly which DSpace Crosswalks will be called when generating the AIP METS manifest.
2. You can export your AIPs using one of the special options/flags.

AIP Details: METS Structure

This METS Structure is based on the structure decided for the original AipPrototype, developed as part of the MIT & UCSD PLEDGE project.

- `mets` element
    - `@PROFILE` fixed value="http://www.dspace.org/schema/aip/1.0/mets.xsd" (this is how we identify an AIP manifest)
    - `@OBJID` URN-format persistent identifier (i.e. Handle) if available, or else a unique identifier. (e.g. "hdl:123456789/1")
    - `@LABEL` title if available
    - `@TYPE` DSpace object type, one of "DSpace ITEM", "DSpace COLLECTION", "DSpace COMMUNITY" or "DSpace SITE".
    - `@ID` is a globally unique identifier, built using the Handle and the Object type (e.g. `dspace-COLLECTION-hdl:123456789/3`).
- `mets/metsHdr` element
    - `@LASTMODDATE` last-modified date for a DSpace Item, or nothing for other objects.
    - `agent` element:
        - `@ROLE` = "CUSTODIAN",
        - `@TYPE` = "OTHER",
        - `@OTHERTYPE` = "DSpace Archive",
        - `name` = *Site handle*. (Note: The Site Handle is of the format `[handle_prefix]/0`, e.g. "123456789/0")
    - `agent` element:
        - `@ROLE` = "CREATOR",
        - `@TYPE` = "OTHER",
        - `@OTHERTYPE` = "DSpace Software",
        - `name` = "DSpace [version]" (Where "[version]" is the specific version of DSpace software which created this AIP, e.g. "1.7.0")
- `mets/dmdSec` element(s)
    - By default, two `dmdSec` elements are included for all AIPs:
        1. object's descriptive metadata crosswalked to MODS (specified by `mets/dmdSec/mdWrap@MDTYPE="MODS"`). See #MODS Schema section below for more information.
        2. object's descriptive metadata in DSpace native DIM intermediate format, to serve as a complete and precise record for restoration or ingestion into another DSpace. Specified by `mets/dmdSec/mdWrap@MDTYPE="OTHER",@OTHERMDTYPE="DIM"`. See #DIM (DSpace Intermediate Metadata) Schema section below for more information.

    - For Collection AIPs, additional dmdSec elements may exist which describe the Item Template for that Collection. Since an Item template is not an actual Item (i.e. it only includes metadata), it is stored within the Collection AIP. The Item Template's dmdSec elements will be referenced by a `div @TYPE="DSpace ITEM Template"` in the METS `structMap`.
    - When the `mdWrap` `@TYPE` value is `OTHER`, the element *MUST* include a value for the `@OTHERTYPE` attribute which names the crosswalk that produced (or interprets) that metadata, e.g. `DIM`.
- `mets/amdSec` element(s)
    - One or more amdSec elements are include for all AIPs. The first amdSec element contains administrative metadata (technical, source, rights, and provenance) for the entire archival object. Additional amdSec elements may exist to describe parts of the archival object (e.g. Bitstreams or Bundles in an Item).
        - `techMD` elements. By default, two types of `techMD` elements may be included:
            - `PREMIS` metadata about an object may be included here (*currently only specified for Bitstreams (files)*). Specified by `mdWrap@MDTYPE="PREMIS"`. See #PREMIS Schema section below for more information.
            - `DSPACE-ROLES` metadata may appear here to describe the Groups or EPeople related to this object (_currently only specified for Site, Community and Collection). Specified by `mdWrap@MDTYPE="OTHER",@OTHERMDTYPE="DSPACE-ROLES"`. See #DSPACE-ROLES Schema section below for more information.
        - `rightsMD` elements. By default, there are four possible types of `rightsMD` elements which may be included:
            - `METSRights` metadata may appear here to describe the permissions on this object. Specified by `mdWrap@MDTYPE="OTHER",@OTHERMDTYPE="METSRIGHTS"`. See #METSRights Schema section below for more information.
            - `DSpaceDepositLicense` if the object is an Item and it has a deposit license, it is contained here. Specified by `mdWrap@MDTYPE="OTHER",@OTHERMDTYPE="DSpaceDepositLicense"`.
            - `CreativeCommonsRDF` If the object is an Item with a Creative Commons license expressed in RDF, it is included here. Specified by `mdWrap@MDTYPE="OTHER",@OTHERMDTYPE="CreativeCommonsRDF"`.
            - `CreativeCommonsText` If the object is an Item with a Creative Commons license in plain text, it is included here. Specified by `mdWrap@MDTYPE="OTHER",@OTHERMDTYPE="CreativeCommonsText"`.
        - `sourceMD` element. By default, there is only one type of `sourceMD` element which may appear:
            - `AIP-TECHMD` metadata may appear here. This stores basic technical/source metadata about in object in a DSpace native format. Specified by `mdWrap@MDTYPE="OTHER",@OTHERMDTYPE="AIP-TECHMD"`. See #AIP Technical Metadata Schema (AIP-TECHMD) section below for more information.
        - `digiprovMD` element.
            - *Not used at this time.*
- `mets/fileSec` element
    - For ITEM objects:
        - Each distinct Bundle in an Item goes into a `fileGrp`. The `fileGrp` has a `@USE` attribute which corresponds to the Bundle name.
        - Bitstreams in bundles become `file` elements under `fileGrp`.
        - `mets/fileSec/fileGrp/file`elements
            - Set `@SIZE` to length of the bitstream. There is a redundant value in the <techMD> but it is more accessible here.
            - Set `@MIMETYPE`, `@CHECKSUM`, `@CHECKSUMTYPE` to corresponding bitstream values. There is redundant info in the <techMD>. (For DSpace, the `@CHECKSUMTYPE="MD5"` at all times)
            - SET `@SEQ` to bitstream's SequenceID if it has one.
            - SET `@ADMID` to the list of <amdSec> element(s) which describe this bitstream.
    - For COLLECTION and COMMUNITY objects:
        - *Only* if the object has a *logo bitstream*, there is a `fileSec` with one `fileGrp` child of `@USE="LOGO"`.
        - The `fileGrp` contains one `file` element, representing the logo Bitstream. It has the same `@MIMETYPE`, `@CHECKSUM`, `@CHECKSUMTYPE` attributes as the Item content bitstreams, but does NOT include metadata section references (e.g. `@ADMID`) or a `@SEQ` attribute.
        - See the main `structMap` for the `fptr` reference to this logo file.
- `mets/structMap` - Primary structure map, `@LABEL="DSpace Object", @TYPE="LOGICAL"`
    - For ITEM objects:
        1. Top-Level `div` with `@TYPE="DSpace Object Contents"`.
            - For every Bitstream in Item it contains a `div` with `@TYPE="DSpace BITSTREAM"`. Each Bitstream `div` has a single `fptr` element which references the bitstream location.
        - If Item has primary bitstream, put it in `structMap/div/fptr` (i.e. directly under the `div` with `@TYPE="DSpace Object Contents"`)
    - For COLLECTION objects:
        1. Top-Level `div` with `@TYPE="DSpace Object Contents"`.
            - For every Item in the Collection, it contains a `div` with `@TYPE="DSpace ITEM"`. Each Item `div` has up to two child `mptr`elements:
                a. One linking to the Handle of that Item. Its `@LOCTYPE="HANDLE"`, and `@xlink:href` value is the raw Handle.
                b. (Optional) one linking to the location of the local AIP for that Item (if known). Its `@LOCTYPE="URL"`, and `@xlink:href` value is a relative link to the AIP file on the local filesystem.
            - If Collection has a Logo bitstream, there is an `fptr` reference to it in the very first `div`.
            - If the Collection includes an Item Template, there will be a `div` with `@TYPE="DSpace ITEM Template"` within the very first `div`. This `div @TYPE="DSpace ITEM Template"` must have a `@DMDID` specified, which links to the dmdSec element(s) that contain the metadata for the Item Template.
    - For COMMUNITY objects:
        1. Top-Level `div` with `@TYPE="DSpace Object Contents"`.
            - For every Sub-Community in the Community it contains a `div` with `@TYPE="DSpace COMMUNITY"`. Each Community `div` has up to two `mptr`elements:
                a. One linking to the Handle of that Community. Its `@LOCTYPE="HANDLE"`, and `@xlink:href` value is the raw Handle.
                b. (Optional) one linking to the location of the local AIP file for that Community (if known). Its `@LOCTYPE="URL"`, and `@xlink:href` value is a relative link to the AIP file on the local filesystem.
            - For every Collection in the Community there is a `div` with `@TYPE="DSpace COLLECTION"`. Each Collection `div` has up to two `mptr`elements:
                a. One linking to the Handle of that Collection. Its `@LOCTYPE="HANDLE"`, and `@xlink:href` value is the raw Handle.

            b. (Optional) one linking to the location of the local AIP file for that Collection (if known). Its `@LOCTYPE="URL"`, and `@xlink:href` value is a relative link to the AIP file on the local filesystem.
        - If Community has a Logo bitstream, there is an `fptr` reference to it in the very first `div`.
    - For SITE objects:
        1. Top-Level `div` with `@TYPE="DSpace Object Contents"`.
            - For every Top-level Community in Site, it contains a `div` with `@TYPE="DSpace COMMUNITY"`. Each Item `div` has up to two child `mptr`elements:
                a. One linking to the Handle of that Community. Its `@LOCTYPE="HANDLE"`, and `@xlink:href` value is the raw Handle.
                b. (Optional) one linking to the location of the local AIP for that Community (if known). Its `@LOCTYPE="URL"`, and `@xlink:href` value is a relative link to the AIP file on the local filesystem.
- `mets/structMap` - Structure Map to indicate object's Parent, `@LABEL="Parent", @TYPE="LOGICAL"`
    - Contains one `div` element which has the unique attribute value `TYPE="AIP Parent Link"` to identify it as the older of the *parent pointer*.
        - It contains a `mptr` element whose `xlink:href` attribute value is the raw Handle of the parent object, e.g. `1721.1/4321`.

Metadata in METS

The following tables describe how various metadata schemas are populated (via DSpace Crosswalks) in the METS file for an AIP.

DIM (DSpace Intermediate Metadata) Schema

DIM Schema is essentially a way of representing DSpace internal metadata structure in XML. DSpace's internal metadata is very similar to a Qualified Dublin Core in its structure, and is primarily meant for descriptive metadata. However, DSpace's metadata allows for custom elements, qualifiers or schemas to be created (so it is extendable to any number of schemas, elements, qualifiers). These custom fields/schemas may or may not be able to be translated into normal Qualified Dublin Core. So, the DIM Schema must be able to express metadata schemas, elements or qualifiers which may or may not exist within Qualified Dublin Core.

In the METS structure, DIM metadata always appears within a `dmdSec` inside an `<mdWrap MDTYPE="OTHER" OTHERMDTYPE="DIM">` element. For example:

```
<dmdSec ID="dmdSec_2190">
   <mdWrap MDTYPE="OTHER" OTHERMDTYPE="DIM">
    ...
   </mdWrap>
</dmdSec>
```

By default, DIM metadata is always included in AIPs. It is controlled by the following configuration in your `dspace.cfg`:

```
aip.disseminate.dmd = MODS, DIM
```

DIM Descriptive Elements for Item objects

As all DSpace Items already have user-assigned DIM (essentially Qualified Dublin Core) metadata fields, those fields are just exported into the DIM Schema within the METS file.

DIM Descriptive Elements for Collection objects

For Collections, the following fields are translated to the DIM schema:

DIM Metadata Field	Database field or value
dc.description	'introductory_text' field
dc.description.abstract	'short_description' field
dc.description.tableofcontents	'side_bar_text' field
dc.identifier.uri	Collection's handle
dc.provenance	'provenance_description' field
dc.rights	'copyright_text' field
dc.rights.license	'license' field
dc.title	'name' field

DIM Descriptive Elements for Community objects

For Communities, the following fields are translated to the DIM schema:

DIM Metadata Field	Database field or value
dc.description	'introductory_text' field
dc.description.abstract	'short_description' field
dc.description.tableofcontents	'side_bar_text' field
dc.identifier.uri	Handle of Community
dc.rights	'copyright_text' field
dc.title	'name' field

DIM Descriptive Elements for Site objects

For the Site Object, the following fields are translated to the DIM schema:

Metadata Field	Value
dc.identifier.uri	Handle of Site (format: `[handle_prefix]/0`)
dc.title	Name of Site (from dspace.cfg 'dspace.name' config)

MODS Schema

By default, all DSpace descriptive metadata (DIM) is also translated into the MODS Schema by utilizing DSpace's `MODSDisseminationCrosswalk`. DSpace's DIM to MODS crosswalk is defined within your `[dspace]/config/crosswalks/mods.properties` configuration file. This file allows you to customize the MODS that is included within your AIPs.

For more information on the MODS Schema, see http://www.loc.gov/standards/mods/mods-schemas.html

In the METS structure, MODS metadata always appears within a `dmdSec` inside an `<mdWrap MDTYPE="MODS">` element. For example:

```
<dmdSec ID="dmdSec_2189">
   <mdWrap MDTYPE="MODS">
    ...
   </mdWrap>
</dmdSec>
```

By default, MODS metadata is always included in AIPs. It is controlled by the following configuration in your `dspace.cfg`:

```
aip.disseminate.dmd = MODS, DIM
```

The MODS metadata is included within your AIP to support interoperability. It provides a way for other systems to interact with or ingest the AIP without needing to understand the DIM Schema. You may choose to disable MODS if you wish, however this may decrease the likelihood that you'd be able to easily ingest your AIPs into a non-DSpace system (unless that non-DSpace system is able to understand the DIM schema). When restoring/ingesting AIPs, DSpace will always first attempt to restore DIM descriptive metadata. Only if no DIM metadata is found, will the MODS metadata be used during a restore.

AIP Technical Metadata Schema (AIP-TECHMD)

The AIP Technical Metadata Schema is a way to translate technical metadata about a DSpace object into the DIM Schema. It is kept separate from DIM as it is considered technical metadata rather than descriptive metadata.

In the METS structure, AIP-TECHMD metadata always appears within a `sourceMD` inside an `<mdWrap MDTYPE="OTHER" OTHERMDTYPE="AIP-TECHMD">` element. For example:

```
<amdSec ID="amd_2191">
    ...
    <sourceMD ID="sourceMD_2198">
       <mdWrap MDTYPE="OTHER" OTHERMDTYPE="AIP-TECHMD">
       ...
       </mdWrap>
    </sourceMD>
    ...
</amdSec>
```

By default, AIP-TECHMD metadata is always included in AIPs. It is controlled by the following configuration in your `dspace.cfg`:

```
aip.disseminate.sourceMD = AIP-TECHMD
```

AIP Technical Metadata for Item

Metadata Field	Value
dc.contributor	Submitter's email address
dc.identifier.uri	Handle of Item
dc.relation.isPartOf	Owning Collection's Handle (*as a URN*)
dc.relation.isReferencedBy	All other Collection's this item is linked to (*Handle URN of each non-owner*)
dc.rights.accessRights	*"WITHDRAWN"* if item is withdrawn

AIP Technical Metadata for Bitstream

Metadata Field	Value
dc.title	Bitstream's name/title
dc.title.alternative	Bitstream's source
dc.description	Bitstream's description
dc.format	Bitstream Format Description
dc.format.medium	Short Name of Format
dc.format.mimetype	MIMEType of Format
dc.format.supportlevel	System Support Level for Format (necessary to recreate Format during restore, if the format isn't know to DSpace by default)
dc.format.internal	Whether Format is internal (necessary to recreate Format during restore, if the format isn't know to DSpace by default)

- Outstanding Question: Why are we recording the file format support status? That's a DSpace property, rather than an Item property. Do DSpace instances rely on objects to tell them their support status?
    - Possible answer (from Larry Stone): Format support and other properties of the BitstreamFormat are recorded here in case the Item is restored in an empty DSpace that doesn't have that format yet, and the relevant bits of the format entry have to be reconstructed from the AIP. --lcs

AIP Technical Metadata for Collection

Metadata Field	Value
dc.identifier.uri	Handle of Collection
dc.relation.isPartOf	Owning Community's Handle (*as a URN*)
dc.relation.isReferencedBy	All other Communities this Collection is linked to (*Handle URN of each non-owner*)

AIP Technical Metadata for Community

Metadata Field	Value
dc.identifier.uri	Handle of Community
dc.relation.isPartOf	Handle of Parent Community (*as a URN*)

AIP Technical Metadata for Site

Metadata Field	Value
dc.identifier.uri	Site Handle (format: `[handle_prefix]/0`)

PREMIS Schema

At this point in time, the PREMIS Schema is only used to represent technical metadata about DSpace Bitstreams (i.e. Files). The PREMIS metadata is generated by DSpace's `PREMISCrosswalk`. Only the PREMIS Object Entity Schema is used.

In the METS structure, PREMIS metadata always appears within a `techMD` inside an `<mdWrap MDTYPE="PREMIS">` element. PREMIS metadata is **always** wrapped within a `<premis:premis>` element. For example:

```
<amdSec ID="amd_2209">
    ...
    <techMD ID="techMD_2210">
       <mdWrap MDTYPE="PREMIS">
          <premis:premis>
            ...
          </premis:premis>
       </mdWrap>
    </techMD>
    ...
</amdSec>
```

Each Bitstream (file) has its own `amdSec` within a METS manifest. So, there will be a separate PREMIS `techMD` for each Bitstream within a single Item.

By default, PREMIS metadata is always included in AIPs. It is controlled by the following configuration in your `dspace.cfg`:

```
aip.disseminate.techMD = PREMIS, DSPACE-ROLES
```

PREMIS Metadata for Bitstream

The following Bitstream information is translated into PREMIS for each DSpace Bitstream (file):

Metadata Field	Value
<premis:objectIdentifier>	Contains Bitstream direct URL
<premis:objectCategory>	Always set to "File"
<premis:fixity>	Contains MD5 Checksum of Bitstream
<premis:format>	Contains File Format information of Bistream
<premis:originalName>	Contains original name of file

DSPACE-ROLES Schema

All DSpace Groups and EPeople objects are translated into a custom `DSPACE-ROLES` XML Schema. This XML Schema is a very simple representation of the underlying DSpace database model for Groups and EPeople. The `DSPACE-ROLES` Schemas is generated by DSpace's `RoleCrosswalk`.

Only the following DSpace Objects utilize the DSPACE-ROLES Schema in their AIPs:

- Site AIP – all Groups and EPeople are represented in DSPACE-ROLES Schema
- Community AIP – only Community-based groups (e.g. `COMMUNITY_1_ADMIN`) are represented in DSPACE-ROLES Schema
- Collection AIP – only Collection-based groups (e.g. `COLLECTION_2_ADMIN`, `COLLECTION_2_SUBMIT`, etc.) are represented in DSPACE-ROLES Schema

In the METS structure, DSPACE-ROLES metadata always appears within a `techMD` inside an `<mdWrap MDTYPE="OTHER" OTHERMDTYPE="DSPACE-ROLES">` element. For example:

```
<amdSec ID="amd_2068">
    ...
    <techMD ID="techMD_2070">
       <mdWrap MDTYPE="OTHER" OTHERMDTYPE="DSPACE-ROLES">
            ...
       </mdWrap>
    </techMD>
    ...
</amdSec>
```

By default, DSPACE-ROLES metadata is always included in AIPs. It is controlled by the following configuration in your `dspace.cfg`:

```
aip.disseminate.techMD = PREMIS, DSPACE-ROLES
```

Example of DSPACE-ROLES Schema for a SITE AIP

Below is a general example of the structure of a DSPACE-ROLES XML file, as it would appear in a SITE AIP.

```
<DSpaceRoles>
  <Groups>
    <Group ID="1" Name="Administrator">
      <Members>
        <Member ID="1" Name="bsmith@myu.edu" />
      </Members>
    </Group>
    <Group ID="0" Name="Anonymous" />
    <Group ID="70" Name="COLLECTION_hdl:123456789/57_ADMIN">
      <Members>
        <Member ID="1" Name="bsmith@myu.edu" />
      </Members>
    </Group>
    <Group ID="75" Name="COLLECTION_hdl:123456789/57_DEFAULT_READ">
      <MemberGroups>
        <MemberGroup ID="0" Name="Anonymous" />
      </MemberGroups>
    </Group>
    <Group ID="71" Name="COLLECTION_hdl:123456789/57_SUBMIT">
      <Members>
        <Member ID="1" Name="bsmith@myu.edu" />
      </Members>
    </Group>
    <Group ID="72" Name="COLLECTION_hdl:123456789/57_WORKFLOW_STEP_1">
      <MemberGroups>
        <MemberGroup ID="1" Name="Administrator" />
      </MemberGroups>
    </Group>
    <Group ID="73" Name="COLLECTION_hdl:123456789/57_WORKFLOW_STEP_2">
      <MemberGroups>
        <MemberGroup ID="1" Name="Administrator" />
      </MemberGroups>
    </Group>
    <Group ID="8" Name="COLLECTION_hdl:123456789/6703_DEFAULT_READ" />
    <Group ID="9" Name="COLLECTION_hdl:123456789/2_ADMIN">
      <Members>
        <Member ID="1" Name="bsmith@myu.edu" />
      </Members>
    </Group>
  </Groups>
  <People>
    <Person ID="1">
      <Email>bsmith@myu.edu</Email>
      <Netid>bsmith</Netid>
      <FirstName>Bob</FirstName>
      <LastName>Smith</LastName>
      <Language>en</Language>
      <CanLogin />
    </Person>
    <Person ID="2">
      <Email>jjones@myu.edu</Email>
      <FirstName>Jane</FirstName>
      <LastName>Jones</LastName>
      <Language>en</Language>
      <CanLogin />
      <SelfRegistered />
    </Person>
  </People>
</DSpaceRoles>
```

Why are there Group Names with Handles?

You may have noticed several odd looking group names in the above example, where a Handle is embedded in the name (e.g. "COLLECTION_hdl:123456789/57_SUBMIT"). This is a translation of a Group name which included a Community or Collection *Internal ID* (e.g. "COLLECTION_45_SUBMIT"). Since you are exporting these Groups outside of DSpace, the *Internal ID* may no longer be valid or be understandable. Therefore, before export, these Group names are all translated to include an externally understandable identifier, in the form of a Handle. If you use this AIP to restore your groups later, they will be translated back to the normal DSpace format (i.e. the handle will be translated back to the new *Internal ID*).

Orphaned Groups are Renamed on Export

If a Group name includes a Community or Collection *Internal ID* (e.g. "COLLECTION_45_SUBMIT"), and that Community or Collection no longer exists, then the Group is considered "Orphaned".

- In 1.8.2 and above, the Group is renamed using the following format: "ORPHANED_[object-type]_GROUP_[obj-id]_[group-type]" (e.g. "ORPHANED_COLLECTION_GROUP_10_ADMIN").
- Prior to 1.8.2, the Group was renamed with a random key: "GROUP_[random-hex-key]_[object-type]_[group-type]" (e.g. "GROUP_123eb3a_COLLECTION_ADMIN"). *This old format was discontinued as giving the groups a randomly generated name caused the SITE AIP to have a different checksum every time it was regenerated (see https://github.com/DSpace/DSpace/issues/4492).*

The reasoning is that we were unable to translate an *Internal ID* into an *External ID* (i.e. Handle). If we are unable to do that translation, re-importing or restoring a group with an *old* internal ID could cause conflicts or instability in your DSpace system. In order to avoid such conflicts, these groups are renamed using a random, unique key.

Example of DSPACE-ROLES Schema for a Community or Collection

Below is a general example of the structure of a DSPACE-ROLES XML file, as it would appear in a Community or Collection AIP.

This specific example is for a Collection, which has associated Administrator, Submitter, and Workflow approver groups. In this very simple example, each group only has one Person as a member of it. Please notice that the Person's information (Name, NetID, etc) is NOT contained in this content (however they are available in the DSPACE-ROLES example for a SITE, as shown above)

```
<DSpaceRoles>
  <Groups>
    <Group ID="9" Name="COLLECTION_hdl:123456789/2_ADMIN" Type="ADMIN">
      <Members>
        <Member ID="1" Name="bsmith@myu.edu" />
      </Members>
    </Group>
    <Group ID="13" Name="COLLECTION_hdl:123456789/2_SUBMIT" Type="SUBMIT">
      <Members>
        <Member ID="2" Name="jjones@myu.edu" />
      </Members>
    </Group>
    <Group ID="10" Name="COLLECTION_hdl:123456789/2_WORKFLOW_STEP_1" Type="WORKFLOW_STEP_1">
      <Members>
        <Member ID="1" Name="bsmith@myu.edu" />
      </Members>
    </Group>
    <Group ID="11" Name="COLLECTION_hdl:123456789/2_WORKFLOW_STEP_2" Type="WORKFLOW_STEP_2">
      <Members>
        <Member ID="2" Name="jjones@myu.edu" />
      </Members>
    </Group>
    <Group ID="12" Name="COLLECTION_hdl:123456789/2_WORKFLOW_STEP_3" Type="WORKFLOW_STEP_3">
      <Members>
        <Member ID="1" Name="bsmith@myu.edu" />
      </Members>
    </Group>
  </Groups>
</DSpaceRoles>
```

METSRights Schema

All DSpace Policies (permissions on objects) are translated into the METSRights schema. This is different than the above DSPACE-ROLES schema, which only represents Groups and People objects. Instead, the METSRights schema is used to translate the permission statements (e.g. a group named "Library Admins" has Administrative permissions on a Community named "University Library"). But the METSRights schema doesn't represent who is a member of a particular group (that is defined in the DSPACE-ROLES schema, as described above).

METSRights should always be used with DSPACE-ROLES

The METSRights Schema must be used in conjunction with the DSPACE-ROLES Schema for Groups, People and Permissions to all be restored properly. As mentioned above, the METSRights metadata can only be used to restore permissions (i.e. DSpace policies). The DSPACE-ROLES metadata must also exist if you wish to restore the actual Group or EPeople objects to which those permissions apply.

All DSpace Object's AIPs (except for the SITE AIP) utilize the METSRights Schema in order to define what permissions people and groups have on that object. Although there are several sections to the METSRights Schema, DSpace AIPs *only use* the `<RightsDeclarationMD>` section, as this is what is used to describe rights on an object.

In the METS structure, METSRights metadata always appears within a `rightsMD` inside an `<mdWrap MDTYPE="OTHER" OTHERMDTYPE="METSRIGHTS">` element. For example:

```
  <amdSec ID="amd_2068">
      ...
      <rightsMD ID="rightsMD_2074">
         <mdWrap MDTYPE="OTHER" OTHERMDTYPE="METSRIGHTS">
              ...
         </mdWrap>
      </rightsMD>
      ...
  </amdSec>
```

By default, METSRights metadata is always included in AIPs. It is controlled by the following configuration in your `dspace.cfg`:

```
aip.disseminate.rightsMD = DSpaceDepositLicense:DSPACE_DEPLICENSE, \
    CreativeCommonsRDF:DSPACE_CCRDF, CreativeCommonsText:DSPACE_CCTEXT, METSRIGHTS
```

Example of METSRights Schema for an Item

An Item AIP will almost always contain several METSRights metadata sections within its METS Manifest. A separate METSRights metadata section is used to describe the permissions on:

- the Item itself
- each Bundle (group of files) in the Item
- each Bitstream (file) within an Item's bundle

Below is an example of a METSRights sections for a publicly visible Bitstream, Bundle or Item. Notice it specifies that the "GENERAL PUBLIC" has the permission to DISCOVER or DISPLAY this object.

```
<rights:RightsDeclarationMD xmlns:rights="http://cosimo.stanford.edu/sdr/metsrights/" RIGHTSCATEGORY="LICENSED">
  <rights:Context CONTEXTCLASS="GENERAL PUBLIC">
    <rights:Permissions DISCOVER="true" DISPLAY="true" MODIFY="false" DELETE="false" />
  </rights:Context>
</rights:RightsDeclarationMD>
```

As of DSpace 3, DSpace policies/permissions may also have a "start-date" or "end-date" (to support Embargo functionality). Such a policy on an Item may look like this. Notice it specifies that the "GENERAL PUBLIC" has the permission to DISCOVER or DISPLAY this object *starting on* 2015-01-01, while the Group "Staff" has permission to DISCOVER or DISPLAY this object *until* 2015-01-01.

```
<rights:RightsDeclarationMD xmlns:rights="http://cosimo.stanford.edu/sdr/metsrights/" RIGHTSCATEGORY="LICENSED">
  <rights:Context CONTEXTCLASS="GENERAL PUBLIC" start-date="2015-01-01" in-effect="false">
    <rights:Permissions DISCOVER="true" DISPLAY="true" MODIFY="false" DELETE="false" />
  </rights:Context>
  <rights:Context CONTEXTCLASS="MANAGED_GRP" end-date="2015-01-01" in-effect="true">
    <rights:UserName USERTYPE="GROUP">Staff</rights:UserName>
    <rights:Permissions DISCOVER="true" DISPLAY="true" MODIFY="false" DELETE="false" />
  </rights:Context>
</rights:RightsDeclarationMD>
```

Example of METSRights Schema for a Collection

A Collection AIP contains one METSRights section, which describes the permissions different Groups or People have within the Collection

Below is an example of a METSRights sections for a publicly visible Collection, which also has an Administrator group, a Submitter group, and a group for each of the three DSpace workflow approval steps. You'll notice that each of the groups is provided with very specific permissions within the Collection. Submitters & Workflow approvers can "ADD CONTENTS" to a collection (but cannot delete the collection). Administrators have full rights.

```
<rights:RightsDeclarationMD xmlns:rights="http://cosimo.stanford.edu/sdr/metsrights/" RIGHTSCATEGORY="LICENSED">
  <rights:Context CONTEXTCLASS="MANAGED_GRP">
    <rights:UserName USERTYPE="GROUP">COLLECTION_hdl:123456789/2_SUBMIT</rights:UserName>
    <rights:Permissions DISCOVER="true" DISPLAY="true" MODIFY="true" DELETE="false" OTHER="true"
```

```
OTHERPERMITTYPE="ADD CONTENTS" />
  </rights:Context>
  <rights:Context CONTEXTCLASS="MANAGED_GRP">
    <rights:UserName USERTYPE="GROUP">COLLECTION_hdl:123456789/2_WORKFLOW_STEP_3</rights:UserName>
    <rights:Permissions DISCOVER="true" DISPLAY="true" MODIFY="true" DELETE="false" OTHER="true"
OTHERPERMITTYPE="ADD CONTENTS" />
  </rights:Context>
  <rights:Context CONTEXTCLASS="MANAGED_GRP">
    <rights:UserName USERTYPE="GROUP">COLLECTION_hdl:123456789/2_WORKFLOW_STEP_2</rights:UserName>
    <rights:Permissions DISCOVER="true" DISPLAY="true" MODIFY="true" DELETE="false" OTHER="true"
OTHERPERMITTYPE="ADD CONTENTS" />
  </rights:Context>
  <rights:Context CONTEXTCLASS="MANAGED_GRP">
    <rights:UserName USERTYPE="GROUP">COLLECTION_hdl:123456789/2_WORKFLOW_STEP_1</rights:UserName>
    <rights:Permissions DISCOVER="true" DISPLAY="true" MODIFY="true" DELETE="false" OTHER="true"
OTHERPERMITTYPE="ADD CONTENTS" />
  </rights:Context>
  <rights:Context CONTEXTCLASS="MANAGED_GRP">
    <rights:UserName USERTYPE="GROUP">COLLECTION_hdl:123456789/2_ADMIN</rights:UserName>
    <rights:Permissions DISCOVER="true" DISPLAY="true" COPY="true" DUPLICATE="true" MODIFY="true" DELETE="true"
PRINT="true" OTHER="true" OTHERPERMITTYPE="ADMIN" />
  </rights:Context>
  <rights:Context CONTEXTCLASS="GENERAL PUBLIC">
    <rights:Permissions DISCOVER="true" DISPLAY="true" MODIFY="false" DELETE="false" />
  </rights:Context>
</rights:RightsDeclarationMD>
```

Example of METSRights Schema for a Community

A Community AIP contains one METSRights section, which describes the permissions different Groups or People have within that Community.

Below is an example of a METSRights sections for a publicly visible Community, which also has an Administrator group. As you'll notice, this content looks very similar to the Collection METSRights section (as described above)

```
<rights:RightsDeclarationMD xmlns:rights="http://cosimo.stanford.edu/sdr/metsrights/" RIGHTSCATEGORY="LICENSED">
  <rights:Context CONTEXTCLASS="MANAGED_GRP">
    <rights:UserName USERTYPE="GROUP">COMMUNITY_hdl:123456789/10_ADMIN</rights:UserName>
    <rights:Permissions DISCOVER="true" DISPLAY="true" COPY="true" DUPLICATE="true" MODIFY="true" DELETE="true"
PRINT="true" OTHER="true" OTHERPERMITTYPE="ADMIN" />
  </rights:Context>
  <rights:Context CONTEXTCLASS="GENERAL PUBLIC">
    <rights:Permissions DISCOVER="true" DISPLAY="true" MODIFY="false" DELETE="false" />
  </rights:Context>
</rights:RightsDeclarationMD>
```

Ant targets and options

1 Options
2 Targets

Ant targets should be run as the service user

A word of warning: in order to ensure proper permissions and file ownership are maintained, you are advised to run these ant targets as the service user (commonly 'dspace' or 'tomcat'). Running them as any other user may cause permission problems

Options

DSpace allows three property values to be set using the -D<property>=<value> option. They may be used in other contexts than noted below, but take care to understand how a particular property will affect a target's outcome.

overwrite

Whether to overwrite configuration files in [dspace]/config. If true, files from [dspace]/config and subdirectories are backed up with .old extension and new files are installed from [dspace-src]/dspace/config and subdirectories; if false, existing config files are untouched, and new files are written beside them with .new extension.	
Possible values:	true, false
Default:	true
Context:	update, init_configs

config

If a path is specified, ant uses values from the specified file and installs it in [dspace]/config in the appropriate contexts.	
Possible values:	path to configuration file to be used
Default:	[dspace-src]/config/dspace.cfg
Context:	update, update_configs, update_code, update_webapps, init_configs, fresh_install, test_database, setup_database, load_registries, clean_database

wars

If true, builds .war files; if false, no .war files are built.	
Possible values:	true, false
Default:	true
Context:	update, update_webapps, fresh_install

Targets

Target	Effect
update	Creates backup copies of the [dspace]/bin, /etc, /lib, and /webapps directories with the form /<directory>.bak-<date-time>. Creates new copies of [dspace]/config, /etc, and /lib directories. Does not affect data files or the database. (See *overwrite*, *config*, *war* options.)
update_configs	Updates the [dspace]/config directory with new configuration files. (See *config* option.)
update_code	Creates backup copies of the [dspace]/bin, /etc, and /lib directories with the form /<directory>.bak-<date-time>. Creates new copies of [dspace]/config, /etc, and /lib directories. (See *config* option.)
update_webapps	Updates [dspace]/webapps directory. (See *config*, *war* options.)
init_configs	Writes configuration files to [dspace]/config. (See *overwrite*, *config* options.)

install_code	Deletes existing [dspace]/bin, /lib, and /etc directories, and installs new copies; overwrites /solr application files, leaving data intact. (See *config* option.)
fresh_install	Performs a fresh installation of the software, including the database & config. (See *config*, *war* options.)
test_database	Tests database connection using parameters specified in dspace.cfg. (See *config* option.)
clean_backups	Removes [dspace]/bin, /etc, /lib, and /webapps directories with .bak* extensions.

Command Line Operations

1 Executing command line operations
2 Available operations
2.1 General use
2.2 Legacy statistics
2.3 SOLR Statistics

The DSpace command launcher or CLI interface offers the execution of different maintenance operations. As most of these are already documented in related parts of the documentation, this page is mainly intended to provide an overview of all available CLI operations, with links to the appropriate documentation.

Executing command line operations

The CLI interface is found at `[dspace]/bin/dspace`. Execute it without arguments or with the `-h` option to see all available operations. Execute `dspace` *op* `-h` to see details about the *op* operation.

Examples:

bin/dspace -h
bin/dspace cleanup -h
bin/dspace cleanup
bin/dspace cleanup --verbose

Available operations

Some operations can also be run as "Processes" (or Scripts) from the administrative User Interface or REST API. Those Scripts have a detailed description in our REST API documentation at https://github.com/DSpace/RestContract/blob/main/scripts/

General use

- bitstore-migrate: Migrate all files (bitstreams) from one assetstore (bitstore) to another
- checker: Run the checksum checker
- checker-emailer: Send emails related to the checksum checker
- classpath: Calculate and display the DSpace classpath
- cleanup: Remove deleted bitstreams from the assetstore
- community-filiator: Tool to manage community and sub-community relationships
- create-administrator: Create a DSpace administrator account (see Installing DSpace)
- curate: Perform curation tasks on DSpace objects
- database: Perform various tasks / checks of the DSpace database
- doi-organiser: Transmit information about DOIs to the registration agency.
- dsprop: View the value of a DSpace property from any configuration file (see Configuration Reference)
- dsrun: Run a (DSpace) Java class directly (used mainly for test purposes)
- embargo-lifter: Pre DSpace 3.0 embargo manager tool used to check, list and lift embargoes
- export: Export items or collections
- filter-media: Perform the media filtering to extract full text from documents and to create thumbnails
- generate-sitemaps: Generate search engine and html sitemaps (see Search Engine Optimization)
- harvest: Manage the OAI-PMH harvesting of external collections (see OAI harvesting docs)
- import: Import items into DSpace (see Importing and Exporting Items via Simple Archive Format (SAF))
- index-authority: import authorities and keep SOLR authority index up to date
- index-discovery: Update Discovery (Solr) search and browse Index
- itemupdate: Item update tool for altering metadata and bitstream content in items (see Updating Items via Simple Archive Format)
- make-handle-config: Run the handle server simple setup command
- metadata-export: Export metadata for batch editing
- metadata-import: Import metadata after batch editing
- migrate-embargo: Embargo manager tool used to migrate old version of Embargo to the new one included in dspace3
- oai: OAI script manager
- packager: Execute a packager
- process-cleaner: Delete old Processes from the system
- rdfizer: tool to convert contents to RDF
- read : execute a stream of commands from a file or pipe
- registry-loader: Load entries into a registry (see Metadata and Bitstream Format Registries)
- structure-builder: Build DSpace community and collection structure (see Exporting and Importing Community and Collection Hierarchy)
- sub-daily: Send daily subscription notices
- test-email: Test the DSpace email server settings are OK
- update-handle-prefix: Update handle records and metadata when moving from one Handle prefix to another
- user: Create, List, Update, Delete EPerson (user) records
- validate-date: Test date-time format rules
- version: Show DSpace version and other troubleshooting information

Legacy statistics

DSpace 7.x does not yet support

Legacy/log based statistics are not available in DSpace 7.x. They are under discussion as this feature is not widely used. Tentatively, they are scheduled for possible removal. See https://github.com/DSpace/DSpace/issues/2852

Legacy statistics parse the DSpace log files and compile information based on the "[dspace]/config/dstat.cfg" configuration file. They are no longer actively maintained, but still exist in the codebase because there is information they report on that is not yet accessible in (or replaced by) SOLR Statistics. Where possible, we recommend using SOLR Statistics and/or Google Analytics for more accurate data.

- stat-general: Compile the general statistics
- stat-initial: Compile the initial statistics
- stat-monthly: Compile the monthly statistics
- stat-report-general: Create the general statistics report
- stat-report-initial: Create the initial statistics report
- stat-report-monthly: Create the monthly statistics report

SOLR Statistics

Scripts for the statistics that are stored in SOLR:

- solr-export-statistics:Export Solr statistics data to CSV (for backup or moving to another server)
- solr-import-statistics: Import Solr statistics data from CSV (for restoration, or moving to another server)
- solr-reindex-statistics: Reindex Solr statistics data (for upgrades or updates to Solr schema)
- stats-log-converter: Convert dspace.log files ready for import into solr statistics
- stats-log-importer: Import previously converted log files into solr statistics
- stats-util: Statistics Client for Maintenance of Solr Statistics Indexes

Database Utilities

1 "database" command
2 Guide to Flyway Migration States

"database" command

This command can be used at any time to manage or upgrade the Database. It will also assist in troubleshooting PostgreSQL and Oracle connection issues with the database.

Command used:	`[dspace]/bin/dspace database`
Java class:	org.dspace.storage.rdbms.DatabaseUtils
Valid Arguments:	**Description**
`test`	Test the database connection settings (in `[dspace]/config/dspace.cfg` or `local.cfg`) are OK and working properly. This command also validates the database version is compatible with DSpace.
`info / status`	Provide detailed information about the DSpace database itself. This includes the database type, version, driver, schema, and any successful/failed/pending database migrations. This command, along with "test", is very useful in debugging issues with your database.
`migrate`	Migrate the database to the latest version (if not already on the latest version). This uses FlywayDB along with embedded migrations scripts to automatically update your database to the latest version. Additional 'migrate' options: • "`migrate ignored`" will run a migration which *also includes* any database migrations which are flagged as "Ignored" (or "Skipped") by the "info" command. If these "Ignored" migrations succeed, they will now be noted (in the "info" command) as having run "Out Of Order" (i.e. they were successful, but they were executed out of the normal, numerical order). • "`migrate force`" (*available in 7.1 and later*) will run a migration *even when no new migrations exist* (i.e. no migrations are currently flagged as "Pending" when using the "info" command). This can be used to force the post-migration ("callback") scripts to run. Normally, these post-migration scripts only run after a new migrations are applied. They will (re-)initialize your database with required objects, like the "Site" object, default groups (Administrator/Anonymous) and default metadata registry and bitstream format registry entries.
`repair`	Attempt to "repair" any migrations which are flagged as "Failed" by the "info" command and/or resolve failed checksum validation. This runs the FlywayDB repair command. Please note however, this will NOT automatically repair corrupt or broken data in your database. It merely tries to re-run previously "Failed" migrations and/or realign the checksums of the applied migrations to the ones of the available migrations.
`skip`	*(Available in 7.5 and later)* Allows you to "skip" individual database migrations. Skipping a migration will flag it as having run successfully (either "Success" or "Out of Order" status), but the migration *will not be executed.* *WARNING: You should ONLY skip migrations which are no longer required or have become obsolete.* Skipping a REQUIRED migration may result in DSpace failing to startup or function properly. The only fix to that scenario would be to run the migration manually (by executing the SQL directly on the database). Therefore, this "skip" command should ONLY be used when the migration is known to be obsolete or no longer valid. All other usages are unsupported.
`update-sequences`	Update database sequences after running a bulk ingest (e.g. AIP Backup and Restore) or data migration.
`validate`	Validate the checksums of all previously run database migrations. This runs the FlywayDB 'validate' command.
`clean`	Completely and permanently delete all tables and data in this database. WARNING: There is no turning back! If you run this command, you will lose your entire database and all its contents. This command is only useful for testing or for reverting your database to a "fresh install" state (e.g. running "`dspace database clean`" followed by "`dspace database migrate`" will return your database to a fresh install state) By default the 'clean' command is **disabled** (to avoid accidental data loss). In order to enable it, you must first set `db.cleanDisabled=false` in either your local.cfg or dspace.cfg.

Guide to Flyway Migration States

Whenever you run the "info" or "status" option (e.g. ./dspace database info), you'll see a table listing all Flyway migrations which were run on your database.

In the "State" column of that table you'll see the various states that Flyway returns for each migration:

- "Baseline" - This state simply notes when your database was originally created. In other words, it's the **first** migration in the table
- "Success" - The state means that the migration was successful. It completed without any errors.
- "Out of Order" - This state means the migration was successful *but was run in a different order than the default.* By default, Flyway will run all migrations in numerical order (based on the "Version" column in that table). **These "Out of Order" entries are normal and will occur in almost every production DSpace.** These entries are a sign of *backported bug fixes* which required a database migration. Those backported fixes may be required to run in *non-numerical order.*
    - For example: If we found a database issue that impacts both 7.x and 8.x, we may create a migration numbered "7.9.[date]" which is *safe to run on both 7.x and 8.x.* If your site is already on 8.x, this migration would be flagged as "Out of Order" because it starts with "7.9.*" and was run **after** other "8.0.*" migrations were already run.
- "Ignored" - This state means the migration was **skipped** (because it's not in numerical order) and *has not yet been run.* To run it, you'd run "./dspace database migrate ignored". Ignored migrations will move into the "Out of Order" state if they are successfully run.
- "Pending" - This state means the migration *has not yet been run*. It's waiting for the next time you call "./dspace database migrate".

Executing streams of commands

You can pass a sequence of commands into the `dspace` command-line tool using the `read` command.

Execute commands...	this way
...from a file	`[dspace]/bin/dspace read a-command-file`
...in a pipeline	`some-other-command | [dspace]/bin/dspace read -` `some-other-command | [dspace]/bin/dspace read`

Handle.Net Registry Support

DSpace comes with support for CNRI's Handle.Net Registry (HNR). This feature is *completely optional*, as DSpace functions the same with or without using a Handle Server/Registry.

A few things to keep in mind:

- You'll notice that while you've been playing around with a test server, DSpace has apparently been creating (fake) handles for you looking like *hdl:123456789/24* and so forth. These aren't really Handles, since the global Handle system doesn't actually know about them, and lots of other DSpace test installs will have created the same IDs. They're only really Handles once you've registered a prefix with CNRI (see below) and have correctly set up the Handle server included in the DSpace distribution. This Handle server communicates with the rest of the global Handle infrastructure so that anyone that understands Handles can find the Handles your DSpace has created.
- If you want to use the Handle system, you'll need to set up a Handle server. One is included with DSpace.
- If you want to use the Handle system, you'll need to obtain a Handle prefix from the central CNRI Handle site. This requires a small yearly fee to CNRI
- Again, all of this is **completely optional.** But, the key benefit is that it provides you with persistent, permanent URLs (of the form `https://hdl.handle.net/[prefix]/[suffix]`) for every object within your DSpace site. Those persistent URLs may be useful for citations or even during upgrades/migrations, as DSpace + Handle.Net ensures that these URLs always go to the right object, even if your site's main URL changes.

A Handle server runs as a separate process that receives TCP requests from other Handle servers, and issues resolution requests to a global server or servers if a Handle entered locally does not correspond to some local content. The Handle protocol is based on TCP, so it will need to be installed on a server that can send and receive TCP on port 2641.

You can either use a Handle server running on the same machine as DSpace, or you can install it on a separate machine. Installing it on the same machine is a little bit easier. If you install it on a separate machine, you can use one Handle server for more than one DSpace installation.

- To install your Handle resolver on the host where DSpace runs
- To install a Handle resolver on a separate machine
- To install a Handle resolver on a separate machine using template handles
- Updating Existing Handle Prefixes

To install your Handle resolver on the host where DSpace runs

We recommend configuring your Handle server **without a passphrase**, as the current DSpace `start-handle-server` scripts do not yet support startup with a passphrase.

If you choose to set a passphrase, you may need to start the Handle Server via: `[dspace]\bin\dspace dsrun net.handle.server.Main [dspace]\handle-server`

1. To configure your DSpace installation to run the handle server, run the following command:

   ```
   [dspace]/bin/make-handle-config
   ```

   a. If you are using Windows, the proper command is:

      ```
      [dspace]/bin/dspace dsrun net.handle.server.SimpleSetup [dspace]/handle-server
      ```

      Ensure that *[dspace]/handle-server* matches whatever you have in *dspace.cfg* for the *handle.dir* property. You will need to answer a series of qestions to configure the server. For the most part, you can use the default options, except you should choose to **not** encrypt your certificates when prompted.
2. Edit the resulting *[dspace]/handle-server/config.dct* file to include the following lines in the *"server_config"*clause:

   ```
   "storage_type" = "CUSTOM"
   "storage_class" = "org.dspace.handle.HandlePlugin"
   "enable_txn_queue" = "no"
   ```

   This tells the Handle server to get information about individual Handles from the DSpace code and to disable transaction replication. If you used the make-handle-config script, these should already be set in your config.dct file.
3. Once the configuration file has been generated, you will need to go to https://hdl.handle.net/4263537/5014 to upload the generated sitebndl.zip file. The upload page will ask you for your contact information. An administrator will then create the naming authority/prefix on the root service (known as the Global Handle Registry), and notify you when this has been completed. You will not be able to continue the handle server installation until you receive further information concerning your naming authority.
4. When CNRI has sent you your naming authority prefix, you will need to edit the *config.dct* file. The file will be found in */[dspace]/handle-server*. Look for *"300:0.NA/123456789"*. Replace 123456789 with the assigned naming authority prefix sent to you. Also change the value of `handle.prefix` in `[dspace]/config/local.cfg` from "123456789" to your assigned naming authority prefix, so that DSpace will use that prefix in assigning new Handles.
5. Now start your handle server (as the dspace user):

```
[dspace]/bin/start-handle-server
```

a. If you are using Windows, there is a corresponding 'start-handle-server.bat' script:

```
[dspace]/bin/start-handle-server.bat
```

Note that since the DSpace code manages individual Handles, administrative operations such as Handle creation and modification aren't supported by DSpace's Handle server.

To install a Handle resolver on a separate machine

This works with DSpace 7.4 and later.

The Handle server you use must be dedicated to resolve Handles from DSpace. You cannot use a Handle server that is in use with other software already. You can use CNRI's Handle Software -- all you have to do is to add to it a plugin that is provided by DSpace. The following instructions were tested with CNRI's Handle software version 9.1.0. You can do the following steps on another machine than the machine DSpace runs on, but you have to copy some files from the machine on which DSpace is installed.

1. Set the following two configuration properties for every DSpace backend that your are running:

DSpace backend configuration to activate the endpoints used by the remote handle resolver

```
handle.remote-resolver.enabled = true
handle.hide.listhandles = false
```

2. Download the CNRI Handle Software: http:s//www.handle.net/download.html. In the tarball you'll find an `README.txt` with installation instructions -- follow it.
3. After installing the CNRI Handle Software you should have two directories: once that contains the CNRI software and one that contains the configuration of you local Handle Server. For the rest of this instruction we assume that the directory containing the CNRI Software is `/hs/handle-9.1.0` and the directory containing the configuration of your local server is `/hs/srv_1`. (We use the same paths here as CNRIs README.txt.)
4. Download the plugin from https://github.com/DSpace/Remote-Handle-Resolver/releases. Select a release. You can get the source and build it yourself, or just use the JAR file included in the release. In either case, once you have a `dspace-remote-handle-resolver-VERSION.jar`, copy it to the directory containing the CNRI software (`/hs/handle-9.1.0/lib`).
5. Create the directory `/hs/srv_1/logs`.
6. Create the following two files in `/hs/srv_1`.

log4j-handle-plugin.properties

```
log4j.rootCategory=INFO, A1
log4j.appender.A1=org.apache.log4j.DailyRollingFileAppender
log4j.appender.A1.File=/hs/srv_1/logs/handle-plugin.log
log4j.appender.A1.DatePattern= '.' yyyy-MM-dd
log4j.appender.A1.layout=org.apache.log4j.PatternLayout
log4j.appender.A1.layout.ConversionPattern=%d %-5p %c @ %m%n
log4j.logger.org.apache.axis.handlers.http.HTTPAuthHandler=INFO
```

Change the path in the third line, if necessary. It must point to the DSpace 7 Rest API (as configured in $dspace.server.url).

handle-dspace-plugin.cfg

```
dspace.handle.endpoint1 = http: //dspace.example.org/server
```

If you run more than one DSpace Installation, you may add more DSpace Endpoints. Just increase the number at the end of the key for each: `endpoint2`, `endpoint3`....

7. Edit the file `/hs/srv_1/config.dct` to include the following lines in the "`server_config`" clause:

```
"storage_type" = "CUSTOM"
"storage_class" = "org.dspace.handle.MultiRemoteDSpaceRepositoryHandlePlugin"
```

8. Edit `/hs/handle-9.1.0/bin/hdl`:
   a. Find a line that contains `exec java ... net.handle.server.Main ...`

    b. Add "`-Dlog4j.configuration=file:///hs/srv_1/log4j-handle-plugin.properties -Ddspace.handle.plugin.configuration=/hs/srv_1/handle-dspace-plugin.cfg`" right in front of `net.handle.server.Main`.
9. If your handle server is running, restart it.

Please note: The Handle Server will only start if it is able to connect to at least one running DSpace Installation. It only resolves the handles of the DSpace Installations that were running when it was started.

To install a Handle resolver on a separate machine using template handles

Instead of using the described plugin above, you can configure a Handle server (version 8+) to resolve handles based on a template. Template handle require less configuration than the plugin, and do not require an additional download. However, there are two things to keep in mind when using template handles:

1. Handles that don't exist will still generate a Handle record with a URL, even though resolving that URL will show an error page.
2. Handle records can only be generated based on the handle and the template. If you need to look up information in DSpace in or to geneate the correct url for a given handle, you will need to use a storage plugin instead.

The Handle server you use must be dedicated to resolve Handles from DSpace. You cannot use a Handle server that is in use with other software already. The following instructions were tested with CNRI's Handle software version 9.1.0.

1. Download the CNRI Handle Software: https://www.handle.net/download.html.
2. In the tarball you'll find an `README.txt` with installation instructions. Follow the directions to install and configure your Handle server. Importantly, make sure your prefixes are set correctly in the "auto_homed_prefixes" setting.
3. Edit `the server's config.dct` file to include the following line in the "`server_config`" clause:

```
"namespace" = "<namespace><template delimiter'/'><value type='URL' index='1' data='https://demo.dspace.org/handle/${handle}'/></template></namespace>"
```

   In the "namespace" section, replace "`https://demo.dspace.org/handle/`" with the url endpoint for your DSpace server. The "${handle}" part of the template will be replaced with the full handle to be resolved.
4. If your handle server is running, restart it.

This configuration is a minimal example of how to configure template handles for DSpace. For more details about configuring template handles, see the Handle Technical Manual, Chapter 11 (PDF download).

Updating Existing Handle Prefixes

If you need to update the handle prefix on items created before the CNRI registration process you can run the *[dspace]/bin/dspace update-handle-prefix script*. You may need to do this if you loaded items prior to CNRI registration (e.g. setting up a demonstration system prior to migrating it to production). The script takes the current and new prefix as parameters. For example:

```
[dspace]/bin/dspace update-handle-prefix 123456789 1303
```

This script will change any handles currently assigned prefix 123456789 to prefix 1303, so for example handle 123456789/23 will be updated to 1303/23 in the database.

Logical Item Filtering and DOI Filtered Provider for DSpace

- Section One: DSpace Logical Item filtering (org.dspace.content.logic.*)
  - LogicalStatement
  - Filters
  - Operators
  - Conditions
  - Configuring Filters in Spring
  - Running Tests on the Command Line
  - Using Filters in other Spring Services
- Section Two: DOI Filtered Provider
  - New FilteredProvider: DOIIdentifierProvider
  - New Curation Task:

Section One: DSpace Logical Item filtering (org.dspace.content.logic.*)

Inspired by the powerful conditional filters in XOAI, this component offers a simple but flexible way to write logical statements and tests, and use the results of those tests in other services or DSpace code.

LogicalStatement

LogicalStatement is a simple interface ultimately implemented by all the other interfaces and classes described below. It just requires that a class implements a `Boolean getResult(context, item)` method.

Filters

Filters are at the root of any test definition, and it is the filter ID that is used to load up the filter in spring configurations for other services, or with DSpace Service Manager.

A filter bean is defined with a single “statement” property - this could be an Operator, to begin a longer logical statement, or a Condition, to perform a simple check.

There is one simple implementation of Filter included - DefaultFilter.

Operators

Operators are the basic logical building blocks that implement operations like AND, OR, NOT, NAND and NOR. An Operator can contain any number of other Operators or Conditions.

So statements like this can be created:

(**x** AND (**y** OR **z**) AND **a** AND (**b** OR NOT(**d**))

Conditions

Conditions are where the actual DSpace item evaluation code is written. A condition accepts a Map<String, Object> map of parameters. Conditions don’t contain any other LogicalStatement classes – the are at the bottom of the chain.

A condition could be something like MetadataValueMatchCondition, where a regex pattern and field name are passed as parameters, then tested against actual item metadata. If the regex matches, the boolean result is true.

Typically, commonly used Conditions will be defined as beans elsewhere in the spring config and then referenced inside Filters and Operators to create more complex statements.

Configuring Filters in Spring

Conditions, Operators and Filters are all defined in `${dspace}/config/spring/api/item-filters.xml`

Here’s a complete example of a filter definition that implements the same rules as the XOAI openAireFilter. As an exercise, some statements will be defined as beans externally, and some will be defined inline as part of the filter.

New Condition: driver-document-type_condition

This condition creates a new bean to test metadata values. In this case, we’re implementing “ends with” for a list of type patterns.

```
<!-- dc.type ends with any of the listed values, as per XOAI "driverDocumentTypeCondition" -->
    <bean id="driver-document-type_condition"
          class="org.dspace.content.logic.condition.MetadataValuesMatchCondition">
        <property name="parameters">
            <map>
                <entry key="field" value="dc.type" />
                <entry key="patterns">
                    <list>
                        <value>article$</value>
                        <value>bachelorThesis$</value>
                        <value>masterThesis$</value>
                        <value>doctoralThesis$</value>
                        <value>book$</value>
                        <value>bookPart$</value>
                        <value>review$</value>
                        <value>conferenceObject$</value>
                        <value>lecture$</value>
                        <value>workingPaper$</value>
                        <value>preprint$</value>
                        <value>report$</value>
                        <value>annotation$</value>
                        <value>contributionToPeriodical$</value>
                        <value>patent$</value>
                        <value>dataset$</value>
                        <value>other$</value>
                    </list>
                </entry>
            </map>
        </property>
    </bean>
```

New Condition: item-is-public_condition

This condition accepts group and action parameters, then inspects item policies for a match - if the supplied group can perform the action, the result is true.

```
<bean id="item-is-public_condition"
          class="org.dspace.content.logic.condition.ReadableByGroupCondition">
        <property name="parameters">
            <map>
                <entry key="group" value="Anonymous" />
                <entry key="action" value="READ" />
            </map>
        </property>
</bean>
```

New Filter: openaire_filter

Here is the full definition for the OpenAIRE filter.

The first statement is an And Operator, with many sub-statements – four Conditions, and an Or statement.

The first two statements in this Operator are simple Conditions defined in-line, and just check for a non-empty value in a couple of metadata fields.

The third statement is a reference to the document type Condition we made earlier:
`<ref bean="driver-document-type_condition" />`

The fourth statement is another Operator, in this case an Or Operator with two Conditions (the is-public Condition we defined earlier, and an in-line definition of as “is-withdrawn” Condition)

The fifth statement is an in-line definition of a Condition that checks dc.relation metadata for a valid OpenAIRE identifier.

So the full logic implemented is:

`(has-title AND has-author AND has-driver-type AND (is-public OR is-withdrawn) AND has-valid-relation)`

```
<!-- An example of an OpenAIRE compliance filter based on the same rules in xoai.xml
      some sub-statements are defined within this bean, and some are referenced from earlier definitions
-->
<bean id="openaire_filter" class="org.dspace.content.logic.DefaultFilter">
    <property name="statement">
        <bean class="org.dspace.content.logic.operator.And">
            <property name="statements">
                <list>
                    <!-- Has a non-empty title -->
                    <bean id="has-title_condition"
                          class="org.dspace.content.logic.condition.MetadataValueMatchCondition">
                        <property name="parameters">
                            <map>
                                <entry key="field" value="dc.title" />
                                <entry key="pattern" value=".*" />
                            </map>
                        </property>
                    </bean>
                    <!-- AND has a non-empty author -->
                    <bean id="has-author_condition"
                          class="org.dspace.content.logic.condition.MetadataValueMatchCondition">
                        <property name="parameters">
                            <map>
                                <entry key="field" value="dc.contributor.author" />
                                <entry key="pattern" value=".*" />
                            </map>
                        </property>
                    </bean>
                    <!-- AND has a valid DRIVER document type (defined earlier) -->
                    <ref bean="driver-document-type_condition" />
                    <!-- AND (the item is publicly accessible OR withdrawn) -->
                    <bean class="org.dspace.content.logic.operator.Or">
                        <property name="statements">
                            <list>
                                <!-- item is public, defined earlier -->
                                <ref bean="item-is-public_condition" />
                                <!-- OR item is withdrawn, for tombstoning -->
                                <bean class="org.dspace.content.logic.condition.IsWithdrawnCondition">
                                    <property name="parameters"><map></map></property>
                                </bean>
                            </list>
                        </property>
                    </bean>
                    <!-- AND the dc.relation is a valid OpenAIRE identifier
                          (starts with "info:eu-repo/grantAgreement/") -->
                    <bean id="has-openaire-relation_condition"
                          class="org.dspace.content.logic.condition.MetadataValueMatchCondition">
                        <property name="parameters">
                            <map>
                                <entry key="field" value="dc.relation" />
                                <entry key="pattern" value="^info:eu-repo/grantAgreement/" />
                            </map>
                        </property>
                    </bean>
                </list>
            </property>
        </bean>
    </property>
</bean>
```

Running Tests on the Command Line

There is a launcher command that can arbitrarily run tests on an item or all items, eg.

`${dspace}/bin/dspace dsrun org.dspace.content.logic.TestLogicRunner -f openaire_filter -i 123456789/100`

A simple `true` or `false` is printed for each item tested.

Using Filters in other Spring Services

The Filter beans can be referenced (or defined) in other services, for instance, here is adding the bean we configured earlier, as a `filterService` to a new `FilteredDOIIdentifierProvider`:

```
<bean id="org.dspace.identifier.DOIIdentifierProvider"
      class="org.dspace.identifier.FilteredDOIIdentifierProvider"
      scope="singleton">
    <property name="configurationService"
              ref="org.dspace.services.ConfigurationService" />
    <property name="DOIConnector"
              ref="org.dspace.identifier.doi.DOIConnector" />
    <property name="filterService"
              ref="openaire_filter"/>
</bean>
```

In the provider, we just define the property with the other services and class variables:

```
private Filter filterService;
```

And make sure there is a setter for it:

```
@Required
public void setFilterService(Filter filterService) {
    this.filterService = filterService;
}
```

Then you can actually run the tests with the service, like this:

```
try {
    Boolean result = filterService.getResult(context, (Item) dso);
    // do something with result
} catch(LogicalStatementException e) {
    // ... handle exception ...
}
```

In the TestLogicRunner, you can see a way to get the filters by name using the DSpaceServiceManager as well.

Section Two: DOI Filtered Provider

New FilteredProvider: DOIIdentifierProvider

DOIIdentifierProvider now extends a base FilteredIdentifierProvider, which looks for any configured filters and only allows minting DOIs for items where the filter returns **true**

This filter is always applied to the DOI consumer and other internal DOI service calls, and is applied by default to the `doi-organiser` tool (though it can be optionally skipped with a command-line argument)

The filter is a spring property configured in identifier-service.xml, in the provider bean declaration.

The filterService property is *optional*. If it is missing from spring configuration, all items will get DOIs minted as per normal and the provider's filter service will be null.

It is defined as follows:

```
<bean id="org.dspace.identifier.DOIIdentifierProvider"
    class="org.dspace.identifier.FilteredDOIIdentifierProvider"
    scope="singleton"> <property name="configurationService"
    ref="org.dspace.services.ConfigurationService" />
    <property name="DOIConnector" ref="org.dspace.identifier.doi.DOIConnector" />
    <property name="filterService" ref="openaire_filter"/>
</bean>
```

Where the "openaire_filter" reference is the ID of a filter bean defined in item-filters.xml

New Curation Task:

In DSpace 5 and 6 implementations of this feature, JSPUI and XMLUI buttons were added to the Edit Item administrative pages so that DOIs could be manually registered (queued for registration) by administrators, explicitly **skipping** the filter.

In the DSpace 7 implementation, this feature can be used via the existing curation task framework, either in the CLI or in the Angular UI (when curation tasks are implemented).

Configuration

This task is configured in ${dspace}/config/modules/curate.cfg as 'registerdoi' with the label "Register DOI".

There is a configuration file in ${dspace}/config/modules/doi-curation.cfg that can be used to customise the behaviour regarding filter skipping, and distribution over multiple items.

```
### DOI registration curation task configuration module


##
# Should any logical filters be skipped when registering DOIs? (ie. *always* register, never filter out the item)
# Default: true
#doi-curation.skip-filter = true


##
# Should we allow the curation task to be distributed over communities / collections of items or the whole
repository?
# This *could* be dangerous if run accidentally over more items than intended.
# Default: false
#doi-curation.distributed = false
```

Mediafilters for Transforming DSpace Content

1 MediaFilters: Transforming DSpace Content
- 1.1 Overview
- 1.2 Available Media Filters
- 1.3 Enabling/Disabling MediaFilters
- 1.4 Executing (via Command Line)
- 1.5 Creating Custom MediaFilters
  - 1.5.1 Creating a simple Media Filter
  - 1.5.2 Creating a Dynamic or "Self-Named" Format Filter
- 1.6 Configuration parameters

MediaFilters: Transforming DSpace Content

Overview

DSpace can apply filters or transformations to files/bitstreams, creating new content. Filters are included that extract text for **full-text searching**, and create **thumbnails** for items that contain images. The media filters are controlled by the `dspace filter-media` script which traverses the asset store, invoking all configured `MediaFilter` or `FormatFilter` classes on files/bitstreams (see Configuring Media Filters for more information on how they are configured).

Available Media Filters

Below is a listing of all currently available Media Filters, and what they actually do:

Name	Java Class	Function	Default input formats	Enabled by Default?
Text Extractor (7.3 or above)	`org.dspace.app.mediafilter.TikaTextExtractionFilter`	As of 7.3, all text extraction for Full text indexing takes place in a single filter. This filter uses Apache Tika which supports a wide variety of formats (e.g. Microsoft products, PDF, HTML, Text, etc). Additional formats may be configured from the Tika supported formats list at https://tika.apache.org/2.3.0/formats.html	Adobe PDF, Microsoft formats (Word, PPT, Excel), CSV, HTML, RTF, Text, OpenDocument formats (Text, Spreadsheet, Presentation)	yes
PDF Text Extractor (7.2 or below)	`org.dspace.app.mediafilter.PDFFilter`	extracts the full text of Adobe PDF documents (only if text-based or OCRed) for full text indexing. (Uses the Apache PDFBox tool)	Adobe PDF	yes
HTML Text Extractor (7.2 or below)	`org.dspace.app.mediafilter.HTMLFilter`	extracts the full text of HTML documents for full text indexing. (Uses Swing's HTML Parser)	HTML, Text	yes
Word Text Extractor (7.2 or below)	`org.dspace.app.mediafilter.PoiWordFilter`	extracts the full text of Microsoft Word and Microsoft Word XML documents for full text indexing. (Uses the "Apache POI" tools.)	Microsoft Word, Microsoft Word XML	yes
Excel Text Extractor (7.2 or below)	`org.dspace.app.mediafilter.ExcelFilter`	extracts the full text of Microsoft Excel documents for full text indexing. (Uses the "Apache POI" tools.)	Microsoft Excel, Microsoft Excel XML	yes
PowerPoint Text Extractor (7.2 or below)	`org.dspace.app.mediafilter.PowerPointFilter`	extracts the full text of slides and notes in Microsoft PowerPoint and PowerPoint XML documents for full text indexing. (Uses the Apache POI tools.)	Microsoft Powerpoint, Microsoft Powerpoint XML	yes
PDFBox JPEG Thumbnail	`org.dspace.app.mediafilter.PDFBoxThumbnail`	creates thumbnail images of the first page of PDF files	Adobe PDF	yes
JPEG Thumbnail	`org.dspace.app.mediafilter.JPEGFilter`	creates thumbnail images of GIF, JPEG and PNG files	BMP, GIF, JPEG, image/png	yes
Branded Preview JPEG	`org.dspace.app.mediafilter.BrandedPreviewJPEGFilter`	creates a branded preview image for GIF, JPEG and PNG files	BMP, GIF, JPEG, image/png	no

ImageMagick Image Thumbnail Generator	`org.dspace.app.mediafilter.ImageMagickImageThumbnailFilter`	Uses ImageMagick to generate thumbnails for image bitstreams. Requires installation of ImageMagick on your server. See ImageMagick Media Filters.	BMP, GIF, image/png, JPG, TIFF, JPEG, JPEG 2000	no
ImageMagick PDF Thumbnail Generator	`org.dspace.app.mediafilter.ImageMagickPdfThumbnailFilter`	Uses ImageMagick and Ghostscript to generate thumbnails for PDF bitstreams. Requires installation of ImageMagick and Ghostscript on your server. See ImageMagick Media Filters.	Adobe PDF	no

Please note that the `filter-media` script will automatically update the DSpace search index by default.

Enabling/Disabling MediaFilters

The media filter plugin configuration `filter.plugins` in `dspace.cfg` contains a list of all enabled media/format filter plugins (see Configuring Media Filters for more information). By modifying the value of `filter.plugins` you can disable or enable MediaFilter plugins. The `filter.plugins` setting can be set multiple times to enable multiple filters. Each filter must be enabled via its name (see "Name" column in the table above).

```
# Enable the default Text Extractor (for 7.3 or above)
filter.plugins = Text Extractor

# Enable the JPEG thumbnail creator
filter.plugins = JPEG Thumbnail

# Enable the PDF thumbnail creator
filter.plugins = PDFBox JPEG Thumbnail
```

Executing (via Command Line)

The media filter system is intended to be run from the command line (or regularly as a cron task):

```
[dspace]/bin/dspace filter-media
```

With no options, this traverses the asset store, applying media filters to bitstreams, and skipping bitstreams that have already been filtered.

Available Command-Line Options:

- **Help** : `[dspace]/bin/dspace filter-media -h`
  - Display help message describing all command-line options.
- **Force mode** : `[dspace]/bin/dspace filter-media -f`
  - Apply filters to ALL bitstreams, even if they've already been filtered. If they've already been filtered, the previously filtered content is overwritten.
- **Identifier mode** : `[dspace]/bin/dspace filter-media -i 123456789/2`
  - Restrict processing to the community, collection, or item named by the identifier - by default, all bitstreams of all items in the repository are processed. The identifier must be a Handle, not a DB key. This option may be combined with any other option.
- **Maximum mode** : `[dspace]/bin/dspace filter-media -m 1000`
  - Suspend operation after the specified maximum number of items have been processed - by default, no limit exists. This option may be combined with any other option.
- **Plugin mode** : `[dspace]/bin/dspace filter-media -p "PDF Text Extractor","Word Text Extractor"`
  - Apply ONLY the filter plugin(s) listed (separated by commas). By default all named filters listed in the *filter.plugins* field of *dspace.cfg* are applied. This option may be combined with any other option. *WARNING:* multiple plugin names must be separated by a comma (i.e. ',') and NOT a comma followed by a space (i.e. ', ').
- **Skip mode** : `[dspace]/bin/dspace filter-media -s 123456789/9,123456789/100`
  - SKIP the listed identifiers (separated by commas) during processing. The identifiers must be Handles (not DB Keys). They may refer to items, collections or communities which should be skipped. This option may be combined with any other option. *WARNING:* multiple identifiers must be separated by a comma (i.e. ',') and NOT a comma followed by a space (i.e. ', ').
  - NOTE: If you have a large number of identifiers to skip, you may maintain this list, one identifier per line, within a separate file (e.g. *filter-skiplist.txt*). Use the following format to call the program.
    - `[dspace]/bin/dspace filter-media -s $(paste -sd, - < filter-skiplist.txt)`
- **Verbose mode** : `[dspace]/bin/dspace filter-media -v`
  - Print all extracted text and other filter details to STDOUT.

Creating Custom MediaFilters

Adding your own filters is done by creating a class which *implements* the `org.dspace.app.mediafilter.FormatFilter` interface. See the Creating a new Media/Format Filter topic and comments in the source file `FormatFilter.java` for more information. In theory filters could be implemented in any programming language (C, Perl, etc.) However, they need to be invoked by the Java code in the Media Filter class that you create.

Creating a simple Media Filter

New Media Filters **must implement** the *org.dspace.app.mediafilter.FormatFilter* interface. More information on the methods you need to implement is provided in the *FormatFilter.java* source file. For example:

```
public class MySimpleMediaFilter implements FormatFilter
```

Alternatively, you could extend the *org.dspace.app.mediafilter.MediaFilter* class, which just defaults to performing no pre/post-processing of bitstreams before or after filtering.

```
public class MySimpleMediaFilter extends MediaFilter
```

You must give your new filter a "name", by adding it and its name to the *plugin.named.org.dspace.app.mediafilter.FormatFilter* field in *dspace.cfg*. In addition to naming your filter, make sure to specify its input formats in the *filter.<class path>.inputFormats* config item. Note the input formats must match the *short description* field in the Bitstream Format Registry (i.e. *bitstreamformatregistry* table).

```
plugin.named.org.dspace.app.mediafilter.FormatFilter = \
        org.dspace.app.mediafilter.MySimpleMediaFilter = My Simple Text Filter, \ ...

filter.org.dspace.app.mediafilter.MySimpleMediaFilter.inputFormats =
        Text
```

If you neglect to define the *inputFormats* for a particular filter, the *MediaFilterManager* will never call that filter, since it will never find a bitstream which has a format matching that filter's input format(s).

If you have a complex Media Filter class, which actually performs different filtering for different formats (e.g. conversion from Word to PDF **and** conversion from Excel to CSV), you should define this as described in Chapter 13.3.2.2 .

Creating a Dynamic or "Self-Named" Format Filter

If you have a more complex Media/Format Filter, which actually performs **multiple** filtering or conversions for different formats (e.g. conversion from Word to PDF **and** conversion from Excel to CSV), you should have define a class which implements the *FormatFilter* interface, while also extending the Chapter 13.3.2.2 *SelfNamedPlugin* class. For example:

```
public class MyComplexMediaFilter extends SelfNamedPlugin implements FormatFilter
```

Since *SelfNamedPlugins* are self-named (as stated), they must provide the various names the plugin uses by defining a getPluginNames() method. Generally speaking, each "name" the plugin uses should correspond to a different type of filter it implements (e.g. "Word2PDF" and "Excel2CSV" are two good names for a complex media filter which performs both Word to PDF and Excel to CSV conversions).

Self-Named Media/Format Filters are also configured differently in *dspace.cfg*. Below is a general template for a Self Named Filter (defined by an imaginary *MyComplexMediaFilter* class, which can perform both Word to PDF and Excel to CSV conversions):

```
#Add to a list of all Self Named filters
plugin.selfnamed.org.dspace.app.mediafilter.FormatFilter = \
        org.dspace.app.mediafilter.MyComplexMediaFilter
#Define input formats for each "named" plugin this filter implements
        filter.org.dspace.app.mediafilter.MyComplexMediaFilter.Word2PDF.inputFormats = Microsoft Word
        filter.org.dspace.app.mediafilter.MyComplexMediaFilter.Excel2CSV.inputFormats = Microsoft Excel
```

As shown above, each Self-Named Filter class must be listed in the `plugin.selfnamed.org.dspace.app.mediafilter.FormatFilter` item in `dspace.cfg`. In addition, each Self-Named Filter **must** define the input formats for *each named plugin* defined by that filter. In the above example the *MyComplexMediaFilter* class is assumed to have defined two named plugins, `Word2PDF` and `Excel2CSV`. So, these two valid plugin names ("Word2PDF" and "Excel2CSV") **must** be returned by the `getPluginNames()` method of the `MyComplexMediaFilter` class.

These named plugins take different input formats as defined above (see the corresponding *inputFormats* setting).

If you neglect to define the `inputFormats` for a particular named plugin, the `MediaFilterManager` will never call that plugin, since it will never find a bitstream which has a format matching that plugin's input format(s).

For a particular Self-Named Filter, you are also welcome to define additional configuration settings in *dspace.cfg*. To continue with our current example, each of our imaginary plugins actually results in a different output format (Word2PDF creates "Adobe PDF", while Excel2CSV creates "Comma Separated Values"). To allow this complex Media Filter to be even more configurable (especially across institutions, with potential different "Bitstream Format Registries"), you may wish to allow for the output format to be customizable for each named plugin. For example:

```
#Define output formats for each named plugin
filter.org.dspace.app.mediafilter.MyComplexMediaFilter.Word2PDF.output Format = Adobe PDF
filter.org.dspace.app.mediafilter.MyComplexMediaFilter.Excel2CSV.outputFormat = Comma Separated Values
```

Any custom configuration fields in *dspace.cfg* defined by your filter are ignored by the *MediaFilterManager*, so it is up to your custom media filter class to read those configurations and apply them as necessary. For example, you could use the following sample Java code in your *MyComplexMediaFilter* class to read these custom *outputFormat* configurations from *dspace.cfg*:

```
#Get "outputFormat" configuration from dspace.cfg
String outputFormat =  ConfigurationManager.getProperty(MediaFilterManager.FILTER_PREFIX + "." +
MyComplexMediaFilter.class.getName() + "." + this.getPluginInstanceName() + ".outputFormat");
```

Configuration parameters

Property	**textextractor.max-chars** (only in 7.3 or above)
Example Value	textextractor.max-chars = 100000
Informational Note	By default, the "Text Extractor" only extracts the first 100,000 characters of text for full-text indexing. This setting allows you to increase or decrease that default. Set to -1 for no maximum. Keep in mind that larger values (or -1) are more likely to encounter OutOfMemoryException errors when extracting text from very large files. In those scenarios, you may wish to consider instead enabling "textextractor.use-temp-file" below to better control memory usage.
Property	**textextractor.use-temp-file** (only in 7.3 or above)
Example Value	textextractor.use-temp-file = false
Informational Note	By default, the "Text Extractor" will perform all text extraction in memory (i.e. textextractor.use-temp-file=false). This ensures text extraction runs quickly, but it has the risk of hitting OutOfMemoryException errors if you either increase "textextractor.max-chars" or simply don't have much available memory on the server. In those scenarios, you can set "textextractor.use-temp-file=true" in order to tell the text extraction process to extract all text using a temporary file. This *decreases* the memory usage of the text extraction process, but will run slightly slower.
Property	**filter.org.dspace.app.mediafilter.publicPermission**
Example Value	filter.org.dspace.app.mediafilter.publicPermission = JPEGFilter
Informational Note	By default mediafilter derivatives / thumbnails inherit the permissions of the parent bitstream, but you can override this, in case you want to make publicly accessible derivative / thumbnail content, typically the thumbnails of objects for the browse list. List the MediaFilter names that would get public accessible permissions. Any media filters not listed will instead inherit the permissions of the parent bitstream.

ImageMagick Media Filters

ImageMagick Media Filters

1 ImageMagick Media Filters
- 1.1 Overview
- 1.2 Installation
- 1.3 DSpace Configuration
  - 1.3.1 Thumbnail Dimensions
  - 1.3.2 Conversion Utility Path
  - 1.3.3 Supported file formats
  - 1.3.4 Overwriting Existing Thumbnails
  - 1.3.5 Flatten
  - 1.3.6 ICC Profiles
  - 1.3.7 Override ImageMagick Default Density
- 1.4 Additional Customization
- 1.5 Possible Errors / Issues
  - 1.5.1 "convert.im6: not authorized" errors
  - 1.5.2 "convert-im6.q16: cache resources exhausted" errors

As of DSpace 7.6, the ImageMagick media filter also supports creating thumbnails of video (MP4) files, provided that "ffmpeg" is installed locally. See instructions below.

Overview

The ImageMagick Media Filters provide consistent, high quality thumbnails for image bitstreams, PDF bitstreams and video (MP4) bitstreams.

These filters require a separate software installation of the conversion utilities: ImageMagick, Ghostscript (to support PDF thumbnails) and/or ffmpeg (to support MP4 thumbnails).

The media filters use the library im4java to invoke the conversion utilities. This library constructs a conversion command launches a sub-process to perform the generation of media files.

Installation

Before ImageMagick Media Filters can be used, you must setup ImageMagick (and optionally Ghostscript) as follows:

1. Install ImageMagick on your server. The installation process differs based on your operating system. For example, on Debian/Ubuntu, it's similar to this:

```
apt-get install imagemagick
```

2. If you wish to generate PDF thumbnails, install Ghostscript on your server. The installation process differs based on your operating system. For example, on Debian/Ubuntu, it's similar to this:

```
apt-get install ghostscript
```

3. (New in 7.6) If you wish to generate MP4 (video) thumbnails, install FFmpeg on your server. The installation process differs based on your operating system. For example, on Debian/Ubuntu, it's similar to this:

```
apt-get install ffmpeg
```

4. The ImageMagick, Ghostscript, and FFmpeg executables should be accessible from the same directory (e.g. `/usr/bin`)
   a. This directory MUST be defined in the `org.dspace.app.mediafilter.ImageMagickThumbnailFilter.ProcessStarter` configuration as describe below.

DSpace Configuration

In the **filter.plugins** section of your **dspace.cfg** (or local.cfg) file, specify the ImageMagick media filters you wish to use.

local.cfg

```
# Make sure to always keep this plugin enabled if you want to support search within text documents
filter.plugin = Text Extractor

# NOTE: When "ImageMagick Image Thumbnail" is enabled, the default "JPEG Thumbnail" should NOT be enabled
filter.plugins = ImageMagick Image Thumbnail

# NOTE: When "ImageMagick PDF Thumbnail" is enabled, the default "PDFBox JPEG Thumbnail" should NOT be enabled
# Requires Ghostscript to also be installed
filter.plugins = ImageMagick PDF Thumbnail

# New in 7.6, this will generate thumbnails from video files.
# Requires ffmpeg to also be installed
filter.plugins = ImageMagick Video Thumbnail
```

This will activate the following settings which are already present in **dspace.cfg** (these do NOT need to be added, as they already exist)

```
plugin.named.org.dspace.app.mediafilter.FormatFilter = org.dspace.app.mediafilter.
ImageMagickImageThumbnailFilter = ImageMagick Image Thumbnail
plugin.named.org.dspace.app.mediafilter.FormatFilter = org.dspace.app.mediafilter.ImageMagickPdfThumbnailFilter
= ImageMagick PDF Thumbnail
plugin.named.org.dspace.app.mediafilter.FormatFilter = org.dspace.app.mediafilter.
ImageMagickVideoThumbnailFilter = ImageMagick Video Thumbnail
```

These media filters contain the several properties which can be configured.

Thumbnail Dimensions

The following properties are used to define the dimensions of the generated thumbnails:

```
# maximum width and height of generated thumbnails
thumbnail.maxwidth  = 80
thumbnail.maxheight = 80
```

Conversion Utility Path

The following property provides a path to the ImageMagick (convert), GhostScript (ghostscript), and ffmpeg utilities.

```
org.dspace.app.mediafilter.ImageMagickThumbnailFilter.ProcessStarter = /usr/bin
```

Supported file formats

Each of these ImageMagick filters has its own list of configurable file formats. The defaults are usually best, but may be updated if you have custom bitstream formats. These settings already exist in your dspace.cfg.

```
filter.org.dspace.app.mediafilter.ImageMagickImageThumbnailFilter.inputFormats = BMP, GIF, PNG, JPG, TIFF,
JPEG, JPEG 2000
filter.org.dspace.app.mediafilter.ImageMagickPdfThumbnailFilter.inputFormats = Adobe PDF
filter.org.dspace.app.mediafilter.ImageMagickVideoThumbnailFilter.inputFormats = Video MP4
```

Overwriting Existing Thumbnails

The ImageMagick media filters can differentiate thumbnails created by the DSpace default thumbnail generator and thumbnails that were manually uploaded by a user. The media filter reads the bitstream description field to make this determination. A regular expression can be provided to define the set of thumbnails that should be overwritten by the ImageMagick thumbnail generator. Thumbnail descriptions matching this pattern will be overwritten even if the -f option is not passed to the filter media process.

```
org.dspace.app.mediafilter.ImageMagickThumbnailFilter.replaceRegex = ^Generated Thumbnail$
```

The ImageMagick media filter will use the bitstream description field to identify bitstreams that it has created using the following setting. Bitstreams containing this label will be overwritten only if the -f filter is applied.

```
org.dspace.app.mediafilter.ImageMagickThumbnailFilter.bitstreamDescription = IM Thumbnail
```

Thumbnail descriptions that do not match either of the patterns listed above are presumed to be manually uploaded thumbnails. These thumbnails will not be replaced even if the -f option is passed to the filter media process.

Flatten

DSpace uses the JPEG format for thumbnails. While JPEG doesn't support transparency, PDF, PNG and other formats do. As those formats are used as outgoing material in DSpace, DSpace has to care about transparency during the generation of the thumbnails. In combinations of specific versions of ImageMagick and Ghostscript it may occur that completely transparent areas will become black. As a solution ImageMagick recommends to flatten images extracted from PDFs before they are stored as JPEG.

Since DSpace 5.2 the ImageMagick media filter flattens thumbnails extracted from PDFs. If you run into problems caused by flattening of the extracted images, you can switch the flattening off by setting the following property in **dspace.cfg** to false:

```
org.dspace.app.mediafilter.ImageMagickThumbnailFilter.flatten = false
```

ICC Profiles

PDFs optimized for physical printing often use the CMYK color space. On the web, however, the de facto color system is sRGB. By default, DSpace's ImageMagick-based thumbnailing system will create thumbnails that use the same color space as the source PDF. Most web browsers are not able to correctly display images that use the CMYK color space, which leads to images with visibly inaccurate colors.

If you are using Ghostscript version 9 or above, it is possible for DSpace to correctly convert images from CMYK to sRGB by providing it with appropriate ICC color profiles to use during thumbnail creation. Default ones are provided by most Ghostscript installations (version 9 or above). The following configuration options tell DSpace where those ICC profiles are located.

```
# org.dspace.app.mediafilter.ImageMagickThumbnailFilter.cmyk_profile = /usr/share/ghostscript/9.18/iccprofiles
/default_cmyk.icc
# org.dspace.app.mediafilter.ImageMagickThumbnailFilter.srgb_profile = /usr/share/ghostscript/9.18/iccprofiles
/default_rgb.icc
```

You may need to adjust those paths for your OS or the version of Ghostscript that you have.

Providing ICC profiles to ImageMagick is optional. If these configuration properties are unset, no profiles will be supplied to ImageMagick, and thumbnails produced from PDFs using the CMYK color space will also use CMYK. The transformation from CMYK to RGB is optional.

Override ImageMagick Default Density

It is possible to override ImageMagick's default density of 72 DPI when creating PDF thumbnails. This increases the quality of resulting thumbnails at the expense of slightly longer execution times and higher memory usage. Any integer over 72 will help, but recommend 144 for a "2x" supersample. See the following configuration option in `dspace.cfg` :

```
# org.dspace.app.mediafilter.ImageMagickThumbnailFilter.density = 144
```

The effect is most notable on PDFs with a lot of text, gradients, or curved lines. See the pull request implementing this feature for more information and comparisons.

Additional Customization

The ImageMagick conversion software provides a large number of conversion options. Subclasses of these media filters could be written to take advantage of the additional conversion properties available in the software.

Note: The PDF thumbnail generator is hard-coded to generate a thumbnail from the first page of the PDF.

Possible Errors / Issues

"convert.im6: not authorized" errors

On Ubuntu (possibly other OSes), you may see errors like these when attempting to generate PDF thumbnails:

```
ERROR filtering, skipping bitstream:
        Item Handle: 1234/5678
        Bundle Name: ORIGINAL
        File Size: 30406135
        Checksum: c1df4b3a4755e9bed956383b61fc5042 (MD5)
        Asset Store: 0
org.im4java.core.CommandException: org.im4java.core.CommandException: convert.im6: not authorized `/tmp
/impdfthumb6294641076817830415.pdf' @ error/constitute.c/ReadImage/454.

OR

org.im4java.core.CommandException: convert-im6.q16: attempt to perform an operation not allowed by the security
policy 'PDF' @ error/constitute.c/IsCoderAuthorized/421
```

These may be caused by a change in your ImageMagick policy configuration on your server.

In Ubuntu, the default "policy.xml" was recently updated to **exclude** all Ghostscript formats (including PDF, PS, etc). See this ticket: https://bugs.launchpad.net/ubuntu/+source/imagemagick/+bug/1796563

- This exclusion was implemented to workaround a security vulnerability in Ghostscript reported here: https://www.kb.cert.org/vuls/id/332928
- According to that vulnerability report, this was patched in Ghostscript v9.24 (or above)

The newly added lines in the `/etc/ImageMagick/policy.xml` are these ones:

```
<!-- disable ghostscript format types -->
<policy domain="coder" rights="none" pattern="PS" />
<policy domain="coder" rights="none" pattern="EPS" />
<policy domain="coder" rights="none" pattern="PDF" />
<policy domain="coder" rights="none" pattern="XPS" />
```

To fix the error above requires you to re-enable ImageMagick to process Ghostscript format types. That can be done by simply commenting out those new "policy" lines in the configuration file (surround them with <!-- and --> to comment out)

Be aware that you MUST ensure you are running Ghostscript v9.24 or later to ensure that you are not at risk for the above security vulnerability in older versions of Ghostscript.

"convert-im6.q16: cache resources exhausted" errors

On Debian, and possibly other OSes, you may see errors like these when attempting to generate video thumbnails (especially if video files are big):

```
File: video.mp4.jpg
ERROR filtering, skipping bitstream:
        Item Handle: 1234/5678
        Bundle Name: ORIGINAL
        File Size: 146761357
        Checksum: 735ceb1b6b249afc84a5bb1b87ae0c02 (MD5)
        Asset Store: 0
org.im4java.core.CommandException: convert-im6.q16: cache resources exhausted `/tmp/magick-64dziU-
1nQJjQHZYu4_R1fFP4l9en5iL.pam' @ error/cache.c/OpenPixelCache/4095.
```

These may be caused by too conservative resource policies in your `policy.xml` file. As an example, default values are located at `/etc/ImageMagick-6/policiy.xml` in Debian 11 (Bullseye), and are:

/etc/ImageMagick-6/policiy.xml

```
<policymap>
  <!-- <policy domain="resource" name="temporary-path" value="/tmp"/> -->
  <policy domain="resource" name="memory" value="256MiB"/>
  <policy domain="resource" name="map" value="512MiB"/>
  <policy domain="resource" name="width" value="16KP"/>
  <policy domain="resource" name="height" value="16KP"/>
  <!-- <policy domain="resource" name="list-length" value="128"/> -->
  <policy domain="resource" name="area" value="128MP"/>
  <policy domain="resource" name="disk" value="1GiB"/>
```

To avoid the `cache resources exhausted` error, try increasing the resource limits policies. You may want to start by increasing the `memory` and `disk` policies (disk cache is used when the memory limit is reached). The actual values have to be adjusted depending on the size of your video bitstreams and the actual resources available in your installation. For example:

/etc/ImageMagick-6/policiy.xml

```
<policymap>
  <!-- <policy domain="resource" name="temporary-path" value="/tmp"/> -->
  <policy domain="resource" name="memory" value="4GiB"/> <!-- memory limit increased from 256MiB to 4GiB -->
  <policy domain="resource" name="map" value="512MiB"/>
  <policy domain="resource" name="width" value="16KP"/>
  <policy domain="resource" name="height" value="16KP"/>
  <!-- <policy domain="resource" name="list-length" value="128"/> -->
  <policy domain="resource" name="area" value="128MP"/>
  <policy domain="resource" name="disk" value="4GiB"/> <!-- disk limit increased from 1GiB to 4GiB -->
```

For a detailed description of the ImageMagick limits, see https://imagemagick.org/script/command-line-options.php#limit.

Once the limits are properly set, a successful execution of the filter should show a message similar to:

```
File: video.mp4.jpg
FILTERED: bitstream 12345678-abcd-efgh-ijkl-1234567890ab (item: 1234/5678) and created 'video.mp4.jpg'
```

Performance Tuning DSpace

1 Bare Minimum Requirements
2 Blocking Aggressive Bots
3 Performance Tuning the Frontend (UI)
    3.1 Use "cluster mode" of PM2 to avoid Node.js using a single CPU
    3.2 Give Node.js more memory
    3.3 Limit which pages are processed via Server Side Rendering (SSR)
    3.4 Turn on (or increase) caching of Server-Side Rendered pages
4 Performance Tuning the Backend (REST API)
    4.1 Give Tomcat More Memory
        4.1.1 Give Tomcat More Java Heap Memory
        4.1.2 Give Tomcat More Java PermGen Memory
        4.1.3 Choosing the size of memory spaces allocated to DSpace Backend
    4.2 Give the Command Line Tools More Memory
        4.2.1 Give the Command Line Tools More Java Heap Memory
        4.2.2 Give the Command Line Tools More Java PermGen Space Memory
5 Give PostgreSQL Database More Memory
6 Performance Tuning Solr

The software DSpace relies on does not come out of the box optimized for large repositories. Here are some tips to make it all run faster.

Bare Minimum Requirements

As of this writing, DSpace 7 is likely to require 4GB of memory at a *bare minimum*. However, with that little memory, you may quickly hit memory issues with any significant user activity or bulk uploading. So, *we recommend running DSpace with at least 8-12GB* (or more for very large or very active sites).

This minimum would roughly include...

- 2GB of memory for the Frontend (UI) / Node.js. Highly active sites will need more.
- 1GB of memory for the Backend (REST API) / JVM / Tomcat. Highly active sites will need more.
- 512MB of memory for PostgreSQL database. Highly active sites will need more.
- 512MB of memory for Solr. Highly active sites may need more.
- Extra memory may be required for command line scripts (which get kicked off in a separate JVM)

Keep in mind, because the frontend & backend can be run on separate servers, you can split this memory across two (or more) servers. You can even choose to run PostgreSQL or Solr either alongside the backend or on their own dedicated server.

The DSpace frontend (UI) will often require several CPUs, especially if you wish to use "cluster mode" (see below) to better scale your application. A smaller application may be able to use 4-6 CPU cores, while highly active sites may require additional CPU power. CPU is most often necessary for the frontend's Angular Serve Side Rendering (again see "cluster mode" notes below) and for any batch processing / command line scripts on backend.

Blocking Aggressive Bots

DSpace itself doesn't have built-in tools to allow you to block various aggressive bots. However, with the growth of Artificial Intelligence, many DSpace sites have experienced aggressive harvesting of their metadata / files by various AI-related bots. These aggressive bots can cause performance problems with DSpace sites if not dealt with in some way.

If you are experiencing this sort of aggressive harvesting, we'd recommend looking at the following resources (none of these are managed by DSpace, but all are community efforts by various library or repository-based developers):

- Aggressive AI Harvesting of Digital Resources
- https://wiki.code4lib.org/Blocking_Bots and the #bots channel on the code4lib Slack team.

Performance Tuning the Frontend (UI)

Use "cluster mode" of PM2 to avoid Node.js using a single CPU

If you are using PM2 to run the User Interface, you may want to start it using PM2's "Cluster Mode". This allows Node.js applications to be scaled across multiple CPUs by using the Node.js cluster module. See the PM2 Cluster Mode documentation at https://pm2.keymetrics.io/docs/usage/cluster-mode/

There are two ways to enable cluster mode. Choose one.

1. First, is by adding the "exec_mode" and "instances" settings to your JSON configuration as follows. You also may want to set the "max_memory_restart" option to avoid PM2 using too much memory. These three settings are described in more detail below. NOTE: make sure to start (or restart) your site to enable these settings (e.g. `pm2 start dspace-ui.json`)

dspace-ui.json

```
{
    "apps": [
        {
           "name": "dspace-ui",
           "cwd": "/full/path/to/dspace-ui-deploy",
           "script": "dist/server/main.js",
           "instances": "max",
           "exec_mode": "cluster",
           "env": {
              "NODE_ENV": "production"
           },
           "max_memory_restart": "500M"
        }
    ]
}
```

a. Setting "exec_mode" to "cluster" will enable cluster mode,
b. The "instances" setting allows you to customize how many CPUs are available to PM2 ("max" = all CPUs. But you also can specify a number like "8" = 8 CPUs.)
c. The "max_memory_restart" setting is *optional* but tells PM2 how much memory to allow **per instance.** The example above has a maximum of 500MB. If the number of 'instances' is 8, that would mean PM2 could use up to 8 x 500MB = 4GB of memory. Therefore, you may wish to modify the values of "instances" and/or "max_memory_restart" to better control the memory available to PM2.

2. Alternatively, you can use command line flags to specify the same settings described above. The "-i" flag enables cluster mode and specifies the number of instances. The "--max-memory-restart" flag limits the memory per instance.

```
# Start the "dspace-ui" app. Cluster it across all available CPUs with a maximum memory of 500MB per CPU.
# This command is equivalent to the example cluster settings in the "dspace-ui.json" file above.
pm2 start dspace-ui.json -i max --max-memory-restart 500M
```

Give Node.js more memory

On machines with >2GB of memory available, Node will only use a maximum of 2GB of memory by default (see https://github.com/nodejs/node/issues/28202). This 2GB of memory should be enough to build & run the User Interface, but it's possible that highly active sites may require 4GB or more.

If you want to increase the memory available to Node.js, you can set the NODE_OPTIONS environment variable:

```
# Increase memory limit to 4GB (4096MB) by setting "max-old-space-size"
# in your NODE_OPTIONS environment variable
export NODE_OPTIONS=--max-old-space-size=4096
```

Limit which pages are processed via Server Side Rendering (SSR)

While enabling Server Side Rendering (SSR) is extremely important for Search Engine Optimization, it can also be very resource intensive for large pages or highly active sites. Server Side Rendering involves building the entire HTML for the page in Node.js (on your server) before sending the page back to the client/user. Most humans only encounter SSR briefly, when they initially visit your site. However, bots *may only interact* with SSR, especially if they are unable to process Javascript. This is true even for Google Scholar, whose bots will only use SSR generated pages to index your site.

In order to maximum the performance of SSR, by default, DSpace will minimize the pages and Angular components that are processed during server side rendering. You may wish to review the default settings to ensure they are appropriate for your site. See the Server Side Rendering (SSR) Settings

Turn on (or increase) caching of Server-Side Rendered pages

As of DSpace 7.5, we now provide basic, in-memory caching of server-side rendered (SSR) pages. Server-side rendering is used to pre-generate full HTML pages to pass back to users (primarily anonymous users and bots). This is necessary for Search Engine Optimization (SEO) as some web crawlers cannot use Javascript. It also can be used to immediately show the first HTML page to users while the Javascript app loads in the user's browser.

While server-side-rendering is highly recommended on all sites, it can result in Node.js having to pre-generate many HTML pages at once when a site has a large number of simultaneous users/bots. This may cause Node.js to spend a lot of time processing server-side-rendered content, slowing down the entire site.

Therefore, DSpace provides some basic caching of server-side rendered pages, which allows the same pre-generated HTML to be sent to many users /bots at once & decreases the frequency of server-side rendering.

These settings are documented at User Interface Configuration: Cache Settings - Server Side Rendering (SSR)

Performance Tuning the Backend (REST API)

Give Tomcat More Memory

Give Tomcat More Java Heap Memory

Java Heap Memory Recommendations

At the time of writing, DSpace recommends you should give Tomcat >= 512MB of Java Heap Memory to ensure optimal DSpace operation. Most larger sized or highly active DSpace installations however tend to allocate more like 1024MB (1GB) to 2048MB (2G) or more of Java Heap Memory.

Performance tuning in Java basically boils down to memory. If you are seeing "`java.lang.OutOfMemoryError: Java heap space`" errors, this is a sure sign that Tomcat isn't being provided with enough Heap Memory.

Tomcat is especially memory hungry, and will benefit from being given lots of RAM. To set the amount of memory available to Tomcat, use either the `JAVA_OPTS` or `CATALINA_OPTS` environment variable, e.g:

```
CATALINA_OPTS=-Xmx512m -Xms512m
```

OR

```
JAVA_OPTS=-Xmx512m -Xms512m
```

The above example sets the maximum Java Heap memory to 512MB.

Difference between JAVA_OPTS and CATALINA_OPTS

You can use either environment variable. `JAVA_OPTS` is also used by other Java programs (besides just Tomcat). `CATALINA_OPTS` is *only used* by Tomcat. So, if you only want to tweak the memory available to Tomcat, it is recommended that you use `CATALINA_OPTS`. If you set **both** `CATALINA_OPTS` and `JAVA_OPTS`, Tomcat will default to using the settings in `CATALINA_OPTS`.

If the machine is dedicated to DSpace a decent rule of thumb is to give tomcat half of the memory on your machine. **At a minimum, you should give Tomcat >= 512MB of memory for optimal DSpace operation.** (*NOTE: As your DSpace instance gets larger in size, you may need to increase this number to the several GB range.*) The latest guidance is to also set `-Xms` to the same value as `-Xmx` for server applications such as Tomcat.

Give Tomcat More Java PermGen Memory

Java PermGen Memory Recommendations

At the time of writing, DSpace recommends you should give Tomcat >= 128MB of PermGen Space to ensure optimal DSpace operation.

If you are seeing "`java.lang.OutOfMemoryError: PermGen space`" errors, this is a sure sign that Tomcat is running out PermGen Memory. (More info on PermGen Space: https://frankkieviet.blogspot.com/2006/10/classloader-leaks-dreaded-permgen-space.html)

To increase the amount of PermGen memory available to Tomcat (default=64MB), use either the `JAVA_OPTS` or `CATALINA_OPTS` environment variable, e.g:

```
CATALINA_OPTS=-XX:MaxPermSize=128m
```

OR

```
JAVA_OPTS=-XX:MaxPermSize=128m
```

The above example sets the maximum PermGen memory to 128MB.

Difference between JAVA_OPTS and CATALINA_OPTS

You can use either environment variable. `JAVA_OPTS` is also used by other Java programs (besides just Tomcat). `CATALINA_OPTS` is *only used* by Tomcat. So, if you only want to tweak the memory available to Tomcat, it is recommended that you use `CATALINA_OPTS`. If you set **both** `CATALINA_OPTS` and `JAVA_OPTS`, Tomcat will default to using the settings in `CATALINA_OPTS`.

Please note that you can obviously set **both** Tomcat's Heap space and PermGen Space together similar to:
`CATALINA_OPTS=-Xmx512m -Xms512m -XX:MaxPermSize=128m`

On an Ubuntu machine (10.04) at least, the file `/etc/default/tomcat6` appears to be the best place to put these environmental variables.

Choosing the size of memory spaces allocated to DSpace Backend

psi-probe is a webapp that can be deployed in DSpace and be used to watch memory usage of the other webapps deployed in the same instance of Tomcat (in our case, the DSpace server webapp).

1. Download the latest version of psi-probe from https://github.com/psi-probe/psi-probe
2. Unzip probe.war into [dspace]/webapps/

```
cd [dspace]/webapps/
unzip ~/probe-3.1.0.zip
unzip probe.war -d probe
```

3. Add a Context element in Tomcat's configuration, and make it privileged (so that it can monitor the other webapps):
   EITHER in `$CATALINA_HOME/conf/server.xml`

```
<Context docBase="[dspace]/webapps/probe" privileged="true" path="/probe" />
```

   OR in `$CATALINA_HOME/conf/Catalina/localhost/probe.xml`

```
<Context docBase="[dspace]/webapps/probe" privileged="true" />
```

4. Edit `$CATALINA_HOME/conf/tomcat-users.xml` to add a user for logging into psi-probe (see more in https://github.com/psi-probe/psi-probe/wiki/InstallationApacheTomcat)

```
<?xml version='1.0' encoding='utf-8'?>
<tomcat-users>
  <user username="admin" password="t0psecret" roles="manager" />
</tomcat-users>
```

5. Restart Tomcat
6. Open http://yourdspace.com:8080/probe/ (edit domain and port number as necessary) in your browser and use the username and password from tomcat-users.xml to log in.

In the "System Information" tab, go to the "Memory utilization" menu. Note how much memory Tomcat is using upon startup and use a slightly higher value than that for the `-Xms` parameter (initial Java heap size). Watch how big the various memory spaces get over time (hours or days), as you run various common DSpace tasks that put load on memory, including indexing, reindexing, importing items into the oai index etc. These maximum values will determine the `-Xmx` parameter (maximum Java heap size). Watching PS Perm Gen grow over time will let you choose the value for the `-XX:MaxPermSize` parameter.

Give the Command Line Tools More Memory

Give the Command Line Tools More Java Heap Memory

Similar to Tomcat, you may also need to give the DSpace Java-based command-line tools more Java Heap memory. If you are seeing "`java.lang.OutOfMemoryError: Java heap space`" errors, when running a command-line tool, this is a sure sign that it isn't being provided with enough Heap Memory.

By default, DSpace only provides 256MB of maximum heap memory to its command-line tools.

If you'd like to provide **more** memory to command-line tools, you can do so via the `JAVA_OPTS` environment variable (which is used by the `[dspace]/bin/dspace` script). Again, it's the same syntax as above:

```
JAVA_OPTS=-Xmx512m -Xms512m
```

This is especially useful for big batch jobs, which may require additional memory.

You can also edit the `[dspace]/bin/dspace` script and add the environmental variables to the script directly.

Give the Command Line Tools More Java PermGen Space Memory

Similar to Tomcat, you may also need to give the DSpace Java-based command-line tools more PermGen Space. If you are seeing "`java.lang.OutOfMemoryError: PermGen space`" errors, when running a command-line tool, this is a sure sign that it isn't being provided with enough PermGen Space.

By default, Java only provides 64MB of maximum PermGen space.

If you'd like to provide **more** PermGen Space to command-line tools, you can do so via the `JAVA_OPTS` environment variable (which is used by the `[dspace]/bin/dspace` script). Again, it's the same syntax as above:

```
JAVA_OPTS=-XX:MaxPermSize=128m
```

This is especially useful for big batch jobs, which may require additional memory.

Please note that you can obviously set **both** Java's Heap space and PermGen Space together similar to:
`JAVA_OPTS=-Xmx512m -Xms512m -XX:MaxPermSize=128m`

Give PostgreSQL Database More Memory

On many Linux distros PostgreSQL comes out of the box with an incredibly conservative configuration - it uses only 8Mb of memory! To put some more fire in its belly edit the `shared_buffers` parameter in `postgresql.conf`. The memory usage is 8KB multiplied by this value. The advice in the Postgres docs is not to increase it above 1/3 of the memory on your machine.

For More PostgreSQL Tips

For more hints/tips with PostgreSQL configurations and performance tuning, see also:

- PostgresPerformanceTuning
- PostgresqlConfiguration

Performance Tuning Solr

Solr has it's own detailed documentation with recommendations for "Taking Solr to Production". We recommend following the recommendations from Solr, especially related to "Ulimit settings" (for Unix-based systems) and "Avoiding Swapping" (for Unix-based systems). See the Solr documentation for more details.

Ping or Healthcheck endpoints for confirming DSpace services are functional

For some installations of DSpace, it might be helpful to have a URL you can configure as a healthcheck for some sort of monitoring system (Monit, Eye). Some installations use load balancers, and those load balancers need a URL to check to confirm the system is functioning correctly. Here are some suggestions for you to use.

Frontend

```
/home
```

Be sure to append that path to the main URL of your DSpace instance's frontend URL. For example: https://demo7.dspace.org/home

Backend

```
/server/api/core/collections
```

```
/server/api/core/sites
```

Be sure to append these paths to the main URL of your DSpace instance's backend URL. For example: https://api7.dspace.org/server/api/core/collections

Both of those endpoints will throw an error if Solr is down or similar, and both are anonymously available (no login required).

Another possibility is the healthcheck actuator:

```
/server/actuator/health
```

which returns a JSON document containing a `status` scalar:

```
{"status":"UP"}
```

Scheduled Tasks via Cron

Several DSpace features **require** that a script is run regularly (via cron, or similar). Some of these features include:

- the e-mail subscription feature that alerts users of new items being deposited;
- the 'media filter' tool, that generates thumbnails of images and extracts the full-text of documents for indexing;
- the (optional) 'checksum checker' that tests the bitstreams in your repository for corruption;
- and the (optional) registration of DOIs using DataCite as registration agency.

There are some optional periodic tasks as well:

- Updating the geolocation database used to enrich usage statistics. At this writing, the database publisher issues monthly updates.

These regularly scheduled tasks should be setup via either cron (for Linux/Mac OSX) or Windows Task Scheduler (for Windows).

Recommended Cron Settings

If you are on Linux or Mac OSX, **you should add these cron settings under the OS account which is running Tomcat (and owns the [dspace] installation directory)**. For example, login as that user and type the following to edit the user's crontab.

```
crontab -e
```

Avoid scheduling cron tasks at midnight

Generally, we recommend avoiding scheduling any DSpace cron tasks to run at exactly midnight. DSpace's log4j settings default to rolling over the log file on a daily basis. Therefore any cron tasks which trigger at midnight have a risk of causing logging issues, or having their logs split over two log files.

While every DSpace installation is unique, in order to get the most out of DSpace, we highly recommend enabling these basic cron settings (the settings are described in the comments):

```
## SAMPLE CRONTAB FOR A PRODUCTION DSPACE
## You obviously may wish to tweak this for your own installation,
## but this should give you an idea of what you likely wish to schedule via cron.
##
## NOTE: You may also need to add additional sysadmin related tasks to your crontab
## (e.g. zipping up old log files, or even removing old logs, etc).

#-----------------
# GLOBAL VARIABLES
#-----------------
# Full path of your local DSpace Installation (e.g. /home/dspace or /dspace or similar)
# MAKE SURE TO CHANGE THIS VALUE!!!
DSPACE = [dspace]

# Shell to use
SHELL=/bin/sh

# Add all major 'bin' directories to path
PATH=/usr/local/sbin:/usr/local/bin:/sbin:/bin:/usr/sbin:/usr/bin

# Set JAVA_OPTS with defaults for DSpace Cron Jobs.
# Only provides 512MB of memory by default (which should be enough for most sites).
JAVA_OPTS="-Xmx512M -Xms512M -Dfile.encoding=UTF-8"

#--------------
# HOURLY TASKS (Recommended to be run multiple times per day, if possible)
# At a minimum these tasks should be run daily.
#--------------

# Send information about new and changed DOIs to the DOI registration agency
# NOTE: ONLY NECESSARY IF YOU REGISTER DOIS USING DATACITE AS REGISTRATION AGENCY (Disabled by default)
# 0 4,12,20 * * *  $DSPACE/bin/dspace doi-organiser -u -q ; $DSPACE/bin/dspace doi-organiser -s -q ; $DSPACE/bin
/dspace doi-organiser -r -q ; $DSPACE/bin/dspace doi-organiser -d -q

#----------------
# DAILY TASKS
# (Recommended to be run once per day. Feel free to tweak the scheduled times below.)
#----------------
```

```
# Update the OAI-PMH index with the newest content at 12:15am every day
# REQUIRED to update content available in OAI-PMH (However, it can be removed if you do not enable OAI-PMH)
15 0 * * * $DSPACE/bin/dspace oai import > /dev/null

# Clean and Update the Discovery indexes at 12:15am every day
# (This ensures that any deleted documents are cleaned from the Discovery search/browse index)
# RECOMMENDED to ensure your search/browse index stays fresh.
15 0 * * * $DSPACE/bin/dspace index-discovery > /dev/null

# run the index-authority script once a day at 12:45am to ensure the Solr Authority cache is up to date
45 0 * * * $DSPACE/bin/dspace index-authority > /dev/null

# Cleanup Web Spiders from DSpace Statistics Solr Index at 01:00 every day
# (This removes any known web spiders from your usage statistics)
# RECOMMENDED if you are running Solr Statistics.
0 1 * * * $DSPACE/bin/dspace stats-util -i

# Send out "daily" update subscription e-mails at 02:00 every day
# (This sends an email to any users who have "subscribed" to a Community/Collection, notifying them of newly
added content.)
# (This job must run before the `filter-media` task, because `filter-media` may update the Solr index, which
could delay sending subscription emails about the new content until the following day.)
# REQUIRED for daily "Email Subscriptions" to work properly.
0 2 * * * $DSPACE/bin/dspace subscription-send -f D

# Run the media filter at 03:00 every day.
# (This task ensures that thumbnails are generated for newly add images,
# and ensures full text search is available for newly added PDF/Word/PPT/HTML documents)
# REQUIRED for Thumbnails to be generated & full-text indexing to work.
0 3 * * * $DSPACE/bin/dspace filter-media

#----------------
# WEEKLY TASKS
# (Recommended to be run once per week, but can be run more or less frequently, based on your local needs
/policies)
#----------------
# Send out "weekly" update subscription e-mails at 02:00 every Sunday
# (This sends an email to any users who have "subscribed" to a Community/Collection, notifying them of newly
added content.)
# REQUIRED for weekly "Email Subscriptions" to work properly.
0 2 * * 0 $DSPACE/bin/dspace subscription-send -f W

# Run the checksum checker at 04:00 every Sunday
# By default it runs through every file (-l) and also prunes old results (-p)
# (This re-verifies the checksums of all files stored in DSpace. If any files have been changed/corrupted,
checksums will differ.)
# OPTIONAL, but useful if you want to enable regular regular checksum validation of files stored in DSpace.
# 0 4 * * 0 $DSPACE/bin/dspace checker -l -p
# NOTE: LARGER SITES MAY WISH TO USE DIFFERENT OPTIONS. The above "-l" option tells DSpace to check
*everything*.
# If your site is very large, you may need to only check a portion of your content per week. The below
commented-out task
# would instead check all the content it can within *one hour*. The next week it would start again where it
left off.
# 0 4 * * 0 $DSPACE/bin/dspace checker -d 1h -p
# Mail the results of the checksum checker (see above) to the configured "mail.admin" at 05:00 every Sunday.
# (This ensures the system administrator is notified whether any checksums were found to be different.)
# 0 5 * * 0 $DSPACE/bin/dspace checker-emailer

#----------------
# MONTHLY TASKS
# (Recommended to be run once per month, but can be run more or less frequently, based on your local needs
/policies)
#----------------
# Send out "monthly" update subscription e-mails at 02:00, on the first of every month
# (This sends an email to any users who have "subscribed" to a Community/Collection, notifying them of newly
added content.)
# REQUIRED for monthly "Email Subscriptions" to work properly.
0 2 1 * * $DSPACE/bin/dspace subscription-send -f M
```

```
# Permanently delete any bitstreams flagged as "deleted" in DSpace, on the first of every month at 01:00
# (This ensures that any files which were deleted from DSpace are actually removed from your local filesystem.
#  By default they are just marked as deleted, but are not removed from the filesystem.)
# REQUIRED to fully remove deleted content files from the "assetstore" folder
0 1 1 * * $DSPACE/bin/dspace cleanup > /dev/null
```

Search Engine Optimization

Please be aware that individual search engines also have their own guidelines and recommendations for inclusion. While the guidelines below apply to **most** DSpace sites, you may also wish to review these guidelines for specific search engines:

- "Indexing Repositories: Pitfalls and Best Practices" talk from Anurag Acharya (co-creator of Google Scholar) presented at the Open Repositories 2015 conference
- Google Scholar Inclusion Guidelines
- Bing Webmaster Guidelines

Ensuring your DSpace is indexed

Anyone who has analyzed traffic to their DSpace site (e.g. using Google Analytics or similar) will notice that a significant (and in many cases a majority) of visitors arrive via a search engine such as Google or Yahoo. Hence, to help maximize the impact of content and thus encourage further deposits, it is important to ensure that your DSpace instance is indexed effectively.

DSpace comes with tools that ensure major search engines (Google, Bing, Yahoo, Google Scholar) are able to easily and effectively index all your content. However, many of these tools provide some basic setup. Here's how to ensure your site is indexed.

For the optimum indexing, you should:

1. Check SEO Validator status to detect any obvious issues *(This tool is only available in 7.6.4 and later)*
2. Keep your DSpace up to date. We are constantly adding new indexing improvements in new releases
3. Ensure your DSpace is visible to search engines.
4. Ensure your proxy is passing X-Forwarded headers to the User Interface
5. Ensure the user interface is using server-side rendering *(enabled by default)*
6. Ensure the sitemaps feature is enabled. *(enabled by default)*
7. Ensure your robots.txt allows access to item "splash" pages and full text.
8. Ensure item metadata appears in HTML headers correctly.
9. Avoid redirecting file downloads to Item landing pages
10. Turn OFF any generation of PDF cover pages
11. As an aside, it's worth noting that OAI-PMH is generally not useful to search engines. OAI-PMH has its own uses, but do not expect search engines to use it.

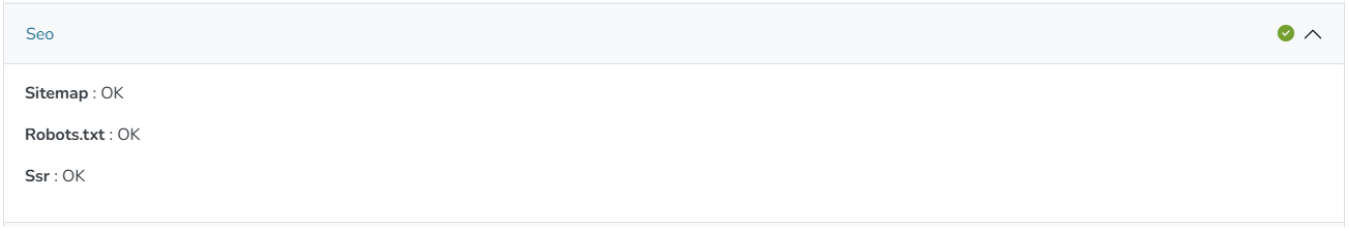
Check SEO Validator status

DSpace now has a basic Search Engine Optimization (SEO) validator which can provide you feedback on how well your site may align with the *some* of these Search Engine Optimization policies.

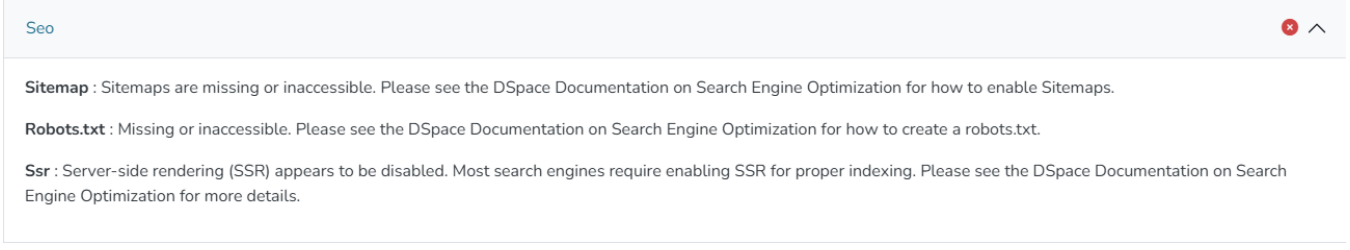
At this time, **this validation tool can only check three things:**

- Ensure the user interface is using server-side rendering
- Ensure the sitemaps feature is enabled and appear to be configured correctly
- Ensure your robots.txt exists and links correctly to the sitemaps (may also detect basic issues with a proxy not passing X-Forwarded headers)

This validation tool can be found in the Admin User Interface on the "Health" page. Look for the section named "SEO". If everything looks good, you'll see a green checkbox similar to this:



If there are issues detected, you'll see a red warning with details on what needs to be addressed.



If issues are detected, you should use the documentation on this wiki page to address the detected issues.

Even if you see a green checkmark on this page, you should **still** review all the Search Engine Optimization guidelines on this page. As noted above, **this validator cannot detect all possible SEO issues,** so manual verification is still required.

Keep your DSpace up to date

We are constantly adding new indexing improvements to DSpace. In order to ensure your site gets all of these improvements, you should strive to keep it up-to-date. For example:

- As of DSpace 7.0, Sitemaps are enabled by default (see below)
- As of DSpace 5.0, the DSpace robots.txt file now includes references to Sitemaps by default (see https://github.com/DSpace/DSpace/issues/5302) , and also blocks known bad bots (see https://github.com/DSpace/DSpace/issues/5701).
- As of DSpace 4.0, DSpace has provided several enhancements, which were requested by the Google Scholar team. These included providing users (and web indexers) a way to browse content by the date it was added to DSpace (see https://github.com/DSpace/DSpace/issues/4851), ensuring the "dc.date.issued" field is set more accurately (see https://github.com/DSpace/DSpace/issues/4850), and enhancing the logic behind the "citation_pdf_url" HTML <meta> tag (see https://github.com/DSpace/DSpace/issues/4852)
- As of DSpace 1.7, DSpace has improved how its Item-level metadata is made available to Google Scholar. For the 1.7.0 release, the DSpace Developers worked directly with the Google Scholar developers, to ensure DSpace is generating the "citation_*" HTML "<meta>" tags (i.e. Highwire Press tags) that Google Scholar recommends in their Indexing Guidelines.
- As of DSpace 1.5, DSpace has support for sitemaps (both simple HTML pages of links, as well as the sitemaps.org protocol). It also includes item metadata in the HTML HEAD element of item display pages, ensuring that the metadata can be effectively indexed no matter what changes you might have made to your DSpace's layout or style.
- As of DSpace 1.4, DSpace has support for the "if-modified-since" HTTP header. This basically means that if an item (or bitstream therein) has not changed since the last time a search engine's crawler indexed it, that item/bitstream does not have to be re-retrieved, sparing your server.

Additional minor improvements / bug fixes have been made to more recent releases of DSpace.

Ensure your DSpace is visible to search engines

First ensure your DSpace instance is visible, e.g. with: https://www.google.com/webmasters/tools/sitestatus

If your site is not indexed at all, all search engines have a way to add your URL, e.g.:

- Google: http://www.google.com/addurl
- Google Scholar: https://scholar.google.com/intl/en/scholar/inclusion.html
- Bing: https://www.bing.com/webmasters/about

Ensure your proxy is passing X-Forwarded headers to the User Interface

Some HTML tags important for SEO, such as the "citation_pdf_url" tag, require the full URL of your site. The DSpace user interface will automatically attempt to "discover" that URL using HTTP Headers.

Because most DSpace sites use some sort of proxy (e.g. Apache web server or Nginx or similar), this **requires** that the proxy be configured to pass along proper X-Forwarded-* headers, especially X-Forwarded-Host and X-Forwarded-Proto. For example in Apache HTTPD, you can do something like this:

```
# This lets DSpace know it is running behind HTTPS and what hostname is currently used
# (requires installing/enabling mod_headers)
RequestHeader set X-Forwarded-Proto https
RequestHeader set X-Forwarded-Host my.dspace.edu
```

Ensure the user interface is using server-side rendering

In DSpace 7, server-side rendering is *enabled by default (when running in production mode)*. However, it's important to ensure you do *not* disable it in production mode, as most search engine bots cannot index your site if SSR is disabled. Per the frontend Installation instructions, you MUST also be running your user interface in production mode (via either `yarn run serve:ssr` or `yarn start`).

Because the DSpace user interface is based on Angular.io (which is a javascript framework), you MUST have server-side rendering enabled (which is the default) for search engines to fully index your side. Server-side rendering allows your site to still function even when Javascript is turned *off* in a user's browser. Some web crawlers do not support Javascript (e.g. Google Scholar), so they will only interact with this server-side rendered content.

If you are unsure if server-side rendering (SSR) is enabled, you can check to see if your site is accessible when Javascript is turned **off.** For example, in Chrome, you should be able to do the following:

1. Open your site in the Chrome browser
2. Turn off (disable) Javascript using the Chrome instructions: https://developer.chrome.com/docs/devtools/javascript/disable/
3. Click reload in your browser window to reload your site.
    a. If SSR is enabled, then you will still see your site's contents. You should be able to browse & search the site. (Keep in mind, pages may take longer to load because every request requires SSR.) However, all dynamic menus or actions obviously will not work, as all pages will be static HTML.
    b. If SSR is disabled, then you will see a blank white page. You will not be able to see any content on your site.
4. Don't forget to re-enable Javascript after you are done testing (see link above, or just close that window & reopen a new one)

DSpace use Angular Universal for server-side rendering, and it's enabled by default in Production mode via our production environment initialization in `src/environments/environment.production.ts`:

```
// Angular Universal Settings
universal: {
  preboot: true,
  ...
},
```

SSR Performance Tuning

While enabling SSR is extremely important for Search Engine Optimization, it can also be very resource intensive for large pages or highly active sites. Server Side Rendering involves building the entire HTML for the page in Node.js (on your server) before sending the page back to the client /user. Most humans only encounter SSR briefly, when they initially visit your site. However, bots *may only interact* with SSR, especially if they are unable to process Javascript. This is true even for Google Scholar, whose bots will only use SSR generated pages to index your site.

In order to maximum the performance of SSR, by default, DSpace will minimize the pages and Angular components that are processed during server side rendering. You may wish to review the default settings to ensure they are appropriate for your site. See the "Server Side Rendering (SSR) settings" in User Interface Configuration

Ensure the sitemaps feature is enabled

As of DSpace 7, sitemaps are *enabled by default and automatically update on a daily basis.* This is the recommended setup to prefer proper indexing. So, there's nothing you need to do unless you wish to either change their schedule, or disable them.

In the dspace.cfg, the Sitemap generation schedule is controlled by this setting

```
# By default, sitemaps regenerate daily at 1:15am server time
sitemap.cron = 0 15 1 * * ?
```

You can modify this schedule by using the Cron syntax defined at https://www.quartz-scheduler.org/api/2.3.0/org/quartz/CronTrigger.html . Any modifications can be placed in your local.cfg.

If you want to disable this automated scheduler, you can either comment it out, or set it to a single "-" (dash) in your local.cfg

```
# This disables the automatic updates
sitemap.cron = -
```

Again, we **highly recommend** keeping them enabled. However, you may choose to disable this scheduler if you wish to define these in your local system cron settings.

Once you've enabled your sitemaps, they will be accessible at the following URLs:

- HTML Sitemaps: ${dspace.ui.url}/sitemap_index.html
- XML Sitemaps: ${dspace.ui.url}/sitemap_index.xml

So, for example, if your "dspace.ui.url = https://mysite.org" in your "dspace.cfg" configuration file, then the HTML Sitemaps would be at: "http://mysite.org/sitemap_index.html"

By default, the Sitemap URLs also will appear in your UI's `robots.txt` (in order to announce them to search engines):

```
# The URL to the DSpace sitemaps
# XML sitemap is listed first as it is preferred by most search engines
Sitemap: [dspace.ui.url]/sitemap_index.xml
Sitemap: [dspace.ui.url]/sitemap_index.html
```

The generate-sitemaps command

If you wanted to generate your sitemaps manually, you can use a commandline tool to do so.

WARNING: Keep in mind, you do NOT need to run these manually in most situations, as sitemaps are autoupdated on a regular schedule (see documentation above)

```
# Commandline option (run from the backend)
[dspace]/bin/dspace generate-sitemaps
```

This command accepts several options:

Option	meaning
-h --help	Explain the arguments and options.
-s --no_sitemaps	Do not generate a sitemap in sitemaps.org format.
-b -no_htmlmap	Do not generate a sitemap in htmlmap format.

You can configure the list of "all search engines" by setting the value of `sitemap.engineurls` in `dspace.cfg`.

Create a good robots.txt

As of 7.5, DSpace's robots.txt file can be found in the UI's codebase at "src/robots.txt.ejs". This is an "embedded javascript template" (ejs) file, which simply allows for us to insert variable values into the "robots.txt" at runtime. It can be edited as a normal text file.

The trick here is to minimize load on your server, but without actually blocking anything vital for indexing. Search engines need to be able to index item, collection and community pages, and all bitstreams within items – full-text access is critically important for effective indexing, e.g. for citation analysis as well as the usual keyword searching.

If you have restricted content on your site, search engines will not be able to access it; they access all pages as an anonymous user.

Ensure that your robots.txt file is at the top level of your site: i.e. at http://repo.foo.edu/robots.txt, and NOT e.g. http://repo.foo.edu/dspace/robots.txt. If your DSpace instance is served from e.g. http://repo.foo.edu/dspace/, you'll need to add /dspace to all the paths in the examples below (e.g. /dspace/browse-subject).

NEVER BLOCK THESE PATHS

Some URLs can be disallowed without negative impact, but be ABSOLUTELY SURE the following URLs can be reached by crawlers, i.e. DO NOT put these on Disallow: lines, or your DSpace instance might not be indexed properly.

- `/bitstreams`
- `/browse/*` (UNLESS USING SITEMAPS)
- `/collections`
- `/communities`
- `/community-list` (UNLESS USING SITEMAPS)
- `/entities/*`
- `/handle`
- `/items`

Example good robots.txt

DSpace 7 comes with an example robots.txt file (which is copied below). As of 7.5, this file can be found at "src/robots.txt.ejs" in the DSpace 7 UI. This is an "embedded javascript template" (ejs) file, which simply allows for us to insert variable values into the "robots.txt" at runtime. It can be edited as a normal text file.

The highly recommended settings are uncommented. Additional, optional settings are displayed in comments – based on your local configuration you may wish to enable them by uncommenting the corresponding "Disallow:" line.

```
# The URL to the DSpace sitemaps
# XML sitemap is listed first as it is preferred by most search engines
# NOTE: The <%= origin %> variables below will be replaced by the fully qualified URL of your site at runtime.
Sitemap: <%= origin %>/sitemap_index.xml
Sitemap: <%= origin %>/sitemap_index.html

##########################
# Default Access Group
# (NOTE: blank lines are not allowable in a group record)
##########################
User-agent: *
# Disable access to Discovery search and filters; admin pages; processes; submission; workspace; workflow &
profile page
Disallow: /search
Disallow: /admin/*
Disallow: /processes
Disallow: /submit
Disallow: /workspaceitems
Disallow: /profile
Disallow: /workflowitems

# Optionally uncomment the following line ONLY if sitemaps are working
# and you have verified that your site is being indexed correctly.
# Disallow: /browse/*
#
# If you have configured DSpace (Solr-based) Statistics to be publicly
# accessible, then you may not want this content to be indexed
# Disallow: /statistics
#
# You also may wish to disallow access to the following paths, in order
# to stop web spiders from accessing user-based content
# Disallow: /contact
# Disallow: /feedback
# Disallow: /forgot
# Disallow: /login
# Disallow: /register

# NOTE: The default robots.txt also includes a large number of recommended settings to avoid misbehaving bots.
# For brevity, they have been removed from this example, but can be found in src/robots.txt.ejs
```

WARNING: for your additional disallow statements to be recognized under the `User-agent: *` group, they *cannot be separated by white lines* from the declared `user-agent: *` block. A white line indicates the start of a new user agent block. Without a leading user-agent declaration on the first line, blocks are ignored. Comment lines are allowed and will not break the user-agent block.

This is OK:

```
User-agent: *
# Disable access to Discovery search and filters; admin pages; processes
Disallow: /search
Disallow: /admin/*
Disallow: /processes
```

This is **not OK**, as the two lines at the bottom will be completely ignored.

```
User-agent: *
# Disable access to Discovery search and filters; admin pages; processes
Disallow: /search

Disallow: /admin/*
Disallow: /processes
```

To identify if a specific user agent has access to a particular URL, you can use this handy robots.txt tester.

For more information on the robots.txt format, please see the Google Robots.txt documentation.

Ensure Item Metadata appears in the HTML HEAD

Google Scholar Metadata in HTML HEAD

In addition to Dublin Core <meta> tags in the HTML HEAD, DSpace also includes Google Scholar specific metadata fields in each item's HTML display page.

```
<meta property="citation_author" content="Tansley, Robert; Donohue, Timothy"/>
<meta property="citation_title" content="Ensuring your DSpace is indexed" />
```

These meta tags are the "Highwire Press tags" which Google Scholar recommends. If you have heavily customized your metadata fields, or wish to change the default "mappings" to these Highwire Press tags, you may do so by modifying https://github.com/DSpace/dspace-angular/blob/dspace-7_x/src/app/core/metadata/metadata.service.ts (see for example the "setCitationAuthorTags()" method in that service class)

Much more information is available in the Configuration section on Google Scholar Metadata Mappings.

Avoid redirecting file downloads to Item landing pages

Make sure that you never redirect "direct file downloads" (i.e. users who directly jump to downloading a file, often from a search engine) to the associated Item's splash/landing page. In the past, some DSpace sites have added these custom URL redirects in order to facilitate capturing statistics via Google Analytics or similar.

While these URL redirects may seem harmless, they may be flagged as cloaking or spam by Google, Google Scholar and other major search engines. This may hurt your site's search engine ranking or even cause your entire site to be flagged for removal from the search engine.

If you have these URL redirects in place, it is highly recommended to remove them immediately. If you created these redirects to facilitate capturing download statistics in Google Analytics, you should consider upgrading to DSpace 5.0 or above, which is able to automatically record bitstream downloads in Google Analytics (see https://github.com/DSpace/DSpace/issues/5454) without the need for any URL redirects.

Turn OFF any generation of PDF cover pages

While DSpace offers a PDF Citation Cover Page option, this option may affect your content's visibility in search engines like Google Scholar. Google Scholar (and possibly other search engines) specifically extracts metadata by analyzing the contents of the first page of a PDF. Dynamically inserting a custom cover page can break the metadata extraction techniques of Google Scholar and may result in all or much of your site being dropped from the Google Scholar search engine.

For more information, please see the "Indexing Repositories: Pitfalls and Best Practices" talk from Anurag Acharya (co-creator of Google Scholar) presented at the Open Repositories 2015 conference.

In general, OAI-PMH is not useful to Search Engines

Feel free to support OAI-PMH, but be aware that in general it is not useful for search engines:

- No reliable way to determine OAI-PMH base URL for a DSpace site.
- No standard or predictable way to get to item display page or full text from an OAI-PMH record, making effective indexing and presenting meaningful results difficult.
- In most cases provides only access to simple Dublin Core, a subset of available metadata.
- **NOTE:** Back in 2008, Google officially announced they were retiring support for OAI-PMH based Sitemaps. So, OAI-PMH will no longer help you get better indexing through Google. Instead, you should be using the DSpace 'generate-sitemaps' feature described above.

T

Google Scholar Metadata Mappings

While DSpace 7.0 supports Google Scholar `meta` tags, they are no longer configurable & are currently hardcoded into the User Interface codebase. Configurability may be coming back in a later 7.x release (based on user feedback), see https://github.com/DSpace/dspace-angular/issues/1198

Google Scholar, in crawling sites, prefers Highwire Press tags. This schema contains names which are all prefixed by the string "citation_", and provide various metadata about the article/item being indexed.

In DSpace, there is a mapping facility to connect metadata fields with these citation fields in HTML. In order to enable this functionality, the switch needs to be flipped in dspace.cfg:

```
google-metadata.enable = true
```

Once the feature is enabled, the mapping is configured by a separate configuration file located here:

```
[dspace]/config/crosswalks/google-metadata.properties
```

This file contains name/value pairs linking meta-tags with DSpace metadata fields. E.g…

```
google.citation_title = dc.title
google.citation_publisher = dc.publisher
google.citation_author = dc.author | dc.contributor.author | dc.creator
```

There is further documentation in this configuration file explaining proper syntax in specifying which metadata fields to use. If a value is omitted for a meta-tag field, the meta-tag is simply not included in the HTML output.

The values for each item are interpolated when the item is viewed, and the appropriate meta-tags are included in the HTML head tag, on both the Brief Item Display and the Full Item Display in the UI.

Note: In DSpace 5, the field google.citation_authors was changed to google.citation_author.

Troubleshooting Information

You can quickly get some basic information about the DSpace version and the products supporting it by using the `[dspace]/bin/dspace version` command.

```
$ bin/dspace version
DSpace version:  4.0-SNAPSHOT
  SCM revision:  da53991b6b7e9f86c2a7f5292e3c2e9606f9f44c
    SCM branch:  UNKNOWN
            OS:  Linux(amd64) version 3.7.10-gentoo
Discovery enabled.
Lucene search enabled.
           JRE:  Oracle Corporation version 1.7.0_21
   Ant version:  Apache Ant(TM) version 1.8.4 compiled on June 25 2012
 Maven version:  3.0.4
   DSpace home:  /home/dspace
$
```

To troubleshoot a specific error, see our Troubleshoot an error guide

Validating CheckSums of Bitstreams

1 Checksum Checker
- 1.1 Checker Execution Mode
- 1.2 Checker Results Pruning
- 1.3 Checker Reporting
- 1.4 Cron or Automatic Execution of Checksum Checker
- 1.5 Automated Checksum Checkers' Results
- 1.6 Database Query

Checksum Checker

As of DSpace 7.6.6, the "-a" argument was renamed to "-i". However, the longer form is still "--handle". See the full list of arguments below.

Checksum Checker is program that can run to verify the checksum of every item within DSpace. Checksum Checker was designed with the idea that most System Administrators will run it from the cron. Depending on the size of the repository choose the options wisely.

Command used:	`[dspace]/bin/dspace checker`
Java class:	org.dspace.app.checker.ChecksumChecker
Arguments short and (long) forms):	Description
`-L` or `--continuous`	Loop continuously through the bitstreams
`-i` or `--handle`	Specify a handle to check
`-b <bitstream-ids>`	Space separated list of bitstream IDs
`-c` or `--count`	Check count
`-d` or `--duration`	Checking duration
`-h` or `--help`	Calls online help
`-l` or `--looping`	Loop once through bitstreams
`-p <prune>`	Prune old results (optionally using specified properties file for configuration
`-v` or `--verbose`	Report all processing

There are three aspects of the Checksum Checker's operation that can be configured:

- the execution mode
- the logging output
- the policy for removing old checksum results from the database
  The user should refer to Chapter 5. Configuration for specific configuration beys in the *dspace.cfg* file.

Checker Execution Mode

Execution mode can be configured using command line options. Information on the options are found in the previous table above. The different modes are described below.

Unless a particular bitstream or handle is specified, the Checksum Checker will always check bitstreams in order of the least recently checked bitstream. (Note that this means that the most recently ingested bitstreams will be the last ones checked by the Checksum Checker.)

Available command line options

- **Limited-count mode:** `[dspace]/bin/dspace checker -c` To check a specific number of bitstreams. The *-c* option if followed by an integer, the number of bitstreams to check. Example: `[dspace/bin/dspace checker -c 10` This is particularly useful for checking that the checker is executing properly. The Checksum Checker's default execution mode is to check a single bitstream, as if the option was *-c 1*
- **Duration mode:** `[dspace]/bin/dspace checker -d` To run the Check for a specific period of time with a time argument. You may use any of the time arguments below: Example: `[dspace/bin/dspace checker -d 2h`(Checker will run for 2 hours)

s	Seconds
m	Minutes
h	Hours
d	Days
w	Weeks
y	Years

The checker will keep starting new bitstream checks for the specific durations, so actual execution duration will be slightly longer than the specified duration. Bear this in mind when scheduling checks.

- **Specific Bitstream mode:** `[dspace]/bin/dspace checker -b` Checker will only look at the internal bitstream IDs. Example: `[dspace]/bin/dspace checker -b 112 113 4567` Checker will only check bitstream IDs 112, 113 and 4567.
- **Specific Handle mode:** `[dspace]/bin/dspace checker -i` Checker will only check bitstreams within the Community, Community or the item itself. Example: `[dspace]/bin/dspace checker -i 123456/999` Checker will only check this handle. If it is a Collection or Community, it will run through the entire Collection or Community.
- **Looping mode:** `[dspace]/bin/dspace checker -l` or `[dspace]/bin/dspace checker -L` There are two modes. The lowercase 'el' (-l) specifies to check every bitstream in the repository once. This is recommended for smaller repositories who are able to loop through all their content in just a few hours maximum. An uppercase 'L' (-L) specifies to continuously loops through the repository. This is not recommended for most repository systems. **Cron Jobs**. For large repositories that cannot be completely checked in a couple of hours, we recommend the -d option in cron.
- **Pruning mode:** `[dspace]/bin/dspace checker -p` The Checksum Checker will store the result of every check in the checksum_history table. By default, successful checksum matches that are eight weeks old or older will be deleted when the -p option is used. (Unsuccessful ones will be retained indefinitely). Without this option, the retention settings are ignored and the database table may grow rather large!

Note that options `-c`, `-d`, `-b`, `-i`, `-l`, and `-L` are mutually exclusive. For example, it is not possible limit the running time or bitstream count of looping modes.

Checker Results Pruning

As stated above in "Pruning mode", the checksum_history table can get rather large, and that running the checker with the -p assists in the size of the checksum_history being kept manageable. The amount of time for which results are retained in the checksum_history table can be modified by one of two methods:

1. Editing the retention policies in `[dspace]/config/dspace.cfg` See Chapter 5 Configuration for the property keys. OR
2. Pass in a properties file containing retention policies when using the -p option.To do this, create a file with the following two property keys:

```
checker.retention.default = 10y
checker.retention.CHECKSUM_MATCH = 8w
```

You can use the table above for your time units. At the command line: `[dspace]/bin/dspace checker -p retention_file_name <ENTER>`

Checker Reporting

Checksum Checker uses log4j to report its results. By default it will report to a log called `[dspace]/log/checker.log`, and it will report only on bitstreams for which the newly calculated checksum does not match the stored checksum. To report on all bitstreams checked regardless of outcome, use the *-v* (verbose) command line option:

`[dspace]/bin/dspace checker -l -v` (This will loop through the repository once and report in detail about every bitstream checked.

To change the location of the log, or to modify the prefix used on each line of output, edit the `[dspace]/config/templates/log4j.properties` file and run `[dspace]/bin/install_configs`.

Cron or Automatic Execution of Checksum Checker

You should schedule the Checksum Checker to run automatically, based on how frequently you backup your DSpace instance (and how long you keep those backups). The size of your repository is also a factor. For very large repositories, you may need to schedule it to run for an hour (e.g. `-d 1h` option) each evening to ensure it makes it through your entire repository within a week or so. Smaller repositories can likely get by with just running it weekly.

Unix, Linux, or MAC OS. You can schedule it by adding a cron entry similar to the following to the crontab for the user who installed DSpace:

```
0 4 * * 0 [dspace]/bin/dspace checker -d2h -p
```

The above cron entry would schedule the checker to run the checker every Sunday at 400 (4:00 a.m.) for 2 hours. It also specifies to 'prune' the database based on the retention settings in *dspace.cfg*.

Windows OS. You will be unable to use the checker shell script. Instead, you should use Windows Schedule Tasks to schedule the following command to run at the appropriate times:

```
[dspace]/bin/dspace checker -d2h -p
```

(This command should appear on a single line).

Automated Checksum Checkers' Results

Optionally, you may choose to receive automated emails listing the Checksum Checkers' results to the email address specified in the `mail.admin` configu ration property. Schedule it to run **after** the Checksum Checker has completed its processing (otherwise the email may not contain all the results). As of DSpace 4.1, an email is only generated if the selected report contains at least one bitstream needing attention.

Command used:	`[dspace]/bin/dspace checker-emailer`
Java class:	org.dspace.checker.DailyReportEmailer
Arguments short and (long) forms):	Description
`-a` or `--all`	Send all the results (everything specified below)
`-d` or `--deleted`	Send E-mail report for all bitstreams set as deleted for today.
`-m` or `--missing`	Send E-mail report for all bitstreams not found in assetstore for today.
`-c` or `--changed`	Send E-mail report for all bitstreams where checksum has been changed for today.
`-u` or `--unchecked`	Send the Unchecked bitstream report.
`-n` or `--not-processed`	Send E-mail report for all bitstreams set to longer be processed for today.
`-h` or `--help`	Help

You can also combine options (e.g. `-m -c`) for combined reports.

Cron. Follow the same steps above as you would running checker in cron. Change the time but match the regularity. Remember to schedule this **after** Checksum Checker has run. For an example cron setup, see Scheduled Tasks via Cron.

Database Query

A query like the following can be used to check the results of the checker (Postgres):

```
SELECT *
FROM checksum_history
WHERE date_trunc('day', process_start_date) = CURRENT_DATE
AND result != 'CHECKSUM_MATCH'
AND result != 'BITSTREAM_MARKED_DELETED';
```

Example of a more detailed query:

```
SELECT
    ch.process_start_date,
    ch.process_end_date,
    ch.result,
    ch.checksum_expected,
    ch.checksum_calculated,
    b.bitstream_id,
    bfr.short_description,
    b.store_number,
    substring(b.internal_id for 2) || '/' || substring(b.internal_id from 3 for 2) || '/' || substring(b.
internal_id from 5 for 2) || '/' || b.internal_id AS bitstream_path,
    hi.handle AS item_handle,
    hc.handle AS collection_handle
FROM checksum_history ch
JOIN bitstream b
ON ch.bitstream_id = b.uuid
JOIN bitstreamformatregistry bfr
ON b.bitstream_format_id = bfr.bitstream_format_id
LEFT JOIN bundle2bitstream bb
ON b.uuid = bb.bitstream_id
LEFT JOIN item2bundle ib
ON bb.bundle_id = ib.bundle_id
LEFT JOIN item i
ON ib.item_id = i.uuid
LEFT JOIN handle hi
ON i.uuid = hi.resource_id
AND hi.resource_type_id = 2
LEFT JOIN handle hc
ON i.owning_collection = hc.resource_id
AND hc.resource_type_id = 3
WHERE ch.result != 'CHECKSUM_MATCH'
AND date_trunc('day', process_start_date) = CURRENT_DATE
ORDER BY ch.check_id DESC;
```

DSpace Development

This section contains information on how to modify, extend and customize the DSpace source code.

- User Interface Design Principles & Accessibility
- REST API
- REST API v6 (deprecated)
- Advanced Customisation
- Curation Tasks
- Development Tools Provided by DSpace
- Services to support Alternative Identifiers
- Batch Processing
- Workflow

User Interface Design Principles & Accessibility

These guidelines help ensure that all DSpace components have a consistent layout and follow the essential Web Content Accessibility Guidelines (WCAG). These guidelines MUST be followed by anyone who wants to contribute to the project. See also our Code Contribution Guidelines

- Overview
    - Terminology used in this page
- Guiding Principles
- User Interface Design Guidelines
- User Interface Accessibility Guidelines

Overview

These guidelines apply primarily to the "Base Theme" for the DSpace User Interface.

- **Base Theme** (`/src/app/` directories): The primary look & feel of DSpace (e.g. HTML layout, header/footer, etc) is defined by the HTML5 templates under this directory. Each HTML5 template is stored in a subdirectory named for the Angular component where that template is used. The base theme includes very limited styling (CSS, etc), based heavily on *default Bootstrap (4.x) styling*, and only allowing for minor tweaks to improve accessibility (e.g. default Bootstrap's color scheme does not have sufficient color contrast)
- Two additional themes are provided with DSpace out-of-the-box
    - **Custom Theme** (`/src/themes/custom` directories): This directory acts as the scaffolding or template for creating a new custom theme. It provides (empty) Angular components/templates which allow you to change the theme of individual components. Since all files are empty by default, if you enable this theme (without modifying it), it will look *identical* to the Base Theme.
    - **DSpace Theme** (`/src/themes/dspace` directories): This is the default theme for DSpace 7. It is a very simple *example theme* providing a custom color scheme & homepage on top of the Base Theme.
- More information on themes (in general) can be found in the User Interface Customization documentation

Terminology used in this page

The following terms are used frequently in this page, and this is a quick reference to what we mean by these terms:

- template = Angular template
- component = Angular component
- style = CSS/SCSS styles
- theme = a set of templates that work together (along with the styling/CSS) to comprise the look & feel of the site.

Guiding Principles

- All templates in the Base Theme (*/src/app* directories) should only use default Bootstrap styling. Documentation at: *https://getbootstrap.com/docs/4.6/getting-started/introduction/*
    - Exceptions may be made for accessibility purposes. For example, Bootstrap notes their default color scheme does not always have sufficient color contrast
- When Bootstrap Components (accordion, dropdown, etc …) are required you MUST use the included ng-bootstrap library. Documentation at: *https://ng-bootstrap.github.io/#/components/accordion/examples*

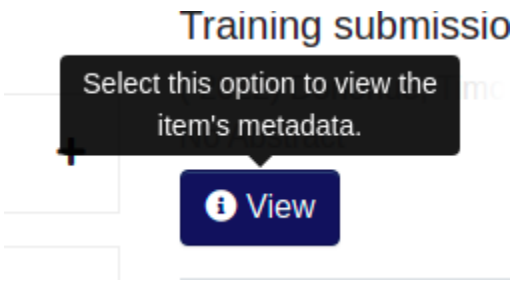
The use of the Bootstrap framework can help in achieving some WCAG goals such as ‘Visual Presentation’ (AAA), 'Parsing' (A), ‘Orientation’ (AA), ‘Reflow’ (AA) and ‘Text Spacing’ (AA). See the Bootstrap chapter ‘*Accessibility*’ for an explanation of WCAG and where to find additional information.

User Interface Design Guidelines

This section provides basic guidelines on User Interface layout/design and the elements (or components) used by the DSpace User Interface. DSpace developers strive to meet these guidelines in order to ensure consistent behavior/layout on all pages. If you find a guideline is not met on a particular page, please report a bug ticket for that user interface page.

- All the buttons should have a text description and an icon:

  If that it’s not possible (e.g. a small button with an icon) always use the ‘name’ and ‘title’ properties.
- Use the tooltip component when you need a better explanation of a button functionality. For example:

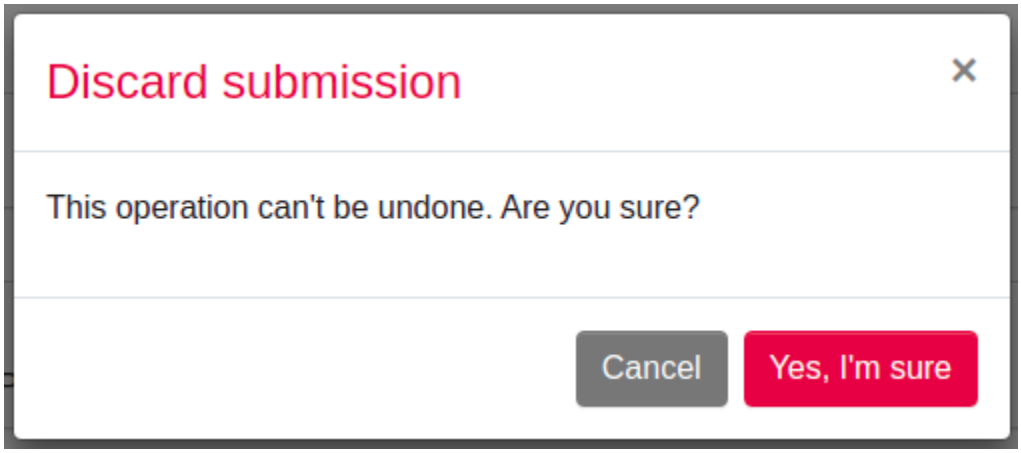


- For UI elements on public pages that are only visible to users with elevated privileges use an inverted color scheme `btn-dark`.
- For the 'anchor' (the '<a>' element) that uses the 'btn' Bootstrap CSS class always use `btn-outline-primary`.
- For the main action button use the Bootstrap CSS class `btn-primary`.
- For buttons like 'Cancel' or 'Back' use che Bootstrap CSS class `btn-outline-secondary`.
- For buttons that open a dropdown list use the Bootstrap CSS class `btn-secondary`.
- In a button series, or group, only one '<button>' has the Bootstrap CSS class `btn-primary`.
- The buttons order, inside a group or a series, should follow the kind of action it performs. At the left side the most 'light' action, like 'Back' or 'Cancel', at the right side the most 'changing' action like 'Delete' or 'Remove'. So, the order is:

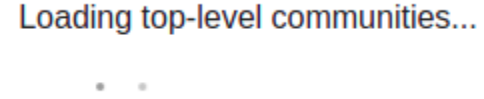
```
Back -> Cancel -> Submit/Edit/Save/Save for later/Deposit -> Discard/Delete/Remove
```

Here an example:

- Where possible, align the buttons to the right.
- Every macro-function inside a page shall be a primary button.
- If, after an action, there is a waiting time, e.g. for the server response, the control (e.g. the button) that launched the action must become disabled and show an animated waiting icon. All other associated controls must be disabled (property '*disabled="true"*').
- Action confirmation should occur on cancellations or non-reversible operations; a modal should appear with a message giving the possibility to confirm or cancel the requested action:

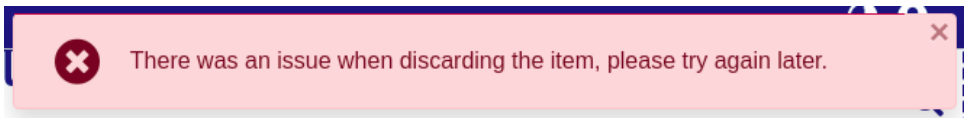


- To inform the user that a certain section of the page is about to be modified as a result of an action (e.g. a page change to move forward in a list) it's necessary to make an animated waiting icon appear:



- All searches that do not return a result must report their absence via a message within a block (e.g. '* *') with the Bootstrap CSS class 'alert-info'.
- The items in the horizontal top navigation menu are links to pages always available to the user (logged or not).
- The items in the left vertical menu are links concerning management, administration and creation or modification of DSpace items. The list can be different according to the permissions of the logged user.

- If a page topic has more logical subdivisions, it's opportune to separate them in more tabs (e.g. the 'Edit collection' page where you can edit metadata, roles and policies).
- Inside the Extended Footer you can insert information about the institution, partnerships, social links, external links or legal information.
- Notifications
    - All the notifications with the whole page scope should be placed on the top right side of the page, hovering on the elements beneath it:



    - Notifications types and color schemes: the corresponding Bootstrap CSS class must be applied:

```
Notification type   Boostrap Class   Description
-------------------------------------------------------------------------------
success             alert-success    to indicate the successful execution of an action
warning             alert-warning    to warn of a possible change to the standard behavior
                                     of some controls (button or action)
info                alert-info       to provide any additional information to the user
                                     (e.g.: no search results)
danger              alert-danger     to inform that an error has occurred
```

    - Besides warnings about changes in behaviors, all the notifications must have a closing button to remove them.
    - The timed notifications are allowed when the notification is purely informational and there is no possibility of interaction (e.g. presence of buttons or forms within the notification).
- For the most common actions and alerts, the following free FontAwesome icons are recommended:

```
Information: fas fa-info-circle
Search     : fas fa-search
Import     : fas fa-file-import
Export     : fas fa-file-export
Add/New    : fas fa-plus
Edit       : fas fa-edit
Save       : fas fa-save
Delete     : fas fa-trash
Cancel     : fas fa-times
Undo       : fas fa-undo
Back       : fas fa-arrow-left

Login      : fas fa-sign-in-alt
Logout     : fas fa-sign-out-alt
```

User Interface Accessibility Guidelines

The DSpace User Interface strives to align with WCAG AA criteria. Some AAA criteria may also be supported. For more information see Accessibility documentation.

Here are specific guidelines we strive to follow in the "Base Theme":

- Web pages have titles that describe their topic or purpose.
- Use the Breadcrumb component on all pages.
- Use headers and labels to describe topics or purposes in order to provide support for the user to navigate, find content, and determine his location. Always provide labels or instructions when content requires user input actions, e.g. forms.
- Elements have full opening and closing tags, are nested according to their own specifications, contain no duplicate attributes, and all IDs are unique.
- Always use ALT properties within the IMAGE tag to describe the content of the images.
- The default language setting of each Web page can be determined programmatically with:

```
<html lang="en">
...
</html>
```

- The text is never justified.
- Text, with the exception of subtitles and images containing text, can be resized up to 200 percent without the aid of assistive technologies and without loss of content or functionality.
- Use '' elements to emphasize something rather than '<b>'. For example, required fields in a form.
- The website operation does not depend on the screen orientation.
- To create Web pages that look and feel predictable, components that have the same functionality within a set of Web pages should be uniquely identified. This means, for example, avoiding buttons with similar functions but different descriptions or using similar icons for different functions. Wherever possible, on different pages or different tabs / page sections, place controls in the same position.
- ARIA Role, ARIA label and ARIA labelledby: use the 'aria-label' and 'role' properties to identify the regions of a page. Ex. to identify two kinds of menu:

```
<div id="leftnav" role="navigaton" aria-label="Primary">
   ...
</div>
<div id="rightnav" role="navigation" aria-label="Secondary">
   ...
</div>
```

Other regions of a page can be the header, footer, the page content, etc. Remember that ‘aria-label’ should be used only when there is no other element in the HTML page that can describe better the element itself. In that case use the ‘aria-labelledby’:

```
<div id="leftnav" role="navigaton" aria-labelledby="menuTitle">
   <h4 id="menuTitle">This is the primary menu</h4>
   ...
</div>
```

It can be also used on a simple text field to provide a label in a situation where there is no text available for a dedicated label but there is other text on the page that can be used to accurately label the control. Ex.:

```
<input name="searchText" type="text" aria-labelledby="searchButton">
<input name="searchButton" id="searchButton" type="submit" value="Search">
```

It’s possible to use them to provide labels to user interface controls (ex.: buttons or inputs in a form).

- For all User Interface components, ‘name’ and ‘role’ must be determined programmatically. To do this use:
    - label elements to associate text labels with form controls
    - ‘aria-labelledby’ and ‘aria-label
- Always prefer the following hierarchy of choices when trying to describe topics or purposes:
    - Plain text with a full description where possible. This will help people with cognitive disabilities who may not immediately know the purpose of the field because the label used by the author is not familiar to them;
    - Label element;
    - ARIA label and ARIA labelledby.
- Identify programmatically the purpose of the inputs using the guidelines described above and the attribute ‘autocomplete’:

```
<input id="fname" type="text" autocomplete="given-name" ... >
```

This property is useful to browsers / user agents to identify the content and provide auto-fill capabilities. The values you can use with 'autocomplete' are described here:

    - https://www.w3.org/TR/WCAG21/#input-purposes
- Including the text of the visible label as part of the accessible name. When speech recognition software processes speech input and looks for matches, it uses the ‘accessible name' of controls, so it’s important that what the user reads in label or description is, at least partially, what is defined in the ‘accessible name' like ‘aria-label’ or ‘aria-labelledby’. E.g. if a button has a visible value of ‘search’ and its ‘aria-label’ has ‘go’ a problem can occur when the user says 'click Search’ :

```
<button aria-label="Go">Search</button>
```

So, if you have an ‘accessible name' available, you can expand it using the label text inside it. All of the following examples are valid:

```
<button>Search</button>
<button aria-label="Search for matches"><i class="fa fa-search"></i></button>

<h4 id="buttonTitle">Search for matches</h4>
<button aria-labelledby="buttonTitle">Search</button>
```

- Order of focus: For example, in a form, use ‘tabindex’ logically (e.g. street number after street name).
- Change the color of an element when it receives FOCUS: e.g. CSS can be used to apply a different color when link elements receive focus.
- Ensure that the information conveyed by color differences is also available in the text; e.g. links also underlined or mandatory form fields highlighted with an asterisk (*);
- Error identification: The element in error is identified and described by text even with client-side controls. Use the property ‘*aria-invalid=”true”*’ inside that element. For example, within a form, apply client-side validations to the input fields and make sure that any error message is comprehensive; where possible, suggestions on how to correct the error, should be provided to the user.

- Provide users with sufficient time to read and use the content. For example, inside the timed notifications (error or success messages) provide a button to stop the timer.

REST API

- Overview
- REST Contract / Documentation
- Finding which REST API Endpoint to use
- REST Configuration
    - REST Spring Boot Configuration
- Technical Design
- DSpace Demo REST-API HAL Browser
- DSpace Python REST Client Library

Overview

The REST API for DSpace is provided as part of the "server" webapp (`[dspace-source]/dspace-server-webapp/`). It is available on the `/api/` subpath of that webapp (i.e. `${dspace.server.url}/api/`), though a human browseable/searchable interface (using the HAL Browser) is also available at the root path (i.e. `${dspace.server.url}`).

The REST API only responds in JSON at this time.

REST Contract / Documentation

The REST Contract is maintained in GitHub at https://github.com/DSpace/RestContract/blob/dspace-7_x/README.md

This contract provides detailed information on how to interact with the API, what endpoints are available, etc. All features/capabilities of the DSpace UI are available in this API.

Finding which REST API Endpoint to use

When first trying the use the DSpace REST API, it can be difficult to determine where to begin. This brief guide provides a few hints on where to start.

First, it's important to be aware that **every single action in the User Interface can be done in the REST API.** So, if you can achieve something in the User Interface, then it's also possible to do via the REST API.

A few key endpoints to be aware of:

- Authentication: https://github.com/DSpace/RestContract/blob/dspace-7_x/authentication.md
- CSRF Tokens (required for all non-GET requests): https://github.com/DSpace/RestContract/blob/dspace-7_x/csrf-tokens.md
- Submission via REST API: https://github.com/DSpace/RestContract/blob/dspace-7_x/submission.md
- Search via REST API (across all object types): https://github.com/DSpace/RestContract/blob/dspace-7_x/search-endpoint.md
    - Some endpoints also provide a "/search" subpath: https://github.com/DSpace/RestContract/blob/dspace-7_x/search-rels.md

How to find which endpoint(s) to use for any feature or action:

1. Open the DSpace User Interface in your browser window. You can even use our Demo Site (https://demo.dspace.org/) if you don't have the User Interface installed or running locally.
2. In your Browser, open the "Developer Tools"
    a. In Chrome, go to "More Tools Developer Tools"
    b. In Firefox, go to "Web Developer Web Developer Tools".
    c. In Microsoft Edge, go to "More Tools Developer Tools".
3. Once in "Developer Tools", open the "Network" tab. This tab will provide information about *every single call that the User Interface makes to the REST API.*
4. Now, perform an action or use a feature in the User Interface in your browser window.
5. Analyze what calls were just sent to the REST API in your "Network" tab. Those are the exact REST API endpoints that were used to perform that action.
    a. NOTE: Some actions may use *multiple endpoints.*
6. Finally, lookup the documentation for those endpoint(s) in the REST Contract / Documentation (see link above)

REST Configuration

The following REST API configurations are provided in `[dspace]/config/rest.cfg` and may be overridden in your local.cfg

Property:	`rest.cors.allowed-origins`
Example Value:	`rest.cors.allowed-origins = ${dspace.ui.url}`

Informational Note:	Allowed Cross-Origin-Resource-Sharing (CORS) origins (in "Access-Control-Allow-Origin" header). Only these origins (client URLs) can successfully authenticate with your REST API *via a web browser*. Defaults to `${dspace.ui.url}` if unspecified (as the UI must have access to the REST API). If you customize that setting, MAKE SURE TO include `${dspace.ui.url}` in that setting if you wish to continue trusting the UI. Multiple allowed origin URLs may be comma separated (or this configuration can be defined multiple times). Wildcard value (*) is NOT SUPPORTED. Keep in mind any URLs added to this setting must be *an exact match with the origin*: mode (http vs https), domain, port, and subpath(s) all must match.. So, for example, these URLs are all considered different origins: "http://mydspace.edu", "http://mydspace.edu:4000" (different port), "https://mydspace.edu" (http vs https), "https://myapp.mydspace.edu" (different domain), and "https://mydspace.edu/myapp" (different subpath). *NOTE #1:* Development or command-line tools may not use CORS and may therefore bypass this configuration. CORS does not provide protection to the REST API / server webapp. Instead, its role is to protect browser-based clients from cookie stealing or other Javascript-based attacks. All modern web browsers use CORS to protect their users from such attacks. Therefore DSpace's CORS support is used to protect users who access the REST API via a web browser application, such as the DSpace UI or custom built Javascript tools/scripts. *NOTE #2:* If you modify this value to allow additional UIs (or Javascript tools) to access your REST API, then you may also need to modify `proxies.trusted.ipranges` to trust the IP address of each UI. Modifying trusted proxies is only necessary if the `X-FORWARDED-FOR` header must be trusted from each additional UIs. (The DSpace UI currently requires the `X-FORWARDED-FOR` header to be trusted). By default, `proxies.trusted.ipranges` will only trust the IP address of the `${dspace.ui.url}` configuration. *NOTE #3:* Although subpath must match, the Origin header itself sent from Angular will never contain a subpath. So if the dspace.ui.url config property ever changes to include a subpath like /myapp, then the expected origin will need to be added to `rest.cors.allowed-origins` , ie. the URL **without** the subpath. (Requires reboot of servlet container, e.g. Tomcat, to reload)
Property:	`rest.cors.allow-credentials`
Example Value:	`rest.cors.allow-credentials = true`
Informational Note:	Whether or not to allow credentials (e.g. cookies) sent by the client/browser in CORS requests (in "Access-Control-Allow-Credentials" header). For DSpace, this MUST be set to "true" to support CSRF checks (which use Cookies) and external authentication via Shibboleth (and similar). Defaults to "true" if unspecified. (Requires reboot of servlet container, e.g. Tomcat, to reload)
Property:	`rest.projections.full.max`
Example Value:	`rest.projections.full.max = 2`
Informational Note:	This property determines the max embeddepth for a FullProjection. This is also used by the SpecificLevelProjection as a fallback in case the property is defined on the bean. Usually, this should be kept as-is for best performance.
Property:	`rest.projection.specificLevel.maxEmbed`
Example Value:	`rest.projection.specificLevel.maxEmbed = 5`

Informational Note:	This property determines the max embed depth for a SpecificLevelProjection. Usually, this should be kept as-is for best performance.
Property:	`rest.properties.exposed`
Example Value:	`rest.properties.exposed = plugin.named.org.dspace.curate.CurationTask` `rest.properties.exposed = google.analytics.key`
Informational Note:	Define which configuration properties are exposed through the `http://<dspace.server.url>/api/config/properties/` REST API endpoint. If a rest request is made for a property which exists, but isn't listed here, the server will respond that the property wasn't found. This property can be defined multiple times to allow access to multiple configuration properties. Generally, speaking, it is ONLY recommended to expose configuration settings where they are necessary for the UI or client, as exposing too many configurations could be a security issue. This is why we only expose the two above settings by default.

REST Spring Boot Configuration

Because the REST API is a Spring Boot web application, you can also configure or override *any* Spring Boot settings in your local.cfg. DSpace's default Spring Boot configuration can be found in `[src]/dspace-server-webapp/src/main/resources/application.properties`. A few common settings from Spring Boot which you may wish to override in your local.cfg include:

Property:	`spring.servlet.multipart.max-file-size`
Example Value:	`spring.servlet.multipart.max-file-size = 512MB`
Informational Note:	Per Spring Boot docs, this setting specifies the maximum size of file that can be uploaded via Spring Boot (and therefore via the DSpace REST API). A value of "-1" removes any limit. DSpace sets this to 512MB by default.
Property:	`spring.servlet.multipart.max-request-size`
Example Value:	`spring.servlet.multipart.max-request-size = 512MB`
Informational Note:	Per Spring Boot docs, this setting specifies the maximum size of a single request via Spring Boot (and therefore via the DSpace REST API). That means if multiple files are uploaded at once, this is the maximum total size of all files. A value of "-1" removes any limit. DSpace sets this to 512MB by default.

Technical Design

The REST API & Server Webapp are built on Spring Boot and Spring HATEOAS, using Spring Security. It also aligns with Spring Data REST (though at this time it doesn't use it directly because of incompatibility with the DSpace data model).

The REST API is stateless, aligns with HATEOAS (Hypertext as the Engine of Application State) principles, returning HAL formatted JSON. This allows the REST API to be easily browsable/interactable via third-party tools that understand HAL & HATEOAS, such as the HAL Browser. JSON Web Tokens (JWT) are used to store state/session information between requests.

For better security, the REST API requires usage of CSRF tokens for all modifying requests.

More information can be found in the REST Contract at https://github.com/DSpace/RestContract/blob/dspace-7_x/README.md#rest-design-principles

DSpace Demo REST-API HAL Browser

- https://demo.dspace.org/server/

DSpace Python REST Client Library

The Library Code is a DSpace Platinum certified Service Provider. They created a python library to use the REST-API of DSpace 7 and upwards out of python: https://github.com/the-library-code/dspace-rest-python.

REST API v6 (deprecated)

- What is DSpace REST API (v4-v6)
  - Installing the REST API (v4-v6)
    - Disabling SSL
  - REST Endpoints
    - Index / Authentication
      - Shibboleth Apache configuration for the REST API
    - Communities
    - Collections
    - Items
    - Bitstreams
    - Handle
    - Hierarchy
    - Schema and Metadata Field Registry
    - Report Tools
  - Model - Object data types
- Introduction to Jersey for developers
- Configuration for DSpace REST
- Recording Proxy Access by Tools
- Additional Information

This documentation describes the old, deprecated REST API

This documentation describes the deprecated DSpace v4-6 REST API. This old API is still available in DSpace 7, but will be removed in DSpace 8.

We highly recommend all users migrate scripts/tools to use the new REST API. This API is no longer actively supported or maintained.

What is DSpace REST API (v4-v6)

The REST API module provides a programmatic interface to DSpace Communities, Collections, Items, and Bitstreams.

DSpace 4 introduced the initial REST API, which did not allow for authentication, and provided only READ-ONLY access to publicly accessible Communities, Collections, Items, and Bitstreams. DSpace 5 builds off of this and allows authentication to access restricted content, as well as allowing Create, Edit and Delete on the DSpace Objects. DSpace 5 REST API also provides improved pagination over resources and searching. There has been a minor drift between the DSpace 4 REST API and the DSpace 5 REST API, so client applications will need to be targeted per version.

Installing the REST API (v4-v6)

This older REST API is disabled in the build by default. To build it, you must run:

```
# The "-Pdspace-rest" flag will build the deprecated "rest" webapp alongside the new "server" webapp
mvn clean package -Pdspace-rest
```

The REST API deploys as a separate "rest" webapp for your servlet container / tomcat. For example, depending on how you deploy webapps, one way would be to alter tomcat-home/conf/server.xml and add:

```
<Context path="/rest"  docBase="/dspace/webapps/rest" />
```

In DSpace 4, the initial/official Jersey-based REST API was added to DSpace. The DSpace 4 REST API provides READ-ONLY access to DSpace Objects.

In DSpace 5, the REST API adds authentication, allows Creation, Update, and Delete to objects, can access restricted materials if authorized, and it requires SSL.

Disabling SSL

For localhost development purposes, SSL can add additional getting-started difficulty, so security can be disabled. To disable DSpace REST's requirement to require security/ssl, alter `[dspace]/webapps/rest/WEB-INF/web.xml` or `[dspace-source]/dspace-rest/src/main/webapp/WEB-INF/web.xml` and comment out the `<security-constraint>` block, and restart your servlet container. Production usages of the REST API should use SSL, as authentication credentials should not go over the internet unencrypted.

REST Endpoints

The REST API is modeled after the DSpace Objects of Communities, Collections, Items, and Bitstreams. The API is not a straight database schema dump of these entities, but provides some wrapping that makes it easy to follow relationships in the API output.

HTTP Header: Accept

Note: You must set your request header's "Accept" property to either JSON (application/json) or XML (application/xml) depending on the format you prefer to work with.

Example usage from command line in XML format with pretty printing:

```
curl -s -H "Accept: application/xml" http://localhost:8080/rest/communities | xmllint --format -
```

Example usage from command line in JSON format with pretty printing:

```
curl -s -H "Accept: application/json" http://localhost:8080/rest/communities | python -m json.tool
```

For this documentation, we will assume that the URL to the "REST" webapp will be http://localhost:8080/rest/ for production systems, this address will be slightly different, such as: https://demo.dspace.org/rest/. The path to an endpoint, will go after the /rest/, such as /rest/communities, all-together this is: http://localhost:8080/rest/communities

Another thing to note is that there are Query Parameters that you can tack on to the end of an endpoint to do extra things. The most commonly used one in this API is "?expand". Instead of every API call defaulting to giving you every possible piece of information about it, it only gives a most commonly used set by default and gives the more "expensive" information when you deliberately request it. Each endpoint will provide a list of available expands in the output, but for getting started, you can start with ?expand=all, to make the endpoint provide all of its information (parent objects, metadata, child objects). You can include multiple expands, such as: ?expand=collections, subCommunities .

Two other query parameters of note are `limit` and `offset.  Endpoints which return arrays of objects, such as` /communities, are "paginated": the full list is broken into "pages" which start at `offset` from the beginning of the list and contain at most `limit` elements. By repeated queries you can retrieve any portion of the array or all of it. Offsets begin at zero. So, to retrieve the sixth through tenth elements of the full list of Collections, you could do this:

```
curl -s -H "Accept: application/json" http://localhost:8080/rest/collections?offset=5\&limit=5
```

Index / Authentication

REST API Authentication has changed in DSpace 6.x. It now uses a `JSESSIONID` cookie (see below). The previous (5.x) authentication scheme using a `rest-dspace-token` is no longer supported.

Method	Endpoint	Description
GET	/	REST API static documentation page

<table>
<tr><td>POST</td><td>/login</td><td>Login to the REST API using a DSpace EPerson (user). It returns a <code>JSESSIONID</code> cookie, that can be used for future authenticated requests.

Example Request:

<pre>
Can use either POST or GET (POST recommended). Must pass the parameters "email" and "password".
curl -v -X POST --data "email=admin@dspace.org&password=mypass" https://dspace.myu.edu/rest/login
</pre>
Example Response:

<pre>
HTTP/1.1 200 OK
Set-Cookie: JSESSIONID=6B98CF8648BCE57DCD99689FE77CB1B8; Path=/rest/; Secure; HttpOnly
</pre>
Example of using JSESSIONID cookie for subsequent (authenticated) requests:

<pre>
curl -v --cookie "JSESSIONID=6B98CF8648BCE57DCD99689FE77CB1B8" https://dspace.myu.edu/rest/status
This should return <authenticated>true</authenticated>, and information about the authenticated user session
</pre>
Invalid email/password combinations will receive an <code>HTTP 401 Unauthorized</code> response.

Please note, special characters need to be HTTP URL encoded.
For example, an email address like <code>dspacedemo+admin@gmail.com</code> (notice the + special character) would need to be encoded as <code>dspacedemo%2Badmin@gmail.com</code>.</td></tr>
<tr><td>GET</td><td>/shibboleth-login</td><td>Login to the REST API using Shibboleth authentication. In order to work, this requires additional Apache configuration. To authenticate, execute the following steps:

1. Call the REST Shibboleth login point with a Cookie jar:

<pre>
curl -v -L -c cookiejar "https://dspace.myu.edu/rest/shibboleth-login"
</pre>
2. This should take you again to the IdP login page. You can submit this form using curl using the same cookie jar. However this is IdP dependant so we cannot provide an example here.

3. Once you submit the form using curl, you should be taken back to the /rest/shibboleth-login URL which will return you the JSESSIONID.

4. Using that JSESSIONID, check if you have authenticated successfully:

<pre>
curl -v "http://localhost:8080/dspace-rest/status" --cookie "JSESSIONID=0633C6379266A283E53F65DF8EF61AB9"
</pre></td></tr>
<tr><td>POST</td><td>/logout</td><td>Logout from the REST API, by providing a <code>JSESSIONID</code> cookie. After being posted this cookie will no longer work.

Example Request:

<pre>
curl -v -X POST --cookie "JSESSIONID=6B98CF8648BCE57DCD99689FE77CB1B8" https://dspace.myu.edu/rest/logout
</pre>
After posting a logout request, cookie is invalidated and the "/status" path should show you as unauthenticated (even when passing that same cookie). For example:

<pre>
curl -v --cookie "JSESSIONID=6B98CF8648BCE57DCD99689FE77CB1B8" https://dspace.myu.edu/rest/status
This should show <authenticated>false</authenticated>
</pre>
Invalid token will result in HTTP 400 Invalid Request</td></tr>
</table>

GET	/test	Returns string "REST api is running", for testing that the API is up.
		
		Example Request:
		
		`curl https://dspace.myu.edu/rest/test`
		
		Example Response:
		
		`REST api is running.`

GET	/status	Receive information about the currently authenticated user token, or the API itself (e.g. version information).
		
		Example Request (XML by default):
		
		`curl -v --cookie "JSESSIONID=6B98CF8648BCE57DCD99689FE77CB1B8" https://dspace.myu.edu/rest/status`
		
		Example Request (JSON):
		
		`curl -v -H "Accept: application/json" --cookie "JSESSIONID=6B98CF8648BCE57DCD99689FE77CB1B8" https://dspace.myu.edu/rest/status`
		
		Example JSON Response:
		
		`{ "okay":true, "authenticated":true, "email":"admin@dspace.org", "fullname":"DSpace Administrator", "sourceVersion":"6.0", "apiVersion":"6" }`

Shibboleth Apache configuration for the REST API

Before Shibboleth authentication for the REST API will work, you need to secure the `/rest/shibboleth-login` endpoint. Add this configuration section to your Apache HTTPD Shibboleth configuration:

```
<Location "/rest/shibboleth-login">
         AuthType shibboleth
         ShibRequireSession On
         # Please note that setting ShibUseHeaders to "On" is a potential security risk.
         # You may wish to set it to "Off". See the mod_shib docs for details about this setting:
         # https://wiki.shibboleth.net/confluence/display/SHIB2/NativeSPApacheConfig#NativeSPApacheConfig-
AuthConfigOptions
         # Here's a good guide to configuring Apache + Tomcat when this setting is "Off":
         # https://www.switch.ch/de/aai/support/serviceproviders/sp-access-rules.html#javaapplications
         ShibUseHeaders On
         require valid-user
</Location>
```

You can test your configuration in 3 different ways:

1. Using a web browser:
    a. Go to `https://dspace.myu.edu/rest/shibboleth-login`, this should redirect you to the login page of your IdP if you don't have a Shibboleth session yet.
    b. Enter your test credentials and this should take you back to the `/rest/shibboleth-login` URL. You should then see a blank page but in the response headers, the JSESSIONID cookie should be present.
    c. Then go to `/rest/status` and you should see information on the current authenticated ePerson.
2. Using curl without a Shibboleth Session
    a. Call the REST Shibboleth login point with a Cookie jar:

```
curl -v -L -c cookiejar "https://dspace.myu.edu/rest/shibboleth-login"
```

b. This should take you again to the IdP login page. You can submit this form using curl using the same cookie jar. However this is IdP dependant so I cannot provide an example here.
c. Once you submit the form using curl, you should be taken back to the `/rest/shibboleth-login` URL which will return you the JSESSIONID.
d. Using that JSESSIONID, check if you have authenticated successfully:

```
curl -v "https://dspace.myu.edu/dspace-rest/status" --cookie
"JSESSIONID=0633C6379266A283E53F65DF8EF61AB9"
```

3. Using curl with a Shibboleth Session (cookie)
   a. When you post the Shibboleth login form, the Shibboleth daemon on the **DSpace server** also returns you a Shibboleth Cookie. This cookie looks like `_shibsession_64656661756c74687...` You can also grab this cookie from your browser.
   b. Double check that the cookie you took is valid:

```
curl -v 'https://dspace-url/Shibboleth.sso/Session' -H 'Cookie:
_shibsession_64656661756c7468747470733a2f2f7265706f7369746f72792e636172646966666d65742e61632e756b2f
73686962626f6c657468=_a8d3ad20d8b655250c7357f7ac0e2910;'
```

   c. This should give you information if the Shibboleth session is valid and on the number of attributes.
   d. Use this cookie to obtain a Tomcat JSESSIONID:

```
curl -v 'https://dspace-url/rest/shibboleth-login' -H 'Cookie:
_shibsession_64656661756c7468747470733a2f2f7265706f7369746f72792e636172646966666d65742e61632e756b2f
73686962626f6c657468=_a8d3ad20d8b655250c7357f7ac0e2910;'
```

   e. Use the returned JSESSIONID to check if you have authenticated successfully:

```
curl -v "http://dspace-url/rest/status" --cookie "JSESSIONID=0633C6379266A283E53F65DF8EF61AB9"
```

Communities

Communities in DSpace are used for organization and hierarchy, and are containers that hold sub-Communities and Collections. (ex: Department of Engineering)

- GET /communities - Returns array of all communities in DSpace.
- GET /communities/top-communities - Returns array of all top communities in DSpace.
- GET /communities/{communityId} - Returns community.
- GET /communities/{communityId}/collections - Returns array of collections of community.
- GET /communities/{communityId}/communities - Returns array of subcommunities of community.
- POST /communities - Create new community at top level. You must post community.
- POST /communities/{communityId}/collections - Create new collections in community. You must post Collection.
- POST /communities/{communityId}/communities - Create new subcommunity in community. You must post Community.
- PUT /communities/{communityId} - Update community. You must put Community
- DELETE /communities/{communityId} - Delete community.
- DELETE /communities/{communityId}/collections/{collectionId} - Delete collection in community.
- DELETE /communities/{communityId}/communities/{communityId2} - Delete subcommunity in community.

Collections

Collections in DSpace are containers of Items. (ex: Engineering Faculty Publications)

- GET /collections - Return all collections of DSpace in array.
- GET /collections/{collectionId} - Return collection with id.
- GET /collections/{collectionId}/items - Return all items of collection.
- POST /collections/{collectionId}/items - Create posted item in collection. You must post an Item

```
curl -v -H "Content-Type: application/json"
--cookie "JSESSIONID=6B98CF8648BCE57DCD99689FE77CB1B8"
--data "{\"metadata\":[ {\"key\": \"dc.title\", \"value\": \"Testing Item\"}, {\"key\": \"dc.contributor.
author\", \"value\": \"Jane Smith\"}]}"
https://dspace-url/rest/[collection-uuid]/items
```

- POST /collections/find-collection - Find collection by passed name.
- PUT /collections/{collectionId} - Update collection. You must put Collection.
- DELETE /collections/{collectionId} - Delete collection from DSpace.
- DELETE /collections/{collectionId}/items/{itemId} - Delete item in collection.

Items

Items in DSpace represent a "work" and combine metadata and files, known as Bitstreams.

- GET /items - Return list of items.
- GET /items/{item id} - Return item.
- GET /items/{item id}/metadata - Return item metadata.
- GET /items/{item id}/bitstreams - Return item bitstreams.
- POST /items/find-by-metadata-field - Find items by metadata entry. You must post a MetadataEntry.
- POST /items/{item id}/metadata - Add metadata to item. You must post an array of MetadataEntry.
- POST /items/{item id}/bitstreams - Add bitstream to item. You must post a Bitstream.
- PUT /items/{item id}/metadata - Update metadata in item. You must put a MetadataEntry.
- DELETE /items/{item id} - Delete item.
- DELETE /items/{item id}/metadata - Clear item metadata.
- DELETE /items/{item id}/bitstreams/{bitstream id} - Delete item bitstream.

Bitstreams

Bitstreams are files. They have a filename, size (in bytes), and a file format. Typically in DSpace, the Bitstream will the "full text" article, or some other media. Some files are the actual file that was uploaded (tagged with bundleName:ORIGINAL), others are DSpace-generated files that are derivatives or renditions, such as text-extraction, or thumbnails. You can download files/bitstreams. DSpace doesn't really limit the type of files that it takes in, so this could be PDF, JPG, audio, video, zip, or other. Also, the logo for a Collection or a Community, is also a Bitstream.

- GET /bitstreams - Return all bitstreams in DSpace.
- GET /bitstreams/{bitstream id} - Return bitstream.
- GET /bitstreams/{bitstream id}/policy - Return bitstream policies.
- GET /bitstreams/{bitstream id}/retrieve - Return data of bitstream.
- POST /bitstreams/{bitstream id}/policy - Add policy to item. You must post a ResourcePolicy
- PUT /bitstreams/{bitstream id}/data - Update data/file of bitstream. You must put the data
- PUT /bitstreams/{bitstream id} - Update metadata of bitstream. You must put a Bitstream, does not alter the file/data
- DELETE /bitstreams/{bitstream id} - Delete bitstream from DSpace.
- DELETE /bitstreams/{bitstream id}/policy/{policy_id} - Delete bitstream policy.

You can access the parent object of a Bitstream (normally an Item, but possibly a Collection or Community when it is its logo) through: /bitstreams/:bitstreamID?expand=parent

As the documentation may state "You must post a ResourcePolicy" or some other object type, this means that there is a structure of data types, that your XML or JSON must be of type, when it is posted in the body.

Handle

In DSpace, Communities, Collections, and Items typically get minted a Handle Identifier. You can reference these objects in the REST API by their handle, as opposed to having to use the internal item-ID.

- GET /handle/{handle-prefix}/{handle-suffix} - Returns a Community, Collection, or Item object that matches that handle.

Hierarchy

Assembling a full representation of the community and collection hierarchy using the communities and collections endpoints can be inefficient. Retrieve a lightweight representation of the nested community and collection hierarchy. Each node of the hierarchy contains minimal information (id, handle, name).

- GET /hierarchy - Retrieve a lightweight representation of the nested community and collection hierarchy.

Schema and Metadata Field Registry

- GET /registries/schema - Return the list of schemas in the registry
- GET /registries/schema/{schema_prefix} - Returns the specified schema
- GET /registries/schema/{schema_prefix}/metadata-fields/{element} - Returns the metadata field within a schema with an unqualified element name
- GET /registries/schema/{schema_prefix}/metadata-fields/{element}/{qualifier} - Returns the metadata field within a schema with a qualified element name
- POST /registries/schema/ - Add a schema to the schema registry
- POST /registries/schema/{schema_prefix}/metadata-fields - Add a metadata field to the specified schema
- GET /registries/metadata-fields/{field_id} - Return the specified metadata field

- PUT /registries/metadata-fields/{field_id} - Update the specified metadata field
- DELETE /registries/metadata-fields/{field_id} - Delete the specified metadata field from the metadata field registry
- DELETE /registries/schema/{schema_id} - Delete the specified schema from the schema registry

Note: since the schema object contains no data fields, the following method has not been implemented: PUT /registries/schema/{schema_id}

Report Tools

Reporting Tools that allow a repository manager to audit a collection for metadata consistency and bitstream consistency. See REST Based Quality Control Reports for more information.

Collection Report Tool on demo.dspace.org

Metadata Query Tool on demo.dspace.org

- GET /reports - Return a list of report tools built on the rest api
- GET /reports/{nickname} - Return a redirect to a specific report
- GET /filters - Return a list of use case filters available for quality control reporting
- GET /filtered-collections - Return collections and item counts based on pre-defined filters
- GET /filtered-collections/{collection_id} - Return items and item counts for a collection based on pre-defined filters
- GET /filtered-items - Retrieve a set of items based on a metadata query and a set of filters

Model - Object data types

Here are all of the data types, not all fields are necessary or supported when posting/putting content, but the output contains this information:

Community Object

{"id":456,"name":"Reports Community","handle":"10766/10213","type":"community","link":"/rest/communities/456","expand":["parentCommunity","collections","subCommunities","logo","all"],"logo":null,"parentCommunity":null,"copyrightText":"","introductoryText":"","shortDescription":"Collection contains materials pertaining to the Able Family","sidebarText":"","countItems":3,"subcommunities":[],"collections":[]}

Collection Object

{"id":730,"name":"Annual Reports Collection","handle":"10766/10214","type":"collection","link":"/rest/collections/730","expand":["parentCommunityList","parentCommunity","items","license","logo","all"],"logo":null,"parentCommunity":null,"parentCommunityList":[],"items":[],"license":null,"copyrightText":"","introductoryText":"","shortDescription":"","sidebarText":"","numberItems":3}

Item Object

{"id":14301,"name":"2015 Annual Report","handle":"123456789/13470","type":"item","link":"/rest/items/14301","expand":["metadata","parentCollection","parentCollectionList","parentCommunityList","bitstreams","all"],"lastModified":"2015-01-12 15:44:12.978","parentCollection":null,"parentCollectionList":null,"parentCommunityList":null,"bitstreams":null,"archived":"true","withdrawn":"false"}

Bitstream Object

{"id":47166,"name":"appearance and physiology 100 percent copied from wikipedia.pdf","handle":null,"type":"bitstream","link":"/rest/bitstreams/47166","expand":["parent","policies","all"],"bundleName":"ORIGINAL","description":"","format":"Adobe PDF","mimeType":"application/pdf","sizeBytes":129112,"parentObject":null,"retrieveLink":"/bitstreams/47166/retrieve","checkSum":{"value":"62778292a3a6dccbe2662a2bfca3b86e","checkSumAlgorithm":"MD5"},"sequenceId":1,"policies":null}

ResourcePolicy Object

[{"id":317127,"action":"READ","epersonId":-1,"groupId":0,"resourceId":47166,"resourceType":"bitstream","rpDescription":null,"rpName":null,"rpType":"TYPE_INHERITED","startDate":null,"endDate":null}]

MetadataEntry Object

{"key":"dc.description.abstract", "value":"This is the description abstract", "language": null}

User Object

{"email":"test@dspace.org","password":"pass"}

Status Object

{"okay":true,"authenticated":true,"email":"test@dspace.org","fullname":"DSpace Test User","token":"6d45daaa-7b02-4ae7-86de-a960838fae5c"}

Introduction to Jersey for developers

The REST API for DSpace is implemented using Jersey, the reference implementation of the Java standard for building RESTful Web Services (JAX-RS 1). That means this API should be easier to expand and maintain than other API approaches, as this approach has been widely adopted in the industry. If this client documentation does not fully answer about how an endpoint works, it is helpful to look directly at the Java REST API code, to see how it is implemented. The code typically has required parameters, optional parameters, and indicates the type of data that will be responded.

There was no central ProviderRegistry that you have to declare your path. Instead, the code is driven by annotations, here is a list of annotations used in the code for CommunitiesResource.java:

- @Path("/communities"), which then allows it to be routed to http://localhost:8080/communities, this is then the base path for all the requests within this class.
- @GET, which indicates that this method responds to GET http requests
- @POST, which indicates that this method responds to POST http requests
- @PUT, which indicates that this method responds to PUT http requests
- @DELETE, which indicates that this method responds to DELETE http requests
- @Path("/{community_id}"), the path is appended to the class level @Path above, this one uses a variable {community_id}. The total endpoint would be http://localhost:8080/rest/communities/123, where 123 is the ID.
- @Consumes({ MediaType.APPLICATION_JSON, MediaType.APPLICATION_XML }), this indicates that this request expects input of either JSON or XML. Another endpoint accepts HTML input.
- @PathParam("community_id") Integer communityId, this maps the path placeholder variable {community_id} to Java int communityID
- @QueryParam("userIP") String user_ip, this maps a query param like ?userIP=8.8.4.4 to Java String user_id variable, and user_id == "8.8.4.4"

Configuration for DSpace REST

Property	rest.stats
Example Value	true
Informational Note	Boolean value indicates whether statistics should be recorded for access via the REST API; Defaults to 'false'.

Recording Proxy Access by Tools

For the purpose of more accurate statistics, a web-based tool may specify who is using it, by adding parameters to the request:

```
http://localhost:8080/rest/items/:ID?userIP=ip&userAgent=userAgent&xforwardedfor=xforwardedfor
```

If no parameters are given, the details of the HTTP request's sender are used in statistics. This enables tools to record the details of their user rather than themselves.

Additional Information

Additional information can be found in the README for dspace-rest, and in the GitHub Pull Request for DSpace REST (Jersey).

Usage examples can be found at: https://github.com/BrunoNZ/dspace-rest-requests

REST Based Quality Control Reports

- Accessing the Query Tool
- Tutorial
- Summary
- API Calls Used in these Reports
- Report Screen Shots
    - Collection QC Report
    - Metadata Query Report
- Installation and Configuration
    - Installing in DSpace 6
    - Disabling the REST Reports
    - Configuring Access of the Reporting Tools
    - Configure the REST Reports that can be requested by name
    - Configure Item handle resolution
    - Enable User Authentication (Password AuthN only) for REST reports
    - Configure the database-specific format for a regex expression
    - Configure the sets of filters of interest to your repository managers
    - Other filter configuration settings
    - Enabling Sort-able Report Tables
    - Installing in DSpace 5

DSpace 7.0 only supports this when using the older, deprecated REST API v6

In DSpace 7.0, REST Quality Control Reports are currently only supported if you also install the old REST API v6 (deprecated) webapp. Tentative plans to migrate these reports to support the new REST API have begun in https://github.com/DSpace/DSpace/issues/7641

Accessing the Query Tool

Browse to `/rest/static/reports/index.html`

Tutorial

DSpace REST Report Tool Tutorial

The following repository contains a tutorial demonstrating the usage of the REST Base Report Tools: https://github.com/terrywbrady/restReportTutorial/blob/master/README.md

Summary

These reports utilize the DSpace REST API to provide a Collection Manager with

- an overview of their collections
- a tool to query metadata for consistency

When deploying the DSpace REST API, and institution may choose to make the API publicly accessible or to restrict access to the API.

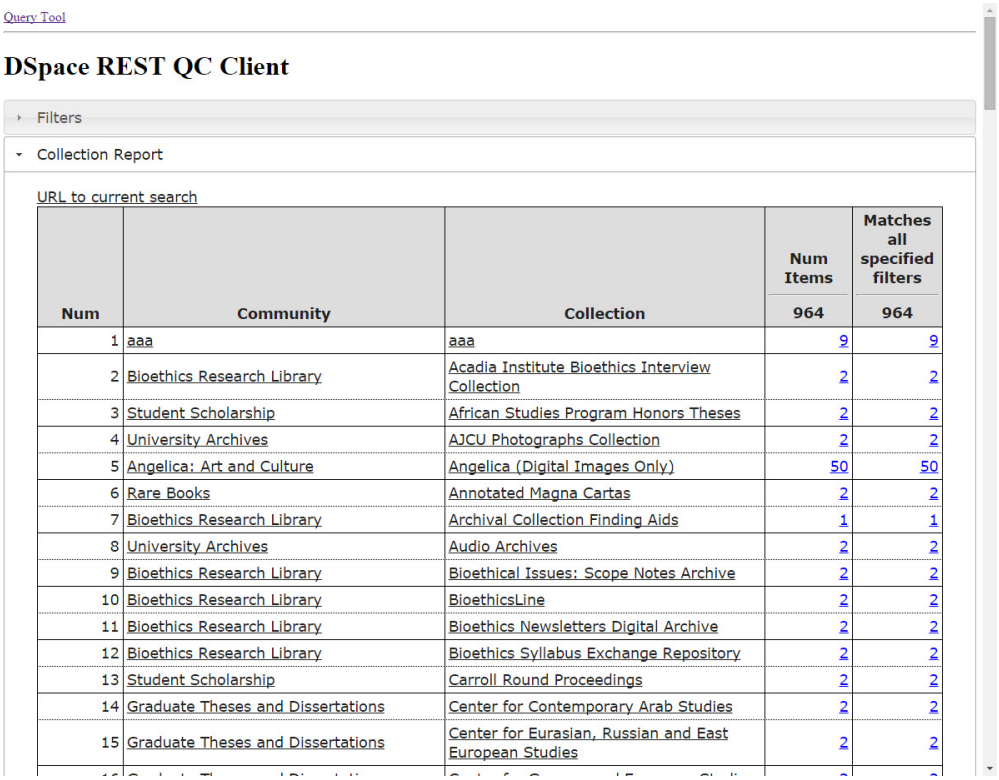
If these reports are deployed in a protected manner, the reporting tools can be configured to bypass DSpace authorization when reporting on collections and items.

API Calls Used in these Reports

REST Reports - Summary of API Calls

Report Screen Shots

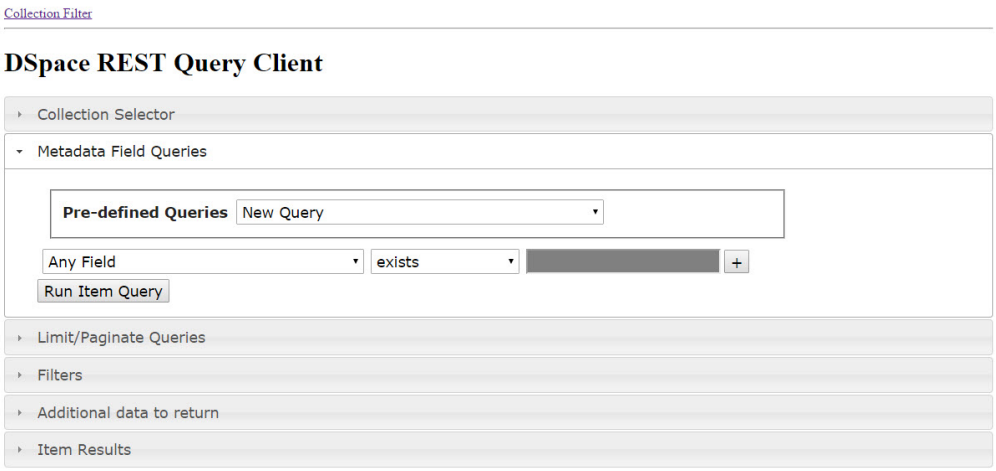
Collection QC Report



Num	Community	Collection	Num Items	Matches all specified filters
			964	964
1	aaa	aaa	9	9
2	Bioethics Research Library	Acadia Institute Bioethics Interview Collection	2	2
3	Student Scholarship	African Studies Program Honors Theses	2	2
4	University Archives	AJCU Photographs Collection	2	2
5	Angelica: Art and Culture	Angelica (Digital Images Only)	50	50
6	Rare Books	Annotated Magna Cartas	2	2
7	Bioethics Research Library	Archival Collection Finding Aids	1	1
8	University Archives	Audio Archives	2	2
9	Bioethics Research Library	Bioethical Issues: Scope Notes Archive	2	2
10	Bioethics Research Library	BioethicsLine	2	2
11	Bioethics Research Library	Bioethics Newsletters Digital Archive	2	2
12	Bioethics Research Library	Bioethics Syllabus Exchange Repository	2	2
13	Student Scholarship	Carroll Round Proceedings	2	2
14	Graduate Theses and Dissertations	Center for Contemporary Arab Studies	2	2
15	Graduate Theses and Dissertations	Center for Eurasian, Russian and East European Studies	2	2

REST Reports - Collection Report Screenshots with Annotated API Calls

Metadata Query Report



REST Reports - Metadata Query Screenshots with Annotated API Calls

Installation and Configuration

Installing in DSpace 6

This code is part of the DSpace 6 code base.

Disabling the REST Reports

The REST reports will be enabled by default in DSpace 6. To disable the execution of these reports, remove the following line from dspace-rest/src/main/webapp/WEB-INF/web.xml

Enable/disable report resources in the REST API

```
<servlet-mapping>
        <servlet-name>default</servlet-name>
        <url-pattern>/static/*</url-pattern>
    </servlet-mapping>
```

Configuring Access of the Reporting Tools

The reports can be configured with anonymous access or the reports can be configured to bypass authorization checks.

Bypassing authorization checks allows collection owners to view the status of all items in the repository without authenticating through the REST API. This option is recommended **if you have secured access to your REST API**.

If your REST API is publicly accessible, deploy the reports with anonymous access and consider providing an authorization token for access to the report calls.

Configure Authorization for REST Reports

```
# Enable/disable authorization for the reporting tools.
# By default, the DSpace REST API will only return communities/collections/items that are accessible to a
particular user.
# If the REST API has been deployed in a protected manner, the reporting tools can be configured to bypass
authorization checks.
# This will allow all items/collections/communities to be returned to the report user.
# Set the rest-reporting-authenticate option to false to bypass authorization
rest.reporting-authenticate = false
```

Configure the REST Reports that can be requested by name

```
# Configure the report pages that can be requested by name
# Create a map of named reports that are available to a report tool user
# Each map entry should be prefixed with rest-report-url
#   The map key is a name for a report
#   The map value is a URL to a report page
# A list of available reports will be available with the call /rest/reports.
# If a request is sent to /rest/reports/[report key], the request will be re-directed to the specified URL
#
# This project currently contains 2 sample reports.  Eventually, additional reports could be introduced through
this mechanism.
rest.report-url.collections = /rest/static/index.html
rest.report-url.item-query = /rest/static/query.html
```

Configure Item handle resolution

Enable the appropriate path to use to resolve an item handle **restReport.js**. (Depends on https://github.com/DSpace/DSpace/pull/1366/files)

Item Handle Resolution

```
this.ROOTPATH = "/handle/"
```

Enable User Authentication (Password AuthN only) for REST reports

Override the following function in your report file to enable/disable password AuthN for the REST reports. (Depends on https://github.com/DSpace/DSpace/pull/1369)

This setting can be found in **restReport.js**

Enable/Disable Password AuthN

```
//disable this setting if Password Authentication is not supported
this.makeAuthLink = function(){return true;};
```

Configure the database-specific format for a regex expression

```
# The REST Report Tools may pass a regular expression test to the database.
# The following configuration setting will construct a SQL regular expression test appropriate to your database
engine
rest.regex-clause = text_value ~ ?
```

Configure the sets of filters of interest to your repository managers

```
# A filter contains a set of tests that will be applied to an item to determine its inclusion in a particular
report.
# Private items and withdrawn items are frequently excluded from DSpace reports.
# Additional filters can be configured to examine other item properties.
# For instance, items containing an image bitstream often have different requirements from a item containing a
PDF.
# The DSpace REST reports come with a variety of filters that examine item properties, item bitstream
properties,
# and item authorization policies.  The existing filters can be used as an example to construct institution
specific filters
# that will test conformity to a set of institutional policies.
# plugin.sequence.org.dspace.rest.filter points to a list of classes that contain available filters.
# Each class must implement the ItemFilterList interface.
#   ItemFilterDefs:     Filters that examine simple item and bitstream type properties
#   ItemFilterDefsMisc: Filters that examine bitstream mime types and dependencies between bitstreams
#   ItemFilterDefsMeta: Filters that examine metadata properties
#   ItemFilterDefsPerm: Filters that examine item and bitstream authorization policies
plugin.sequence.org.dspace.rest.filter.ItemFilterList = \
        org.dspace.rest.filter.ItemFilterDefs,\
        org.dspace.rest.filter.ItemFilterDefsMisc,\
        org.dspace.rest.filter.ItemFilterDefsPerm

#     org.dspace.rest.filter.ItemFilterDefsMeta,\
```

Other filter configuration settings

The configuration file contains other settings that will control the behavior of the filters that you have enabled.

Enabling Sort-able Report Tables

1. Install sortable.js http://www.kryogenix.org/code/browser/sorttable/
2. Add to /dspace/modules/rest/src/main/webapp/static/reports
3. Include sortable.js in index.html and query.html

Uncomment the following in index.html and query.html

```
<!-- <script src="sorttable.js"></script> -->
```

4. Enable sortable in the report code in restCollReport.js and restQueryReport.js

CHANGE

```
var CollReport = function() {
    Report.call(this);
    //If sortable.js is included, uncomment the following
    //this.hasSorttable = function(){return true;}

var QueryReport = function() {
    Report.call(this);
    //If sortable.js is included, uncomment the following
    //this.hasSorttable = function(){return true;}
```

CHANGE TO

```
var CollReport = function() {
    Report.call(this);
    //If sortable.js is included, uncomment the following
    this.hasSorttable = function(){return true;}

var QueryReport = function() {
    Report.call(this);
    //If sortable.js is included, uncomment the following
    this.hasSorttable = function(){return true;}
```

Installing in DSpace 5

This feature is not a part of the DSpace 5 code base. Please see the following notes to enable a DSpace 5 compatible version of these reports.

1. Install https://github.com/DSpace/DSpace/pull/1568
2. Change the following code into restCollReport.js and restQuery.js to pull the correct id for each DSpace Object

Change the following in restCollReport.js and restQuery.js

```
var CollReport = function() {
Report.call(this);
var QueryReport = function() {
Report.call(this);
```

Change TO

```
var CollReport = function() {
Report.call(this);
this.getId = function(obj) {return obj.id;}
var QueryReport = function() {
Report.call(this);
this.getId = function(obj) {return obj.id;}
```

REST Reports - Collection Report Screenshots with Annotated API Calls

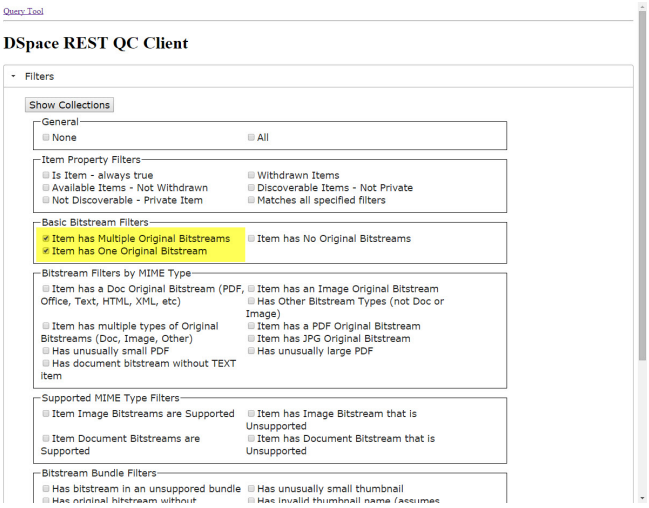
DSpace 7.0 only supports this when using the older, deprecated REST API v6

In DSpace 7.0, REST Quality Control Reports are currently only supported if you also install the old REST API v6 (deprecated) webapp. Tentative plans to migrate these reports to support the new REST API have begun in https://jira.lyrasis.org/browse/DS-4301.

Initial Report Display

API Call

/rest/filtered-collections?limit=25&expand=topCommunity&offset=0



Select Filters of Interest

API Call

/rest/filters

Query Tool

DSpace REST QC Client

Filters

Collection Report

URL to current search

Num	Community	Collection	Num Items	Item has Multiple Original Bitstreams	Item has One Original Bitstream	Matches all specified filters
			964	163	624	0
1	aaa	aaa	9	0	9	0
2	Bioethics Research Library	Acadia Institute Bioethics Interview Collection	2	0	2	0
3	Student Scholarship	African Studies Program Honors Theses	2	0	2	0
4	University Archives	AJCU Photographs Collection	2	0	2	0
5	Angelica: Art and Culture	Angelica (Digital Images Only)	50	0	14	0
6	Rare Books	Annotated Magna Cartas	2	0	2	0
7	Bioethics Research Library	Archival Collection Finding Aids	1	0	1	0
8	University Archives	Audio Archives	2	0	0	0
9	Bioethics Research Library	Bioethical Issues: Scope Notes Archive	2	1	1	0
10	Bioethics Research Library	BioethicsLine	2	0	0	0
11	Bioethics Research Library	Bioethics Newsletters Digital Archive	2	0	2	0
12	Bioethics Research Library	Bioethics Syllabus Exchange Repository	2	0	2	0
13	Student Scholarship	Carroll Round Proceedings	2	0	2	0

View Filtered Counts

API Call

/rest/filtered-collections/{collection_id}?limit=500&filters=has_multiple_originals.has_one_original

Query Tool

DSpace REST QC Client

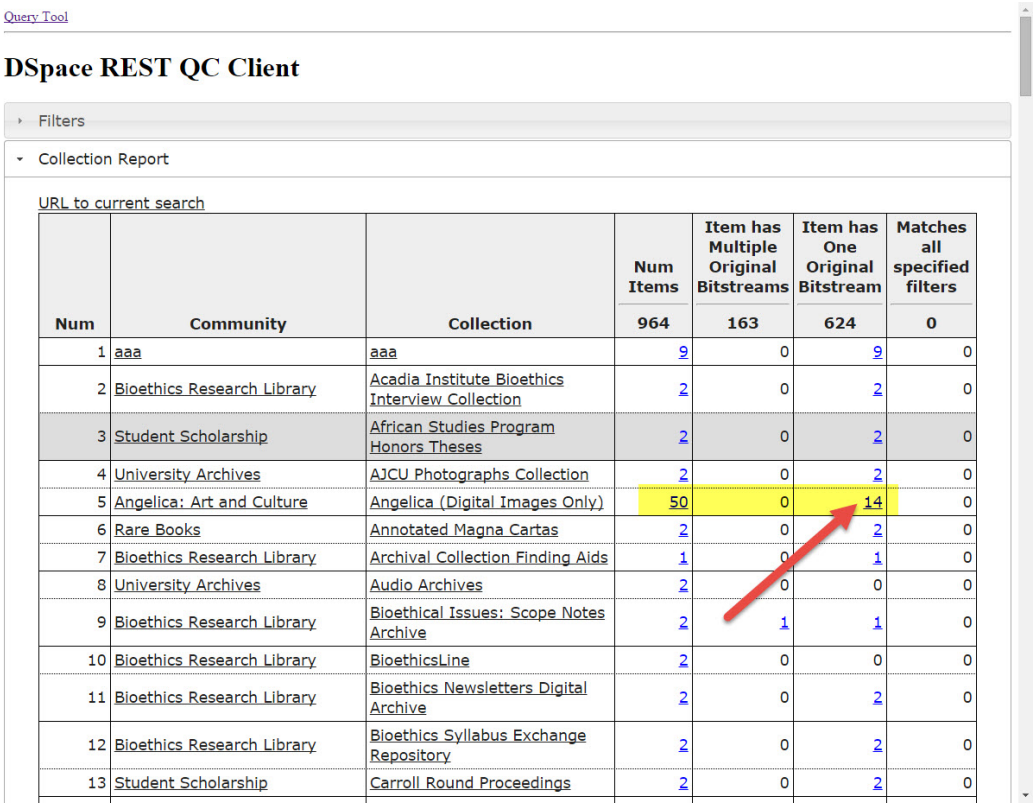
Filters

Collection Report

URL to current search

Num	Community	Collection	Num Items	Item has Multiple Original Bitstreams	Item has One Original Bitstream	Matches all specified filters
			964	163	624	0
1	aaa	aaa	9	0	9	0
2	Bioethics Research Library	Acadia Institute Bioethics Interview Collection	2	0	2	0
3	Student Scholarship	African Studies Program Honors Theses	2	0	2	0
4	University Archives	AJCU Photographs Collection	2	0	2	0
5	Angelica: Art and Culture	Angelica (Digital Images Only)	50	0	14	0
6	Rare Books	Annotated Magna Cartas	2	0	2	0
7	Bioethics Research Library	Archival Collection Finding Aids	1	0	1	0
8	University Archives	Audio Archives	2	0	0	0
9	Bioethics Research Library	Bioethical Issues: Scope Notes Archive	2	1	1	0
10	Bioethics Research Library	BioethicsLine	2	0	0	0
11	Bioethics Research Library	Bioethics Newsletters Digital Archive	2	0	2	0
12	Bioethics Research Library	Bioethics Syllabus Exchange Repository	2	0	2	0
13	Student Scholarship	Carroll Round Proceedings	2	0	2	0

View Items of Interest



Query Tool

DSpace REST QC Client

Filters

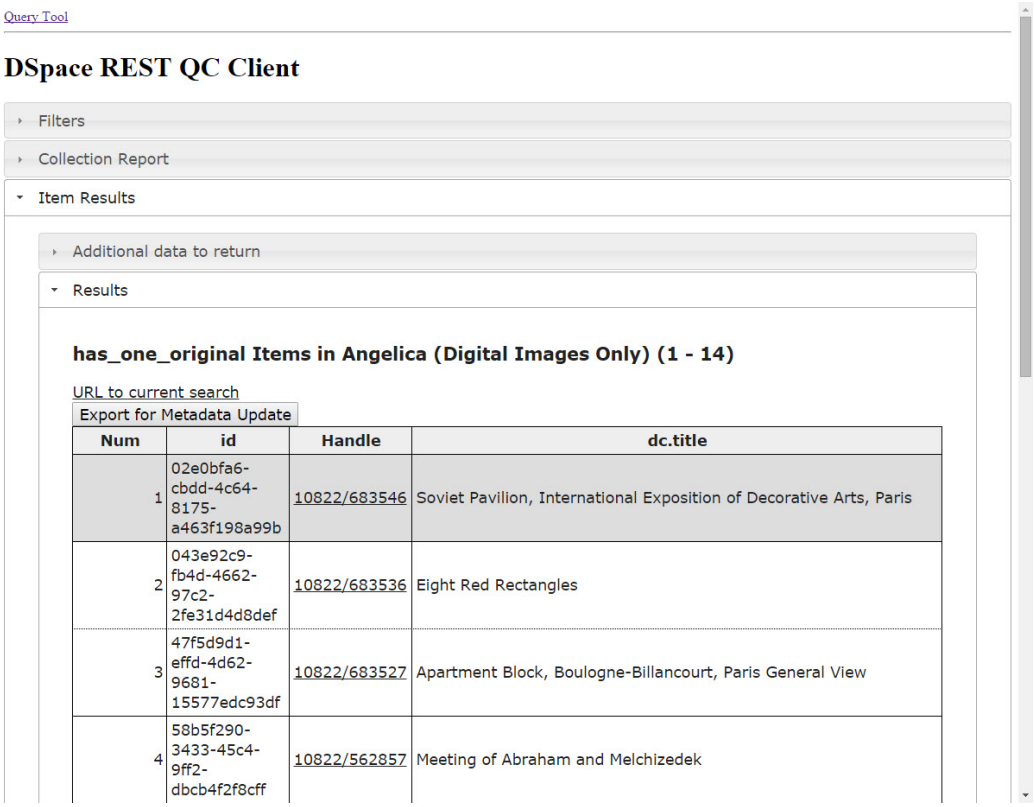
Collection Report

URL to current search

Num	Community	Collection	Num Items	Item has Multiple Original Bitstreams	Item has One Original Bitstream	Matches all specified filters
			964	163	624	0
1	aaa	aaa	9	0	9	0
2	Bioethics Research Library	Acadia Institute Bioethics Interview Collection	2	0	2	0
3	Student Scholarship	African Studies Program Honors Theses	2	0	2	0
4	University Archives	AJCU Photographs Collection	2	0	2	0
5	Angelica: Art and Culture	Angelica (Digital Images Only)	50	0	14	0
6	Rare Books	Annotated Magna Cartas	2	0	2	0
7	Bioethics Research Library	Archival Collection Finding Aids	1	0	1	0
8	University Archives	Audio Archives	2	0	0	0
9	Bioethics Research Library	Bioethical Issues: Scope Notes Archive	2	1	1	0
10	Bioethics Research Library	BioethicsLine	2	0	0	0
11	Bioethics Research Library	Bioethics Newsletters Digital Archive	2	0	2	0
12	Bioethics Research Library	Bioethics Syllabus Exchange Repository	2	0	2	0
13	Student Scholarship	Carroll Round Proceedings	2	0	2	0

API Call

/rest/filtered-collections/{collection_id}?expand=items&limit=100&filters=has_one_original&offset=0



Query Tool

DSpace REST QC Client

Filters

Collection Report

Item Results

Additional data to return

Results

has_one_original Items in Angelica (Digital Images Only) (1 - 14)

URL to current search

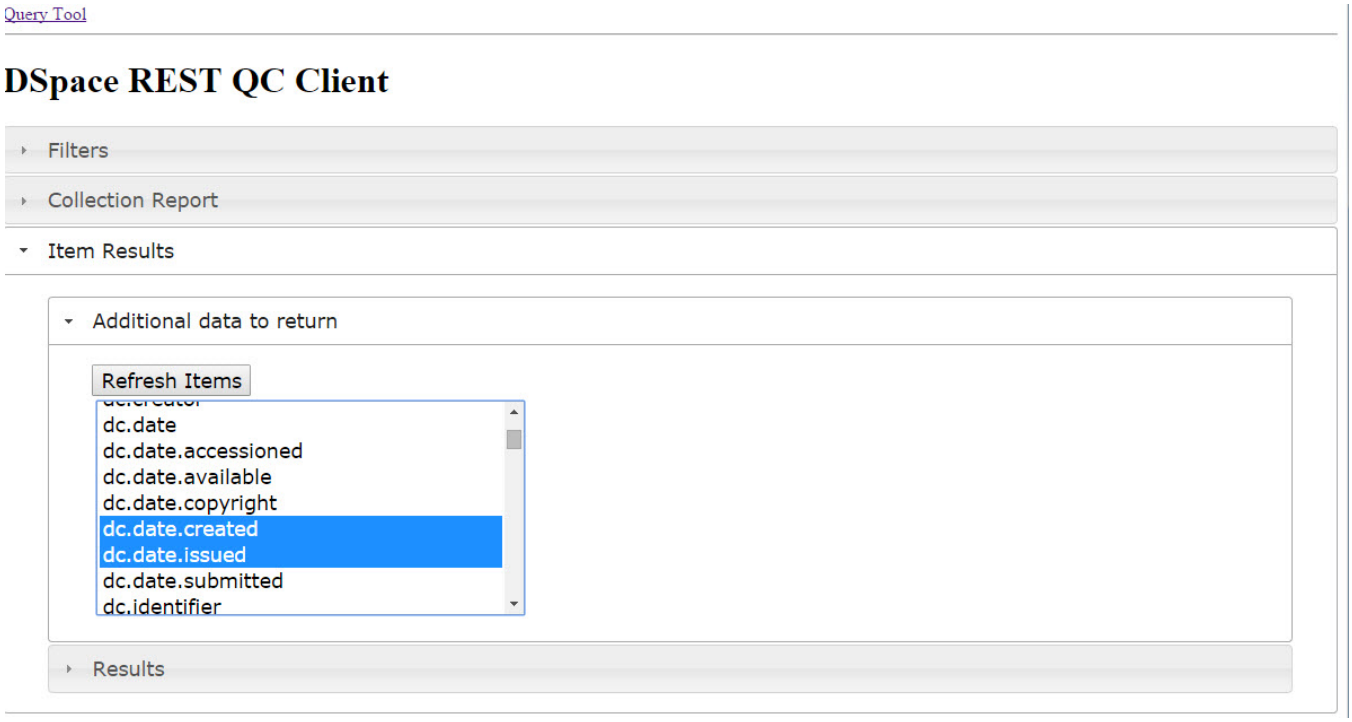
Export for Metadata Update

Num	id	Handle	dc.title
1	02e0bfa6-cbdd-4c64-8175-a463f198a99b	10822/683546	Soviet Pavilion, International Exposition of Decorative Arts, Paris
2	043e92c9-fb4d-4662-97c2-2fe31d4d8def	10822/683536	Eight Red Rectangles
3	47f5d9d1-effd-4d62-9681-15577edc93df	10822/683527	Apartment Block, Boulogne-Billancourt, Paris General View
4	58b5f290-3433-45c4-9ff2-dbcb4f2f8cff	10822/562857	Meeting of Abraham and Melchizedek

Select additional fields to display

API Call

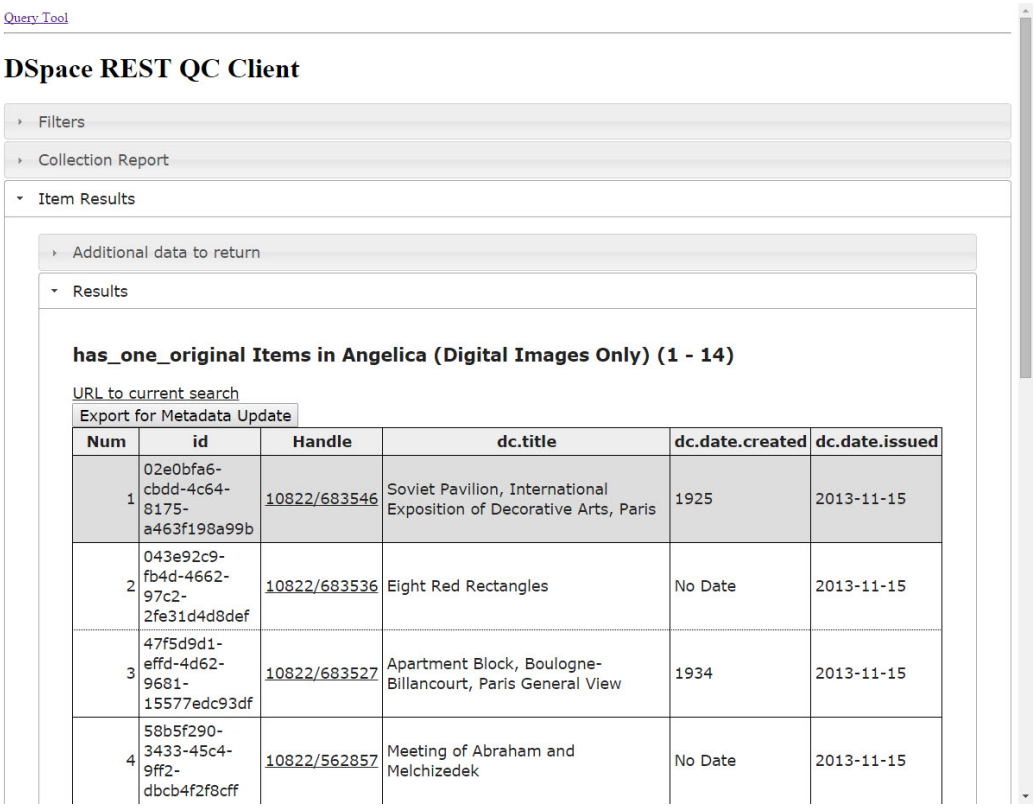
/rest/registries/schema



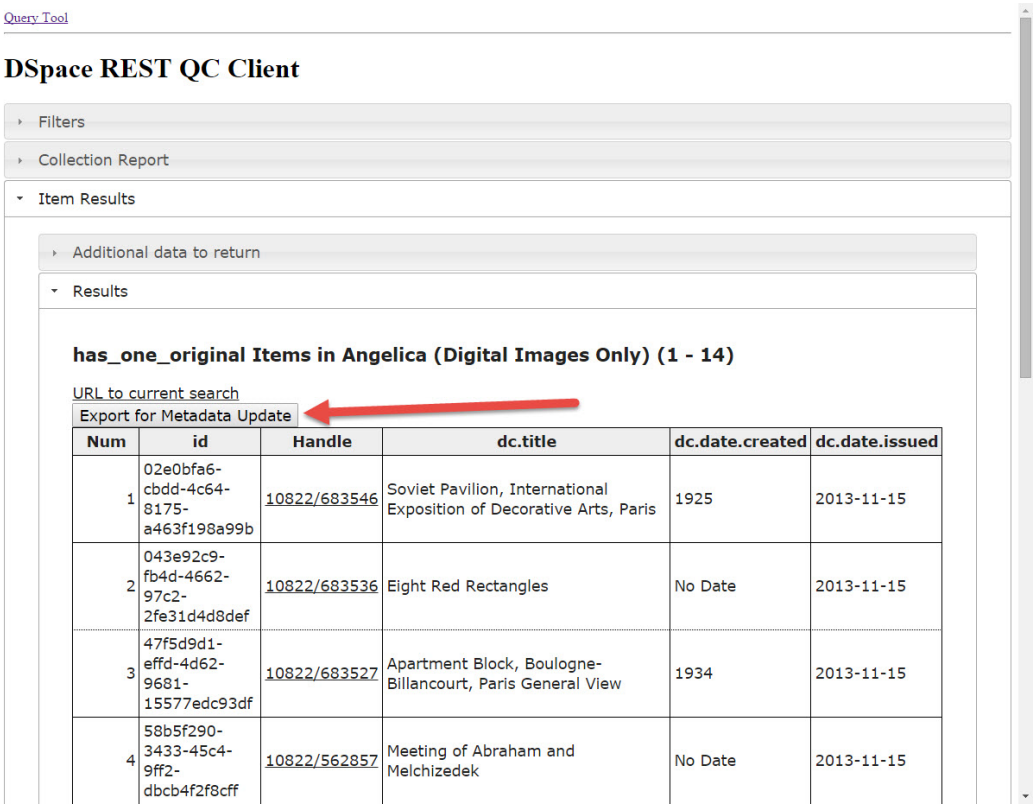
View Updated Results

API Call

/rest/filtered-collections/{collection_id}?expand=items,metadata&limit=100&filters=has_one_original&offset=0&show_fields[]=dc.date.created&show_fields[]=dc.date.issued



Download CSV File for Metadata Update



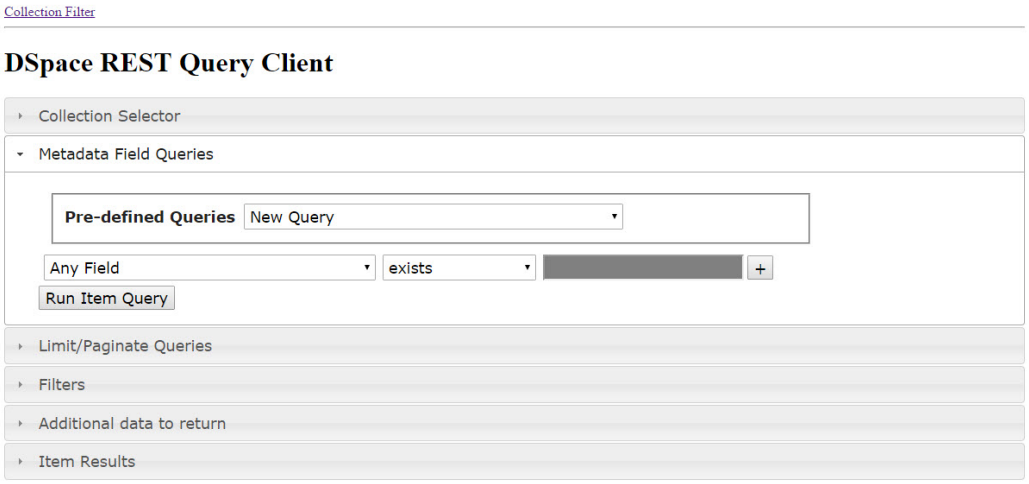
CSV File ready compatible with DSpace Metadata Update

```
"id","dc.title","dc.date.created","dc.date.issued"
"02e0bfa6-cbdd-4c64-8175-a463f198a99b","Soviet Pavilion, International Exposition of Decorative Arts, Paris","1925","2013-11-15"
"043e92c9-fb4d-4662-97c2-2fe31d4d8def","Eight Red Rectangles","No Date","2013-11-15"
"47f5d9d1-effd-4d62-9681-15577edc93df","Apartment Block, Boulogne-Billancourt, Paris General View","1934","2013-11-15"
"58b5f290-3433-45c4-9ff2-dbcb4f2f8cff","Meeting of Abraham and Melchizedek","No Date","2013-11-15"
"600a441e-fcd8-477b-b2a0-1b11eca97d35","Art Institute of Chicago Detail: Wing Detail: Wall MODEL","2005","2013-11-15"
"6401d1d0-3b67-46ab-892a-49549cd84fe3","Ascension","1801","2013-11-15"
"97c920e8-eebd-4548-95af-3c676d0f5c57","DuBarry Pavillion PLAN","1780","2013-11-15"
"a0e75f49-aa3a-4be7-9b13-48bad53a7a51","Saint Stefano Rotondo Detail: Entrance Portico","No Date","2013-11-15"
"b725a94f-5b86-472a-9a1c-58204f660703","Abbey du Thoronet PLAN","1146","2013-11-15"
"cd7dd096-5207-44e1-b480-f114a8ce8458","Soviet Pavilion, International Exposition of Decorative Arts, Paris","1925","2013-11-15"
"d44f90df-b20e-4ba5-a9c5-cd731868e79b","Mata Hari","1906","2013-11-15"
"dd74c102-4617-470f-ab42-c61caf230e88","Sistine Chapel Detail: Interior c1500","1473","2013-11-15"
"e5538842-a919-4ecc-ad16-2cf66993c031","Rheims Cathedral UNDER RESTORATION","1250","2013-11-15"
"eeeb25b9-463d-417b-9f2c-c791f78a69c9","Amiens Cathedral Notre-Dame PLAN","1220","2013-11-15"
```

REST Reports - Metadata Query Screenshots with Annotated API Calls

DSpace 7.0 only supports this when using the older, deprecated REST API v6

In DSpace 7.0, REST Quality Control Reports are currently only supported if you also install the old REST API v6 (deprecated) webapp. Tentative plans to migrate these reports to support the new REST API have begun in https://jira.lyrasis.org/browse/DS-4301.

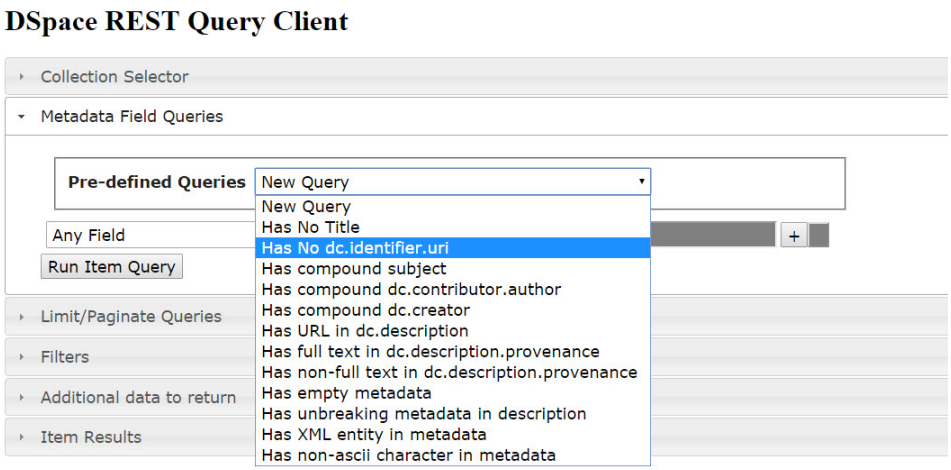


Set collections to Query

API Call

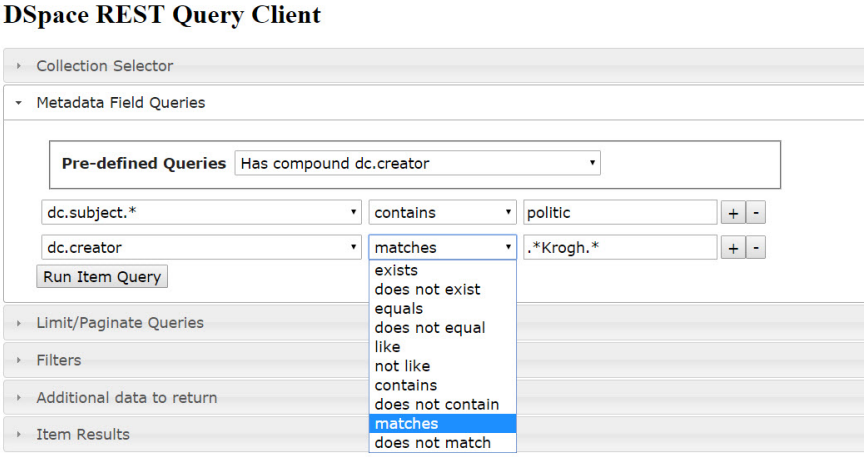
/rest/hierarchy

Pre-defined Queries are Available



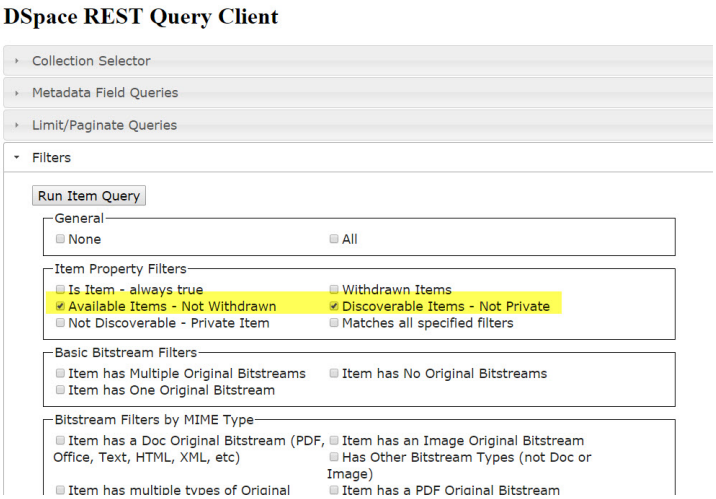
Multiple Metadata Fields can be Queried

API Call
/rest/registries/schema



Apply Filters if Desired

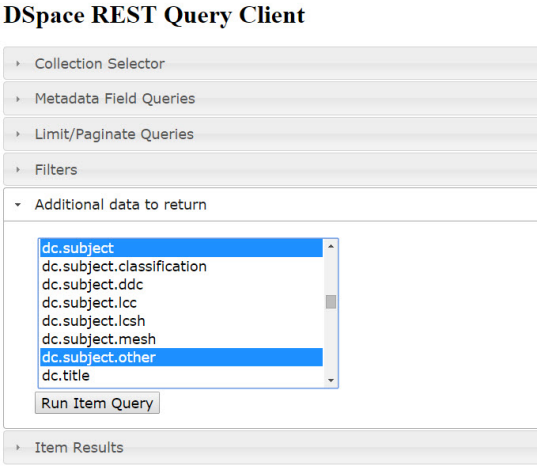
API Call
/rest/filters



Select Additional Fields to Display

API Call

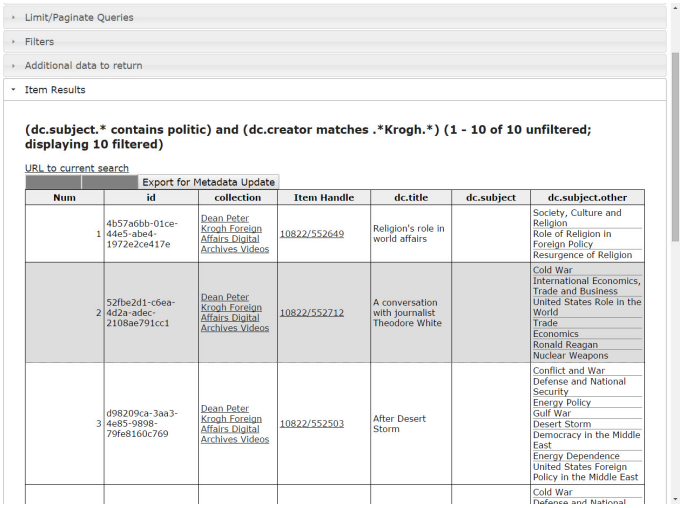
/rest/registries/schema



View Results

API Call

```
rest/filtered-items?

query_field[]=dc.subject.*&query_field[]=dc.creator&query_op[]=contains&query_op[]=matches&query_val[]
=politic&query_val[]=.*Krogh.*

&collSel[]=

&limit=100&offset=0

&expand=parentCollection,metadata

&filters=is_withdrawn,is_discoverable

&show_fields[]=dc.subject&show_fields[]=dc.subject.other
```



Limit/Paginate Queries

Filters

Additional data to return

Item Results

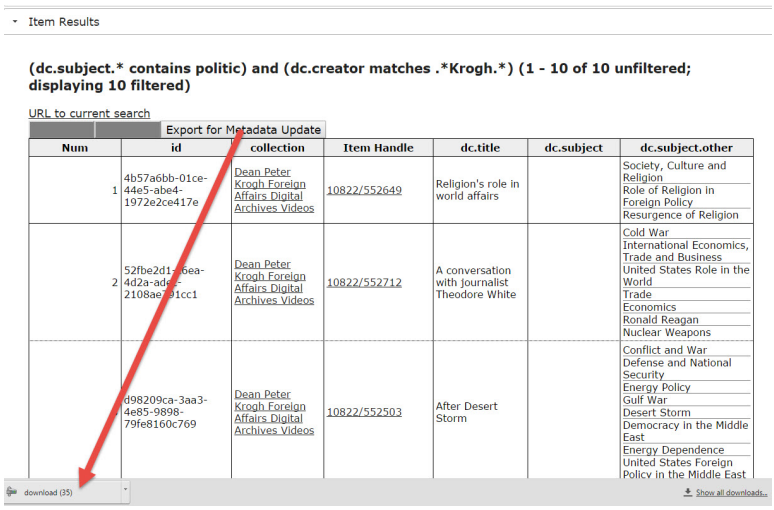
(dc.subject.* contains politic) and (dc.creator matches .*Krogh.*) (1 - 10 of 10 unfiltered; displaying 10 filtered)

URL to current search

Export for Metadata Update

Num	id	collection	Item Handle	dc.title	dc.subject	dc.subject.other
1	4b57a6bb-01ce-44e5-abe4-1972e2ce417e	Dean Peter Krogh Foreign Affairs Digital Archives Videos	10822/552649	Religion's role in world affairs		Society, Culture and Religion Role of Religion in Foreign Policy Resurgence of Religion
2	52fbe2d1-c6ea-4d2a-adec-2108ae791cc1	Dean Peter Krogh Foreign Affairs Digital Archives Videos	10822/552712	A conversation with journalist Theodore White		Cold War International Economics, Trade and Business United States Role in the World Trade Economics Ronald Reagan Nuclear Weapons
3	d98209ca-3aa3-4e85-9898-79fe8160c769	Dean Peter Krogh Foreign Affairs Digital Archives Videos	10822/552503	After Desert Storm		Conflict and War Defense and National Security Energy Policy Gulf War Desert Storm Democracy in the Middle East Energy Dependence United States Foreign Policy in the Middle East
						Cold War Defense and National

Export as CSV for DSpace Metadata Update Process



Item Results

(dc.subject.* contains politic) and (dc.creator matches .*Krogh.*) (1 - 10 of 10 unfiltered; displaying 10 filtered)

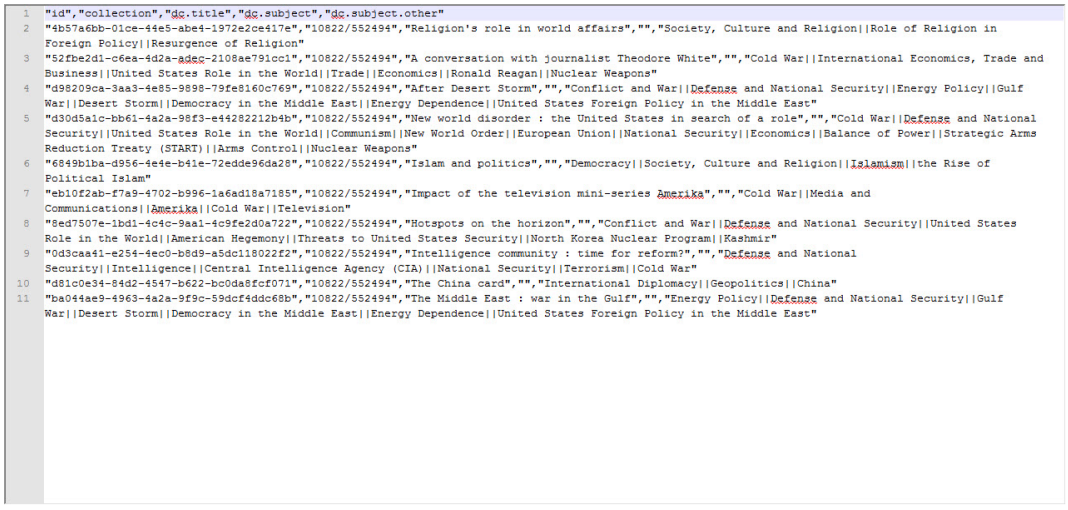
URL to current search

Export for Metadata Update

Num	id	collection	Item Handle	dc.title	dc.subject	dc.subject.other
1	4b57a6bb-01ce-44e5-abe4-1972e2ce417e	Dean Peter Krogh Foreign Affairs Digital Archives Videos	10822/552649	Religion's role in world affairs		Society, Culture and Religion Role of Religion in Foreign Policy Resurgence of Religion
2	52fbe2d1-c6ea-4d2a-adec-2108ae791cc1	Dean Peter Krogh Foreign Affairs Digital Archives Videos	10822/552712	A conversation with journalist Theodore White		Cold War International Economics, Trade and Business United States Role in the World Trade Economics Ronald Reagan Nuclear Weapons
3	d98209ca-3aa3-4e85-9898-79fe8160c769	Dean Peter Krogh Foreign Affairs Digital Archives Videos	10822/552503	After Desert Storm		Conflict and War Defense and National Security Energy Policy Gulf War Desert Storm Democracy in the Middle East Energy Dependence United States Foreign Policy in the Middle East

download (35)

Show all downloads...



```
"id","collection","dc.title","dc.subject","dc.subject.other"
"4b57a6bb-01ce-44e5-abe4-1972e2ce417e","10822/552494","Religion's role in world affairs","","Society, Culture and Religion||Role of Religion in Foreign Policy||Resurgence of Religion"
"52fbe2d1-c6ea-4d2a-adec-2108ae791cc1","10822/552494","A conversation with journalist Theodore White","","Cold War||International Economics, Trade and Business||United States Role in the World||Trade||Economics||Ronald Reagan||Nuclear Weapons"
"d98209ca-3aa3-4e85-9898-79fe8160c769","10822/552494","After Desert Storm","","Conflict and War||Defense and National Security||Energy Policy||Gulf War||Desert Storm||Democracy in the Middle East||Energy Dependence||United States Foreign Policy in the Middle East"
"d30d5a1c-bb61-4a2a-98f3-e44282212b4b","10822/552494","New world disorder : the United States in search of a role","","Cold War||Defense and National Security||United States Role in the World||Communism||New World Order||European Union||National Security||Economics||Balance of Power||Strategic Arms Reduction Treaty (START)||Arms Control||Nuclear Weapons"
"6849b1ba-d956-4e4e-b41e-72edde96da28","10822/552494","Islam and politics","","Democracy||Society, Culture and Religion||Islamism||the Rise of Political Islam"
"eb10f2ab-f7a9-4702-b996-1a6ad18a7185","10822/552494","Impact of the television mini-series Amerika","","Cold War||Media and Communications||Amerika||Cold War||Television"
"8ed7507e-1bd1-4c4c-9aa1-4c9fe2d0a722","10822/552494","Hotspots on the horizon","","Conflict and War||Defense and National Security||United States Role in the World||American Hegemony||Threats to United States Security||North Korea Nuclear Program||Kashmir"
"0d3caa41-e254-4ec0-b8d9-a5dc118022f2","10822/552494","Intelligence community : time for reform?","","Defense and National Security||Intelligence||Central Intelligence Agency (CIA)||National Security||Terrorism||Cold War"
"d81c0e34-84d2-4547-b622-bc0da8fcf071","10822/552494","The China card","","International Diplomacy||Geopolitics||China"
"ba044ae9-4963-4a2a-9f9c-59dcf4ddc68b","10822/552494","The Middle East : war in the Gulf","","Energy Policy||Defense and National Security||Gulf War||Desert Storm||Democracy in the Middle East||Energy Dependence||United States Foreign Policy in the Middle East"
```

REST Reports - Summary of API Calls

- GET /rest - Summary of API Calls
- GET /rest/reports - List of Available Reports
- GET /rest/reports/[report name] - Redirect to a Specific Report
- GET /rest/filters - Return filters to apply to a list of items
- GET /rest/filtered-collections - Return collections and item counts based on pre-defined filters
- GET /rest/filtered-collections/{collection_id} - Return items and item counts for a collection based on pre-defined filters
- GET /rest/filtered-items - Retrieve a set of items based on a metadata query and a set of filters

DSpace 7.x only supports this when using the older, deprecated REST API v6

In DSpace 7.0, REST Quality Control Reports are currently only supported if you also install the old REST API v6 (deprecated) webapp. Tentative plans to migrate these reports to support the new REST API have begun in https://github.com/DSpace/DSpace/issues/7641

GET /rest - Summary of API Calls

The response from this call includes the set of REST report calls that are available.

- GET /reports - Return a list of report tools built on the rest api
- GET /reports/{nickname} - Return a redirect to a specific report
- GET /filters - Return a list of use case filters available for quality control reporting
- GET /filtered-collections - Return collections and item counts based on pre-defined filters
- GET /filtered-collections/{collection_id} - Return items and item counts for a collection based on pre-defined filters
- GET /filtered-items - Retrieve a set of items based on a metadata query and a set of filters

GET /rest/reports - List of Available Reports

The response from this call includes the set of reports that are available

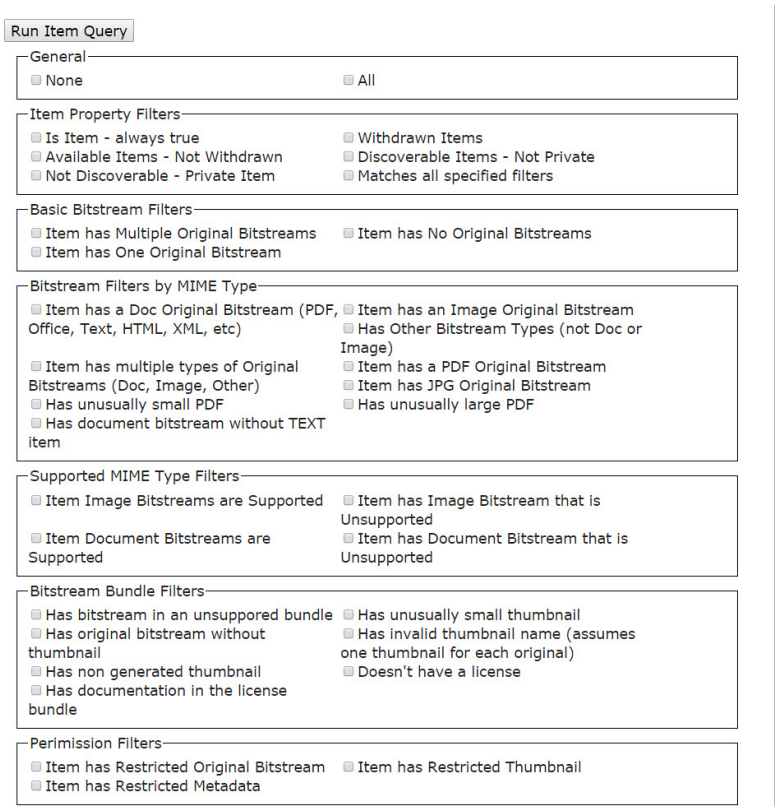
- **collection**: /rest/static/index.html
- **item-query**: /rest/static/query.html

GET /rest/reports/[report name] - Redirect to a Specific Report

This will re-direct to the configured report

GET /rest/filters - Return filters to apply to a list of items

The response will return the list of available filters. These have been configured in rest.cfg.



GET /rest/filtered-collections - Return collections and item counts based on pre-defined filters

This request is similar to the call /rest/collections except that it allows the user to supply a comma separated list of filters to apply to the collection.

The response will contain

- The total count of items for each collection
- The count of items that match each filter
- The count of items that match all filters combined

GET /rest/filtered-collections/{collection_id} - Return items and item counts for a collection based on pre-defined filters

This request is similar to the call /rest/collections/{collection_id} except that it allows the user to supply a comma separated list of filters to apply to the collection.

The response will contain

- The total count of items for the collection
- The count of items that match each filter
- The count of items that match all filters combined

When combined with the expand=items parameter, this call will return the set of items that match a filter or set of filters. It may be necessary to paginate through these results.

GET /rest/filtered-items - Retrieve a set of items based on a metadata query and a set of filters

This request allows a collection owner to construct a complex metadata query against specific metadata fields applying a number of comparison operators.

The search features of DSpace allow an end user to discover items via search. This command allows the collection owner to audit and enforce metadata consistency within a collection.

The query will consist of the following components

- Metadata Field Query

    - Field(s) to be searched
    - Search operator
    - Search value (if applicable)
- Collection scope (optional)
    - Comma separated list of collections to search
- Filters
    - A comma separated list of item filters that will be applied to all results.
    - It may be necessary to paginate through results when applying a highly selective filter

Advanced Customisation

If you are looking for ways to override specific classes or resources in DSpace (specifically in the backend), this page provides a guide for how to do so.

1 Additions module
2 Server Webapp Overlay
3 Rest (Deprecated) Webapp Overlay

Additions module

Location: `[dspace-source]/dspace/modules/additions/`

This module may be used to store dspace-api changes, custom plugins, etc. Classes placed in `[dspace-source]/dspace/modules/additions` will override those located in the `[dspace-source]/dspace-api`

This module may be used to override classes across *all* webapps located in `[dspace-source]/dspace/modules/` directory, as well as in the command line interface. Therefore, this modules is for global overrides only. If you have overrides specific to a single webapp, use the "Maven WAR Overlays" option below.

Server Webapp Overlay

Location: `[dspace-source]/dspace/modules/server/`

This module overlay directory allows you to override any classes, resources or files available (by default) in the Server Webapp. This includes overriding files of any of the following source directories:

- `[dspace-source]/dspace-oai/` (Bundled into the Server Webapp as a JAR)
- `[dspace-source]/dspace-rdf/` (Bundled into the Server Webapp as a JAR)
- `[dspace-source]/dspace-server-webapp/` (The Server Webapp itself)
- `[dspace-source]/dspace-sword/` (Bundled into the Server Webapp as a JAR)
- `[dspace-source]/dspace-swordv2/` (Bundled into the Server Webapp as a JAR)

Java classes place in `[dspace-source]/dspace/modules/server/` will override classes (of the same path/name) in any of the above modules.

You can also override resources (i.e. any files under a /src/main/resources/ directory) which are embedded in one of the JARs by putting them under `[dspace-source]/dspace/modules/server/src/main/resources/`. For example, to override the "`[dspace-source]/dspace-oai/src/main/resources/templates/index.twig.html`" file embedded in the `dspace-oai.jar`, you'd place your own version at `[dspace-source]/dspace/modules/server/src/main/resources/templates/index.twig.html`. This results in the resource/file being copied over into the `WEB-INF/classes/` subdirectory of the "server" webapp, and in that location it will override any file of the same name embedded in a JAR (per Servlet Spec 3.0).

Rest (Deprecated) Webapp Overlay

Location: `[dspace-source]/dspace/modules/rest`

If you have chosen to install the deprecated REST API v6 webapp, you can similar override any classes/files of that separate webapp by just placing those files in the `[dspace-source]/dspace/modules/rest/` directory

DSpace Service Manager

1 Introduction
2 Configuration
    2.1 Configuring Addons to Support Spring Services
    2.2 Configuration Priorities
        2.2.1 Configuring a new Addon
            2.2.1.1 Addon located as resource in jar
            2.2.1.2 Addon located in the [dspace]/config/spring directory
        2.2.2 The Core Spring Configuration
        2.2.3 Utilizing Autowiring to minimize configuration complexity.
    2.3 Accessing the Services Via Service Locator / Java Code
3 Architectural Overview
    3.1 Service Manager Startup in Webapplications and CLI
4 Tutorials

Introduction

The DSpace Spring Service Manager supports overriding configuration at many levels.

Configuration

Configuring Addons to Support Spring Services

Configuring Addons to support Spring happens at two levels. Default Spring configuration is available in the DSpace JAR or WAR resources directory and allows the addon developer to inject configuration into the service manager at load time. The second level is in the deployed [dspace]/config/spring directory where configurations can be provided on a addon module by addon module basis.

This latter method requires the addon to implement a SpringLoader to identify the location to look for Spring configuration and a place configuration files into that location. This can be seen inside the current [dspace-source]/config/modules/spring.cfg

Configuration Priorities

The ordering of the loading of Spring configuration is the following:

1. configPath = "spring/spring-dspace-applicationContext.xml" relative to the current classpath
2. addonResourcePath = "classpath*:spring/spring-dspace-addon-*-services.xml" relative to the current classpath
3. coreResourcePath = "classpath*:spring/spring-dspace-core-services.xml" relative to the current classpath
4. Finally, an array of SpringLoader API implementations that are checked to verify "config/spring/module" can actually be loaded by its existence on the classpath. The configuration of these SpringLoader API classes can be found in dspace.dir/config/modules/spring.cfg.

Configuring a new Addon

There are 2 ways to create a new Spring addon: a new Spring file can be located in the resources directory or in the configuration [dspace]/config/spring directory. A Spring file can also be located in both of these locations but the configuration directory gets preference and will override any configurations located in the resources directory.

Addon located as resource in jar

In the resources directory of a certain module, a Spring file can be added if it matches the following pattern: "spring/spring-dspace-addon-*-services.xml". An example of this can be found in the dspace-discovery-solr block in the DSpace trunk. (`spring-dspace-addon-discovery-services.xml`) Wherever this jar is loaded in a Maven module, the Spring files will be processed into services.

Addon located in the [dspace]/config/spring directory

This directory has the following subdirectories in which Spring files can be placed:

- api: when placed in this module the Spring files will always be processed into services (since all of the DSpace modules are dependent on the API).
- discovery: when placed in this module the Spring files will only be processed when the discovery library is present

The reason why there is a separate directory is that if a service cannot be loaded, the kernel will crash and DSpace will not start.

Configuring an additional subdirectory for a custom module

So you need to indeed create a new directory in [dspace]/config/spring. Next you need to create a class that inherits from the "org.dspace.kernel.config. SpringLoader". This class only contains one method named getResourcePaths(). What we do now at the moment is implement this in the following manner:

```
@Override
public String[] getResourcePaths(ConfigurationService configurationService) {
     StringBuffer filePath = new StringBuffer();
     filePath.append(configurationService.getProperty("dspace.dir"));
     filePath.append(File.separator);
     filePath.append("config");
     filePath.append(File.separator);
     filePath.append("spring");
     filePath.append(File.separator);
     filePath.append("{module.name}"); //Fill in the module name in this string
     filePath.append(File.separator);
     try {

          //By adding the XML_SUFFIX here it doesn't matter if there should be some kind of spring.xml.old file
in there it will only load in the active ones.
          return new String[]{new File(filePath.toString()).toURI().toURL().toString() + XML_SUFFIX};
      } catch (MalformedURLException e) {
          return new String[0];
      }
}
```

After the class has been created you will also need to add it to the "spring.springloader.modules" property located in the [dspace]/config/modules/spring.cfg.
The Spring service manager will check this property to ensure that only the interface implementations which it can find the class for are loaded in.

By doing this way we give some flexibility to the developers so that they can always create their own Spring modules and then Spring will not crash when it can't find a certain class.

The Core Spring Configuration

Utilizing Autowiring to minimize configuration complexity.

Please see the following tutorials:

- DSpace Spring Services Tutorial
- The TAO of DSpace Services

Accessing the Services Via Service Locator / Java Code

Please see the following tutorials:

- DSpace Spring Services Tutorial
- The TAO of DSpace Services

Architectural Overview

Please see Architectural Overview here: DSpace Services Framework

Service Manager Startup in Webapplications and CLI

Please see the DSpace Services Framework

Tutorials

Several good Spring / DSpace Services Tutorials are already available:

- DSpace Spring Services Tutorial
- The TAO of DSpace Services

Curation Tasks

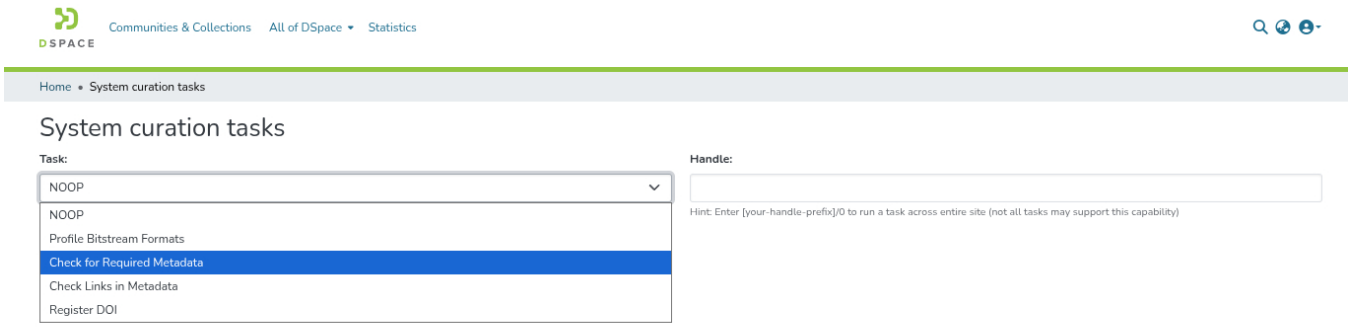
1 What is a Curation Task?
2 Curation Task Configuration (curate.cfg)
3 Writing your own tasks
4 Task Output and Reporting
    4.1 Status Code
    4.2 Result String
    4.3 Reporting Stream
    4.4 Accessing task output in calling code
5 Task Properties
6 Task Annotations
7 Scripted Tasks
    7.1 Interface
        7.1.1 performDso() vs. performId()

This documentation provides a guide for how to programmatically create Curation Tasks. For more information configuring Curation Tasks, see the Curation System section of the documentation

What is a Curation Task?

A **Curation Task** is a pluggable unit of backend logic in DSpace used to **inspect**, **validate**, **report on**, or **modify** repository content. Typically they are used for batch maintenance, metadata validation, content transformation, and repository-wide cleanup or enrichment operations. Tasks operate on DSpace objects (Items, Collections, Communities, or the entire Site) and can be executed from the **command line**, the **administrative UI**, or as a part of **automated workflows**.

Technically, a Curation Task is a Java class that implements `org.dspace.curate.CurationTask` (or `org.dspace.curate.ScriptedTask` for non-Java JVM implementations, see Curation System - Scripted Tasks for more information). Each task returns a standardized status code and may also produce result messages or detailed reports.



Curation task in the administrative UI of DSpace

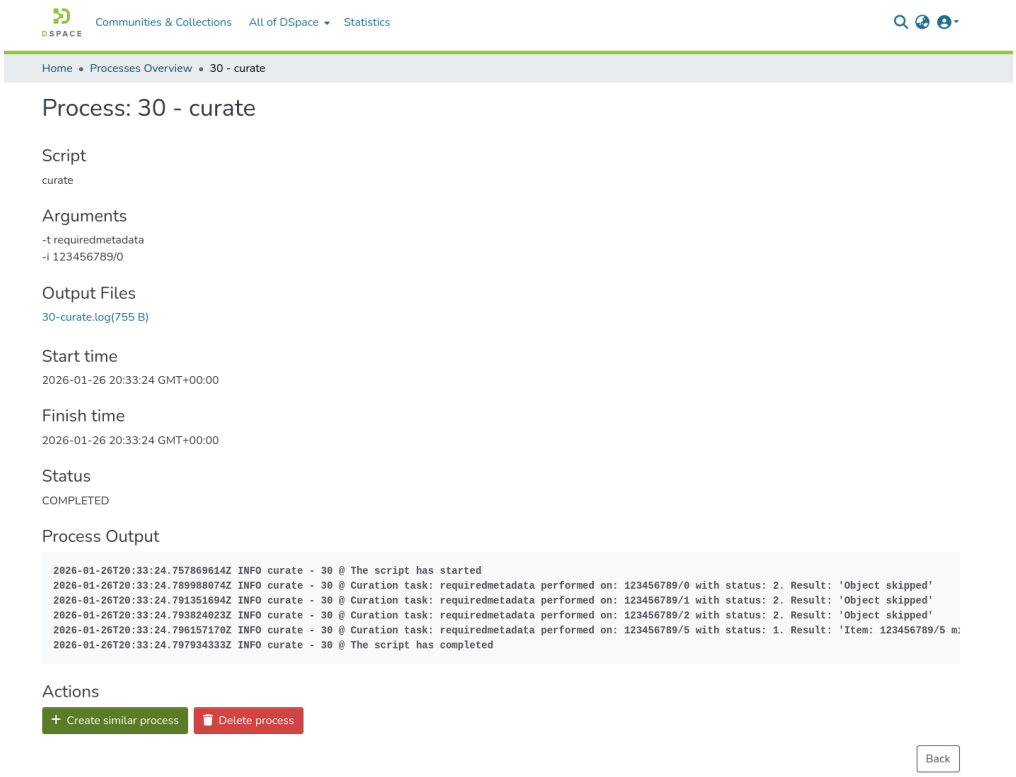
Curation Task Configuration (curate.cfg)

Available Curation Tasks are registered in DSpace backend's `[dspace]/config/modules/curate.cfg` using the DSpace plugin system:

```
plugin.named.org.dspace.curate.CurationTask = org.dspace.ctask.general.NoOpCurationTask = noop
plugin.named.org.dspace.curate.CurationTask = org.dspace.ctask.general.ProfileFormats = profileformats
#plugin.named.org.dspace.curate.CurationTask = org.dspace.ctask.general.ClamScan = vscan
#...
```

Only tasks listed and uncommented in this file are available for execution. Commenting out an entry disables it system-wide without requiring code changes.

The logical task name (e.g. "noop", "profileformats") is what administrators use when invoking the task via the UI or CLI; not the Java class name.



Writing your own tasks

A task is just a java class that can contain arbitrary code, but it must have 2 properties:

First, it must provide a no argument constructor, so it can be loaded by the PluginManager. Thus, all tasks are 'named' plugins, with the taskname being the plugin name.

Second, it must implement the interface `org.dspace.curate.CurationTask`

The `CurationTask` interface is almost a "tagging" interface, and only requires a few very high-level methods be implemented. The most significant is:

```
int perform(DSpaceObject dso);
```

The return value should be a code describing one of 4 conditions:

- 0 : SUCCESS the task completed successfully
- 1 : FAIL the task failed (it is up to the task to decide what 'counts' as failure - an example might be that the virus scan finds an infected file)
- 2 : SKIPPED the task could not be performed on the object, perhaps because it was not applicable
- -1 : ERROR the task could not be completed due to an error

If a task extends the `AbstractCurationTask` class, that is the only method it needs to define.

Task Output and Reporting

Few assumptions are made by CS about what the 'outcome' of a task may be (if any) - it. could e.g. produce a report to a temporary file, it could modify DSpace content silently, etc. But the CS runtime does provide a few pieces of information whenever a task is performed:

Status Code

This is returned to CS by any of a task's `perform` methods. The complete list of values, defined in `Curator`, is:

value	symbol	meaning
-3	CURATE_NOTASK	CS could not find the requested task

-2	CURATE_UNSET	task did not return a status code because it has not yet run
-1	CURATE_ERROR	task could not be performed
0	CURATE_SUCCESS	task performed successfully
1	CURATE_FAIL	task performed, but failed
2	CURATE_SKIP	task not performed due to object not being eligible

In the administrative UI, this code is translated into the word or phrase configured by the *ui.statusmessages* property (discussed in Curation System) for display.

Result String

The task may set a string indicating details of the outcome:

```
curator.setResult("Item " + item.getID() + " was painted " + color);
```

CS does not interpret or assign result strings; the task does it. A task may choose not to assign a result, but the "best practice" for tasks is to assign one whenever possible. Code which invokes `Curator.getResult()` may use the result string for display or any other purpose.

Reporting Stream

For very fine-grained information, a task may write to a *reporting* stream. Unlike the result string, there is no limit to the amount of data that may be pushed to this stream.

```
curator.report("Lorem ipsum dolor sit amet,\n");
curator.report("consectetur adipiscing elit,\n");
curator.report("sed do eiusmod tempor incididunt ut labore et dolore magna aliqua.\n");
```

Accessing task output in calling code

The status code, reporting stream, and the result string are accessed (or set) by methods on the Curator object:

```
Curator curator = new Curator();
curator.setReporter(new OutputStreamWriter(System.out));
curator.addTask("vscan").curate(coll);
int status = curator.getStatus("vscan");
String result = curator.getResult("vscan");
```

Task Properties

Task code may configure itself using ConfigurationService in the normal manner, or by the use of "task properties". See Curation System - Task Properties for discussion of the issues for which task properties were invented. Any code which extends AbstractCurationTask has access to its configured task properties.

The entire "API" for task properties is:

```
public String taskProperty(String name);
public int taskIntProperty(String name, int defaultValue);
public long taskLongProperty(String name, long defaultValue);
public boolean taskBooleanProperty(String name, boolean default);
```

Task Annotations

CS looks for, and will use, certain Java annotations in the task Class definition that can help it invoke tasks more intelligently. An example may explain best. Since tasks operate on DSOs that can either be simple (Items) or containers (Collections, and Communities), there is a fundamental problem or ambiguity in how a task is invoked: if the DSO is a collection, should the CS invoke the task on each member of the collection, or does the task "know" how to do that itself? The decision is made by looking for the @Distributive annotation: if present, CS assumes that the task will manage the details, otherwise CS will walk the collection, and invoke the task on each member. The java class would be defined:

```
@Distributive
public class MyTask implements CurationTask
```

A related issue concerns how non-distributive tasks report their status and results: the status will normally reflect only the last invocation of the task in the container, so important outcomes could be lost. If a task declares itself @Suspendable, however, the CS will cease processing when it encounters a FAIL status. When used in the UI, for example, this would mean that if our virus scan is running over a collection, it would stop and return status (and result) to the scene on the first infected item it encounters. You can even tune @Supendable tasks more precisely by annotating what invocations you want to suspend on. For example:

```
@Suspendable(invoked=Curator.Invoked.INTERACTIVE)
public class MyTask implements CurationTask
```

would mean that the task would suspend if invoked in the UI, but would run to completion if run on the command-line.

Only a few annotation types have been defined so far, but as the number of tasks grow, we can look for common behavior that can be signaled by annotation. For example, there is a @Mutative type: that tells CS that the task may alter (mutate) the object it is working on.

Scripted Tasks

DSpace 1.8 introduced limited (and somewhat experimental) support for deploying and running tasks written in languages other than Java. Since version 6, Java has provided a standard way (API) to invoke so-called scripting or dynamic language code that runs on the Java Virtual Machine (JVM). Scripted tasks are those written in a language accessible from this API. See Curation System - Scripted Tasks for information on configuring and running scripted tasks.

Interface

Scripted tasks must implement a slightly different interface than the **CurationTask** interface used for Java tasks. The appropriate interface for scripting tasks is **ScriptedTask** and has the following methods:

```
public void init(Curator curator, String taskId) throws IOException;
public int performDso(DSpaceObject dso) throws IOException;
public int performId(Context ctx, String id) throws IOException;
```

The difference is that `ScriptedTask` has separate `perform` methods for DSO and identifier. The reason for that is that some scripting languages (e.g. Ruby) don't support method overloading.

performDso() vs. performId()

You may have noticed that the `ScriptedTask` interface has both `performDso()` and `performId()` methods, but only performDso is ever called when curator is launched from command line.

There are a class of use-cases in which we want to construct or create new DSOs (DSpaceObject) given an identifier in a task. In these cases, there may be no live DSO to pass to the task.
You actually **can** get curation system to call `performId()` if you queue a task then process the queue - when reading the queue all CLI has is the handle to pass to the task.

Curation tasks in Jython

As mentioned in the "Scripted Tasks" chapter of Curation Tasks, you can write your curation tasks in several languages, including Jython (a flavour of Python running on JVM).

Instructions are outdated and unproven in DSpace 7.x

Setting up scripted tasks in Jython

1. Download the latest Jython installer jar (e.g. jython-installer-2.7.1.jar) from http://www.jython.org/downloads.html
2. Get `jython.jar` and the `Lib` directory.
    a. either unzip the installer jar:
    ```
    unzip -d [dspace]/lib/ jython-installer-2.7.1.jar jython.jar Lib/
    unzip -d [dspace]/webapps/server/WEB-INF/lib/ jython-installer-2.7.1.jar jython.jar Lib/
    ```
    b. or use it to install Jython:
    ```
    java -jar jython-installer-2.7.1.jar --console
    ```
    Note: Installation location doesn't matter, this is not necessary for DSpace. You can safely delete it after you retrieve `jython.jar` and `Lib`.
3. Install Jython to DSpace classpaths (step 2a already did this for you):
    a. The goal is to put `jython.jar` and the jython `Lib/` directory into **every** DSpace classpath you intend to use, so it must be installed in **both** `[dspace]/lib` and the webapp that deploys to Tomcat (if you want to run from the UI) - `[dspace]/webapps/server/WEB-INF/lib/`. There are no special maven/pom extensions - just copy in the jar and `Lib/`.
    b. You **can** use symlinks if you wish as long as allowLinking (Tomcat <=7, Tomcat 8) is set to true in that context's configuration. However, be warned that Tomcat documentation lists allowLinking="true" as a possible security concern.
    c. Note: Older versions of Jython mention the need for jython-engine.jar to implement JSR-223. Don't worry about that, new Jython versions, e.g. 2.7.1 don't require this.
4. Configure the curation framework to be aware of your new task(s):
    a. set up the location of scripted tasks in the curation system. This means simply adding a property to `[dspace]/config/modules/curate.cfg`:
    ```
    script.dir=${dspace.dir}/ctscripts
    ```
    b. in this directory, create a text file named "`task.catalog`". This is a Java properties file where lines beginning with '#' are comments. Add a line for each task you write. The syntax is following:
    ```
    # logical task name = script engine name|file name|constructor invocation
    mytask=python|mytask.py|MyTask()
    ```
    Notes:
        - **don't** put spaces around the pipe character or you'll get an error similar to this one:
        `ERROR org.dspace.curate.TaskResolver @ Script engine: 'python ' is not installed`
        - The "script engine name" is whatever name (or alias) jython registers in the JVM. You can use both "python" and "jython" as engine name (tested on jython 2.7.1).
        - The logical task name can't conflict with existing (java) task names, but otherwise any single-word token can be used.
        - The file name is just the script file name in the `script.dir` directory
        - "constructor invocation" is the language specific way to create an object that implements the task interface - it's ClassName() for Python
    c. If you want pretty names in the UI, configure other curate.cfg properties - see "`ui.tasknames`" (or groups etc)
5. Write your task.
    In the directory configured above, create your task (with the name configured in "`task.catalog`").
    The basic requirement of any scripted task is that it implements the ScriptedTask Java interface.
    So for our example, the `mytask.py` file might look like this:

```
from org.dspace.curate import ScriptedTask

class MyTask(ScriptedTask):
        def init(self, curator, taskName):
                print "initializing with Jython"

        def performDso(self, dso):
                print "perform on dso"
                return 0

        def performId(self, context, id):
                print "perform on id %s" % (id)
                return 0
```

6. Invoke the task.
   You can do this the same way you would invoke any task (from command line, in the admin UI, etc). The advantage of scripting is that you do not need to restart your servlet container to test changes; each task's source code is reloaded when you launch the task, so you can just put the updated script in place.
   Example of invocation from command line:

   ```
   [dspace]/bin/dspace curate -t mytask -i 123456789/123 -r -
   ```

   Note: "`-r -`" means that the script's standard output will be directed to the console. You can read more details in the "On the command line" chapter of the Curation Tasks page.

See also

- Curation Tasks page in the official documentation
- Nailgun - for speeding up repeated runs of a dspace command from the command line
  Note: since DSpace 4.0, there's a solution for running dspace CLI commands in batch: Executing streams of commands
- Jython webapp for DSpace - general purpose (not curation task) webapp written in Jython, optionally with access to DSpace API

Development Tools Provided by DSpace

Date parser tester

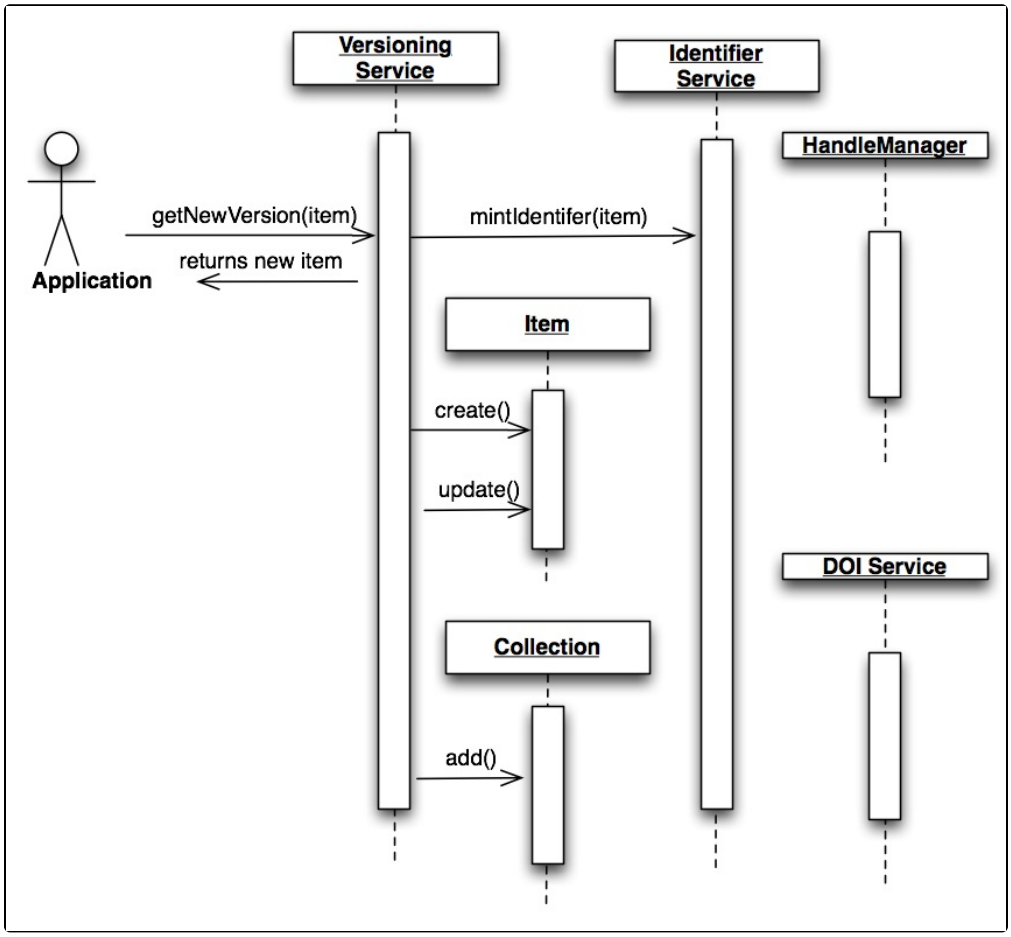
Some parts of DSpace use a custom date/time parser (`org.dspace.util.MultiFormatDateParser`) which is driven by a table of regular expressions, so it can match any of a variety of formats. The table is found in `config/spring/api/discovery-solr.xml`. To test new and altered rules, you can use the DSpace command line tool's `validate-date` command. You can simply pass it a date/time string on the command line (`dspace validate-date` 01-01-2015). You can pipe a stream of strings to be validated, one per line (`dspace validate-date < test.data`). Or you can have it prompt you for each string to be tested (`dspace validate-date`).

Services to support Alternative Identifiers

Together with the Item Level Versioning an Identifier Service was introduced that make it possible to integrate new Identifiers. Currently the Identifier Service is used for Items only, but this may be changed in future versions of DSpace. Identifiers used for different versions are an very important point as part of an versioning strategy. The following documentation describes the Identifier Service in the context of Item Level Versioning, nevertheless the Identifier Service is also used for Items when the Item Level Versioning is switched off.

Versioning and Identifier Service

DSpace Item Versioning is encapsulated as an Extensible Service that may be reimplemented by the local repository maintainers to produce alternate versioning behaviors and Identifier Schemes. Versioning Services layer on top of IdentifierServices dedicated to Encoding, Resolution, Minting and Registration of Identifiers for specific DSpace Items. It is through this highly extensible layering of functionality where local developers can alter the versioning behavior and introduce their own local enhancements. The DSpace Service Manager, based on the Spring Framework, provides the key leverage for this flexibility.



Versioning Service

The *Versioning* Service will be responsible for the replication of one or more Items when a new version is requested. The new version will not yet be preserved in the Repository, it will be preserved when the databases transactional window is completed, thus when errors arise in the *versioning* process, the database will be properly kept in its original state and the application will alert that an exception has occurred that is in need of correction.

The *Versioning* Service will rely on a generic *IdentifierService* that is described below for minting and registering any identifiers that are required to track the revision history of the Items.

```
public interface VersioningService {

    Version createNewVersion(Context c, int itemId);

    Version createNewVersion(Context c, int itemId, String summary);

    void removeVersion(Context c, int versionID);

    void removeVersion(Context c, Item item);

    Version getVersion(Context c, int versionID);

    Version restoreVersion(Context c, int versionID);

    Version restoreVersion(Context c, int versionID, String summary);

    VersionHistory findVersionHistory(Context c, int itemId);

    Version updateVersion(Context c, int itemId, String summary);

    Version getVersion(Context c, Item item);
}
```

Identifier Service

The Identifier Service maintains an extensible set of *IdentifierProvider* services that are responsible for two important activities in Identifier management:

1. Resolution: *IdentifierService* act in a manner similar to the existing HandleManager in DSpace, allowing for resolution of DSpace Items from provided identifiers.
2. Minting: Minting is the act of reserving and returning an identifier that may be used with a specific DSpaceObject.
3. Registering: Registering is the act of recording the existence of a minted identifier with an external persistent resolver service. These services may reside on the local machine (HandleManager) or exist as external services (PURL or EZID DOI registration services)

```
public interface IdentifierService {

    /**
     *
     * @param context
     * @param dso
     * @param identifier
     * @return
     */
    String lookup(Context context, DSpaceObject dso, Class<? extends Identifier> identifier);

    /**
     *
     * This will resolve a DSpaceObject based on a provided Identifier.  The Service will interrogate
the providers in
     * no particular order and return the first successful result discovered.  If no resolution is
successful,
     * the method will return null if no object is found.
     *
     * TODO: Verify null is returned.
     *
     * @param context
     * @param identifier
     * @return
     * @throws IdentifierNotFoundException
     * @throws IdentifierNotResolvableException
     */
    DSpaceObject resolve(Context context, String identifier) throws IdentifierNotFoundException,
IdentifierNotResolvableException;

    /**
     *
     * Reserves any identifiers necessary based on the capabilities of all providers in the service.
     *
     * @param context
     * @param dso
```

```
     * @throws org.dspace.authorize.AuthorizeException
     * @throws java.sql.SQLException
     * @throws IdentifierException
     */
    void reserve(Context context, DSpaceObject dso) throws AuthorizeException, SQLException,
IdentifierException;

    /**
     *
     * Used to Reserve a Specific Identifier (for example a Handle,  hdl:1234.5/6) The provider is
responsible for
     * Detecting and Processing the appropriate identifier, all Providers are interrogated, multiple
providers
     * can process the same identifier.
     *
     * @param context
     * @param dso
     * @param identifier
     * @throws org.dspace.authorize.AuthorizeException
     * @throws java.sql.SQLException
     * @throws IdentifierException
     */
    void reserve(Context context, DSpaceObject dso, String identifier) throws AuthorizeException,
SQLException, IdentifierException;

    /**
     *
     * @param context
     * @param dso
     * @return
     * @throws org.dspace.authorize.AuthorizeException
     * @throws java.sql.SQLException
     * @throws IdentifierException
     */
    void register(Context context, DSpaceObject dso) throws AuthorizeException, SQLException,
IdentifierException;

    /**
     *
     * Used to Register a Specific Identifier (for example a Handle,  hdl:1234.5/6) The provider is
responsible for
     * Detecting and Processing the appropriate identifier, all Providers are interrogated, multiple
providers
     * can process the same identifier.
     *
     * @param context
     * @param dso
     * @param identifier
     * @return
     * @throws org.dspace.authorize.AuthorizeException
     * @throws java.sql.SQLException
     * @throws IdentifierException
     */
    void register(Context context, DSpaceObject dso, String identifier) throws AuthorizeException,
SQLException, IdentifierException;

    /**
     * Delete (Unbind) all identifiers registered for a specific DSpace item. Identifiers are "unbound"
across
     * all providers in no particular order.
     *
     * @param context
     * @param dso
     * @throws org.dspace.authorize.AuthorizeException
     * @throws java.sql.SQLException
     * @throws IdentifierException
     */
    void delete(Context context, DSpaceObject dso) throws AuthorizeException, SQLException,
IdentifierException;

    /**
```

```
     * Used to Delete a Specific Identifier (for example a Handle,  hdl:1234.5/6) The provider is
responsible for
     * Detecting and Processing the appropriate identifier, all Providers are interrogated, multiple
providers
     * can process the same identifier.
     *
     * @param context
     * @param dso
     * @param identifier
     * @throws org.dspace.authorize.AuthorizeException
     * @throws java.sql.SQLException
     * @throws IdentifierException
     */
    void delete(Context context, DSpaceObject dso, String identifier) throws AuthorizeException,
SQLException, IdentifierException;

}
```

Batch Processing

In the current DSpace design, the database transactions are in most of the cases relatively long: from Context creation to the moment the Context is completed. Especially when doing batch processing, that transaction can become very long. The new data access layer introduced in DSpace 6 which is based on Hibernate has built-in cache and auto-update mechanisms. But these mechanisms do not work well with long transactions and even have an exponentially adverse-effect on performance.

Therefore we added a new method `enableBatchMode()` to the DSpace Context class which tells our database connection that we are going to do some batch processing. The database connection (Hibernate in our case) can then optimize itself to deal with a large number of inserts, updates and deletes. Hibernate will then not postpone update statements anymore which is better in the case of batch processing. The method `isBatchModeEnabled()` lets you check if the current Context is in "batch mode".

When dealing with a lot of records, it is also important to deal with the size of the (Hibernate) cache. A large cache can also lead to decreased performance and eventually to "out of memory" exceptions. To help developers to better manage the cache, a method `getCacheSize()` was added to the DSpace Context class that will give you the number of database records currently cached by the database connection. Another new method `uncacheEntity(ReloadableEntity entity)` will allow you to clear the cache (of a single object) and free up (heap) memory. The `uncacheEntity()` method may be used to immediately remove an object from heap memory once the batch processing is finished with it. Besides the `uncacheEntity()` method, the `commit()` method in the DSpace Context class will also clear the cache, flush all pending changes to the database and commit the current database transaction. The database changes will then be visible to other threads.

BUT `uncacheEntity()` and `commit()` come at a price. After calling this method all previously fetched entities (hibernate terminology for database record) are "detached" (pending changes are not tracked anymore) and cannot be combined with "attached" entities. If you change a value in a detached entity, Hibernate will not automatically push that change to the database. If you still want to change a value of a detached entity or if you want to use that entity in combination with attached entities (e.g. adding a bitstream to an item) after you have cleared the cache, you first have to reload that entity. Reloading means asking the database connection to re-add the entity from the database to the cache and get a new object reference to the required entity. From then on, it is important that you use that new object reference. To simplify the process of reloading detached entities, we've added a `reloadEntity(ReloadableEntity entity)` method to the DSpace Context class with a new interface `ReloadableEntity`. This method will give the user a new "attached" reference to the requested entity. All DSpace Objects and some extra classes implement the `ReloadableEntity` interface so that they can be easily reloaded.

Examples on how to use these new methods can be found in the `IndexClient` class. But to summarize, when batch processing it is important that:

1. You put the Context into batch processing mode using the method:

```
boolean originalMode = context.isBatchModeEnabled();
context.enableBatchMode(true);
```

2. Perform necessary batch operations, being careful to call `uncacheEntity()` whenever you complete operations on each object. Alternatively, you can `commit()` the context once the object cache reaches a particular size (see `getCacheSize()`). Remember, once an object is "uncached", you will have to reload it (see `reloadEntity()`) before you can work with it again:

```
final Iterator<Item> itemIterator = itemService.findByCollection(context, collection);

// Loop through all items
while (itemIterator.hasNext()) {
  // Get access to next Item
  Item item = itemIterator.next();

  ... do something with Item ...

  // To prevent memory issues, discard Item from the cache after processing
  context.uncacheEntity(item);
}

// Remember: calling commit() will decache all objects
context.commit();

// So, if you need to reuse your Collection *post* commit(), you'd have to reload it
Collection collection = context.reloadEntity(collection);
```

3. When you're finished with processing the records, you put the context back into its original mode:

```
context.enableBatchMode(originalMode);
```

Workflow

Configuration

Main workflow configuration

As of DSpace 7, the `workflow.xml` configuration file has been migrated to use Spring Bean syntax (instead of a custom XML format). The structure of this XML has changed. If you need help migrating your old `workflow.xml` file (which started with a `<wf-config>` tag) to the new format (using `<bean>` tags), an XSLT script is available: workflow-migration.xsl

The workflow main configuration can be found in the workflow.xml file, located in **`[dspace]/config/spring/api/workflow.xml`** . An example of this workflow configuration file can be found below.

```
<beans>
    <bean class="org.dspace.xmlworkflow.XmlWorkflowFactoryImpl">
        <property name="workflowMapping">
            <util:map>
                <entry key="defaultWorkflow" value-ref="defaultWorkflow"/>
<!--            <entry key="123456789/4" value-ref="selectSingleReviewer"/>-->

<!--            <entry key="123456789/5" value-ref="scoreReview"/>-->
            </util:map>
        </property>
    </bean>

    <!--Standard DSpace workflow-->
    <bean name="defaultWorkflow" class="org.dspace.xmlworkflow.state.Workflow">

        <property name="firstStep" ref="reviewstep"/>
        <property name="steps">
            <util:list>
                <ref bean="reviewstep"/>
                <ref bean="editstep"/>
                <ref bean="finaleditstep"/>
            </util:list>
        </property>
    </bean>

    <bean id="{workflow.id}"
          class="org.dspace.xmlworkflow.state.Workflow">
        <!-- Another workflow configuration-->
    </bean>

    <!-- Role beans.  See below. -->

    <!-- Step beans.  See below. -->

</beans>
```

workflowFactory bean (org.dspace.xmlworkflow.XmlWorkflowFactoryImpl)

The workflow map contains a mapping between collections in DSpace and a workflow configuration, and is defined by the `workflowMapping` property of the workflow factory. Similar to the configuration of the submission process, the mapping can be done based on the handle of the collection. The mapping with "defaultWorkflow" as the value for the collection mapping, will be used for the collections not occurring in other mapping tags. Each mapping is defined by a "`entry`" element with two attributes:

- key: can either be a collection handle or "defaultWorkflow"
- value-ref: the value of this attribute points to one of the workflow configurations defined by the "Workflow" beans

workflow beans (org.dspace.xmlworkflow.state.Workflow)

The workflow bean is a repeatable XML element and represents one workflow process. It requires the following:

- "`name`" attribute: a unique name used for the identification of the workflow and used in the workflow to collection mapping
- "`firstStep`" property: the identifier of the first step of the workflow. This step will be the entry point of this workflow-process. When a new item has been committed to a collection that uses this workflow, the step configured in the "`firstStep`" property will he the first step the item will go through.
- "`steps`" property: a list of all steps within this workflow (in the order they will be processed).

role beans (org.dspace.xmlworkflow.Role)

Each workflow step has defined "role" property. A role represents one or more existing DSpace EPersons or Groups and can be used to assign them to one or more steps in the workflow process. One role is represented by one "role" bean and has the following:

- "id" attribute: a unique identifier (in one workflow process) for the role
- "description" property: optional attribute to describe the role
- "scope" property: optional attribute that is used to find our group and must have one of the following values, which are defined as constant fields of org.dspace.xmlworkflow.Role.Scope:
    - COLLECTION: The collection value specifies that the group will be configured at the level of the collection. This type of groups is the same as the type that existed in the original workflow system. In case no value is specified for the scope attribute, the workflow framework assumes the role is a collection role.
    - REPOSITORY: The repository scope uses groups that are defined at repository level in DSpace. The name attribute should exactly match the name of a group in DSpace.
    - ITEM: The item scope assumes that a different action in the workflow will assign a number of EPersons or Groups to a specific workflow-item in order to perform a step. These assignees can be different for each workflow item.
- "name" property: The name specified in the name attribute of a role will be used to lookup an eperson group in DSpace. The lookup will depend on the scope specified in the "scope" attribute:
    - COLLECTION: The workflow framework will look for a group containing the name specified in the name attribute and the ID of the collection for which this role is used.
    - REPOSITORY: The workflow framework will look for a group with the same name as the name specified in the name attribute.
    - ITEM: in case the item scope is selected, the name of the role attribute is not required.

```
<bean id="reviewer" class="org.dspace.xmlworkflow.Role">
    <property name="scope" value="#{ T(org.dspace.xmlworkflow.Role.Scope).COLLECTION}"/>
    <property name="name" value="Reviewer"/>
    <property name="description" value="The people responsible for this step are able to edit the metadata of
incoming submissions, and then accept or reject them."/>
</bean>
```

step beans (org.dspace.xmlworkflow.state.Step)

The step element represents one step in the workflow process. A step represents a number of actions that must be executed by one specified role. In case no role attribute is specified, the workflow framework assumes that the DSpace system is responsible for the execution of the step and that no user interface will be available for each of the actions in this step. The step element has the following in order to further configure it:

- "name" attribute: The name attribute specifies a unique identifier for the step. This identifier will be used when configuring other steps in order to point to this step. This identifier can also be used when configuring the start step of the workflow item.
- "userSelectionMethod" property: This attribute defines the UserSelectionAction that will be used to determine how to attach users to this step for a workflow-item. The value of this attribute must refer to the identifier of an action bean in the workflow-actions.xml. Examples of the user attachment to a step are the currently used system of a task pool or as an alternative directly assigning a user to a task.
- "role" property: optional attribute that must point to the id attribute of a role element specified for the workflow. This role will be used to define the epersons and groups used by the userSelectionMethod.
- RequiredUsers

```
<bean name="reviewstep" class="org.dspace.xmlworkflow.state.Step">
    <property name="userSelectionMethod" ref="claimaction"/>
    <property name="role" ref="reviewer"/>
    <property name="outcomes">
        <util:map>
            <entry key="#{ T(org.dspace.xmlworkflow.state.actions.ActionResult).OUTCOME_COMPLETE}"

                   value-ref="editstep"/>
        </util:map>
    </property>
    <property name="actions">
        <util:list>
            <ref bean="reviewaction"/>
        </util:list>
    </property>
</bean>
```

Each step contains a number of actions that the workflow item will go through. In case the action has a user interface, the users responsible for the exectution of this step will have to execute these actions before the workflow item can proceed to the next action or the end of the step.

There is also an optional subsection that can be defined for a step part called "outcomes". This can be used to define outcomes for the step that differ from the one specified in the nextStep attribute. Each action returns an integer depending on the result of the action. The default value is "0" and will make the workflow item proceed to the next action or to the end of the step.
In case an action returns a different outcome than the default "0", the alternative outcomes will be used to lookup the next step. The "outcomes" element contains a number of steps, each having a status attribute. This status attribute defines the return value of an action. The value of the element will be used to lookup the next step the workflow item will go through in case an action returns that specified status.

Workflow actions configuration

API configuration

The workflow actions configuration is located in the [dspace]/config/spring/api/ directory and is named "workflow-actions.xml". This configuration file describes the different Action Java classes that are used by the workflow framework. Because the workflow framework uses Spring framework for loading these action classes, this configuration file contains Spring configuration.

This file contains the beans for the actions and user selection methods referred to in the `workflow.xml`. In order for the workflow framework to work properly, each of the required actions must be part of this configuration.

```
<beans
    xmlns="http://www.springframework.org/schema/beans"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xmlns:util="http://www.springframework.org/schema/util"
    xsi:schemaLocation="http://www.springframework.org/schema/beans http://www.springframework.org/schema/beans
/spring-beans-2.0.xsd
                        http://www.springframework.org/schema/util http://www.springframework.org/schema/util
/spring-util-2.0.xsd">

    <!-- At the top are our bean class identifiers --->
    <bean id="{action.api.id}" class="{class.path}" scope="prototype"/>
    <bean id="{action.api.id.2}" class="{class.path}" scope="prototype"/>

     <!-- Below the class identifiers come the declarations for out actions/userSelectionMethods -->

     <!-- Use class workflowActionConfig for an action -->
     <bean id="{action.id}" class="oorg.dspace.xmlworkflow.state.actions.WorkflowActionConfig" scope="prototype">
         <constructor-arg type="java.lang.String" value="{action.id}"/>

         <property name="processingAction" ref="{action.api.id}"/>
         <property name="requiresUI" value="{true/false}"/>
     </bean>

     <!-- Use class UserSelectionActionConfig for a user selection method -->
     <!--User selection actions-->
     <bean id="{action.api.id.2}" class="org.dspace.xmlworkflow.state.actions.UserSelectionActionConfig" scope="pr
ototype">
         <constructor-arg type="java.lang.String" value="{action.api.id.2}"/>

         <property name="processingAction" ref="{user.selection.bean.id}"/>
         <property name="requiresUI" value="{true/false}"/>
     </bean>
</beans>
```

Two types of actions are configured in this Spring configuration file:

- User selection action: This type of action is always the first action of a step and is responsible for the user selection process of that step. In case a step has no role attached, no user will be selected and the `NoUserSelectionAction` is used.
- Processing action: This type of action is used for the actual processing of a step. Processing actions contain the logic required to execute the required operations in each step. Multiple processing actions can be defined in one step. These user and the workflow item will go through these actions in the order they are specified in the workflow configuration unless an alternative outcome is returned by one of them.

User Selection Action

Each user selection action that is used in the workflow configuration refers to a bean definition in the `workflow-actions.xml` file. In order to define a new user selection action, the following XML code is used:

```
<bean id="{action.api.id.2}" class="org.dspace.xmlworkflow.state.actions.UserSelectionActionConfig" scope="prototy
pe">
    <constructor-arg type="java.lang.String" value="{action.api.id.2}"/>

    <property name="processingAction" ref="{user.selection.bean.id}"/>
    <property name="requiresUI" value="{true/false}"/>
</bean>
```

This bean defines a new UserSelectionActionConfig and the following child tags:

- constructor-arg: This is a constructor argument containing the ID of the task. This is the same as the `id` attribute of the bean and is used by the workflow configuration to refer to this action.
- property `processingAction`: This tag refers the the ID of the API bean, responsible for the implementation of the API side of this action. This bean should also be configured in this XML.
- property `requiresUI`: In case this property is true, the workflow framework will expect a user interface for the action. Otherwise the framework will automatically execute the action and proceed to the next one.

Processing Action

Processing actions are configured similarly to the user selection actions. The only difference is that these processing action beans are implementations of the WorkflowActionConfig class instead of the UserSelectionActionConfig class.

Authorizations

Currently, the authorizations are always granted and revoked based on the tasks that are available for certain users and groups. The types of authorization policies that is granted for each of these is always the same:

- READ
- WRITE
- ADD
- DELETE

Database

The workflow uses a separate metadata schema named `workflow`. The fields this schema contains can be found in the `[dspace]/config/registries` directory and in the file `workflow-types.xml`. This schema is only used when using the score reviewing system at the moment, but one could always use this schema if metadata is required for custom workflow steps.

The following tables have been added to the DSpace database. All tables are prefixed with 'cwf_' to avoid any confusion with the existing workflow related database tables:

cwf_workflowitem

The cwf_workflowitem table contains the different workflowitems in the workflow. This table has the following columns:

- workflowitem_id: The identifier of the workflowitem and primary key of this table
- item_id: The identifier of the DSpace item to which this workflowitem refers.
- collection_id: The collection to which this workflowitem is submitted.
- multiple_titles: Specifies whether the submission has multiple titles (important for submission steps)
- published_before: Specifies whether the submission has been published before (important for submission steps)
- multiple_files: Specifies whether the submission has multiple files attached (important for submission steps)

cwf_collectionrole

The cwf_collectionrole table represents a workflow role for one collection. This type of role is the same as the roles that existed in the original workflow meaning that for each collection a separate group is defined to described the role. The cwf_collectionrole table has the following columns:

- collectionrol_id: The identifier of the collectionrole and the primaty key of this table
- role_id: The identifier/name used by the workflow configuration to refer to the collectionrole
- collection_id: The collection identifier for which this collectionrole has been defined
- group_id: The group identifier of the group that defines the collection role

cwf_workflowitemrole

The cwf_workflowitemrole table represents roles that are defined at the level of an item. These roles are temporary roles and only exist during the execution of the workflow for that specific item. Once the item is archived, the workflowitemrole is deleted. Multiple rows can exist for one workflowitem with e.g. one row containing a group and a few containing epersons. All these rows together make up the workflowitemrole The cwf_workflowitemrole table has the following columns:

- workflowitemrole_id: The identifier of the workflowitemrole and the primaty key of this table
- role_id: The identifier/name used by the workflow configuration to refer to the workflowitemrole
- workflowitem_id: The cwf_workflowitem identifier for which this workflowitemrole has been defined
- group_id: The group identifier of the group that defines the workflowitemrole role
- eperson_id: The eperson identifier of the eperson that defines the workflowitemrole role

cwf_pooltask

The cwf_pooltask table represents the different task pools that exist for a workflowitem. These task pools can be available at the beginning of a step and contain all the users that are allowed to claim a task in this step. Multiple rows can exist for one task pool containing multiple groups and epersons. The cwf_pooltask table has the following columns:

- pooltask_id: The identifier of the pooltask and the primaty key of this table
- workflowitem_id: The identifier of the workflowitem for which this task pool exists
- workflow_id: The identifier of the workflow configuration used for this workflowitem
- step_id: The identifier of the step for which this task pool was created
- action_id: The identifier of the action that needs to be displayed/executed when the user selects the task from the task pool
- eperson_id: The identifier of an eperson that is part of the task pool
- group_id: The identifier of a group that is part of the task pool

cwf_claimtask

The cwf_claimtask table represents a task that has been claimed by a user. Claimed tasks can be assigned to users or can be the result of a claim from the task pool. Because a step can contain multiple actions, the claimed task defines the action at which the user has arrived in a particular step. This makes it possible to stop working halfway the step and continue later. The cwf_claimtask table contains the following columns:

- claimtask_id: The identifier of the claimtask and the primary key of this table
- workflowitem_id: The identifier of the workflowitem for which this task exists

- workflow_id: The id of the workflow configuration that was used for this workflowitem
- step_id: The step that is currenlty processing the workflowitem
- action_id: The action that should be executed by the owner of this claimtask
- owner_id: References the eperson that is responsible for the execution of this task

cwf_in_progress_user

The cwf_in_progess_user table keeps track of the different users that are performing a certain step. This table is used because some steps might require multiple users to perform the step before the workflowitem can proceed. The cwf_in_progress_user table contains the following columns:

- in_progress_user_id: The identifier of the in progress user and the primary key of this table
- workflowitem_id: The identifier of the workflowitem for which the user is performing or has performed the step.
- user_id: The identifier of the eperson that is performing or has performe the task
- finished: Keeps track of the fact that the user has finished the step or is still in progress of the execution

DSpace Reference

- Configuration Reference
- DSpace Item State Definitions
- Directories and Files
- Metadata and Bitstream Format Registries
- Architecture
  - Application Layer
  - Business Logic Layer
  - DSpace Services Framework
  - Storage Layer
- History
  - Changes in 7.x
  - Changes in Older Releases

Configuration Reference

There are a numbers of ways in which DSpace may be configured and/or customized. This chapter of the documentation will discuss the configuration of the software and will also reference customizations that may be performed in the chapter following.

For ease of use, the Configuration documentation is broken into several parts:

- General Configuration - addresses general conventions used with configuring the `local.cfg` file, `dspace.cfg` and other configuration files which use similar conventions.
- The local.cfg Configuration Properties File - describes how to use the `local.cfg` file to store all your locally customized configurations
- The dspace.cfg Configuration Properties File - specifies the basic `dspace.cfg` file settings (these settings specify the default configuration for DSpace)
- Optional or Advanced Configuration Settings - contain other more advanced settings that are optional in the `dspace.cfg` configuration file.

The full table of contents follows:

1 General Configuration
- 1.1 Configuration File Syntax
  - 1.1.1 Special Characters
  - 1.1.2 Specifying Multiple Values for Properties
  - 1.1.3 Including other Property Files
- 1.2 Configuration Scheme for Reloading and Overriding
- 1.3 Why are there multiple copies of some config files?

2 The local.cfg Configuration Properties File

3 The dspace.cfg Configuration Properties File
- 3.1 Main DSpace Configurations
- 3.2 General Solr Configuration
- 3.3 DSpace Database Configuration
  - 3.3.1 To provide the database connection pool externally
- 3.4 DSpace Email Settings
  - 3.4.1 Wording of E-mail Messages
    - 3.4.1.1 Templates can set message headers
- 3.5 File Storage
- 3.6 Logging Configuration
- 3.7 General Plugin Configuration
- 3.8 Configuring the Search Engine
- 3.9 Handle Server Configuration
- 3.10 Delegation Administration: Authorization System Configuration
- 3.11 Inheritance of collection default policy (since 7.1)
- 3.12 Login as feature
- 3.13 Restricted Item Visibility Settings
- 3.14 Restricted Bundle Settings
- 3.15 Proxy Settings
- 3.16 Configuring Media Filters
- 3.17 Crosswalk and Packager Plugin Settings
  - 3.17.1 Configurable MODS Dissemination Crosswalk
  - 3.17.2 XSLT-based Crosswalks
    - 3.17.2.1 Testing XSLT Crosswalks
  - 3.17.3 Configurable Qualified Dublin Core (QDC) dissemination crosswalk
  - 3.17.4 Configuring Crosswalk Plugins
  - 3.17.5 Configuring Packager Plugins
- 3.18 Event System Configuration
- 3.19 Embargo
- 3.20 Checksum Checker Settings
- 3.21 Item Export and Download Settings
- 3.22 Subscription Emails
- 3.23 Hiding Metadata
- 3.24 Settings for the Submission Process
- 3.25 Configuring the Sherpa/RoMEO Integration
- 3.26 Configuring Creative Commons License
- 3.27 WEB User Interface Configurations
- 3.28 Item Counts in user interface
- 3.29 Browse Index Configuration
  - 3.29.1 Defining the storage of the Browse Data
  - 3.29.2 Defining the Indexes
  - 3.29.3 Defining Sort Options
  - 3.29.4 Hierarchical Browse Indexes
  - 3.29.5 Other Browse Options
  - 3.29.6 Browse Index Authority Control Configuration
  - 3.29.7 Tag cloud
- 3.30 Links to Other Browse Contexts
- 3.31 Submission License Substitution Variables
- 3.32 Syndication Feed (RSS) Settings
- 3.33 OpenSearch Support
- 3.34 Content Inline Disposition Threshold / Format
- 3.35 Multi-file HTML Document/Site Settings
- 3.36 Sitemap Settings

3.37 Authority Control Settings
3.38 Configuring Multilingual Support
3.38.1 Setting the Default Language for the Application
3.38.2 Supporting More Than One Language
3.38.2.1 Changes in dspace.cfg
3.38.2.2 Related Files
3.39 Upload File Settings
3.40 SFX Server (OpenURL)
3.41 Controlled Vocabulary Settings
4 Optional or Advanced Configuration Settings
4.1 The Metadata Format and Bitstream Format Registries
4.1.1 Metadata Format Registries
4.1.2 Bitstream Format Registry
4.2 Configuring Usage Instrumentation Plugins
4.2.1 The Passive Plugin
4.2.2 The Tab File Logger Plugin
4.3 Behavior of the workflow system
4.4 Recognizing Web Spiders (Bots, Crawlers, etc.)
5 Command-line Access to Configuration Properties

General Configuration

In the following sections you will learn about the different configuration files that you will need to edit so that you may make your DSpace installation work.

DSpace provides a number of textual configuration files which may be used to configure your site based on local needs. These include:

- `[dspace]/config/dspace.cfg` : The primary configuration file, which contains the main configurations for DSpace.
- `[dspace]/config/modules/*.cfg` : Module configuration files, which are specific to various modules/features within DSpace.
- `[dspace]/config/local.cfg` : A (optional, but highly recommended) localized copy of configurations/settings specific to your DSpace (see The local.cfg Configuration Properties File below)
- Additional feature-specific configuration files also exist under `[dspace]/config/`, some of these include:
  - `default.license` : the default deposit license used by DSpace during the submission process (see Submission User Interface documentation)
  - `hibernate.cfg.xml` : The Hibernate class configuration for the DSpace database (almost never requires changing)
  - `item-submission.xml` : the default item submission process for DSpace (see Submission User Interface documentation)
  - `launcher.xml` : The configuration of the DSpace command-line "launcher" (`[dspace]/bin/dspace` , see the DSpace Command Launcher documentation)
  - `log4j2.xml` : The default logging settings for DSpace log files (usually placed in `[dspace]/log`)
  - `submission-forms.xml`: The default deposit forms for DSpace, used by `item-submission.xml` (see Submission User Interface documentation)

As most of these configurations are detailed in other areas of the DSpace documentation (see links above), this section concentrates primarily on the "*.cfg" configuration files (namely `dspace.cfg` and `local.cfg`).

Configuration File Syntax

We will use the `dspace.cfg` as our example for input conventions used throughout the system. These same input conventions apply to all DSpace *.cfg files.

All DSpace *.cfg files use the Apache Commons Configuration properties file syntax. This syntax is very similar to a standard Java properties file, with a few notable enhancements described below.

- Comments all start with a "#" symbol. These lines are ignored by DSpace.
- Other settings appear as property/value pairs of the form: `property.name = property value`
- Certain special characters (namely commas) MUST BE escaped. See the "Special Characters" section below
- Values assigned in the same *.cfg file are "additive", and result in an array of values. See "Specifying Multiple Values for Properties" below.

Some property defaults are "commented out". That is, they have a "#" preceding them, and the DSpace software ignores the config property. This may cause the feature not to be enabled, or, cause a default property to be used.

The property value may contain references to other configuration properties, in the form `${property.name}`. A property may not refer to itself. Examples:

```
dspace.dir = /path/to/dspace
dspace.name = My DSpace

# property.name will be equal to "My DSpace is great!"
property.name = ${dspace.name} is great!

# property2.name will be equal to "/path/to/dspace/rest/of/path"
property2.name = ${dspace.dir}/rest/of/path

# However, this will result in an ERROR, as the property cannot reference itself
property3.name = ${property3.name}
```

Special Characters

Certain characters in `*.cfg` files are considered special characters, and **must** be escaped in any values. The most notable of these special characters include:

- Commas (,) : as they represent lists or arrays of values (see "Specifying Multiple Values for Properties" below)
- Backslashes (\) : as this is the escape character

This means that if a particular setting needs to use one of these special characters in its value, it must be escaped. Here's a few examples:

```
# WRONG SETTING
# This setting is INVALID. DSpace is expecting your site name to be a single value,
# But, this setting would create an array of two values: "DSpace" and "My Institution"
dspace.name = DSpace, My Institution

# CORRECT SETTING (commas is escaped)
# Instead, if the name of your DSpace includes a comma, you need to escape it with "\,"
dspace.name = DSpace\, My Institution

# WRONG SETTING
# As the backslash is the escape character, this won't work
property.name = \some\path

# CORRECT SETTING
# If you want a literal backslash, you need to escape it with "\\"
# So, the below value will be returned as "\some\path"
property.name = \\some\\path
```

Additional examples of escaping special characters are provided in the documentation of the Apache Commons Configuration properties file syntax.

Specifying Multiple Values for Properties

Because DSpace supports the Apache Commons Configuration properties file syntax, it is much easier to specify multiple values for a single setting. All you have to do is repeat the same property name multiple times in the same `*.cfg` file.

For example:

```
# The below settings define *two* AuthenticationMethods that will be enabled, LDAP and Password authentication
# Notice how the same property name is simply repeated, and passed different values.
plugin.sequence.org.dspace.authenticate.AuthenticationMethod = org.dspace.authenticate.LDAPAuthentication
plugin.sequence.org.dspace.authenticate.AuthenticationMethod = org.dspace.authenticate.PasswordAuthentication

# Alternatively, you can also define them as a comma-separated list
# (In this scenario, you would NOT escape the comma, as you want them to be considered multiple values)
# So, this single line is exactly equavalent to the settings above:
plugin.sequence.org.dspace.authenticate.AuthenticationMethod = org.dspace.authenticate.LDAPAuthentication, org.
dspace.authenticate.PasswordAuthentication
```

Please be aware that this ONLY works if you are reusing the exact same configuration in the same configuration file. This causes the values to be "additive" (i.e they are appended to the same list).

However, as you'll see below, the `local.cfg` file always *overrides* settings elsewhere. So, if the above "AuthenticationMethod" plugin was specified in both your `authentication.cfg` and your `local.cfg`, the value(s) in your `local.cfg` would *override* the defaults in your `authentication.cfg` (more on that below).

Additional examples of creating lists or arrays of values are provided in the documentation of the Apache Commons Configuration properties file syntax.

Including other Property Files

Because DSpace supports the Apache Commons Configuration properties file syntax, it also can include/embed property files within other property files by using the "`include=`" setting.

For example, the `dspace.cfg` includes/embeds all of the default `config/modules/*.cfg` files via a series of "`include=`" settings near the bottom of the `dspace.cfg`. As an example, here's a small subset of those include calls:

```
# defines our modules subdirectory
module_dir = modules

# The following lines include specific "authentication*.cfg" files inside your dspace.cfg
# This essentially "embeds" their configurations into your dspace.cfg,
# treating them as if they were a single configuration file.
include = ${module_dir}/authentication.cfg
include = ${module_dir}/authentication-ip.cfg
include = ${module_dir}/authentication-ldap.cfg
include = ${module_dir}/authentication-password.cfg
include = ${module_dir}/authentication-shibboleth.cfg
```

This ability to include properties files within others is very powerful, as it allows you to inherit settings from other files, or subdivide large configuration files. Be aware that this essentially causes DSpace to treat all included configurations *as if they were part of the parent file*. This means that, in the above example, as far as DSpace is concerned, all the settings contained within the `authentication*.cfg` files "appear" as though they are specified in the main `dspace.cfg`.

This ability to include other files is also possible with the `local.cfg` file, should you want to subdivide your localized settings into several locally specific configuration files.

Configuration Scheme for Reloading and Overriding

Known limitations to reloadable configurations

While the DSpace API supports dynamically reloading configurations, the user or machine interfaces *may* still cache some configuration settings. This means that while the API layer may reload a new value, that new value may not always affect/change the behavior of your user interface (until you restart Tomcat).

Also, please be aware that all DSpace configuration values loaded into Spring beans (for example configurations that appear in Spring XML configuration files or in @Value annotations) **are cached by Spring**. This means that they will not be reloadable within Spring beans until Tomcat is restarted.

Because DSpace supports the Apache Commons Configuration, its configurations can now be reloaded without restarting your servlet container (e.g. Tomcat). By default, DSpace checks for changes to any of its runtime configuration files every 5 seconds. If a change has been made, the configuration file is reloaded. The 5 second interval is configurable in the `config-definition.xml` (which defines the configuration scheme DSpace uses).

Additionally, DSpace provides the ability to easily override default configuration settings (in dspace.cfg or modules/*.cfg) using a local.cfg file (see The local.cfg Configuration Properties File) or using System Properties / Environment Varilables.

Both of these features are defined in DSpace's default "configuration scheme" or "configuration definition" in the `[dspace]/config/config-definition.xml` file. This file defines the Apache Commons Configuration settings that DSpace utilizes by default. It is a valid "configuration definition" file as defined by Apache Commons Configuration. See their Configuration Definition File Documentation for more details.

You are welcome to customize the `config-definition.xml` to customize your local configuration scheme as you see fit. Any customizations to this file will require restarting your servlet container (e.g. Tomcat).

By default, the DSpace `config-definition.xml` file defines the following configuration scheme:

- *Configuration File Syntax/Sources:* All DSpace configurations are loaded via Properties files (using the Configuration File Syntax detailed above)
  - Note: Apache Commons Configuration does support other configuration sources such as XML configurations or database configurations (see its Overview documentation). At this time, DSpace does not utilize these other sorts of configurations by default. However, it would be possible to customize your local config-definition.xml to load settings from other locations.
- *Configuration Files/Sources:* By default, only two configuration files are loaded into Apache Commons Configuration for DSpace:
  - `local.cfg` (see The local.cfg Configuration Properties File documentation below)
  - `dspace.cfg` (NOTE: all `modules/*.cfg` are loaded by `dspace.cfg` via "`include=`" statements at the end of that configuration file. They are essentially treated as sub-configs which are embedded/included into the `dspace.cfg`)
- *Configuration Override Scheme:* The configuration override scheme is defined as follows. Configurations specified in earlier locations will automatically override any later values:
  - System Properties (`-D[setting]=[value]`) override all other options
  - Environment Variables.
    - DSpace provides a custom environment variable syntax as follows:
      - All periods (.) in configuration names must be translated to "__P__" (two underscores, capital P, two underscores), e.g. "dspace__P__dir" environment variable will override the "dspace.dir" configuration in local.cfg (or other *.cfg files)
      - All dashes (-) in configuration names must be translated to "__D__" (two underscores, capital D, two underscores), e.g. "authentication__D__ip__P__groupname" environment variable will override the "authentication-ip.groupname" configuration in local.cfg (or other *.cfg files)
  - `local.cfg`
  - `dspace.cfg` (and all `modules/*.cfg` files) contain the default values for all settings.
- *Configuration Auto-Reload:* By default, all configuration files are automatically checked every 5 seconds for changes. If they have changed, they are automatically reloaded.

For more information on customizing our default config-definition.xml file, see the Apache Commons Configuration documentation on the configuration definition file. Internally, DSpace simply uses the `DefaultConfigurationBuilder` class provided by Apache Commons Configuration to initialize our configuration scheme (and load all configuration files).

Customizing the default configuration scheme

Because the `config-definition.xml` file is just a Configuration Definition file for Apache Commons Configuration, you can also choose to customize the above configuration scheme based on your institution's local needs. This includes, but is not limited to, changing the name of "local.cfg", adding additional configuration files/sources, or modifying the override or auto-reload schemes. For more information, see the Configuration Definition File Documentation from Apache Commons Configuration.

Why are there multiple copies of some config files?

It is important to remember that there are **multiple copies of each configuration files in an installation of DSpace**. The primary ones to be aware of are:

1. The "source" configuration file(s) are found under in `[dspace-source]/dspace/config/` or subdirectories. This also includes the `[dspace-source]/local.cfg`
2. The "runtime" configuration file(s) that are found in `[dspace]/config/`

The DSpace server (webapp) and command line programs only look at the *runtime* configuration file(s).

When you are revising/changing your configuration values, it may be tempting to *only edit the runtime file*. **DO NOT** do this. Whenever you rebuild DSpace, it will "reset" your runtime configuration to whatever is in your source directories (the previous runtime configuration is copied to a date suffixed file, should you ever need to restore it).

Instead, we recommend to **always make the same changes to the source version** of the configuration file in addition to the runtime file. In other words, the source and runtime files should always be identical / kept in sync.

One way to keep the two files in synchronization is to edit your files in `[dspace-source]/dspace/config/` and then run the following commands to rebuild DSpace and install the updated configs:

```
cd [dspace-source]/dspace/
mvn package
cd [dspace-source]/dspace/target/dspace-installer
ant update_configs
```

This will copy the source configuration files into the runtime ([dspace]/config) directory. Another option to manually sync the files by copying them to each directory.

Please note that there are additional "ant" commands to help with configuration management:

- "ant update_configs" ==> Moves existing configs in `[dspace]/config/` to *.old files and replaces them with what is in `[dspace-source]/dspace/config/`
- "ant -Doverwrite=false update_configs" ==> Leaves existing configs in `[dspace]/config/` intact. Just copies new configs from `[dspace-source]/dspace/config/` over to *.new files.

The `local.cfg` Configuration Properties File

build.properties has been replaced by local.cfg

As of DSpace 6 and above, the old "`build.properties`" configuration file has been replaced by this new "`local.cfg`" configuration file. For individuals who are familiar with the old `build.properties` file, this new `local.cfg` differs in a few key ways:

- Unlike `build.properties`, the `local.cfg` file can be used to override ANY setting in any other configuration file (`dspace.cfg` or `modules/*.cfg`). To override a default setting, simply copy the configuration into your `local.cfg` and change its value(s).
- Unlike `build.properties`, the `local.cfg` file is not utilized during the compilation process (e.g. mvn package). But, it is automatically copied alongside the final `dspace.cfg` into your installation location (`[dspace]/config/`), where it overrides default DSpace settings with your locally specific settings *at runtime*.
- Like `build.properties`, the `local.cfg` file is expected to be specified in the source directory by default (`[dspace-source]`). There is an example (`[dspace-source]/dspace/config/local.cfg.EXAMPLE`) provided which you can use to create a `[dspace-source]/dspace/config/local.cfg`.

Many configurations have changed names between DSpace 5 (and below) and DSpace 6 (and above)

If you are upgrading from an earlier version of DSpace, you will need to be aware that *many* configuration names/keys have changed. Because Apache Commons Configuration allows for auto-overriding of configurations, all configuration names/keys in different `*.cfg` files MUST be uniquely named (otherwise accidental, unintended overriding may occur).

In order to create this powerful ability to override configurations in your `local.cfg`, all `modules/*.cfg` files had their configurations **renamed** to be prepended with the module name. As a basic example, all the configuration settings within the `modules/oai.cfg` configuration now start with "`oai.`".

Additionally, while the `local.cfg` may look *similar* to the old `build.properties`, many of its configurations have slightly different names. *So, simply copying your build.properties into a local.cfg will NOT work*.

This means that DSpace 5.x (or below) configurations are NOT guaranteed compatible with DSpace 6. While you obviously can use your old configurations as a reference, you will need to start with fresh copy of all configuration files, and reapply any necessary configuration changes (this has always been the recommended procedure). However, as you'll see below, you'll likely want to do that anyways in order to take full advantage of the new `local.cfg` file.

It is possible to easily override default DSpace configurations (from `dspace.cfg` or `modules/*.cfg` files) in your own `local.cfg` configuration file.

A example `[dspace-source]/dspace/config/local.cfg.EXAMPLE` is provided with DSpace. The example only provides a few key configurations which most DSpace sites are likely to need to customize. However, you may add (or remove) any other configuration to your `local.cfg` to customize it as you see fit.

To get started, simply create your own `[dspace-source]/dspace/config/local.cfg` based on the example, e.g.

```
cd [dspace-source]/dspace/config/
cp local.cfg.EXAMPLE local.cfg
```

You can then begin to edit your `local.cfg` with your local settings for DSpace. There are a few key things to note about the `local.cfg` file:

- **Override any default configurations:** Any setting in your `local.cfg` will automatically OVERRIDE a setting of the same name in the `dspace.cfg` or any `modules/*.cfg` file. This also means that you can copy ANY configuration (from `dspace.cfg` or any `modules/*.cfg` file) into your `local.cfg` to specify a new value.
    - For example, specifying `dspace.url` in `local.cfg` will override the default value of `dspace.url` in `dspace.cfg`.
    - Also, specifying `oai.solr.url` in `local.cfg` will override the default value of `oai.solr.url` in `config/modules/oai.cfg`
- **Configuration Syntax:** The `local.cfg` file uses the Apache Commons Configuration Property file syntax (like all *.cfg files) . For more information see the section on Configuration File Syntax above.
    - This means the `local.cfg` also supports enhanced features like the ability to include other config files (via "`include=`" statements).
- **Override local.cfg via System Properties:** As needed, you also are able to OVERRIDE settings in your `local.cfg` by specifying them as System Properties or Environment Variables.
    - For example, if you wanted to change your `dspace.dir` in development/staging environment, you could specify it as a System Property (e.g. `-Ddspace.dir=[new-location]`). This new value will override any value in *both* `local.cfg` and `dspace.cfg`.

When you build DSpace (e.g. mvn package), this `local.cfg` file will be automatically copied to `[dspace]/config/local.cfg`. Similar to the `dspace.cfg`, the "runtime" configuration (used by DSpace) is the one in `[dspace]/config/local.cfg`. See the Why are there multiple copies of some config files? question above for more details on the runtime vs source configuration.

Here's a very basic example of settings you could place into your `local.cfg` file (with inline comments):

```
# This is a simple example local.cfg file which shows off options
# for creating your own local.cfg

# This overrides the default value of "dspace.dir" in dspace.cfg
dspace.dir = C:/dspace/

# This overrides the default value of "dspace.server.url" in dspace.cfg
dspace.server.url = https://dspace.myuniversity.edu/server

# The overrides the default "dspace.ui.url" setting it to the same value as my "baseUrl" above
dspace.ui.url = https://dspace.myuniversity.edu

# If our database settings are the same as the default ones in dspace.cfg,
# then, we may be able to simply customize the db.username and db.password
db.username = myuser
db.password = mypassword

# For DSpace, we want the LDAP and Password authentication plugins enabled
# This overrides the default AuthenticationMethod in /config/modules/authentication.cfg
# Since we specified the same key twice, these two values are appended (see Configuration File Syntax above)
plugin.sequence.org.dspace.authenticate.AuthenticationMethod = org.dspace.authenticate.LDAPAuthentication
plugin.sequence.org.dspace.authenticate.AuthenticationMethod = org.dspace.authenticate.PasswordAuthentication

# We also can reference other configurations in values.
# For instance, we can set the "mail.admin" and the "feedback.recipient" to be the same email
mail.admin = myemail@myuniversity.edu
feedback.recipient = ${mail.admin}

# For the example, we'll override the default oai.path in /config/modules/oai.cfg
# This puts our OAI-PMH interface at ${dspace.server.url}/oaipmh
oai.path = oaipmh

# We'll also override the default oai.solr.url in /config/modules/oai.cfg
# Notice here we're referencing a configuration (solr.server) that only exists in dspace.cfg
# This is allowed. Your local.cfg can reference configs from other *.cfg files.
oai.solr.url = ${solr.server}/oaipmh

# Finally, this local.cfg also supports adding "include=" statements, to include
# additional local configuration files.
# In this example, a local-rest.cfg and local-curate.cfg (in the same directory)
# will automatically be included as part of this local.cfg.
# This allows you to subdivide you local configs in as many (or few) config files
# as you desire.
include = local-rest.cfg
include = local-curate.cfg
```

The `dspace.cfg` Configuration Properties File

Any dspace.cfg setting can be overridden in your local.cfg

Remember, *any* of the below `dspace.cfg` settings can be copied into your local.cfg configuration file and overridden. So, rather than editing the `dspace.cfg` (or any of the `modules/*.cfg`), it's recommended to simply override the default values in your `local.cfg`. That way, your `local.cfg` can serve as the record of which configurations you have actually tweaked in your DSpace, which may help to simplify future upgrades.

The `dspace.cfg` contains basic information about a DSpace installation, including system path information, network host information, and other like items. It is the default configuration file for DSpace, used by DSpace when it is actively running. However, as noted above, any of these default configurations may be overridden in your own `local.cfg` configuration file.

Main DSpace Configurations

Property:	`dspace.dir`

Example Value:	`/dspace`
Informational Note:	Root directory of DSpace installation. Omit the trailing slash '/'. Note that this setting is used by default in other settings, e.g. `assetstore.dir` . (*On Windows be sure to use forward slashes for the directory path!* For example: "C:/dspace" is a valid path for Window.)
Property:	`dspace.server.url`
Example Value:	`https://dspace.myu.edu/server`
Informational Note:	Main URL at which DSpace backend ("server" webapp) is publicly available. If using port 80 (HTTP) or 443 (HTTPS), you may omit the port number. Otherwise the port number must be included. **Do not include a trailing slash ('/').** In Production, you must use HTTPS if you wish to access the REST API from a different server/domain. This configuration should match the User Interface's "rest" settings in the config.yml (specifically the "ssl", "host", "port" and "nameSpace" settings in that file). See User Interface Configuration
Property:	`dspace.server.ssr.url`

Example Value:	`http://localhost:8080/server`
Informational Note:	*(7.6.3 and later)* Optional, separate Server-Side Rendering URL for the REST API. When specified, this URL will be used by the DSpace User Interface during server-side execution. This may be a private or localhost URL, but it must be accessible to the User Interface's server-side code. For example, the "dspace.server.url" (above) should always reference a public URL like "https://mydspace.edu/server". But, you could set "dspace.server.ssr.url" to a localhost URL (e.g. "http://localhost:8080/server") if your REST API is running on the same machine as the User Interface. This configuration would result in all client-side code (running in the user's browser) accessing the REST API via the "dspace.server.url"(a public, HTTPS URL), while the server-side code (triggered by SSR) would access the REST API via "dspace.server.ssr.url" (a localhost HTTP URL). When this configuration is specified you **must** set the same URL in the User Interface's "rest.ssrBaseUrl" settings in its config.yml. See User Interface Configuration
Property:	`dspace.ui.url`
Example Value:	`dspace.ui.url = http://dspacetest.myu.edu:4000`
Informational note	Main URL at which the DSpace frontend (Angular User Interface) is publicly available. If using port 80 (HTTP) or 443 (HTTPS), you may omit the port number. Otherwise the port number must be included. **Do not include a trailing slash ('/').** In Production, you should be using HTTPS for security purposes. This URL should match the URL you type in the browser to access your User Interface. In the backend, this URL is primarily used to build UI-based URLs in sitemaps, email messages, etc. Therefore, it need not be set on initial installation, but it should be configured as soon as your user interface is installed. If you are not using the DSpace UI (and running the backend "headless"), this may be set to the URL of whatever you consider your primary "user interface".
Property:	`dspace.name`
Example Value:	`dspace.name = DSpace at My University`

Informational Note:	Short and sweet site name, used in e-mails, exports and machine interfaces (e.g. OAI-PMH). It is not currently used by the Angular UI.

General Solr Configuration

DSpace uses Solr for various indexing purposes, and uses a pool of open connections to manage communication with Solr. These properties configure the connections between DSpace and Solr.

See also the additional Solr configuration properties for specific indexes such as search, statistics, authority and OAI PMH.

Property:	`solr.server`
Example Value:	`solr.server = http://localhost:8983/solr`
Informational Note:	Base URL to the Solr server. Specific indexes append to this value.
Property:	`solr.client.maxTotalConnections`
Example Value:	`solr.client.maxTotalConnections = 20`
Informational Note:	The maximum number of connections that will be opened between DSpace and Solr.
Property:	`solr.client.maxPerRoute`
Example Value:	`solr.client.maxPerRoute = 15`
Informational Note:	The maximum number of connections that will be opened between DSpace and a specific Solr instance (if you have more than one).
Property:	`solr.client.keepAlive`
Example Value:	`solr.client.keepAlive = 5000`
Informational Note:	The default amount of time that a connection in use will be held open, in milliseconds. Solr may specify a different keep-alive interval and it will be obeyed.
Property:	`solr.client.timeToLive`
Example Value:	`solr.client.timeToLive = 600`
Informational Note:	The maximum amount of time before an open connection will be closed when idle, in seconds. New connections will be opened as needed, subject to the above limits.

DSpace Database Configuration

Many of the database configurations are software-dependent. That is, it will be based on the choice of database software being used. Currently, DSpace properly supports PostgreSQL and Oracle.

Oracle Support Deprecated - Will no longer be supported in 2023

Oracle support has been deprecated in DSpace. It will no longer be supported as of June/July 2023. See https://github.com/DSpace/DSpace/issues/8214

We recommend all users install DSpace on PostrgreSQL (see above)

Property:	`db.url`
Example Value:	`db.url = jdbc:postgresql://localhost:5432/dspace`
Informational Note:	The above value is the default value when configuring with PostgreSQL. When using Oracle, use this value: `jbdc.oracle.thin:@//host:port/dspace`
Property:	`db.username`
Example Value:	`db.username = dspace`
Informational Note:	In the installation directions, the administrator is instructed to create the user "dspace" who will own the database "dspace".
Property:	`db.password`
Example Value:	`db.password = dspacepassword`
Informational Note:	This is the password that was prompted during the installation process (cf. 3.2.3. Installation)
Property:	`db.schema`
Example Value:	`db.schema = public`
Informational Note:	If your database contains multiple schemas, you can avoid problems with retrieving the definitions of duplicate objects by specifying the schema name here that is used for DSpace by uncommenting the entry. This property is optional. For PostgreSQL databases, this is often best set to "public" (default schema). For Oracle databases, the schema is usually equivalent to the username of your database account. So, for Oracle, this may be set to `${db.username}` in most scenarios.
Property:	`db.maxconnections`
Example Value:	`db.maxconnections = 30`
Informational Note:	Maximum number of Database connections in the connection pool
Property:	`db.maxwait`
Example Value:	`db.maxwait = 5000`
Informational Note:	Maximum time to wait before giving up if all connections in pool are busy (in milliseconds).
Property:	`db.maxidle`
Example Value:	`db.maxidle = -1`
Informational Note:	Maximum number of idle connections in pool. (*-1* = unlimited)

Property:	`db.cleanDisabled`
Example Value:	`db.cleanDisabled = true`
Informational Note:	This is a developer-based setting which determines whether you are allowed to run "./dspace database clean" to completely delete all content and tables in your database. This should always be set to "true" in Production to protect against accidentally deleting all your content by running that command. (Default is set to true)

To provide the database connection pool externally

Alternately, you may supply a configured connection pool out of JNDI. The object must be named `jdbc/dspace` (the full path is `java:comp/env/jdbc/dspace`). DSpace will always look up this name and, if found, will use the returned object as its database connection pool. If not found, the above `db.*` properties will be used to create the pool.

If you are using Tomcat, then the object might be defined using a `<Resource>` element, or connected to a `<Resource>` child of `<GlobalNamingResources>` using a `<ResourceLink>` element. See your Servlet container's documentation for details of configuring the JNDI initial context. For example, Tomcat provides a useful JNDI Datasource How-to

Earlier releases of DSpace provided a configuration property `db.jndi` to specify the name to be looked up, but that has been removed. The name is specified in `config/spring/api/core-hibernate.xml` if you really need to change it.

DSpace Email Settings

The configuration of email is simple and provides a mechanism to alert the person(s) responsible for different features of the DSpace software.

DSpace will look up a javax.mail.Session object in JNDI and, if found, will use that to send email. Otherwise it will create a Session using some of the properties detailed below.

Property:	`mail.server`
Example Value:	`mail.server = smtp.my.edu`
Informational Note:	The address on which your outgoing SMTP email server can be reached.
Property:	`mail.server.username`
Example Value:	`mail.server.username = myusername`
Informational Note:	SMTP mail server authentication username, if required. This property is optional.
Property:	`mail.server.password`
Example Value:	`mail.server.password = mypassword`
Informational Note:	SMTP mail server authentication password, if required. This property is optional.
Property:	`mail.server.port`
Example Value:	`mail.server.port = 25`

Informational Note:	The port on which your SMTP mail server can be reached. By default, port 25 is used. Change this setting if your SMTP mailserver is running on another port. This property is optional.
Property:	`mail.from.address`
Example Value:	`mail.from.address = dspace-noreply@myu.edu`
Informational Note:	The "From" address for email. Change the 'myu.edu' to the site's host name.
Property:	`feedback.recipient`
Example Value:	`feedback.recipient = dspace-help@myu.edu`
Informational Note:	When a user clicks on the feedback link/feature, the information will be sent to the email address of choice. This configuration is currently limited to only one recipient. This is also the email address displayed on the contacts page.
Property:	`mail.admin`
Example Value:	`mail.admin = dspace-help@myu.edu`
Example Value:	Email address of the general site administrator (Webmaster). System notifications/reports and other sysadmin emails are sent to this email address.
Property:	`mail.admin.name`
Example Value:	`mail.admin.name = DSpace Administrator`
Example Value:	Name associated with the `mail.admin` email address.
Property:	`alert.recipient`
Example Value:	`alert.recipient = john.doe@myu.edu`
Informational Note:	Enter the recipient for server errors and alerts. This property is optional and defaults to the `${mail.admin}` setting
Property:	`registration.notify`
Example Value:	`registration.notify = mike.smith@myu.edu`
Informational Note:	Enter the recipient that will be notified when a new user registers on DSpace. This property is optional & defaults to no value.
Property:	`mail.charset`

Example Value:	`mail.charset = UTF-8`
Informational Note:	Set the default mail character set. This may be over-ridden by providing a line inside the email template '#set($charset = "encoding")'. Otherwise this default is used.
Property:	`mail.allowed.referrers`
Example Value:	`mail.allowed.referrers = localhost`
Informational Note:	A comma separated list of hostnames that are allowed to refer browsers to email forms. This property is optional. **UNSUPPORTED in DSpace 7.0**
Property:	`mail.extraproperties`

Example Value:	`# Example which can fix "Could not convert socket to TLS" errors (i.e. SMTP over TLS)`
	`mail.extraproperties = mail.smtp.socketFactory.port=587, \`
	`mail.smtp.starttls.enable=true, \`
	`mail.smtp.starttls.required=true, \`
	`mail.smtp.ssl.protocols=TLSv1.2`
	
	`# Different example of using SSL for your Mail libary`
	`mail.extraproperties = mail.smtp.socketFactory.port=465, \`
	`mail.smtp.socketFactory.class=javax.net.ssl.SSLSocketFactory, \`
	`mail.smtp.socketFactory.fallback=false`

Informational Note:	If you need to pass extra settings to the Java mail library. Comma separated, equals sign between the key and the value. This property is optional.
Property:	`mail.server.disabled`
Example Value:	`mail.server.disabled = false`
Informational Note:	An option is added to disable the mailserver. By default, this property is set to '`false`'. By setting value to '`true`', DSpace will not send out emails. It will instead log the subject of the email which should have been sent. This is especially useful for development and test environments where production data is used when testing functionality. This property is optional.
Property:	`mail.session.name`
Example Value:	`mail.session.name = myDSpace`
Informational Note:	Specifies the name of a javax.mail.Session object stored in JNDI under `java:comp/env/mail`. The default value is "Session".
Property:	`default.language`
Example Value:	`default.language = en_US`
Informational Note:	If no other language is explicitly stated in the *submission-forms.xml*, the default language will be attributed to the metadata values. See also Multilingual Support

Property:	`mail.message.headers`

Example Value:	`mail.message.headers = subject`
	
	`mail.message.headers = charset`

Informational Note:	When processing a message template, setting a Velocity variable whose name is one of the values of this configuration property will add or replace a message header of the same name, using the value of the variable as the header's value. See "Templates can set message headers".
Property:	`mail.welcome.enabled`
Example Value:	`mail.welcome.enabled = true`
Informational Note:	Enable a "welcome letter" to the newly-registered user. By default this is `false`. See the `welcome` email template.

Wording of E-mail Messages

Sometimes DSpace automatically sends e-mail messages to users, for example, to inform them of a new work flow task, or as a subscription e-mail alert. The wording of e-mails can be changed by editing the relevant file in `[dspace]/config/emails` . Each file is commented. Be careful to keep the right number 'placeholders' (e.g. *${params[2]}*) for the template's positional parameters.

Each file is a Velocity template. You can use the full Velocity Template Language to help you customize messages. There are two Velocity variables pre-defined by DSpace when processing an e-mail template:

- `params` is the array of message parameters provided by the DSpace code which is sending the message. These are indexed by number, starting at zero.
- `config` is the table of DSpace configuration properties (such as `dspace.name`). These are looked up using `config.get(property name)`.

Sample message template

```
## This is a comment.  It will not be part of the message.
This is the body of the message.  The code which sends it supplied two parameters:  ${params[0]} and ${params
[1]}.
The name of this DSpace instance is ${config.get('dspace.name')} and you can browse it at ${config.get('dspace.
url')}.
```

Also see the template `config/emails/register` for an example of each.

Note: You should replace the contact-information "`dspace-help@myu.edu or call us at xxx-555-xxxx`" with your own contact details in:
`config/emails/change_password`
`config/emails/register`

Configure set of property keys accessible by Velocity Templates (since 7.6.7)

For security reasons, the list of configuration properties available to use in Email or LDN templates should be as restrictive as possible, to avoid any unwanted information disclosure.

By default, an attempt to access a disallowed or missing configuration property will result in a null or blank string. You can also run Velocity in "strict mode" which will throw a fatal exception if a message is constructed with an invalid configuration property.

If you have custom email templates accessing configuration properties outside of these defaults, make sure to configure them here.

```
# Allowed configuration properties, to pass in a "config" map to email and LDN templates.
# This allows templates to easily access dynamic configuration properties, without
# exposing sensitive information to the templating engine
message.templates.allowed-config = dspace.name
message.templates.allowed-config = dspace.shortname
message.templates.allowed-config = dspace.ui.url
message.templates.allowed-config = mail.helpdesk
message.templates.allowed-config = mail.message.helpdesk.telephone
message.templates.allowed-config = mail.admin
message.templates.allowed-config = mail.admin.name

# Whether to run Velocity in strict mode (null parameter values in templates for LDN or Email will result
# in an Exception instead of a blank string)
# Default: false (this can introduce unwanted side-effects if e.g. a submitter eperson is deleted for a
workflow task)
#message.templates.strict_mode = false
```

Templates can set message headers

A template can set specific message headers by defining Velocity variables which have been enabled for this use by naming them as values of the DSpace configuration property `mail.message.headers`. In most cases the name of the Velocity variable will become the header's name, and the value of the variable, the header's value. For example: `#set(My-Header, "Hello World!")` in a template will result in the message having a header `My-Header: Hello World!` if DSpace's mail.message.headers includes "My-Header".

A few Velocity variable names are special in DSpace email templates:

name	meaning
subject	supplies the Subject: header's value.
charset	sets the `charset` parameter of the `Content-Type:` header of the bodypart, when there is a single bodypart. It also causes the subject value to be treated as being encoded in this charset. If not set, the charset defaults to US-ASCII as specified in RFC 2046. If there are multiple bodyparts, all are assumed to be encoded in US-ASCII and `charset` has no effect on them.

Sample message template

```
## This is a comment.  It will not be part of the sent message.
#set($subject = "This will be the Subject: of the message")
This is the body of the message.
```

File Storage

Beginning with DSpace 6, your file storage location (aka bitstore) is now defined in the `[dspace]/config/spring/api/bitstore.xml` Spring configuration file. By default it is defined as the `[dspace]/assetstore/`. More information on modifying your file storage location can be found at Configuring the Bitstream Store in the Storage Layer documentation.

DSpace supports multiple options for storing your repository bitstreams (uploaded files). The files are not stored in the database, instead they are provided via a configured "assetstore" or "bitstore".

By default, the assetstore is simply a directory on your server (`[dspace]/assetstore/`) under which bitstreams (files) are stored by DSpace.

At this time, DSpace supports two primary locations for storing your files:

1. Your local filesystem (used by default), specifically under the `[dspace]/assetstore/` directory
2. OR, Amazon S3 (requires your own Amazon S3 account)

More information on configuring or customizing the storage location of your files can be found in the Storage Layer documentation.

Logging Configuration

Logging configuration has now moved to `${dspace.dir}/config/log4j2.xml`

Property:	`log.init.config`

Example Value:	`log.init.config = ${dspace.dir}/config/log4j2.xml`

Informational Note:	This is where your logging configuration file is located. You may override the default log4j configuration by providing your own. Existing alternatives are:
	`log.init.config = ${dspace.dir}/config/log4j2.xml`
	`log.init.config = ${dspace.dir}/config/log4j2-console.xml`

Property:	`log.dir` (*defined in log4j2.xml*)
Example value:	`log.dir = ${log4j:configParentLocation}/../log`
Informational Note:	This is where to put the DSpace logs. The default setting (shown above) writes all DSpace logs to the ${dspace.dir}/log/ directory.
Property:	`loglevel.dspace` (*defined in log4j2.xml*)
Example value:	`loglevel.dspace = INFO`
Informational Note:	Log level for all DSpace-specific code (org.dspace.* packages). By default, DSpace only provides general INFO logs (in order to keep log sizes reasonable). As necessary, you can temporarily change this setting to any of the following (ordered for most information to least): DEBUG, INFO, WARN, ERROR, FATAL Please be aware we do not recommend running at the DEBUG level in Production for significant periods of time, as it will cause the logs to be extremely large in size.
Property:	`loglevel.other` (*defined in log4j2.xml*)
Example value:	`loglevel.other = INFO`
Informational Note:	Log level for other third-party tools/APIs used by DSpace (non-DSpace specific code). By default, DSpace only provides general INFO logs (in order to keep log sizes reasonable). As necessary, you can temporarily change this setting to any of the following (ordered for most information to least): DEBUG, INFO, WARN, ERROR, FATAL Please be aware we do not recommend running at the DEBUG level in Production for significant periods of time, as it will cause the logs to be extremely large in size.

General Plugin Configuration

Property:	**`plugin.classpath`**
Example Value:	/opt/dspace/plugins/aPlugin.jar:/opt/dspace/moreplugins
Informational Note:	Search path for third-party plugin classes. This is a colon-separated list of directories and JAR files, each of which will be searched for plugin classes after looking in all the places where DSpace classes are found. In this way you can designate one or more locations for plugin files which will not be affected by DSpace upgrades.

Configuring the Search Engine

DSpace's search module is also known as "Discovery" and utilizes Apache Solr for indexing. It provides up-to-date features, such as filtering/faceting, hit highlighting, search snippets, etc.

Detailed documentation is available for customization, see Discovery.

Handle Server Configuration

The CNRI Handle system is a 3rd party service for maintaining persistent URL's. For a nominal fee, you can register a handle prefix for your repository. As a result, your repository items will be also available under the links http://handle.net/<<handle prefix>>/<<item id>>. As the base url of your repository might change or evolve, the persistent handle.net URL's secure the consistency of links to your repository items. For complete information regarding the Handle server, the user should consult Handle.Net Registry Support.

Property:	`handle.canonical.prefix`
Example Value	`handle.canonical.prefix = http://hdl.handle.net/` `handle.canonical.prefix = ${dspace.ui.url}/handle/`
Informational Note:	Canonical Handle URL prefix. By default, DSpace is configured to use http://hdl.handle.net/ as the canonical URL prefix when generating `dc.identifier.uri` during submission, and in the 'identifier' displayed in item record pages. If you do not subscribe to CNRI's handle service, you can change this to match the persistent URL service you use, or you can force DSpace to use your site's URL, e.g. `handle.canonical.prefix = ${dspace.ui.url}/handle/`. Note that this will not alter `dc.identifer.uri` metadata for existing items (only for subsequent submissions).
Property:	`handle.prefix`
Example Value	`handle.prefix = 1234.56789`
Informational Note:	The default installed by DSpace is `123456789` but you will replace this upon receiving a handle from CNRI.
Property:	`handle.dir`
Example Value:	`handle.dir = ${dspace.dir}/handle-server`
Informational Note:	The default files, as shown in the Example Value is where DSpace will install the files used for the Handle Server.
Property	`handle.additional.prefixes`
Example Value	`handle.additional.prefixes = 1234.56789.0, 1234.56789.1, 987`

Informational Note:	List any additional prefixes that need to be managed by this handle server. For example, any handle prefixes that came from an external repository whose items have now been added to this DSpace. Multiple additional prefixes may be added in a comma separated list.

Delegation Administration: Authorization System Configuration

It is possible to delegate the administration of Communities and Collections. This functionality eliminates the need for an Administrator Superuser account for these purposes. An EPerson that will be attributed Delegate Admin rights for a certain community or collection will also "inherit" the rights for underlying collections and items. As a result, a community admin will also be collection admin for all underlying collections. Likewise, a collection admin will also gain admin rights for all the items owned by the collection.

Authorization to execute the functions that are allowed to user with WRITE permission on an object will be attributed to be the ADMIN of the object (e.g. community/collection/admin will be always allowed to edit metadata of the object). The default will be "*true*" for all the configurations.

Community Administration: Subcommunities and Collections	
Property:	`core.authorization.community-admin.create-subelement`
Example Value:	`core.authorization.community-admin.create-subelement = true`
Informational Note:	Authorization for a delegated community administrator to create subcommunities or collections.
Property:	`core.authorization.community-admin.delete-subelement`
Example Value:	`core.authorization.community-admin.delete-subelement = true`
Informational Note:	Authorization for a delegated community administrator to delete subcommunities or collections.
Community Administration: Policies and The group of administrators	
Property:	`core.authorization.community-admin.policies`
Example Value:	`core.authorization.community-admin.policies = true`
Informational Note:	Authorization for a delegated community administrator to administrate the community policies.
Property:	`core.authorization.community-admin.admin-group`
Example Value:	`core.authorization.community-admin.admin-group = true`
Informational Note:	Authorization for a delegated community administrator to edit the group of community admins.
Community Administration: Collections in the above Community	
Property:	`core.authorization.community-admin.collection.policies`
Example Value:	`core.authorization.community-admin.collection.policies = true`
Informational Note:	Authorization for a delegated community administrator to administrate the policies for underlying collections.
Property:	`core.authorization.community-admin.collection.template-item`
Example Value:	`core.authorization.community-admin.collection.template-item = true`
Informational Note:	Authorization for a delegated community administrator to administrate the item template for underlying collections.
Property:	`core.authorization.community-admin.collection.submitters`
Example Value:	`core.authorization.community-admin.collection.submitters = true`
Informational Note:	Authorization for a delegated community administrator to administrate the group of submitters for underlying collections.
Property:	`core.authorization.community-admin.collection.workflows`

Example Value:	`core.authorization.community-admin.collection.workflows = true`
Informational Note:	Authorization for a delegated community administrator to administrate the workflows for underlying collections.
Property:	`core.authorization.community-admin.collection.admin-group`
Example Value:	`core.authorization.community-admin.collection.admin-group = true`
Informational Note:	Authorization for a delegated community administrator to administrate the group of administrators for underlying collections.
Community Administration: Items Owned by Collections in the Above Community	
Property:	`core.authorization.community-admin.item.delete`
Example Value:	`core.authorization.community-admin.item.delete = true`
Informational Note:	Authorization for a delegated community administrator to delete items in underlying collections.
Property:	`core.authorization.community-admin.item.withdraw`
Example Value:	`core.authorization.community-admin.item.withdraw = true`
Informational Note:	Authorization for a delegated community administrator to withdraw items in underlying collections.
Property:	`core.authorization.community-admin.item.reinstate`
Example Value:	`core.authorization.community-admin.item.reinstate = true`
Informational Note:	Authorization for a delegated community administrator to reinstate items in underlying collections.
Property:	`core.authorization.community-admin.item.policies`
Example Value:	`core.authorization.community-admin.item.policies = true`
Informational Note:	Authorization for a delegated community administrator to administrate item policies in underlying collections.
Community Administration: Bundles of Bitstreams, related to items owned by collections in the above Community	
Property:	`core.authorization.community-admin.item.create-bitstream`
Example Value:	`core.authorization.community-admin.item.create-bitstream = true`
Informational Note:	Authorization for a delegated community administrator to create additional bitstreams in items in underlying collections.
Property:	`core.authorization.community-admin.item.delete-bitstream`
Example Value:	`core.authorization.community-admin.item.delete-bitstream = true`
Informational Note:	Authorization for a delegated community administrator to delete bitstreams from items in underlying collections.
Property:	`core.authorization.community-admin.item.cc-license`
Example Value:	`core.authorization.community-admin.item.cc-license = true`
Informational Note:	Authorization for a delegated community administrator to administer licenses from items in underlying collections.

Collection Administration:
The properties for collection administrators work similar to those
of community administrators,
with respect to collection administration.
	`core.authorization.collection-admin.policies`
	`core.authorization.collection-admin.template-item`
	`core.authorization.collection-admin.submitters`
	`core.authorization.collection-admin.workflows`
	`core.authorization.collection-admin.admin-group`

Collection Administration:
Item owned by the above Collection. The properties for collection
administrators work similar to those of
community administrators,
with respect to administration of
items in underlying collections.
	`core.authorization.collection-admin.item.delete`
	`core.authorization.collection-admin.item.withdraw`
	`core.authorization.collection-admin.item.reinstatiate`
	`core.authorization.collection-admin.item.policies`

Collection Administration:
Bundles of bitstreams, related to items owned by collections in the
above Community. The properties for collection administrators
work similar to those of community administrators, with respect to
administration of bitstreams related to items in
underlying collections.
	`core.authorization.collection-admin.item.create-bitstream`
	`core.authorization.collection-admin.item.delete-bitstream`
	`core.authorization.collection-admin.item-admin.cc-license`

Item Administration. The properties for item administrators work similar to those of community and collection administrators, with respect to administration of items in underlying collections.	`core.authorization.item-admin.policies`

Item Administration:
Bundles of bitstreams, related to items owned by collections in the
above Community. The properties for item administrators work
similar to those of community and collection administrators,
with respect to administration of bitstreams
related to items in underlying collections.
	`core.authorization.item-admin.create-bitstream`
	`core.authorization.item-admin.delete-bitstream`
	`core.authorization.item-admin.cc-license`

Inheritance of collection default policy (since 7.1)

Property:	core.authorization.installitem.inheritance-read.append-mode
Example Value:	core.authorization.installitem.inheritance-read.append-mode = false
Informational Note:	Determine if the DEFAULT READ policies of the collection should be always appended to the policies of the new item (property set to **true**) or used only when no other READ policy has been defined in the submission process (property set to **false**). Please note that also in append mode an open access default policy will be NOT inherited if other policies have been defined in the submission (i.e. if the item was restricted)

Login as feature

Property:	`webui.user.assumelogin`
Example Value:	`webui.user.assumelogin = true`
Informational Note:	Determine if super administrators (those whom are in the Administrators group) can login as another user from the "edit eperson" page. This is useful for debugging problems in a running DSpace instance, especially in the workflow process. The default value is false, i.e., no one may assume the login of another user.

Restricted Item Visibility Settings

By default RSS feeds and subscription emails will include ALL items regardless of permissions set on them. If you wish to only expose items through these channels where the ANONYMOUS user is granted READ permission, then set the following options to false.

Property:	`harvest.includerestricted.rss`
Example Value:	`harvest.includerestricted.rss = true`
Informational Note:	When set to 'true' (default), items that haven't got the READ permission for the ANONYMOUS user, will be included in RSS feeds anyway.

Property:	`harvest.includerestricted.subscription`
Example Value:	`harvest.includerestricted.subscription = true`
Informational Note:	When set to true (default), items that haven't got the READ permission for the ANONYMOUS user, will be included in Subscription emails anyway.

Restricted Bundle Settings

Some bundles are intended only to be accessed and processed internally, such as TEXT, LICENSE, and SWORD. This configuration section allows any bundle to be restricted to only site administrators or the system bypass.

Property:	`core.authorization.restricted-bundle`
Example Value:	`core.authorization.restricted-bundle` = TEXT
Informational Note:	All bundles of the given name will have polciies stripped on item install or creation, or when inheriting from a new collection.

Proxy Settings

These settings for proxy are commented out by default. Uncomment and specify both properties if proxy server is required for external http requests. Use regular host name without port number.

Property:	`http.proxy.host`
Example Value	`http.proxy.host = proxy.myu.edu`
Informational Note	Enter the host name without the port number. *Only currently used for Creative Commons licensing feature (to contact their API), and Sitemap generation (to ping search server regarding updates)*
Property:	`http.proxy.port`
Example Value	`http.proxy.port = 2048`
Informational Note	Enter the port number for the proxy server. *Only currently used for Creative Commons licensing feature (to contact their API), and Sitemap generation (to ping search server regarding updates)*
Property	`useProxies`
Example Value:	`useProxies = true`
Informational Note:	As of DSpace 7 (and above), this setting defaults to **true**. If "useProxies" is enabled, the authentication and statistics logging code will read the X-Forwarded-For header in order to determine the correct client IP address. As the User Interface uses Angular Universal (for SEO support), the proxy server that comes with Angular Universal is always enabled. By default, only your local server (127.0.0.1) and the public IP address of `dspace.ui.url` are "trusted" as a proxy. If your DSpace instance is protected by external proxy server, you may need to update the "proxies.trusted.ipranges" property below. This also affects IPAuthentication, and should be enabled for that to work properly if your installation uses a proxy server.
Property	proxies.trusted.ipranges
Example Value:	proxies.trusted.ipranges = 127.0.0.1

Informational Note:	By default, only proxies running on localhost (127.0.0.1) and the `dspace.ui.url` (public IP address) are "trusted". This allows our Angular User Interface to communicate with the REST API via a trusted proxy, which is required for Angular Universal (for SEO support). You can specify a range by only listing the first three ip-address blocks, e.g. 128.177.243 You can list multiple IP addresses or ranges by comma-separating them.
Property	proxies.trusted.include_ui_ip
Example Value:	proxies.trusted.include_ui_ip = true
Informational Note:	This setting specifies whether to automatically trust IP address of the `dspace.ui.url` as a proxy. By default, this is always set to true to ensure the UI is fully trusted by the backend. However, if you are not using the Angular UI, you may choose to set this to "false" in order to only trust proxies running on localhost (127.0.0.1) by default.
Property	server.forward-headers-strategy
Example Value:	server.forward-headers-strategy = FRAMEWORK
Informational Note:	This is a Spring Boot setting which may be overridden/specified in your local.cfg. By default, Spring Boot does not automatically use X-Forwarded-* Headers when generating links (and similar) in the REST API. When using a proxy in front of the REST API, you may need to modify this setting: • NATIVE = allows your web server to natively support standard Forwarded headers • FRAMEWORK = enables Spring Framework's built in filter to manage these headers in Spring Boot. (This value may be useful to set for DSpace if you find that X-Forwarded headers are not working) • NONE = default value. Forwarded headers are ignored For more information see the Spring Boot docs at https://docs.spring.io/spring-boot/docs/current/reference/html/howto.html#howto-use-behind-a-proxy-server

Configuring Media Filters

Media or Format Filters are classes used to generate derivative or alternative versions of content or bitstreams within DSpace. For example, the PDF Media Filter will extract textual content from PDF bitstreams, the JPEG Media Filter can create thumbnails from image bitstreams.

Media Filters are configured as Named Plugins, with each filter also having a separate configuration setting (in *dspace.cfg*) indicating which formats it can process. The default configuration is shown below.

Property:	`filter.plugins`

Example Value:	`filter.plugins = PDF Text Extractor`
	`filter.plugins = Html Text Extractor`
	`filter.plugins = Word Text Extractor`
	`filter.plugins = JPEG Thumbnail`

Informational Note:	This setting lists the names of all enabled MediaFilter or FormatFilter plugins. To enable multiple plugins, list them on separate lines (as shown above) or provide a comma separated list.
Property:	`plugin.named.org.dspace.app.mediafilter.FormatFilter`

Example Value:	`plugin.named.org.dspace.app.mediafilter.FormatFilter = org.dspace.app.mediafilter.PDFFilter = PDF Text Extractor`
	`plugin.named.org.dspace.app.mediafilter.FormatFilter = org.dspace.app.mediafilter.HTMLFilter = HTML Text Extractor`
	`plugin.named.org.dspace.app.mediafilter.FormatFilter = org.dspace.app.mediafilter.WordFilter = Word Text Extractor`
	`plugin.named.org.dspace.app.mediafilter.FormatFilter = org.dspace.app.mediafilter.JPEGFilter = JPEG Thumbnail`
	`plugin.named.org.dspace.app.mediafilter.FormatFilter = org.dspace.app.mediafilter.BrandedPreviewJPEGFilter = Branded Preview JPEG`

Informational Note:	Assign "human-understandable" names to each filter. These names are used to enable/disable plugins using "filter.plugins" setting above. As with the previous setting, multiple plugins can be listed here on separate lines (or comma separated)

Property:	`filter.org.dspace.app.mediafilter.PDFFilter.inputFormats`
	`filter.org.dspace.app.mediafilter.HTMLFilter.inputFormats`
	`filter.org.dspace.app.mediafilter.WordFilter.inputFormats`
	`filter.org.dspace.app.mediafilter.JPEGFilter.inputFormats`
	`filter.org.dspace.app.mediafilter.BrandedPreviewJPEGFilter.inputFormats`

Example Value:	`filter.org.dspace.app.mediafilter.PDFFilter.inputFormats = Adobe PDF`
	`filter.org.dspace.app.mediafilter.HTMLFilter.inputFormats = HTML, Text`
	`filter.org.dspace.app.mediafilter.WordFilter.inputFormats = Microsoft Word`
	`filter.org.dspace.app.mediafilter.JPEGFilter.inputFormats = BMP, GIF, JPEG, \`
	`image/png`
	`filter.org.dspace.app.mediafilter.BrandedPreviewJPEGFilter.inputFormats = BMP, \`
	`GIF, JPEG, image/png`

Informational Note:	Configure each filter's input format(s). These must match format names in the DSpace file format registry.
Property:	filter.org.dspace.app.mediafilter.publicPermission
Example Value:	filter.org.dspace.app.mediafilter.publicPermission = JPEGFilter
	Optionally, configure filter(s) which should **always** create publicly accessible bitstreams (e.g. useful if you want thumbnails to always be publicly accessible). By default, any bitstreams created by a filter will inherit the same permissions as the original file (e.g. if original image is access restricted, then thumbnail will also be access restricted by default).
Property:	`pdffilter.largepdfs`
Example Value:	`pdffilter.largepdfs = true`
Informational Note:	It this value is set for "true", all PDF extractions are written to temp files as they are indexed. This is slower, but helps to ensure that PDFBox software DSpace uses does not eat up all your memory.
Property:	`pdffilter.skiponmemoryexception`
Example Value:	`pdffilter.skiponmemoryexception = true`

Informational Note:	If this value is set for "true", PDFs which still result in an "Out of Memory" error from PDFBox are skipped over. These problematic PDFs will never be indexed until memory usage can be decreased in the PDFBox software.

Names are assigned to each filter using the `plugin.named.org.dspace.app.mediafilter.FormatFilter` field (e.g. by default the PDFilter is named "PDF Text Extractor".

Finally, the appropriate `filter.<class path>.inputFormats` defines the valid input formats which each filter can be applied. These format names **must match** the `short description` field of the Bitstream Format Registry.

You can also implement more dynamic or configurable Media/Format Filters which extend `SelfNamedPlugin` .

More Information on MediaFilters

For more information on Media/Format Filters, see the section on Mediafilters for Transforming DSpace Content.

Crosswalk and Packager Plugin Settings

The subsections below give configuration details based on the types of crosswalks and packager plugins you need to implement.

More Information on Packagers & Crosswalks

For more information on using Packagers and Crosswalks, see the section on Importing and Exporting Content via Packages.

Configurable MODS Dissemination Crosswalk

The MODS crosswalk is a self-named plugin. To configure an instance of the MODS crosswalk, add a property to the DSpace configuration starting with "c`rosswalk.mods.properties.`"; the final word of the property name becomes the plugin's name. For example, a property name `crosswalk.mods.properties.MODS` defines a crosswalk plugin named "`MODS`".

The value of this property is a path to a separate properties file containing the configuration for this crosswalk. The pathname is relative to the DSpace configuration directory, i.e. the `config` subdirectory of the DSpace install directory. Example from the `dspace.cfg` file:

Properties:	`crosswalk.mods.properties.MODS` `crosswalk.mods.properties.mods`
Example Values:	`crosswalk.mods.properties.MODS = crosswalks/mods.properties` `crosswalk.mods.properties.mods = crosswalks/mods.properties`
Informational Note:	This defines a crosswalk named MODS whose configuration comes from the file `[dspace]/config/crosswalks/mods.properties` . (In the above example, the lower-case name was added for OAI-PMH)

The MODS crosswalk properties file is a list of properties describing how DSpace metadata elements are to be turned into elements of the MODS XML output document. The property name is a concatenation of the metadata schema, element name, and optionally the qualifier. For example, the *contributor.author* element in the native Dublin Core schema would be: *dc.contributor.author*. The value of the property is a line containing two segments separated by the vertical bar ("`|`"_): The first part is an XML fragment which is copied into the output document. The second is an XPath expression describing where in that fragment to put the value of the metadata element. For example, in this property:

```
dc.contributor.author = <mods:name>
                            <mods:role>
                                <mods:roleTerm type="text">author</mods:roleTerm>
                                </mods:role>
                            <mods:namePart>%s</mods:namePart>
                        </mods:name>
```

Some of the examples include the string "`%s`" in the prototype XML where the text value is to be inserted, but don't pay any attention to it, it is an artifact that the crosswalk ignores. For example, given an author named *Jack Florey*, the crosswalk will insert

```
<mods:name>
    <mods:role>
        <mods:roleTerm type="text">author</mods:roleTerm>
    </mods:role>
    <mods:namePart>Jack Florey</mods:namePart>
</mods:name>
```

into the output document. Read the example configuration file for more details.

XSLT-based Crosswalks

The XSLT crosswalks use XSL stylesheet transformation (XSLT) to transform an XML-based external metadata format to or from DSpace's internal metadata. XSLT crosswalks are much more powerful and flexible than the configurable MODS and QDC crosswalks, but they demand some esoteric knowledge (XSL stylesheets). Given that, you can create all the crosswalks you need just by adding stylesheets and configuration lines, without touching any of the Java code.

The default settings in the `dspace.cfg` file for submission crosswalk:

Properties:	`crosswalk.submission.MODS.stylesheet`
Example Value:	`crosswalk.submission.MODS.stylesheet = crosswalks/mods-submission.xsl`
Informational Note:	Configuration XSLT-driven submission crosswalk for MODS

As shown above, there are three (3) parts that make up the properties "key":

```
crosswalk.submission.PluginName.stylesheet =
          1        2        3               4
```

`crosswalk` first part of the property key.
`submission` second part of the property key.
`PluginName` is the name of the plugin. The *path* value is the path to the file containing the crosswalk stylesheet (relative to `/[dspace]/config`).
Here is an example that configures a crosswalk named "LOM" using a stylesheet in `[dspace]/config/crosswalks/d-lom.xsl`:
`crosswalk.submission.LOM.stylesheet = crosswalks/d-lom.xsl`
A dissemination crosswalk can be configured by starting with the property key *crosswalk.dissemination*. Example:
`crosswalk.dissemination.PluginName.stylesheet = path`
The *PluginName* is the name of the plugin (!) . The *path* value is the path to the file containing the crosswalk stylesheet (relative to `/[dspace]/config`).

You can make two different plugin names point to the same crosswalk, by adding two configuration entries with the same path:

```
crosswalk.submission.MyFormat.stylesheet = crosswalks/myformat.xslt
     crosswalk.submission.almost_DC.stylesheet = crosswalks/myformat.xslt
```

The dissemination crosswalk must also be configured with an XML Namespace (including prefix and URI) and an XML schema for its output format. This is configured on additional properties in the DSpace configuration:

```
crosswalk.dissemination.PluginName.namespace.Prefix = namespace-URI
     crosswalk.dissemination.PluginName.schemaLocation = schemaLocation value
```

For example:

```
crosswalk.dissemination.qdc.namespace.dc = http://purl.org/dc/elements/1.1/
     crosswalk.dissemination.qdc.namespace.dcterms = http://purl.org/dc/terms/
     crosswalk.dissemination.qdc.schemalocation = http://purl.org/dc/elements/1.1/ \
     http://dublincore.org/schemas/xmls/qdc/2003/04/02/qualifieddc.xsd
```

If you remove all XSLTDisseminationCrosswalks please disable the XSLTDisseminationCrosswalk in the list of selfnamed plugins. If no XSLTDisseminationCrosswalks are configured but the plugin is loaded the PluginManager will log an error message ("Self-named plugin class "org.dspace.content.crosswalk.XSLTDisseminationCrosswalk" returned null or empty name list!").

Testing XSLT Crosswalks

The XSLT crosswalks will automatically reload an XSL stylesheet that has been modified, so you can edit and test stylesheets without restarting DSpace. You can test a crosswalk by using a command-line utitlity. To test a dissemination crosswalk you have to run:

```
[dspace]/bin/dspace dsrun org.dspace.content.crosswalk.XSLTDisseminationCrosswalk <plugin name> <handle>
[output-file]
```

For example, you can test the marc plugin on the handle 123456789/3 with:

```
[dspace]/bin/dspace dsrun org.dspace.content.crosswalk.XSLTDisseminationCrosswalk marc 123456789/3
```

Informations from the script will be printed to stderr while the XML output of the dissemination crosswalk will be printed to stdout. You can give a third parameter containing a filename to write the output into a file, but be careful: the file will be overwritten if it exists. When you are working on XSLTCrosswalks it is very helpful to get the original XML on which the XSLT processor works. Use the crosswalk dim to get the original XML:

```
[dspace]/bin/dspace dsrun org.dspace.content.crosswalk.XSLTDisseminationCrosswalk dim 123456789/3
```

Testing a submission crosswalk works quite the same way. Use the following command-line utility, it calls the crosswalk plugin to translate an XML document you submit, and displays the resulting intermediate XML (DIM). Invoke it with:

```
[dspace]/bin/dspace dsrun
     org.dspace.content.crosswalk.XSLTIngestionCrosswalk [-l] <plugin name> <input-file>
```

where <*plugin name*> is the name of the crosswalk plugin to test (e.g. "LOM"), and <*input-file*> is a file containing an XML document of metadata in the appropriate format.

Add the `-l` option to pass the ingestion crosswalk a list of elements instead of a whole document, as if the List form of the ingest() method had been called. This is needed to test ingesters for formats like DC that get called with lists of elements instead of a root element.

Configurable Qualified Dublin Core (QDC) dissemination crosswalk

The QDC crosswalk is a self-named plugin. To configure an instance of the QDC crosswalk, add a property to the DSpace configuration starting with "`crosswalk.qdc.properties.`"; the final word of the property name becomes the plugin's name. For example, a property name `crosswalk.qdc.properties.QDC` defines a crosswalk plugin named "`QDC`".

The following is from *dspace.cfg* file:

Properties:	`crosswalk.qdc.namspace.qdc.dc`
Example Value:	`crosswalk.qdc.namspace.qdc.dc = http://purl.org/dc/elements/1.1_`
Properties:	`crosswalk.qdc.namspace.qdc.dcterms`
Example Value:	`crosswalk.qdc.namspace.qdc.dc = http://purl.org/dc/terms/_`
Properties:	`crosswalk.qdc.schemaLocation.QDC`

Example Value:	`crosswalk.qdc.schemaLocation.QDC = http://www.purl.org/dc/terms \`
	`http://dublincore.org/schemas/xmls/qdc/2006/01/06/dcterms.xsd \`
	`http://purl.org/dc/elements/1.1 \`
	`http://dublincore.org/schemas/xmls/qdc/2006/01/06/dc.xsd`

Properties:	`crosswalk.qdc.properties.QDC`
Example Value:	`crosswalk.qdc.properties.QDC = crosswalks/QDC.properties`
Informational Note:	Configuration of the QDC Crosswalk dissemination plugin for Qualified DC. *(Add lower-case name for OAI-PMH. That is, change QDC to qdc.)*}}

In the property key "`crosswalk.qdc.properties.QDC`" the value of this property is a path to a separate properties file containing the configuration for this crosswalk. The pathname is relative to the DSpace configuration directory `/[dspace]/config` . Referring back to the "Example Value" for this property key, one has `crosswalks/qdc.properties` which defines a crosswalk named `QDC` whose configuration comes from the file `[dspace]/config/crosswalks/qdc.properties` .

You will also need to configure the namespaces and schema location strings for the XML output generated by this crosswalk. The namespaces properties names are formatted:

`crosswalk.qdc.namespace.`*prefix* = *uri*

where *prefix* is the namespace prefix and *uri* is the namespace URI. See the above Property and Example Value keys as the default dspace.cfg has been configured.

The QDC crosswalk properties file is a list of properties describing how DSpace metadata elements are to be turned into elements of the Qualified DC XML output document. The property name is a concatenation of the metadata schema, element name, and optionally the qualifier. For example, the `contributor.author` element in the native Dublin Core schema would be: `dc.contributor.author` . The value of the property is an XML fragment, the element whose value will be set to the value of the metadata field in the property key.

For example, in this property:

```
dc.coverage.temporal = <dcterms:temporal />
```

the generated XML in the output document would look like, e.g.:
```
<dcterms:temporal>Fall, 2005</dcterms:temporal>
```

Configuring Crosswalk Plugins

Ingestion crosswalk plugins are configured as named or self-named plugins for the interface `org.dspace.content.crosswalk.IngestionCrosswalk`. Dissemination crosswalk plugins are configured as named or self-named plugins for the interface `org.dspace.content.crosswalk.DisseminationCrosswalk`.

You can add names for existing crosswalks, add new plugin classes, and add new configurations for the configurable crosswalks as noted below.

Configuring Packager Plugins

Package ingester plugins are configured as named or self-named plugins for the interface `org.dspace.content.packager.PackageIngester` . Package disseminator plugins are configured as named or self-named plugins for the interface `org.dspace.content.packager.PackageDisseminator` .

You can add names for the existing plugins, and add new plugins, by altering these configuration properties. See the Plugin Manager architecture for more information about plugins.

Event System Configuration

If you are unfamiliar with the Event System in DSpace, and require additional information with terms like "Consumer" and "Dispatcher" please refer to Event Mechanism.

Property:	`event.dispatcher.default.class`
Example Value:	`event.dispatcher.default.class = org.dspace.event.BasicDispatcher`
Informational Note:	This is the default synchronous dispatcher (Same behavior as traditional DSpace).
Property:	`event.dispatcher.default.consumers`
Example Value:	`event.dispatcher.default.consumers = search, browse, eperson`
Informational Note:	This is the default synchronous dispatcher (Same behavior as traditional DSpace).
Property:	`event.dispatcher.noindex.class`
Example Value:	`event.dispatcher.noindex.class = org.dspace.event.BasicDispatcher`
Informational Note:	The noindex dispatcher will not create search or browse indexes (useful for batch item imports).
Property:	`event.dispatcher.noindex.consumers`
Example Value:	`event.dispatcher.noindex.consumers = eperson`
Informational Note:	The noindex dispatcher will not create search or browse indexes (useful for batch item imports).
Property:	`event.consumer.discovery.class`
Example Value:	`event.consumer.discovery.class = org.dspace.discovery.IndexEventConsumer`
Informational Note:	Consumer to maintain the search/browse (Discovery) index.
Property:	`event.consumer.discovery.filters`
Example Value:	`event.consumer.discovery.filters = Community | Collection | Item | Bundle | Site+Add | Create | Modify | Modify_Metadata | Delete | Remove`
Informational Note:	Consumer to maintain the search/browse (Discovery) index.
Property:	`event.consumer.eperson.class`
Example Value:	`event.consumer.eperson.class = org.dspace.eperson.EPersonConsumer`

Informational Note:	Consumer related to EPerson changes
Property:	`event.consumer.eperson.filters`
Example Value:	`event.consumer.eperson.filters = EPerson+Create`
Informational Note:	Consumer related to EPerson changes
Property:	`event.consumer.test.class`
Example Value:	`event.consumer.test.class = org.dspace.event.TestConsumer`
Informational Note:	Test consumer for debugging and monitoring. **Commented out by default.**
Property:	`event.consumer.test.filters`
Example Value:	`event.consumer.test.filters = All+All`
Informational Note:	Test consumer for debugging and monitoring. **Commented out by default.**
Property:	`testConsumer.verbose`
Example Value:	`testConsumer.verbose = true`
Informational Note:	Set this to true to enable testConsumer messages to standard output. **Commented out by default.**

Embargo

DSpace embargoes utilize standard metadata fields to hold both the "terms" and the "lift date". Which fields you use are configurable, and no specific metadata element is dedicated or predefined for use in embargo. Rather, you specify exactly what field you want the embargo system to examine when it needs to find the terms or assign the lift date.

Property:	`embargo.field.terms`
Example Value:	`embargo.field.terms = SCHEMA.ELEMENT.QUALIFIER`
Informational Note:	Embargo terms will be stored in the item metadata. This property determines in which metadata field these terms will be stored. An example could be dc.embargo.terms
Property:	`embargo.field.lift`
Example Value:	`embargo.field.lift = SCHEMA.ELEMENT.QUALIFIER`
Informational Note:	The Embargo lift date will be stored in the item metadata. This property determines in which metadata field the computed embargo lift date will be stored. You may need to create a DC metadata field in your Metadata Format Registry if it does not already exist. An example could be dc.embargo.liftdate
Property:	`embargo.terms.open`
Example Value:	`embargo.terms.open = forever`
Informational Note:	You can determine your own values for the embargo.field.terms property (see above). This property determines what the string value will be for indefinite embargos. The string in terms field to indicate indefinite embargo.
Property:	`plugin.single.org.dspace.embargo.EmbargoSetter`
Example Value:	`plugin.single.org.dspace.embargo.EmbargoSetter = org.dspace.embargo.DefaultEmbargoSetter`

Informational Note:	To implement the business logic to set your embargos, you need to override the EmbargoSetter class. If you use the value DefaultEmbargoSetter, the default implementation will be used.
Property:	`plugin.single.org.dspace.embargo.EmbargoLifter`
Example Value:	`plugin.single.org.dspace.embargo.EmbargoLifter = org.dspace.embargo.DefaultEmbargoLifter`
Informational Note:	To implement the business logic to lift your embargos, you need to override the EmbargoLifter class. If you use the value DefaultEmbargoLifter, the default implementation will be used.

More Embargo Details

More details on Embargo configuration, including specific examples can be found in the Embargo section of the documentation.

Checksum Checker Settings

DSpace comes with a Checksum Checker script (`[dspace]/bin/dspace checker`) which can be scheduled to verify the checksum of every item within DSpace. Since DSpace calculates and records the checksum of every file submitted to it, this script is able to determine whether or not a file has been changed (either manually or by some sort of corruption or virus). The idea being that the earlier you can identify a file has changed, the more likely you'd be able to recover it (assuming it was not a wanted change).

Property:	`plugin.single.org.dspace.checker.BitstreamDispatcher`
Example Value:	`plugin.single.org.dspace.checker.BitstreamDispatcher = org.dspace.checker.SimpleDispatcher`
Informational Note:	The Default dispatcher is case non is specified.
Property:	`checker.retention.default`
Example Value:	`checker.retention.default = 10y`
Informational Note:	This option specifies the default time frame after which all checksum checks are removed from the database (defaults to 10 years). This means that after 10 years, all successful or unsuccessful matches are removed from the database.
Property:	`checker.retention.CHECKSUM_MATCH`
Example Value:	`checker.retention.CHECKSUM_MATCH = 8w`
Informational Note:	This option specifies the time frame after which a successful match will be removed from your DSpace database (defaults to 8 weeks). This means that after 8 weeks, all successful matches are automatically deleted from your database (in order to keep that database table from growing too large).

More Checksum Checking Details

For more information on using DSpace's built-in Checksum verification system, see the section on Validating CheckSums of Bitstreams.

Item Export and Download Settings

It is possible for an authorized user to request a complete export and download of a DSpace item in a compressed zip file. This zip file may contain the following:
dublin_core.xml
license.txt
contents *(listing of the contents)*
handle *file itself and the extract file if available*

The configuration settings control several aspects of this feature:

Property:	`org.dspace.app.itemexport.work.dir`
Example Value:	`org.dspace.app.itemexport.work.dir = ${dspace.dir}/exports`

Informational Note:	The directory where the exports will be done and compressed.
Property:	`org.dspace.app.itemexport.download.dir`
Example Value:	`org.dspace.app.itemexport.download.dir = ${dspace.dir}/exports/download`
Informational Note	The directory where the compressed files will reside and be read by the downloader.
Property:	`org.dspace.app.itemexport.life.span.hours`
Example Value:	`org.dspace.app.itemexport.life.span.hours = 48`
Informational Note	The length of time in hours each archive should live for. When new archives are created this entry is used to delete old ones.
Property:	`org.dspace.app.itemexport.max.size`
Example Value:	`org.dspace.app.itemexport.max.size = 200`
Informational Note	The maximum size in Megabytes (Mb) that the export should be. This is enforced before the compression. Each bitstream's size in each item being exported is added up, if their cumulative sizes are more than this entry the export is not kicked off.

Subscription Emails

DSpace, through some advanced installation and setup, is able to send out an email to collections that a user has subscribed. The user who is subscribed to a collection is emailed each time an item id added or modified. The following property key controls whether or not a user should be notified of a modification.

Property:	`eperson.subscription.onlynew`
Example Value:	`eperson.subscription.onlynew = true`
Informational Note:	For backwards compatibility, the subscription emails by default include any modified items. The property key is COMMENTED OUT by default.

Hiding Metadata

It is possible to hide metadata from public consumption, so that it's only available to users with WRITE permissions on the Item. (NOTE: Prior to 7.6.1, Administrator privileges were required for hidden metadata. This was modified to allow users to submit hidden metadata fields, as well as allow Community /Collection Admins to see hidden metadata.)

Property:	`metadata.hide.dc.description.provenance`
Example Value:	`metadata.hide.dc.description.provenance = true`
Informational Note:	Hides the metadata in the property key above except to the administrator. Fields named here are hidden in the following places UNLESS the logged-in user has WRITE permissions on the Item: 1. REST API (and therefore the User Interface) 2. RDF (everywhere as there is currently no possibility to authenticate) 3. OAI-PMH server (everywhere as there is currently no possibility to authenticate) To designate a field as hidden, add a property here in the form: `metadata.hide.SCHEMA.ELEMENT.QUALIFIER = true`. This default configuration hides the `dc.description.provenance` field, since that usually contains email addresses which ought to be kept private and is mainly of interest to administrators.

Settings for the Submission Process

Property:	`webui.submit.upload.required`
Example Value:	`webui.submit.upload.required = true`
Informational Note:	Whether or not a file is **required** to be uploaded during the "Upload" step in the submission process. The default is true. If set to "false", then the submitter (human being) has the option to skip the uploading of a file.

Configuring the Sherpa/RoMEO Integration

DSpace has integration with the Sherpa/RoMEO API in order to allow importing data from Sherpa/RoMEO during the submission You must register for a free API key (see below for details).

Property:	sherpa.romeo.url
Example Value:	sherpa.romeo.url = https://v2.sherpa.ac.uk/cgi/retrieve
Informational Note:	The Sherpa/RoMEO endpoint.
Property:	sherpa.romeo.apikey
Example Value:	`sherpa.romeo.apikey = YOUR-API-KEY`
Informational Note:	Allow to use a specific API key to raise the usage limit (500 calls/day for unregistred user). You MUST register for a free api access key at https://v2.sherpa.ac.uk/api/

The functionality rely on understanding to which Journal (ISSN) is related the submitting item. This is done out of box looking to some item metadata but a different strategy can be used as for example look to a metadata authority in the case that the Sherpa/RoMEO autocomplete for Journal is used (see AuthorityControlSettings)

The strategy used to discover the Journal related to the submission item is defined in the spring file **/config/spring/api/sherpa.xml**

```
<bean class="org.dspace.app.sherpa.submit.SHERPASubmitConfigurationService"
                id="org.dspace.app.sherpa.submit.SHERPASubmitConfigurationService">
                <property name="issnItemExtractors">
                        <list>
                                <bean class="org.dspace.app.sherpa.submit.MetadataValueISSNExtractor">
                                        <property name="metadataList">
                                                <list>
                                                        <value>dc.identifier.issn</value>
                                                </list>
                                        </property>
                                </bean>
                                <!-- Use the follow if you have the SHERPARoMEOJournalTitle enabled
                                <bean class="org.dspace.app.sherpa.submit.MetadataAuthorityISSNExtractor">
                                        <property name="metadataList">
                                                <list>
                                                        <value>dc.title.alternative</value>
                                                </list>
                                        </property>
                                </bean>  -->
                        </list>
                </property>
        </bean>
```

Configuring Creative Commons License

The following configurations are for the Creative Commons license step in the submission process. Submitters are given an opportunity to select a Creative Common license to accompany the item. Creative Commons licenses govern the use of the content. For further details, refer to the Creative Commons website at http://creativecommons.org .

Creative Commons licensing is optionally available and may be configured for any given collection that has a defined submission sequence, or be part of the "default" submission process. This process is described in the Submission User Interface section of this manual. There is a Creative Commons step already defined, but it is commented out, so enabling Creative Commons licensing is typically just a matter of uncommenting that step.

When enabled, the Creative Commons public API is utilized. This allows DSpace to store metadata references to the selected CC license, while also storing the CC License as a bitstream. The following CC License information are captured:

- The URL of the CC License is stored in the "dc.rights.uri" metadata field (or whatever field is configured in the "cc.license.uri" setting below)
- The name of the CC License is stored in the "dc.rights" metadata field (or whatever field is configured in the "cc.license.name" setting below). This only occurs if "cc.submit.setname=true" (default value)
- The RDF version of the CC License is stored in a bitstream named "license_rdf" in the CC-LICENSE bundle (as long as "cc.submit.addbitstream=true", which is the default value)

The following configurations (in dspace.cfg) relate to the Creative Commons license process:

Property:	`cc.api.rooturl`
Example Value:	`cc.api.rooturl = http://api.creativecommons.org/rest/1.5`
Informational Note:	Generally will never have to assign a different value - this is the base URL of the Creative Commons service API.
Property:	`cc.license.uri`
Example Value:	`cc.license.uri = dc.rights.uri`
Informational Note:	The field that holds the Creative Commons license URI.
Property:	`cc.license.name`
Example Value:	`cc.license.name = dc.rights`
Informational Note:	The field that holds the Creative Commons license Name.
Property:	`cc.submit.setname`
Example Value:	`cc.submit.setname = true`
Informational Note:	If true, the license assignment will add the field configured with the "cc.license.name" with the name of the CC license; if false, only "cc.license.uri" field is added.
Property:	`cc.submit.addbitstream`
Example Value:	`cc.submit.addbitstream = true`
Informational Note:	If true, the license assignment will add a bitstream with the CC license RDF; if false, only metadata field(s) are added.
Property:	`cc.license.classfilter`
Example Value:	`cc.license.classfilter = recombo,mark`
Informational Note:	This list defines the values that will be excluded from the license (class) selection list, as defined by the web service at the URL: http://api.creativecommons.org/rest/1.5/classes
Property:	`cc.license.jurisdiction`

Example Value:	`# Configure specific jurisdiction via language code:`
	`cc.license.jurisdiction = nz`
	`# Set property to 'empty' to overwrite default value from dspace.cfg:`
	`cc.license.jurisdiction =`

Informational Note:	Should a jurisdiction be used? If so, which one? See http://creativecommons.org/international/ for a list of possible codes (e.g. nz = New Zealand, uk = England and Wales, jp = Japan) Commenting out this field will cause DSpace to select the latest, unported CC license (currently version 4.0). However, as Creative Commons 4.0 does not provide jurisdiction specific licenses, if you specify this setting, your DSpace will continue to use older, Creative Commons 3.0 jurisdiction licenses. If commenting out this field in your local.cfg doesn't seem to work, double check your dspace.cfg. If cc.license.jurisdiction is set to a specific value in dspace.cfg, commenting it out in local.cfg will result in your DSpace using the value configured in dspace.cfg. Possible workaround: Set the property to an 'empty value' like in the example above.
Property	`cc.license.locale`
Example Value:	`cc.license.locale` = en
Informational Note:	Locale to be used (in the form: language or language_country), e.g. "en" or "en_US" If no default locale is defined the Creative Commons default locale will be used.

WEB User Interface Configurations

General Web User Interface Configurations

Property:	`webui.licence_bundle.show`
Example Value:	`webui.licence_bundle.show = false`
Informational Note:	Sets whether to display the contents of the license bundle (often just the deposit license in the standard DSpace installation). **UNSUPPORTED in DSpace 7.x**
Property:	`thumbnail.maxwidth`
Example Value:	`thumbnail.maxwidth = 80`
Informational Note:	This property sets the maximum width of generated thumbnails that are being displayed on item pages.
Property:	`thumbnail.maxheight`
Example Value:	`thumbnail.maxheight = 80`
Informational Note:	This property sets the maximum height of generated thumbnails that are being displayed on item pages.
Property:	`webui.preview.maxwidth`
Example Value:	`webui.preview.maxwidth = 600`
Informational Note:	This property sets the maximum width for the preview image. Only used for BrandedPreviewJPEGFilter
Property:	`webui.preview.maxheight`
Example Value:	`webui.preview.maxheight = 600`
Informational Note:	This property sets the maximum height for the preview image. Only used for BrandedPreviewJPEGFilter
Property:	`webui.preview.brand`
Example Value:	`webui.preview.brand = My Institution Name`
Informational Note:	This is the brand text that will appear with the image. Only used for BrandedPreviewJPEGFilter
Property:	`webui.preview.brand.abbrev`
Example Value:	`webui.preview.brand.abbrev = MyOrg`
Informational Note:	An abbreviated form of the full Branded Name. This will be used when the preview image cannot fit the normal text. Only used for BrandedPreviewJPEGFilter
Property:	`webui.preview.brand.height`
Example Value:	`webui.preview.brand.height = 20`

Informational Note:	The height (in px) of the brand. Only used for BrandedPreviewJPEGFilter
Property:	`webui.preview.brand.font`
Example Value:	`webui.preview.brand.font = SansSerif`
Informational Note:	This property sets the font for your Brand text that appears with the image. Only used for BrandedPreviewJPEGFilter
Property:	`webui.preview.brand.fontpoint`
Example Value:	`webui.preview.brand.fontpoint = 12`
Informational Note:	This property sets the font point (size) for your Brand text that appears with the image. Only used for BrandedPreviewJPEGFilter
Property:	`webui.preview.dc`
Example Value:	`webui.preview.dc = rights`
Informational Note:	The Dublin Core field that will display along with the preview. This field is optional. Only used for BrandedPreviewJPEGFilter

Item Counts in user interface

Available in 7.6 or later

Optionally, you can enable item counts to be displayed in the user interface for every Community and Collection. This uses the same configuration that was in place for DSpace 6 and earlier.

Property:	`webui.strengths.show`
Example Value:	`webui.strengths.show = false`
Informational Note:	When "true" this will display the count of archived items (in the User Interface's browse screens). By default this is "false" (disabled). When enabled, the counts may be counted in real time, or fetched from the cache (see next option).
Property:	`webui.strengths.cache`
Example Value:	`webui.strengths.cache = false`
Informational Note:	When showing the strengths (i.e. item counts), should they be counted in real time, or fetched from the cache. Counts fetched in real time will perform an actual count of the index contents every time a page with this feature is requested, which may not scale. If you set the property key is set to cache ("true"), the counts will be cached on first load.

Browse Index Configuration

The browse indexes for DSpace can be extensively configured. These configurations are used by Discovery. This section of the configuration allows you to take control of the indexes you wish to browse, and how you wish to present the results. The configuration is broken into several parts: defining the indexes, defining the fields upon which users can sort results, defining truncation for potentially long fields (e.g. authors), setting cross-links between different browse contexts (e.g. from an author's name to a complete list of their items), how many recent submissions to display, and configuration for item mapping browse.

Property:	`webui.browse.index.<n>`
Example Value:	`webui.browse.index.1 = dateissued:item:dateissued` `webui.browse.index.2 = author:metadata:dc.contributor.*\,dc.creator:text`
Informational Note:	This is an example of how one "Defines the Indexes". See "Defining the Indexes" in the next sub-section.
Property:	`webui.itemlist.sort-option.<n>`
Example Value:	`webui.itemlist.sort-option.1 = title:dc.title:title`
Informational Note:	This is an example of how one "Defines the Sort Options". See "Defining Sort Options" in the following sub-section.

Defining the storage of the Browse Data

Optionally, you may configure a custom implementation use for the Browse DAOs both for read operations (create/update operations are handled by Event Consumers). However, as of DSpace 6, DSpace only includes one out-of-the-box option:

- SOLR Browse Engine (SOLR DAOs), default since DSpace 4.0 - This enables Apache Solr to be utilized as a backend for all browsing of DSpace. This option requires that you have Discovery (Solr search/browse engine) enabled in your DSpace.

Property:	`browseDAO.class`
Example Value:	browseDAO.class = org.dspace.browse.SolrBrowseDAO
Informational Note:	This property configures the Java class that is used for READ operations by the Browse System. **You need to have Discovery enabled (this is the default since DSpace 4.0) to use the Solr Browse DAOs**

Defining the Indexes

If you make changes in this section be sure to update your SOLR indexes running the Discovery Maintenance Script, see Discovery

DSpace comes with four default indexes pre-defined: author, title, date issued, and subjects. Users may also define additional indexes or re-configure the current indexes for different levels of specificity. For example, the default entries that appear in the dspace.cfg as default installation:

```
webui.browse.index.1 = dateissued:item:dateissued
webui.browse.index.2 = author:metadata:dc.contributor.*\,dc.creator:text
webui.browse.index.3 = title:item:title
webui.browse.index.4 = subject:metadata:dc.subject.*:text
#webui.browse.index.5 = dateaccessioned:item:dateaccessioned
```

There are two types of indexes which are provided in this default integration:

- "`item`" indexes which have a format of `webui.browse.index.<n> = <index-name> : item : <sort-type> : (asc | desc)`
- "`metadata`" indexes which have a format of `webui.browse.index.<n> = <index-name> : metadata : <comma-separated-list-of-metadata-fields> : (date | text) : (asc | dec) : <sort-type>`

Please notice that the punctuation is paramount in typing this property key in the `dspace.cfg` file. The following table explains each element:

Element	Definition and Options (if available)
`webui.browse.index.<n>`	*n* is the index number. The index numbers must start from 1 and increment continuously by 1 thereafter. Deviation from this will cause an error during install or a configuration update. So anytime you add a new browse index, remember to increase the number. (Commented out index numbers may be used over again).
`<index-name>`	The name by which the index will be identified. In order for the DSpace UI to display human-friendly description for this index, you'll need to update the UI's language packs (e.g. `src/assets/i18n/en.json5`) to include a key using this index name, for example: • `browse.metadata.<index-name> = "MyField",` • `browse.metadata.<index-name>.breadcrumbs = "Browse by MyField",`
`(metadata|item)`	Only two options are available: "`metadata`" or "`item`" • "`metadata`" indexes allow you to index all items based on one or more metadata fields. The list of fields should be provided as part of the "metadata" configuration. Only items which have values for these fields will appear in this index (e.g. if you have a "metadata" index for "`dc.subject.*`", an item will not appear in that browse/search if it doesn't have a "`dc.subject.*`" value). The browse index will have to parts: first it lists all values of the specified metadata fields. If the user select one of these values the index lists all items in which the specified metadata field is assigned with the selected value. ◦ Note: If you set a <sort-type> to be used, this sort type is not used on the values of the metadata fields but on the order of the items when listing all items that have a specific value of the metadata field. • "`item`" indexes provide you with a browseable list of ALL items in the site, sorted by a particular metadata field. The field this index is sorted by is referenced by `<sort-option-name>` (which should refer to a corresponding "`webui.itemlist.sort-option.<n>`" setting... see Defining Sort Options below for more information)
`<schema-prefix>`	(Only for "metadata" indexes) The schema used for the field to be index. First part of a metadata field name. The default is dc (for Dublin Core).
`<element>`	(Only for "metadata" indexes) The schema element. Second part of a metadata field name. In Dublin Core, for example, the author element is referred to as "Contributor". The user should consult the default Dublin Core Metadata Registry table in Appendix A.

`<qualifier>`	(Only for "metadata" indexes) This is the qualifier to the <element> component. Third part of a metadata field name. The user has two choices: an asterisk "*" or a proper qualifier of the element. The asterisk is a wildcard and causes DSpace to index all types of the schema element. For example, if you have the element "contributor" and the qualifier "*" then you would index all contributor data regardless of the qualifier. Another example, you have the element "subject" and the qualifier "lcsh" would cause the indexing of only those fields that have the qualifier "lcsh". (This means you would only index Library of Congress Subject Headings and not all data elements that are subjects.)
`<sort-type>`	(Optional, should be set for "item" indexes) This refers to the sort type / data type of the field: • `date` the index type will be treated as a date object and sorted as such • `text` the index type will be treated as plain text and sorted as such • (any other value refers to a custom <sort-type> which should be defined in a corresponding `webui.itemlist.sort-option.<n>` setting. See Defining Sort Options below for more information.)
`<sort-order>`	(Optional) The default sort order. Choose `asc` (ascending) or `desc` (descending). Ascending is the default value, but descending may be useful for date-based indexes (e.g. to display most recent submissions first)

Defining Sort Options

If you make changes in this section be sure to update your SOLR indexes running the Discovery Maintenance Script, see Discovery

Sort options/types will be available when browsing a list of items (either on "`item`" index type above or after selecting a specific value for "metadata" indexes). You can define an arbitrary number of fields to sort on. For example, the default entries that appear in the *dspace.cfg* as default installation:

```
webui.itemlist.sort-option.1 = title:dc.title:title
webui.itemlist.sort-option.2 = dateissued:dc.date.issued:date
webui.itemlist.sort-option.3 = dateaccessioned:dc.date.accessioned:date
```

The format of each entry is `web.browse.sort-option.<n> = <sort-type-name>:<schema-prefix>.<element>.<qualifier>:<datatype>`. Please notice the punctuation used between the different elements. The following table explains the each element:

Element	Definition and Options (if available)
`webui.itemlist.sort-option.<n>`	*n* is an arbitrary number you choose.
`<sort-type-name>`	The name by which the sort option will be identified. This is the name by which it is referred in the "webui.browse.index" settings (see Defining the Indexes).
`<schema-prefix>`	The schema used for the field to be sorted on in the index. The default is dc (for Dublin Core).
`<element>`	The schema element. In Dublin Core, for example, the author element is referred to as "Contributor". The user should consult the default Dublin Core Metadata Registry table in Appendix A.
`<qualifier>`	This is the qualifier to the <element> component. The user has two choices: an asterisk "*" or a proper qualifier of the element.
`<datatype>`	This refers to the datatype of the field: `date` the sort field will be treated as a date object `text` the sort field will be treated as plain text. `title` the sort field will be treated like a title, which will include a link to the item page

Hierarchical Browse Indexes

No configuration is necessary for hierarchical browse indexes (Browse by Subject Category). These are automatically generated based on the used controlled vocabularies in your submission forms. Default DSpace has one hierarchical browse index (Browse by Subject Category), since "srsc" is the only vocabulary used in the default submission-forms.xml.

Please note that when using another vocabulary, the UI's language packs (e.g. src/assets/i18n/en.json5) will need to be updated as well, e.g.:

- `"menu.section.browse_global_by_srsc": "By Subject Category"`
- `"browse.metadata.srsc.breadcrumbs": "Browse by Subject Category"`
- `"browse.comcol.by.srsc": "By Subject Category"`

Starting with DSpace 7.6.1, these Hierarchical "Browse By" options can be **disabled** via the below configuration:

Property:	`webui.browse.vocabularies.disabled`
Example Value:	`webui.browse.vocabularies.disabled = srsc`

Informational Note:	By default, all controlled vocabularies used within your submission forms (submission-forms.xml) will be enabled in the Browse By menu of the User Interface. If you wish to disable any from display in the UI, you can list them in this configuration. Multiple values can be comma separated (or this config can be repeated). Changes to this configuration will not take effect until your servlet engine (e.g. Tomcat) is restarted.

Other Browse Options

We set other browse values in the following section.

Property:	`webui.browse.metadata.show-freq.< n >`
Example Value:	`webui.browse.metadata.show-freq.1 = false`
Informational Note:	This enable/disable the show of frequencies (count) in metadata browse <n> refers to the browse configuration. As default frequencies are shown for all metadata browse
Property:	`plugin.named.org.dspace.sort.OrderFormatDelegate`
Example Value:	`plugin.named.org.dspace.sort.OrderFormatDelegate = \` `org.dspace.sort.OrderFormatTitleMarc21=title`

Informational Note:	This sets the option for how the indexes are sorted. All sort normalizations are carried out by the OrderFormatDelegate. The plugin manager can be used to specify your own delegates for each datatype. The default datatypes (and delegates) are:
	`author = org.dspace.sort.OrderFormatAuthor`
	`title = org.dspace.sort.OrderFormatTitle`
	`text = org.dspace.sort.OrderFormatText`
	If you redefine a default datatype here, the configuration will be used in preferences to the default. However, if you do not explicitly redefine a datatype, then the default will still be used in addition to the datatypes you do specify. As of DSpace release 1.5.2, the multi-lingual MARC21 title ordering is configured as default, as shown in the example above. To use the previous title ordering (before release 1.5.2), comment out the configuration in your *dspace.cfg* file.

Browse Index Authority Control Configuration

Property:	`webui.browse.index.<n>`
Example Value:	`webui.browse.index.5 = lcAuthor:metadataAuthority:dc.contributor.author:authority`
Informational Note:	

Tag cloud

Apart from the single (type=metadata) and full (type=item) browse pages, tag cloud is a new way to display the unique values of a metadata field.

To enable “tag cloud” browsing for a specific index you need to declare it in the dspace.cfg configuration file using the following option:

Property:	webui.browse.index.tagcloud.<*n*>
Example Value:	webui.browse.index.tagcloud.1 = true

Informational Note:	Enable/Disable tag cloud in browsing for a specific index. ‘n’ is the index number of the specific index which needs to be of type ‘metadata’. Possible values: true, false Default value is false. If no option exists for a specific index, it is assumed to be false. You do not have to re-index discovery when you change this configuration **UNSUPPORTED in DSpace 7.0**

Tag cloud configuration

The appearance configuration for the tag cloud is located in the Discovery xml configuration file (*dspace/config/spring/api/discovery.xml*). Without configuring the appearance, the default one will be applied to the tag cloud

In this file, there must be a bean named “*browseTagCloudConfiguration*” of class “*org.dspace.discovery.configuration.TagCloudConfiguration*”. This bean can have any of the following properties. If some is missing, the default value will be applied.

displayScore	Should display the score of each tag next to it? Default: false
shouldCenter	Should display the tag as center aligned in the page or left aligned? Possible values: true | false. Default: true
totalTags	How many tags will be shown. Value -1 means all of them. Default: -1
cloudCase	The letter case of the tags. Possible values: Case.LOWER | Case.UPPER | Case.CAPITALIZATION | Case.PRESERVE_CASE | Case.CASE_SENSITIVE Default: Case.PRESERVE_CASE
randomColors	If the 3 css classes of the tag cloud should be independent of score (random=yes) or based on the score. Possible values: true | false . Default: true
fontFrom	The font size (in em) for the tag with the lowest score. Possible values: any decimal. Default: 1.1
fontTo	The font size (in em) for the tag with the lowest score. Possible values: any decimal. Default: 3.2
cuttingLevel	The score that tags with lower than that will not appear in the rag cloud. Possible values: any integer from 1 to infinity. Default: 0
ordering	The ordering of the tags (based either on the name or the score of the tag) Possible values: Tag.NameComparatorAsc | Tag.NameComparatorDesc | Tag.ScoreComparatorAsc | Tag.ScoreComparatorDesc Default: Tag.GreekNameComparatorAsc

When tagCloud is rendered there are some CSS classes that you can change in order to change the tagcloud appearance.

Class	Note
tagcloud	General class for the whole tagcloud
tagcloud_1	Specific tag class for tag of type 1 (based on score)
tagcloud_2	Specific tag class for tag of type 2 (based on score)
tagcloud_3	Specific tag class for tag of type 3 (based on score)

Links to Other Browse Contexts

We can define which fields link to other browse listings. This is useful, for example, to link an author's name to a list of just that author's items. The effect this has is to create links to browse views for the item clicked on. If it is a "single" type, it will link to a view of all the items which share that metadata element in common (i.e. all the papers by a single author). If it is a "full" type, it will link to a view of the standard full browse page, starting with the value of the link clicked on.

Property:	`webui.browse.link.<n>`
Example Value:	`webui.browse.link.1 = author:dc.contributor.*`

Informational Note:	This is used to configure which fields should link to other browse listings. This should be associated with the name of one of the browse indexes (`webui.browse.index.n`) with a metadata field listed in `webui.itemlist.columns` above. If this condition is not fulfilled, cross-linking will not work. Note also that crosslinking only works for metadata fields not tagged as `title` in `webui.itemlist.columns`.

The format of the property key is *webui.browse.link.<n> = <index name>:<display column metadata>* Please notice the punctuation used between the elements.

Element	Definition and Options (if available)
`webui.browse.link.` *n*	{{*n* is an arbitrary number you choose
`<index name>`	This need to match your entry for the index name from *webui.browse.index* property key.
`<display column metadata>`	Use the DC element (and qualifier)

Examples of some browse links used in a real DSpace installation instance:

`webui.browse.link.1 = author:dc.contributor.*`
Creates a link for all types of contributors (authors, editors, illustrators, others, etc.)
`webui.browse.link.2 = subject:dc.subject.lcsh`
Creates a link to subjects that are Library of Congress only. In this case, you have a browse index that contains only LC Subject Headings
`webui.browse.link.3 = series:dc.relation.ispartofseries`
Creates a link for the browse index "Series". Please note this is again, a customized browse index and not part of the DSpace distributed release.

Submission License Substitution Variables

Property:	`plugin.named.org.dspace.content.license.`
	`LicenseArgumentFormatter`
	
	(property key broken up for display purposes only)

Example Value:	`plugin.named.org.dspace.content.license.LicenseArgumentFormatter = \`
	`org.dspace.content.license.SimpleDSpaceObjectLicenseFormatter = collection, \`
	`org.dspace.content.license.SimpleDSpaceObjectLicenseFormatter = item, \`
	`org.dspace.content.license.SimpleDSpaceObjectLicenseFormatter = eperson`

Informational Note:	It is possible include contextual information in the submission license using substitution variables. The text substitution is driven by a plugin implementation.

Syndication Feed (RSS) Settings

Supported as of 7.3 and above.

Please note that Syndication (RSS/Atom) feeds **require** that OpenSearch is enabled to function. When enabled, a syndication feed will be available on the DSpace homepage (for entire site), and on each community/collection homepage (specific to that community/collection). Because Syndication Feeds use OpenSearch, all OpenSearch settings also apply to Syndication Feeds.

Property:	`websvc.opensearch.enable`
Example Value:	`webui.opensearch.enable = true`
Informational Note:	By default, OpenSearch & Syndication feeds are set to true (on) . Change key to "false" to disable. NOTE this setting affects OpenSearch Support as well

Property:	`webui.feed.localresolve`
Example Value:	`webui.feed.localresolve = false`
Informational Note:	By default, (set to false), URLs returned by the feed will point at the global handle resolver (e.g. http://hdl.handle.net/123456789/1). If set to ***true*** the local server URLs are used (e.g. http://myserver.myorg/handle/123456789/1).
Property:	`webui.feed.item.title`
Example Value:	`webui.feed.item.title = dc.title`
Informational Note:	This property customizes each single-value field displayed in the feed information for each item. Each of the fields takes a **single** metadata field. The form of the key is <scheme prefix>.<element>.<qualifier> In place of the qualifier, one may leave it blank to exclude any qualifiers or use the wildcard "*" to include all qualifiers for a particular element.
Property:	`webui.feed.item.date`
Example Value:	`webui.feed.item.date = dc.date.issued`
Informational Note:	This property customizes each single-value field displayed in the feed information for each item. Each of the fields takes a **single** metadata field. The form of the key is <scheme prefix>.<element>.<qualifier> In place of the qualifier, one may leave it blank to exclude any qualifiers or use the wildcard "*" to include all qualifiers for a particular element.
Property:	`webui.feed.item.description`

Example Value:	`webui.feed.item.description = dc.title, dc.contributor.author, \`
	`dc.contributor.editor, dc.description.abstract, \`
	`dc.description`

Informational Note:	One can customize the metadata fields to show in the feed for each item's description. Elements are displayed in the order they are specified in *dspace.cfg*.Like other property keys, the format of this property key is: *webui.feed.item.description = <scheme prefix>.<element>.<qualifier>*. In place of the qualifier, one may leave it blank to exclude any qualifiers or use the wildcard "*" to include all qualifiers for a particular element.
Property:	`webui.feed.item.author`
Example Value:	`webui.feed.item.author = dc.contributor.author`

Informational Note:	The name of field to use for authors (Atom only); repeatable.
Property:	`webui.feed.logo.url`
Example Value:	`webui.feed.logo.url = ${dspace.url}/themes/mysite/images/mysite-logo.png`
Informational Note:	Customize the image icon included with the site-wide feeds. This must be an absolute URL.
Property:	`webui.feed.item.dc.creator`
Example Value:	`webui.feed.item.dc.creator = dc.contributor.author`
Informational Note:	This optional property adds *structured* DC elements as XML elements to the feed description. They are not the same thing as, for example, *webui.feed.item.description*. Useful when a program or stylesheet will be transforming a feed and wants separate author, description, date, etc.
Property:	`webui.feed.item.dc.date`
Example Value:	`webui.feed.item.dc.date = dc.date.issued`
Informational Note:	This optional property adds *structured* DC elements as XML elements to the feed description. They are not the same thing as, for example, *webui.feed.item.description*. Useful when a program or stylesheet will be transforming a feed and wants separate author, description, date, etc.
Property:	`webui.feed.item.dc.description`
Example Value:	`webui.feed.item.dc.description = dc.description.abstract`
Informational Note:	This optional property adds *structured* DC elements as XML elements to the feed description. They are not the same thing as, for example, *webui.feed.item.description*. Useful when a program or stylesheet will be transforming a feed and wants separate author, description, date, etc.
Property:	`webui.feed.podcast.collections`

Example Value:	`webui.feed.podcast.collections = 1811/45183,1811/47223`
Informational Note:	This optional property enables Podcast Support on the RSS feed for the specified collection handles. The podcast is iTunes compatible and will expose the bitstreams in the items for viewing and download by the podcast reader. Multiple values are separated by commas. For more on using /enabling Media RSS Feeds to share content via iTunesU, see: Enable Media RSS Feeds
Property:	`webui.feed.podcast.communities`
Example Value:	`webui.feed.podcast.communities = 1811/47223`
Informational Note:	This optional property enables Podcast Support on the RSS feed for the specified community handles. The podcast is iTunes compatible and will expose the bitstreams in the items for viewing and download by the podcast reader. Multiple values are separated by commas. For more on using /enabling Media RSS Feeds to share content via iTunesU, see: Enable Media RSS Feeds
Property:	`webui.feed.podcast.mimetypes`
Example Value:	`webui.feed.podcast.mimetypes = audio/x-mpeg,application/pdf`
Informational Note:	This optional property for Podcast Support, allows you to choose which MIME types of bitstreams are to be enclosed in the podcast feed. Multiple values are separated by commas. For more on using/enabling Media RSS Feeds to share content via iTunesU, see: Enable Media RSS Feeds
Property:	`webui.feed.podcast.sourceuri`
Example Value:	`webui.feed.podcast.sourceuri = dc.source.uri`
Informational Note:	This optional property for the Podcast Support will allow you to use a value for a metadata field as a replacement for actual bitstreams to be enclosed in the RSS feed. A use case for specifying the external sourceuri would be if you have a non-DSpace media streaming server that has a copy of your media file that you would prefer to have the media streamed from. For more on using/enabling Media RSS Feeds to share content via iTunesU, see: Enable Media RSS Feeds

OpenSearch Support

OpenSearch is a small set of conventions and documents for describing and using "search engines", meaning any service that returns a set of results for a query. See extensive description in the *Business Layer section* of the documentation.

Please note that RSS/Atom feeds **require** that OpenSearch is enabled to function.

OpenSearch uses all the configuration properties for DSpace RSS to determine the mapping of metadata fields to feed fields. Note that a new field for authors has been added (used in Atom format only).

Property:	`websvc.opensearch.enable`

Example Value:	`websvc.opensearch.enable = true/false`
Informational Note:	Whether or not OpenSearch is enabled. By default, the feature is enabled to support RSS/Atom feeds. Change to "false" to disable.
Property:	`websvc.opensearch.svccontext`
Example Value:	`websvc.opensearch.svccontext = search`
Informational Note:	The URL path where OpenSearch is made available on the backend. For example, "search" means it is available at `${dspace.server.url}/search`
Property:	`websvc.opensearch.uicontext`
Example Value:	`websvc.opensearch.uicontext = opensearch/search`
Informational Note:	Context for HTML request URLs. Change only for non-standard servlet mapping.
Property:	`websvc.opensearch.autolink`
Example Value:	`websvc.opensearch.autolink = true`
Informational Note:	Present autodiscovery link in every page head.
Property:	`websvc.opensearch.validity`
Example Value:	`websvc.opensearch.validity = 48`
Informational Note:	Number of hours to retain results before recalculating. This applies to the Manakin interface only.
Property:	`websvc.opensearch.shortname`
Example Value:	`websvc.opensearch.shortname = DSpace`
Informational Note:	A short name used in browsers for search service. It should be sixteen (16) or fewer characters.
Property:	`websvc.opensearch.longname`
Example Value:	`websvc.opensearch.longname = ${dspace.name}`
Informational Note:	A longer name up to 48 characters.
Property:	`websvc.opensearch.description`
Example Value:	`websvc.opensearch.description = ${dspace.name} DSpace repository`
Informational Note:	`Brief service description`
Property:	`websvc.opensearch.faviconurl`
Example Value:	`websvc.opensearch.faviconurl = http://www.dspace.org/images/favicon.ico`
Informational Note:	Location of favicon for service, if any. They must by 16 x 16 pixels. You can provide your own local favicon instead of the default.
Property:	`websvc.opensearch.samplequery`
Example Value:	`websvc.opensearch.samplequery = photosynthesis`
Informational Note:	Sample query. This should return results. You can replace the sample query with search terms that should actually yield results in your repository.
Property:	`websvc.opensearch.tags`

Example Value:	`websc.opensearch.tags = IR DSpace`
Informational Note:	Tags used to describe search service.
Property:	`websvc.opensearch.formats`
Example Value:	`websvc.opensearch.formats = html,atom,rss`
Informational Note:	Result formats offered. Use one or more comma-separated from the list: html, atom, rss. Please note that html is required for auto discovery in browsers to function, and must be the first in the list if present.

Content Inline Disposition Threshold / Format

The following configurations may be used to change the disposition behavior of the browser. This allows you to specify when a file (bitstream) in DSpace should be downloaded, or attempt to be opened in a user's browser.

Property:	`webui.content_disposition_threshold`
Example value:	`webui.content_disposition_threshold = 8388608`
Informational Note:	The default filesize is set to 8MB. When a file/bitstream being viewed is larger than 8MB, the user's browser will download the file to their local machine and the user will have to open it manually. All files smaller than this threshold will be sent "inline" to the user's browser, allowing the browser to decide whether to open it within the browser or download it. The value provided is always in bytes. For example: 4 MB = 4194304, 8 MB = 8388608, 16 MB = 16777216 NOTE: This threshold is only applied if the file/bitstream does NOT match the below "webui.content_disposition_format" list.
Property:	`webui.content_disposition_inline (Available in 7.6.7+)`
Example value:	`webui.content_disposition_inline = application/pdf, image/gif, image/jpeg, image/png, audio/mpeg, video/mpeg, video/mp4`
Informational Note:	Set which file mimetypes or file extensions are allowed to be opened inline in a user's browser. By default, ALL files will be downloaded, regardless of the threshold above, unless specified in this configuration. NOTE: For security reasons, some file formats (e.g. HTML, XML, RDF, JS) will always be downloaded regardless of the settings here. This blocks these formats from executing embedded JavaScript when opened inline, protecting the site from potential XSS attacks. For example, setting this to "application/pdf, image/gif, image/jpeg, image/png" will ensure that all files/bitstreams matching those MIME Types will be opened inline in the user's browser. But, all other files will be downloaded. File extensions may also be used to reference formats. For example, setting "pdf, jpg" will ensure all files ending in ".pdf" or ".jpg" will be opened in a user's browser. (NOTE: This setting replaces "webui.content_disposition_format", which is no longer supported. That prior setting listed formats **not allowed** to be opened inline, while the new setting lists formats that **are allowed** to be opended inline.)
Property:	`webui.content_disposition_format (Available only in 7.0-7.6.6)`
Example value:	`webui.content_disposition_format = text/html, text/richtext`
Informational Note:	Set which file mimetypes or file extensions will be forced to download, regardless of the "threshold" set above. Multiple values may be provided by setting this property several times, or by passing it a comma-separated list. For example, setting this to "text/html, text/richtext" will ensure that all files/bitstreams matching those MIME Types will always be downloaded (and never open inline in the user's browser). File extensions may also be used to reference formats. For example, setting "pdf, xls" will ensure all files ending in ".pdf" or ".xls" will always be downloaded.

Multi-file HTML Document/Site Settings

The setting is used to configure the "depth" of request for html documents bearing the same name.

Property:	`webui.html.max-depth-guess`
Example Value:	`webui.html.max-depth-guess = 3`
Informational Note:	When serving up composite HTML items in the UI, how deep can the request be for us to serve up a file with the same name? For example, if one receives a request for "*foo/bar/index.html*" and one has a bitstream called just "*index.html*", DSpace will serve up the former bitstream (*foo/bar /index.html*) for the request if *webui.html.max-depth-guess* is 2 or greater. If *webui.html.max-depth-guess* is 1 or less, then DSpace would not serve that bitstream, as the depth of the file is greater. If webui.html.max-depth-guess is zero, the request filename and path must always exactly match the bitstream name. The default is set to 3. **UNSUPPORTED IN DSpace 7.0**

Sitemap Settings

To aid web crawlers index the content within your repository, you can make use of sitemaps. For best SEO, Sitemaps are enabled by default and update automatically (see cron setting).

Property:	`sitemap.dir`
Example Value:	`sitemap.dir = ${dspace.dir}/sitemaps`
Informational Note:	The directory where the generate sitemaps are stored.
Property:	`sitemap.engineurls`
Example Value:	sitemap.engineurls = http://www.google.com/webmasters/sitemaps/ping?sitemap=
Informational Note:	Comma-separated list of search engine URLs to "ping" when a new Sitemap has been created. Include everything except the Sitemap UL itself (which will be URL-encoded and appended to form the actual URL "pinged").Add the following to the above parameter if you have an application ID with Yahoo: *http://search.yahooapis.com/SiteExplorerService/V1/updateNotification?appid=REPLACE_ME?url=_* . *(Replace the component _REPLACE_ME* with your application ID). There is no known "ping" URL for MSN/Live search.
Property:	`sitemap.cron`
Example Value:	`sitemap.cron = 0 15 1 * * ?`

Informational Note:	The DSpace sitemaps are regenerated on a regular basis based on the Cron syntax provided in this configuration. By default, sitemaps are updated daily at 1:15am local time. Cron syntax is defined at https://www.quartz-scheduler.org/api/2.3.0/org/quartz/CronTrigger.html Remove (comment out) this config to disable the sitemap scheduler. Sitemap scheduler can also be disabled by setting to "-" (single dash) in local.cfg.

Authority Control Settings

Two features fall under the header of Authority Control: Choice Management and Authority Control of Item ("DC") metadata values. Authority control is a fully optional feature in DSpace 1.6. Implemented out of the box are the Library of Congress Names service, and the Sherpa Romeo authority plugin.

For an in-depth description of this feature, please consult: Authority Control of Metadata Values

Property:	`plugin.named.org.dspace.content.authority.ChoiceAuthority`

Example Value:	`plugin.named.org.dspace.content.authority.ChoiceAuthority = \`
	`org.dspace.content.authority.SampleAuthority = Sample, \`
	`org.dspace.content.authority.SHERPARoMEOPublisher = SRPublisher, \`
	`org.dspace.content.authority.SHERPARoMEOJournalTitle = SRJournalTitle, \`
	`org.dspace.content.authority.SolrAuthority = SolrAuthorAuthority`

Informational Note:	List of all enabled authority control plugins
Property:	`plugin.selfnamed.org.dspace.content.authority.ChoiceAuthority`
Example Value:	`plugin.selfnamed.org.dspace.content.authority.ChoiceAuthority = \` `org.dspace.content.authority.DCInputAuthority`
Property:	`lcname.url`
Example Value:	`lcname.url = http://alcme.oclc.org/srw/search/lcnaf_`
Informational Note:	Location (URL) of the Library of Congress Name Service
Property:	`sherpa.romeo.url / sherpa.romeo.apikey`
Informational Note:	Please refers to the Sherpa/RoMEO Publishers Policy Database Integration section for details about such properties. See Configuring the Sherpa/RoMEO Publishers Policy Database Integration
Property:	`orcid.api.url`
Example Value:	`orcid.api.url = https://pub.orcid.org/v3.0`
Informational Note:	Location (URL) of the ORCID v3 Public API
Property:	`authority.minconfidence`

Example Value:	`authority.minconfidence = ambiguous`
Informational Note:	This sets the default lowest confidence level at which a metadata value is included in an authority-controlled browse (and search) index. It is a symbolic keyword, one of the following values (listed in descending order): accepted, uncertain, ambiguous, notfound, failed, rejected, novalue, unset. See `org.dspace.content.authority.Choices` source for descriptions.

Configuring Multilingual Support

See Multilingual Support for more details/examples.

Setting the Default Language for the Application

Property:	`default.locale`
Example Value:	`default.locale = en`
Informational Note:	The default language for the application is set with this property key. This is a locale according to i18n and might consist of country, country_language or country_language_variant. If no default locale is defined, then the server default locale will be used. The format of a local specifier is described here: http://java.sun.com/j2se/1.4.2/docs/api/java/util/Locale.html

Supporting More Than One Language

Changes in dspace.cfg

Property:	`webui.supported.locales`
Example Value:	`webui.supported.locales = en, de`
`or perhaps`	`webui.supported.locales = en, en_ca, de`
Informational Note:	All the locales that are supported by this instance of DSpace. Comma separated list. **UNSUPPORTED IN DSpace 7.0.** *However, the DSpace 7 UI has a similar "languages" setting in environment.*.ts*

The table above, if needed and is used will result in:

- a language switch in the default header
- the user will be enabled to choose his/her preferred language, this will be part of his/her profile
- wording of emails
  - mails to registered users, e.g. alerting service will use the preferred language of the user
  - mails to unregistered users, e.g. suggest an item will use the language of the session
- according to the language selected for the session, using dspace-admin Edit News will edit the news file of the language according to session

Related Files

If you set webui.supported.locales make sure that all the related additional files for each language are available. *LOCALE* should correspond to the locale set in *webui.supported.locales*, e. g.: for webui.supported.locales = en, de, fr, there should be:

- `[dspace-source]/dspace/modules/server/src/main/resources/Messages.properties`
- `[dspace-source]/dspace/modules/server/src/main/resources/Messages_en.properties`
- `[dspace-source]/dspace/modules/server/src/main/resources/Messages_de.properties`
- `[dspace-source]/dspace/modules/server/src/main/resources/Messages_fr.properties`

Files to be localized:

- `[dspace-source]/dspace/modules/server/src/main/resources/Messages_LOCALE.properties`
- `[dspace-source]/dspace/config/submission-forms_LOCALE.xml`
- `[dspace-source]/dspace/config/default_LOCALE.license - should be pure ASCII`
- `[dspace-source]/dspace/config/emails/change_password_LOCALE`
- `[dspace-source]/dspace/config/emails/feedback_LOCALE`
- `[dspace-source]/dspace/config/emails/internal_error_LOCALE`
- `[dspace-source]/dspace/config/emails/register_LOCALE`
- `[dspace-source]/dspace/config/emails/submit_archive_LOCALE`
- `[dspace-source]/dspace/config/emails/submit_reject_LOCALE`
- `[dspace-source]/dspace/config/emails/submit_task_LOCALE`
- `[dspace-source]/dspace/config/emails/subscription_LOCALE`
- `[dspace-source]/dspace/config/emails/suggest_LOCALE`

Upload File Settings

Property:	`upload.temp.dir`
Example Value:	`upload.temp.dir = ${dspace.dir}/upload`
Informational Note:	This property sets where DSpace temporarily stores uploaded files.

SFX Server (OpenURL)

SFX Server is an OpenURL Resolver.

Property:	`sfx.server.url`
Example Value:	`sfx.server.url` = `http://sfx.myu.edu:8888/sfx?`
	`sfx.server.url` = `http://worldcatlibraries.org/registry/gateway?`
Informational Note:	SFX query is appended to this URL. If this property is commented out or omitted, SFX support is switched off.

All the parameters mapping are defined in `[dspace]/config/sfx.xml` file. The program will check the parameters in `sfx.xml` and retrieve the correct metadata of the item. It will then parse the string to your resolver.

For the following example, the program will search the first query-pair which is DOI of the item. If there is a DOI for that item, your retrieval results will be, for example:
`http://researchspace.auckland.ac.nz/handle/2292/5763`

Example. For setting DOI in `sfx.xml`

```
<query-pairs>
    <field>
        <querystring>rft_id=info:doi/</querystring>
        <dc-schema>dc</dc-schema>
        <dc-element>identifier</dc-element>
        <dc-qualifier>doi</dc-qualifier>
    </field>
</query-pairs>
```

If there is no DOI for that item, it will search next query-pair based on the `[dspace]/config/sfx.xml` and then so on.

Example of using ISSN, volume, issue for item without DOI
`[http://researchspace.auckland.ac.nz/handle/2292/4947]`

For parameter passing to the `<querystring>`

```
<querystring>rft_id=info:doi/</querystring>
```

Please refer to these:
`[http://ocoins.info/cobgbook.html]`
`[http://ocoins.info/cobg.html]`

Program assume won't get empty string for the item, as there will at least author, title for the item to pass to the resolver.

For contributor author, program maintains original DSpace SFX function of extracting author's first and last name.

```
<field>
    <querystring>rft.aulast=</querystring>
    <dc-schema>dc</dc-schema>
    <dc-element>contributor</dc-element>
    <dc-qualifier>author</dc-qualifier>
</field>
<field>
    <querystring>rft.aufirst=</querystring>
    <dc-schema>dc</dc-schema>
    <dc-element>contributor</dc-element>
    <dc-qualifier>author</dc-qualifier>
</field>
```

Controlled Vocabulary Settings

DSpace now supports controlled vocabularies to confine the set of keywords that users can use while describing items.

The need for a limited set of keywords is important since it eliminates the ambiguity of a free description system, consequently simplifying the task of finding specific items of information.

The controlled vocabulary add-on allows the user to choose from a defined set of keywords organized in an tree (taxonomy) and then use these keywords to describe items while they are being submitted.

We have also developed a small search engine that displays the classification tree (or taxonomy) allowing the user to select the branches that best describe the information that he/she seeks.

The taxonomies are described in XML following this (very simple) structure:

```
<node id="acmccs98" label="ACMCCS98">
    <isComposedBy>
        <node id="A." label="General Literature">
            <isComposedBy>
                <node id="A.0" label="GENERAL"/>
                <node id="A.1" label="INTRODUCTORY AND SURVEY"/>
            </isComposedBy>
        </node>
    </isComposedBy>
</node>
```

You are free to use any application you want to create your controlled vocabularies. A simple text editor should be enough for small projects. Bigger projects will require more complex tools. You may use Protegé to create your taxonomies, save them as OWL and then use a XML Stylesheet (XSLT) to transform your documents to the appropriate format. Future enhancements to this add-on should make it compatible with standard schemas such as OWL or RDF.

New vocabularies should be placed in `[dspace]/config/controlled-vocabularies/` and must be according to the structure described. A validation XML Schema (named `controlledvocabulary.xsd`) is also available in that directory.

Vocabularies need to be associated with the correspondent DC metadata fields. Edit the file `[dspace]/config/input-forms.xml` and place a *"vocabulary"* tag under the *"field"* element that you want to control. Set value of the *"vocabulary"* element to the name of the file that contains the vocabulary, leaving out the extension (the add-on will only load files with extension "*.xml"). For example:

```
<field>
    <dc-schema>dc</dc-schema>
    <dc-element>subject</dc-element>
    <dc-qualifier></dc-qualifier>
    <!-- An input-type of twobox MUST be marked as repeatable -->
    <repeatable>true</repeatable>
    <label>Subject Keywords</label>
    <input-type>twobox</input-type>
    <hint> Enter appropriate subject keywords or phrases below. </hint>
    <required></required>
    <vocabulary [closed="false"]>nsi</vocabulary>
</field>
```

The vocabulary element has an optional boolean attribute **closed** that can be used to force input only with the JavaScript of controlled-vocabulary add-on. The default behavior (i.e. without this attribute) is as set **closed="false"**. This allow the user also to enter the value in free way.

The following vocabularies are currently available by default:

- **nsi** - *nsi.xml* - The Norwegian Science Index
- **srsc** - *srsc.xml* - Swedish Research Subject Categories

Optional or Advanced Configuration Settings

The following section explains how to configure either optional features or advanced features that are not necessary to make DSpace "out-of-the-box"

The Metadata Format and Bitstream Format Registries

The *[dspace]/config/registries* directory contains three XML files. These are used to load the *initial* contents of the Dublin Core Metadata registry and Bitstream Format registry and SWORD metadata registry. After the initial loading (performed by *ant fresh_install* above), the registries reside in the database; the XML files are not updated.

In order to change the registries, you may adjust the XML files before the first installation of DSpace. On an already running instance it is recommended to change bitstream registries via DSpace admin UI, but the metadata registries can be loaded again at any time from the XML files without difficulty. The changes made via admin UI are not reflected in the XML files.

Metadata Format Registries

The default metadata schema is Dublin Core, so DSpace is distributed with a default Dublin Core Metadata Registry. Currently, the system requires that every item have a Dublin Core record.

There is a set of Dublin Core Elements, which is used by the system and should not be removed or moved to another schema. See Appendix: Default Dublin Core Metadata registry.

Note: altering a Metadata Registry has no effect on corresponding parts, e.g. item submission interface, item display, item import and vice versa. Every metadata element used in submission interface or item import must be registered before using it.

Note also that deleting a metadata element will delete all its corresponding values.

If you wish to add more metadata elements, you can do this in one of two ways. Via the DSpace admin UI you may define new metadata elements in the different available schemas. But you may also modify the XML file (or provide an additional one), and re-import the data as follows:

```
[dspace]/bin/dspace registry-loader -metadata [xml file]
```

The XML file should be structured as follows:

```
<dspace-dc-types>
    <dc-type>
        <schema>dc</schema>
        <element>contributor</element>
        <qualifier>advisor</qualifier>
        <scope_note>Use primarily for thesis advisor.</scope_note>
    </dc-type>
</dspace-dc-types>
```

The set of metadata registry files which is read by the MetadataImporter tool is configured by the `metadata.registry.load` property in `dspace.cfg` or `local.cfg`. If you wish to use the importer to load a new metadata namespace from a new file, you will need to add the path to your new registry file as an additional value of this property before running the tool.

Bitstream Format Registry

The bitstream formats recognized by the system and levels of support are similarly stored in the bitstream format registry. This can also be edited at install-time via *[dspace]/config/registries/bitstream-formats.xml* or by the administration Web UI. The contents of the bitstream format registry are entirely up to you, though the system requires that the following two formats are present:

- *Unknown*
- *License*
  Deleting a format will cause any existing bitstreams of this format to be reverted to the unknown bitstream format.

Configuring Usage Instrumentation Plugins

A usage instrumentation plugin is configured as a Spring bean in the applicationContext.xml for each of the various user interface web applications. It will require the injection of an instance of EventService, which it will use to register itself on the UsageEvent bus. See the configuration file for examples.

More than one such plugin may be configured – each will receive all usage events.

If you wish to write your own, it must extend the abstract class org.dspace.usage.AbstractUsageEventListener.

The Passive Plugin

The Passive plugin is provided as the class org.dspace.usage.PassiveUsageEventListener. It absorbs events without effect, and serves as a simple example of how to write a UsageEvent listener.

The Tab File Logger Plugin

The Tab File Logger plugin is provided as the class org.dspace.usage.TabFileUsageEventListener. It writes event records to a file in tab-separated column format. If left unconfigured, it will write to `[DSpace]/log/usage-events.tsv`. To specify the file path, provide an absolute path, or a path relative to log.dir, as the value for usageEvent.tabFileLogger.file in dspace.cfg.

Behavior of the workflow system

DSpace contains a workflow system to review submissions as described in detail as part of the architecture of the business logic layer and in Configurable Workflow. The file **[dspace]/config/modules/workflow.cfg** contains additional properties to configure details of the workflow system.

The property `workflow.reviewer.file-edit` controls whether files may be added/edited/removed during review (set to true) or whether files can be downloaded during review only.

[dspace]/config/modules/workflow.cfg

```
#Allow the reviewers to add/edit/remove files from the submission
#When changing this property you might want to alert submitters in the license that reviewers can alter their
files
workflow.reviewer.file-edit=false
```

The workflow system will send notifications on new Items waiting to be reviewed to all EPersons that may resolve those. Tasks can be taken to avoid that two EPersons work on the same task at the same time without knowing from each other. When a EPerson returns a task to the pool without resolving it (by accepting or rejecting the submission), another E-Mail is sent. In case you only want to be notified of completely new tasks entering a step of the workflow system, you may switch off notifications on tasks returned to the pool by setting workflow.notify.returend.tasks to false in config/modules/workflow.cfg as shown below:

[dspace]/config/modules/workflow.cfg

```
# Notify reviewers about tasks returned to the pool
workflow.notify.returned.tasks = false
```

By default notifications are sent for tasks returned to the pool.

Recognizing Web Spiders (Bots, Crawlers, etc.)

DSpace can often recognize that a given access request comes from a web spider that is indexing your repository. These accesses can be flagged for separate treatment (perhaps exclusion) in usage statistics. This requires patterns to match against incoming requests. These patterns exist in files that you will find in `config/spiders`.

In the `spiders` directory itself, you will find a number of files provided by iplists.com. These files contain network address patterns which have been discovered to identify a number of known indexing services and other spiders. You can add your own files here if you wish to exclude more addresses that you know of. You will need to include your files' names in the list configured in `config/modules/solr-statistics.cfg`. The `iplists.com-*.txt` files can be updated using a tool provided by DSpace. See SOLR Statistics for details.

In the `spiders` directory you will also find two subdirectories. `agents` contains files filled with regular expressions, one per line. An incoming request's `User-Agent` header is tested with each expression found in any of these files until an expression matches. If there is a match, the request is marked as being from a spider, otherwise not. `domains` similarly contains files filled with regular expressions which are used to test the domain name from which the request comes. You may add your own files of regular expressions to either directory if you wish to test requests with patterns of your own devising.

Command-line Access to Configuration Properties

You can resolve a configuration property name to its value using the command `dspace dsprop -p some.property.name`. The output is undecorated and may be suitable for use in scripts.

The `dsprop command has these options:`

name	argument	meaning
--property -p	name	the name of the desired configuration property. This option is required.
--module -m	name	the name of the module in which the property is found. If omitted, the value of --property is the entire name. If used, the name will be composed as module.property. For example, "`-m dspace -p url`" will look up the value of `dspace.url`.
--raw -r		if used, this prevents the substitution of other property values into the value of the requested property. It is also useful to see all of the propery values when a specific property has an **array of values** (i.e. the configuration supports specifying multiple values). Otherwise, by default , dsprop may only return the first value in the array.

--help -h -?		Display help similar to this table.

DSpace Item State Definitions

Workspace item

An item that is under submission and active edit by an authorized user. The workspace item is visible only to the submitter and the system administrators. (Currently there is no simple way to find/browse such items other than with the direct item ID or to use the supervisor functionality). Using the supervisor functionality, a system admin can allow other authorized user to see/edit the item in the workspace state.

Expected use cases:

- Self deposit
- Collaboration over an in-progress submission for a small group of researchers. (This use case is implemented only with major limitations, using the supervision feature – concurrency, lack of delegation: supervision must be defined by the system administrators, etc.)

Workflow Item

An item that is under review for quality control and policy compliance. The workflow item is visible to the original submitter (currently only basic metadata are visible out-of-box in the mydspace summary list), users assigned to the specific workflow step where the item resides, and system administrators. (Currently there is no simple way to find/browse such items other than with the direct item ID or to use the abort workflow functionality).

Expected use cases:

- Quality control
- Improvements to the bibliographic record (metadata available in workflow can be different than those asked of the submitter)
- Check of policy / copyright

Withdrawn item

It is the removal of an Item from the archive. However, a withdrawn item is still available to Administrative users (and may optionally be restored to the archive at a later date). A withdrawn item disappears from DSpace (except from Administrative screens) and the item appears to be deleted.

Expected use cases:

- Staging area for item to be removed when copyright issues arise with publisher. If the copyright issue is confirmed, the item will be permanently deleted or kept in the withdrawn state for future reference.
- Logical deletion delegated to community/collection admin, where permanent deletion is reserved to system administrators
- Logical deletion, where permanent deletion is not an option for an organization
- Removal of an old version of an item, forcing redirect to a new up-to-date version of the item (this use case is not currently implemented out-of-box in DSpace)

By design, withdrawing an item is reversible. As an administrator, you can reverse the withdrawing of an item, through the action "reinstate". As a mechanism to support this, a resource policy state "WITHDRAWN_READ" was introduced.

When as item is withdrawn, all READ policies associated with the item and its underlying bundles and bitstreams, are changed into WITHDRAWN_READ policies. This achieves 2 things:

1. The READ policy information in itself is still preserved, and can get switched back to normal READ policies if the item gets reinstated.
2. As long as the item is withdrawn, those WITHDRAWN_READ policies should not give any users or groups read rights.

WITHDRAWN_READ was introduced in DS-3097 after it was observed that even though an item was withdrawn, the related bitstreams were still accessible.

Non-Discoverable item (also known as Private item)

A non-discoverable (or "private") item is one that is simply **hidden** from all search/browse/OAI results, and is therefore only accessible via direct link (or bookmark). By default, all Items are *discoverable*, meaning they will appear in search/browse/OAI results.

It's important to clarify that non-discoverable items *may or may not be access restricted.* It is possible for an Item to be anonymously visible, but non-discoverable, so that you can only access the item if you are given a link to it.

This state should only refer to the discoverable nature of the item. A non-discoverable (or "private") item will not be included in any system that aims to help users to find items. So it will not appear in:

- Browse
- Recent submission
- Search result
- OAI-PMH (at least for the ListRecords and ListIdentifiers verb; though the OAI-PMH specification is not clear about inconsistent implementation of the ListRecords and GetRecord verb)
- REST list and search methods

It should be accessible under the actual Authorization Policies of DSpace using direct URL or query method such as:

- Splash page access (i.e. /handle/<xxxxx>/<yyyyy>)

- OAI-PMH GetRecord verb
- REST direct access /rest/item/<item-id> or equivalent

Expected use cases:

- Provide a light rights awareness feature where discovery is not enabled for search and/or browse
- Hide “special items” such as repository presentations, guides or support materials
- Hide an old version of an Item in cases where real versioning is not appropriate or liked
- Hide specific types of item such as “Item used to record Journal record: Journal Title, ISSN, Publisher etc.” used as authority file for metadata (dc.relation.ispartof) of “normal item”

Archived/Published item

An item that is in a stable state, available in the repository under the defined Authorization Policies. Changes to these items are possible only for a restricted group of users (administrators) and should produce versioning according to the Institution's policy.

Embargoed Item

Are a special case of Archived/Published Item. The item has some time based access policy attached to it and/or the underlying bitstreams. Specifically, read permission for someone (EPerson Group) starting from a defined date. Typically embargo is applied to the bitstreams so that "fulltext" has initially very limited access (normally administrators or other "repository staff" groups) and only after a defined date will the fulltext become visible to all users (Anonymous group). This scenario is used to implement typical "embargo requirements" from publishers -- see Delayed Open Access.

If the metadata of the item should be visible only to a specific group of users, it is possible to define an embargo policy also for the ITEM itself. A READ policy for a specific group will mean that only the users in that group will be able to access the item splash page. Note that the DSpace REST API & UI is fully rights aware (see Discovery documentation for more information, especially the section on "Access Rights Awareness"), meaning that an embargoed item is hidden automatically until the embargo expires.

Directories and Files

1 Overview
2 Source Directory Layout
3 Installed Directory Layout
4 Contents of Server Web Application
5 Log Files
    5.1 log4j2.xml File.

Overview

A complete DSpace installation consists of three separate directory trees:

- **The source directory:**: This is where (surprise!) the source code lives. Note that the config files here are used only during the initial install process. After the install, config files should be changed in the install directory. It is referred to in this document as *[dspace-source]*.
- **The install directory:**: This directory is populated during the install process and also by DSpace as it runs. It contains config files, command-line tools (and the libraries necessary to run them), and usually -- although not necessarily -- the contents of the DSpace archive (depending on how DSpace is configured). After the initial build and install, changes to config files should be made in this directory. It is referred to in this document as *[dspace]*.
- **The web deployment directory:**: This directory is generated by the web server the first time it finds a dspace.war file in its webapps directory. It contains the unpacked contents of dspace.war, i.e. the JSPs and java classes and libraries necessary to run DSpace. Files in this directory should never be edited directly; if you wish to modify your DSpace installation, you should edit files in the source directory and then rebuild. The contents of this directory aren't listed here since its creation is completely automatic. It is usually referred to in this document as *[tomcat]/webapps /dspace*.

Source Directory Layout

- *[dspace-source]*
    - *LICENSE* - DSpace source code license.
    - *README* - Obligatory basic information file.
    - *dspace/* - Directory which contains all build and configuration information for DSpace
        - *bin/* - Some shell and Perl scripts for running DSpace command-line tasks. Primary among them is the 'dspace' commandline utility
        - *config/* - Configuration files:
            - local.cfg.EXAMPLE - an example "local.cfg" file, which can be used to store all your local configuration overrides. See Configuration Reference.
            - *controlled-vocabularies/* - Fixed, limited vocabularies used in metadata entry
            - *crosswalks/* - Metadata crosswalks - property files or XSL stylesheets
            - *emails/* - Text and layout templates for emails sent out by the system.
            - entities/ - Configuration files for Configurable Entities
            - modules/ - Configurations for modules / individual features within DSpace
            - registries/ - **Initial** contents of the bitstream format registry and Dublin Core element/qualifier registry. These are only used on initial system setup, after which they are maintained in the database.
            - spring/ - Spring XML configurations used by DSpace for various features.
            - *dspace.cfg* - The Main DSpace configuration file
            - *dc2mods.cfg* - Mappings from Dublin Core metadata to MODS for the METS export.
            - *default.license* - The default license that users must grant when submitting items.
            - *dstat.cfg* , *dstat.map* - Configuration for statistical reports.
            - *submission-forms.xml* , *item-submission.xml* - Submission UI configuration files
        - *modules/* - The Web UI modules "overlay" directory. DSpace uses Maven to automatically look here for any customizations you wish to make to DSpace Web interfaces. See also Advanced Customisation
        - *solr/* - Solr configuration files for all Solr indexes used by DSpace.
        - *src/* - Maven configurations for DSpace System. This directory contains the Maven and Ant build files for DSpace.
        - *target/* - (Only exists after building DSpace) This is the location Maven uses to build your DSpace installation package.
            - *dspace-installer*- The location of the DSpace Installation Package (which can then be installed by running *ant update*)

- The Source Release contains the following additional directories :-
    - *dspace-api* - Java API source module (to build the dspace-api.jar)
    - *dspace-oai* - OAI-PMH source module (to build to dspace-oai.jar)
    - *dspace-rdf* - RDF source module (to build to dspace-rdf.jar)
    - *dspace-rest* - REST API v6 (deprecated) source module
    - dspace-server-webapp - Primary backend webapp which hosts the REST API, along with any other enabled modules (OAI, RDF, SWORD, etc).
    - *dspace-services* - Common Services module
    - *dspace-sword* - SWORD (Simple Web-service Offering Repository Deposit) deposit service source module
    - *dspace-swordv2* - SWORDv2 source module
    - *pom.xml* - DSpace Parent Project definition

Installed Directory Layout

Below is the basic layout of a DSpace installation using the default configuration. These paths can be configured if necessary.

- *[dspace]*

- *assetstore/* - assetstore files. This is where all the files uploaded into DSpace are stored by default. See Storage Layer.
- *bin/* - shell scripts for DSpace command-line tasks. Primary among them is the 'dspace' commandline utility
- *config/* - configuration, with sub-directories as above
- *etc/* - Administrative and database management files
- *exports/* - temporary storage for any export packages
- *handle-server/* - Handles server files and configuration
- *imports/* - temporary storage for any import packages
- *lib/* - JARs, including dspace-api.jar, containing the DSpace classes
- *log/* - Log files
- *reports/* - Reports generated by statistical report generator
- *solr/* - Solr search/browse indexes
- *triplestore/* - RDF triple store index files (when enabled)
- *upload/* - temporary directory used during file uploads etc.
- *webapps/* - location where DSpace installs all Web Applications

Contents of Server Web Application

DSpace's Ant build file creates a webapps/*server/* directory with the following structure:

- server/
  - index.html - Root page of the third party HAL Browser (used to browse/search REST API)
  - login.html - (Custom) Login page for HAL Browser (supporting DSpace authentication plugins)
  - js/ - Javascript overrides for HAL Browser (main HAL Browser code is brought in via Spring REST dependencies)

Log Files

The first source of potential confusion is the log files. Since DSpace uses a number of third-party tools, problems can occur in a variety of places. Below is a table listing the main log files used in a typical DSpace setup. The locations given are defaults, and might be different for your system depending on where you installed DSpace and the third-party tools. The ordering of the list is roughly the recommended order for searching them for the details about a particular problem or error.

Log File	What's In It
[dspace]/log/dspace.log.yyyy-mm-dd	Main DSpace log file. This is where the DSpace code writes a simple log of events and errors that occur within the DSpace code. You can control the verbosity of this by editing the *[dspace]/config/log4j2.xml* file.
[dspace]/log/dspace-cli.log.yyyy-mm-dd	DSpace log file for "command-line" tasks only. Any task that is run from the command-line (e.g. [dspace]/bin/dspace script) will log to this file **instead** of the main log file. You can control the verbosity of this by editing the *[dspace]/config/log4j2-cli.xml* file.
[dspace]/log/handle-plugin.log	The Handle server runs as a separate process from the DSpace Web UI (which runs under Tomcat's JVM). Due to a limitation of log4j's 'rolling file appenders', the DSpace code running in the Handle server's JVM must use a separate log file. The DSpace code that is run as part of a Handle resolution request writes log information to this file. You can control the verbosity of this by editing *[dspace]/config/log4j2-handle-plugin.xml*
[dspace]/log/handle-server.log	This is the log file for CNRI's Handle server code. If a problem occurs within the Handle server code, before DSpace's plug-in is invoked, this is where it may be logged.
[tomcat]/logs/catalina.out	This is where Tomcat's standard output is written. Many errors that occur within the Tomcat code are logged here. For example, if Tomcat can't find the DSpace code (*dspace.jar*), it would be logged in *catalina.out*.
[tomcat]/logs/hostname_log.yyyy-mm-dd.txt	If you're running Tomcat stand-alone (without Apache), it logs some information and errors for specific Web applications to this log file. *hostname* will be your host name (e.g. *dspace.myu.edu*) and *yyyy-mm-dd* will be the date.
[tomcat]/logs/apache_log.yyyy-mm-dd.txt	If you're using Apache, Tomcat logs information about Web applications running through Apache (*mod_webapp*) in this log file (*yyyy-mm-dd* being the date.)
[apache]/error_log	Apache logs to this file. If there is a problem with getting *mod_webapp* working, this is a good place to look for clues. Apache also writes to several other log files, though *error_log* tends to contain the most useful information for tracking down problems.
PostgreSQL log	PostgreSQL also writes a log file. This one doesn't seem to have a default location, you probably had to specify it yourself at some point during installation. In general, this log file rarely contains pertinent information--PostgreSQL is pretty stable, you're more likely to encounter problems with connecting via JDBC, and these problems will be logged in *dspace.log*.

log4j2.xml File.

the file *[dspace]/config/log4j2.xml* controls how and where log files are created. There are three sets of configurations in that file, called A1, and A2. These are used to control the logs for DSpace (as a whole), and the checksum checker respectively. As implied by the name, this configuration use Log4j v2. For more information on syntax, see https://logging.apache.org/log4j/2.x/manual/configuration.html

Metadata and Bitstream Format Registries

1 Default Dublin Core Metadata Registry (DC)
2 Dublin Core Terms Registry (DCTERMS)
3 Local Metadata Registry (local)
4 Default Bitstream Format Registry

Default Dublin Core Metadata Registry (DC)

The default DSpace Dublin Core Metadata Registry was originally derived from the 15 Dublin Core elements. This registry initializes the default schema, where **dc** is used to identify the namespace. As this registry is meant to track the Dublin Core standard, it's recommended that the local DSpace administrator not add/remove metadata fields from this namespace; the "local" namespace should be used instead (see below).

element	qualifier	scope note
contributor		A person, organization, or service responsible for the content of the resource. Catch-all for unspecified contributors.
contributor	advisor	Use primarily for thesis advisor.
contributor	author[1]	Author(s) of the work (used by default)
contributor	editor	
contributor	illustrator	
contributor	other	
coverage	spatial	Spatial characteristics of content.
coverage	temporal	Temporal characteristics of content.
creator		May be used as an alternative to "contributor.author"
date		Use qualified form if possible.
date	accessioned[1]	Date DSpace takes possession of item.
date	available[1]	Date or date range item became available to the public.
date	copyright	Date of copyright.
date	created	Date of creation or manufacture of intellectual content if different from date.issued.
date	issued[1]	Date of publication or distribution.
date	submitted	Recommend for theses/dissertations.
identifier		Catch-all for unambiguous identifiers not defined by qualified form; use identifier.other for a known identifier common to a local collection instead of unqualified form.
identifier	citation[2]	Human-readable, standard bibliographic citation of non-DSpace format of this item
identifier	govdoc[2]	A government document number
identifier	isbn[2]	International Standard Book Number
identifier	issn[2]	International Standard Serial Number
identifier	sici	Serial Item and Contribution Identifier
identifier	ismn[2]	International Standard Music Number
identifier	other[2]	A known identifier type common to a local collection.
identifier	uri[1]	Uniform Resource Identifier
description[1]		Catch-all for any description not defined by qualifiers.
description	abstract[1]	Abstract or summary.
description	provenance[1]	The history of custody of the item since its creation, including any changes successive custodians made to it.
description	sponsorship[2]	Information about sponsoring agencies, individuals, or contractual arrangements for the item.

description	statementofresponsibility	To preserve statement of responsibility from MARC records.
description	tableofcontents	A table of contents for a given item.
description	uri	Uniform Resource Identifier pointing to description of this item.
format[2]		Catch-all for any format information not defined by qualifiers.
format	extent[2]	Size or duration.
format	medium[2]	Physical medium.
format	mimetype[2]	Registered MIME type identifiers.
language		Catch-all for non-ISO forms of the language of the item, accommodating harvested values.
language	iso[2]	Current ISO standard for language of intellectual content, including country codes (e.g. "en_US").
publisher[2]		Entity responsible for publication, distribution, or imprint.
relation		Catch-all for references to other related items.
relation	isformatof	References additional physical form.
relation	ispartof	References physically or logically containing item.
relation[1]	ispartofseries	Series name and number within that series, if available.
relation	haspart	References physically or logically contained item.
relation	isversionof	References earlier version.
relation	hasversion	References later version.
relation	isbasedon	References source.
relation	isreferencedby	Pointed to by referenced resource.
relation	requires	Referenced resource is required to support function, delivery, or coherence of item.
relation	replaces	References preceeding item.
relation	isreplacedby	References succeeding item.
relation	uri	References Uniform Resource Identifier for related item
rights		Terms governing use and reproduction.
rights	uri	References terms governing use and reproduction.
source		Do not use; only for harvested metadata.
source	uri	Do not use; only for harvested metadata.
subject[2]		Uncontrolled index term.
subject	classification	Catch-all for value from local classification system. Global classification systems will receive specific qualifier
subject	ddc	Dewey Decimal Classification Number
subject	lcc	Library of Congress Classification Number
subject	lcsh	Library of Congress Subject Headings
subject	mesh	MEdical Subject Headings
subject	other	Local controlled vocabulary; global vocabularies will receive specific qualifier.
title[1]		Title statement/title proper.
title	alternative[2]	Varying (or substitute) form of title proper appearing in item, e.g. abbreviation or translation
type[1]		Nature or genre of content.

[1] Used by several functional areas of DSpace. **DO NOT REMOVE WITHOUT INVESTIGATING THE CONSEQUENCES**

[2] This field is included in the default DSpace Submission User Interface. Removing this field from your registry will break the default DSpace submission form.

Dublin Core Terms Registry (DCTERMS)

The Dublin Core Terms (DCTERMS) registry was introduced in DSpace 4. This registry initializes an optional metadata schema, where **dcterms** is used to identify the namespace. In DSpace 4, none of these fields are used by any of the system functionality out of the box. The registry and schema were added as a first step to facilitate the future migration of the DSpace specific DC schema, to this schema that complies to current Dublin Core standards.

The main advantage of the DCTERMS schema is that no field name details gets lost during harvesting, as opposed to harvesting of so called "simple" dublin core, where the qualifiers from the above schema are omitted during harvesting.

As this registry is meant to track the Dublin Core Terms standard, it's recommended that the local DSpace administrator not add/remove metadata fields from this namespace; the "local" namespace should be used instead (see below).

term	scope note
abstract	A summary of the resource.
accessRights	Information about who can access the resource or an indication of its security status. May include information regarding access or restrictions based on privacy, security, or other policies.
accrualMethod	The method by which items are added to a collection.
accrualPeriodicity	The frequency with which items are added to a collection.
accrualPolicy	The policy governing the addition of items to a collection.
alternative	An alternative name for the resource.
audience	A class of entity for whom the resource is intended or useful.
available	Date (often a range) that the resource became or will become available.
bibliographicCitation	Recommended practice is to include sufficient bibliographic detail to identify the resource as unambiguously as possible.
conformsTo	An established standard to which the described resource conforms.
contributor	An entity responsible for making contributions to the resource. Examples of a Contributor include a person, an organization, or a service.
coverage	The spatial or temporal topic of the resource, the spatial applicability of the resource, or the jurisdiction under which the resource is relevant.
created	Date of creation of the resource.
creator	An entity primarily responsible for making the resource.
date	A point or period of time associated with an event in the lifecycle of the resource.
dateAccepted	Date of acceptance of the resource.
dateCopyrighted	Date of copyright.
dateSubmitted	Date of submission of the resource.
description	An account of the resource.
educationLevel	A class of entity, defined in terms of progression through an educational or training context, for which the described resource is intended.
extent	The size or duration of the resource.
format	The file format, physical medium, or dimensions of the resource.
hasFormat	A related resource that is substantially the same as the pre-existing described resource, but in another format.
hasPart	A related resource that is included either physically or logically in the described resource.
hasVersion	A related resource that is a version, edition, or adaptation of the described resource.
identifier	An unambiguous reference to the resource within a given context.
instructionalMethod	A process, used to engender knowledge, attitudes and skills, that the described resource is designed to support.
isFormatOf	A related resource that is substantially the same as the described resource, but in another format.
isPartOf	A related resource in which the described resource is physically or logically included.

isReferencedBy	A related resource that references, cites, or otherwise points to the described resource.
isReplacedBy	A related resource that supplants, displaces, or supersedes the described resource.
isRequiredBy	A related resource that requires the described resource to support its function, delivery, or coherence.
issued	Date of formal issuance (e.g., publication) of the resource.
isVersionOf	A related resource of which the described resource is a version, edition, or adaptation.
language	A language of the resource.
license	A legal document giving official permission to do something with the resource.
mediator	An entity that mediates access to the resource and for whom the resource is intended or useful.
medium	The material or physical carrier of the resource.
modified	Date on which the resource was changed.
provenance	A statement of any changes in ownership and custody of the resource since its creation that are significant for its authenticity, integrity, and interpretation.
publisher	An entity responsible for making the resource available.
references	A related resource that is referenced, cited, or otherwise pointed to by the described resource.
relation	A related resource.
replaces	A related resource that is supplanted, displaced, or superseded by the described resource.
requires	A related resource that is required by the described resource to support its function, delivery, or coherence.
rights	Information about rights held in and over the resource.
rightsHolder	A person or organization owning or managing rights over the resource.
source	A related resource from which the described resource is derived.
spatial	Spatial characteristics of the resource.
subject	The topic of the resource.
tableOfContents	A list of subunits of the resource.
temporal	Temporal characteristics of the resource.
title	A name given to the resource.
type	The nature or genre of the resource.
valid	Date (often a range) of validity of a resource.

Local Metadata Registry (local)

Editing the DC and DCTERMS schemas is recommended against because it may complicate the upgrade path in case a newer version of DSpace needs to make changes or migrations in these standard metadata fields. Therefore, an empty metadata schema called "local" is provided (since DSpace 6), which can be used by the DSpace administrator as a namespace for custom local metadata fields. Such custom fields would be anything that does not fit into DC or DCTERMS. Future DSpace migrations will not touch fields in the "local" namespace.

element	qualifier	scope note
<empty by default>		<fields to be populated by DSpace administrator if needed>

Default Bitstream Format Registry

Mimetype	Short Description	Description	Support Level	Internal	Extensions
application/octet-stream[1]	Unknown	Unknown data format	Unknown	false	
text/plain[1]	License	Item-specific license agreed upon to submission	Known	true	
application/marc	MARC	Machine-Readable Cataloging records	Known	false	

application/mathematica	Mathematica	Mathematica Notebook	Known	false	ma
application/msword	Microsoft Word	Microsoft Word	Known	false	doc
application/pdf	Adobe PDF	Adobe Portable Document Format	Known	false	pdf
application/postscript	Postscript	Postscript Files	Known	false	ai, eps, ps
application/sgml	SGML	SGML application (RFC 1874)	Known	false	sgm, sgml
application/vnd.ms-excel	Microsoft Excel	Microsoft Excel	Known	false	xls
application/vnd.ms-powerpoint	Microsoft Powerpoint	Microsoft Powerpoint	Known	false	ppt
application/vnd.ms-project	Microsoft Project	Microsoft Project	Known	false	mpd, mpp, mpx
application/vnd.visio	Microsoft Visio	Microsoft Visio	Known	false	vsd
application/wordperfect5.1	WordPerfect	WordPerfect 5.1 document	Known	false	wpd
application/x-dvi	TeX dvi	TeX dvi format	Known	false	dvi
application/x-filemaker	FMP3	Filemaker Pro	Known	false	fm
application/x-latex	LateX	LaTeX document	Known	false	latex
application/x-photoshop	Photoshop	Photoshop	Known	false	pdd, psd
application/x-tex	TeX	Tex/LateX document	Known	false	tex
audio/basic	audio/basic	Basic Audio	Known	false	au, snd
audio/x-aiff	AIFF	Audio Interchange File Format	Known	false	aif, aifc, aiff
audio/x-mpeg	MPEG Audio	MPEG Audio	Known	false	abs, mpa, mpega
audio/x-pn-realaudio	RealAudio	RealAudio file	Known	false	ra, ram
audio/x-wav	WAV	Broadcase Wave Format	Known	false	wav
image/gif	GIF	Graphics Interchange Format	Known	false	gif
image/jpeg	JPEG	Joint Photographic Experts Group/JPEG File Interchange Format (JFIF)	Known	false	jpeg, jpg
image/png	image/png	Portable Network Graphics	Known	false	png
image/tiff	TIFF	Tag Image File Format	Known	false	tif, tiff
image/x-ms-bmp	BMP	Microsoft Windows bitmap	Known	false	bmp
image/x-photo-cd	Photo CD	Kodak Photo CD image	Known	false	pcd
text/css	CSS	Cascading Style Sheets	Known	false	css
text/html	HTML	Hypertext Markup Language	Known	false	htm, html
text/plain	Text	Plain Text	Known	false	asc, txt
text/richtext	RTF	Rich Text Format	Known	false	rtf
text/xml	XML	Extensible Markup Language	Known	false	xml
video/mpeg	MPEG	Moving Picture Experts Group	Known	false	mpe, mpeg, mpg
video/quicktime	Video Quicktime	Video Quicktime	Known	false	mov, qt

[1] Used by several functional areas of DSpace. **DO NOT REMOVE WITHOUT INVESTIGATING THE CONSEQUENCES**

Architecture

1 Overview

Overview

The DSpace system is organized into three layers, each of which consists of a number of components.

- **Application Layer** - All external/public facing interfaces/tools. These include the Web User Interface, REST API, OAI-PMH, RDF, and SWORD (v and v2) interfaces. Also includes the Command Line interface, and various tools that can be used to import/export data to/from DSpace.
- **Business Logic Layer** - Primarily the Java API layer ([dspace-source]/dspace-api and dspace-services), which provides the core business logic for all the various application interfaces.
- **Storage Layer** - A subset of the dspace-api (*org.dspace.storage.* classes)* whose role is to manage all content storage (metadata, relationships, bitstreams) for all business layer objects. This layer is provides access to a relational database (Postgres or Oracle usually) via Hibernate ORM & using FlywayDB for migrations/updates. It also defines a custom BitStoreService for storing files (bitstreams) via storage plugins (currently supporting filesystem storage or Amazon S3 storage).

DSpace System Architecture

The storage layer is responsible for physical storage of metadata and content. The business logic layer deals with managing the content of the archive, users of the archive (e-people), authorization, and workflow. The application layer contains components that communicate with the world outside of the individual DSpace installation, for example the Web user interface and the Open Archives Initiative protocol for metadata harvesting service.

Each layer only invokes the layer below it; the application layer may not use the storage layer directly, for example. Each component in the storage and business logic layers has a defined public API. The union of the APIs of those components are referred to as the Storage API (in the case of the storage layer) and the DSpace Java API (in the case of the business logic layer), and the DSpace REST API (in the case of the application layer). In the Application Layer, it's worth noting that the Web User Interface only accesses DSpace via the REST API.

It is important to note that each layer is *trusted*. Although the logic for *authorising actions* is in the business logic layer, the system relies on individual applications in the application layer to correctly and securely *authenticate* e-people. If a 'hostile' or insecure application were allowed to invoke the Java API directly, it could very easily perform actions as any e-person in the system.

The reason for this design choice is that authentication methods will vary widely between different applications, so it makes sense to leave the logic and responsibility for that in these applications.

The source code is organized to cohere very strictly to this three-layer architecture.

The storage and business logic layer APIs are extensively documented with Javadoc-style comments. Generate the HTML version of these by entering the [dspace-source]/dspace directory and running:

```
mvn javadoc:javadoc
```

The resulting documentation will be at *[dspace-source]dspace-api/target/site/apidocs/index.html*. The package-level documentation of each package usually contains an overview of the package and some example usage. This information is not repeated in this architecture document; this and the Javadoc APIs are intended to be used in parallel.

The REST API provides not only JavaDocs, but also a public contract. See REST API.

Each layer is described in a separate section:

- Storage Layer
  - RDBMS
  - Bitstream Store
- Business Logic Layer
  - Core Classes
  - Content Management API
  - Workflow System
  - Administration Toolkit
  - E-person/Group Manager
  - Authorisation
  - Handle Manager/Handle Plugin
  - Search
  - Browse API
  - History Recorder
  - Checksum Checker
- Application Layer
  - Web User Interface
  - OAI-PMH Data Provider
  - Item Importer and Exporter
  - Transferring Items Between DSpace Instances
  - Registration
  - METS Tools
  - Media Filters

- Sub-Community Management

Application Layer

The following explains the components of the Application Layer.

1 Web User Interface
    1.1 Web UI Files
2 REST API
3 OAI-PMH Data Provider
4 RDF / Linked Data Provider
5 SWORD v1 Service / Server
6 SWORD v2 Service / Server
7 DSpace Command Line Launcher
    7.1 Command Launcher Structure

Web User Interface

The DSpace Web UI is the largest and most-used component in the application layer. As of DSpace 7, it has been rebuilt on Angular.io communicating via the REST API to the rest of DSpace.

Web UI Files

The web User Interface code is managed in a separate GitHub Project:

https://github.com/DSpace/dspace-angular/

Quick setup and configuration instructions can be found in the README of that project.

REST API

This component defines the main public API of the Application Layer. See REST API section of the documentation.

OAI-PMH Data Provider

See OAI section of the documentation

RDF / Linked Data Provider

See Linked (Open) Data section of the documentation

SWORD v1 Service / Server

See SWORDv1 Server section of the documentation

SWORD v2 Service / Server

See SWORDv2 Server section of the documentation

DSpace Command Line Launcher

The DSpace Command Launcher brings together the various command and scripts into a standard-practice for running CLI runtime programs. See Command Line Operations

Command Launcher Structure

There are two components to the command launcher: the dspace script and the launcher.xml. The DSpace command calls a java class which in turn refers to *launcher.xml* that is stored in the *[dspace]/config* directory

launcher.xml is made of several components:

- *<command>* begins the stanza for a command
- *<name>_name of command_</name>* the name of the command that you would use.
- *<description>_the description of the command_</description>*
- *<step> </step>* User arguments are parsed and tested.
- *<class>_<the java class that is being used to run the CLI program>_</class>*

See Command Line Operations for additional details.

Business Logic Layer

1 Core Classes
- 1.1 The Configuration Service
- 1.2 Constants
- 1.3 Context
- 1.4 Email
- 1.5 LogManager
- 1.6 Utils

2 Content Management API
- 2.1 Other Classes
- 2.2 Modifications
- 2.3 What's In Memory?
- 2.4 Dublin Core Metadata
- 2.5 Support for Other Metadata Schemas
- 2.6 Packager Plugins

3 Plugin Service
- 3.1 Concepts
- 3.2 Using the Plugin Service
  - 3.2.1 Types of Plugin
  - 3.2.2 Self-Named Plugins
  - 3.2.3 Obtaining a Plugin Instance
  - 3.2.4 Lifecycle Management
  - 3.2.5 Getting Meta-Information
- 3.3 Implementation
  - 3.3.1 LegacyPluginServiceImpl Class
  - 3.3.2 SelfNamedPlugin Class
  - 3.3.3 Errors and Exceptions
- 3.4 Configuring Plugins
  - 3.4.1 Configuring Singleton (Single) Plugins
  - 3.4.2 Configuring Sequence of Plugins
  - 3.4.3 Configuring Named Plugins
- 3.5 Use Cases
  - 3.5.1 Managing the MediaFilter plugins transparently
  - 3.5.2 A Singleton Plugin
  - 3.5.3 Plugin that Names Itself
  - 3.5.4 Stackable Authentication

4 Workflow System
5 Administration Toolkit
6 E-person/Group Manager
7 Authorization
- 7.1 Special Groups
- 7.2 Miscellaneous Authorization Notes

8 Handle Manager/Handle Plugin
9 Search
- 9.1 Harvesting API

10 Browse API
- 10.1 Using the API

11 Checksum checker
12 OpenSearch Support
13 Embargo Support
- 13.1 What is an Embargo?
- 13.2 Embargo Model and Life-Cycle

Core Classes

The *org.dspace.core* package provides some basic classes that are used throughout the DSpace code.

The Configuration Service

The configuration service is responsible for reading the main *dspace.cfg* properties file, managing the 'template' configuration files for other applications such as Apache, and for obtaining the text for e-mail messages.

The system is configured by editing the relevant files in `[dspace]/config`, as described in the configuration section.

When editing configuration files for applications that DSpace uses, such as Apache Tomcat, you may want to edit the copy in `[dspace-source]` and then run `ant update` or `ant overwrite_configs` rather than editing the 'live' version directly! This will ensure you have a backup copy of your modified configuration files, so that they are not accidentally overwritten in the future.

The *ConfigurationService* class can also be invoked as a command line tool:

- `[dspace]/bin/dspace dsprop property.name` This writes the value of *property.name* from *dspace.cfg* to the standard output, so that shell scripts can access the DSpace configuration. If the property has no value, nothing is written.

For many more details on configuration in DSpace, see Configuration Reference

Constants

This class contains constants that are used to represent types of object and actions in the database. For example, authorization policies can relate to objects of different types, so the *resourcepolicy* table has columns *resource_id*, which is the internal ID of the object, and *resource_type_id*, which indicates whether the object is an item, collection, bitstream etc. The value of *resource_type_id* is taken from the *Constants* class, for example *Constants.ITEM*.

Here are a some of the most commonly used constants you might come across:

DSpace types

- Bitstream: 0
- Bundle: 1
- Item: 2
- Collection: 3
- Community: 4
- Site: 5
- Group: 6
- Eperson: 7

DSpace actions

- Read: 0
- Write: 1
- Delete: 2
- Add: 3
- Remove: 4

Refer to the org.dspace.core.Constants for all of the Constants.

Context

The *Context* class is central to the DSpace operation. Any code that wishes to use the any API in the business logic layer must first create itself a *Context* object. This is akin to opening a connection to a database (which is in fact one of the things that happens.)

A context object is involved in most method calls and object constructors, so that the method or object has access to information about the current operation. When the context object is constructed, the following information is automatically initialized:

- A connection to the database. This is a transaction-safe connection. i.e. the 'auto-commit' flag is set to false.
- A cache of content management API objects. Each time a content object is created (for example *Item* or *Bitstream*) it is stored in the *Context* object. If the object is then requested again, the cached copy is used. Apart from reducing database use, this addresses the problem of having two copies of the same object in memory in different states.
  The following information is also held in a context object, though it is the responsibility of the application creating the context object to fill it out correctly:

- The current authenticated user, if any
- Any 'special groups' the user is a member of. For example, a user might automatically be part of a particular group based on the IP address they are accessing DSpace from, even though they don't have an e-person record. Such a group is called a 'special group'.
- Any extra information from the application layer that should be added to log messages that are written within this context. For example, the Web UI adds a session ID, so that when the logs are analyzed the actions of a particular user in a particular session can be tracked.
- A flag indicating whether authorization should be circumvented. This should only be used in rare, specific circumstances. For example, when first installing the system, there are no authorized administrators who would be able to create an administrator account!As noted above, the public API is *trusted*, so it is up to applications in the application layer to use this flag responsibly.
  Typical use of the context object will involve constructing one, and setting the current user if one is authenticated. Several operations may be performed using the context object. If all goes well, *complete* is called to commit the changes and free up any resources used by the context. If anything has gone wrong, *abort* is called to roll back any changes and free up the resources.

You should always *abort* a context if *any* error happens during its lifespan; otherwise the data in the system may be left in an inconsistent state. You can also *commit* a context, which means that any changes are written to the database, and the context is kept active for further use.

Email

Sending e-mails is pretty easy. Just use the configuration manager's *getEmail* method, set the arguments and recipients, and send.

The e-mail texts are stored in `[dspace]/config/emails`. They are processed by the standard *java.text.MessageFormat*. At the top of each e-mail are listed the appropriate arguments that should be filled out by the sender. Example usage is shown in the *org.dspace.core.Email* Javadoc API documentation.

LogManager

The log manager consists of a method that creates a standard log header, and returns it as a string suitable for logging. Note that this class does not actually write anything to the logs; the log header returned should be logged directly by the sender using an appropriate Log4J call, so that information about where the logging is taking place is also stored.

The level of logging can be configured on a per-package or per-class basis by editing `[dspace]/config/log4j.properties`. You will need to stop and restart Tomcat for the changes to take effect.

A typical log entry looks like this:

2002-11-11 08:11:32,903 INFO org.dspace.app.webui.servlet.DSpaceServlet @ anonymous:session_id=BD84E7C194C2CF4BD0EC3A6CAD0142BB: view_item:handle=1721.1/1686

This is breaks down like this:

Date and time, milliseconds	*2002-11-11 08:11:32,903*
Level (*FATAL*, *WARN*, *INFO* or *DEBUG*)	*INFO*
Java class	*org.dspace.app.webui.servlet.DSpaceServlet*
	@
User email or *anonymous*	*anonymous*
	:
Extra log info from context	*session_id=BD84E7C194C2CF4BD0EC3A6CAD0142BB*
	:
Action	*view_item*
	:
Extra info	*handle=1721.1/1686*

The above format allows the logs to be easily parsed and analyzed. The `[dspace]/bin/log-reporter` script is a simple tool for analyzing logs. Try:

```
[dspace]/bin/log-reporter --help
```

It's a good idea to 'nice' this log reporter to avoid an impact on server performance.

Utils

Utils contains miscellaneous utility method that are required in a variety of places throughout the code, and thus have no particular 'home' in a subsystem.

Content Management API

The content management API package *org.dspace.content* contains Java classes for reading and manipulating content stored in the DSpace system. This is the API that components in the application layer will probably use most.

Classes corresponding to the main elements in the DSpace data model (*Community*, *Collection*, *Item*, *Bundle* and *Bitstream*) are sub-classes of the abstract class *DSpaceObject*. The *Item* object handles the Dublin Core metadata record.

Each class generally has one or more static *find* methods, which are used to instantiate content objects. Constructors do not have public access and are just used internally. The reasons for this are:

- "Constructing" an object may be misconstrued as the action of creating an object in the DSpace system, for example one might expect something like:

  ```
  Context dsContent = new Context();
  Item myItem = new Item(context, id)
  ```

  to construct a brand new item in the system, rather than simply instantiating an in-memory instance of an object in the system.
- *find* methods may often be called with invalid IDs, and return *null* in such a case. A constructor would have to throw an exception in this case. A *null* return value from a static method can in general be dealt with more simply in code.
- If an instantiation representing the same underlying archival entity already exists, the *find* method can simply return that same instantiation to avoid multiple copies and any inconsistencies which might result.

Collection, *Bundle* and *Bitstream* do not have *create* methods; rather, one has to create an object using the relevant method on the container. For example, to create a collection, one must invoke *createCollection* on the community that the collection is to appear in:

```
Context context = new Context();
Community existingCommunity = Community.find(context, 123);
Collection myNewCollection = existingCommunity.createCollection();
```

The primary reason for this is for determining authorization. In order to know whether an e-person may create an object, the system must know which container the object is to be added to. It makes no sense to create a collection outside of a community, and the authorization system does not have a policy for that.

Items are first created in the form of an implementation of InProgressSubmission. An *InProgressSubmission* represents an item under construction; once it is complete, it is installed into the main archive and added to the relevant collection by the *InstallItem* class. The *org.dspace.content* package provides an implementation of *InProgressSubmission* called *WorkspaceItem*; this is a simple implementation that contains some fields used by the Web submission UI. The *org.dspace.workflow* also contains an implementation called *WorkflowItem* which represents a submission undergoing a workflow process.

In the previous chapter there is an overview of the item ingest process which should clarify the previous paragraph. Also see the section on the workflow system.

Community and *BitstreamFormat* do have static *create* methods; one must be a site administrator to have authorization to invoke these.

Other Classes

Classes whose name begins *DC* are for manipulating Dublin Core metadata, as explained below.

The *FormatIdentifier* class attempts to guess the bitstream format of a particular bitstream. Presently, it does this simply by looking at any file extension in the bitstream name and matching it up with the file extensions associated with bitstream formats. Hopefully this can be greatly improved in the future!

The *ItemIterator* class allows items to be retrieved from storage one at a time, and is returned by methods that may return a large number of items, more than would be desirable to have in memory at once.

The *ItemComparator* class is an implementation of the standard *java.util.Comparator* that can be used to compare and order items based on a particular Dublin Core metadata field.

Modifications

When creating, modifying or for whatever reason removing data with the content management API, it is important to know when changes happen in-memory, and when they occur in the physical DSpace storage.

Primarily, one should note that no change made using a particular *org.dspace.core.Context* object will actually be made in the underlying storage unless *complete* or *commit* is invoked on that *Context*. If anything should go wrong during an operation, the context should always be aborted by invoking *abort*, to ensure that no inconsistent state is written to the storage.

Additionally, some changes made to objects only happen in-memory. In these cases, invoking the *update* method lines up the in-memory changes to occur in storage when the *Context* is committed or completed. In general, methods that change any metadata field only make the change in-memory; methods that involve relationships with other objects in the system line up the changes to be committed with the context. See individual methods in the API Javadoc.

Some examples to illustrate this are shown below:

`Context context = new Context();`
`Bitstream b = Bitstream.find(context, 1234);`
`b.setName("newfile.txt");`
`b.update();`
`context.complete();`
	Will change storage

`Context context = new Context();`
`Bitstream b = Bitstream.find(context, 1234);`
`b.setName("newfile.txt");`
`b.update();`
`context.abort();`
	Will not change storage (context aborted)

`Context context = new Context();`
`Bitstream b = Bitstream.find(context, 1234);`
`b.setName("newfile.txt");`
`context.complete();`
	The new name **will not** be stored since *update* was not invoked

``
Context context = new Context();
Bitstream bs = Bitstream.find(context, 1234);
Bundle bnd = Bundle.find(context, 5678);
bnd.add(bs);
context.complete();
``
	The bitstream **will** be included in the bundle, since *update* doesn't need to be called

What's In Memory?

Instantiating some content objects also causes other content objects to be loaded into memory.

Instantiating a *Bitstream* object causes the appropriate *BitstreamFormat* object to be instantiated. Of course the *Bitstream* object does not load the underlying bits from the bitstream store into memory!

Instantiating a *Bundle* object causes the appropriate *Bitstream* objects (and hence *BitstreamFormat*s) to be instantiated.

Instantiating an *Item* object causes the appropriate *Bundle* objects (etc.) and hence *BitstreamFormat*s to be instantiated. All the Dublin Core metadata associated with that item are also loaded into memory.

The reasoning behind this is that for the vast majority of cases, anyone instantiating an item object is going to need information about the bundles and bitstreams within it, and this methodology allows that to be done in the most efficient way and is simple for the caller. For example, in the Web UI, the servlet (controller) needs to pass information about an item to the viewer (JSP), which needs to have all the information in-memory to display the item without further accesses to the database which may cause errors mid-display.

You do not need to worry about multiple in-memory instantiations of the same object, or any inconsistencies that may result; the *Context* object keeps a cache of the instantiated objects. The *find* methods of classes in *org.dspace.content* will use a cached object if one exists.

It may be that in enough cases this automatic instantiation of contained objects reduces performance in situations where it is important; if this proves to be true the API may be changed in the future to include a *loadContents* method or somesuch, or perhaps a Boolean parameter indicating what to do will be added to the *find* methods.

When a *Context* object is completed, aborted or garbage-collected, any objects instantiated using that context are invalidated and should not be used (in much the same way an AWT button is invalid if the window containing it is destroyed).

Dublin Core Metadata

The Metadatum class is a simple container that represents a single Dublin Core-like element, optional qualifier, value and language. Note that since DSpace 1.4 the *MetadataValue* and associated classes are preferred (see Support for Other Metadata Schemas). The other classes starting with *DC* are utility classes for handling types of data in Dublin Core, such as people's names and dates. As supplied, the DSpace registry of elements and qualifiers corresponds to the Library Application Profile for Dublin Core. It should be noted that these utility classes assume that the values will be in a certain syntax, which will be true for all data generated within the DSpace system, but since Dublin Core does not always define strict syntax, this may not be true for Dublin Core originating outside DSpace.

Below is the specific syntax that DSpace expects various fields to adhere to:

Element	Qualifier	Syntax	Helper Class
date	Any or unqualified	ISO 8601 in the UTC time zone, with either year, month, day, or second precision. Examples: `2000`, `2002-10`, `2002-08-14`, `1999-01-01T14:35:23Z`	*DCDate*
contributor	Any or unqualified	In general last name, then a comma, then first names, then any additional information like "Jr.". If the contributor is an organization, then simply the name. Examples:_Doe, John Smith, John Jr. van Dyke, Dick Massachusetts Institute of Technology _	*DCPersonName*
language	*iso*	A two letter code taken ISO 639, followed optionally by a two letter country code taken from ISO 3166. Examples:_en fr en_US _	*DCLanguage*
relation	*ispartofseries*	The series name, following by a semicolon followed by the number in that series. Alternatively, just free text._MIT-TR; 1234 My Report Series; ABC-1234 NS1234 _	*DCSeriesNumber*

Support for Other Metadata Schemas

To support additional metadata schemas a new set of metadata classes have been added. These are backwards compatible with the DC classes and should be used rather than the DC specific classes wherever possible. Note that hierarchical metadata schemas are not currently supported, only flat schemas (such as DC) are able to be defined.

The *MetadataField* class describes a metadata field by schema, element and optional qualifier. The value of a *MetadataField* is described by a *MetadataValue* which is roughly equivalent to the older Metadatum class. Finally the *MetadataSchema* class is used to describe supported schemas. The DC schema is supported by default. Refer to the javadoc for method details.

Packager Plugins

The Packager plugins let you *ingest* a package to create a new DSpace Object, and *disseminate* a content Object as a package. A package is simply a data stream; its contents are defined by the packager plugin's implementation.

To ingest an object, which is currently only implemented for Items, the sequence of operations is:

1. Get an instance of the chosen *PackageIngester* plugin.
2. Locate a Collection in which to create the new Item.
3. Call its *ingest* method, and get back a *WorkspaceItem*.
   The packager also takes a *PackageParameters* object, which is a property list of parameters specific to that packager which might be passed in from the user interface.

Here is an example package ingestion code fragment:

```
Collection collection =  find target collection
     InputStream source = ...;
     PackageParameters params = ...;
     String license = null;

    PackageIngester sip = (PackageIngester) PluginManager
            .getNamedPlugin(PackageIngester.class, packageType);

    WorkspaceItem wi = sip.ingest(context, collection, source, params, license);
```

Here is an example of a package dissemination:

```
OutputStream destination = ...;
     PackageParameters params = ...;
     DSpaceObject dso = ...;

     PackageIngester dip = (PackageDisseminator) PluginManager
             .getNamedPlugin(PackageDisseminator.class, packageType);

     dip.disseminate(context, dso, params, destination);
```

Plugin Service

In DSpace 6, the old "PluginManager" was replaced by `org.dspace.core.service.PluginService` which performs the same activities/actions.

The PluginService is a very simple component container. It creates and organizes components (plugins), and helps select a plugin in the cases where there are many possible choices. It also gives some limited control over the life cycle of a plugin.

Concepts

The following terms are important in understanding the rest of this section:

- **Plugin Interface** A Java interface, the defining characteristic of a plugin. The consumer of a plugin asks for its plugin by interface.
- **Plugin** a.k.a. Component, this is an instance of a class that implements a certain interface. It is interchangeable with other implementations, so that any of them may be "plugged in", hence the name. A Plugin is an instance of any class that implements the plugin interface.
- **Implementation class** The actual class of a plugin. It may implement several plugin interfaces, but must implement at least one.
- **Name** Plugin implementations can be distinguished from each other by name, a short String meant to symbolically represent the implementation class. They are called "named plugins". Plugins only need to be named when the caller has to make an active choice between them.
- **SelfNamedPlugin class** Plugins that extend the *SelfNamedPlugin* class can take advantage of additional features of the Plugin Manager. Any class can be managed as a plugin, so it is not necessary, just possible.
- **Reusable** Reusable plugins are only instantiated once, and the Plugin Manager returns the same (cached) instance whenever that same plugin is requested again. This behavior can be turned off if desired.

Using the Plugin Service

Types of Plugin

The Plugin Service supports three different patterns of usage:

1. **Singleton Plugins** There is only one implementation class for the plugin. It is indicated in the configuration. This type of plugin chooses an implementation of a service, for the entire system, at configuration time. Your application just fetches the plugin for that interface and gets the configured-in choice. See the `getSinglePlugin()` method.
2. **Sequence Plugins** You need a sequence or series of plugins, to implement a mechanism like Stackable Authentication or a pipeline, where each plugin is called in order to contribute its implementation of a process to the whole. The Plugin Manager supports this by letting you configure a sequence of plugins for a given interface. See the `getPluginSequence()` method.
3. **Named Plugins** Use a named plugin when the application has to choose one plugin implementation out of many available ones. Each implementation is bound to one or more names (symbolic identifiers) in the configuration. The name is just a string to be associated with the

combination of implementation class and interface. It may contain any characters except for comma (,) and equals (=). It may contain embedded spaces. Comma is a special character used to separate names in the configuration entry. Names must be unique within an interface: No plugin classes implementing the same interface may have the same name. Think of plugin names as a controlled vocabulary – for a given plugin interface, there is a set of names for which plugins can be found. The designer of a Named Plugin interface is responsible for deciding what the name means and how to derive it; for example, names of metadata crosswalk plugins may describe the target metadata format. See the `getNamedPlugin()` method and the `getAllPluginNames()` methods.

Self-Named Plugins

Named plugins can get their names either from the configuration or, for a variant called self-named plugins, from within the plugin itself.

Self-named plugins are necessary because one plugin implementation can be configured itself to take on many "personalities", each of which deserves its own plugin name. It is already managing its own configuration for each of these personalities, so it makes sense to allow it to export them to the Plugin Manager rather than expecting the plugin configuration to be kept in sync with it own configuration.

An example helps clarify the point: There is a named plugin that does crosswalks, call it *CrosswalkPlugin*. It has several implementations that crosswalk some kind of metadata. Now we add a new plugin which uses XSL stylesheet transformation (XSLT) to crosswalk many types of metadata – so the single plugin can act like many different plugins, depending on which stylesheet it employs.

This XSLT-crosswalk plugin has its own configuration that maps a Plugin Name to a stylesheet – it has to, since of course the Plugin Manager doesn't know anything about stylesheets. It becomes a self-named plugin, so that it reads its configuration data, gets the list of names to which it can respond, and passes those on to the Plugin Manager.

When the Plugin Service creates an instance of the XSLT-crosswalk, it records the Plugin Name that was responsible for that instance. The plugin can look at that Name later in order to configure itself correctly for the Name that created it. This mechanism is all part of the SelfNamedPlugin class which is part of any self-named plugin.

Obtaining a Plugin Instance

The most common thing you will do with the Plugin Service is obtain an instance of a plugin. To request a plugin, you must always specify the plugin interface you want. You will also supply a name when asking for a named plugin.

A sequence plugin is returned as an array of _Object_s since it is actually an ordered list of plugins.

See the getSinglePlugin(), getPluginSequence(), getNamedPlugin() methods.

Lifecycle Management

When *PluginService* fulfills a request for a plugin, a new instance is always created.

Getting Meta-Information

The *PluginService* can list all the names of the Named Plugins which implement an interface. You may need this, for example, to implement a menu in a user interface that presents a choice among all possible plugins. See the getAllPluginNames() method.

Note that it only returns the plugin name, so if you need a more sophisticated or meaningful "label" (i.e. a key into the I18N message catalog) then you should add a method to the plugin itself to return that.

Implementation

Note: The *PluginService* refers to interfaces and classes internally only by their names whenever possible, to avoid loading classes until absolutely necessary (i.e. to create an instance). As you'll see below, self-named classes still have to be loaded to query them for names, but for the most part it can avoid loading classes. This saves a lot of time at start-up and keeps the JVM memory footprint down, too. As the Plugin Manager gets used for more classes, this will become a greater concern.

The only downside of "on-demand" loading is that errors in the configuration don't get discovered right away. The solution is to call the *checkConfiguration()* method after making any changes to the configuration.

LegacyPluginServiceImpl Class

The *LegacyPluginServiceImpl* class is the default PluginService implementation. While it is possible to implement your own version of PluginService, no other implementations are provided with DSpace

Here are the public methods, followed by explanations:

- `Object getSinglePlugin(Class interfaceClass)` - Returns an instance of the singleton (single) plugin implementing the given interface. There must be exactly one single plugin configured for this interface, otherwise the *PluginConfigurationError* is thrown. Note that this is the only "get plugin" method which throws an exception. It is typically used at initialization time to set up a permanent part of the system so any failure is fatal. See the *plugin.single* configuration key for configuration details.
- `Object[] getPluginSequence(Class interfaceClass)` - Returns instances of all plugins that implement the interface *interfaceClass*, in an *Array*. Returns an empty array if no there are no matching plugins. The order of the plugins in the array is the same as their class names in the configuration's value field. See the *plugin.sequence* configuration key for configuration details.
- `Object getNamedPlugin(Class interfaceClass, String name)` - Returns an instance of a plugin that implements the interface *interfaceClass* and is bound to a name matching name. If there is no matching plugin, it returns null. The names are matched by *String.equals()*. See the *plugin.named* and *plugin.selfnamed* configuration keys for configuration details.

- `String[] getAllPluginNames(Class interfaceClass)` - Returns all of the names under which a named plugin implementing the interface *interfaceClass* can be requested (with *getNamedPlugin()*). The array is empty if there are no matches. Use this to populate a menu of plugins for interactive selection, or to document what the possible choices are. The names are NOT returned in any predictable order, so you may wish to sort them first. Note: Since a plugin may be bound to more than one name, the list of names this returns does not represent the list of plugins. To get the list of unique implementation classes corresponding to the names, you might have to eliminate duplicates (i.e. create a Set of classes).

SelfNamedPlugin Class

A named plugin implementation must extend this class if it wants to supply its own Plugin Name(s). See Self-Named Plugins for why this is sometimes necessary.

```
abstract class SelfNamedPlugin
{
    // Your class must override this:
    // Return all names by which this plugin should be known.
    public static String[] getPluginNames();

    // Returns the name under which this instance was created.
    // This is implemented by SelfNamedPlugin and should NOT be
        overridden.
    public String getPluginInstanceName();
}
```

Errors and Exceptions

```
public class PluginConfigurationError extends Error
{
    public PluginConfigurationError(String message);
}
```

An error of this type means the caller asked for a single plugin, but either there was no single plugin configured matching that interface, or there was more than one. Either case causes a fatal configuration error.

```
public class PluginInstantiationException extends RuntimeException
{
    public PluginInstantiationException(String msg, Throwable cause)
}
```

This exception indicates a fatal error when instantiating a plugin class. It should only be thrown when something unexpected happens in the course of instantiating a plugin, e.g. an access error, class not found, etc. Simply not finding a class in the configuration is not an exception.

This is a *RuntimeException* so it doesn't have to be declared, and can be passed all the way up to a generalized fatal exception handler.

Configuring Plugins

All of the Plugin Service's configuration comes from the DSpace Configuration Service (see Configuration Reference). You can configure these characteristics of each plugin:

1. **Interface**: Classname of the Java interface which defines the plugin, including package name. e.g. *org.dspace.app.mediafilter.FormatFilter*
2. **Implementation Class**: Classname of the implementation class, including package. e.g. *org.dspace.app.mediafilter.PDFFilter*
3. **Names**: (Named plugins only) There are two ways to bind names to plugins: listing them in the value of a plugin.named.interface key, or configuring a class in *plugin.selfnamed.interface* which extends the *SelfNamedPlugin* class.
4. **Reusable option**: (Optional) This is declared in a *plugin.reusable* configuration line. Plugins are reusable by default, so you only need to configure the non-reusable ones.

Configuring Singleton (Single) Plugins

This entry configures a Single Plugin for use with getSinglePlugin():

```
plugin.single.interface = classname
```

For example, this configures the class *org.dspace.checker.SimpleDispatcher* as the plugin for interface *org.dspace.checker.BitstreamDispatcher*:

```
plugin.single.org.dspace.checker.BitstreamDispatcher=org.dspace.checker.SimpleDispatcher
```

Configuring Sequence of Plugins

This kind of configuration entry defines a Sequence Plugin, which is bound to a sequence of implementation classes. The key identifies the interface, and the value is a comma-separated list of classnames:
plugin.sequence.interface = classname, ...
The plugins are returned by *getPluginSequence()* in the same order as their classes are listed in the configuration value.

For example, this entry configures Stackable Authentication with three implementation classes:

```
plugin.sequence.org.dspace.eperson.AuthenticationMethod = \
            org.dspace.eperson.X509Authentication, \
            org.dspace.eperson.PasswordAuthentication, \
            edu.mit.dspace.MITSpecialGroup
```

Configuring Named Plugins

There are two ways of configuring named plugins:

1. **Plugins Named in the Configuration** A named plugin which gets its name(s) from the configuration is listed in this kind of entry:_plugin.named. interface = classname = name [, name..] [classname = name..]_The syntax of the configuration value is: classname, followed by an equal-sign and then at least one plugin name. Bind more names to the same implementation class by adding them here, separated by commas. Names may include any character other than comma (,) and equal-sign (=).For example, this entry creates one plugin with the names GIF, JPEG, and image /png, and another with the name TeX:

   ```
   plugin.named.org.dspace.app.mediafilter.MediaFilter = \
           org.dspace.app.mediafilter.JPEGFilter = GIF, JPEG, image/png \
           org.dspace.app.mediafilter.TeXFilter = TeX
   ```

   This example shows a plugin name with an embedded whitespace character. Since comma (,) is the separator character between plugin names, spaces are legal (between words of a name; leading and trailing spaces are ignored).This plugin is bound to the names "Adobe PDF", "PDF", and "Portable Document Format".

   ```
   plugin.named.org.dspace.app.mediafilter.MediaFilter = \
         org.dspace.app.mediafilter.TeXFilter = TeX \
         org.dspace.app.mediafilter.PDFFilter =  Adobe PDF, PDF, Portable Document Format
   ```

   NOTE: Since there can only be one key with plugin.named. followed by the interface name in the configuration, all of the plugin implementations must be configured in that entry.
2. **Self-Named Plugins** Since a self-named plugin supplies its own names through a static method call, the configuration only has to include its interface and classname:*plugin.selfnamed.interface = classname [, classname..] The following example first demonstrates how the plugin class, XsltDisseminationCrosswalk* is configured to implement its own names "MODS" and "DublinCore". These come from the keys starting with *crosswalk.dissemination.stylesheet.*. The value is a stylesheet file. The class is then configured as a self-named plugin:

   ```
   crosswalk.dissemination.stylesheet.DublinCore = xwalk/TESTDIM-2-DC_copy.xsl
   crosswalk.dissemination.stylesheet.MODS = xwalk/mods.xsl

   plugin.selfnamed.crosswalk.org.dspace.content.metadata.DisseminationCrosswalk = \
           org.dspace.content.metadata.MODSDisseminationCrosswalk, \
           org.dspace.content.metadata.XsltDisseminationCrosswalk
   ```

   NOTE: Since there can only be one key with *plugin.selfnamed.* followed by the interface name in the configuration, all of the plugin implementations must be configured in that entry. The *MODSDisseminationCrosswalk* class is only shown to illustrate this point.

Use Cases

Here are some usage examples to illustrate how the Plugin Service works.

Managing the MediaFilter plugins transparently

The MediaFilterService implementation relies heavily on the Plugin Service. The MediaFilter classes become plugins named in the configuration. Refer to the Configuration Reference for further details.

A Singleton Plugin

This shows how to configure and access a single anonymous plugin, such as the BitstreamDispatcher plugin:

Configuration:

plugin.single.org.dspace.checker.BitstreamDispatcher=org.dspace.checker.SimpleDispatcher

The following code fragment shows how dispatcher, the service object, is initialized and used:

```
BitstreamDispatcher dispatcher = (BitstreamDispatcher)PluginManager.getSinglePlugin(BitstreamDispatcher.class);

int id = dispatcher.next();

while (id != BitstreamDispatcher.SENTINEL)
{
     /*
        do some processing here
     */

     id = dispatcher.next();
}
```

Plugin that Names Itself

This crosswalk plugin acts like many different plugins since it is configured with different XSL translation stylesheets. Since it already gets each of its stylesheets out of the DSpace configuration, it makes sense to have the plugin give PluginService the names to which it answers instead of forcing someone to configure those names in two places (and try to keep them synchronized).

Here is the configuration file listing both the plugin's own configuration and the *PluginService* config line:

```
crosswalk.dissemination.stylesheet.DublinCore = xwalk/TESTDIM-2-DC_copy.xsl
crosswalk.dissemination.stylesheet.MODS = xwalk/mods.xsl

plugin.selfnamed.org.dspace.content.metadata.DisseminationCrosswalk = \
  org.dspace.content.metadata.XsltDisseminationCrosswalk
```

This look into the implementation shows how it finds configuration entries to populate the array of plugin names returned by the *getPluginNames()* method. Also note, in the *getStylesheet()* method, how it uses the plugin name that created the current instance (returned by *getPluginInstanceName()*) to find the correct stylesheet.

```
public class XsltDisseminationCrosswalk extends SelfNamedPlugin
{
    ....
    private final String prefix =
        "crosswalk.dissemination.stylesheet.";
    ....
    public static String[] getPluginNames()
    {
        List aliasList = new ArrayList();
        Enumeration pe = ConfigurationManager.propertyNames();

        while (pe.hasMoreElements())
        {
            String key = (String)pe.nextElement();
            if (key.startsWith(prefix))
                aliasList.add(key.substring(prefix.length()));
        }
        return (String[])aliasList.toArray(new
        String[aliasList.size()]);
    }

    // get the crosswalk stylesheet for an instance of the plugin:
    private String getStylesheet()
    {
        return ConfigurationManager.getProperty(prefix +
        getPluginInstanceName());
    }
}
```

Stackable Authentication

The Stackable Authentication mechanism needs to know all of the plugins configured for the interface, in the order of configuration, since order is significant. It gets a Sequence Plugin from the Plugin Manager. Refer to the Configuration Section on Stackable Authentication for further details.

Workflow System

The primary classes are:

org.dspace.content. WorkspaceItem	contains an Item before it enters a workflow
org.dspace.workflow. WorkflowItem	contains an Item while in a workflow
org.dspace.workflow. WorkflowService	responds to events, manages the WorkflowItem states. There are two implementations, the traditional, default workflow (described below) and Configurable Workflow.
org.dspace.content. Collection	contains List of defined workflow steps
org.dspace.eperson.Group	people who can perform workflow tasks are defined in EPerson Groups
org.dspace.core.Email	used to email messages to Group members and submitters

The default workflow system models the states of an Item in a state machine with 5 states (SUBMIT, STEP_1, STEP_2, STEP_3, ARCHIVE.) These are the three optional steps where the item can be viewed and corrected by different groups of people. Actually, it's more like 8 states, with STEP_1_POOL, STEP_2_POOL, and STEP_3_POOL. These pooled states are when items are waiting to enter the primary states. Optionally, you can also choose to enable the enhanced, Configurable Workflow, if you wish to have more control over your workflow steps/states. (*Note: the remainder of this description relates to the traditional, default workflow. For more information on the Configurable Workflow option, visit Configurable Workflow* .)

The WorkflowService is invoked by events. While an Item is being submitted, it is held by a WorkspaceItem. Calling the start() method in the WorkflowService converts a WorkspaceItem to a WorkflowItem, and begins processing the WorkflowItem's state. Since all three steps of the workflow are optional, if no steps are defined, then the Item is simply archived.

Workflows are set per Collection, and steps are defined by creating corresponding entries in the List named workflowGroup. If you wish the workflow to have a step 1, use the administration tools for Collections to create a workflow Group with members who you want to be able to view and approve the Item, and the workflowGroup[0] becomes set with the ID of that Group.

If a step is defined in a Collection's workflow, then the WorkflowItem's state is set to that step_POOL. This pooled state is the WorkflowItem waiting for an EPerson in that group to claim the step's task for that WorkflowItem. The WorkflowManager emails the members of that Group notifying them that there is a task to be performed (the text is defined in config/emails,) and when an EPerson goes to their 'My DSpace' page to claim the task, the WorkflowManager is invoked with a claim event, and the WorkflowItem's state advances from STEP_x_POOL to STEP_x (where x is the corresponding step.) The EPerson can also generate an 'unclaim' event, returning the WorkflowItem to the STEP_x_POOL.

Other events the WorkflowService handles are advance(), which advances the WorkflowItem to the next state. If there are no further states, then the WorkflowItem is removed, and the Item is then archived. An EPerson performing one of the tasks can reject the Item, which stops the workflow, rebuilds the WorkspaceItem for it and sends a rejection note to the submitter. More drastically, an abort() event is generated by the admin tools to cancel a workflow outright.

Administration Toolkit

The *org.dspace.administer* package contains some classes for administering a DSpace system that are not generally needed by most applications.

The *CreateAdministrator* class is a simple command-line tool, executed via `[dspace]/bin/dspace create-administrator`, that creates an administrator e-person with information entered from standard input. This is generally used only once when a DSpace system is initially installed, to create an initial administrator who can then use the Web administration UI to further set up the system. This script does not check for authorization, since it is typically run before there are any e-people to authorize! Since it must be run as a command-line tool on the server machine, generally this shouldn't cause a problem. A possibility is to have the script only operate when there are no e-people in the system already, though in general, someone with access to command-line scripts on your server is probably in a position to do what they want anyway!

The *DCType* class is similar to the *org.dspace.content.BitstreamFormat* class. It represents an entry in the Dublin Core type registry, that is, a particular element and qualifier, or unqualified element. It is in the *administer* package because it is only generally required when manipulating the registry itself. Elements and qualifiers are specified as literals in *org.dspace.content.Item* methods and the *org.dspace.content.Metadatum* class. Only administrators may modify the Dublin Core type registry.

The *org.dspace.administer.RegistryLoader* class contains methods for initializing the Dublin Core type registry and bitstream format registry with entries in an XML file. Typically this is executed via the command line during the build process (see *build.xml* in the source.) To see examples of the XML formats, see the files in *config/registries* in the source directory. There is no XML schema, they aren't validated strictly when loaded in.

E-person/Group Manager

DSpace keeps track of registered users with the *org.dspace.eperson.EPerson* class. The class has methods to create and manipulate an *EPerson* such as get and set methods for first and last names, email, and password. (Actually, there is no *getPassword()* method, an MD5 hash of the password is stored, and can only be verified with the *checkPassword()* method.) There are find methods to find an EPerson by email (which is assumed to be unique,) or to find all EPeople in the system.

The *EPerson* object should probably be reworked to allow for easy expansion; the current EPerson object tracks pretty much only what MIT was interested in tracking - first and last names, email, phone. The access methods are hardcoded and should probably be replaced with methods to access arbitrary name/value pairs for institutions that wish to customize what EPerson information is stored.

Groups are simply lists of *EPerson* objects. Other than membership, *Group* objects have only one other attribute: a name. Group names must be unique, so (for groups associated with workflows) we have adopted naming conventions where the role of the group is its name, such as *COLLECTION_100_ADD*. Groups add and remove EPerson objects with *addMember()* and *removeMember()* methods. One important thing to know about groups is that they store their membership in memory until the *update()* method is called - so when modifying a group's membership don't forget to invoke *update()* or your changes will be lost! Since group membership is used heavily by the authorization system a fast *isMember()* method is also provided.

Two specific groups are created when DSpace is installed: Administrator (which can bypass authorization) and Anonymous (which is assigned to all sessions that are not logged in). The code expects these groups to exist. They cannot be renamed or deleted.

Another kind of Group is also implemented in DSpace, special Groups. The *Context* object for each session carries around a List of Group IDs that the user is also a member of, currently the MITUser Group ID is added to the list of a user's special groups if certain IP address or certificate criteria are met.

Authorization

The primary classes are:

org.dspace.authorize.AuthorizeService	does all authorization, checking policies against Groups
org.dspace.authorize.ResourcePolicy	defines all allowable actions for an object
org.dspace.eperson.Group	all policies are defined in terms of EPerson Groups

The authorization system is based on the classic 'police state' model of security; no action is allowed unless it is expressed in a policy. The policies are attached to resources (hence the name *ResourcePolicy*,) and detail who can perform that action. The resource can be any of the DSpace object types, listed in *org.dspace.core.Constants* (*BITSTREAM*, *ITEM*, *COLLECTION*, etc.) The 'who' is made up of EPerson groups. The actions are also in *Constants. java* (*READ*, *WRITE*, *ADD*, etc.) The only non-obvious actions are *ADD* and *REMOVE*, which are authorizations for container objects. To be able to create an Item, you must have *ADD* permission in a Collection, which contains Items. (Communities, Collections, Items, and Bundles are all container objects.)

Currently most of the read policy checking is done with items, communities and collections are assumed to be openly readable, but items and their bitstreams are checked. Separate policy checks for items and their bitstreams enables policies that allow publicly readable items, but parts of their content may be restricted to certain groups.

Three new attributes have been introduced in the *ResourcePolicy* class as part of the DSpace Embargo Contribution:

- *rpname*: resource policy name
- *rptype*: resource policy type
- *rpdescription*: resource policy description

While *rpname* and *rpdescription _are fields manageable by the users the _rptype* is a fields managed by the system. It represents a type that a resource policy can assume beteween the following:

- TYPE_SUBMISSION: all the policies added automatically during the submission process
- TYPE_WORKFLOW: all the policies added automatically during the workflow stage
- TYPE_CUSTOM: all the custom policies added by users
- TYPE_INHERITED: all the policies inherited by the *DSO* father.

An custom policy, created for the purpose of creating an embargo could look like:

```
policy_id: 4847
resource_type_id: 2
resource_id: 89
action_id: 0
eperson_id:
epersongroup_id: 0
start_date: 2013-01-01
end_date:
rpname: Embargo Policy
rpdescription:  Embargoed through 2012
rptype: TYPE_CUSTOM
```

The *AuthorizeService* class'
authorizeAction(Context, object, action) is the primary source of all authorization in the system. It gets a list of all of the ResourcePolicies in the system that match the object and action. It then iterates through the policies, extracting the EPerson Group from each policy, and checks to see if the EPersonID from the Context is a member of any of those groups. If all of the policies are queried and no permission is found, then an *AuthorizeException* is thrown. An *authorizeAction()* method is also supplied that returns a boolean for applications that require higher performance.

ResourcePolicies are very simple, and there are quite a lot of them. Each can only list a single group, a single action, and a single object. So each object will likely have several policies, and if multiple groups share permissions for actions on an object, each group will get its own policy. (It's a good thing they're small.)

Special Groups

All users are assumed to be part of the public group (ID=0.) DSpace admins (ID=1) are automatically part of all groups, much like super-users in the Unix OS. The Context object also carries around a List of special groups, which are also first checked for membership. These special groups are used at MIT to indicate membership in the MIT community, something that is very difficult to enumerate in the database! When a user logs in with an MIT certificate or with an MIT IP address, the login code adds this MIT user group to the user's Context.

Miscellaneous Authorization Notes

Where do items get their read policies? From the their collection's read policy. There once was a separate item read default policy in each collection, and perhaps there will be again since it appears that administrators are notoriously bad at defining collection's read policies. There is also code in place to enable policies that are timed, have a start and end date. However, the admin tools to enable these sorts of policies have not been written.

Handle Manager/Handle Plugin

The *org.dspace.handle* package contains two classes; *HandleService* is used to create and look up Handles, and *HandlePlugin* is used to expose and resolve DSpace Handles for the outside world via the CNRI Handle Server code.

Handles are stored internally in the *handle* database table in the form:

1721.123/4567

Typically when they are used outside of the system they are displayed in either URI or "URL proxy" forms:

```
hdl:1721.123/4567
http://hdl.handle.net/1721.123/4567
```

It is the responsibility of the caller to extract the basic form from whichever displayed form is used.

The *handle* table maps these Handles to resource type/resource ID pairs, where resource type is a value from *org.dspace.core.Constants* and resource ID is the internal identifier (database primary key) of the object. This allows Handles to be assigned to any type of object in the system, though as explained in the functional overview, only communities, collections and items are presently assigned Handles.

HandleService contains static methods for:

- Creating a Handle
- Finding the Handle for a *DSpaceObject*, though this is usually only invoked by the object itself, since *DSpaceObject* has a *getHandle* method
- Retrieving the *DSpaceObject* identified by a particular Handle
- Obtaining displayable forms of the Handle (URI or "proxy URL").
  HandlePlugin is a simple implementation of the Handle Server's *net.handle.hdllib.HandleStorage* interface. It only implements the basic Handle retrieval methods, which get information from the *handle* database table. The CNRI Handle Server is configured to use this plug-in via its *config.dct* file.

Note that since the Handle server runs as a separate JVM to the DSpace Web applications, it uses a separate 'Log4J' configuration, since Log4J does not support multiple JVMs using the same daily rolling logs. This alternative configuration is located at `[dspace]/config/log4j-handle-plugin.properties`. The `[dspace]/bin/start-handle-server` script passes in the appropriate command line parameters so that the Handle server uses this configuration.

In additional to Handles, DSpace also provides basic support for DOIs (Digital Object Identifiers). For more information visit DOI Digital Object Identifier.

Search

DSpace's search code is a simple, configurable API which currently wraps Apache Solr. See Discovery for more information on how to customize the default search settings, etc.

Harvesting API

The *org.dspace.search* package also provides a 'harvesting' API. This allows callers to extract information about items modified within a particular timeframe, and within a particular scope (all of DSpace, or a community or collection.) Currently this is used by the Open Archives Initiative metadata harvesting protocol application, and the e-mail subscription code.

The *Harvest.harvest* is invoked with the required scope and start and end dates. Either date can be omitted. The dates should be in the ISO8601, UTC time zone format used elsewhere in the DSpace system.

HarvestedItemInfo objects are returned. These objects are simple containers with basic information about the items falling within the given scope and date range. Depending on parameters passed to the *harvest* method, the *containers* and *item* fields may have been filled out with the IDs of communities and collections containing an item, and the corresponding *Item* object respectively. Electing not to have these fields filled out means the harvest operation executes considerable faster.

In case it is required, *Harvest* also offers a method for creating a single *HarvestedItemInfo* object, which might make things easier for the caller.

Browse API

The browse API uses the same underlying technology as the Search API (Apache Solr, see also Discovery). It maintains indexes of dates, authors, titles and subjects, and allows callers to extract parts of these:

- **Title**: Values of the Dublin Core element **title** (unqualified) are indexed. These are sorted in a case-insensitive fashion, with any leading article removed. For example: "The DSpace System" would appear under 'D' rather than 'T'.
- **Author**: Values of the **contributor** (any qualifier or unqualified) element are indexed. Since *contributor* values typically are in the form 'last name, first name', a simple case-insensitive alphanumeric sort is used which orders authors in last name order. Note that this is an index of *authors*, and not *items by author*. If four items have the same author, that author will appear in the index only once. Hence, the index of authors may be greater or smaller than the index of titles; items often have more than one author, though the same author may have authored several items. The author indexing in the browse API does have limitations:
    - Ideally, a name that appears as an author for more than one item would appear in the author index only once. For example, 'Doe, John' may be the author of tens of items. However, in practice, author's names often appear in slightly differently forms, for example:

      ```
      Doe, John
      Doe, John Stewart
      Doe, John S.
      ```

      Currently, the above three names would all appear as separate entries in the author index even though they may refer to the same author. In order for an author of several papers to be correctly appear once in the index, each item must specify *exactly* the same form of their name, which doesn't always happen in practice.
    - Another issue is that two authors may have the same name, even within a single institution. If this is the case they may appear as one author in the index. These issues are typically resolved in libraries with *authority control records*, in which are kept a 'preferred' form of the author's name, with extra information (such as date of birth/death) in order to distinguish between authors of the same name. Maintaining such records is a huge task with many issues, particularly when metadata is received from faculty directly rather than trained library catalogers.
- **Date of Issue**: Items are indexed by date of issue. This may be different from the date that an item appeared in DSpace; many items may have been originally published elsewhere beforehand. The Dublin Core field used is **date.issued**. The ordering of this index may be reversed so 'earliest first' and 'most recent first' orderings are possible. Note that the index is of *items by date*, as opposed to an index of *dates*. If 30 items have the same issue date (say 2002), then those 30 items all appear in the index adjacent to each other, as opposed to a single 2002 entry. Since dates in DSpace Dublin Core are in ISO8601, all in the UTC time zone, a simple alphanumeric sort is sufficient to sort by date, including dealing with varying granularities of date reasonably. For example:

  ```
  2001-12-10
  2002
  2002-04
  2002-04-05
  2002-04-09T15:34:12Z
  2002-04-09T19:21:12Z
  2002-04-10
  ```

- **Date Accessioned**: In order to determine which items most recently appeared, rather than using the date of issue, an item's accession date is used. This is the Dublin Core field **date.accessioned**. In other aspects this index is identical to the date of issue index.
- **Items by a Particular Author**: The browse API can perform is to extract items by a particular author. They do not have to be primary author of an item for that item to be extracted. You can specify a scope, too; that is, you can ask for items by author X in collection Y, for example.This particular flavor of browse is slightly simpler than the others. You cannot presently specify a particular subset of results to be returned. The API call will simply return all of the items by a particular author within a certain scope. Note that the author of the item must *exactly* match the author passed in to the API; see the explanation about the caveats of the author index browsing to see why this is the case.
- **Subject**: Values of the Dublin Core element **subject** (both unqualified and with any qualifier) are indexed. These are sorted in a case-insensitive fashion.

Using the API

The API is generally invoked by creating a *BrowseScope* object, and setting the parameters for which particular part of an index you want to extract. This is then passed to the relevant *Browse* method call, which returns a *BrowseInfo* object which contains the results of the operation. The parameters set in the *B rowseScope* object are:

- How many entries from the index you want
- Whether you only want entries from a particular community or collection, or from the whole of DSpace
- Which part of the index to start from (called the *focus* of the browse). If you don't specify this, the start of the index is used
- How many entries to include before the *focus* entry

To illustrate, here is an example:

- We want **7** entries in total
- We want entries from collection *x*
- We want the focus to be 'Really'
- We want **2** entries included before the focus.

The results of invoking *Browse.getItemsByTitle* with the above parameters might look like this:

```
Rabble-Rousing Rabbis From Sardinia
        Reality TV: Love It or Hate It?
FOCUS>  The Really Exciting Research Video
        Recreational Housework Addicts: Please Visit My House
        Regional Television Variation Studies
        Revenue Streams
        Ridiculous Example Titles:  I'm Out of Ideas
```

Note that in the case of title and date browses, *Item* objects are returned as opposed to actual titles. In these cases, you can specify the 'focus' to be a specific item, or a partial or full literal value. In the case of a literal value, if no entry in the index matches exactly, the closest match is used as the focus. It's quite reasonable to specify a focus of a single letter, for example.

Being able to specify a specific item to start at is particularly important with dates, since many items may have the save issue date. Say 30 items in a collection have the issue date 2002. To be able to page through the index 20 items at a time, you need to be able to specify exactly which item's 2002 is the focus of the browse, otherwise each time you invoked the browse code, the results would start at the first item with the issue date 2002.

Author browses return *String* objects with the actual author names. You can only specify the focus as a full or partial literal *String*.

Another important point to note is that presently, the browse indexes contain metadata for all items in the main archive, regardless of authorization policies. This means that all items in the archive will appear to all users when browsing. Of course, should the user attempt to access a non-public item, the usual authorization mechanism will apply. Whether this approach is ideal is under review; implementing the browse API such that the results retrieved reflect a user's level of authorization may be possible, but rather tricky.

Checksum checker

Checksum checker is used to verify every item within DSpace. While DSpace calculates and records the checksum of every file submitted to it, the checker can determine whether the file has been changed. The idea being that the earlier you can identify a file has changed, the more likely you would be able to record it (assuming it was not a wanted change).

`org.dspace.checker.CheckerCommand` class, is the class for the checksum checker tool, which calculates checksums for each bitstream whose ID is in the *most_recent_checksum* table, and compares it against the last calculated checksum for that bitstream.

OpenSearch Support

DSpace is able to support the OpenSearch protocol. For those not acquainted with the standard, a very brief introduction, with emphasis on what possibilities it holds for current use and future development.

OpenSearch is a small set of conventions and documents for describing and using 'search engines', meaning any service that returns a set of results for a query. It is nearly ubiquitous, but also nearly invisible, in modern web sites with search capability. If you look at the page source of Wikipedia, Facebook, CNN, etc you will find buried a link element declaring OpenSearch support. It is very much a lowest-common-denominator abstraction (think Google box), but does provide a means to extend its expressive power. This first implementation for DSpace supports *none* of these extensions, many of which are of potential value, so it should be regarded as a foundation, not a finished solution. So the short answer is that DSpace appears as a 'search-engine' to OpenSearch-aware software.

Another way to look at OpenSearch is as a RESTful web service for search, very much like SRW/U, but considerably simpler. This comparative loss of power is offset by the fact that it is widely supported by web tools and players: browsers understand it, as do large metasearch tools.

How Can It Be Used

- Browser Integration: Many recent browsers (IE7+, FF2+) can detect, or 'autodiscover', links to the document describing the search engine. Thus you can easily add your or other DSpace instances to the drop-down list of search engines in your browser. This list typically appears in the upper right corner of the browser, with a search box. In Firefox, for example, when you visit a site supporting OpenSearch, the color of the drop-down list widget changes color, and if you open it to show the list of search engines, you are offered an opportunity to add the site to the list. IE works nearly the same way but instead labels the web sites 'search providers'. When you select a DSpace instance as the search engine and enter a search, you are simply sent to the regular search results page of the instance.
- Flexible, interesting RSS Feeds. Because one of the formats that OpenSearch specifies for its results is RSS (or Atom), you can turn any search query into an RSS feed. So if there are keywords highly discriminative of content in a collection or repository, these can be turned into a URL that a feed reader can subscribe to. Taken to the extreme, one could take any search a user makes, and dynamically compose an RSS feed URL for it in the page of returned results. To see an example, if you have a DSpace with OpenSearch enabled, try:

  ```
  # The Opensearch feature is available from the dspace.server.url
  [dspace.server.url]/opensearch/search?query=<your query>
  # e.g. https://demo.dspace.org/server/opensearch/search?query=<your query>
  ```

  The default format returned is Atom 1.0, so you should see an Atom document containing your search results.
- You can extend the syntax with a few other parameters, as follows:

Parameter	Values
format	atom, rss, html

scope	UUID of a collection or community to restrict the search to
rpp	number indicating the number of results per page (i.e. per request)
start	number of page to start with (if paginating results)
sort	metadata field to sort by, for example "dc.title" (see `webui.itemlist.sort-option` in `dspace.cfg`)
sort_direction	ASC or DESC (default ASC)

Multiple parameters may be specified on the query string, using the "&" character as the delimiter, e.g.:

```
https://demo.dspace.org/server/opensearch/search?query=<your query>&format=rss&scope=40250cb0-09d3-4b22-b5c5-c39bc815f6ea
```

- Cheap metasearchSearch aggregators like A9 (Amazon) recognize OpenSearch-compliant providers, and so can be added to metasearch sets using their UIs. Then you site can be used to aggregate search results with others.
- When OpenSearch is enabled in DSpace, informational "link" tags will be embedded into the HTML of every page. These link tags will allow tools to easily discover the OpenSearch Service Document (`/service`) and RSS / Atom feeds. The links appear in the HTML head tag and look like this:

```
<link href="https://demo.dspace.org/server/opensearch/search?format=atom&query=*" type="application/atom+xml" rel="alternate" title="Sitewide Atom feed">
<link href="https://demo.dspace.org/server/opensearch/search?format=rss&query=*" type="application/rss+xml" rel="alternate" title="Sitewide RSS feed">
<link href="https://demo.dspace.org/server/opensearch/search/service" type="application/atom+xml" rel="search" title="DSpace">
```

Configuration is through the `dspace.cfg` file. See OpenSearch Support for more details.

Embargo Support

What is an Embargo?

An embargo is a temporary access restriction placed on content, commencing at time of accession. It's scope or duration may vary, but the fact that it eventually expires is what distinguishes it from other content restrictions. For example, it is not unusual for content destined for DSpace to come with permanent restrictions on use or access based on license-driven or other IP-based requirements that limit access to institutionally affiliated users. Restrictions such as these are imposed and managed using standard administrative tools in DSpace, typically by attaching specific policies to Items or Collections, Bitstreams, etc. The embargo functionally introduced in 1.6, however, includes tools to automate the imposition and removal of restrictions in managed timeframes.

Embargo Model and Life-Cycle

Functionally, the embargo system allows you to attach 'terms' to an item before it is placed into the repository, which express how the embargo should be applied. What do 'we mean by terms' here? They are really any expression that the system is capable of turning into (1) the time the embargo expires, and (2) a concrete set of access restrictions. Some examples:
"2020-09-12" - an absolute date (i.e. the date embargo will be lifted)"6 months" - a time relative to when the item is accessioned"forever" - an indefinite, or open-ended embargo"local only until 2015" - both a time and an exception (public has no access until 2015, local users OK immediately)"Nature Publishing Group standard" - look-up to a policy somewhere (typically 6 months)
These terms are 'interpreted' by the embargo system to yield a specific date on which the embargo can be removed or 'lifted', and a specific set of access policies. Obviously, some terms are easier to interpret than others (the absolute date really requires none at all), and the 'default' embargo logic understands only the most basic terms (the first and third examples above). But as we will see below, the embargo system provides you with the ability to add in your own 'interpreters' to cope with any terms expressions you wish to have. This date that is the result of the interpretation is stored with the item and the embargo system detects when that date has passed, and removes the embargo ("lifts it"), so the item bitstreams become available. Here is a more detailed life-cycle for an embargoed item:

1. **Terms Assignment.** The first step in placing an embargo on an item is to attach (assign) 'terms' to it. If these terms are missing, no embargo will be imposed. As we will see below, terms are carried in a configurable DSpace metadata field, so assigning terms just means assigning a value to a metadata field. This can be done in a web submission user interface form, in a SWORD deposit package, a batch import, etc. - anywhere metadata is passed to DSpace. The terms are not immediately acted upon, and may be revised, corrected, removed, etc, up until the next stage of the life-cycle. Thus a submitter could enter one value, and a collection editor replace it, and only the last value will be used. Since metadata fields are multivalued, theoretically there can be multiple terms values, but in the default implementation only one is recognized.
2. **Terms interpretation/imposition.** In DSpace terminology, when an item has exited the last of any workflow steps (or if none have been defined for it), it is said to be 'installed' into the repository. At this precise time, the 'interpretation' of the terms occurs, and a computed 'lift date' is assigned, which like the terms is recorded in a configurable metadata field. It is important to understand that this interpretation happens only once, (just like the installation), and cannot be revisited later. Thus, although an administrator can assign a new value to the metadata field holding the terms after the item has been installed, this will have no effect on the embargo, whose 'force' now resides entirely in the 'lift date' value. For this reason, you cannot embargo content already in your repository (at least using standard tools). The other action taken at installation time is the actual imposition of the embargo. The default behavior here is simply to remove the read policies on all the bundles and bitstreams except for the "LICENSE" or "METADATA" bundles. See the section on *Extending Embargo Functionality* for how to alter this behavior. Also note

that since these policy changes occur before installation, there is no time during which embargoed content is 'exposed' (accessible by non-administrators). The terms interpretation and imposition together are called 'setting' the embargo, and the component that performs them both is called the embargo 'setter'.

3. **Embargo Period.** After an embargoed item has been installed, the policy restrictions remain in effect until removed. This is not an automatic process, however: a 'lifter' must be run periodically to look for items whose 'lift date' is past. Note that this means the effective removal of an embargo is **not** the lift date, but the earliest date after the lift date that the lifter is run. Typically, a nightly cron-scheduled invocation of the lifter is more than adequate, given the granularity of embargo terms. Also note that during the embargo period, all metadata of the item remains visible. This default behavior can be changed. One final point to note is that the 'lift date', although it was computed and assigned during the previous stage, is in the end a regular metadata field. That means, if there are extraordinary circumstances that require an administrator (or collection editor , anyone with edit permissions on metadata) to change the lift date, they can do so. Thus, they can 'revise' the lift date without reference to the original terms. This date will be checked the next time the 'lifter' is run. One could immediately lift the embargo by setting the lift date to the current day, or change it to 'forever' to indefinitely postpone lifting.
4. **Embargo Lift.** When the lifter discovers an item whose lift date is in the past, it removes (lifts) the embargo. The default behavior of the lifter is to add the resource policies *that would have been added* had the embargo not been imposed. That is, it replicates the standard DSpace behavior, in which an item inherits it's policies from its owning collection. As with all other parts of the embargo system, you may replace or extend the default behavior of the lifter (see section V. below). You may wish, e.g. to send an email to an administrator or other interested parties, when an embargoed item becomes available.
5. **Post Embargo.** After the embargo has been lifted, the item ceases to respond to any of the embargo life-cycle events. The values of the metadata fields reflect essentially historical or provenance values. With the exception of the additional metadata fields, they are indistinguishable from items that were never subject to embargo.

More Embargo Details

More details on Embargo configuration, including specific examples can be found in the Embargo section of the documentation.

DSpace Services Framework

1 Architectural Overview
    1.1 DSpace Kernel
        1.1.1 Kernel registration
    1.2 Service Manager
2 Basic Usage
    2.1 Standalone Applications
    2.2 Application Frameworks (Spring, Guice, etc.)
    2.3 Web Applications
3 Providers and Plugins
    3.1 Activators
    3.2 Provider Stacks
4 Core Services
    4.1 Caching Service
    4.2 Configuration Service
    4.3 EventService
    4.4 RequestService
    4.5 SessionService
5 Examples
    5.1 Configuring Event Listeners
6 Tutorials

The DSpace Services Framework is a backporting of the DSpace 2.0 Development Group's work in creating a reasonable and abstractable "Core Services" layer for DSpace components to operate within. The Services Framework represents a "best practice" for new DSpace architecture and implementation of extensions to the DSpace application. DSpace Services are best described as a "Simple Registry" where plugins can be "looked up" or located. The DS2 (DSpace 2.0) core services are the main services that make up a DS2 system. These includes services for things like user and permissions management and storage and caching. These services can be used by any developer writing DS2 plugins (e.g. statistics), providers (e.g. authentication), or user interfaces.

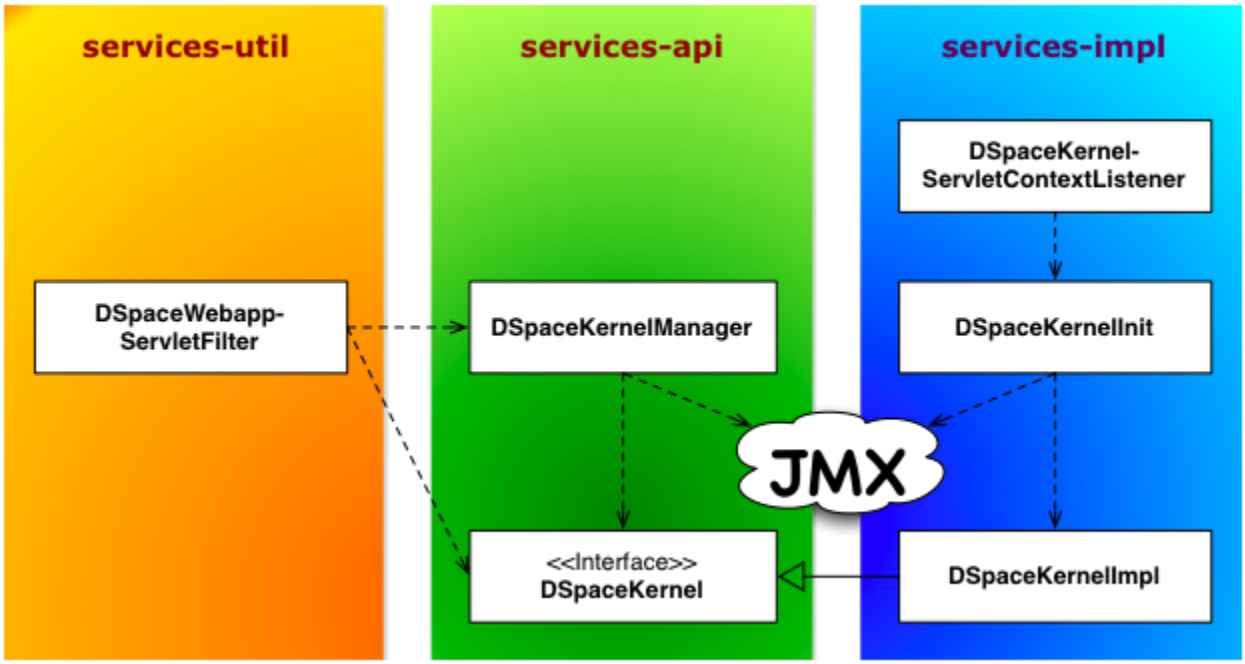
Architectural Overview

DSpace Kernel

The DSpace Kernel manages the start up and access services in the DSpace Services framework. It is meant to allow for a simple way to control the core parts of DSpace and allow for flexible ways to startup the kernel. For example, the kernel can be run inside a single webapp along with a frontend UI or it can be started as part of the servlet container so that multiple webapps can use a single kernel (this increases speed and efficiency). The kernel is also designed to happily allow multiple kernels to run in a single servlet container using identifier keys.

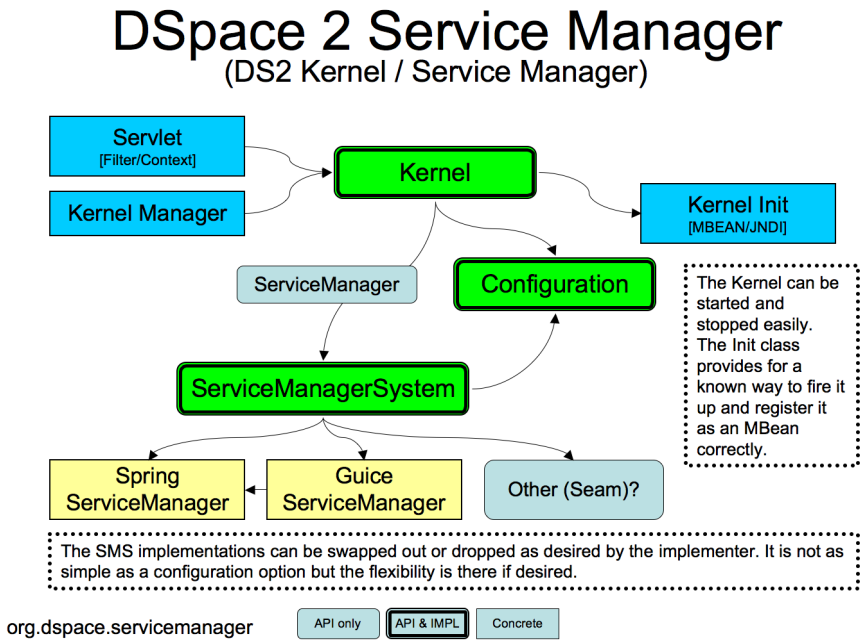
Kernel registration

The kernel will automatically register itself as an MBean when it starts up so that it can be managed via JMX. It allows startup and shutdown and provides direct access to the ServiceManager and the ConfigurationService. All the other core services can be retrieved from the ServiceManager by their APIs.



Service Manager

The ServiceManager abstracts the concepts of service lookups and lifecycle control. It also manages the configuration of services by allowing properties to be pushed into the services as they start up (mostly from the ConfigurationService). The ServiceManagerSystem abstraction allows the DSpace ServiceManager to use different systems to manage its services. The current implementations include Spring and Guice. This allows DSpace 2 to have very little service management code but still be flexible and not tied to specific technology. Developers who are comfortable with those technologies can consume the services from a parent Spring ApplicationContext or a parent Guice Module. The abstraction also means that we can replace Spring/Guice or add other dependency injection systems later without requiring developers to change their code. The interface provides simple methods for looking up services by interface type for developers who do not want to have to use or learn a dependency injection system or are using one which is not currently supported.



The DS2 kernel is compact so it can be completely started up in a unit test (technically integration test) environment. (This is how we test the kernel and core services currently). This allows developers to execute code against a fully functional kernel while developing and then deploy their code with high confidence.

Basic Usage

To use the Framework you must begin by instantiating and starting a DSpaceKernel. The kernel will give you references to the ServiceManager and the ConfigurationService. The ServiceManager can be used to get references to other services and to register services which are not part of the core set.

Access to the kernel is provided via the Kernel Manager through the DSpace object, which will locate the kernel object and allow it to be used.

Standalone Applications

For standalone applications, access to the kernel is provided via the Kernel Manager and the DSpace object which will locate the kernel object and allow it to be used.

```
/* Instantiate the Utility Class */
DSpace dspace = new DSpace();


/* Access get the Service Manager by convenience method */
ServiceManager manager = dspace.getServiceManager();


/* Or access by convenience method for core services */
EventService service = dspace.getEventService();
```

The DSpace launcher (

```
bin/dspace
```

) initializes a kernel before dispatching to the selected command.

Application Frameworks (Spring, Guice, etc.)

Similar to Standalone Applications, but you can use your framework to instantiate an `org.dspace.utils.DSpace` object.

```
<bean id="dspace" class="org.dspace.utils.DSpace"/>
```

Web Applications

In web applications, the kernel can be started and accessed through the use of Servlet Filter/ContextListeners which are provided as part of the DSpace 2 utilities. Developers don't need to understand what is going on behind the scenes and can simply write their applications and package them as webapps and take advantage of the services which are offered by DSpace 2.

Providers and Plugins

For developers (how we are trying to make your lives easier): The DS2 ServiceManager supports a plugin/provider system which is runtime hot-swappable. The implementor can register any service/provider bean or class with the DS2 kernel ServiceManager. The ServiceManager will manage the lifecycle of beans (if desired) and will instantiate and manage the lifecycle of any classes it is given. This can be done at any time and does not have to be done during Kernel startup. This allows providers to be swapped out at runtime without disrupting the service if desired. The goal of this system is to allow DS2 to be extended without requiring any changes to the core codebase or a rebuild of the code code.

Activators

Developers can provide an activator to allow the system to startup their service or provider. It is a simple interface with 2 methods which are called by the ServiceManager to startup the provider(s) and later to shut them down. These simply allow a developer to run some arbitrary code in order to create and register services if desired. It is the method provided to add plugins directly to the system via configuration as the activators are just listed in the configuration file and the system starts them up in the order it finds them.

Provider Stacks

Utilities are provided to assist with stacking and ordering providers. Ordering is handled via a priority number such that 1 is the highest priority and something like 10 would be lower. 0 indicates that priority is not important for this service and can be used to ensure the provider is placed at or near the end without having to set some arbitrarily high number.

Core Services

The core services are all behind APIs so that they can be reimplemented without affecting developers who are using the services. Most of the services have plugin/provider points so that customizations can be added into the system without touching the core services code. For example, let's say a deployer has a specialized authentication system and wants to manage the authentication calls which come into the system. The implementor can simply implement an AuthenticationProvider and then register it with the DS2 kernel's ServiceManager. This can be done at any time and does not have to be done during Kernel startup. This allows providers to be swapped out at runtime without disrupting the DS2 service if desired. It can also speed up development by allowing quick hot redeploys of code during development.

Caching Service

Provides for a centralized way to handle caching in the system and thus a single point for configuration and control over all caches in the system. Provider and plugin developers are strongly encouraged to use this rather than implementing their own caching. The caching service has the concept of scopes so even storing data in maps or lists is discouraged unless there are good reasons to do so.

Configuration Service

The ConfigurationService controls the external and internal configuration of DSpace 2. It reads Properties files when the kernel starts up and merges them with any dynamic configuration data which is available from the services. This service allows settings to be updated as the system is running, and also defines listeners which allow services to know when their configuration settings have changed and take action if desired. It is the central point to access and manage all the configuration settings in DSpace 2.

Manages the configuration of the DSpace 2 system. Can be used to manage configuration for providers and plugins also.

EventService

Handles events and provides access to listeners for consumption of events.

RequestService

In DS2 a request is an atomic transaction in the system. It is likely to be an HTTP request in many cases but it does not have to be. This service provides the core services with a way to manage atomic transactions so that when a request comes in which requires multiple things to happen they can either all succeed or all fail without each service attempting to manage this independently. In a nutshell this simply allows identification of the current request and the ability to discover if it succeeded or failed when it ends. Nothing in the system will enforce usage of the service, but we encourage developers who are interacting with the system to make use of this service so they know if the request they are participating in with has succeeded or failed and can take appropriate actions.

SessionService

In DS2 a session is like an HttpSession (and generally is actually one) so this service is here to allow developers to find information about the current session and to access information in it. The session identifies the current user (if authenticated) so it also serves as a way to track user sessions. Since we use HttpSession directly it is easy to mirror sessions across multiple servers in order to allow for no-interruption failover for users when servers go offline.

Examples

Configuring Event Listeners

Event Listeners can be created by overriding the the EventListener interface:

In Spring:

```
<?xml version="1.0" encoding="UTF-8"?>
<beans>

    <bean id="dspace" class="org.dspace.utils.DSpace"/>

    <bean id="dspace.eventService"
          factory-bean="dspace"
          factory-method="getEventService"/>

    <bean class="org.my.EventListener">
         <property name="eventService" >
                   <ref bean="dspace.eventService"/>
            </property>
    </bean>
</beans>
```

(org.my.EventListener will need to register itself with the EventService, for which it is passed a reference to that service via the eventService property.)

or in Java:

```
DSpace dspace = new DSpace();

EventService eventService = dspace.getEventService();

EventListener listener = new org.my.EventListener();
eventService.registerEventListener(listener);
```

(This registers the listener externally – the listener code assumes it is registered.)

Tutorials

Several tutorials on Spring / DSpace Services are available:

- DSpace Spring Services Tutorial
- The TAO of DSpace Services

Storage Layer

In this section, we explain the storage layer: the database structure, maintenance, and the bitstream store and configurations. The bitstream store, also known as assetstore or bitstore, holds the uploaded, ingested, or generated files (documents, images, audio, video, datasets, ...), where as the database holds all of the metadata, organization, and permissions of content.

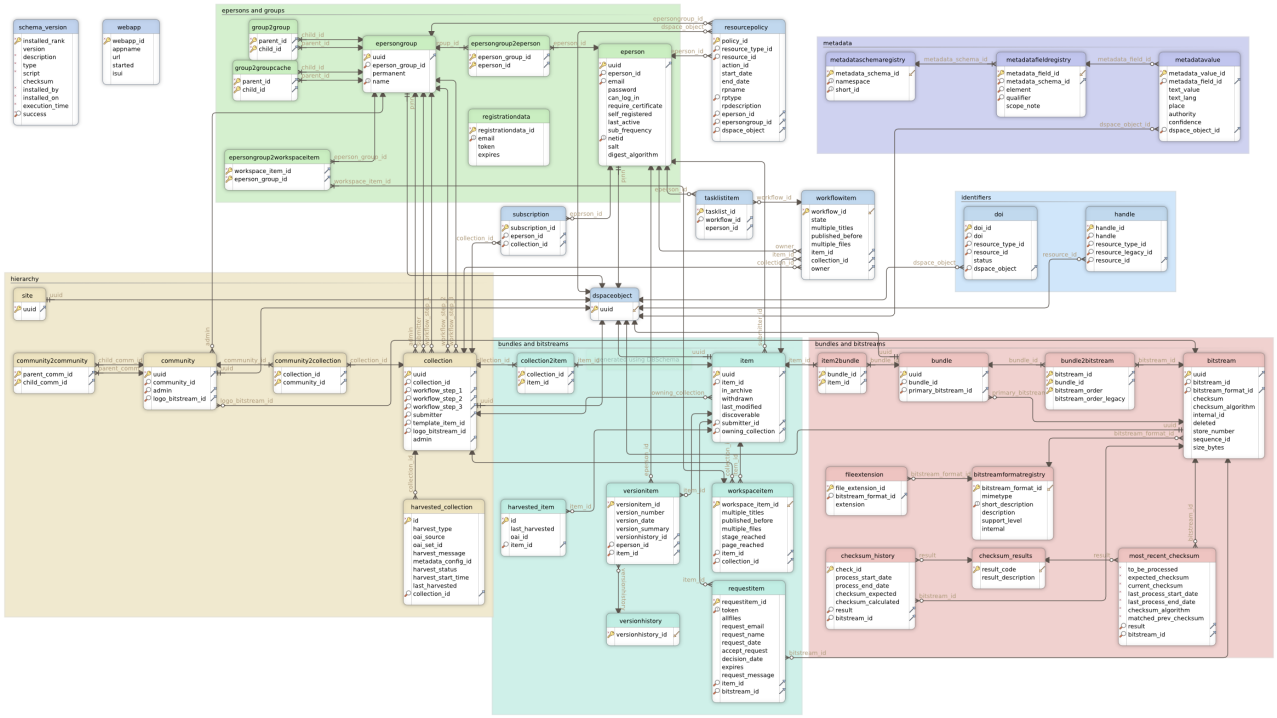
1 RDBMS / Database Structure
- 1.1 Maintenance and Backup
- 1.2 Configuring the RDBMS Component
- 1.3 Custom RDBMS tables, columns or views

2 Bitstream Store
- 2.1 Cleanup
- 2.2 Backup
- 2.3 Configuring the Bitstream Store
  - 2.3.1 Configuring Traditional Storage
  - 2.3.2 Configuring Amazon S3 Storage
- 2.4 Migrate BitStores

RDBMS / Database Structure

DSpace uses a relational database to store all information about the organization of content, metadata about the content, information about e-people and authorization, and the state of currently-running workflows.

DSpace 6 database schema (Postgres). Right-click the image and choose "Save as" to save in full resolution. Instructions on updating this schema diagram are in How to update database schema diagram.



- DSpace uses FlywayDB to perform automated database initialization and upgrades. Flyway's role is to initialize the database tables (and default content) prior to Hibernate initialization.
  - The `org.dspace.storage.rdbms.DatabaseUtils` class manages all Flyway API calls, and executes the SQL migrations under the `org.dspace.storage.rdbms.sqlmigration` package and the Java migrations under the `org.dspace.storage.rdbms.migration` package.
  - Once all database migrations have run, a series of Flyway Callbacks are triggered to initialize the (empty) database with required default content. For example, callbacks exist for adding default DSpace Groups (`GroupServiceInitializer`), default Metadata & Format Registries (`DatabaseRegistryUpdater`), and the default Site object (`SiteServiceInitializer`). All Callbacks are under the `org.dspace.storage.rdbms` package.
  - While Flyway is automatically initialized and executed during startup, various Database Utilities are also available on the command line. These utilities allow you to manually trigger database upgrades or check the status of your database.
- DSpace uses Hibernate ORM as the object relational mapping layer between the DSpace database and the DSpace code.
  - The main Hibernate configuration can be found at `[dspace]/config/hibernate.cfg.xml`
  - Hibernate initialization is triggered via Spring (beans) defined `[dspace]/config/spring/api/core-hibernate.xml`. This Spring configuration pulls in some settings from DSpace Configuration, namely all Database (db.*) settings defined there.

- All DSpace Object Classes provide a DAO (Data Access Object) implementation class that extends a GenericDAO interface defined in o `rg.dspace.core.GenericDAO` class. The default (abstract) implementation is in `org.dspace.core.AbstractHibernateDAO` cla ss.
- The DSpace Context object (`org.dspace.core.Context`) provides access to the configured `org.dspace.core.DBConnection` (D atabase Connection), which is HibernateDBConnection by default. The `org.dspace.core.HibernateDBConnection` class provides access the the Hibernate Session interface (org.hibernate.Session) and its Transactions.
  - Each Hibernate Session opens a single database connection when it is created, and holds onto it until the Session is closed. A Session may consist of one or more Transactions. Sessions are NOT thread-safe (so individual objects cannot be shared between threads).
  - Hibernate will intelligently cache objects in the current Hibernate Session (on object access), allowing for optimized performance.
  - DSpace provides methods on the Context object to specifically remove (`Context.uncacheEntity()`) or reload (`Context. reloadEntity()`) objects within Hibernate's Session cache.
  - DSpace also provides special Context object "modes" to optimize Hibernate performance for read-only access (`Mode. READ_ONLY`) or batch processing (`Mode.BATCH_EDIT`). These modes can be specified when constructing a new Context object.

Most of the functionality that DSpace uses can be offered by any standard SQL database that supports transactions. However, at this time, DSpace only provides Flyway migration scripts for PostgreSQL and Oracle (and has only been tested with those database backends). Additional database backends should be possible, but would minimally require creating custom Flyway migration scripts for that database backend.

Maintenance and Backup

When using PostgreSQL, it's a good idea to perform regular 'vacuuming' of the database to optimize performance. By default, PostgreSQL performs autom atic vacuuming on your behalf. However, if you have this feature disabled, then we recommend scheduling the `vacuumdb` command to run on a regular basis.

```
# clean up the database nightly
40 2 * * * /usr/local/pgsql/bin/vacuumdb --analyze dspace > /dev/null 2>&1
```

Backups: The DSpace database can be backed up and restored using usual PostgreSQL Backup and Restore methods, for example with `pg_dump` and `p sql`. However when restoring a database, you will need to perform these additional steps:

- After restoring a backup, you will need to reset the primary key generation sequences so that they do not produce already-used primary keys. Do this by executing the SQL in `[dspace]/etc/postgres/update-sequences.sql`, for example with:

  ```
  psql -U dspace -f [dspace]/etc/update-sequences.sql
  ```

Configuring the RDBMS Component

The database manager is configured with the following properties in `dspace.cfg`:

`db.url`	The JDBC URL to use for accessing the database. This should not point to a connection pool, since DSpace already implements a connection pool.
`db.driver`	JDBC driver class name. Since presently, DSpace uses PostgreSQL-specific features, this should be *org.postgresql.Driver*.
`db. username`	Username to use when accessing the database.
`db. password`	Corresponding password ot use when accessing the database.

Custom RDBMS tables, columns or views

When at all possible, we recommend creating custom database tables or views within a *separate schema* from the DSpace database tables. Since the DSpace database is initialized and upgraded automatically using Flyway DB, the upgrade process may stumble or throw errors if you've directly modified the DSpace database schema, views or tables. Flyway itself assumes it has full control over the DSpace database schema, and it is not "smart" enough to know what to do when it encounters a locally customized database.

That being said, if you absolutely need to customize your database tables, columns or views, it is possible to create *custom Flyway migration scripts* , which should make your customizations easier to manage in future upgrades. (Keep in mind though, that you may still need to maintain/update your custom Flyway migration scripts if they ever conflict directly with future DSpace database changes. The only way to "future proof" your local database changes is to try and make them as independent as possible, and avoid directly modifying the DSpace database schema as much as possible.)

If you wish to add custom Flyway migrations, they may be added to the following locations:

- Custom Flyway SQL migrations may be added anywhere under the `org.dspace.storage.rdbms.sqlmigration` package (e.g. `[src]/dspace-api/src/main/resources/org/dspace/storage/rdbms/sqlmigration` or subdirectories)
- Custom Flyway Java migrations may be added anywhere under the `org.dspace.storage.rdbms.migration` package (e.g. `[src]/dspace-api/src/main/java/org/dspace/storage/rdbms/migration/` or subdirectories)
- Additionally, for backwards support, custom SQL migrations may also be placed in the `[dspace]/etc/[db-type]/` folder (e.g. `[dspace]/etc/postgres/` for a PostgreSQL specific migration script)

Adding Flyway migrations to any of the above location will cause Flyway to auto-discover the migration. It will be run in the order in which it is named. Our DSpace Flyway script naming convention follows Flyway best practices and is as follows:

- SQL script names: `V[version]_[date]__[description].sql`
  - E.g. `V5.0_2014.09.26__DS-1582_Metadata_For_All_Objects.sql` is a SQL migration script created for DSpace 5.x (`V5.0`) on Sept 26, 2014 (`2014_09_24`). Its purpose was to fulfill the needs of ticket DS-1582, which was to migrate the database in order to support adding metadata on all objects.
  - More examples can be found under the `org.dspace.storage.rdbms.sqlmigration` package
- Java migration script naming convention: `V[version]_[date]__[description].java`
  - E.g. `V5_0_2014_09_25__DS_1582_Metadata_For_All_Objects_drop_constraint.java` is a Java migration created for DSpace 5.x (`V5_0`) on Sept 25, 2014 (`2014_09_25`). Its purpose was to fulfill the needs of ticket DS-1582, specifically to drop a few constraints.
  - More examples can be found under the `org.dspace.storage.rdbms.migration` package
- Flyway will execute migrations in order, based on their Version and Date. So, V1.x (or V1_x) scripts are executed first, followed by V3.0 (or V3_0), etc. If two migrations have the same version number, the date is used to determine ordering (earlier dates are run first).

Bitstream Store

DSpace offers two means for storing content.

1. Storage in a mounted file system on the server (DSBitStore)
2. Storage using AWS S3 (Simple Storage Service), (S3BitStore).

Both are achieved using a simple, lightweight BitStore API, providing actions of Get, Put, About, Remove. Higher level operations include Store, Register, Checksum, Retrieve, Cleanup, Clone, Migrate. Digital assets are stored on the bitstores by being transferred to the bitstore when it is uploaded or ingested. The exception to this is for "registered" objects, that the assets are put onto the filesystem ahead of time out-of-band, and during ingest, it just maps the database to know where the object already resides. The storage interface is such that additional storage implementations (i.e. other cloud storage providers) can be added with minimal difficulty.

DSBitStore stores content on a path on the filesystem. This could be locally attached normal filesystem, a mounted drive, or a mounted networked filesystem, it will all be treated as a local filesystem. All DSpace needs to be configured with for a filesystem, is the filesystem path, i.e. /dspace/assetstore, /opt/data/assetstore. The DSBitStore uses a "Directory Scatter" method of storing an asset within 3 levels of subfolders, to minimize any single folder having too many objects for normal filesystem performance.

S3BitStore uses Amazon Web Services S3 (Simple Storage Service) to offer limitless cloud storage into a bucket, and each distinct asset will have a unique key. S3 is a commercial service (costs money), but is available at low price point, and is fully managed, content is automatically replicated, 99.999999999% object durability, integrity checked. Since S3 operates within the AWS network, using other AWS services, such virtual server on EC2 will provide lower network latency than local "on premises" servers. Additionally there could be some in-bound / out-bound bandwidth costs associated with DSpace application server outside of the AWS network communicating with S3, compared to AWS-internal EC2 servers. S3 has a checksum computing operation, in which the S3 service can return the checksum from the storage service, without having to shuttle the bits from S3, to your application server, and then computing the checksum. S3BitStore requires an S3 bucketName, accessKey, secretKey, and optionally specifying the AWS region, or a subfolder within the bucket.

There can be multiple bitstream stores. Each of these bitstream stores can be traditional storage or S3 storage. This means that the potential storage of a DSpace system is not bound by the maximum size of a single disk or file system and also that filesystem and S3storage can be combined in one DSpace installation. Both filesystem and S3 storage are specified by configuration. Also see Configuring the Bitstream Store below.

Stores are numbered, starting with zero, then counting upwards. Each bitstream entry in the database has a store number, used to retrieve the bitstream when required. An example of having multiple asset stores configured is that assetstore0 is /dspace/assetstore, when the filesystem gets nearly full, you could then configure a second filesystem path assetstore1 at /data/assetstore1, later, if you wanted to use S3 for storage, assetstore2 could be s3://dspace-assetstore-xyz. In this example various bitstreams (database objects) refer to different assetstore for where the files reside. It is typically simplest to just have a single assetstore configured, and all assets reside in that one. If policy dictated, infrequently used masters could be moved to slower/cheaper disk, where as access copies are on the fastest storage. This could be accomplished through migrating assets to different stores.

Bitstreams also have an 38-digit internal ID, different from the primary key ID of the bitstream table row. This is not visible or used outside of the bitstream storage manager. It is used to determine the exact location (relative to the relevant store directory) that the bitstream is stored in traditional storage. The first three pairs of digits are the directory path that the bitstream is stored under. The bitstream is stored in a file with the internal ID as the filename.

For example, a bitstream with the internal ID *12345678901234567890123456789012345678* is stored in the directory:

```
[dspace]/assetstore/12/34/56/12345678901234567890123456789012345678
```

The reasons for storing files this way are:

- Using a randomly-generated 38-digit number means that the 'number space' is less cluttered than simply using the primary keys, which are allocated sequentially and are thus close together. This means that the bitstreams in the store are distributed around the directory structure, improving access efficiency.
- The internal ID is used as the filename partly to avoid requiring an extra lookup of the filename of the bitstream, and partly because bitstreams may be received from a variety of operating systems. The original name of a bitstream may be an illegal UNIX filename.

- When storing a bitstream, the *BitstreamStorageService* DOES set the following fields in the corresponding database table row:
  - *bitstream_id*
  - *size*
  - *checksum*
  - *checksum_algorithm*
  - *internal_id*
  - *deleted*
  - *store_number*
- The remaining fields are the responsibility of the *Bitstream* content management API class.

The bitstream storage manager is fully transaction-safe. In order to implement transaction-safety, the following algorithm is used to store bitstreams:

1. A database connection is created, separately from the currently active connection in the current DSpace context.
2. An unique internal identifier (separate from the database primary key) is generated.
3. The bitstream DB table row is created using this new connection, with the *deleted* column set to *true*.
4. The new connection is _commit_ted, so the 'deleted' bitstream row is written to the database
5. The bitstream itself is stored in a file in the configured 'asset store directory', with a directory path and filename derived from the internal ID
6. The *deleted* flag in the bitstream row is set to *false*. This will occur (or not) as part of the current DSpace *Context*.

This means that should anything go wrong before, during or after the bitstream storage, only one of the following can be true:

- No bitstream table row was created, and no file was stored
- A bitstream table row with *deleted=true* was created, no file was stored
- A bitstream table row with *deleted=true* was created, and a file was stored
  None of these affect the integrity of the data in the database or bitstream store.

Similarly, when a bitstream is deleted for some reason, its *deleted* flag is set to true as part of the overall transaction, and the corresponding file in storage is *not* deleted.

Cleanup

The above techniques mean that the bitstream storage manager is transaction-safe. Over time, the bitstream database table and file store may contain a number of 'deleted' bitstreams. The *cleanup* method of *BitstreamStorageService* goes through these deleted rows, and actually deletes them along with any corresponding files left in the storage. It only removes 'deleted' bitstreams that are more than one hour old, just in case cleanup is happening in the middle of a storage operation.

This cleanup can be invoked from the command line via the *cleanup* command, which can in turn be easily executed from a shell on the server machine using `[dspace]/bin/dspace cleanup`. You might like to have this run regularly by *cron*, though since DSpace is read-lots, write-not-so-much it doesn't need to be run very often.

```
# Clean up any deleted files from local storage on first of the month at 2:40am
40 2 1 * * [dspace]/bin/dspace cleanup > /dev/null 2>&1
```

Backup

The bitstreams (files) in traditional storage may be backed up very easily by simply 'tarring' or 'zipping' the `[dspace]/assetstore/` directory (or whichever directory is configured in *dspace.cfg*). Restoring is as simple as extracting the backed-up compressed file in the appropriate location.

It is important to note that since the bitstream storage manager holds the bitstreams in storage, and information about them in the database, that a database backup and a backup of the files in the bitstream store must be made at the same time; the bitstream data in the database must correspond to the stored files.

Of course, it isn't really ideal to 'freeze' the system while backing up to ensure that the database and files match up. Since DSpace uses the bitstream data in the database as the authoritative record, it's best to back up the database before the files. This is because it's better to have a bitstream in storage but not the database (effectively non-existent to DSpace) than a bitstream record in the database but not storage, since people would be able to find the bitstream but not actually get the contents.

With DSpace 1.7 and above, there is also the option to backup both files and metadata via the AIP Backup and Restore feature.

Configuring the Bitstream Store

Changed in DSpace 7.4

While the old **bitstore.xml** file (defined at `[dspace]/config/spring/api/bitstore.xml`) still exists in DSpace 7.4, a new configuration file has been added for configuring the Bitstream Store: `[dspace]/config/modules/assetstore.cfg`

If you have previously configured **bitstore.xml**, and only have a single assetstore, we recommend resetting **bitstore.xml** to the default configuration, and use the new **assetstore.cfg** file for configuring your Bitstream Store. This is of course very general advice, and your specific situation may require more care, particularly if you have more than one Bitstream Store (see below, and this likely doesn't apply to you, but it could).

BitStores (aka assetstores) are configured with `[dspace]/config/modules/assetstore.cfg`

Configuring Traditional Storage

By default, DSpace uses a traditional filesystem bitstore of `[dspace]/assetstore/`

To configure traditional filesystem bitstore, as a specific directory, configure the bitstore like this:

```
#---------------------------------------------------------------#
#-----------------STORAGE CONFIGURATIONS------------------------#
#---------------------------------------------------------------#
# Configuration properties used by the bitstore.xml config file #
#                                                               #
#---------------------------------------------------------------#

# assetstore.dir, look at DSPACE/config/spring/api/bitstore.xml for more options
assetstore.dir = ${dspace.dir}/assetstore

# Configures the primary store to be local or S3.
# This value will be used as `incoming` default store inside the `bitstore.xml`
# Possible values are:
#     - 0: to use the `localStore`;
#     - 1: to use the `s3Store`.
# If you want to add additional assetstores, they must be added to that bitstore.xml
# and new values should be provided as key-value pairs in the `stores` map of the
# `bitstore.xml` configuration.
assetstore.index.primary = 0
```

This would configure store number 0 named localStore, which is a `DSBitStore` (filesystem), at the filesystem path of `${dspace.dir}/assetstore` (i.e. `[dspace]/assetstore/`)

It is also possible to use multiple local filesystems. The following example is specific to the older **bitstore.xml** configuration, and it should still work, but is un-tested with DSpace 7.4. In the example below, key #0 is localStore at `${dspace.dir}/assetstore`, and key #1 is localStore2 at `/data/assetstore2`. Note that incoming is set to store "1", which in this case refers to localStore2. That means that any new files (bitstreams) uploaded to DSpace will be stored in localStore2, but some existing bitstreams may still exist in localStore.

```
<bean name="org.dspace.storage.bitstore.BitstreamStorageService" class="org.dspace.storage.bitstore.
BitstreamStorageServiceImpl">
    <property name="incoming" value="1"/>
    <property name="stores">
        <map>
            <entry key="0" value-ref="localStore"/>
            <entry key="1" value-ref="localStore2"/>
        </map>
    </property>
</bean>
<bean name="localStore" class="org.dspace.storage.bitstore.DSBitStoreService" scope="singleton">
    <property name="baseDir" value="${dspace.dir}/assetstore"/>
</bean>
<bean name="localStore2" class="org.dspace.storage.bitstore.DSBitStoreService" scope="singleton">
    <property name="baseDir" value="/data/assetstore2"/>
</bean>
```

Configuring Amazon S3 Storage

To use Amazon S3 as a bitstore, in **assetstore.cfg** point **assetstore.index.primary** to 1, set **assetstore.s3.enabled** to true, and configure it with **awsAccessKey**, **awsSecretKey**, and **bucketName**. NOTE: Before you can specify these settings, you obviously will have to create an account in the Amazon AWS console, and create an IAM user with credentials and privileges to an existing S3 bucket. You can also use an IAM Role, which would allow your Amazon EC2 instance to talk directly to your S3 bucket without using any credentials. If you do configure an IAM Role with AWS, you'll still need to provide values for **awsAccessKey** and **awsSecretKey**, but they are un-used by DSpace, and can be anything at all. In the example below, we assume you are using an IAM Role, and are providing the text "use-the-role-please" as the value for these two keys. This is of course not a valid AWS key, for either example.

```
# Configures the primary store to be local or S3.
# This value will be used as `incoming` default store inside the `bitstore.xml`
# Possible values are:
#     - 0: to use the `localStore`;
#     - 1: to use the `s3Store`.
# If you want to add additional assetstores, they must be added to that bitstore.xml
# and new values should be provided as key-value pairs in the `stores` map of the
# `bitstore.xml` configuration.
```

```
assetstore.index.primary = 1

#------------------------------------------------------------#
#-------------- Amazon S3 Specific Configurations -----------#
#------------------------------------------------------------#
# The below configurations are only used if the primary storename
# is set to 's3Store' or the 's3Store' is configured as a secondary store
# in your bitstore.xml

# Enables or disables the store initialization during startup, without initialization the store won't work.
# if changed to true, a lazy initialization will be tried on next store usage, be careful an excecption could
be thrown
assetstore.s3.enabled = true

# For using a relative path (xx/xx/xx/xxx...) set to true, default it false
# When true: it splits the path into subfolders, each of these
# are 2-chars (2-bytes) length, the last is the filename and could have
# at max 3-chars (3-bytes).
# When false: is used the absolute path using full filename.
assetstore.s3.useRelativePath = false

# S3 bucket name to store assets in. If unspecified, by default DSpace will
# create a bucket based on the hostname of `dspace.ui.url` setting.
assetstore.s3.bucketName = your-bucket-name-goes-here

# Subfolder to organize assets within the bucket, in case this bucket
# is shared. Optional, default is root level of bucket
assetstore.s3.subfolder = your-optional-subfolder-goes-here

# Optional custom S3 endpoint URI. Leave this blank / commented to use the Amazon default
# (First available in 7.6.6)
# assetstore.s3.endpoint =

# please don't use root credentials in production but rely on the aws credentials default
# discovery mechanism to configure them (ENV VAR, EC2 Iam role, etc.)
# The preferred approach for security reason is to use the IAM user credentials, but isn't always possible.
# More information about credentials here: https://docs.aws.amazon.com/sdk-for-java/v1/developer-guide
/credentials.html
# More information about IAM usage here: https://docs.aws.amazon.com/sdk-for-java/v1/developer-guide/java-dg-
roles.html
assetstore.s3.awsAccessKey = use-the-role-please
assetstore.s3.awsSecretKey = use-the-role-please

# If the credentials are left empty,
# then this setting is ignored and the default AWS region will be used.
# NOTE: AWS is funny about regions, it's probably best to explicitly provide the one you intend to use,
# if you want to keep track of where your bitstreams are
assetstore.s3.awsRegionName = us-west-2

# The target throughput for transfer requests in Gbps. Higher value means more connections will be established
with S3.
# (First available in 7.6.6)
assetstore.s3.targetThroughputGbps = 10.0

# Sets the minimum part size for transfer parts. Decreasing the minimum part size causes multipart transfer to
be split
# into a larger number of smaller parts. (First available in 7.6.6)
assetstore.s3.minPartSizeBytes = 8388608

# Specifies the maximum number of S3 connections that should be established during a transfer.
# If not provided, it will be based on targetThroughputGbps
# (First available in 7.6.6)
assetstore.s3.maxConcurrency =

# The algorithm the S3 client will use to create a checksum when doing putObject.
# (First available in 7.6.6)
assetstore.s3.s3ChecksumAlgorithm = CRC32
```

The incoming property specifies which assetstore receives incoming assets (i.e. when new files are uploaded, they will be stored in the "incoming" assetstore). This defaults to store 0. NOTE: in the **assetstore.cfg** file, this setting is called **assetstore.index.primary**.

S3BitStore has parameters for awsAccessKey, awsSecretKey, bucketName, awsRegionName (optional), and subfolder (optional).

- `awsAccessKey` and `awsSecretKey` are created from the Amazon AWS console. You'll want to create an IAM user, and generate a Security Credential, which provides you the accessKey and secret. Since you need permission to use S3, you could give this IAM user a quick & dirty policy of AmazonS3FullAccess (for all S3 buckets that you own), or for finer grain controls, you can assign an IAM user to have certain permissions to certain resources, such as read/write to a specific subfolder within a specific s3 bucket.
- `bucketName` is a globally unique name that distinguishes your S3 bucket. It has to be unique among all other S3 users in the world.
- `awsRegionName` is a region in AWS where S3 will be stored. Default is US Eastern. Consider distance to primary users, and pricing when choosing the region.
- `subfolder` is a folder within the S3 bucket, where you could organize the assets to be in. If you wanted to re-use a bucket for multiple purposes (bucketname/assets vs bucketname/backups) or DSpace instances (bucketname/XYZDSpace or bucketname/ABCDSpace or bucketname /ABCDSpaceProduction).

Migrate BitStores

There is a command line migration tool to move all the assets within a bitstore, to another bitstore. `bin/dspace bitstore-migrate`

```
[dspace]/bin/dspace bitstore-migrate
usage: BitstoreMigrate
 -a,--source <arg>        Source assetstore store_number (to lose content). This is a number such as 0 or 1
 -b,--destination <arg>   Destination assetstore store_number (to gain content). This is a number such as 0 or
1.
 -d,--delete              Delete file from losing assetstore. (Default: Keep bitstream in old assetstore)
 -h,--help                Help
 -p,--print               Print out current assetstore information
 -s,--size <arg>          Batch commit size. (Default: 1, commit after each file transfer)

[dspace]/bin/dspace bitstore-migrate -p
store[0] == DSBitStore, which has 2 bitstreams.
store[1] == S3BitStore, which has 2 bitstreams.
Incoming assetstore is store[1]

[dspace]/bin/dspace bitstore-migrate -a 0 -b 1


[dspace]/bin/dspace bitstore-migrate -p
store[0] == DSBitStore, which has 0 bitstreams.
store[1] == S3BitStore, which has 4 bitstreams.
Incoming assetstore is store[1]
```

History

- Changes in 7.x
- Changes in Older Releases

Changes in 7.x

- Changes in DSpace 7.6.7
- Changes in DSpace 7.6.6
- Changes in DSpace 7.6.5
- Changes in DSpace 7.6.4
- Changes in DSpace 7.6.3
- Changes in DSpace 7.6.2
- Changes in DSpace 7.6.1
- Changes in DSpace 7.6
- Changes in DSpace 7.5
- Changes in DSpace 7.4
- Changes in DSpace 7.3
- Changes in DSpace 7.2
- Changes in DSpace 7.1.1
- Changes in DSpace 7.1
- Changes in DSpace 7.0

Changes in DSpace 7.6.7

- UI Changes: https://github.com/DSpace/dspace-angular/milestone/35?closed=1
- REST API Changes: https://github.com/DSpace/DSpace/milestone/83?closed=1
- REST Contract Changes: https://github.com/DSpace/RestContract/milestone/26?closed=1

Changes in DSpace 7.6.6

- UI Changes: https://github.com/DSpace/dspace-angular/milestone/34?closed=1
- REST API Changes: https://github.com/DSpace/DSpace/milestone/82?closed=1
- REST Contract Changes:]https://github.com/DSpace/RestContract/milestone/25?closed=1 *(No changes)*

Changes in DSpace 7.6.5

- UI Changes: https://github.com/DSpace/dspace-angular/milestone/31?closed=1
- REST API Changes: https://github.com/DSpace/DSpace/milestone/79?closed=1
- REST Contract Changes: https://github.com/DSpace/RestContract/milestone/22?closed=1 *(No changes)*

Changes in DSpace 7.6.4

- UI Changes: https://github.com/DSpace/dspace-angular/milestone/28?closed=1
- REST API Changes: https://github.com/DSpace/DSpace/milestone/75?closed=1
- REST Contract Changes: https://github.com/DSpace/RestContract/milestone/19?closed=1

Changes in DSpace 7.6.3

- UI Changes: https://github.com/DSpace/dspace-angular/milestone/26?closed=1
- REST API Changes: https://github.com/DSpace/DSpace/milestone/74?closed=1
- REST Contract Changes: https://github.com/DSpace/RestContract/milestone/16?closed=1

Changes in DSpace 7.6.2

- UI Changes: https://github.com/DSpace/dspace-angular/milestone/23?closed=1
- REST API Changes: https://github.com/DSpace/DSpace/milestone/71?closed=1

Changes in DSpace 7.6.1

- UI Changes: https://github.com/DSpace/dspace-angular/milestone/22?closed=1
- REST API Changes: https://github.com/DSpace/DSpace/milestone/70?closed=1

Changes in DSpace 7.6

- UI Changes: https://github.com/DSpace/dspace-angular/milestone/20?closed=1

- REST API Changes: https://github.com/DSpace/DSpace/milestone/68?closed=1

Changes in DSpace 7.5

- UI Changes: https://github.com/DSpace/dspace-angular/milestone/19?closed=1
- REST API Changes: https://github.com/DSpace/DSpace/milestone/67?closed=1

Changes in DSpace 7.4

- UI Changes: https://github.com/DSpace/dspace-angular/milestone/18?closed=1
- REST API Changes: https://github.com/DSpace/DSpace/milestone/66?closed=1

Changes in DSpace 7.3

- UI Changes: https://github.com/DSpace/dspace-angular/milestone/17?closed=1
- REST API Changes: https://github.com/DSpace/DSpace/milestone/65?closed=1

Changes in DSpace 7.2

- UI Changes: https://github.com/DSpace/dspace-angular/milestone/14?closed=1
- REST API Changes: https://github.com/DSpace/DSpace/milestone/37?closed=1

Changes in DSpace 7.1.1

- UI Changes: *(No changes, this was a backend only release. The frontend v7.1 is fully compatible with backend v7.1.1)*
- REST API Changes: *Only included a single commit to fix a severe log4j vulnerability by upgrading to log4j v2.15.0: Commit 8662d82*

Changes in DSpace 7.1

- UI Changes: https://github.com/DSpace/dspace-angular/milestone/12?closed=1
- REST API Changes: https://github.com/DSpace/DSpace/milestone/36?closed=1

Changes in DSpace 7.0

Changes in 7.0

- UI Changes: https://github.com/DSpace/dspace-angular/milestone/15?closed=1
- REST API Changes: https://github.com/DSpace/DSpace/milestone/21?closed=1

Changes in 7.0 Beta 5

- UI Changes: https://github.com/DSpace/dspace-angular/milestone/11?closed=1
- REST API Changes: https://github.com/DSpace/DSpace/milestone/35?closed=1

Changes in 7.0 Beta 4

- UI Changes: https://github.com/DSpace/dspace-angular/milestone/10?closed=1
- REST API Changes: https://github.com/DSpace/DSpace/milestone/33?closed=1

Changes in 7.0 Beta 3

- UI Changes: https://github.com/DSpace/dspace-angular/milestone/9?closed=1
- REST API Changes: https://github.com/DSpace/DSpace/milestone/34?closed=1

Changes in 7.0 Beta 2

- UI Changes: https://github.com/DSpace/dspace-angular/milestone/8?closed=1
- REST API Changes: https://github.com/DSpace/DSpace/milestone/38?closed=1

Changes in 7.0 Beta 1

- UI Changes: https://github.com/DSpace/dspace-angular/milestone/7?closed=1
- REST API Changes: https://github.com/DSpace/DSpace/milestone/32?closed=1

Changes in 7.0 Preview Release (Alpha)

- UI Changes: https://github.com/DSpace/dspace-angular/milestone/16?closed=1
- REST API Changes: https://github.com/DSpace/DSpace/milestone/39?closed=1

Changes in Older Releases

All historical changes from older releases of DSpace may be found in the online documentation for those older releases.

Learning DSpace

The DSpace Community Advisory Team (DCAT) is developing this user-facing guide to DSpace 7. All are welcome to participate.

Pages

- Community and Collection management
  - 'Assigning Roles' in Collections: How to Set Up Workflows and Grant Privileges
  - Collection Management
    - Create Collection
    - Delete Collection
    - Edit Collection
    - Export Collection
  - Community Management
    - Create a Community
    - Delete Community
    - Edit Community
- Content (Item) management
  - Add item
  - Delete item
  - Edit Item
    - Authorizations (Manage access to an item)
    - Collection Mapper
    - Edit Bitstream
    - Edit Metadata
    - Edit Relationship
    - Make an Item Private
    - Move an Item
    - Reinstate an item
    - Versioned Item
    - Withdraw an item
  - Embargo an item
  - Lease an item
- DSpace 7 Demo Quick Start
- Management sidebar
- Menus
- Registry management
  - Metadata Registry Management
- Request-a-copy
- Search - Advanced
- User management
  - Add or Manage an E-Person
  - Create or manage a user group

Videos

https://new.d2t.co/knowledge-center

For repository/DSpace administrators

Community and Collection management

Content (Item) management

User management

Registry management

Draft page not yet transferred -

- Administrative search

Pages not yet created; please feel free to add, and start writing, these pages for topics requested by the DCAT DSpace Community Advisory Team:

- Perform curation tasks
- Manage active workflow tasks

For submitters

Add item

Embargo an item

Lease an item

For DSpace users

Request-a-copy

Editing Guidelines

- Crop images before loading.
- Add a border around each image.
- Add an image as "large" if text is small.
- Avoid tables, if possible.

Transfer images from Google Doc to the Wiki

1. Download the Google Doc as an HTML file. This will include an images folder.
2. Crop the images, if needed. (GIMP, free open source; cropping instructions)
3. Upload those images to the Wiki storage area.

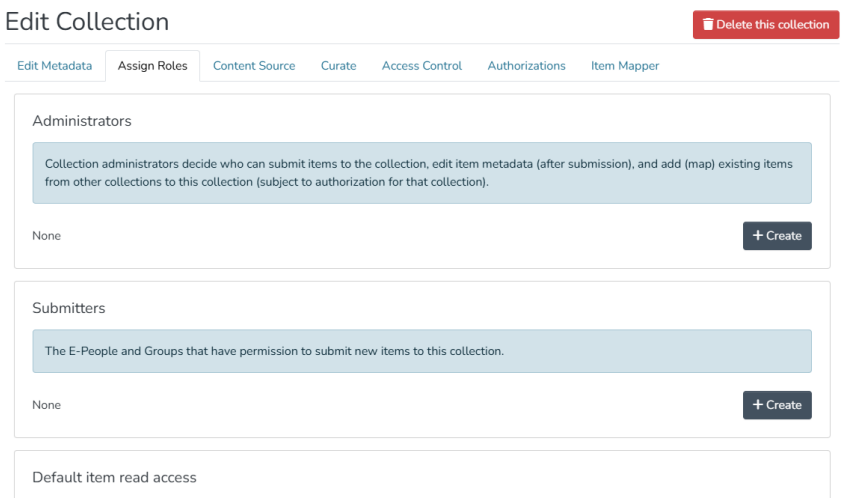
Community and Collection management

Documentation for repository managers.

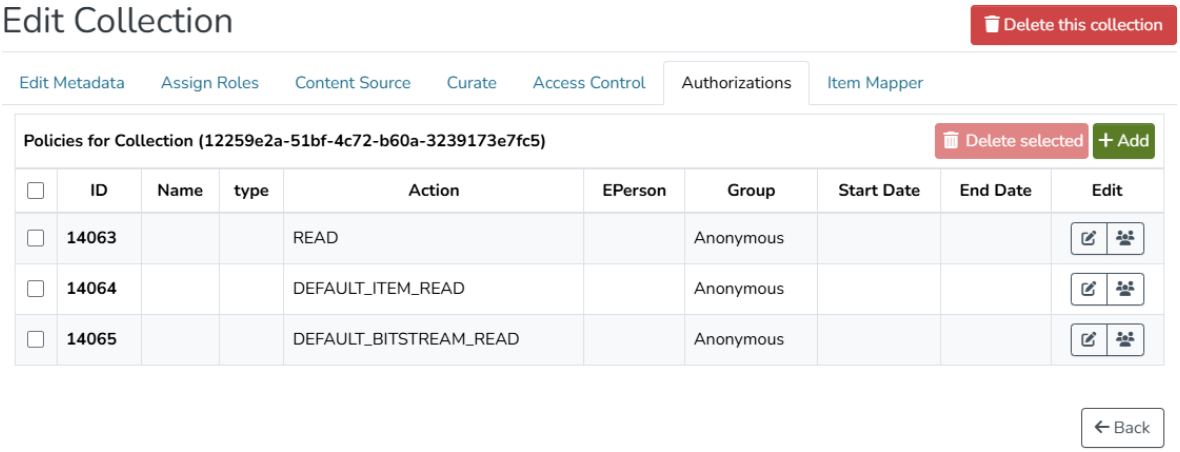
- 'Assigning Roles' in Collections: How to Set Up Workflows and Grant Privileges
- Collection Management
  - Create Collection
  - Delete Collection
  - Edit Collection
  - Export Collection
- Community Management
  - Create a Community
  - Delete Community
  - Edit Community

'Assigning Roles' in Collections: How to Set Up Workflows and Grant Privileges

After creating a collection, if you would like individuals other than the repository administrator to be able to submit items or edit materials after submission, or if you want materials to be reviewed or edited by your team before they're added to the collection, you need to 'assign roles':



Assigning roles also creates a policy related to that role and the user/group to which is assigned. These policies can be viewed in the 'Authorizations' tab. No use cases have been identified by the DSpace community for why an administrator would need to add or modify policies through this section of the interface; it is generally simpler to add policies by assigning roles. Below are the default authorization policies for a collection:



There are two types of roles: (1) those which grant privileges within the collection, and (2) those which set up the workflow for the collection. These are reviewed below.

(1) Roles granting privileges within a collection:

1.Administrator

The DSpace interface states 'Collection administrators decide who can submit items to the collection, edit item metadata (after submission), and add (map) existing items from other collections to this collection (subject to authorization for that collection).'

Implications:

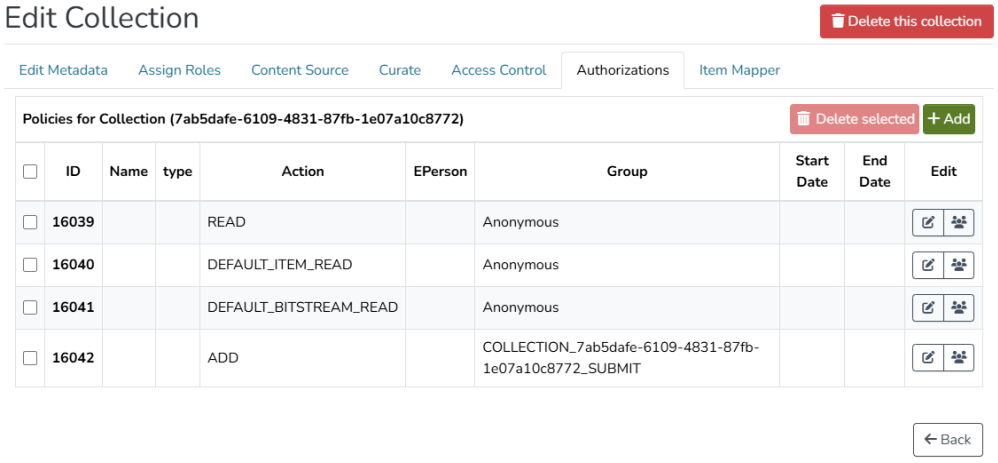
- If you give an individual administrator privileges on a collection, they can submit materials to the collection, delete the collection, withdraw materials, change embargo dates, change the workflow on their own, and assign other users to be collection administrators
- It is redundant to give a top-level administrator a role as a collection admin; they already have these privileges via their current role
- Administrators, however, will not have materials submitted to the collection appear in their 'Workflow Tasks' in 'My DSpace'; this only occurs for individuals assigned to a role of the second type.
- This role may also be assigned for communities

2. Submitters

The DSpace interface states that Submitters are 'the E-People and Groups that have permission to submit new items to this collection.' By default, in a typical DSpace installation, only top-level Administrators and collection administrators should be able to submit to a collection.

Implications:

- If you would like to allow for anonymous submissions to a collection, you must 'Create' a group under submitters, and then add the 'Anonymous' subgroup to that group.
- Creating a submitters group adds an 'ADD' policy under 'Authorizations'; this is however not information you will likely ever need to use:



3. Default item read access

The DSpace interface states that those with this role are the 'E-People and Groups that can read new items submitted to this collection. Changes to this role are not retroactive. Existing items in the system will still be viewable by those who had read access at the time of their addition.' By default, this role is set to anonymous.

4. Default item bitstream read access

E-People and Groups that can read new bitstreams submitted to this collection. Changes to this role are not retroactive. Existing bitstreams in the system will still be viewable by those who had read access at the time of their addition.

(2) Roles setting up workflow for a collection

- The number of roles configured here determine the number of steps in your collection workflow. For example, assigning a populated group to Reviewers, Editors, and Final Editors creates a three-step approval process.
- You cannot review or edit materials you submit to a collection, either by yourself or while impersonating another user. They will not appear in your workflow.
- Neither editors nor final editors can edit the metadata of a submission AFTER it has been accepted.

1.Reviewers

The DSpace interface states that 'Reviewers are able to accept or reject incoming submissions. However, they are not able to edit the submission's metadata.'

2. Editors

'Editors are able to edit the metadata of incoming submissions, and then accept or reject them.'

3. Final Editors

'Final editors are able to edit the metadata of incoming submissions, but will not be able to reject them.'

Collection Management

The collection is a level within a community or sub-community that holds items. This document provides an overview of creating, editing, and deleting a collection.

The documentation below assumes that the user has the relevant authorizations. For example, the admin menu and edit buttons would appear to a user having collection administration permission.

If you’re unsure about collection administration permissions assigned to your account for the target community, contact your system administrator.

- Create Collection
- Delete Collection
- Edit Collection
- Export Collection

Create Collection

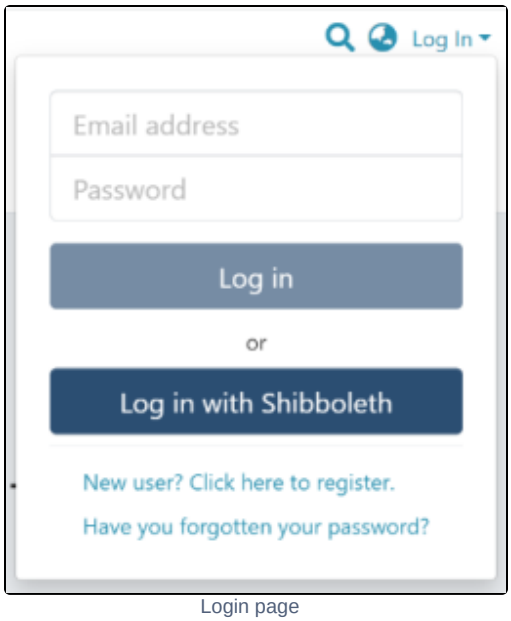
- Audience
- Create Collection

Audience

Repository Administrator
Community Administrator
Collection Administrator

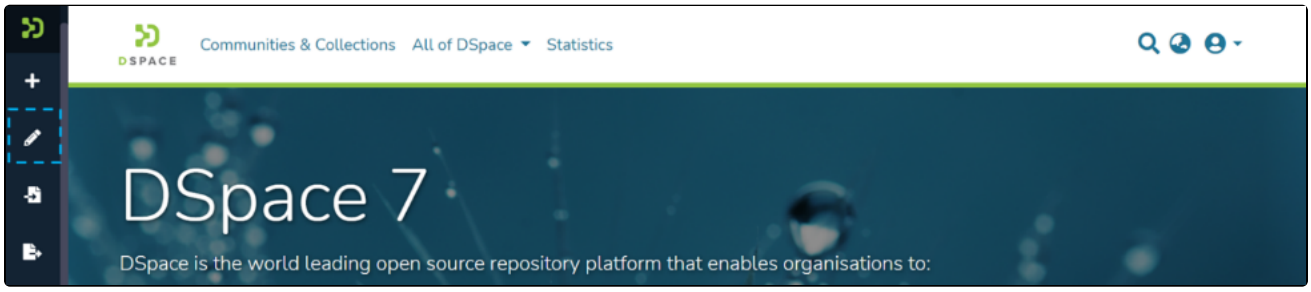
Create Collection

Step 1: Login using your credentials

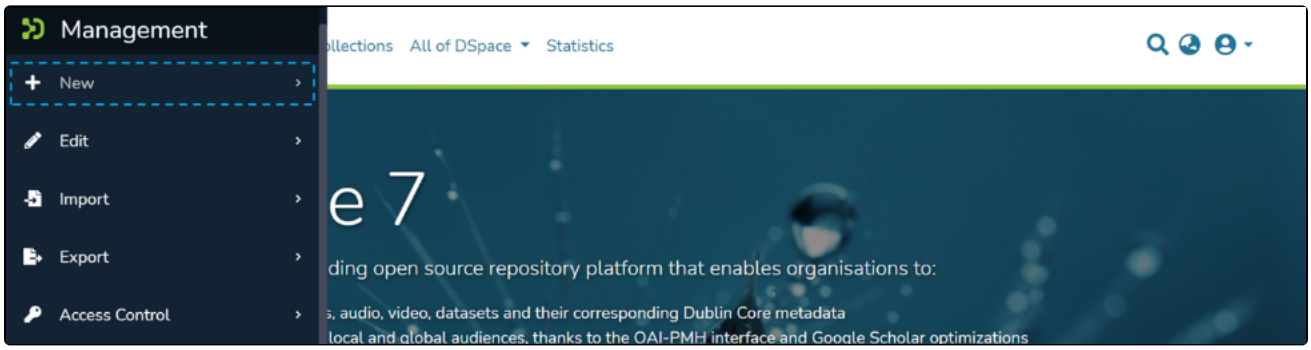


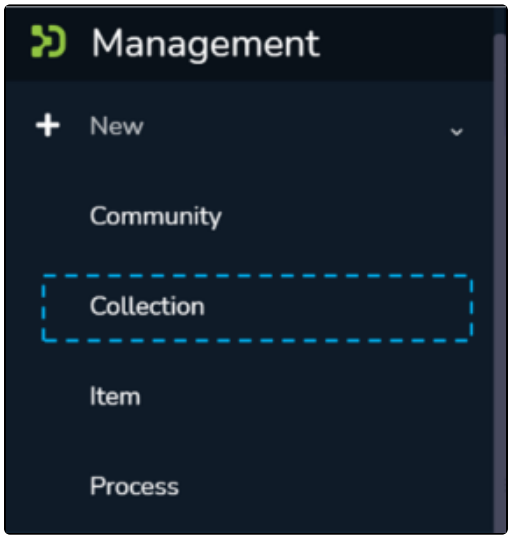
Login page

Step 2: Rollover your cursor on the “+” sign

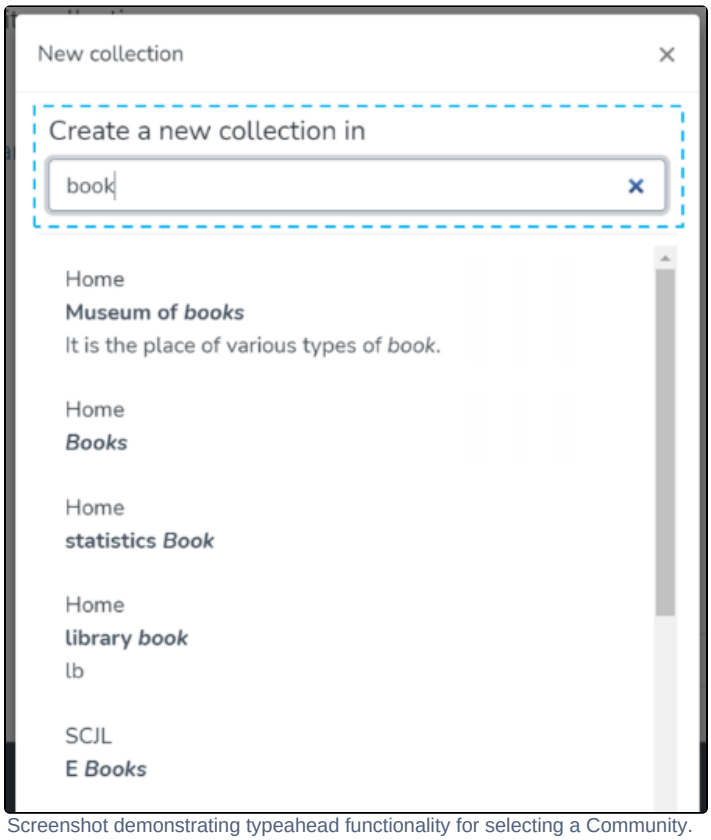


Step 3: Click on “New” and click on “Collection” to proceed with the collection creation process.



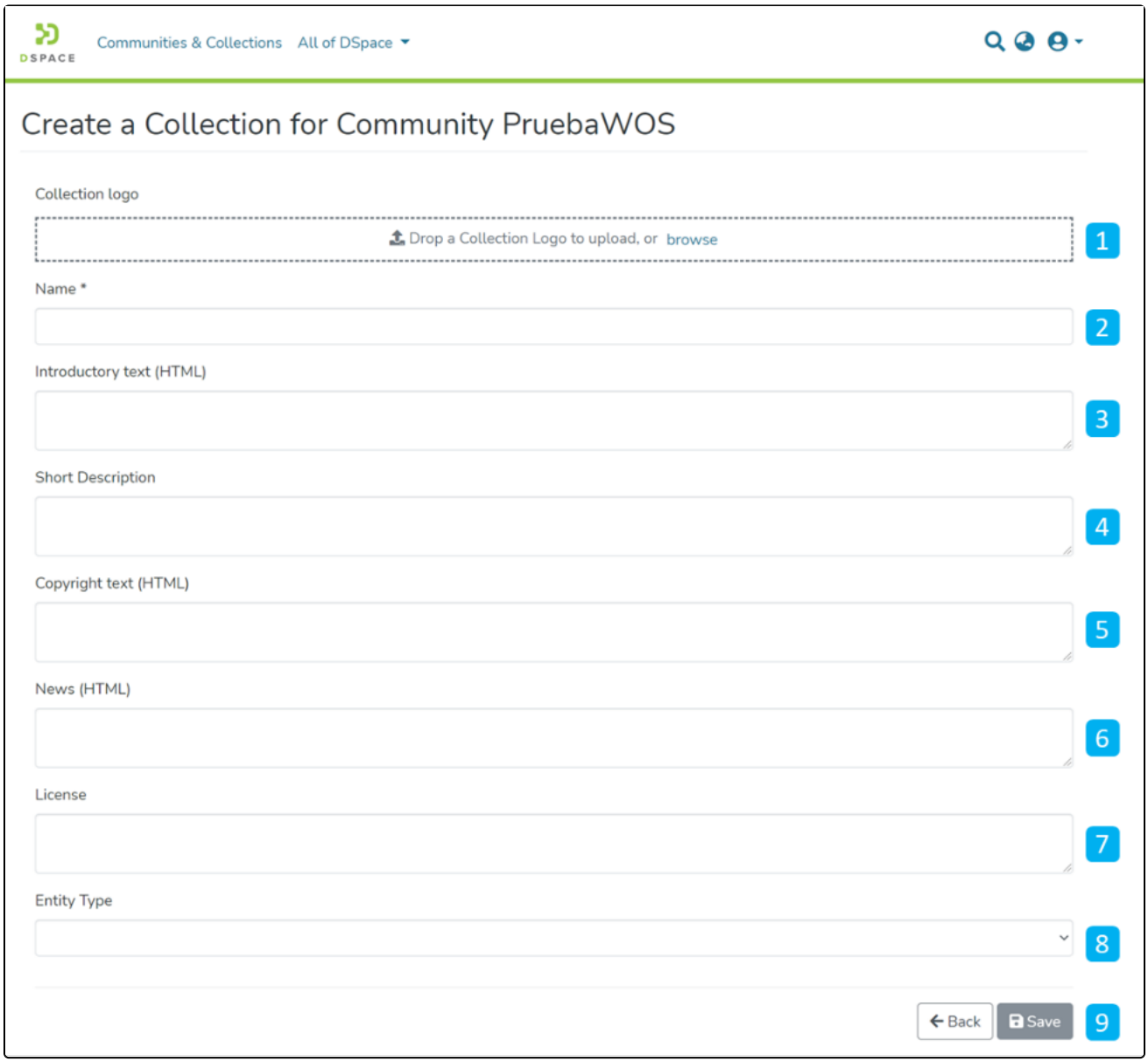


Step 4: A pop-up window showing a list of communities will appear. Type the community's name in the search field where you want to add this collection. Upon typing a few letters of the community's name, a list of communities having those letters or words will appear. Click on the community name for initiating collection creation.



Screenshot demonstrating typeahead functionality for selecting a Community.

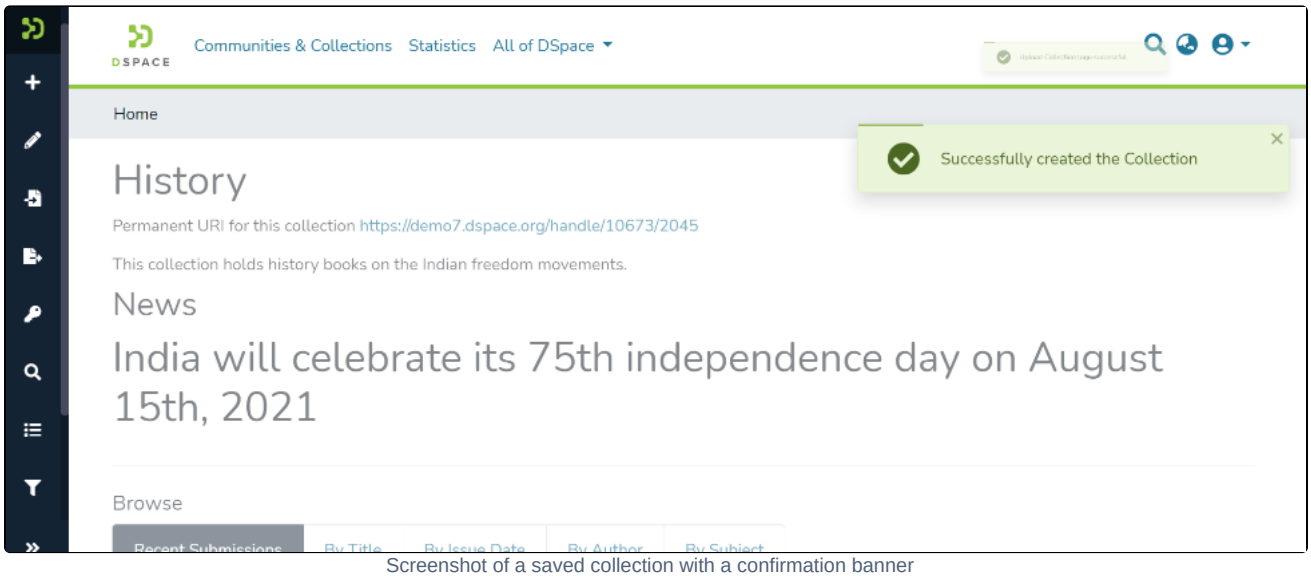
Step 5: A Create Collection form opens to populate information regarding the collection.



Create Collection form

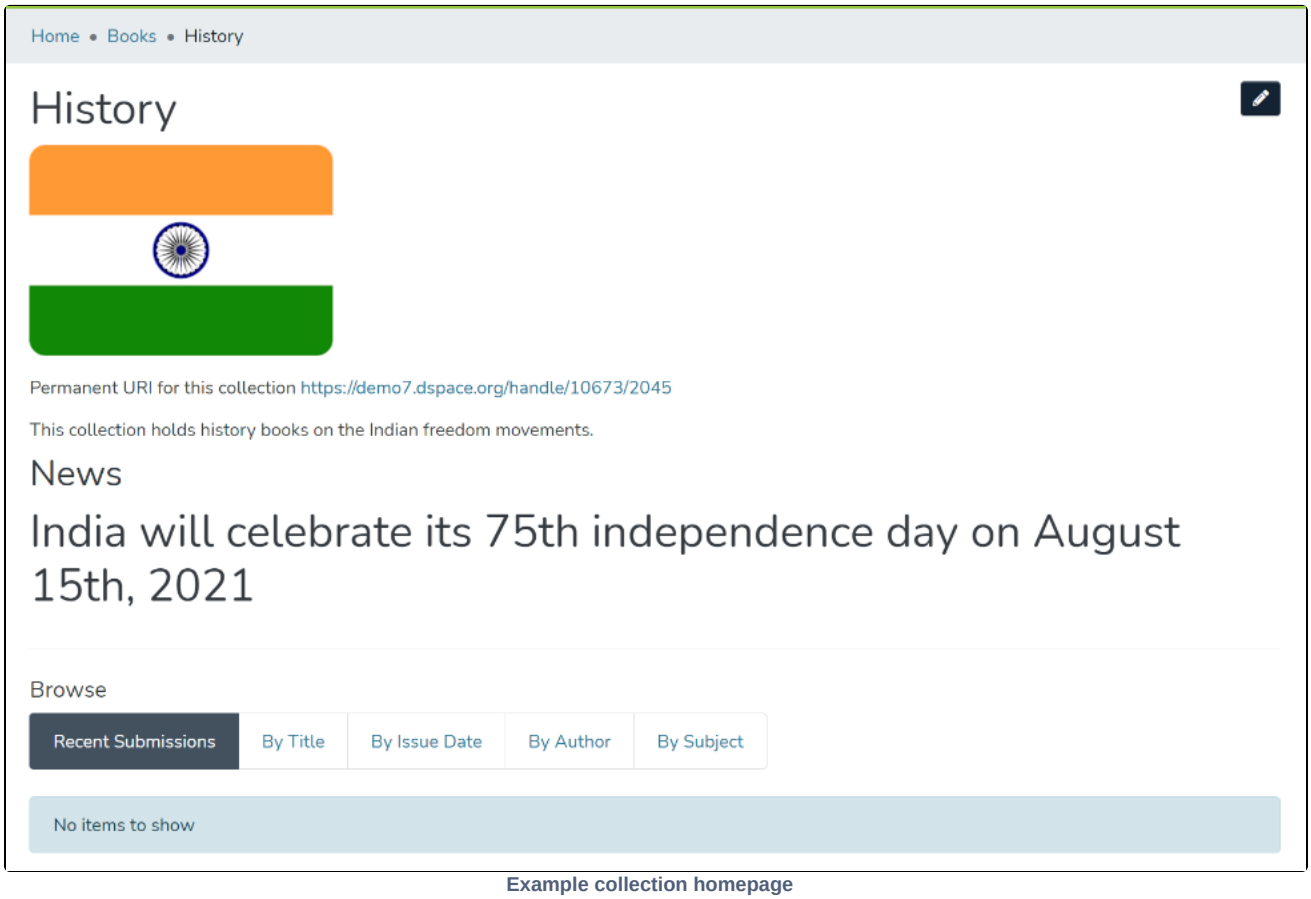
1. *Collection logo – Click on the 'browse' link to select an image file to add as the collection's logo. It is advisable to maintain uniform dimensions of the logo across the repository.*
2. *Name – The collection's name is a mandatory field and marked in '*.'*
3. *Introduction text (HTML) – Users can enter introductory text providing an overview of the contents stored in this collection. One can utilize HTML tags to format the text or continue entering the plain text content.*
4. *Short Description – This field can have a one-line description of the collection that appears with the collection name on the community homepage.*
5. *Copyright text (HTML) – Users can enter copyrights related information here. Fields marked with (HTML) support HTML tags-based formatting.*
6. *New (HTML) – Enter news about this collection. Users can update this by going to this section via Edit Collection.*
7. *License – Add license-related information here.*
8. *Entity Type – Select Entity from the drop-down that must be uploaded in the collection.*
9. *Action Button – Users can click on the appropriate button as determined. Clicking on the Save button will add the collection into the repository.*

Step 6: Click on the 'Save' button to complete the Collection creation process. A success prompt will pop up upon collection creation, and the application will automatically open the collection homepage.



Screenshot of a saved collection with a confirmation banner

Success prompt upon collection creation



Example collection homepage

Collection homepage

Delete Collection

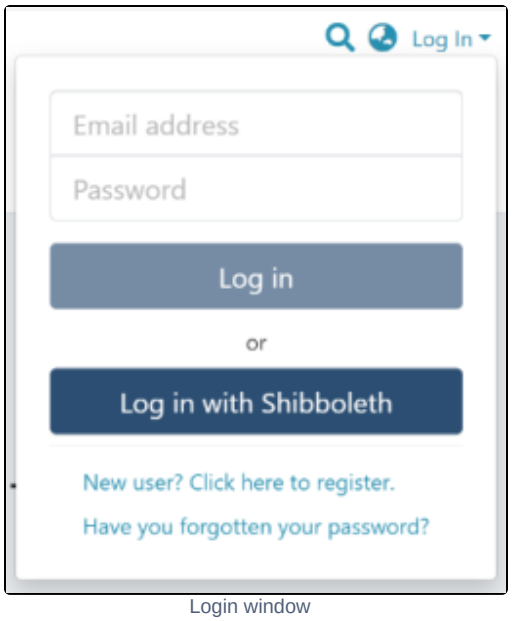
- Audience
- Delete Collection

Audience

Repository Administrator
Community Administrator
Collection Administrator

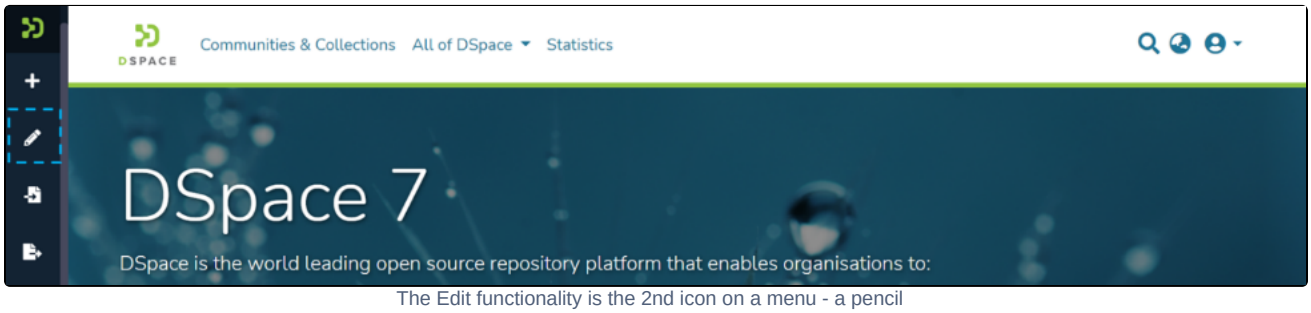
Delete Collection

Step 1: Login using your credentials



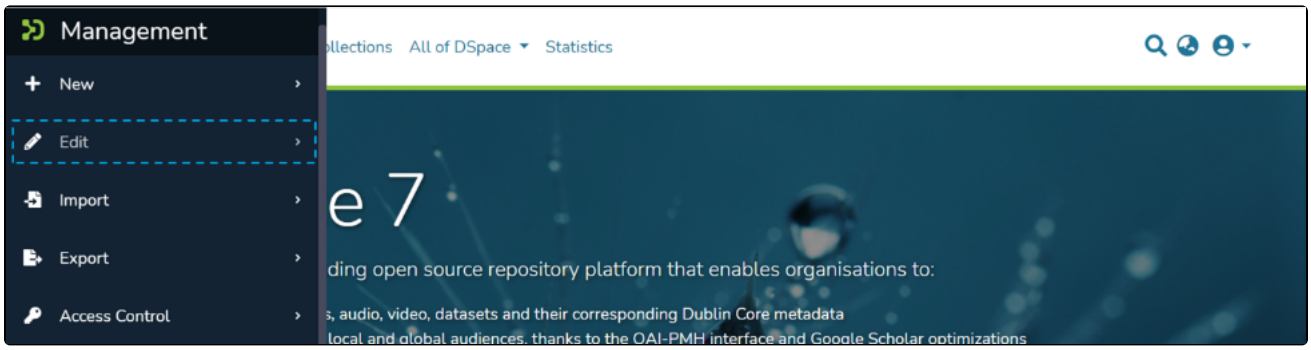
Login window

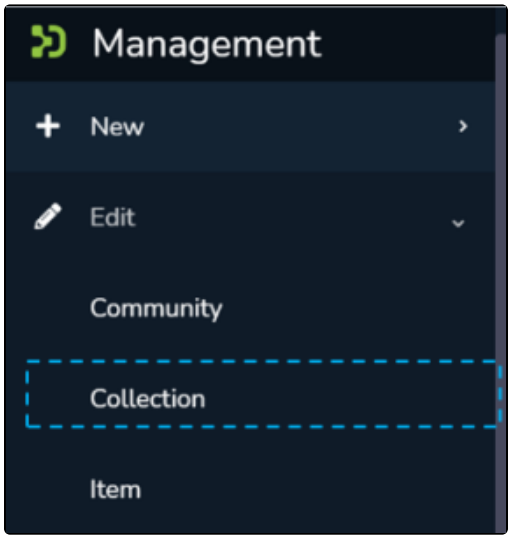
Step 2: There are multiple ways to initiate the delete collection process. One of them is by going to the target collection using Admin options. Rollover your cursor on the”Edit“ sign.



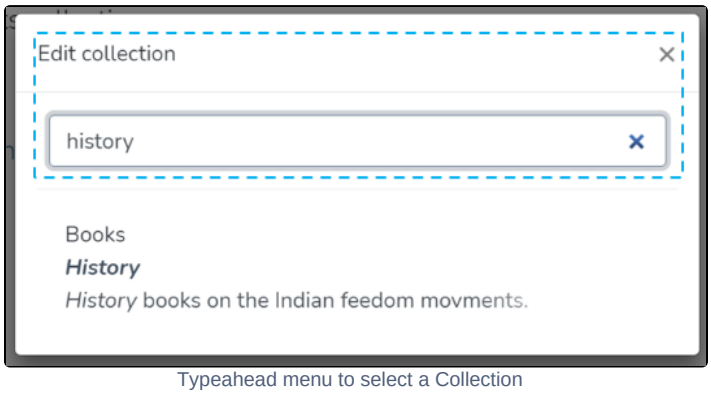
The Edit functionality is the 2nd icon on a menu - a pencil

Step 3: Click on “Edit” and click on “Collection” to proceed with the edit collection process.



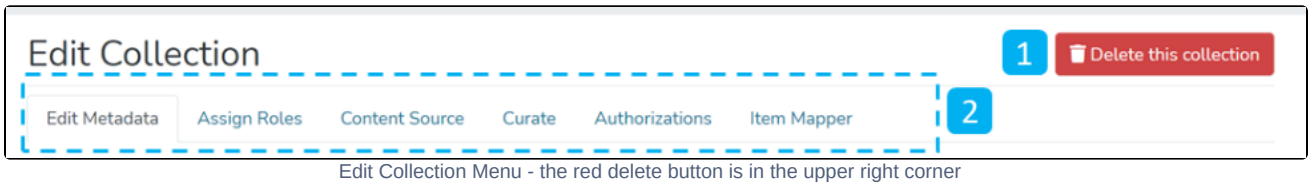


Step 4: A pop-up with the list of collections will appear. Type the Collection's name in the search field you want to delete. A list of collections having typed values will appear upon typing a few letters of the Collection's title. Click on the Collection to continue with the deletion.



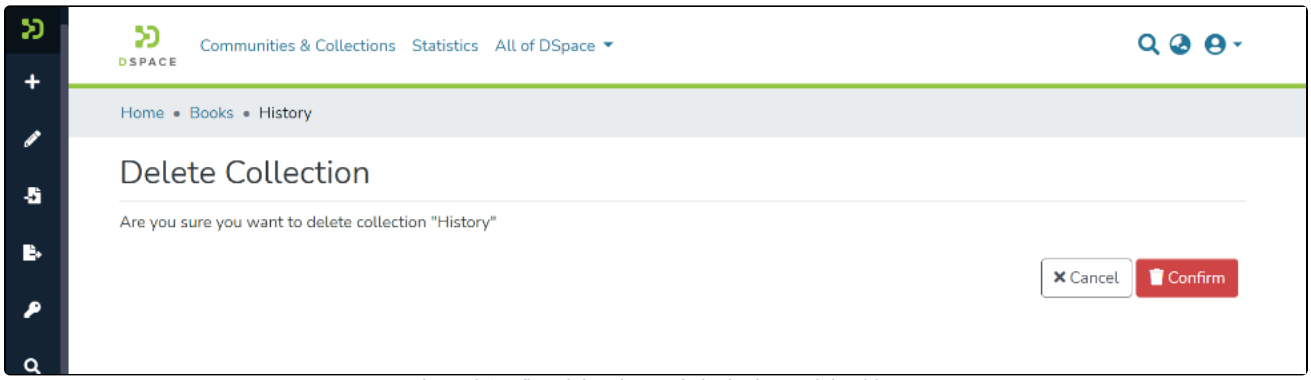
Typeahead menu to select a Collection

Step 5: The application will take the user to the edit collection form. To initiate the collection deletion, the user must click the 'Delete this collection' button.



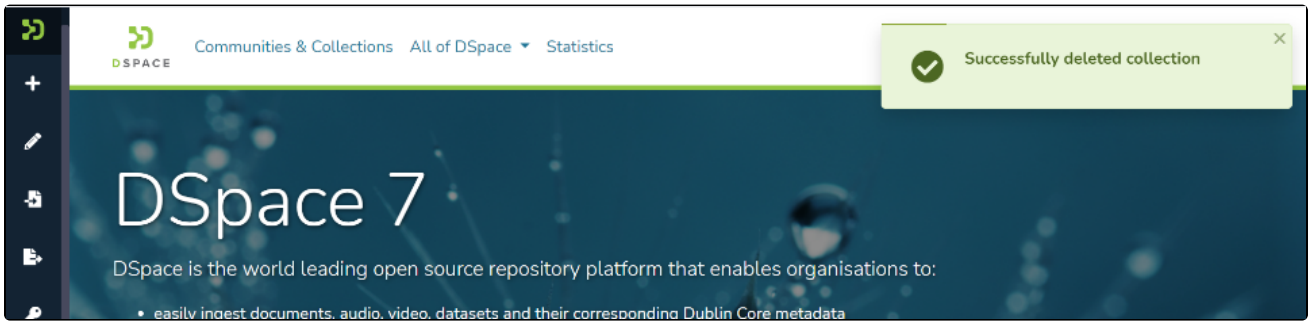
Edit Collection Menu - the red delete button is in the upper right corner

Step 6: Click on the Confirm button to continue with the collection deletion or click on the Cancel button to return to the previous page.



The red Confirm delete button is in the lower right side

Success prompt confirming the deletion will appear, and the DSpace homepage will open.



Edit Collection

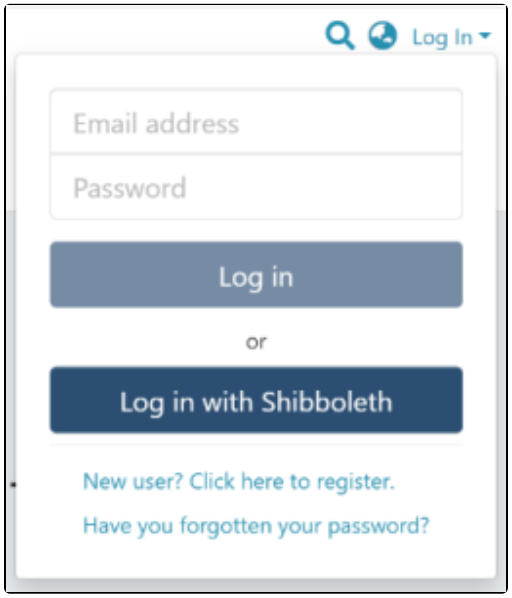
- Audience
- Edit Collection
- Edit Metadata
    - Template Item
- Assign Roles
- Content Source
- Curate
- Authorizations
- Item Mapper
    - Manage mapped items
    - Map new items

Audience

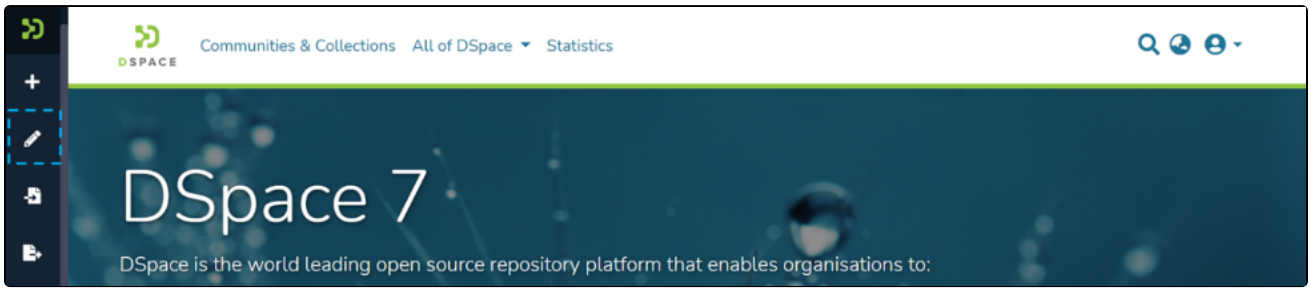
Repository Administrator
Community Administrator
Collection Administrator

Edit Collection

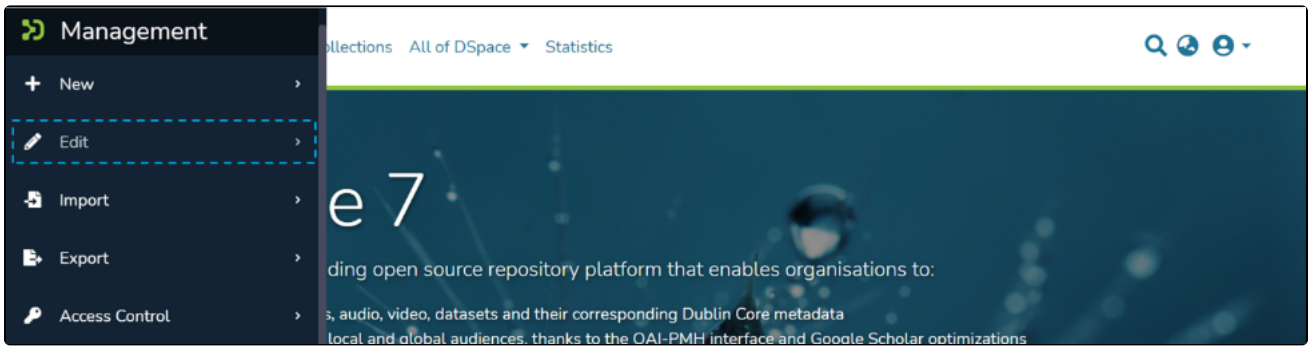
Step 1: Login using your credentials

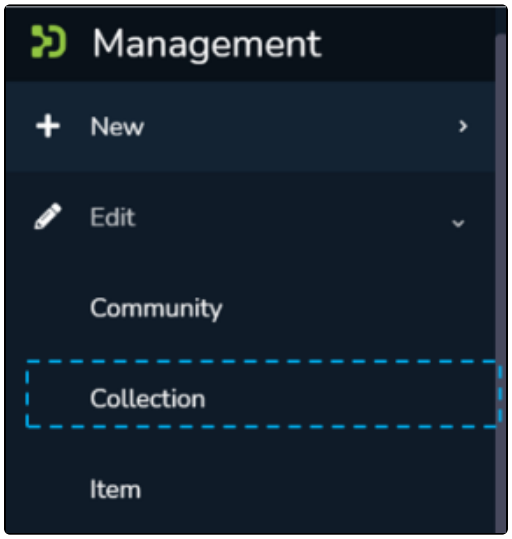


Step 2: There are multiple ways to initiate the Edit collection process. One of them is by going to the target collection using Admin options. Rollover your cursor on the "Edit" sign.



Step 3: Click on “Edit” and click on “Collection” to proceed with the edit collection process.





Step 4: A pop-up window showing list of collections will appear. Type the collection's name in the search field you want to edit. Upon typing a few letters of the collection's name, a list of collections with those letters or words will appear. Click on collection for initiating editing.

Step 5: The application will take the user to the edit collection form. The user can perform a range of actions to edit the collection. Each tab is explained in a separate process in this document.

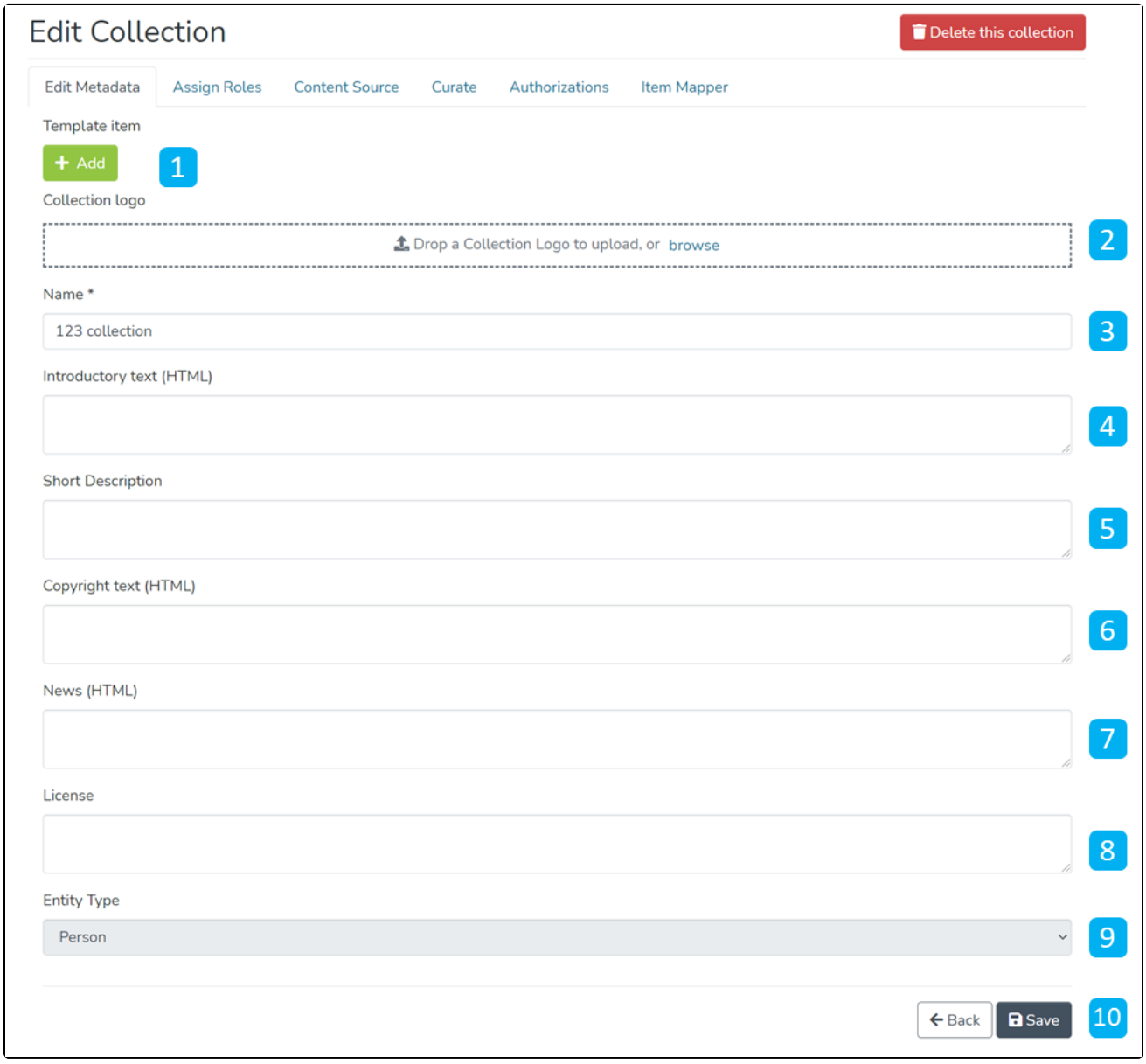
1. *Delete this collection – The button provided for deleting the collection. Detailed steps are explained in the latter part of this document.*

2. *Tabs – Edit collection has a variety of activities involved, which are grouped in a logical manner across various tabs. Below is the summary of these tabs*
    a. *Edit Metadata – Tab covers activities related to editing Collection's profile information*
    b. *Assign Roles – This tab allows users to create specific roles for the collection*
    c. *Content Source – This tab enables harvesting the contents from various sources using OAI standards*
    d. *Curate – Users can set up various workflows related to content curation in this tab*
    e. *Authorizations – Under this tab, users can manage various groups created for managing different access rights and workflows specific to the collection*

Edit Metadata

Step 6: Edit Metadata tab allows users to update the collection's profile-related information, a.k.a. collection Metadata.

Various actions on this tab are explained immediately after the Edit Metadata illustration is added below.

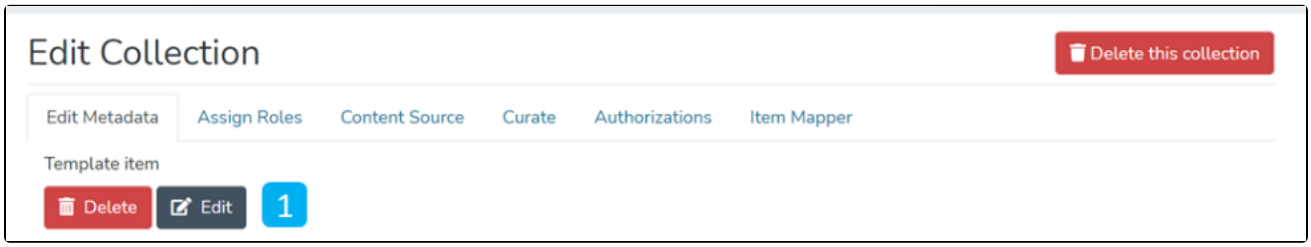


1. Template Item – Users can add metadata elements and values during item submission in this collection. Item submitters can update or delete these pre-populated values during the submission process.
2. Collection logo – Click on the delete button to remove the existing logo or add it if no logo exists.
3. Name – Update the existing collection name in this field.
4. Introduction text (HTML) – Update introductory text if already added or can add new text. One can utilize HTML tags to format the text or continue entering the plain text content.
5. Short Description – Update the collection description or add a fresh short description for the collection.
6. Copyright text (HTML) – Update copyrights related information in this field. Fields marked with (HTML) support HTML tags-based formatting.

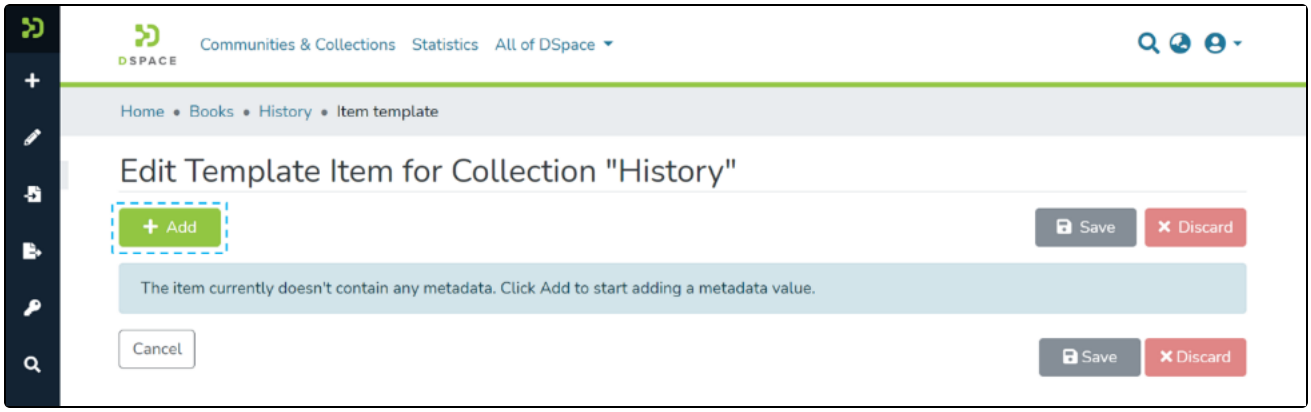
7. New (HTML) – Add/Update news specific to this collection in the field.
8. License – Add/Update license-related information here.
9. Entity Type – Once added an entity to the collection, the value remains constant and uneditable.
10. Action Button – Clicking on the Save button will update the metadata information for the collection.

Template Item

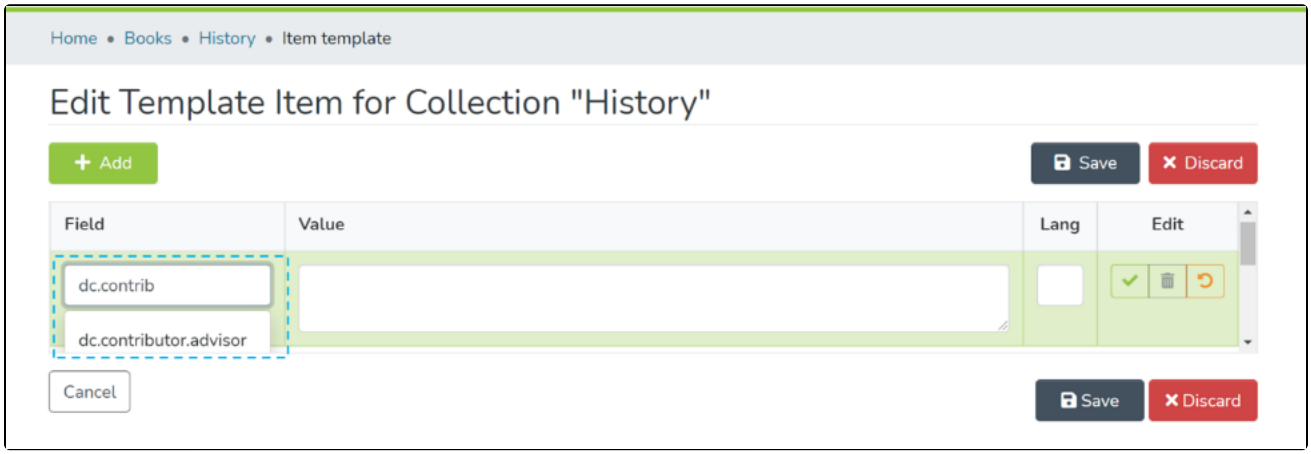
Step 7: Click the 'Edit' button under the Template Item section to add metadata elements with pre-populated values for the item submission process.



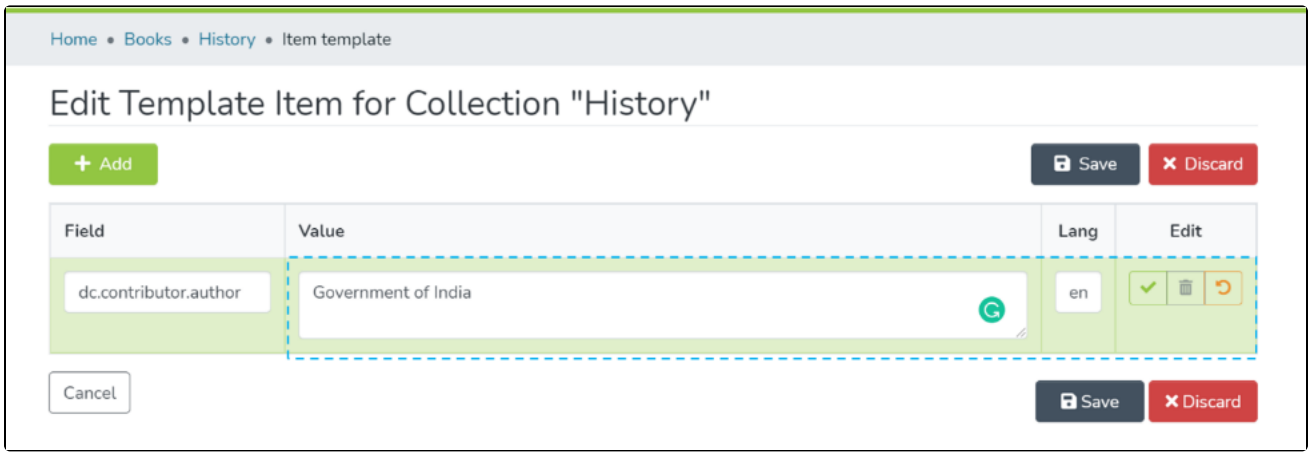
Step 8: Click on the Add button to add the metadata element.



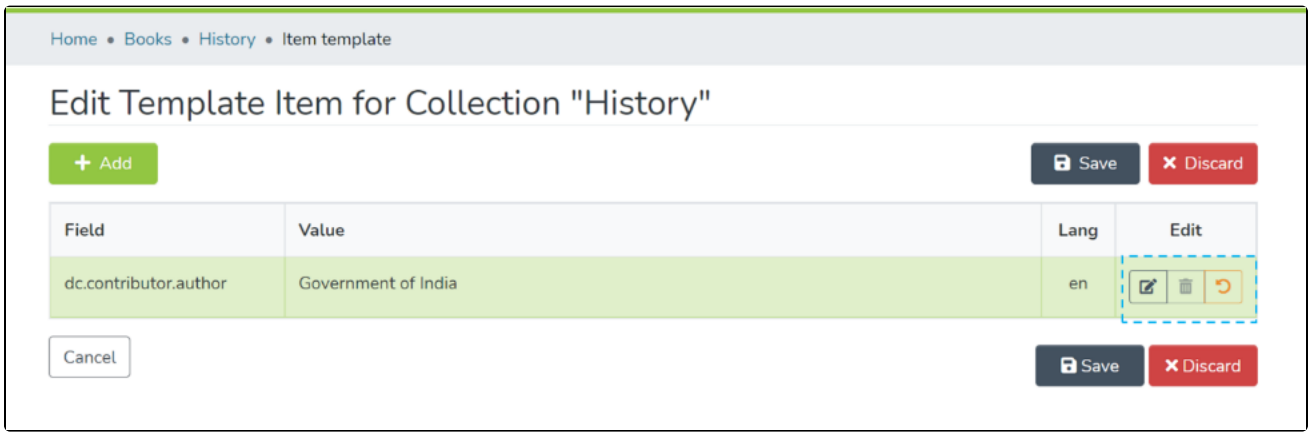
Step 9: Users can start typing metadata elements as demonstrated below and select the appropriate component from the drop-down list.



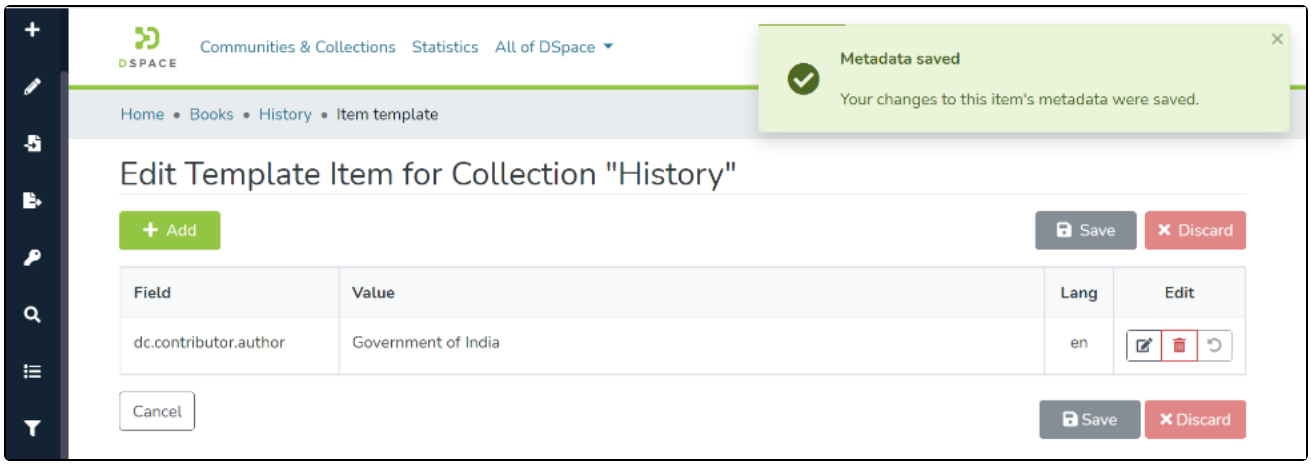
Step 10: Enter the desired value in the Value field, and enter the ISO code of the language used. Then, click on add button for adding the template metadata element.



Step 11: Users can click on the action buttons appearing to the right of the element added for updating or deleting the value-added in the element.



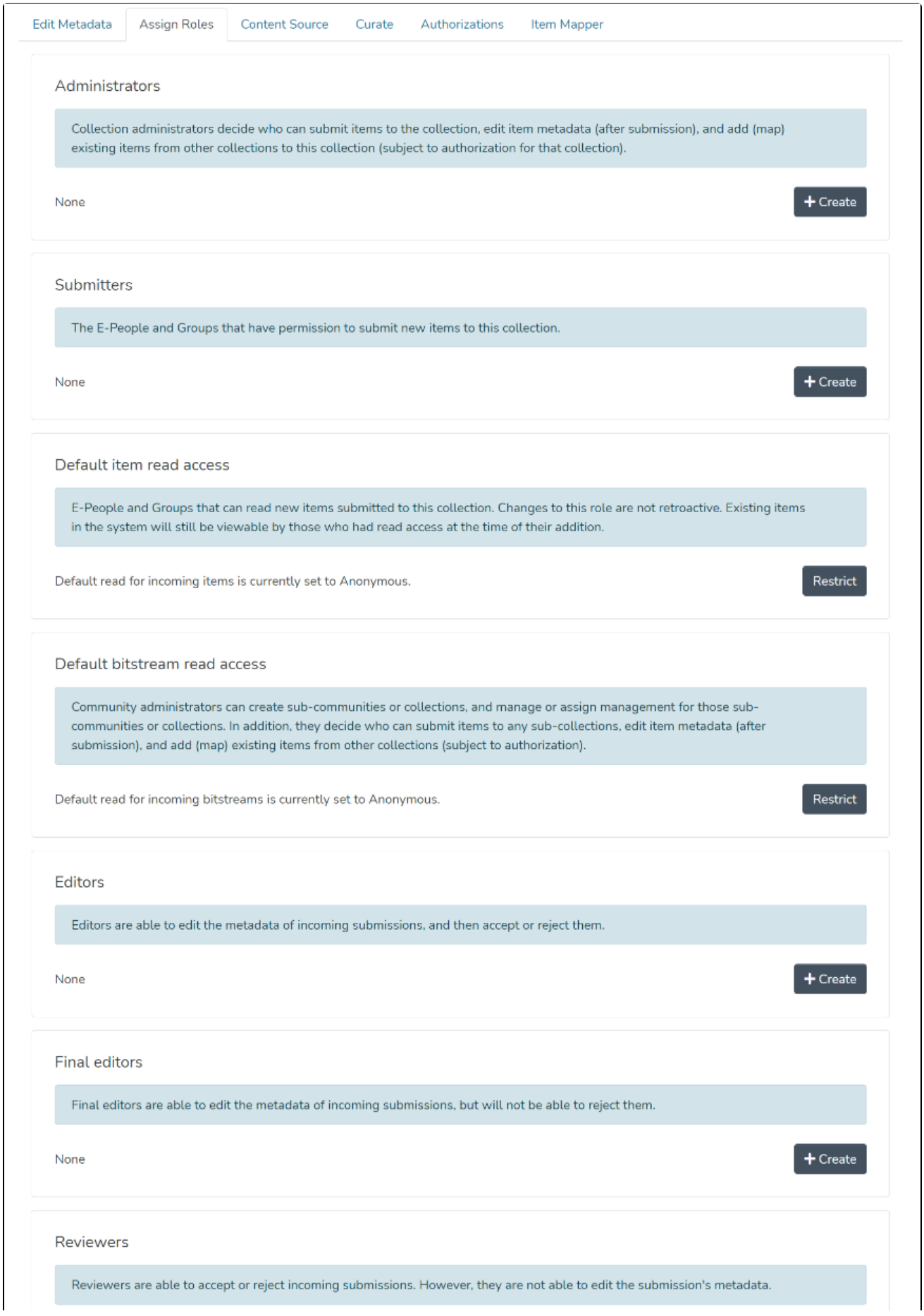
Step 12: Click on the Save button to finish the template edit process. A pop-up notification confirming successful updates of the metadata element will appear, as demonstrated below.

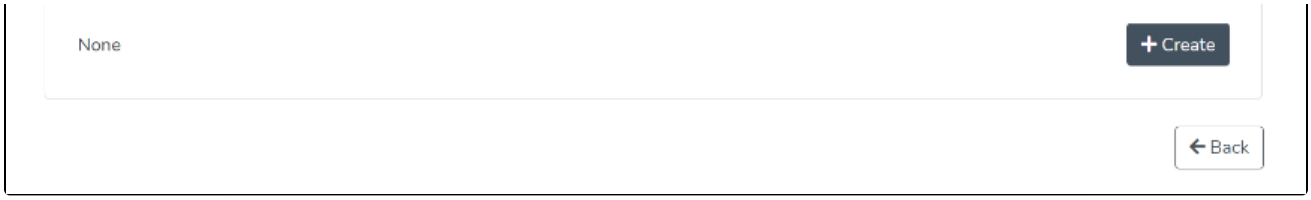


Step 13: Click on the Save button appearing at the bottom of the Edit Metadata tab to save all updates. A success prompt will appear, and the collection homepage will open.

Assign Roles

Step 14: This tab helps assign users to roles designed for the collection. These roles include administrative, maker-checker, and content consumption activities. The description for each role is provided below the screenshot.

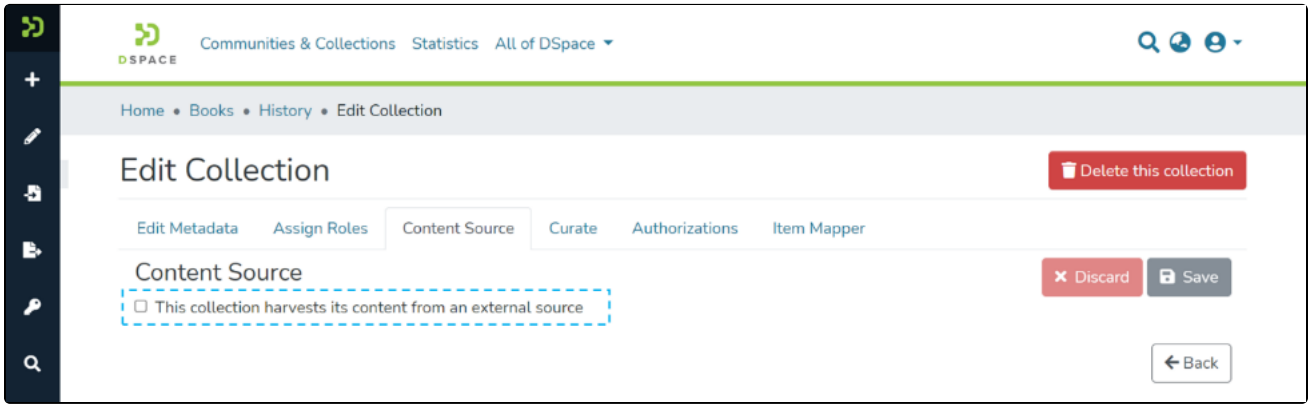

Edit Metadata
Assign Roles
Content Source
Curate
Authorizations
Item Mapper
Administrators
Collection administrators decide who can submit items to the collection, edit item metadata (after submission), and add (map) existing items from other collections to this collection (subject to authorization for that collection).
None
+ Create
Submitters
The E-People and Groups that have permission to submit new items to this collection.
None
+ Create
Default item read access
E-People and Groups that can read new items submitted to this collection. Changes to this role are not retroactive. Existing items in the system will still be viewable by those who had read access at the time of their addition.
Default read for incoming items is currently set to Anonymous.
Restrict
Default bitstream read access
Community administrators can create sub-communities or collections, and manage or assign management for those sub-communities or collections. In addition, they decide who can submit items to any sub-collections, edit item metadata (after submission), and add (map) existing items from other collections (subject to authorization).
Default read for incoming bitstreams is currently set to Anonymous.
Restrict
Editors
Editors are able to edit the metadata of incoming submissions, and then accept or reject them.
None
+ Create
Final editors
Final editors are able to edit the metadata of incoming submissions, but will not be able to reject them.
None
+ Create
Reviewers
Reviewers are able to accept or reject incoming submissions. However, they are not able to edit the submission's metadata.



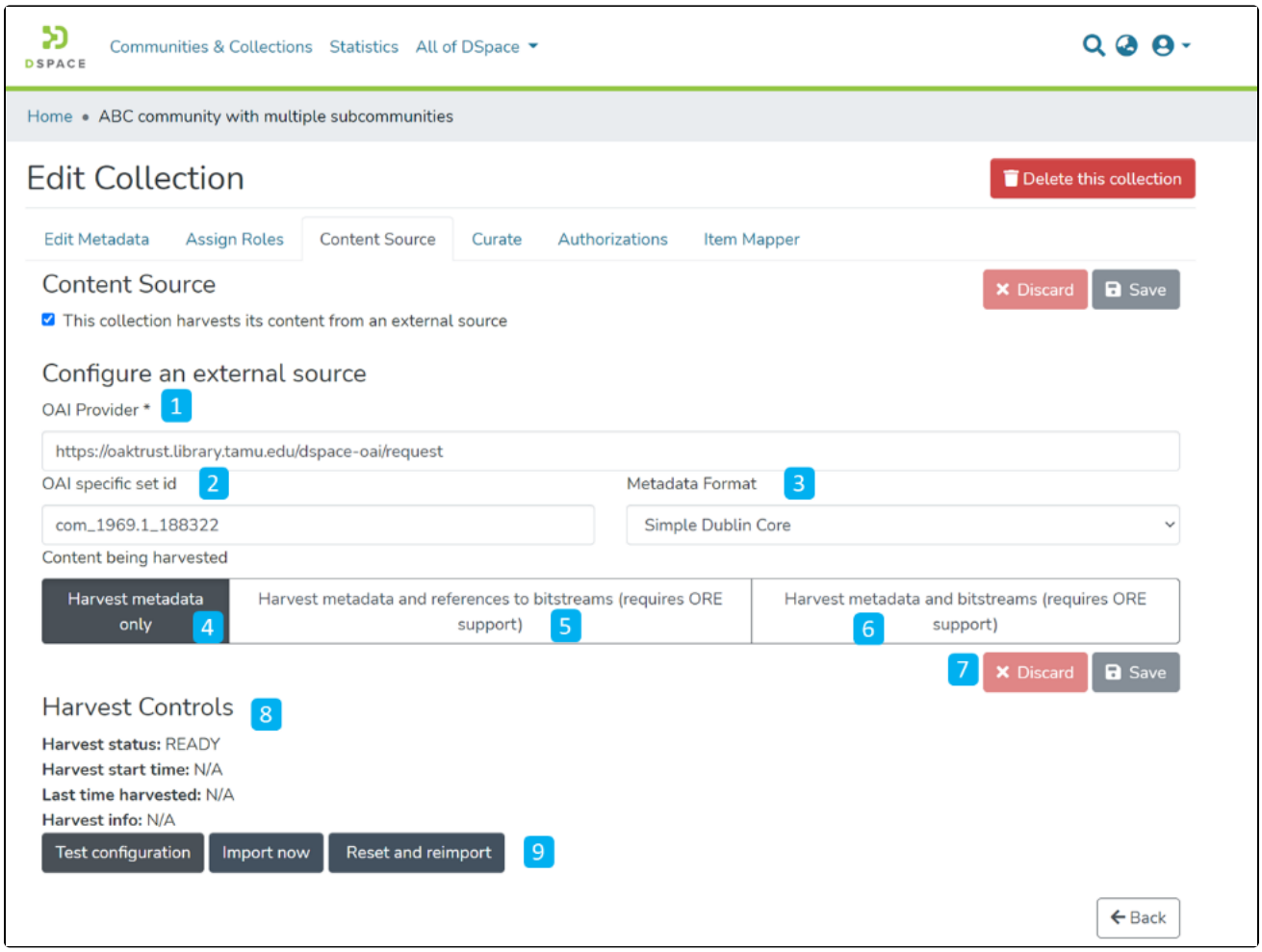
1. Administrators – The collection administrator can assign rights like item submission, edit item metadata, and map existing items from other collections to this collection. Click the create button to create a dedicated Administrator group for the collection.
2. Submitters – Users or User groups part of this group can submit items to the collection. Click on the create button to add specific users and user groups to perform item submission.
3. Default item read access - E-People and Groups can read new items submitted to this collection. Changes to this role are not retroactive. Existing items in the system will still be viewable by those who had read access at the time of their addition. Click the restrict button to restrict default item read access rights to a specific user group.
4. Default bitstream read access – E-People and Groups added in this section can read bitstreams (attachments) in items by default. Click the restrict button to restrict default bitstream read access rights to a specific user group.
5. Editors - Editors can edit the metadata of submissions and then accept or reject them. Click on the create button to add the workflow step of editing metadata and assigning roles to specific users or user groups.
6. Final editors - Final editors can edit the metadata of incoming submissions but can not reject them. Click the create button to add this workflow step to the collection and assign a role to specific users or user groups.
7. Reviewers - Reviewers can accept or reject incoming submissions. However, they can not edit the metadata. Click the create button to add this workflow step to the collection and assign a role to specific users or user groups.

Content Source

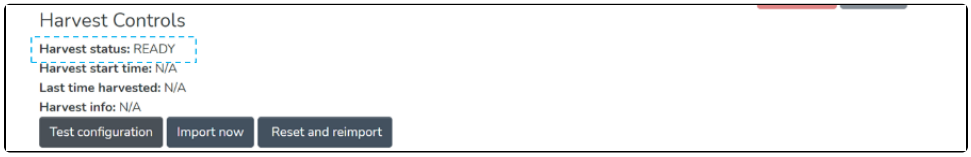
Step 16: This tab enables harvesting the content from external sources using OAI standards. Users can start harvesting by clicking the checkbox, “This collection harvests its content from an external source.”



Step 17: Users will see various parameters related to OAI-based content harvesting upon clicking the checkbox as explained in the previous step. Below is the explanation of elements appearing under Configure an external source header.

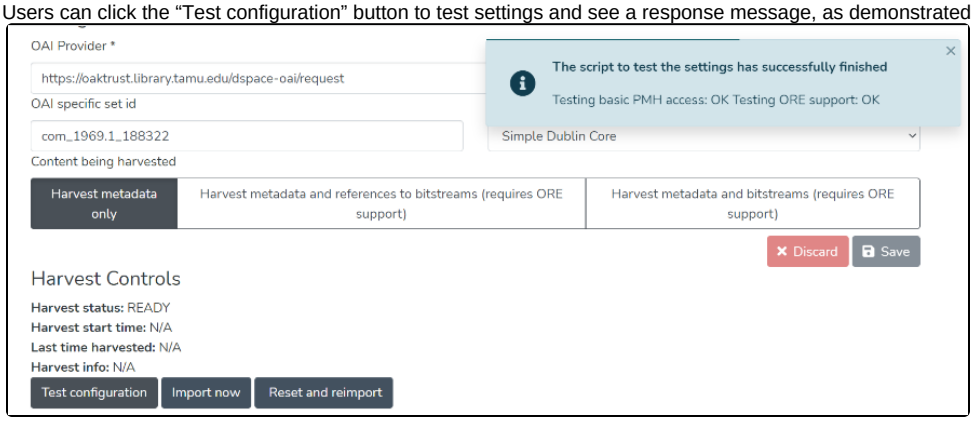


1. OAI Provider – Enter the source OAI provider's URL.
2. OAI specific set id – Enter the set ID to source content.
3. Metadata Format – Select suitable metadata format using the dropdown list, e.g., Simple Dublin Core, Qualified Dublin Core, and DSpace Intermediate metadata.
4. Harvest metadata only – Select this option to harvest only metadata from the source.
5. Harvest metadata and references to bitstreams (requires ORE support) – Click on this option to harvest metadata and reference links to corresponding bitstreams.
6. Harvest metadata and bitstream (requires ORE support) – Use this option to harvest both metadata and corresponding bitstreams into the target repository.
7. Click on the 'Save' button to update harvesting settings.
8. Upon clicking the save button and subject to successful validation of values entered, "Harvest Status" will turn to "Ready," as demonstrated in the screenshot below.



9. After successfully configuring an OAI profile, these buttons will get active, and the user can start harvesting data immediately.

10. Users can click the “Test configuration” button to test settings and see a response message, as demonstrated on the screen below.



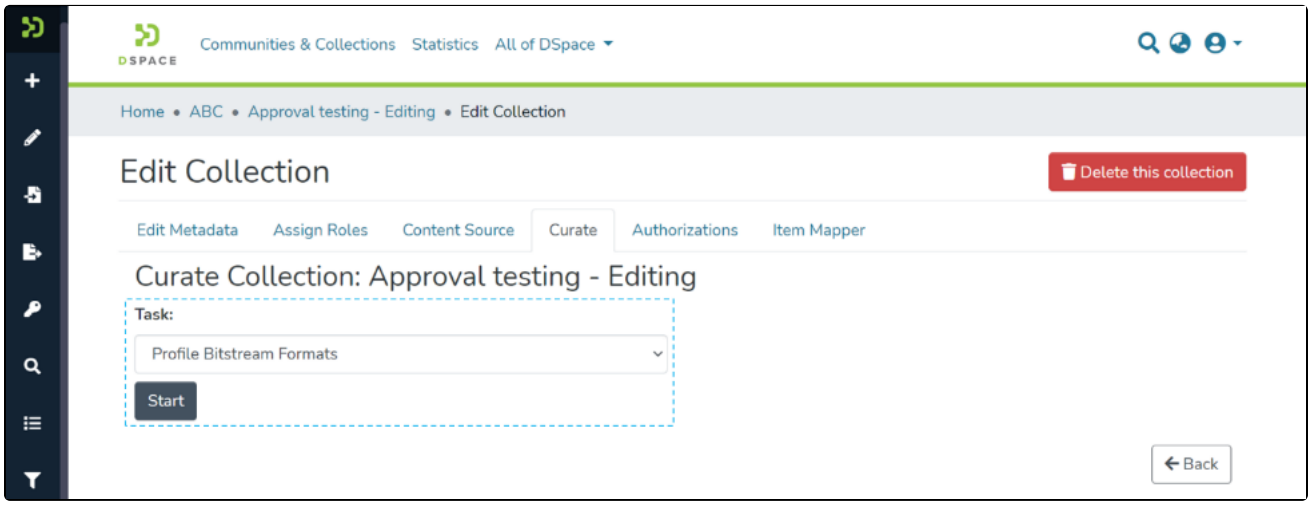
11. Upon successfully testing settings, click on the “Import now” button to harvest metadata immediately.

Curate

Step 18: The Curate tab provides various workflows for curating items stored in the collection. Below are standard flows, and there can be customized curation workflows as well

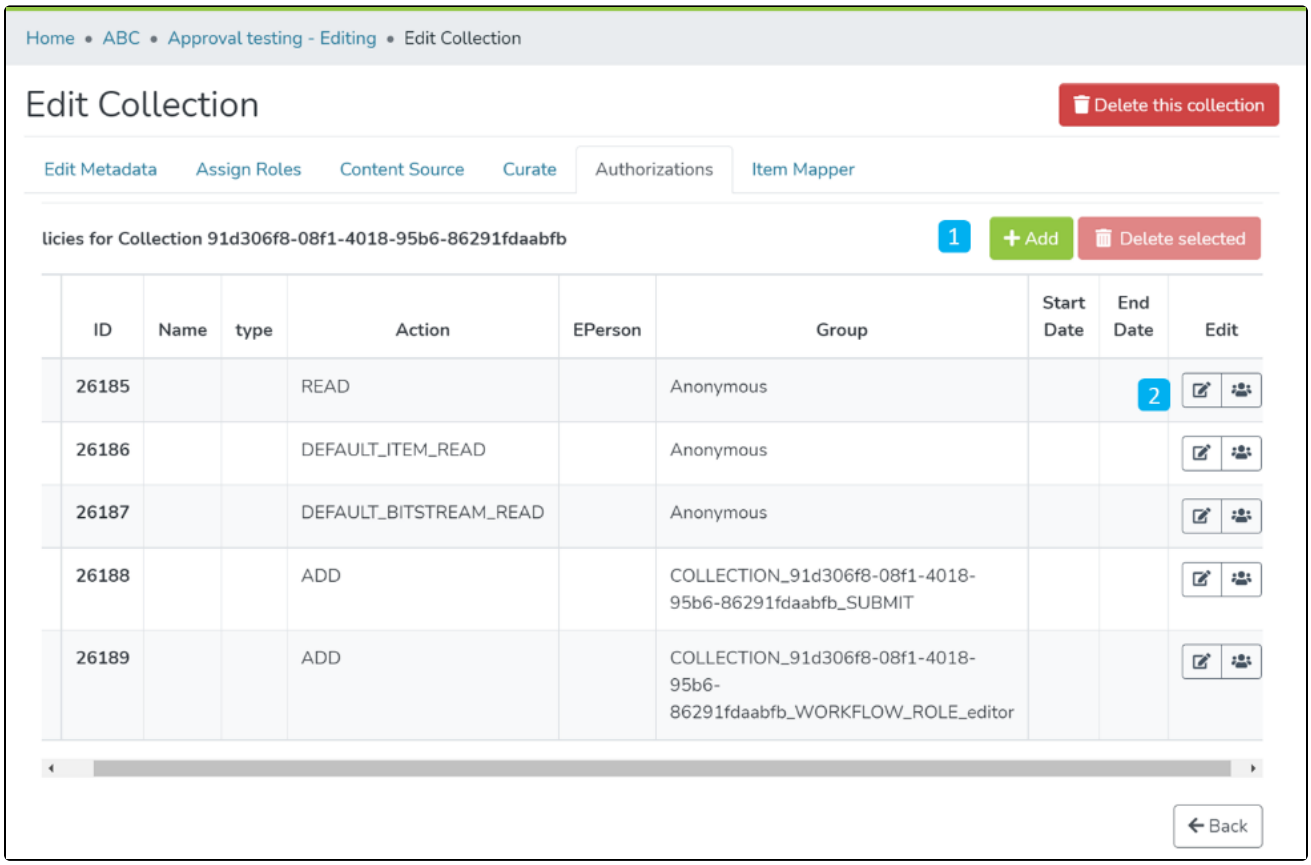
1. Profile bitstream formats
2. Check for Required Metadata
3. Check Links in Metadata

Users must select a workflow from the dropdown list and click the “Start” button to initiate the curation process.



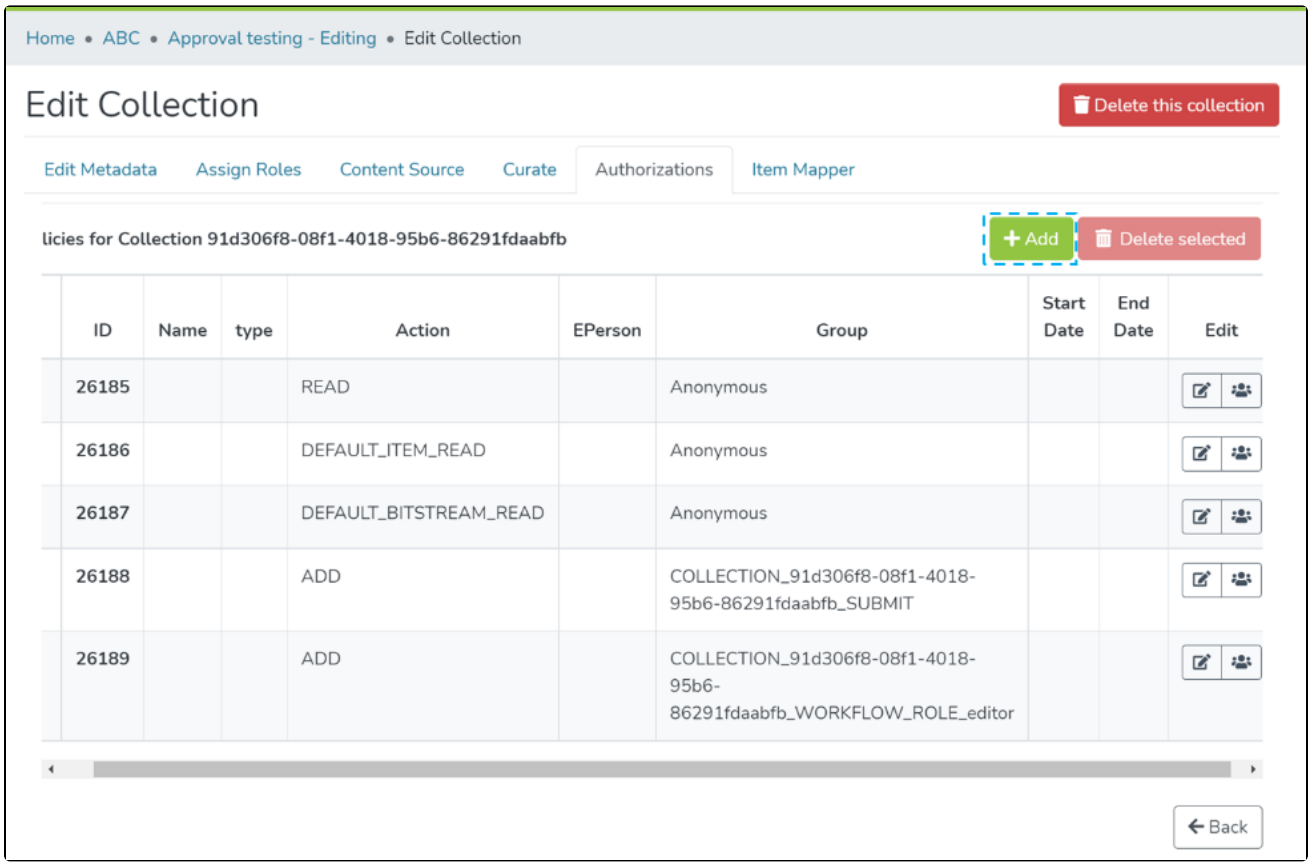
Authorizations

Step 19: Authorizations tab has all the policies defined for the collection. These are in addition to policies created from the “Assign Roles” tab. Key actions available in this tab are explained below.

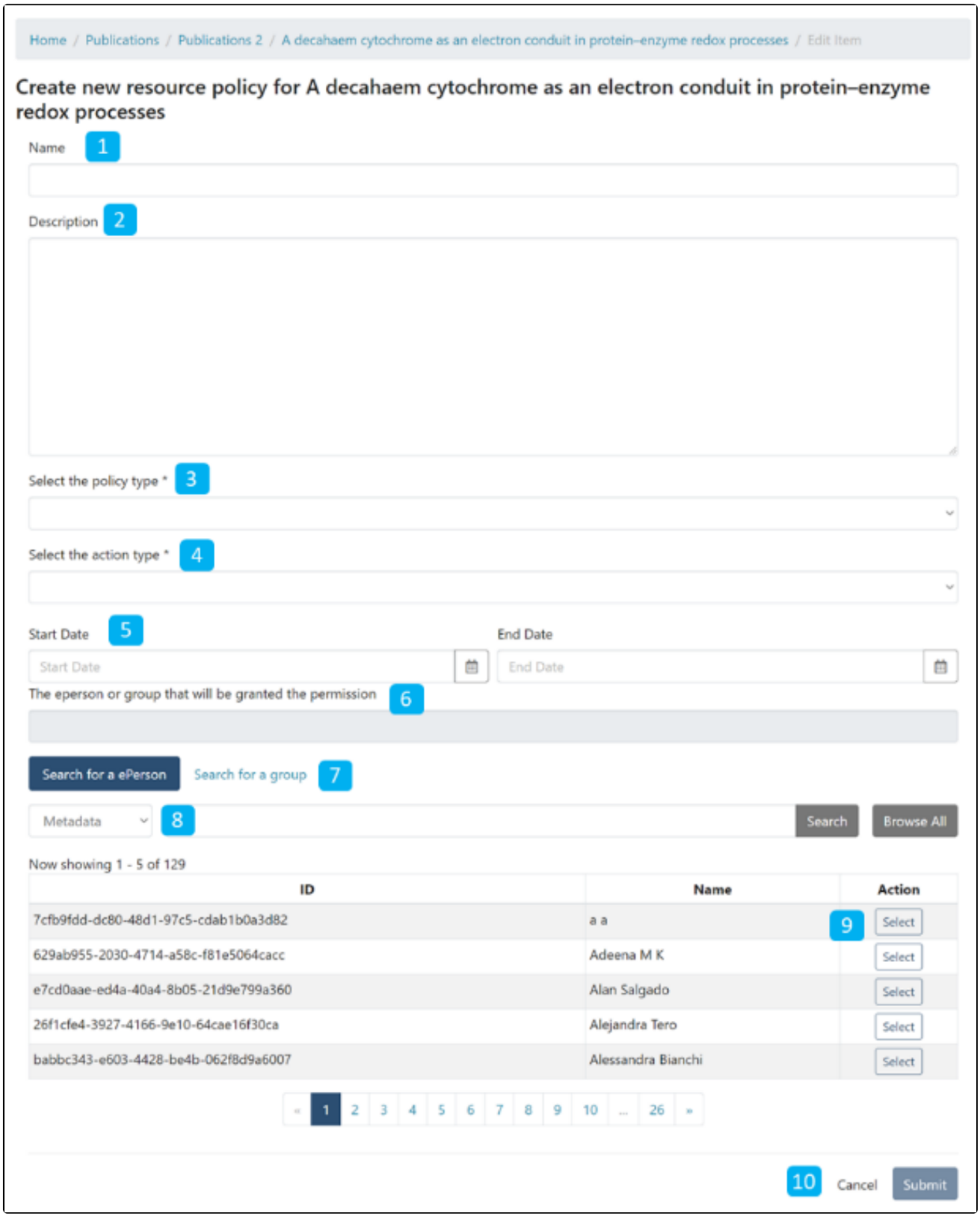


1. Manage Policies – Click the Add button or select policies from the table to create a new policy. Next, click on the Delete selected button for a batch deletion of the policies.
2. Edit policy and members in a policy – Click the edit button to edit an individual policy or click on the group icon to edit the user group.

Step 20: Click on Add button to create a new Authorization policy.



Step 21: Users can add information in the fields available in this form to the policy and save it by clicking the submit button. Please see the description of each field followed by the below screenshot.



1. **Name:** Enter the Policy name in this field.
2. **Description:** Enter the Policy description here for future reference and understanding of other users.
3. **Select the policy type:** The user can select one of the following policy classification types from the list
    a. TYPE_SUBMISSION: a policy in place during the submission
    b. TYPE_WORKFLOW: a policy in place during the approval workflow
    c. TYPE_INHERITED: a policy that has been inherited from a container (the collection)
    d. TYPE_CUSTOM: a policy defined by the user during the submission or workflow phase
4. **Select the action type:** The user can select one of the following actions from the dropdown list:

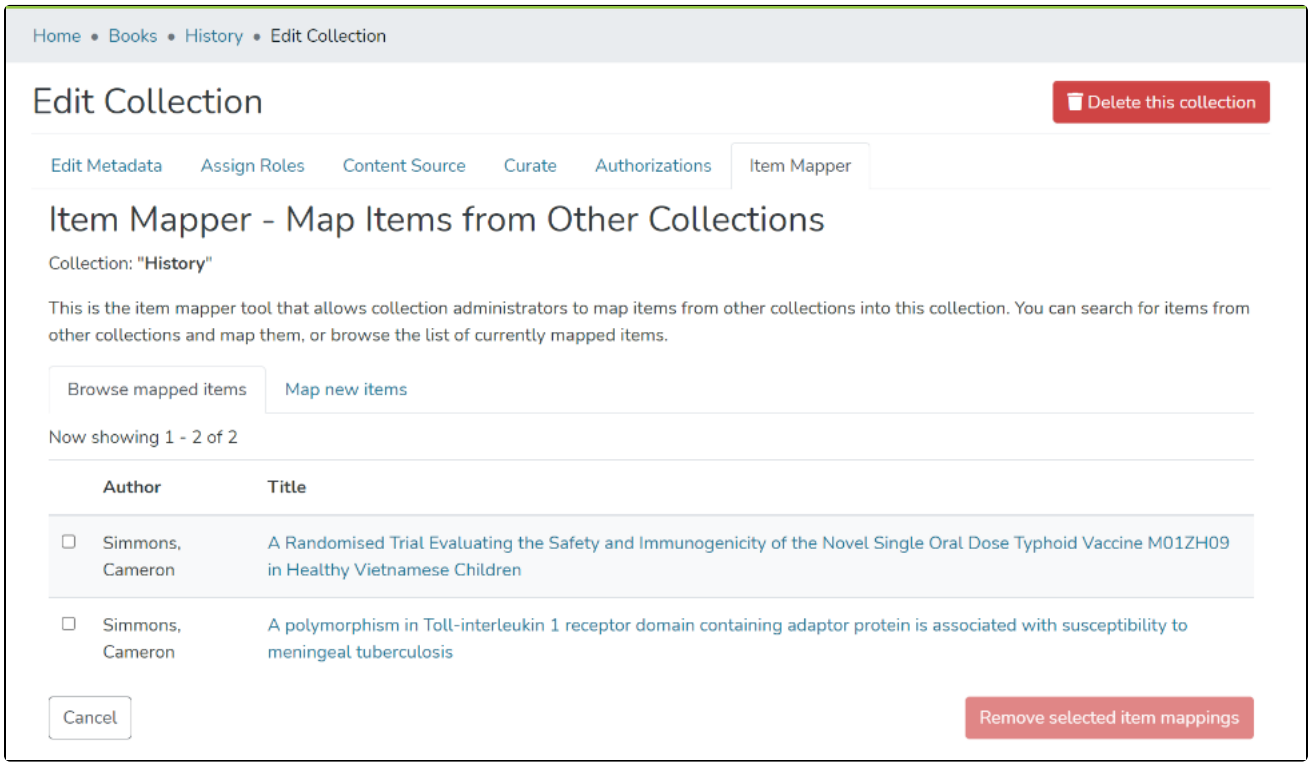
    a. READ - Ability to view the specific object
    b. WRITE - Ability to edit the specific object
    c. ADD - Ability to add an object to a parent object (e.g. add a Bitstream to a Bundle, add a Bundle to an Item, add an Item to a Collection , add a Collection to a Community, etc.)
    d. REMOVE - Ability to remove an object from a parent object (i.e. the opposite of "ADD")
    e. ADMIN - Full Admin permissions on the specific object
    f. DELETE - Ability to delete the specific object
    g. WITHDRAWN_READ - Ability to view a withdrawn Item (only valid for Items)
    h. DEFAULT_BITSTREAM_READ - Default "READ" permissions added to any Bitstreams of a newly submitted Item (only valid for Collections)
    i. DEFAULT_ITEM_READ - Default "READ" permissions added to any newly submitted Item (only valid for Collections)
5. **Start date – end date:** The user can select the start date and end date for using the policy, should they want to apply it for a fixed period.
6. **The eperson or group that will be granted the permission:** List of users/groups selected for granting permission under the policy
7. **Search for an ePerson / Search for a group:** Select ePerson or group for searching the entity
8. **Search field:** Enter keywords for searching the ePerson/Group
9. **ePerson/Group list:** Click on the select button against the user/group you want to add to the policy
10. **Submit/Cancel button:** Click on the Submit button to complete policy creation or click on the Cancel button to cancel the entire process.

You'll see a confirmation prompt upon successfully creating the policy, as shown below. After that, the user will be back on the Authorizations screen.

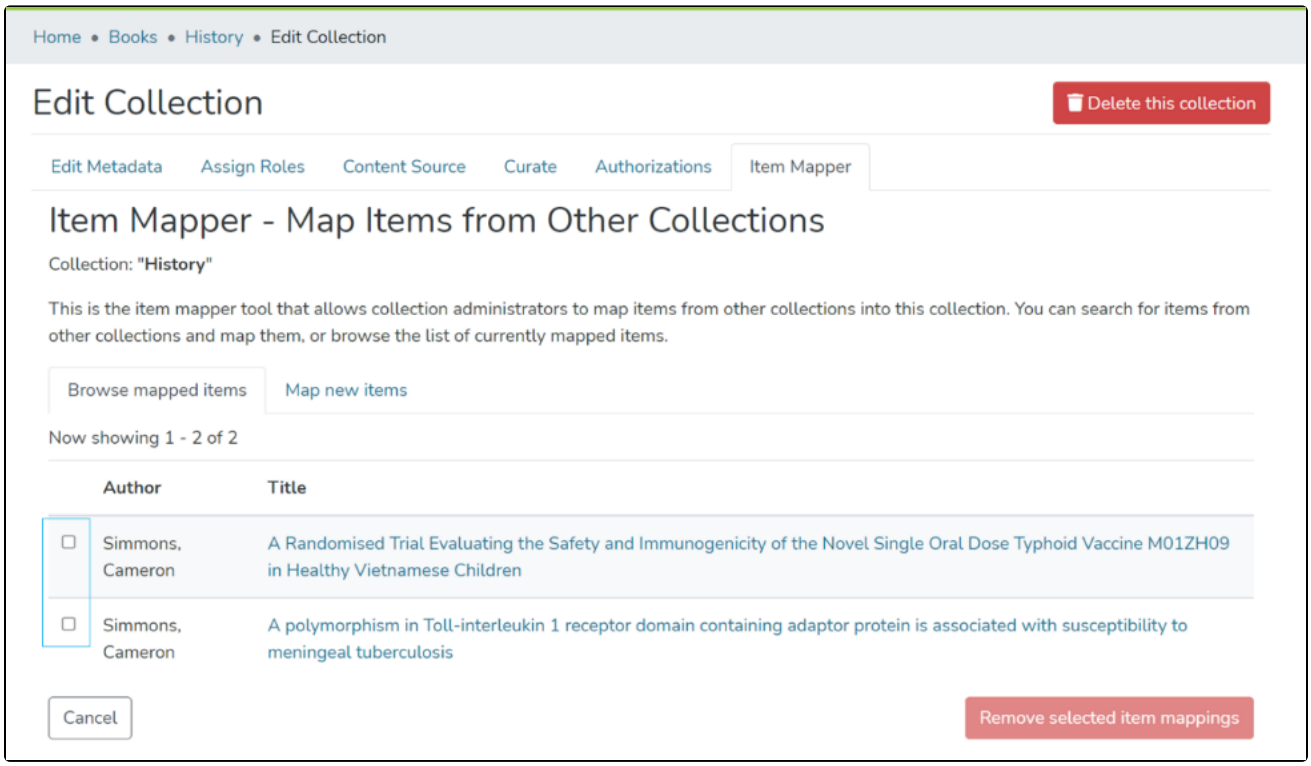
Item Mapper

Manage mapped items

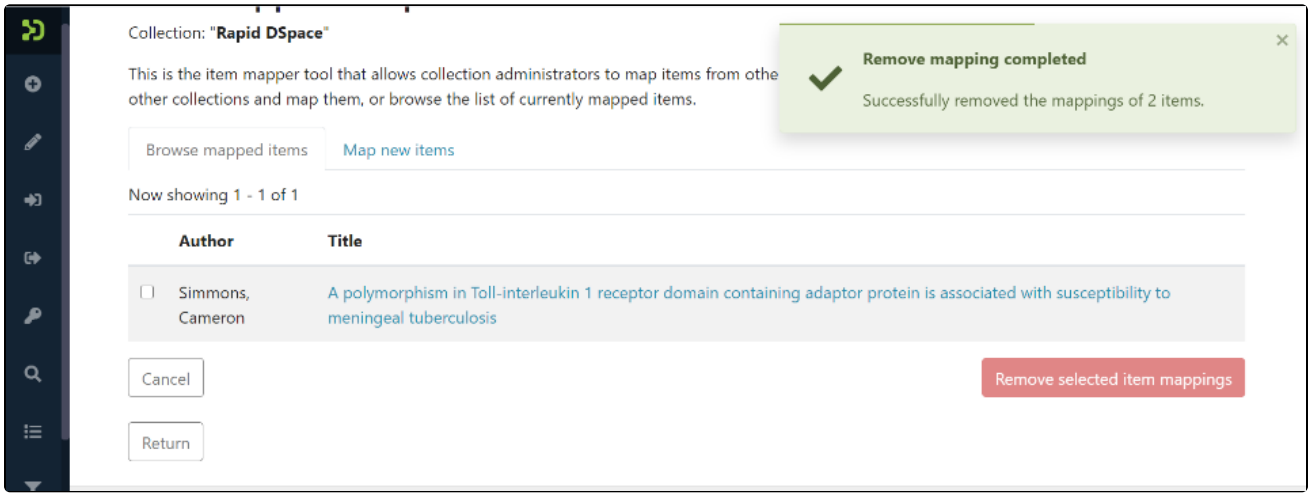
Step 22: The item mapper tab allows users to map items from other collections and manage mapped items.



Step 23: You'll see items mapped with collections under the "Browse mapped items" tab. Click on the checkbox appearing with each item to select the item (s) required to be unmapped.

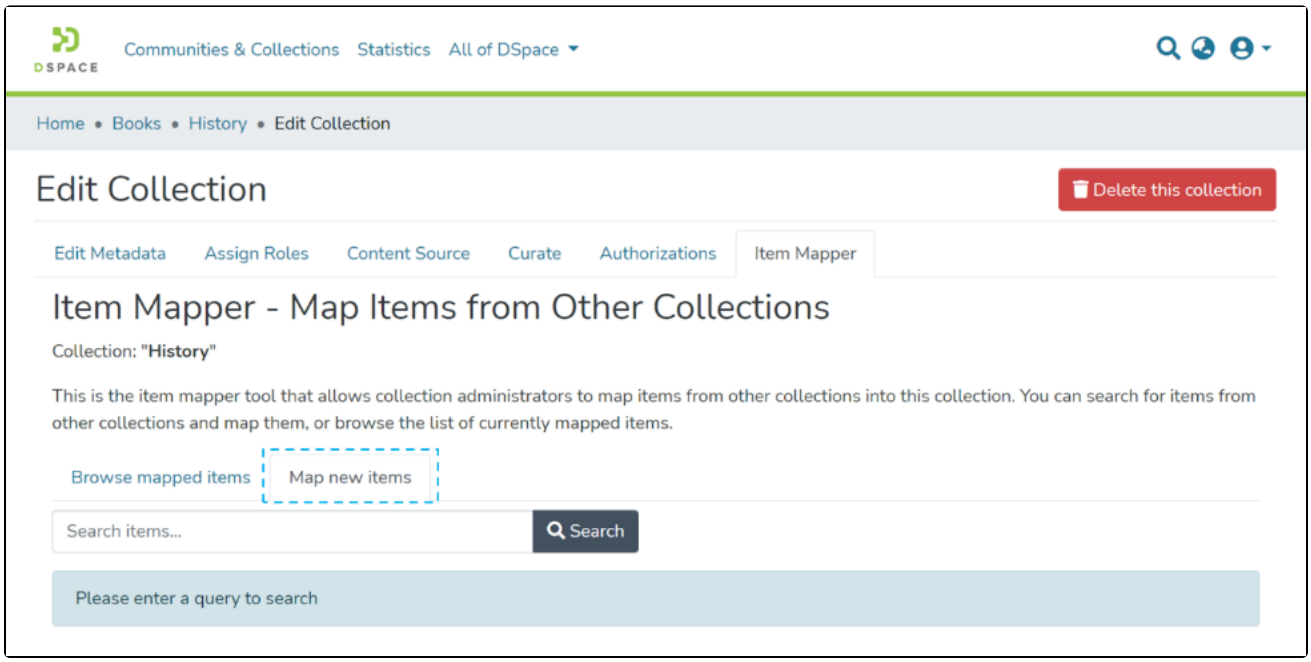


Step 24: After selecting items required to be unmapped, please click on “Remove selected item mappings” to complete the operation. Click on the “Cancel” button on the left of “Remove selected item mappings” to cancel the process.



Map new items

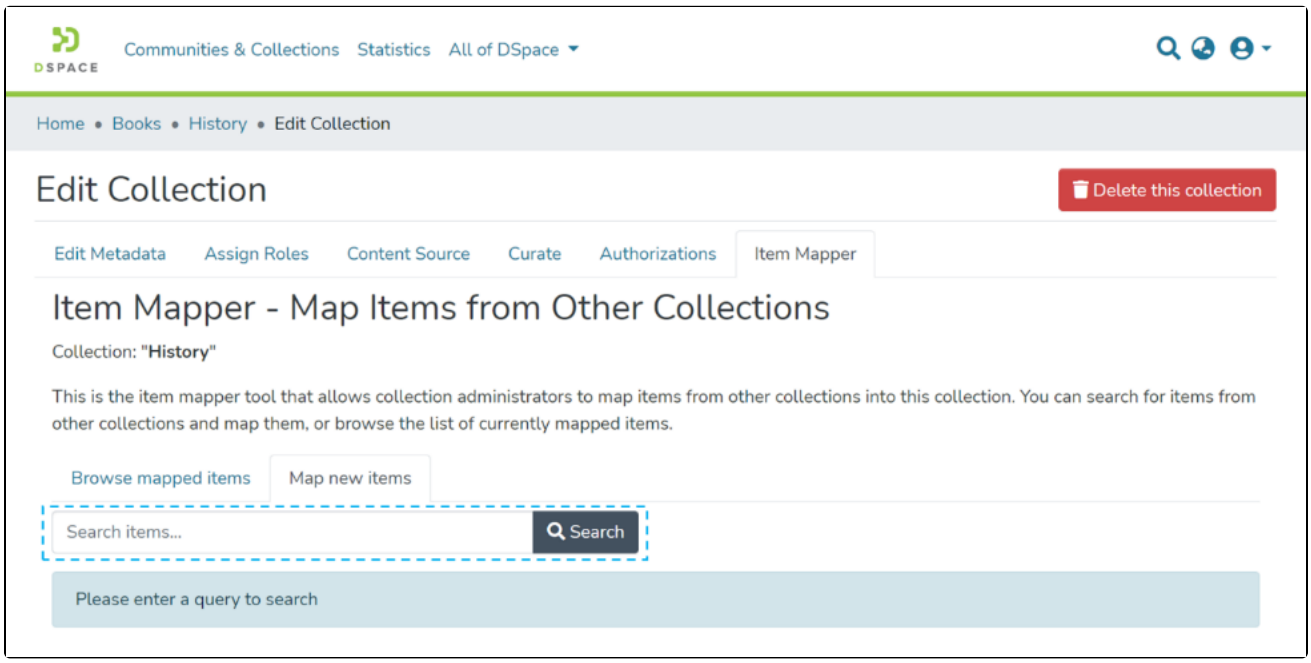
Step 25: Click on the “Map new items” to search items for mapping.



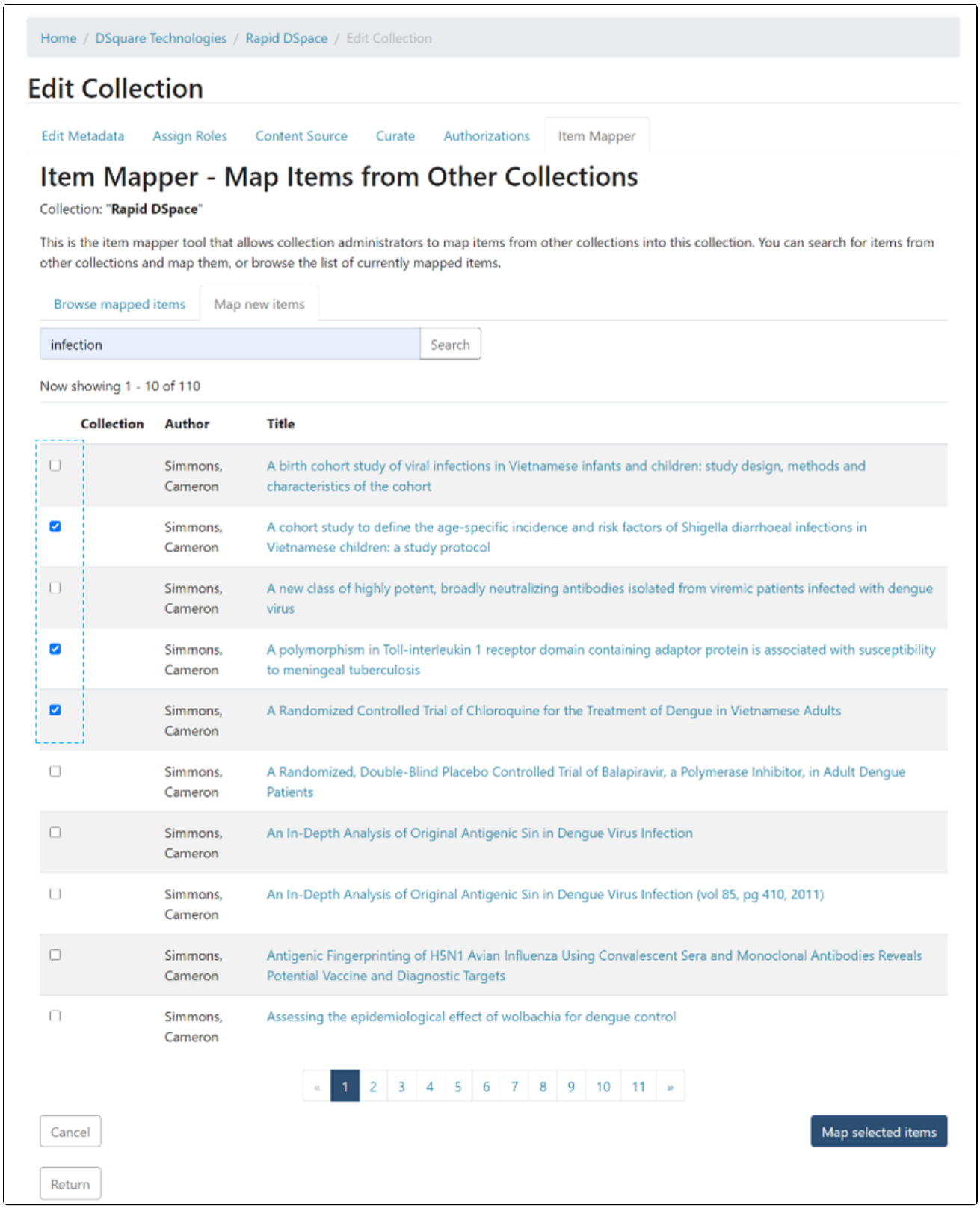
Step 26: After confirming the non-existence of the target item in the existing mapped items list, please click on “Map new items.” Then, enter keywords /keyphrases in the search field to search for target items.

Click on the “Search” button as highlighted on the below screen.

You must know that you can enter keywords or keyphrases from any metadata field. Search field under “Map new items” work exactly like the basic search field of DSpace.

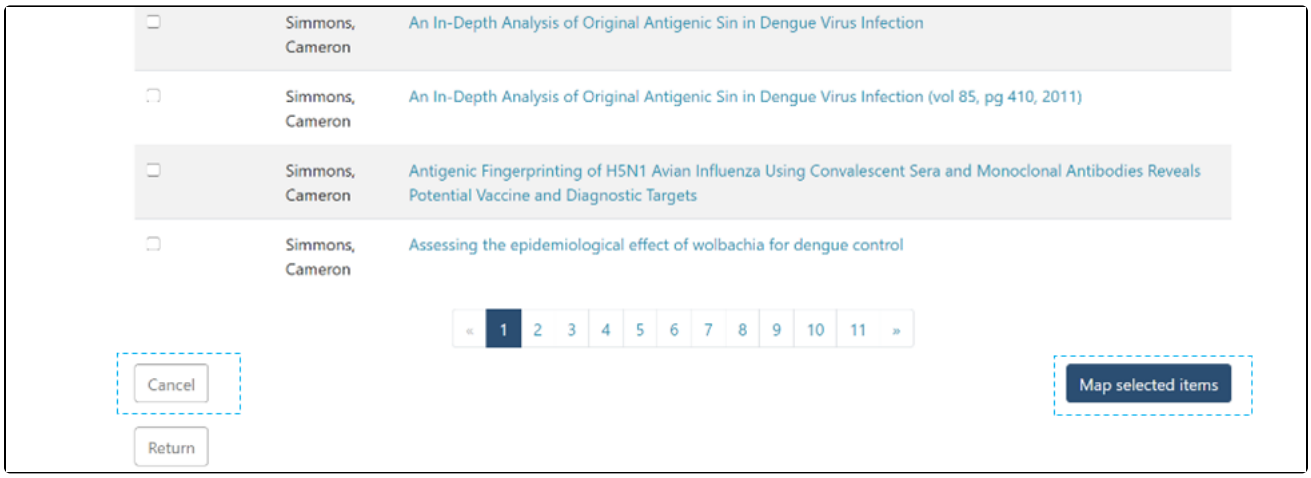


Step 27: Users can select target items from the search results by clicking the checkbox appearing with items.

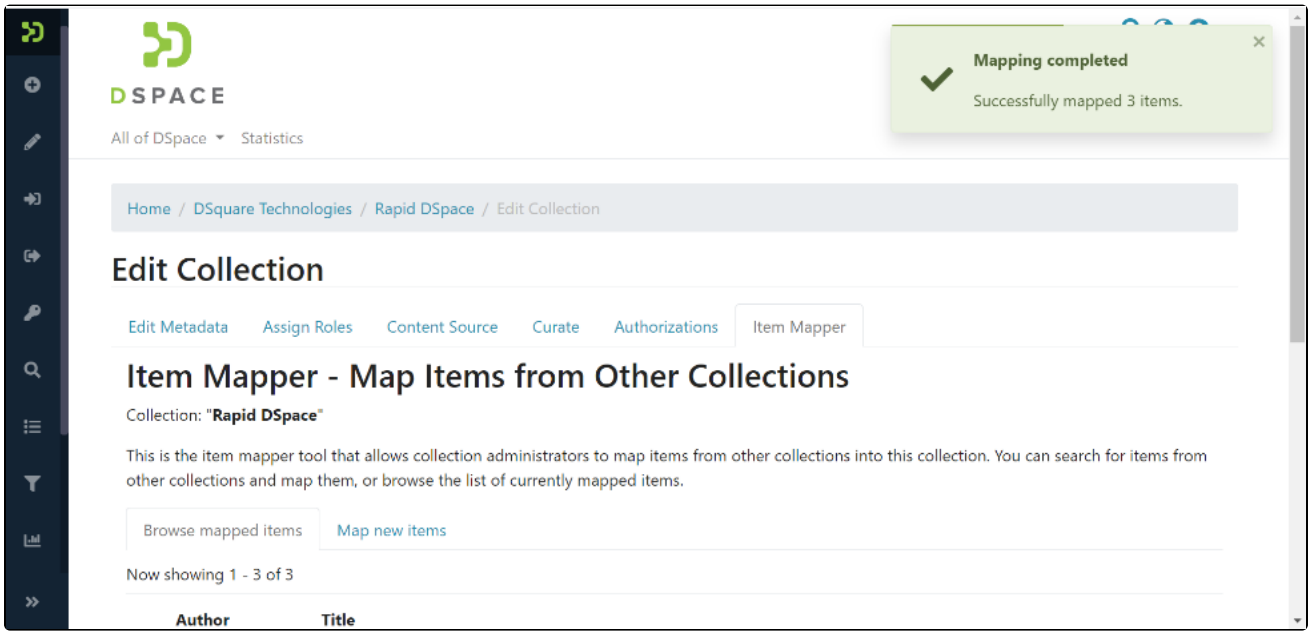


Step 28: After selecting target items, please click on the “Map selected items” button at the bottom of the page to complete the item mapping process.

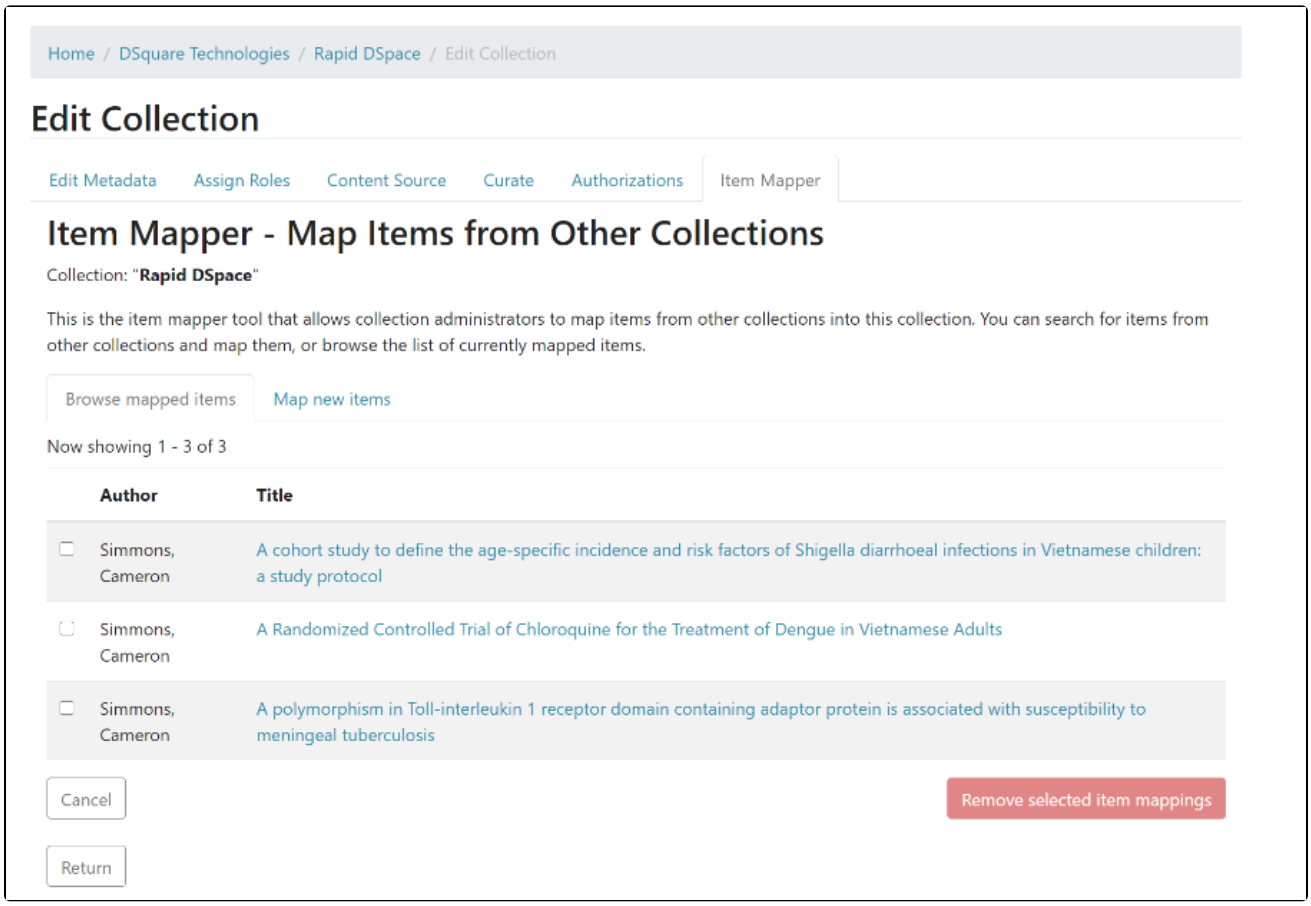
Click the “Cancel” button to cancel the activity and return to the collection edit page.



Step 29: A prompt confirming successful mapping of items will appear upon completing the task, as demonstrated below.



Mapped items will appear in the collection and under the “Browse mapped items” tab, as demonstrated below.


Home / DSquare Technologies / Rapid DSpace / Edit Collection
Edit Collection
Edit Metadata
Assign Roles
Content Source
Curate
Authorizations
Item Mapper
Item Mapper - Map Items from Other Collections
Collection: "Rapid DSpace"
This is the item mapper tool that allows collection administrators to map items from other collections into this collection. You can search for items from other collections and map them, or browse the list of currently mapped items.
Browse mapped items
Map new items
Now showing 1 - 3 of 3
Author
Title
Simmons, Cameron
A cohort study to define the age-specific incidence and risk factors of Shigella diarrhoeal infections in Vietnamese children: a study protocol
Simmons, Cameron
A Randomized Controlled Trial of Chloroquine for the Treatment of Dengue in Vietnamese Adults
Simmons, Cameron
A polymorphism in Toll-interleukin 1 receptor domain containing adaptor protein is associated with susceptibility to meningeal tuberculosis
Cancel
Remove selected item mappings
Return

Export Collection

DSpace provides a feature of exporting metadata of any collection into CSV format. Users can utilize this CSV file for multiple purposes like creating ad-hoc reports, importing metadata into other systems, or for any use case as per its requirements.

- Audience
- Exporting a collection

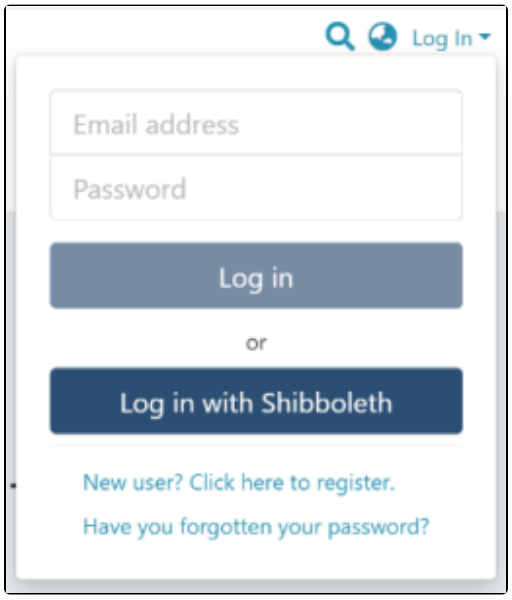
Audience

1. Repository Administrator
2. Community Administrator
3. Collection Administrator
4. Basic user

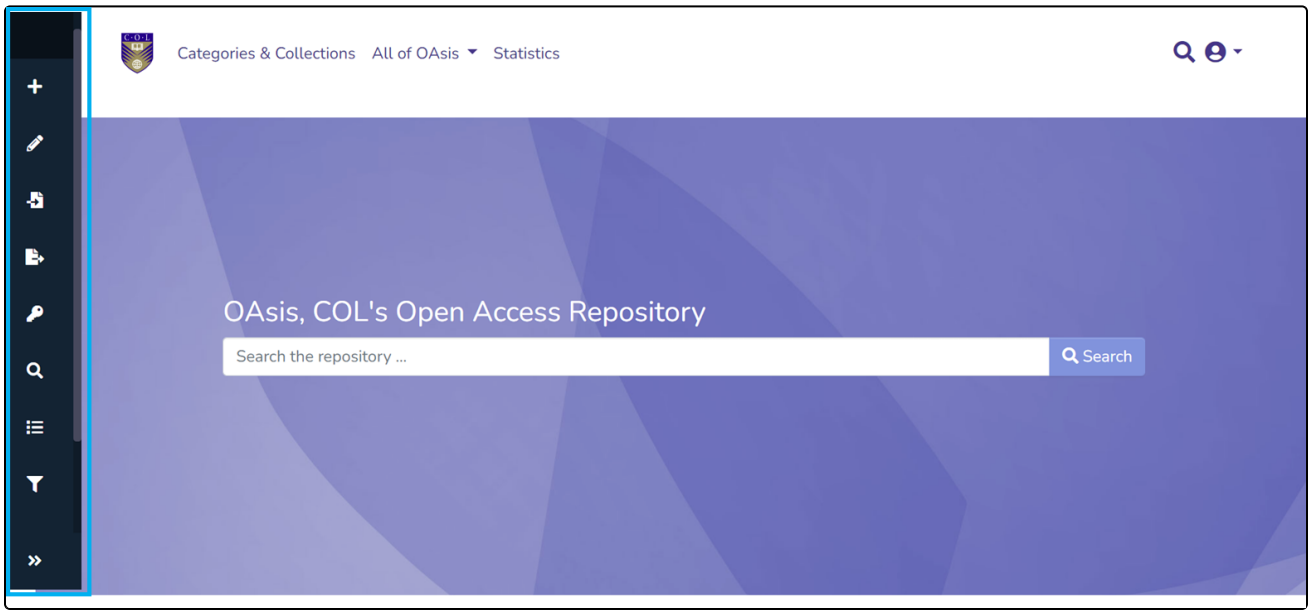
Exporting a collection

Users log in using their log-in credentials and follow the steps mentioned below to export a collection's metadata.

Step 1: Go to the DSpace home page and click on the "Log In" link at the top right corner of the screen, as illustrated below.

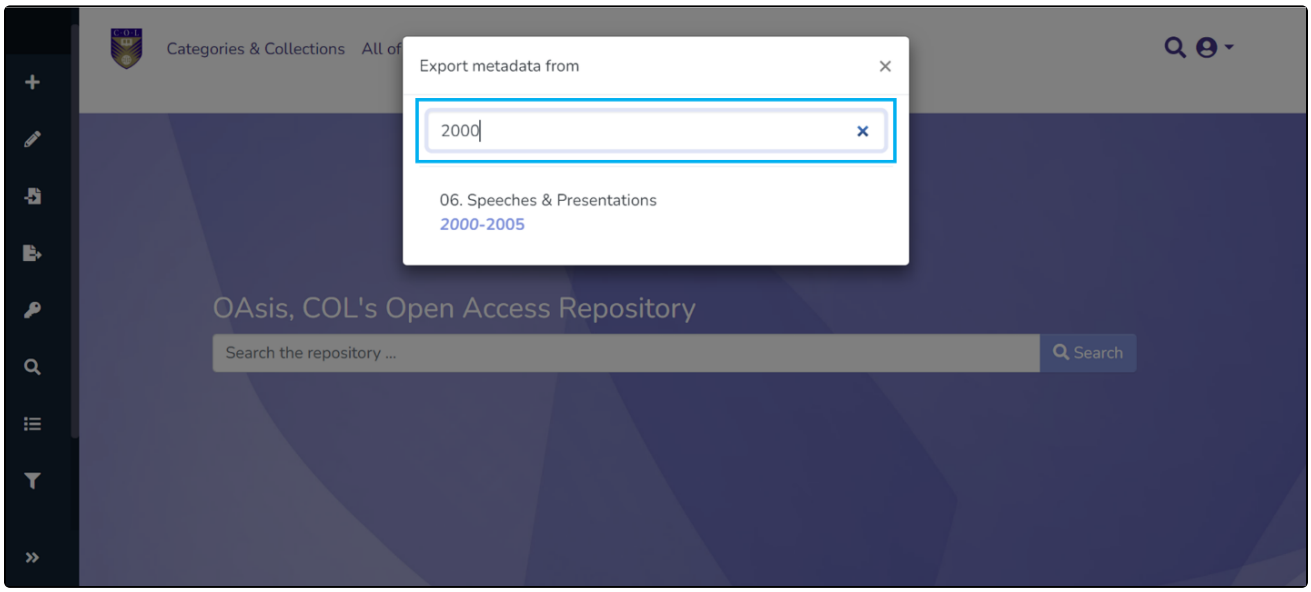


Step 2: Users will see the admin menu on the left-hand side of the screen, as highlighted in the illustration.

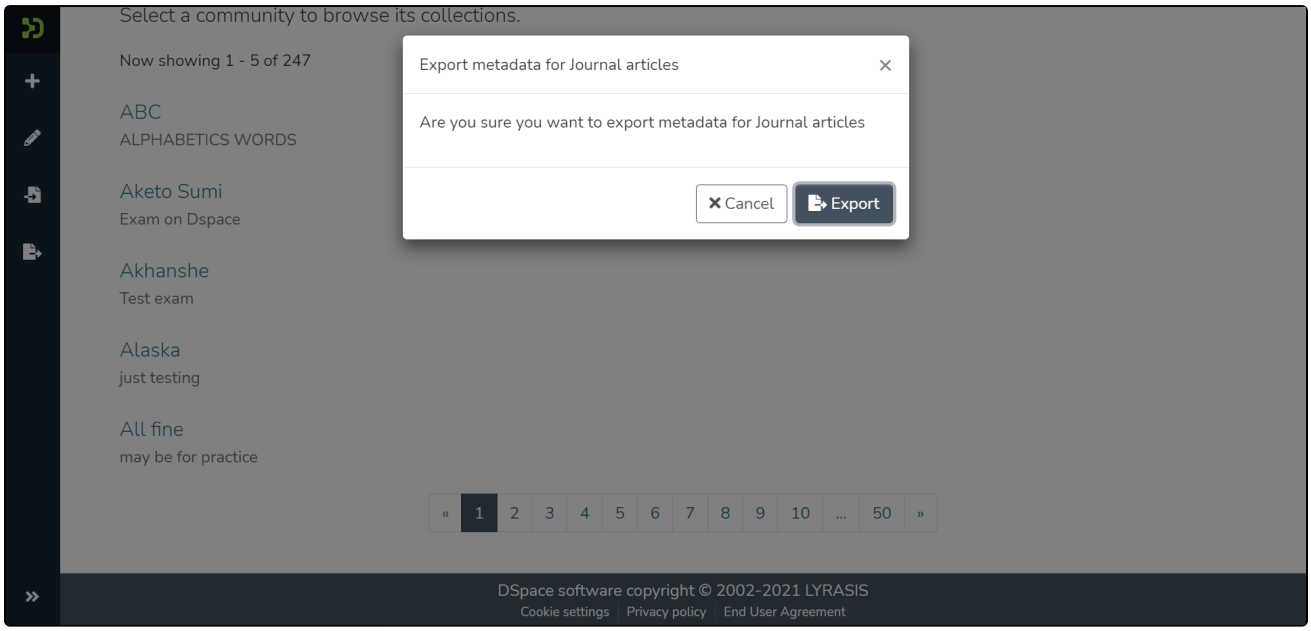


Step 3: Rollover your cursor over the Export menu and click on metadata.

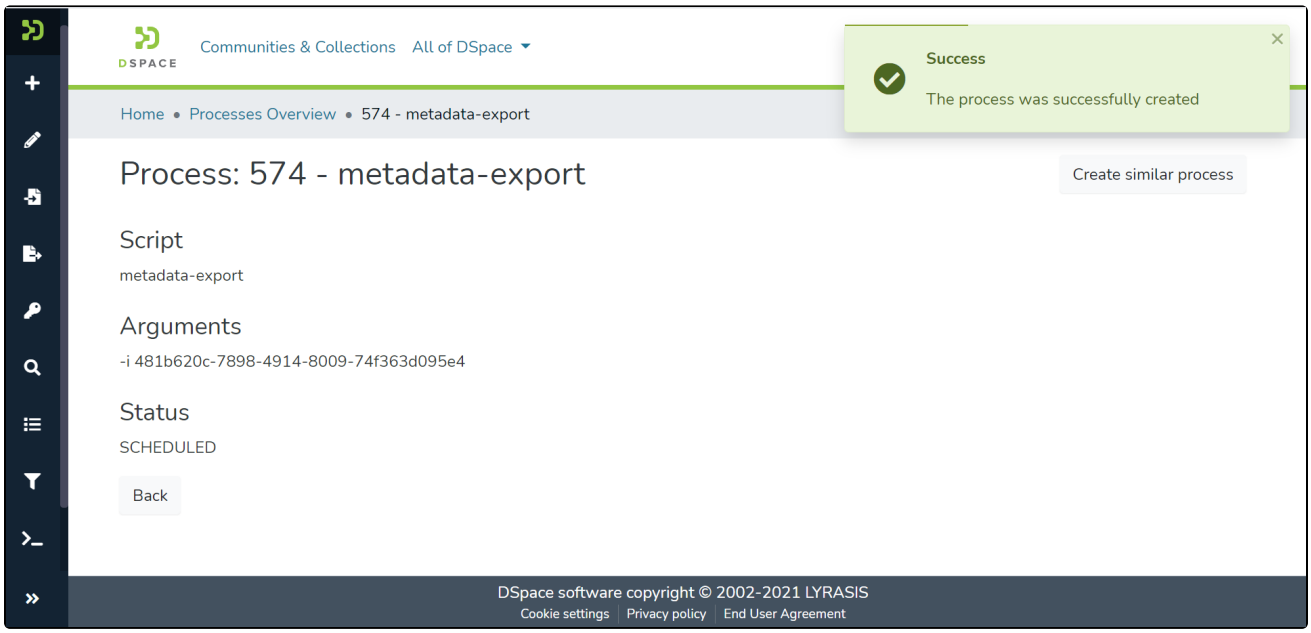
Step 4: Type the collection's name in the textbox and click on the target collection from the list appearing in the popup.



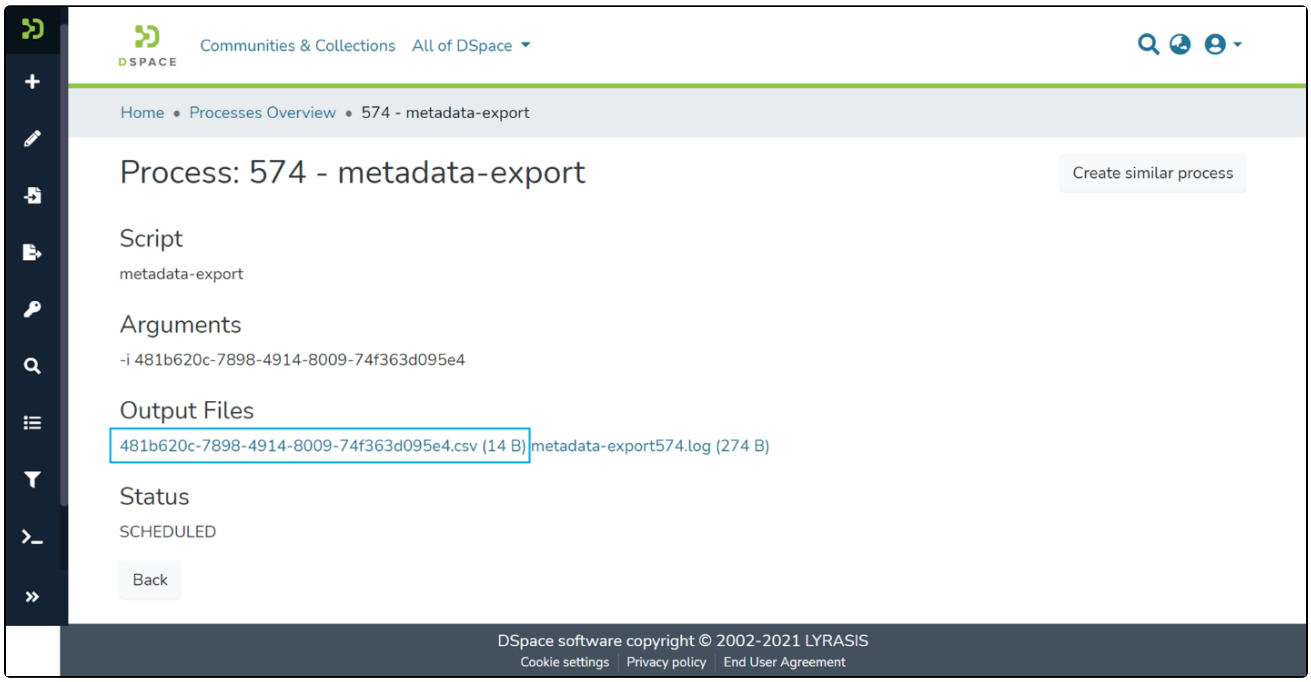
Step 5: Click on the “Export” button in the popup to continue with the item metadata-export or click the “Cancel” button to cancel the process.



Users will see the success prompt confirming the creation of the export process upon successful completion of the process, or else the application will show the failure promptly.

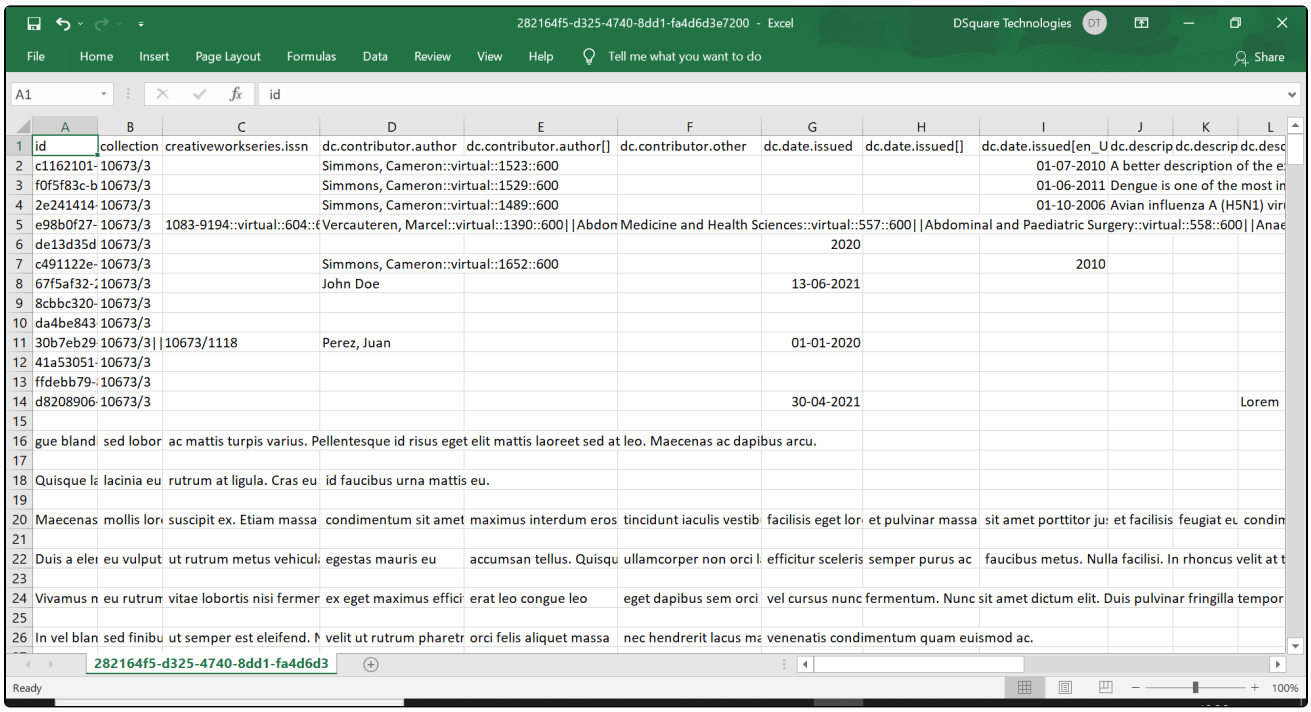


Step 6: Users will be redirected to the metadata export page with a csv download link, as highlighted in the screenshot below. Click on the link to download the file.

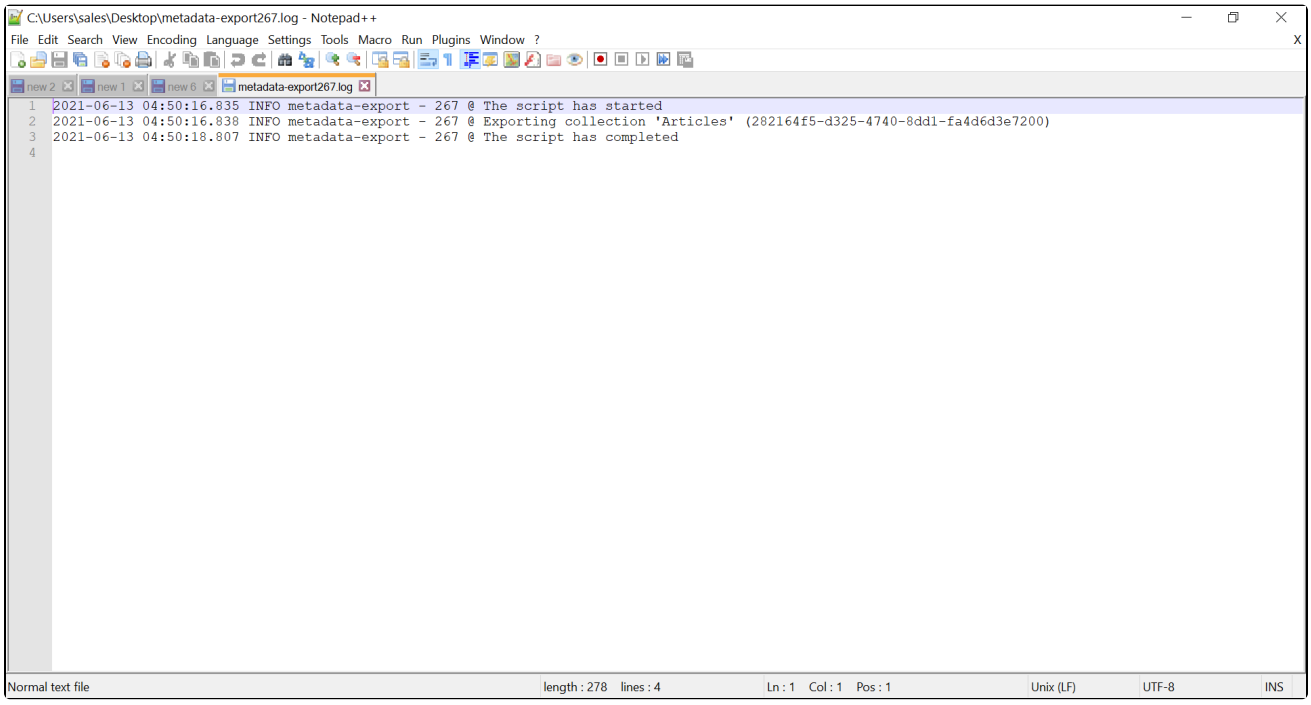


Users can perform the following actions on this page:

- Click on the CSV file link to download the metadata CSV. This file contains metadata of items stored in the exported collection.



- Click on the log file link to download. The Logfile contains details of steps performed during the export job.



Community Management

The community is the primary storage level in the DSpace’s storage hierarchy that holds sub-community and collections. This document provides an overview of creating, editing, and deleting a community. The documentation below assumes that the user has the relevant authorizations. For example, the admin menu and edit buttons would appear to a user having community administration permission.

If you’re unsure about community administration permissions assigned to your account for the target community, contact your system administrator.

- Create a Community
- Delete Community
- Edit Community

Create a Community

- Audience
- Create Community

A Community is the primary storage level in the DSpace's storage hierarchy that holds Sub-Communities and Collections. This document provides an overview of creating, editing, and deleting a Community. The documentation below assumes that the user has the relevant authorizations. For example, the admin menu and edit buttons would appear to a user having community administration permission.

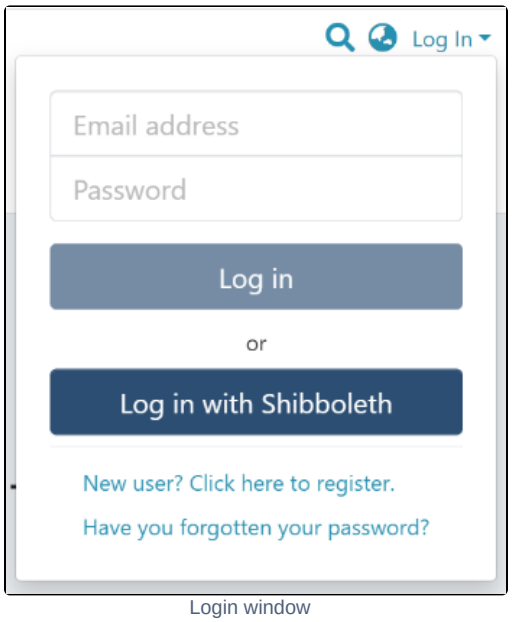
If you're unsure about community administration permissions assigned to your account for the target community, contact your system administrator.

Audience

1. Repository Administrator
2. Community Administrator

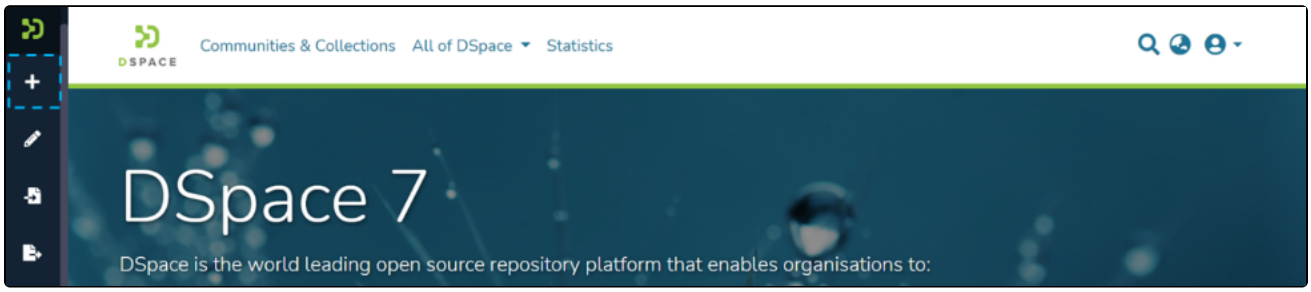
Create Community

Step 1: Login using your credentials

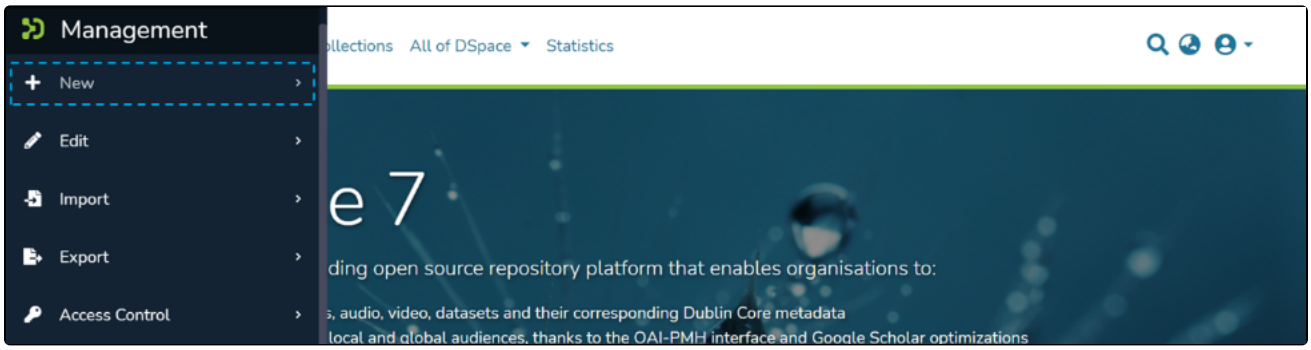


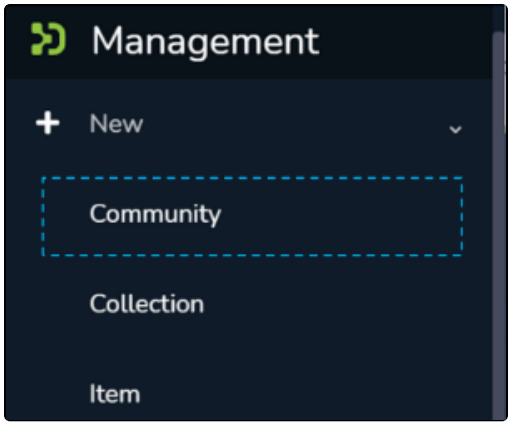
Login window

Step 2: Rollover your cursor on the "+" sign



Step 3: Click on the "New" link and click on "Community" to proceed with the community creation.





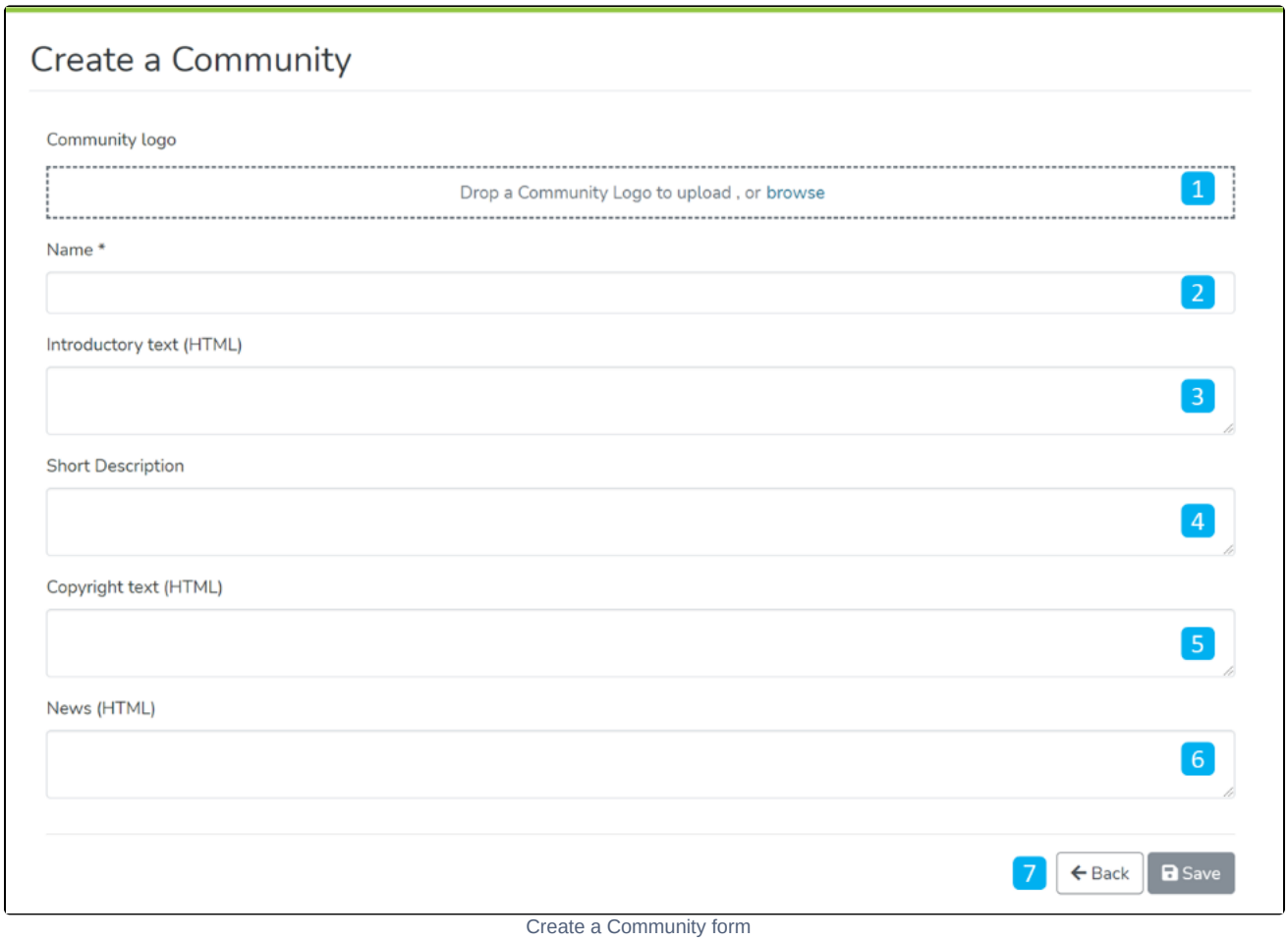
Step 4: A popup providing the option to either create a Parent community or a sub-community will appear, with a list showing existing communities. Create your new community by either:

1. Click on “Create a new top-level community” to create a top-level community.
   … or…
2. Typing the name of the existing parent community in the search field to add a sub-community within. Upon typing a few letters of the community’s name, a list of communities having those letters or words will appear. Click on the community name to create a sub-community.

Step 5: As per the user’s choice in the previous step, the application will open the create community or create a sub-community form to populate information regarding the community’s profile. Below is the explanation of the information that needs to be populated on this form.

It is important to understand that both “Create Community” and “Create Sub-community” forms are identical. The critical difference between both is that the “Create Community” form helps create a top-level community while the latter helps create a sub-community within a community or a sub-community.

The description provided below the following screenshot remains identical for both Community and Sub-community creation.



Create a Community form

1. Community logo – Select the community's logo by clicking on the 'browse' link to select an image file. It is advisable to maintain uniform dimensions of the logo across the repository.
2. Name – The community's name is a mandatory field marked with '*'.
3. Introductory text (HTML) – Users can add introductory text providing an overview of the contents stored in the community. One can utilize HTML tags to format the text or continue entering plain text content.
4. Short Description – This field can have a one-line description of the community that displays with the community name in the list of communities on the parent community page (or on the DSpace's in the case of a top-level community.
5. Copyright text (HTML) – Users can enter copyright information here. Fields marked with (HTML) support HTML tag-based formatting.
6. News (HTML) – Enter news about this community. Users can update this by regularly going to this section via the editing community.
7. Action Buttons – Users can click on the appropriate button as determined. Clicking on the Save button will add the community to the repository.

Step 6: Click on the 'Save' button to complete the Community creation. A success prompt will pop up upon community creation, and the user will be re-directed to the community homepage.

Screenshot of a saved Community with confirmation banners

Success prompt upon community creation

Example Community homepage

Community homepage

Delete Community

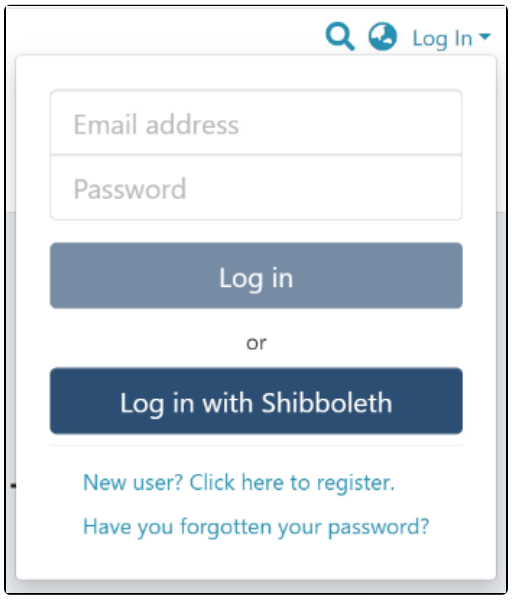
- Audience
- Edit Community

Audience

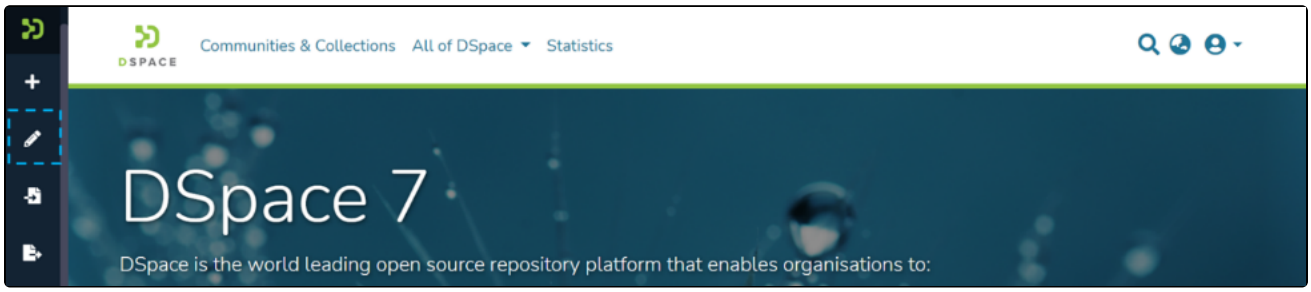
1. Repository Administrator
2. Community Administrator

Edit Community

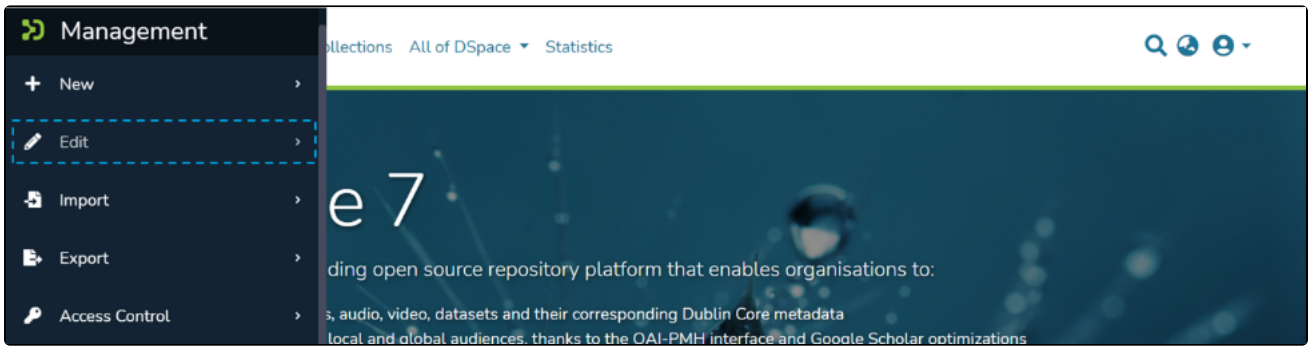
Step 1: Login using your credentials

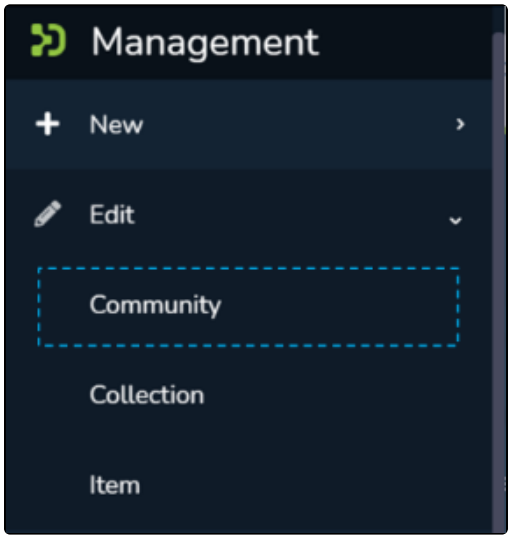


Step 2: There are multiple ways to navigate to the controls to delete a community. One of them is by going to the target community and clicking on the button with the pencil icon next to the community title ie the ‘Edit community’ button. Alternatively, follow the steps provided here.

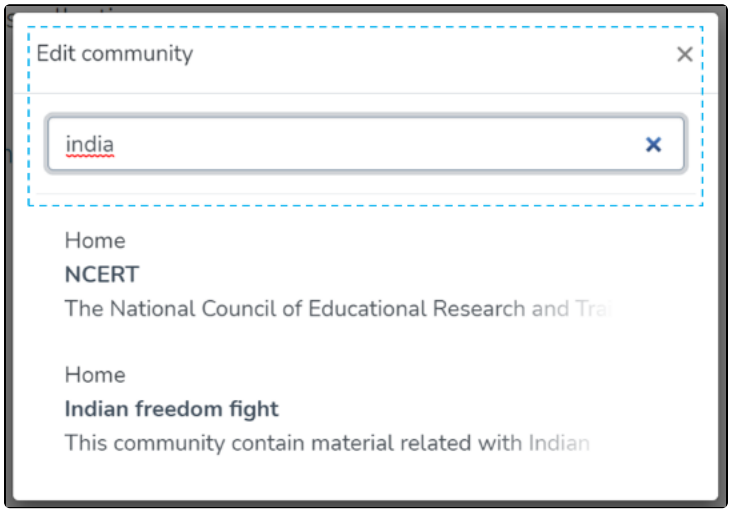


Step 3: Click on “Edit” and click on “Community” to proceed with the edit community process.

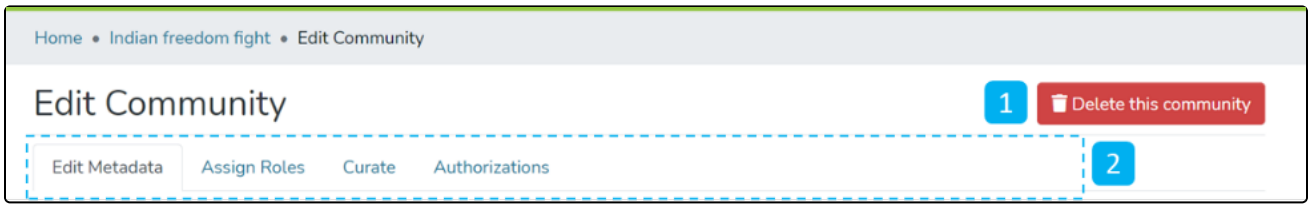




Step 4: A popup showing a search box and a list of communities will appear. Type the name of the community you want to edit in the search field. Upon typing a few letters of the community's name, a list of the communities having those letters or words will appear. Click on the target community to initiate editing.

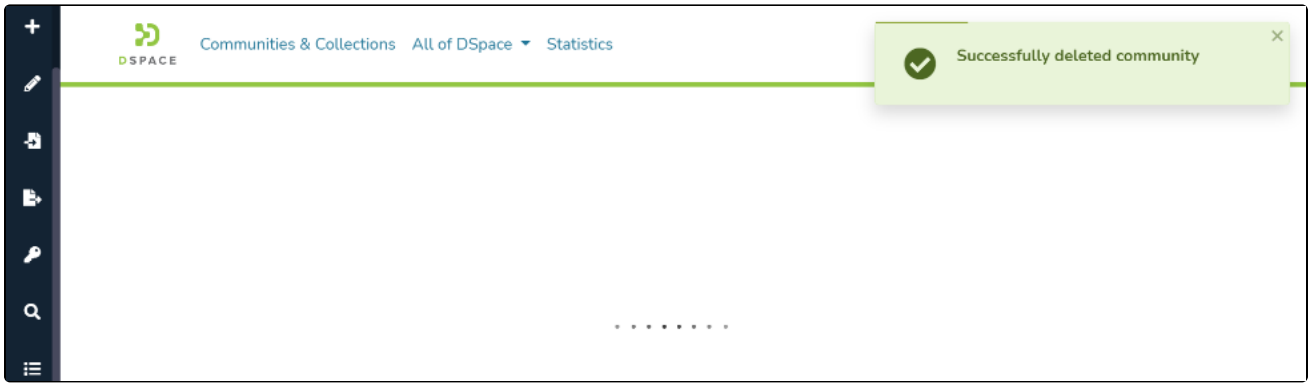


Step 5: The application will take the user to the edit community form. To initiate the community deletion process, the user needs to click on the 'Delete this community' button.



Step 6: Click on the Confirm button to continue with the community deletion or click on the Cancel button to return to the previous page.

Users will be re-directed to the homepage of DSpace upon successful completion of the community delete, and a popup confirming the community deletion will appear.



Edit Community

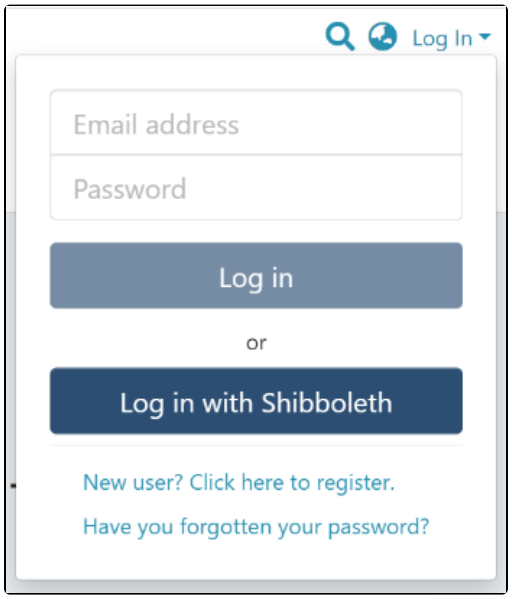
- Audience
- Edit Community
- Edit Metadata
- Assign Roles
- Curate
- Authorizations

Audience

1. Repository Administrator
2. Community Administrator

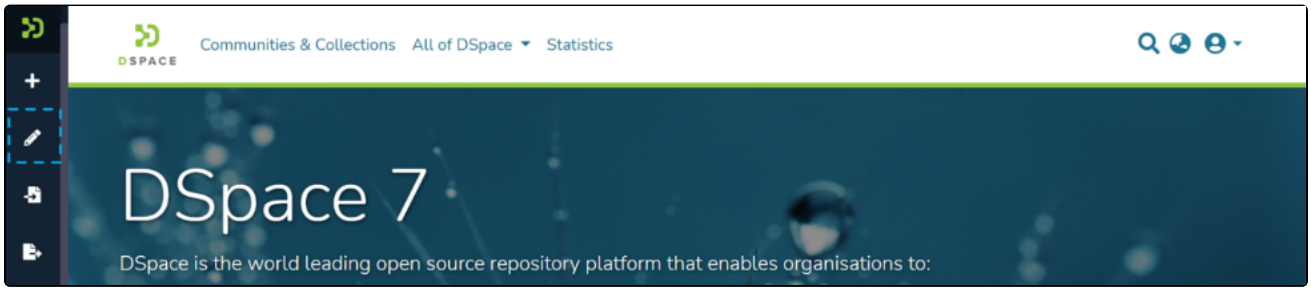
Edit Community

Step 1: Login using your credentials

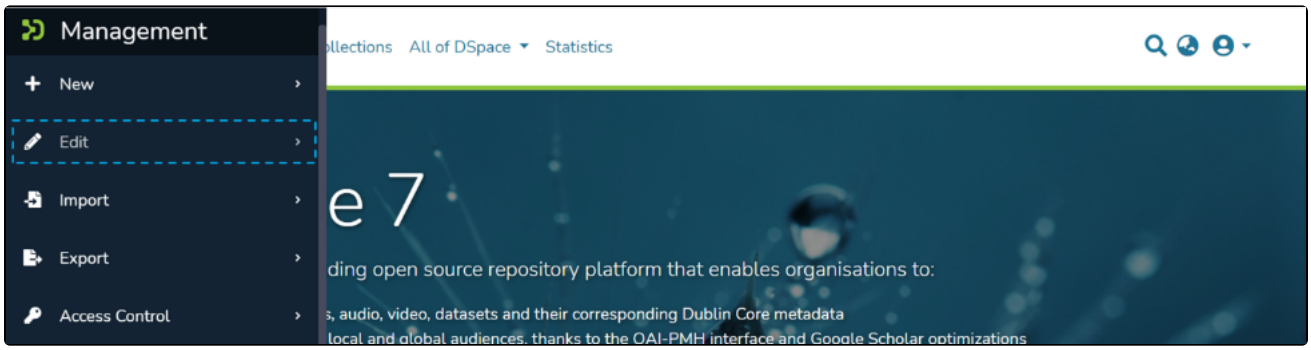


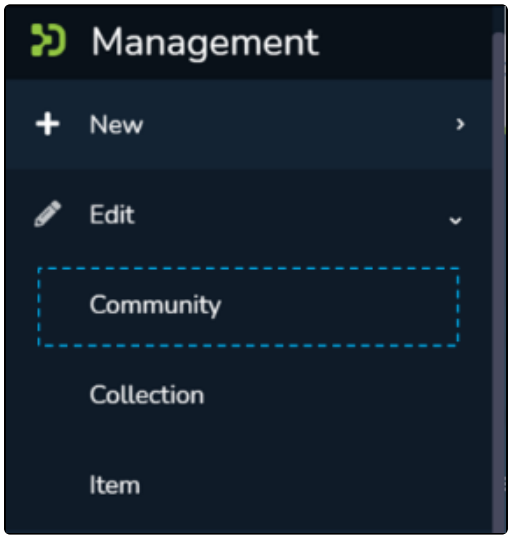
Step 2: There are multiple ways to start editing a community. One of them is by going to the target community and clicking on the Edit button, the button with the pencil icon, beside the page title. Alternatively, follow the steps provided here.

Rollover cursor on the pencil icon in the admin menu.

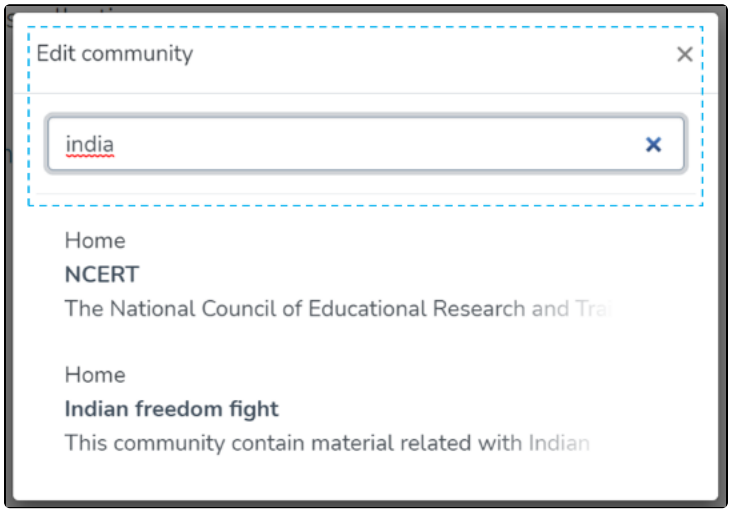


Step 3: Click on “Edit” and click on “Community” to proceed with the edit community process.

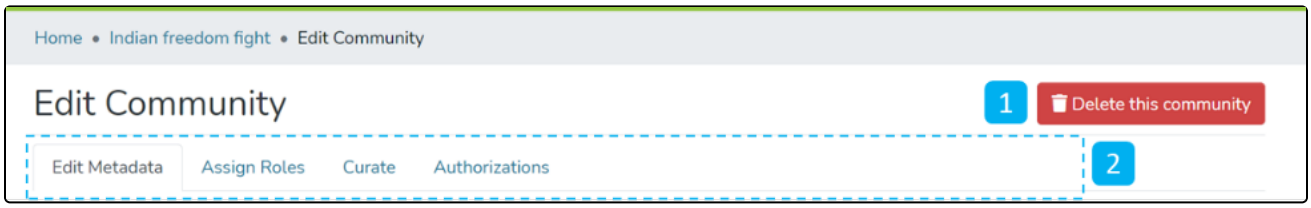




Step 4: A popup showing a list of the communities in the DSpace and a search box will appear. If you are already on the page of the community, it will appear at the top of the list, so you can select it by clicking on it. Otherwise, type the name of the community you want to edit in the search field. Upon typing a few letters of the community's name, a list of the community(ies) having those word(s) will appear. Click on the target community to initiate editing.



The application will take the user to the edit community form to perform various actions to edit the community. Each tab is explained in a separate process in this document.



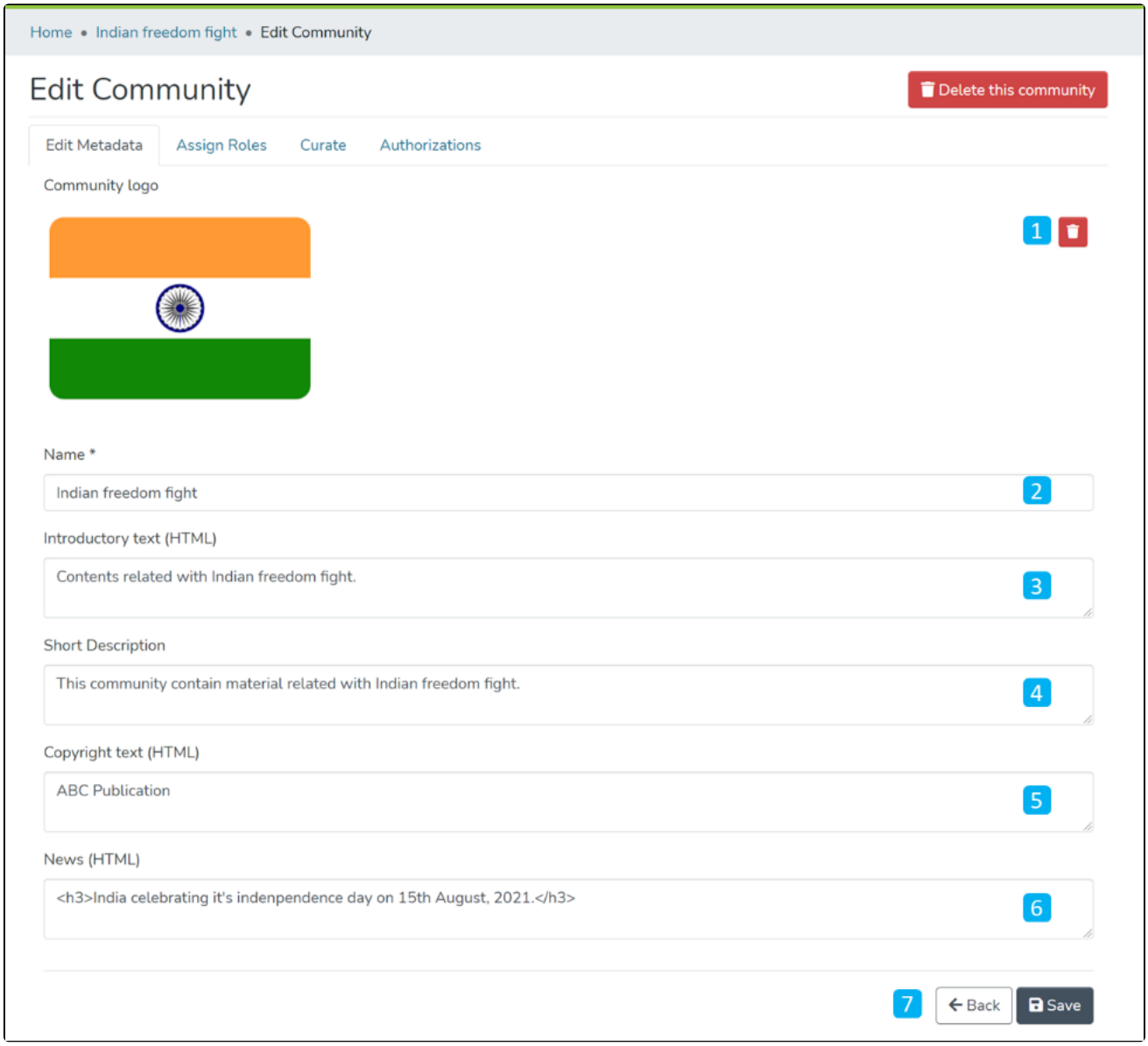
1. Delete this community – The button provided for deleting the community. Detailed steps are explained in the latter part of this page.
2. Tabs – Edit community has a variety of functions, which are grouped logically across various tabs. Below is the summary of these tabs

    a. Edit Metadata – Tab covers activities related to editing the community's profile information.
    b. Assign Roles – This tab allows users to create specific roles for the community, usually, the role of Administrator of the community, see further detail below.
    c. Curate – Users can set up various workflows related to content curation in this tab
    d. Authorizations – Under this tab, users can manage various groups and their different access rights in the community, for example, this tab could be used to grant an individual the administrator role, see further detail below.

Edit Metadata

The Edit Metadata tab allows users to update the community's profile-related information, a.k.a. community metadata.

Various actions on this tab are explained immediately after the Edit Metadata illustration is added below.



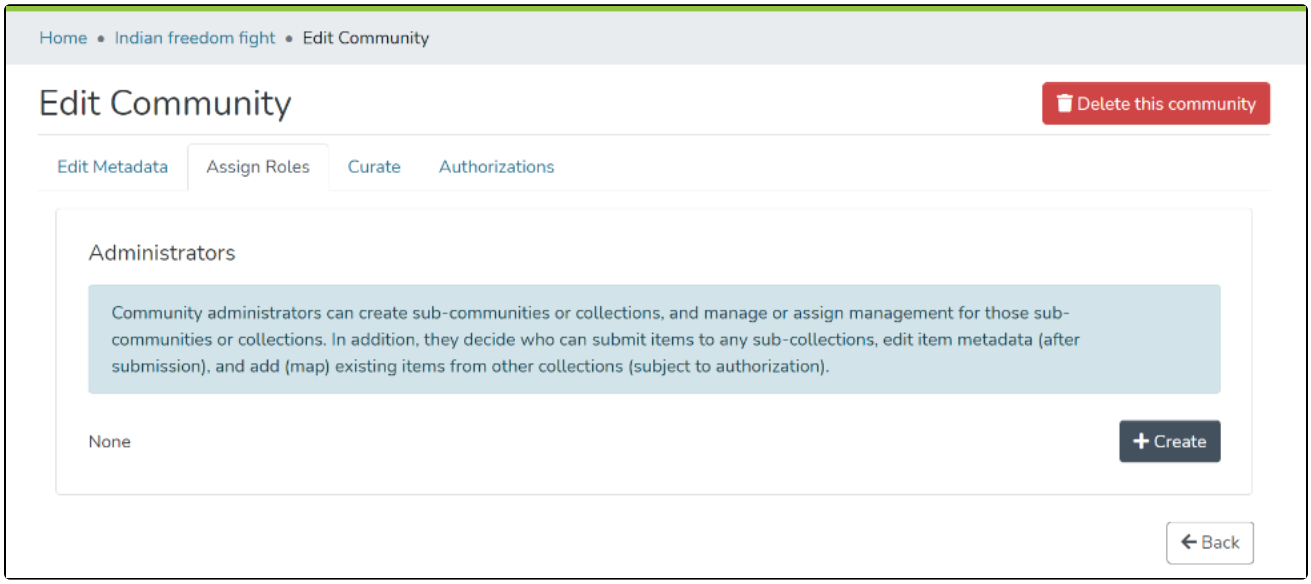
1. Community logo – Click on the delete button to remove the existing logo. If no logo exists, then a widget allowing the user to add a logo is displayed here.
2. Name – Update the existing community's name in this field.
3. Introduction text (HTML) – Update introductory text if already added or can add new text. One can utilize HTML tags to format the text or continue entering plain text content.
4. Short Description – Update the description of the community or add a fresh short description for the community.
5. Copyright text (HTML) – Update copyright-related information in this field. This is usually displayed at the foot of the community landing page. Fields marked with (HTML) support HTML tags-based formatting.
6. News (HTML) – Add/Update news specific to this community in the field. This is usually displayed with the heading 'News', underneath the community's introductory text, and above the list of collections and sub-communities.
7. Action Button – Clicking on the Save button will update the metadata information for the community.

Click on the 'Save' button to save the information updated in the 'Edit Community' tab. A success prompt will appear, confirming the successful edit of the community.

Assign Roles

This tab allows authorized users to create a Community administrator role. Click on the "create" button to assign a community administrator role.

The roles available on this tab are explained below this illustration.



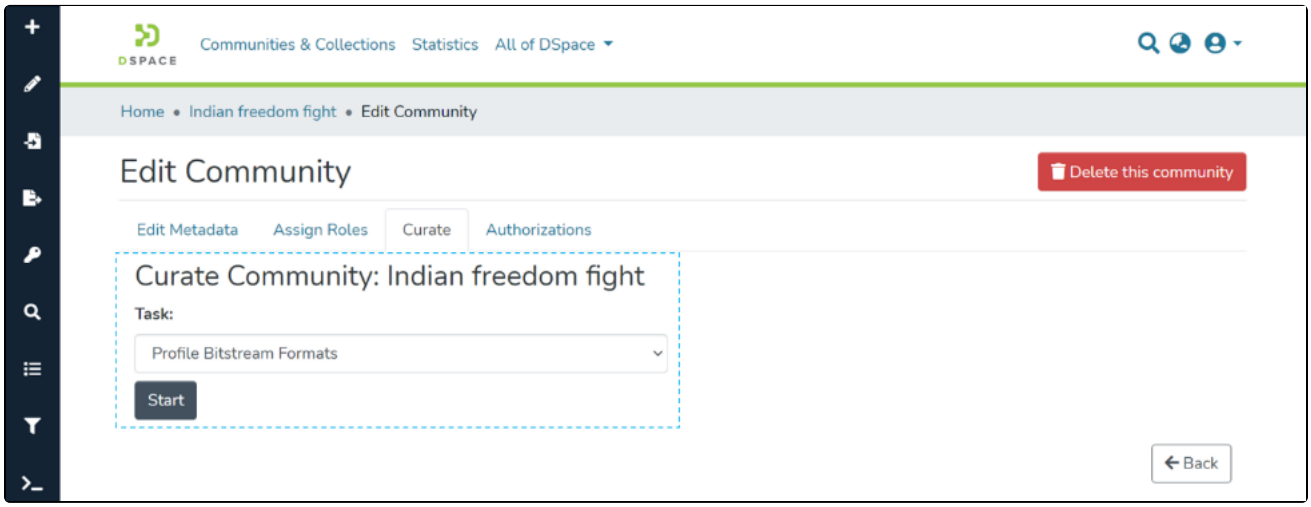
Administrators - Community administrators can create and manage sub-communities and collections. This user profile can also assign rights to edit item metadata and map existing items from other collections.

Curate

This tab provides various workflows for curating items stored in the community. Below are standard flows, and there can be customized curation workflows as well

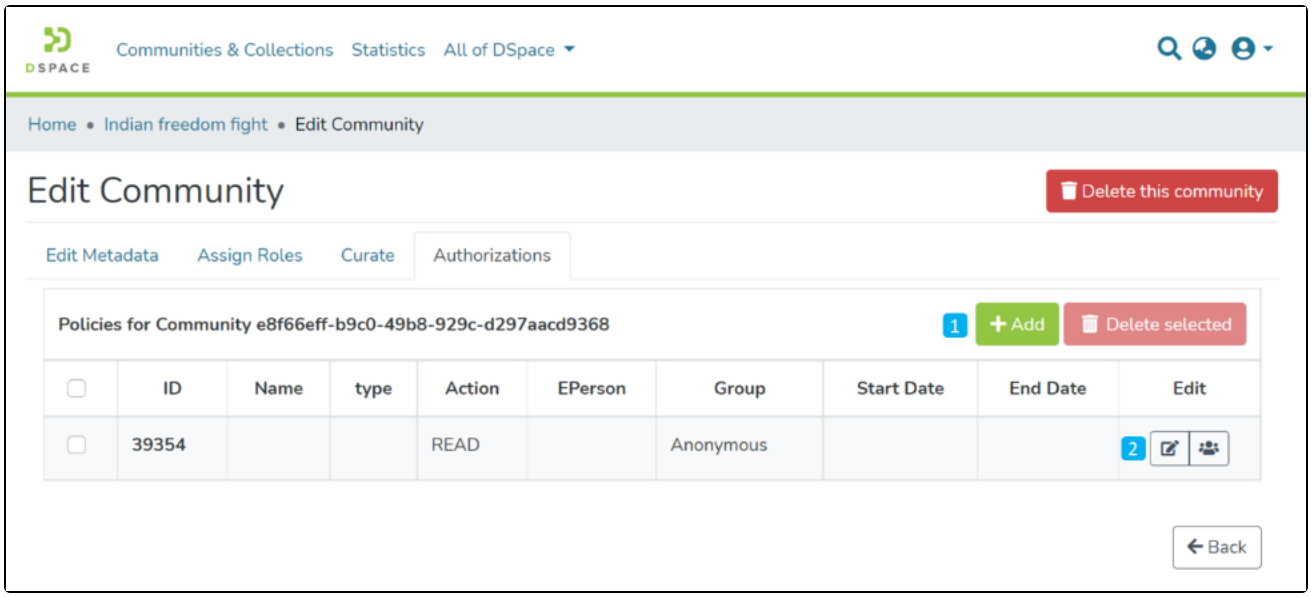
1. Profile bitstream formats
2. Check for Required Metadata
3. Check Links in Metadata

Users must select a workflow from the dropdown list and click the “Start” button to initiate the curation process.



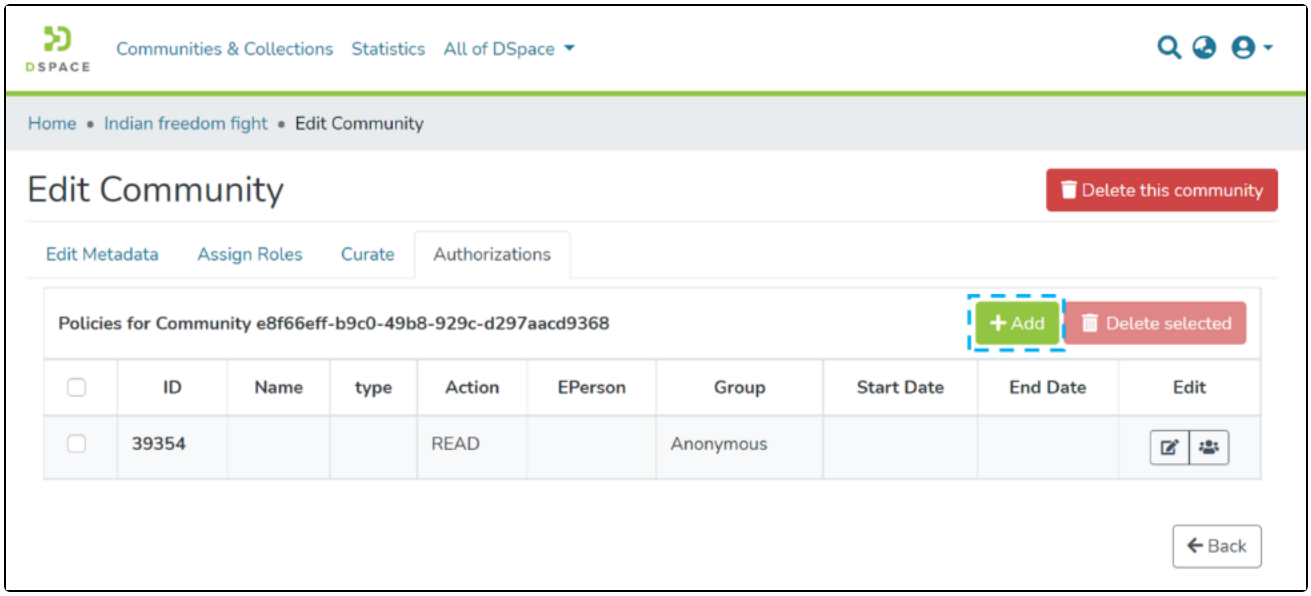
Authorizations

Users can view and edit community resource policies defined for the community, in the Authorizations tab. Users can create policies in addition to the standard policies created from the Assign Roles tab. Following are the key actions in this tab.

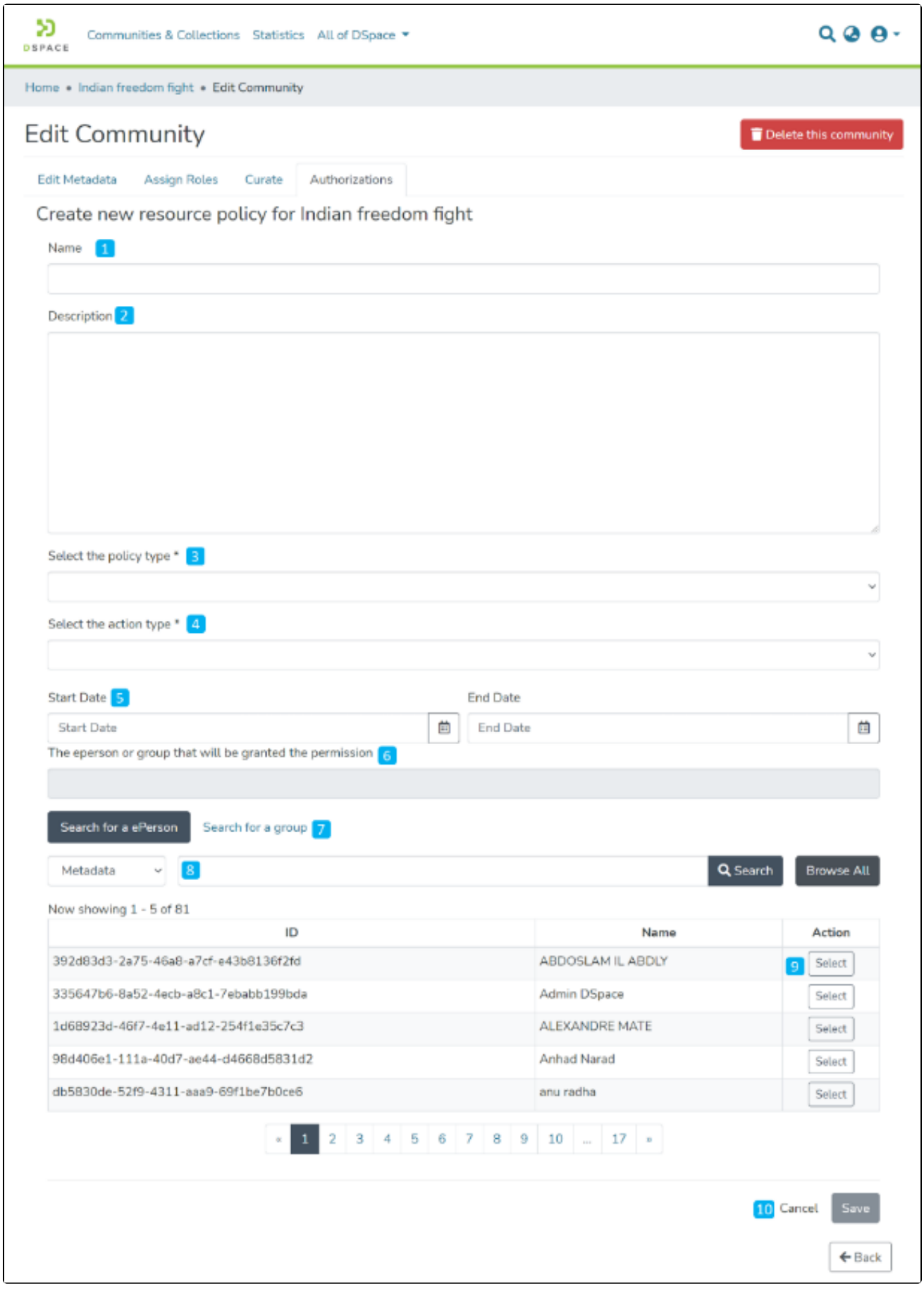


1. Manage Policies
   – Click on the 'Add' button to create a new resource policy or select policies from the table, see further detail below.
   - Alternatively, click on the 'Delete selected' button for a batch deletion of the policies.
2. Edit policy and members in a policy – Click on the Edit icon to edit an individual policy or click on the group icon to edit the user group (eg. to add or remove individual ePersons).

Click on Add button to create a new Authorization policy



Users can enter the information to create the policy and click on the submit button. Please see the description of each field followed by the below screenshot.


DSPACE
Communities & Collections
Statistics
All of DSpace
Home • Indian freedom fight • Edit Community
Edit Community
Delete this community
Edit Metadata
Assign Roles
Curate
Authorizations
Create new resource policy for Indian freedom fight
Name 1
Description 2
Select the policy type * 3
Select the action type * 4
Start Date 5
End Date
Start Date
End Date
The eperson or group that will be granted the permission 6
Search for a ePerson
Search for a group 7
Metadata
8
Search
Browse All
Now showing 1 - 5 of 81
ID
Name
Action
392d83d3-2a75-46a8-a7cf-e43b8136f2fd
ABDOSLAM IL ABDLY
9
Select
335647b6-8a52-4ecb-a8c1-7ebabb199bda
Admin DSpace
Select
1d68923d-46f7-4e11-ad12-254f1e35c7c3
ALEXANDRE MATE
Select
98d406e1-111a-40d7-ae44-d4668d5831d2
Anhad Narad
Select
db5830de-52f9-4311-aaa9-69f1be7b0ce6
anu radha
Select
« 1 2 3 4 5 6 7 8 9 10 ... 17 »
10
Cancel
Save
Back

1. **Name:** Enter the Policy name in this field.
2. **Description:** Enter the Policy description here for future reference and understanding of other users.
3. **Select the policy type:** The user can select one of the following policy classification types from the list
    a. TYPE_SUBMISSION: a policy in place during the submission
    b. TYPE_WORKFLOW: a policy in place during the approval workflow
    c. TYPE_INHERITED: a policy that has been inherited from a container (the community)
    d. TYPE_CUSTOM: a policy defined by the user during the submission or workflow phase
4. **Select the action type:** The user can select one of the following actions from the dropdown list. For example, select “READ” to assign read rights to the user or user group.:
    a. READ - Ability to view the specific object
    b. WRITE - Ability to edit the specific object
    c. ADD - Ability to add an object to a parent object (e.g. add a Bitstream to a Bundle, add a Bundle to an Item, add an Item to a Collection , add a Collection to a Community, etc.)
    d. REMOVE - Ability to remove an object from a parent object (i.e. the opposite of "ADD")
    e. ADMIN - Full Admin permissions on the specific object
    f. DELETE - Ability to delete the specific object
    g. WITHDRAWN_READ - Ability to view a withdrawn Item (only valid for Items)
    h. DEFAULT_BITSTREAM_READ - Default "READ" permissions added to any Bitstreams of a newly submitted Item (only valid for Collections)
    i. DEFAULT_ITEM_READ - Default "READ" permissions added to any newly submitted Item (only valid for Collections)
5. **Start date – end date:** The user can select the start date and end date of the period for which the policy will be active, should they want to apply this policy for a fixed period only. If the start date is left blank, the policy comes into effect immediately.
6. **The ePerson or group that will be granted the permission:** List of users/groups selected for granting permission under the policy
7. **Search for an ePerson / Search for a group:** Select ePerson or group to add
8. **Search field:** Enter keywords for searching the ePerson/Group
9. **ePerson/Group list:** Click on the select button against the user/group you want to add to the policy
10. **Submit/Cancel button:** Click on the Submit button to complete policy creation or click on the Cancel button to cancel the entire process.

Upon successfully creating the policy, you’ll see a confirmation prompt, and the user will be back on the Authorizations screen.

Content (Item) management

Documentation for repository managers.

- Add item
- Delete item
- Edit Item
    - Authorizations (Manage access to an item)
    - Collection Mapper
    - Edit Bitstream
    - Edit Metadata
    - Edit Relationship
    - Make an Item Private
    - Move an Item
    - Reinstate an item
    - Versioned Item
    - Withdraw an item
- Embargo an item
- Lease an item

Add item

- Target Audience
- Overview
- Submission Form Highlights
- Item Submission Process

Target Audience

Content Submitters

Overview

The item submission process lets authorized users deposit contents using metadata and bitstreams. It primarily consists of components.

1. Target collection where the item needs to be submitted
2. Submission form using which metadata and bitstreams related to the item are submitted
3. The submission form also helps in defining Access rights around an item

Submission Form Highlights

1. Bitstream upload section

This section allows users to upload bitstream(s) by browsing or drag & drop mechanism.

2. Target Collection

It is a location where the item will be submitted.

3. General Metadata section

Users can define general or primary metadata about an item in this section.

4. Additional Metadata section

This section allows users to add secondary or additional metadata.

5. Bitstreams Management

Bitstreams uploaded by users list in this section. Using various options, they can further define these bitstreams or remove them from the attachment list.

6. Deposit License

Users need to accept the license in this section to submit the item to the repository.

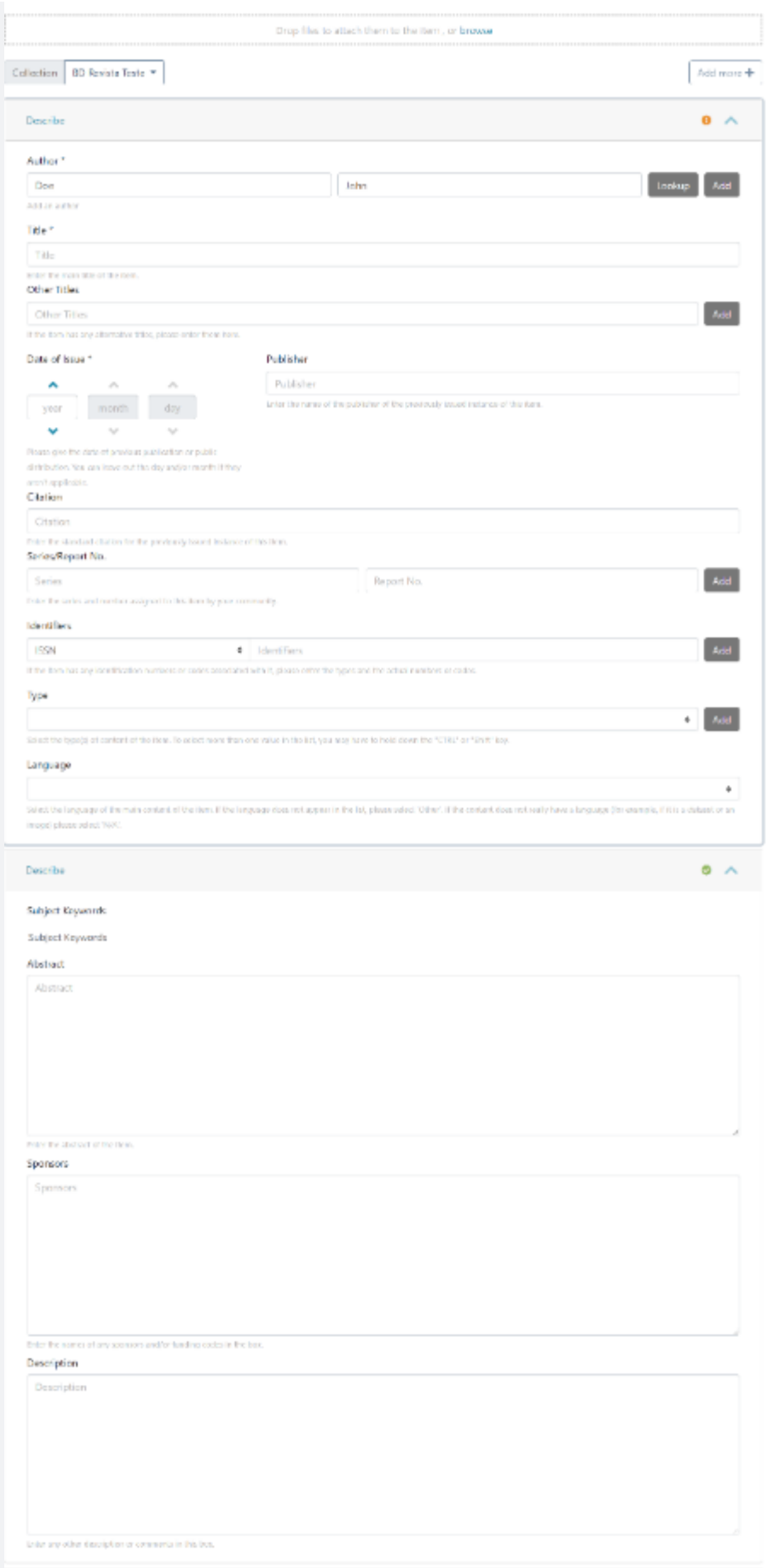
7. Manage Item Submission

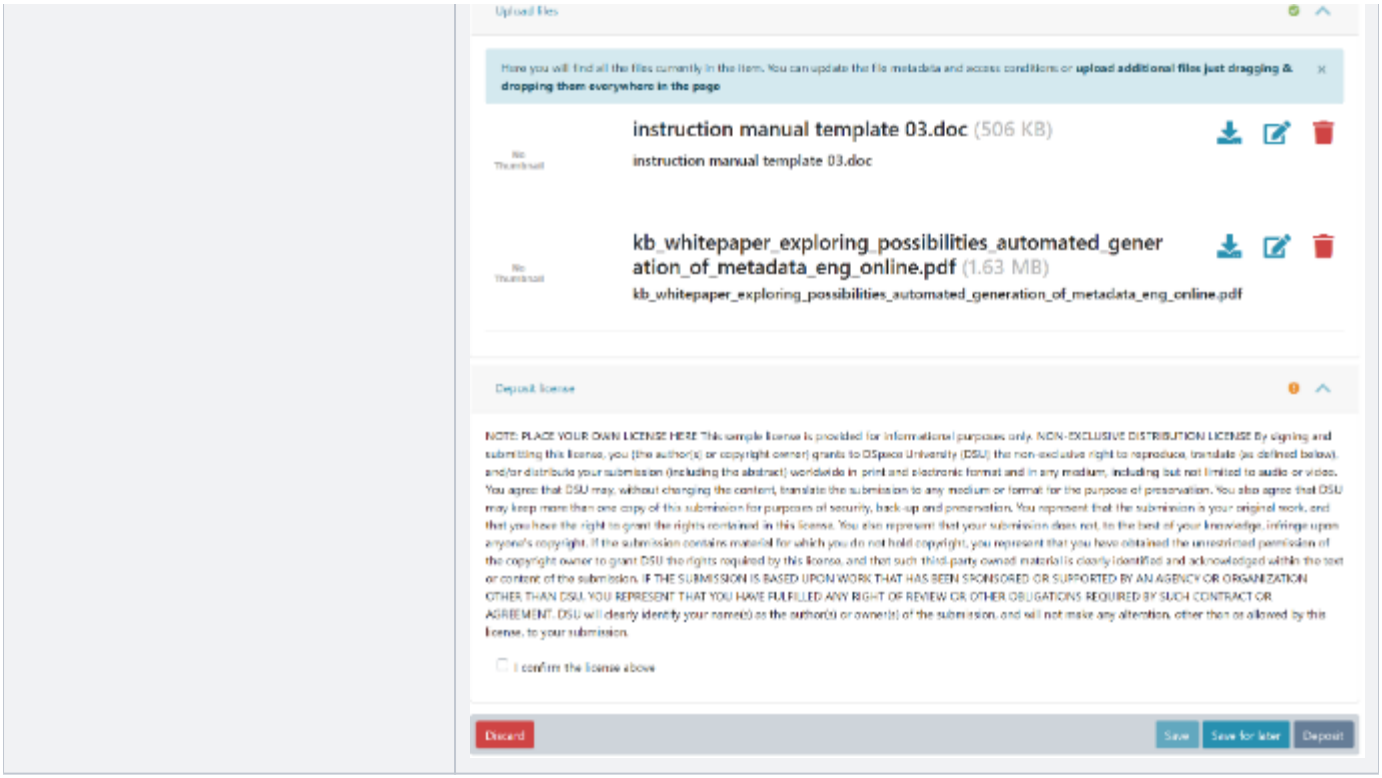
a. *Discard:* The user can discard the submission by clicking this button. Action will delete all information populated in the form permanently.

b. *Save:* This button helps save information in the submission form and helps resume information update should the process gets interrupted.

c. *Save for Later:* Save information in the MyDSpace section to update later.

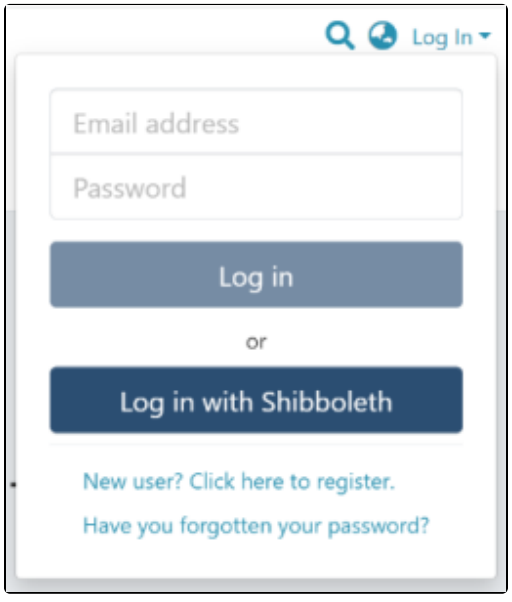
d. *Deposit:* Click this button to complete the submission. The item will go to the next step as per the workflow defined for the collection.



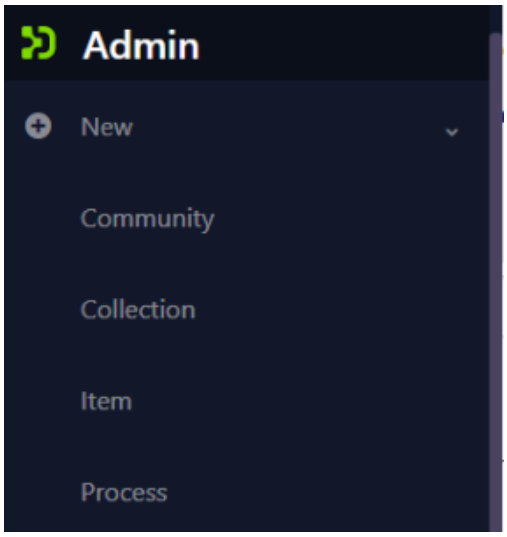


Item Submission Process

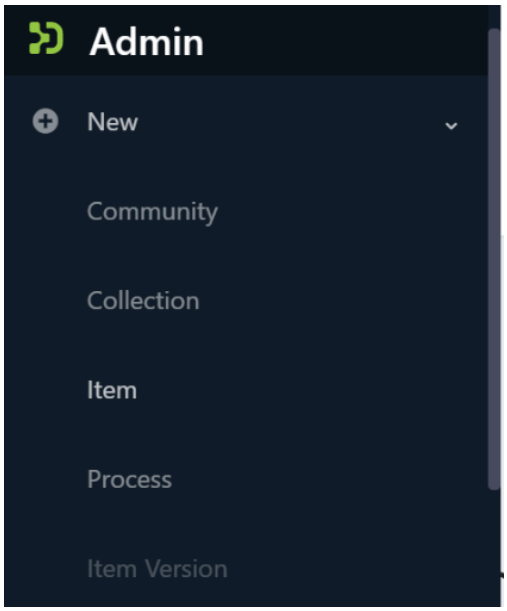
Step 1: Login using your credentials



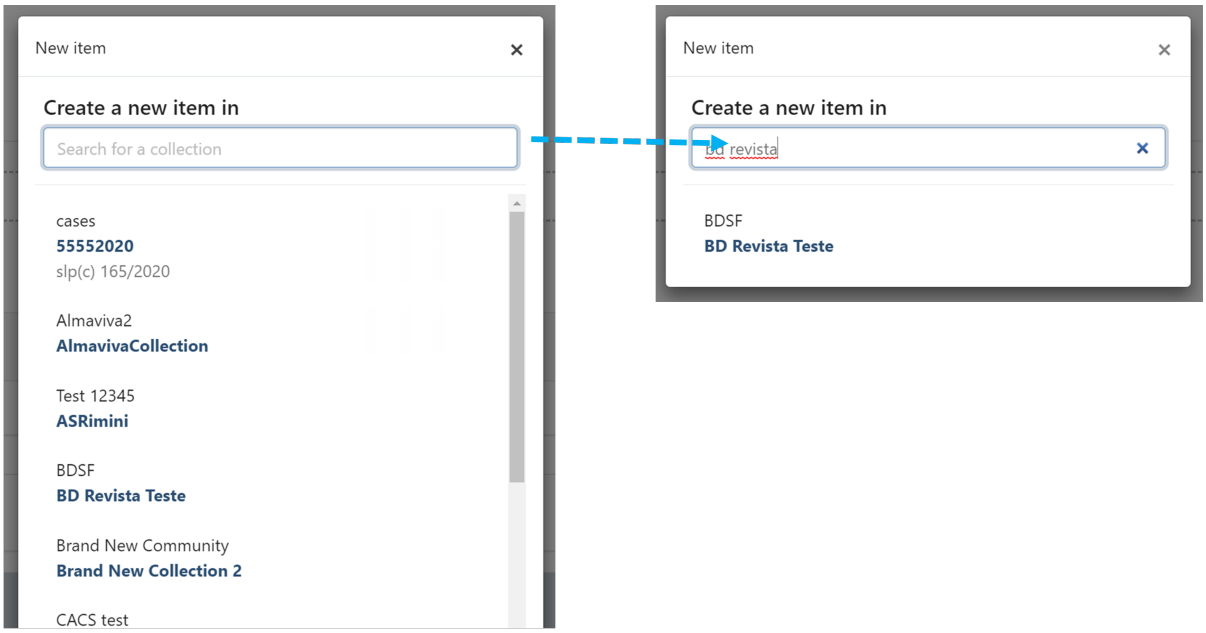
Step 2: Roll over the cursor on the “+” sign



Step 3: Click on “New” and click on “item” for proceeding further in the Item addition process

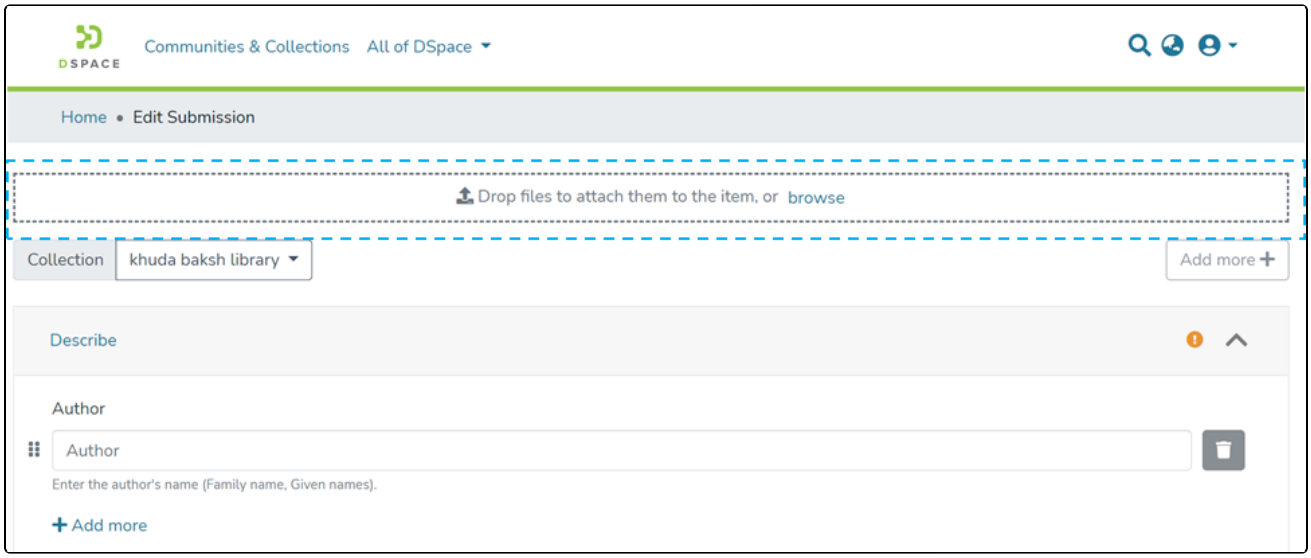


Step 4: A popup window with a collection list will appear. The user can select the target collection by typing its name or scrolling down the collection list. Then, click on the collection to initiate item submission.

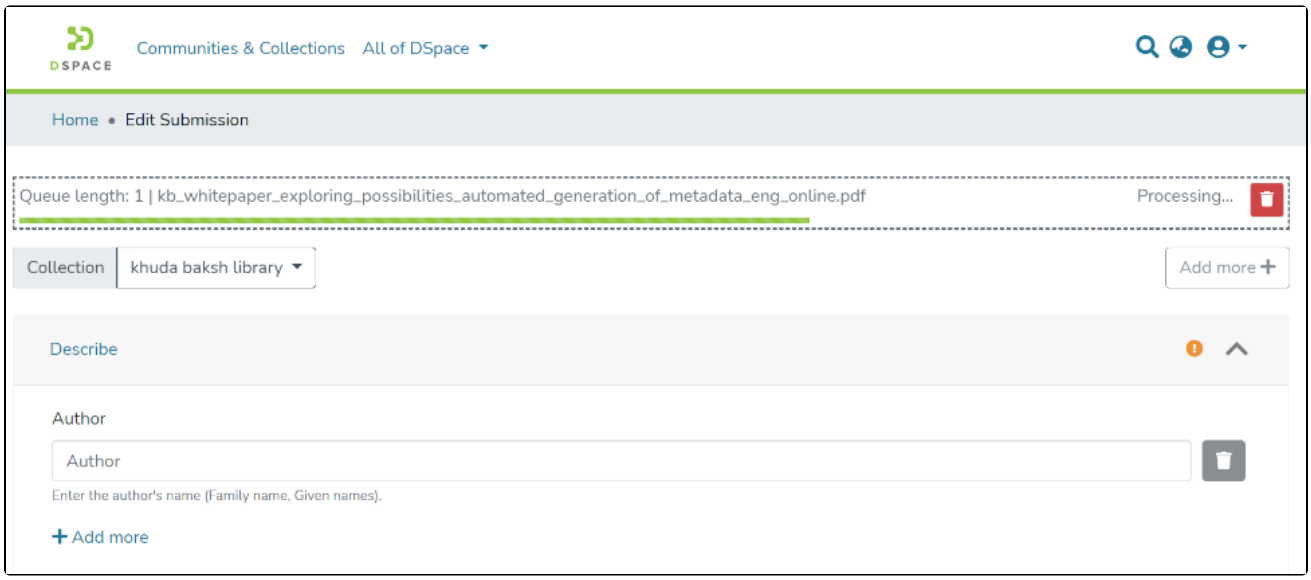


Step 5: Users will see the item submission form after selecting the target collection. The first step is to upload the attachment(s) in the item. In DSpace terminology, an attachment is known as a “bitstream”.

Click on the “browse” link to upload attachment(s). Users can upload multiple files by selecting them together or dragging in the space.



A progress bar showing bitstream upload progress will appear, as demonstrated in the illustration below. In addition, after a successful bitstream upload, a prompt confirming success or failure will appear.

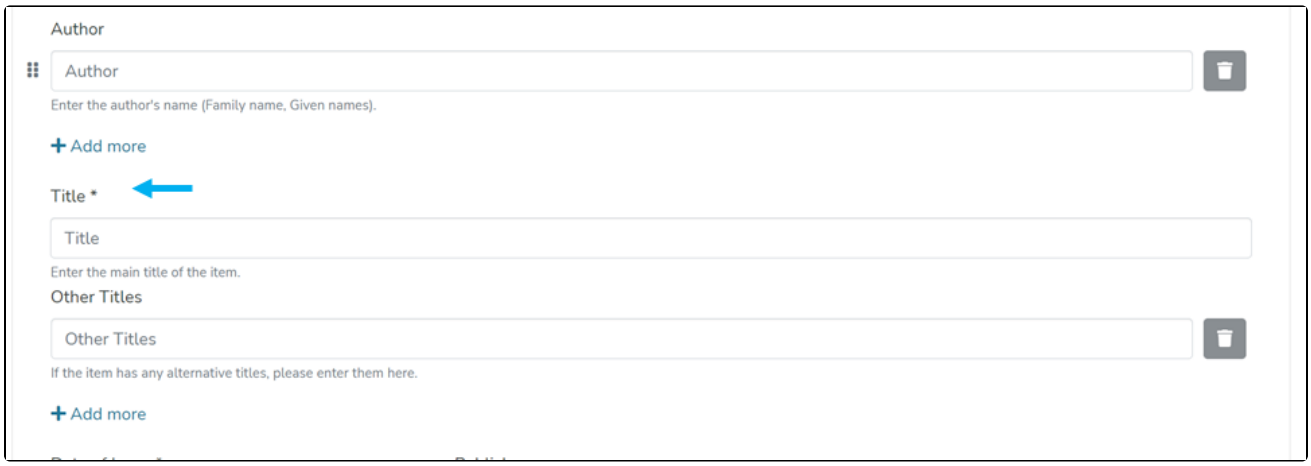


Bitstream upload in progress

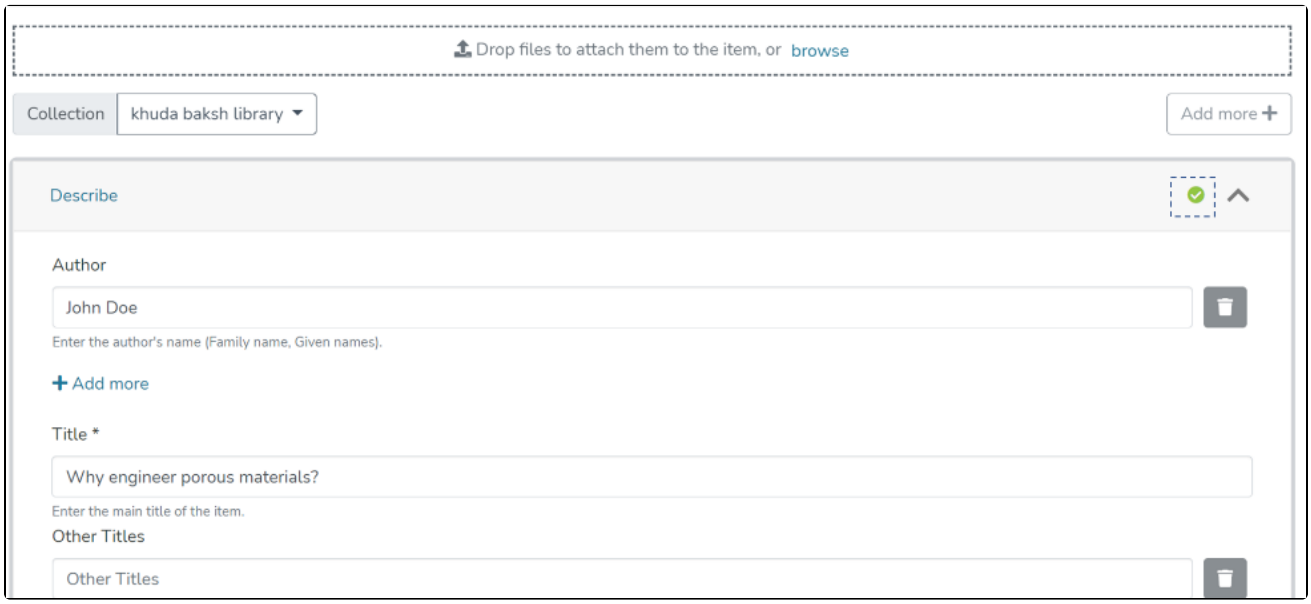
Bitstream Upload Successful

Step 6: After bitstream upload, the next step is to describe the item by adding metadata.

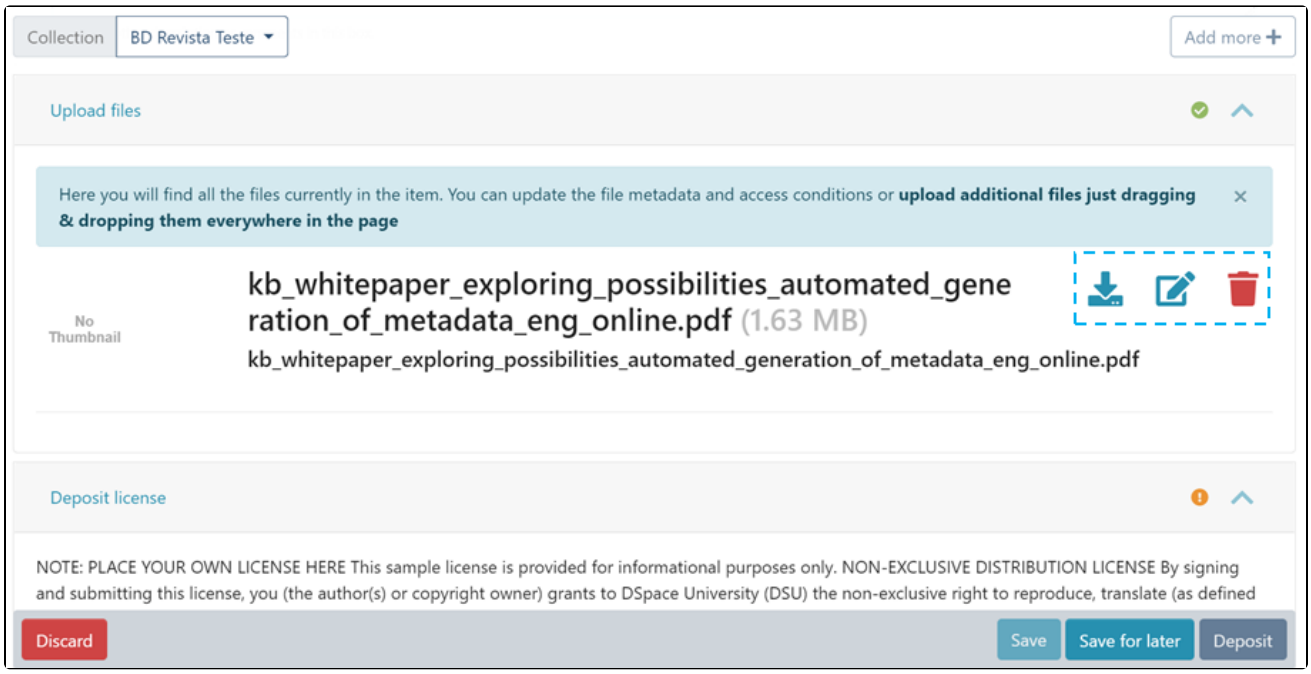
Metadata fields marked with "*" are mandatory, and users need to populate information in these fields to complete the submission mandatorily. A few examples in the standard submission form are Author, Title, and Date of Issue.



Users will notice an alert mark at the top right of the “Describe” tab turning from Amber to green once mandatory fields have values. Below is an illustration showing the state of the “Describe” section having values in all mandatory fields.



Step 7: The user can further update bitstreams by clicking on Buttons appearing next to the bitstream title.



Download: Click this button to download bitstream on a local machine.

Edit: Update bitstream details and access rights using this button. More explanation is provided below.

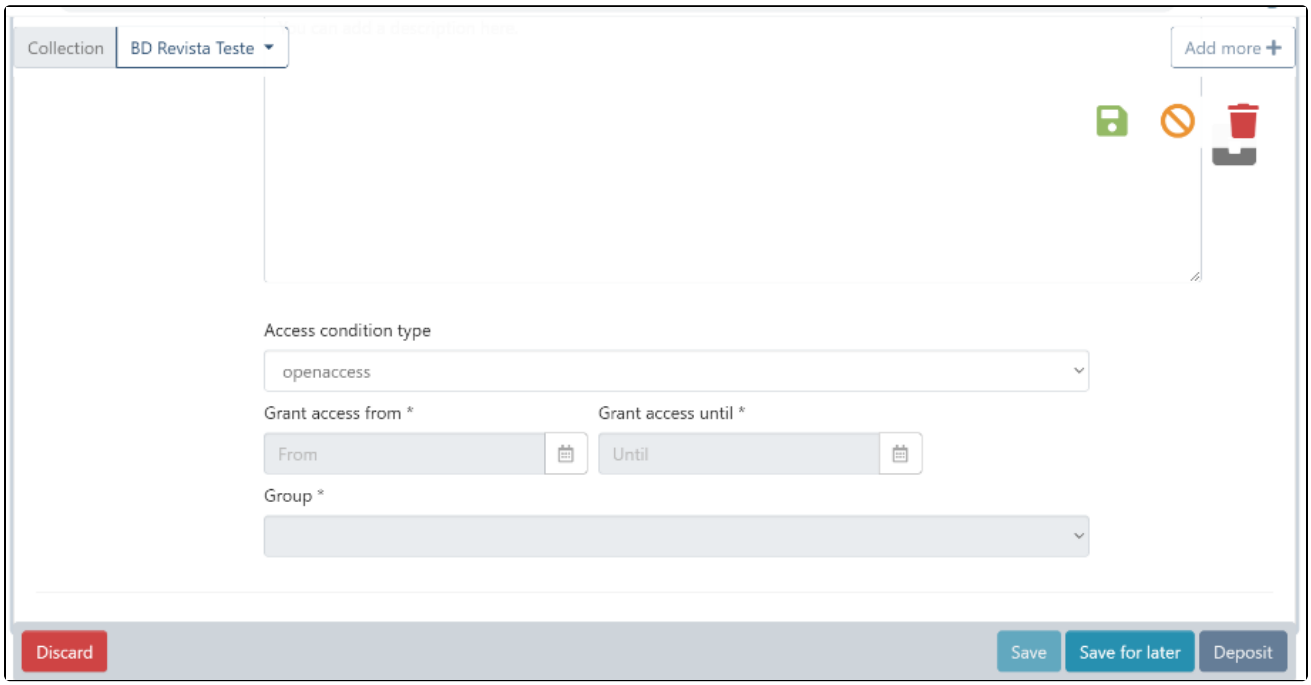
Delete: Clicking this button will delete the bitstream from the submission form.

Step 8: By clicking the edit button next to the bitstream, users can update bitstream information, as explained below.

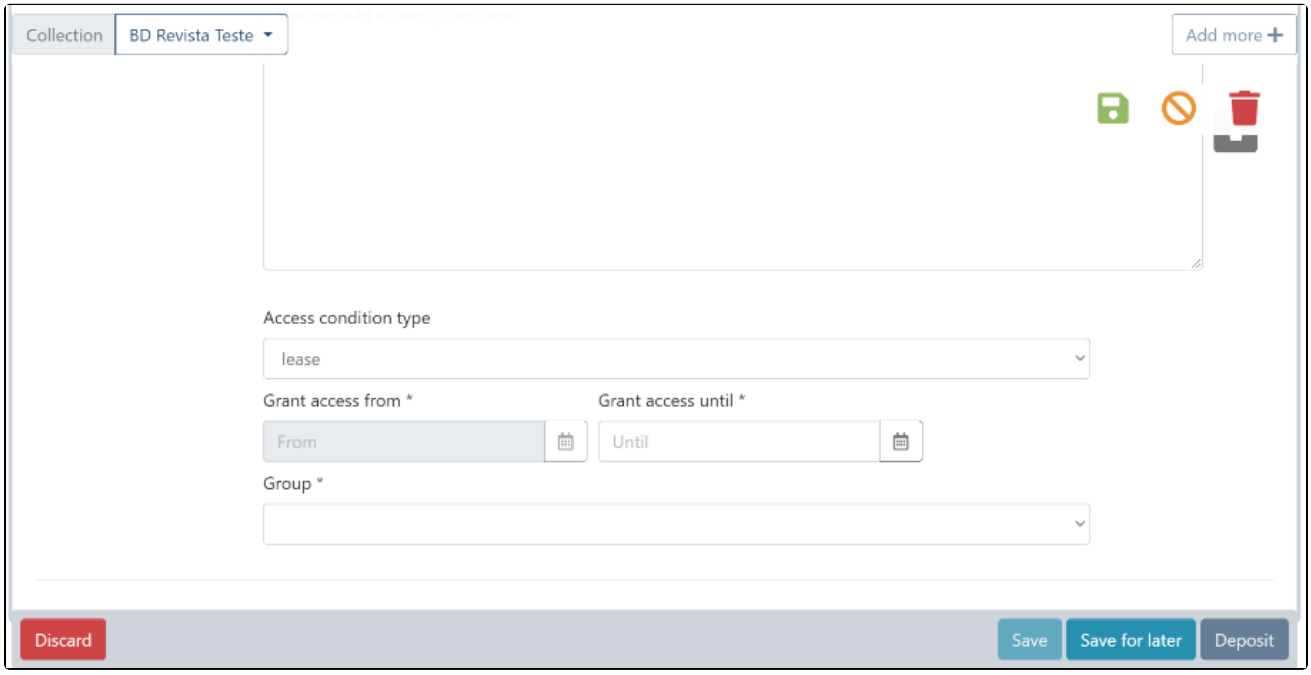
Update the bitstream title and add descriptions to describe the attachment further. Please refer to the below illustration demonstrating both functions.

Users can define access conditions for the bitstream by selecting the appropriate option from the dropdown list. These options are:

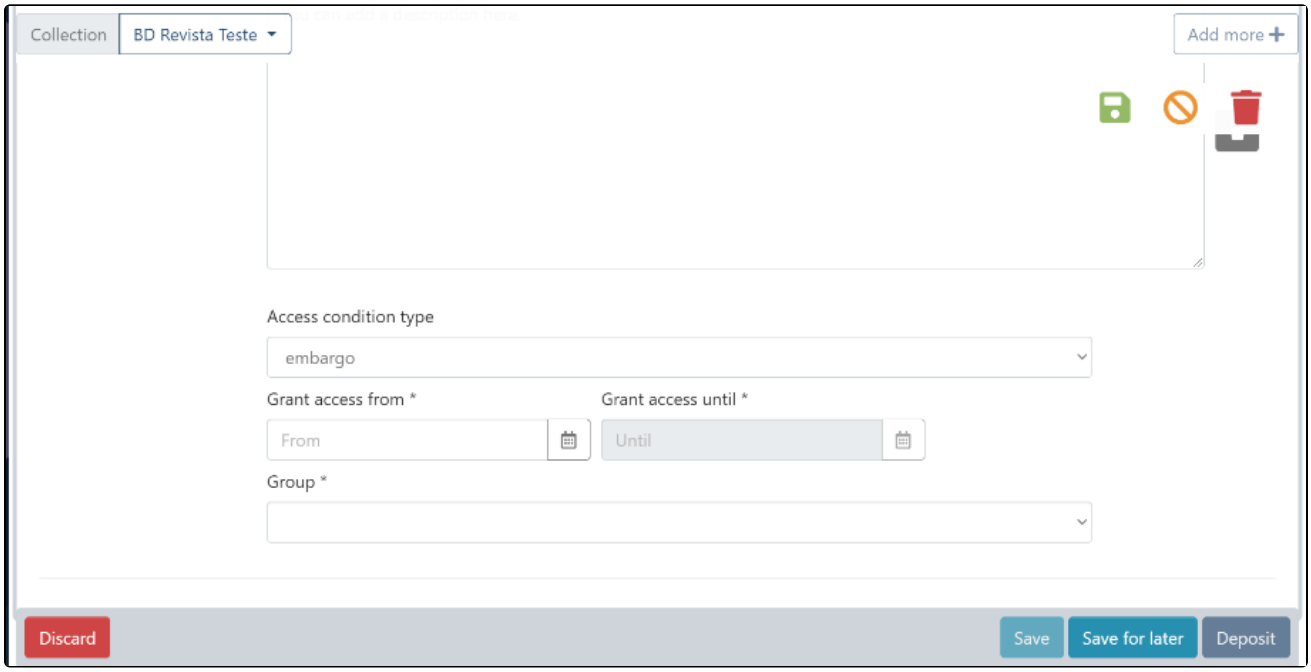
- Open Access: Select this option to make the bitstream available without any restriction.



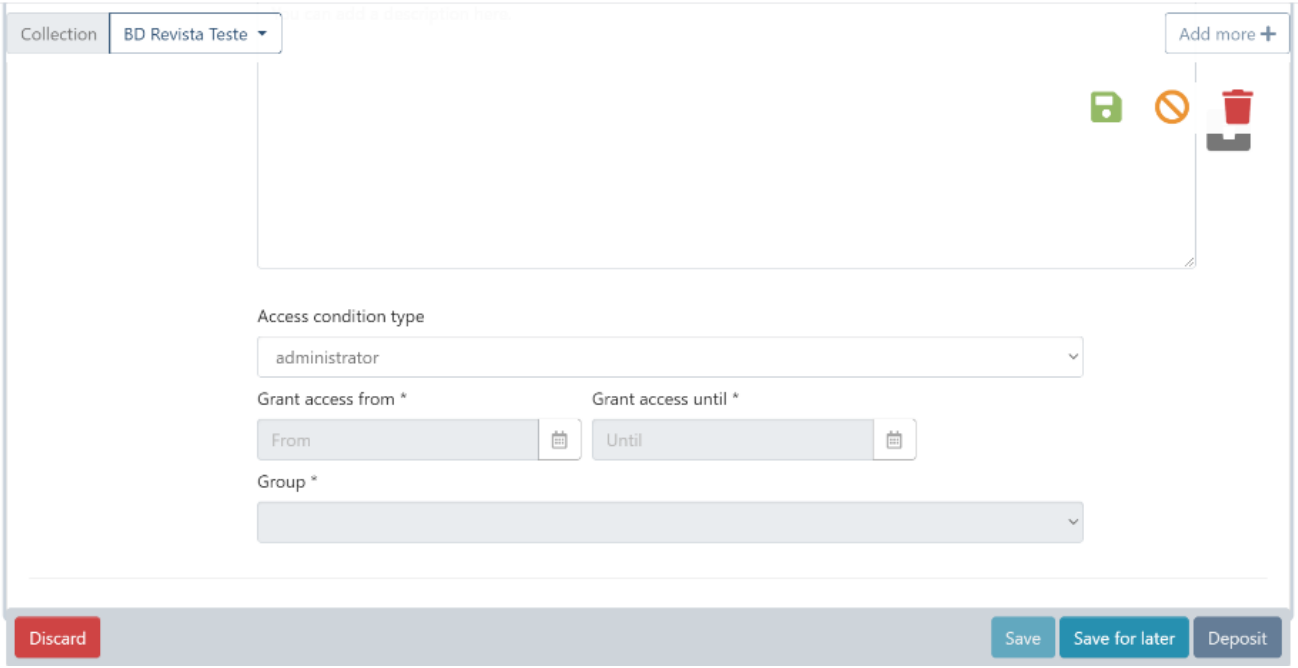
- Lease: This option is applicable when a user wants to keep bitstream accessible until a specific date in the future. The bitstream will not be available as open-access content after the defined date under the “Grant access until” option.



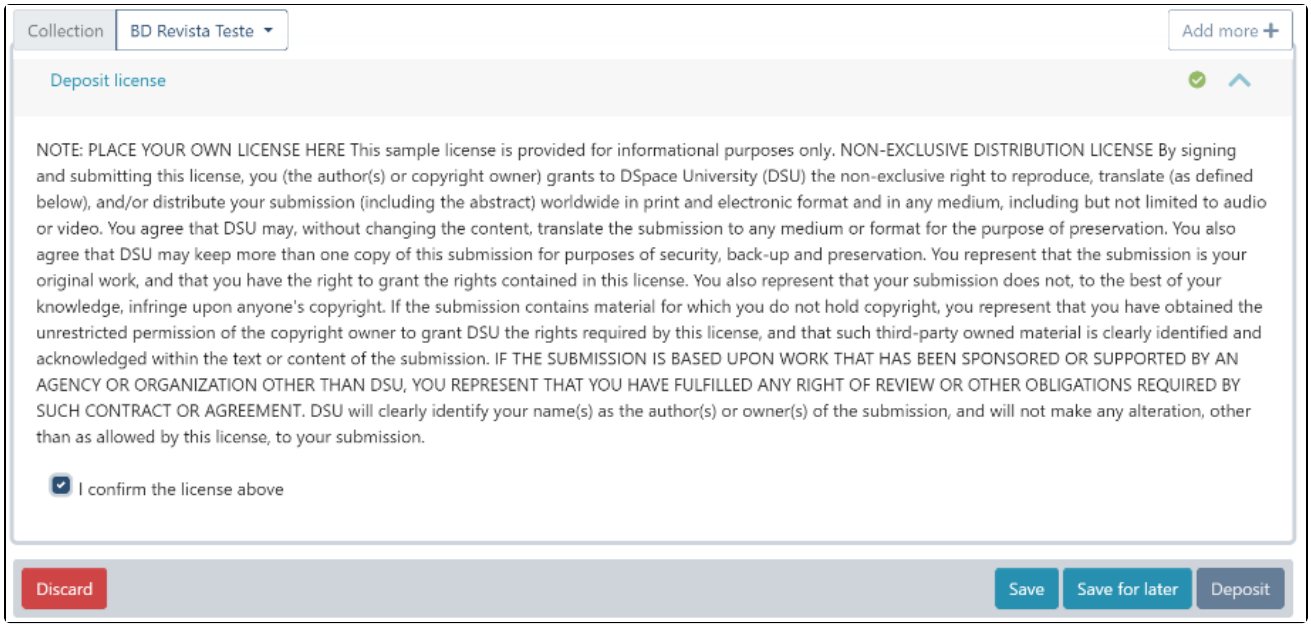
- Embargo: In contrast to a lease, an embargo allow the user to keep bitstream access restricted until a future date. This date is defined in the “Grant access from” field. The bitstream will be available as open-access content to users after this date.



- Administrator: Select this option if the bitstream’s access remains limited to administrators.



Step 9: Finally, users must click on the “I confirm the license above” checkbox to accept the deposit license and click on the “Deposit” button to complete the item submission.



Delete item

- Target Audience
- Process Overview
- Item Delete Process

Target Audience

Content Submitters

Community Administrators

System Administrators

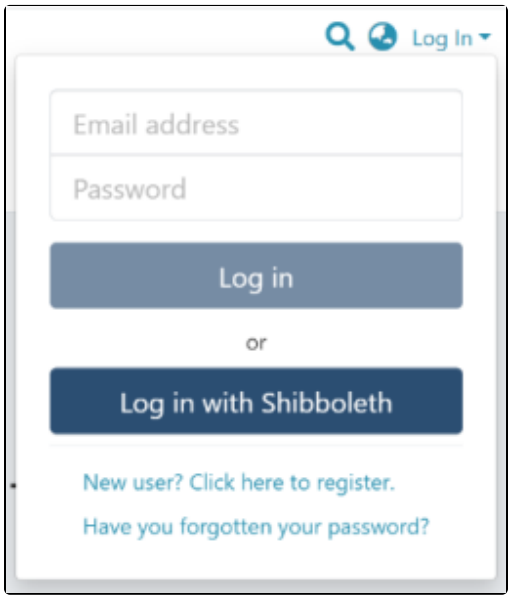
Process Overview

As the name suggests, the Permanently Delete option is exercised when the authorized user(s) wants to permanently delete any item (Metadata + bitstreams) from the repository.

Apart from permanently deleting an item, options like “Withdraw item from repository” and “Make item Private” can temporarily help disable the content access.

Item Delete Process

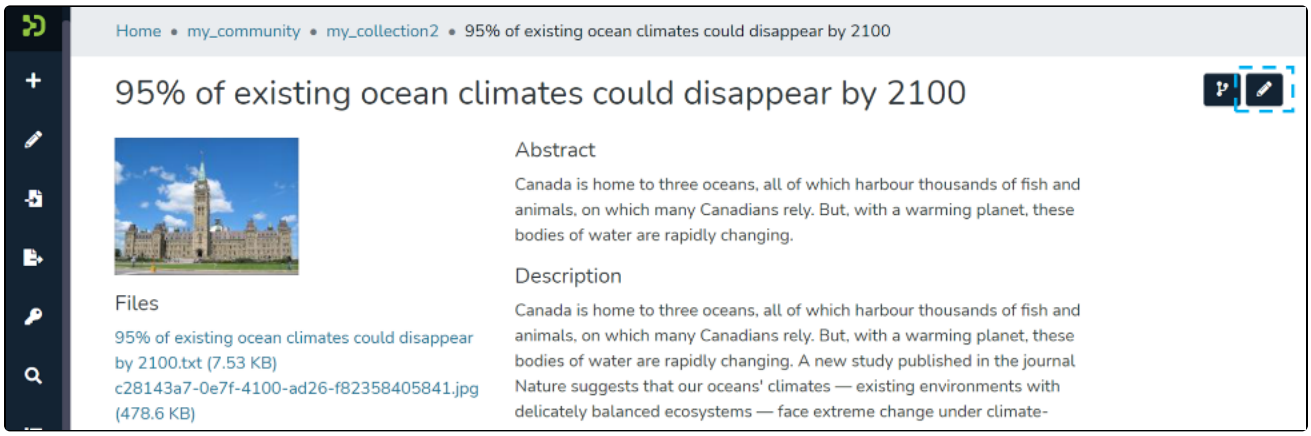
Step 1: Login using your credentials



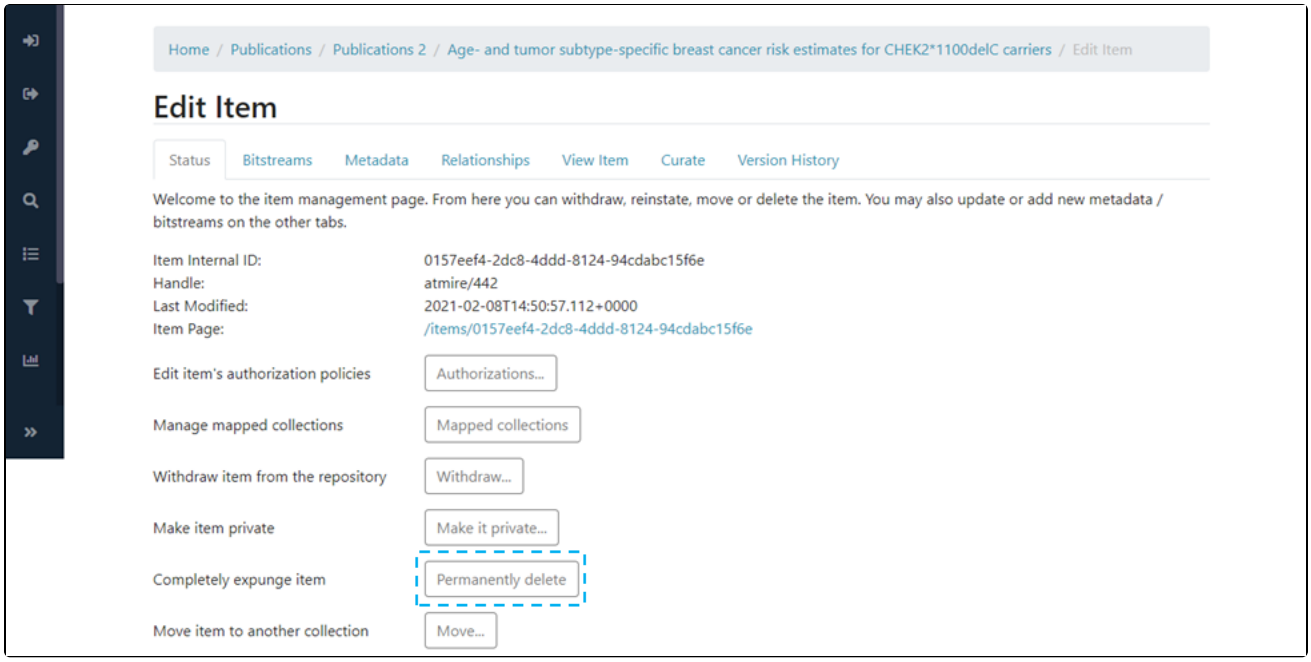
Step 2: Go to the target Item using a convenient method i.e.

1. Using search & filters options
2. Browsing through Communities & Collections
3. Browsing using Metadata elements listed under the Browse menu

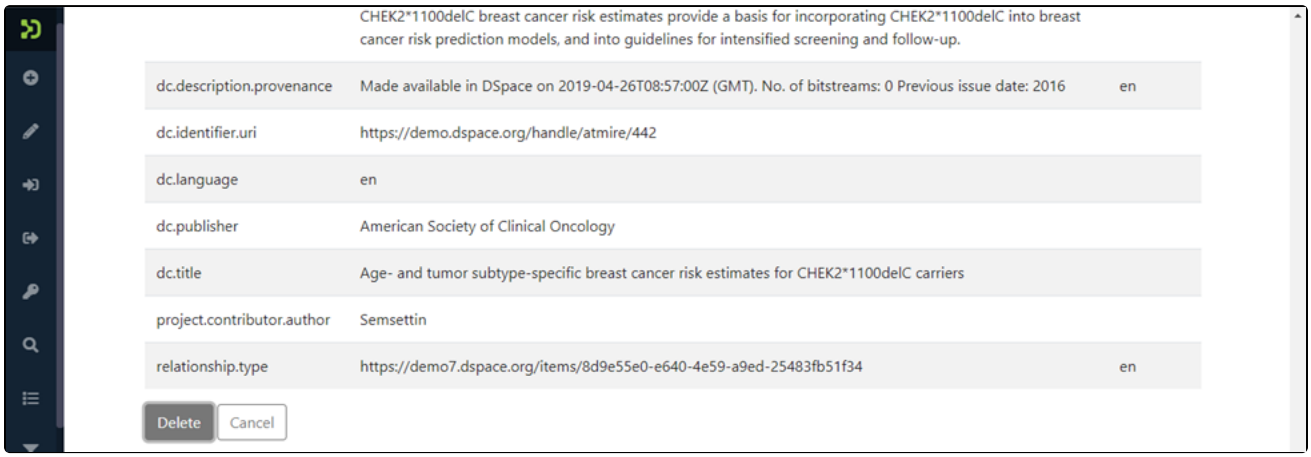
Step 3: Click on the “Edit” button as highlighted on the screen below. This button will appear to the user having edit rights on the target item.



Step 4: Click the “Permanently Delete” Button to delete the item.



Step 5: Click on the Delete button on the confirmation screen. Should you want to continue with deletion, click on cancel to cancel the Permanent deletion of the item from DSpace.



Edit Item

- Audience
- Edit Item Overview
  - Status tab
  - Bitstreams tab
  - Metadata tab
  - Relationships tab
  - Version History Tab

Audience

Content Submitters

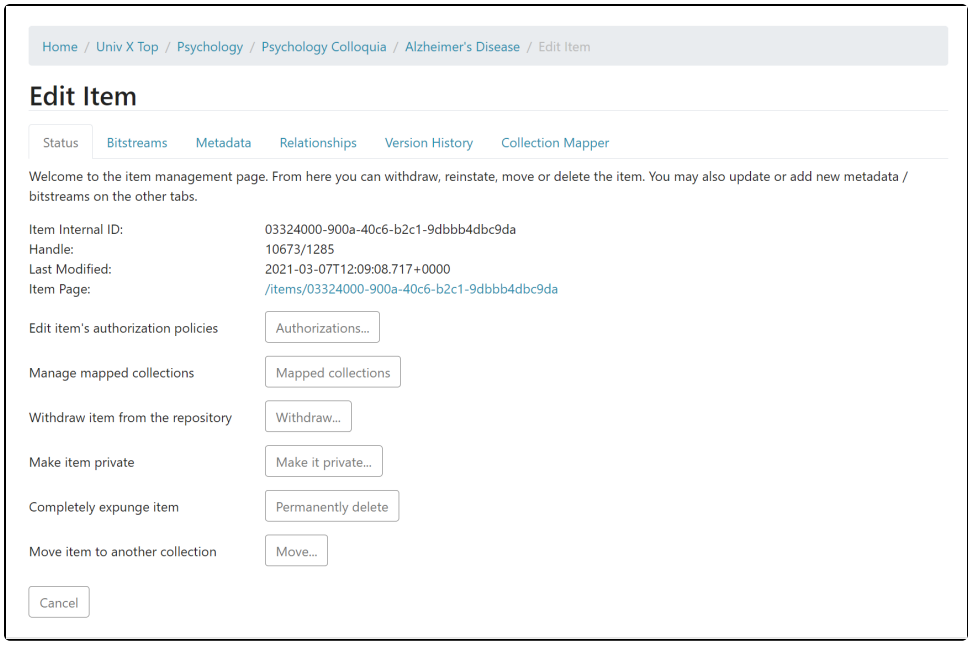
Community Administrators

System Administrators

Edit Item Overview

The Edit Item process enables authorized users to edit various properties of items deposited in DSpace. Edit item serves the following purposes, each contained in different tabs within the Edit Item form.

Status tab



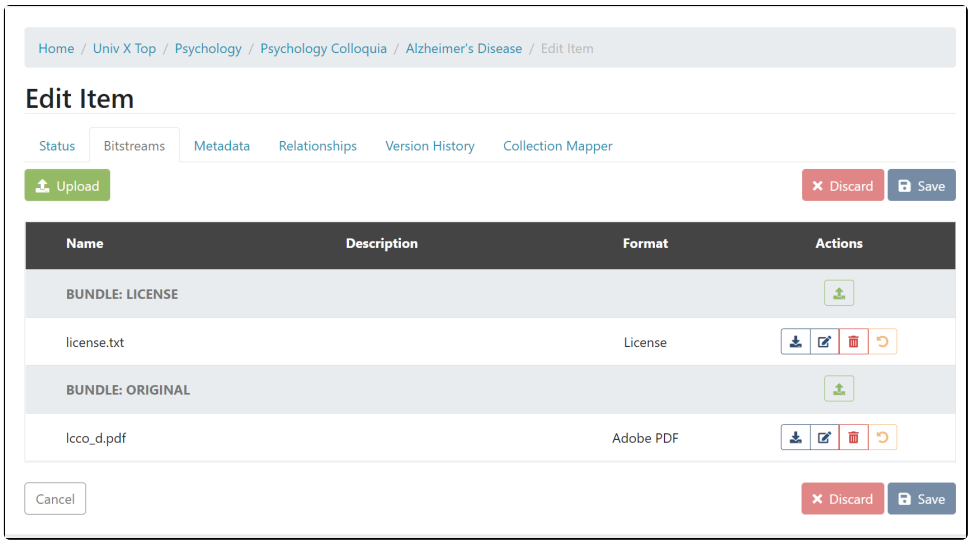
Update authorization policies

Managing mapped collections

Update item status (Withdrawn/Private/Expunge)

Move item to another collection

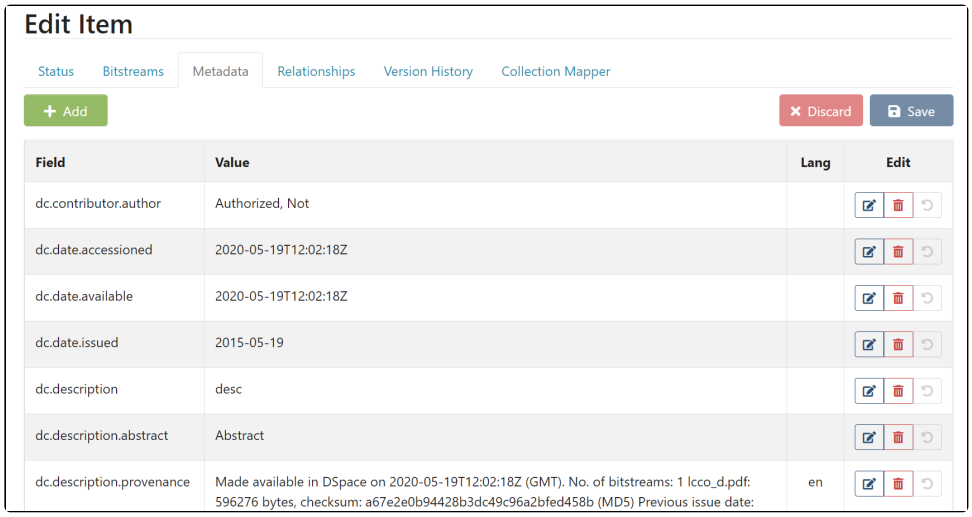
Bitstreams tab



Add or remove licenses or other bitstreams (i.e. files) attached to item

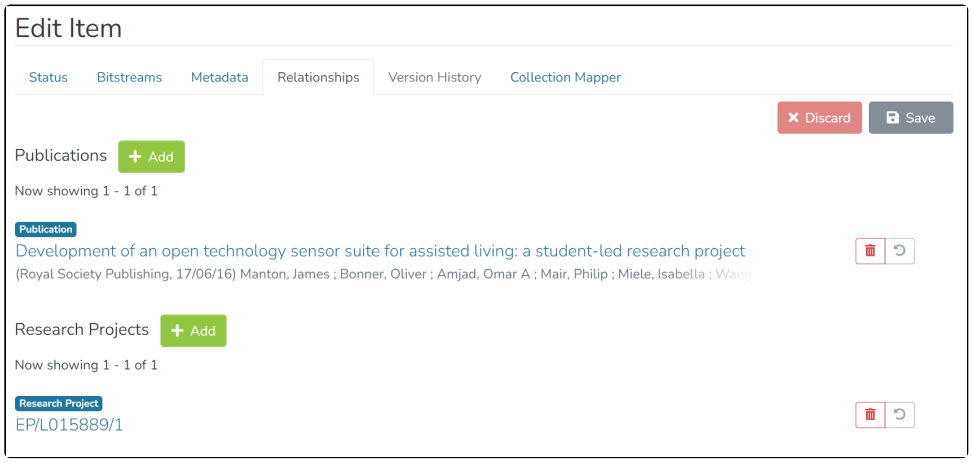
Edit metadata associated with the bitstreams, including filename, file format and file description

Metadata tab

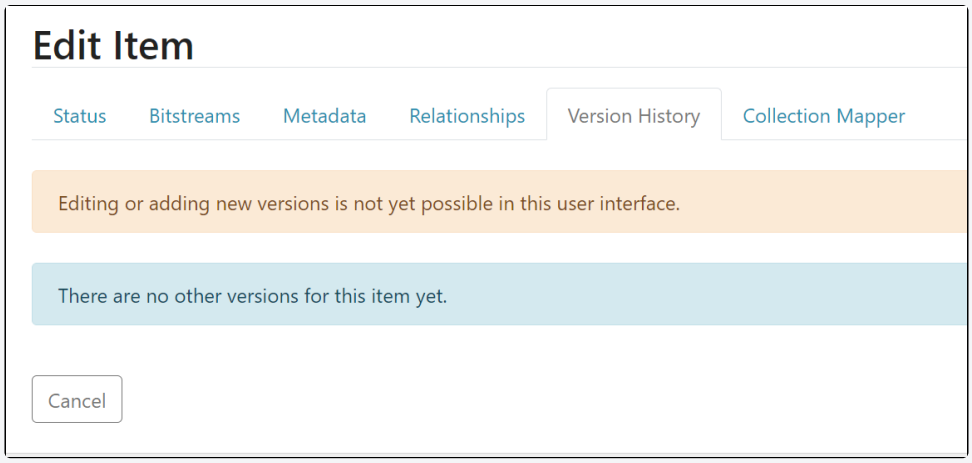


Add or deletie metadata fields (i.e. elements added to Item) or edit values in existing fields. See further detail below.

Relationships tab



Add or delete relationships with other items or editing existing relationship with items. See further details below.

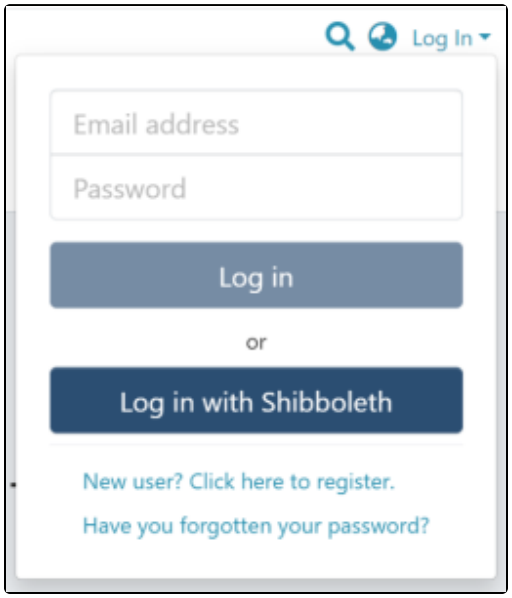
Version History Tab	 **Create and manage item versions in DSpace.**

Authorizations (Manage access to an item)

- Overview
- Add Authorization Policy
- Manage Policy
- Delete Policy

Overview

Step 1: Login using your credentials

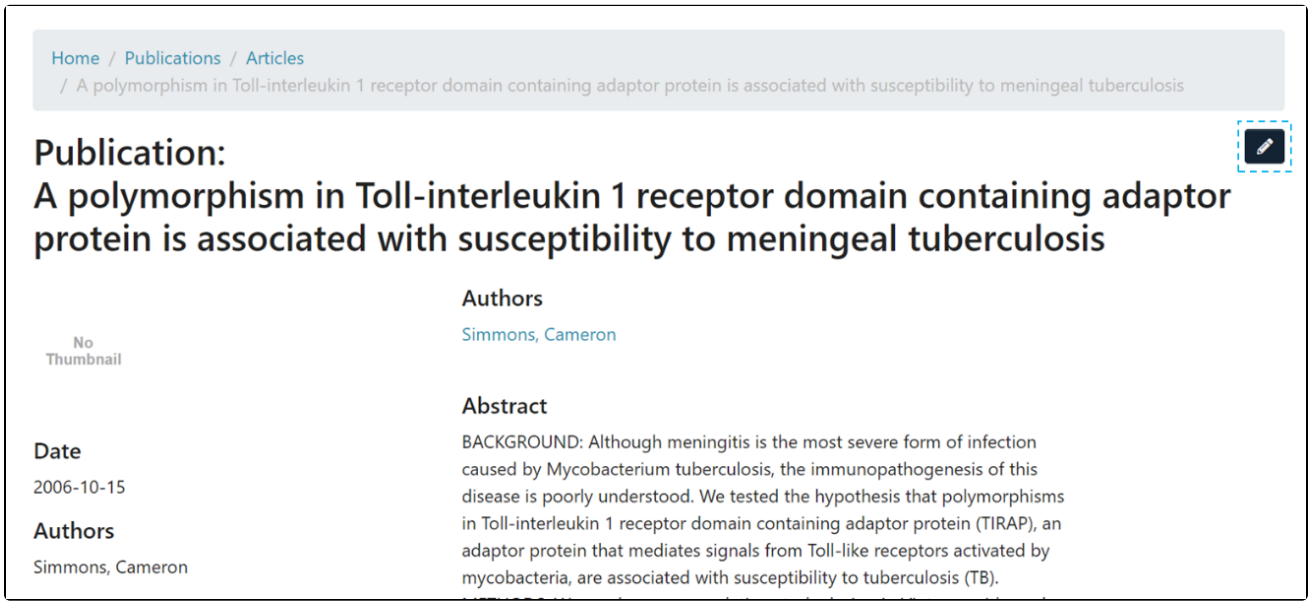


Step 2: Go to the Item you want to edit

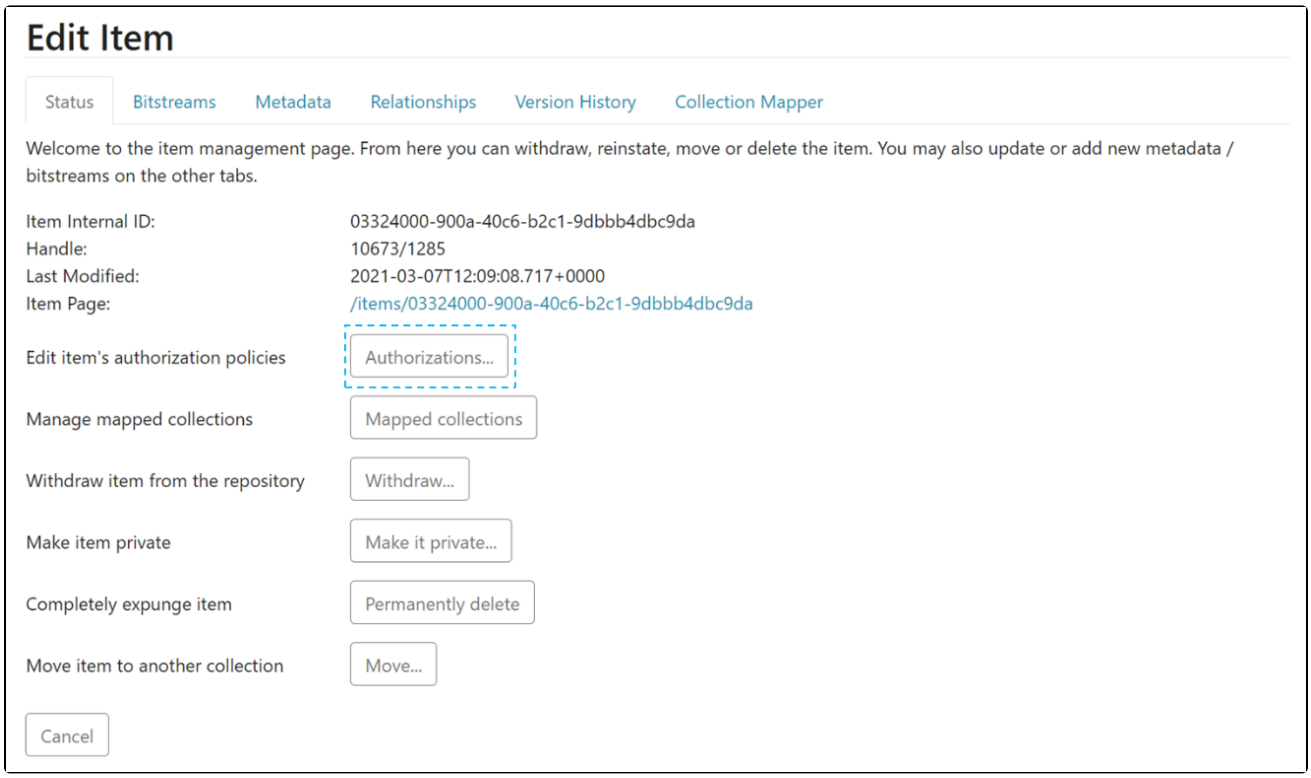
Users can reach an item through multiple methods, as listed below:

1. Search an item
2. Browse communities and collections
3. Finding an item in the Administration section at Edit > Item

Click the "Edit" button on the item title's right-hand side.



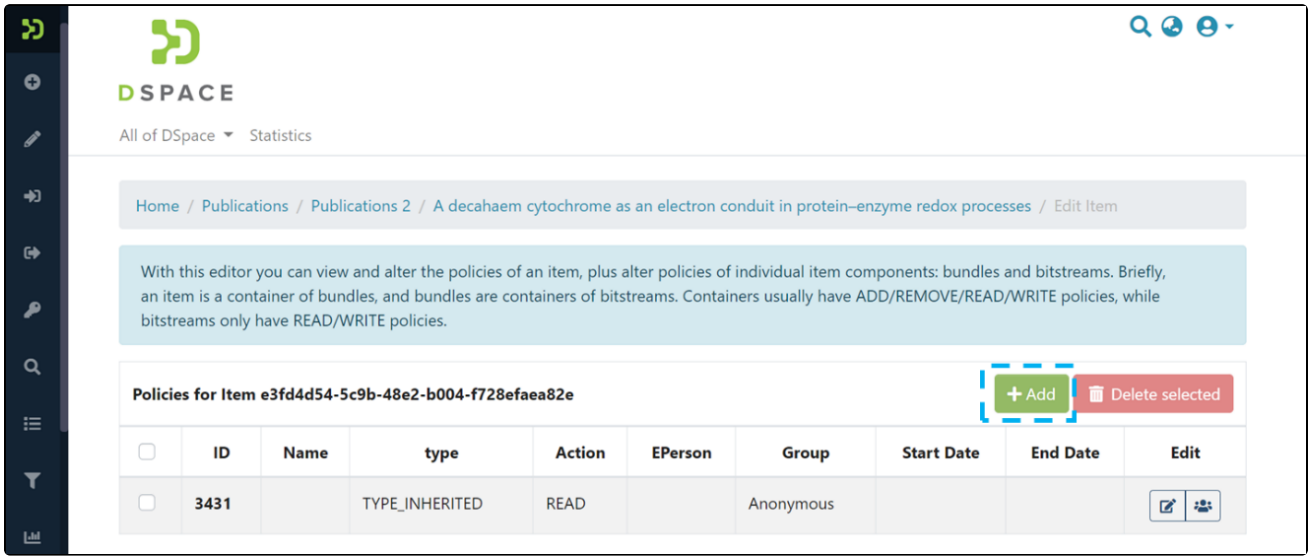
Step 3: Click the "Authorizations" button under the "Status" tab to manage the Item's authorization policies.



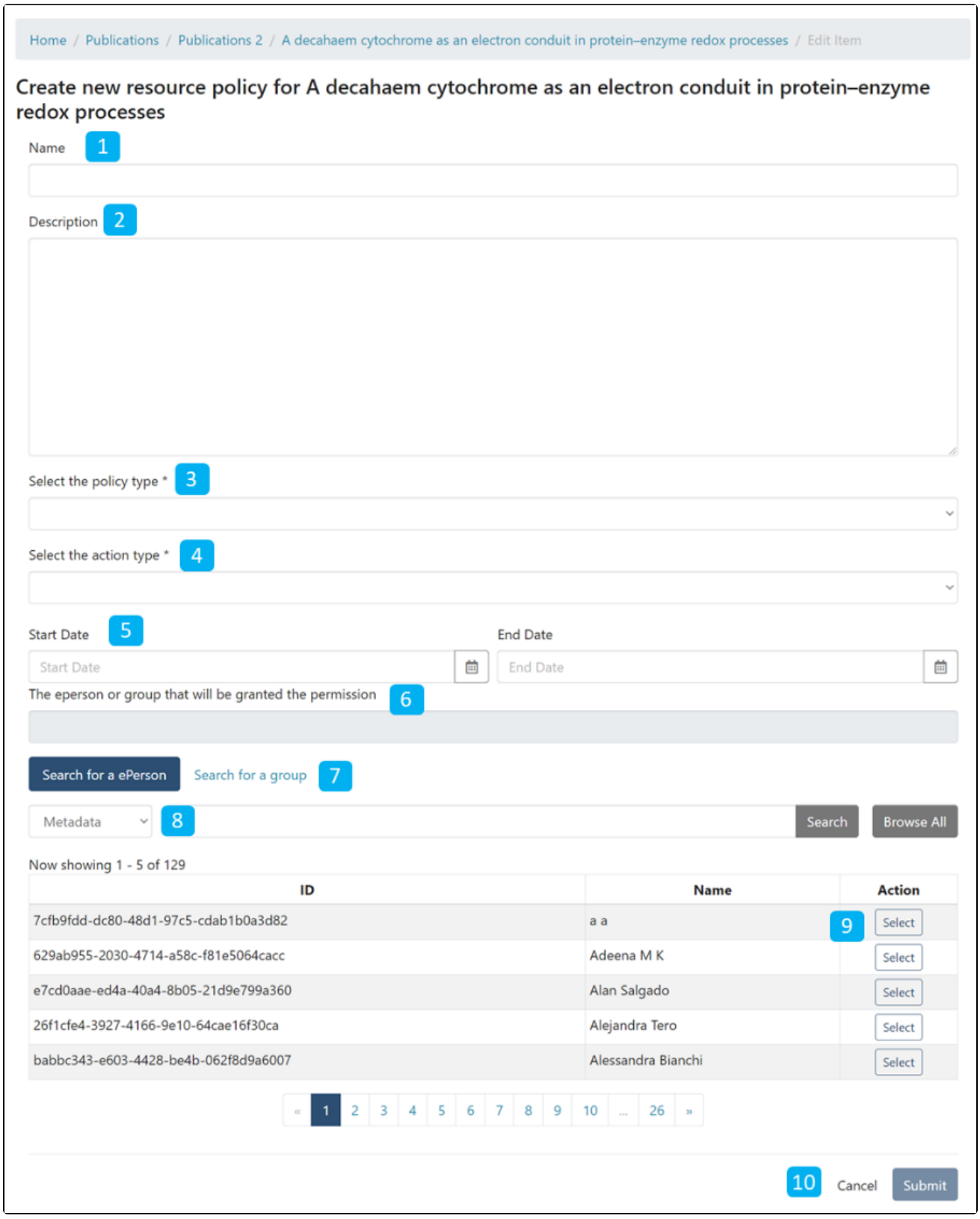
> *Users can create different policies for both Item and bitstreams. These are:*
>
> 1. *Managing ADD/REMOVE/READ/WRITE policies for an item*
> 2. *Managing READ/WRITE policies for a bitstream*

Add Authorization Policy

Step 1: Click the "Add" button to create a new resource policy for the Item.



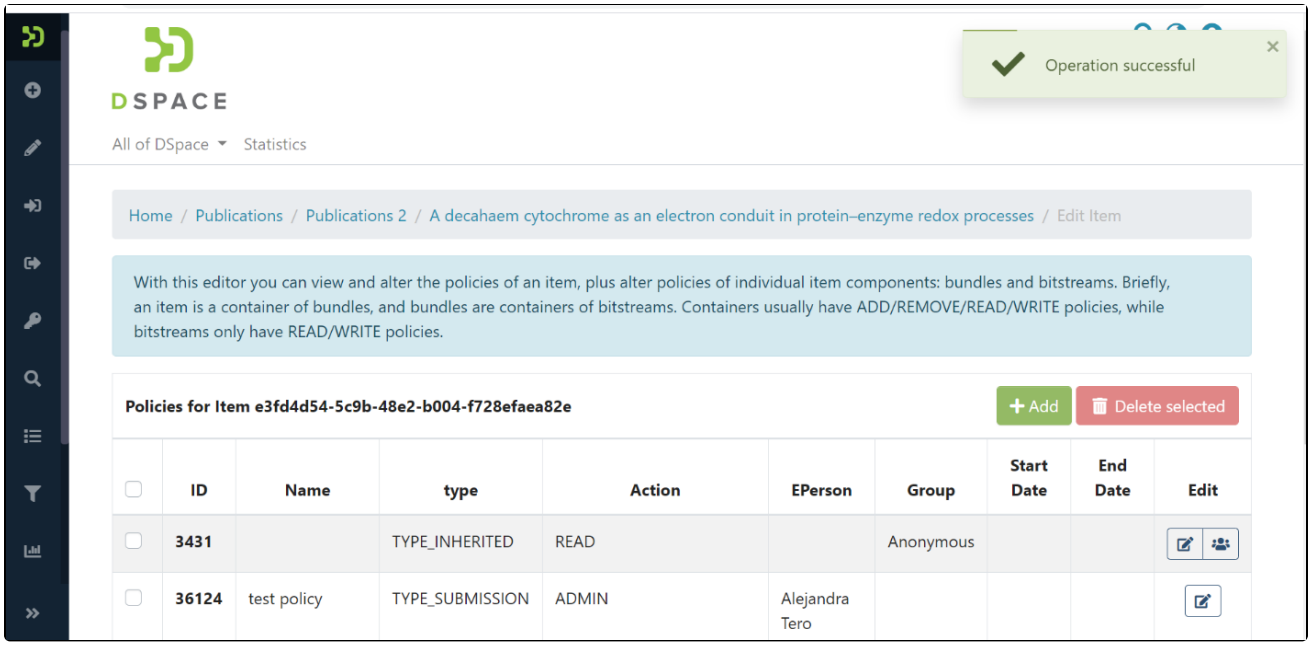
Step 2: Users can populate information on the "Create new resource policy" page about the policy and click on the "submit" button. Please see the description of each field followed by the below screenshot.



1. **Name:** Enter Policy name.
2. **Description:** Enter the Policy description for future reference and understanding of other users.
3. **Select the policy type (Required):** The user can select one of the following policy types:
    a. TYPE_SUBMISSION: a policy in place during the submission
    b. TYPE_WORKFLOW: a policy in place during the approval workflow
    c. TYPE_INHERITED: a policy that has been inherited from a container (the collection)
    d. TYPE_CUSTOM: a policy defined by the user during the submission or workflow phase

    e. null: if the information is not available
4. **Select the action type (Required):** The user can select one of the following actions from the drop-down list:
    a. READ - Ability to view the specific object
    b. WRITE - Ability to edit the specific object
    c. ADD - Ability to add an object to a parent object (e.g. add a Bitstream to a Bundle, add a Bundle to an Item, add an Item to a Collection , add a Collection to a Community, etc.)
    d. REMOVE - Ability to remove an object from a parent object (i.e. the opposite of "ADD")
    e. ADMIN - Full Admin permissions on the specific object
    f. DELETE - Ability to delete the specific object
    g. WITHDRAWN_READ - Ability to view a withdrawn Item (only valid for Items)
    h. DEFAULT_BITSTREAM_READ - Default "READ" permissions added to any Bitstreams of a newly submitted Item (only valid for Collections)
    i. DEFAULT_ITEM_READ - Default "READ" permissions added to any newly submitted Item (only valid for Collections)
5. **Start date – end date:** The user can select a start date and end date for using the policy, should they want to apply it for a fixed period.
6. **The eperson or group granted the permission is the l**ist of users/groups granted authorization under the policy.
7. **Search for an ePerson / Search for a group:** Click on the "ePerson" or "group" to search for the entity.
8. **Search field:** Enter keywords for searching the ePerson/Group.
9. **ePerson/Group list:** Click on the select button against the user/group you want to add to the policy
10. **Submit/Cancel button:** Click on the "Submit" button to complete policy creation or click on the Cancel button to cancel the entire process.

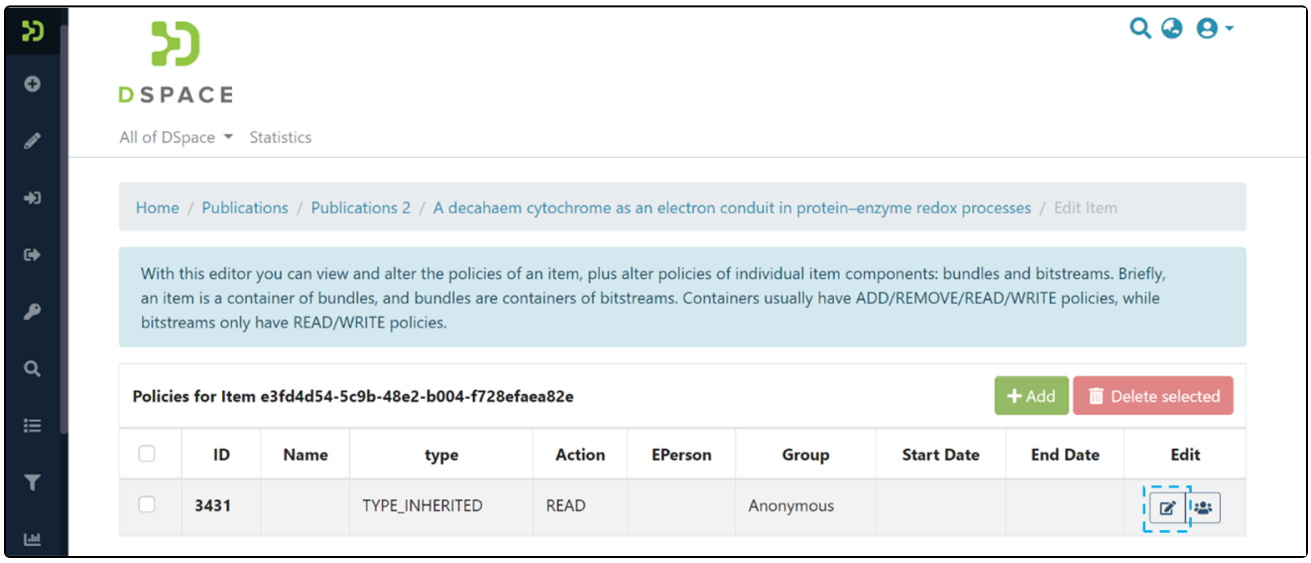
As shown below, users will see a success prompt upon the policy creation and be back on the Manage Policies screen.



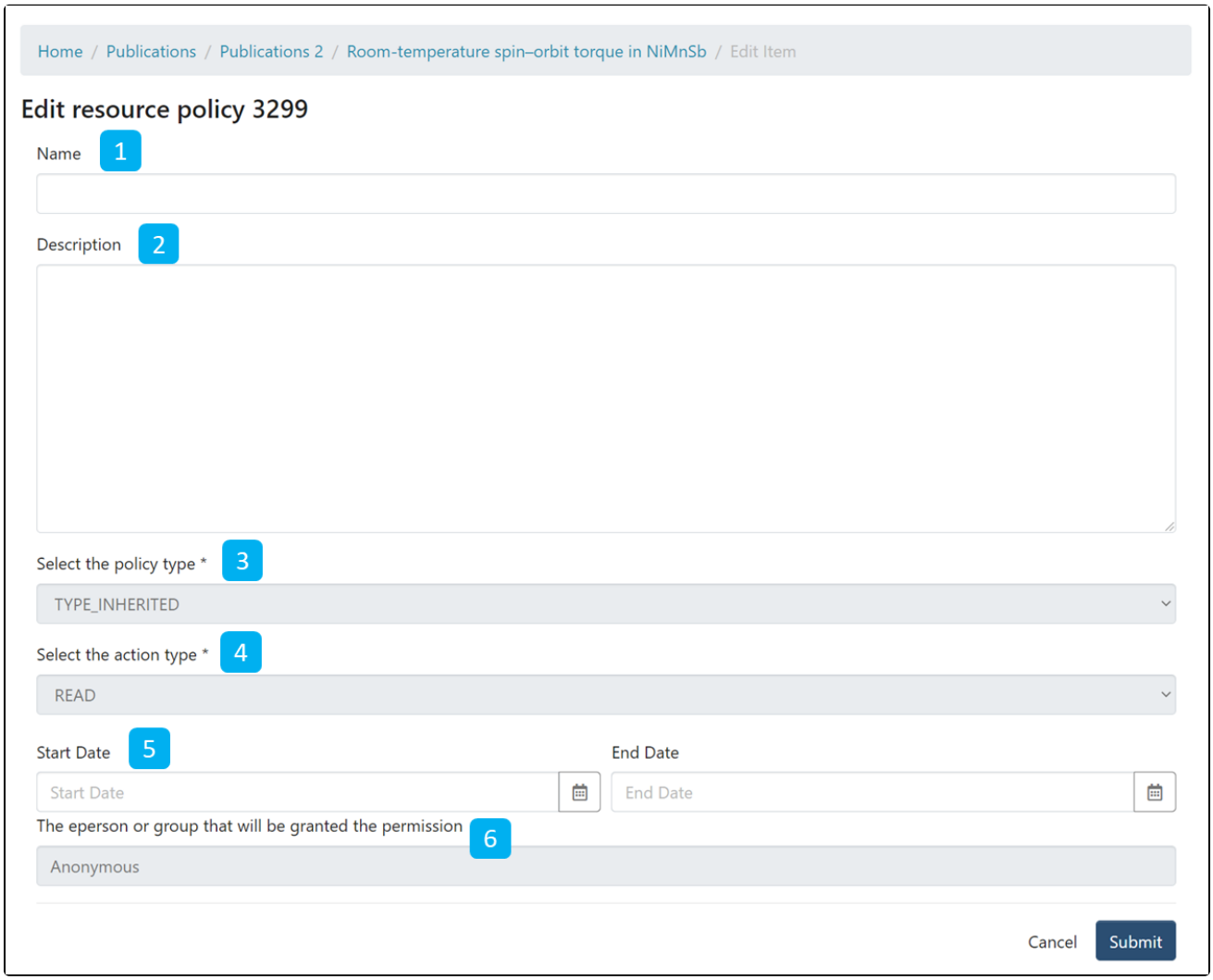
Manage Policy

Step 1: Click on the Edit policy icon appearing against each policy to update it.

The user group button next to the Edit policy icon will take users to the user group management. Please refer concerned section for more details.



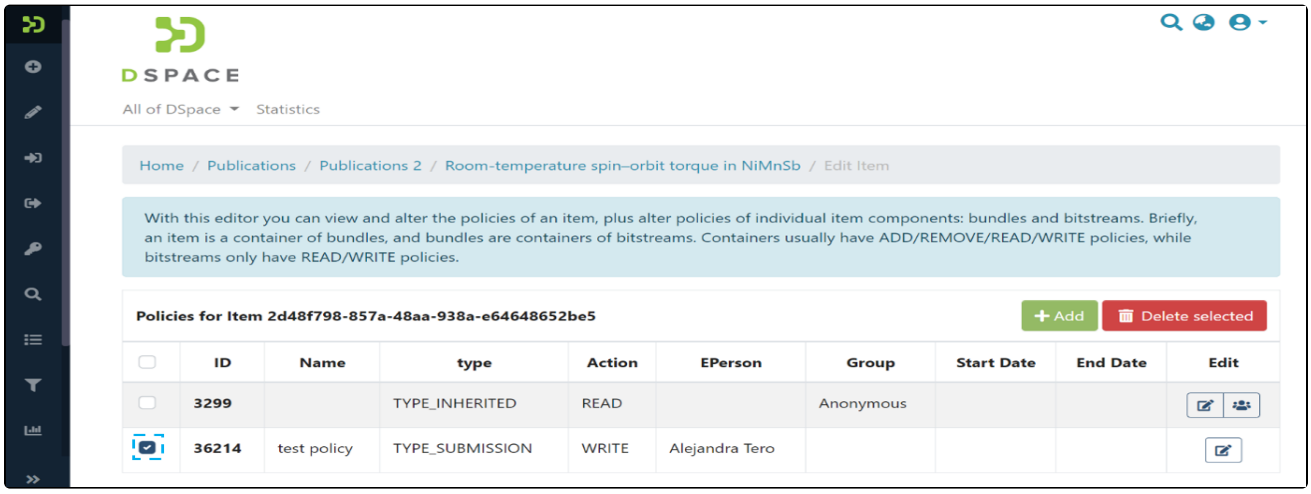
Step 2: Update policy information on the "Edit resource policy" page and click on the "Submit" button. Please see the description of each field appearing on the "Edit resource policy" page after the screenshot.



1. **Name:** Enter Policy name.
2. **Description:** Enter the Policy description for future reference and understanding of other users.
3. **Select the policy type:** The user can select one of the following policy types:
    a. TYPE_SUBMISSION: a policy in place during the submission

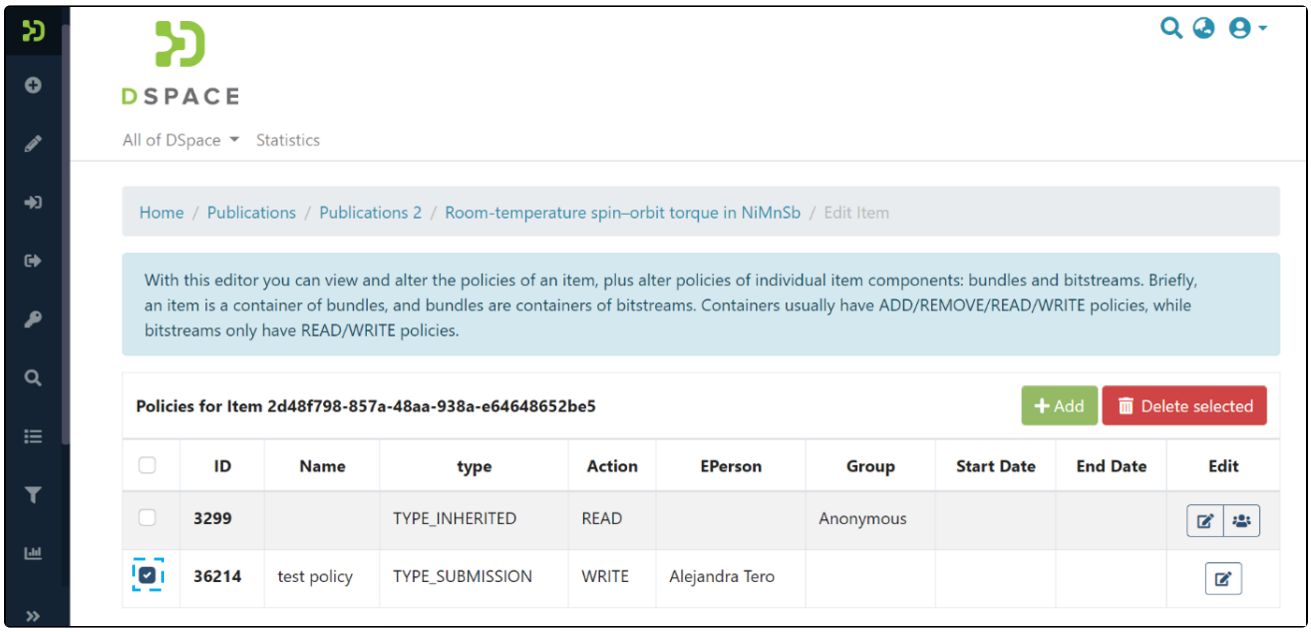
   b. TYPE_WORKFLOW: a policy in place during the approval workflow
   c. TYPE_INHERITED: a policy that has been inherited from a container (the collection)
   d. TYPE_CUSTOM: a policy defined by the user during the submission or workflow phase
   e. null: if the information is not available
4. **Select the action type:** The user can select one of the following actions from the drop-down list:
   a. READ - Ability to view the specific object
   b. WRITE - Ability to edit the specific object
   c. ADD - Ability to add an object to a parent object (e.g. add a Bitstream to a Bundle, add a Bundle to an Item, add an Item to a Collection , add a Collection to a Community, etc.)
   d. REMOVE - Ability to remove an object from a parent object (i.e. the opposite of "ADD")
   e. ADMIN - Full Admin permissions on the specific object
   f. DELETE - Ability to delete the specific object
   g. WITHDRAWN_READ - Ability to view a withdrawn Item (only valid for Items)
   h. DEFAULT_BITSTREAM_READ - Default "READ" permissions added to any Bitstreams of a newly submitted Item (only valid for Collections)
   i. DEFAULT_ITEM_READ - Default "READ" permissions added to any newly submitted Item (only valid for Collections)
5. **Start date – end date:** The user can select a start date and end date for using the policy, should they want to apply it for a fixed period.
6. **The eperson or group granted the permission is the l**ist of users/groups granted authorization under the policy.
7. **Search for an ePerson / Search for a group:** Click on the "ePerson" or "group" to search for the entity.
8. **Search field:** Enter keywords for searching the ePerson/Group.
9. **ePerson/Group list:** Click on the select button against the user/group you want to add to the policy
10. **Submit/Cancel button:** Click on the "Submit" button to complete policy creation or click on the Cancel button to cancel the entire process.

As shown below, users will see a success prompt upon the policy creation and be back on the Manage Policies screen.



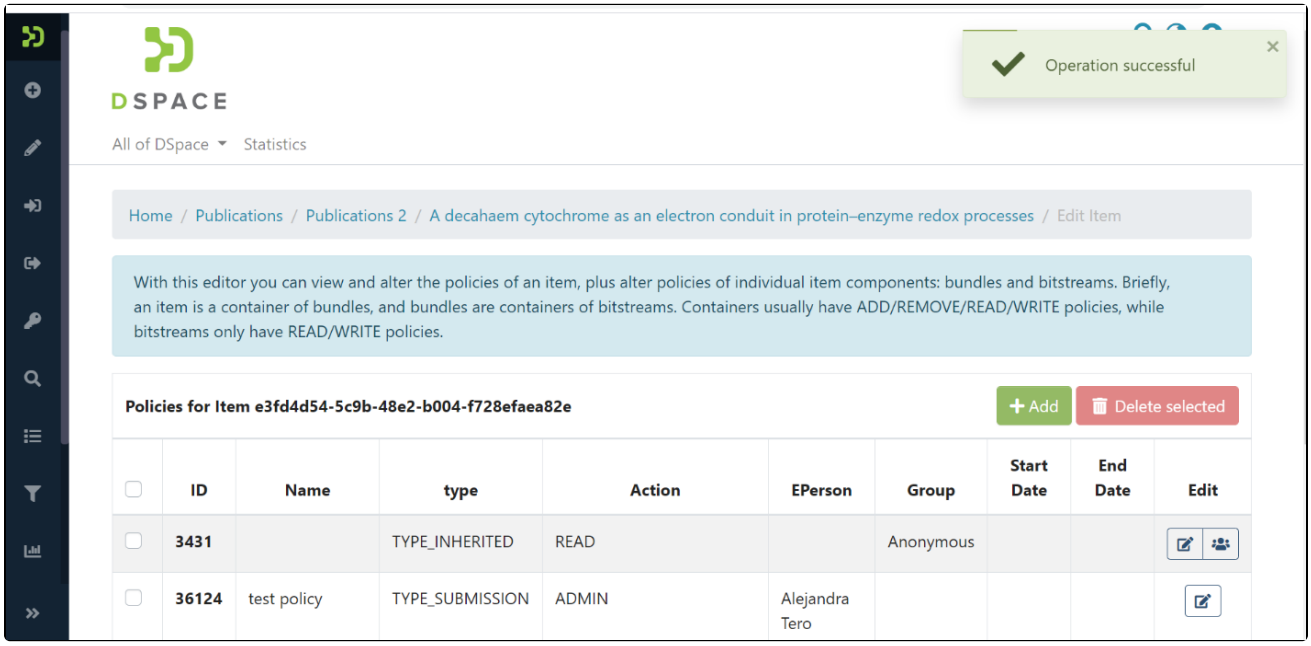
Delete Policy

Step 1: Click on the check box on the left-hand side of each policy, and the "Delete Selected" button will be activated.



Step 2: Click the "Delete selected" button to delete the policy. Please note that deleted policy is irrecoverable.

You'll see success prompt upon deletion of the selected policy.

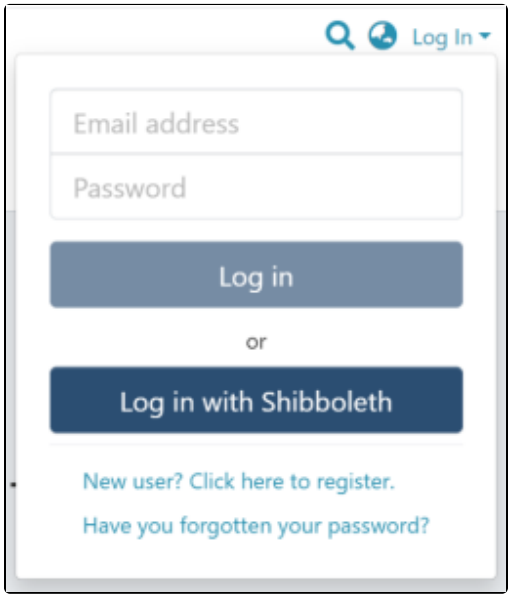


Collection Mapper

- Manage mapped items to collections
- Manage Mapped Collections
- Map new collections

Manage mapped items to collections

Step 1: Login using your credentials

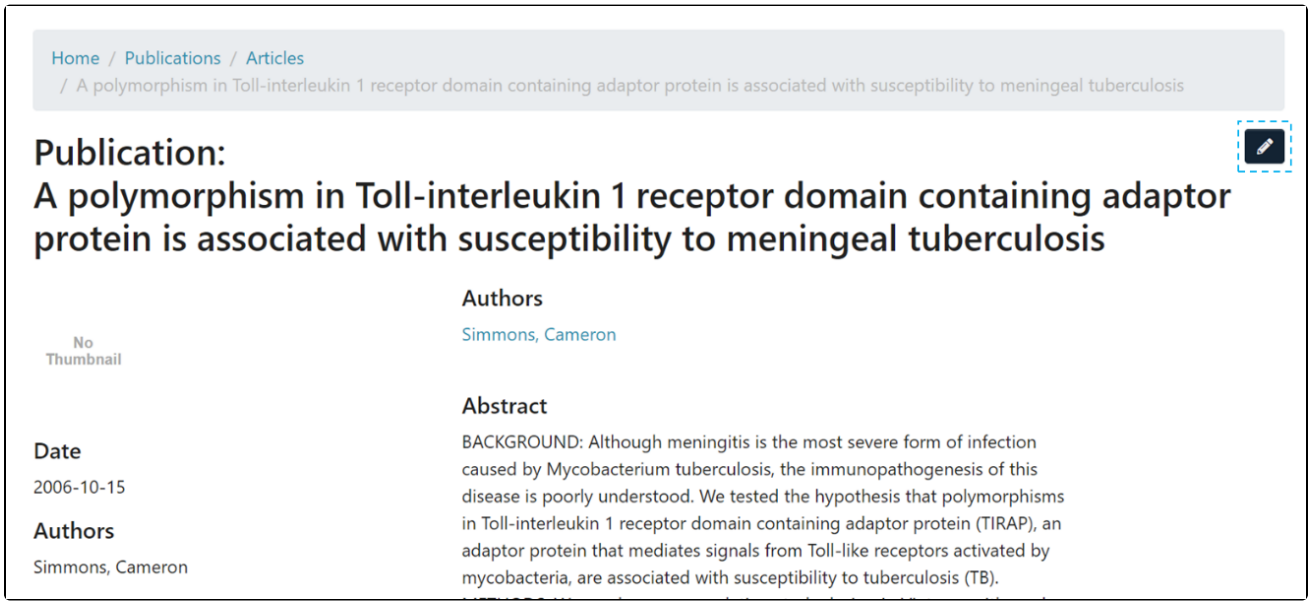


Step 2: Go to the Item you want to edit

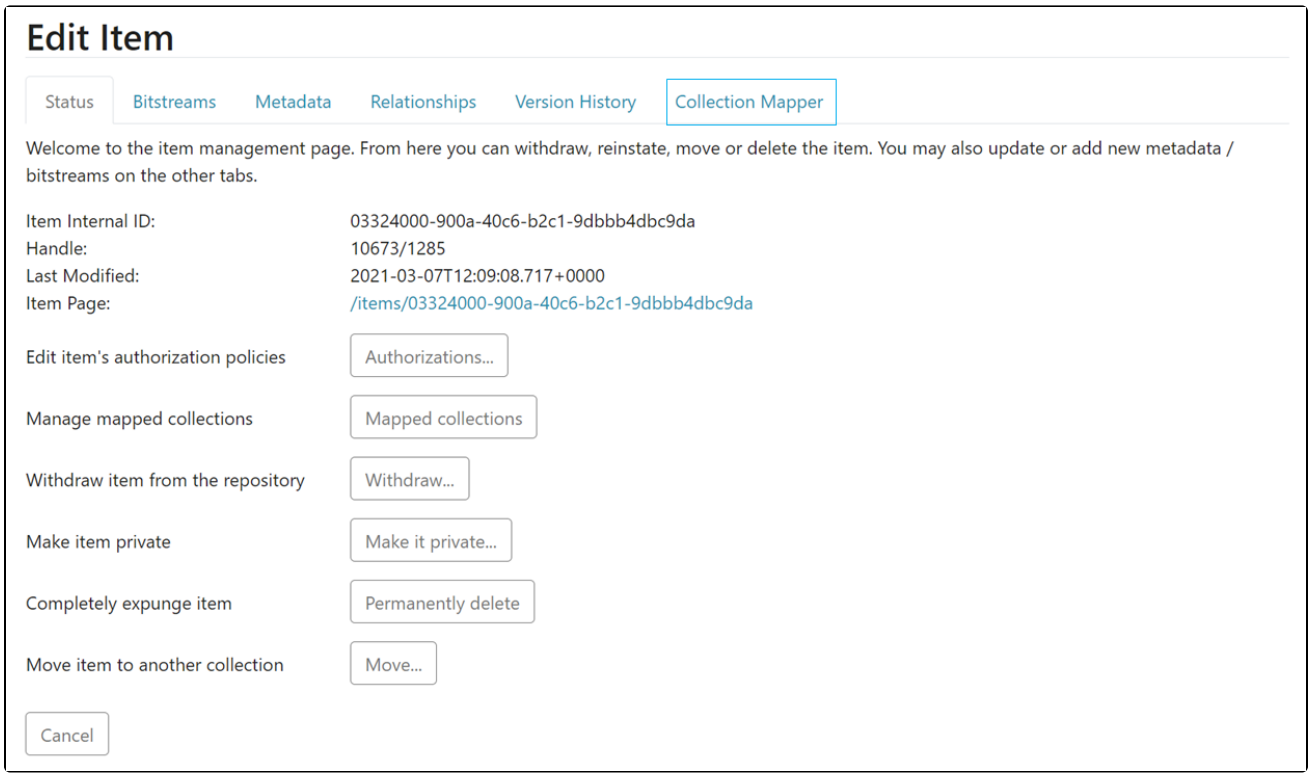
Users can reach an item through multiple methods, as listed below:

1. Search an item
2. Browse communities and collections
3. Finding an item in the Administration section at Edit > Item

Click the "Edit" button on the item title's right-hand side.



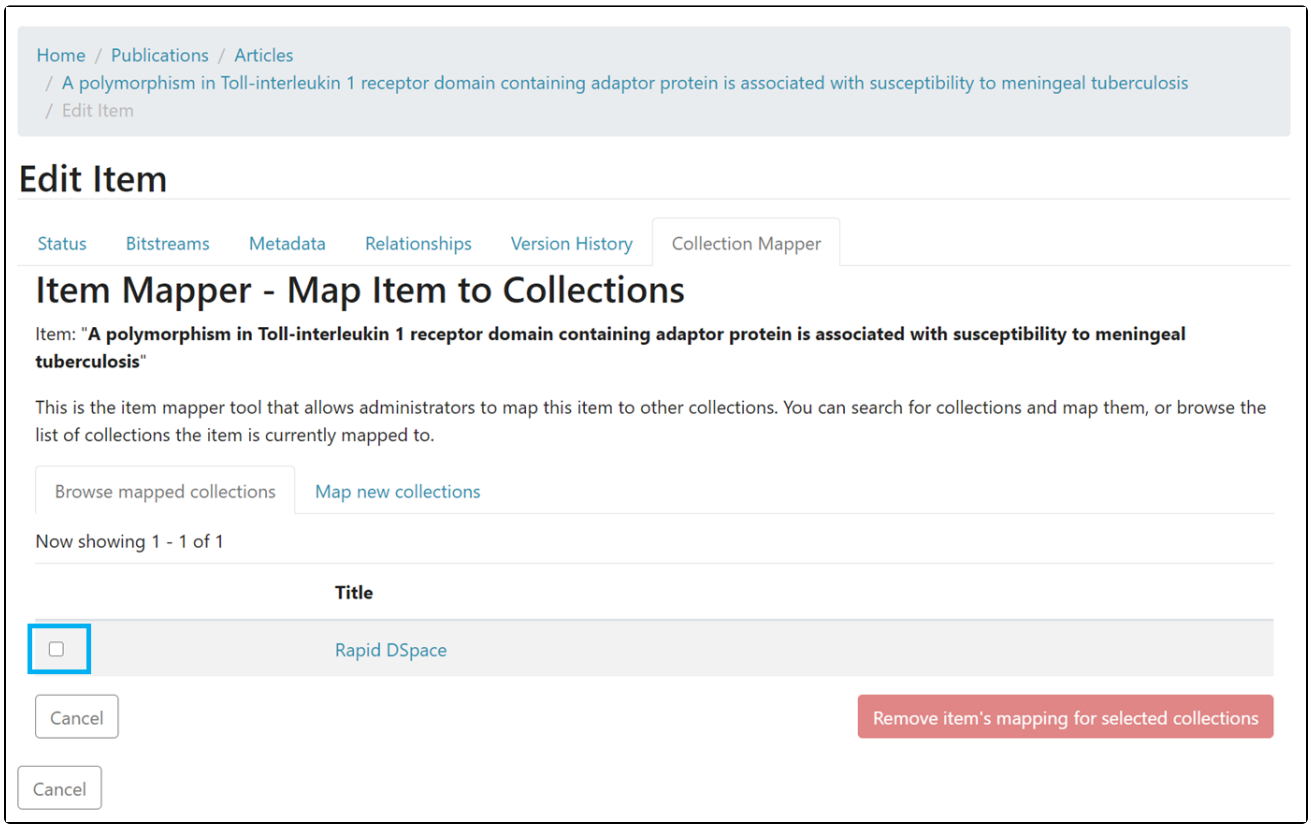
Step 3: Click the "Collection Mapper" tab to move to the section.



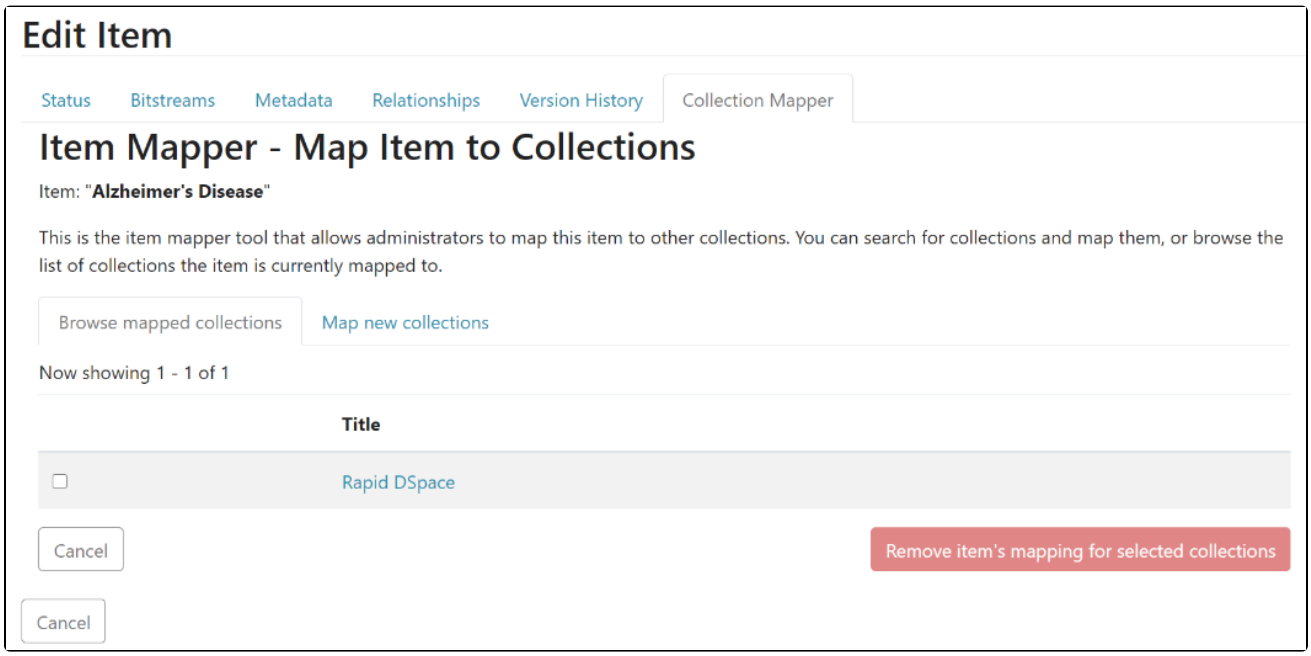
> *Users can perform multiple functions in the Collection Mapper tab. These are:*
>
> 1. *Manage existing collections mapped to the Item*
> 2. *Map fresh collections to the Item*

Manage Mapped Collections

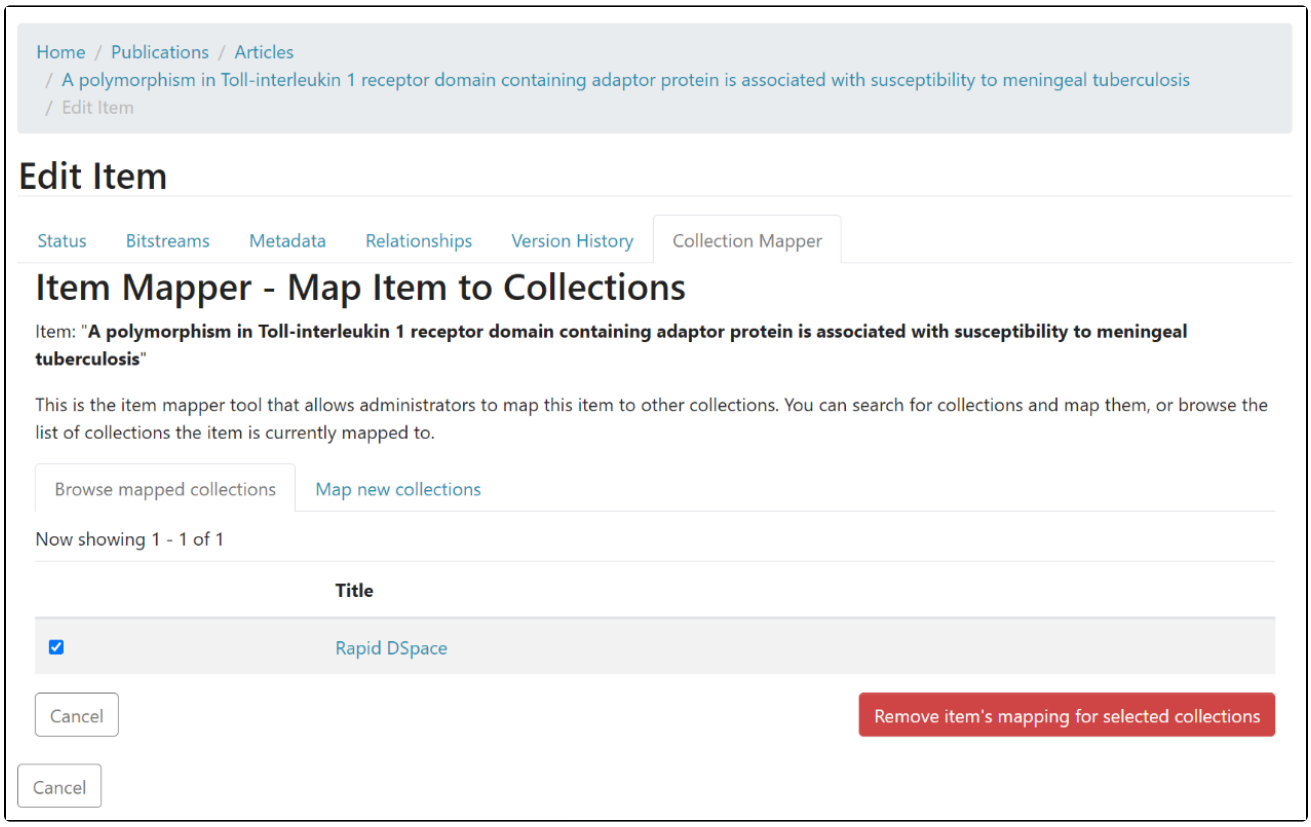
Step 1: If you want to delete an existing collection mapped with the Item, click the checkbox on the left of the collection name to select it.



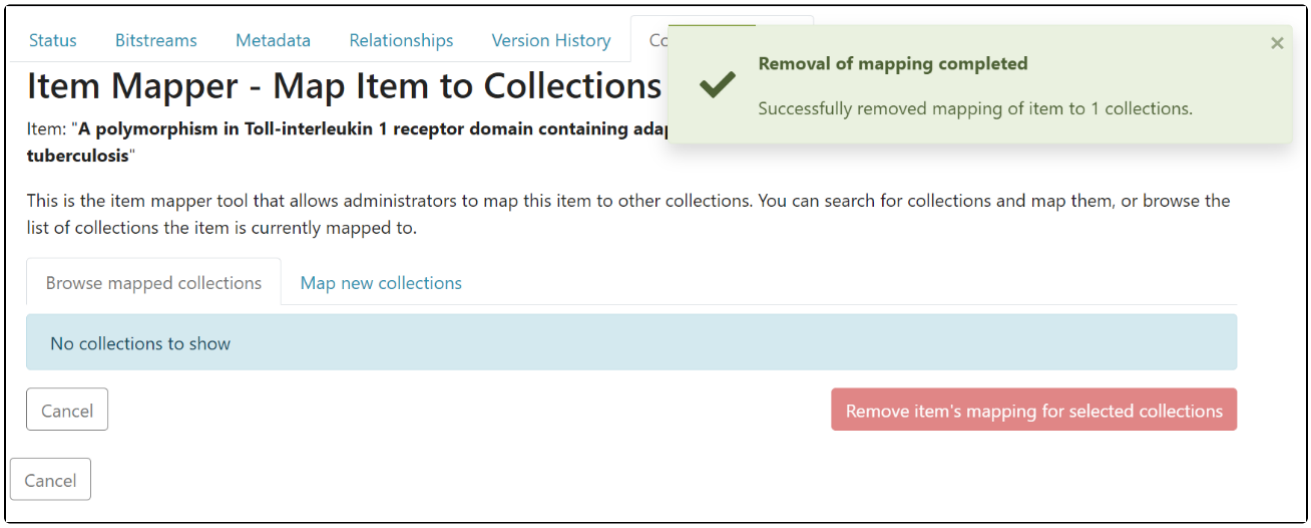
The "Remove item's mapping for selected collections" button will activate upon selecting the collection.



Step 2: Click on “Remove item’s mapping for selected collections” to unmap selected collection(s) or click “Cancel” to cancel the operation.

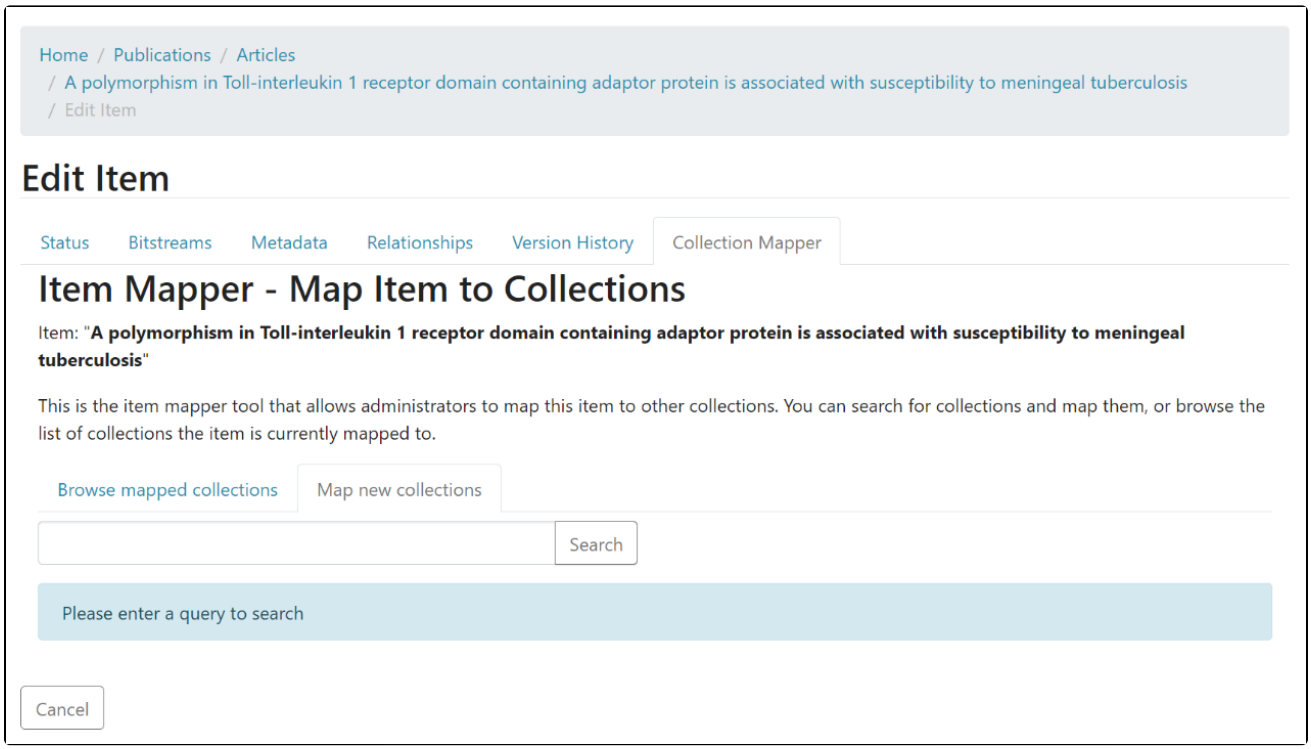


A prompt confirming successful unmapping of the collection will appear, and the selected collection will disappear from the list in the “Browse mapped collections” tab.

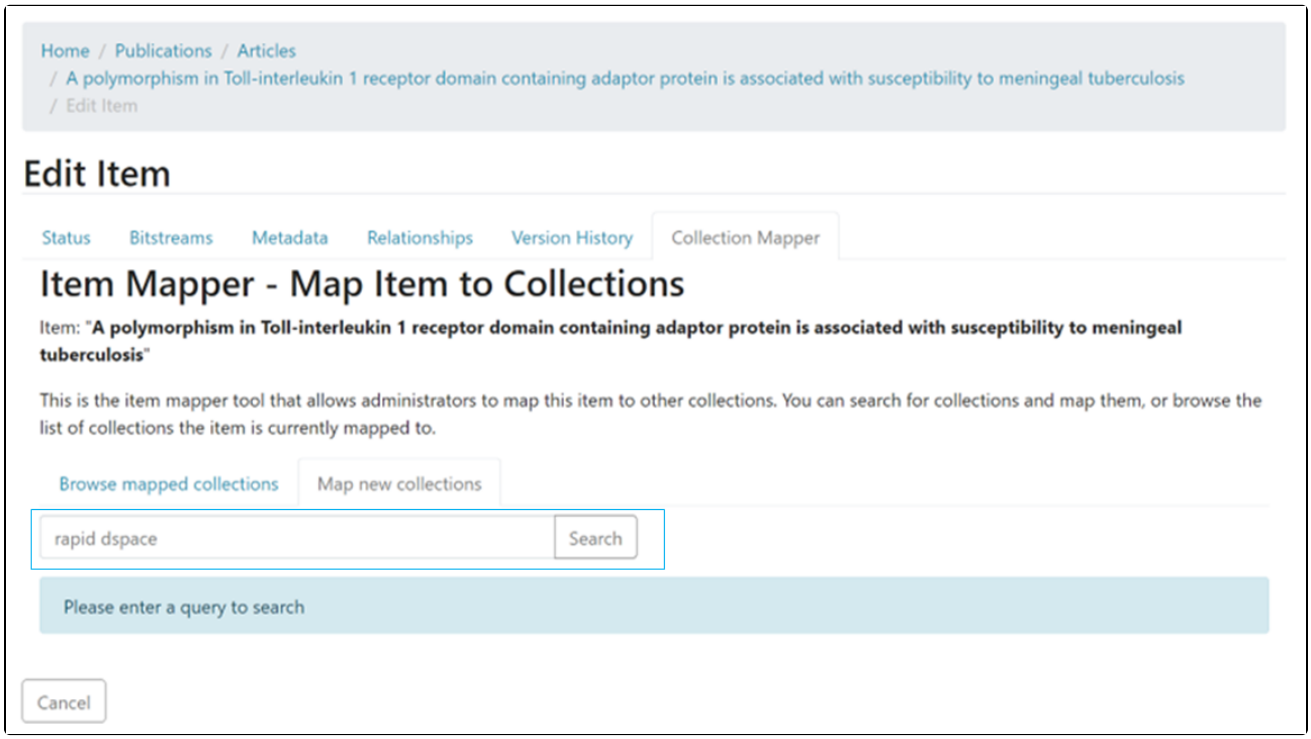


Map new collections

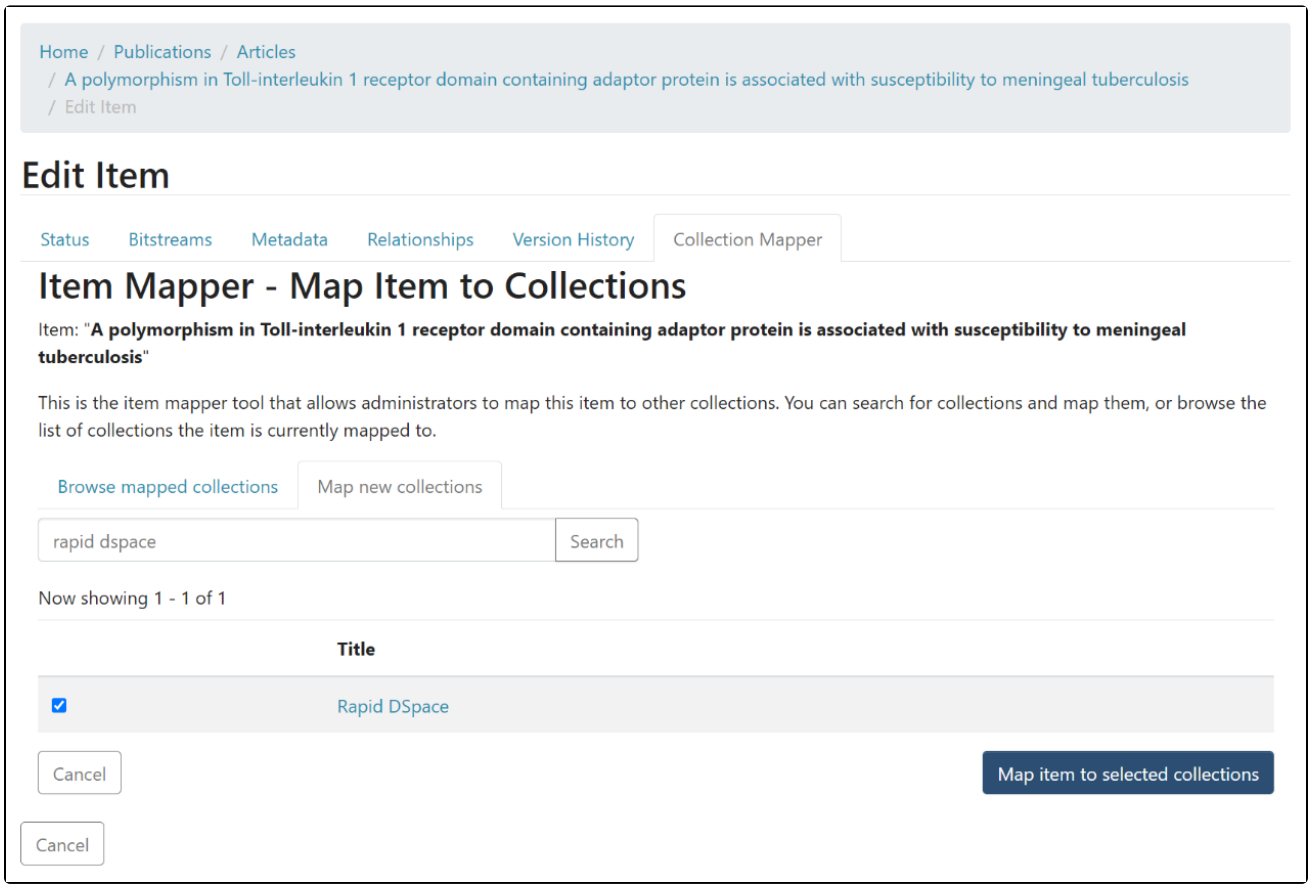
Step 1: Click on the “Map new collections” tab to map a fresh collection with the Item.



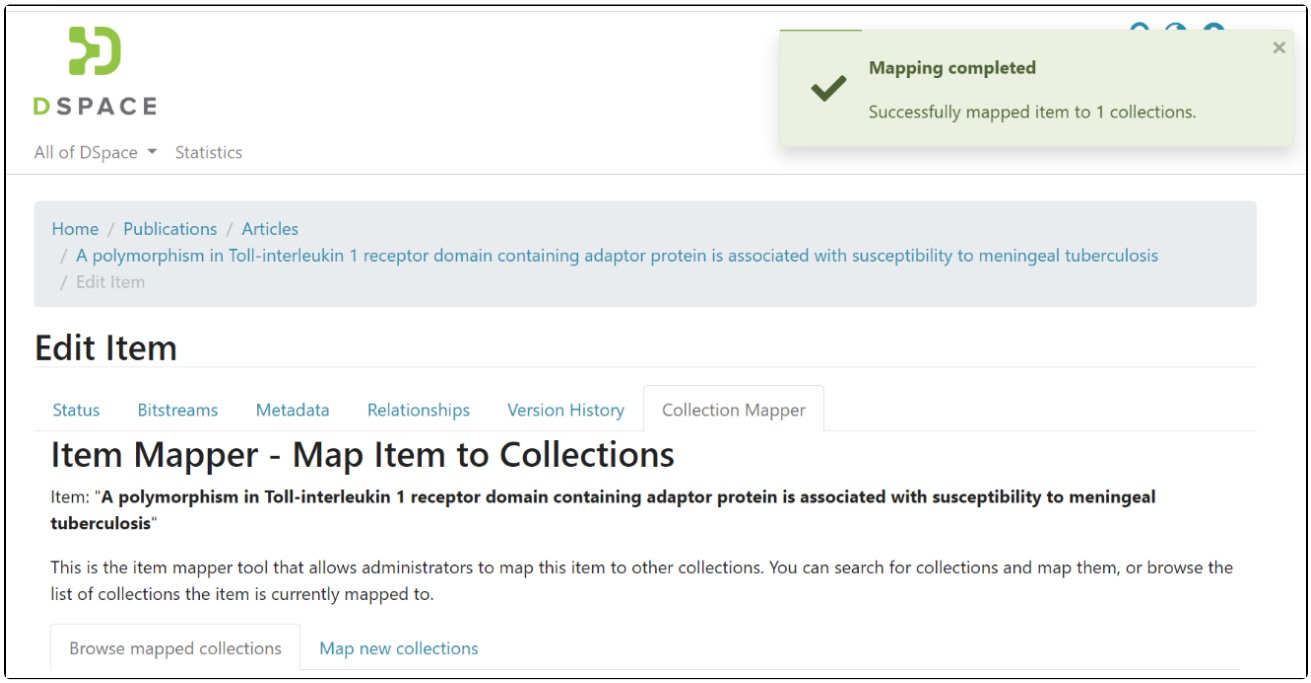
Step 2: Enter the name of the collection you want to map with this Item and click on the “Search” button.



Step 3: Click the checkbox on the left of target collections to select them. Click the “Map item to selected collections” button to complete mapping, or use the “Cancel” button to cancel the operation.



A success prompt confirming collection mapping will appear. The selected collection will appear under the “Browse mapped collections” tab.

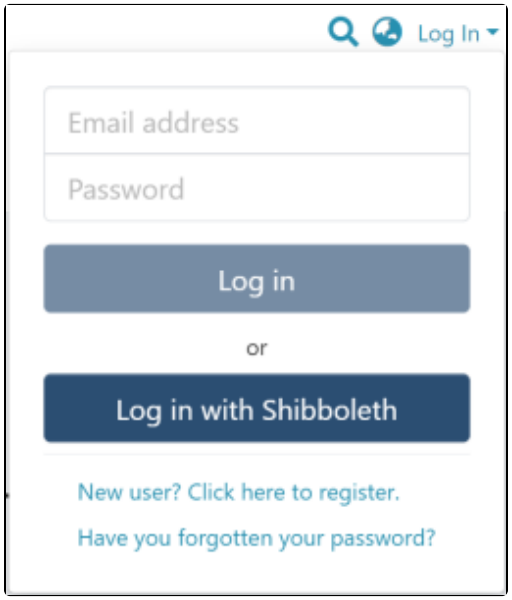


Edit Bitstream

- Edit Bitstream Process
- Add a Bitstream or Bundle in an item
- Edit a Bitstream or Bundle in an item

Edit Bitstream Process

Step 1: Login using your credentials

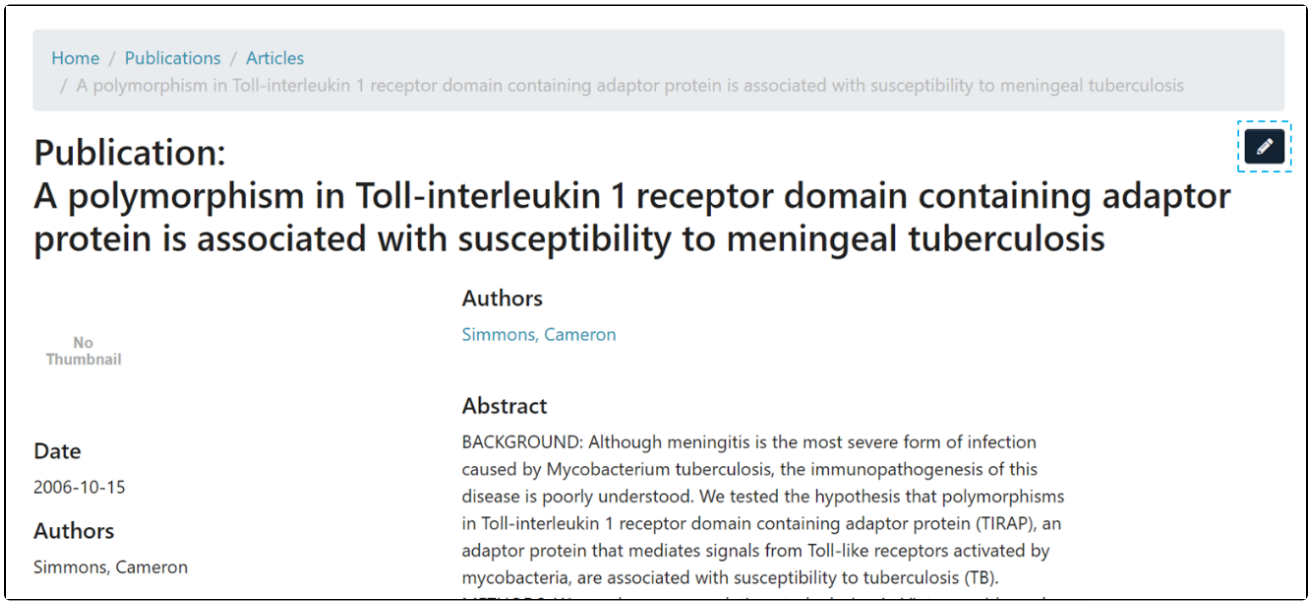


Step 2: Go to the Item you want to edit

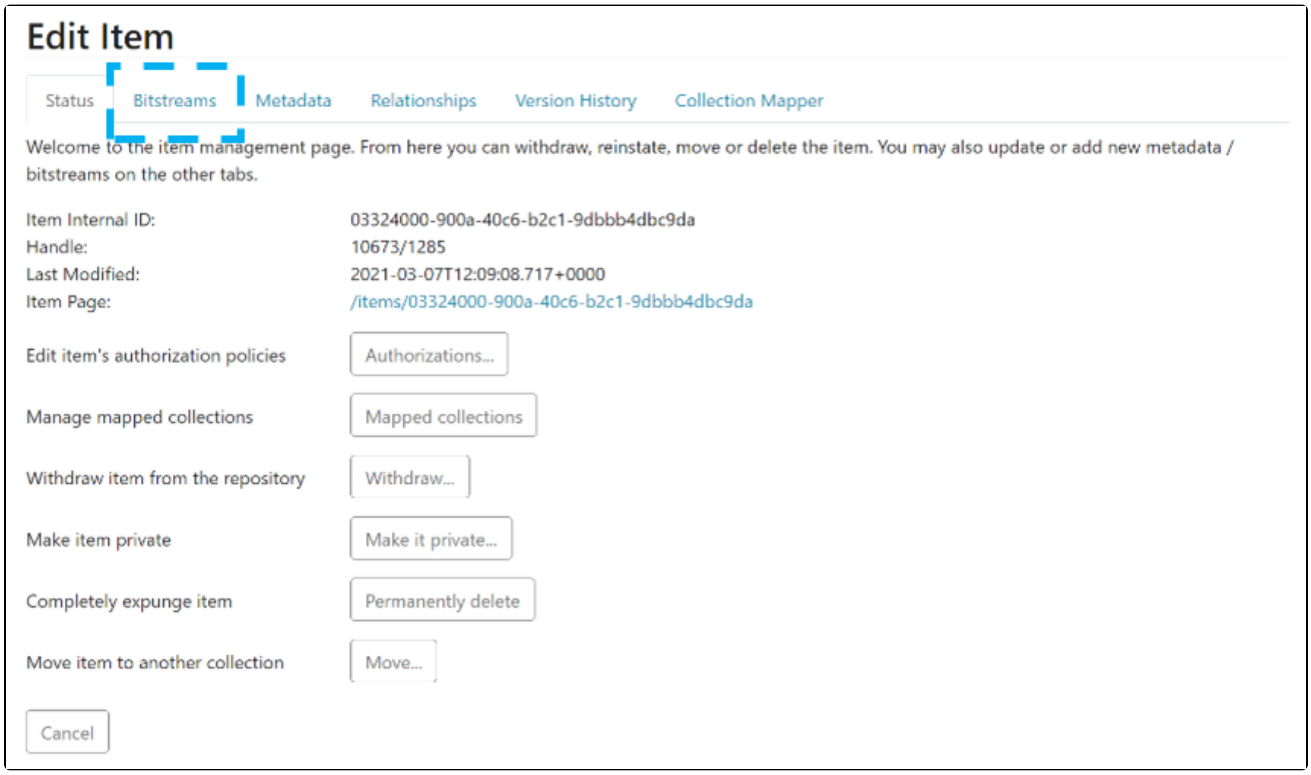
Users can reach an item through multiple methods, as listed below:

1. Search for an item
2. Browse communities and collections
3. Finding an item in the Administration section at Edit > Item

Click on the “Edit” button appearing on the right-hand side of the item title.



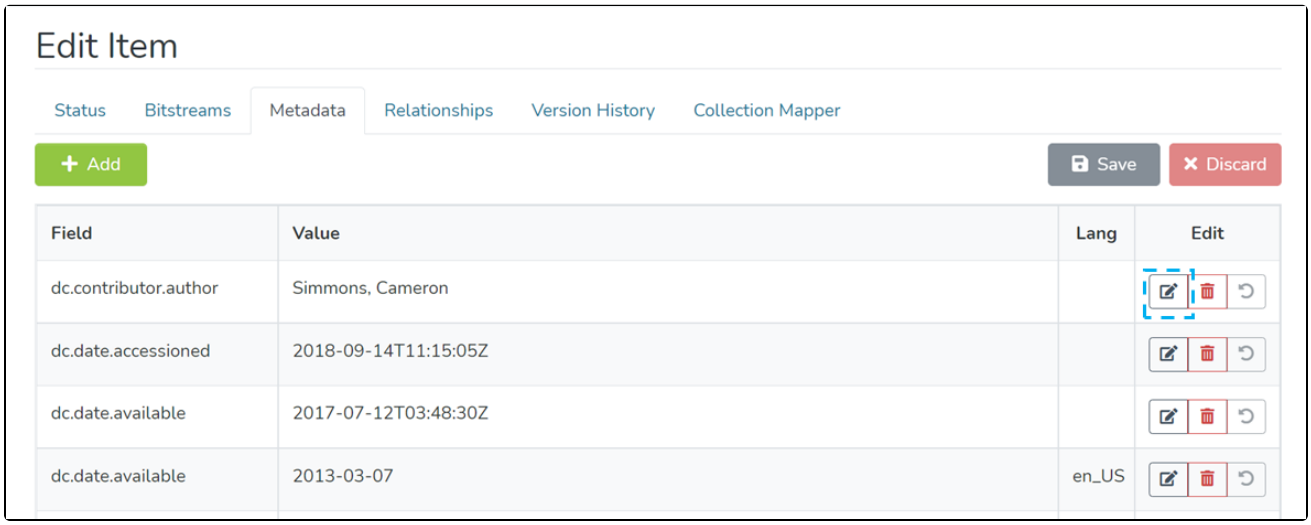
Step 3: Click on the “Bitstreams” tab to edit the metadata.



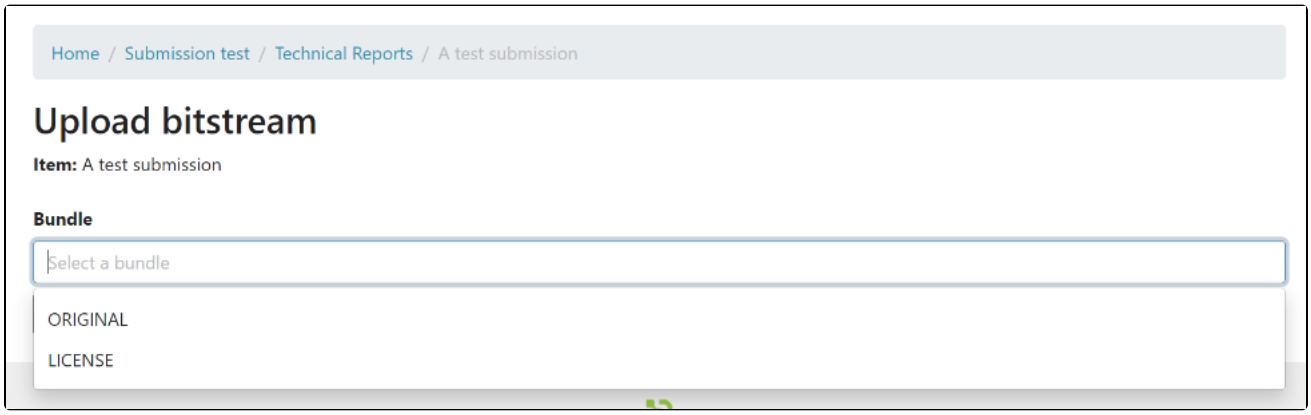
> *Users can perform multiple functions in the bitstream tab. These are:*
>
> 1. *Adding a bitstream or bundle in an item*
> 2. *Updating or deleting an existing bundle in an item*
> 3. *Update/delete/add bitstream(s) in an existing bundle*

Add a Bitstream or Bundle in an item

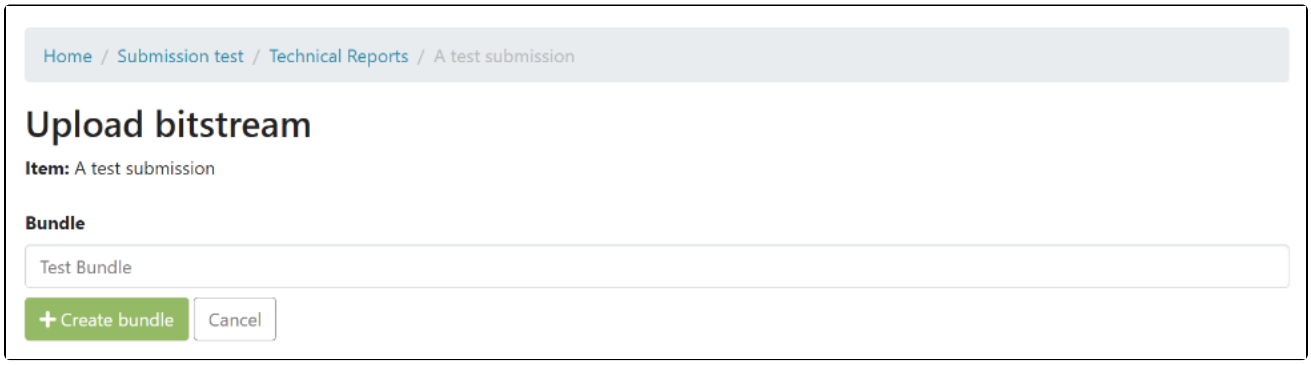
Step 1: Click on the “Upload” button to upload a bitstream in existing bundles or create a new bundle to add a bitstream.



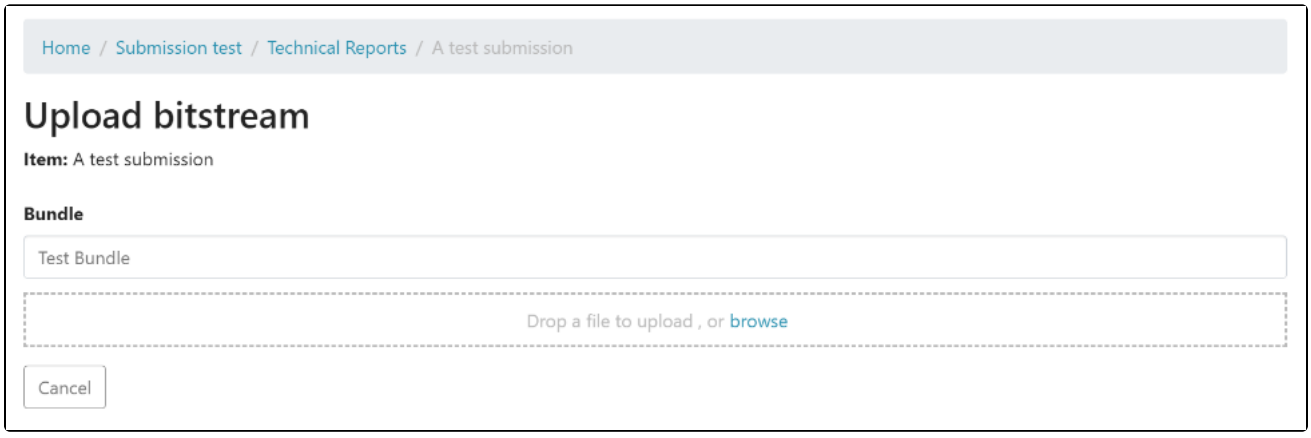
Step 2: Enter the bundle name or select existing names appearing in the dropdown list.



Step 3: Click on the “Create bundle” button to create a bundle or click on “Cancel” to cancel the operation.



Step 4: Drag and drop the bitstream(s) you want to attach to the bundle, or you can click on the “browse” link appearing in the file upload section.

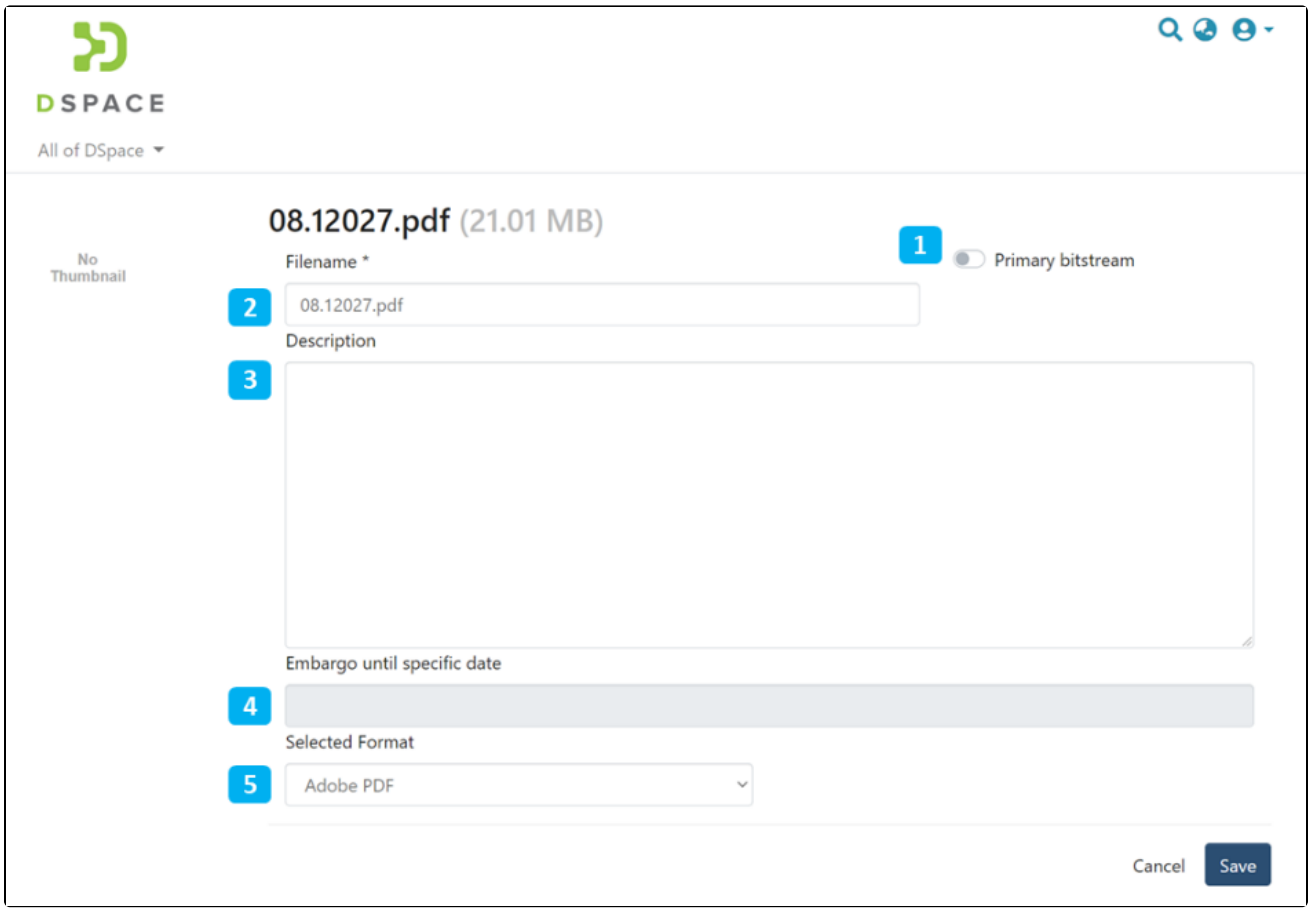


Step 5: After a successful bitstream(s) upload, the user can add more details about the bitstream on the next screen. Click on the “Save” button at the bottom of the page to save details, or click the “Cancel” button to discard updates.

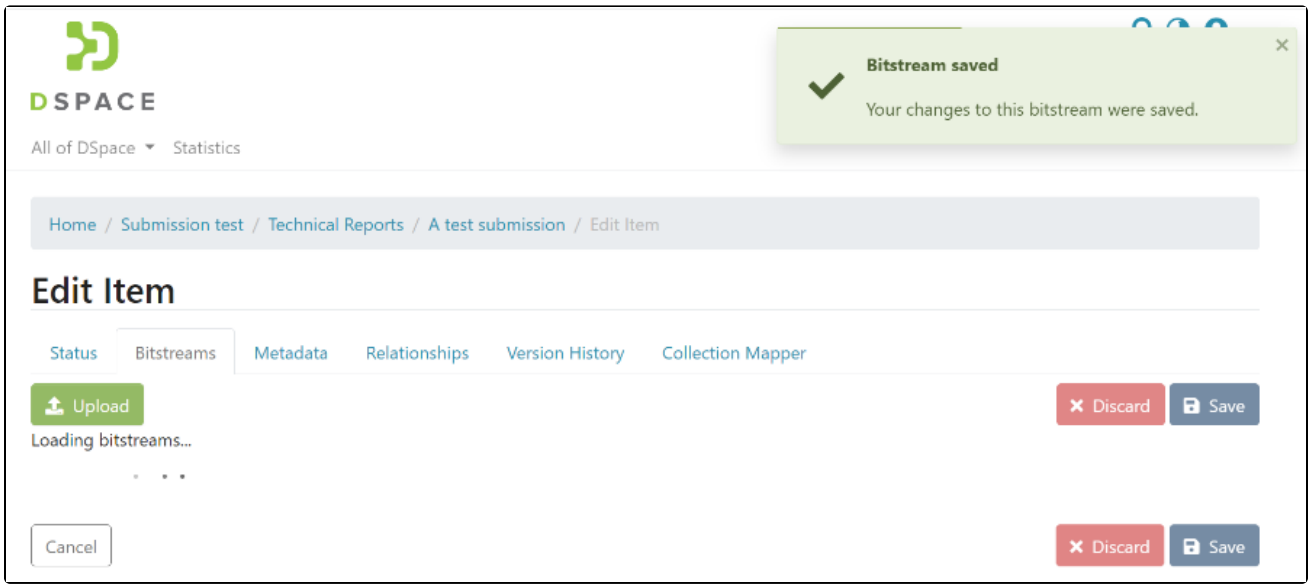
Below is the description of various fields appearing on this page

1. Primary bitstream: Click on this button to set the bitstream as the primary bitstream for that bundle. If another bitstream was the primary bitstream beforehand, it will be replaced with this one. The thumbnail for the primary bitstream of the ORIGINAL bundle will be used as the main thumbnail for the item. (Note that the image media viewer currently doesn't take primary bitstreams into account, see https://github.com/DSpace/dspace-angular/issues/2310)
2. The default value appearing in this field is the attachment’s filename. However, users can replace it with the value of their choice.

3. Description: Users add a description of the attachment in this field.
4. Embargo until date: Users can select a future date to restrict public access to the attachment. Additionally, users can grant access to a specific set of users by selecting user groups.
5. Selected format: If the file extension of the uploaded attachment exists in the DSpace's bitstream registry, the user will see the registered value for extension in this field. Users can change the value by selecting another one using the dropdown.

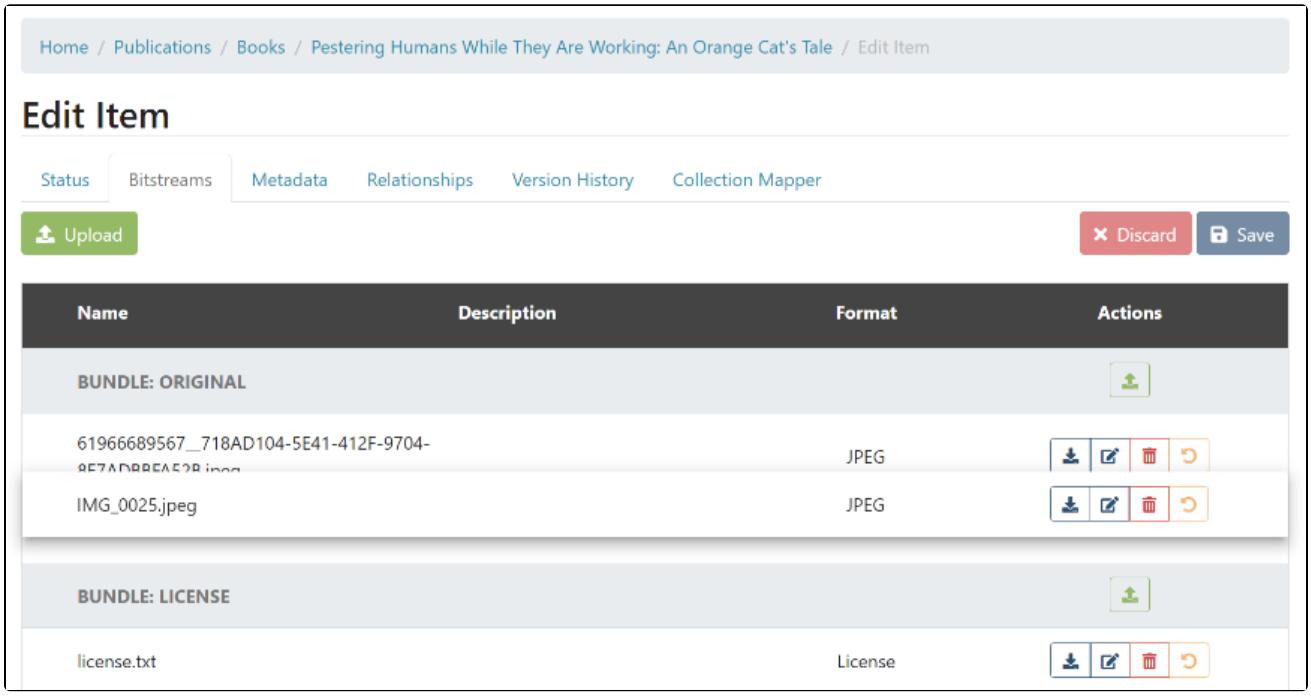


After clicking the "Save" button, the user will be redirected to the bitstream tab. A prompt confirming success or failure will appear.



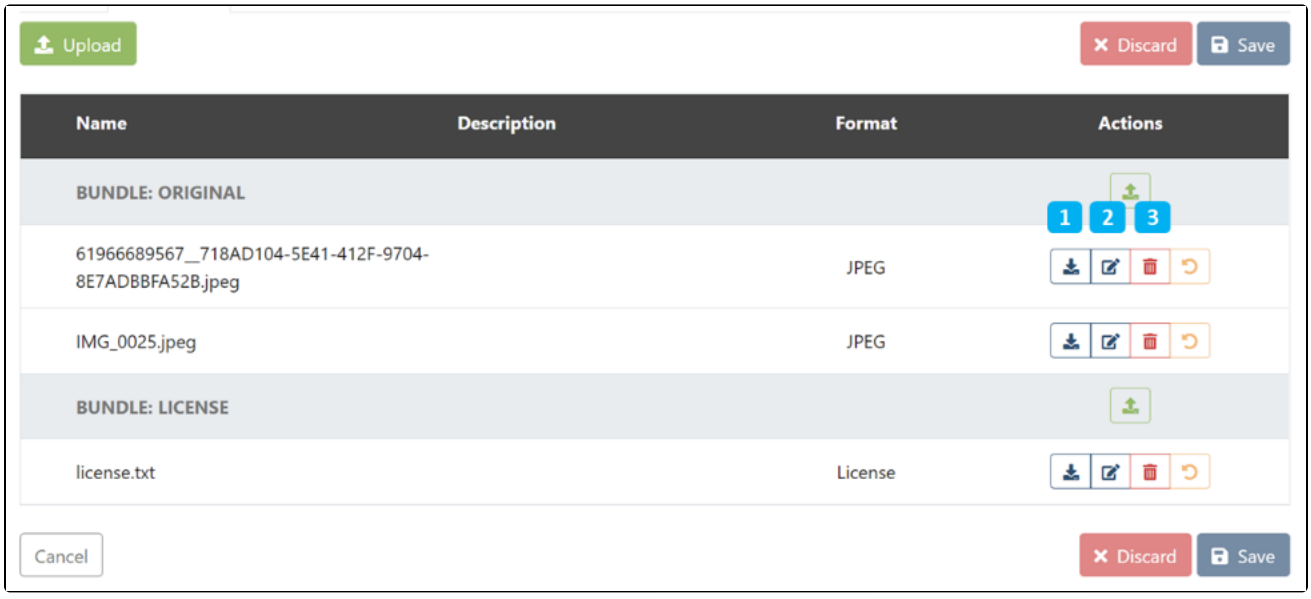
Edit a Bitstream or Bundle in an item

Step 1: Click on a bitstream and drag it above or below another bitstream(s) in the bundle to change the bitstream's sequence.



Step 2: Apart from adjusting the bitstream's sequence in a bundle, below are other options available:

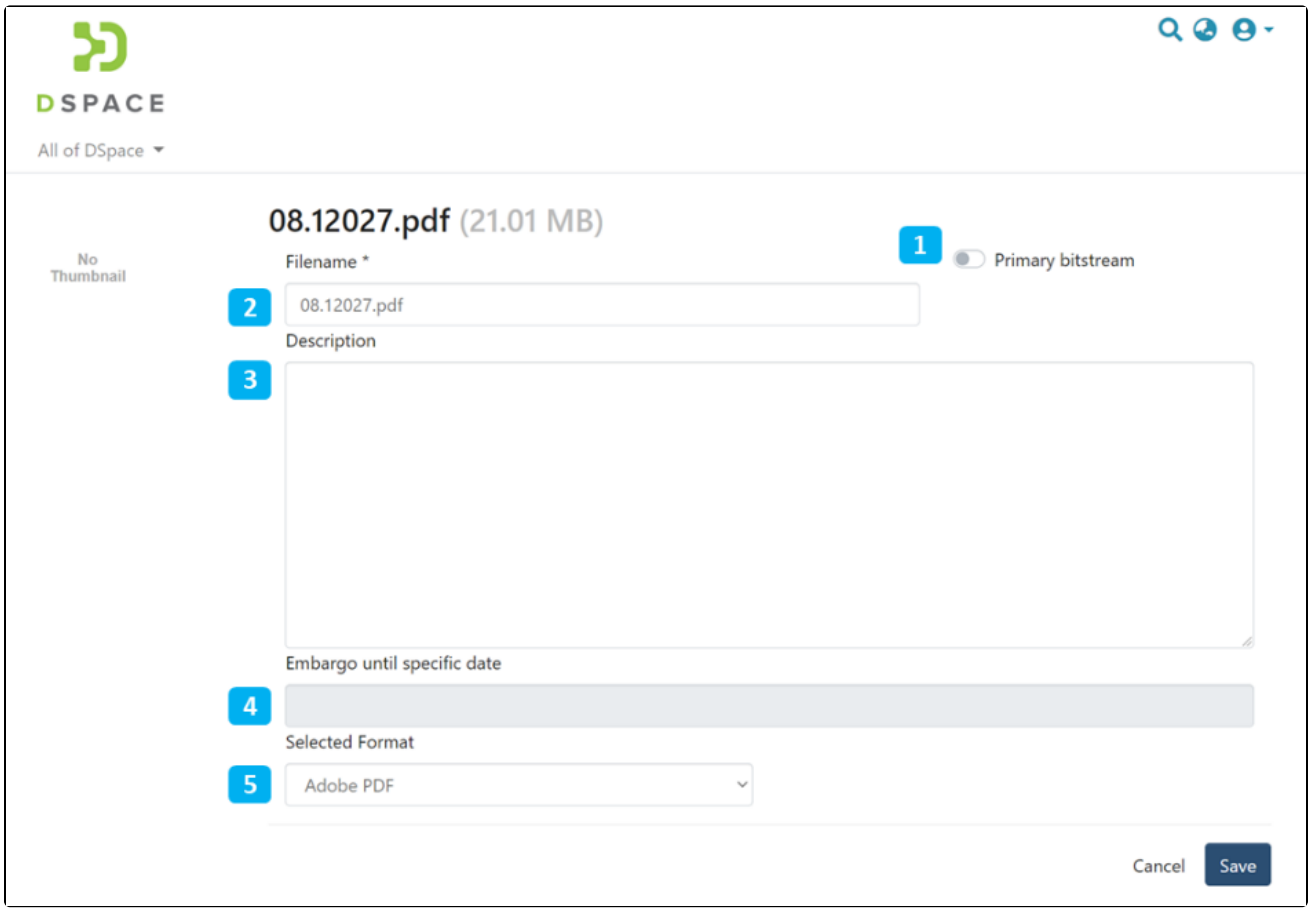
1. Download Bitstream: Click on this button to download the attachment on the local device.
2. Edit Bitstream: Click on the "Edit bitstream" button to edit details. More details are given in the following steps.
3. Delete Bitstream: Click on the Delete Bitstream button to delete bitstream from the bundle.



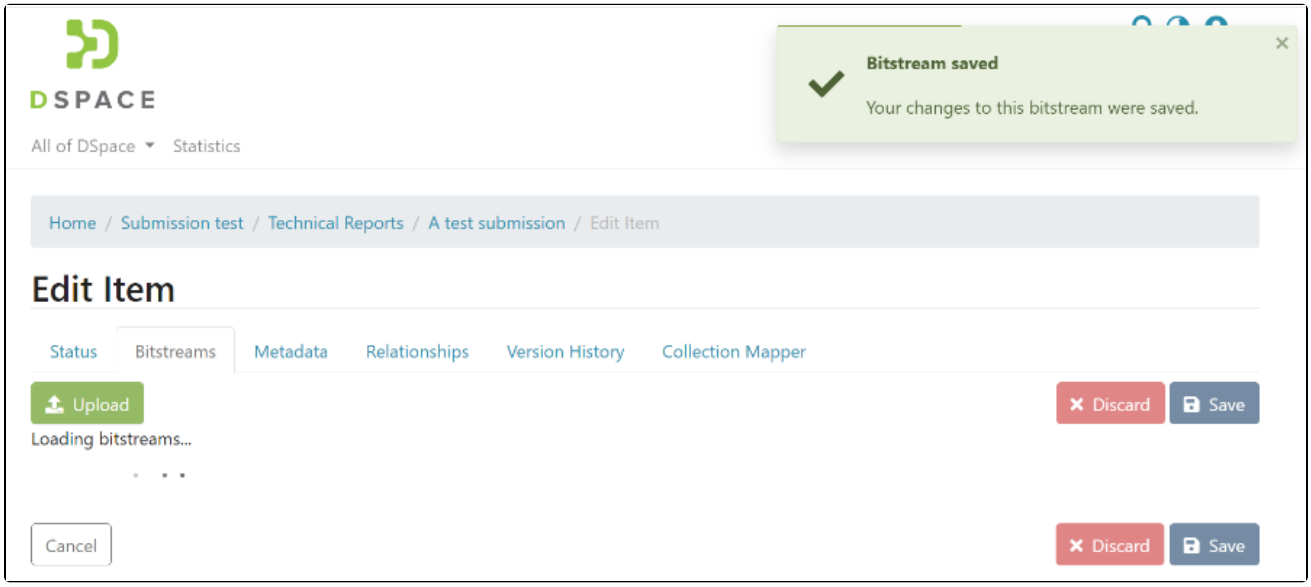
Step 3: Click on the Edit button shown on the screen above to edit the bitstream details. Below is the description of various fields appearing on this form

1. Primary bitstream: Click on this button to set the bitstream as the primary bitstream for that bundle. If another bitstream was the primary bitstream beforehand, it will be replaced with this one. The thumbnail for the primary bitstream of the ORIGINAL bundle will be used as the main thumbnail for the item. (Note that the image media viewer currently doesn't take primary bitstreams into account, see https://github.com/DSpace/dspace-angular/issues/2310)
2. Filename: The default value appearing in this field is the attachment's filename. Users can replace it with the value of their choice.

3. Description: Users add a description of the attachment in this field.
4. Embargo until date: Users can select a future date to restrict public access to the attachment. Additionally, users can grant access to a specific set of users by selecting user groups.
5. Selected format: If the file extension of the uploaded attachment exists in the DSpace's bitstream registry, the user will see the registered value for extension in this field. Users can change the value by selecting another one using the dropdown.

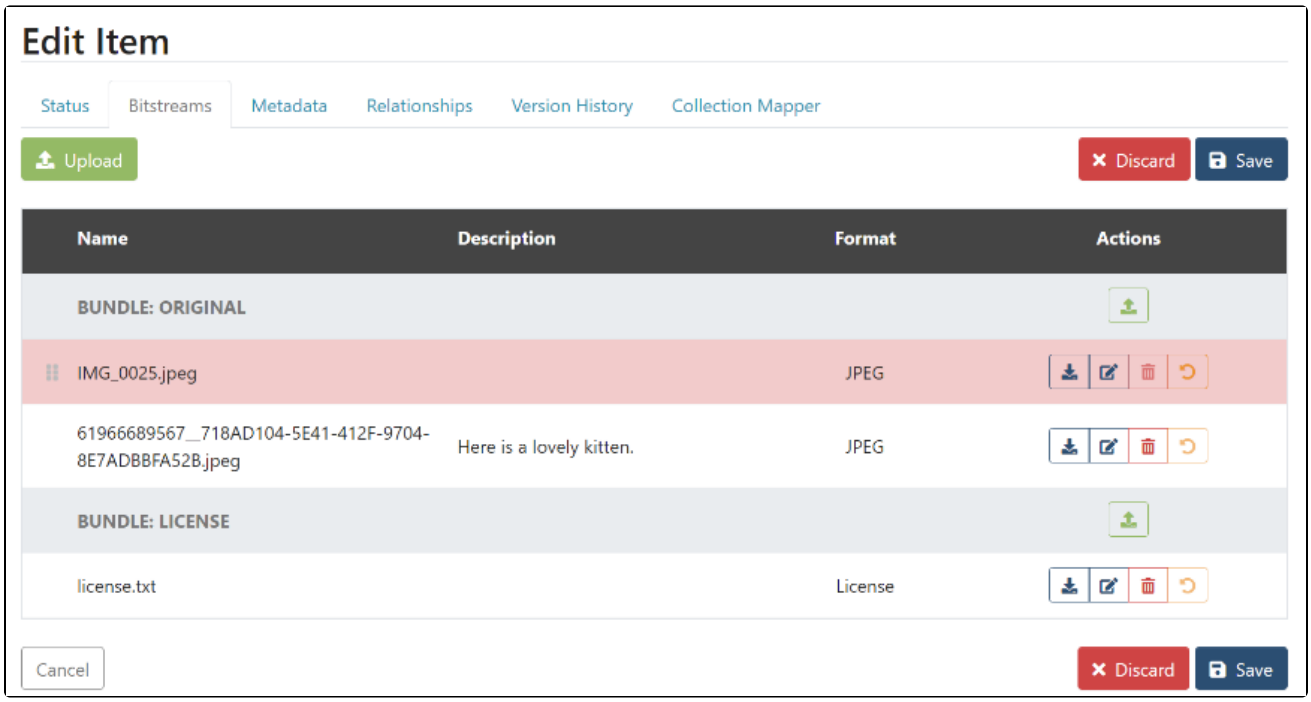


After clicking the "Save" button, the user will be redirected to the bitstream tab. A prompt confirming success or failure will appear.



Step 4: Click on the Delete button to delete a bitstream. The attachment you want to delete will be highlighted in the red background for confirmation.

Click on the "Save" button appearing below the bitstreams list to continue with the deletion. Otherwise, click on the "Discard" button to cancel the process.

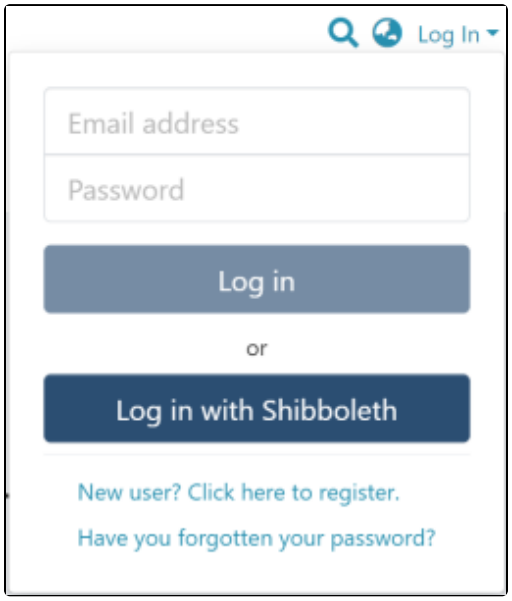

Edit Item
Status
Bitstreams
Metadata
Relationships
Version History
Collection Mapper
Upload
Discard
Save
Name
Description
Format
Actions
BUNDLE: ORIGINAL
IMG_0025.jpeg
JPEG
61966689567__718AD104-5E41-412F-9704-8E7ADBBFA52B.jpeg
Here is a lovely kitten.
JPEG
BUNDLE: LICENSE
license.txt
License
Cancel
Discard
Save

Edit Metadata

- Edit Metadata Process
- Add a metadata field
- Edit an existing metadata field
- Delete an existing metadata field

Edit Metadata Process

Step 1: Login using your credentials

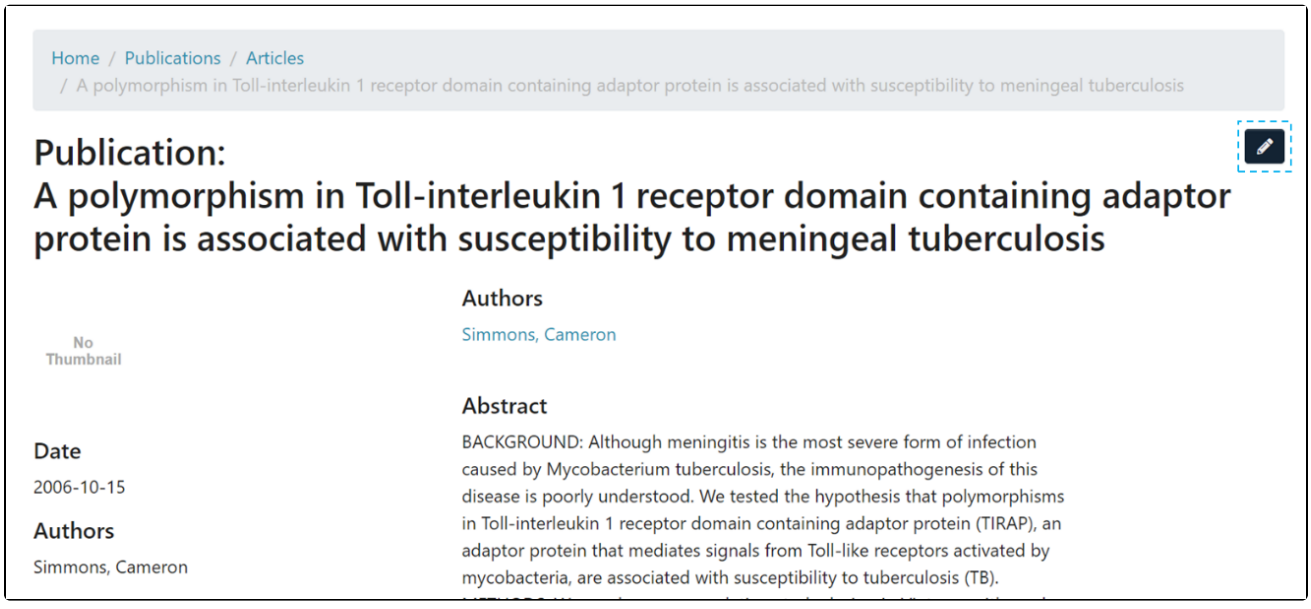


Step 2: Go to the Item you want to edit

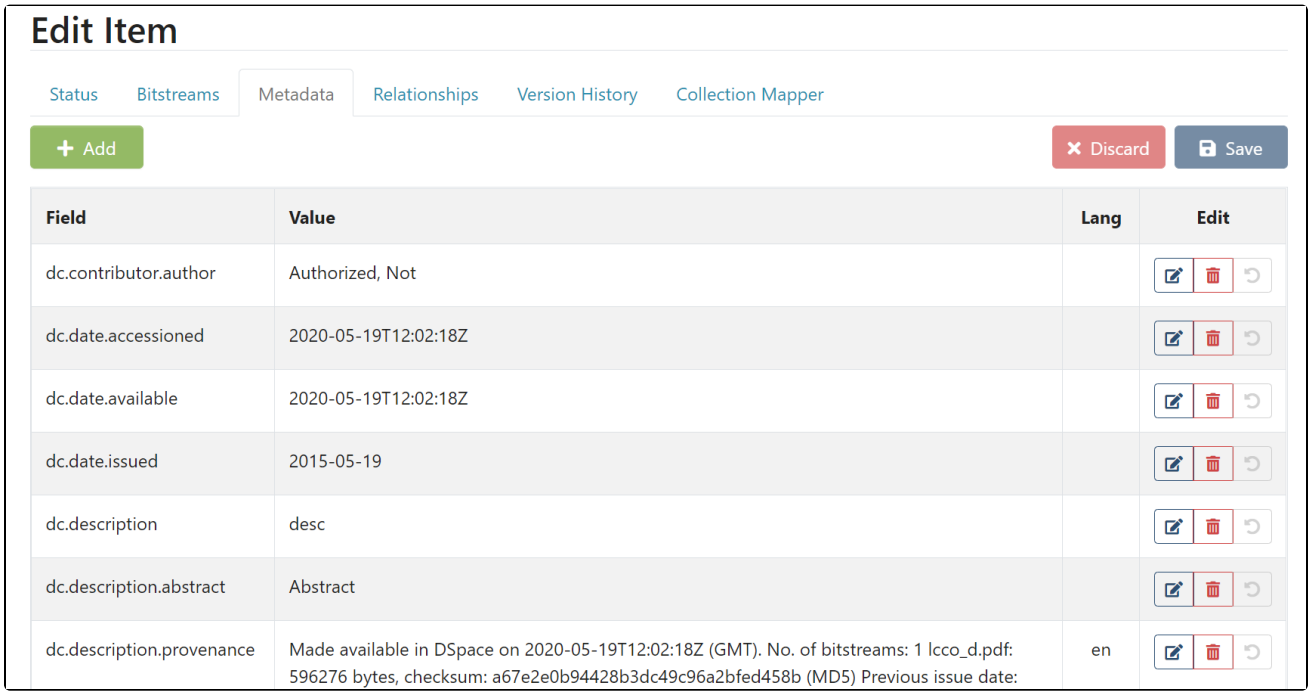
Users can reach an item through multiple methods, as listed below:

1. Search for an item
2. Browse communities and collections
3. Finding an item in the Administration section at Edit > Item

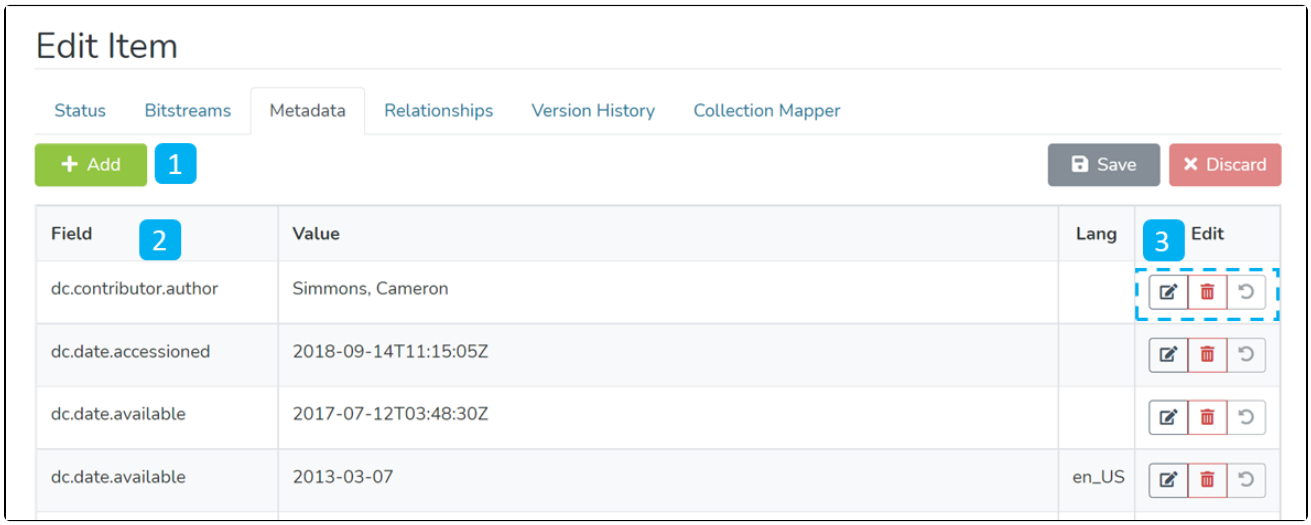
Click on the “Edit” button appearing on the right-hand side of the item title.



Step 3: Click on the “Metadata” tab to edit the metadata.



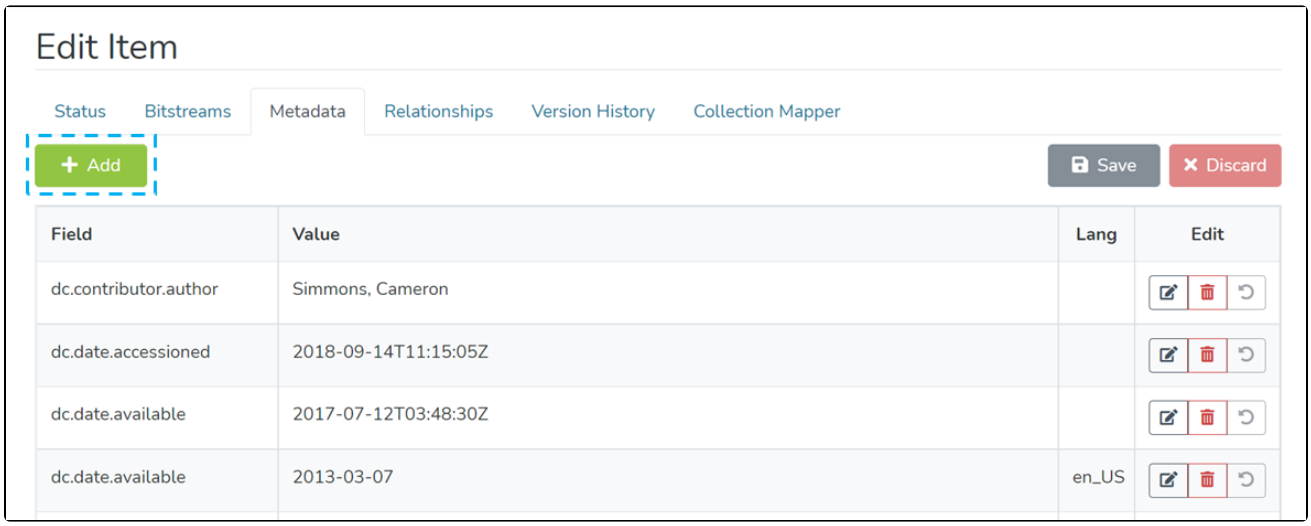
Step 4: Users can perform multiple actions in the Edit Metadata section, which are listed after the screenshot.



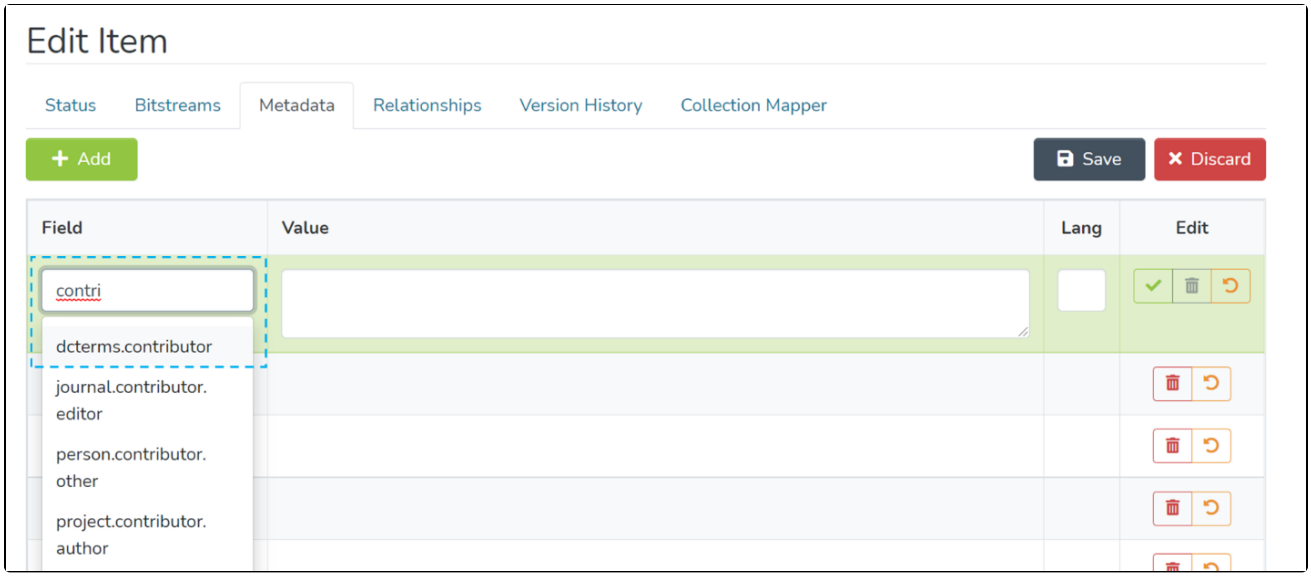
1. **Add:** Button used for adding new metadata elements in the existing Item.
2. **Metadata fields:** The fields column shows the metadata element’s value appearing in the “Value” column.
3. **Edit:** This panel contains various options to update the specific metadata field. They are
    a. Edit value – Users click on this button to edit the existing metadata value
    b. Delete metadata field – Click on this button for deleting the metadata field from the Item
    c. Undo changes – Click on this button for undo changes made in the metadata field

Add a metadata field

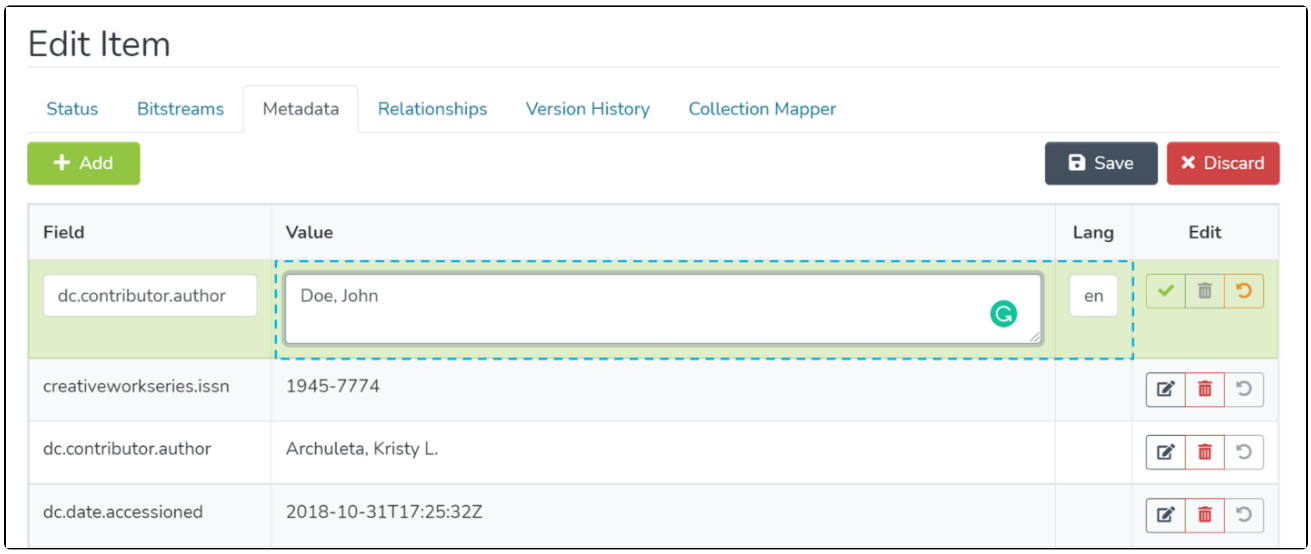
Step 1: Click on the “Add” button to add a metadata field.



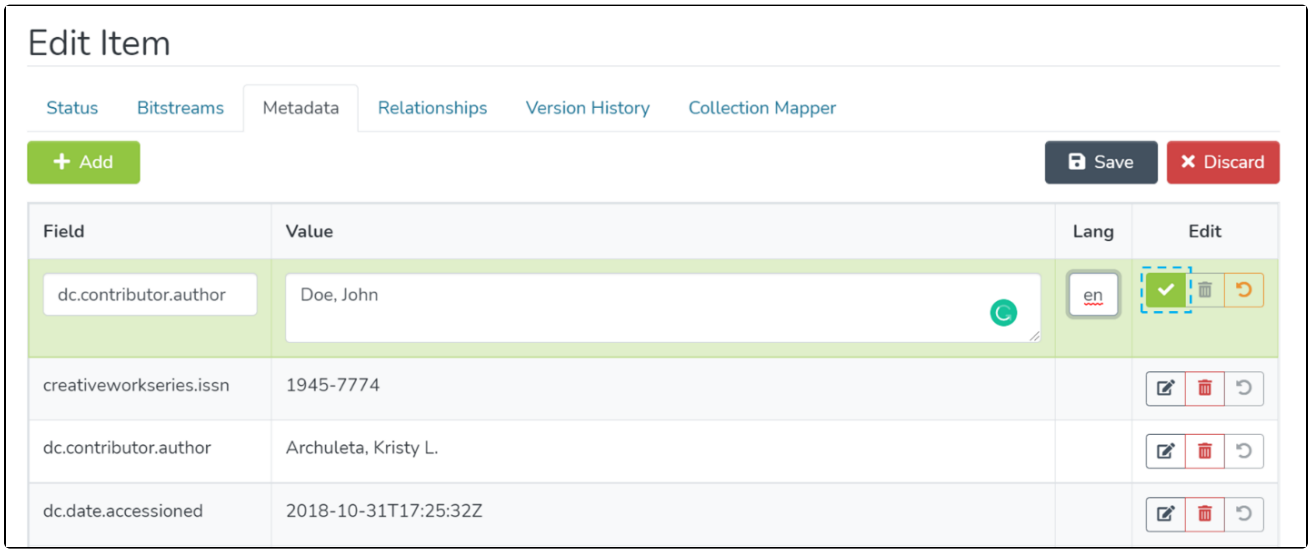
Step 2: Upon typing a few characters of the metadata element, users will notice a drop-down list showing metadata elements matching the entered value. Users can select the appropriate metadata element from the drop-down list.



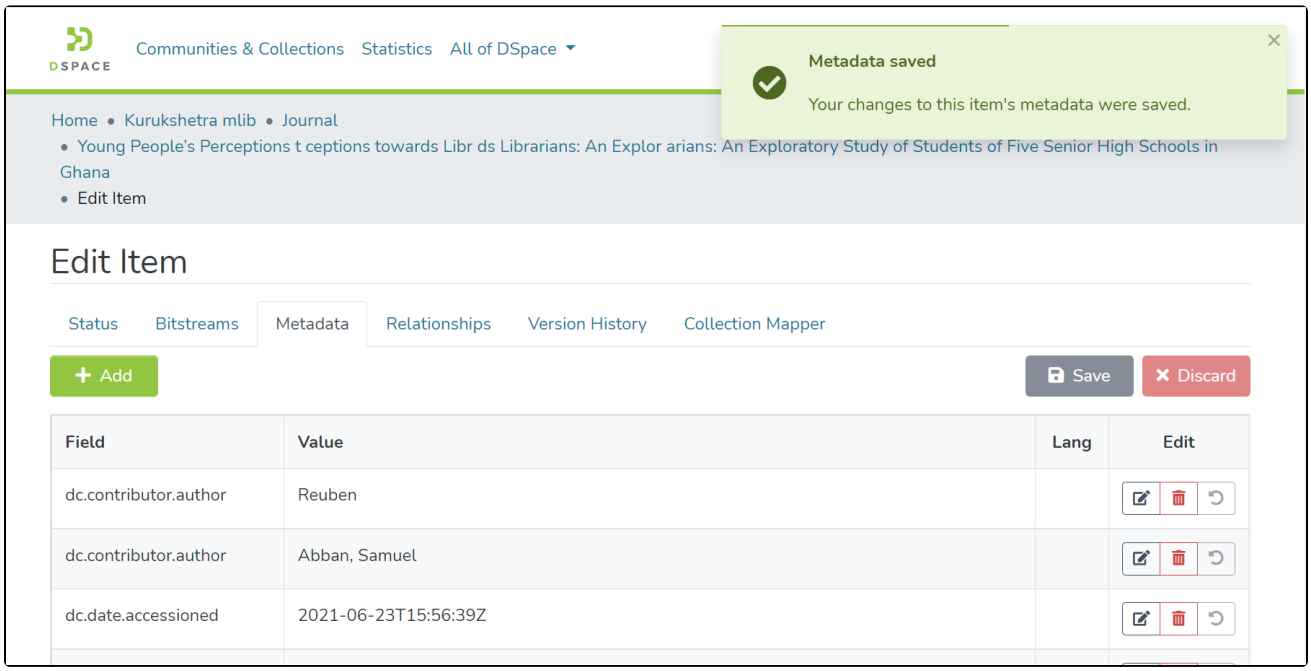
Step 3: After selecting the required metadata element, enter the metadata value in the “Value” field and the ISO code of the language under the Lang field. For example, enter "en" for English.



Step 4: Click on the “Complete” button as highlighted below to update the field.

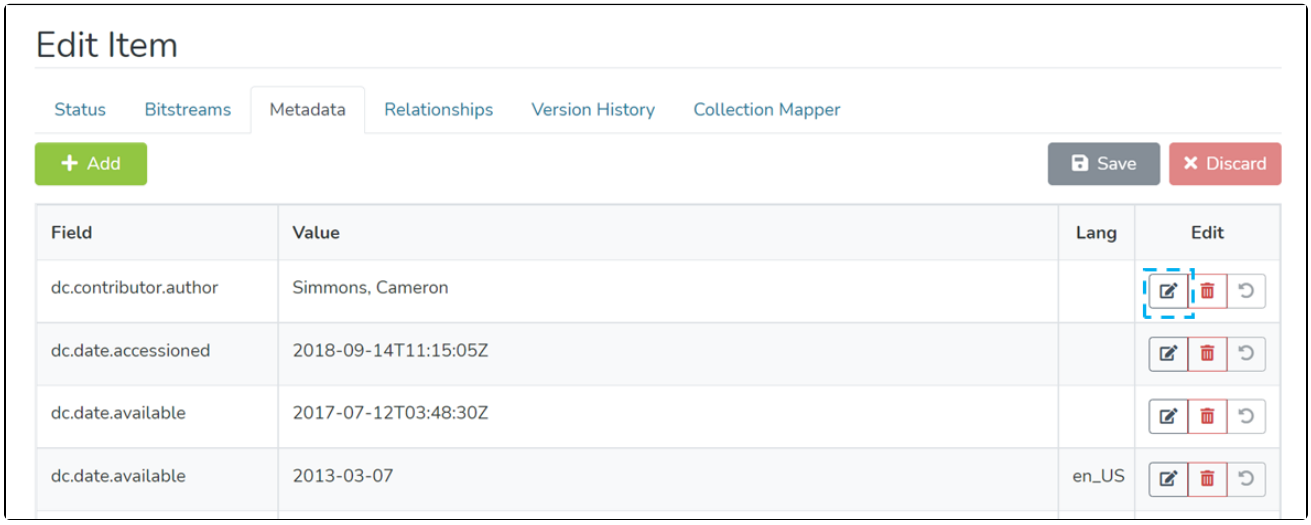


Step 5: Click on the “Save” button to continue saving changes or “Discard” to cancel changes made in the metadata fields. A success prompt confirming Metadata updates will appear as shown below.

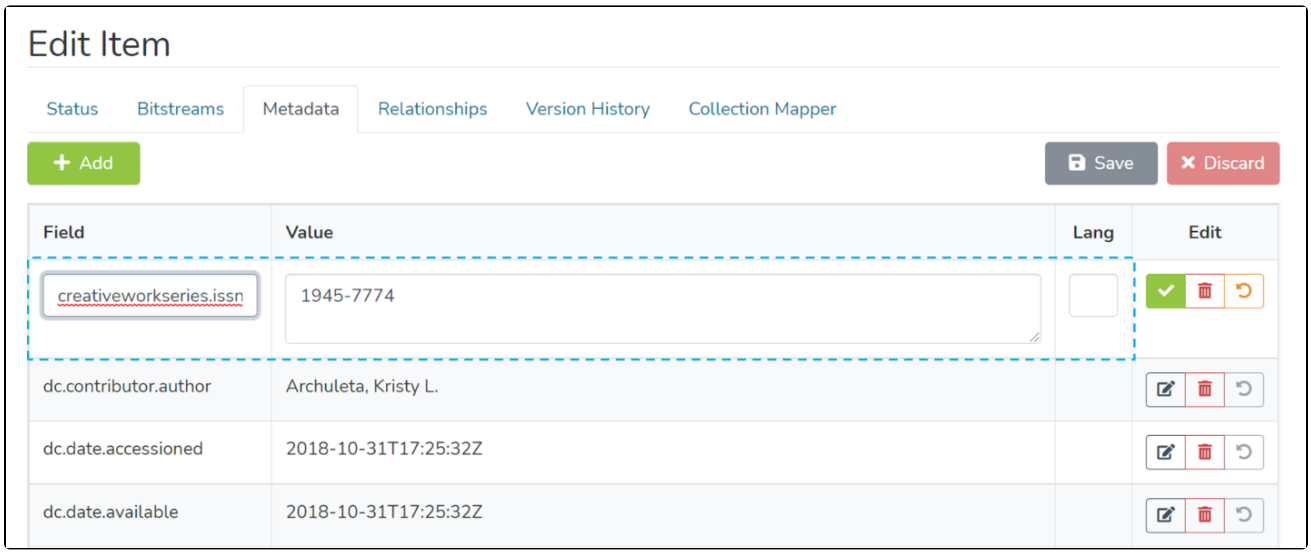


Edit an existing metadata field

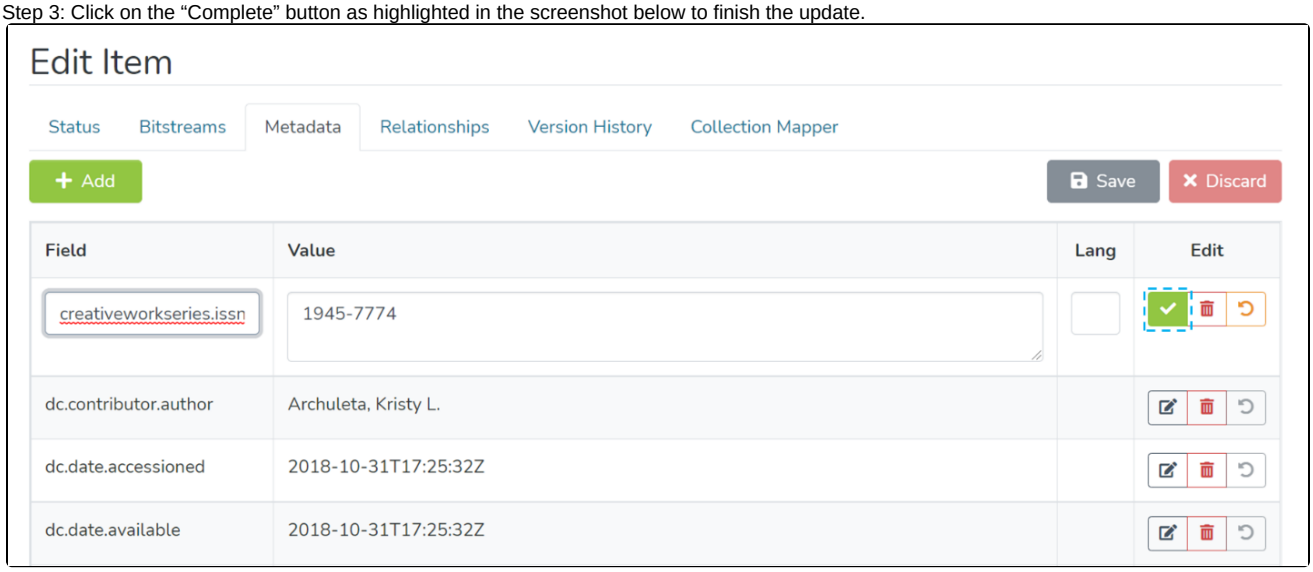
Step 1: Click on the "Edit" button to edit an existing metadata field.



Step 2: The metadata field becomes editable after clicking the "Edit" button to edit the metadata field element, value, and language.

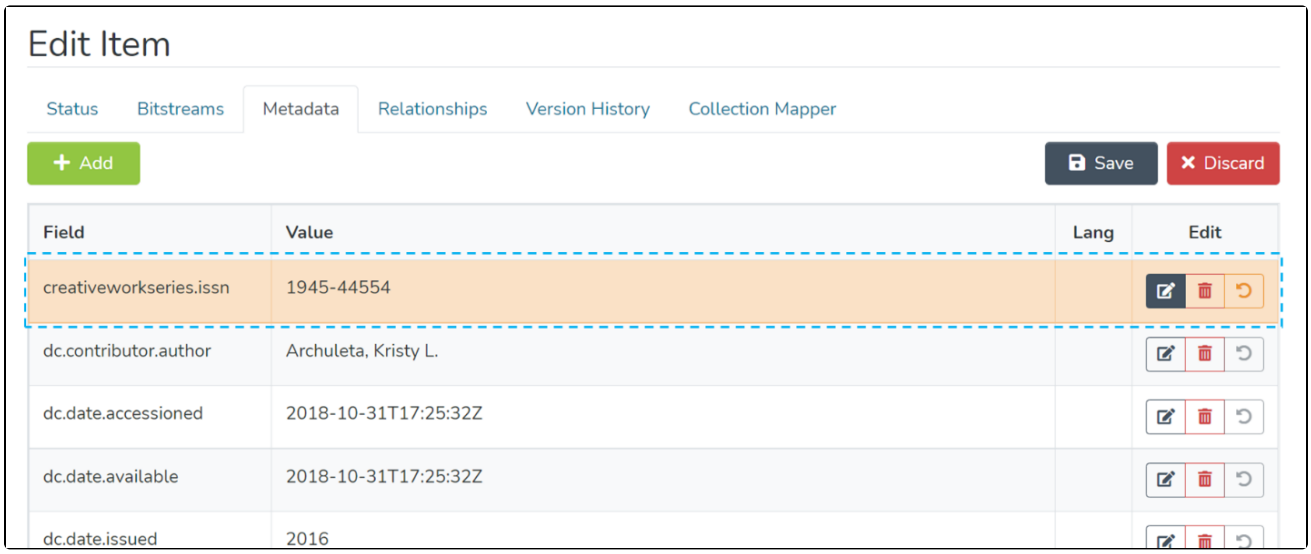


Step 3: Click on the “Complete” button as highlighted in the screenshot below to finish the update.

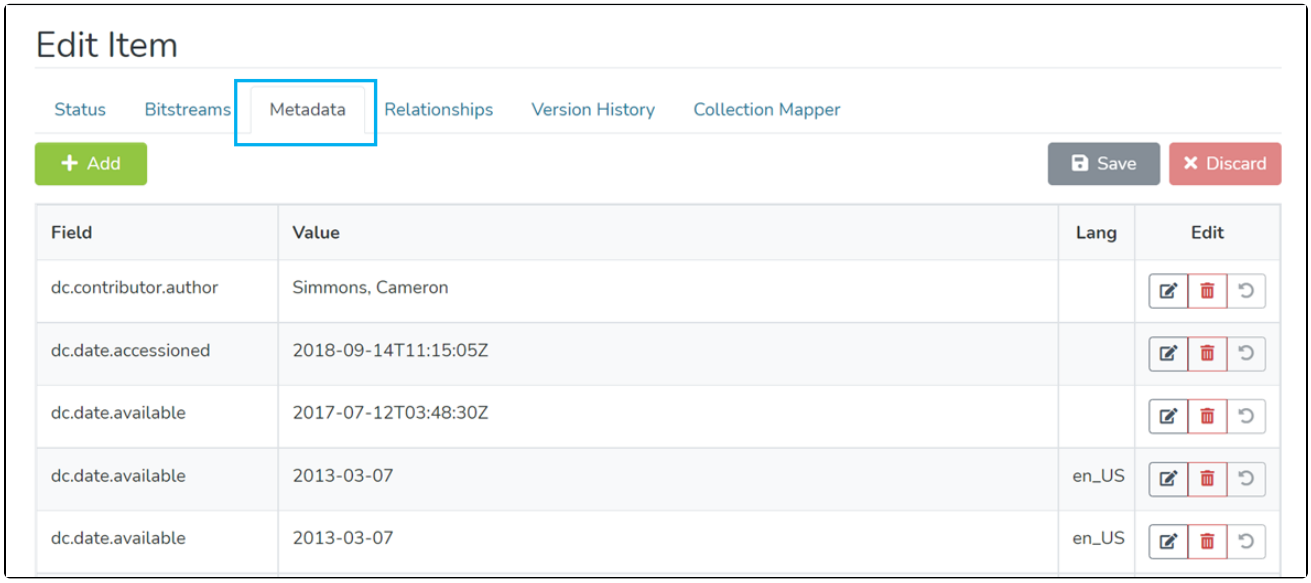


Note: image20 and image21 seem identical

The metadata field will highlight the successful addition of the metadata, as shown in the screenshot below. Click on the undo button if you want to undo the addition of the metadata field.

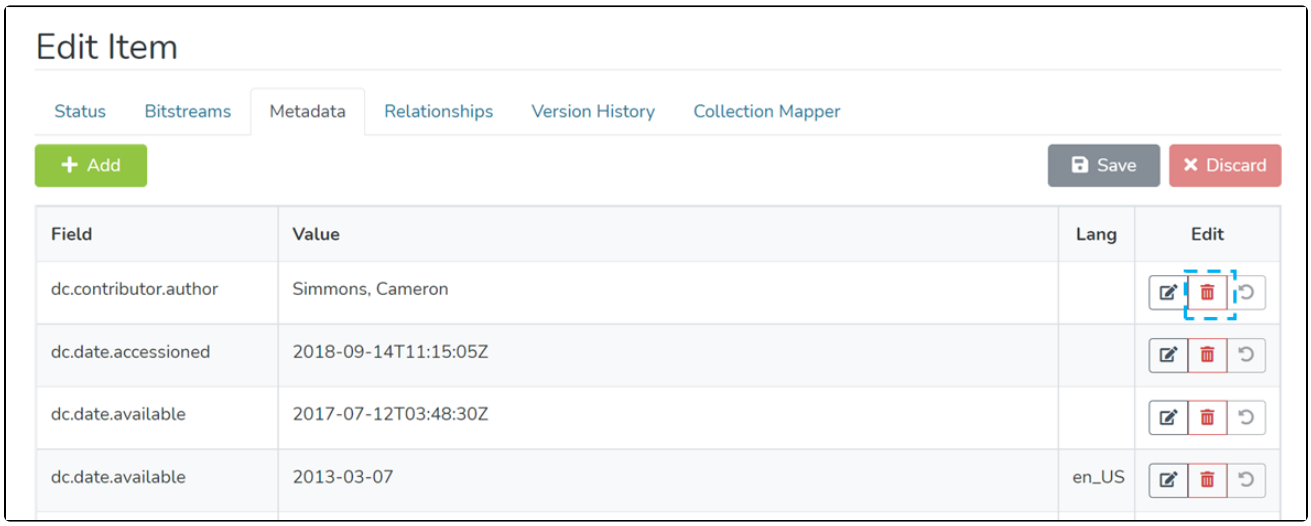


Step 4: Click on the “Save” button to continue saving changes or “Discard” to cancel changes made in the metadata fields. A success prompt confirming Metadata updates will appear as shown below.

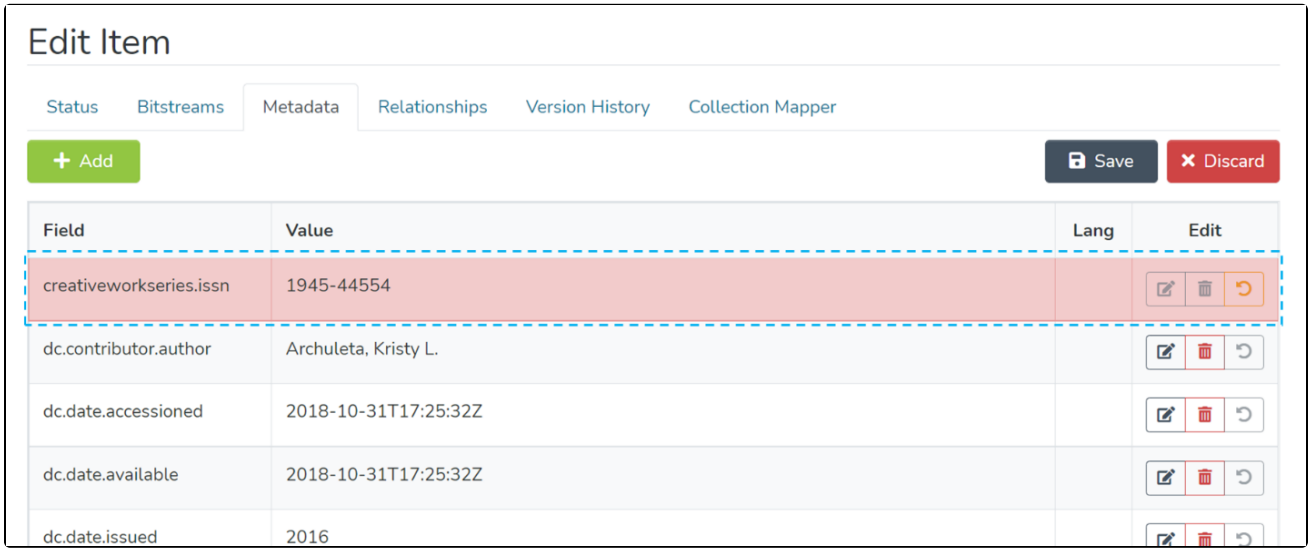


Delete an existing metadata field

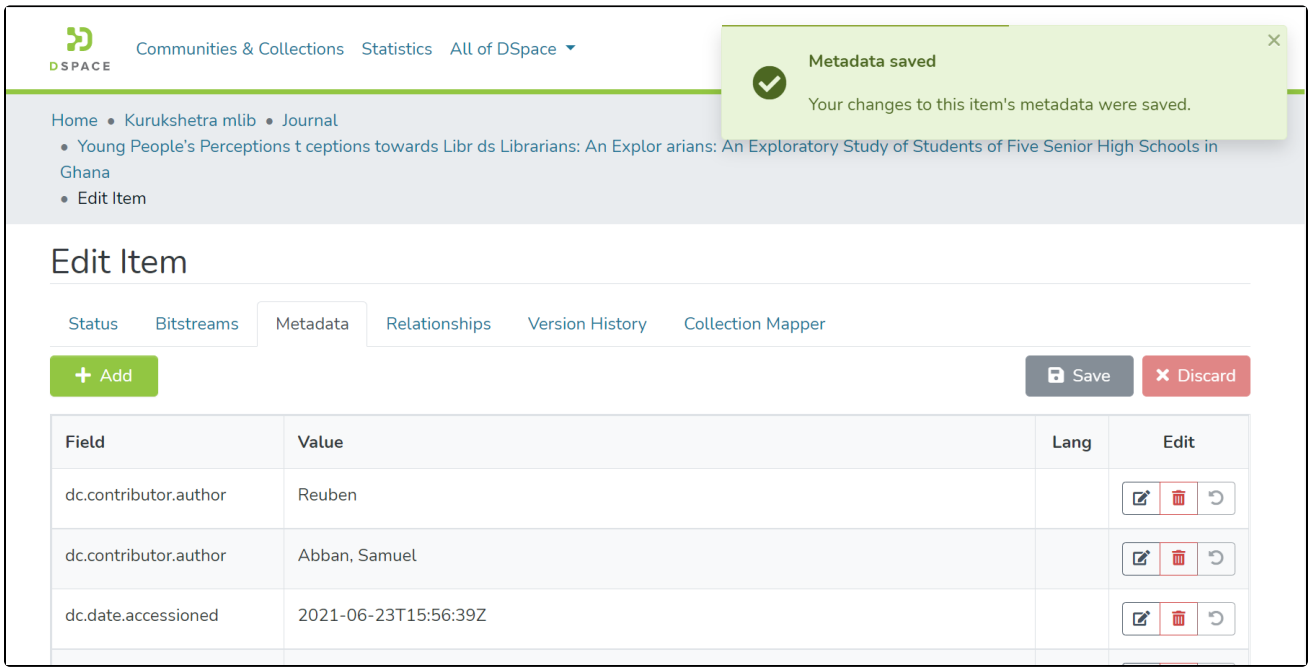
Step 1: Click on the “Delete” button to initiate the metadata field deletion.



Step 2: The deleted metadata field will be highlighted in red.



Step 3: Click on the “Save” button to continue saving changes or “Discard” to cancel changes made in the metadata fields. A success prompt confirming Metadata updates will appear as shown below.


DSPACE
Communities & Collections
Statistics
All of DSpace
Metadata saved
Your changes to this item's metadata were saved.
Home • Kurukshetra mlib • Journal
• Young People's Perceptions t ceptions towards Libr ds Librarians: An Explor arians: An Exploratory Study of Students of Five Senior High Schools in Ghana
• Edit Item
Edit Item
Status
Bitstreams
Metadata
Relationships
Version History
Collection Mapper
+ Add
Save
Discard
Field
Value
Lang
Edit
dc.contributor.author
Reuben
dc.contributor.author
Abban, Samuel
dc.date.accessioned
2021-06-23T15:56:39Z

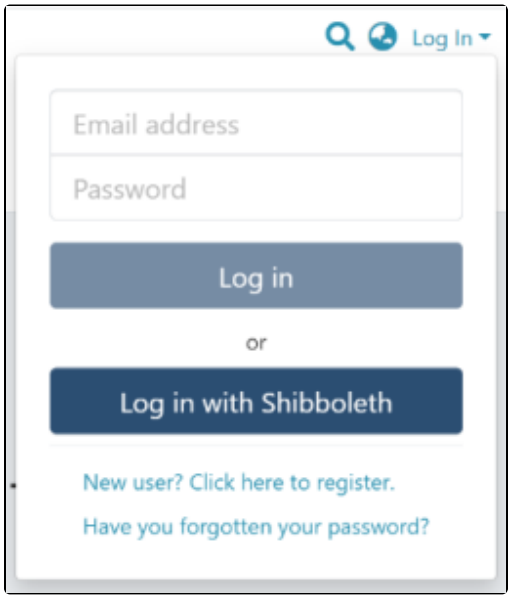
Edit Relationship

- Relationship Management
- Add Relationships with other items
- Delete a Relationship

Relationship Management

Entity-relationship is a new concept introduced in the DSpace version 7, helping authorized users logically link two or more items by defining relationships among them. A few good examples are the Relationship between an article & Author, a Journal and Journal Article, Organization Unit and Individuals in the unit, etc.

Step 1: Login using your credentials

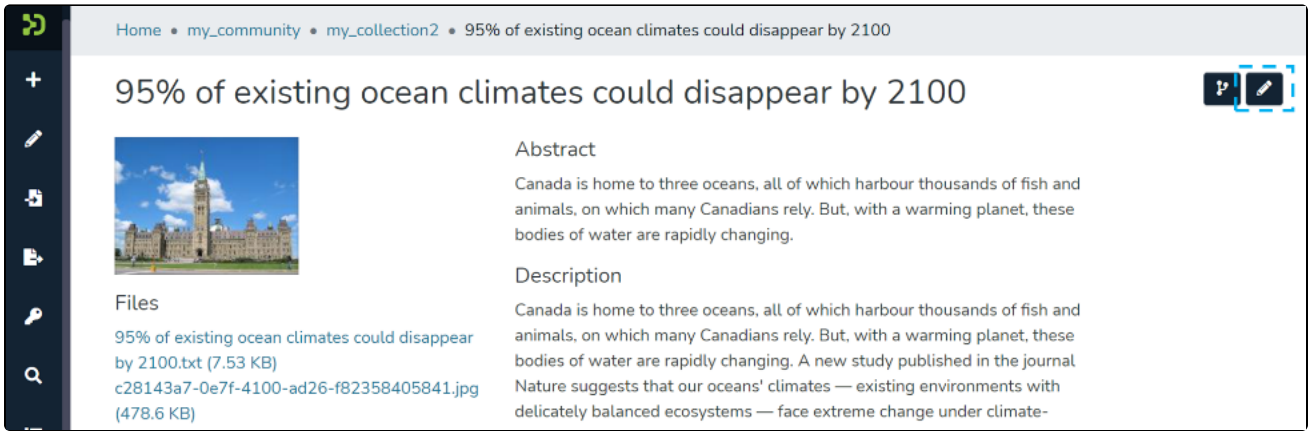


Step 2: Go to the Item you want to edit

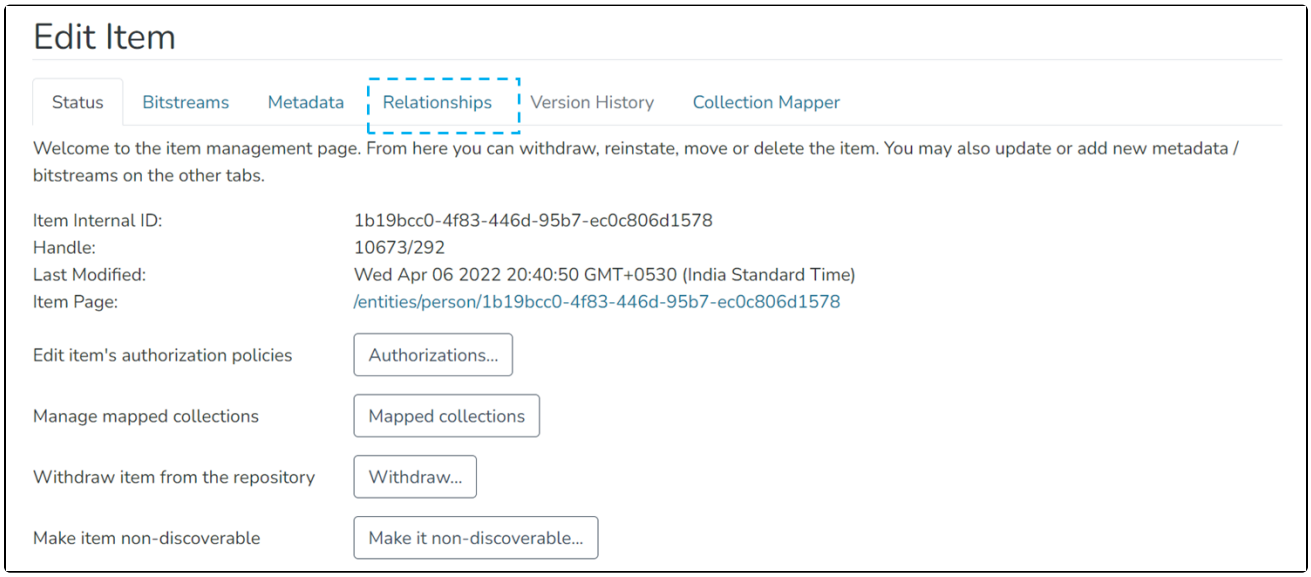
Users can reach an item through multiple methods, which are listed below

1. Finding an item using search functionalities of the DSpace
2. Reaching an item by browsing communities and collections
3. Finding an item in the Administration section at Edit > Item

Click on the “Edit” button appearing towards the right-hand side of the item title.

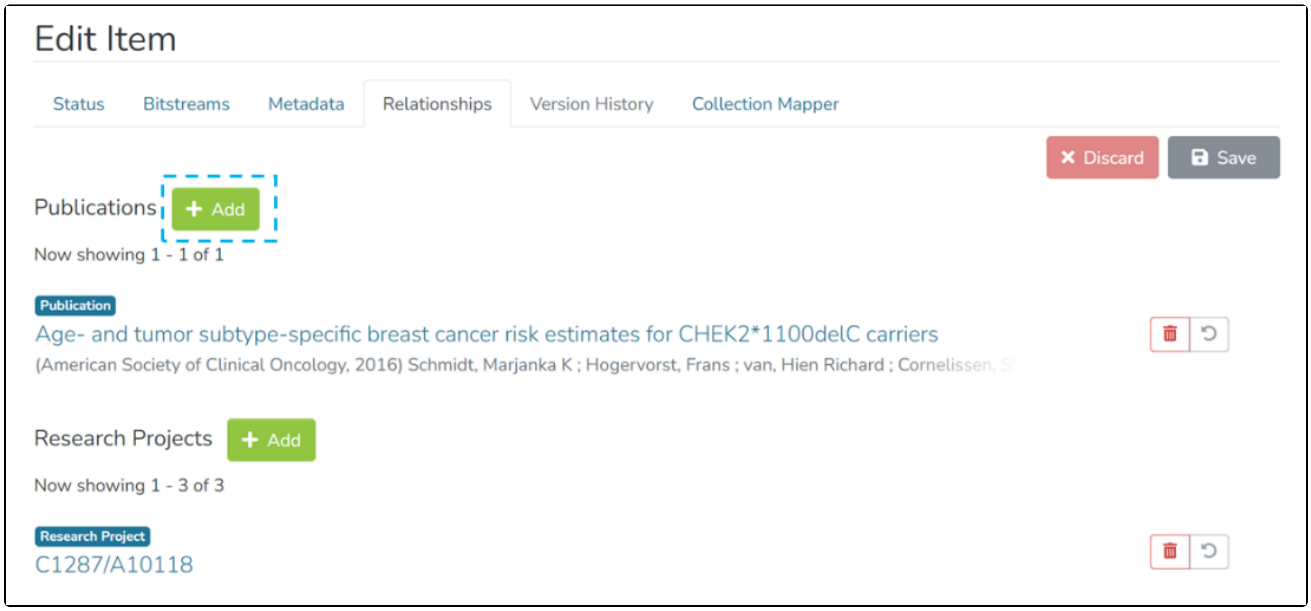


Step 3: Click on the “Relationships” tab.

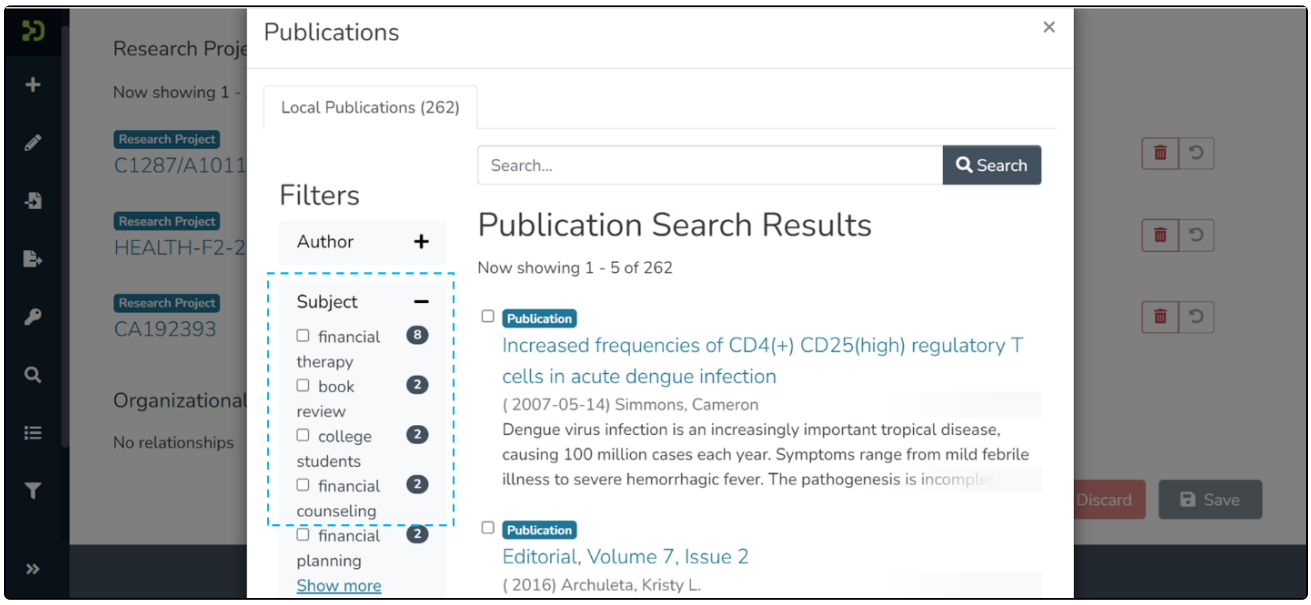


Add Relationships with other items

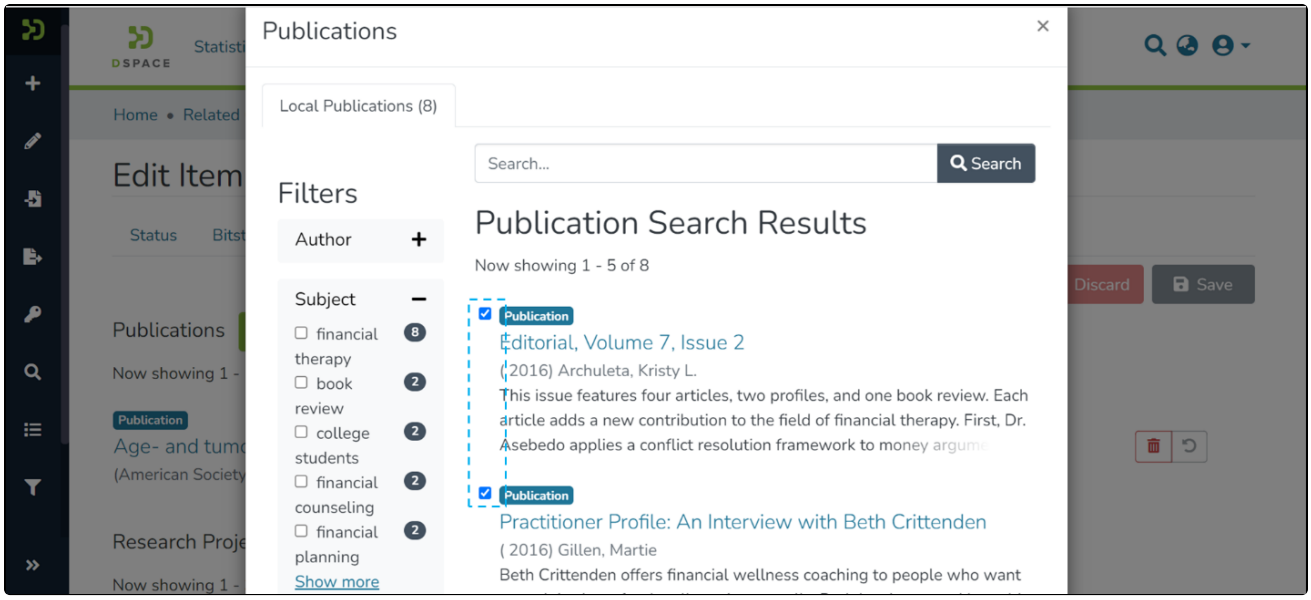
Step 1: Click on the “Add” button next to the relationship type to map Item (s) under each relationship type. For example, click on the “Add” button next to Publications to add an item as a related publication.



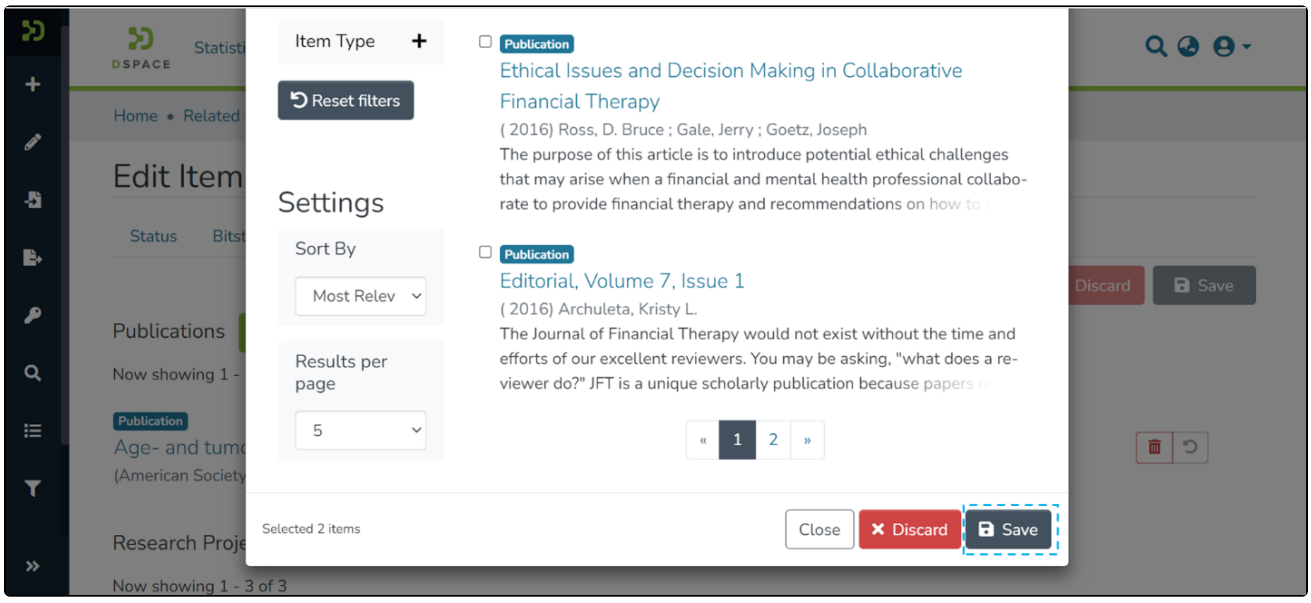
Step 2: Identify target items for addition using Filters and Search functions. Please note that process is identical to the advanced search process.



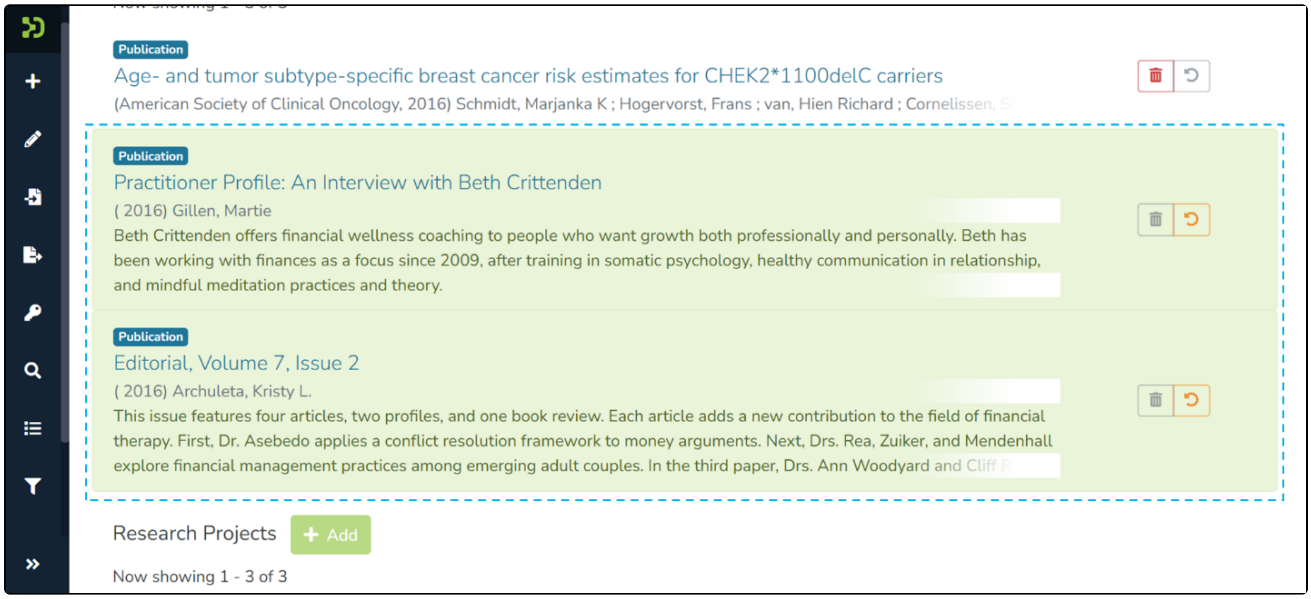
Step 3: Click checkboxes appearing on the left-hand side of each Item to select them for the Relationship.



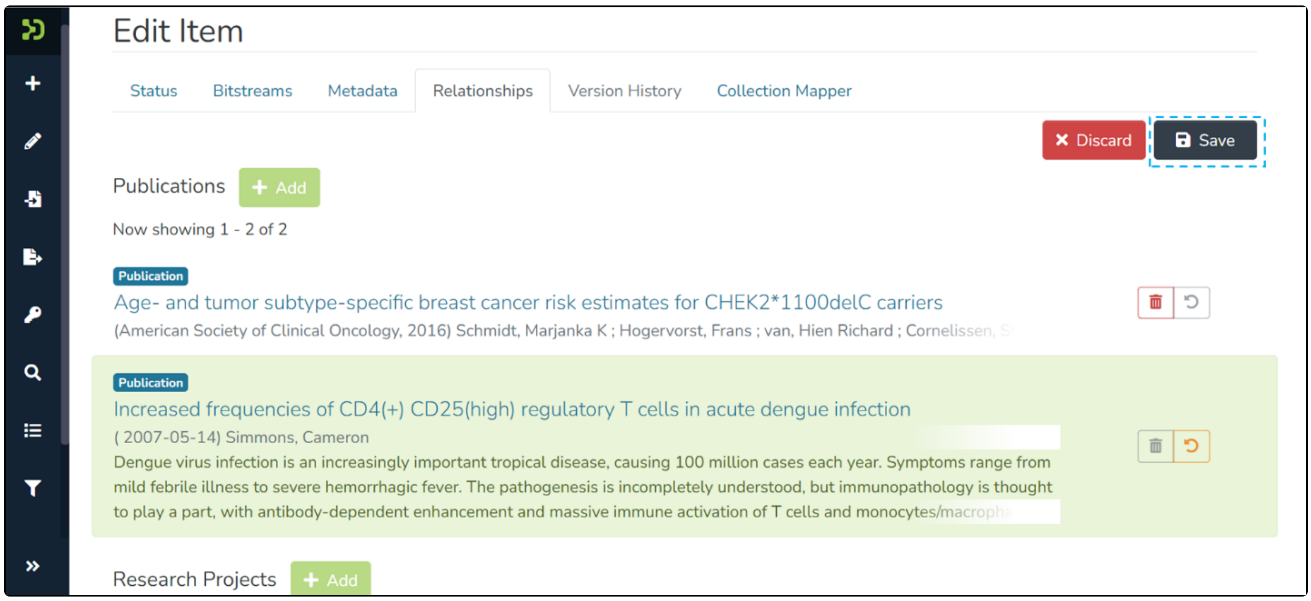
Step 4: Scroll down to the bottom of the screen and click on the “Save” button to complete the process.



The application will redirect users to the source item's relationship tab, highlighting newly added items for easy identification.



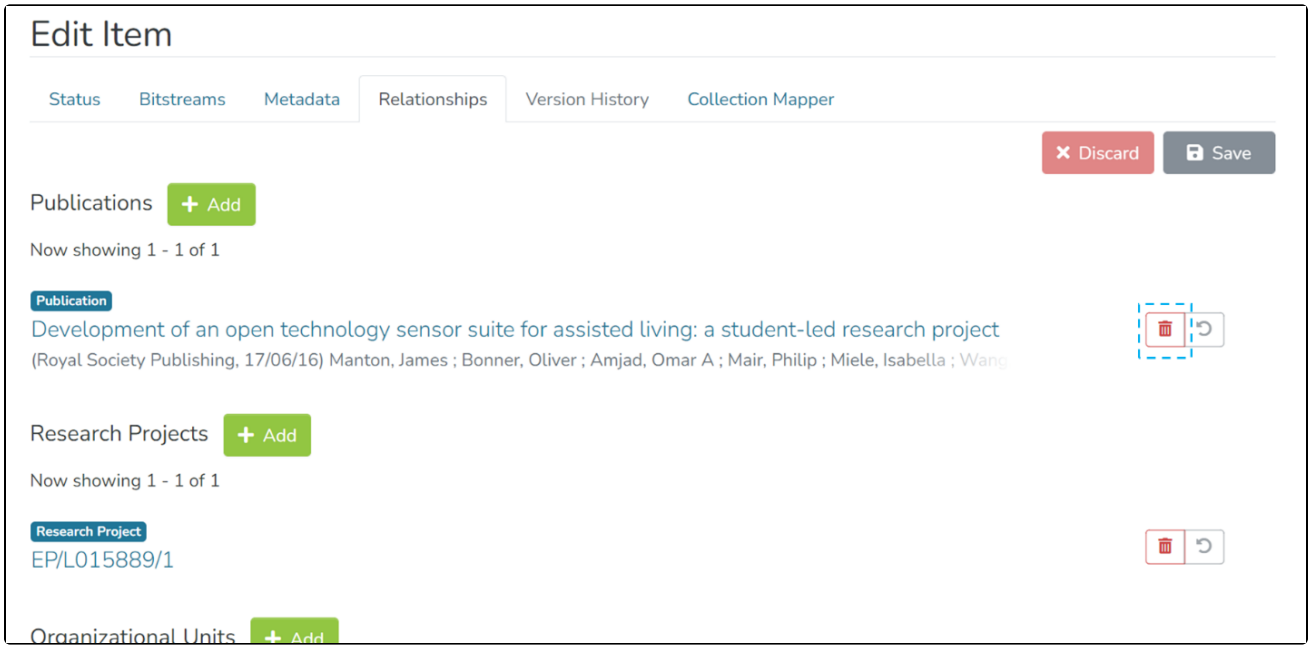
Step 5: Click on the “Save” button to confirm the item addition under the selected relationship type or click “Discard” to undo the addition.



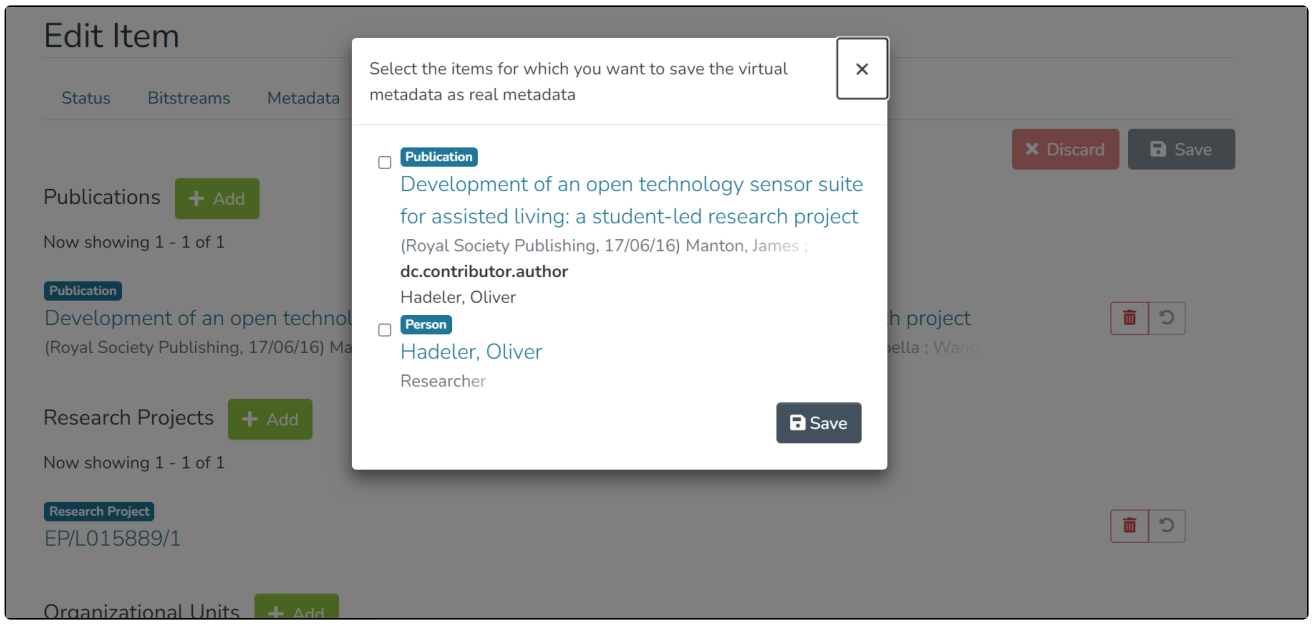
Users will see a prompt confirming the successful mapping of items under the selected relationship type.

Delete a Relationship

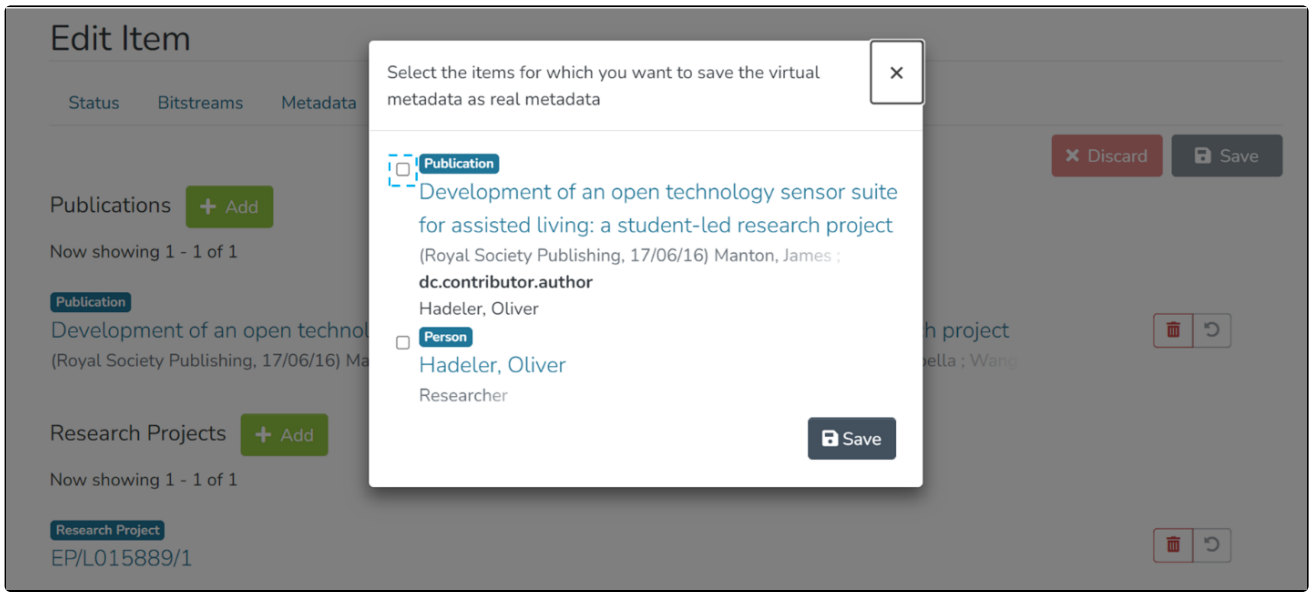
Step 1: Click on the “delete” button appearing a bitstream and drag it above or below another bitstream(s) in the bundle to change the bitstream’s sequence.



Step 2: A prompt showing items concerning the selected Item will appear in the prompt. Click the checkbox appearing next to items. Please refer screenshots below for a better understanding.

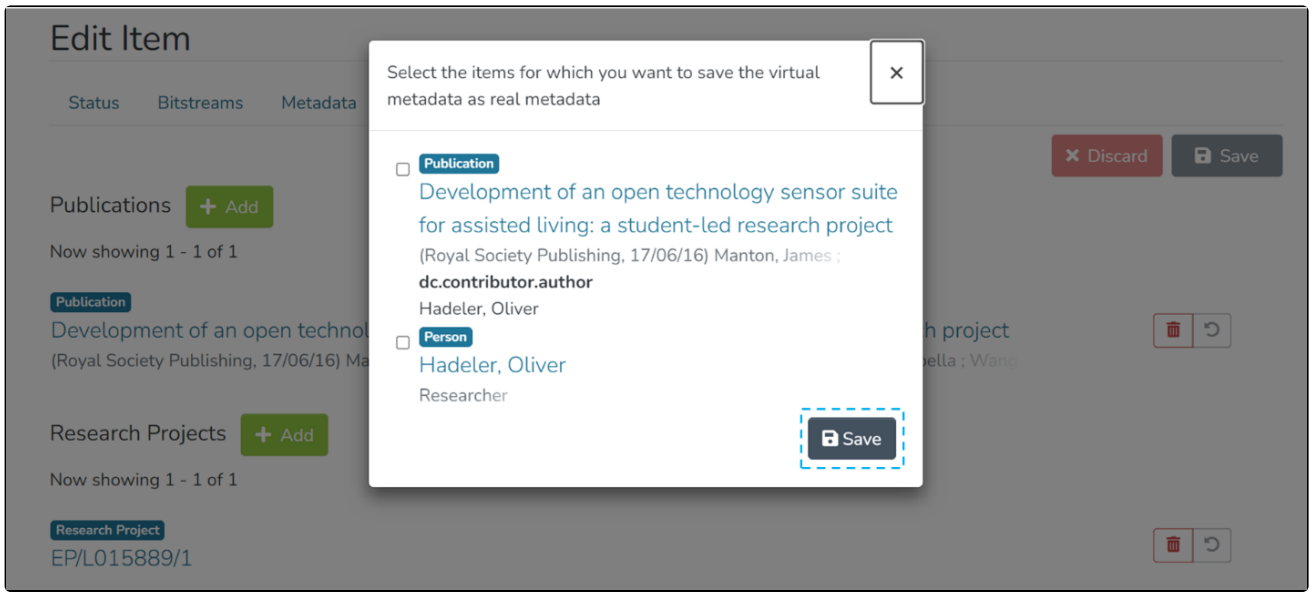


List of items concerning the selected Item.



Click on the checkbox highlighted before the Item

Step 3: Click the “Save” button to complete the selection process and go back to the Relationship tab of the source item.



Step 4: Click on the “Save” button to confirm the deletion under the selected relationship type or click “Discard” to undo the action. Users will see a prompt confirming the successful deletion of items under the selected relationship type.

Make an Item Private

- Audience
- Make Item private

Audience

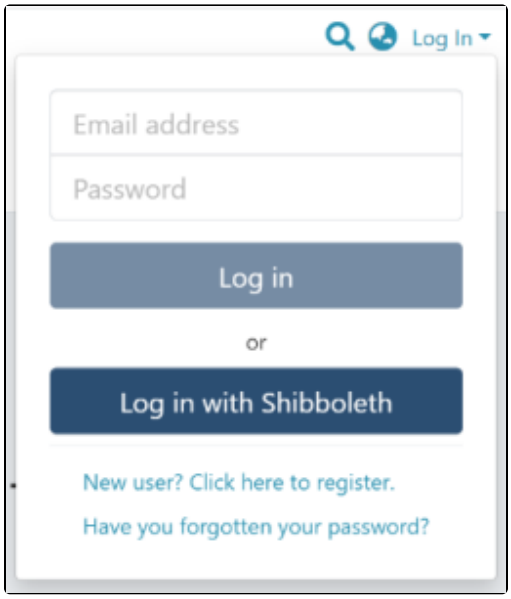
Content Submitters

Community Administrators

System Administrators

Make Item private

Step 1: Login using your DSpace credentials

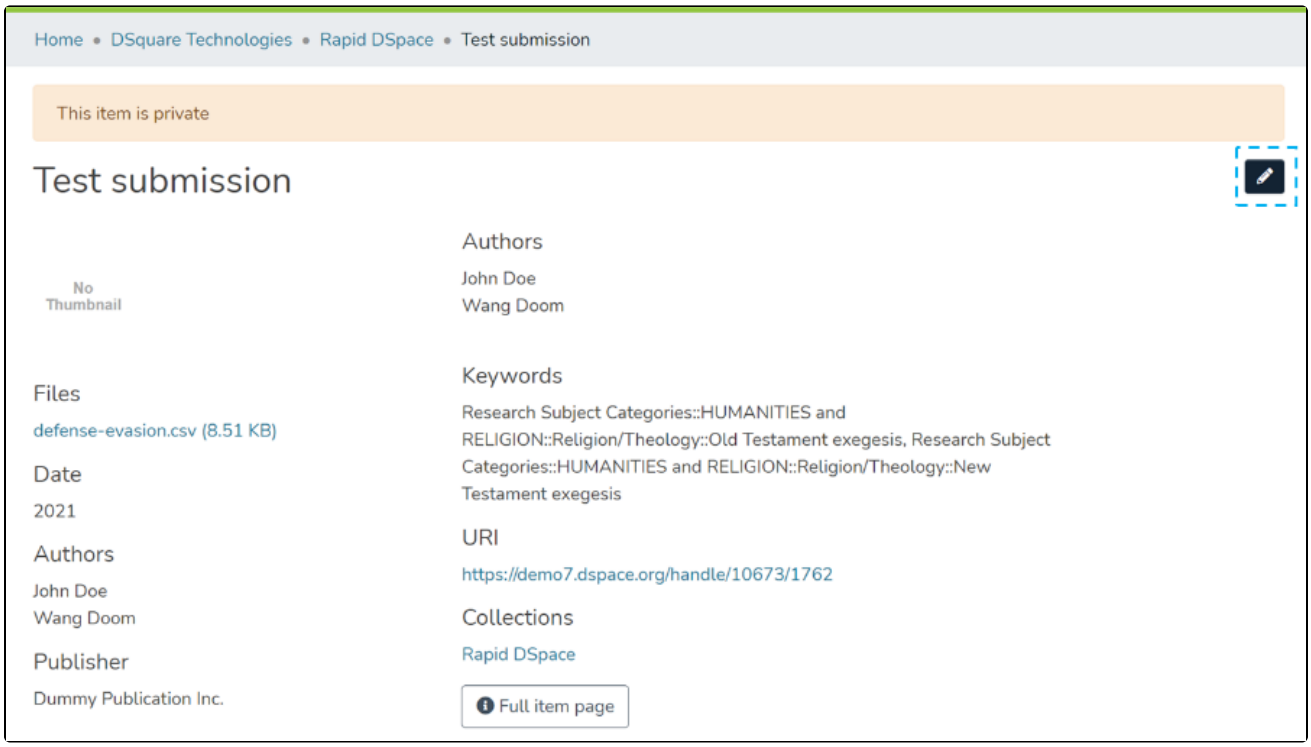


Step 2: Go to the item you want to edit

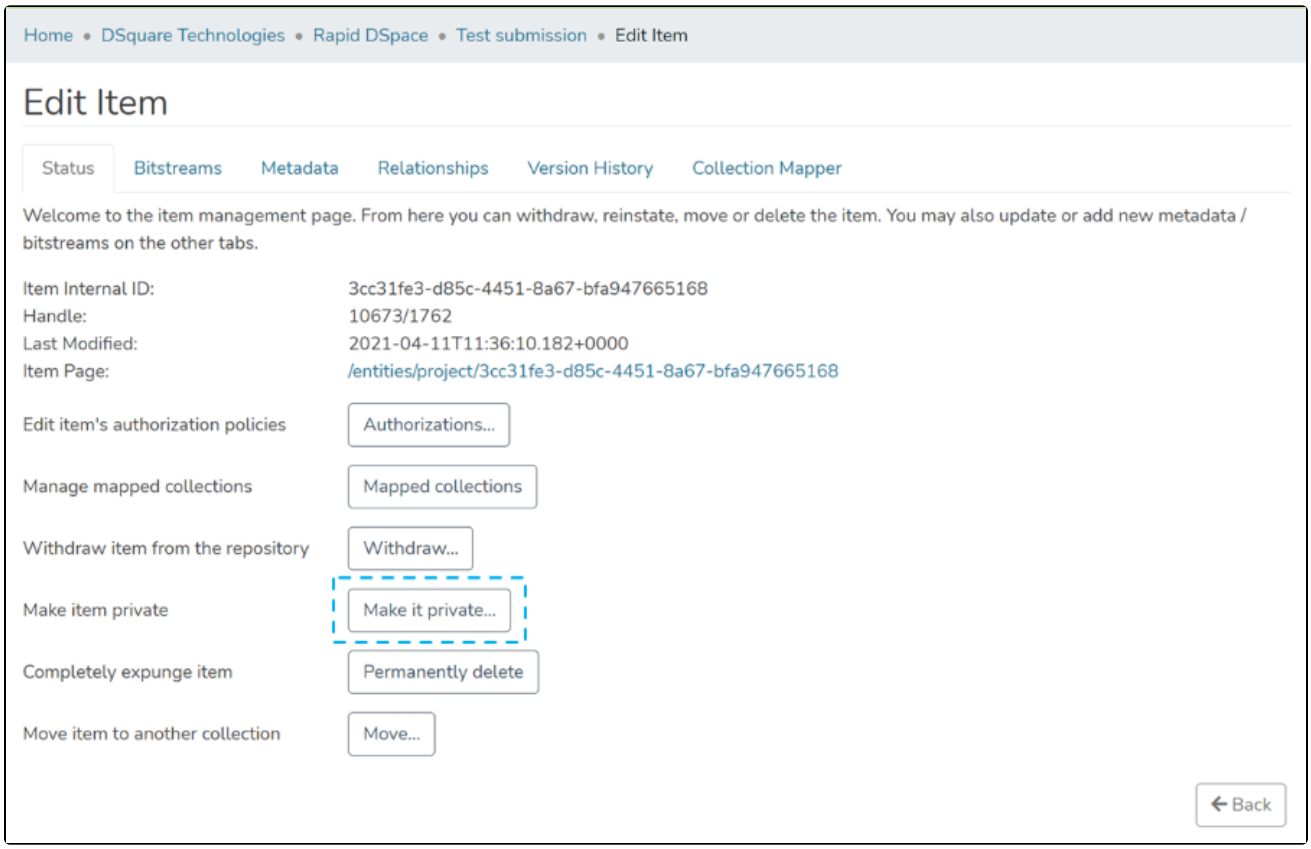
Users can reach an item through multiple methods, as listed below:

1. Search an item
2. Browse communities and collections
3. Finding an item in the Administration section at Edit > Item

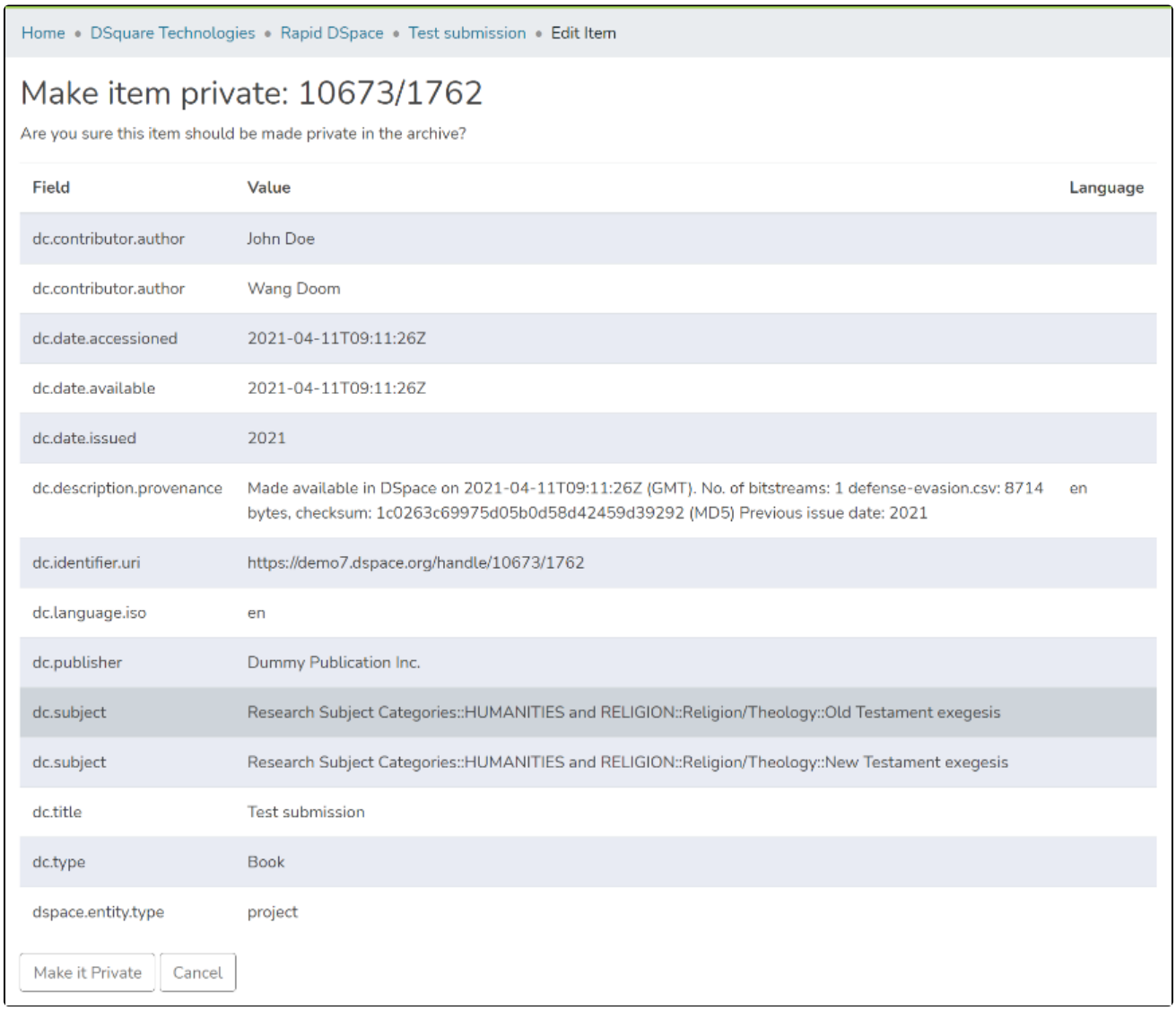
Click on the “Edit” button appearing towards the right-hand side of the item title.



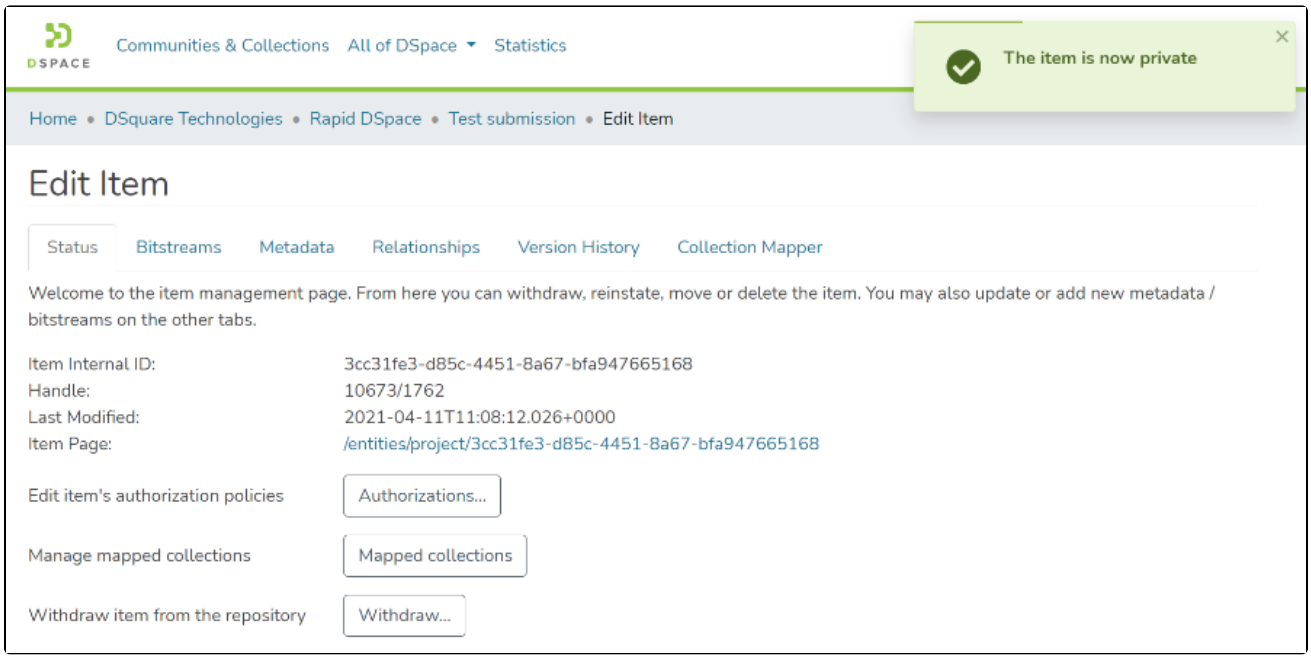
Step 3: Click on the “Make it private” button under the “Status” tab to make the selected Item private.



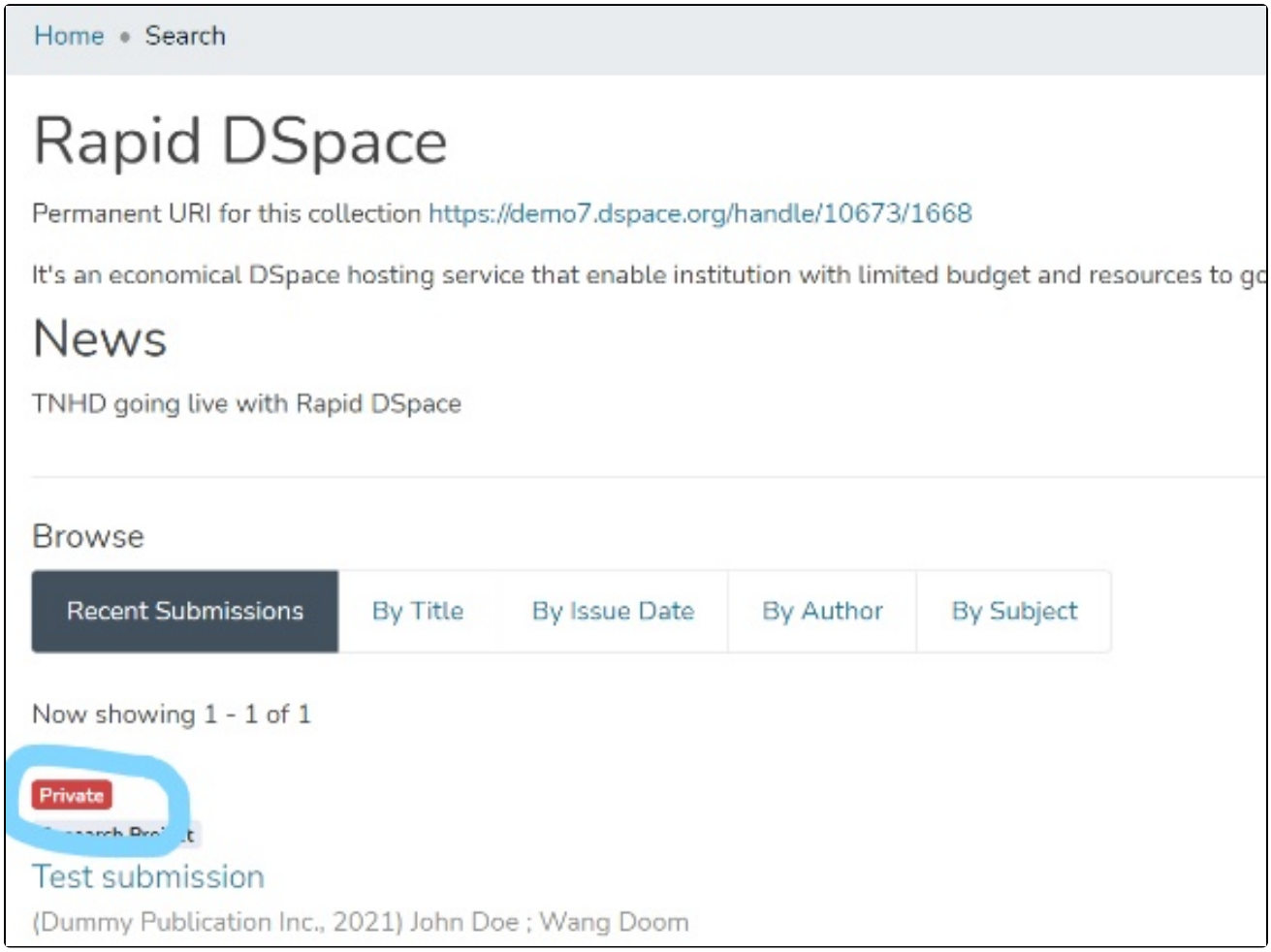
Step 4: Click on the “Make it Private” button to make the selected Item private or click the “Cancel” button to cancel the operation.



Step 5: You will see a success prompt confirming that the Item is private, as shown below.



Step 6: You will notice that the Item will appear with a “Private” tag.



Move an Item

- Audience
- Move an item

Audience

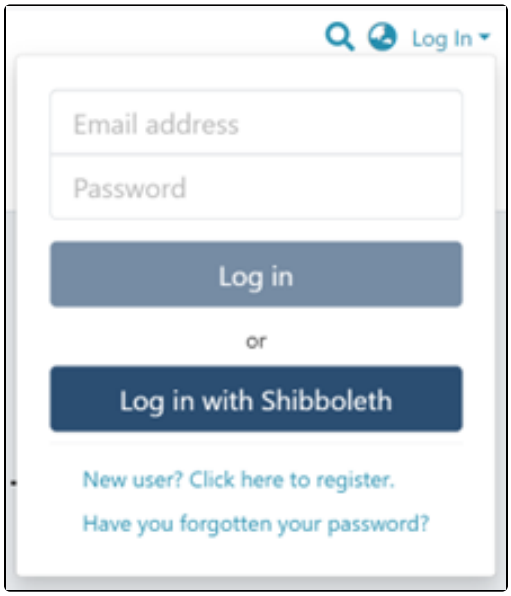
Content Submitters

Community Administrators

System Administrators

Move an item

Step 1: Login using your credentials

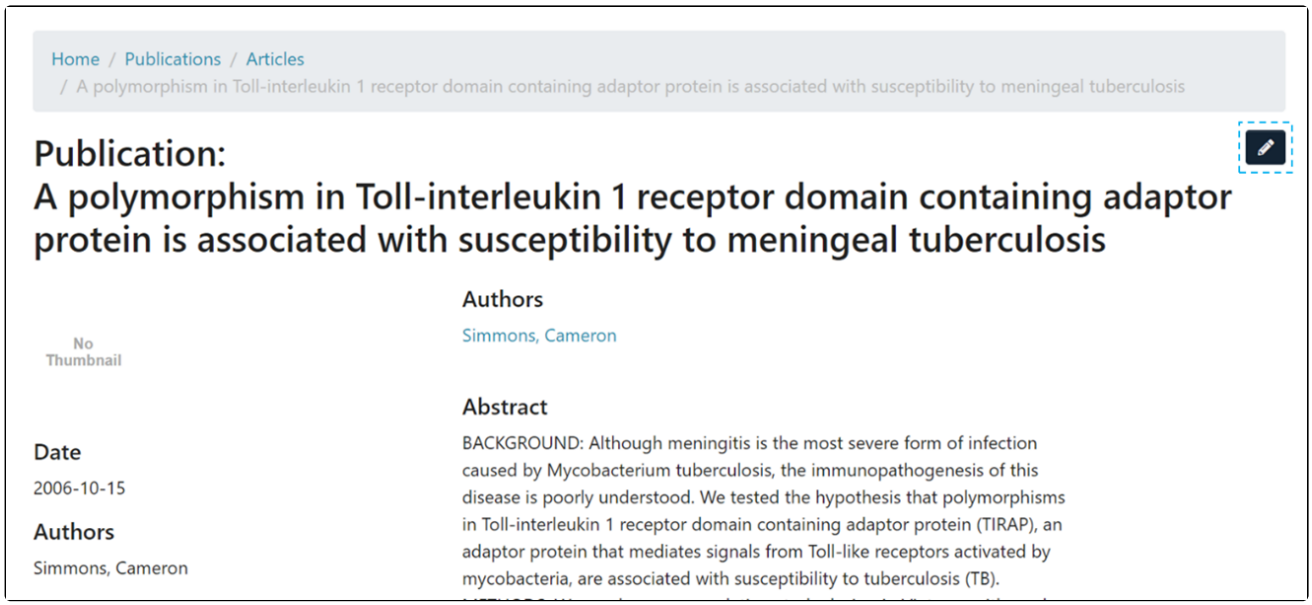


Step 2: Go to the item you want to edit

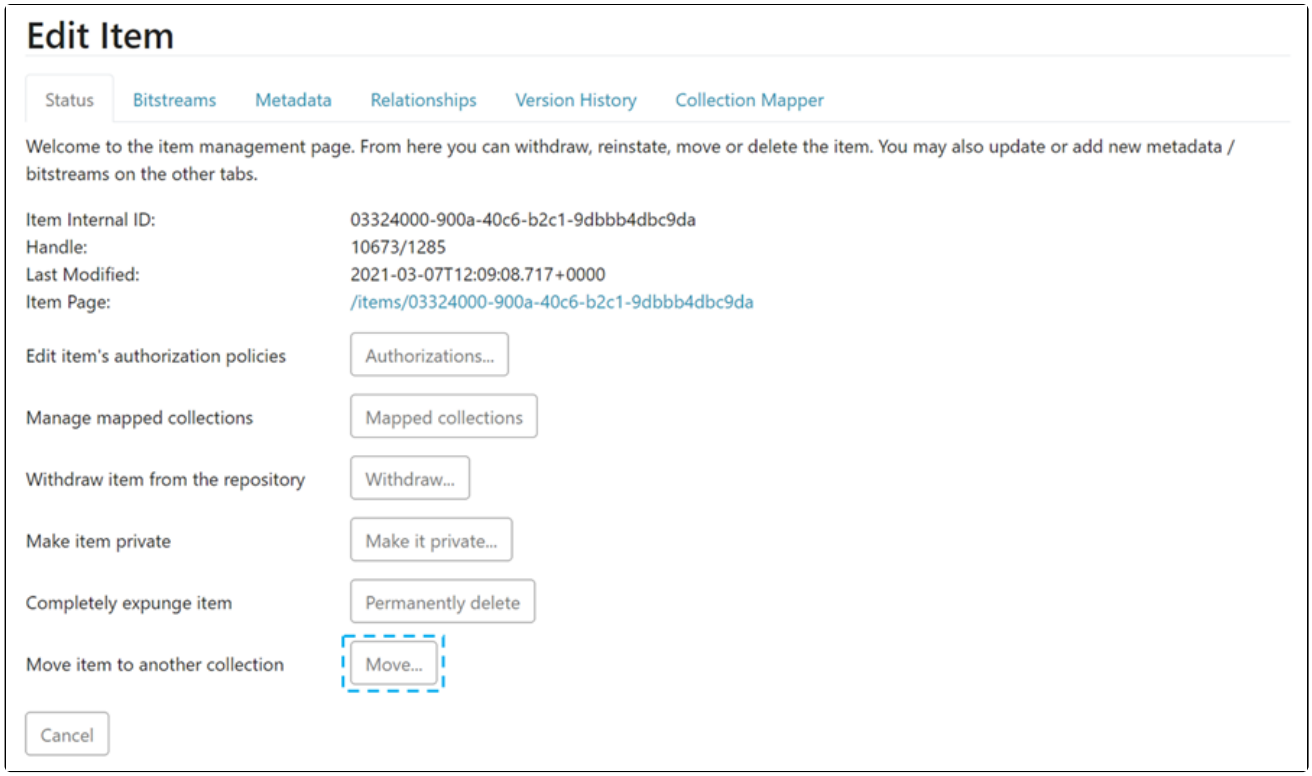
Users can reach an item through multiple methods, as listed below:

1. Search an item
2. Browse communities and collections
3. Finding an item in the Administration section at Edit > Item

Click on the “Edit” button appearing on the right-hand side of the item title.



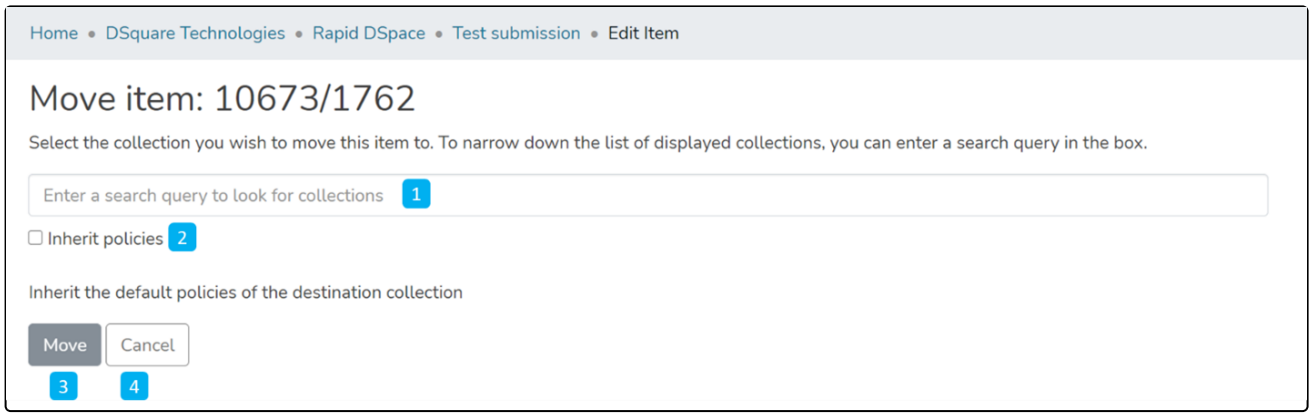
Step 3: Click on the “Status” tab and click the “Move” button.



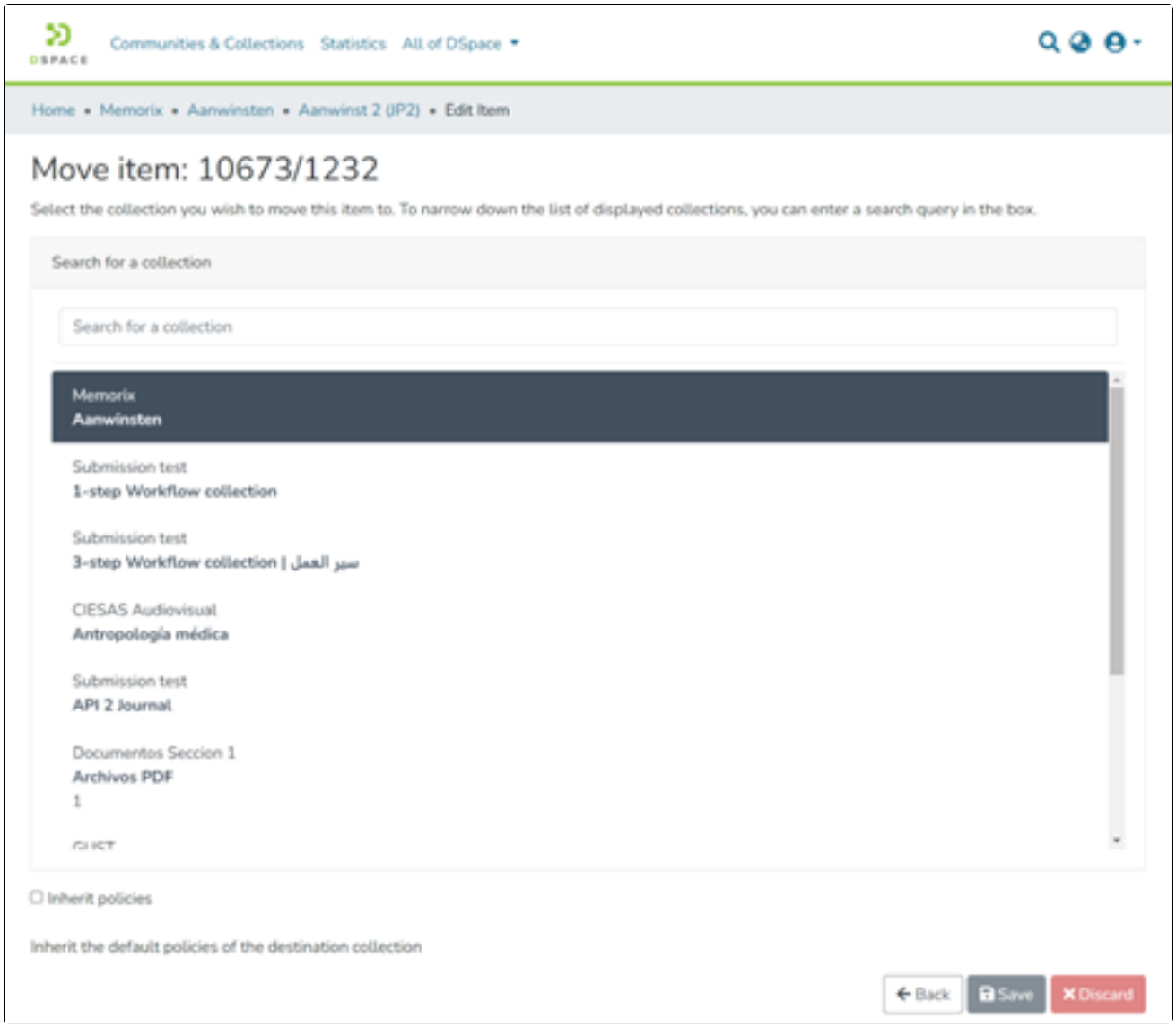
Step 4: Understanding the Move item page

The field for entering the collection name: Enter the target collection name to move the item or select the collection from the drop-down list, as demonstrated in the following step.

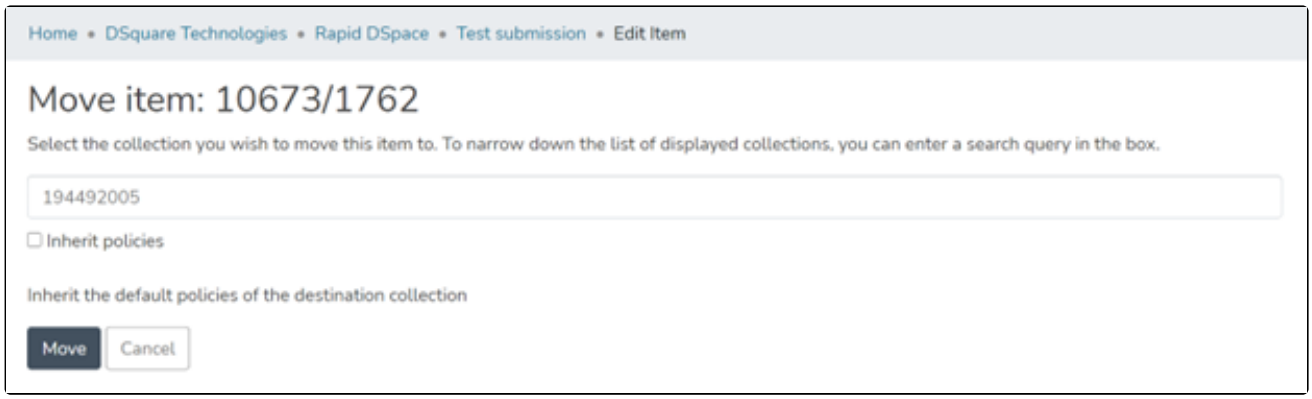
1. **Inherit policies:** Click on this check box to update the item’s policies according to the collection’s policies.
2. **Move:** Click the “Move” button to complete the operation.
3. **Cancel:** Click the “Cancel” button to cancel the operation.



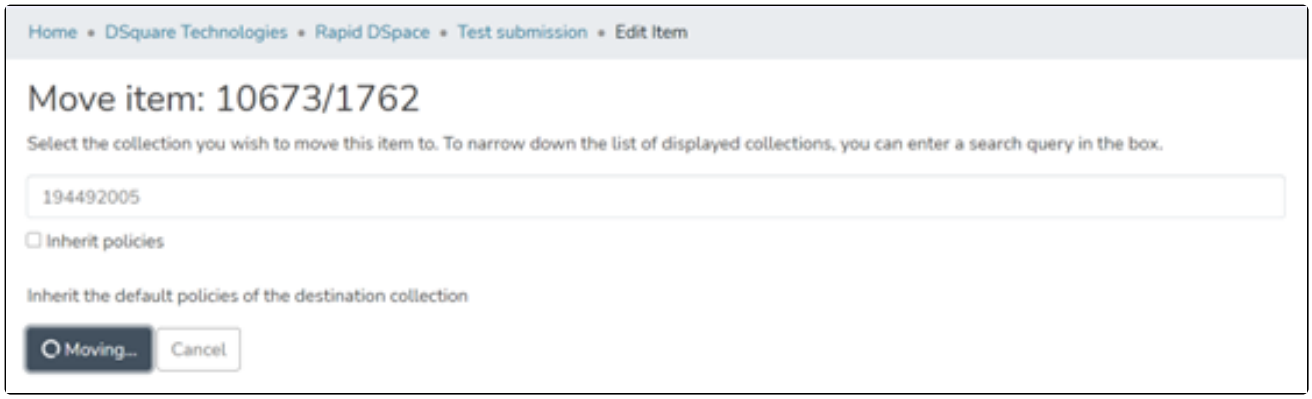
Step 5: Click on the Collection name and type the target collection name to move the item or scroll the collection list to identify the appropriate collection.



Step 6: Click on the “Move” button after selecting the target collection.

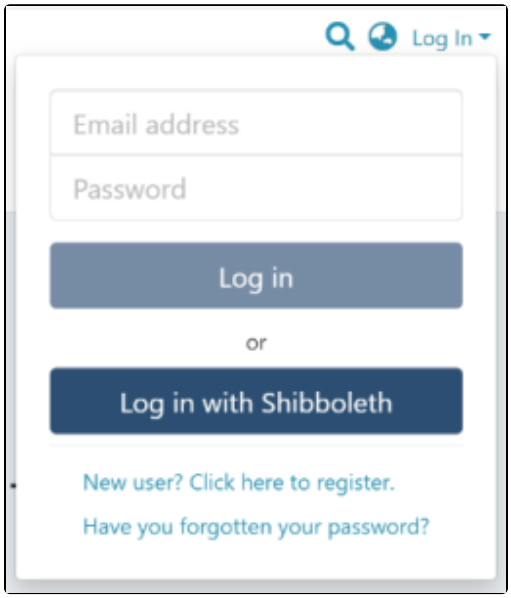


Step 7: The item will move to the target collection upon completing the operation.



Reinstate an item

Step 1: Login using the DSpace credentials

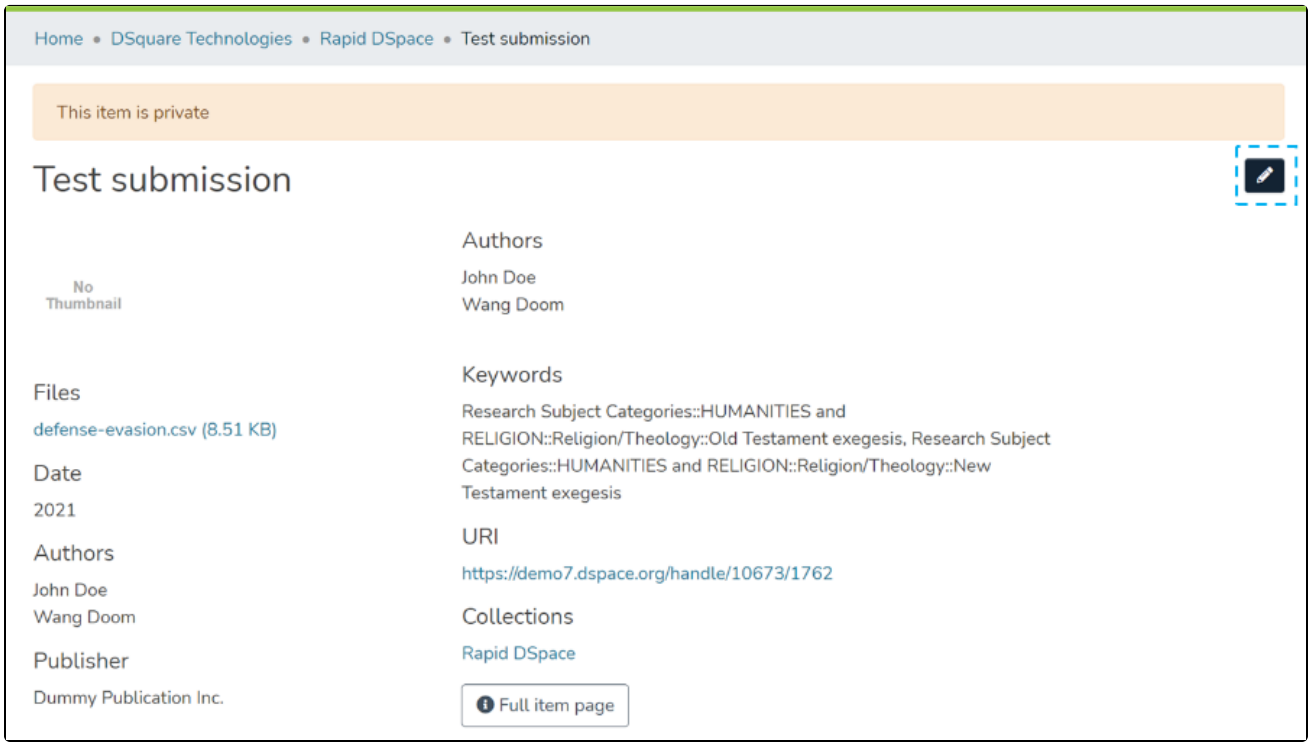


Step 2: Users can go to the item they want to edit

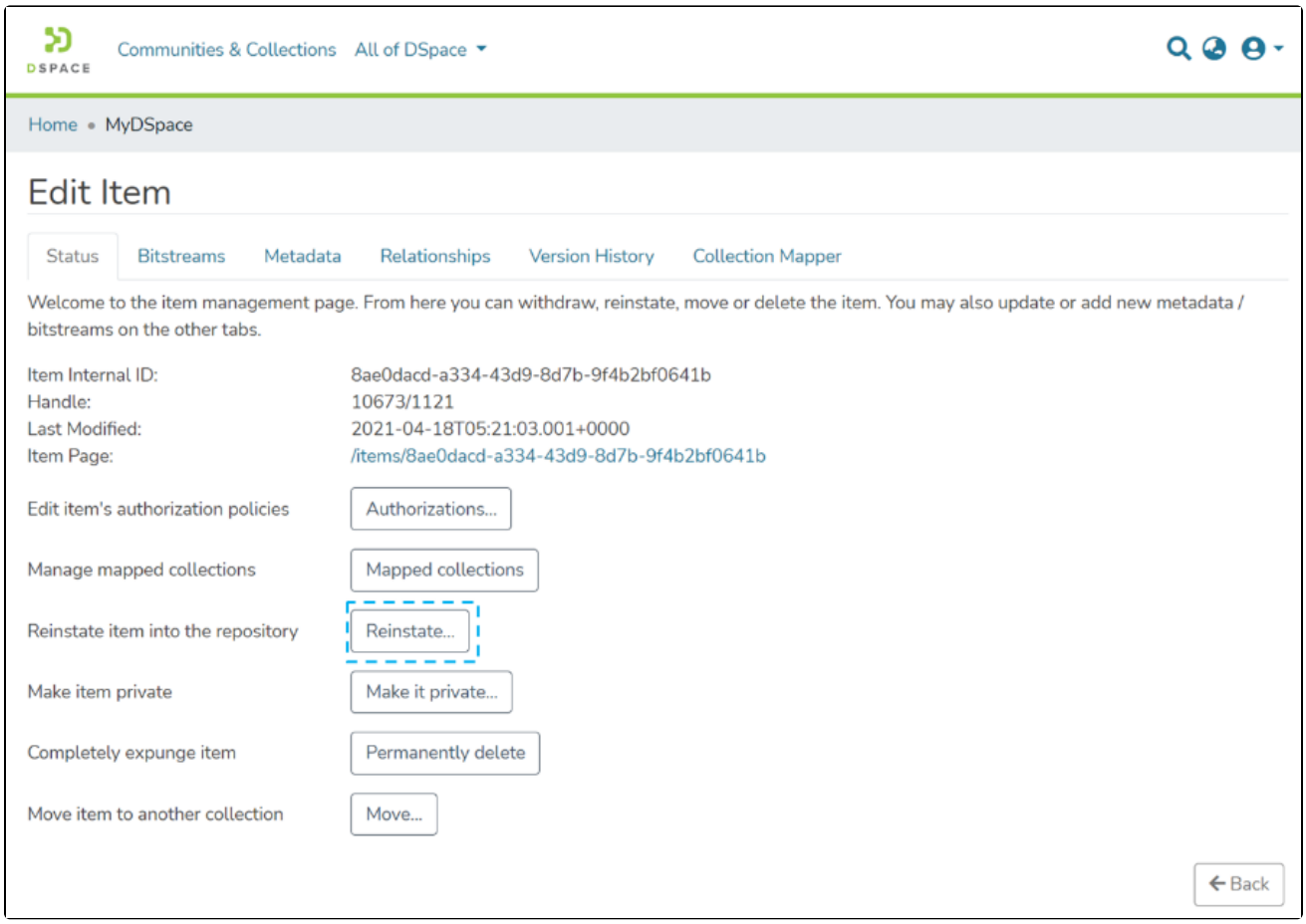
Users can reach an item through multiple methods, which are listed below

1. Finding an item using search functionalities of the DSpace
2. Getting an item by browsing communities and collections
3. Finding an item in the Administration section at Edit > Item

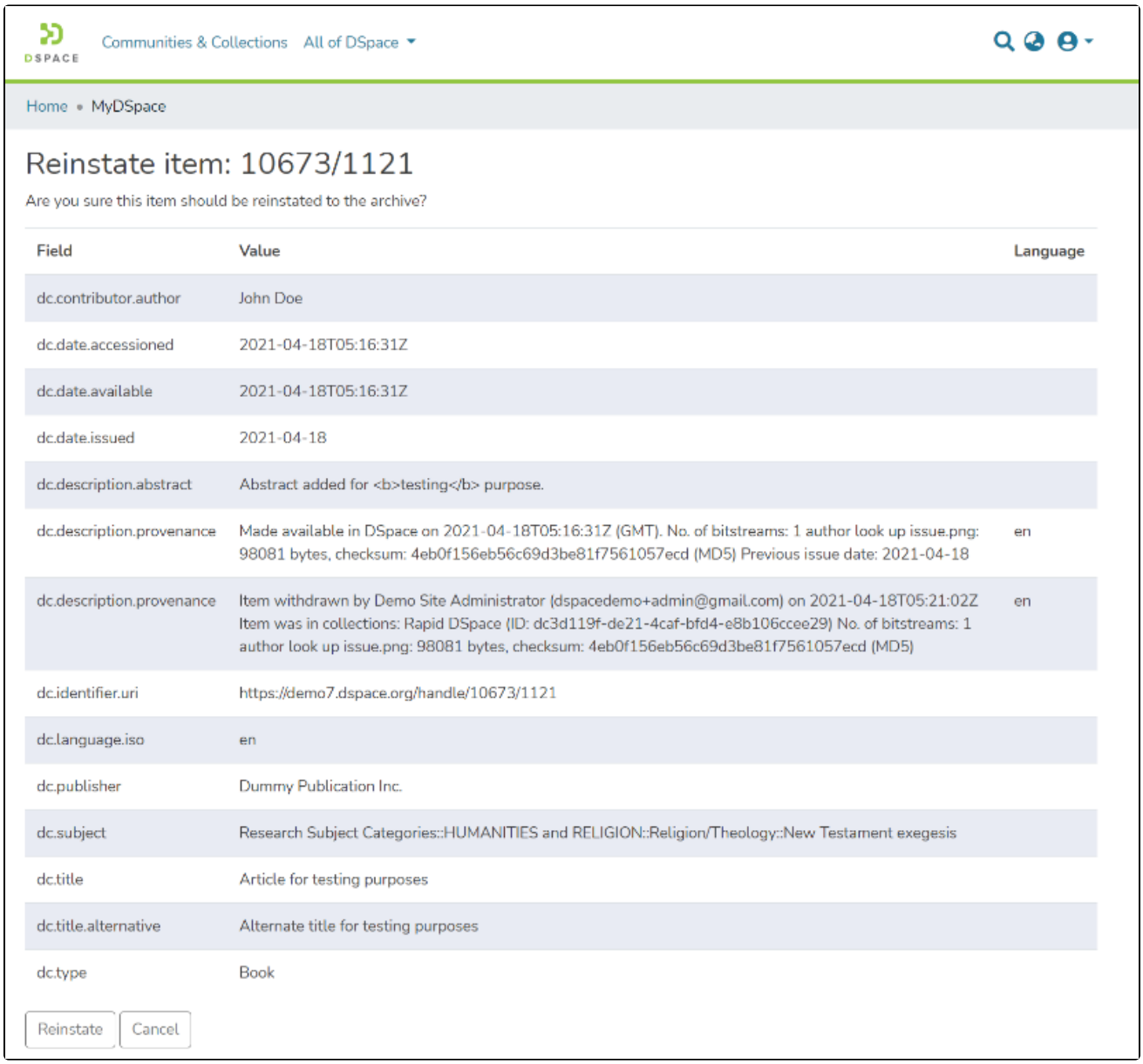
Click on the “Edit” button on the right-hand side of the item title.



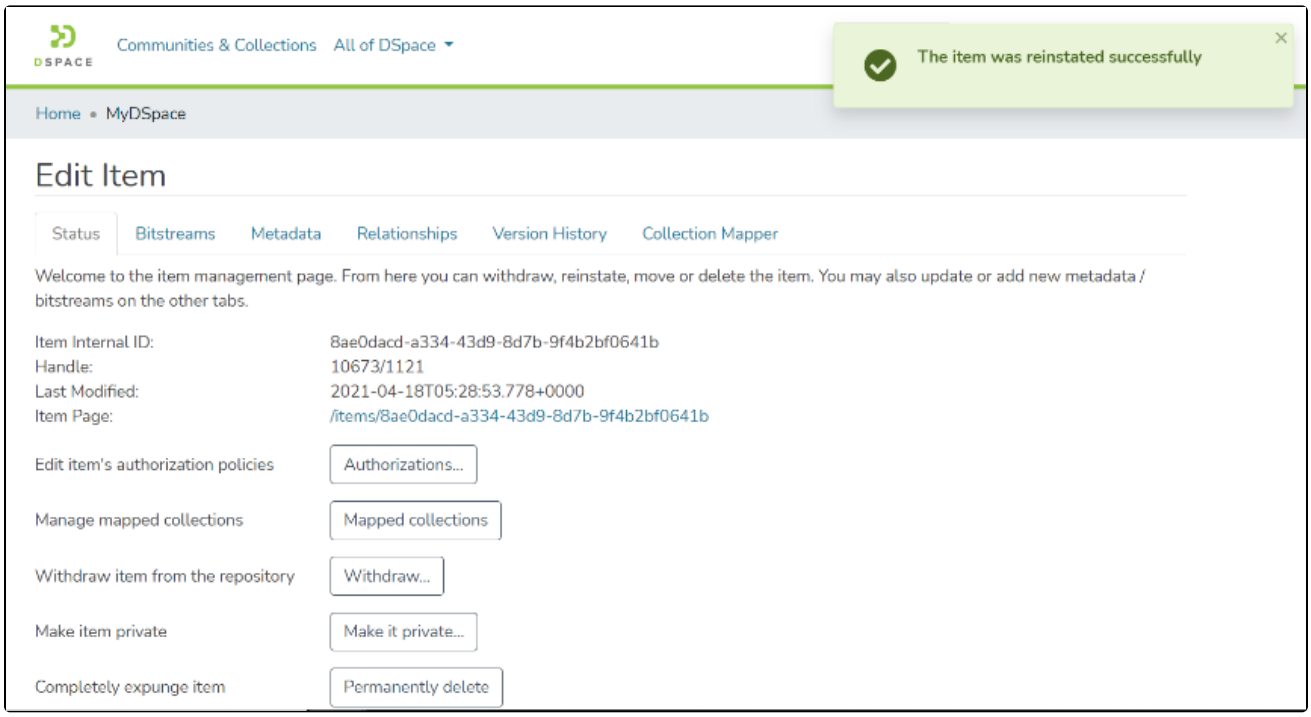
Step 3: Click the “Reinstate” button under the “Status” tab to reinstate the item into the archive.



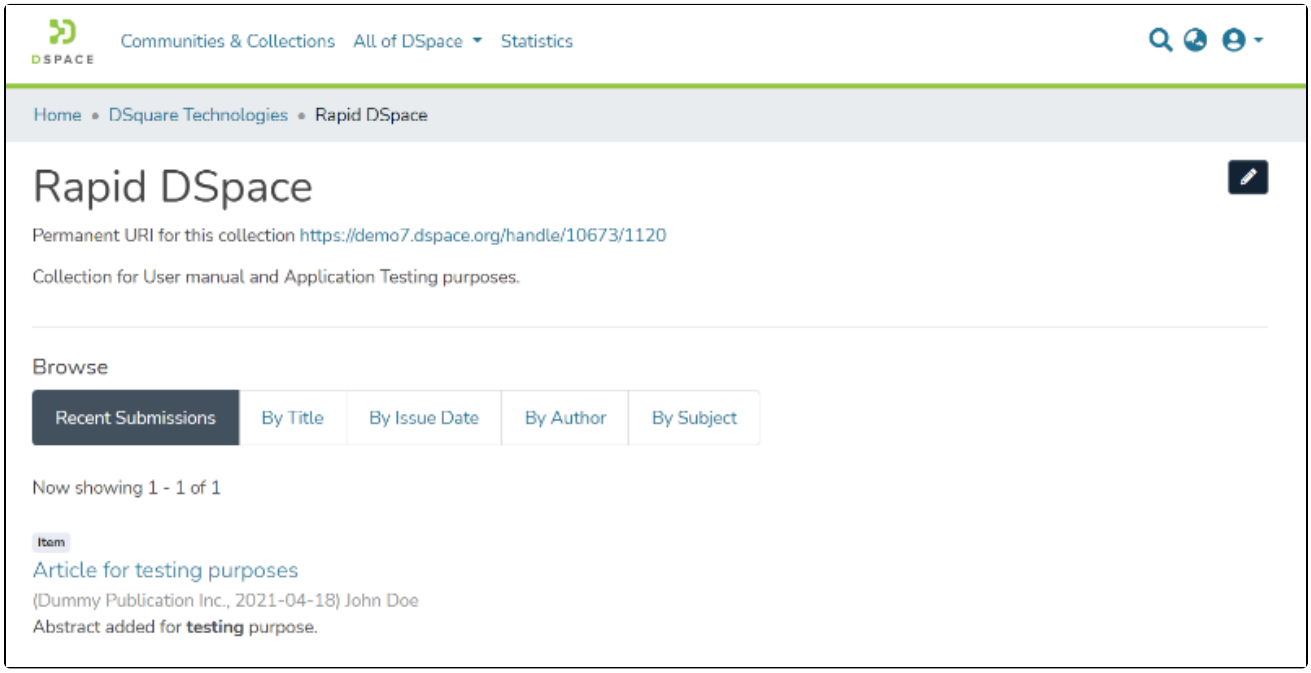
Step 4: Click the “Reinstate” button to reinstate the item or click the “Cancel” button to cancel the operation.



Step 5: Users will see a success prompt confirming the item reinstate, as shown below.



Step 6: Users will notice that the “Withdrawn” tag appearing earlier on top of the item does not appear anymore.



Versioned Item

- Audience
- Create a version
  - Few important facts
- Access Item's versions
  - Additional information for the version creator

DSpace provides version creation and version management functionality. This functionality enables authorized users to create multiple versions of an item to manage changes in its metadata and attachment while keeping track of differences between two versions. Users can also roll back to the previous version.

Audience

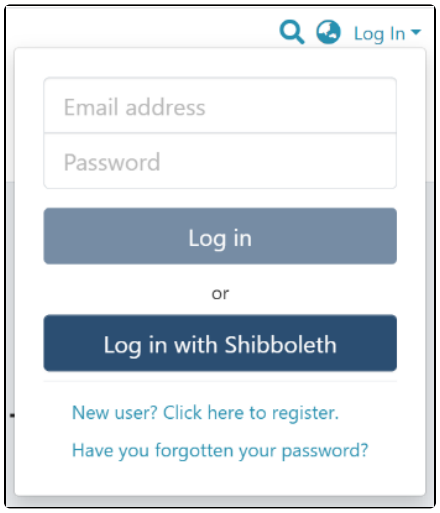
Content Submitters

Community Administrators

System Administrators

Create a version

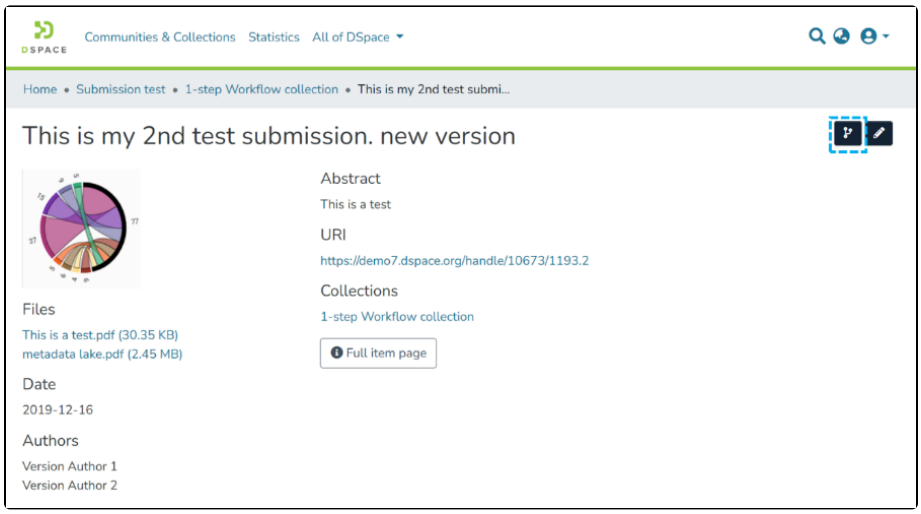
Step 1: Login using the DSpace credentials



Step 2: Users can reach an item to create a version through various methods, which are listed below

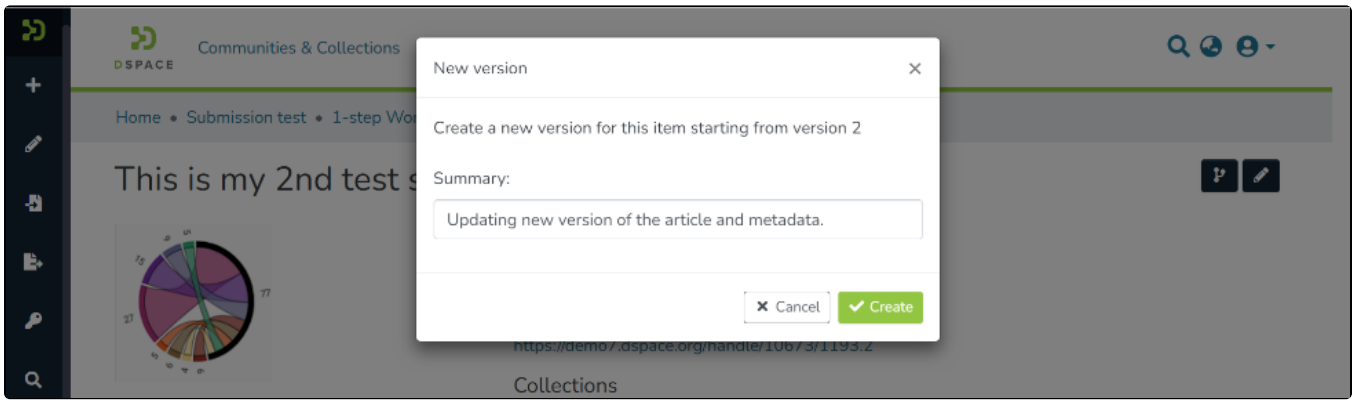
1. Finding an item using search functionalities of the DSpace
2. Reaching an item by browsing communities and collections
3. Finding an item in the Administration section at Edit > Item

Step 3: Users will see the "Create Version" button on the item detail page highlighted below. Click it to create a new version of an item.

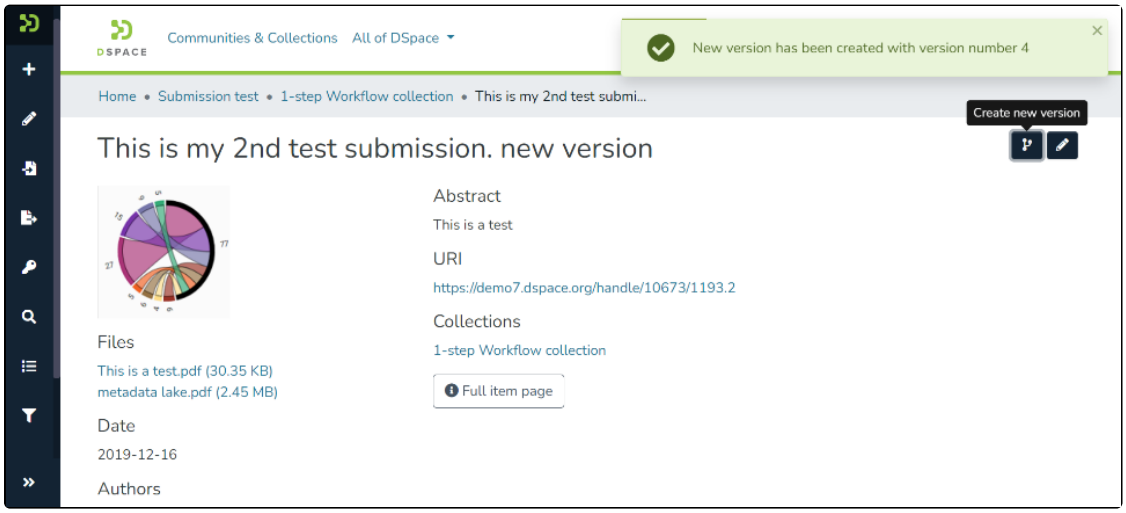


Step 4: After clicking the “Create Version” button, users will see a prompt seeking a summary of the new version. Please enter a summary of changes users will make in the latest version.

Later, this summary plays an essential role in tracking changes made in the version that helps the broader user group and auditors.



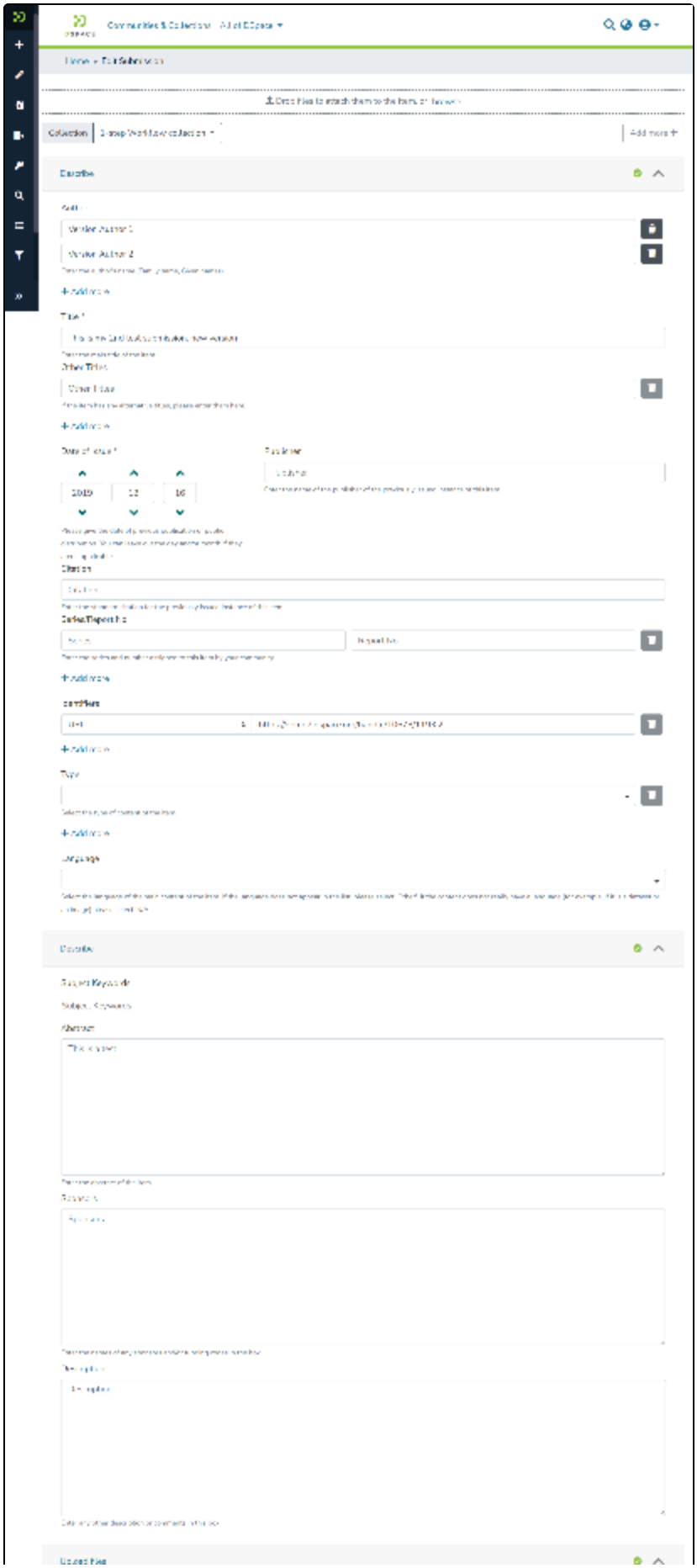
Step 5: Users will see a success prompt confirming a new version creation, as shown below. A page similar to the item submission process will appear with the item’s existing metadata and attachments in an editable mode.

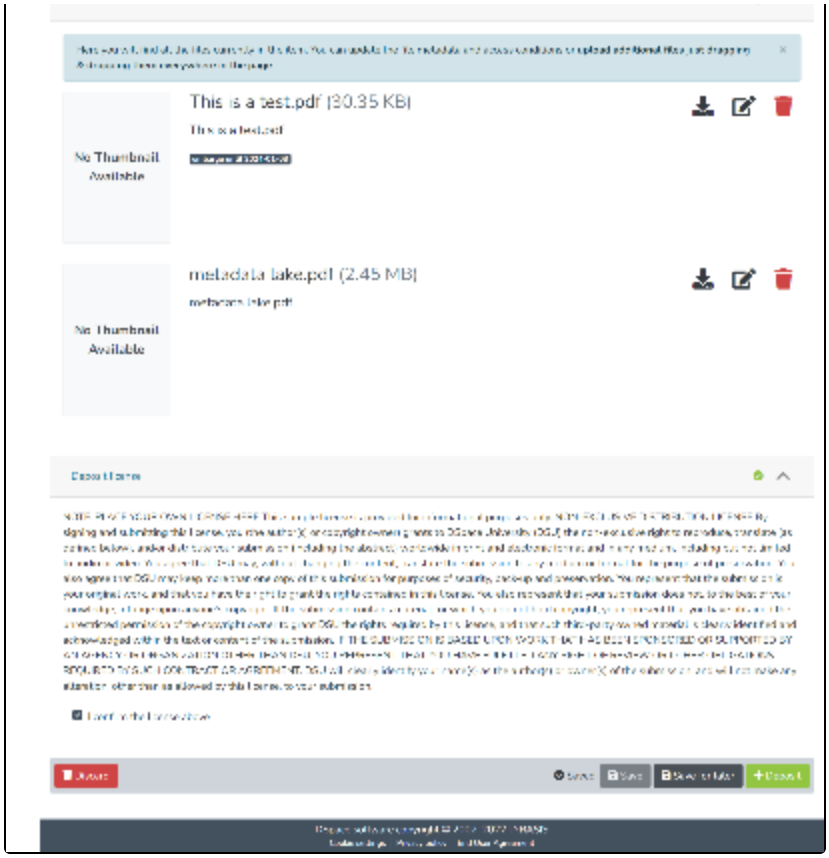


Step 6: Users can update required metadata and attachments on this page the same way they would have done during the item submission process.

Few important facts

- *Users can update and add new metadata during the version update process*
- *Like the metadata, one can also update attachments by updating/removing existing attachments and adding new ones.*
- *It's possible to assign a new collection to the latest version. However, it does not change the storage location of the old version.*
- *Suppose the collection where the latest version needs to be stored has approval workflows assigned. The newest version will be published after necessary approvals.*
- *Users can save the draft version during updates and pick it up from their workspace to complete later.*





Step 7: Click on the “Deposit” button to complete the version creation process. Apart from clicking the “Deposit” button, users can perform the following actions during the version creation:

- Discard: Click this button to cancel the version creation process.
- Save: Keep saving the updates intermittently and continue updating version details.
- Save for later: Save the progress and pick up the item from the workspace later to complete the version creation.

As briefed above, if the target collection has the approval workflow assigned to it, then the latest version will appear to users having an approval role for acceptance. However, if no workflow is set to the collection, the new version will be published for public access.

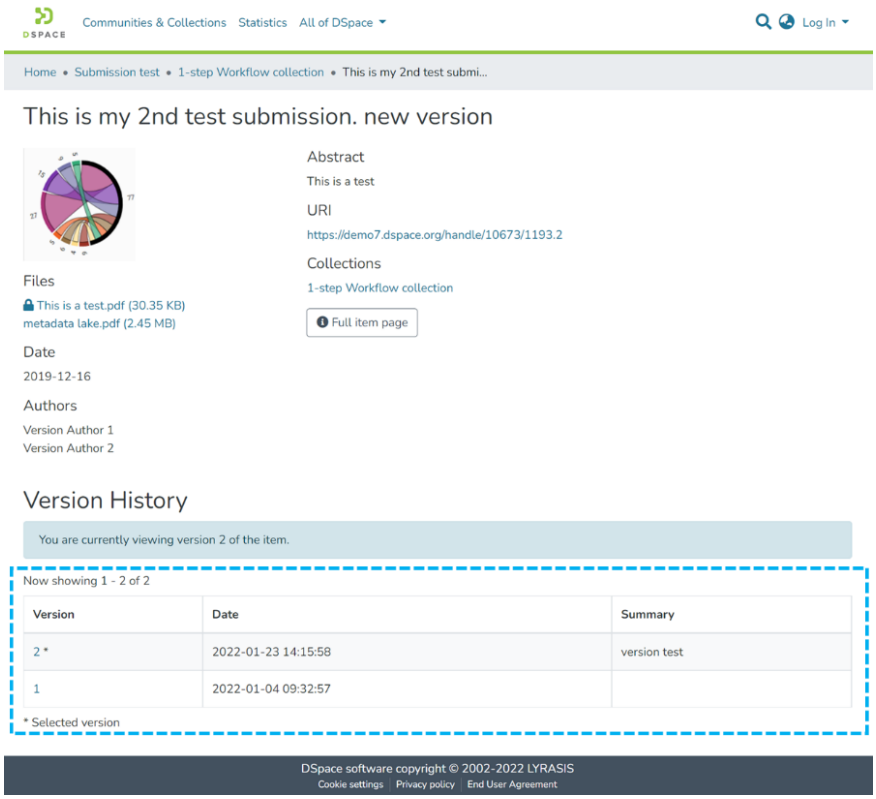
Access Item’s versions

Step 1: Users can reach an item through various methods, which are listed below

1. Finding an item using search functionalities of the DSpace
2. Reaching an item by browsing communities and collections

Step 2: Users can scroll down the item details page to see its version history, as illustrated below. Version history table shows the following details:

- **Version:** The version number of the item. The illustration shows that the selected version has * next to it.
- **Date:** Version creation date and time as per the server.
- **Summary:** Summary added by the user during version creation.

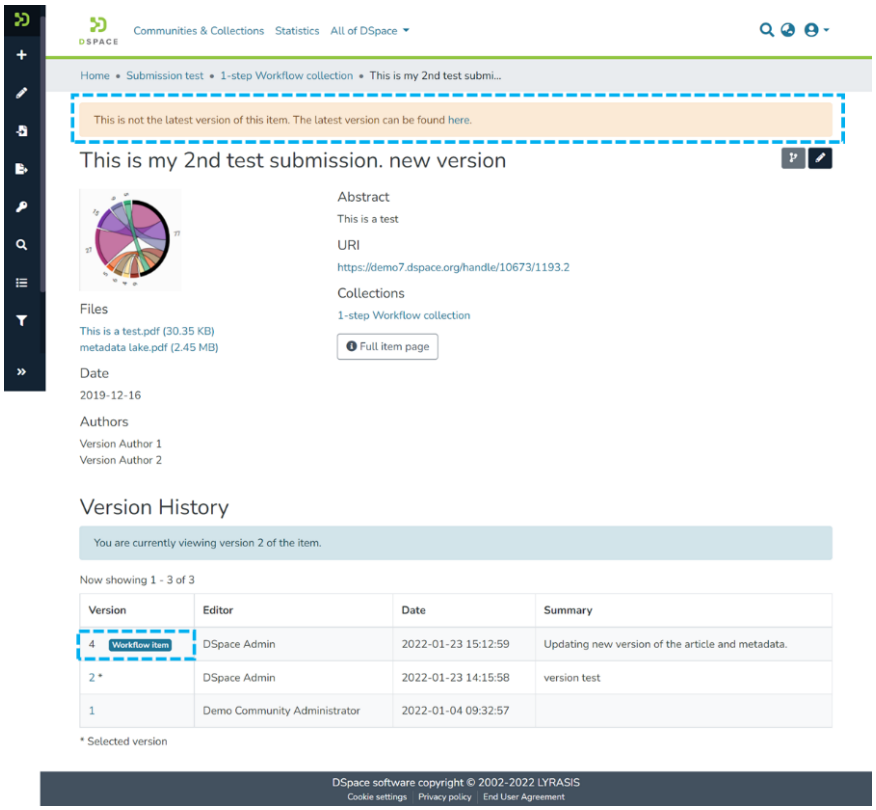


Step 3: The item details page shows information from the latest version. Users can click on the previous version id to see it.

Additional information for the version creator

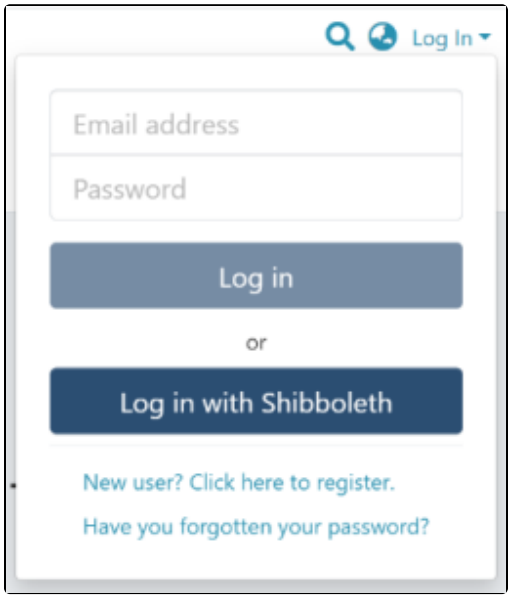
In addition to the above details, the version creator will see the following information related to versioning.

- ***The version in the approval workflow:*** *If a version is unpublished due to pending approval, then the "Workflow Item" tag will appear next to such versions. These versions are not visible to all users.*
- ***Alert about the latest version:*** *An alert confirms the page is not the newest version, and a link to the newest version appears at the top of the page.*


Communities & Collections
Statistics
All of DSpace
Home • Submission test • 1-step Workflow collection • This is my 2nd test submi...
This is not the latest version of this item. The latest version can be found here.
This is my 2nd test submission. new version
Abstract
This is a test
URI
https://demo7.dspace.org/handle/10673/1193.2
Collections
1-step Workflow collection
Full item page
Files
This is a test.pdf (30.35 KB)
metadata lake.pdf (2.45 MB)
Date
2019-12-16
Authors
Version Author 1
Version Author 2
Version History
You are currently viewing version 2 of the item.
Now showing 1 - 3 of 3
Version
Editor
Date
Summary
4 Workflow item
DSpace Admin
2022-01-23 15:12:59
Updating new version of the article and metadata.
2 *
DSpace Admin
2022-01-23 14:15:58
version test
1
Demo Community Administrator
2022-01-04 09:32:57
* Selected version
DSpace software copyright © 2002-2022 LYRASIS
Cookie settings
Privacy policy
End User Agreement

Withdraw an item

Step 1: Login using the DSpace credentials

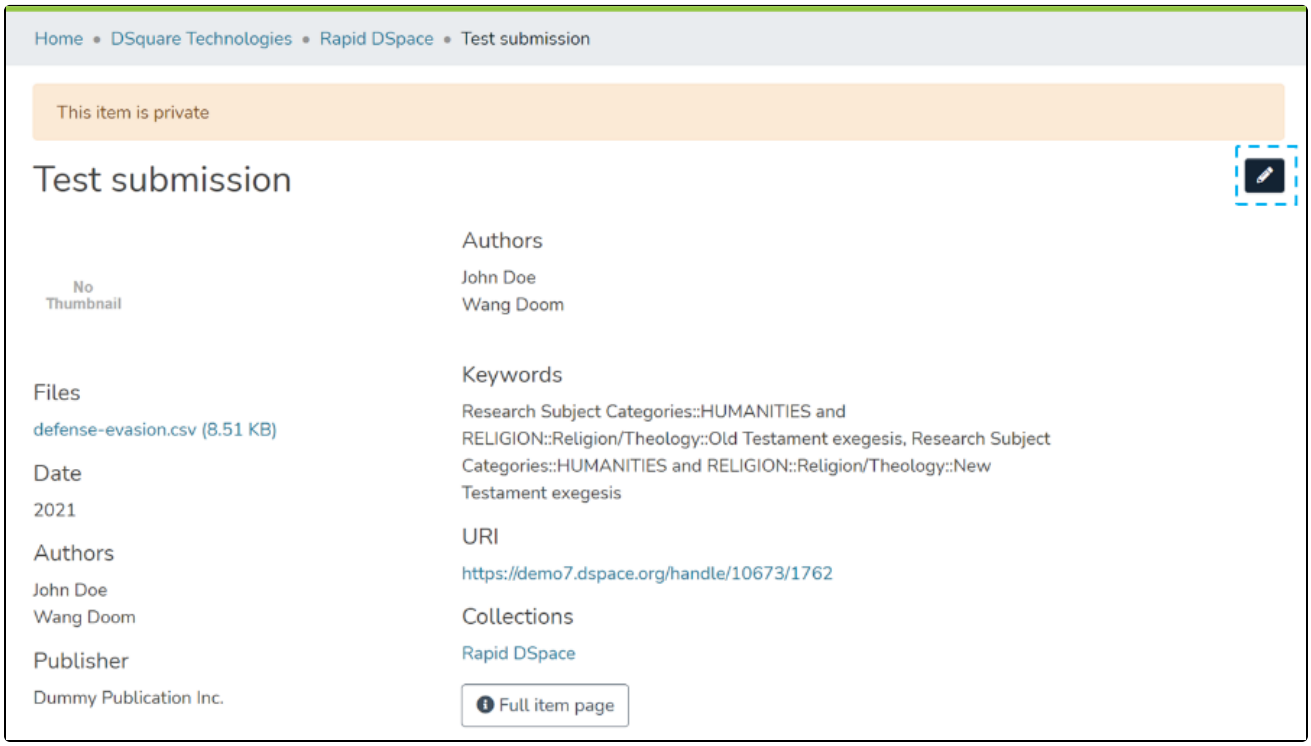


Step 2: Users can go to the item they want to edit

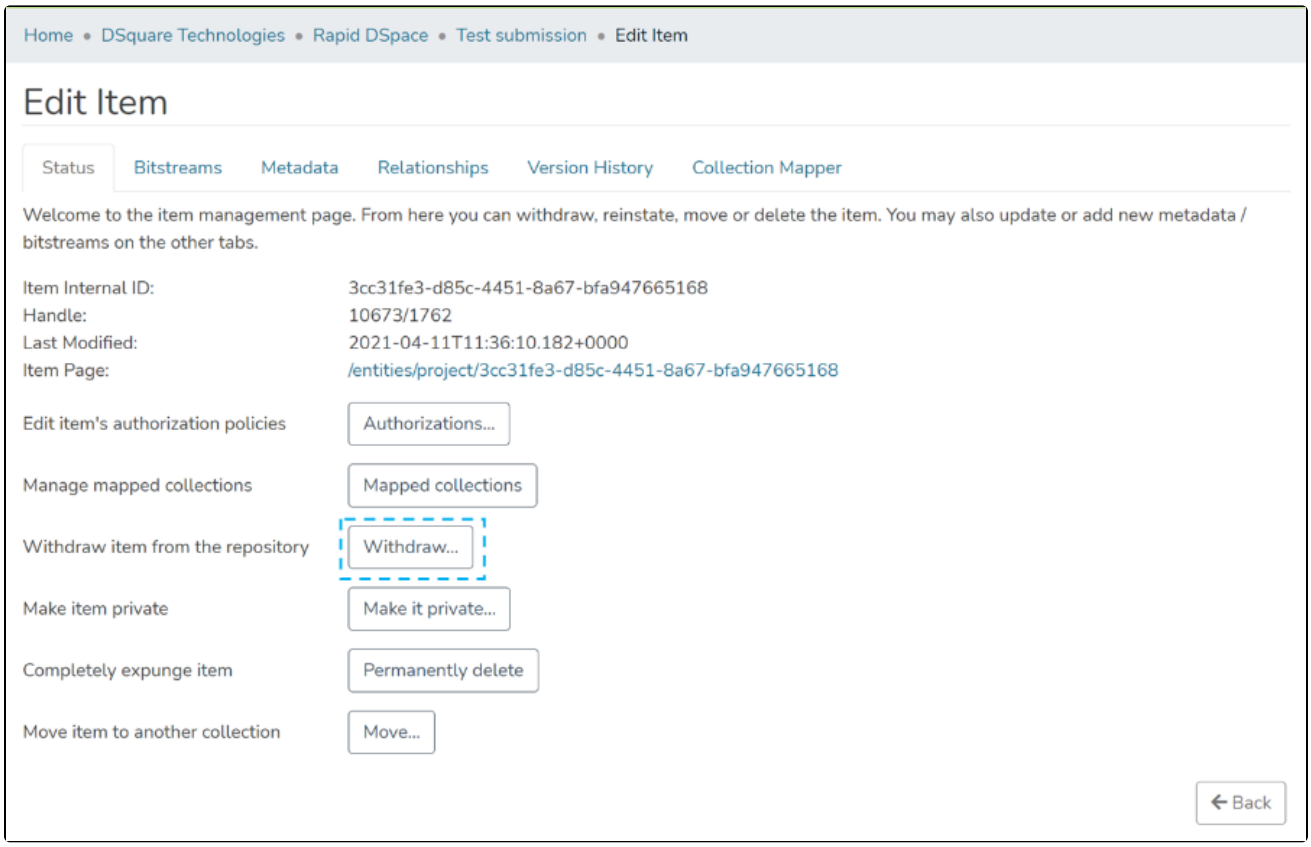
Users can reach an item through multiple methods, which are listed below

1. Finding an item using search functionalities of the DSpace
2. Reaching an item by browsing communities and collections
3. Finding an item in the Administration section at Edit > Item

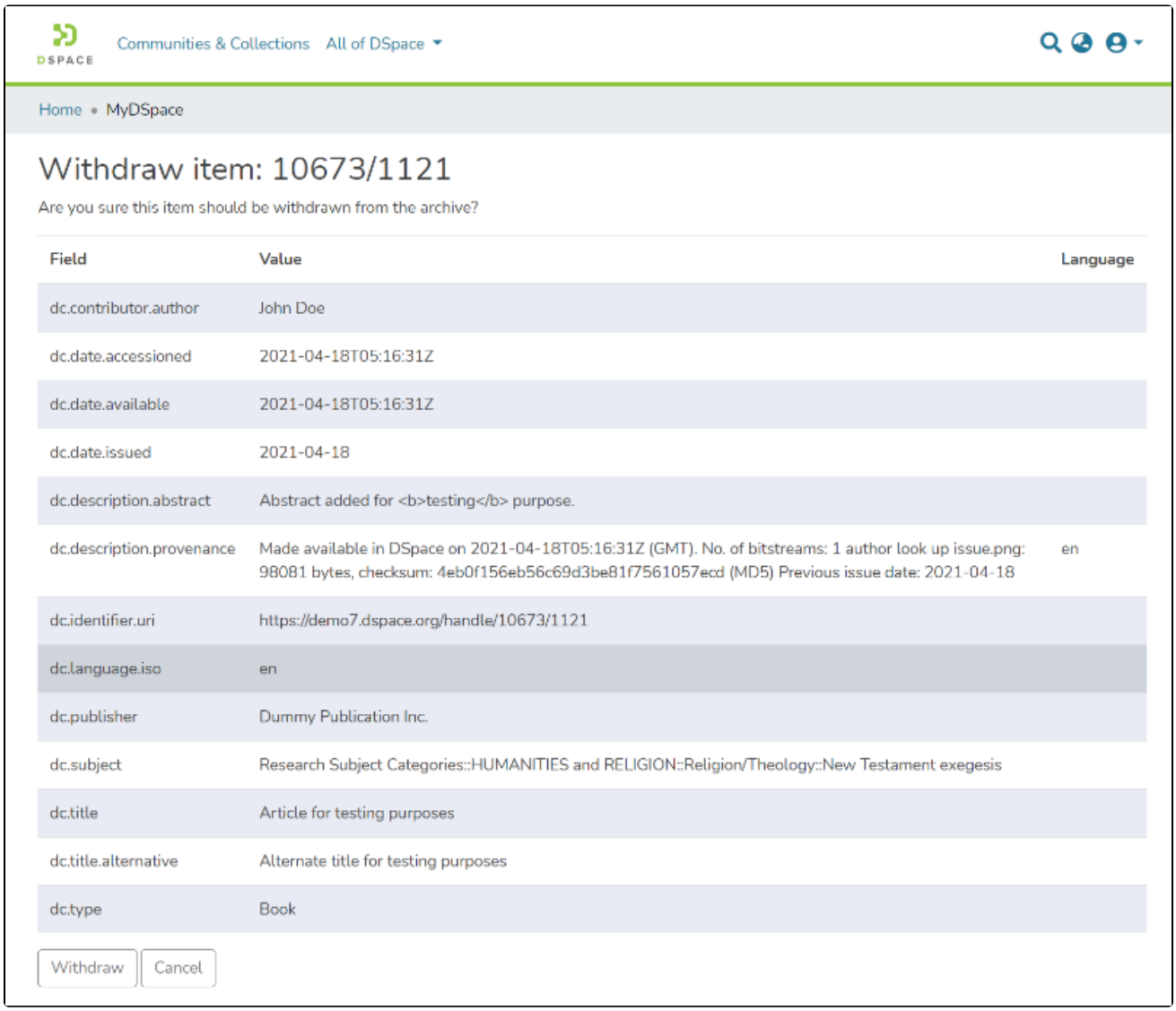
Click on the “Edit” button on the right-hand side of the item title.



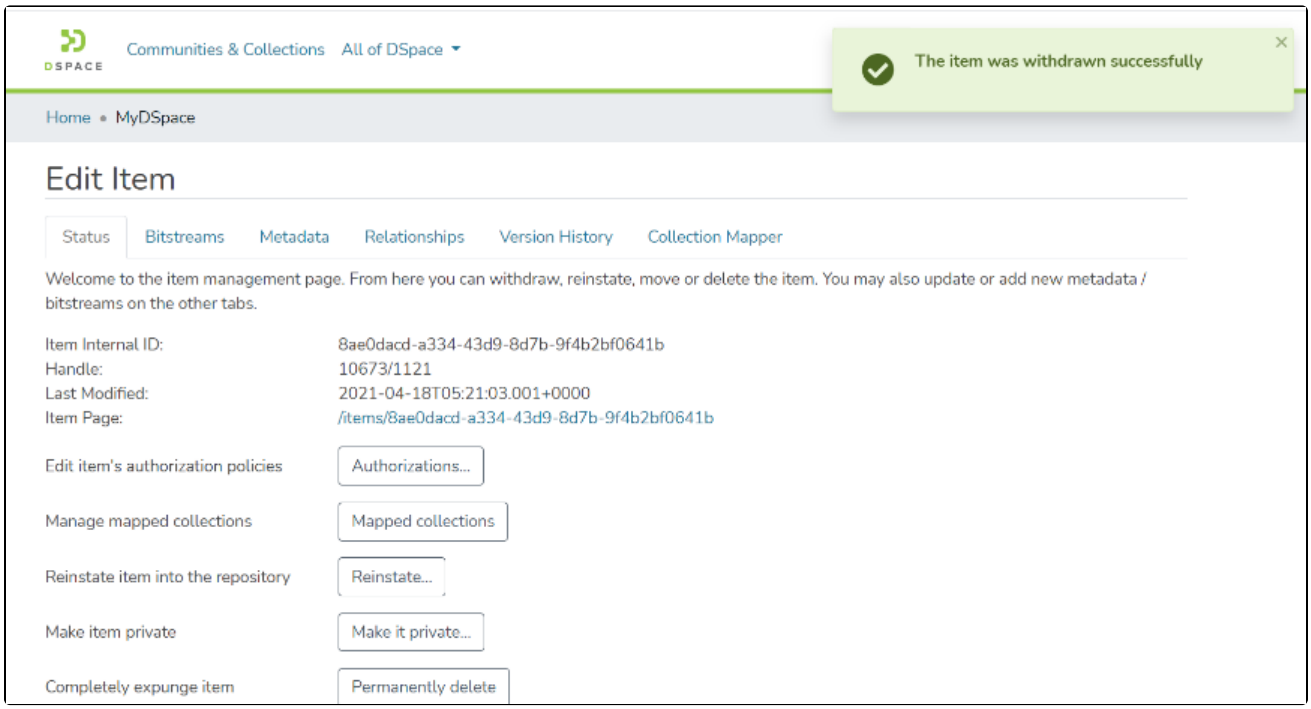
Step 3: Click the “Withdraw” button under the “Status” tab to withdraw the item from the archive.



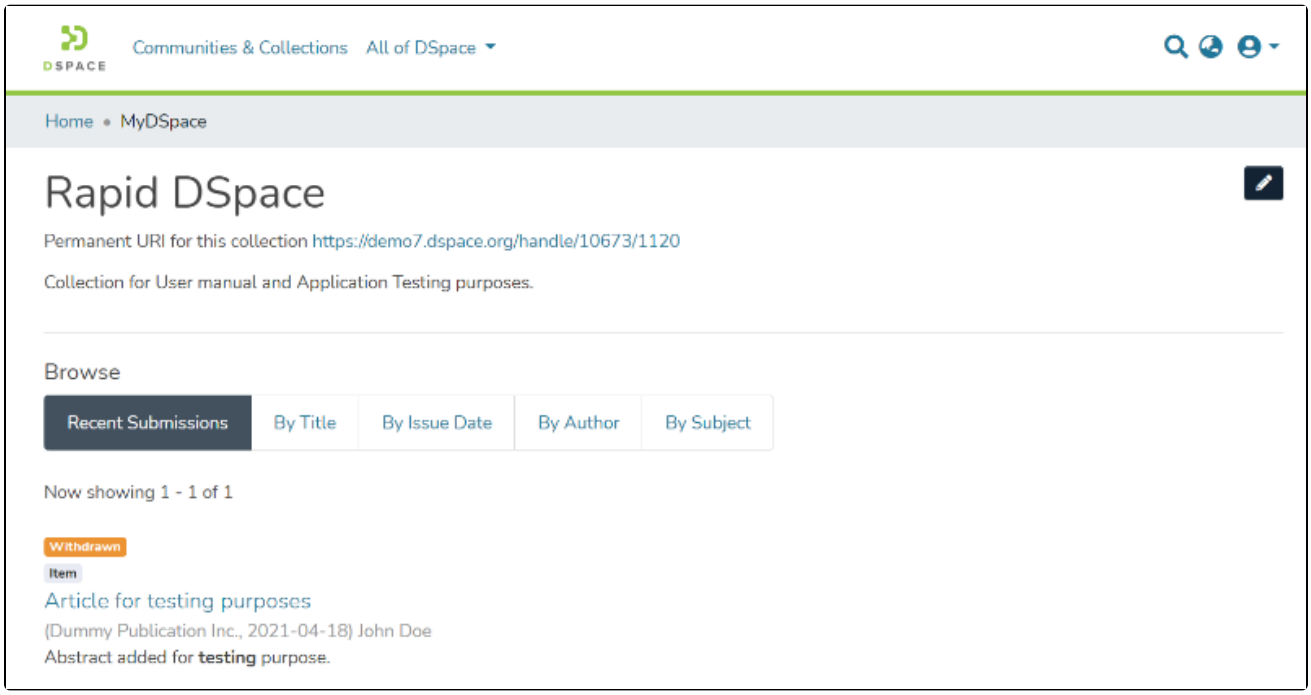
Step 4: Click on the “Withdraw” button to withdraw the item or click the “Cancel” button to cancel the operation.



Step 5: Users will see a success prompt confirming the item withdrawal, as shown below.



Step 6: Users will notice that the item will appear with a “Withdrawn” tag.



Embargo an item

“Embargo an item” helps restrict the Item’s attachment’s access until a future date. A user can embargo an item while submitting it or later by editing it. Both methods to embargo an item are explained below.

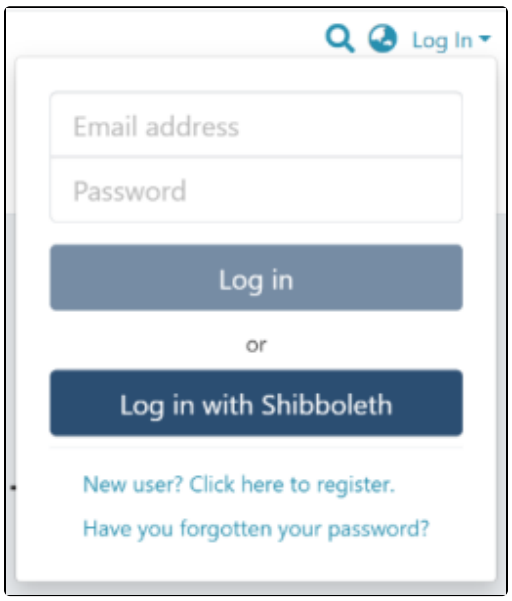
- Audience
- Embargo an item during the item submission
- Embargo an item via edit item

Audience

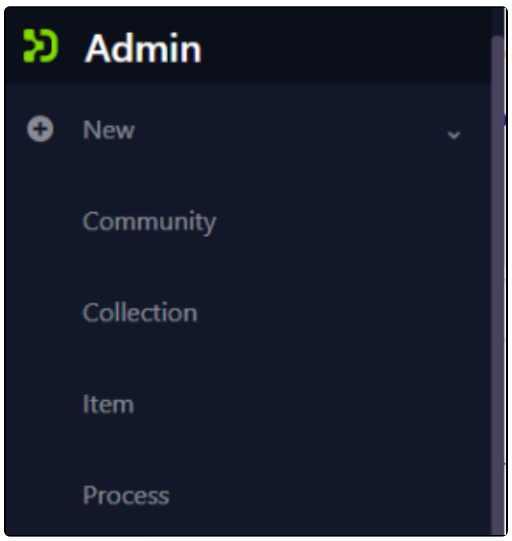
1. Repository Administrator
2. Community Administrator
3. Collection Administrator
4. Item Administrator/submitter

Embargo an item during the item submission

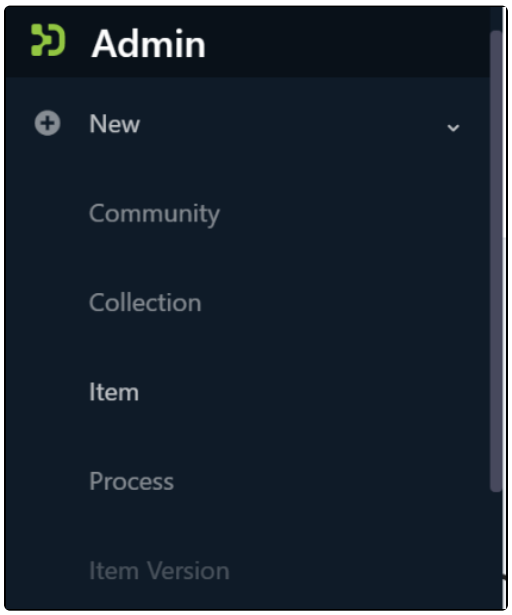
Step 1: Login using your credentials



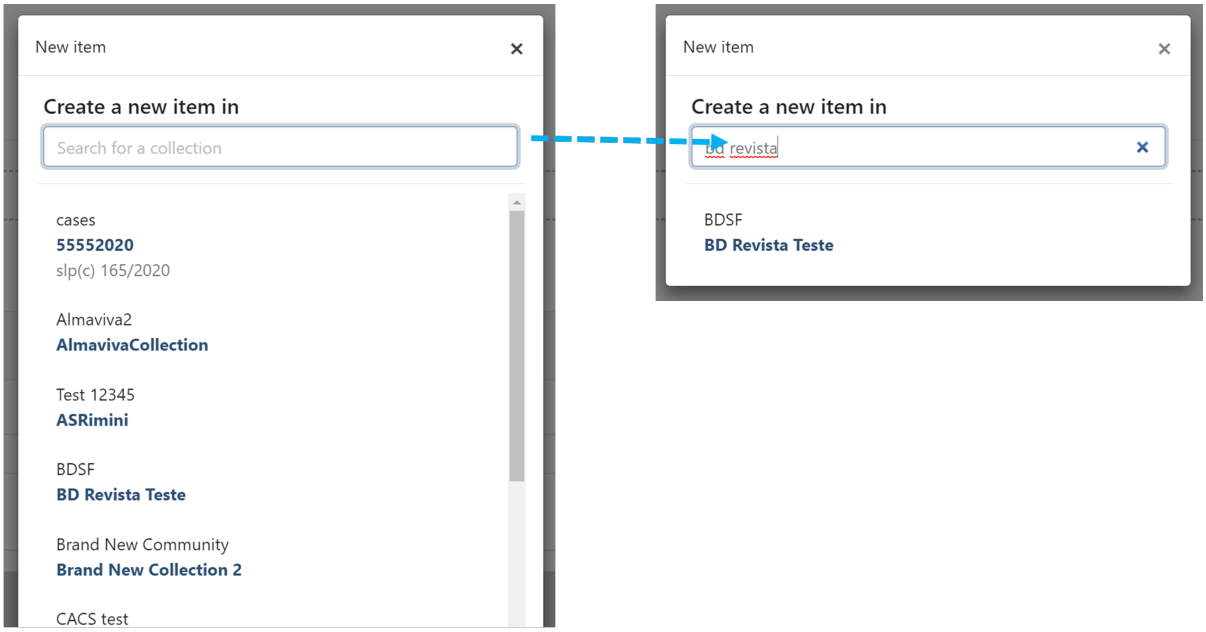
Step 2: Roll over the cursor on the “+” sign.



Step 3: Click on “New” and click on “item” to proceed further in the Item addition process

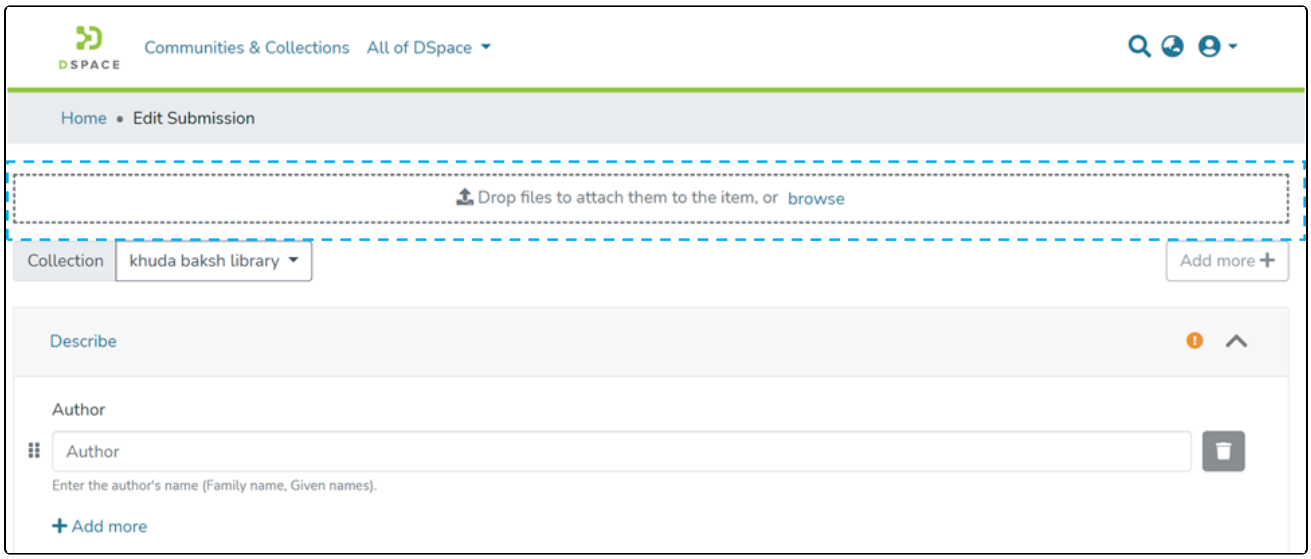


Step 4: A popup window with a collection list will appear. The user can select the target collection by typing its name or scrolling down the collection list. Then, click on the collection to initiate item submission.

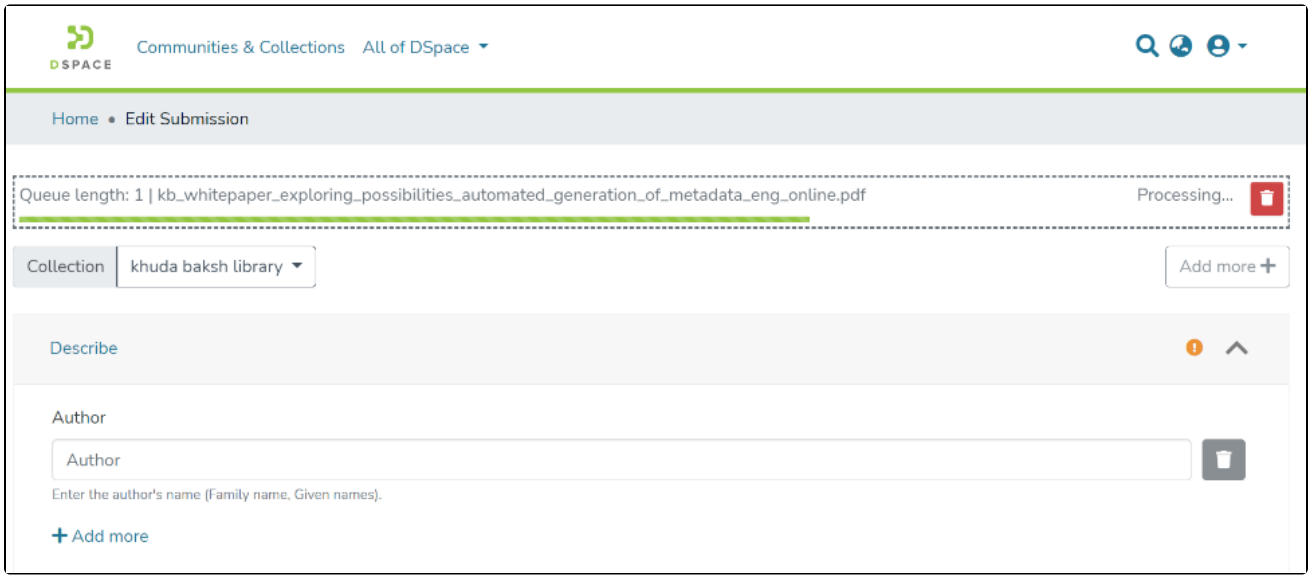


Step 5: Users will see the item submission form after selecting the target collection. The first step is to upload attachment(s) in the Item. In DSpace terminology, an attachment is known as a “bitstream.”

Click on the “browse” link to upload attachment(s). Users can upload multiple files by selecting them together or dragging them into the space.



A progress bar showing bitstream upload progress will appear, as demonstrated in the illustration below. In addition, after a successful bitstream upload, a prompt confirming success or failure will appear.



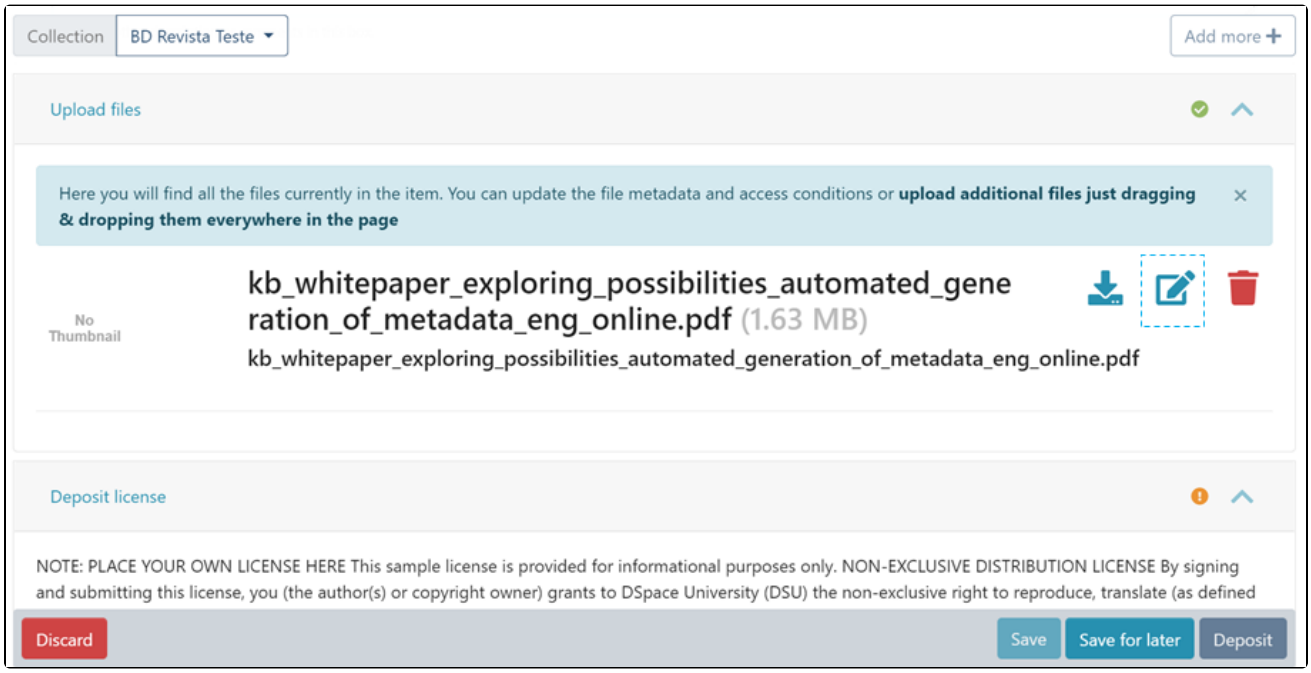
Bitstream upload in progress

Bitstream Upload Successful

Step 6: After uploading bitstream, the next step is to describe the Item by adding metadata.

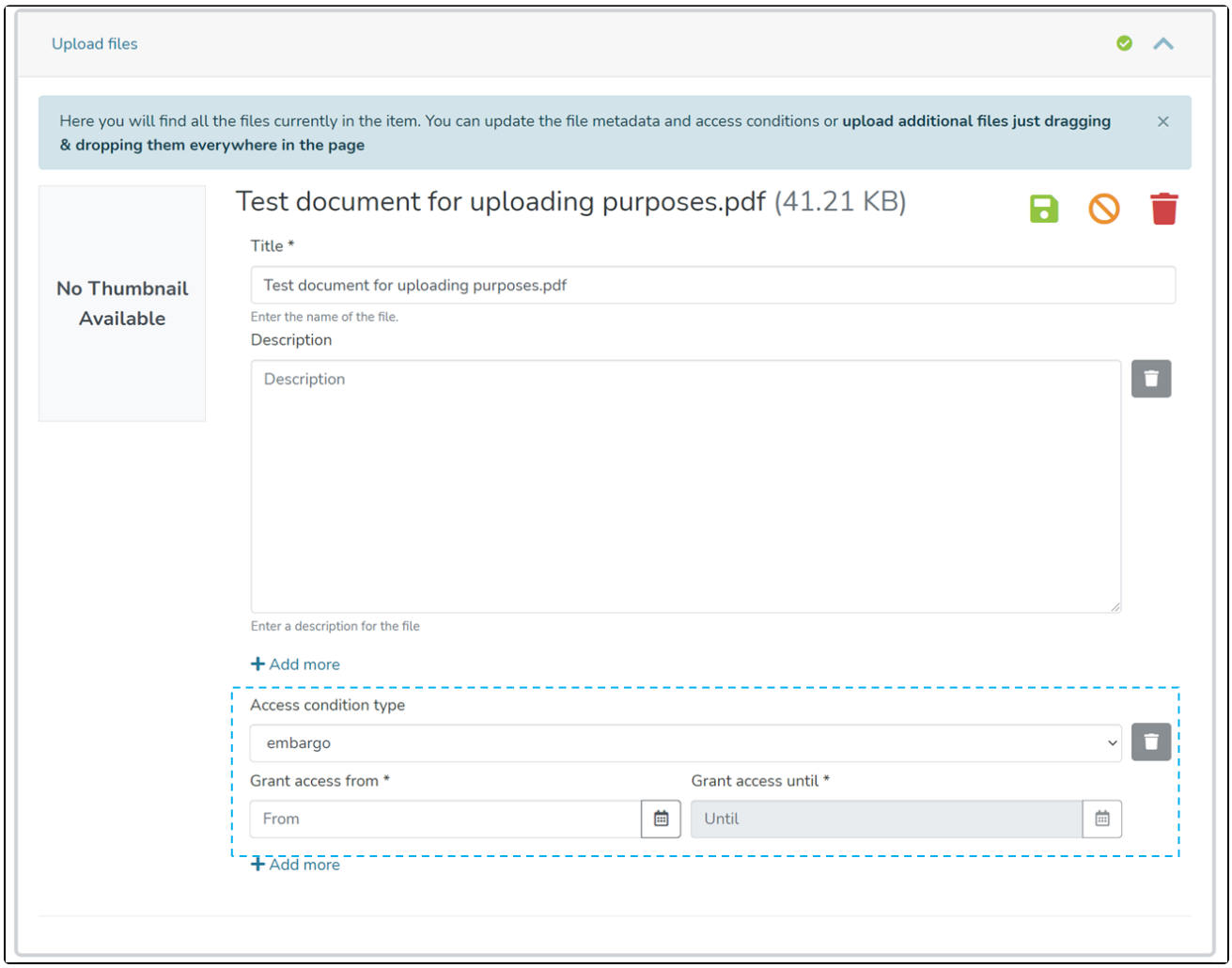
Please refer **Add Item** process for detailed documentation on populating information in the metadata fields.

Step 7: Click on the edit button against any attachment to add embargo policy.



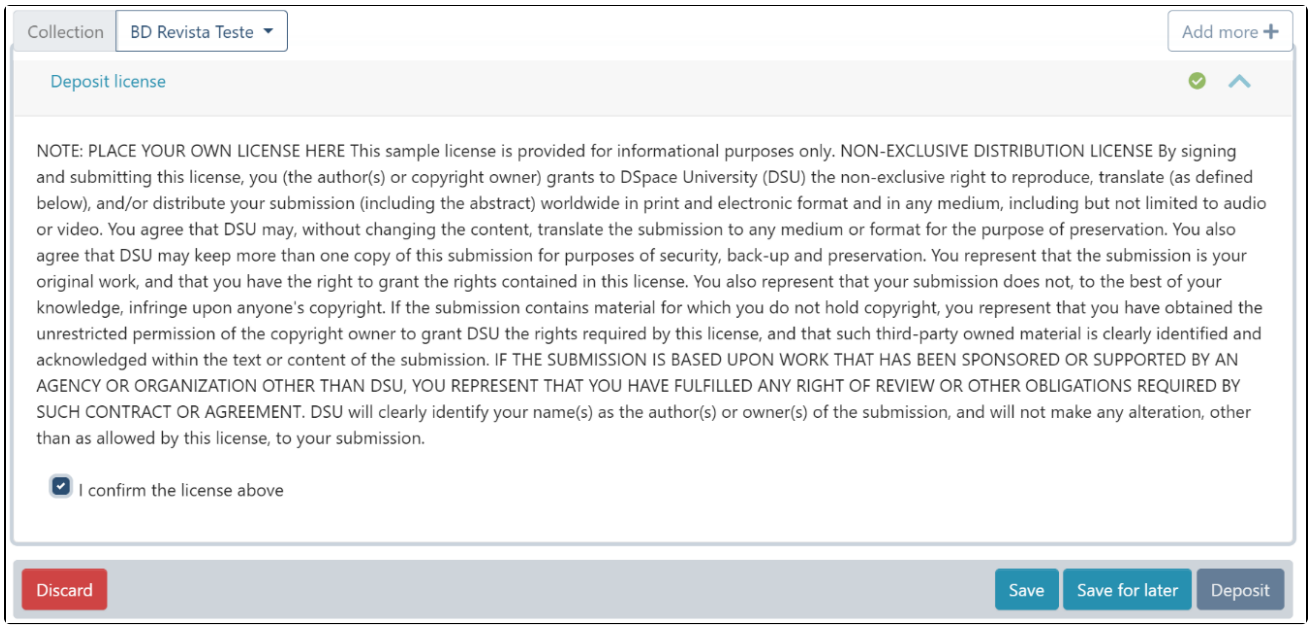
Step 8: Users can apply multiple policies on an attachment, and there are various available options. Click on the dropdown list under the “Access condition” type and select embargo, as highlighted in the screenshot below.

After selecting embargo in the dropdown list, the “Grant access from” date field will be activated. Next, users can choose the future date, after which the attachment should be accessible to the larger set of DSpace users.



Users can add multiple policies to the attachment by clicking the “Add more” link. For example, a user can define an embargo on an item until a future date. Likewise, a lease policy can keep the attachment open access until another date in the future.

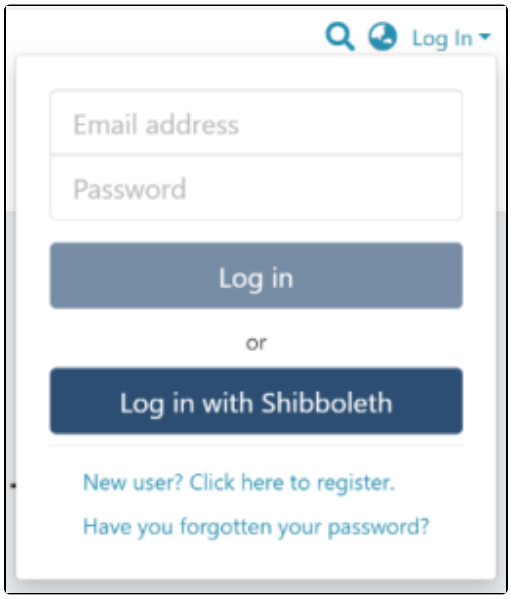
Step 9: After updating all information, the submitter clicks on the “I confirm the license above” checkbox to accept the repository’s license.



Step 10: Click on the “Deposit” button to submit the Item in DSpace. Users will get to see a confirmation prompt upon successful submission of the Item.

Embargo an item via edit item

Step 1: Login using your credentials

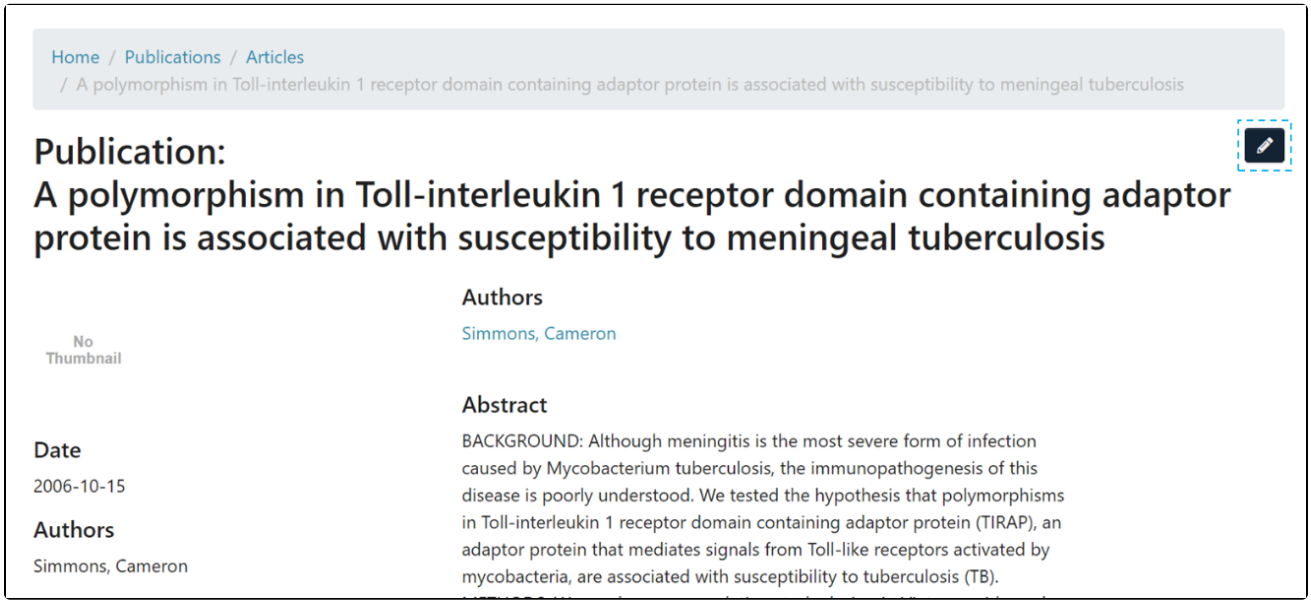


Step 2: Go to the Item you want to edit

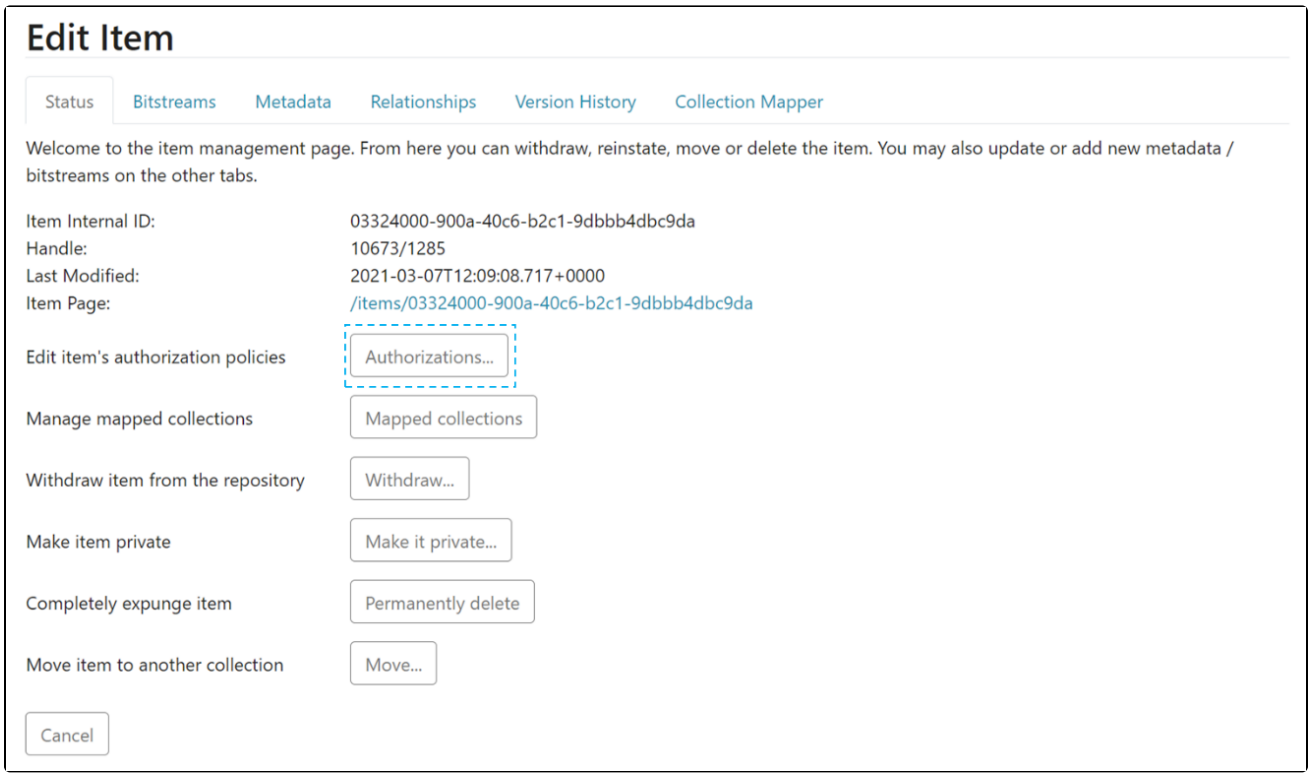
Users can reach an item through multiple methods, as listed below:

1. Search an item
2. Browse communities and collections
3. Finding an item in the Administration section at Edit > Item

Click on the “Edit” button appearing on the right-hand side of the item title.



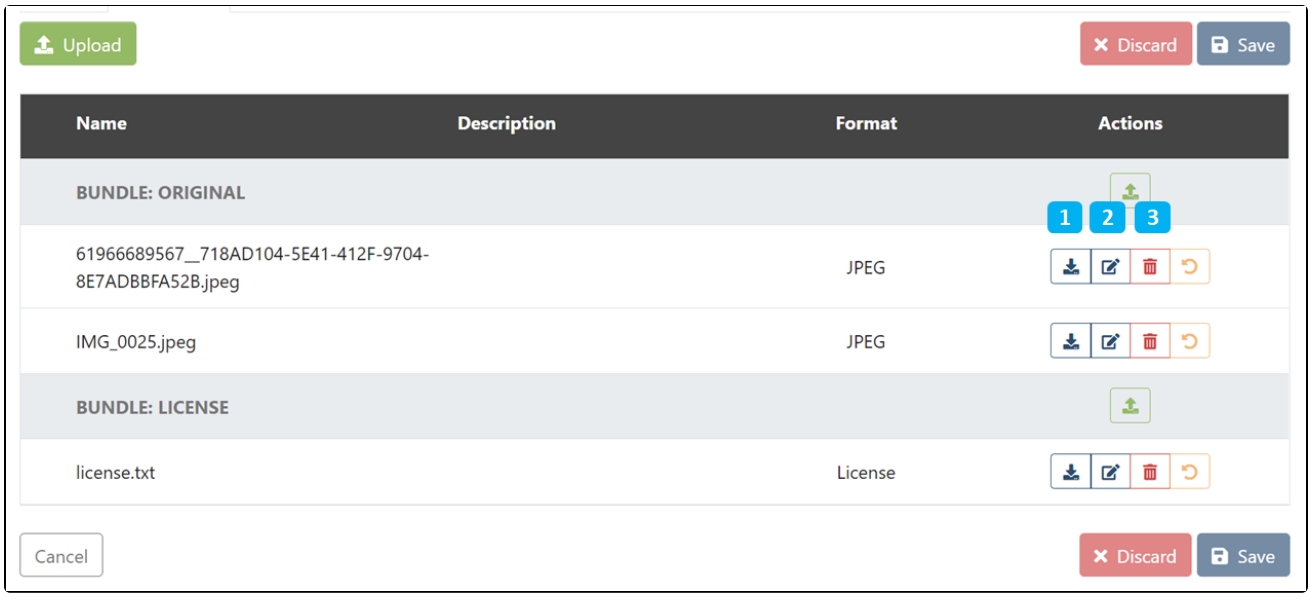
Step 3: Click on the “Authorizations” button under the “Status” tab to continue with adding the embargo policy.



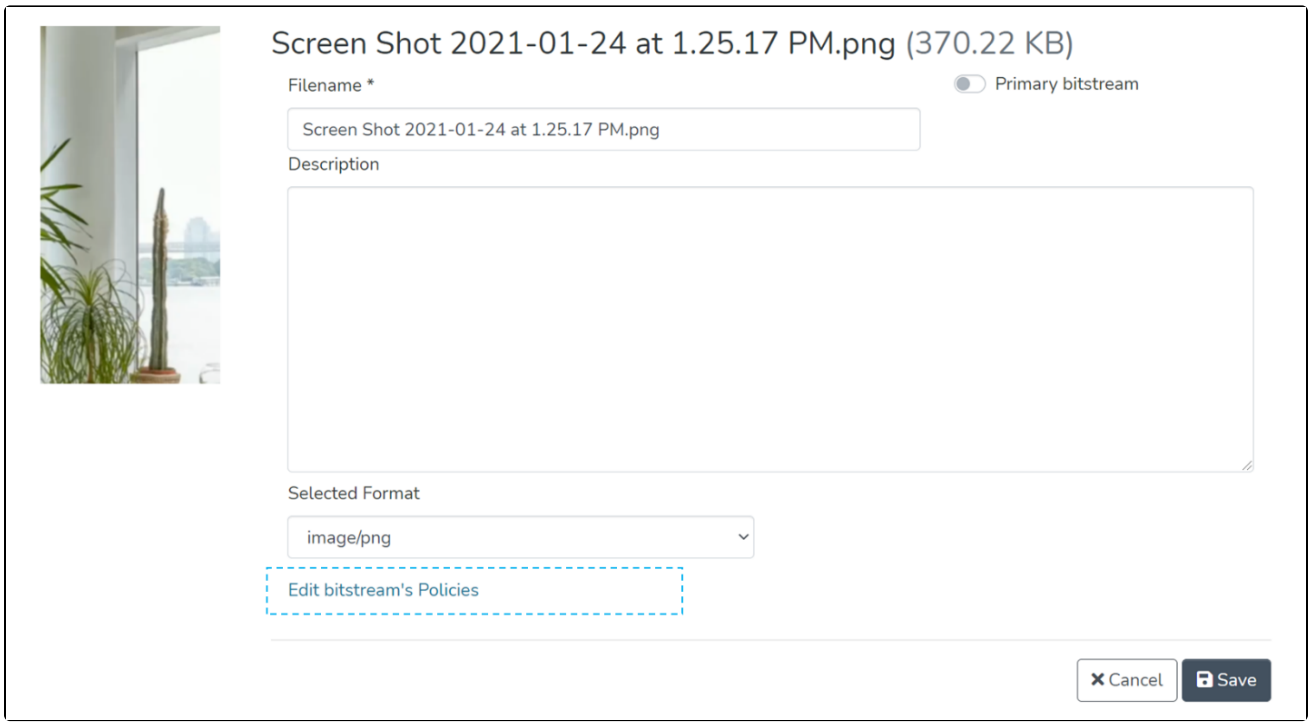
Step 4: The user will see multiple options against each attachment as explained below:

1. **Download Bitstream:** Click on this button to download the attachment on your local device for view.
2. **Edit Bitstream:** Click on the “Edit bitstream” button for editing details. Explained in the next step
3. **Delete Bitstream:** Click on the “Delete Bitstream” button to delete bitstream from the bundle.

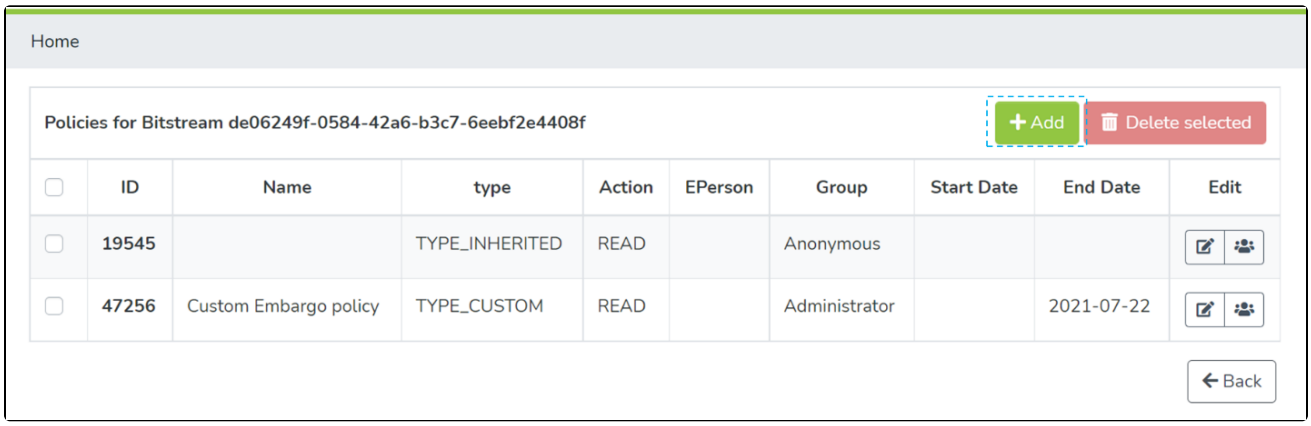
Click on the “edit bitstream” button to embargo the attachment.



Step 5: Click on “Edit bitstream’s Policies” to continue with the embargo process.

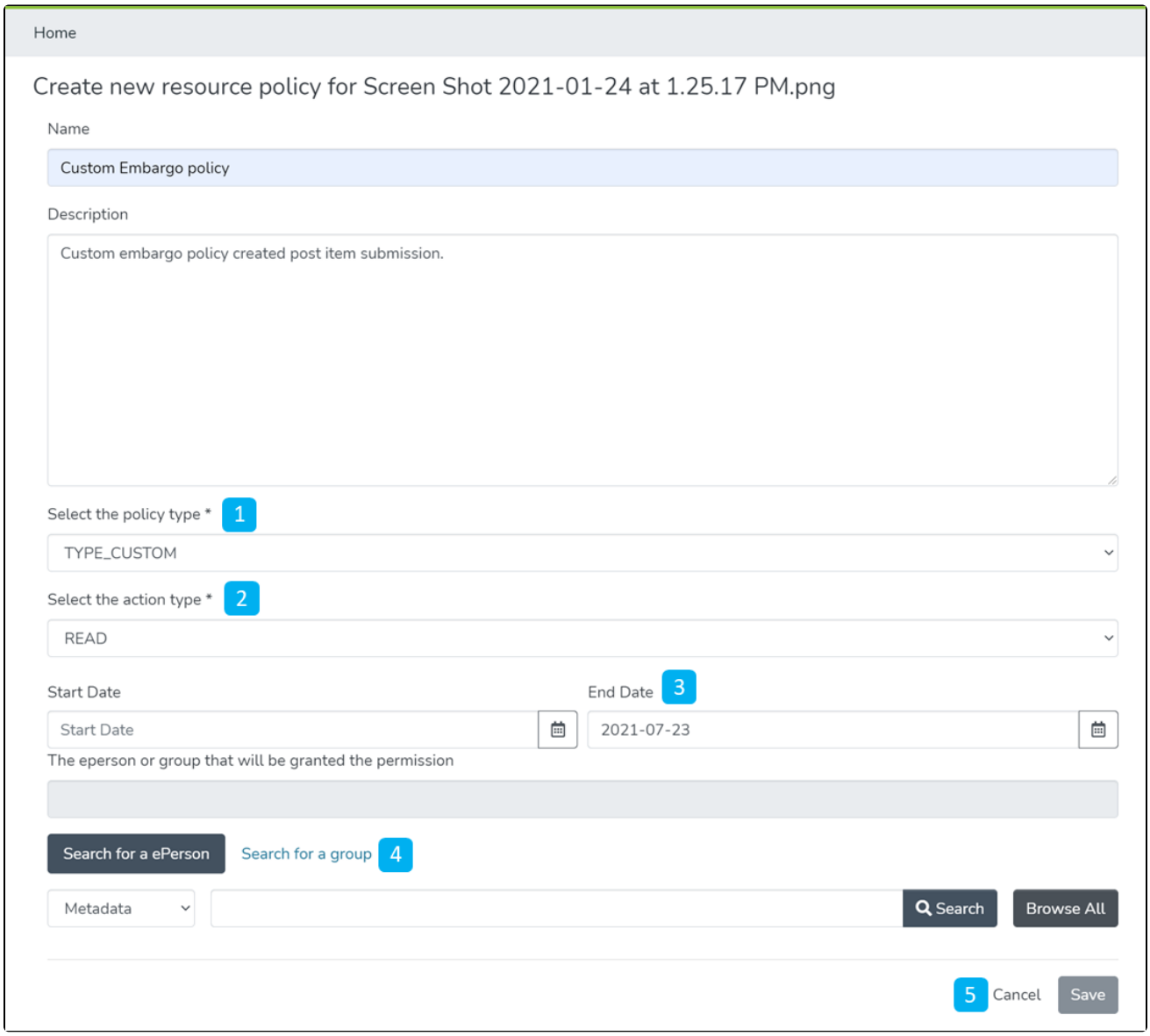


Step 6: Click on the “Add” button to create the custom embargo policy for the attachment.



Step 7: Enter details for creating the embargo policy on this form and perform the following actions:

1. Select ‘TYPE_CUSTOM’ under ‘Select the policy type’ dropdown menu
2. Select ‘READ’ under the ‘Select the action type’ dropdown list.
3. Select a future date under the Start Date to determine when the embargo will end. (The "Start Date" represents the embargo date, as it is when the policy will begin to take effect,)
4. Select E-Person or a User group
5. Click on Save to create the policy



Detailed documentation on various possibilities on this form is available under ‘Edit Bitstream’ user documentation.

Users will see a success prompt upon creating the policy and will be redirected to the bitstream policy page.

Lease an item

“Lease an item” helps restrict the Item’s attachment’s access after a future date. A user can lease an item during the submission or later by editing it. Both methods to lease an item are explained below.

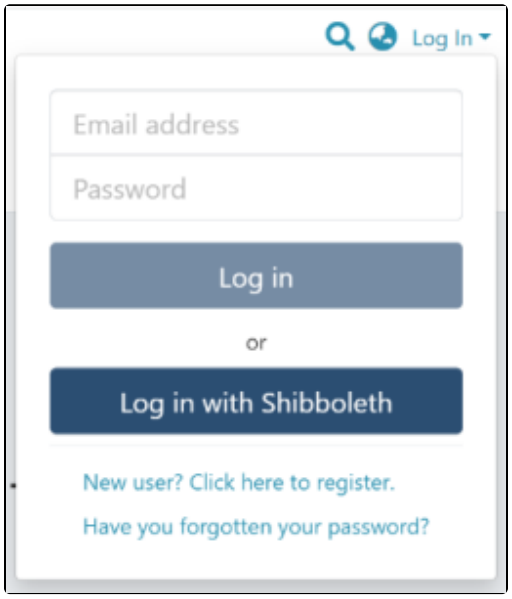
- Audience
- Lease an item during the item submission
- Lease an item via edit item

Audience

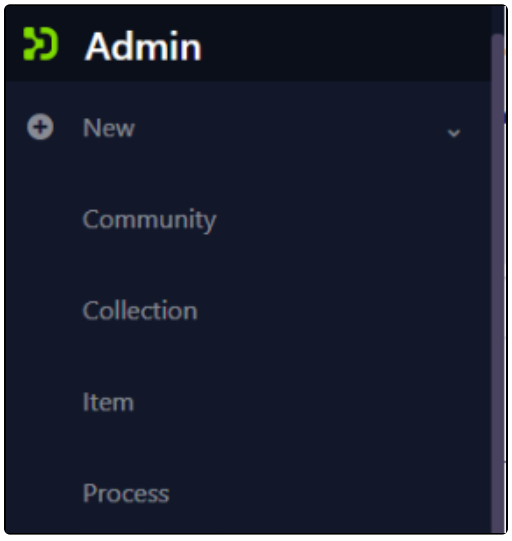
1. Repository Administrator
2. Community Administrator
3. Collection Administrator
4. Item Administrator/submitter

Lease an item during the item submission

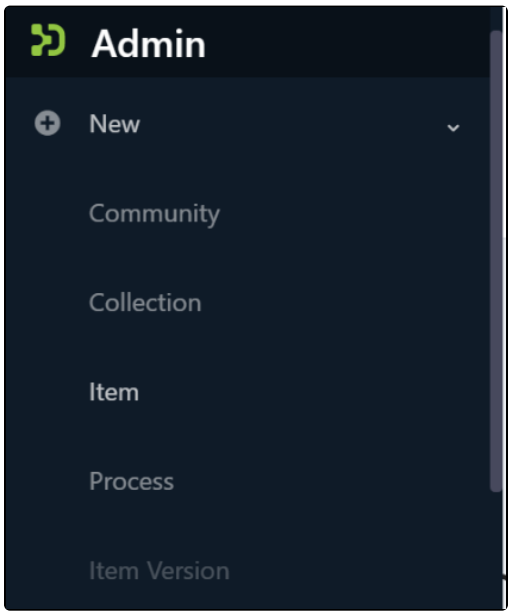
Step 1: Login using your credentials



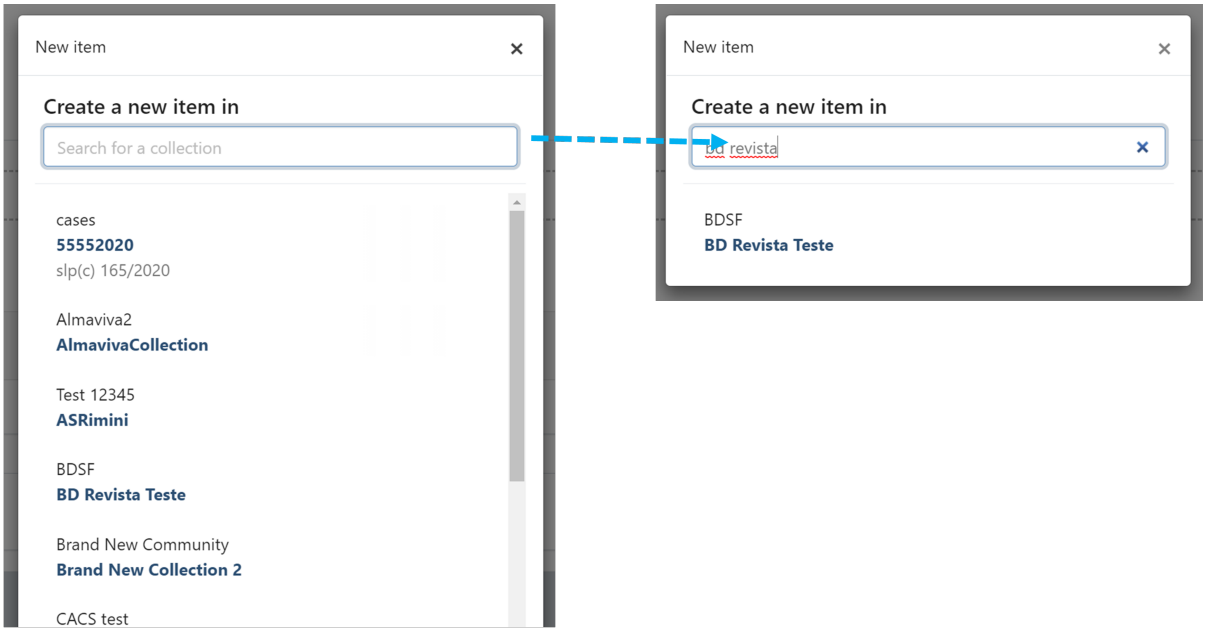
Step 2: Roll over the cursor on the “+” sign.



Step 3: Click on “New” and click on “item” to proceed further in the Item addition process

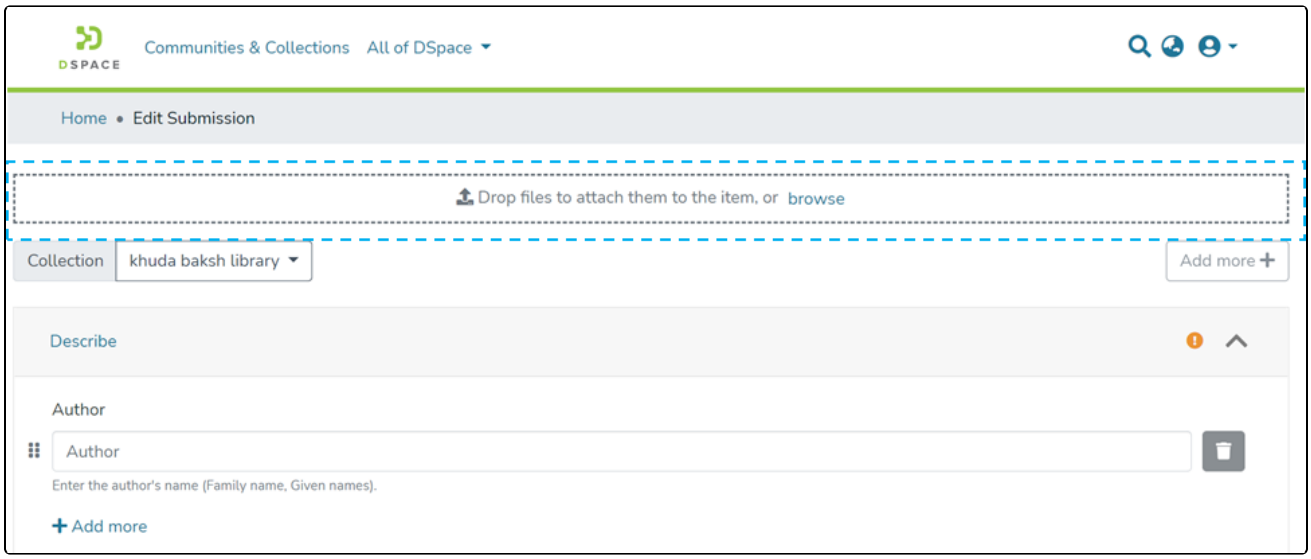


Step 4: A popup window with a collection list will appear. The user can select the target collection by typing its name or scrolling down the collection list. Then, click on the collection to initiate item submission.

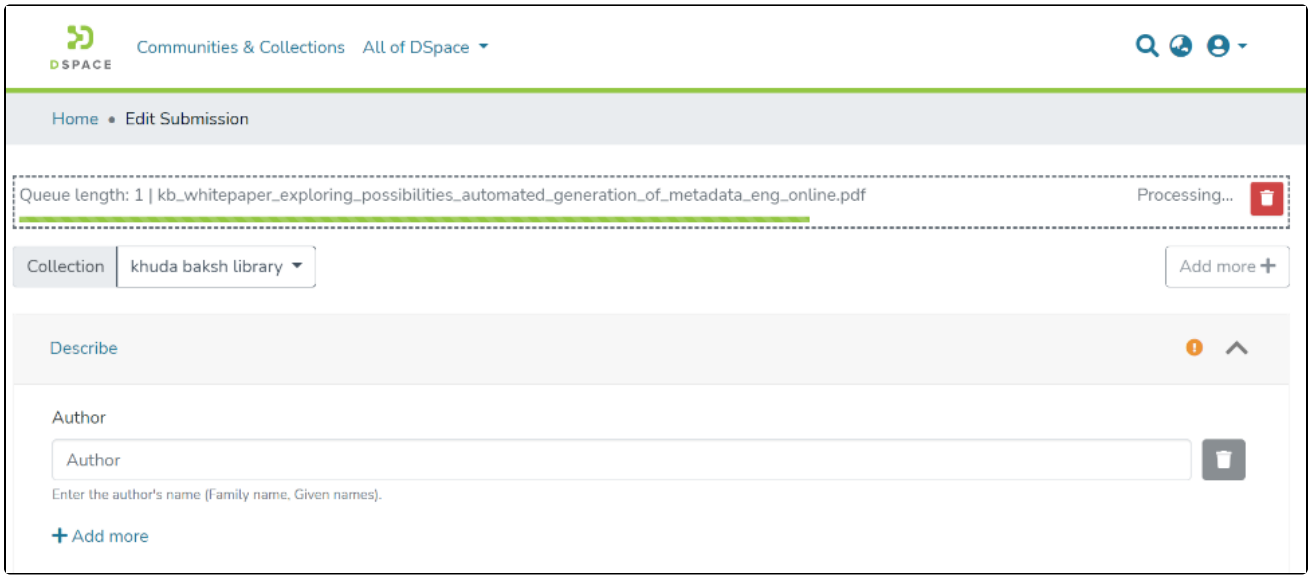


Step 5: Users will see the item submission form after selecting the target collection. The first step is to upload attachment(s) in the Item. In DSpace terminology, an attachment is known as a “bitstream.”

Click on the “browse” link to upload attachment(s). Users can upload multiple files by selecting them together or dragging them into the space.



A progress bar showing bitstream upload progress will appear, as demonstrated in the illustration below. In addition, after a successful bitstream upload, a prompt confirming success or failure will appear.



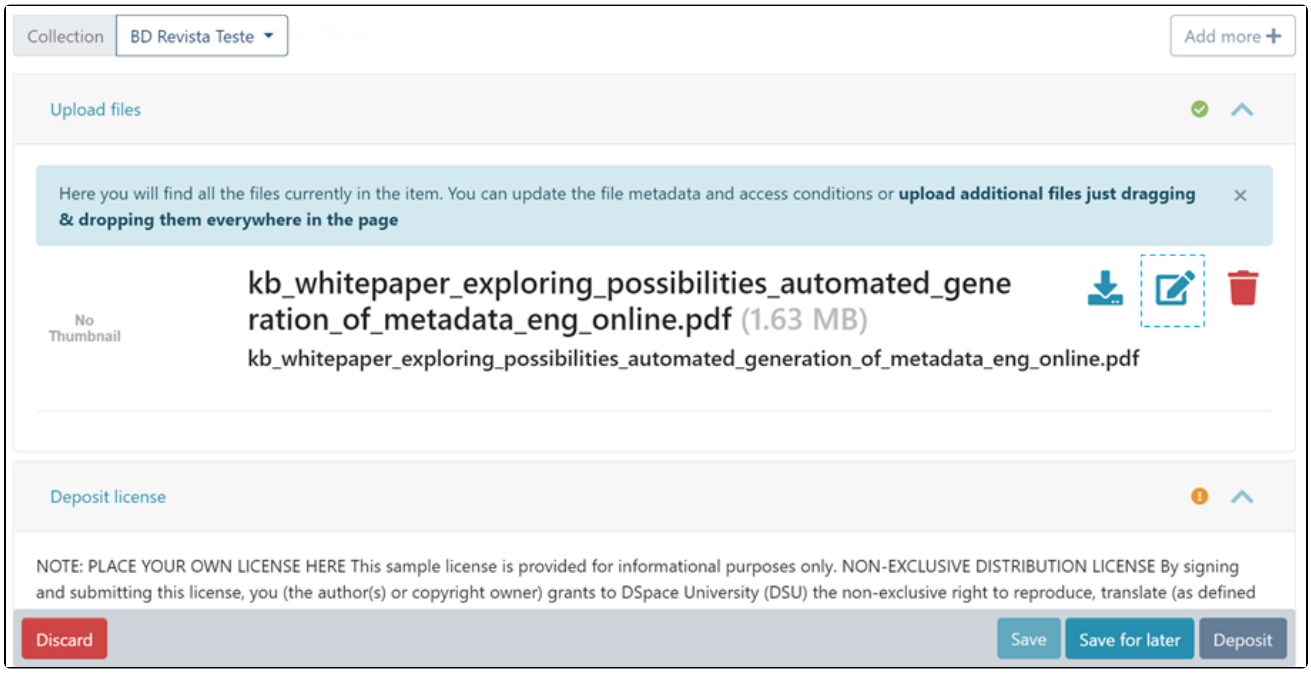
Bitstream upload in progress

Bitstream Upload Successful

Step 6: After uploading bitstream, the next step is to describe the Item by adding metadata.

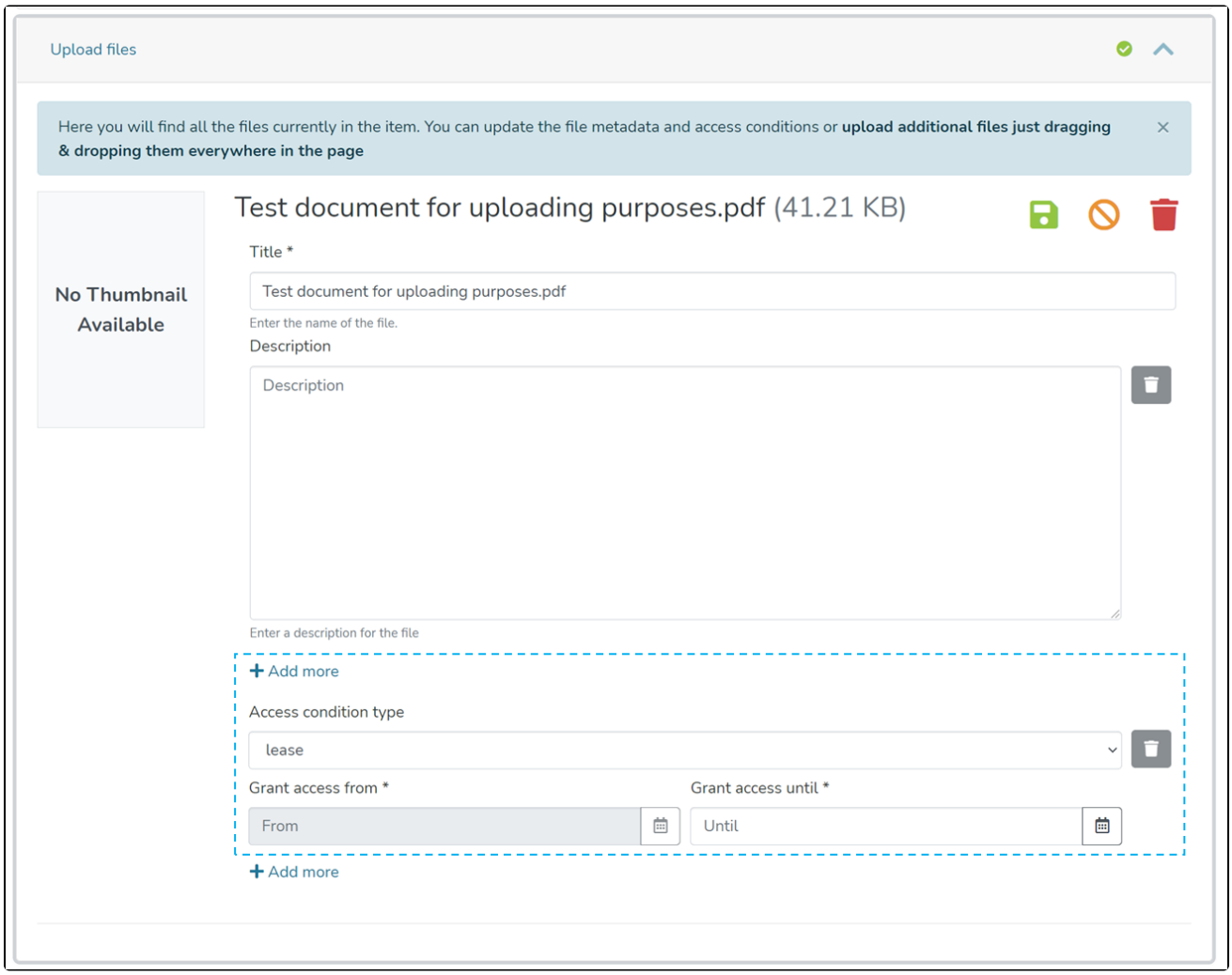
Please refer **Add Item** process for detailed documentation on populating information in the metadata fields.

Step 7: Click on the “edit” button against any attachment to lease it.



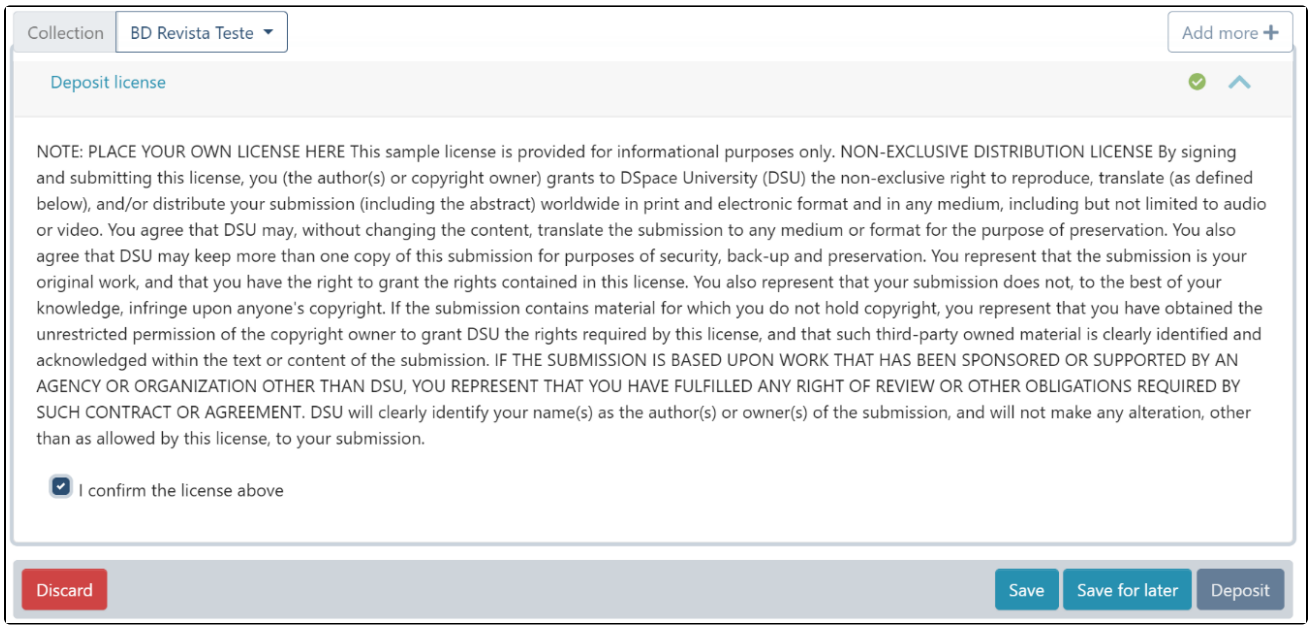
Step 8: Users can apply multiple policies on an attachment from available options. Click on the dropdown list under the “Access condition” and select lease, as highlighted in the screenshot below.

After selecting the lease, the “Grant access until” date field will be activated. Next, choose the future date, after which the attachment should be restricted to the larger set of DSpace users.



Users can add multiple policies to the attachment by clicking the “Add more” link. For example, a user can define a lease on an item until a future date. And after that date, another policy can be defined for the next period.

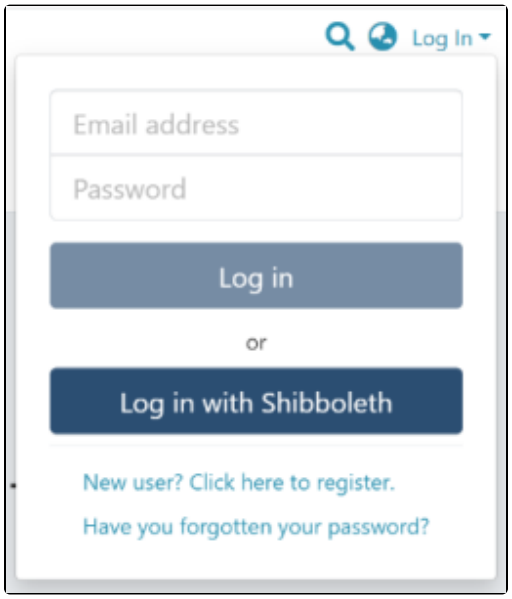
Step 9: After updating all information, the submitter clicks on the “I confirm the license above” checkbox to accept the repository’s license.



Step 10: Click on the “Deposit” button to submit the Item in DSpace. Users will get to see a confirmation prompt upon successful submission of the Item.

Lease an item via edit item

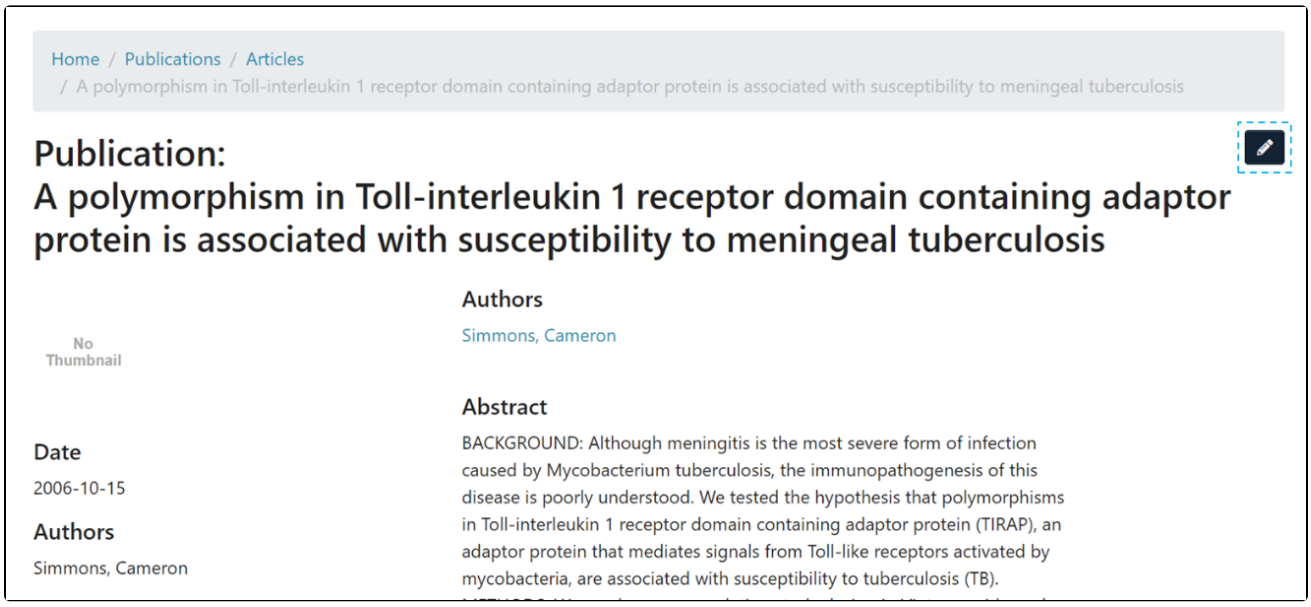
Step 1: Login using your credentials



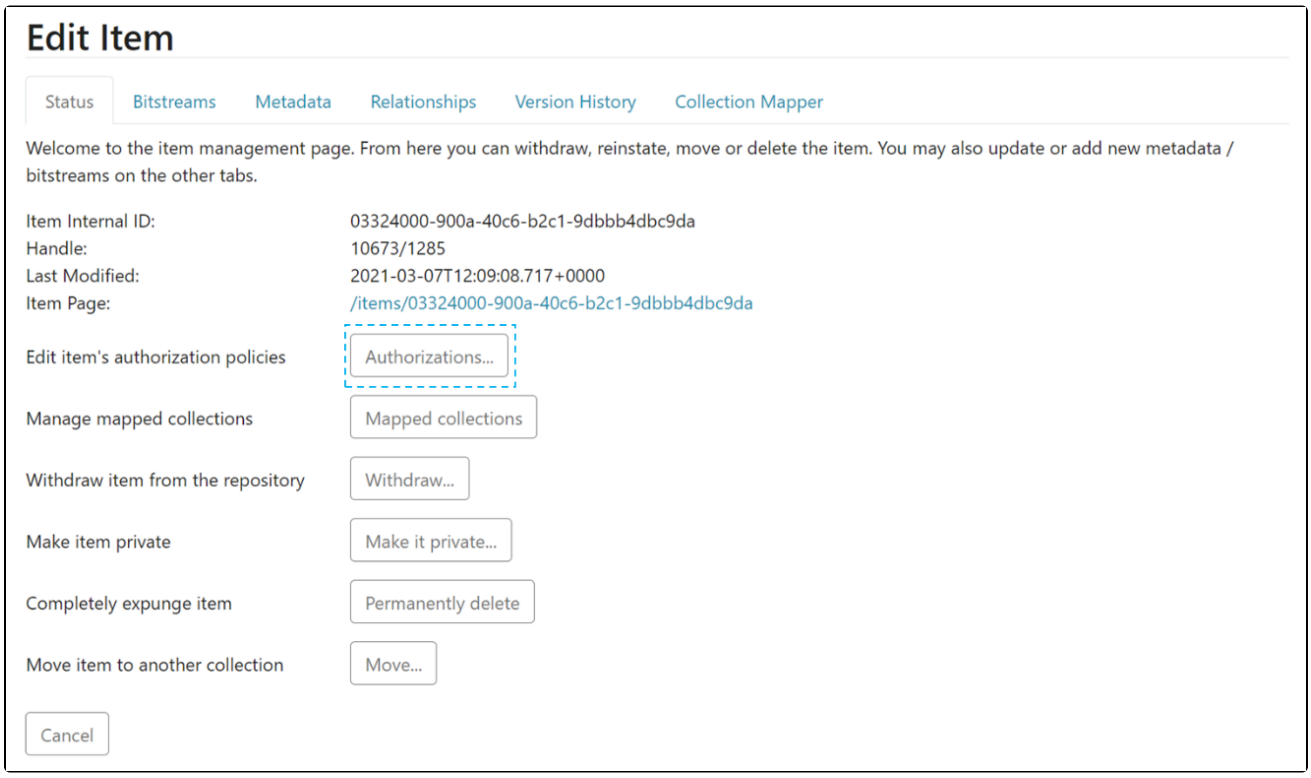
Step 2: Go to the Item you want to edit

Users can reach an item through multiple methods, as listed below:

1. Search an item
2. Browse communities and collections
3. Finding an item in the Administration section at Edit > Item
4. Click the “Edit” button on the right side of the item title.



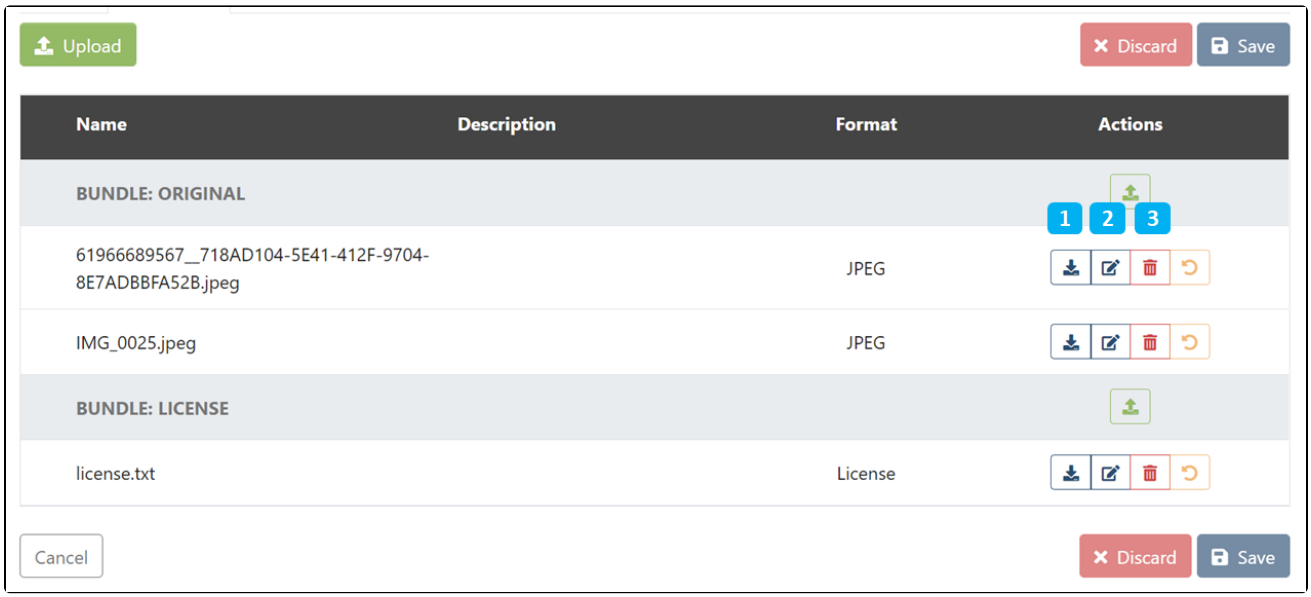
Step 3: Click on the “Authorizations” button under the “Status” tab to continue with leasing attachment(s).



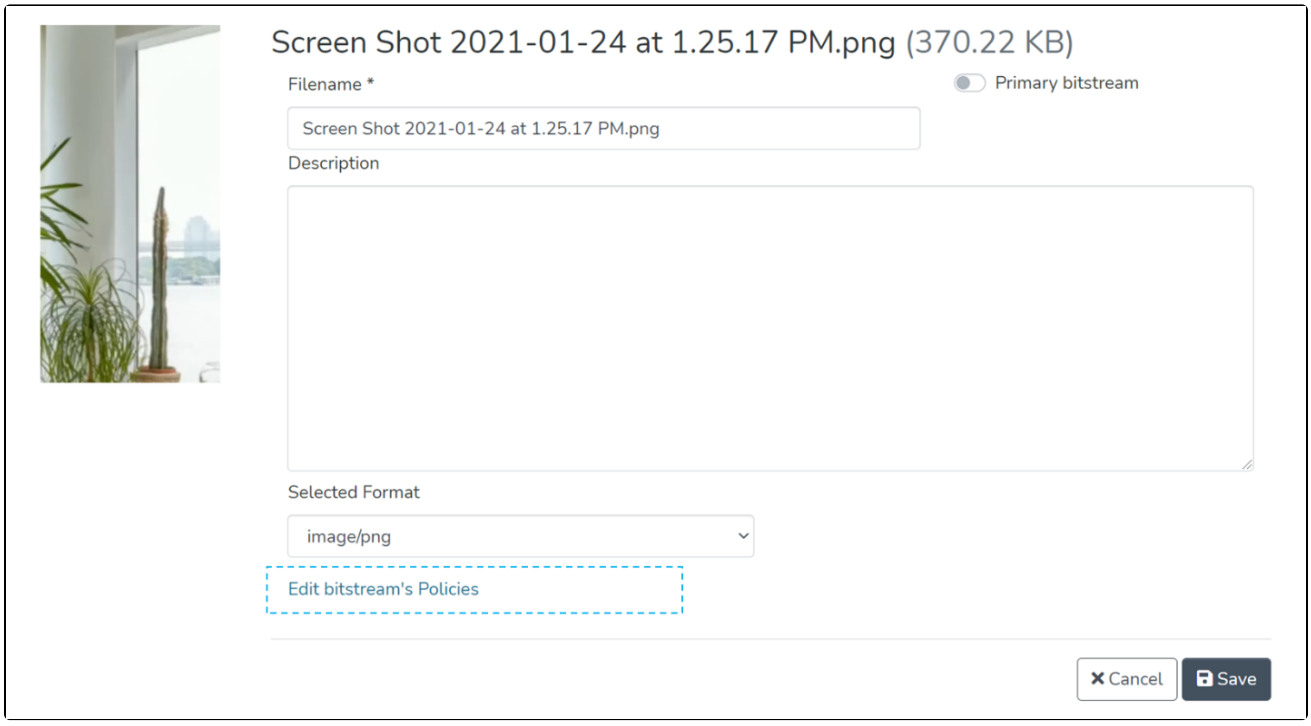
Step 4: The user will see multiple options against each attachment as explained below:

1. **Download Bitstream:** Click on this button to download the attachment on your local device for view.
2. **Edit Bitstream:** Click on the “Edit bitstream” button for editing details. Explained in the next step
3. **Delete Bitstream:** Click on the “Delete Bitstream” button to delete bitstream from the bundle.

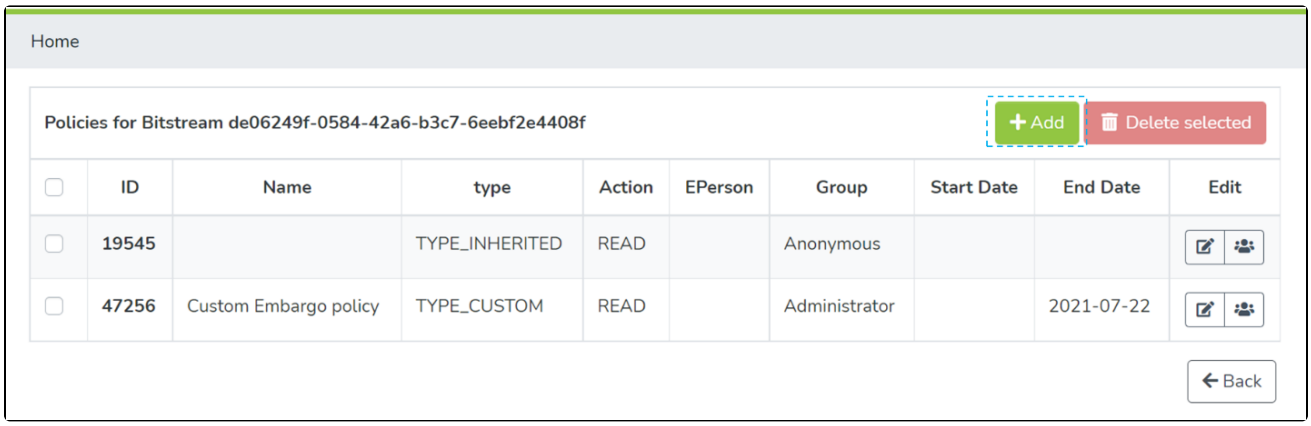
Click on the “edit bitstream” button to lease the attachment.



Step 5: Click on “Edit bitstream’s Policies” to continue the leasing process.

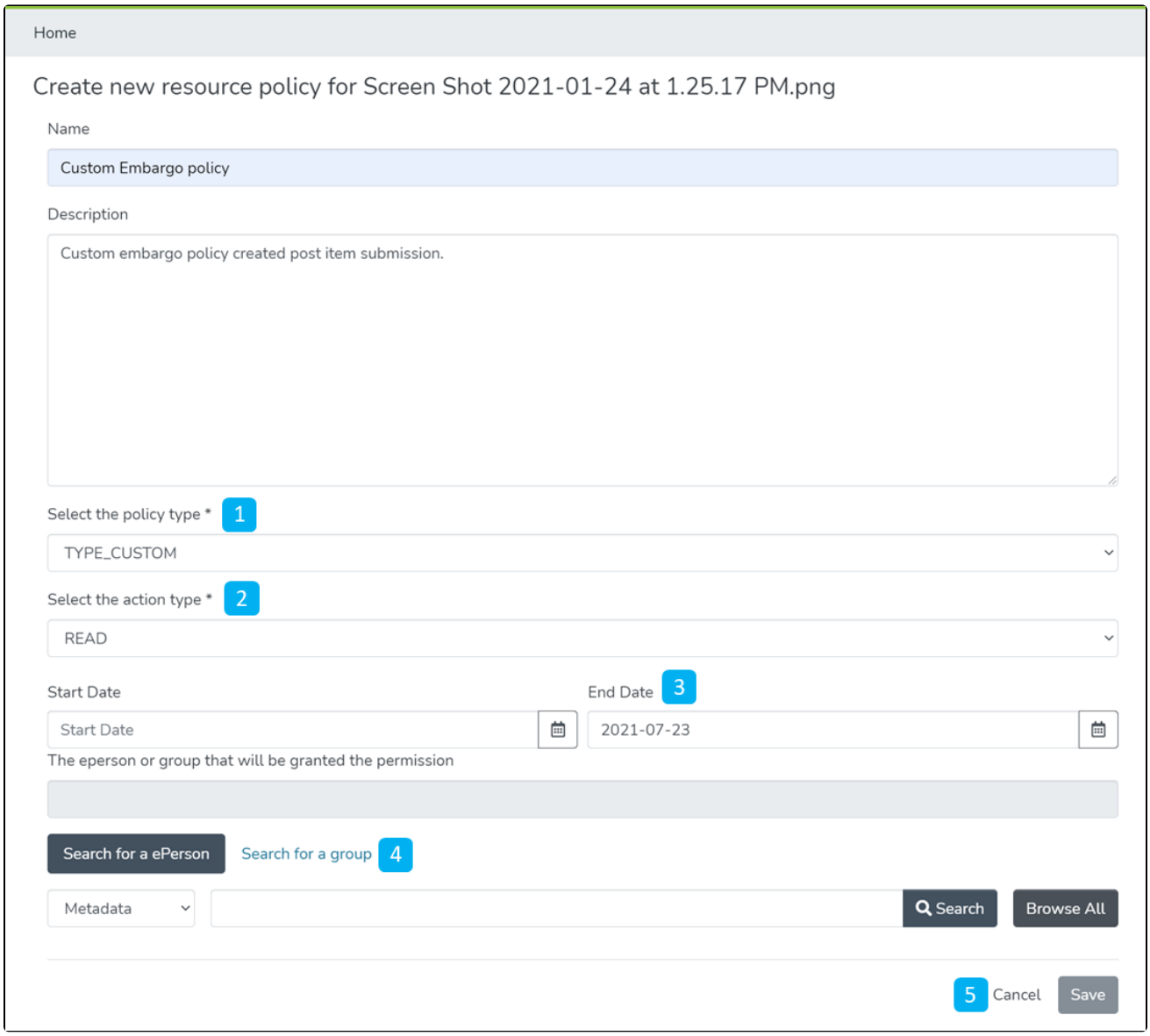


Step 6: Click on the “Add” button to create the custom lease policy for the attachment.



Step 7: Enter details for creating the lease policy on this form and perform the following actions:

1. Select ‘TYPE_CUSTOM’ under ‘Select the policy type’ dropdown menu
2. Select ‘READ’ under the ‘Select the action type’ dropdown list.
3. Select a future date under the End Date to determine when the embargo will end.
4. Select E-Person or a User group
5. Click on Save to create the policy



Detailed documentation on various possibilities on this form is available under ‘Edit Bitstream’ user documentation.

Users will see a success prompt upon creating the policy and be redirected to the bitstream policy page.

DSpace 7 Demo Quick Start

Front end

https://demo.dspace.org

OAI-PMH

https://demo.dspace.org/server/oai/request?verb=Identify

REST-API

https://demo.dspace.org/server/

SWORD v1

https://demo.dspace.org/server/sword/servicedocument

SWORD v2

https://demo.dspace.org/server/swordv2/servicedocument

Media viewer (video and image), IIIF Mirador player are disabled on Demo

Creative Commons licenses are not enabled on Demo

DSpace 6 Demo (for comparison)

Front end

https://demo6.dspace.org/

https://demo6.dspace.org/xmlui/

OAI-PMH

https://demo6.dspace.org/oai/request?verb=Identify

REST-API

https://demo6.dspace.org/rest

SWORD v1

https://demo6.dspace.org/xmlui/sword/servicedocument

SWORD v2

https://demo6.dspace.org/xmlui/swordv2/servicedocument

Browse thumbnails are disabled

Management sidebar

Many of the administrative functions can be accessed from the Management sidebar. This list maps the menu to more detailed information.

- New
  - Community
  - Collection
  - Add item
  - Process
- Edit
  - Community
  - Collection
  - Item
- Import
  - Metadata
- Export
  - Metadata
- Access Control
  - People
  - Groups
- Admin Search
- Registries
  - Metadata
  - Format
- Curation Task
- Processes
- Administer Workflow
- Health

New

Quickly create or edit objects from anywhere in the system. Either browse to the object first, or search for it using the Admin sidebar. - Release Notes

Community

Collection

Add item

Process

Processes UI (video) allows Administrators to run backend scripts/processes while monitoring their progress & completion. - Release Notes

See Command Line Operations for more detail about these commands.

Edit

- Quickly create or edit objects from anywhere in the system. Either browse to the object first, or search for it using the Admin sidebar.
- Bitstream Editing (video) has a drag-and-drop interface for re-ordering bitstreams and makes adding and editing bitstreams more intuitive.
- Metadata Editing (video) introduces suggest-as-you-type for field name selection of new metadata. - Release Notes

Community

- Edit
- Delete

Collection

- Edit
- Delete

Item

- Edit
- Delete

Import

You can drop or browse CSV files that contain batch metadata operations on files. When "Validate Only" selected, the uploaded CSV will be validated. You will receive a report of detected changes, but no changes will be saved.

Metadata

Export

Metadata

Access Control

Login As (Impersonate) another account allows Administrators to debug issues that a specific user is seeing, or do some work on behalf of that user. (Login as an Admin, Click "Access Control" in sidebar, Click "People". Search for the user account & edit it. Click the "Impersonate EPerson" button. You will be authenticated as that user until you click "Stop Impersonating EPerson" in the upper right.) - Release Notes

People

Groups

Admin Search

Administrative Search (video) combines retrieval of withdrawn items and private items, together with a series of quick action buttons. - Release Notes

Registries

Metadata

Format

Curation Task

Processes

Processes UI (video) allows Administrators to run backend scripts/processes while monitoring their progress & completion. - Release Notes

Details about each of the available processes/scripts can be found in the "scripts" directory of the REST API docs: https://github.com/DSpace/RestContract/blob/main/script

Additional information can also be found in the Command Line Operations documentation.

Administer Workflow

Administer Active Workflows (video) allows Administrators to see every submission that is currently in the workflow approval process. - Release Notes

Health

Admin "Health" menu provides basic control panel functionality *(based on 6.x Control Panel).* When logged in as an Administrator, select "Health" from the side menu. You'll see a "Status" tab which provides useful information about the status of the DSpace backend, and an "Info" tab which provides an overview of backend configurations and Java information. - Release Notes

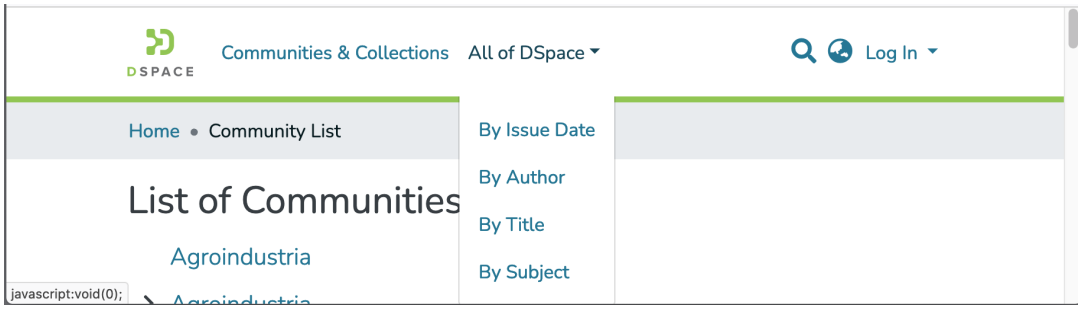
Menus

- Communities & Collections
- All of DSpace
  - By Issue Date
  - By Author
  - By Title
  - By Subject
- Statistics
- Search box
- Language
- Log In
  - Profile
  - MyDSpace

Communities & Collections

This is an alphabetical and hierarchical list of communities in the repository. Communities may contain sub-communities and collections. Use the left chevons to expand the hierarchy.

All of DSpace



Browse by Issue Date, Author, Title, or Subject. Use the gear icon to set the sorting and number of results per page.

By Issue Date

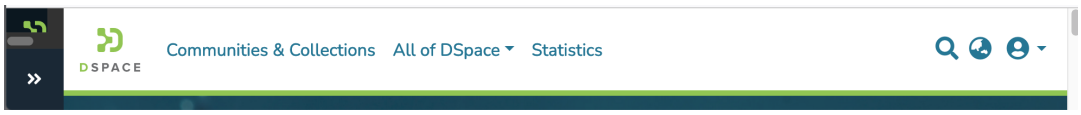
By Author

By Title

By Subject

Statistics

(Once logged in with correct access)



On the Home page, this lists the 10 items with most visits to the item page. (This does not count the downloads of the files contained in the item.)

On a Community page, this lists the total cumulative visits to the community's main page (not all the collections and items that community contains), visits to the community main page broken out by month for the past 7 months, and by top cumulative country and city views.

On a Collection page, this lists the total cumulative visits to the collection's main page (not all the items that collection contains), visits to the collection page broken out by month for the past 7 months, and by top cumulative country and city views.

On an Item page, this lists the total cumulative visits to the item page, (This does not count the downloads of the files contained in the item.), visits to that page broken out by month for the past 7 months, and by top cumulative country and city views.

Search box

Use this search box to retrieve search results that can be further refined by search filters, such as date, subject, author, and item type.

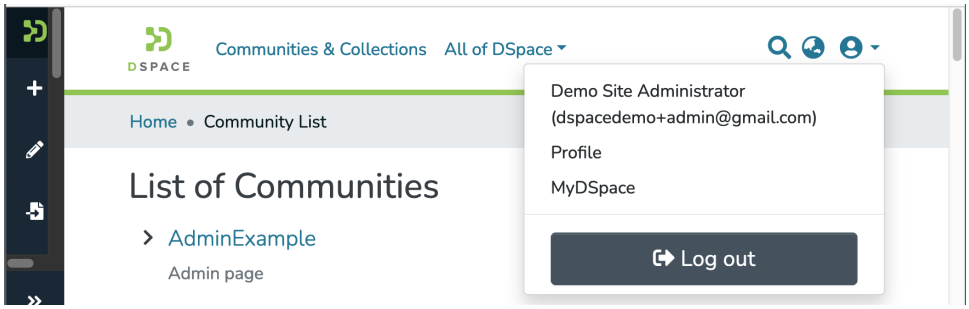
See Search - Advanced for tips for creating Boolean searches.

Language

Choose your preferred supported language for the DSpace system. Repository files and metadata will be in their source language.

Log In

(Once logged in)



Profile

If enabled in the repository, one can set up a Researcher Profile.

One can reset their password and view your group memberships.

MyDSpace

View or edit your submissions:

- Items that you are preparing (Status=Workspace),
- Items that are in the approval process (Status=Workflow),
- Items that have been approved and completed (Status=Archived).

Registry management

Info for repository managers, covering topics such as metadata registry management and format registry management.

- Metadata Registry Management

Metadata Registry Management

The metadata registry maintains a list of various metadata schemas in DSpace. These metadata schemas consist of different metadata elements. However, DSpace requires the qualified Dublin Core schema.

- Audience
- Create Metadata Registry
- Metadata Registry Management
- Delete Metadata Schema

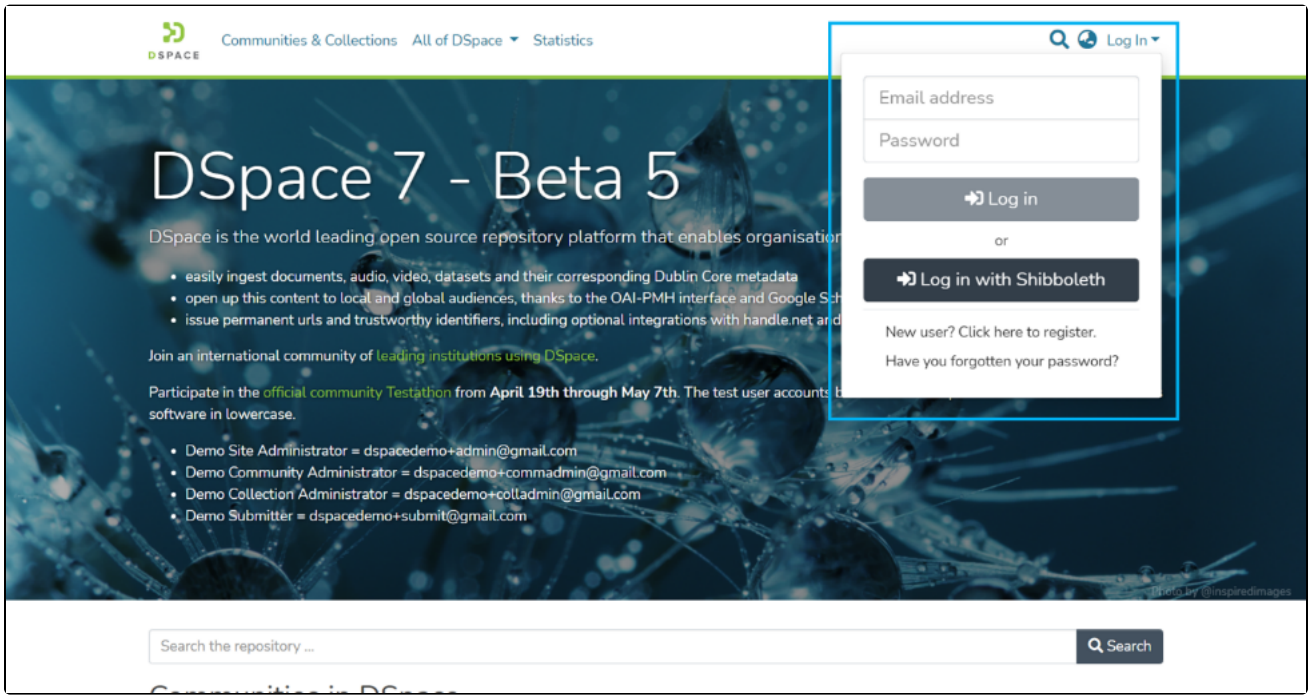
Audience

1. Repository Administrator
2. Community Administrator

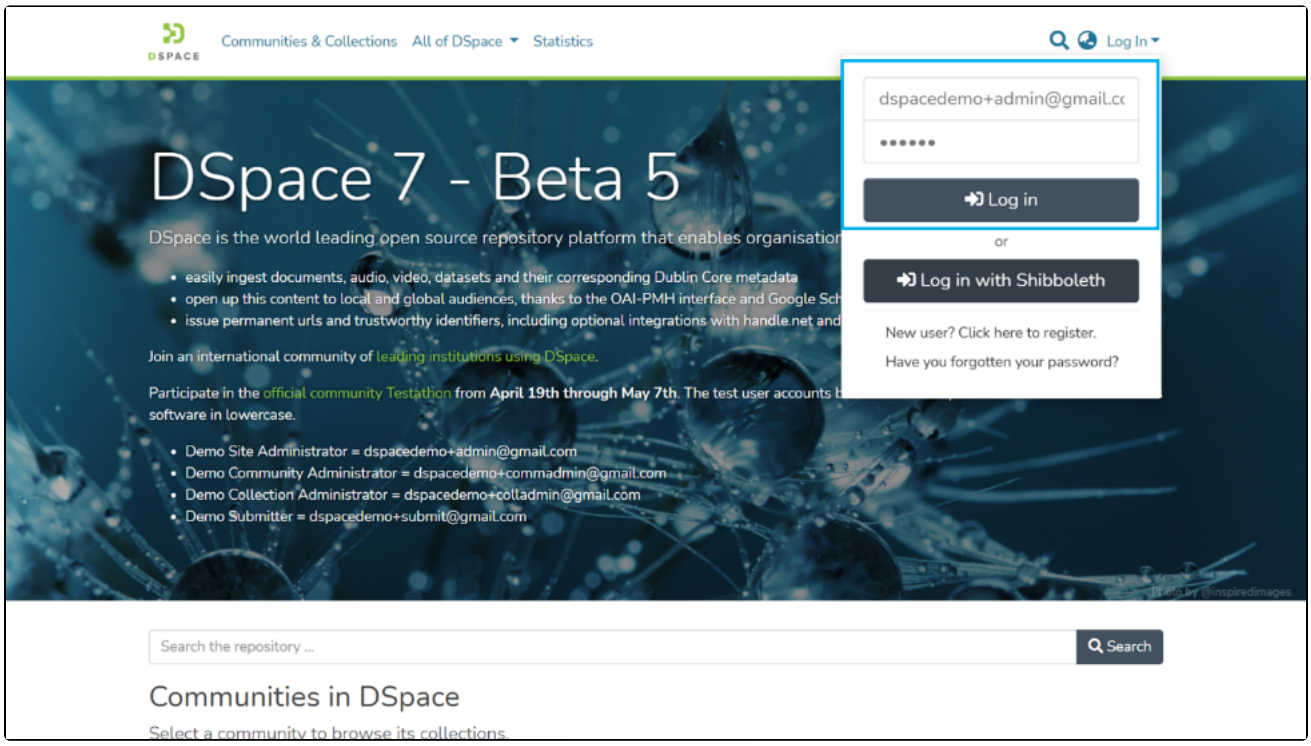
Create Metadata Registry

This process is for System and Community Administrator user profiles.

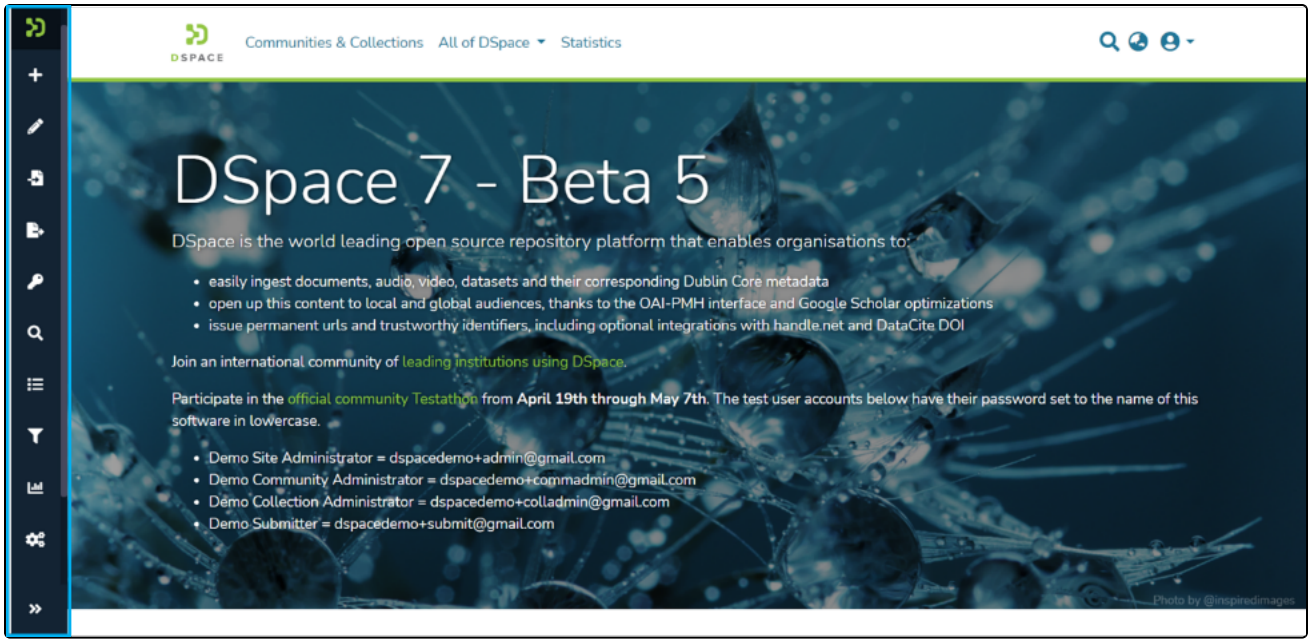
Step 1: Click on the “Log In” link appearing at the top right corner of the DSpace home page, and the pop-up will open, as illustrated in the below screen.



Step 2: Enter your user id and password and click on the login button for logging in to DSpace.

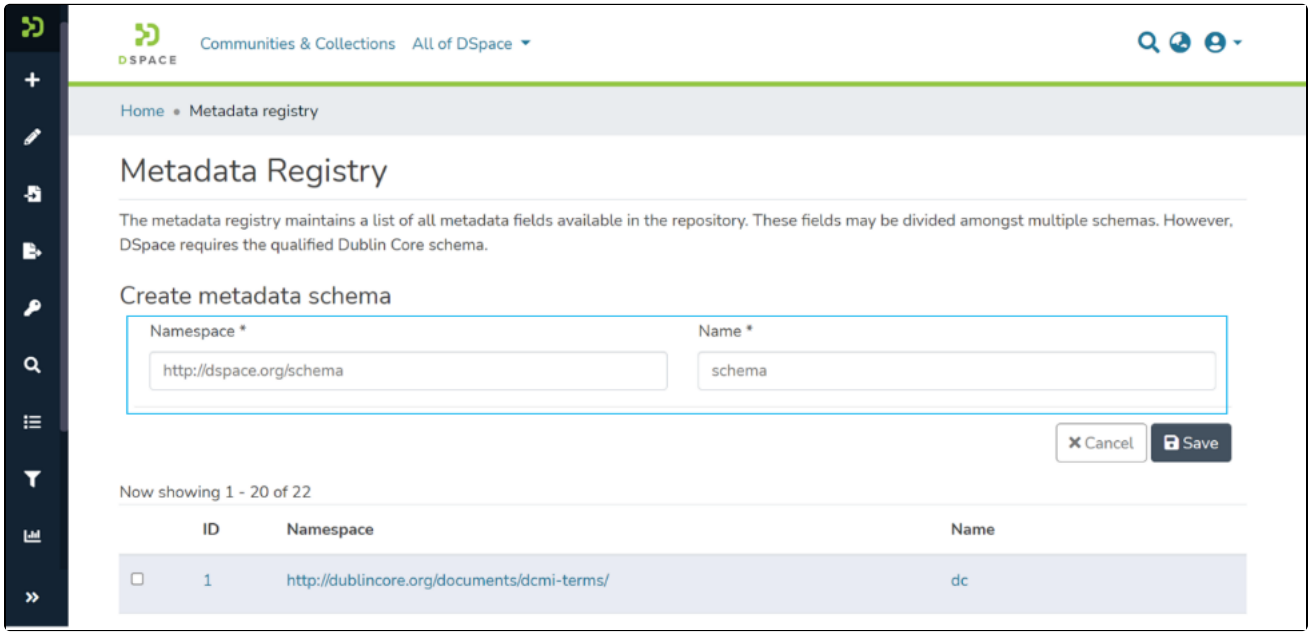


Step 3: Users with administrative rights will see the admin menu on the left-hand side of the screen, as shown in the below illustration.

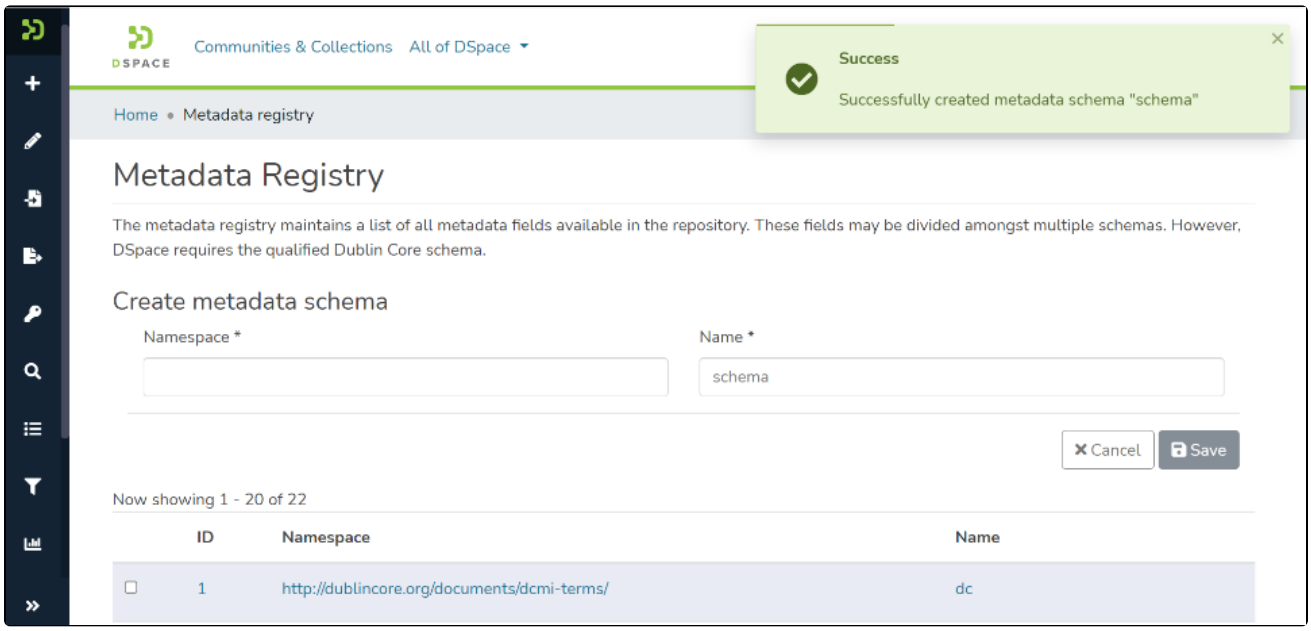


Step 4: Roll over your cursor over the administration menu and click on Registries. Click on Metadata to go to the Metadata Registries.

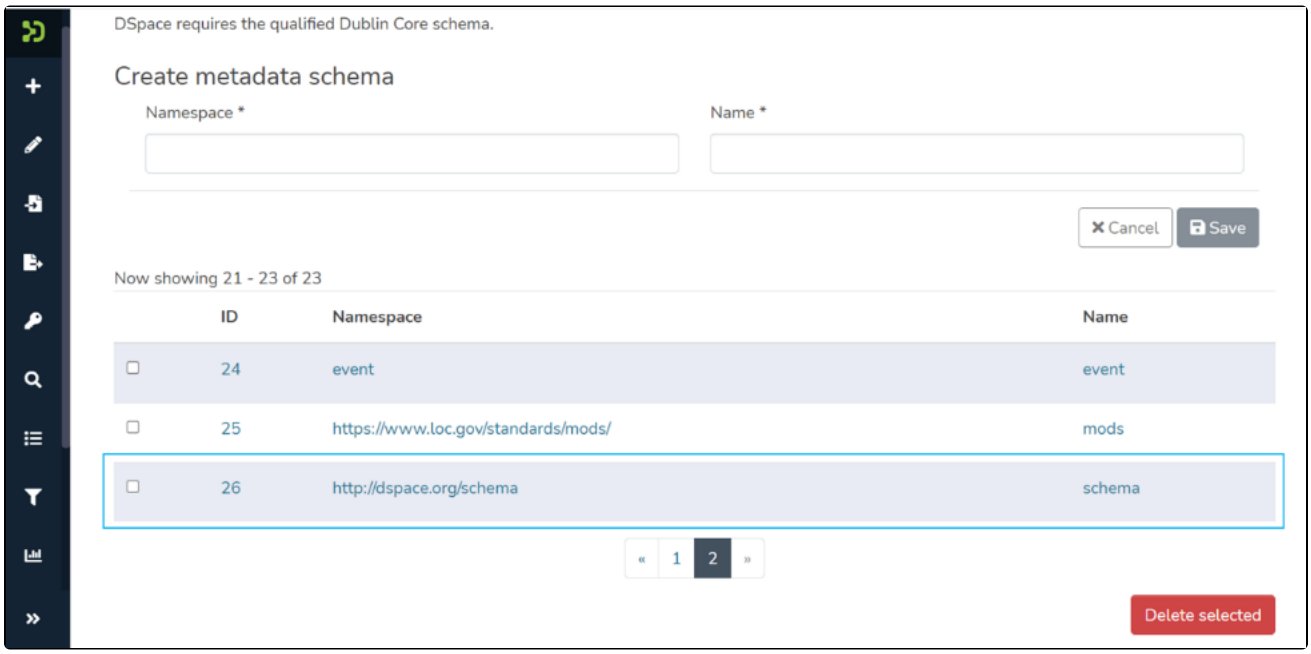
Step 5: Enter the value of your choice in the ‘Namespace’ field and the short value in the ‘Name’ field.



Step 6: Click on the Save button to create a Metadata schema. A success prompt will appear upon metadata schema creation in the DSpace.

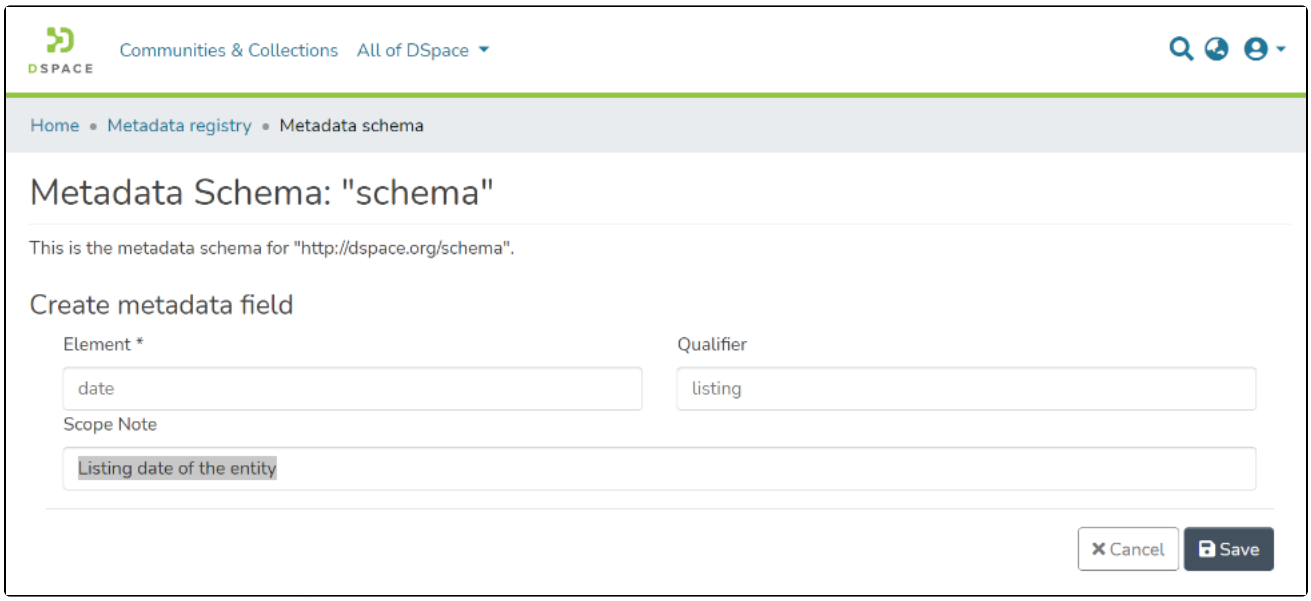


Step 7: Scroll down the list to see the schema added. Click on the namespace or name value to access the metadata schema.

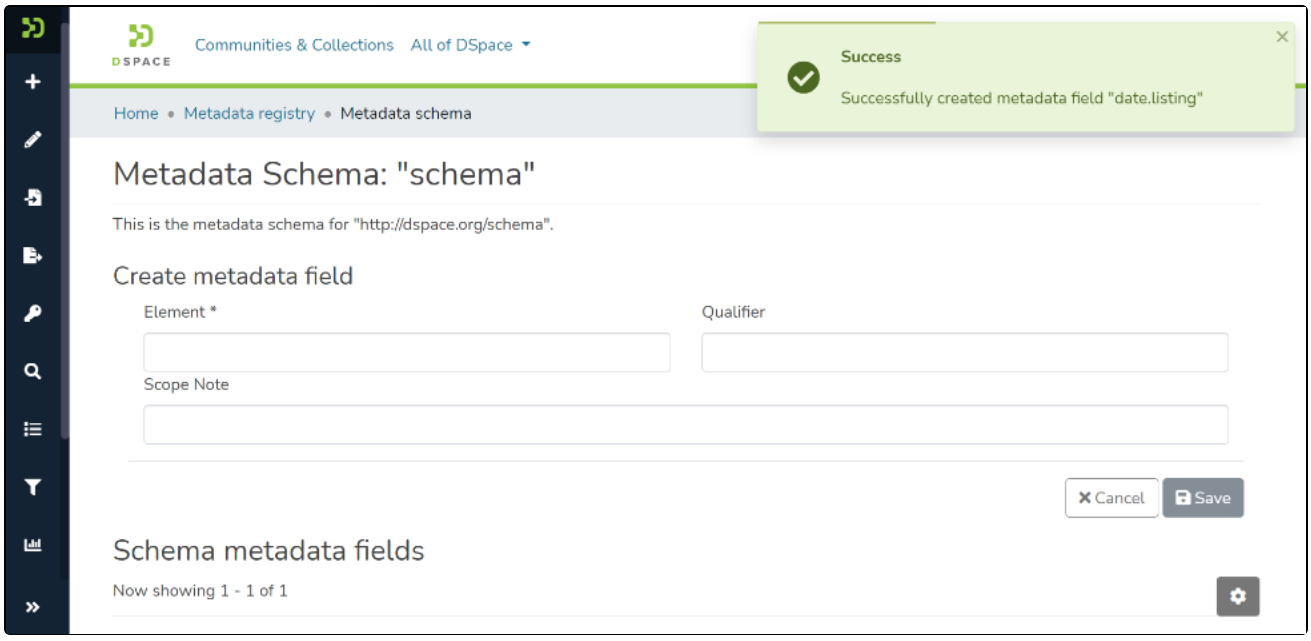


Step 8: Enter desired values in ‘Element,’ ‘Qualifier,’ and ‘scope note.’

1. Element is the primary value in the metadata schema. For example, suppose the date is required to be added as a metadata element. In that case, the element’s value can be the date.
2. Qualifier – Use the Qualifier to segregate the metadata element further. For example, suppose you want to add multiple dates under an element. In that case, users can add various qualifiers for each date type. For example, there can be a content listing date, content de-listing date, and other types of dates.
3. Scope Note – Users can enter the definition of the metadata element created to benefit other users.



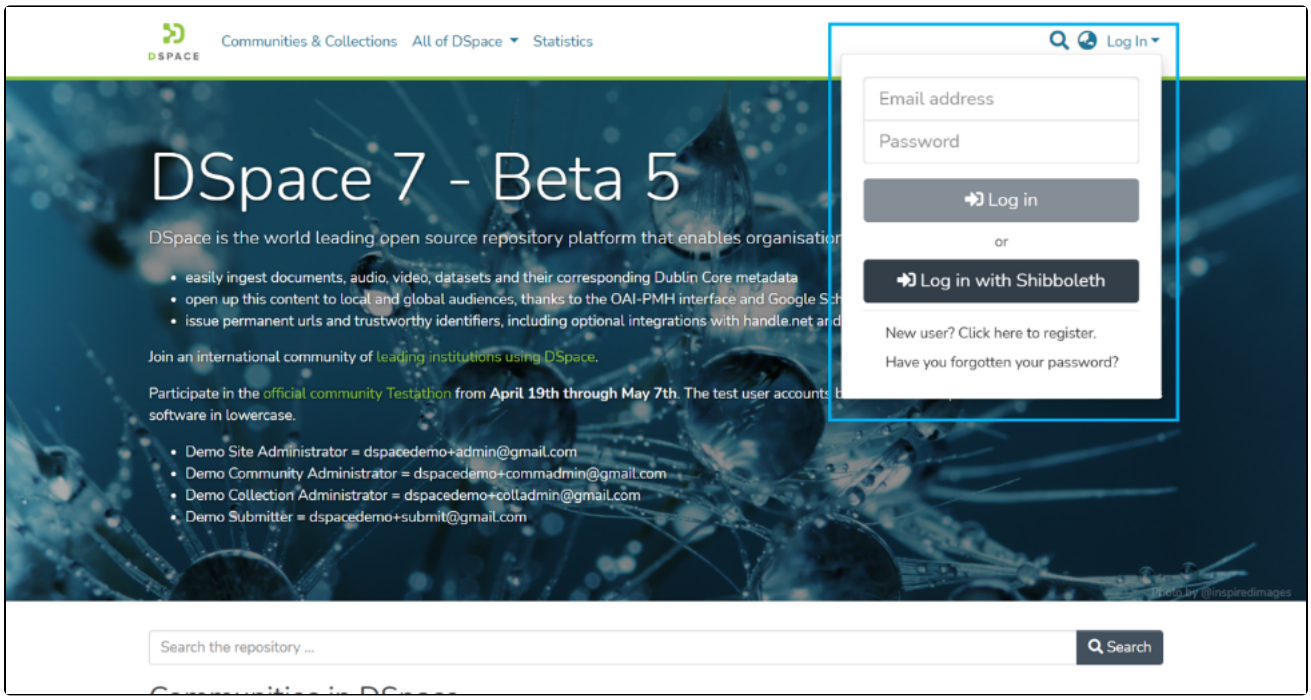
Step 9: Click on the Save button to add an element to the selected metadata schema. You will see a confirmation prompt upon the successful addition of the element.



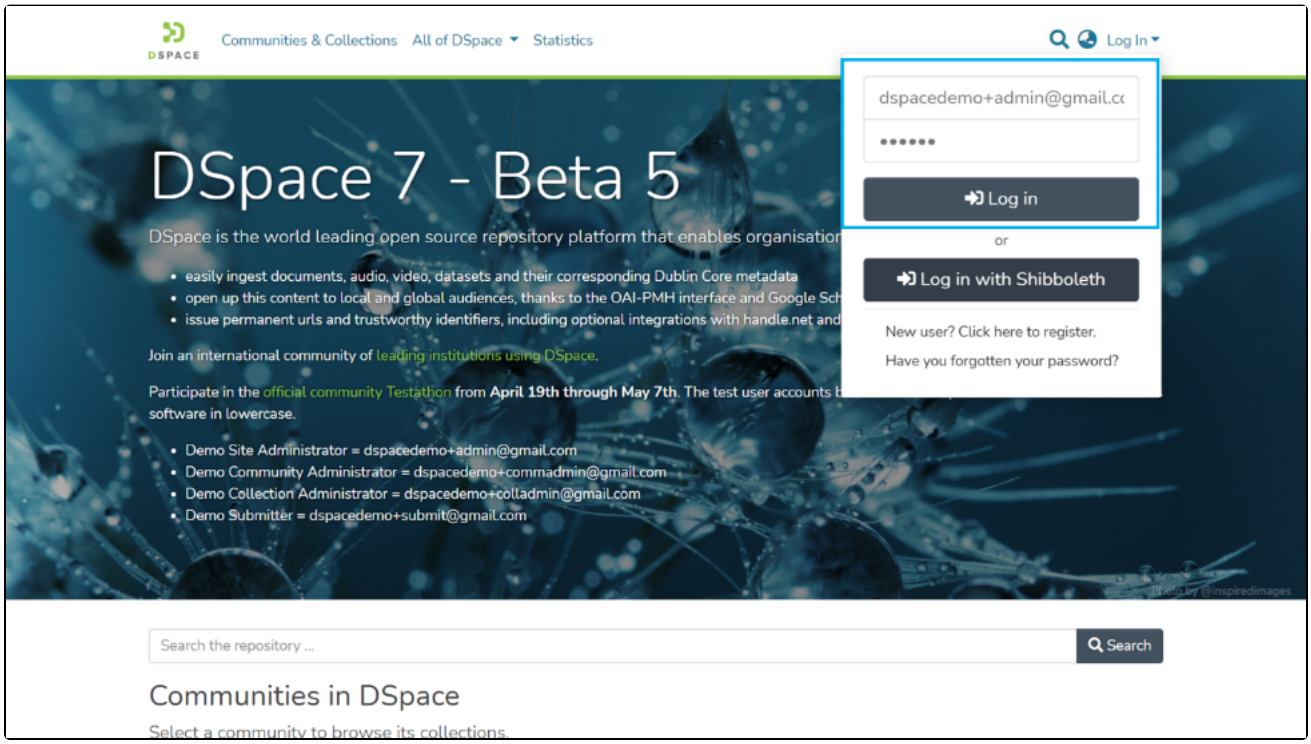
Metadata Registry Management

This process is for System and Community Administrator user profiles. Under metadata registry management, users can update or delete metadata elements or metadata registries.

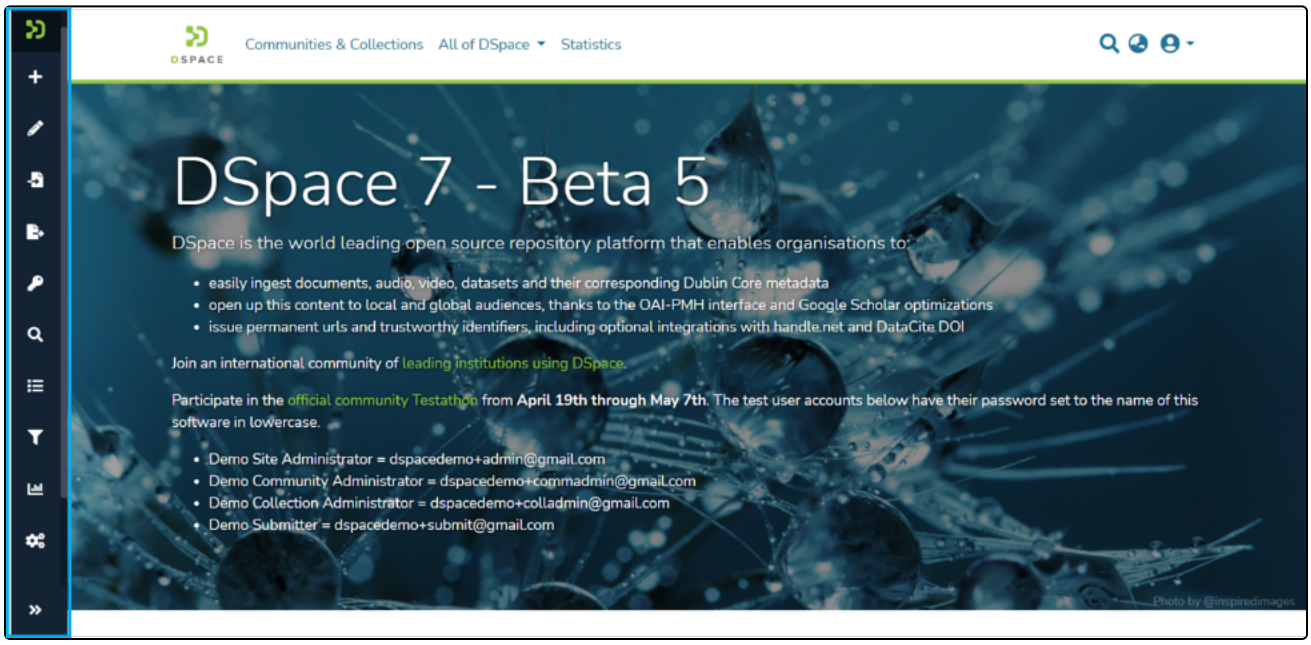
Step 1: Go to the home page of DSpace and click on the “Log In” link appearing at the top right corner of the screen, and the pop-up will open, as illustrated in the below screen.



Step 2: Enter your user id and password and click on the login button for logging in to DSpace.

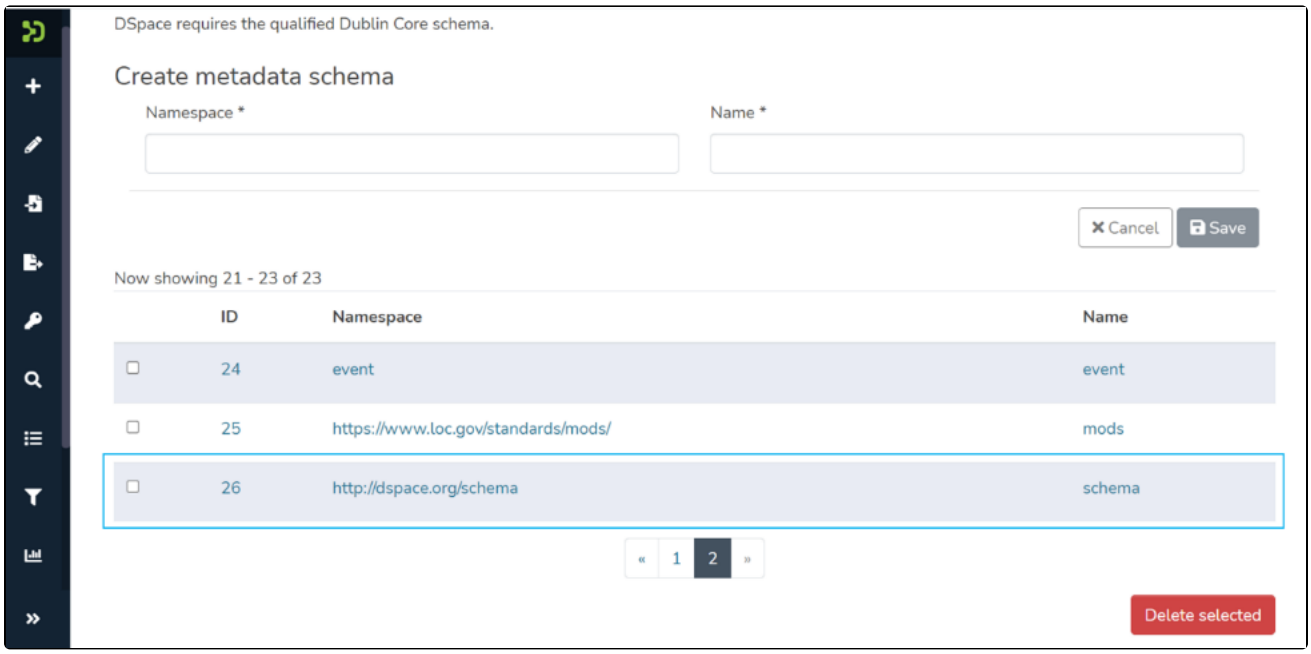


Step 3: Users with administrative rights will see the admin menu on the left-hand side of the screen, as shown in the below illustration.

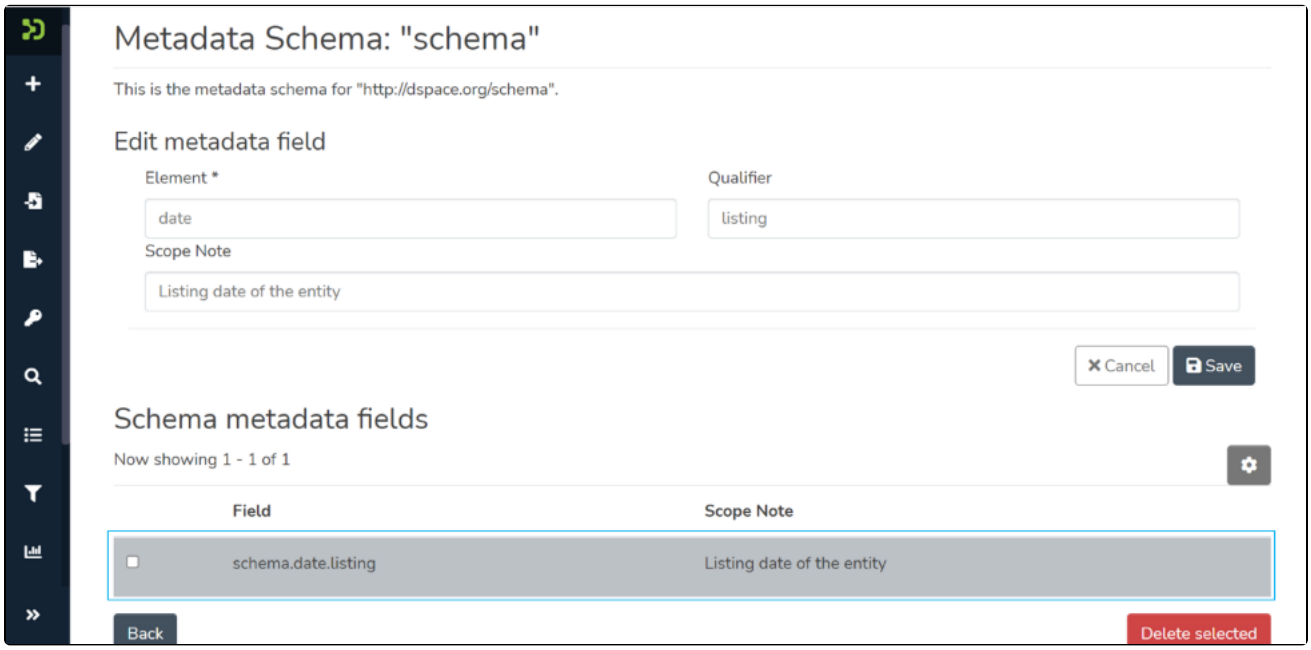


Step 4: Roll over your cursor over the administration menu and click on Registries. Click on Metadata to go to the Metadata Registries.

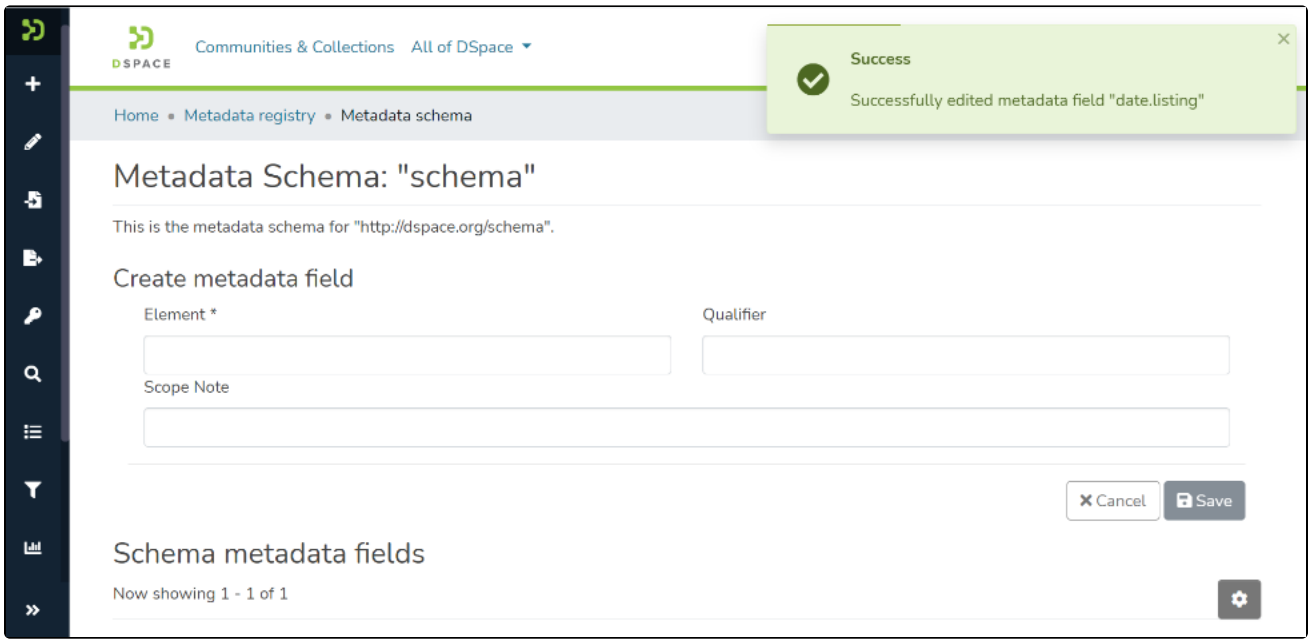
Step 5: Click on the namespace or name of the Metadata schema you want to edit.



Step 6: Click on the metadata element you want to edit. Upon clicking the target element, its values will be populated in the corresponding fields under the Edit Metadata fields section, as shown below.

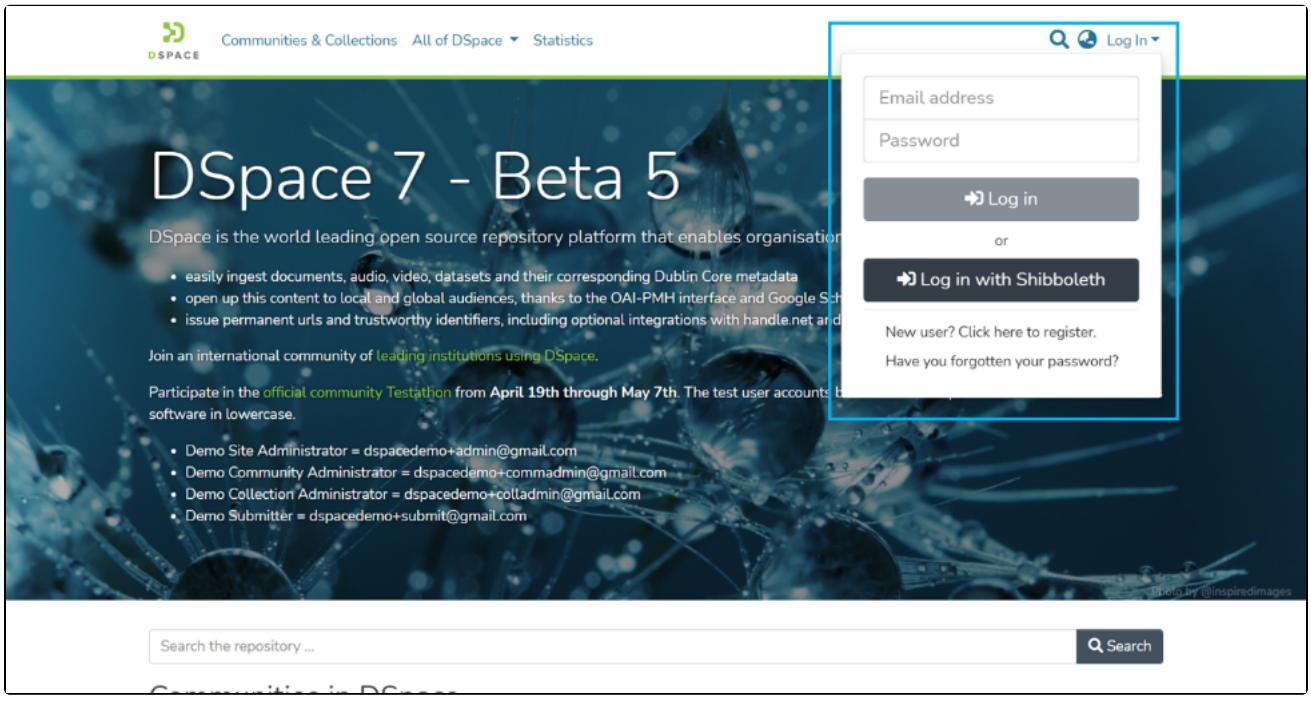


Step 7: Update the value under the target field(s) and click on the ‘Save’ button to save them. A success prompt will appear upon successful update. In addition, an updated metadata element will appear in the metadata schema.

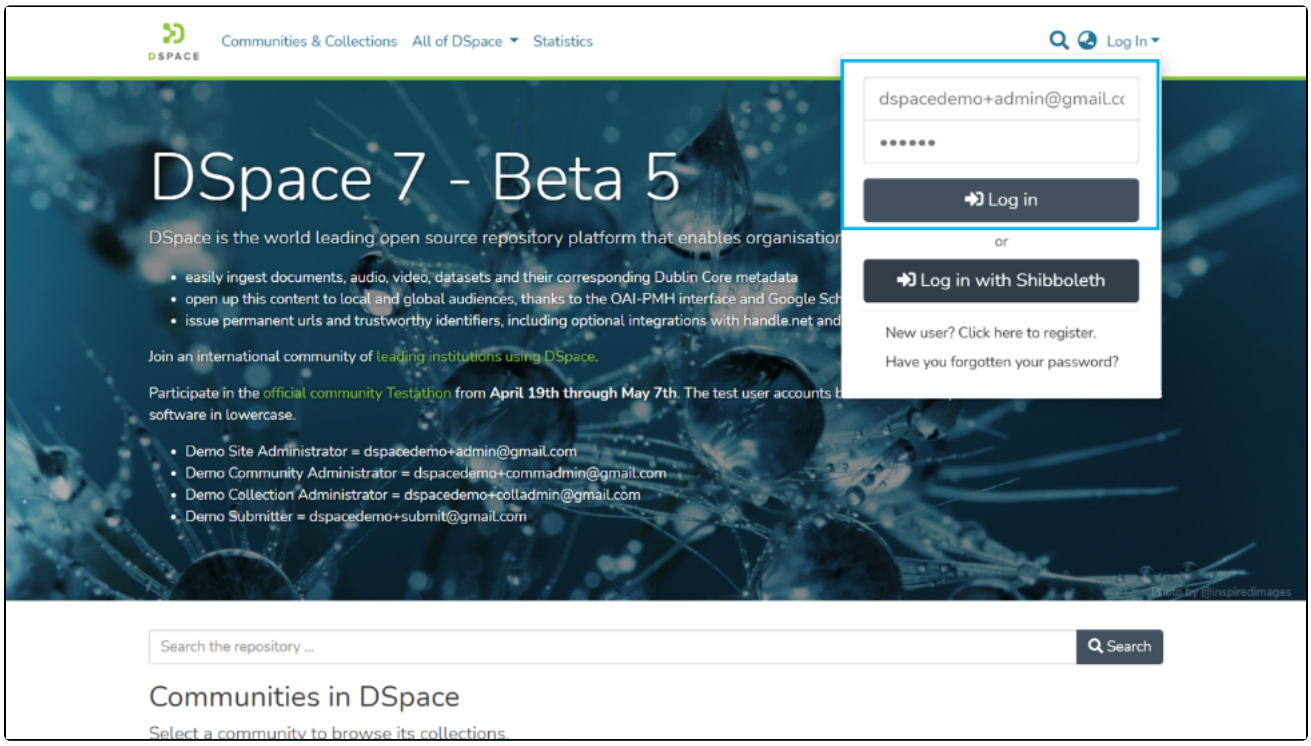


Delete Metadata Schema

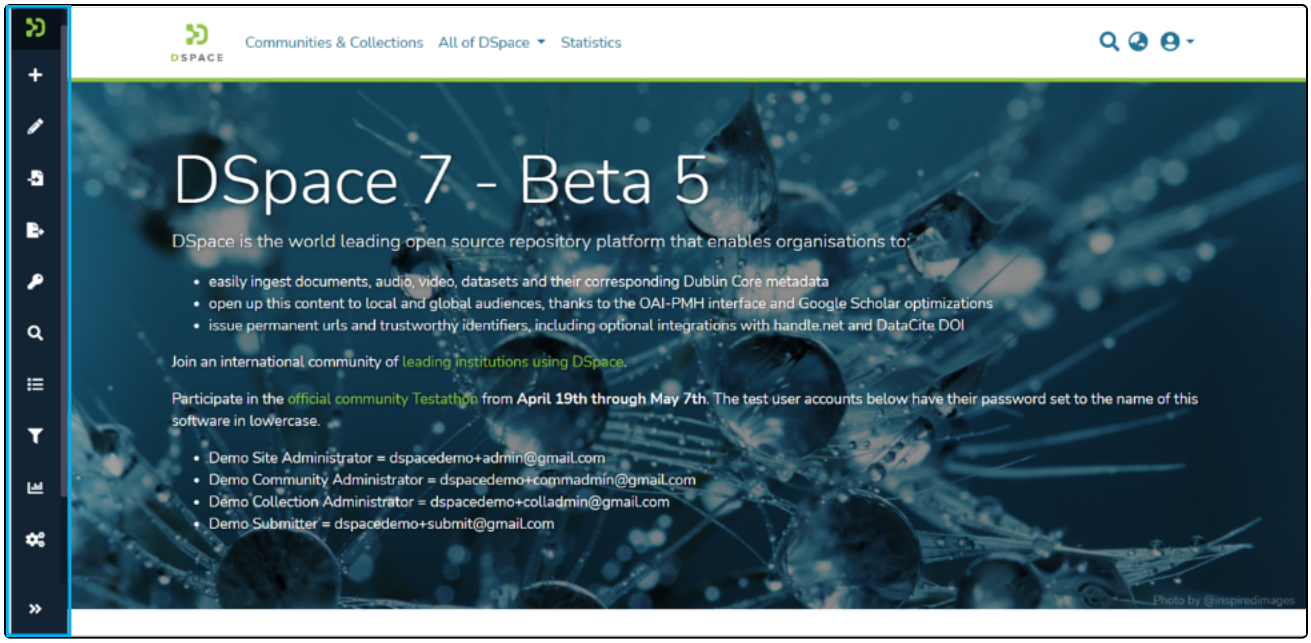
Step 1: Go to the home page of DSpace and click on the “Log In” link appearing at the top right corner of the screen, and the pop-up will open, as illustrated in the below screen.



Step 2: Enter your user id and password and click on the login button for logging in to DSpace.

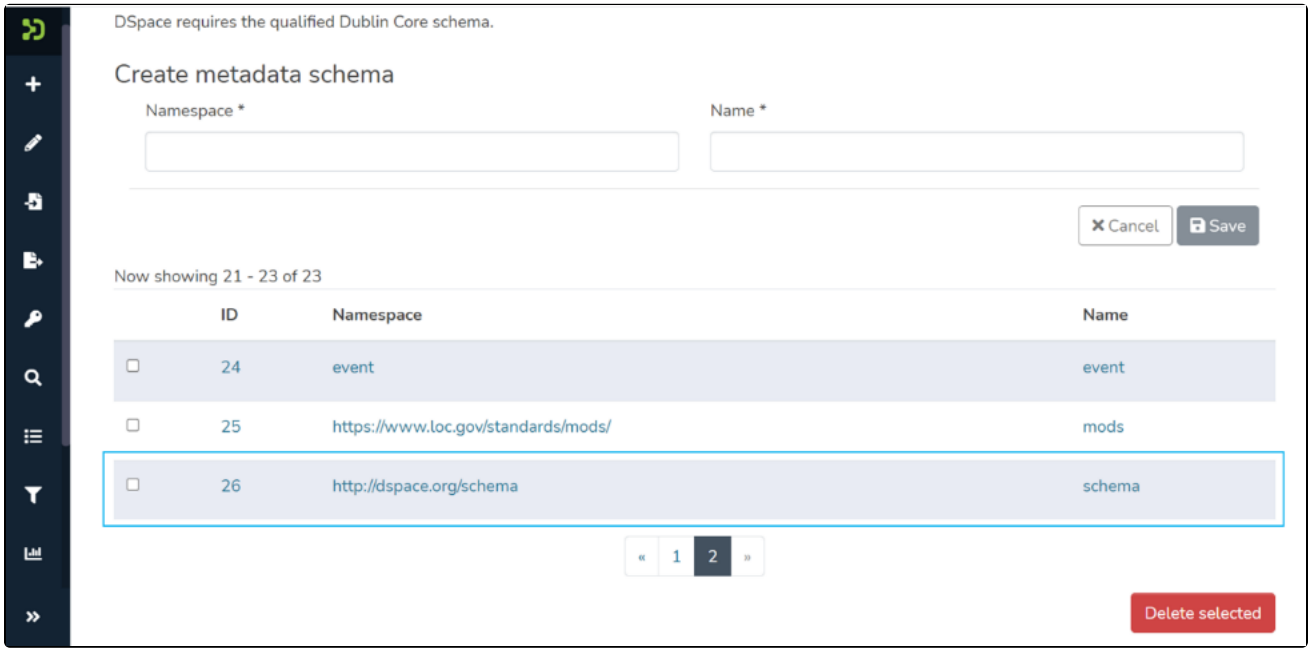


Step 3: Users with administrative rights will see the admin menu on the left-hand side of the screen, as shown in the below illustration.



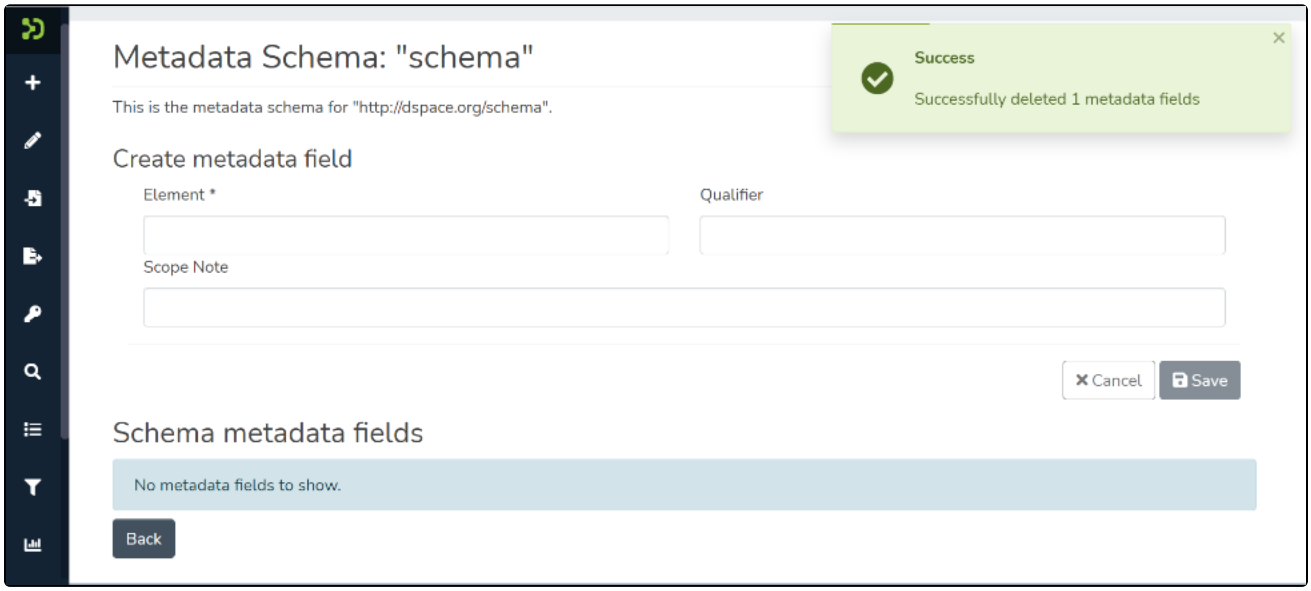
Step 4: Roll over your cursor over the administration menu and click on Registries. Click on Metadata to go to the Metadata Registries.

Step 5: Click on the namespace or name of the Metadata schema under which you want to delete an element.

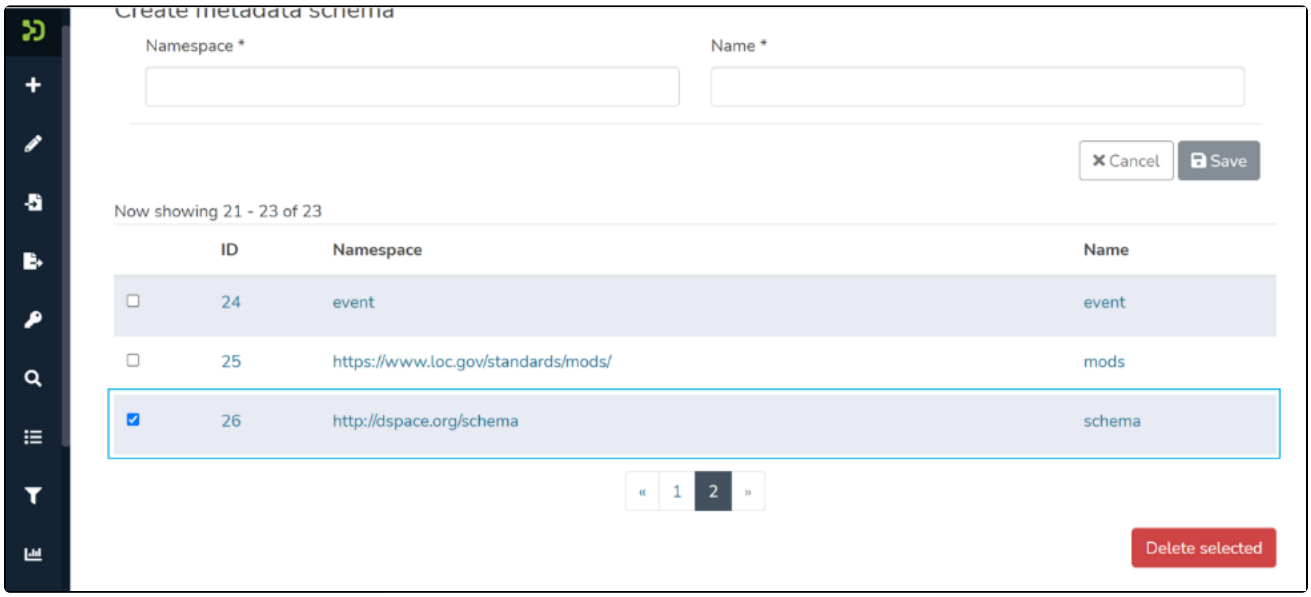


Step 6: Click on the checkbox appearing on the left of the metadata element you want to delete.

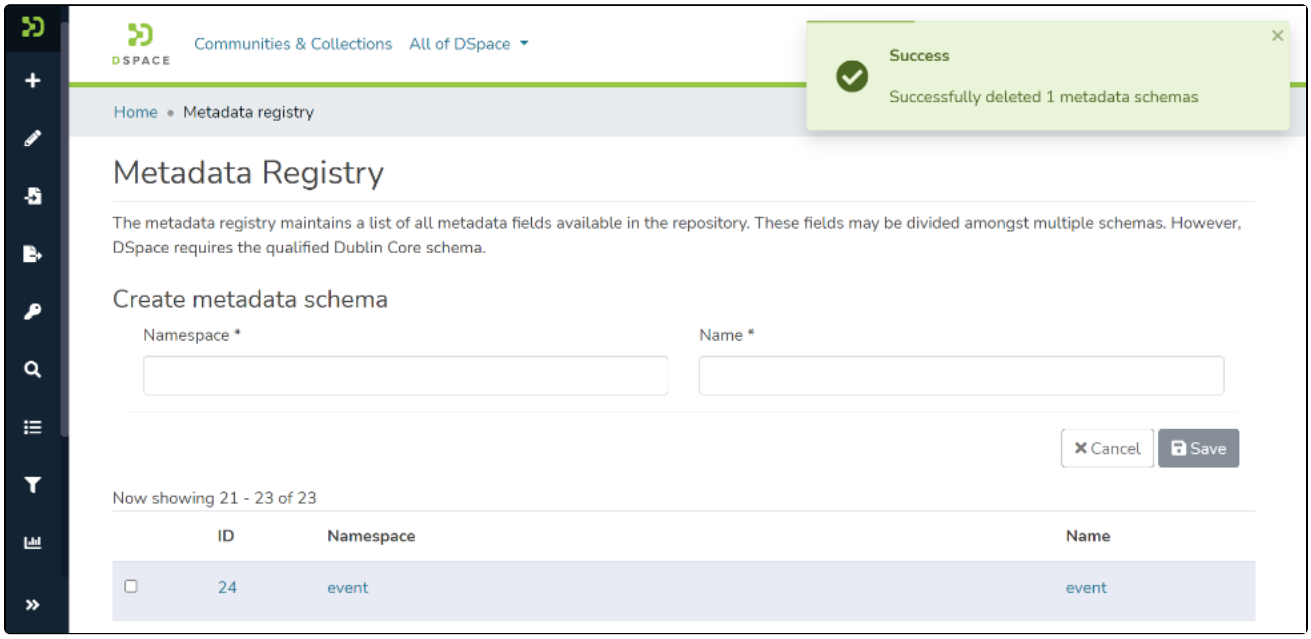
Step 7: Click on the ‘Delete selected’ button appearing in red at the bottom of the page. Upon successful deletion of the element, you will see a prompt confirming deletion of the field.



Step 8: To delete the entire metadata schema, click on the checkbox appearing on the left of the target metadata schema(s).



Step 9: Click on the ‘Delete selected’ button appearing in red at the bottom of the page. Upon successful deletion of the element, you will see a prompt confirming deletion of the field.



Request-a-copy

Scope

Covering the use of the request-a-copy feature which allows users to request bitstreams which are under embargo. This information does not cover how to activate or de-activate the feature.

Use Case

The repository manager wants to be able to advise their depositors on the existence of the feature, that they should be prepared to receive requests.

Audience

Repository managers, anonymous users, submitters

Feature description

If the file(s) (ie bitstreams) in a deposit are under embargo, they will not be available for users to download directly. However, users can send the depositor a request for the file(s) through DSpace, by double-clicking on a filename hyperlink and completing the form. DSpace sends the details of the request to the depositor by email. The depositor is offered the option to agree or decline to the request. If the depositor then uses the link in the email and the form it opens in DSpace to transmit their agreement to the request, the system will attempt to email the file(s) to the user.

Known limitation

Of course some files are too large for many email servers to cope with. For example files over 150 MB have been impossible for some systems to send.

Search - Advanced

Advanced Search (including boolean options) is already supported in the DSpace 7 search page. Boolean keywords can be used, and you can also specify to search within specific fields by name. Some examples:

- *Basic searching:* Searching `test power` will return results with both these words in them (this is equivalent to an "AND" boolean search). E.g. https://demo.dspace.org/search?query=test%20power
- *Boolean searching options*
  - Searching `test AND power` will return results with both these words in them. E.g. https://demo.dspace.org/search?query=test%20AND%20power
  - Searching `test OR power` will return results with either of these words in them. E.g. https://demo.dspace.org/search?query=test%20OR%20power
  - Searching `test NOT power` will return results with "test" but not including "power". E.g. https://demo.dspace.org/search?query=test%20NOT%20power
- *Phrase searching:* Searching `"test power"` (in quotes) will return results with the exact phrase "test power" in them. E.g. https://demo.dspace.org/search?query=%22test%20power%22
- *Searching within specific fields*
  - Searching `dc.title:test` will only return results where the `dc.title` includes "test". E.g. https://demo.dspace.org/search?query=dc.title:test
  - Searching `dc.title:test AND dc.description.abstract:green` will only return results where the `dc.title` field includes "test" and the `dc.description.abstract` field returns "green". E.g. https://demo.dspace.org/search?query=dc.title:test%20and%20dc.description.abstract:green
  - Searching `dc.subject:fin*` will only return results where one (or more) dc.subject fields start with "fin" (e.g. finance, financial, finish, etc), e.g. https://demo.dspace.org/search?query=dc.subject:fin*
- *Wildcard searching:*
  - Searching `test pow*` will return results including "test" and any word starting with "pow". E.g. https://demo.dspace.org/search?query=test%20pow*
  - Searching `dc.description.abstract:*` will return results that include the "dc.description.abstract" metadata field (with any value in it). E.g. https://demo.dspace.org/search?query=dc.description:*
- *Range searching:*
  - Searching `dc.date.issued:[1999 TO 2003]` will return results that have a "dc.date.issued" metadata field that has a date between 1999 and 2003 (inclusive). E.g. https://demo.dspace.org/search?query=dc.date.issued:%5B1999%20TO%202003%5D
  - Searching `dc.date.issued:[2010 TO *]` will return results that have a "dc.date.issued" metadata field that has a date after (or including) 2010 . E.g. https://demo.dspace.org/search?query=dc.date.issued:%5B2010%20TO%20*%5D
- *Special characters:* Some characters have special meaning in searches, e.g. colon (:), asterisk (*), boolean operators, etc. If you need to search for these characters exactly, surround them with double quotes.
  - Searching `"test:power"` will search for that string *exactly* (including the colon character). (NOTE: Without the quotes, DSpace would attempt to perform "Searching within specific fields" (see above) as the colon is a special character.)

DSpace supports all Solr search syntax options, as all searches in DSpace are sent directly to Solr. For more examples, see the Solr documentation for the "Specifying Terms for the Standard Query Parser".

User management

Documentation for repository managers.

- Add or Manage an E-Person
- Create or manage a user group

Add or Manage an E-Person

An E Person is a user in DSpace who can be assigned various rights to perform activities or manage content access in the repository.

This section provides details about various methods to create or update an E Person in DSpace. For example, DSpace allows users to self-register. In addition, users with administrative rights can create and update E Person in the system. Both methods are explained in the latter part of this document.

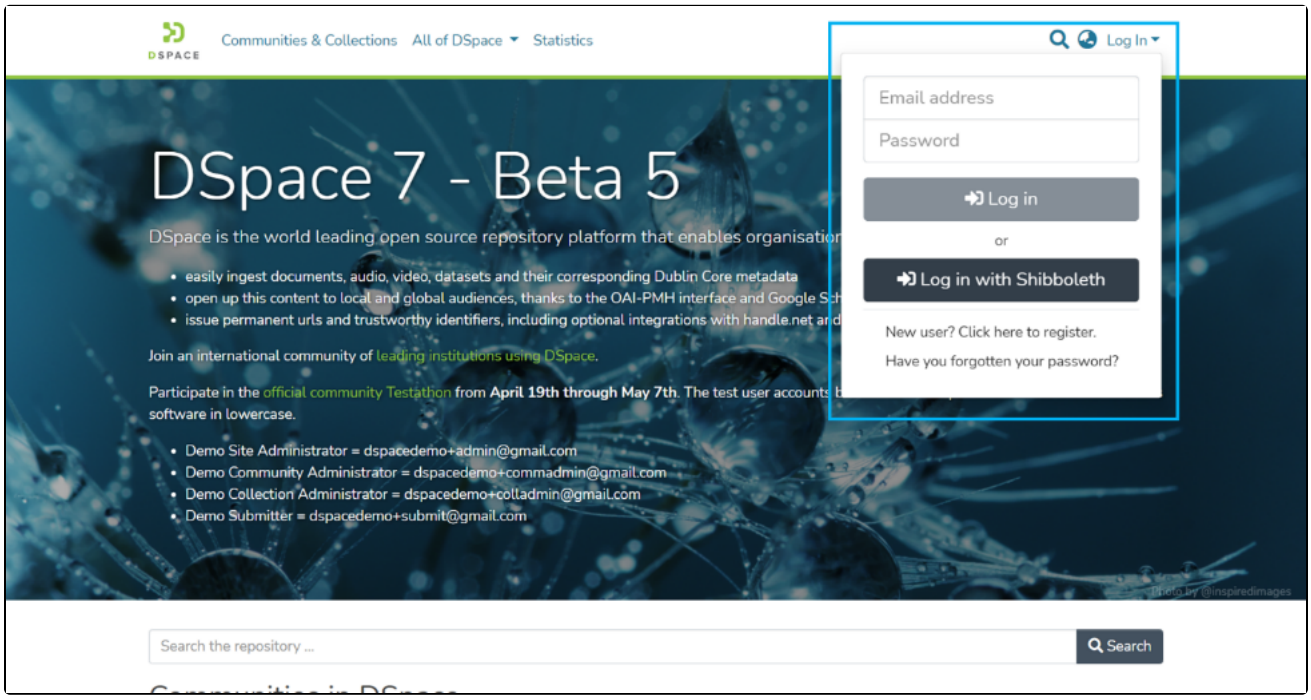
- Audience
- Add E Person – Self Registration
- Add E Person – Registration by Administrator
- Update an Eperson
    - Update an Eperson – Update details & Manage Log in
    - Update an Eperson – Manage user groups membership
    - Update an Eperson – Delete Eperson

Audience

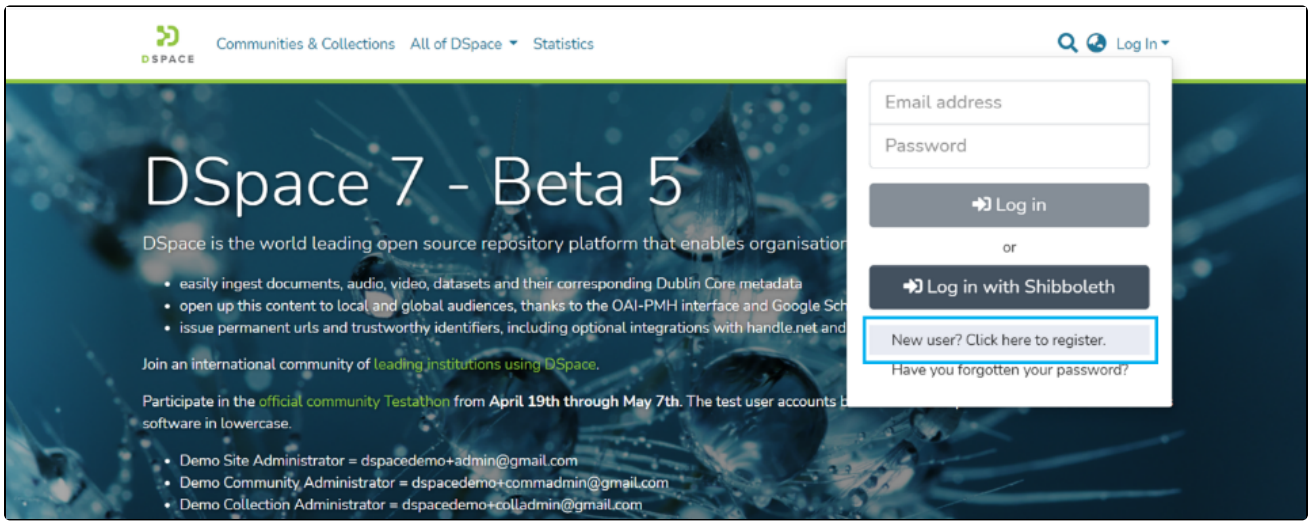
1. Repository Administrator
2. Community Administrator
3. Collection Administrator
4. Base User

Add E Person – Self Registration

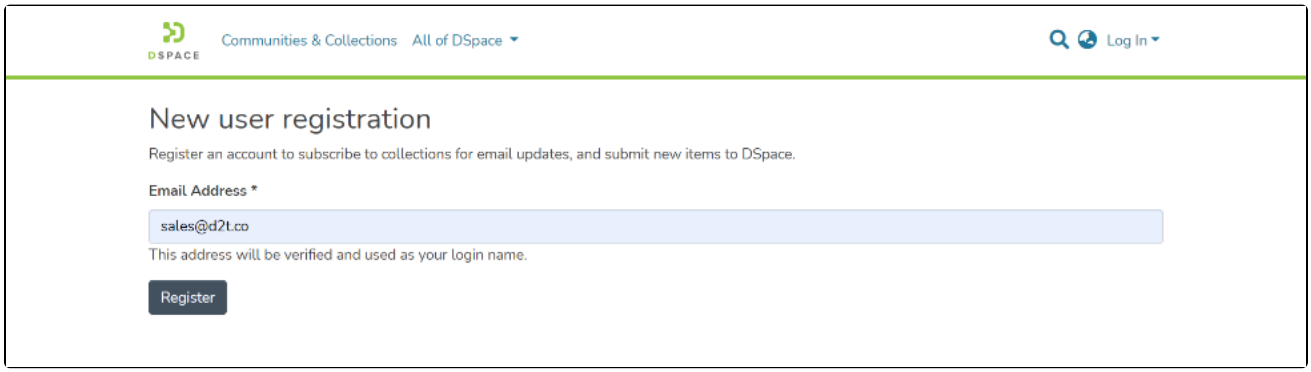
Step 1: Go to DSpace's home page and click on the "Log In" link appearing at the top right corner of the screen, and the pop-up will open, as illustrated below screen.



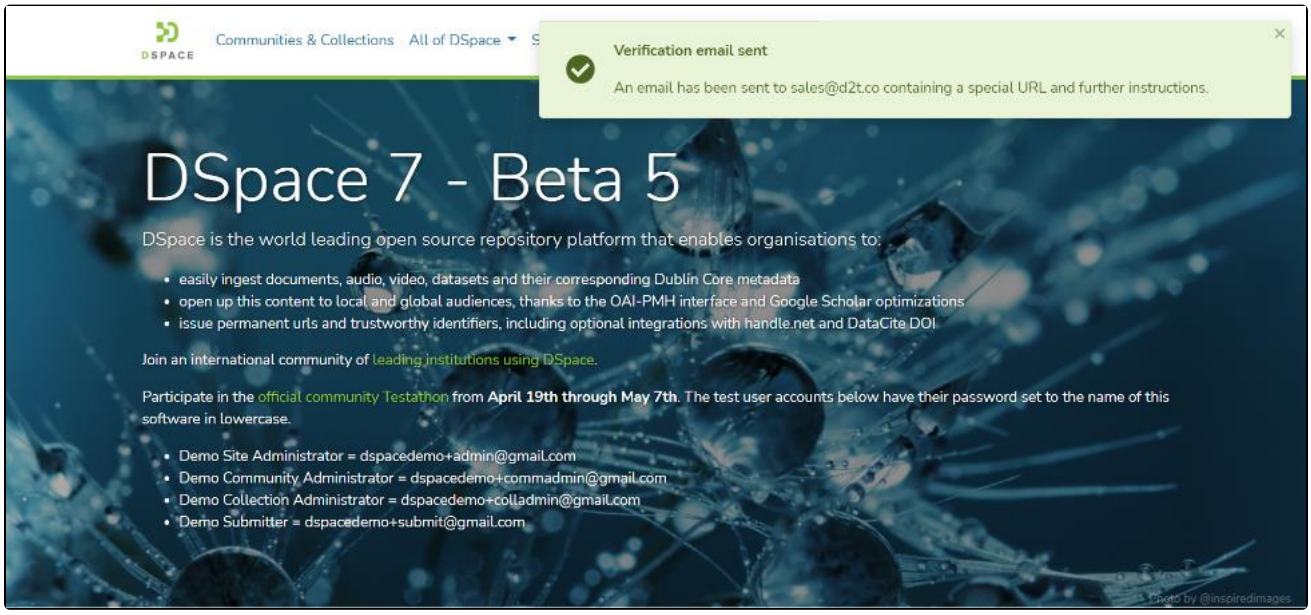
Step 2: Click on the New User to add an E Person in DSpace.



Step 3: Enter the user's Email ID who needs to be registered as E Person in DSpace and click on the Register button.

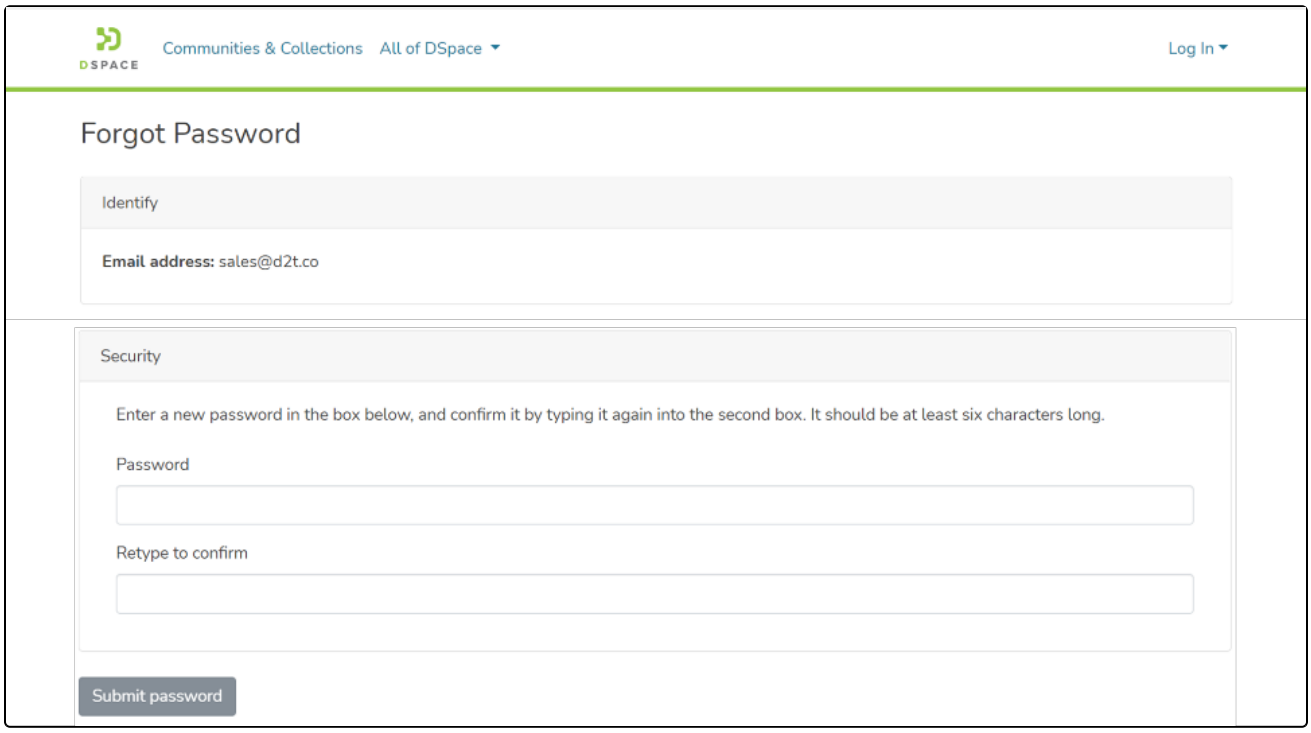


Step 4: After clicking the register button, an email will be sent to the user's email id, and the user will be redirected to the home page. This E Mail will go from the communication mail ID registered in the DSpace instance.



Step 5: Click on the unique registration link received in the email to continue with the registration process. Suppose your email client or server security settings do not show hyperlinks. In that case, you can copy the link and paste it into your browser.

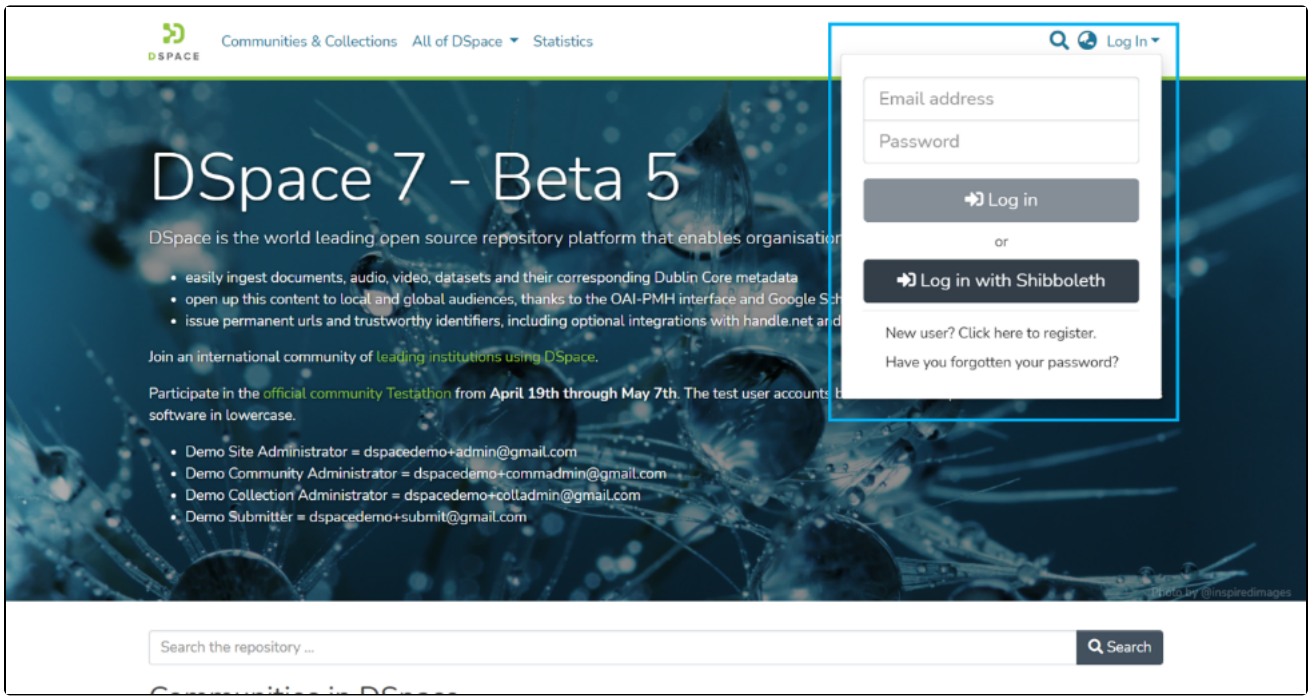
Step 6: Enter a password of your choice and re-enter the same password. Followed by this, click on the “Submit Password” button to complete the registration process.



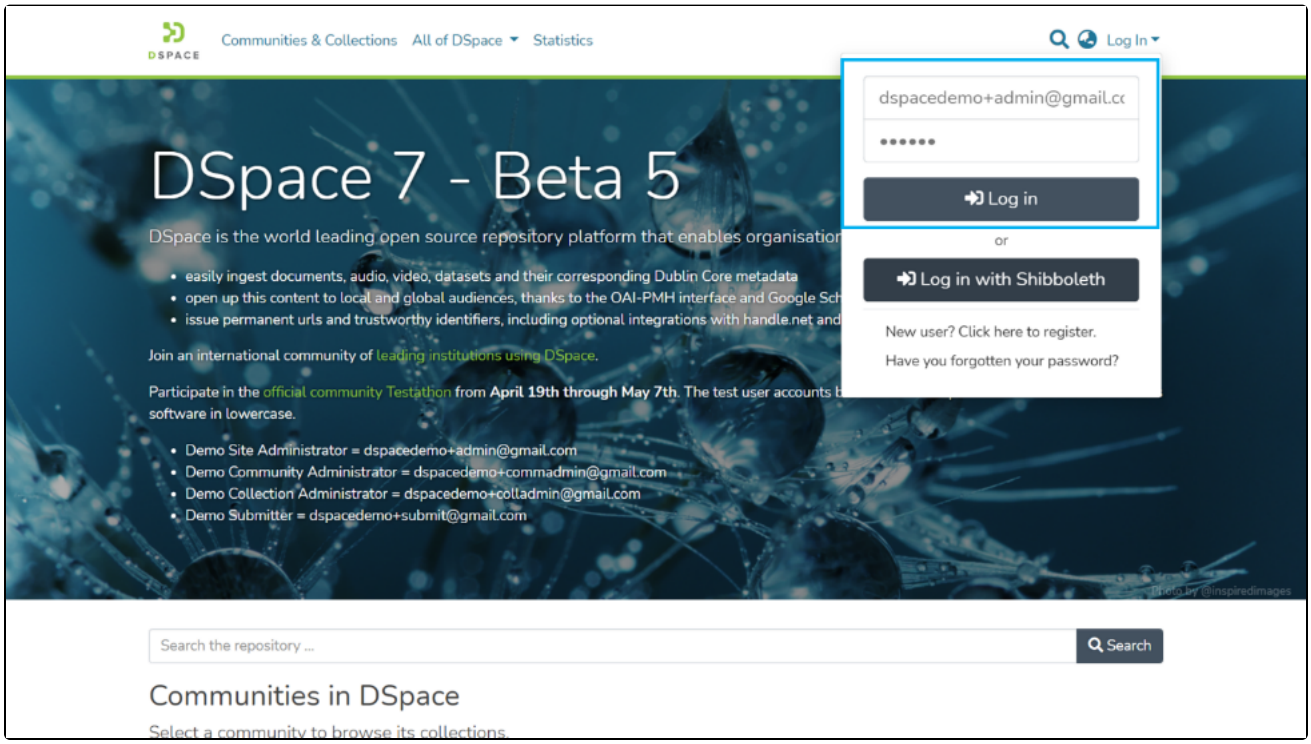
Add E Person – Registration by Administrator

This process is for the user having System, Community, and Collection Administrator rights. Only users with these rights can manage E People.

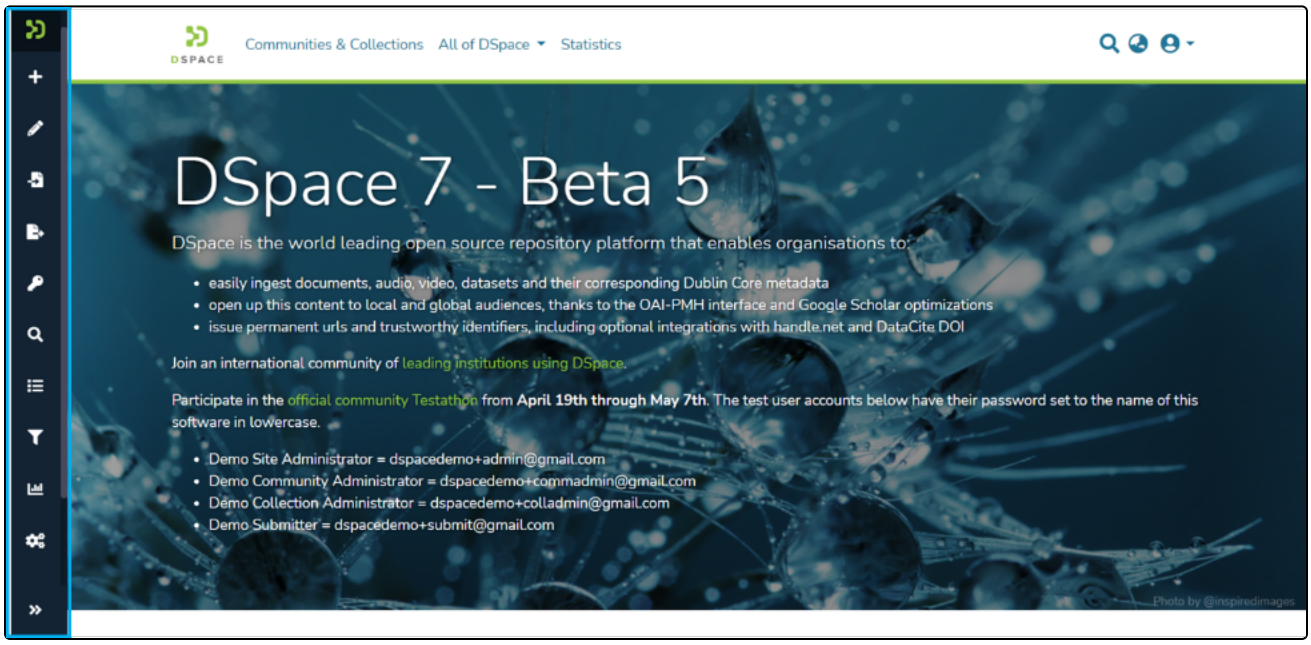
Step 1: Go to DSpace’s home page and click on the “Log In” link appearing at the top right corner of the screen, and the pop-up will open, as illustrated below screen.



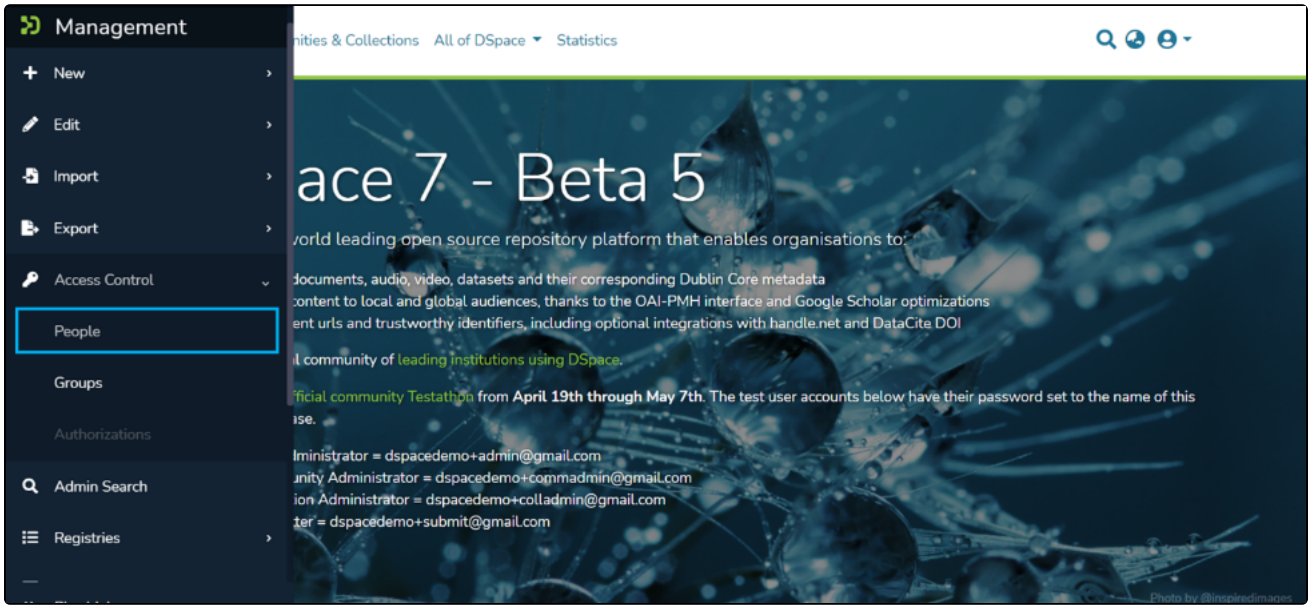
Step 2: Enter your user id and password and click on the login button for logging in to DSpace



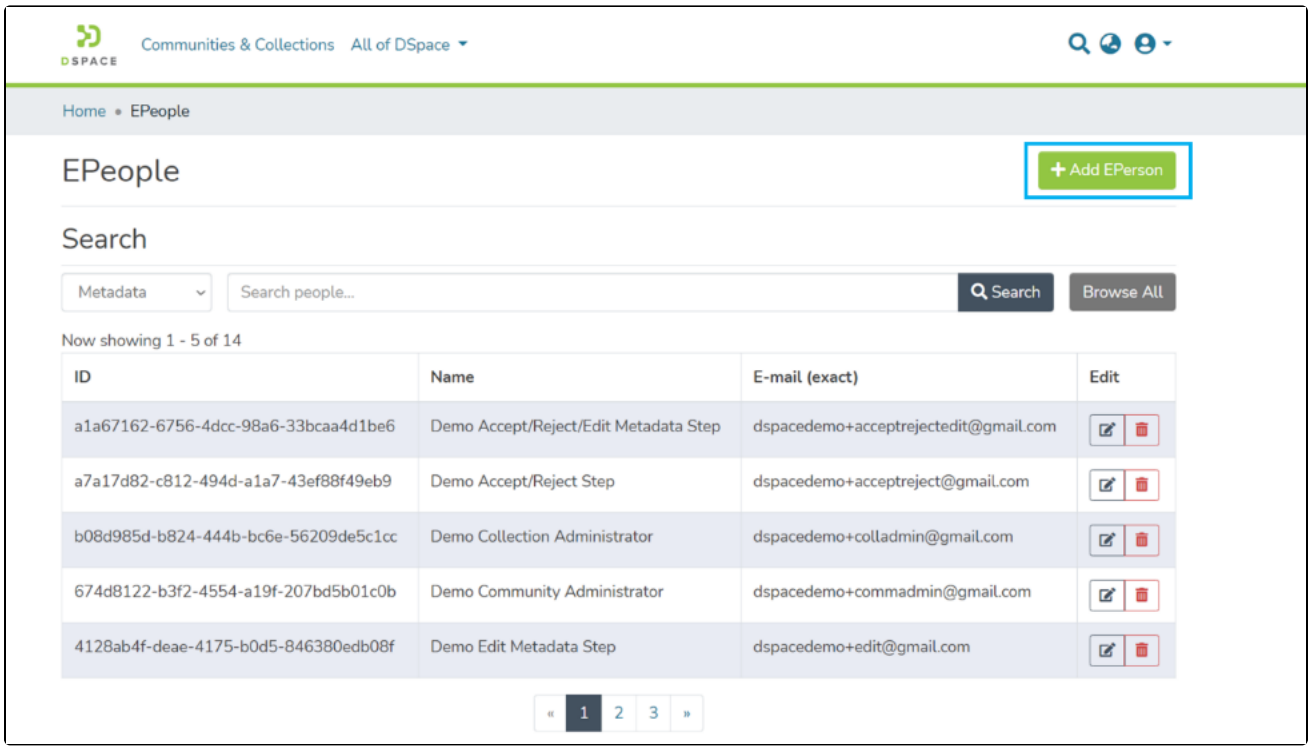
Step 3: Users with administrative rights will see the admin menu on the left-hand side of the screen, as shown below illustration.



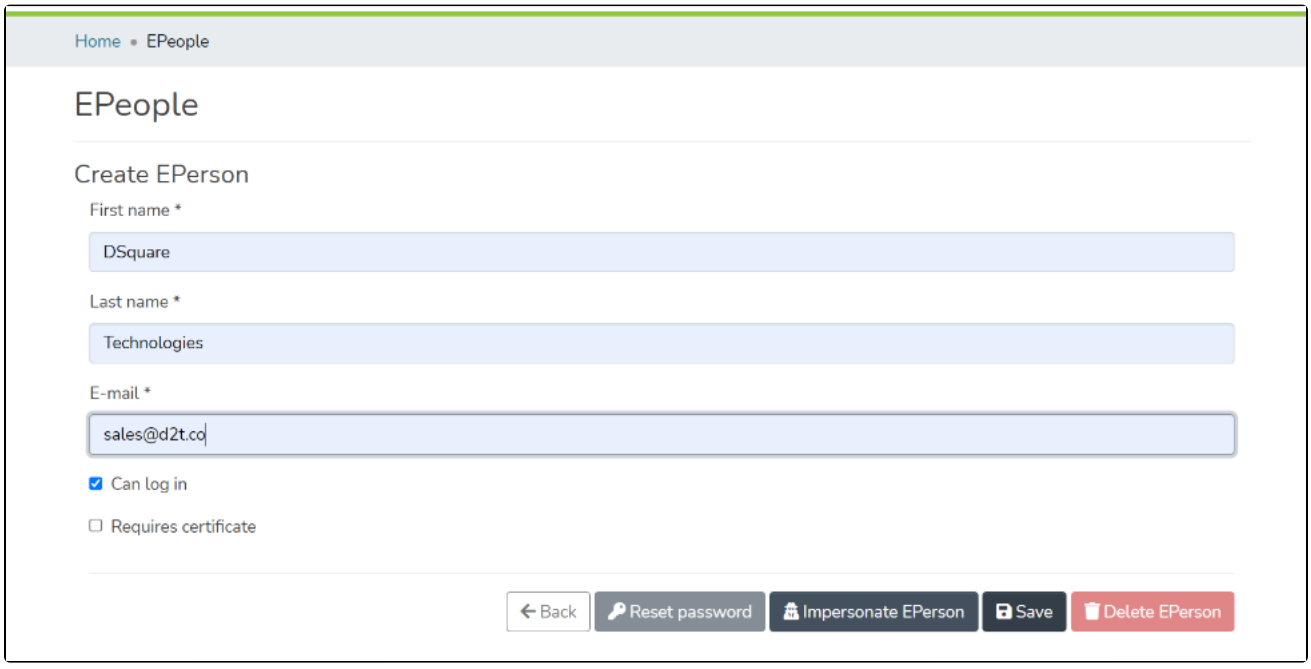
Step 4: Rollover your cursor over the administration menu and click on Access control. Click on the People link to go to the EPeople module.



Step 5: Click on the “Add EPerson” button for initiating the E-Person creation process.

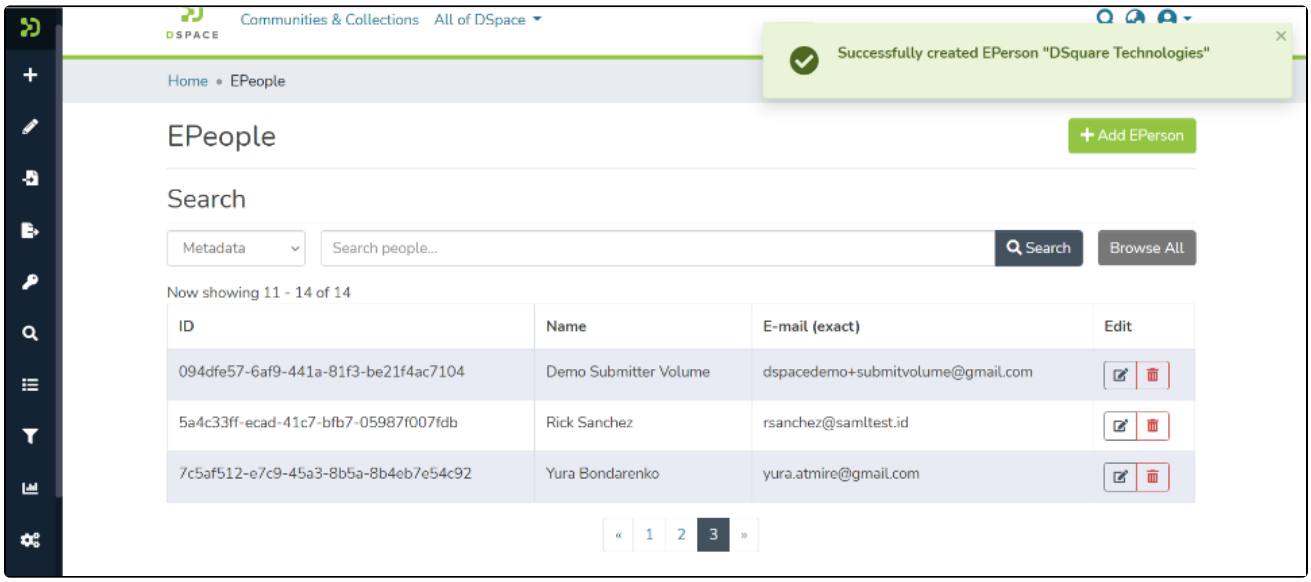


Step 6: Enter “First Name”, “Last Name,” and “E-Mail”. Select the “Can log in” check box for enabling id to login into the DSpace.



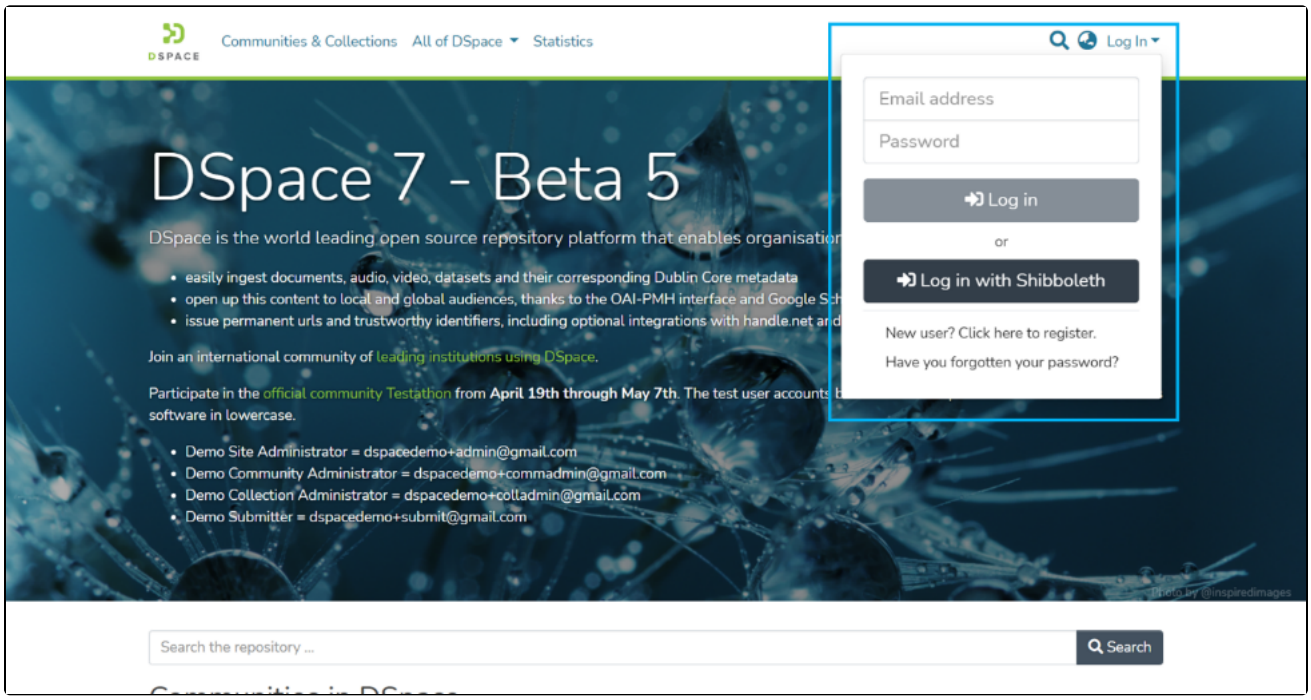
Step 7: Upon populating values in mandatory fields “Save” button will get activated. Click on the Save button to complete the process.

Step 8: Upon successfully adding EPerson in DSpace, you will see a success prompt on the screen.

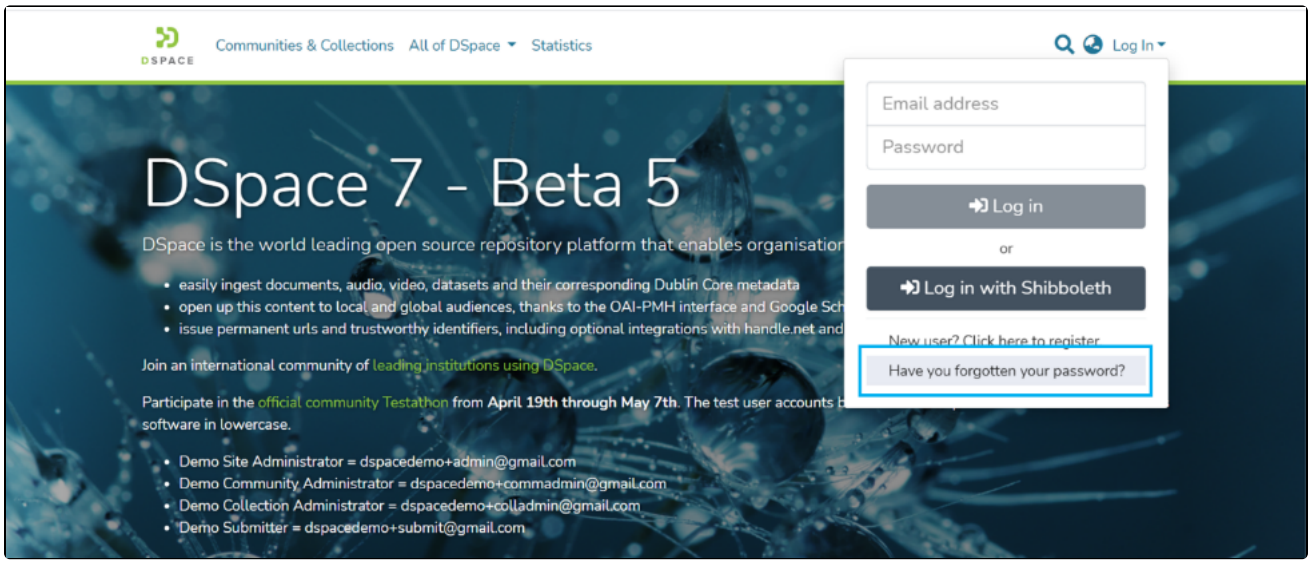


Step 9: The administrator can inform the user to generate its password using forgot password mechanism. The same is explained in the following steps.

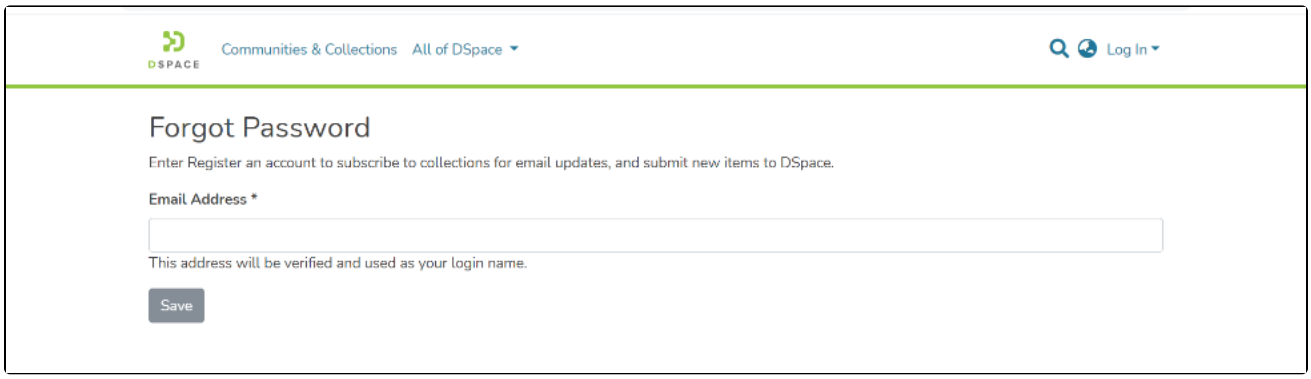
Step 10: The user to go to the home page of DSpace and click on the “Log In” link appearing at the top right corner of the screen and pop up will open, as illustrated below screen.



Step 11: Click on the “Have you forgotten your password?” password link reset page.



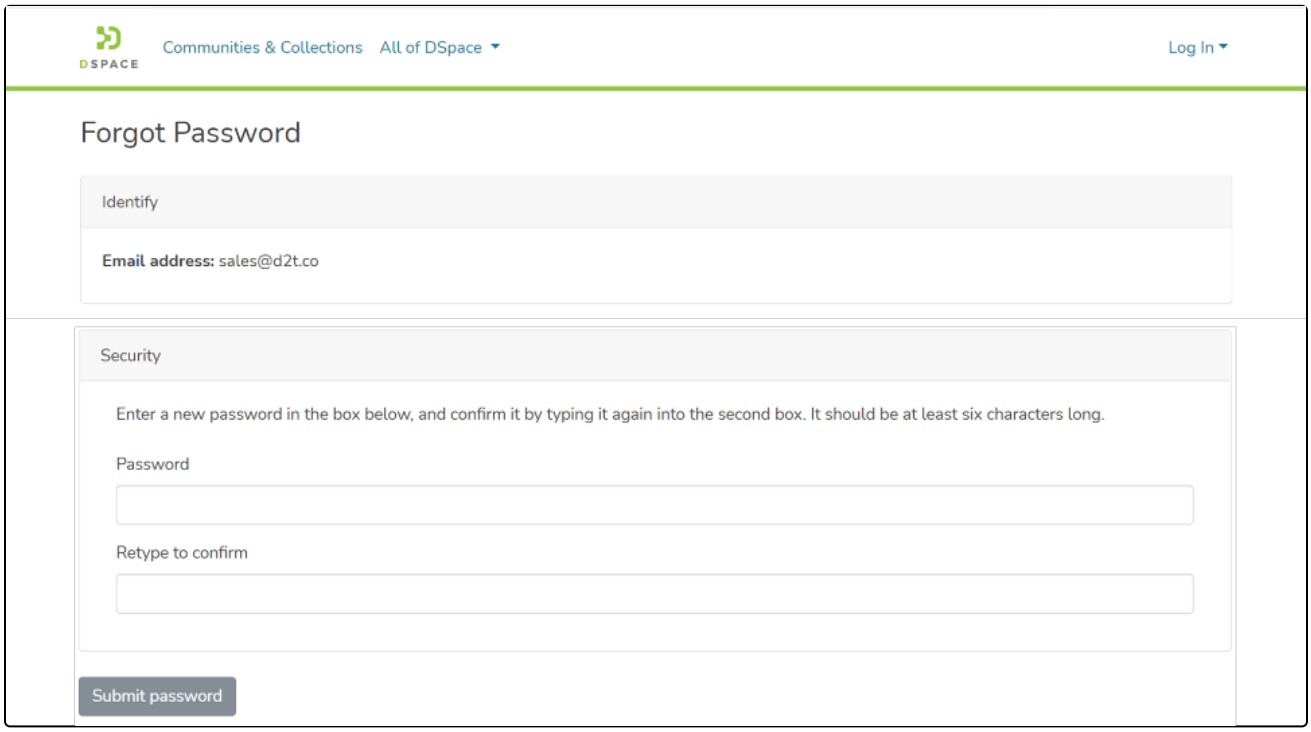
Step 12: Enter the Email ID used for Eperson creation in the DSpace and click on the “Save” button.



Step 13: A prompt confirming a password reset link dispatch to the registered email id will appear and the user will be redirected to the home page. This E Mail will go from the communication mail ID registered in the DSpace instance.

Step 14: Click on the unique registration link received in the email to continue the password reset process. Suppose the user's email client or server security settings do not show hyperlinks. In that case, the user can copy the link and paste it into its browser.

Step 15: Enter a password of your choice and re-enter the same password. Followed by this, click on the "Submit Password" button to complete the password generation process.



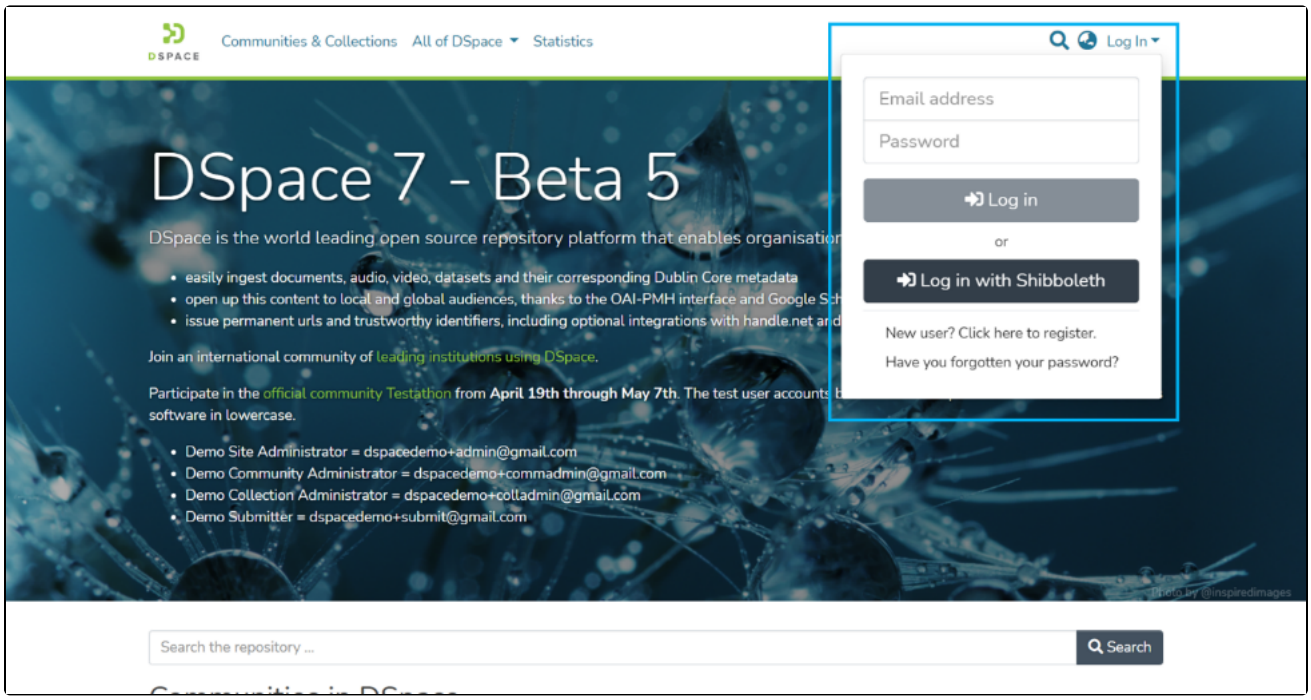
Update an Eperson

Eperson updates can be performed by users having System, Community, and Collection Administrator rights. These users can perform the following activities:

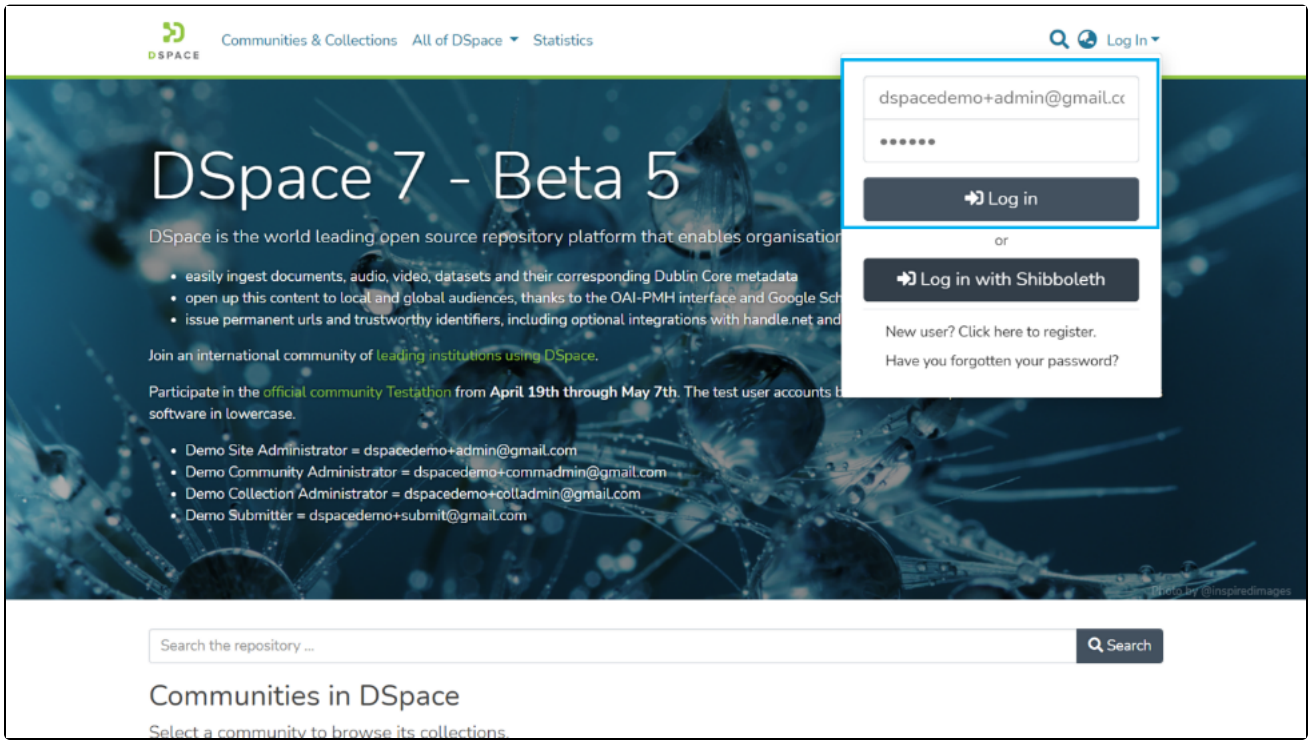
1. Update Eperson details
2. Manage Eperson Log in
3. Impersonate Eperson
4. Manage user groups membership
5. Delete Eperson from DSpace

Update an Eperson – Update details & Manage Log in

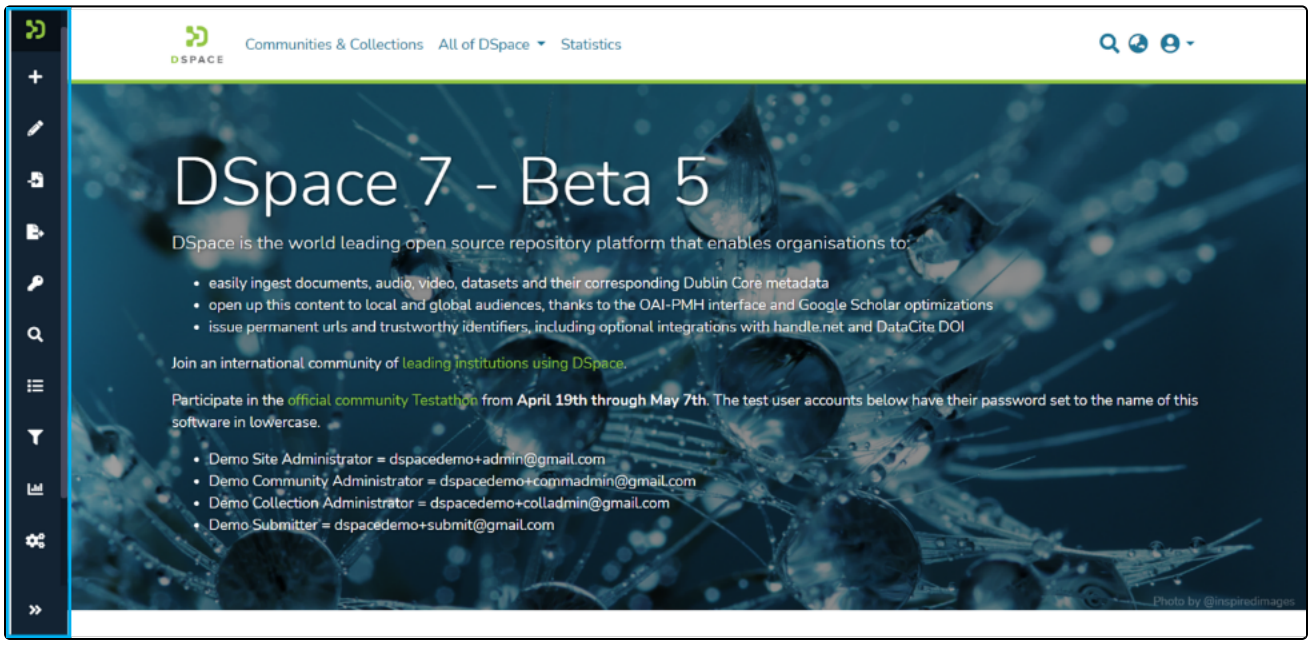
Step 1: Go to DSpace's home page and click on the "Log In" link appearing at the top right corner of the screen, and the pop-up will open, as illustrated below screen.



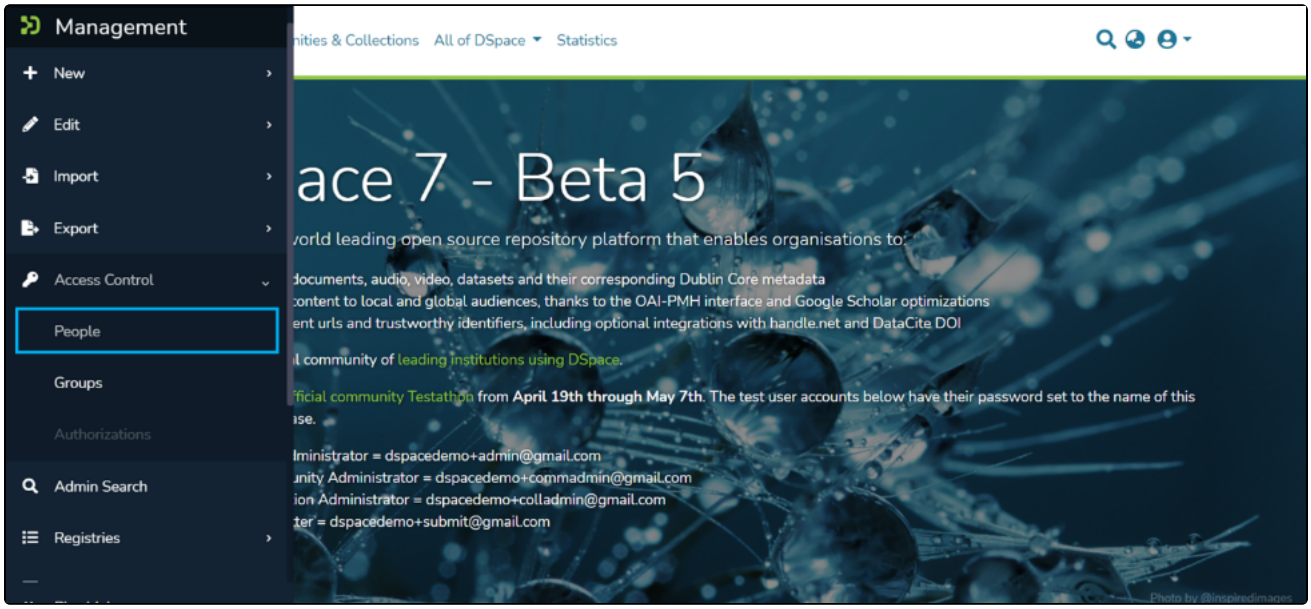
Step 2: Enter your user id and password and click on the login button for logging in to DSpace.



Step 3: Users with administrative rights will see the admin menu on the left-hand side of the screen, as shown below illustration.



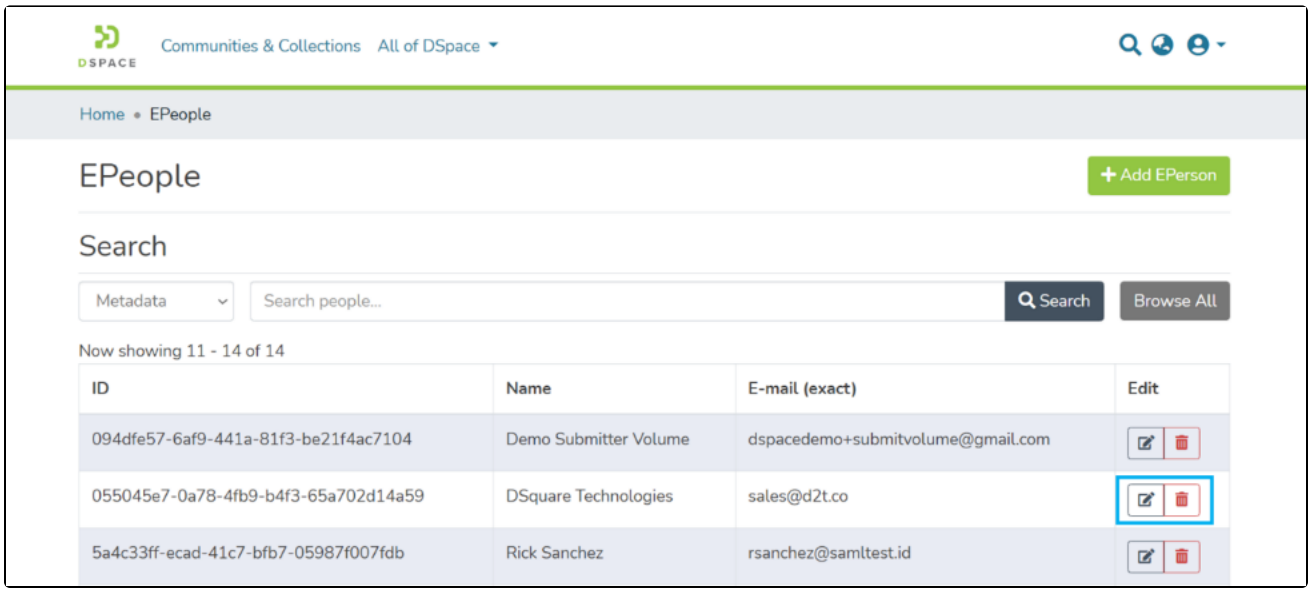
Step 4: Rollover your cursor over the administration menu and click on Access control. Click on the People link to go to the EPeople module.



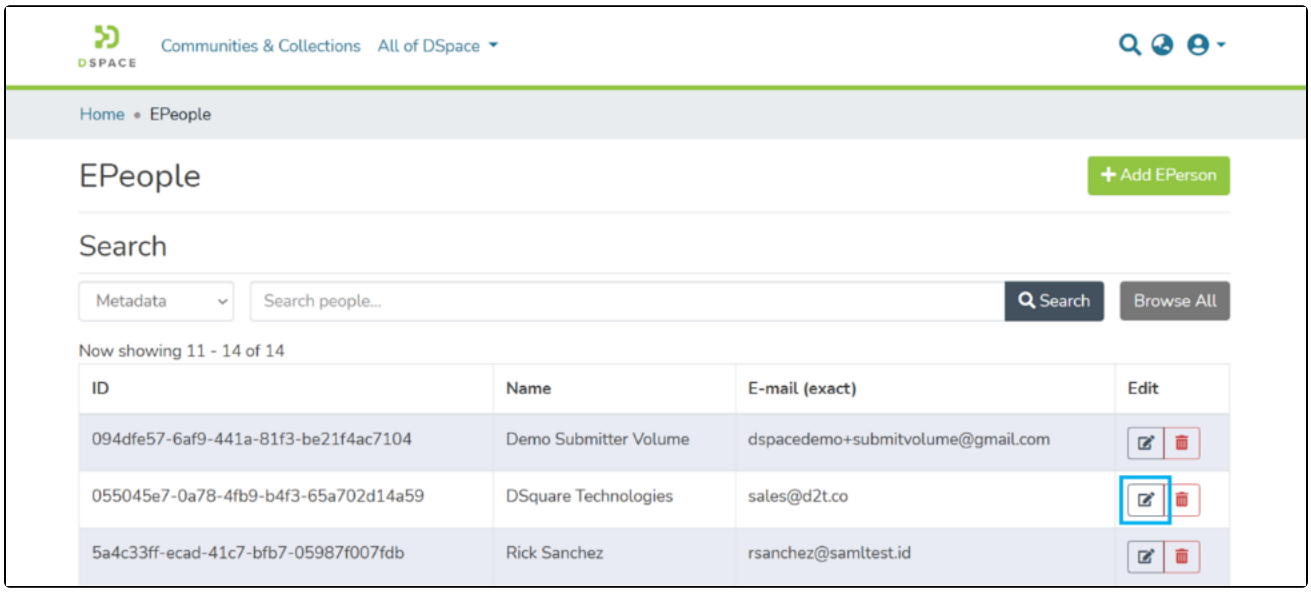
Step 5: Scroll or search the eperson's name in the epeople list.

Step 6: Epeople table has the following elements:

- ID – Unique ID generated in DSpace for each Eperson
- Name – Full name of Eperson as entered during registration
- Email – Email id used for registering user
- Edit – Buttons showing various actions available for Eperson management. It has "Edit" and "Delete buttons. Click on the "Edit" button if you want to edit details of the Eperson, or click on the "Delete" button if you want to delete Eperson from DSpace permanently.



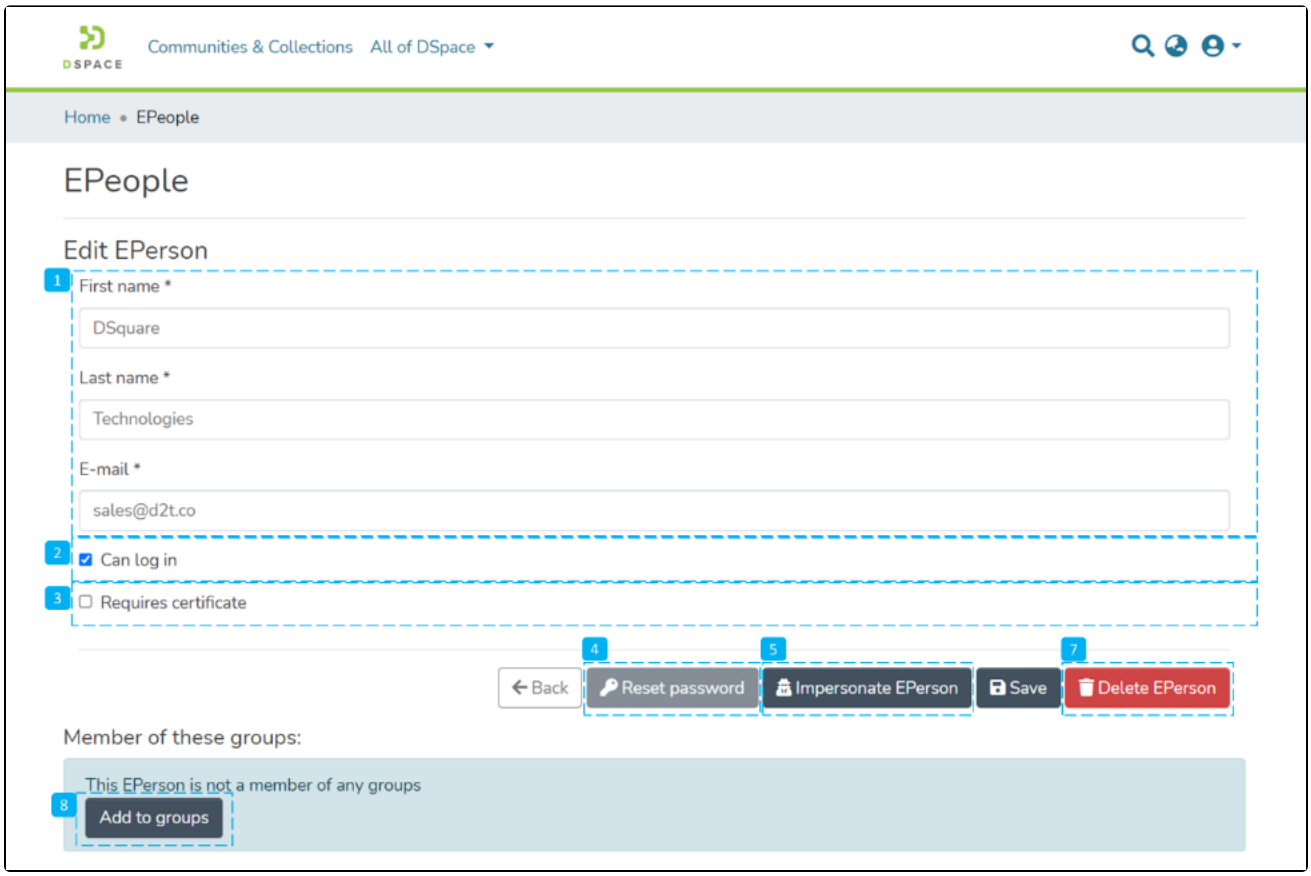
Step 7: Click on the “Edit” button to continue with Eperson editing.



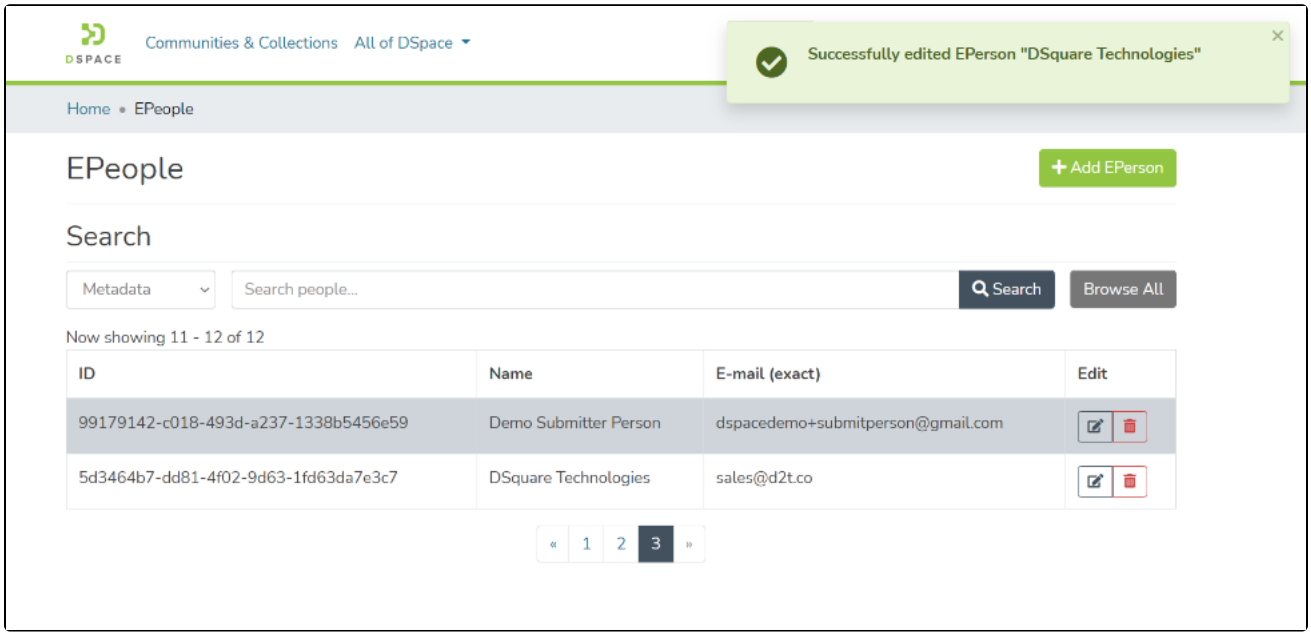
Step 8: You can make the following updates on the Edit Eperson page

1. Update Eperson details – You can update First Name, Last Name, and Email ID as the user’s identification details.
2. Enable/Disable User Login – Uncheck this option to disable the user’s login into the DSpace without deleting it. This option is helpful for scenarios where users need to be disabled from logging in and performing specific actions temporarily.
3. Requires Certificate – Configure this option if a certificate is to be used for the login.
4. Reset Password – Password can be reset for Eperson using this option. A Password field should be present for utilizing this option.
5. Impersonate Eperson – The user with rights to Impersonate EPerson can impersonate the selected Eperson and perform all activities that the Eperson is entitled to.
6. Delete Eperson – Using this option, Eperson can be deleted permanently from DSpace
7. Update User group(s) – This button allows the administrator to add selected Eperson to multiple user groups.

You can make options # 1 to 3 and click on Save for updating the Eperson record.

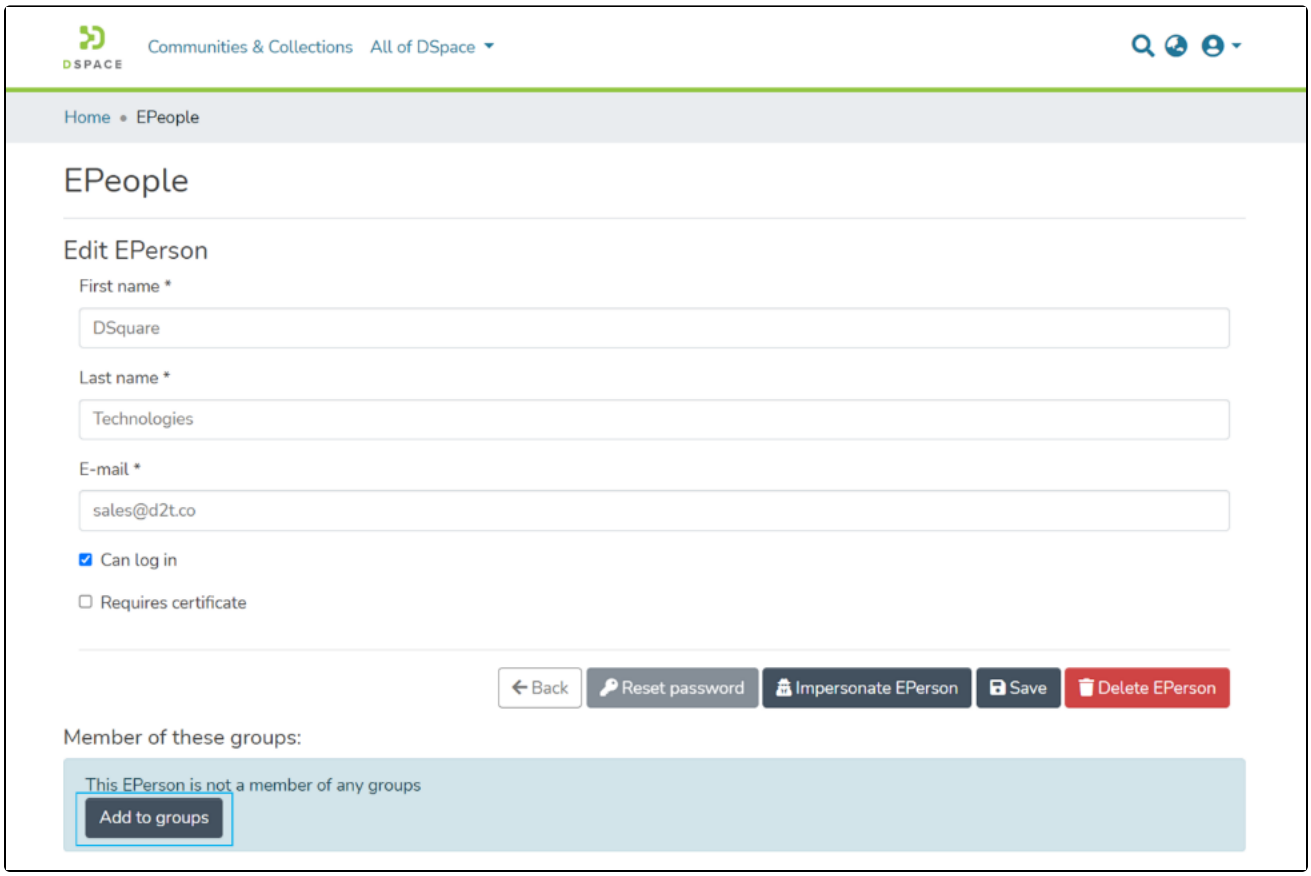


Step 9: Upon successful execution of the update, you will notice a success prompt on the screen.

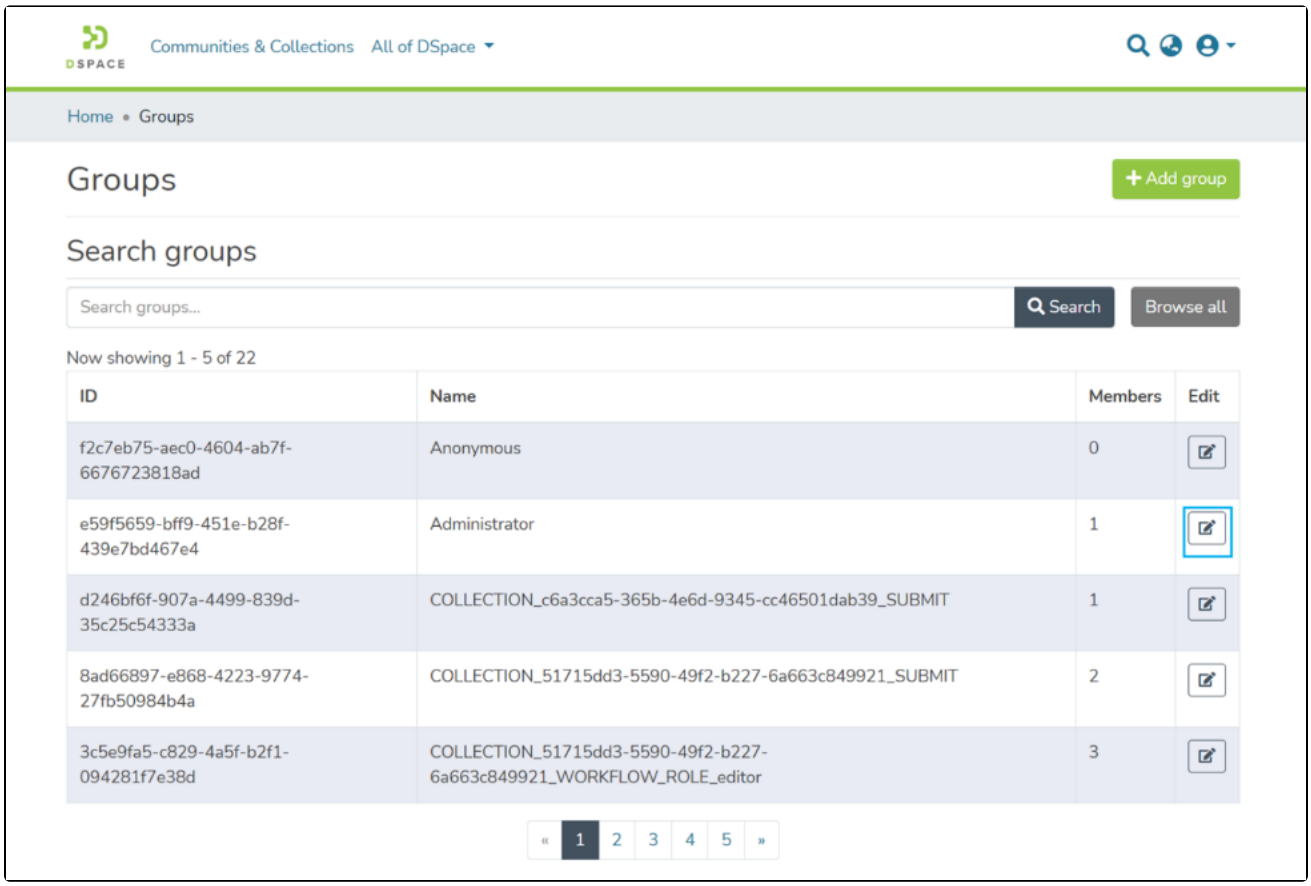


Update an Eperson – Manage user groups membership

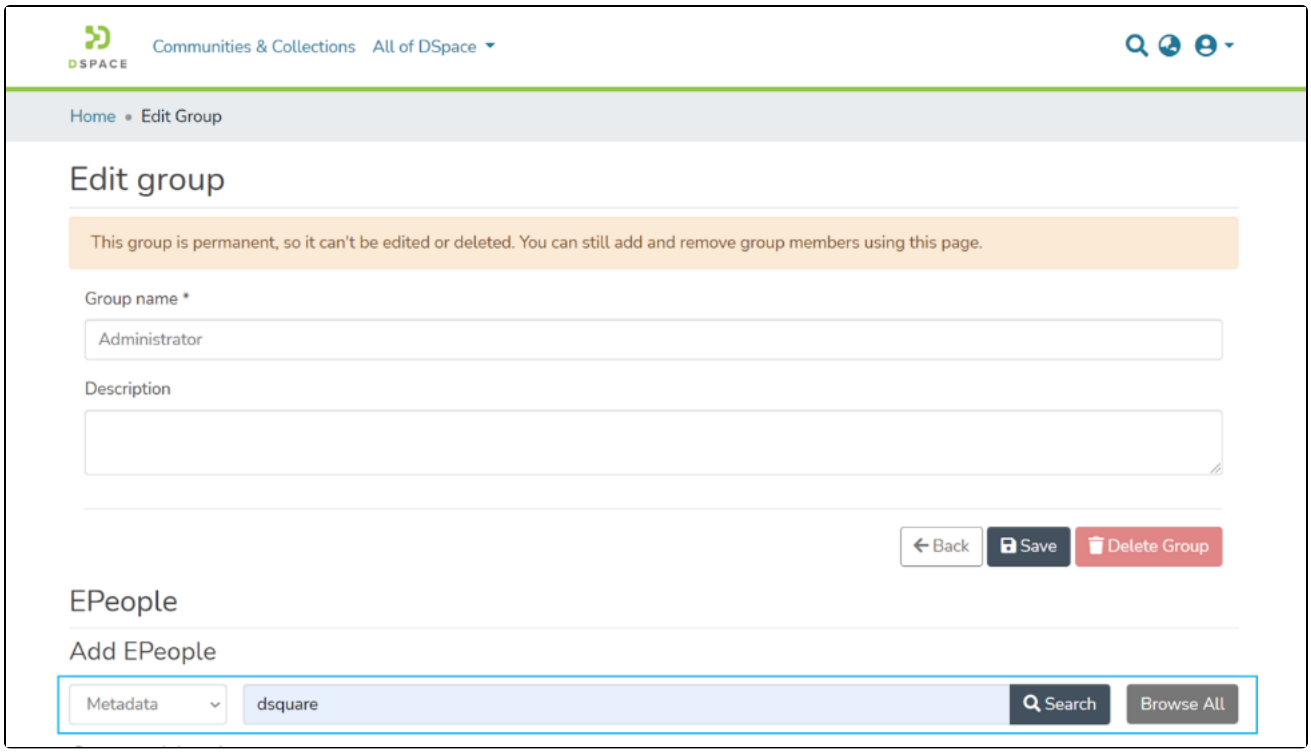
Step 10: Click on the “Add to group” button to initiate the process of adding selected Eperson to a user group.



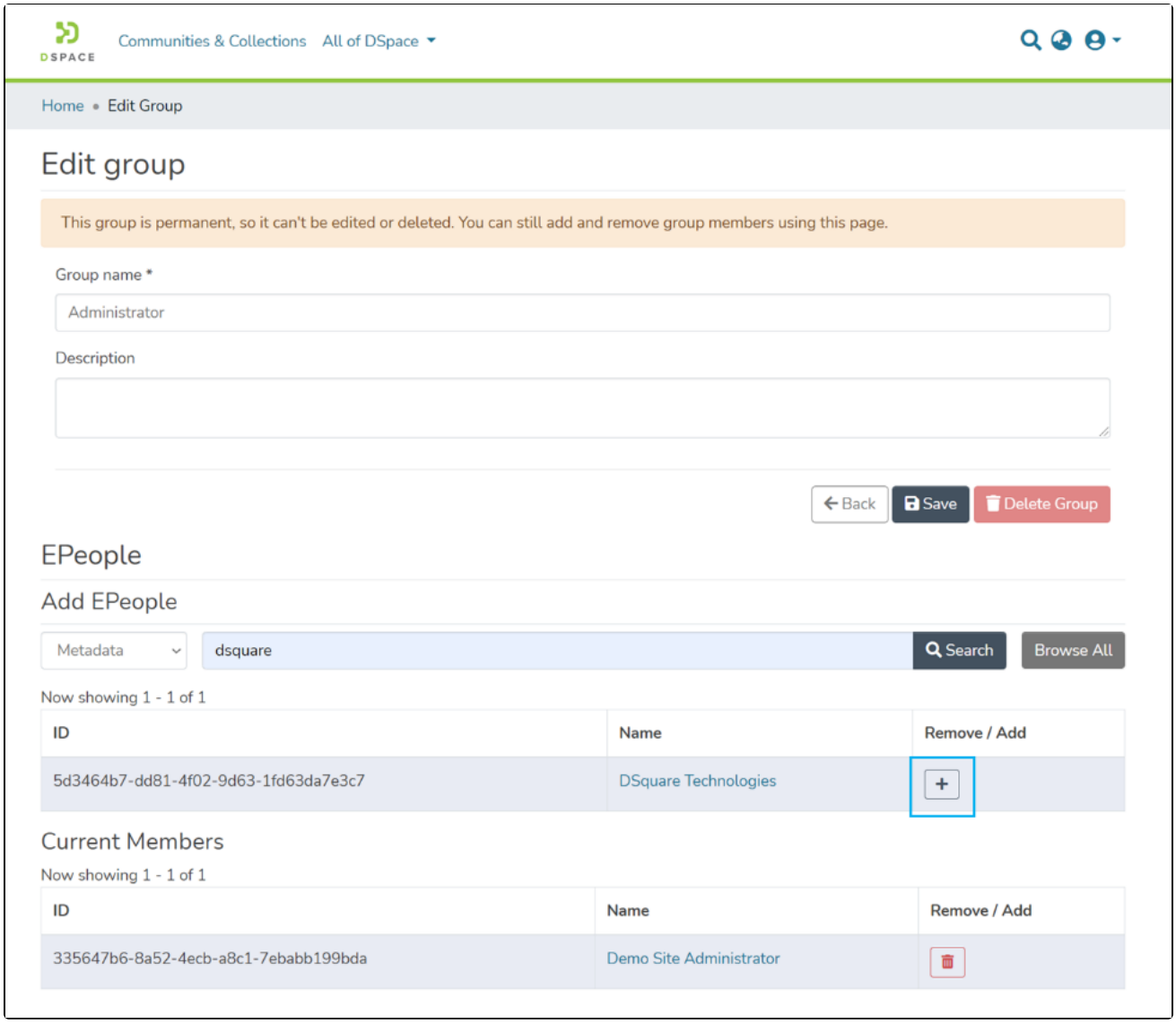
Step 11: Please click on the “Edit” button next to the user group you want to select for this Eperson.



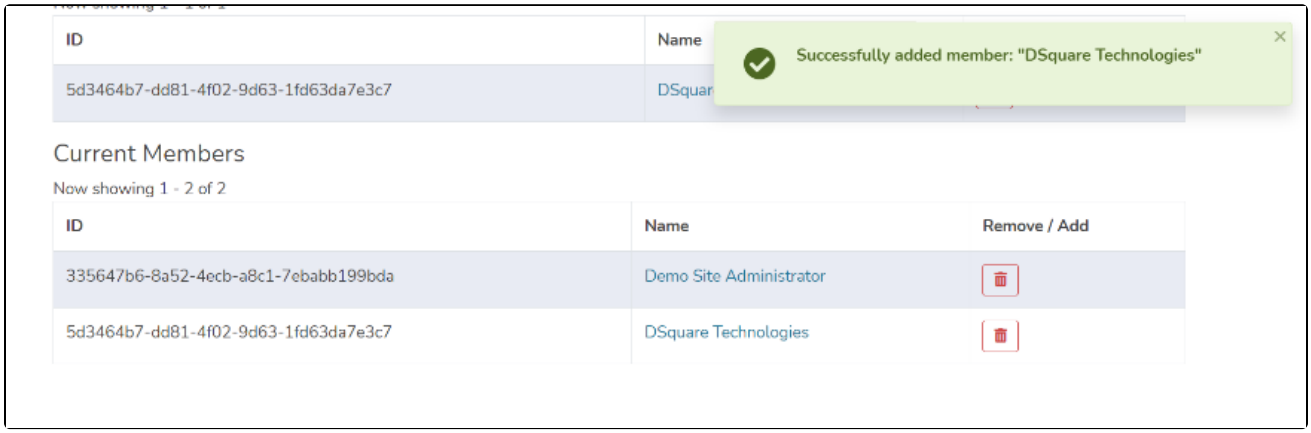
Step 12: Enter the eperson's name or any other metadata value in the search field to find the target user.



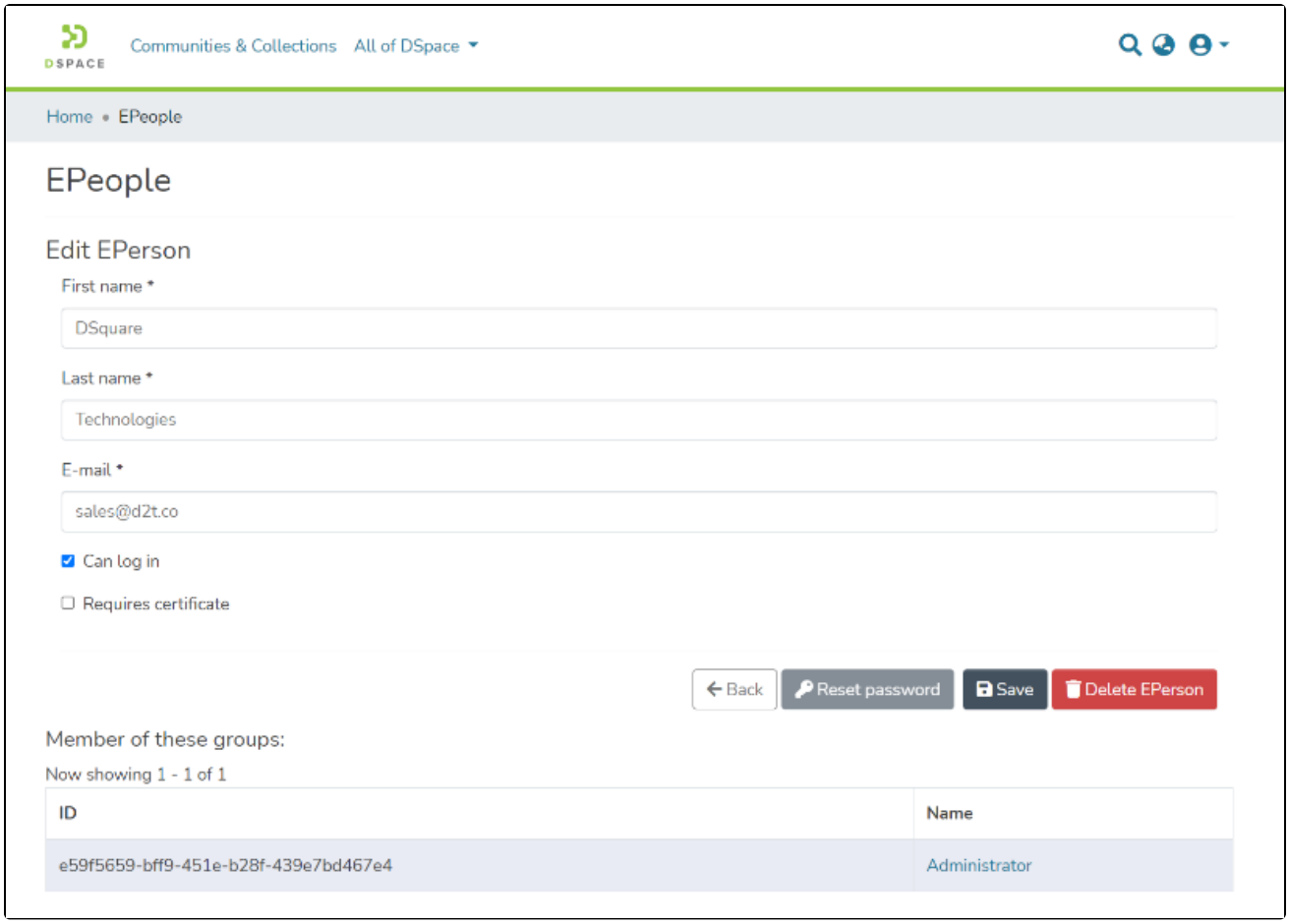
Step 13: You will see Epeople appearing as a result of a search made. Click on the + button appearing next to Eperson you want to add to this group.



Step 14: Upon successfully adding Eperson to the group, a success prompt will appear on the screen, and Eperson will appear in the Current Members list of the group.

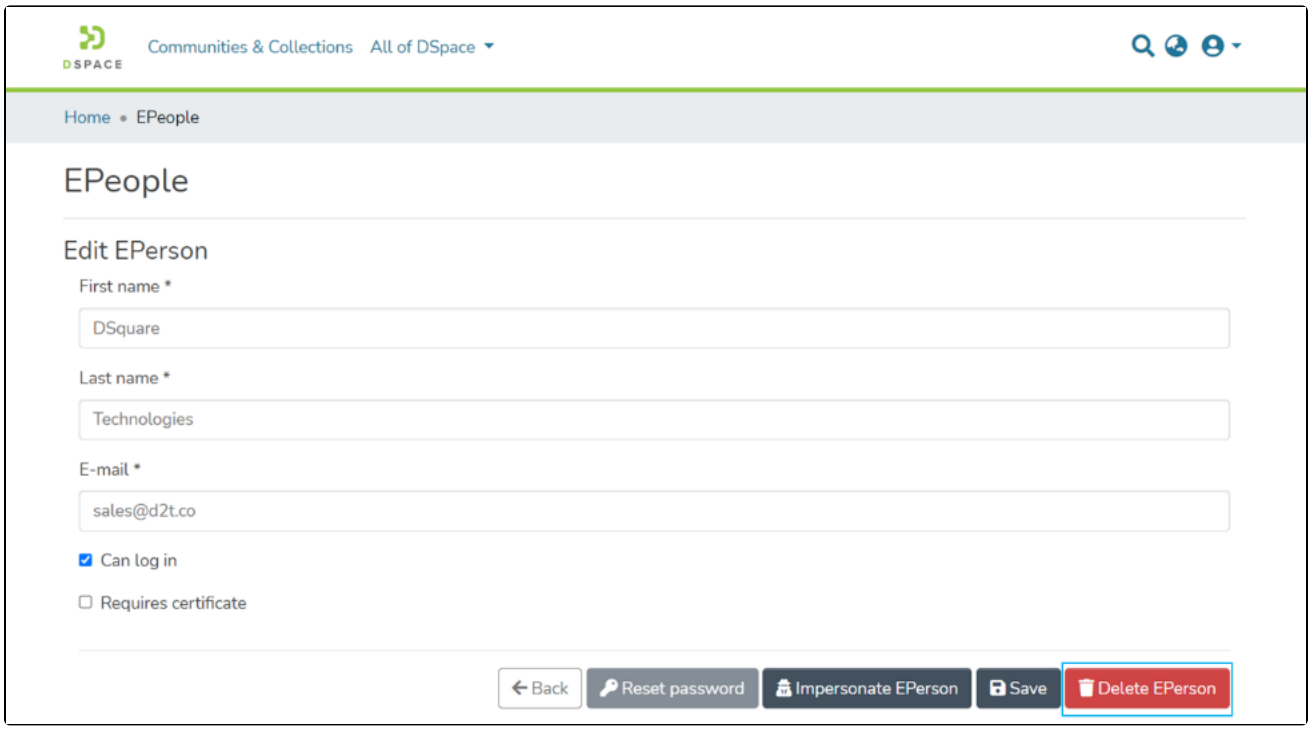


Step 15: You will see this group on the profile page of Eperson as well.

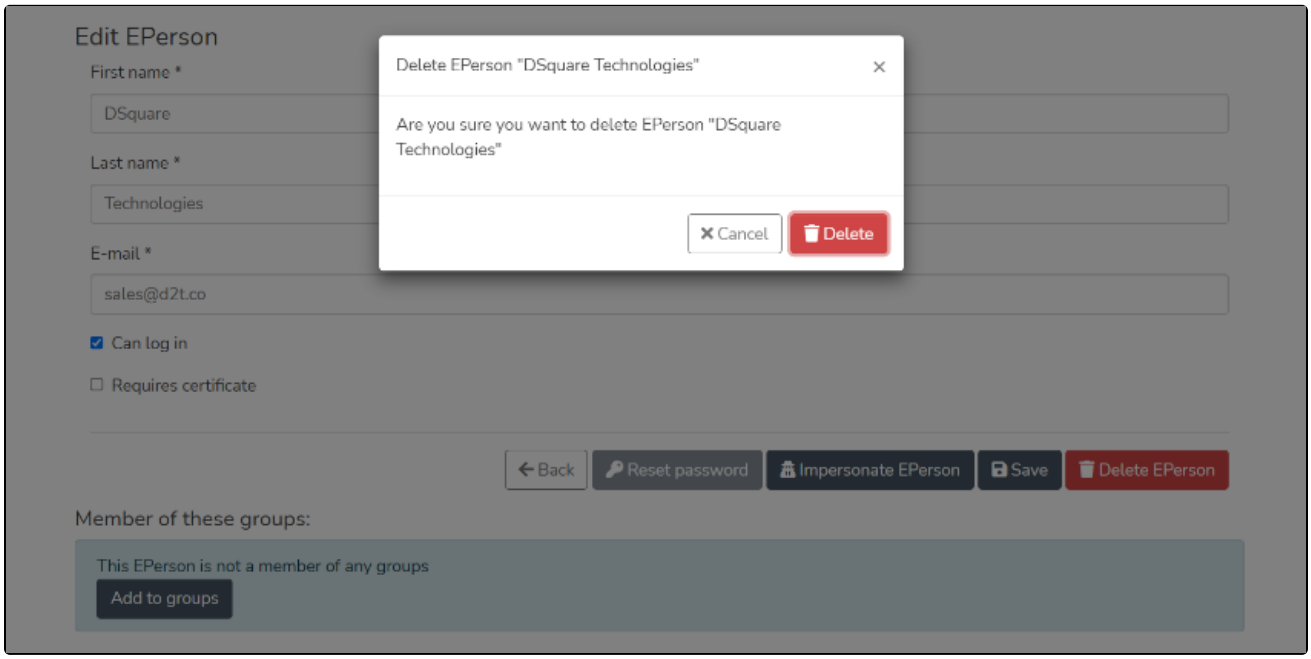


Update an Eperson – Delete Eperson

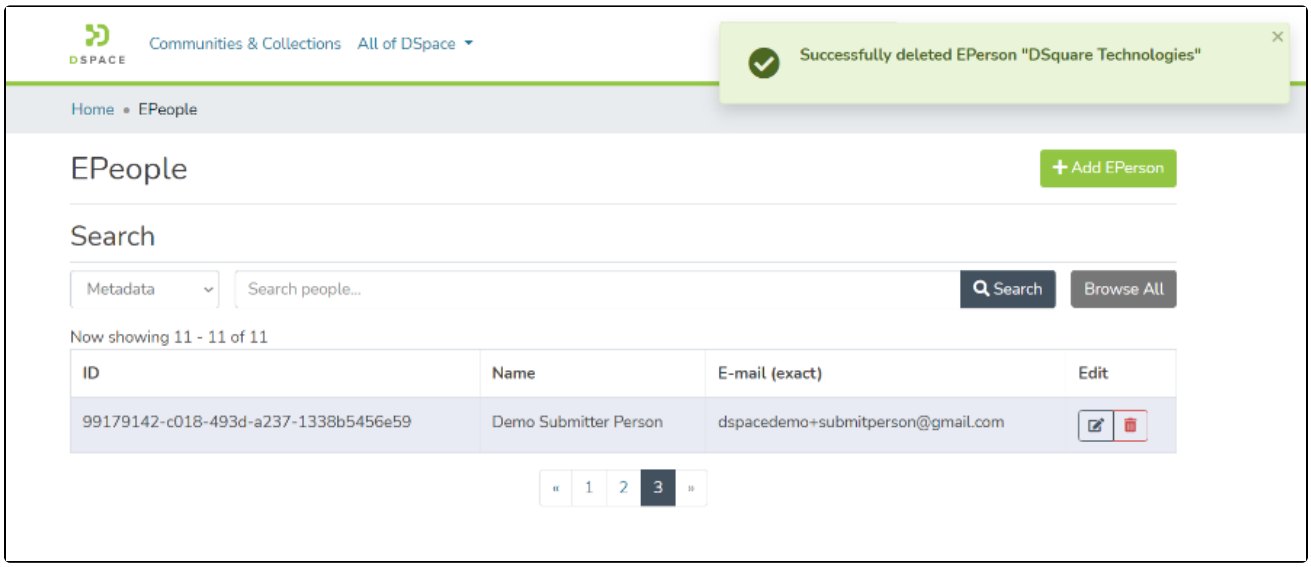
Step 16: Click the “Delete Eperson” button if you want to delete Eperson from DSpace permanently.



Step 17: DSpace will show you a confirmation prompt to re-confirm your decision of deleting the selected Eperson. Click on “Delete” if you want to continue with the deletion, or click on “Cancel” if you want to cancel the deletion.



Step 18: Successful deletion of the user will be confirmed by displaying a success promptly. You will be redirected to the Epeople page.


DSPACE
Communities & Collections
All of DSpace
Successfully deleted EPerson "DSquare Technologies"
Home • EPeople
EPeople
+ Add EPerson
Search
Metadata
Search people...
Search
Browse All
Now showing 11 - 11 of 11
ID
Name
E-mail (exact)
Edit
99179142-c018-493d-a237-1338b5456e59
Demo Submitter Person
dspacedemo+submitperson@gmail.com
« 1 2 3 »

Create or manage a user group

User groups in DSpace are meant for creating a group of E-People. These groups can be assigned access rights to allow users to perform multiple activities or manage content access in the repository.

This section provides steps to add or update a User group in the application.

- Audience
- Add a user group
- Manage a user group
    - Update group details
    - Delete Group
    - Add/Manage EPeople
    - Add/Manage subgroups

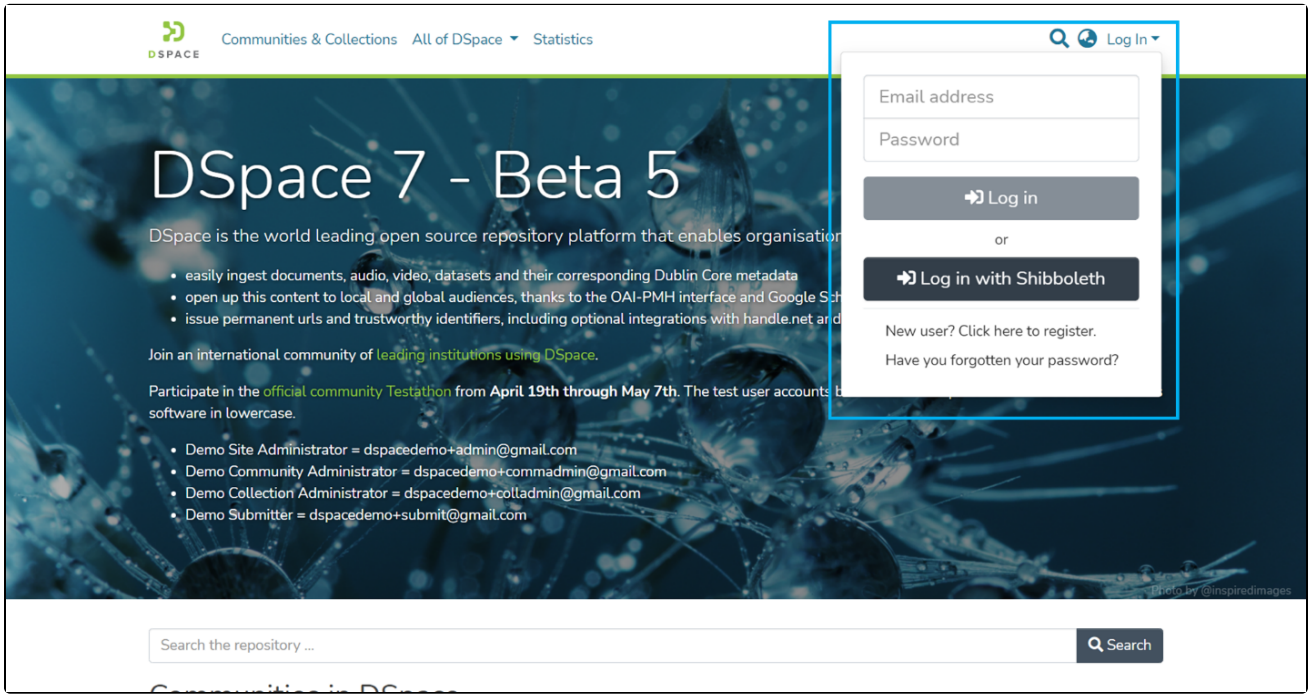
Audience

- Repository Administrator
- Community Administrator

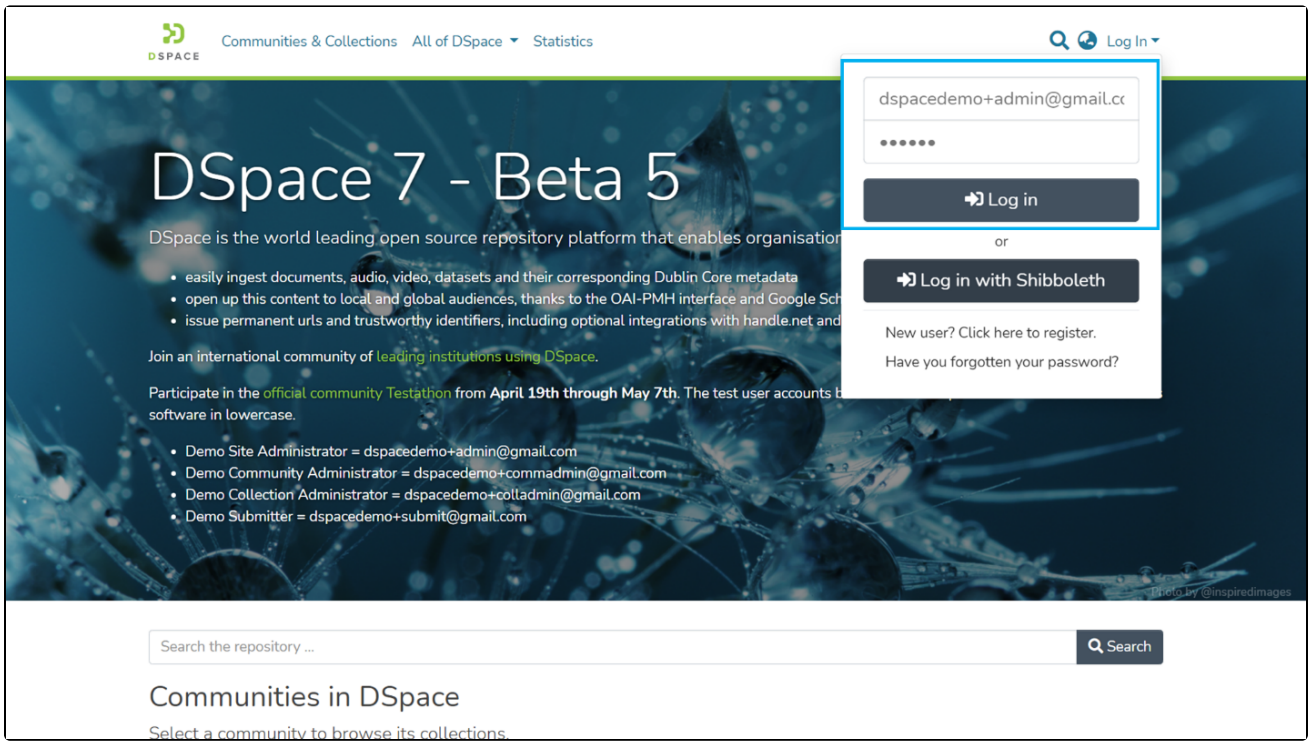
Add a user group

This process is for users having System and Community Administrator rights. As these user profiles can add and manage E People.

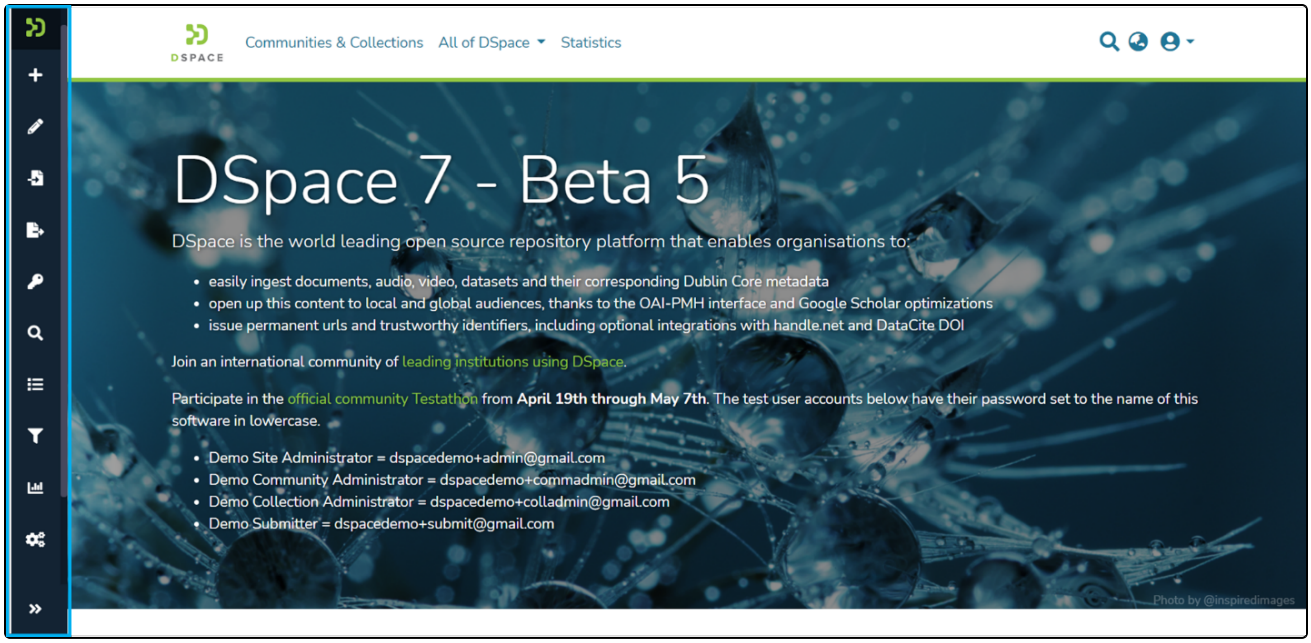
Step 1: Click on the "Log In" link at the top right corner of the DSpace's homepage, and the pop-up will open, as illustrated below the screen.



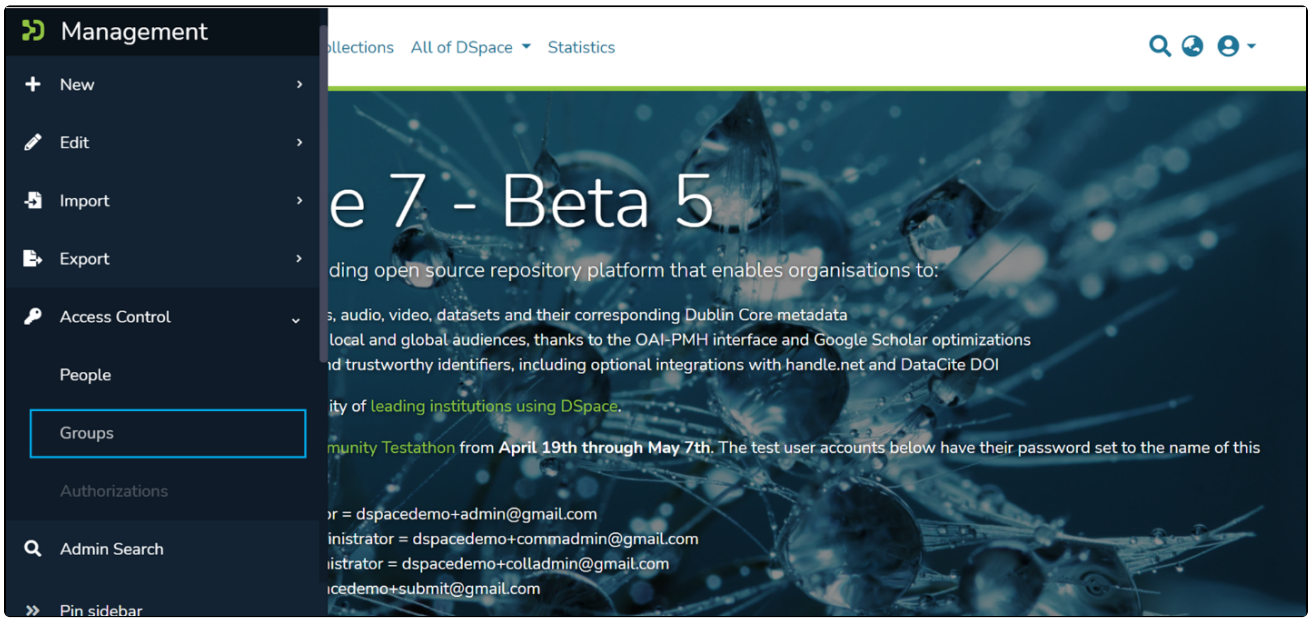
Step 2: Enter your user id and password and click on the "Log in" button to enter into DSpace.



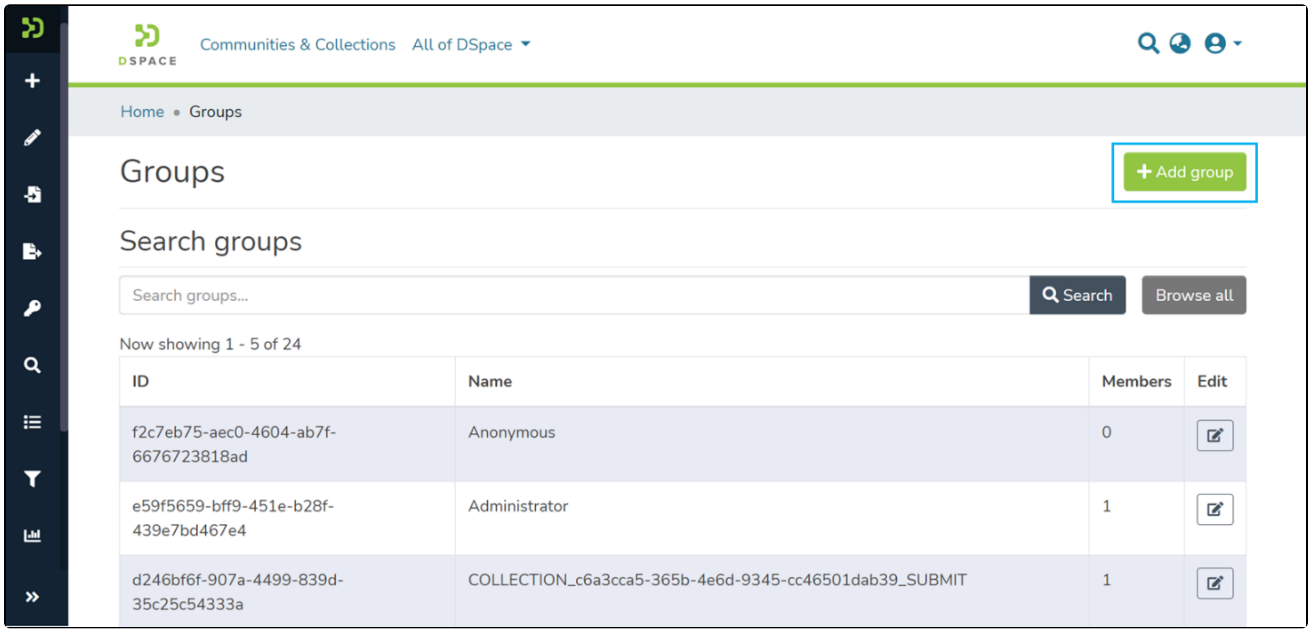
Step 3: Users with administrative rights will see the admin menu on the left-hand side of the screen, as shown in the below illustration.



Step 4: Rollover your cursor over the administration menu, click on "Access Control," and click on the "Groups" link.

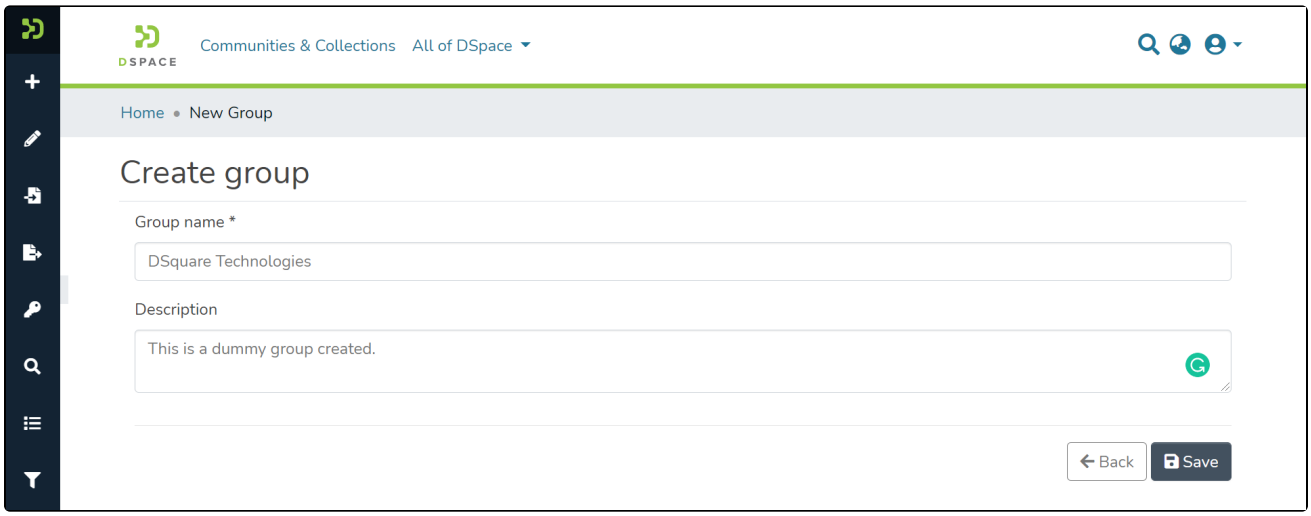


Step 5: Click on the "Add Group" button to create a Group.

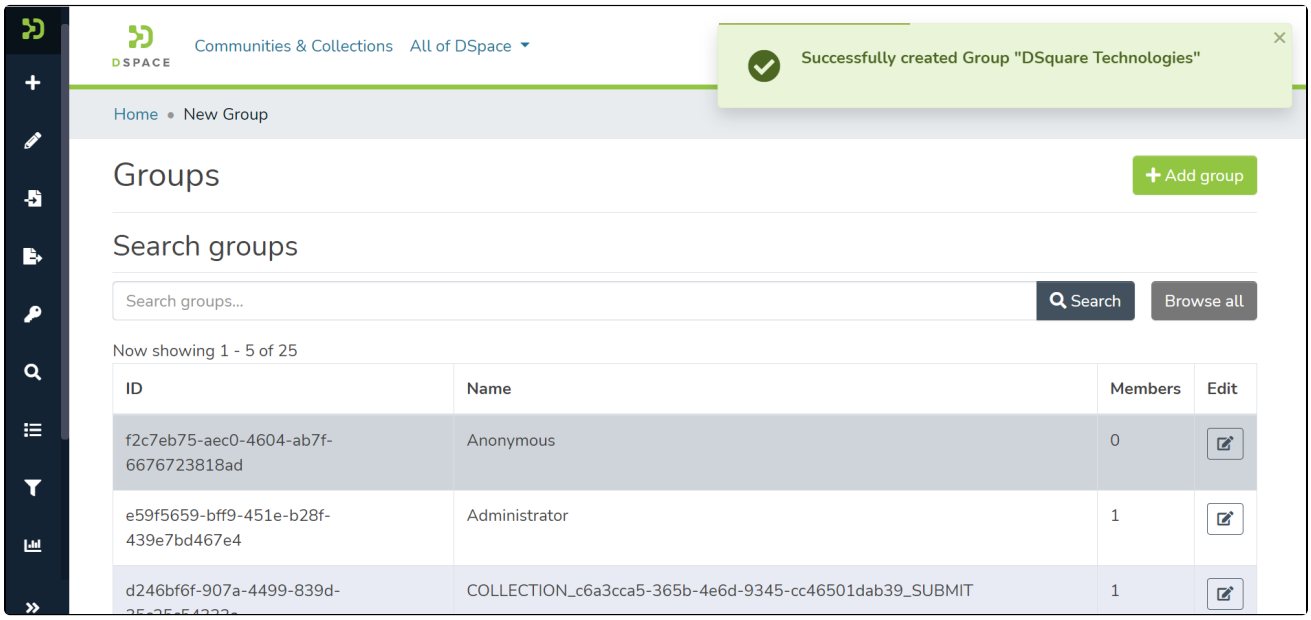


Step 6: The "Group Name" is compulsory. To benefit a broader user base, a description of the Group is a good practice.

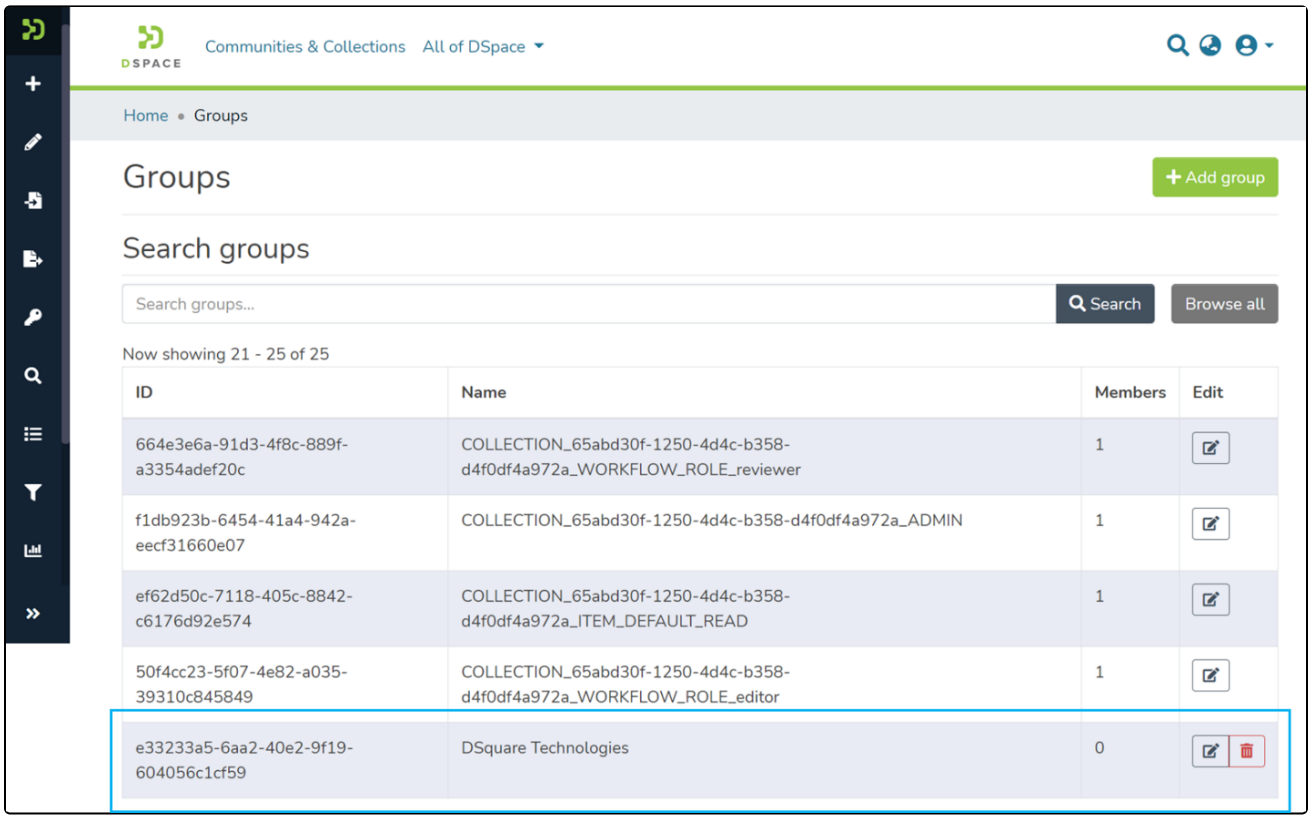
The "Save" button will get activated upon entering the group name.



Step 7: Upon successfully adding Group in DSpace, you will see a success prompt on the screen.



Step 8: The Group created will appear in the Group's list as highlighted below.



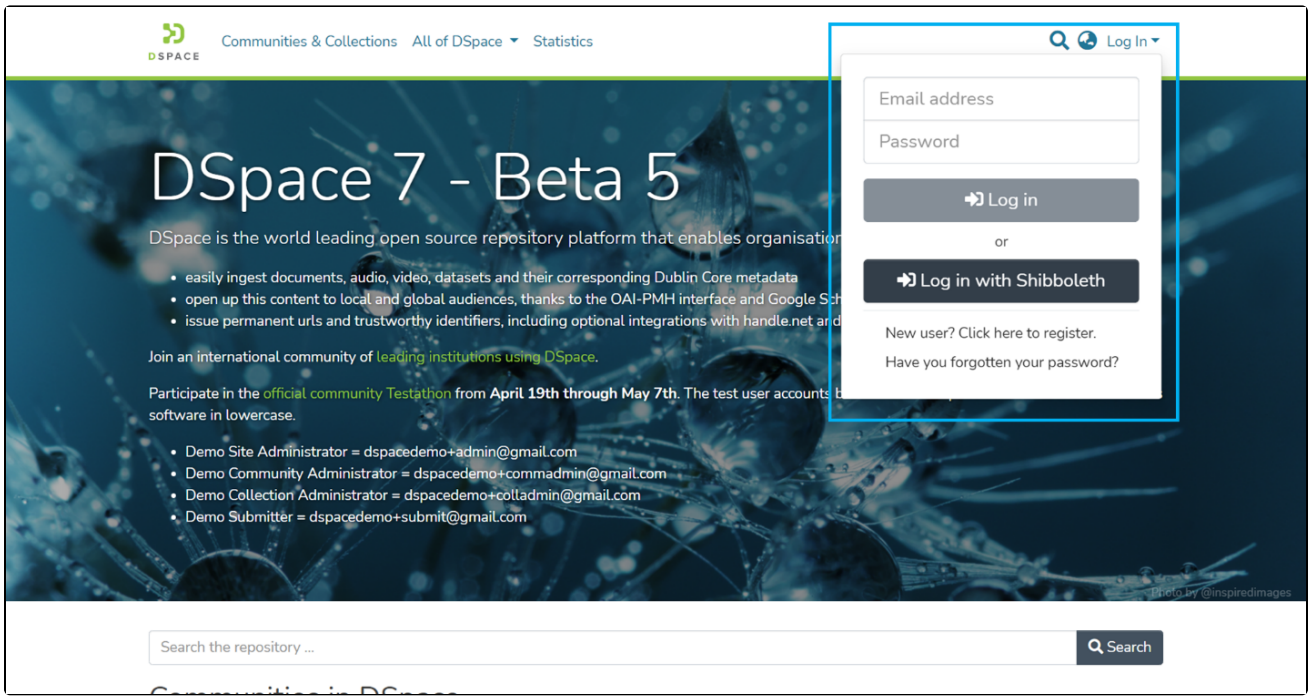
Manage a user group

The System, Community, and Collection administrator profiles can update user group(s). These users can perform the following activities:

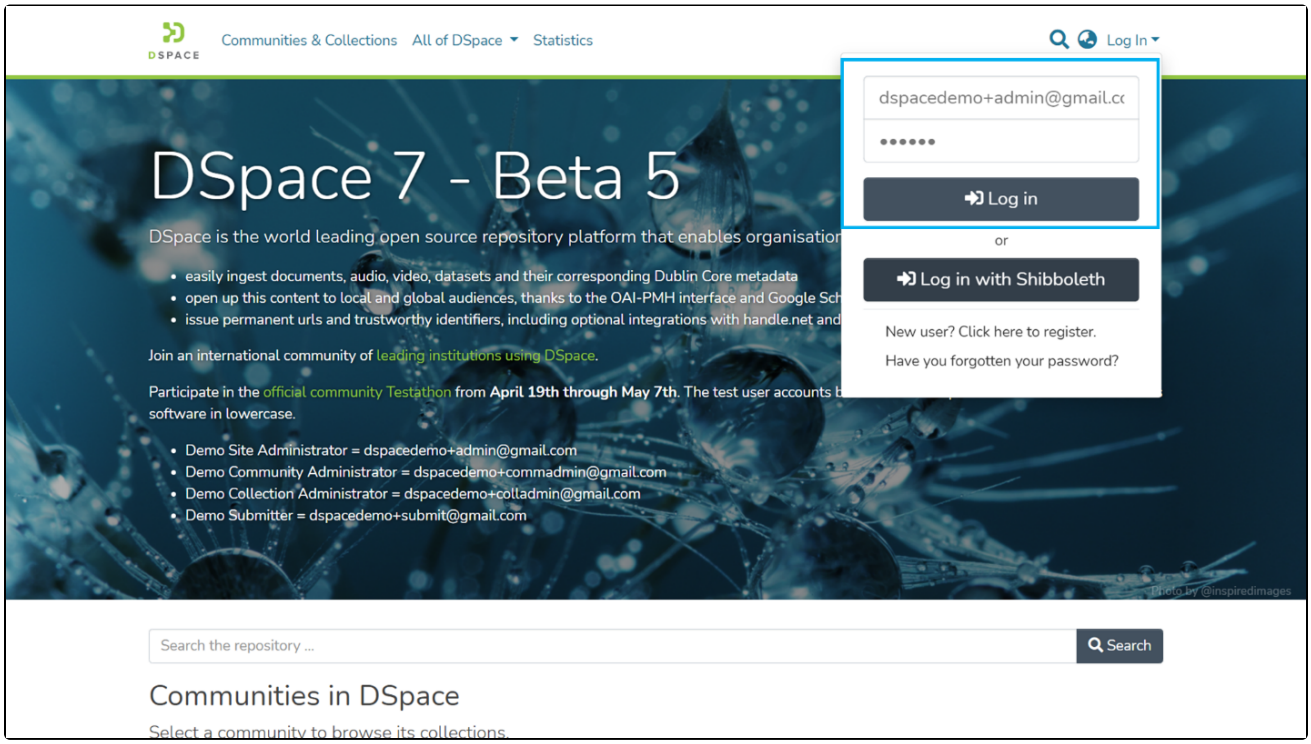
- Update group details
- Delete group
- Add/remove E-people
- Add/Remove Subgroups

Update group details

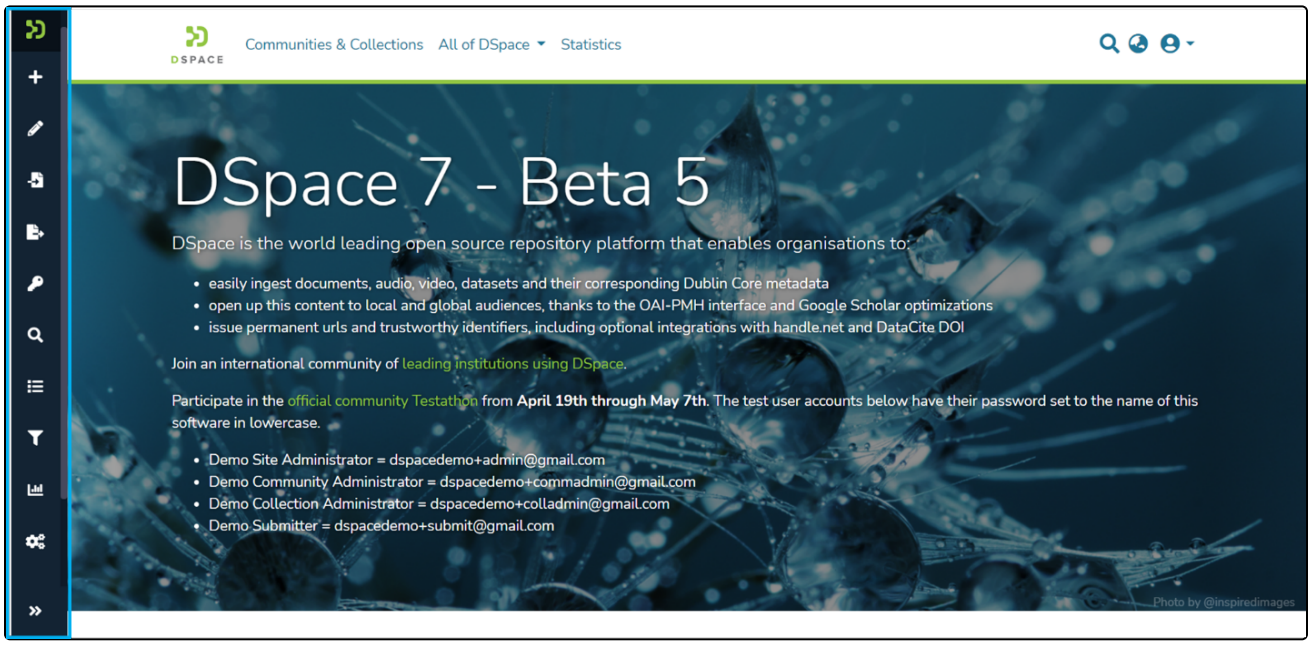
Step 1: Click on the "Log In" link at the top right corner of the DSpace's homepage, and the pop-up will open, as illustrated below the screen.



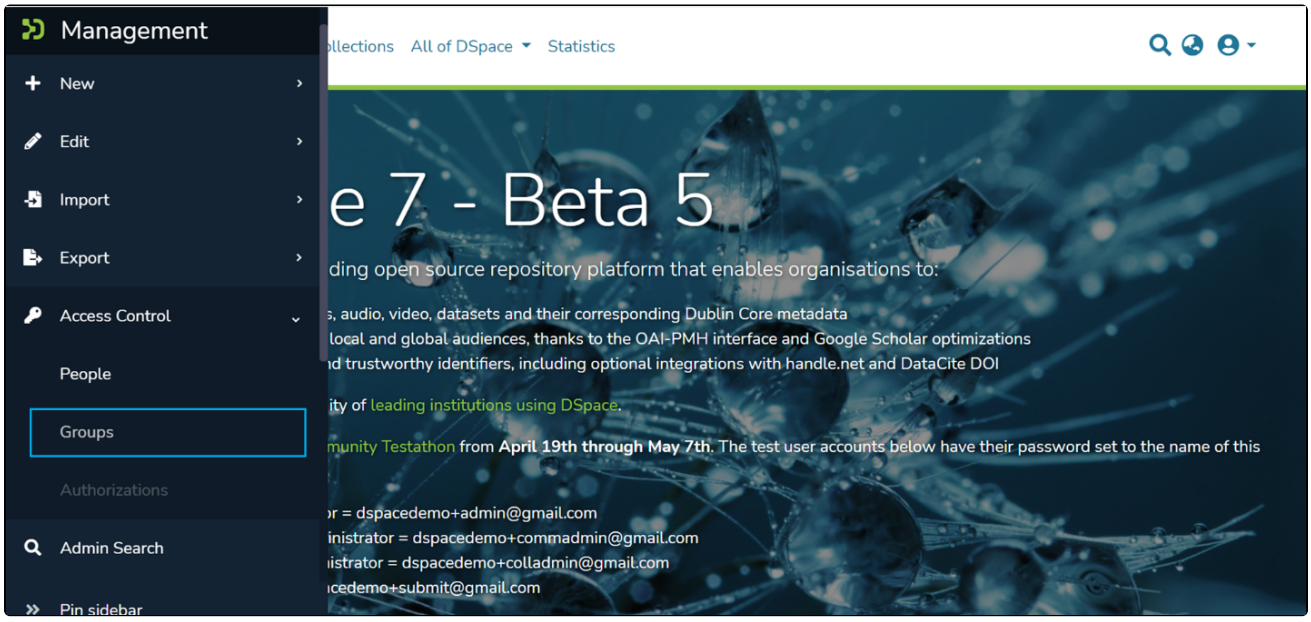
Step 2: Enter your user id and password and click on the "Log in" button to enter into DSpace.



Step 3: Users with administrative rights will see the admin menu on the left-hand side of the screen, as shown in the below illustration.



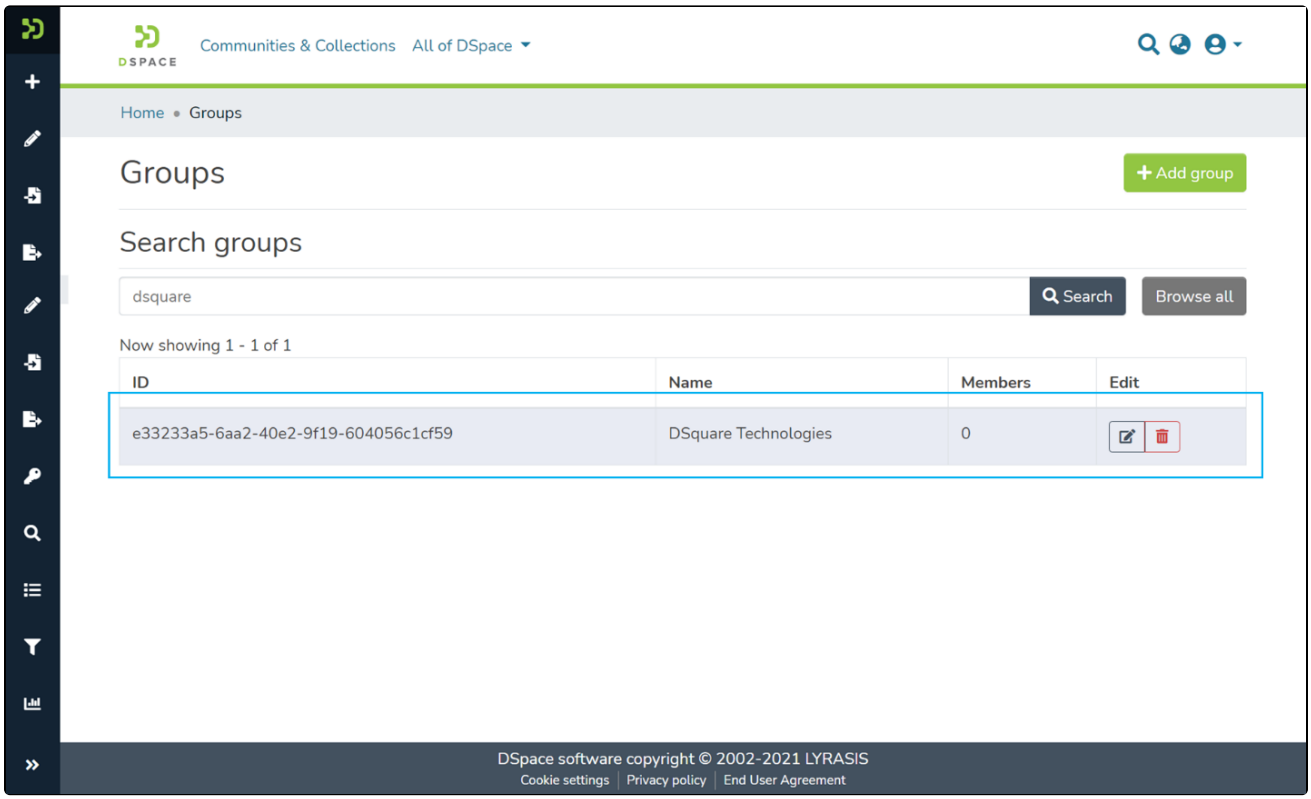
Step 4: Rollover your cursor over the administration menu, click on "Access Control," and click on the "Groups" link.



Step 5: Users can find the Group required to be edited or deleted by scrolling down the Groups list or entering Group's metadata in the search field on the group page. Please see the illustration below showing an example.

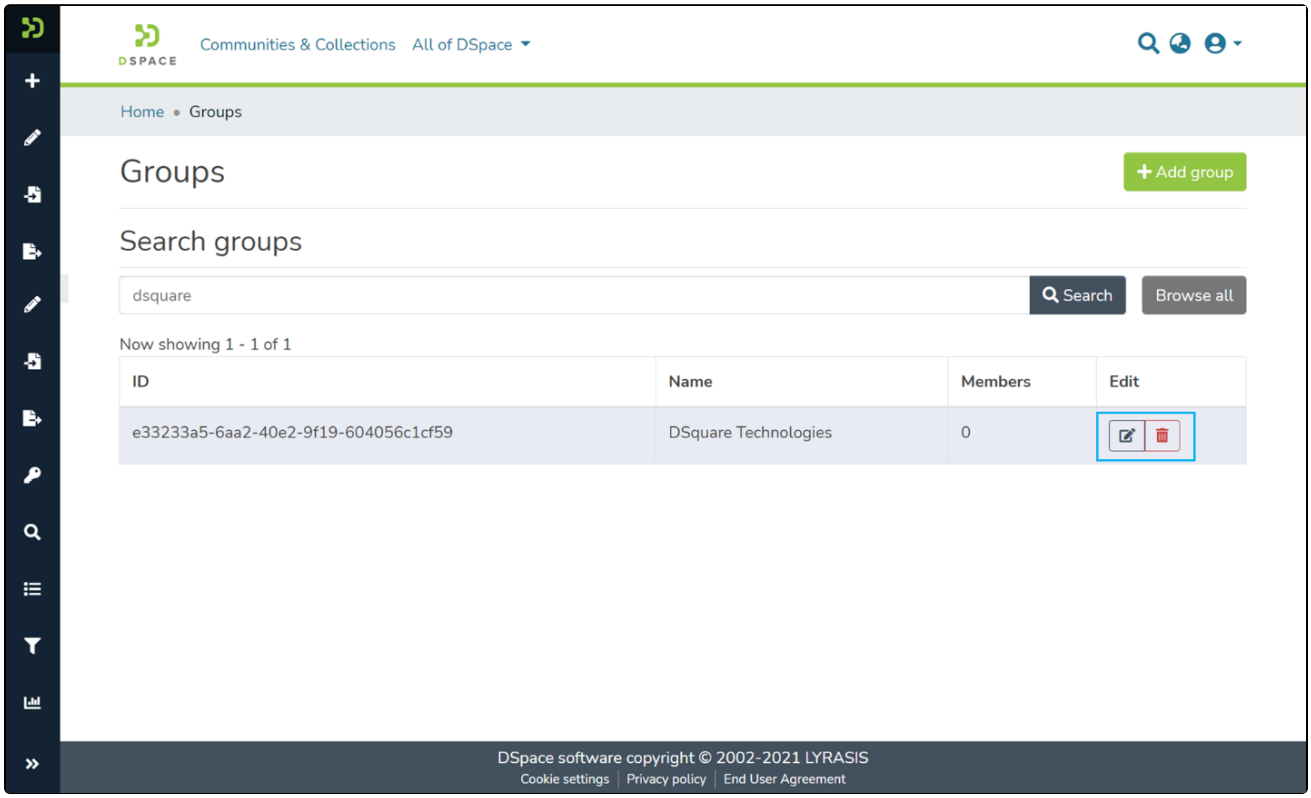
The user group table shows the following details

- **ID:** Unique ID generated in DSpace for each Group
- **Name:** Name of Group as entered during group creation
- **Members:** A count of Epeople were added to the Group as members.



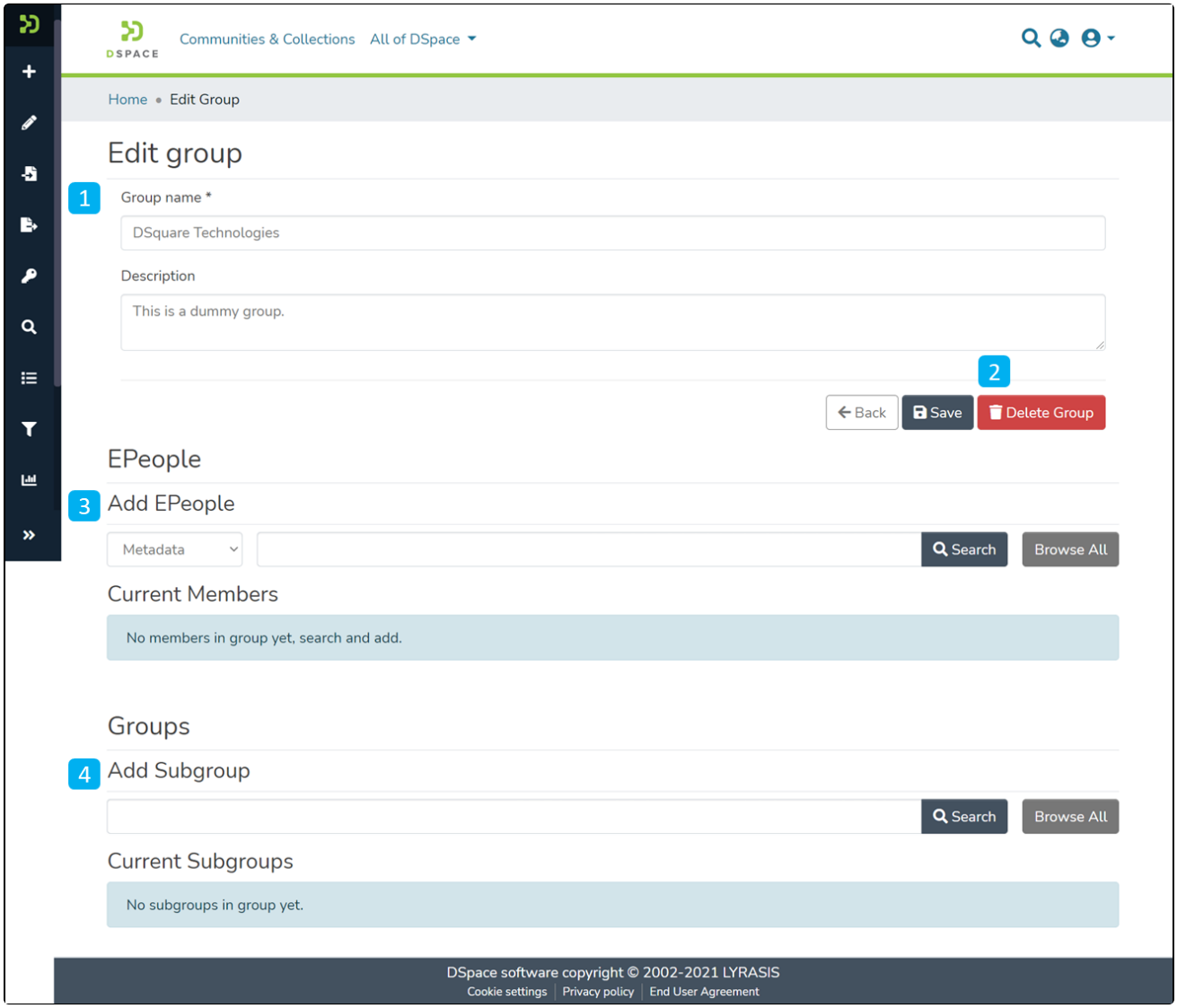
Step 6: Click on the

1. "Edit" button in the Edit column to edit the Group.
2. "Delete" button to delete the Group from DSpace permanently.



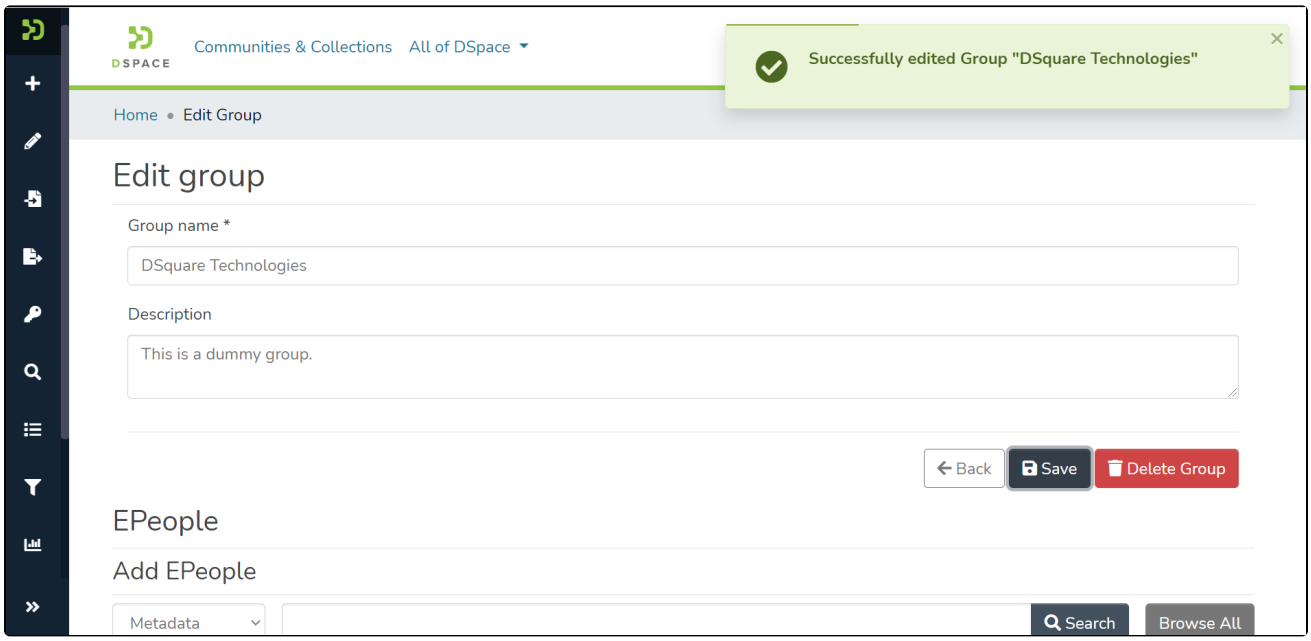
Step 7: You can make the following updates on the Edit group page

1. **Update group details:** Update the Group name and description.
2. **Delete Group:** Permanently delete the Group from DSpace.
3. **Add/Manage People:** Add Epeople to the Group using the search and browse function. Or delete existing group members using the delete option.
4. **Add/Manage subgroups:** Add existing groups as subgroups using the search and browse function. Or delete existing subgroups using the delete option.



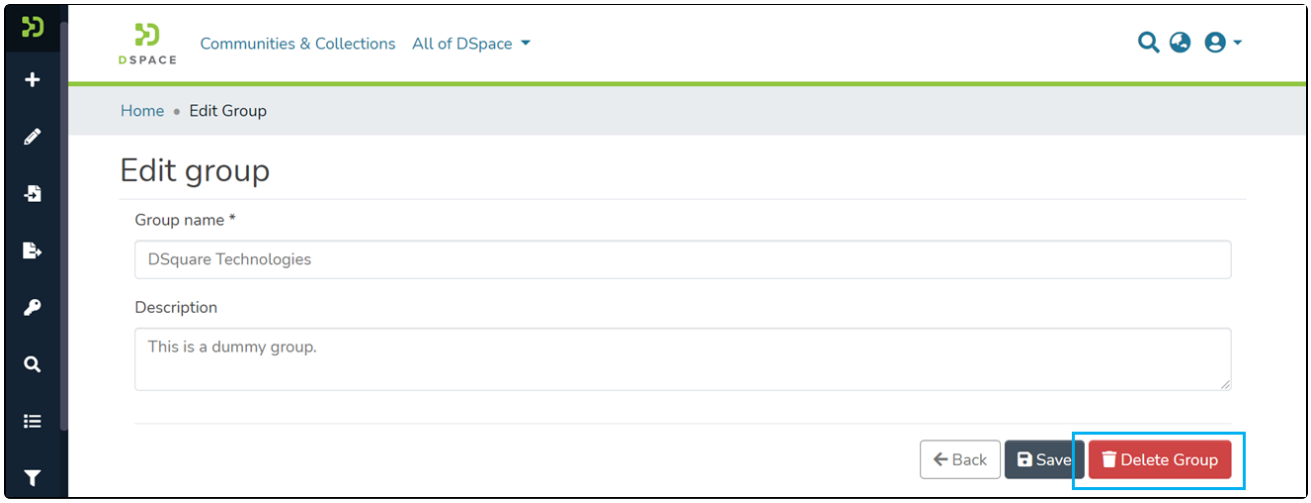
Step 8: Update the group name and description to update the group details and click on the "Save" button.

Step 9: A success prompt will appear, confirming a successful update as displayed below.



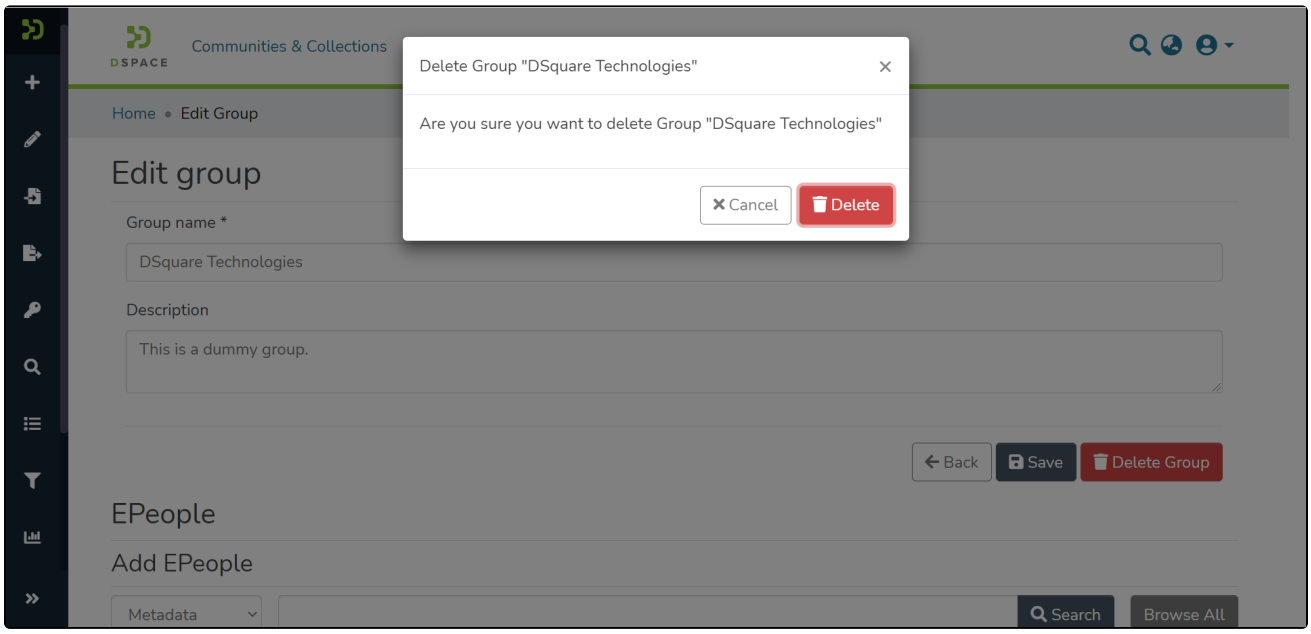
Delete Group

Step 10: Click on the "Delete Group" button to permanently delete the Group from DSpace.

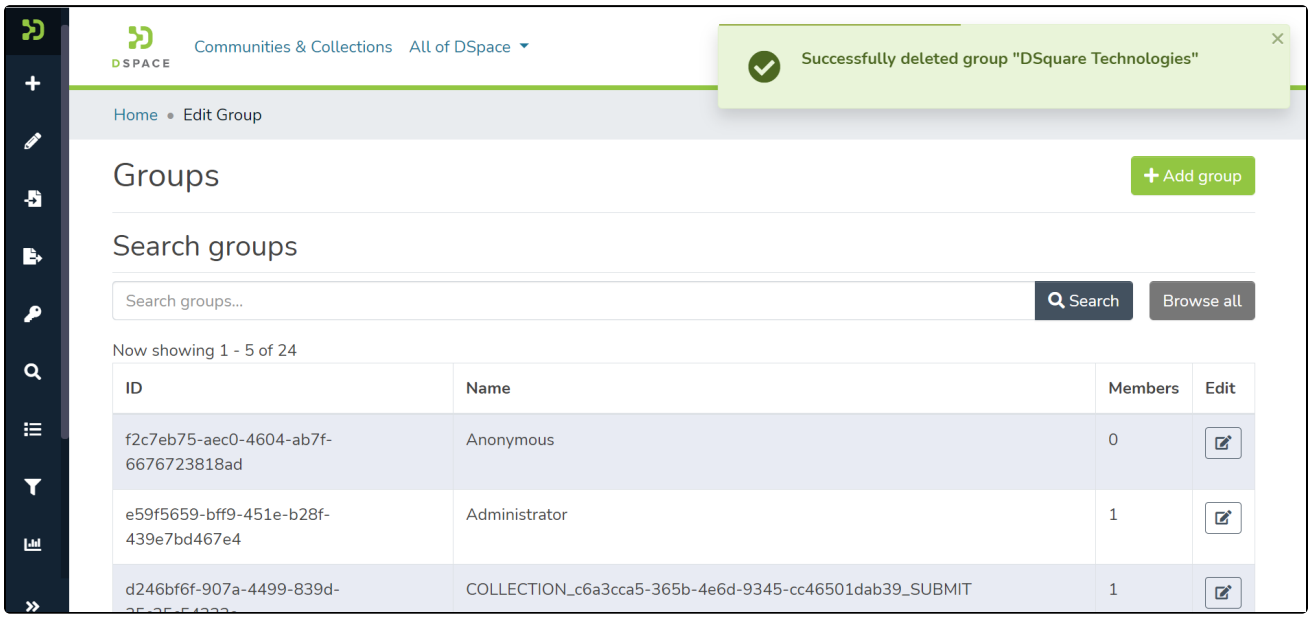


Step 11: DSpace will show you a confirmation prompt to re-assess your decision of deleting the Group. Click on "Delete" if you want to continue, or click "Cancel" to return.

Please note that the Group, once deleted, can not be recovered.

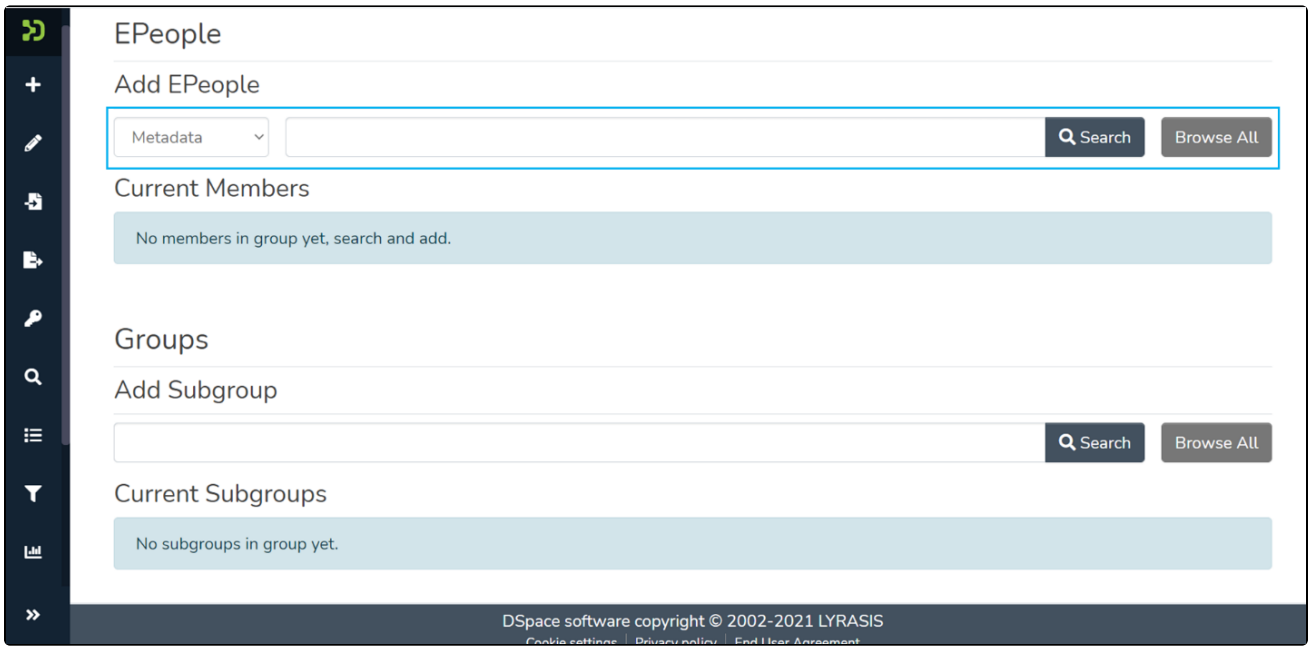


Step 12: Users will see a success prompt confirming the user group deletion, as demonstrated below.

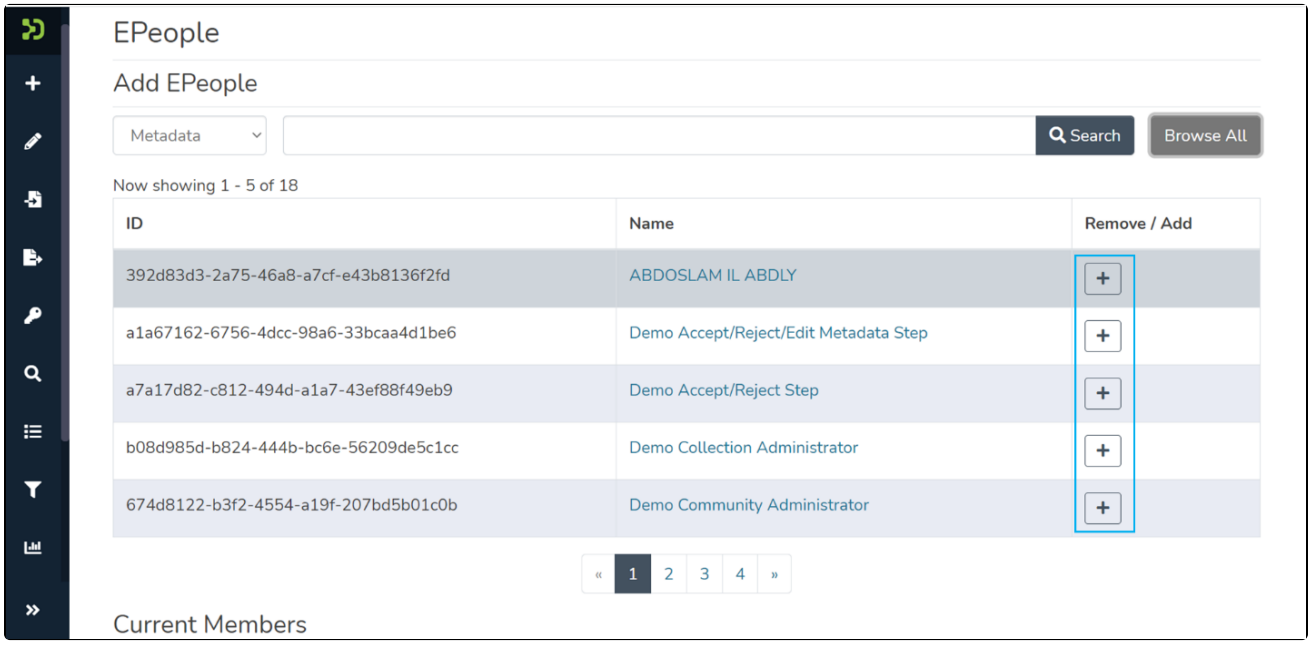


Add/Manage EPeople

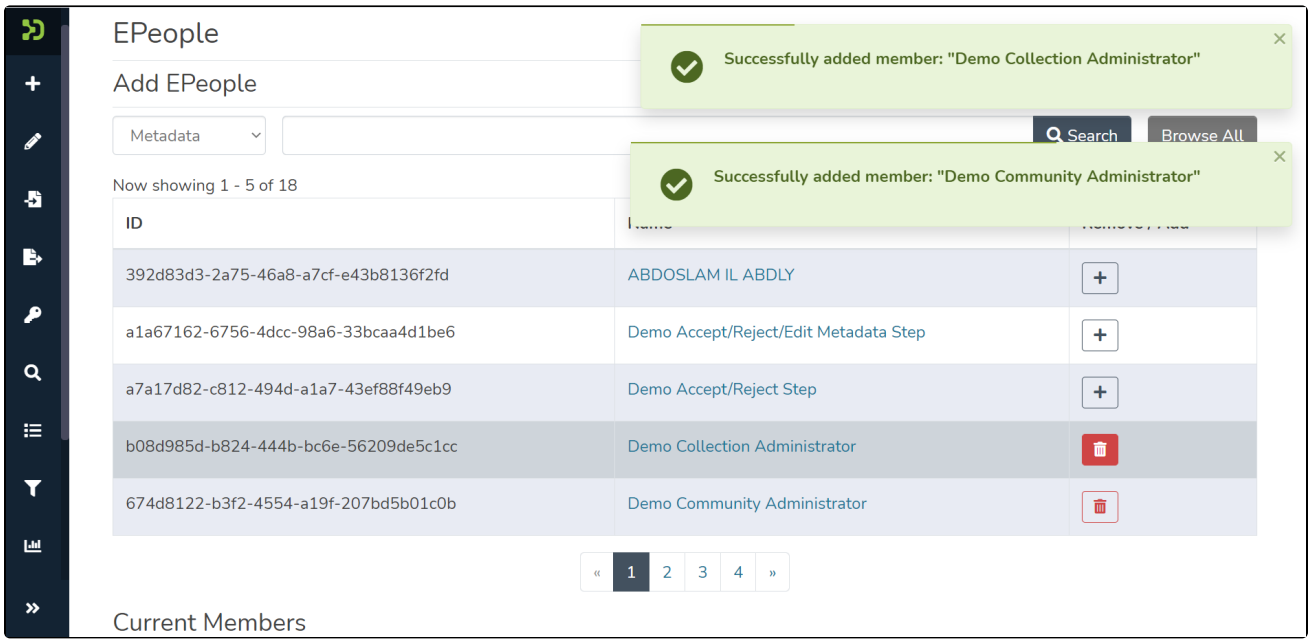
Step 13: Scroll down to the Epeople section to find the user(s). Use metadata values to search Epeople and click the "Search" button, or click the "Browse All" button to list EPeople.



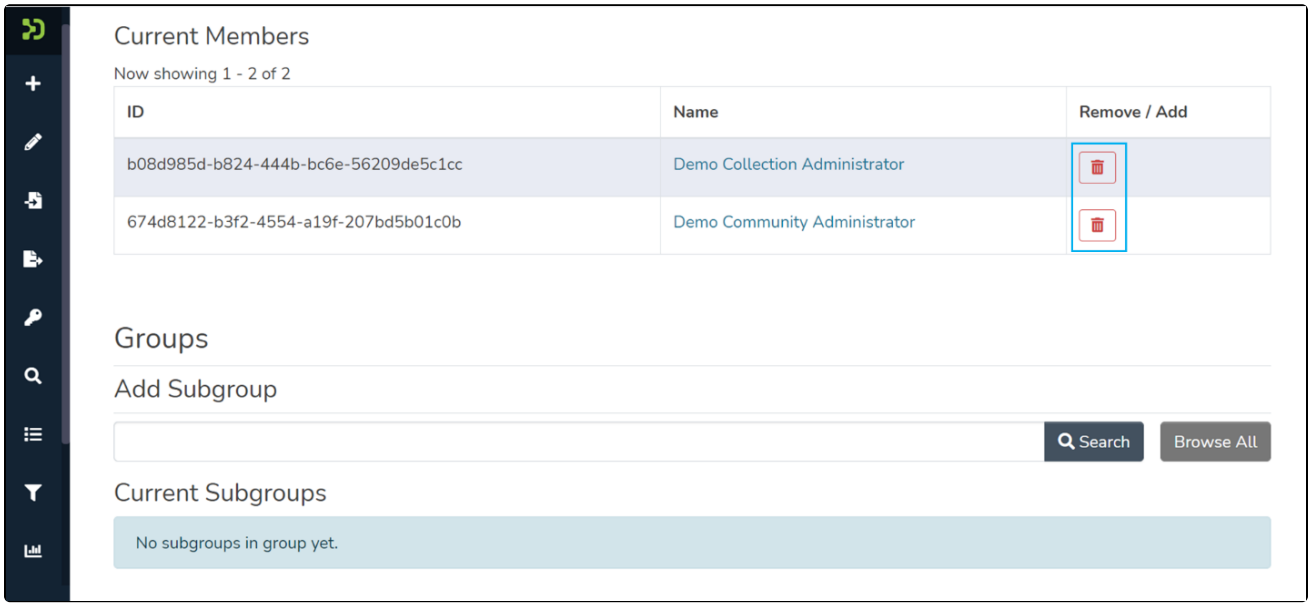
Step 14: Click on the add button '+' next to each user name to add that user as a group member.



Step 15: A prompt will appear confirming the addition of users to the Group. As illustrated below, the "Delete" button will appear next to the user names.



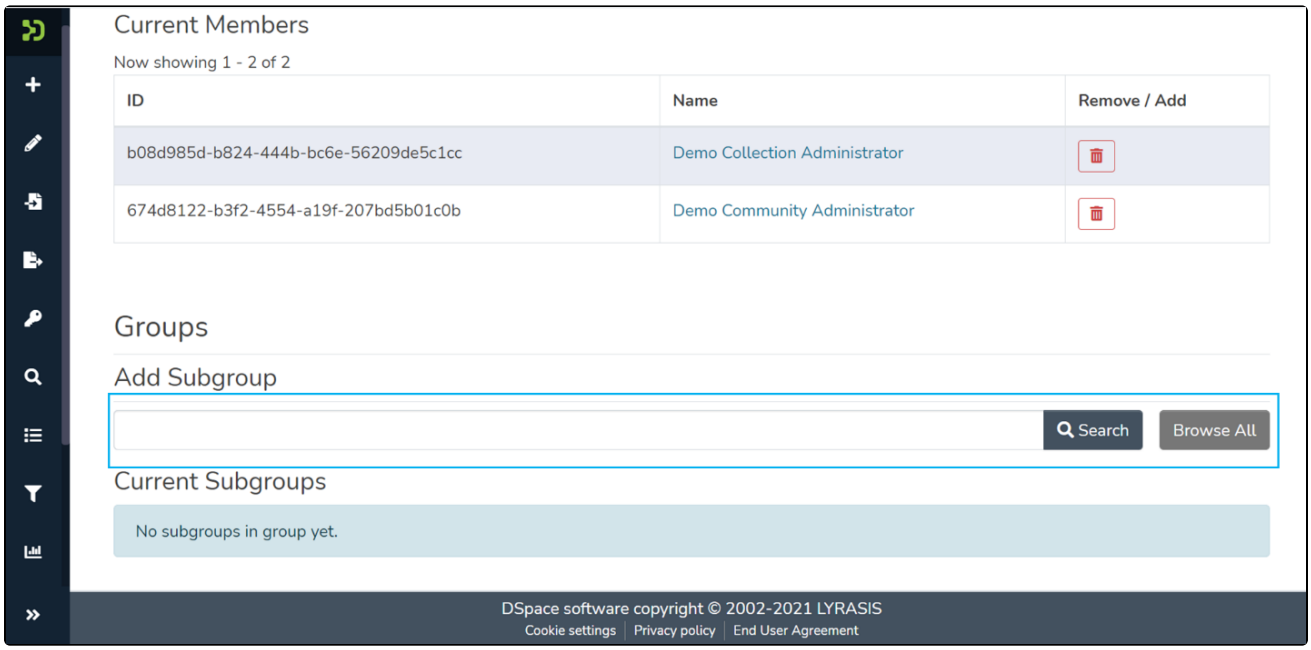
Success prompt confirming user added as members



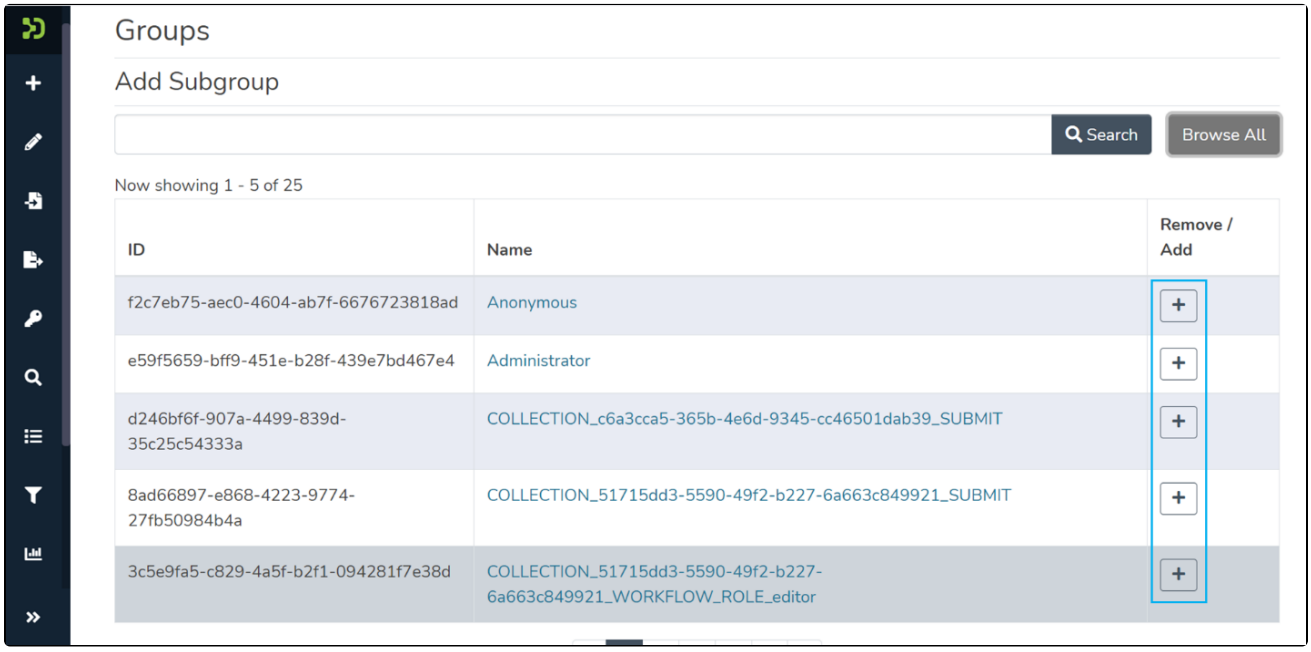
Users appearing as Members under the Current Members section

Add/Manage subgroups

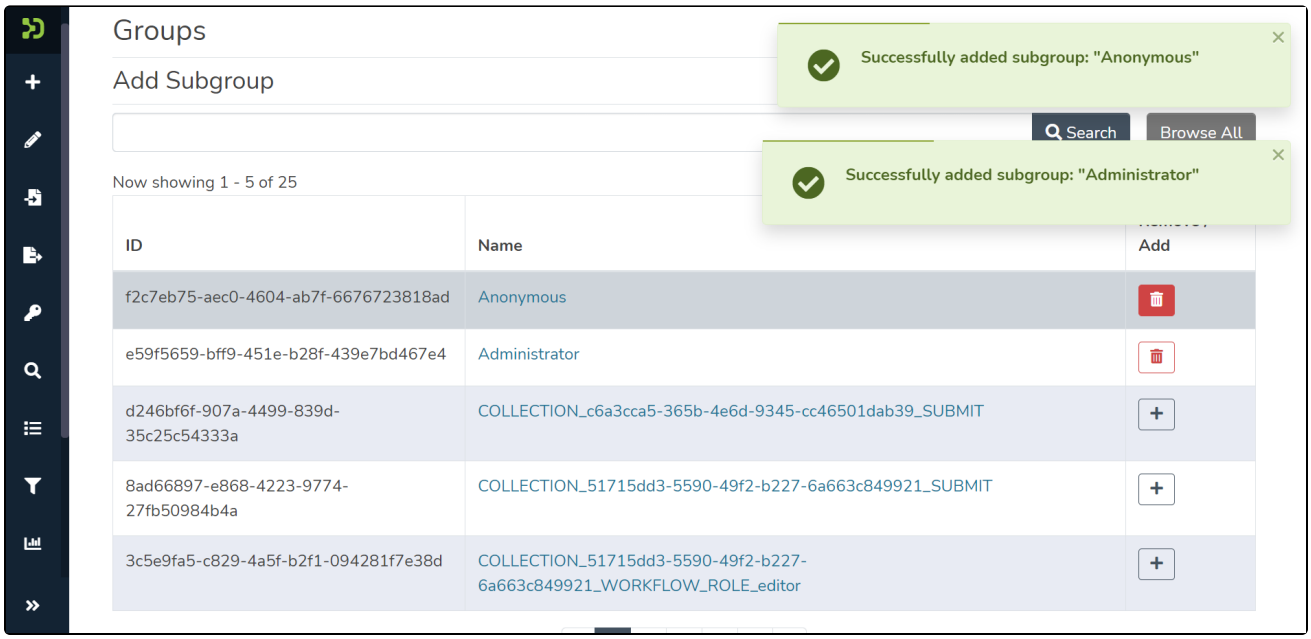
Step 16: Scroll down to the Group section to find the Group (s). Enter a group name and click the "Search" button, or click the "Browse All" button to list groups.



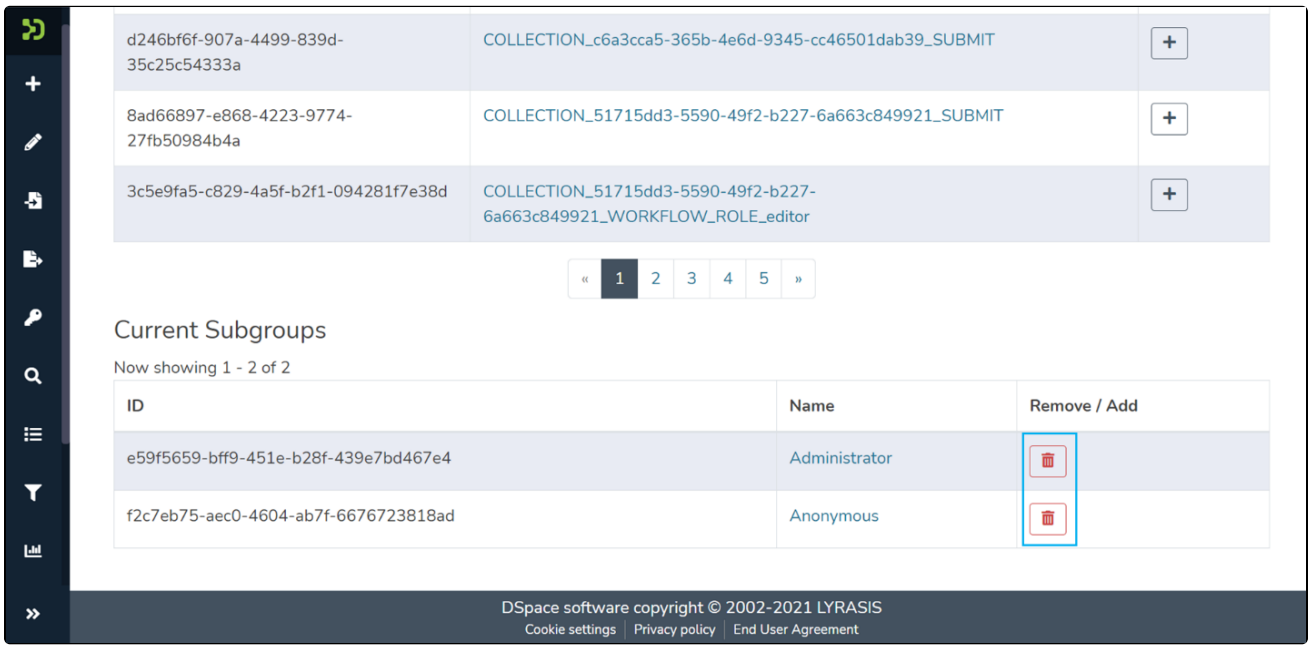
Step 17: Click on the add button '+' next to each group name to add it as a sub-group.



Step 18: A prompt will appear to confirm a subgroup in the Group. As illustrated below, the "Delete" button will appear next to the group names.



Success prompt confirming groups added as subgroups



Groups appearing as subgroups under the Current Subgroups section